Learning coefficients on matrix-factorization fibers and saddles

An expert proof and a companion for human verification

MAIS-O77, depth two

GPT Sol 5.6, GPT Astra 6.0,
directed by Niels Feld*

17 October 2026, 15:54 UTC
Statement-review edition 1.60

Abstract

Let $I, O, H \in \mathbb{N}$ with $I, O \geq 1$, and let $\Phi \in \mathbb{R}^{O \times I}$ have rank $r < H$. We study the loss $L_\Phi(A, B) = \frac{1}{2}\|BA - \Phi\|_F^2$ on $\mathbb{R}^{H \times I} \times \mathbb{R}^{O \times H}$, with the Euclidean norm and Lebesgue measure in all matrix entries. At an exact fit of factor ranks a, b, set $p = O - b$, $q = I - a$, $h = H + r - a - b$, and $s = Oa + b(I - a)$. The local two-sided band-volume pair is $((s + d_0(p, q, h))/2, \mu_0(p, q, h))$, where d_0 is the minimum of $(h - t)(q - t) + pt$ over integers $0 \leq t \leq \min(h, q)$ and μ_0 counts its minimizers. We determine all rank strata attaining the least threshold and the greatest multiplicity attained there. Given an orthonormal singular expansion of Φ with distinct positive singular values, every point of every specified nonoptimal saddle rung has pair $(1, 1)$.

Part I presents the mathematics for human refereeing and reproduces the Lean definitions and main statement needed to audit what was formalized. Part II explains the proofs beside the Lean code. The final Lean theorem has no external Wishart hypothesis: its dependency closure proves the required Gram and spectral integral identities. The conclusion concerns radius-first band-volume bounds; it does not assert a formal meromorphic-continuation theorem. As reported on 17 October 2026, Niels Feld has personally reviewed the main Lean statement and its upstream definitions and judges that they faithfully express the intended O77 question under its stated hypotheses. Independent refereeing and complete personal understanding of the proof remain separate tasks.

How to read and review this edition. Read Part I as a preprint. In Part II, compare each mathematical definition and proof with the following Lean block; elementary coordinate and algebraic helpers are grouped where they serve one calculation. The module headings follow dependency order. A generic intermediate theorem may have hypotheses that its later O77 client proves; this does not add those hypotheses to the final answer. The essential comparison is between the ordinary meaning and the full Lean type, including every parameter and the definitions it unfolds.

Code and checking record. The public checked artifact is `O77Full_2026_10_06_20_50_UTC.lean`, identified by its SHA-256 in Section 90. The repository includes the successful 7 October compilation and separate `leanchecker` replay log. The 55-module terminal development,

*Niels Feld has reviewed the main Lean statement and the upstream definitions needed to interpret it, and judges that they express the intended O77 question. He is still developing a complete understanding of the proof. His expertise is in algebraic geometry, motivic homotopy theory, and K-theory, rather than singular learning theory.

including all proof scripts and seven axiom guards, is unchanged in this editorial update. Six supplementary modules remain outside that terminal dependency closure. The earlier September certification anchor and the October preservation record are retained as historical provenance.

AI assistance and statement review (17 October 2026). The mathematics, formalization, and editorial revisions were developed predominantly by language models, directed by Niels Feld. Feld reports:

> I have personally reviewed the main Lean statement and the upstream definitions needed to interpret it. In my judgment, they faithfully express an answer to the natural-language MAIS-O77 question in its two-sided band-volume formulation, under the stated hypotheses.

This records Feld's mathematical judgment about the formal specification. He has also worked through the mathematical exposition, but does not yet claim a complete personal understanding or an independent verification of every part of the proof. The recorded kernel checks and this statement review provide distinct evidence. Independent scrutiny of both the specification and exposition remains welcome. This edition does not claim completed independent refereeing, a Comparator run, or a formal zeta-pole identification. The dated record and its exact source identity appear in Section 90.1.

Contents

I Human-facing preprint: statement reviewed **7**

1 Introduction and conventions **7**

2 The local invariant and the exact O77 specification **8**

3 The common local-volume calculus **19**

4 Rank geometry and canonical coordinates **21**

5 The explicit depth-two elimination chart **23**

6 The exact Gaussian Gram and eigenvalue identities **26**

7 The residual matrix-product germ **31**

8 Finite minimization and the complete minimum locus **36**

9 The all-points fiber answer **39**

10 The saddle branch at an arbitrary rung point **40**

11 Conclusion of the proof **44**

II Detailed mathematical proofs beside the frozen Lean development **45**

12 O77: finite residual arithmetic **45**

13 The complete numerical minimum locus **64**

14 Rectangular matrix and chart algebra **80**

15 The canonical multiplication derivative **98**

16 The exact algebraic elimination chart **112**

17 Direct saddle-band calculus **126**

18 Full-dimensional saddle-band assembly **140**

19 Spectral-prefix arithmetic **153**

20 Ordered-Laguerre analysis **184**

21 Abstract residual interfaces: reusable conditional assembly **248**

22 Concrete Gaussian rows and Sylvester identity **255**

23 Arbitrary quadratic Gaussians and smaller-Gram spectra **264**

24 Permutation chambers and sorted spectra **275**

25 The concrete real-Wishart interface: exact intermediate spectral interface 289

26 The conditional Gaussian Laplace estimate 296

27 Exact dilation and the one-shell estimate 311

28 Global localization and the residual-origin pair 326

29 Module `O77AdaptedSections` 349

30 Module `O77NumberedCoordinates` 353

31 Module `O77Analytic` 361

32 Module `O77CanonicalRanks` 380

33 Module `O77Transport` 383

34 Module `O77NormalForm` 390

35 Module `O77Coordinates` 401

36 Module `O77Canonical` 416

37 Module `O77BasisChange` 428

38 Module `O77FiberRanks` 435

39 Module `O77FiberClassification` 437

40 Module `O77SaddlePreparation` 443

41 Module `O77MatrixFrames` 450

42 Module `O77RungGeometry` 454

43 Module `O77SaddleAttachment` 467

44 Module `O77RungCompletion` 477

45 Module `O77PowerLogUniqueness` 482

46 Module `O77SemanticMinimum` 485

47 Module `O77Answer` 493

48 One-dimensional Gram spectra and squared radii 496

49 Rectangular orientations, small ranks, and normalization 501

50 The higher-rank interface and the O77 contract 505

51 Nonnegative polar integration, including infinite tests 505

52 Symmetric entry measure and Schur complements 509

53 The rank-deficient exceptional set has entry measure zero 515

54 Exact independent-entry coordinates for the Schur cone 518

55 Constructing coordinates adapted to the actual rows 525

56 Projecting one row and integrating its orthogonal radius 527

57 Splitting the last matrix row in entry measure 532

58 A measurable border kernel and its cone identity 534

59 The Gaussian Gram pushforward theorem 539

60 From the Gram cone to the exact Wishart interface 542

61 The spectral tangent in independent upper entries 545

62 Half-normalized Cayley coordinates 553

63 Orthogonal conjugation in upper-entry measure 558

64 The genuine orthogonal spectral map 563

65 Null zero loci of real polynomials 570

66 Repeated spectra and the singular boundary 573

67 Constructed local inverses and exact local integration 577

68 The determinant of the skew-coordinate pullback 581

69 The full moving spectral Jacobian 587

70 Attach the actual density to the sorted extension 595

71 Exact local weighted spectral integration 598

72 Charts at every actual simple-spectrum matrix 600

73 The exact finite ambiguity of an eigenframe 604

74 Every spectral orbit meets the Cayley chart 609

75 Counting actual Cayley fibers 612

76 Separating eigenvalue and angular entry coordinates 617

77 Ordered spectra and the full-orthant factor 624

78 The spectral map is locally invertible on its ordered source 627

79 A change-of-variables formula with the actual fiber count 630

80 The image and finite fibers of the global spectral map 638

81 Angular density and parameter independence **642**

82 Global spectral integration and the unconditional answer **646**

83 Supplementary modules outside the terminal dependency closure **651**

84 Auxiliary module `O77DimensionLocalResidual` **651**

85 Auxiliary module `O77ResidualTranspose` **654**

86 Auxiliary module `O77LowRankFiber` **657**

87 Auxiliary module `O77LowRankAnswer` **659**

88 Auxiliary module `O77PublishedInputAudit` **660**

89 Auxiliary module `O77MultiplicityTranslation` **661**

90 Certification anchor and reproducibility **664**

Part I
Human-facing preprint: statement reviewed

1 Introduction and conventions

The problem asks for the small-volume behavior of the squared error of matrix multiplication, both along its zero fiber and on specified nonoptimal saddle rungs [1, 2]. We prove the complete rankwise answer on the zero fiber, identify all rank strata minimizing the threshold, and show that every point on every nonoptimal rung has signed-band pair $(1, 1)$. The first part gives an ordinary mathematical proof. The second part presents the Lean development together with explanations of its statements and proof steps. The main development's mathematical declarations and proofs are those of the certified file; obsolete editorial comments have been removed.

The distinction between the two readings is precise. An ordinary proof may invoke familiar finite-dimensional integration theorems with their hypotheses; the Lean proof derives the required matrix and spectral integrals from its library foundations. In particular, the final theorem has no Wishart premise. The argument below includes the Gram and spectral changes of variables, so the normalization used in the residual calculation can be checked without guessing a density convention.

The discussion concerns two factors only. It makes no assertion about arbitrary-depth networks, stochastic gradient dynamics, or a formal meromorphic continuation theorem. The mathematical invariant is the radius-first two-sided volume predicate defined next. Earlier work on reduced-rank regression motivates the matrix normal form and residual calculation [7]; all ingredients needed here are stated and justified below.

Prior contributions and acknowledgments. The present development builds on publicly posted work in the MAIS discussion. Rob Sneiderman (`Robby955`) posted the overlapping exact-fit rank formula, its analytic matrix normal form, and the residual Laguerre calculation in issue #3 and the accompanying O70 preprint [3, 7]; he credits Claude Opus 5 for AI assistance. Svyatoslav Novikov (`kumino`) posted the O77 candidate solution in issue #12, including the all-points nonoptimal-saddle conclusion, and expressly deferred credit for the overlapping fiber formula to Sneiderman [4]. Novikov attributes the construction, drafting, and AI auditing to OpenAI Codex under his direction. His scalar counterexample to the opposing-staircases conjecture, in issue #5, is related context rather than a proof of the general O77 saddle assertion [5].

Mario Brcic (`mbrcic`) has developed a separate Lean treatment in the *AI Safety Formalization Atlas* and contributed detailed formalization and specification audits in comments on issues #3, #5, and #12 [6]. In the inspected version, its band-volume saddle theorem is unconditional, while its fiber result takes the Wishart eigenvalue law as a hypothesis. We credit these mathematical and formalization contributions without treating their existence as an independent referee report on the present notes. The problem and agenda themselves are due to the MAIS project directed by Lionel Levine, with the AI contributions recorded in [1, 2]. No priority claim is made here for the previously posted fiber or saddle formulas.

For natural numbers m, n, write $\mathbb{R}^{m\times n} = \mathbb{R}^{m\times n}$. For a real matrix X, write X^{T} for transpose, and, for matrices $X, Y \in \mathbb{R}^{m\times n}$ of the same shape, put $\langle X, Y\rangle_F = \sum_{i=1}^{m}\sum_{j=1}^{n} X_{ij}Y_{ij}$ for the Frobenius inner product. Its associated norm is $\|X\|_F = \sqrt{\langle X, X\rangle_F}$, called the Frobenius norm. Cartesian products of matrix spaces carry the Euclidean norm whose square is the sum of all squared entries, and product Lebesgue measure in those entries. The norm $\|X\|_{\mathrm{op}}$ is the induced Euclidean operator norm. For a linear map, ker and im denote its kernel

and image; $\mathrm{GL}_n(\mathbb{R})$ is the group of invertible real $n \times n$ matrices, and I_n is the identity matrix. We write $Df(w)[v]$ for a derivative applied to a vector, and $D^2 f(w)[v_1, v_2]$ for the bilinear second derivative. A C^j map has continuous derivatives through order j. An empty-coordinate space is one point of measure one; empty determinants and products equal one. For two positive real functions F, G on the same parameter set, $F \asymp G$ in a specified limit means that there exist real constants $0 < c \le C < \infty$ such that $cG \le F \le CG$ for every parameter sufficiently near that limit. Constants may depend on fixed dimensions, points, and radii, but not on the varying asymptotic parameter.

2 The local invariant and the exact O77 specification

Standing dimensions and domains. The matrix-factorization problem has natural dimensions I, O, H, with $I, O \ge 1$. Its target is a real matrix $\Phi \in \mathbb{R}^{O \times I}$, and $r = \operatorname{rank} \Phi$; the main theorem assumes $r < H$. A point in parameter space is always $w = (A, B)$ with $A \in \mathbb{R}^{H \times I}$ and $B \in \mathbb{R}^{O \times H}$. All ranks are dimensions over $\mathbb{R}$. Auxiliary lemmas explicitly quantify their own dimensions and may include zero-dimensional spaces. Within a lemma, its local letters and hypotheses take precedence over these standing conventions. We use $\mathbb{N} = \{0, 1, 2, \ldots\}$.

Definition 2.1 (Matrix loss and Euclidean measure)**.** *Let $I, O, H \in \mathbb{N}$ with $I, O \ge 1$ and $H \ge 1$, and fix $\Phi \in \mathbb{R}^{O \times I}$. Set $W = \mathbb{R}^{H \times I} \times \mathbb{R}^{O \times H}$, of real dimension $H(I + O)$. For every $w = (A, B) \in W$, define its Euclidean norm by*

$$\|(A, B)\|^2 = \sum_{i=1}^{H} \sum_{j=1}^{I} A_{ij}^2 + \sum_{i=1}^{O} \sum_{j=1}^{H} B_{ij}^2.$$

Equip W with product Lebesgue measure in the entries of A and B. Any enumeration of those entries identifies it with $\mathbb{R}^{H(I+O)}$ and its ordinary Euclidean measure. For every $(A, B) \in W$, put

$$L_\Phi(A, B) = \frac{1}{2} \sum_{i=1}^{O} \sum_{j=1}^{I} ((BA)_{ij} - \Phi_{ij})^2, \qquad F_\Phi = \{(A, B) \in W : BA = \Phi\}.$$

Definition 2.2 (A local volume predicate)**.** *Let $d \in \mathbb{N}$, let $f : \mathbb{R}^d \to \mathbb{R}$ be a function, and let $w \in \mathbb{R}^d$. For real $\delta, \varepsilon > 0$, real λ, and $m \in \mathbb{N}$ with $m \ge 1$, define*

$$V_{f,w,\delta}(\varepsilon) = \mathrm{vol}\{z \in B(w, \delta) : |f(z) - f(w)| < \varepsilon\}, \qquad G_{\lambda,m}(\varepsilon) = \varepsilon^\lambda \big(\log(1/\varepsilon)\big)^{m-1}.$$

Here $B(w, \delta) = \{z \in \mathbb{R}^d : \|z - w\|_2 < \delta\}$ is the open Euclidean ball. The symbol vol *denotes extended nonnegative Lebesgue measure (outer measure on arbitrary sets); the sets used for the polynomial loss are Borel. The assertion $\mathsf{BandPair}(f, w; \lambda, m)$, defined for every $\lambda \in \mathbb{R}$ and $m \in \mathbb{N}$, requires $\lambda \ge 0$ and $m \ge 1$ and, under these guards, means*

$$\begin{aligned} &\exists \delta_0 > 0 \; \forall \delta \in (0, \delta_0) \; \exists c_\delta, C_\delta, \varepsilon_\delta > 0, \\ &\qquad c_\delta \le C_\delta, \quad \varepsilon_\delta < e^{-1}, \quad \forall \varepsilon \in (0, \varepsilon_\delta), \quad c_\delta G_{\lambda,m}(\varepsilon) \le V_{f,w,\delta}(\varepsilon) \le C_\delta G_{\lambda,m}(\varepsilon). \end{aligned}$$

Outside these guards the predicate is false. The constants may depend on the point and on the fixed radius; they do not depend on ε. If $g : \mathbb{R}^d \to \mathbb{R}$ is nonnegative near w and $g(w) = 0$, then $|g(z) - g(w)| = g(z)$ on every sufficiently small ball. Thus the same predicate $\mathsf{BandPair}(g, w; \lambda, m)$ uses the sublevel inequality $g(z) < \varepsilon$ there; no second predicate or notation is needed.

The formulation is intentionally a predicate, not a choice of a pair before its existence is proved. Local functions defined only on an open neighbourhood are handled by requiring all the balls in the definition to lie in that neighbourhood; any total extension agrees on those balls and is irrelevant. A local diffeomorphism is a map between open neighbourhoods with an explicit inverse, not merely an invertible derivative at the base point.

Lemma 2.3 (Uniqueness and the constant germ)**.** *For every $d \in \mathbb{N}$, every function $f : \mathbb{R}^d \to \mathbb{R}$, and every $w \in \mathbb{R}^d$, if both $\mathsf{BandPair}(f, w; \lambda, m)$ and $\mathsf{BandPair}(f, w; \lambda', m')$ hold for $\lambda, \lambda' \in \mathbb{R}$ and $m, m' \in \mathbb{N}$, then $\lambda = \lambda'$ and $m = m'$. If f is constant on a neighborhood of w, then $\mathsf{BandPair}(f, w; 0, 1)$ holds. The constant-germ convention is used only to handle an absent residual factor; it does not assign a finite zeta pole to an identically zero function.*

Proof. Fix $0 < \delta$ below both witness radii, and take $\varepsilon > 0$ below both resulting cutoffs. The two lower bounds make the band volume positive; the upper bounds make it finite. Combining a lower bound from either pair with the upper bound from the other gives constants $0 < c \le C < \infty$, independent of ε, for which

$$c \le \varepsilon^{\lambda - \lambda'} \big(\log(1/\varepsilon)\big)^{m - m'} \le C.$$

The integer difference $m - m'$ here is formed in $\mathbb{Z}$. Set $t = \log(1/\varepsilon)$ and take logarithms. The expression $-(\lambda - \lambda')t + (m - m') \log t$ is bounded as $t \to \infty$. Dividing by t and using $\log t / t \to 0$ proves $\lambda = \lambda'$. Then boundedness of $(m - m') \log t$ proves $m = m'$.

If f is constant on $B(w, \delta_0)$, then for each $0 < \delta < \delta_0$ the band is $B(w, \delta)$ for every $\varepsilon > 0$. Its volume is positive and finite. Take both comparison constants equal to this volume and take $\varepsilon_\delta = e^{-2}$. This proves the pair $(0, 1)$, including $d = 0$, when the volume is one. □

Measure and strict inequalities. The measure in this definition is the actual nonnegative extended-real measure of the band set. It is finite on a fixed bounded window. The Euclidean norm is specified by its entries; the default norm on a function or matrix type is not used to redefine the invariant. For every $d \in \mathbb{N}$, function $g : \mathbb{R}^d \to \mathbb{R}$, and $\varepsilon > 0$, one has

$$\{z : g(z) < \varepsilon\} \subseteq \{z : g(z) \le \varepsilon\} \subseteq \{z : g(z) < 2\varepsilon\}.$$

Intersecting these sets with any fixed ball shows that replacing a strict sublevel by a weak sublevel leaves its power-logarithmic order unchanged, whenever that order exists. Constant rescaling of ε preserves both exponents, as proved below. No assumption that every level set is null is needed.

Definition 2.4 (Minimality defined from the actual volume predicate)**.** *Let $d \in \mathbb{N}$, $f : \mathbb{R}^d \to \mathbb{R}$, $S \subseteq \mathbb{R}^d$, and $w \in \mathbb{R}^d$. Say that w is a threshold minimizer of f on S if*

$$\begin{gathered} w \in S \quad \text{and there exist } \lambda \in \mathbb{R},\ m \in \mathbb{N} \text{ such that} \\ \mathsf{BandPair}(f, w; \lambda, m) \quad \text{and} \\ \forall z \in S\ \forall \lambda' \in \mathbb{R}\ \forall m' \in \mathbb{N}, \quad \mathsf{BandPair}(f, z; \lambda', m') \Longrightarrow \lambda \le \lambda'. \end{gathered}$$

The existential witnesses and the universal comparison belong to the same assertion. Minimality is defined from the measured bands themselves, before any numerical rank formula is introduced.

Definition 2.5 (Feasible rank data and the residual table)**.** *Let $I, O, H, r \in \mathbb{N}$ satisfy $r \le \min(I, O, H)$. For natural numbers a, b, say that (a, b) is feasible for (I, O, H, r) when*

$$r \le a \le \min(H, I), \qquad r \le b \le \min(H, O), \qquad a + b \le H + r.$$

For every such feasible pair define the natural numbers

$$p = O - b, \quad q = I - a, \quad h = H + r - a - b, \quad s = Oa + b(I - a) = Oa + Ib - ab = OI - pq.$$

Independently, for every $p, q, h \in \mathbb{N}$ and every integer $0 \le t \le \min(q, h)$, define the cost $c_{p,q,h}(t)$. Its minimum and the number of minimizing integers are

$$c_{p,q,h}(t) = (h - t)(q - t) + pt = hq + t(p - h - q) + t^2, \tag{1}$$

$$d_0(p, q, h) = \min_{\substack{t \in \mathbb{N} \\ 0 \le t \le \min(q,h)}} c_{p,q,h}(t), \tag{2}$$

$$\mu_0(p, q, h) = \#\{t \in \mathbb{N} : 0 \le t \le \min(q, h),\ c_{p,q,h}(t) = d_0(p, q, h)\}, \tag{3}$$

$$\Lambda(a, b) = \frac{s + d_0(p, q, h)}{2}, \qquad \mathfrak{m}(a, b) = \mu_0(p, q, h). \tag{4}$$

In the last line, (I, O, H, r) is fixed, (a, b) is feasible, and p, q, h, s are its four residual and regular dimensions above; thus Λ and $\mathfrak{m}$ also depend on the suppressed parameters (I, O, H, r). All arithmetic differences outside a stated natural-number formula are taken in $\mathbb{Z}$ or $\mathbb{R}$ after casting. The edge convention follows directly: if $q = 0$ or $h = 0$, the index set is $\{0\}$ and its cost is zero. If $p = 0$ and $q, h > 0$, the cost vanishes only at $t = \min(q, h)$. Hence in all absent-residual cases $d_0 = 0$ and $\mu_0 = 1$. For $0 \le t \le \min(q, h)$, the subtractions in the first expression for $c_{p,q,h}$ are nonnegative. The implementation computes its minimum using the second expression over $\mathbb{Z}$, then proves the agreement before passing to $\mathbb{N}$ and $\mathbb{R}$.

Definition 2.6 (Specified saddle rungs)**.** *For the saddle assertion, fix $I, O, H \in \mathbb{N}$ and $\Phi \in \mathbb{R}^{O \times I}$ with $r = \operatorname{rank} \Phi < H$. Fix orthonormal families $(u_j)_{j=1}^r \subset \mathbb{R}^O$ and $(v_j)_{j=1}^r \subset \mathbb{R}^I$, and positive real numbers $(\sigma_j)_{j=1}^r$ such that $\sigma_i > \sigma_j$ whenever $1 \le i < j \le r$, and suppose*

$$\Phi = \sum_{j=1}^{r} \sigma_j u_j v_j^{\mathsf{T}}.$$

For every integer $k \in \{0, \dots, r\}$ put $\Phi_k = \sum_{j=1}^{k} \sigma_j u_j v_j^{\mathsf{T}}$ and define the following subset of $W = \mathbb{R}^{H \times I} \times \mathbb{R}^{O \times H}$:

$$C_k = \{(A, B) : BA = \Phi_k,\ Av_j = 0,\ B^{\mathsf{T}} u_j = 0 \text{ for every } k < j \le r\}.$$

Thus $C_r = F_\Phi$. This is the explicit rung definition from A7, not just the equation $BA = \Phi_k$. The singular data are supplied as witnesses. Only this last, saddle part of the answer assumes distinct positive singular values; the fiber statements apply to every target of the stated rank.

Remark 2.7 (What the predicate certifies). The target is exactly the power and logarithmic order in O77's band-volume formulation. There is no need to formalize general meromorphic continuation to retain m. A later bridge identifying the volume pair with a zeta pole is a separate, optional theorem. At a saddle the invariant remains a signed-band invariant around the actual loss level.

2.1 The closed minimum formulas

Before stating the theorem, define its numerical minimum condition and value. Fix natural numbers I, O, H, r with $r \le \min(I, O, H)$, and let $a, b \in \mathbb{N}$ denote a feasible pair. Put

$$\chi = (I + O + H + r) \bmod 2, \qquad L_{\text{even}} = \frac{2(H + r)(I + O) - (I - O)^2 - (H + r)^2}{8}.$$

All differences in this formula are differences of real numbers. Define $\mathcal{M}(a, b)$ and Λ_{gl} by the following table, restricted to feasible pairs. The strict regimes are mutually exclusive under $r \le \min(I, O, H)$; when none holds the last row applies.

Dimension regime	$\mathcal{M}(a,b)$	Λ_{gl}
$I+H<O+r$	$a=r$	$[H(I-r)+Or]/2$
$O+H<I+r$	$b=r$	$[H(O-r)+Ir]/2$
$I+O<H+r$	every feasible pair	$OI/2$
$O+r\le I+H$, $I+r\le O+H$, $H+r\le I+O$	$2a+O\le I+H+r+\chi$ and $2b+I\le O+H+r+\chi$	$L_{\mathrm{even}}+\chi/8$

The last row's rank inequalities use only sums of natural numbers. They are exactly the inequalities in `MinimumRankCondition`; we use this form throughout the statement audit. Finally define

$$m_{\mathrm{gl}}=\begin{cases}2, & \chi=1 \text{ and all three balanced inequalities hold,}\\ 1, & \text{otherwise.}\end{cases}$$

At this point these are numerical definitions; the theorem identifies them with extrema of actual measured pairs.

2.2 The complete answer and its Lean meaning

Theorem 2.8 (The complete measured answer to O77)**.** *For every $I,O,H\in\mathbb{N}$ with $I,O\ge 1$, and every $\Phi\in\mathbb{R}^{O\times I}$ of rank $r=\operatorname{rank}\Phi<H$, the following statements hold:*

1. *Every $w=(A,B)\in F_\Phi$ has feasible ranks $a=\operatorname{rank}A$, $b=\operatorname{rank}B$ and $\Lambda(a,b)>0$. For every $\lambda\in\mathbb{R}$ and $m\in\mathbb{N}$,*

$$\mathsf{BandPair}(L_\Phi,w;\lambda,m)\quad\Longleftrightarrow\quad\lambda=\Lambda(a,b)\text{ and }m=\mathfrak{m}(a,b).$$

2. *For every $a,b\in\mathbb{N}$, the pair (a,b) is feasible if and only if there are matrices $A\in\mathbb{R}^{H\times I}$ and $B\in\mathbb{R}^{O\times H}$ satisfying $BA=\Phi$, $\operatorname{rank}A=a$, and $\operatorname{rank}B=b$.*
3. *For every $w=(A,B)\in F_\Phi$, put $a=\operatorname{rank}A$ and $b=\operatorname{rank}B$. Then w is a threshold minimizer of L_Φ on F_Φ, in the measured sense of Definition 2.4, if and only if its ranks satisfy $\mathcal{M}(a,b)$.*
4. *The set*

$$\{\lambda\in\mathbb{R}:\ \exists w\in F_\Phi\ \exists m\in\mathbb{N},\ \mathsf{BandPair}(L_\Phi,w;\lambda,m)\}$$

   has least element Λ_{gl}, including attainment.

5. *The set*

$$\left\{m\in\mathbb{N}:\ \begin{array}{l}\exists w\in F_\Phi\text{ minimizing the threshold of }L_\Phi\text{ on }F_\Phi,\\ \exists\lambda\in\mathbb{R},\ \mathsf{BandPair}(L_\Phi,w;\lambda,m)\end{array}\right\}$$

   has greatest element m_{gl}, including attainment.

6. *For every singular expansion satisfying Definition 2.6, for every $k\in\mathbb{N}$ with $k\le r$ the set C_k is nonempty, and for every $k\in\mathbb{N}$ with $k<r$ and every $w\in C_k$, one has $\mathsf{BandPair}(L_\Phi,w;1,1)$.*

The first five clauses have no singular-value simplicity hypothesis. The sixth quantifies over the supplied strictly decreasing singular data and uses both annihilation conditions in C_k; it is not a claim about every solution of $BA=\Phi_k$. When $r=0$, the fiber statements remain substantive, C_0 is nonempty, and the assertion for $k<r$ is vacuous. The proof occupies the remaining sections of this part.

The exact Lean statement and the definitions needed to interpret it are reproduced in the statement-audit package below. The first five clauses refer to actual volumes on the fixed fiber; the last clause quantifies over the supplied singular data.

A further attained-point consequence. Under the hypotheses of Theorem 2.8, there exists a point of F_Φ with ranks (r, r) that attains both the least threshold and the greatest multiplicity among threshold minimizers. This additional identification of a representative is a consequence of the baseline realization and arithmetic results; it is not a separate field of the six-clause proposition `AnswersO77`.

Two small examples distinguish the assertions. For $I = O = H = 2$ and $\Phi = 0$, a point of ranks $(2, 0)$ has no residual factor but has pair $(2, 1)$, whereas the minimum threshold is $3/2$. An absent residual therefore does not characterize the minimum locus. For $I = O = H = 2$ and $\operatorname{rank}\Phi = 1$, ranks $(1, 1)$ and $(2, 1)$ both minimize the threshold 2, but their multiplicities are 2 and 1. Thus clause (5) is a maximum over the minimum-threshold locus, not a claim that the pair is constant on that locus.

2.3 The complete statement package for a human referee

The following package collects all project definitions, abbreviations, and structures occurring transitively in the *statement* `O77.AnswersO77Phi`. Consequently its meaning can be checked here without reconstructing a chain of definitions from Part II. Each displayed Lean declaration is taken from the certified source; these repetitions are reading aids, not additional declarations in the extracted program. The final theorem and the two supplementary theorem headers are also reproduced below. Results used only inside proof terms belong to the proof companion in Part II.

How to read parameters and library notation. Throughout this package, dimensions $I, O, H, r, a, b, p, q, h, K$ are natural numbers, including zero unless an explicit inequality excludes it. Braces around a declaration such as `IOH:ℕ` universally binds implicit parameters: it does not omit their quantification. A parenthesized argument is explicit. A hypothesis after a colon is an argument of the theorem, and a field of a structure is part of its specification. The real number names `ℝ` and `Real`, and the natural number names `ℕ` and `Nat`, are synonyms. The following Mathlib vocabulary supplies the remaining meanings.

- `Finn` is the ordered finite set $\{0, \dots, n-1\}$. Its coercion or `val` is the corresponding natural number. `MatrixmnℝR` means functions $m \to n \to \mathbb{R}$; its multiplication is the ordinary finite matrix product. The rank `A.rank` is the rank of the associated real linear map.
- A finite sum `∑i,...` runs over the whole indicated finite type. `Fintype` supplies that finite index set. The product `X×Y`, its projections `w.1`, `w.2`, and function addition and subtraction have their ordinary coordinate meanings.
- `MeasureTheory.volume` on $\mathbb{R}$ is Lebesgue measure; `Measure.pi` is finite product measure and `Measure.prod` is binary product measure. Measure values lie in `ENNReal`$= [0, +\infty]$. For real x, `ENNReal.ofRealx` is $\max(x, 0)$ viewed in this extended half-line.
- In `eps^lam`, the exponent `lam:ℝ` selects real power `Real.rpow`; in `log...^k`, the natural number `k` selects an ordinary integer power. `Real.log` and `Real.exp` are natural logarithm and exponential.
- Natural subtraction is truncated: `a-b` in $\mathbb{N}$ means $\max(a - b, 0)$. Subtraction in $\mathbb{Z}$ or $\mathbb{R}$ is signed. An annotation such as `(h:Int)` or `(H:ℝ)` casts *before* the surrounding operations; `Int.toNatz` means $\max(z, 0)$ as a natural number. The expression n mod 2 is the remainder modulo two; Lean writes this operator with a percent sign.
- For an ordered type and a set S, `IsLeastSx` means $x \in S$ and $x \le y$ for every $y \in S$; `IsGreatestSx` means $x \in S$ and $y \le x$ for every $y \in S$. Both include attainment, not merely an infimum or supremum.

Real matrices and parameter space. The first abbreviation is in namespace `O77.Reconstruction0905`. The second is in `O77.Native`, with `RM` opened from `O77.Reconstruction0905`. Thus $w = (A, B)$ has $A \in \mathbb{R}^{H\times I}$, $B \in \mathbb{R}^{O\times H}$, and $w.2 * w.1 = BA$.

```
abbrev RM (m n : Nat) := Matrix (Fin m) (Fin n) ℝ

abbrev Parameters (I O H : ℕ) := RM H I × RM O H
```

The measure is fixed by matrix entries. The following three declarations are in `O77.Residual`. In `frobeniusSq`, the symbols m, n are arbitrary finite *index types*, not dimensions. In `matrixLebesgue`, p, h likewise denote index types; their later instantiations are `FinH`, `FinI`, or `FinO`. These definitions give $\sum_{i,j} A_{ij}^2$ and Lebesgue measure in every real entry. No unspecified default matrix norm or Hausdorff measure is substituted.

```
def frobeniusSq {m n : Type*} [Fintype m] [Fintype n]
    (A : Matrix m n ℝ) : ℝ :=
  ∑ i, ∑ j, (A i j) ^ 2

noncomputable def coordinateLebesgue ι( : Type*) [Fintype ι] :
    MeasureTheory.Measure ι( → ℝ) :=
  MeasureTheory.Measure.pi fun _ : ι => MeasureTheory.volume

noncomputable def matrixLebesgue (p h : Type*) [Fintype p] [Fintype h] :
    MeasureTheory.Measure (Matrix p h ℝ) :=
  MeasureTheory.Measure.pi fun _ : p => coordinateLebesgue h
```

The next declarations lie in `O77.Native`, which opens `O77.Residual` and the abbreviation `RM` above. For every $I, O, H \in \mathbb{N}$, every $\Phi \in \mathbb{R}^{O\times I}$, every $w \in \mathbb{R}^{H\times I} \times \mathbb{R}^{O\times H}$, and every $R, \varepsilon \in \mathbb{R}$, these are total definitions. The norm is $\sqrt{\|A\|_F^2 + \|B\|_F^2}$, and the loss is $\frac{1}{2}\|BA - \Phi\|_F^2$. The strict inequalities define an open ball and a two-sided band about the *actual* value $L_\Phi(w)$.

```
def entryMeasure (I O H : ℕ) : Measure (Parameters I O H) :=
  (matrixLebesgue (Fin H) (Fin I)).prod (matrixLebesgue (Fin O) (Fin H))

def ambientSq {I O H : ℕ} (w : Parameters I O H) : ℝ :=
  frobeniusSq w.1 + frobeniusSq w.2

def entryNorm {I O H : ℕ} (w : Parameters I O H) : ℝ :=
  Real.sqrt (ambientSq w)

def loss {I O H : ℕ} (Phi : RM O I) (w : Parameters I O H) : ℝ :=
  frobeniusSq (w.2 * w.1 - Phi) / 2

def entryBall {I O H : ℕ} (w : Parameters I O H) (R : ℝ) : Set (Parameters I O H) :=
  {v | entryNorm (v-w) < R}

def bandSet {I O H : ℕ} (Phi : RM O I) (w : Parameters I O H) (R eps : ℝ) :
    Set (Parameters I O H) :=
  {v | v ∈ entryBall w R ∧ |loss Phi v - loss Phi w| < eps}

def bandVolume {I O H : ℕ} (Phi : RM O I) (w : Parameters I O H) (R eps : ℝ) : ENNReal :=
  entryMeasure I O H (bandSet Phi w R eps)
```

The Borel structures here are the usual finite-coordinate product structures. The Borel-space instances used in the source prove their compatibility with topology; they supply no free measure parameter. Empty index products have the canonical one-point product measure of mass one. In particular, the positive-radius ball in a zero-dimensional space has mass one. At a fiber point $BA = \Phi$ the displayed loss vanishes, so the band reduces to the nonnegative sublevel set $L_\Phi(v) < \varepsilon$. Without that hypothesis the absolute value must remain.

The power–logarithm comparison, inside out. The next three declarations are in `O77.Residual`. The arguments are a real exponent λ, a natural number k, a positive scale variable ε, and, in the third declaration, an arbitrary function $V : \mathbb{R} \to [0, +\infty]$. Their total definitions are shown, but the comparison is tested only on $0 < \varepsilon < \varepsilon_0 < e^{-1}$. On that interval the logarithm and all factors are positive, so `ENNReal.ofReal` introduces no truncation.

```
def powerLogZeroReal (lam : Real) (k : Nat) (eps : Real) : Real :=
  eps ^ lam * (Real.log (1 / eps)) ^ k

def powerLogZero (lam : Real) (k : Nat) (eps : Real) : ENNReal :=
  ENNReal.ofReal (powerLogZeroReal lam k eps)

def EThetaZeroPowerLog (V : Real -> ENNReal) (lam : Real) (k : Nat) : Prop :=
  ∃ c C eps0 : Real,
    0 < c ∧ c ≤ C ∧ 0 < eps0 ∧ eps0 < Real.exp (-1) ∧
      ∀ eps : Real, 0 < eps -> eps < eps0 ->
        ENNReal.ofReal c * powerLogZero lam k eps ≤ V eps ∧
          V eps ≤ ENNReal.ofReal C * powerLogZero lam k eps
```

Thus `EThetaZeroPowerLogVlamk` asserts a two-sided bound by $\varepsilon^{\lambda}(\log(1/\varepsilon))^{k}$. Its real constants c, C, ε_0 are chosen once, before every smaller ε; $0 < c \le C$ forces both comparison constants to be positive and finite. The radius wrapper is in `O77.Transport0906`, opening this residual predicate. Here $V : \mathbb{R} \to \mathbb{R} \to [0, +\infty]$ takes the radius first and band width second; ρ, δ denote the outer and actual radii.

```
def RadiusOrder (V : ℝ → ℝ → ENNReal) (lam : ℝ) (m : ℕ) : Prop :=
  0 ≤ lam ∧ 1 ≤ m ∧ ∃ rho : ℝ, 0 < rho ∧
    ∀ delta, 0 < delta → delta < rho →
      EThetaZeroPowerLog (V delta) lam (m-1)
```

This order of quantifiers permits c, C, ε_0 to depend on δ, and prohibits dependence on the subsequently varying ε. It requires *every* sufficiently small positive radius. The guard $m \ge 1$ ensures that the natural subtraction $m - 1$ is exactly the intended logarithmic degree; $m = 0$ does not accidentally encode degree zero. The guard $\lambda \ge 0$ is likewise part of the predicate, not a fact postponed to the theorem. The final binding is in `O77.Native`, opening `O77.Transport0906.RadiusOrder`:

```
def HasLocalBandPair {I O H : ℕ} (Phi : RM O I) (w : Parameters I O H)
    (lam : ℝ) (m : ℕ) : Prop :=
  RadiusOrder (bandVolume Phi w) lam m
```

Consequently `HasLocalBandPairPhiwlamm` is precisely $\mathsf{BandPair}(L_\Phi, w; \lambda, m)$ from Definition 2.2, with the specified entry measure. It contains no rank formula or assumed geometric normal form.

Rank feasibility and the finite arithmetic table. The following declarations are in `O77`. Every argument is a natural number. `RankDataIOHrab` records exactly the seven inequalities for ranks $r = \operatorname{rank}\Phi$, $a = \operatorname{rank}A$, $b = \operatorname{rank}B$; it contains no existence or volume assertion.

```
structure RankData (I O H r a b : Nat) : Prop where
  r_le_a : r ≤ a
  r_le_b : r ≤ b
  a_le_I : a ≤ I
  a_le_H : a ≤ H
  b_le_O : b ≤ O
  b_le_H : b ≤ H
  sum_le : a + b ≤ H + r

def residualHidden (H r a b : Nat) : Nat := (H + r) - (a + b)
def regularDim (I O a b : Nat) : Nat := O * a + b * (I - a)
```

Under these inequalities, `residualHiddenHrab` equals $H+r-a-b \geq 0$ and `regularDimIOab` equals $Oa+b(I-a)$. The sum $H+r$ is formed before subtracting $a+b$; there is no intermediate truncated subtraction of $a+b$ from H.

The next three declarations are in `O77.Arithmetic`. For $B, D \in \mathbb{Z}$ and $t \in \mathbb{N}$, `quadraticBDt` is $B+t^2-Dt$, computed in $\mathbb{Z}$. For an arbitrary $f : \mathbb{N} \to \mathbb{Z}$ and $K \in \mathbb{N}$, `minimumUpTofK` recursively computes the minimum over the nonempty set $\{0, \ldots, K\}$. `List.range(K+1)` lists those integers once each, and `filter` retains those satisfying the equality, whose Boolean decision is `decide`. This is finite enumeration and equality testing; no minimization oracle occurs in the definitions.

```
def quadratic (B D : Int) (t : Nat) : Int :=
  B + (t : Int) * (t : Int) - D * (t : Int)

def minimumUpTo (f : Nat → Int) : Nat → Int
  | 0 => f 0
  | K + 1 => min (minimumUpTo f K) (f (K + 1))

def minimizerList (f : Nat → Int) (K : Nat) : List Nat :=
  (List.range (K + 1)).filter (fun t => decide (f t = minimumUpTo f K))
```

Returning to `O77`, let $p, q, h \in \mathbb{N}$ be residual matrix dimensions and $0 \leq t \leq \min(h, q)$ an integer. The displayed `residualCost` is $(h-t)(q-t)+pt$ on this guarded range. The algorithm instead uses its signed polynomial expression $hq+t^2-(h+q-p)t$ through `residualQuadratic`.

```
def residualCost (p q h t : Nat) : Nat := (h - t) * (q - t) + p * t

def residualQuadratic (p q h : Nat) : Nat → Int :=
  Arithmetic.quadratic ((h : Int) * (q : Int))
    ((h : Int) + (q : Int) - (p : Int))

def residualMin (p q h : Nat) : Nat :=
  (Arithmetic.minimumUpTo (residualQuadratic p q h) (min h q)).toNat

def residualMinimizerList (p q h : Nat) : List Nat :=
  Arithmetic.minimizerList (residualQuadratic p q h) (min h q)

def residualMultiplicity (p q h : Nat) : Nat :=
  (residualMinimizerList p q h).length

def candidateTwiceLambda (I O H r a b : Nat) : Nat :=
  regularDim I O a b + residualMin (O - b) (I - a) (residualHidden H r a b)

def candidateMultiplicity (I O H r a b : Nat) : Nat :=
  residualMultiplicity (O - b) (I - a) (residualHidden H r a b)
```

On the stated finite range the two costs agree and are nonnegative; therefore the `toNat` in `residualMin` preserves the minimum. The list contains every minimizing integer exactly once, so its length is $\mu_0(p, q, h)$, not the logarithmic degree $\mu_0 - 1$. For a feasible rank pair (a, b) and fixed dimensions and target rank, `candidateTwiceLambda` is $s + d_0(p, q, h)$ and `candidateMultiplicity` is $\mu_0(p, q, h)$, where $p = O - b$, $q = I - a$, $h = H + r - a - b$, and $s = Oa + b(I - a)$. Their names identify numerical candidates; the measured equality is asserted separately by `AnswersO77` below.

Minimum locus and attained extrema. In `O77.Native`, threshold minimality uses actual matrix pairs and their measured band predicates. Its comparison point z ranges over the *same fixed fiber* as w. The real numbers `lam`, `mu` and natural numbers `m`, `n` are bound by the displayed existential and universal quantifiers.

```
def IsThresholdMinimizer {I O H : ℕ} (Phi : RM O I) (w : Parameters I O H) : Prop :=
```

```
  w.2*w.1 = Phi ∧ ∃ lam : ℝ, ∃ m : ℕ, HasLocalBandPair Phi w lam m ∧
    ∀ z : Parameters I O H, z.2*z.1 = Phi → ∀ mu : ℝ, ∀ n : ℕ,
      HasLocalBandPair Phi z mu n → lam ≤ mu
```

The closed rank condition lies in `O77`; the closed real-valued threshold lies in `O77.Native`. Both are total functions of natural dimension and rank data. Their use in the contract is guarded by actual fiber membership or feasibility, as appropriate. The `if/else` branches are tested in their displayed order; the final branch is the complementary balanced case.

```
def MinimumRankCondition (I O H r a b : Nat) : Prop :=
  if I+H < O+r then a = r else
  if O+H < I+r then b = r else
  if I+O < H+r then True else
    2*a+O ≤ I+H+r+(I+O+H+r)%2 ∧
    2*b+I ≤ O+H+r+(I+O+H+r)%2

def globalThresholdFormula (I O H r : ℕ) : ℝ :=
  if I+H < O+r then ((H : ℝ)*((I : ℝ)-r)+(O : ℝ)*r)/2 else
  if O+H < I+r then ((H : ℝ)*((O : ℝ)-r)+(I : ℝ)*r)/2 else
  if I+O < H+r then ((O : ℝ)*I)/2 else
    (2*((H : ℝ)+r)*((I : ℝ)+O)-((I : ℝ)-O)^2-((H : ℝ)+r)^2+
      ((I+O+H+r)%2 : ℕ))/8
```

In the threshold formula the leading real casts force the differences $I-r$, $O-r$, $I-O$ and all polynomial arithmetic to be in $\mathbb{R}$. The balanced expression has denominator 8. Its parity term is computed in $\mathbb{N}$ and then coerced into $\mathbb{R}$ by the surrounding sum. These are exactly the closed expressions of Section 2.1.

Supplied singular data and every point of a rung. The following structure is in `O77.RungGeometry0906`. For $I, O, r \in \mathbb{N}$, it stores r positive numbers and two orthonormal families in $\mathbb{R}^O$ and $\mathbb{R}^I$. `StrictAntisigma` means $\sigma_i > \sigma_j$ whenever $i < j$ in `Finr`. For real vectors x, y with the same finite index type, `dotProductxy` is $\sum_\ell x_\ell y_\ell$; the conditional value 1 on $i = j$ and 0 otherwise expresses orthonormality. Thus no balancedness, loss value, or Hessian conclusion is a structure field.

```
structure SingularData (I O r : ℕ) where
  sigma : Fin r → ℝ
  sigma_pos : ∀ j, 0 < sigma j
  decreasing : StrictAnti sigma
  u : Fin r → Fin O → ℝ
  v : Fin r → Fin I → ℝ
  orth_u : ∀ i j, dotProduct (u i) (u j) = if i = j then 1 else 0
  orth_v : ∀ i j, dotProduct (v i) (v j) = if i = j then 1 else 0
```

The next declarations are in its nested namespace `O77.RungGeometry0906.SingularData`, under the source context shown at the beginning of the listing. `RM` and `Parameters` refer to the abbreviations already displayed. The outer product `Matrix.vecMulVecuv` has (i, j)-entry $u_i v_j$, and • multiplies all entries by the real scalar. Lean's index j represents mathematical mode $j + 1$.

```
variable {I O r : ℕ} (D : SingularData I O r)

def «prefix» (k : ℕ) : RM O I :=
  ∑ j : Fin r, if j.val < k then D.sigma j • Matrix.vecMulVec (D.u j) (D.v j) else 0

def target : RM O I := D.«prefix» r

def OnRung {H : ℕ} (k : ℕ) (w : Parameters I O H) : Prop :=
  w.2 * w.1 = D.«prefix» k ∧ ∀ j : Fin r, k ≤ j.val →
    w.1.mulVec (D.v j) = 0 ∧ w.2.transpose.mulVec (D.u j) = 0
```

These definitions make sense for every $k \in \mathbb{N}$; the final theorem restricts to $k \le r$ or $k < r$ explicitly. The condition `k≤j.val` is precisely the discarded-mode condition $j + 1 > k$. `mulVec` is ordinary matrix-vector multiplication, and `transpose.mulVec` gives $B^{\mathsf{T}} u_j$. Hence `OnRung` requires both annihilation conditions in addition to the product equation. When $r = 0$ the families are empty, the target is zero, and the statement about $k < r$ has no instances.

The six-clause contract and the final theorem. In namespace `O77`, the source opens `RM` from `O77.Reconstruction0905` and `Parameters`, `HasLocalBandPair`, `IsThresholdMinimizer`, `globalThresholdFormula` from `O77.Native`. With these name resolutions, the entire contract is:

```
def AnswersO77 {I O H : ℕ} (Phi : RM O I) : Prop :=
  ∀( w : Parameters I O H, w.2*w.1 = Phi →
    RankData I O H Phi.rank w.1.rank w.2.rank ∧
    0 < (candidateTwiceLambda I O H Phi.rank w.1.rank w.2.rank : ℝ)/2 ∧
    ∀ lam : ℝ, ∀ m : ℕ, HasLocalBandPair Phi w lam m ↔
      lam = (candidateTwiceLambda I O H Phi.rank w.1.rank w.2.rank : ℝ)/2 ∧
      m = candidateMultiplicity I O H Phi.rank w.1.rank w.2.rank) ∧
  ∀( a b : ℕ, RankData I O H Phi.rank a b ↔
    ∃ w : Parameters I O H, w.2*w.1 = Phi ∧ w.1.rank = a ∧ w.2.rank = b) ∧
  ∀( w : Parameters I O H, w.2*w.1 = Phi →
    (IsThresholdMinimizer Phi w ↔ MinimumRankCondition I O H Phi.rank w.1.rank w.2.rank)) ∧
  IsLeast {lam : ℝ | ∃ w : Parameters I O H, w.2*w.1 = Phi ∧
    ∃ m : ℕ, HasLocalBandPair Phi w lam m} (globalThresholdFormula I O H Phi.rank) ∧
  IsGreatest {m : ℕ | ∃ w : Parameters I O H, IsThresholdMinimizer Phi w ∧
    ∃ lam : ℝ, HasLocalBandPair Phi w lam m}
    (if (I+O+H+Phi.rank)%2 = 1 ∧ O+Phi.rank ≤ I+H ∧
      I+Phi.rank ≤ O+H ∧ H+Phi.rank ≤ I+O then 2 else 1) ∧
  ∀( D : O77.RungGeometry0906.SingularData I O Phi.rank, D.target = Phi →
    ∀( k : ℕ, k ≤ Phi.rank → ∃ w : Parameters I O H, D.OnRung k w) ∧
    ∀( k : ℕ, k < Phi.rank → ∀ w : Parameters I O H, D.OnRung k w →
      HasLocalBandPair Phi w 1 1))
```

Read its top-level conjunctions in order.

1. For every exact fit, feasibility holds, the proposed exponent is positive, and the measured relation holds *if and only if* its two arguments equal the proposed pair.
2. Every feasible rank pair is realized over the given matrix Φ, and every realized pair is feasible.
3. For every exact fit, semantic threshold minimality is equivalent to the closed rank condition. The premise $BA = \Phi$ precedes the *whole* equivalence; it is not absorbed into only one side.
4. The closed threshold is an attained least element of the set of actual measured exponents on the fiber.
5. The displayed parity value is an attained greatest multiplicity among threshold minimizers. This does not assert that multiplicity is constant on that locus.
6. For every singular datum D representing the fixed target, all rungs $0 \le k \le \operatorname{rank} \Phi$ are inhabited, and every point of every rung $k < \operatorname{rank} \Phi$ has pair $(1, 1)$.

The variable r in the preceding definitions becomes `Phi.rank` throughout this contract. The contract is a proposition for arbitrary dimensions and target; the original problem's domain assumptions are arguments of the following theorem, also in `O77`.

```
theorem answersO77 {I O H : ℕ} (Phi : Matrix (Fin O) (Fin I) ℝ)
    (hI : 0 < I) (hO : 0 < O) (hrH : Phi.rank < H) : AnswersO77 (H := H) Phi :=
  answersO77_of_wishart GlobalSpectralIntegral.realWishartEigenvalueFormula
    Phi hI hO hrH
```

There are precisely three explicit hypotheses: $I > 0$, $O > 0$, and $\operatorname{rank} \Phi < H$. In particular $H > 0$ follows automatically. The named Wishart result occurs in the *proof term*, as a theorem

proved in the development, not as an input to this theorem or a field of `Answers077`. This distinction is visible directly in the statement: there is no `hW` binder. The full proof and its geometric and integral dependencies appear in Part II.

The baseline-rank corollary. The additional assertion that ranks (r, r) attain both extrema is consistent with, but is not a literal additional conjunct of, `Answers077`. Its certified supporting statements are included here so that the natural-language corollary can also be audited. First, in `077`, the numerical baseline is merely evaluation of the candidate at (r, r):

```
def baselineTwiceLambda (I O H r : Nat) : Nat :=
  candidateTwiceLambda I O H r r r
```

The next header, in `077.Native`, uses an explicit measured-pair hypothesis `hPairs`. For the present target that hypothesis is already supplied by the first clause of `Answers077`, by taking `lam` and `m` to equal their candidate values. The rank inequality $\operatorname{rank}\Phi \le H$ follows from the strict hypothesis of the main theorem. Proof bodies are omitted in these two header excerpts only.

```
theorem baseline_attained_of_pairs {I O H : ℕ} (Phi : RM O I)
    (hPairs : ∀ w : Parameters I O H, w.2*w.1 = Phi →
      HasLocalBandPair Phi w
        ((O77.candidateTwiceLambda I O H Phi.rank w.1.rank w.2.rank : ℝ)/2)
        (O77.candidateMultiplicity I O H Phi.rank w.1.rank w.2.rank))
    (hrH : Phi.rank ≤ H) :
    ∃ w : Parameters I O H, w.2*w.1 = Phi ∧
      w.1.rank = Phi.rank ∧ w.2.rank = Phi.rank ∧
      HasLocalBandPair Phi w ((O77.baselineTwiceLambda I O H Phi.rank : ℝ)/2)
        (O77.candidateMultiplicity I O H Phi.rank Phi.rank Phi.rank) ∧
      IsThresholdMinimizer Phi w
```

This gives a point with both ranks $r = \operatorname{rank}\Phi$ that is a threshold minimizer and carries the baseline pair. The following header is in `077`; it computes that pair's multiplicity for any $I, O, H, r \in \mathbb{N}$ satisfying $r \le I, O, H$:

```
theorem baselineMultiplicity_formula {I O H r : Nat}
    (hrI : r ≤ I) (hrO : r ≤ O) (hrH : r ≤ H) :
    candidateMultiplicity I O H r r r =
      if (I+O+H+r)%2 = 1 ∧ O+r ≤ I+H ∧ I+r ≤ O+H ∧ H+r ≤ I+O
      then 2 else 1
```

For $r = \operatorname{rank}\Phi$, the first two bounds follow from the dimensions of Φ and the third from the main hypothesis. The right side is exactly the greatest value in clause 5. Clause 4 and threshold minimality identify the exponent at this point with the attained minimum. Thus the same baseline point attains both extrema, without turning the minimum-threshold locus into a constant-multiplicity locus.

Boundary of the statement audit. Every project symbol in the definition of `Answers077` has now been expanded recursively to standard Lean or Mathlib vocabulary. The recursion concerns the theorem's *meaning*: chart maps, Gram densities and other definitions used only to construct the proof are developed in Part II. The preceding package is therefore enough to inspect the claimed dimensions, measure, loss, quantifiers, finite optimization and rung conditions before reading the analytic proof.

3 The common local-volume calculus

The lemmas in this section apply to the predicate of Definition 2.2. Here and throughout the section, a Euclidean space of dimension $d \in \mathbb{N}$ carries its Euclidean norm and Lebesgue measure; $B_d(w, R)$ denotes its open ball of center w and radius $R > 0$. The dimension $d = 0$ is allowed. Whenever a pair is mentioned, its parameters are $\lambda \in \mathbb{R}$ with $\lambda \geq 0$ and $m \in \mathbb{N}$ with $m \geq 1$. The comparison function is $G_{\lambda,m}(\varepsilon) = \varepsilon^{\lambda}(\log(1/\varepsilon))^{m-1}$ for $0 < \varepsilon < e^{-1}$. All comparison constants below are independent of ε. The arguments use inequalities, finite-dimensional change of variables, and product measure; they do not assume that every analytic germ admits a volume pair.

Lemma 3.1 (Constant rescaling of the small parameter)**.** *For every $c > 0$, $\lambda \geq 0$, and integer $m \geq 1$, there are constants $c_1, c_2, \varepsilon_0 > 0$ such that, for every $0 < \varepsilon < \varepsilon_0$,*

$$c_1 G_{\lambda,m}(\varepsilon) \leq G_{\lambda,m}(\varepsilon/c) \leq c_2 G_{\lambda,m}(\varepsilon).$$

One can choose ε_0 so that both ε and ε/c lie in $(0, e^{-1})$.

Proof. Take $\varepsilon_0 = \min\{e^{-2}, ce^{-2}, e^{-2|\log c|-1}\} > 0$. Then $0 < \varepsilon < \varepsilon_0$ ensures both $\varepsilon, \varepsilon/c < e^{-1}$ and $\log(1/\varepsilon) > 2|\log c|$. The power contributes the constant factor $c^{-\lambda}$, and

$$\tfrac{1}{2}\log(1/\varepsilon) \leq \log(c/\varepsilon) \leq \tfrac{3}{2}\log(1/\varepsilon).$$

Raise these positive quantities to the nonnegative integer $m - 1$ and multiply by $c^{-\lambda}\varepsilon^{\lambda}$. Thus one may use $c_1 = c^{-\lambda}2^{-(m-1)}$ and $c_2 = c^{-\lambda}(3/2)^{m-1}$, after decreasing ε_0 as stated. This also treats $m = 1$, for which there is no logarithmic factor. □

Lemma 3.2 (Windows and bounded densities)**.** *Let $d \in \mathbb{N}$, let f be a continuous real function on an open neighbourhood of $w \in \mathbb{R}^d$, and fix $\lambda \in \mathbb{R}$ and $m \in \mathbb{N}$. Suppose* $\mathsf{BandPair}(f, w; \lambda, m)$ *has witness radius $\delta_0 > 0$. Let $0 < \delta_- \leq \delta_+ < \delta_0$ and let Ω be a measurable set satisfying $B_d(w, \delta_-) \subseteq \Omega \subseteq B_d(w, \delta_+)$. Let $\pi : \Omega \to [0, \infty)$ be measurable, and suppose that there are real constants $0 < a_\pi \leq b_\pi < \infty$ with $a_\pi \leq \pi(z) \leq b_\pi$ for almost every $z \in \Omega$. Then the weighted band volume*

$$W(\varepsilon) = \int_{\Omega} \mathbf{1}_{\{|f(z)-f(w)|<\varepsilon\}}\pi(z)\, dz$$

is bounded above and below by positive constant multiples of $G_{\lambda,m}(\varepsilon)$ for every sufficiently small $\varepsilon > 0$. Here $\mathbf{1}_E$ denotes the indicator of a set E.

Proof. For every $\varepsilon > 0$,

$$a_\pi V_{f,w,\delta_-}(\varepsilon) \leq W(\varepsilon) \leq b_\pi V_{f,w,\delta_+}(\varepsilon).$$

Use the lower band estimate for radius δ_- and the upper one for radius δ_+, and take the minimum of their two admissible ε thresholds. More generally, to establish the radius-first predicate it suffices to perform such a comparison for each sufficiently small ambient ball, using inner and outer product windows fixed before ε varies. This last observation is simply the quantifier order in Definition 2.2. □

Lemma 3.3 (Comparability of nonnegative germs)**.** *Let $d \in \mathbb{N}$ and $w \in \mathbb{R}^d$. Let f, g be continuous real functions on an open neighbourhood Ω of w. Suppose that $f(w) = g(w) = 0$, that $f, g \geq 0$ on Ω, and that there are constants $c, C > 0$ with $c\, g(z) \leq f(z) \leq C\, g(z)$ for every $z \in \Omega$. Then, for every $\lambda \geq 0$ and integer $m \geq 1$,*

$$\mathsf{BandPair}(g, w; \lambda, m) \iff \mathsf{BandPair}(f, w; \lambda, m).$$

Proof. Fix a ball about w contained in Ω and below the witness radius for the known pair. On that ball, for every $\varepsilon > 0$,

$$\{g < \varepsilon/C\} \subseteq \{f < \varepsilon\} \subseteq \{g < \varepsilon/c\}.$$

All three sets in this display are intersected with that ball. If the known pair for g has cutoff ε_δ at the chosen radius, restrict ε below $\min\{c, C\}\varepsilon_\delta$ as well as the two cutoffs from Lemma 3.1. The two inclusions then give the required lower and upper estimates at this radius. Since the radius was arbitrary below the common neighbourhood and witness radii, they prove the radius-first assertion. The same argument with $C^{-1}f \le g \le c^{-1}f$ proves the converse. For signed functions one may apply this statement to the centered absolute values $z \mapsto |f(z) - f(w)|$ and $z \mapsto |g(z) - g(w)|$, provided these nonnegative functions satisfy the stated comparison. □

Lemma 3.4 (Local C^1 coordinate invariance)**.** *Let $d \in \mathbb{N}$, let $\mathcal{U}, \mathcal{V} \subseteq \mathbb{R}^d$ be open, and let $\psi : \mathcal{U} \to \mathcal{V}$ be a C^1 diffeomorphism. Fix $w' \in \mathcal{U}$, set $w = \psi(w')$, and let $f : \mathcal{V} \to \mathbb{R}$ be continuous. Then, for every $\lambda \ge 0$ and integer $m \ge 1$,*

$$\mathsf{BandPair}(f, w; \lambda, m) \iff \mathsf{BandPair}(f \circ \psi, w'; \lambda, m).$$

Both predicates use the Lebesgue measures in their respective Euclidean coordinates.

Proof. Continuity of the derivatives and invertibility at the two base points allow us to choose neighbourhoods on which the absolute Jacobian determinants of ψ and ψ^{-1} have finite positive upper and lower bounds. Choose these neighbourhoods before choosing a ball radius. For each sufficiently small $\delta > 0$, the image $\Omega_\delta = \psi(B_d(w', \delta))$ contains a ball about w and is contained in a ball about w of radius smaller than the witness radius for $\mathsf{BandPair}(f, w; \lambda, m)$. The inner inclusion follows from continuity of ψ^{-1} at w; the outer inclusion follows from continuity of ψ at w'.

Change variables in the band indicator on $B_d(w', \delta)$. Its integral becomes the weighted band volume on Ω_δ with density $z \mapsto |\det D\psi^{-1}(z)|$. Apply Lemma 3.2. Its constants and inner radius may depend on δ, exactly as permitted by the radius-first predicate. Repeat with ψ^{-1} for the reverse implication. In dimension zero, the determinant and the mass of the unique point both equal one. □

Exact-fit basis changes and positive levels. Fix dimensions $I, O, H \in \mathbb{N}$ with $I, O, H \ge 1$. For matrices $A \in \mathbb{R}^{H \times I}$, $B \in \mathbb{R}^{O \times H}$, and $\Phi \in \mathbb{R}^{O \times I}$, fix invertible real matrices G_I, G_H, G_O of sizes I, H, O, respectively. The transformed factors and target are

$$A' = G_H A G_I^{-1}, \qquad B' = G_O B G_H^{-1}, \qquad \Phi' = G_O \Phi G_I^{-1}.$$

Their errors satisfy $B'A' - \Phi' = G_O(BA - \Phi)G_I^{-1}$. The inequality $\|PEQ\|_F \le \|P\|_{\mathrm{op}}\|E\|_F\|Q\|_{\mathrm{op}}$ applied first to $G_O(BA - \Phi)G_I^{-1}$ and then to $G_O^{-1}(B'A' - \Phi')G_I$ gives

$$\frac{\|BA - \Phi\|_F^2}{\|G_O^{-1}\|_{\mathrm{op}}^2 \|G_I\|_{\mathrm{op}}^2} \le \|B'A' - \Phi'\|_F^2 \le \|G_O\|_{\mathrm{op}}^2 \|G_I^{-1}\|_{\mathrm{op}}^2 \|BA - \Phi\|_F^2.$$

All four operator norms are finite and positive, since the matrices are invertible in positive dimensions. At an exact fit both losses vanish. Apply Lemma 3.3 to the two losses expressed in the same parameter coordinates, then Lemma 3.4 to the invertible parameter transformation. This preserves the pair. The parameter transformation has its usual positive constant absolute Jacobian determinant; no volume-preservation assertion is needed.

At a positive-loss center, norm equivalence does not compare the centered scalar losses. A determinant-one nonorthogonal output shear can change their multiplicity, as recorded below

in the counterexamples. This restriction concerns changing the scalar loss through its output metric. Precomposing one fixed scalar loss with an actual coordinate diffeomorphism is covered at every center by Lemma 3.4. The fiber proof applies the additional error comparison only at exact fits; the saddle proof retains the original loss and uses orthogonal coordinates.

Lemma 3.5 (Free coordinates)**.** *Let $d, e \in \mathbb{N}$, let f be a continuous real-valued function on an open neighbourhood of $w \in \mathbb{R}^d$, and fix $v_* \in \mathbb{R}^e$, $\lambda \in \mathbb{R}$, and $m \in \mathbb{N}$. If* $\mathsf{BandPair}(f, w; \lambda, m)$*, then the function $F(z, v) = f(z)$ satisfies* $\mathsf{BandPair}(F, (w, v_*); \lambda, m)$ *in $\mathbb{R}^d \times \mathbb{R}^e$.*

Proof. For each $R > 0$, the product of the two balls of radius $R/2$ about w and v_* is contained in the ambient product-space ball of radius R, which is contained in the product of the two balls of radius R. On each product window, Tonelli's theorem makes the band volume equal to the band volume in the z variables times the positive finite volume of the fixed v ball. For every sufficiently small R, both z radii lie below the assumed witness radius. The two comparisons therefore establish the asserted pair. A dimension-zero factor has volume one. □

Lemma 3.6 (Adding separated quadratic directions)**.** *Let $d, s \in \mathbb{N}$, and let g be a continuous nonnegative real function on a neighbourhood of $0 \in \mathbb{R}^d$, with $g(0) = 0$. Suppose* $\mathsf{BandPair}(g, 0; \lambda, m)$*, where $\lambda \geq 0$ and $m \geq 1$ is an integer. For $(u, y) \in \mathbb{R}^s \times \mathbb{R}^d$ near $(0, 0)$, define $F(u, y) = \|u\|^2 + g(y)$. Then*

$$\mathsf{BandPair}(F, (0, 0); \lambda + s/2, m).$$

Proof. For $s = 0$ the assertion is the assumed pair. Suppose $s > 0$, and choose positive radii R, ρ with ρ below the witness radius for g. Let ω_s be the Lebesgue volume of $B_s(0, 1)$, and put $V_{g,\rho}(\varepsilon) = \mathrm{vol}\{y \in B_d(0, \rho) : g(y) < \varepsilon\}$. For $0 < \varepsilon < R^2$, in the fixed product window $B_s(0, R) \times B_d(0, \rho)$, nonnegativity of both summands gives

$$\begin{aligned} \omega_s(\varepsilon/2)^{s/2} V_{g,\rho}(\varepsilon/2) &\leq \mathrm{vol}\{(u, y) : \|u\|^2 + g(y) < \varepsilon\} \\ &\leq \omega_s \varepsilon^{s/2} V_{g,\rho}(\varepsilon). \end{aligned}$$

The set in the middle is restricted to the stated product window. For the lower bound use the product of $\{\|u\|^2 < \varepsilon/2\}$ and $\{g(y) < \varepsilon/2\}$; for the upper bound use the corresponding bounds with ε. Lemma 3.1 now gives the power $\varepsilon^{\lambda + s/2}$ and the same logarithmic degree $m - 1$. For each sufficiently small ambient radius, choose inner and outer product windows as in Lemma 3.5; this establishes every radius required by the predicate. If g is identically zero near the origin, its pair is $(0, 1)$ and the same argument applies. □

4 Rank geometry and canonical coordinates

In this section let $I, O, H \in \mathbb{N}$ and fix a target $\Phi \in \mathbb{R}^{O \times I}$ of rank $r \leq H$. For every pair of proposed factor ranks $a, b \in \mathbb{N}$, "feasible" means the inequalities in Definition 2.5. When these hold, write

$$\alpha = a - r, \quad \beta = b - r, \quad p = O - b, \quad q = I - a, \quad h = H + r - a - b, \quad s = Oa + bq.$$

All six block sizes $r, \alpha, \beta, h, p, q$ are nonnegative integers. The equalities $I = r + \alpha + q$, $H = r + \alpha + \beta + h$, and $O = r + \beta + p$ will specify the decompositions used below. A subscript on one of V, W, U in a direct-sum decomposition denotes its dimension.

Lemma 4.1 (Feasibility and realization)**.** *For every $a, b \in \mathbb{N}$, there exist matrices $A \in \mathbb{R}^{H \times I}$ and $B \in \mathbb{R}^{O \times H}$ with $BA = \Phi$, $\operatorname{rank} A = a$, and $\operatorname{rank} B = b$ if and only if (a, b) is feasible.*

Proof. The product-rank and matrix-size bounds give all the required inequalities except $a+b \le H+r$. Regard A and B as linear maps $\mathbb{R}^I \to \mathbb{R}^H \to \mathbb{R}^O$. The restriction of B to $\operatorname{im} A$ has image $\operatorname{im}\Phi$ and rank r. Rank-nullity therefore gives

$$\dim(\operatorname{im} A \cap \ker B) = a - r \le \dim \ker B = H - b,$$

which is the missing inequality.

Conversely, suppose (a, b) is feasible. Split input coordinates into blocks of dimensions (r, α, q), hidden coordinates into (r, α, β, h), and output coordinates into (r, β, p). Define a canonical pair (A_*, B_*) by making A_* the identity from the first two input blocks to their respective hidden copies and making B_* the identity from the first and third hidden blocks to their respective output copies, with all other blocks zero. Then the factor ranks are a and b, and $\Phi_* = B_* A_*$ is the standard rank-r matrix. To realize the prescribed target, choose vectors $v_1, \dots, v_r \in \mathbb{R}^I$ whose classes form a basis of $\mathbb{R}^I / \ker\Phi$, and complete them by a basis of $\ker\Phi$. Then $\Phi v_1, \dots, \Phi v_r$ form a basis of $\operatorname{im}\Phi$; complete these vectors to a basis of $\mathbb{R}^O$. Let G_I, G_O have the respective basis vectors as columns. They are invertible, and $\Phi G_I = G_O \Phi_*$ holds column by column: its first r columns are Φv_i, and the remaining columns are zero. Consequently $\Phi = G_O \Phi_* G_I^{-1}$. The matrices $A = A_* G_I^{-1}$ and $B = G_O B_*$ now satisfy $BA = \Phi$. Multiplication by an invertible matrix preserves rank, so their ranks are a, b. Empty bases and empty identity matrices give the same construction when a dimension or r is zero. □

Lemma 4.2 (Canonical form of a pair)**.** *Let $A \in \mathbb{R}^{H\times I}$ and $B \in \mathbb{R}^{O\times H}$ satisfy $BA = \Phi$, and put $a = \operatorname{rank} A$, $b = \operatorname{rank} B$. With the block sizes just defined, there are bases giving decompositions*

$$\begin{aligned} \mathbb{R}^I &= V_r \oplus V_\alpha \oplus V_q, \\ \mathbb{R}^H &= W_r \oplus W_\alpha \oplus W_\beta \oplus W_h, \\ \mathbb{R}^O &= U_r \oplus U_\beta \oplus U_p, \end{aligned}$$

in which A and B have the canonical matrices (A_*, B_*) of Lemma 4.1. In particular, Φ_* is the identity $V_r \to U_r$ and zero on the other input blocks. Consequently the three ranks $(\operatorname{rank} A, \operatorname{rank} B, \operatorname{rank}(BA))$ classify the orbits of matrix pairs under

$$(G_O, G_H, G_I) \cdot (A, B) = (G_H A G_I^{-1}, G_O B G_H^{-1}),$$

where $G_O \in \mathrm{GL}_O(\mathbb{R})$, $G_H \in \mathrm{GL}_H(\mathbb{R})$, and $G_I \in \mathrm{GL}_I(\mathbb{R})$.

Proof. Set $W_\alpha = \operatorname{im} A \cap \ker B$ and choose a complement W_r in $\operatorname{im} A$. The restriction of B identifies W_r with $U_r = \operatorname{im}\Phi$. Choose a complement U_β of U_r in $\operatorname{im} B$ and lift a basis of U_β through B; its span is a subspace W_β on which B is an isomorphism onto U_β. Choose a complement W_h of W_α in $\ker B$. The four hidden summands are independent: applying B to a vanishing sum first kills its W_r and W_β components, and the chosen complement inside $\ker B$ kills the remaining two. Their dimensions sum to H, so they exhaust the hidden space.

Lift bases of W_r and W_α through A to obtain V_r and V_α, and set $V_q = \ker A$. These are independent and exhaust the input by rank-nullity. Choose U_p as a complement of $\operatorname{im} B$ in the output. Use the lifted and image bases so that the four indicated isomorphisms have identity matrices. This gives the canonical pair. The three ranks are invariant under the displayed action, and the construction assigns the same canonical pair to any two pairs having these ranks, which proves the orbit assertion. □

Lemma 4.3 (Dimension identities and the derivative)**.** *Let $A \in \mathbb{R}^{H\times I}$, $B \in \mathbb{R}^{O\times H}$ have ranks a, b, and let $r = \operatorname{rank}(BA)$; define the six block sizes as above. For the multiplication map $\mathcal{P}(A, B) = BA$, its derivative is*

$$D\mathcal{P}_{(A,B)}(\dot A, \dot B) = B\dot A + \dot B A \qquad (\dot A \in \mathbb{R}^{H\times I},\ \dot B \in \mathbb{R}^{O\times H}).$$

This linear map has rank $s = Oa + Ib - ab = Oa + bq$. In the canonical coordinates its image consists of all $O \times I$ matrices whose bottom-right $p \times q$ block is zero. The integer

$$g = a^2 + (H - a)a + \alpha q + O\beta + bh$$

is nonnegative and satisfies $s + ph + hq + g = H(I + O)$.

Proof. The bilinear product rule gives the derivative formula. The term $B\dot{A}$ ranges over all matrices whose columns lie in $\operatorname{im} B$; this space has dimension bI. The term $\dot{B}A$ ranges over all matrices whose rows lie in the row space of A; this space has dimension Oa. In the canonical coordinates these are, respectively, the matrices supported in the first b rows and those supported in the first a columns. Their intersection is an arbitrary $b \times a$ block, of dimension ab. This proves the rank and image assertions. Every summand defining g is a nonnegative block size. Substituting $I = a + q$, $O = b + p$, $H = a + \beta + h$, $a = r + \alpha$, and $b = r + \beta$ proves the total-dimension identity. □

5 The explicit depth-two elimination chart

Fix nonnegative integers $r, \alpha, \beta, h, p, q$ for this section, and set

$$a = r + \alpha, \quad b = r + \beta, \quad I = a + q, \quad H = a + \beta + h, \quad O = b + p.$$

Let (A_*, B_*) and $\Phi_* = B_*A_*$ be the canonical pair and target of Lemma 4.2. The variables $A \in \mathbb{R}^{H \times I}$ and $B \in \mathbb{R}^{O \times H}$ will vary near this pair; their nearby ranks are not assumed constant. Put $s = Oa + bq$ and let g have the value in Lemma 4.3. Every block below uses the fixed coordinate splittings determined by these integers. All block products and Frobenius norms use real entries. The formulas include zero-size blocks: I_0 denotes the empty identity, $\det I_0 = 1$, and empty sums of squares are zero.

Lemma 5.1 (Left gauge and its inverse)**.** *Write $A = \left(\begin{smallmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{smallmatrix}\right)$ with row sizes $(a, H - a)$ and column sizes (a, q). On the open set $\det A_{11} \neq 0$, define*

$$L(A) = \begin{pmatrix} A_{11}^{-1} & 0 \\ -A_{21}A_{11}^{-1} & I_{H-a} \end{pmatrix}, \qquad L(A)^{-1} = \begin{pmatrix} A_{11} & 0 \\ A_{21} & I_{H-a} \end{pmatrix},$$

$$X = A_{11}^{-1}A_{12} \in \mathbb{R}^{a \times q}, \qquad S = A_{22} - A_{21}X \in \mathbb{R}^{H-a \times q},$$

$$\bar{A} = L(A)A = \begin{pmatrix} I_a & X \\ 0 & S \end{pmatrix}, \qquad \bar{B} = BL(A)^{-1} \in \mathbb{R}^{O \times H}.$$

The map $(A, B) \mapsto (A_{11}, A_{21}, X, S, \bar{B})$ is an analytic coordinate change on this open set, and $\bar{B}\bar{A} = BA$. Its inverse is $A_{12} = A_{11}X$, $A_{22} = S + A_{21}X$, and $B = \bar{B}L(A)$, with A_{11} and A_{21} retained unchanged.

Proof. Multiply the two displayed $H \times H$ matrices in both orders to verify the inverse. Multiplication by A gives the displayed value of $\bar{A}$. Substitution of A_{12} and A_{22} verifies both coordinate round trips; associativity gives $\bar{B}\bar{A} = BA$. Matrix inversion is analytic on the nonzero-determinant open set, since its entries are cofactors divided by the determinant. At the canonical pair $A_{11} = I_a$, so this set contains the base point, also when $a = 0$. □

Lemma 5.2 (Exact error expansion)**.** *On the domain of Lemma 5.1, write $\bar{B} = (B_I \mid D \mid Y)$ with column sizes (a, β, h). Thus $B_I \in \mathbb{R}^{O \times a}$, $D \in \mathbb{R}^{O \times \beta}$, and $Y \in \mathbb{R}^{O \times h}$. Split*

$$X = \begin{pmatrix} X_R \\ X_P \end{pmatrix}, \qquad S = \begin{pmatrix} S_Q \\ S_Z \end{pmatrix},$$

where the row sizes are (r, α) for X and (β, h) for S. In particular $S_Z \in \mathbb{R}^{h\times q}$. Let $C_I \in \mathbb{R}^{O\times a}$ consist of the first a columns of Φ_*, and let $J \in \mathbb{R}^{O\times r}$ be the matrix of the first-coordinate inclusion $\mathbb{R}^r \hookrightarrow \mathbb{R}^O$. Define

$$U = B_I - C_I \in \mathbb{R}^{O\times a}, \qquad P(D) = (J \mid D) \in \mathbb{R}^{O\times b}, \qquad T = \begin{pmatrix} X_R \\ S_Q \end{pmatrix} \in \mathbb{R}^{b\times q}.$$

Then

$$BA - \Phi_* = \big(U \mid UX + P(D)T + YS_Z\big). \tag{5}$$

The identity holds for every pair on the gauge domain, including pairs outside the exact-fit fiber.

Proof. The block product is $\bar{B}\bar{A} = (B_I \mid B_I X + DS_Q + YS_Z)$. Subtract $\Phi_* = (C_I \mid 0)$, use $C_I X = JX_R$, and combine $JX_R + DS_Q = P(D)T$. Each right-hand block has size $O \times q$. □

Lemma 5.3 (Output normalization and triangular shear)**.** *With the notation of Lemma 5.2, let $P_t \in \mathbb{R}^{b\times b}$ and $P_b \in \mathbb{R}^{p\times b}$ be the top and bottom row blocks of $P(D)$. Let $D_* \in \mathbb{R}^{O\times\beta}$ be the corresponding block of B_*. For $D = D_*$ one has $P_t = I_b$ and $P_b = 0$. On the additional open condition $\det P_t \neq 0$, define*

$$R(D) = \begin{pmatrix} P_t^{-1} & 0 \\ -P_b P_t^{-1} & I_p \end{pmatrix}, \qquad R(D)^{-1} = \begin{pmatrix} P_t & 0 \\ P_b & I_p \end{pmatrix}.$$

These are $O\times O$ matrices and $R(D)P(D) = \binom{I_b}{0}$. Define $Y_1 \in \mathbb{R}^{b\times h}$, $Y_0 \in \mathbb{R}^{p\times h}$, and $T' \in \mathbb{R}^{b\times q}$ by

$$R(D)Y = \begin{pmatrix} Y_1 \\ Y_0 \end{pmatrix}, \qquad T' = T + Y_1 S_Z.$$

The change of variables retains D, S_Z, Y_1 and is inverted by $T = T' - Y_1 S_Z$ and $Y = R(D)^{-1}\binom{Y_1}{Y_0}$. Writing $(BA - \Phi_*)_{\text{right}}$ for the last q columns of the product error, one has

$$R(D)\big((BA - \Phi_*)_{\text{right}} - UX\big) = \begin{pmatrix} T' \\ Y_0 S_Z \end{pmatrix}. \tag{6}$$

Proof. Block multiplication verifies both inverses and $R(D)P(D) = \binom{I_b}{0}$. Apply $R(D)$ to the right-hand part of (5) after subtracting UX; its two row blocks are $T + Y_1 S_Z$ and $Y_0 S_Z$. The change from T to T' is a triangular shear because Y_1 and S_Z remain coordinates, so the stated subtraction is its inverse. □

Proposition 5.4 (Chart and explicit inverse)**.** *There are open neighbourhoods $\mathcal{U}$ of (A_*, B_*) in $\mathbb{R}^{H\times I} \times \mathbb{R}^{O\times H}$ and $\mathcal{V}$ of zero in the product of nine matrix spaces*

$$\begin{aligned} &\mathbb{R}^{O\times a} \times \mathbb{R}^{b\times q} \times \mathbb{R}^{p\times h} \times \mathbb{R}^{h\times q} \\ &\quad \times \mathbb{R}^{a\times a} \times \mathbb{R}^{H-a\times a} \times \mathbb{R}^{\alpha\times q} \times \mathbb{R}^{O\times\beta} \times \mathbb{R}^{b\times h}. \end{aligned}$$

such that the following map is an analytic diffeomorphism $\Psi : \mathcal{U} \to \mathcal{V}$:

$$\Psi(A, B) = \big(U, T', Y_0, S_Z,\ A_{11} - I_a, A_{21}, X_P, D - D_*, Y_1\big).$$

It maps (A_*, B_*) to zero. The first two blocks contain s regular coordinates, the next two are residual factors of sizes $p \times h$ and $h \times q$, and the last five contain g further coordinates.

Proof. Given the nine output blocks, recover A_{11} and D by restoring the constants I_a, D_*. Compute $P_t, P_b, R(D)$ from D. Then reconstruct

$$T = T' - Y_1 S_Z, \qquad X = \begin{pmatrix} X_R \\ X_P \end{pmatrix}, \qquad S = \begin{pmatrix} S_Q \\ S_Z \end{pmatrix}, \qquad B_I = U + C_I, \qquad Y = R(D)^{-1} \begin{pmatrix} Y_1 \\ Y_0 \end{pmatrix},$$

where X_R and S_Q are the first r and last β rows of T. Finally put

$$A_{12} = A_{11} X, \quad A_{22} = S + A_{21} X, \quad \bar{B} = (B_I \mid D \mid Y), \quad B = \bar{B} L(A).$$

These formulas are analytic and inverse to Ψ wherever the two determinants $\det A_{11}$ and $\det P_t(D)$ are nonzero. The conditions are open in both coordinate descriptions and both determinants equal one at the canonical point. At that point all nine centered blocks vanish. The block counts are $Oa + bq = s$, $ph + hq$, and $a^2 + (H - a)a + \alpha q + O\beta + bh = g$; their sum is the original ambient dimension by Lemma 4.3. Thus no ambient coordinates have been discarded. □

Lemma 5.5 (Two-sided norm comparison)**.** *There is an open neighbourhood $\mathcal{U}_0 \subseteq \mathcal{U}$ of (A_*, B_*) such that every $(A, B) \in \mathcal{U}_0$ satisfies*

$$\frac{1}{12}\big(\|u\|^2 + \|Y_0 S_Z\|_F^2\big) \le 2L_{\Phi_*}(A, B) \le 6\big(\|u\|^2 + \|Y_0 S_Z\|_F^2\big),$$

where $u = (U, T')$ and $\|u\|^2 = \|U\|_F^2 + \|T'\|_F^2$. The constants apply uniformly to all nine coordinate blocks in this neighbourhood.

Proof. For a pair in the chart domain put $V = \binom{T'}{Y_0 S_Z} \in \mathbb{R}^{O \times q}$. Equations (5)–(6) give

$$2L_{\Phi_*}(A, B) = \|U\|_F^2 + \|UX + R(D)^{-1} V\|_F^2, \qquad \|V\|_F^2 = \|T'\|_F^2 + \|Y_0 S_Z\|_F^2.$$

At the base point $X = 0$, $P_t = I_b$, and $P_b = 0$. By continuity, shrink the neighbourhood so that $\|X\|_{\mathrm{op}} < 1/2$, $\|P_t - I_b\|_{\mathrm{op}} < 1/2$, and $\|P_b\|_{\mathrm{op}} < 1/2$ throughout it. The Neumann bound gives $\|P_t^{-1}\|_{\mathrm{op}} \le 2$. For $v_1 \in \mathbb{R}^b$ and $v_2 \in \mathbb{R}^p$, the block formulas imply

$$\|R(D)(v_1, v_2)\|^2 \le 6\|v_1\|^2 + 2\|v_2\|^2 \le 6(\|v_1\|^2 + \|v_2\|^2),$$
$$\|R(D)^{-1}(v_1, v_2)\|^2 \le \tfrac{11}{4}\|v_1\|^2 + 2\|v_2\|^2 \le 3(\|v_1\|^2 + \|v_2\|^2).$$

Here $\|P_t\|_{\mathrm{op}} \le 3/2$ and $\|x + y\|^2 \le 2\|x\|^2 + 2\|y\|^2$ give the second bound. Applying these inequalities column by column yields

$$\|UX\|_F \le \tfrac{1}{2}\|U\|_F, \quad \|R(D)^{-1} V\|_F \le \sqrt{3}\|V\|_F, \quad \|R(D)^{-1} V\|_F \ge \|V\|_F / \sqrt{6}.$$

Consequently the upper estimate for the loss is $\frac{3}{2}\|U\|_F^2 + 6\|V\|_F^2 \le 6(\|U\|_F^2 + \|V\|_F^2)$. For the lower estimate use $\|x + y\|^2 \ge \frac{1}{2}\|y\|^2 - \|x\|^2$, giving $2L_{\Phi_*}(A, B) \ge \frac{3}{4}\|U\|_F^2 + \frac{1}{12}\|V\|_F^2$. These are the asserted bounds. □

From the chart to native volume. The model function in the chart is $N = \|U\|_F^2 + \|T'\|_F^2 + \|Y_0 S_Z\|_F^2$; it is independent of the last five coordinate blocks. This independence is an assertion about N, rather than about the original loss. Regrouping and numbering the entries in the nine blocks preserves product Lebesgue measure. The nonlinear inverse Ψ^{-1} has instead the density $|\det D\Psi^{-1}|$, which is bounded above and away from zero on a sufficiently small common neighbourhood by Lemma 3.4. Choose that neighbourhood together with the loss bounds before choosing an ambient radius. Inner and outer windows then give both volume inequalities for each sufficiently small fixed radius, with constants depending on that radius.

Lemmas 3.3, 3.5, and 3.6 therefore give the following implication: if the residual function $(Y_0, S_Z) \mapsto \|Y_0 S_Z\|_F^2$ has pair (λ_0, m_0) at $(0,0)$, where $\lambda_0 \geq 0$ and $m_0 \geq 1$ is an integer, then L_{Φ_*} has pair $(\lambda_0 + s/2, m_0)$ at (A_*, B_*). The Lean implementation of this step is `O77.Canonical0906.canonical_volume_sandwich` and `O77.Canonical0906.canonical_pair_of_residual`; its intermediate norm comparison uses positive uniform constants without fixing the particular numerical bounds above.

6 The exact Gaussian Gram and eigenvalue identities

6.1 The integrals and their measures

Throughout this section, $k, d \in \mathbb{N}$ satisfy $k \leq d$; the parameters ρ, T are real and nonnegative unless another domain is stated explicitly. The symbol I_k denotes the $k \times k$ identity matrix. For these dimensions and parameters define

$$\eta_{k,d} = \frac{d-k-1}{2}, \qquad G_{k,d}(\rho, T) = \int_{\mathbb{R}^{k\times d}} e^{-\|X\|_F^2} \det(I_k + TXX^{\mathsf{T}})^{-\rho}\, dX.$$

The exponent uses real subtraction after casting k, d to $\mathbb{R}$, and dX is the product of Lebesgue measures in the kd entries of X. All sums and products indexed by i in this section run from 1 to k; those indexed by $i < j$ run over $1 \leq i < j \leq k$. Define the strict ordered positive chamber

$$\mathcal{C}_k = \{s = (s_1, \ldots, s_k) \in \mathbb{R}^k : 0 < s_1 < \cdots < s_k\}.$$

For any real exponent η, define on this chamber the density

$$\mathcal{L}_{\eta,\rho,T}(s) = e^{-\sum_i s_i} \prod_i s_i^{\eta} \prod_{i<j} (s_j - s_i) \prod_i (1 + Ts_i)^{-\rho}.$$

For $s \in [0,\infty)^k$, let $s^{\uparrow}$ be its nondecreasing rearrangement. The same formula for $\mathcal{L}_{\eta,\rho,T}$ on weakly ordered tuples is interpreted using total real powers. With ds the product Lebesgue measure on $\mathbb{R}^k$, the implemented integral is

$$J_k(\eta, \rho, T) = \int_{[0,\infty)^k}^{-} \mathrm{ofReal}\big(\mathcal{L}_{\eta,\rho,T}(s^{\uparrow})\big)\, ds.$$

Here $\mathrm{ofReal}(x) = \max(x, 0)$, viewed in $[0,\infty]$, and $\int^-$ denotes a nonnegative extended-real integral. In this section nonnegative integral identities are first understood in $[0,\infty]$; they are read as ordinary real identities only after finiteness has been established. For $G_{k,d}$, finiteness already follows from

$$0 < e^{-\|X\|_F^2} \det(I_k + TXX^{\mathsf{T}})^{-\rho} \leq e^{-\|X\|_F^2},$$

because XX^{T} is positive semidefinite and $\rho, T \geq 0$. The sorted density is measurable. Order the finitely many coordinate permutations and use the first one that sorts each tuple. The sets on which each permutation is selected are Borel, since they are defined by finitely many coordinate inequalities and exclusions. On each such set, sorting is a fixed coordinate permutation. Exponentials, finite sums and products, and the total real-power functions are Borel, so the displayed integrand is Borel as well. The displayed density extends to the boundary using the library's total real-power convention. This boundary convention has no effect: coordinate-zero and coordinate-tie hyperplanes are null, and away from them the sorted density equals

$$e^{-\sum_i s_i} \prod_i s_i^{\eta} \prod_{i<j} |s_i - s_j| \prod_i (1 + Ts_i)^{-\rho}.$$

This equality is the formal sorted/absolute-density adapter; it is not a redefinition of `laguerre OrthantLIntegral`. For $k = 0$ the density is the constant one on the single empty tuple; hence these particular empty-coordinate integrals equal one.

For $x > 0$, the Euler Gamma function is $\Gamma(x) = \int_0^\infty e^{-t} t^{x-1}\, dt$; this integral is positive and finite. Only positive arguments of Γ occur below.

Theorem 6.1 (Exact real Wishart identity)**.** *For every $k, d \in \mathbb{N}$ with $k \le d$, there exists a real constant $C_{k,d} > 0$, independent of both test parameters, such that for every $\rho, T \in [0, \infty)$,*

$$G_{k,d}(\rho, T) = C_{k,d} J_k(\eta_{k,d}, \rho, T).$$

The same constant gives the right-oriented identity on $Z \in \mathbb{R}^{d\times k}$ with integrand $e^{-\|Z\|_F^2} \det(I_d + TZZ^{\mathsf{T}})^{-\rho}$. For $k = 0$, $C_{0,d} = 1$. For $k = 1 \le d$, $C_{1,d} = \pi^{d/2}/\Gamma(d/2)$.

This is a classical Gaussian eigenvalue formula; one published normalization is [10, Theorem 3.1 and equations (3)–(4), (24)]. The proof below gives the same identity in the exact entry-coordinate measures used by the formal development. It includes the square case $k = d$, where $\eta_{k,d} = -1/2$.

6.2 The Gaussian Gram pushforward

Let $\mathrm{Sym}_k(\mathbb{R}) = \{S \in \mathbb{R}^{k\times k} : S^{\mathsf{T}} = S\}$. Identify this real vector space with the independent entries $(S_{ij})_{1\le i\le j\le k}$, and let dS be product Lebesgue measure in those $k(k+1)/2$ coordinates. This is `upperLebesgue`, not the Frobenius-normalized volume on the symmetric subspace: an off-diagonal symmetric coordinate vector has Frobenius norm $\sqrt{2}$. Define

$$\mathcal{P}_k = \{S \in \mathrm{Sym}_k(\mathbb{R}) : x^{\mathsf{T}} S x > 0 \text{ for every } 0 \ne x \in \mathbb{R}^k\}.$$

The notation $S > 0$ below means $S \in \mathcal{P}_k$. For every Borel measurable test function $F : \mathrm{Sym}_k(\mathbb{R}) \to [0, \infty]$, including tests taking the value ∞, we have

$$\int_{\mathbb{R}^{k\times d}} e^{-\|X\|_F^2} F(XX^{\mathsf{T}})\, dX = A_{k,d} \int_{\mathcal{P}_k} e^{-\operatorname{tr} S} \det(S)^{\eta_{k,d}} F(S)\, dS, \tag{7}$$

where $\operatorname{tr} S = \sum_{i=1}^k S_{ii}$ is the trace and

$$A_{k,d} = \prod_{i=0}^{k-1} \frac{\pi^{(d-i)/2}}{\Gamma((d-i)/2)} > 0.$$

The identity holds as a nonnegative integral identity, without a preliminary integrability assumption on F. The Lean theorem permits Borel tests on all square matrices. The two formulations agree: restricting such a test to symmetric matrices gives this statement, and a Borel test on the symmetric subspace extends measurably to all matrices by composition with $X \mapsto (X + X^{\mathsf{T}})/2$. For $k = 0$ the product defining $A_{k,d}$ is empty and equals one; every Gamma argument in a nonempty product is positive because $i < k \le d$.

We prove (7) by induction on the number of rows, with $d \in \mathbb{N}$ fixed. For zero rows the matrix space and the symmetric cone each consist of the empty matrix, of measure one; trace is zero and determinant is one. Both sides are therefore $F(0)$.

For the induction step suppose $0 \le k < d$, and fix a full-row-rank matrix $X \in \mathbb{R}^{k\times d}$. Put $B = XX^{\mathsf{T}} > 0$. For a new row $y \in \mathbb{R}^{1\times d}$ set $z = Xy^{\mathsf{T}}$. The orthogonal projection of y^{T} onto the row space of X is $X^{\mathsf{T}} B^{-1} z$: applying X gives z, and its difference from y^{T} lies in $\ker X$, the orthogonal complement of the row space. Its squared norm is $z^{\mathsf{T}} B^{-1} z$. Choose an orthonormal basis of the row space and express the restriction of X in that basis by a

square matrix R. Then $RR^{\mathsf{T}} = B$, so $|\det R| = \det(B)^{1/2}$. Thus the change from parallel orthonormal coordinates to z contributes $\det(B)^{-1/2}$ to dy.

The perpendicular space has dimension $d-k \geq 1$. Polar integration there, followed by $t = \|y_\perp\|^2$, gives

$$\int_{\mathbb{R}^{d-k}} f(\|v\|^2)\,dv = \frac{\pi^{(d-k)/2}}{\Gamma((d-k)/2)} \int_0^\infty f(t)t^{(d-k)/2-1}\,dt$$

for every nonnegative measurable f. Indeed, the surface measure of the unit sphere is $2\pi^{(d-k)/2}/\Gamma((d-k)/2)$ and $r^{d-k-1}dr = \frac{1}{2}t^{(d-k)/2-1}dt$. In particular the factor $1/2$ from the squared-radius substitution is included. Applying the parallel and perpendicular changes of variables to a nonnegative Borel test F on symmetric $(k+1)\times(k+1)$ matrices gives the explicit fixed-row identity

$$\begin{aligned}&\int_{\mathbb{R}^{1\times d}} e^{-\|y\|^2} F\begin{pmatrix} B & Xy^{\mathsf{T}} \\ yX^{\mathsf{T}} & \|y\|^2 \end{pmatrix} dy \\ &= \frac{\pi^{(d-k)/2}}{\Gamma((d-k)/2)} \det(B)^{-1/2} \int_{\mathbb{R}^k}\int_0^\infty e^{-z^{\mathsf{T}}B^{-1}z-t} t^{(d-k)/2-1} F\begin{pmatrix} B & z \\ z^{\mathsf{T}} & z^{\mathsf{T}}B^{-1}z+t \end{pmatrix} dt\,dz.\end{aligned}$$

All factors on the right are measurable in (B,z,t) for $B>0$; extend the kernel by zero outside that open cone. Tonelli then gives a measurable function of B. Hence the induction hypothesis applies to this kernel after multiplying by $e^{-\|X\|_F^2}$ and integrating in X. The bases chosen for a fixed X no longer occur in the formula, so no measurable family of bases is required.

Put $u = z^{\mathsf{T}}B^{-1}z + t$ and $S = \left(\begin{smallmatrix} B & z \\ z^{\mathsf{T}} & u \end{smallmatrix}\right)$. For $v \in \mathbb{R}^k$ and $c \in \mathbb{R}$, completing the square gives

$$(v,c)^{\mathsf{T}} S (v,c) = (v + cB^{-1}z)^{\mathsf{T}} B (v + cB^{-1}z) + c^2 t.$$

Thus $S > 0$ exactly when $B > 0$ and $t > 0$. Block elimination also gives $\det S = \det(B)t$, and $\operatorname{tr} S = \operatorname{tr} B + z^{\mathsf{T}}B^{-1}z + t$. The map $(B,z,t) \mapsto (B,z,u)$ has Jacobian one: its derivative is block triangular with identity on the B,z coordinates and derivative 1 in t. The independent entries of S are exactly those of B, then z, then u, up to a coordinate permutation of absolute determinant one. The determinant powers supplied by the induction hypothesis and the fixed-row formula are

$$\det(B)^{(d-k-1)/2} \det(B)^{-1/2} t^{(d-k)/2-1} = \big(\det(B)t\big)^{(d-k-2)/2} = \det(S)^{\eta_{k+1,d}}.$$

The exponential becomes $e^{-\operatorname{tr} S}$, and the constants multiply by $\pi^{(d-k)/2}/\Gamma((d-k)/2)$. This is precisely (7) for $k+1$.

For completeness, the exceptional matrices X with deficient row rank form a null set. For $k \geq 1$ they lie in the zero set of the leading $k \times k$ minor, a nonzero polynomial since it takes the value 1 at $[I_k\ 0]$. To see that any nonzero real polynomial has a null zero set, induct on its number of variables. In the last variable write it as $\sum_j p_j(x')t^j$ and choose a coefficient p_j that is not the zero polynomial. Outside the null zero set of that coefficient, the one-variable polynomial has finitely many roots. Fubini on bounded boxes proves nullity, and the countable union of those boxes gives the assertion on the whole space. At $k = 0$ there is no exceptional set. Integrals over these null sets vanish even for nonnegative tests taking the value ∞. The last induction step, from $d-1$ to d rows, has perpendicular dimension one, so this argument includes the square case.

6.3 The spectral map and its actual multiplicity

Fix $k \in \mathbb{N}$. We now prove the eigenvalue change of variables in the same symmetric entry measure. Let

$$\operatorname{Skew}_k(\mathbb{R}) = \{K \in \mathbb{R}^{k\times k} : K^{\mathsf{T}} = -K\}.$$

This space is identified with the $k(k-1)/2$ entries K_{ij} for $i<j$; its diagonal entries are zero and its lower entries are their negatives. Write dK for product Lebesgue measure in these independent entries. For $K \in \mathrm{Skew}_k(\mathbb{R})$ define the Cayley matrix

$$Q(K) = (I - K/2)^{-1}(I + K/2).$$

Here and throughout this subsection $I = I_k$. The matrix $I - K/2$ is invertible because $\langle x, Kx\rangle = 0$ for every $x \in \mathbb{R}^k$; its kernel equation forces $\|x\|^2 = 0$. Direct multiplication proves that $Q(K)$ is orthogonal and that $Q(K) + I$ is invertible. Conversely, an orthogonal matrix $Q \in \mathbb{R}^{k\times k}$ with $Q + I$ invertible has skew-symmetric inverse Cayley coordinate $K = 2(Q - I)(Q + I)^{-1}$.

Write $\mathrm{diag}(s)$ for the diagonal matrix with entries $s_1, \dots, s_k$. The spectral map $\Theta : \mathrm{Skew}_k(\mathbb{R}) \times \mathcal{C}_k \to \mathrm{Sym}_k(\mathbb{R})$ is

$$\Theta(K, s) = Q(K)\,\mathrm{diag}(s)Q(K)^{\mathsf{T}}, \qquad s \in \mathcal{C}_k.$$

Both its source and target have dimension $k(k+1)/2$. We use the same upper-index order on each: the source coordinate in the (i,i) slot is s_i, and that in the (i,j) slot, $i<j$, is K_{ij}; the target coordinate in the (i,j) slot is the symmetric entry S_{ij}. Thus the signed determinant below uses a specified common coordinate order. At $(K,s) = (0,s)$, a tangent vector in the skew factor is a matrix $\dot K \in \mathrm{Skew}_k(\mathbb{R})$. The derivative of orthogonal conjugation in this direction is the commutator $[\dot K, \mathrm{diag}(s)] = \dot K\,\mathrm{diag}(s) - \mathrm{diag}(s)\dot K$. Its independent (i,j) entry is $(s_j - s_i)\dot K_{ij}$. The diagonal derivative is the identity. Thus its determinant is the Vandermonde.

At a general K, let $\dot K$ be a skew tangent and let $\dot s \in \mathbb{R}^k$. Differentiating the inverse in the Cayley formula, and using $Q(K) + I = 2(I - K/2)^{-1}$, gives

$$DQ_K(\dot K) = (I - K/2)^{-1}\dot K(I - K/2)^{-1}, \qquad Q(K)^{\mathsf{T}}DQ_K(\dot K) = (I - K/2)^{-\mathsf{T}}\dot K(I - K/2)^{-1}.$$

The second expression is skew-symmetric. Differentiating $Q(K)^{\mathsf{T}}Q(K) = I$ now writes the whole spectral derivative as

$$D\Theta_{(K,s)}(\dot K, \dot s) = Q(K)\left(\mathrm{diag}(\dot s) + \left[(I - K/2)^{-\mathsf{T}}\dot K(I - K/2)^{-1}, \mathrm{diag}(s)\right]\right)Q(K)^{\mathsf{T}}.$$

Here $M^{-\mathsf{T}}$ means $(M^{-1})^{\mathsf{T}}$ for an invertible matrix M. This factors the derivative into three linear maps: a congruence on the skew coordinates, the diagonal/commutator map already computed at $K = 0$, and orthogonal conjugation on the target.

For $k \geq 1$ and a real $k\times k$ matrix M, the determinant of $\dot K \mapsto M\dot K M^{\mathsf{T}}$ on skew coordinates is $(\det M)^{k-1}$. Under the identification of a skew matrix with a bivector, this map is $\bigwedge^2 M$. After extending scalars to $\mathbb{C}$ and triangularizing M, its diagonal entries are $\lambda_i\lambda_j$ for $i<j$, where λ_i are the diagonal entries of that triangular matrix. Their product is $\prod_i \lambda_i^{k-1} = (\det M)^{k-1}$, proving the identity also for singular M. Apply it to $M = (I - K/2)^{-\mathsf{T}}$. Orthogonal conjugation has absolute determinant one in symmetric entry coordinates: it preserves Frobenius volume, and that volume is $2^{k(k-1)/4}$ times independent-entry volume. This fixed ratio cancels between source and target. Multiplying the three determinants therefore gives

$$|\det D\Theta(K,s)| = \underbrace{|\det(I - K/2)|^{-(k-1)}}_{a_k(K)} \prod_{i<j}(s_j - s_i). \tag{8}$$

The explicit differential, skew-congruence determinant, and volume comparison are proved in the corresponding modules in Part II. In (8), a_k is a positive finite continuous function on the skew space; it depends on k and K, but not on s. For $k = 0$ both coordinate spaces are singletons and the Jacobian identity is the empty-product identity $1 = 1$.

The map is not asserted to have a constant number of inverse images. An orthogonal frame here means a matrix $Q \in O(k) = \{Q \in \mathbb{R}^{k\times k} : Q^{\mathsf{T}}Q = I_k\}$. For such a Q, define its admissible-sign count by

$$n(Q) = \#\{\varepsilon \in \{-1,1\}^k : \det(Q\,\mathrm{diag}(\varepsilon) + I) \neq 0\}.$$

Here $\varepsilon = (\varepsilon_1, \dots, \varepsilon_k)$ ranges over the 2^k sign vectors, and $\mathrm{diag}(\varepsilon)$ changes the sign of each column of Q. Thus $n(Q) \leq 2^k$. It is also positive: multilinearity of the determinant gives

$$\sum_{\varepsilon\in\{-1,1\}^k} \det(Q\,\mathrm{diag}(\varepsilon) + I) = 2^k.$$

For each $S \in \mathcal{P}_k$ with pairwise distinct eigenvalues, let $s \in \mathcal{C}_k$ be its uniquely ordered eigenvalue tuple and choose $Q \in O(k)$ with $S = Q\,\mathrm{diag}(s)Q^{\mathsf{T}}$. Every other frame for this same ordered tuple is $Q\,\mathrm{diag}(\varepsilon)$ for a unique sign vector. The admissible-sign count is invariant under such a sign change, by reindexing the sign vectors. The inverse Cayley formula therefore identifies the actual fiber $\Theta^{-1}(\{S\})$ with exactly $n(Q)$ frames, and this fiber is nonempty. The image is precisely the positive-definite simple-spectrum locus. Repeated eigenvalues are null: the characteristic discriminant is a polynomial in symmetric entries and does not vanish on a diagonal matrix with distinct entries.

Here is the change-of-variables argument with the actual fiber multiplicity. The domain of Θ is open in Euclidean coordinates, and (8) is nonzero there. The inverse function theorem supplies open sets on which Θ is a diffeomorphism onto its image. Second countability gives a countable such cover $(U_j)_{j\geq 0}$. Put $P_j = U_j \setminus \bigcup_{i<j} U_i$. These are disjoint Borel sets covering the domain; their images are Borel because each lies inside a chart on which Θ is a homeomorphism. For any target S, each piece contributes at most one inverse image. Consequently

$$\#\Theta^{-1}(\{S\}) = \sum_{j\geq 0} \mathbf{1}_{\Theta(P_j)}(S).$$

This proves measurability of the actual fiber cardinality. On the image it is the positive integer $n(Q)$ computed above. For a nonnegative Borel function f on the image, apply injective change of variables on each P_j to $f(S)/\#\Theta^{-1}(\{S\})$. Sum by Tonelli. Each S in the image appears exactly as many times as there are inverse images, giving

$$\int_{\Theta(\mathrm{Skew}_k(\mathbb{R})\times\mathcal{C}_k)} f(S)\,dS = \int_{\mathrm{Skew}_k(\mathbb{R})\times\mathcal{C}_k} \frac{f(\Theta(K,s))|\det D\Theta(K,s)|}{n(Q(K))}\,dK\,ds.$$

Division is only by positive finite integers. Thus this identity is valid for nonnegative integrals before any finiteness conclusion, and no constant covering degree has been assumed.

The function $K \mapsto n(Q(K))$ is measurable: it is the sum of the indicators of the finitely many open conditions $\det(Q(K)\,\mathrm{diag}(\varepsilon) + I) \neq 0$. Consequently $a_k(K)/n(Q(K))$ is positive, finite, measurable, and independent of s, d, ρ, T. Define the nonnegative integral

$$D_k^{\mathrm{ord}} = \int_{\mathbb{R}^{k(k-1)/2}} \frac{a_k(K)}{n(Q(K))}\,dK.$$

For $S = Q\,\mathrm{diag}(s)Q^{\mathsf{T}}$, $\mathrm{tr}\,S = \sum_i s_i$, $\det S = \prod_i s_i$, and $\det(I + TS) = \prod_i (1 + Ts_i)$. Applying the preceding change of variables and Tonelli therefore gives, before asserting finiteness,

$$\int_{\mathcal{P}_k} e^{-\mathrm{tr}\,S} \det(S)^{\eta_{k,d}} \det(I + TS)^{-\rho}\,dS = D_k^{\mathrm{ord}} \int_{\mathcal{C}_k} \mathcal{L}_{\eta_{k,d},\rho,T}(s)\,ds. \tag{9}$$

Although D_k^{ord} was defined in $[0,\infty]$, calibration now proves $0 < D_k^{\mathrm{ord}} < \infty$: in the already established identity (9), set $d = k$ and $\rho = T = 0$. The left side is positive and finite by (7)

applied to the Gaussian mass and $A_{k,k} > 0$. A nonnegative extended-real product that is positive and finite has both factors positive and finite. This uses the already parameterized identity, not an attempted inference of that identity from one calibration value.

The $k!$ disjoint strict permutation chambers cover the positive orthant off a null set, and coordinate permutations preserve measure. Thus J_k is $k!$ times the ordered integral. Combining (7) and (9) proves the Wishart theorem with

$$C_{k,d} = A_{k,d}\, D_k^{\mathrm{ord}}/k! \in (0, \infty).$$

For $k = 0$ all factors are one; for $k = 1$ the skew-coordinate space has one point, the branch count is one, and the radial constant gives the stated $C_{1,d}$. Finally transposition preserves matrix-entry measure, and Sylvester's identity $\det(I_d + TZZ^{\mathsf{T}}) = \det(I_k + TZ^{\mathsf{T}}Z)$ gives the right-oriented identity with the same constant.

The literal formal conclusion is `O77.GlobalSpectralIntegral.realWishartEigenvalueFormula`. Its constituent identities are the general nonnegative Gram pushforward in Section 59, the reciprocal Jacobian formula in Section 79, and the final spectral integration in Section 82. The namespace `External` in the name of the resulting proposition is historical naming; a proof of that proposition is supplied, not a new assumption of the final theorem.

7 The residual matrix-product germ

In this section fix three dimensions $p, q, h \in \mathbb{N}$, independently of the matrix-factorization ranks. Their later specialization will be $p = O - b$, $q = I - a$, $h = H + r - a - b$. Set $E_{p,q,h} = \mathbb{R}^{p\times h} \times \mathbb{R}^{h\times q}$ and define

$$K_{p,q,h}(Y, Z) = \|YZ\|_F^2, \qquad (Y, Z) \in \mathbb{R}^{p\times h} \times \mathbb{R}^{h\times q}.$$

The real dimension of $E_{p,q,h}$ is $D = ph + hq$. Its norm is $\|(Y, Z)\| = (\|Y\|_F^2 + \|Z\|_F^2)^{1/2}$, and $dw = dY\, dZ$ denotes product Lebesgue measure in these D entries. The symbol K below is an abbreviation for the real-valued function $K_{p,q,h}$, and is no longer the skew matrix used in the preceding section. The quantities $d_0(p, q, h)$ and $\mu_0(p, q, h)$ are the finite minimum and its number of minimizers from Definition 2.5.

Theorem 7.1 (Residual-origin pair)**.** *For every $p, q, h \in \mathbb{N}$, with the norm and measure just specified,* $\mathsf{BandPair}(K_{p,q,h}, 0; d_0(p, q, h)/2, \mu_0(p, q, h))$ *holds. Equivalently, the residual germ at the zero pair $0 = (0, 0) \in E_{p,q,h}$ has radius-first pair*

$$\left(\frac{d_0(p, q, h)}{2}, \mu_0(p, q, h)\right).$$

If $pqh = 0$, this is the neutral pair $(0, 1)$. For positive dimensions the proof uses the Wishart identity just proved, then establishes the power and logarithmic order directly.

7.1 R1: Gaussian integration row by row

Fix $Z \in \mathbb{R}^{h\times q}$ and $T \in [0, \infty)$. For $Y \in \mathbb{R}^{p\times h}$, write $y_i \in \mathbb{R}^{1\times h}$ for its ith row, $1 \le i \le p$. Define $M_T(Z) = I_h + TZZ^{\mathsf{T}} \in \mathbb{R}^{h\times h}$. Direct finite-sum algebra gives

$$T\|YZ\|_F^2 + \|Y\|_F^2 = \sum_{i=1}^{p} y_i M_T(Z) y_i^{\mathsf{T}}.$$

The Gram matrix ZZ^{T} is positive semidefinite, so $M_T(Z)$ is positive definite. The measure on $\mathbb{R}^{p\times h}$ is literally the finite product of the row measures; Tonelli therefore factors the nonnegative exponential into p row integrals.

For $M \in \mathcal{P}_h$, the real spectral theorem gives $M = U\,\mathrm{diag}(\delta_1, \dots, \delta_h)U^{\mathsf{T}}$ with $U \in O(h)$ and $\delta_i > 0$. The substitution $x = Uz$ has absolute Jacobian one and makes $x^{\mathsf{T}}Mx = \sum_i \delta_i z_i^2$. For each diagonal factor, $v = \sqrt{\delta_i}\, z_i$ and the scalar Gaussian integral give

$$\int_{\mathbb{R}} e^{-\delta_i z_i^2}\, dz_i = \delta_i^{-1/2} \int_{\mathbb{R}} e^{-v^2}\, dv = \sqrt{\pi}\, \delta_i^{-1/2}.$$

Tonelli's theorem for the nonnegative product therefore gives

$$\int_{\mathbb{R}^h} e^{-x^{\mathsf{T}}Mx}\, dx = \prod_{i=1}^{h} \left(\sqrt{\pi}\, \delta_i^{-1/2}\right) = \pi^{h/2} (\det M)^{-1/2}.$$

The right side is finite, establishing integrability before any use of a totalized real integral. At $h = 0$ both the integral and the empty product are one. Consequently

$$\int_{\mathbb{R}^{p\times h}} e^{-T\|YZ\|_F^2 - \|Y\|_F^2}\, dY = \pi^{ph/2} \det(M_T(Z))^{-p/2}.$$

7.2 R2: the determinant through the smaller Gram matrix

For the fixed residual dimensions define $k = \min(h, q)$ and $d = \max(h, q)$, so $0 \le k \le d$. For each $Z \in \mathbb{R}^{h\times q}$ and $T \in [0, \infty)$, Sylvester's rectangular determinant identity gives

$$\det(I_h + TZZ^{\mathsf{T}}) = \det(I_q + TZ^{\mathsf{T}}Z).$$

Use the $h \times h$ Gram matrix when $h \le q$ and the $q \times q$ Gram matrix when $q \le h$. Both are real symmetric positive semidefinite. The real spectral theorem and the determinant formula for a Hermitian shift give nonnegative eigenvalues $s_1, \dots, s_k$ with

$$\det(I_h + TZZ^{\mathsf{T}}) = \prod_{i=1}^{k} (1 + Ts_i).$$

No step deletes zero eigenvalues from the larger Gram matrix informally; the proof switches to the smaller matrix before taking the product.

7.3 R3: from the eigenvalue orthant to one ordered chamber

In the eigenvalue space $\mathbb{R}^k$, remove the finite union of coordinate-zero hyperplanes $\{s : s_i = 0\}$, $1 \le i \le k$, and coordinate-tie hyperplanes $\{s : s_i = s_j\}$, $1 \le i < j \le k$. Each is the kernel of a nonzero linear functional and hence has Lebesgue measure zero. Off this null set there is a unique strict sorting permutation. The strict permutation chambers are therefore pairwise disjoint and cover the orthant almost everywhere.

Coordinate permutations preserve the finite-product Lebesgue measure. Hence every Borel measurable function $F : [0, \infty)^k \to [0, \infty]$ invariant under all coordinate permutations satisfies

$$\int_{[0,\infty)^k} F(s)\, ds = k! \int_{0<s_1<\cdots<s_k} F(s)\, ds.$$

The weak and strict ordered chambers have the same integral because their difference is contained in the null boundary. For $k = 0$, there is one empty permutation and $0! = 1$; both integrals equal $F(())$, the value at the unique empty tuple.

7.4 R4: application of the exact Wishart identity

We now apply the exact Wishart identity proved above, with $k = \min(h, q)$, $d = \max(h, q)$ and $\rho = p/2$. Its exponent is $\eta_{k,d} = (d - k - 1)/2 > -1$; it equals $-1/2$ when $k = d$. Thus the square case belongs to the same proof. The variance-$1/2$ entry Gaussian is exactly the weight $e^{-\|Z\|_F^2}$ occurring after R1, and the spectral weight is $e^{-\sum_i s_i}$. The common normalizer handles either rectangular orientation without dependence on $p/2$ or T. The sorted extension is permutation invariant, so R3 contributes the visible factor $k!$ when replacing its full orthant by the strict ordered chamber. The residual power–log conclusion is derived next from this exact identity.

7.5 R5: the ordered estimate and the Gaussian Laplace order

For this estimate, let $k \in \mathbb{N}$, $\eta \in (-1, \infty)$, and $\rho \in (0, \infty)$ be fixed, and let $T > 0$ vary. For the one-based indices $i = 1, \dots, k$, define

$$A_i = \eta + i > 0, \qquad \nu = \sum_{i=1}^{k} \min(A_i, \rho), \qquad \ell = \#\{i : A_i = \rho\}.$$

Since the A_i increase strictly, ℓ is zero or one. We first prove directly that

$$\int_{\mathcal{C}_k} \mathcal{L}_{\eta,\rho,T}(s)\, ds \asymp T^{-\nu} (\log T)^{\ell} \qquad (T \longrightarrow \infty). \tag{10}$$

Thus the chamber estimate used here is part of the proof, rather than a second literature premise.

For every fixed real $A > 0$ and the fixed $\rho > 0$, the elementary model integral satisfies, as $T \to \infty$,

$$\int_0^\infty e^{-x} x^{A-1} (1 + Tx)^{-\rho}\, dx \asymp \begin{cases} T^{-A}, & A < \rho, \\ T^{-\rho} \log T, & A = \rho, \\ T^{-\rho}, & A > \rho. \end{cases}$$

To verify this for $T \geq 2$, split the domain at $1/T$ and 1. On $(0, 1/T)$ the two bounded factors e^{-x} and $(1+Tx)^{-\rho}$ give an integral comparable to T^{-A}. On $(1/T, 1)$ the integral is comparable to $T^{-\rho} \int_{1/T}^1 x^{A-\rho-1}\, dx$, and

$$\int_{1/T}^1 x^{A-\rho-1}\, dx = \begin{cases} (T^{\rho-A} - 1)/(\rho - A), & A < \rho, \\ \log T, & A = \rho, \\ (1 - T^{-(A-\rho)})/(A - \rho), & A > \rho. \end{cases}$$

For $T \geq 2$, the first expression is bounded above and below by positive constants times $T^{\rho-A}$, and the third by positive constants. For example the lower constants use $1 - 2^{-(\rho-A)} > 0$ in the first case and $1 - 2^{-(A-\rho)} > 0$ in the third. This yields exactly the three orders displayed above after multiplication by $T^{-\rho}$. On $(1, \infty)$ the upper bound is a finite constant times $T^{-\rho}$, by exponential decay. The first two intervals already supply every required lower bound. This also proves integrability before any conversion from nonnegative to real integrals.

On the ordered chamber the Vandermonde obeys

$$0 \leq \prod_{i<j} (s_j - s_i) \leq \prod_{i=1}^{k} s_i^{i-1}.$$

If $2s_i \leq s_j$ whenever $i < j$, the reverse inequality holds with factor $2^{-\binom{k}{2}}$. Consequently the chamber density lies below the separated product $\prod_i e^{-s_i} s_i^{A_i - 1} (1 + Ts_i)^{-\rho}$. Enlarging the

domain to the full positive product and applying Tonelli and the scalar estimates gives the upper bound in (10).

For the lower bound first suppose no A_i equals ρ. For each $A_i < \rho$ choose an interval $[a_i/T, b_i/T]$ with fixed endpoints $0 < a_i < b_i$. For each $A_i > \rho$ choose a fixed interval $[a_i, b_i] \subset (0,1)$ with $a_i < b_i$. Choose the endpoints so that $2b_i < a_{i+1}$ for successive indices in each group. This is what we mean here by two-separated intervals: every point of the later interval is greater than twice every point of the earlier interval. For all sufficiently large T the two groups are also two-separated. Their product lies in $\mathcal{C}_k$. On a small interval, the substitution $s_i = x/T$ gives

$$\int_{a_i/T}^{b_i/T} e^{-s_i} s_i^{A_i-1}(1+Ts_i)^{-\rho}\,ds_i = T^{-A_i}\int_{a_i}^{b_i} e^{-x/T}x^{A_i-1}(1+x)^{-\rho}\,dx.$$

For $T \geq 1$ the last integral is bounded below by its strictly positive value with e^{-x} in place of $e^{-x/T}$. On a fixed interval $[a_i, b_i] \subset (0,1)$, taking $T \geq 1/a_i$ gives $(1+Ts_i)^{-\rho} \geq 2^{-\rho}T^{-\rho}s_i^{-\rho}$. Its integral is therefore at least $2^{-\rho}T^{-\rho}\int_{a_i}^{b_i} e^{-x}x^{A_i-\rho-1}\,dx$, a positive constant times $T^{-\rho}$. Multiply these estimates and the Vandermonde lower bound to obtain the required lower bound $\prod_{A_i<\rho} T^{-A_i}\prod_{A_i>\rho} T^{-\rho}$. The interval choices are fixed before T varies.

If $A_j = \rho$, use the same small intervals for $i < j$ and fixed intervals for $i > j$, but let the jth coordinate range over $[a/T, b]$. Choose $a > 2\max_{i<j} b_i$ when the preceding group is nonempty, and $b > 0$ smaller than half the first following fixed endpoint; absent preceding or following groups impose no condition. Choose $b < 1$ and $a \geq 1$. This product is again two-separated for large T. Its resonant integral is bounded below by a positive constant times

$$T^{-\rho}\int_{a/T}^{b}\frac{dx}{x} = T^{-\rho}\log(bT/a) \asymp T^{-\rho}\log T.$$

Together with the other factors this proves the matching lower bound. For $k = 0$ the integral is one, and $\nu = \ell = 0$.

Return to fixed positive natural dimensions p, q, h. Specialize the preceding estimate to $k = \min(h,q)$, $d = \max(h,q)$, $\eta = (d-k-1)/2$ and $\rho = p/2$. All quotients and subtractions in these exponents are in $\mathbb{R}$. For $0 \leq j \leq k$ set

$$B_j = \sum_{i=1}^{j} A_i + (k-j)\rho = \frac{j(d-k+j)+p(k-j)}{2}.$$

The successive differences $B_j - B_{j-1} = A_j - \rho$ increase strictly. Thus $\min_j B_j = \sum_i \min(A_i, \rho) = \nu$, and the number of minimizing indices is $1+\ell$. For each integer $j \in \{0,\dots,k\}$, put $t = k-j$. Since $\{h,q\} = \{k,d\}$, we have $(h-t)(q-t) = j(d-k+j)$; hence $B_j = c_{p,q,h}(t)/2$, where $c_{p,q,h}$ is the cost in (1). This substitution is a bijection of the two finite index sets. Therefore $\nu = d_0(p,q,h)/2$ and $1+\ell = \mu_0(p,q,h)$.

For $T \geq 0$, define the residual Gaussian Laplace integral on $E_{p,q,h}$ by

$$J_G(T) = \int e^{-TK_{p,q,h}(Y,Z)}e^{-\|Y\|_F^2-\|Z\|_F^2}\,dY\,dZ.$$

The row integral and the Wishart identity therefore give

$$J_G(T) = \pi^{ph/2}C_{k,d}k!\int_{\mathcal{C}_k}\mathcal{L}_{\eta_{k,d},p/2,T}(s)\,ds.$$

Every displayed constant is positive and finite, independent of T. It follows that

$$J_G(T) \asymp T^{-d_0(p,q,h)/2}(\log T)^{\mu_0(p,q,h)-1}.$$

7.6 R6: quartic localization to each fixed ball

Put

$$N(Y,Z) = \|Y\|_F^2 + \|Z\|_F^2, \qquad D = ph + hq, \qquad \overline{B}_R = \{w \in E_{p,q,h} : N(w) \le R^2\},$$

and, for every fixed $R > 0$, define

$$J_R(T) = \int_{\overline{B}_R} e^{-TK_{p,q,h}(Y,Z)}\, dY\, dZ.$$

Here and below the integration variable is $w = (Y,Z) \in E_{p,q,h}$, and R is fixed before the variable $T \ge 0$ is chosen. The use of a closed ball here makes the shell boundaries explicit; the public open-ball predicate will follow by inclusions in R7. Simultaneous dilation $\delta_c(Y,Z) = (cY, cZ)$ satisfies

$$K(\delta_c w) = c^4 K(w), \qquad N(\delta_c w) = c^2 N(w), \qquad d(\delta_c w) = c^D\, dw \quad (c > 0).$$

The measure identity is the ordinary change of variables in all original matrix entries. The residual is jointly homogeneous of degree four.

On the initial ball, $e^{-N} \ge e^{-R^2}$, so $e^{-R^2} J_R(T) \le J_G(T)$. For the opposite comparison partition the complement of the closed ball into the pairwise disjoint half-open shells

$$S_{R,n} = \{(2^n R)^2 < N \le (2^{n+1}R)^2\}, \qquad n \ge 0.$$

Every point outside the initial ball belongs to exactly one such shell; a point at a dyadic boundary belongs to the preceding shell. On $S_{R,n}$ we have $e^{-N} \le e^{-4^n R^2}$. Under $w = 2^{n+1}u$ the shell maps into $\overline{B}_R$. Since $T \ge 0$ and $K \ge 0$, quartic homogeneity gives $e^{-T2^{4(n+1)}K(u)} \le e^{-TK(u)}$. Consequently

$$\int_{S_{R,n}} e^{-TK-N}\, dw \le 2^{(n+1)D} e^{-4^n R^2} J_R(T).$$

The ratio of consecutive coefficients is $2^D e^{-3\cdot 4^n R^2} \to 0$, so the series of coefficients converges. The initial closed ball contributes at most $J_R(T)$. Tonelli over the disjoint shells therefore gives

$$J_G(T) \le A_R J_R(T), \qquad A_R = 1 + \sum_{n \ge 0} 2^{(n+1)D} e^{-4^n R^2} \in (0, \infty).$$

Both comparisons hold for every $T \ge 0$, with R fixed first. Hence the Gaussian power–log order from R5 holds on every positive fixed closed ball. Positivity of R is essential for shell coverage and summability; no claim at radius zero is used.

7.7 R7: Laplace-to-sublevel transfer and strict balls

For $R > 0$, the quadratic ball $\overline{B}_R = \{N \le R^2\}$ is closed because N is continuous, and bounded because every entry has absolute value at most R. It is therefore compact. Its original coordinate measure is therefore finite. For a fixed $R > 0$, put

$$F_R(\varepsilon) = \mathrm{vol}\{w \in \overline{B}_R : K(w) < \varepsilon\}.$$

We give the elementary transfer from J_R to F_R explicitly. For every $\varepsilon > 0$, the upper bound is $F_R(\varepsilon) \le e J_R(1/\varepsilon)$. For fixed real $a > 0$ and $T > 0$, split the integral over $\overline{B}_R$ at the strict level $K(w) < a/T$. On its complement $K \ge a/T$, so

$$J_R(T) \le F_R(a/T) + e^{-a/2} J_R(T/2).$$

Write $\lambda = d_0(p,q,h)/2$ and $\ell = \mu_0(p,q,h) - 1$, so $\lambda, \ell \geq 0$. Choose positive constants c, C for the two-sided estimate of J_R. For sufficiently large T (so that the upper estimate also applies at $T/2$ and $\log(T/2) > 0$),

$$\frac{J_R(T/2)}{J_R(T)} \leq \frac{C}{c}\, 2^{\lambda} \left(\frac{\log(T/2)}{\log T}\right)^{\ell} \leq \frac{C}{c}\, 2^{\lambda}.$$

Choose a single $a > 0$ with $e^{-a/2}(C/c)2^{\lambda} \leq 1/2$. Then the splitting inequality gives $F_R(a/T) \geq J_R(T)/2$ for all sufficiently large T. Substitute $T = a/\varepsilon$, so $J_R(a/\varepsilon) \asymp \varepsilon^{\lambda}(\log(1/\varepsilon))^{\ell}$ by Lemma 3.1. Together with the upper estimate this yields

$$F_R(\varepsilon) \asymp \varepsilon^{d_0(p,q,h)/2} \left(\log \frac{1}{\varepsilon}\right)^{\mu_0(p,q,h)-1} \qquad (\varepsilon \downarrow 0).$$

There is no appeal here to a converse zeta theorem or to a general Tauberian theorem. The argument preserves strict sublevels.

For $\delta > 0$, define $B(0,\delta) = \{w \in E_{p,q,h} : \|w\| < \delta\}$. To recover the open-ball predicate, use

$$\overline{B}_{\delta/2} \subset B(0,\delta) \subset \overline{B}_{\delta}.$$

The estimates on these two closed balls give the lower and upper bounds on the open ball. Constants are selected only after δ is fixed, exactly as required by Definition 2.2. No sphere-null theorem is needed. Finally, if $pqh = 0$, then $K \equiv 0$, $d_0 = 0$ and $\mu_0 = 1$; every positive-radius open ball has positive finite volume, including the empty-dimensional case where its measure is one. This gives the neutral pair directly.

8 Finite minimization and the complete minimum locus

Lemma 8.1 (Discrete convexity and parity)**.** *For all $p,q,h \in \mathbb{N}$, $1 \leq \mu_0(p,q,h) \leq 2$. If $p,q,h > 0$, then $d_0(p,q,h) > 0$, and $\mu_0(p,q,h) = 2$ precisely when*

$$p+q+h \text{ is odd}, \qquad p \leq q+h, \quad q \leq p+h, \quad h \leq p+q.$$

For every other triple, $\mu_0(p,q,h) = 1$.

Proof. The zero-dimensional cases are already established. For positive dimensions, $c_{p,q,h}(0) = hq > 0$ and $c_{p,q,h}(t) \geq pt > 0$ for $t > 0$. Thus the finite minimum is positive. Over $\mathbb{Z}$ the first differences are

$$c_{p,q,h}(t+1) - c_{p,q,h}(t) = 2t + 1 + p - h - q,$$

strictly increasing by 2. A finite sequence with such first differences decreases, has at most one zero difference, then increases. Its minimizers form one point or two adjacent points. A zero difference at $0 \leq t < \min(h,q)$ is equivalent to $h+q-p$ odd and $1 \leq h+q-p \leq 2\min(h,q)-1$. This is the stated odd-parity condition and $|h-q| < p < h+q$. In odd parity the triangle equalities are impossible, so strict and weak triangle inequalities agree for this test. □

Proposition 8.2 (Closed residual table)**.** *Fix $p,q,h \in \mathbb{N}$ and write $\lambda_0 = d_0(p,q,h)/2$ and $m_0 = \mu_0(p,q,h)$. If $pqh = 0$, then $(\lambda_0, m_0) = (0,1)$. For $p,q,h > 0$, their values are given by the following disjoint cases:*

Region	(λ_0, m_0)
$p > q+h$	$(hq/2, 1)$
$q > p+h$	$(hp/2, 1)$
$h > p+q$	$(pq/2, 1)$
All three weak triangle inequalities	$\left(\dfrac{2h(p+q) - (p-q)^2 - h^2 + \chi}{8},\ 1+\chi\right)$, *where* $\chi = (p+q+h) \bmod 2$.

The branches are disjoint and exhaustive, with the zero-dimensional branch taking precedence.

Proof. The zero-dimensional cases follow directly from the finite cost: if $q = 0$ or $h = 0$, the only index is $t = 0$; if $p = 0$ and $q, h > 0$, the unique zero occurs at $t = \min(h, q)$. Suppose now that $p, q, h > 0$. For each allowed integer t, viewed as a real number, completing the square gives

$$c_{p,q,h}(t) = hq - \left(\frac{h+q-p}{2}\right)^2 + \left(t - \frac{h+q-p}{2}\right)^2.$$

If $p > q+h$, the vertex is negative, so the minimum on the allowed integers is uniquely at $t = 0$, of value hq. If $q > p + h$, the vertex is greater than h and $h < q$; the minimum is uniquely at $t = h$, of value ph. If $h > p + q$, the vertex is greater than q and $q < h$; the minimum is uniquely at $t = q$, of value pq.

In the remaining case the three weak triangle inequalities place the vertex in $[0, \min(h, q)]$. If $p + q + h$ is even, the vertex is an allowed integer and its squared distance from the nearest allowed integer is zero. If that sum is odd, the vertex is a half-integer. The triangle inequalities are then strict, so both adjacent integers are allowed and have squared distance $1/4$. Consequently, with $\chi = (p + q + h) \bmod 2$,

$$\lambda_0 = \frac{hq}{2} - \frac{(h+q-p)^2}{8} + \frac{\chi}{8} = \frac{2h(p+q) - (p-q)^2 - h^2 + \chi}{8}.$$

There is one nearest integer in the even case and two in the odd case. This also covers triangle walls: equality in a triangle inequality forces even parity and puts the integer vertex at an endpoint. □

Lemma 8.3 (Global comparison identities)**.** *Let $I, O, H, r, a, b \in \mathbb{N}$, assume $r \le \min(I, O, H)$, and suppose (a, b) is feasible for these dimensions and target rank. Put*

$$\alpha = a - r, \quad \beta = b - r, \quad p = O - b, \quad q = I - a, \quad h = H + r - a - b, \quad s = Oa + b(I - a),$$

and let $\chi = (I + O + H + r) \bmod 2 \in \{0, 1\}$. Then

$$\begin{aligned} p - q - h &= 2\alpha - (I + H - O - r), \\ q - p - h &= 2\beta - (O + H - I - r), \\ h - p - q &= H + r - I - O, \\ p + q + h &\equiv I + O + H + r \pmod 2. \end{aligned}$$

Let $L_{\text{even}} = [2(H + r)(I + O) - (I - O)^2 - (H + r)^2]/8$, as in Section 2.1. The following equalities hold over $\mathbb{Q}$, with all dimensions first cast to $\mathbb{Q}$:

$$\begin{aligned} \frac{s}{2} + \frac{2h(p+q) - (p-q)^2 - h^2}{8} &= L_{\text{even}}, \\ \frac{s + hq}{2} - L_{\text{even}} &= \frac{(p-q-h)^2}{8}, \\ \frac{s + hp}{2} - L_{\text{even}} &= \frac{(q-p-h)^2}{8}, \\ s + pq &= OI. \end{aligned}$$

Proof. Substitute the definitions and expand. Feasibility makes the dimension subtractions equal to their ordinary integer differences. After these conversions the displayed identities are polynomial identities; the parity identity follows because the two sums differ by $2(a + b)$. □

Theorem 8.4 (O77(a): all minimizing rank strata)**.** *Let $I, O, H, r \in \mathbb{N}$ satisfy $r \le \min(I, O, H)$. For every feasible $(a, b) \in \mathbb{N}^2$, use p, q, h, s and $\Lambda(a, b), \mathfrak{m}(a, b)$ from Definition 2.5. The numerical expression Λ_{gl} of Section 2.1 is the minimum of $\Lambda(a, b)$ over all feasible rank pairs, and $\mathcal{M}(a, b)$ characterizes exactly its equality locus. The minimum is attained at (r, r), so also*

$$\Lambda_{\mathrm{gl}} = \frac{r(I + O - r)}{2} + \frac{d_0(O - r, I - r, H - r)}{2}.$$

Set $\chi = (I + O + H + r) \bmod 2$. In the balanced global regime

$$O + r \le I + H, \qquad I + r \le O + H, \qquad H + r \le I + O,$$

every feasible pair has excess

$$\Lambda(a, b) - \Lambda_{\mathrm{gl}} = \begin{cases} 0, & \text{the residual is balanced,} \\ ((p - q - h)^2 - \chi)/8, & p > q + h, \\ ((q - p - h)^2 - \chi)/8, & q > p + h. \end{cases}$$

Here "the residual is balanced" means that p, q, h satisfy the three weak triangle inequalities in Proposition 8.2. If a residual dimension vanishes, $d_0(p, q, h) = 0$ and $\mu_0(p, q, h) = 1$; the displayed excess formulas remain valid, but the excess itself need not vanish. For example, $I = O = H = 2$, $r = 0$, $(a, b) = (2, 0)$ has $\Lambda - \Lambda_{\mathrm{gl}} = 1/2$. On the minimizing locus the largest multiplicity is 1 in the three strictly dominated regimes and $1 + \chi$ in the balanced regime; the pointwise multiplicity remains the table in Proposition 8.2.

Proof. The three strict regimes are mutually exclusive: adding any two contradictory domination inequalities would force r to exceed one of the admissible dimensions. If none holds, the weak balanced inequalities hold. The pair (r, r) is feasible.

In the output-dominated regime $I+H < O+r$, the identity $p-q-h = 2(a-r)-(I+H-O-r)$ is positive for every feasible pair. The residual table, including its zero-dimensional edge values, gives

$$2\Lambda(a, b) = s + hq = Oa + (H + r - a)(I - a).$$

Subtracting its value at $a = r$ gives the explicit identity

$$2\Lambda(a, b) - \big[Or + H(I - r)\big] = (a - r)\big(a - (I + H - O)\big).$$

The first factor is nonnegative and the second is strictly positive, since $a \ge r > I + H - O$. Equality therefore holds exactly at $a = r$, independently of the feasible value of b.

In the input-dominated regime $O + H < I + r$, the corresponding formulas are

$$2\Lambda(a, b) = s + hp = Ib + (H + r - b)(O - b),$$

$$2\Lambda(a, b) - \big[Ir + H(O - r)\big] = (b - r)\big(b - (O + H - I)\big).$$

The second factor is again strictly positive, so equality holds exactly at $b = r$. In the hidden-dominated regime $I + O < H + r$, the defect $h - p - q = H + r - I - O$ is positive on every stratum, and $2\Lambda = s + pq = OI$ on all feasible pairs.

In the balanced regime $h - p - q \le 0$. At most one of $p - q - h$ and $q - p - h$ is positive, because their sum is $-2h \le 0$. The table and Lemma 8.3 give exactly the three excess formulas. A positive defect has parity χ. If $\chi = 0$ it is at least 2 and the excess is positive. If $\chi = 1$ its excess vanishes exactly when the defect is 1. Thus minimization is equivalent to

$$p - q - h \le \chi, \qquad q - p - h \le \chi.$$

Substituting the affine defect identities and collecting terms gives

$$2a + O \le I + H + r + \chi, \qquad 2b + I \le O + H + r + \chi.$$

These are exactly the defining inequalities for $\mathcal{M}(a,b)$ in the balanced row. At $(a,b) = (r,r)$ they follow from the two balanced dimension inequalities $O + r \le I + H$ and $I + r \le O + H$. Thus this row also has the claimed attained minimum.

For a zero residual dimension, a balanced residual has its other two dimensions equal, hence even total parity and table value zero; in a dominated residual the corresponding product value is zero. Thus the preceding formulas include all edges. Multiplicities follow from Proposition 8.2. At (r,r) the residual is balanced in the balanced global regime; odd parity excludes a zero residual dimension there, so it realizes multiplicity 2 when $\chi = 1$. □

From arithmetic back to O77. Fix $I, O, H \in \mathbb{N}$ with $I, O \ge 1$ and a target $\Phi \in \mathbb{R}^{O\times I}$ of rank $r < H$. Applying the fiber formula established next, the union

$$\bigcup_{\substack{(a,b)\ \text{feasible} \\ \Lambda(a,b)=\Lambda_{\mathrm{gl}}}} \{(A,B) \in F_\Phi : \operatorname{rank} A = a, \operatorname{rank} B = b\}$$

is exactly the required minimum-threshold locus, not just a collection of examples. Lemma 4.1 proves that every listed feasible stratum really occurs for the fixed target.

9 The all-points fiber answer

Theorem 9.1 (The pair at every exact fit)**.** *Let $I, O, H \in \mathbb{N}$ with $I, O, H \ge 1$, let $\Phi \in \mathbb{R}^{O\times I}$ have rank $r < H$, and let $w_* = (A_*, B_*) \in \mathbb{R}^{H\times I} \times \mathbb{R}^{O\times H}$ satisfy $B_*A_* = \Phi$. Set $a = \operatorname{rank} A_*$ and $b = \operatorname{rank} B_*$. With $\Lambda(a,b)$ and $\mathfrak{m}(a,b)$ as in Definition 2.5, the rank pair is feasible and*

$$\mathsf{BandPair}(L_\Phi, w_*; \Lambda(a,b), \mathfrak{m}(a,b)).$$

Consequently the pair depends only on (a,b), and the entire minimum-threshold locus is the union of the feasible rank strata listed in Theorem 8.4.

Assembly of Theorem 9.1. At the chosen exact fit, $B_*A_* = \Phi$ implies $L_\Phi(w_*) = 0$ by its definition. The absolute band is therefore the nonnegative sublevel set. Apply feasibility and pass to adapted bases. The basis changes are fixed invertible linear maps, not generally isometries; two-sided Frobenius bounds and local C^1 coordinate invariance preserve the pair. Apply the determinant-open elimination chart and its uniform comparison to reduce the loss to

$$(u, Y_0, Z, v) \longmapsto \|u\|^2 + \|Y_0 Z\|_F^2.$$

Here $u \in \mathbb{R}^s$, $Y_0 \in \mathbb{R}^{p\times h}$, $Z \in \mathbb{R}^{h\times q}$, and $v \in \mathbb{R}^g$, with p, q, h, s from Definition 2.5 and g the nonnegative gauge dimension from Lemma 4.3. The free gauge variables v do not change the pair. Theorem 7.1 supplies the residual factor, and the separated s-dimensional quadratic term adds $s/2$ without changing multiplicity. The positive factor $1/2$ in the original loss is harmless. This gives $((s + d_0(p,q,h))/2, \mu_0(p,q,h))$.

The fixed neighborhood, Jacobian bounds, compatible windows, and nonorthogonal basis transport in this argument establish the pair in the original matrix entries. □

10 The saddle branch at an arbitrary rung point

Throughout this section fix integers $I, O \geq 1$, $H \in \mathbb{N}$, and $r \in \mathbb{N}$ with $r < H$, together with vectors $u_1, \dots, u_r \in \mathbb{R}^O$, $v_1, \dots, v_r \in \mathbb{R}^I$ and real numbers $\sigma_1 > \cdots > \sigma_r > 0$ as in Definition 2.6. Thus both vector families are orthonormal, the target is $\Phi = \sum_{j=1}^{r} \sigma_j u_j v_j^{\mathsf{T}}$, and $C_k \subset \mathbb{R}^{H\times I} \times \mathbb{R}^{O\times H}$ is defined for each integer k with $0 \leq k \leq r$. Empty families and sums are allowed when $r = 0$. Write $L = L_\Phi$ throughout this section. The parameter space has the Euclidean inner product obtained by adding the two Frobenius inner products, and its volume is product Lebesgue measure in the matrix entries. In particular $D^2L(w)[D, D]$ below denotes the actual second Fréchet derivative, evaluated twice on the perturbation D.

The proof applies at every chosen point of every nonoptimal rung. One discarded singular mode supplies a positive two-plane and a negative direction. The implicit-function theorem then constructs critical centers in nearby planar sections. Integrating uniform planar annulus estimates over a fixed tail window proves the band estimate in the whole parameter space.

Lemma 10.1 (Rung criticality, level, and nonemptiness)**.** *For every integer k with $0 \leq k \leq r$ and every $w = (A, B) \in C_k$, put $E = BA - \Phi \in \mathbb{R}^{O\times I}$. Then*

$$E = -\sum_{j=k+1}^{r} \sigma_j u_j v_j^{\mathsf{T}}, \quad B^{\mathsf{T}}E = 0, \quad EA^{\mathsf{T}} = 0, \quad L(w) = \frac{1}{2}\sum_{j=k+1}^{r} \sigma_j^2, \quad \operatorname{rank}(BA) = k.$$

Thus $DL(w) = 0$. Moreover C_k is nonempty for every integer k with $0 \leq k \leq r$. For this nonemptiness assertion alone it is enough to assume $r \leq H$.

Proof. Use the annihilation conditions in Definition 2.6; the two displayed matrix products vanish term by term. For arbitrary perturbations $\dot{A} \in \mathbb{R}^{H\times I}$ and $\dot{B} \in \mathbb{R}^{O\times H}$, differentiating the finite sum of squares gives

$$DL(A, B)[\dot{A}, \dot{B}] = \langle E, B\dot{A} + \dot{B}A\rangle_F = \langle B^{\mathsf{T}}E, \dot{A}\rangle_F + \langle EA^{\mathsf{T}}, \dot{B}\rangle_F.$$

Here $\langle M, N\rangle_F = \sum_{i,j} M_{ij}N_{ij}$ for two matrices of the same size. For $1 \leq i, j \leq r$,

$$\langle u_i v_i^{\mathsf{T}}, u_j v_j^{\mathsf{T}}\rangle_F = \langle u_i, u_j\rangle\langle v_i, v_j\rangle = \begin{cases} 1, & i = j, \\ 0, & i \neq j. \end{cases}$$

Hence the squared norm of E is $\sum_{j=k+1}^{r} \sigma_j^2$. The image of Φ_k is contained in $\operatorname{span}(u_1, \dots, u_k)$, and $\Phi_k v_j = \sigma_j u_j$ for $1 \leq j \leq k$. Since these k vectors are independent and $\sigma_j > 0$, the image has dimension exactly k.

For nonemptiness, choose the first k standard coordinate vectors $e_1, \dots, e_k \in \mathbb{R}^H$ and set

$$A_k = \sum_{j=1}^{k} \sigma_j e_j v_j^{\mathsf{T}}, \qquad B_k = \sum_{j=1}^{k} u_j e_j^{\mathsf{T}}.$$

The choice is possible because $k \leq r \leq H$. Using $e_i^{\mathsf{T}}e_j = 0$ for $i \neq j$ and 1 for $i = j$ gives $B_kA_k = \sum_{j=1}^{k} \sigma_j u_j v_j^{\mathsf{T}} = \Phi_k$. For each $\ell > k$, every term in $A_k v_\ell$ contains $v_j^{\mathsf{T}} v_\ell = 0$, and every term in $B_k^{\mathsf{T}} u_\ell$ contains $u_j^{\mathsf{T}} u_\ell = 0$. Thus all rung conditions hold. At $k = 0$ both matrices are zero and the same conclusions follow from empty sums. This is the factorization used by the Lean construction; no balance condition is required. □

Lemma 10.2 (Exact discarded-mode restriction)**.** *Fix integers k, j with $0 \leq k < j \leq r$ and a point $w = (A, B) \in C_k$. Put $u = u_j \in \mathbb{R}^O$, $v = v_j \in \mathbb{R}^I$ and $\sigma = \sigma_j > 0$. For every $x, y \in \mathbb{R}^H$ define the matrix perturbation $D(x, y) = (\Delta A, \Delta B)$ by $\Delta A = xv^{\mathsf{T}}$ and $\Delta B = uy^{\mathsf{T}}$. Then*

$$L(A + \Delta A, B + \Delta B) - L(A, B) = \tfrac{1}{2}\|Bx\|^2 + \tfrac{1}{2}\|A^{\mathsf{T}}y\|^2 - \sigma\langle x, y\rangle + \tfrac{1}{2}\langle x, y\rangle^2. \tag{11}$$

Define $Q : \mathbb{R}^H \times \mathbb{R}^H \to \mathbb{R}$ *by*

$$Q(x, y) = \|Bx\|^2 + \|A^\mathsf{T} y\|^2 - 2\sigma\langle x, y\rangle.$$

For every $t \in \mathbb{R}$ *the exact restriction is*

$$L(w + tD(x, y)) - L(w) = \frac{t^2}{2}Q(x, y) + \frac{t^4}{2}\langle x, y\rangle^2, \qquad D^2L(w)[D(x, y), D(x, y)] = Q(x, y).$$

The map D *is an injective linear isometry from* $\mathbb{R}^H \times \mathbb{R}^H$*, with squared norm* $\|x\|^2 + \|y\|^2$*, onto its image in the parameter space.*

Proof. The perturbed error is $E+Bxv^\mathsf{T}+uy^\mathsf{T}A+u\langle x, y\rangle v^\mathsf{T}$. The two linear terms are orthogonal because $B^\mathsf{T}u = 0$ and $Av = 0$. Each is orthogonal to the quadratic term for the same reason. Their inner products with E vanish by $Ev = -\sigma u$ and $E^\mathsf{T}u = -\sigma v$. Finally $\langle E, uv^\mathsf{T}\rangle_F = -\sigma$ and u, v are unit vectors. Expanding the squared Frobenius norm gives (11). Replacing x, y by tx, ty gives the displayed identity for every $t \in \mathbb{R}$, because $D(tx, ty) = tD(x, y)$. The loss is polynomial, so the chain rule identifies the second derivative at $t = 0$ of $t \mapsto L(w + tD(x, y))$ with $D^2L(w)[D(x, y), D(x, y)]$. Differentiating the explicit polynomial twice gives $Q(x, y)$; the quartic term has zero second derivative at zero. Finally,

$$\|xv^\mathsf{T}\|_F^2 + \|uy^\mathsf{T}\|_F^2 = \|x\|^2\|v\|^2 + \|u\|^2\|y\|^2 = \|x\|^2 + \|y\|^2.$$

Linearity follows entry by entry from the definition of D. The norm identity proves that D is an isometry and has zero kernel. □

Lemma 10.3 (Two positive directions and one negative direction)**.** *Let* $A \in \mathbb{R}^{H\times I}$*,* $B \in \mathbb{R}^{O\times H}$ *and* $\sigma > 0$*, and define* Q *by the displayed formula in Lemma 10.2. If* $H \geq 2$ *and* $\operatorname{rank}(BA) < H$*, then* Q *is positive definite on the* H*-dimensional subspace* $\{(x, -x) : x \in \mathbb{R}^H\}$*, and there exist* $x, y \in \mathbb{R}^H$ *with* $Q(x, y) < 0$*.*

Proof. On $P = \{(x, -x) : x \in \mathbb{R}^H\}$,

$$Q(x, -x) = \|Bx\|^2 + \|A^\mathsf{T}x\|^2 + 2\sigma\|x\|^2 > 0 \quad (x \neq 0).$$

If both B were injective and A surjective, then $\operatorname{im}(BA) = B(\mathbb{R}^H)$ would have dimension H, a contradiction. Hence either $\ker B \neq 0$ or $\ker A^\mathsf{T} \neq 0$; the latter equivalence follows from finite-dimensional rank-nullity for A and its transpose.

If $0 \neq z \in \ker B$, then $Q(z, tz) = t^2\|A^\mathsf{T}z\|^2 - 2\sigma t\|z\|^2 < 0$ for sufficiently small $t > 0$. A concrete choice is $t = \sigma\|z\|^2/(\|A^\mathsf{T}z\|^2 + 1)$: the product $t\|A^\mathsf{T}z\|^2$ is strictly less than $\sigma\|z\|^2$. If instead $0 \neq z \in \ker A^\mathsf{T}$, use $Q(tz, z)$ with the analogous choice involving $\|Bz\|^2$. □

Lemma 10.4 (A positive plane and nearby lower values give an ambient band)**.** *Let* E *be a finite-dimensional real Euclidean space of dimension* n*, and let* $f : E \to \mathbb{R}$ *be continuous on* E *and* C^2 *on an open neighbourhood of* 0*. Assume* $f(0) = 0$ *and* $Df(0) = 0$*. Let* $P \subset E$ *be a two-dimensional linear subspace such that* $D^2f(0)[v, v] > 0$ *for every* $v \in P \setminus \{0\}$*. Assume also that for every real* $\rho > 0$ *there exists* $x \in E$ *satisfying* $\|x\| < \rho$ *and* $f(x) < 0$*. Then* $\mathsf{BandPair}(f, 0; 1, 1)$*, with the norm of* E *and its Euclidean Lebesgue measure. Global continuity matches the formal radial-control interface; the matrix loss satisfies it because it is polynomial.*

Proof. Complete an orthonormal basis of the specified plane and write the resulting coordinates as $(y, z) \in \mathbb{R}^2 \times \mathbb{R}^d$, where $d = n - 2 \geq 0$. We use f also for its pullback to these coordinates. This change preserves both norm and volume. Identify the planar differential $D_yf(y, z)$ with its gradient in $\mathbb{R}^2$. Its derivative in y at $(0, 0)$ is the positive-definite symmetric matrix

$D^2_{yy}f(0,0)$, hence an invertible matrix. The implicit-function theorem applies because f is C^2 and $D_yf(0,0)=0$. It gives a C^1 map c near $0\in\mathbb{R}^d$, with $c(0)=0$ and $D_yf(c(z),z)=0$.

The minimum of $D^2_{yy}f(0,0)[v,v]$ on the planar unit circle is strictly positive. Continuity of the Hessian in operator norm therefore gives a neighbourhood of $(0,0)$ and fixed constants $0<\kappa\le K$ on which all planar Hessians are bounded below by κ and above by K. Shrink the domain of c and choose $R_0,S_0>0$ so that $(c(z)+x,z)$ stays in this neighbourhood whenever $\|z\|<S_0$ and $\|x\|\le R_0$. Thus for every such z,x and every $v\in\mathbb{R}^2$,

$$\kappa\|v\|^2\le D^2_{yy}f(c(z)+x,z)[v,v]\le K\|v\|^2.$$

These radii and constants are fixed before the band radius is chosen. Put $g(z)=f(c(z),z)$ and, for a unit vector $e\in\mathbb{R}^2$, put $H_{z,e}(t)=f(c(z)+\sqrt{t}\,e,z)-g(z)$ for $0\le t\le R_0^2$. For each ordinary radius $0\le a\le R_0$, the fundamental theorem of calculus and the critical-center identity give

$$D_yf(c(z)+ae,z)[e]=\int_0^a D^2_{yy}f(c(z)+be,z)[e,e]\,db.$$

Since $\|e\|=1$, this lies between κa and Ka. Integrating with respect to a from $\sqrt{s}$ to $\sqrt{t}$ gives

$$\frac{\kappa}{2}(t-s)\le H_{z,e}(t)-H_{z,e}(s)\le\frac{K}{2}(t-s)\qquad(0\le s\le t\le R_0^2).\tag{12}$$

Thus $H_{z,e}$ is continuous and strictly increasing. Taking $s=0$ also gives $f(c(z)+x,z)\ge g(z)$ for every $\|x\|\le R_0$; for $x\neq0$ use $e=x/\|x\|$ and $t=\|x\|^2$, while $x=0$ is equality. This argument never differentiates $\sqrt{t}$ at zero.

Choose $\delta_0>0$ so small that $\delta_0<S_0$, $\delta_0<R_0/2$, and $\|c(z)\|<R_0/2$ whenever $\|z\|<\delta_0$. Restricting the tail radius to δ_0 also gives the uniform center-smallness bound relative to R_0. Fix any $0<\delta<\delta_0$. For the upper bound, a section of the ambient δ-ball at z lies in the disk of radius R_0 about $c(z)$. By the lower secant inequality, the squared radii where $|g(z)+H_{z,e}(t)|<\varepsilon$ occupy an interval of length at most $4\varepsilon/\kappa$. Polar area is one half of the integral of squared-radius length over the angle, so the section area is at most $4\pi\varepsilon/\kappa$, uniformly in z and its level shift $g(z)$. Tonelli over the finite-volume tail ball gives an upper bound $C_\delta\varepsilon$ for the full ambient band.

For the lower bound, set $R=\delta/4$. By continuity choose $0<S<\delta/4$ such that $\|c(z)\|<\delta/4$ and $|g(z)|<\kappa R^2/8$ for $\|z\|<S$. There is a point (y_*,z_*) with $f(y_*,z_*)<0$ sufficiently near zero that $\|z_*\|<S$ and $\|y_*-c(z_*)\|<R_0$. The minimum inequality gives $g(z_*)<0$. Write $\gamma=-g(z_*)>0$. Choose an open tail ball U about z_*, contained in $B(0,S)$, on which $g(z)<-\gamma/2$. This fixed ball has positive finite volume; if the tail dimension were zero, the negative value would contradict $g(0)=0$, so that case cannot occur under the hypotheses.

For $0<\varepsilon<\min(\gamma/4,\kappa R^2/8)$ and $z\in U$, both levels $-g(z)-\varepsilon$ and $-g(z)+\varepsilon$ lie strictly between 0 and $\kappa R^2/2\le H_{z,e}(R^2)$. The intermediate-value theorem and strict monotonicity give their unique preimages. The upper secant inequality shows that the intervening squared-radius interval has length at least $4\varepsilon/K$. Its planar band area is therefore at least $4\pi\varepsilon/K$. Every point used lies in the ambient δ-ball: $\|y\|\le\|c(z)\|+R<\delta/2$ and $\|z\|<\delta/4$. Integrating over U gives the lower bound $(4\pi/K)\,\mathrm{vol}(U)\varepsilon>0$. The tail window and all constants were fixed before ε. Shrink its cutoff below e^{-1} and, if necessary, decrease the lower constant to obtain exactly Definition 2.2. □

Proposition 10.5 (Preparation at an arbitrary rung point)**.** *For every integer k with $0\le k<r$ and every point $w_*=(A,B)\in C_k$, there exist orthogonal coordinates on the parameter space centered at w_*. Write $e:\mathbb{R}^2\times\mathbb{R}^d\to\mathbb{R}^{H\times I}\times\mathbb{R}^{O\times H}$ for their linear isometry, with $d=H(I+O)-2$, and define the centered loss by $f(y,z)=L_\Phi(w_*+e(y,z))-L_\Phi(w_*)$. These coordinates can be chosen so that f has the critical centers, uniform radial secants, and nearby*

lower values constructed in Lemma 10.4. These are the geometric data represented by the Lean record `O77.SaddleDirect.CenteredFamily` *and the separate lower-value predicate; the record contains no band-volume conclusion.*

Proof. Choose $j = k+1$ and set $u = u_j$, $v = v_j$, and $\sigma = \sigma_j$ as in Lemma 10.2. Since $k < r < H$, we have $H \geq 2$. Choose orthonormal vectors $e_1, e_2 \in \mathbb{R}^H$. The two parameter vectors

$$D(e_1, -e_1)/\sqrt{2}, \qquad D(e_2, -e_2)/\sqrt{2}$$

are orthonormal by the isometry in Lemma 10.2. For every $x \in \operatorname{span}(e_1, e_2)$, the Hessian on their plane satisfies

$$D^2L(w_*)[D(x,-x)/\sqrt{2}, D(x,-x)/\sqrt{2}] = \tfrac{1}{2}Q(x,-x) \geq \sigma\|x\|^2.$$

It is therefore positive definite with respect to the actual ambient Euclidean norm. Lemma 10.1 gives $\operatorname{rank}(BA) = k < H$, so Lemma 10.3 also supplies a vector (x, y) with $Q(x, y) < 0$. Along its actual matrix perturbation $D = (xv^{\mathsf{T}}, uy^{\mathsf{T}})$, the exact restriction is

$$L(w_* + tD) - L(w_*) = \frac{t^2}{2}\big(Q(x,y) + t^2\langle x, y\rangle^2\big).$$

For each prescribed $\delta > 0$, take

$$t = \min\left\{\frac{1}{2}, \frac{-Q(x,y)}{2(1+\langle x,y\rangle^2)}, \frac{\delta}{2(1+\|D\|)}\right\} > 0.$$

Then $t\|D\| < \delta$ and, since $t \leq 1/2$, $t^2\langle x, y\rangle^2 \leq -Q(x, y)/2$. The expression in parentheses in the exact restriction is at most $Q(x, y)/2 < 0$. Thus $w_* + tD$ belongs to the open δ-ball and has strictly smaller loss. This proves the required assertion in every neighbourhood, also when the quartic coefficient vanishes. The loss is a polynomial and is critical at w_* by Lemma 10.1; hence its actual derivatives satisfy the hypotheses of Lemma 10.4. That lemma constructs the centers and secants and proves the full-dimensional band estimate. The orthogonal coordinate change preserves the original Frobenius entry volume exactly. □

Theorem 10.6 (The pair at every nonoptimal rung point)**.** *For the dimensions and singular data fixed at the start of this section, for every integer k with $0 \leq k < r$ and every $w_* \in C_k$,*

$$\mathsf{BandPair}(L_\Phi, w_*; 1, 1).$$

Every C_k with $0 \leq k \leq r$ is nonempty, including when $r = 0$. For $r = 0$, only the nonoptimal-rung pair assertion is vacuous. The formula is pointwise on the whole specified rung, not merely at a balanced representative or at an infimum over the rung.

Assembly of Theorem 10.6. If $r = 0$, there is no index $k < r$. Otherwise fix an arbitrary $k < r$ and an arbitrary $w_* \in C_k$. Lemma 10.1 gives criticality and nonemptiness, while Lemmas 10.2 and 10.3 supply the positive two-plane and a negative direction. Proposition 10.5 converts these derivatives into the centered-family and lower-value data. The full-dimensional estimate in Lemma 10.4 gives the full-dimensional radius-first band pair $(1, 1)$; the orthogonal coordinate map and translation preserve the pair. No genericity, balance condition, or choice of a preferred representative has entered, so the argument is pointwise on the entire rung.

□

Necessary calibration. For the function $f : \mathbb{R}^2 \to \mathbb{R}$ given by $f(x, y) = xy$, Tonelli gives, on the square $(-a, a)^2$ and for $0 < \varepsilon < a^2$,

$$\operatorname{vol}\{(x, y) \in (-a, a)^2 : |xy| < \varepsilon\} = 4 \int_0^a \min\{a, \varepsilon/x\}\, dx = 4\varepsilon\big(1 + \log(a^2/\varepsilon)\big).$$

Here the integral is split at $x = \varepsilon/a$; the value at $x = 0$ is irrelevant to the integral. For any $\delta > 0$, the square $(-\delta/\sqrt{2}, \delta/\sqrt{2})^2$ lies inside the open δ-ball, which in turn lies inside $(-\delta, \delta)^2$. The two square formulas therefore bound its ball band volume above and below by positive constants times $\varepsilon \log(1/\varepsilon)$ for sufficiently small ε. Thus the pair at zero is $(1, 2)$. Indefiniteness alone does not give the $(1, 1)$ estimate; the two positive directions in our argument matter. For every fixed $\sigma > 0$, in the excluded scalar case $I = O = H = r = 1$, the loss $L(a, b) = \frac{1}{2}(ab - \sigma)^2$ satisfies

$$|L(a, b) - L(0, 0)| = |ab|\, |ab/2 - \sigma|.$$

On a sufficiently small ball the second factor is bounded above and below by positive constants. The pair is therefore $(1, 2)$ at $(0, 0)$. This example explains why the hypothesis $H > r$ cannot simply be removed.

11 Conclusion of the proof

Proof of Theorem 2.8. Fix the dimensions $I, O \geq 1$, $H \in \mathbb{N}$ and the target $\Phi \in \mathbb{R}^{O \times I}$ of rank $r < H$ from Theorem 2.8. At any exact fit $w = (A, B)$ put $a = \operatorname{rank}(A)$, $b = \operatorname{rank}(B)$, $s = Oa + b(I - a)$, $p = O - b$, $q = I - a$, and $h = H + r - a - b$. Theorem 9.1 supplies the pair $((s + d_0(p, q, h))/2, \mu_0(p, q, h))$. Its threshold is positive: if $s > 0$ this follows from $s/2$; if $s = 0$, the positive input/output dimensions force $a = b = 0$ and hence $r = 0$. Thus $p = O, q = I, h = H$ are positive. For an allowed integer t, the cost is $hq > 0$ when $t = 0$, while for $t > 0$ it is at least $pt > 0$. The minimum of this nonempty finite family is therefore $d_0 > 0$. Uniqueness, proved in Lemma 2.3, gives the full if-and-only-if pair classification. Lemma 4.1 realizes every feasible pair over the fixed target. Consequently comparing thresholds at actual points is equivalent to comparing Λ over the feasible numerical ranks. Theorem 8.4 identifies the entire equality locus and proves both extremal formulas. Its baseline ranks (r, r) are realized, so the least threshold and greatest multiplicity are attained. Finally, Lemma 10.1 inhabits every rung and Theorem 10.6 gives the pair at each nonoptimal point. For $r = 0$ only the pointwise nonoptimal assertion is vacuous; the zero rung is still inhabited. □

The construction proves the six clauses of `O77.Answers077`. The final declaration `O77.answers077` supplies the proved Wishart theorem to the intermediate assembly theorem `O77.answers077_of_wishart`; no Wishart hypothesis remains in its type. The exact declarations and the immutable proof bodies appear in Sections 47 and 82.

Part II
Detailed mathematical proofs beside the frozen Lean development

Conventions for the detailed proofs. Every dimension and finite index bound is a natural number, unless a different scalar type is explicitly stated. A real matrix has the row and column sizes displayed when it is introduced; matrix multiplication always contracts the matching inner index. Integration in matrix or vector entries means the product of one-dimensional Lebesgue measures. An empty coordinate product is a singleton of mass one. Sums and products over empty index sets are 0 and 1, respectively. Whenever a dimension occurs in a real exponent or a signed polynomial, it is cast before subtraction; a subtraction explicitly made in $\mathbb{N}$ is truncated. In each lemma, all variables mentioned in its hypotheses and conclusion are universally quantified unless introduced by an existential quantifier or chosen in its proof.

Notation in the code. `ℝ` is Lean's actual real field, `Matrix(Finm)(Finn)ℝ` is the space of rectangular matrices, and `RM` abbreviates this matrix type. `∫⁻` is the nonnegative Lebesgue integral in `ENNReal`; finiteness is established before using a real-valued integral identity or cancelling a potentially infinite quantity. `HasFDerivAt`, `HasStrictFDerivAt`, and `ContDiff` express actual derivatives and differentiability, not assumed formulas for a named Hessian. Function and matrix types may carry non-Euclidean default norms; all native volume statements use the explicit entry norm and entry measure. Each local coordinate type and its normalization is introduced in the chapter that uses it.

Public statements and helper statements. A structure that stores a center, a linear equivalence, or a derivative bound is an intermediate interface. Its fields must be proved when it is constructed. The proofs below keep those constructors visible. `AnswersO77` is defined only from the actual measured predicate, ranks, and rung membership. The last main module proves it without a Wishart, chart, or realization parameter. The scalar and zero-dimensional controls are retained; their role is to test normalization, not to replace universal proofs.

12 O77: finite residual arithmetic

This chapter develops the finite arithmetic, exact matrix identities, and basic analytic lemmas used by the later modules. The ordinary argument is followed immediately by the corresponding frozen Lean block. Arithmetic conclusions are initially numerical; the measured interpretation is supplied by the actual-point constructions later in this part.

Imports and scope. The two initial fragments supply the original imports and open the namespace. They are retained exactly, as are the later namespace boundaries. `autoImplicit false` makes undeclared parameters an error. This chapter uses finite entry functions for its elementary algebra and Mathlib's actual real numbers and measures for analysis.

```
import Mathlib.Analysis.SpecialFunctions.PolarCoord
import Mathlib.MeasureTheory.Integral.Prod
import Mathlib.MeasureTheory.Measure.Prod
import Mathlib.MeasureTheory.Group.Measure
import Mathlib.MeasureTheory.Measure.Lebesgue.VolumeOfBalls
import Mathlib.Tactic.FunProp
import Mathlib.Tactic.FieldSimp
import Mathlib.Tactic.Linarith
import Mathlib.Tactic.Positivity
```

```
import Mathlib.Tactic.Ring
import Mathlib.Analysis.SpecialFunctions.Gamma.Basic
import Mathlib.Analysis.SpecialFunctions.Integrals.Basic
import Mathlib.MeasureTheory.Integral.IntervalIntegral.Basic
import Mathlib.LinearAlgebra.Matrix.PosDef
import Mathlib.LinearAlgebra.Matrix.SchurComplement
import Mathlib.Algebra.Order.Star.Real
import Mathlib.MeasureTheory.Integral.Pi
import Mathlib.Analysis.Matrix.MeasurableSpace
import Mathlib.Analysis.Matrix.Normed
import Mathlib.Analysis.SpecialFunctions.Gaussian.FourierTransform
import Mathlib.Analysis.Matrix.PosDef
import Mathlib.MeasureTheory.Measure.Lebesgue.Basic
import Mathlib.Data.Fin.Tuple.Sort
import Mathlib.Data.Fintype.Perm
import Mathlib.MeasureTheory.Measure.Lebesgue.EqHaar
import Mathlib.MeasureTheory.Integral.Lebesgue.Map
```

```
import Std

set_option autoImplicit false

namespace O77
```

12.1 Numerical rank data, without geometric conclusions

Definition 12.1 (Numerical feasibility and block shapes)**.**

Corresponding Lean declaration(s): `O77.RankData`,`O77.BlockShape`,`O77.rankShape`.

Let $I, O, H, r, a, b \in \mathbb{N}$. *The predicate* RankData(I, O, H, r, a, b) *consists exactly of*

$$r \le a, \quad r \le b, \quad a \le I, \quad a \le H, \quad b \le O, \quad b \le H, \quad a + b \le H + r.$$

Define $h = (H + r) - (a + b)$ *and* $s = Oa + b(I - a)$ *as total natural-number functions. Their interpretation as dimensions is asserted only with the feasibility inequalities. A block shape is a tuple* $d = (r, \alpha, \beta, h, p, q) \in \mathbb{N}^6$*, with*

$$I_d = r + \alpha + q, \qquad O_d = r + \beta + p, \qquad H_d = r + \alpha + \beta + h.$$

The associated shape of numerical rank data is $(r, a - r, b - r, (H + r) - (a + b), O - b, I - a)$*. All subtractions in this definition are natural subtractions. A proof of feasibility has no coordinate, volume, normal-form, or answer field.*

The inequalities are proved for actual matrix ranks in `O77AdaptedSections`; their sufficiency over each fixed target is proved in `O77FiberClassification`. They are not extra geometric fields of this record.

```
/-- Numerical feasibility only; no geometric or asymptotic conclusion is a field. -/
structure RankData (I O H r a b : Nat) : Prop where
  r_le_a : r ≤ a
  r_le_b : r ≤ b
  a_le_I : a ≤ I
  a_le_H : a ≤ H
  b_le_O : b ≤ O
  b_le_H : b ≤ H
  sum_le : a + b ≤ H + r

def residualHidden (H r a b : Nat) : Nat := (H + r) - (a + b)
def regularDim (I O a b : Nat) : Nat := O * a + b * (I - a)
```

```
structure BlockShape where
  r : Nat
  alpha : Nat
  beta : Nat
  h : Nat
  p : Nat
  q : Nat

def BlockShape.input (d : BlockShape) : Nat := d.r + d.alpha + d.q
def BlockShape.output (d : BlockShape) : Nat := d.r + d.beta + d.p
def BlockShape.hidden (d : BlockShape) : Nat :=
  d.r + d.alpha + d.beta + d.h

def rankShape (I O H r a b : Nat) : BlockShape :=
  ⟨r, a - r, b - r, residualHidden H r a b, O - b, I - ⟩a
```

Lemma 12.2 (Reconstruction and feasibility of a shape)**.**

Corresponding Lean declaration(s): `077.rankShape_dimensions`,`077.shape_feasible`.

If $D : \mathsf{RankData}(I, O, H, r, a, b)$*, its associated shape has dimensions* I, O, H*. Conversely, every* $d = (r, \alpha, \beta, h, p, q) \in \mathbb{N}^6$ *has feasible rank data* $(I_d, O_d, H_d, r, r+\alpha, r+\beta)$*.*

Proof. For the forward assertion, $r \le a \le I$ gives $r+(a-r)+(I-a) = I$, and $r \le b \le O$ gives the analogous equality for O. Moreover $r+(a-r)+(b-r) = a+b-r$. Since $a+b \le H+r$ and $a, b \ge r$, adding $(H+r)-(a+b)$ to this last value gives H. For the converse, $r \le r+\alpha, r+\beta$; the input and output bounds follow by adding q and p, respectively. Both ranks are at most H_d, because the complementary summands are nonnegative. Finally $(r+\alpha)+(r+\beta) \le H_d+r$ is equivalent to $0 \le h$. These are exactly the seven fields of the predicate, including all zero-size cases. □

```
theorem rankShape_dimensions {I O H r a b : Nat}
    (D : RankData I O H r a b) :
    (rankShape I O H r a b).input = I ∧
    (rankShape I O H r a b).output = O ∧
    (rankShape I O H r a b).hidden = H := by
  rcases D with ⟨hra, hrb, haI, haH, hbO, hbH, ⟩hab
  simp only [rankShape, BlockShape.input, BlockShape.output,
    BlockShape.hidden, residualHidden]
  omega

theorem shape_feasible (d : BlockShape) :
    RankData d.input d.output d.hidden d.r
      (d.r + d.alpha) (d.r + d.beta) := by
  constructor <;>
    (try simp only [BlockShape.input, BlockShape.output, BlockShape.hidden]) <;>
    omega
```

Lemma 12.3 (Regular and residual generator dimensions)**.**

Corresponding Lean declaration(s): `077.regularDim_add_residual`.

For natural I, O, a, b *with* $a \le I$ *and* $b \le O$*,*

$$\big(Oa + b(I-a)\big) + (O-b)(I-a) = OI.$$

Proof. The hypotheses give $a+(I-a) = I$ and $b+(O-b) = O$ in $\mathbb{N}$. Distributivity transforms the left side into $Oa+(b+(O-b))(I-a) = O(a+(I-a)) = OI$. Neither of these subtraction cancellations is used without its inequality. □

```
theorem regularDim_add_residual {I O a b : Nat}
    (ha : a ≤ I) (hb : b ≤ O) :
    regularDim I O a b + (O - b) * (I - a) = O * I := by
  unfold regularDim
  have ha' : a + (I - a) = I := by omega
  have hb' : b + (O - b) = O := by omega
  grind
```

12.2 Reusable finite minima with complete witnesses

Definition 12.4 (Minimum on an initial interval)**.**

Corresponding Lean declaration(s): `O77.Arithmetic.minimumUpTo`,`O77.Arithmetic.IsMinimumOn`,`O77.Arithmetic.minimizerList`.

For a function $f : \mathbb{N} \to \mathbb{Z}$ and a bound $K \in \mathbb{N}$, define $M_f(0) = f(0)$ and $M_f(K+1) = \min(M_f(K), f(K+1))$ recursively. For an index $t \in \mathbb{N}$, define

$$\mathsf{IsMin}_K(f,t) \quad \Longleftrightarrow \quad t \le K \text{ and } \forall u \in \mathbb{N}\,(u \le K \Rightarrow f(t) \le f(u)).$$

Let $\mathcal{M}_f(K)$ be the list obtained from $[0,1,\dots,K]$ by retaining exactly the indices t for which $f(t) = M_f(K)$. The test is decidable because both values are integers. The initial interval is never empty, including when $K = 0$.

This definition is independent of the later vertex formula. In particular, the proposed minimizers are not used to define what it means to minimize.

```
namespace Arithmetic

/-- A minimum of the nonempty interval 0,...,K, computed without choice. -/
def minimumUpTo (f : Nat → Int) : Nat → Int
  | 0 => f 0
  | K + 1 => min (minimumUpTo f K) (f (K + 1))

def IsMinimumOn (f : Nat → Int) (K t : Nat) : Prop :=
  t ≤ K ∧ ∀ u : Nat, u ≤ K → f t ≤ f u

def minimizerList (f : Nat → Int) (K : Nat) : List Nat :=
  (List.range (K + 1)).filter (fun t => decide (f t = minimumUpTo f K))
```

Lemma 12.5 (Lower bound and attainment)**.**

Corresponding Lean declaration(s): `O77.Arithmetic.minimumUpTo_le`,`O77.Arithmetic.minimumUpTo_attained`.

For every $f : \mathbb{N} \to \mathbb{Z}$, $K \in \mathbb{N}$, and $t \le K$, one has $M_f(K) \le f(t)$. There is a $t \le K$ such that $f(t) = M_f(K)$.

Proof. Both assertions are proved by induction on K. At $K = 0$, the sole index is 0 and the value is $f(0)$. Suppose the assertions hold at K. If $t \le K+1$, either $t = K+1$, in which case the right entry of the minimum bounds $M_f(K+1)$, or $t \le K$, in which case $M_f(K+1) \le M_f(K) \le f(t)$. For attainment, choose an old witness $t \le K$. When $M_f(K) \le f(K+1)$, this same witness attains the new minimum. Otherwise the new index $K+1$ attains it. These cases also cover an equality of the two candidate values; the witness need not be unique. □

```
theorem minimumUpTo_le (f : Nat → Int) {K t : Nat}
    (ht : t ≤ K) : minimumUpTo f K ≤ f t := by
  induction K with
  | zero =>
```

```
      have ht0 : t = 0 := by omega
      subst t
      exact Int.le_refl _
  | succ K ih =>
      by_cases heq : t = K + 1
      · subst t
        exact Int.min_le_right _ _
      · have htK : t ≤ K := by omega
        exact Int.le_trans (Int.min_le_left _ _) (ih htK)

theorem minimumUpTo_attained (f : Nat → Int) (K : Nat) :
    ∃ t : Nat, t ≤ K ∧ f t = minimumUpTo f K := by
  induction K with
  | zero => exact ⟨0, Nat.le_refl _, ⟩rfl
  | succ K ih =>
      rcases ih with ⟨t, ht, ⟩hval
      by_cases hle : minimumUpTo f K ≤ f (K + 1)
      · refine ⟨t, by omega, ?⟩_
        rw [minimumUpTo, Int.min_def, ite_eq_left hle, ← hval]
      · refine ⟨K + 1, Nat.le_refl _, ?⟩_
        rw [minimumUpTo, Int.min_def, ite_eq_right hle]
```

Lemma 12.6 (Semantic characterization and nonnegative values)**.**

Corresponding Lean declaration(s): `O77.Arithmetic.isMinimumOn_iff_value`,`O77.Arithmetic.mem_minimizerList`,`O77.Arithmetic.minimumUpTo_nonnegative`.

For every $f : \mathbb{N} \to \mathbb{Z}$ and every $K, t \in \mathbb{N}$,

$$\mathsf{IsMin}_K(f,t) \iff [t \le K \text{ and } f(t) = M_f(K)] \iff t \in \mathcal{M}_f(K).$$

If $f(u) \ge 0$ for every $u \le K$, then $M_f(K) \ge 0$.

Proof. If t is minimizing, compare $f(t)$ with the value at an attaining index from Lemma 12.5. This gives $f(t) \le M_f(K)$, whereas that lemma also gives $M_f(K) \le f(t)$. Antisymmetry gives equality. Conversely, the equality and the lower-bound property give $f(t) \le f(u)$ for every admissible u. The filtered list contains t exactly when $t < K + 1$ and $f(t) = M_f(K)$; for natural indices $t < K + 1$ is equivalent to $t \le K$. Finally the attained value is one of the assumed nonnegative values, proving the last assertion. □

```
theorem isMinimumOn_iff_value (f : Nat → Int) (K t : Nat) :
    IsMinimumOn f K t ↔ t ≤ K ∧ f t = minimumUpTo f K := by
  constructor
  · rintro ⟨ht, ⟩hmin
    rcases minimumUpTo_attained f K with ⟨u, hu, ⟩hval
    have hlow := minimumUpTo_le f ht
    have hupp := hmin u hu
    exact ⟨ht, by ⟩omega
  · rintro ⟨ht, ⟩hval
    refine ⟨ht, ?⟩_
    intro u hu
    rw [hval]
    exact minimumUpTo_le f hu

theorem mem_minimizerList (f : Nat → Int) (K t : Nat) :
    t ∈ minimizerList f K ↔ IsMinimumOn f K t := by
  rw [isMinimumOn_iff_value]
  simp only [minimizerList, List.mem_filter, List.mem_range, decide_eq_true_eq]
  omega

theorem minimumUpTo_nonnegative (f : Nat → Int) (K : Nat)
```

```
    (hf : ∀ t : Nat, t ≤ K → 0 ≤ f t) :
    0 ≤ minimumUpTo f K := by
  rcases minimumUpTo_attained f K with ⟨t, ht, ⟩hval
  rw ←[ hval]
  exact hf t ht
```

Lemma 12.7 (Nonemptiness and restriction to the actual interval)**.**

Corresponding Lean declaration(s): `O77.Arithmetic.minimizerList_nonempty`,`O77.Arithmetic.minimumUpTo_congr`,`O77.Arithmetic.minimizerList_congr`.

The list $\mathcal{M}_f(K)$ is nonempty. If $f, g : \mathbb{N} \to \mathbb{Z}$ satisfy $f(t) = g(t)$ for all $t \le K$, then $M_f(K) = M_g(K)$ and $\mathcal{M}_f(K) = \mathcal{M}_g(K)$ as lists, not only as sets.

Proof. An attaining index belongs to $\mathcal{M}_f(K)$ by the preceding lemma. For congruence choose attaining indices t for f and u for g. Then $M_g(K) \le g(t) = f(t) = M_f(K)$ and $M_f(K) \le f(u) = g(u) = M_g(K)$, proving equality. At each entry of the common list $[0, \dots, K]$, both equality tests now have the same truth value. Filtering by identical tests on all entries gives identical lists. No condition is imposed on either function outside the finite interval. □

```
theorem minimizerList_nonempty (f : Nat → Int) (K : Nat) :
    minimizerList f K ≠ [] := by
  rcases minimumUpTo_attained f K with ⟨t, ht, ⟩hval
  apply List.ne_nil_of_mem (a := t)
  rw [mem_minimizerList, isMinimumOn_iff_value]
  exact ⟨ht, ⟩hval

theorem minimumUpTo_congr (f g : Nat → Int) (K : Nat)
    (heq : ∀ t : Nat, t ≤ K → f t = g t) :
    minimumUpTo f K = minimumUpTo g K := by
  rcases minimumUpTo_attained f K with ⟨t, ht, ⟩htval
  rcases minimumUpTo_attained g K with ⟨u, hu, ⟩huval
  have h1 := minimumUpTo_le g ht
  have h2 := minimumUpTo_le f hu
  rw ←[ heq t ht] at h1
  rw [heq u hu] at h2
  omega

theorem minimizerList_congr (f g : Nat → Int) (K : Nat)
    (heq : ∀ t : Nat, t ≤ K → f t = g t) :
    minimizerList f K = minimizerList g K := by
  unfold minimizerList
  apply List.filter_congr
  intro t ht
  have htK : t ≤ K := by have := List.mem_range.mp ht; omega
  rw [heq t htK, minimumUpTo_congr f g K heq]
```

12.3 Integer quadratics: a reusable complete minimization theorem

Definition 12.8 (Quadratic with a signed vertex parameter)**.**

Corresponding Lean declaration(s): `O77.Arithmetic.quadratic`.

For $B, D \in \mathbb{Z}$ and $t \in \mathbb{N}$, write $Q_{B,D}(t) = B + t^2 - Dt$, where t is coerced to $\mathbb{Z}$ before the displayed operations. The signed value D is not truncated.

```
/-- Constant term B and signed vertex parameter D stay independent. -/
def quadratic (B D : Int) (t : Nat) : Int :=
  B + (t : Int) * (t : Int) - D * (t : Int)
```

Lemma 12.9 (Difference and increment identities)**.**

Corresponding Lean declaration(s): `077.Arithmetic.quadratic_difference,077.Arithmetic.quadratic_increment`.

For all $B, D \in \mathbb{Z}$ and $u, t \in \mathbb{N}$,

$$Q_{B,D}(u) - Q_{B,D}(t) = (u - t)(u + t - D), \qquad Q_{B,D}(t + 1) - Q_{B,D}(t) = 2t + 1 - D.$$

All differences in these identities are in $\mathbb{Z}$.

Proof. In the first difference the two constant terms cancel. The remaining expression is $u^2 - t^2 - D(u - t) = (u - t)(u + t - D)$ by distributivity. Substituting $u = t + 1$ gives the second identity, since $(t + 1) - t = 1$. This computation has no range or sign hypotheses. □

```
theorem quadratic_difference (B D : Int) (u t : Nat) :
    quadratic B D u - quadratic B D t =
      ((u : Int) - (t : Int)) * ((u : Int) + (t : Int) - D) := by
  unfold quadratic
  grind

theorem quadratic_increment (B D : Int) (t : Nat) :
    quadratic B D (t + 1) - quadratic B D t = 2 * (t : Int) + 1 - D := by
  unfold quadratic
  grind
```

Lemma 12.10 (Two neighbour inequalities characterize the global minimum)**.**

Corresponding Lean declaration(s): `077.Arithmetic.quadratic_minimum_iff`.

For $B, D \in \mathbb{Z}$ and $K, t \in \mathbb{N}$,

$$\mathsf{IsMin}_K(Q_{B,D}, t) \iff \begin{cases} t \le K, \\ t = 0 \text{ or } 2t - 1 \le D, \\ t = K \text{ or } D \le 2t + 1. \end{cases}$$

The inequalities involving D take place in $\mathbb{Z}$.

Proof. A minimizing index belongs to the interval. If $t > 0$, compare it with $t-1$; Lemma 12.9 gives $2t - 1 - D \le 0$. If $t < K$, compare it with $t + 1$; the same lemma gives $2t + 1 - D \ge 0$. This proves necessity, without asking for a nonexistent neighbour at either endpoint.

Conversely assume the three displayed conditions, and let $u \le K$. At $u = t$ the values agree. If $u < t$, then $t > 0$, $u \le t - 1$, and

$$Q(u)-Q(t) = (t-u)(D-u-t) \ge 0 : \quad t-u \ge 0, \quad D-u-t \ge (2t-1)-u-t = t-u-1 \ge 0.$$

If $u > t$, then $t < K$, $u \ge t + 1$, and

$$Q(u)-Q(t) = (u-t)(u+t-D) \ge 0 : \quad u-t \ge 0, \quad u+t-D \ge u+t-(2t+1) = u-t-1 \ge 0.$$

Each nonnegativity assertion is an ordinary ordered-integer product inequality. Thus $Q(t) \le Q(u)$ for every admissible u. When $K = 0$, the condition reduces to $t = 0$, as required. □

```
/-- Global minimization is equivalent to two integer-neighbour inequalities. -/
theorem quadratic_minimum_iff (B D : Int) (K t : Nat) :
    IsMinimumOn (quadratic B D) K t ↔
      t ≤ K ∧
      (t = 0 ∨ 2 * (t : Int) - 1 ≤ D) ∧
      (t = K ∨ D ≤ 2 * (t : Int) + 1) := by
  constructor
```

```
  · rintro ⟨ht, ⟩hmin
    refine ⟨ht, ?_, ?⟩_
    · by_cases ht0 : t = 0
      · exact Or.inl ht0
      · right
        have hp : t - 1 ≤ K := by omega
        have hcmp := hmin (t - 1) hp
        have hi := quadratic_increment B D (t - 1)
        have heq : t - 1 + 1 = t := by omega
        rw [heq] at hi
        omega
    · by_cases htK : t = K
      · exact Or.inl htK
      · right
        have hn : t + 1 ≤ K := by omega
        have hcmp := hmin (t + 1) hn
        have hi := quadratic_increment B D t
        omega
  · rintro ⟨ht, hl, ⟩hr
    refine ⟨ht, ?⟩_
    intro u hu
    by_cases hut : u = t
    · subst u; exact Int.le_refl _
    · by_cases hlt : u < t
      · have hl' : 2 * (t : Int) - 1 ≤ D := by omega
        have h1 : 0 ≤ (t : Int) - (u : Int) := by omega
        have h2 : 0 ≤ D - (u : Int) - (t : Int) := by omega
        have hp := Int.mul_nonneg h1 h2
        have hd : quadratic B D u - quadratic B D t =
            ((t : Int) - (u : Int)) * (D - (u : Int) - (t : Int)) := by
          unfold quadratic; grind
        omega
      · have hr' : D ≤ 2 * (t : Int) + 1 := by omega
        have h1 : 0 ≤ (u : Int) - (t : Int) := by omega
        have h2 : 0 ≤ (u : Int) + (t : Int) - D := by omega
        have hp := Int.mul_nonneg h1 h2
        have hd := quadratic_difference B D u t
        omega
```

Definition 12.11 (Clamped lower vertex)**.**

Corresponding Lean declaration(s): `O77.Arithmetic.lowerVertex`.

For $D \in \mathbb{Z}$ and $K \in \mathbb{N}$, define

$$v(D,K) = \min\left(\left\lfloor \frac{\max(D,0)}{2} \right\rfloor, K\right) \in \mathbb{N}.$$

In the code, D.toNat denotes $\max(D,0)$ viewed as a natural number, and division by 2 is Euclidean natural division. This truncation belongs to the algorithm for an index*, not to the coefficient D of the quadratic.*

Lemma 12.12 (Bounds and a minimizing witness)**.**

Corresponding Lean declaration(s): `O77.Arithmetic.lowerVertex_bounds`,`O77.Arithmetic.lowerVertex_isMinimum`.

For $v = v(D,K)$,

$$v \le K, \qquad (v = 0 \text{ or } 2v \le D), \qquad (v = K \text{ or } D \le 2v+1).$$

Consequently $\mathsf{IsMin}_K(Q_{B,D}, v)$ *for every $B \in \mathbb{Z}$.*

Proof. The first bound follows from the minimum defining v. If $v > 0$, then $\lfloor \max(D, 0)/2 \rfloor \geq v \geq 1$, so $D \geq 0$ and $2v \leq D$. If $v < K$, the minimum did not clamp at K, hence $v = \lfloor \max(D, 0)/2 \rfloor$. For $D \geq 0$ Euclidean division gives $D = 2v + e$ with $e \in \{0, 1\}$; for $D < 0$ one has $v = 0$ and $D \leq 1$. This proves the last bound in both cases. It implies the right-neighbour condition; the stronger left bound $2v \leq D$ implies $2v - 1 \leq D$. Lemma 12.10 proves the final assertion. □

```
/-- The lower nearest integer to the vertex, clamped to the allowed interval. -/
def lowerVertex (D : Int) (K : Nat) : Nat := min (D.toNat / 2) K

theorem lowerVertex_bounds (D : Int) (K : Nat) :
    lowerVertex D K ≤ K ∧
    (lowerVertex D K = 0 ∨ 2 * (lowerVertex D K : Int) ≤ D) ∧
    (lowerVertex D K = K ∨ D ≤ 2 * (lowerVertex D K : Int) + 1) := by
  unfold lowerVertex
  omega

theorem lowerVertex_isMinimum (B D : Int) (K : Nat) :
    IsMinimumOn (quadratic B D) K (lowerVertex D K) := by
  rw [quadratic_minimum_iff]
  have h := lowerVertex_bounds D K
  omega
```

Lemma 12.13 (Every minimizing index, including a possible tie)**.**

Corresponding Lean declaration(s): `077.Arithmetic.quadratic_minimum_indices`.

For $B, D \in \mathbb{Z}$, $K, t \in \mathbb{N}$, and $v = v(D, K)$,

$$\mathsf{IsMin}_K(Q_{B,D}, t) \iff t = v \text{ or } [v < K,\ D = 2v + 1,\ t = v + 1].$$

Proof. The index v is minimizing. Any other minimizing t has the same value, by Lemma 12.6. The difference identity therefore gives $(t - v)(t + v - D) = 0$. Since $\mathbb{Z}$ has no zero divisors, either $t = v$ or $D = t + v$. In the second case, $t < v$ is impossible: it would imply $v > 0$ and $D = t + v \leq 2v - 1 < 2v \leq D$. Thus, unless $t = v$, one has $t > v$. Since $t \leq K$, this implies $v < K$, and the upper bound from Lemma 12.12 yields $t + v = D \leq 2v + 1$. The natural inequalities $v < t \leq v + 1$ force $t = v + 1$, and $D = 2v + 1$ follows.

Conversely $t = v$ is already known to minimize. If the second alternative holds, then $t = v + 1 \leq K$, $2t - 1 = D$, and $D \leq 2t + 1$. These satisfy both neighbour conditions, including the case $t = K$. The preceding local-test lemma gives minimization. □

```
theorem quadratic_minimum_indices (B D : Int) (K t : Nat) :
    IsMinimumOn (quadratic B D) K t ↔
      t = lowerVertex D K ∨
        (lowerVertex D K < K ∧
         D = 2 * (lowerVertex D K : Int) + 1 ∧
         t = lowerVertex D K + 1) := by
  let v := lowerVertex D K
  have hv := lowerVertex_bounds D K
  change v ≤ K ∧ (v = 0 ∨ 2 * (v : Int) ≤ D) ∧
    (v = K ∨ D ≤ 2 * (v : Int) + 1) at hv
  change IsMinimumOn (quadratic B D) K t ↔
    t = v ∨ (v < K ∧ D = 2 * (v : Int) + 1 ∧ t = v + 1)
  constructor
  · intro ht
    have htval := (isMinimumOn_iff_value _ K t).mp ht
    have hvval := (isMinimumOn_iff_value _ K v).mp
      (lowerVertex_isMinimum B D K)
    have hd := quadratic_difference B D t v
```

```
    have hprod : ((t : Int) - (v : Int)) *
        ((t : Int) + (v : Int) - D) = 0 := by omega
    rcases Int.mul_eq_zero.mp hprod with hsame | hsum
    · left; omega
    · omega
  · intro ht
    rw [quadratic_minimum_iff]
    omega
```

Lemma 12.14 (Multiplicity as an actual count, not a case-definition)**.**

Corresponding Lean declaration(s): `O77.Arithmetic.minimizerList_nodup`,`O77.Arithmetic.quadratic_minimizer_count`.

Every list $\mathcal{M}_f(K)$ has no duplicate entries. For $v = v(D, K)$,

$$\#\mathcal{M}_{Q_{B,D}}(K) = \begin{cases} 2, & v < K \text{ and } D = 2v + 1, \\ 1, & \text{otherwise.} \end{cases}$$

Here # is list length, which equals the cardinality of its represented set because there are no duplicates.

Proof. The list $[0, \ldots, K]$ has no repeated entry; a filtered sublist cannot acquire repetitions. By Lemma 12.13, in the tie case the filtered list has exactly the elements $v, v + 1$, which are distinct, and otherwise it has exactly the element v. Two finite lists without repetitions and with the same members are permutations of one another. Apply this fact to $[v, v + 1]$ or $[v]$ and take lengths. This argument proves the count for the independently defined filtered list; the list itself was not defined by this two-case formula. □

```
theorem minimizerList_nodup (f : Nat → Int) (K : Nat) :
    (minimizerList f K).Nodup := by
  exact List.Nodup.sublist List.filter_sublist List.nodup_range

theorem quadratic_minimizer_count (B D : Int) (K : Nat) :
    (minimizerList (quadratic B D) K).length =
      if lowerVertex D K < K ∧ D = 2 * (lowerVertex D K : Int) + 1
      then 2 else 1 := by
  let v := lowerVertex D K
  change (minimizerList (quadratic B D) K).length =
    if v < K ∧ D = 2 * (v : Int) + 1 then 2 else 1
  by_cases h : v < K ∧ D = 2 * (v : Int) + 1
  · have hn : ([v, v + 1] : List Nat).Nodup := by simp
    have he : (minimizerList (quadratic B D) K).Perm [v, v + 1] := by
      apply (List.perm_ext_iff_of_nodup (minimizerList_nodup _ _) hn).mpr
      intro t
      rw [mem_minimizerList, quadratic_minimum_indices]
      change (t = v ∨ (v < K ∧ D = 2 * (v : Int) + 1 ∧ t = v + 1)) ↔ _
      simp [h]
    simpa [h] using he.length_eq
  · have hn : ([v] : List Nat).Nodup := by simp
    have he : (minimizerList (quadratic B D) K).Perm [v] := by
      apply (List.perm_ext_iff_of_nodup (minimizerList_nodup _ _) hn).mpr
      intro t
      rw [mem_minimizerList, quadratic_minimum_indices]
      change (t = v ∨ (v < K ∧ D = 2 * (v : Int) + 1 ∧ t = v + 1)) ↔ _
      simp only [List.mem_singleton]
      omega
    simpa [h] using he.length_eq
```

12.4 Residual arithmetic and its exact connection to the blueprint

Definition 12.15 (Executable residual data)**.**

Corresponding Lean declaration(s): `O77.residualCost`,`O77.residualQuadratic`,`O77.residualMin`,`O77.residualMinimizerList`,`O77.residualMultiplicity`,`O77.residualVertex`.

For $p, q, h, t \in \mathbb{N}$ put $K = \min(h, q)$ and define

$$c_N(t) = (h - t)(q - t) + pt \in \mathbb{N}, \qquad c_Z(t) = Q_{hq,h+q-p}(t) \in \mathbb{Z}.$$

The coefficient operations in c_Z take place in $\mathbb{Z}$. Define

$$d_N = \big(M_{c_Z}(K)\big).\mathsf{toNat}, \qquad \mathcal{R} = \mathcal{M}_{c_Z}(K), \qquad \mu_N = \#\mathcal{R}, \qquad v = v(h + q - p, K).$$

These are the code's `residualMin`, `residualMinimizerList`, `residualMultiplicity`, and `residualVertex`.

The next two lemmas establish, rather than assume, that $d_N = d_0(p, q, h)$ and $\mu_N = \mu_0(p, q, h)$ from Definition 2.5. The conversion `toNat` is not allowed to discard a negative minimum silently. The nonnegativity proof is part of that identification. The list is a standard-library representation of the finite minimizer set; any alternative finite-set representation must preserve this membership specification.

```
end Arithmetic

/-- The finite cost uses natural subtraction only on guarded indices. -/
def residualCost (p q h t : Nat) : Nat := (h - t) * (q - t) + p * t

def residualQuadratic (p q h : Nat) : Nat → Int :=
  Arithmetic.quadratic ((h : Int) * (q : Int))
    ((h : Int) + (q : Int) - (p : Int))

def residualMin (p q h : Nat) : Nat :=
  (Arithmetic.minimumUpTo (residualQuadratic p q h) (min h q)).toNat

def residualMinimizerList (p q h : Nat) : List Nat :=
  Arithmetic.minimizerList (residualQuadratic p q h) (min h q)

def residualMultiplicity (p q h : Nat) : Nat :=
  (residualMinimizerList p q h).length

def residualVertex (p q h : Nat) : Nat :=
  Arithmetic.lowerVertex ((h : Int) + (q : Int) - (p : Int)) (min h q)
```

Lemma 12.16 (Guarded coercion and no lost negative value)**.**

Corresponding Lean declaration(s): `O77.Arithmetic.residualCost_cast`,`O77.Arithmetic.residualQuadratic_nonnegative`,`O77.Arithmetic.residualMin_cast`.

For all $p, q, h, t \in \mathbb{N}$ with $t \le K$, the integer coercion of $c_N(t)$ is $c_Z(t)$. In particular $c_Z(t) \ge 0$. Moreover the integer coercion of d_N equals $M_{c_Z}(K)$.

Proof. The bound $t \le \min(h, q)$ implies $t \le h, q$, so natural subtraction commutes with integer coercion for both $h-t$ and $q-t$. Expanding gives $(h-t)(q-t)+pt = hq+t^2-(h+q-p)t = c_Z(t)$. A coerced natural number is nonnegative. All values on the finite interval are therefore nonnegative; Lemma 12.6 shows that its attained minimum is nonnegative. For a nonnegative integer z, coercing $z.\mathsf{toNat}$ back to $\mathbb{Z}$ returns z. Apply this only after the nonnegativity has been proved. □

```
namespace Arithmetic

theorem residualCost_cast (p q h t : Nat) (ht : t ≤ min h q) :
    (residualCost p q h t : Int) = residualQuadratic p q h t := by
  have hth : t ≤ h := by omega
  have htq : t ≤ q := by omega
  simp only [residualCost, Int.natCast_add, Int.natCast_mul,
    Int.natCast_sub hth, Int.natCast_sub htq,
    residualQuadratic, quadratic]
  grind

theorem residualQuadratic_nonnegative (p q h t : Nat)
    (ht : t ≤ min h q) : 0 ≤ residualQuadratic p q h t := by
  rw ←[ residualCost_cast p q h t ht]
  omega

theorem residualMin_cast (p q h : Nat) :
    (residualMin p q h : Int) =
      minimumUpTo (residualQuadratic p q h) (min h q) := by
  unfold residualMin
  exact Int.toNat_of_nonneg
    (minimumUpTo_nonnegative _ _ (residualQuadratic_nonnegative p q h))
```

Lemma 12.17 (The exported minimum and minimizers have their intended meaning)**.**

Corresponding Lean declaration(s): `O77.Arithmetic.residualMin_le`,`O77.Arithmetic.residualMin_attained`,`O77.Arithmetic.mem_residualMinimizerList`,`O77.Arithmetic.residualMinimizerList_nodup`.

For all $p, q, h \in \mathbb{N}$, $d_N \le c_N(t)$ whenever $t \le K$, and there exists an admissible t with $c_N(t) = d_N$. Also

$$t \in \mathcal{R} \iff t \le K \text{ and } c_N(t) = d_N, \qquad \mathcal{R} \text{ has no repeated entry.}$$

Thus $d_N = d_0$ and $\mu_N = \mu_0$ as defined independently in the blueprint.

Proof. Apply the lower-bound and attaining-witness assertions to c_Z, and replace each value by its equal natural coercion using Lemma 12.16. Integer coercion preserves and reflects order and equality on natural numbers, giving the first two assertions. The filtered-list membership characterization gives $t \le K$ and $c_Z(t) = M_{c_Z}(K)$; the same coercion identities turn this into the displayed statement. Noduplication follows from the general filtered-list lemma. A least value with an attaining index is the finite minimum in Definition 2.5. The list enumerates exactly the defining minimizer set, once per member; its length is therefore that set's cardinality. No analytic premise enters any of these steps. □

```
theorem residualMin_le (p q h t : Nat) (ht : t ≤ min h q) :
    residualMin p q h ≤ residualCost p q h t := by
  have hle := minimumUpTo_le (residualQuadratic p q h) ht
  rw ←[ residualMin_cast, ← residualCost_cast p q h t ht] at hle
  omega

theorem residualMin_attained (p q h : Nat) :
    ∃ t : Nat, t ≤ min h q ∧ residualCost p q h t = residualMin p q h := by
  rcases minimumUpTo_attained (residualQuadratic p q h) (min h q) with
    ⟨t, ht, ⟩hval
  rw ←[ residualMin_cast, ← residualCost_cast p q h t ht] at hval
  exact ⟨t, ht, by ⟩omega

theorem mem_residualMinimizerList (p q h t : Nat) :
    t ∈ residualMinimizerList p q h ↔
      t ≤ min h q ∧ residualCost p q h t = residualMin p q h := by
```

```
  unfold residualMinimizerList
  rw [mem_minimizerList, isMinimumOn_iff_value]
  constructor
  · rintro ⟨ht, ⟩hval
    rw ←[ residualMin_cast, ← residualCost_cast p q h t ht] at hval
    exact ⟨ht, by ⟩omega
  · rintro ⟨ht, ⟩hval
    refine ⟨ht, ?⟩_
    rw ←[ residualMin_cast, ← residualCost_cast p q h t ht, hval]

theorem residualMinimizerList_nodup (p q h : Nat) :
    (residualMinimizerList p q h).Nodup :=
  minimizerList_nodup _ _
```

Lemma 12.18 (A closed minimizing witness and all residual minimizers)**.**

Corresponding Lean declaration(s): `O77.Arithmetic.residualVertex_bounds`,`O77.Arithmetic.residualMin_eq_cost_vertex`,`O77.Arithmetic.residualMinimizer_indices`.

For $v = v(h+q-p, K)$ one has $v \le K$, $d_0 = c_N(v)$, and

$$t \in \mathcal{R} \iff t = v \text{ or } [v < K,\ h+q = p+2v+1,\ t = v+1].$$

Furthermore either $v = 0$ or $2v \le h+q-p$, and either $v = K$ or $h+q-p \le 2v+1$; these last inequalities use signed integer subtraction.

Proof. Instantiate Lemmas 12.12 and 12.13 with $B = hq$ and $D = h+q-p$ in $\mathbb{Z}$. The minimizing property identifies $c_Z(v)$ with $M_{c_Z}(K)$. Lemma 12.16, applicable because $v \le K$, gives $d_N = c_N(v)$. Finally the integer equality $h+q-p = 2v+1$ is equivalent to the natural equality $h+q = p+2v+1$. This yields all, not merely some, minimizing indices. □

```
theorem residualVertex_bounds (p q h : Nat) :
    residualVertex p q h ≤ min h q ∧
    (residualVertex p q h = 0 ∨
      2 * (residualVertex p q h : Int) ≤ (h : Int) + q - p) ∧
    (residualVertex p q h = min h q ∨
      (h : Int) + q - p ≤ 2 * (residualVertex p q h : Int) + 1) :=
  lowerVertex_bounds _ _

theorem residualMin_eq_cost_vertex (p q h : Nat) :
    residualMin p q h = residualCost p q h (residualVertex p q h) := by
  have hv := (isMinimumOn_iff_value _ (min h q) (residualVertex p q h)).mp
    (lowerVertex_isMinimum ((h : Int) * q) ((h : Int) + q - p) (min h q))
  have ht := (residualVertex_bounds p q h).1
  have he : residualQuadratic p q h (residualVertex p q h) =
      minimumUpTo (residualQuadratic p q h) (min h q) := hv.2
  rw ←[ residualMin_cast, ← residualCost_cast p q h _ ht] at he
  omega

theorem residualMinimizer_indices (p q h t : Nat) :
    t ∈ residualMinimizerList p q h ↔
      t = residualVertex p q h ∨
      (residualVertex p q h < min h q ∧
       h + q = p + 2 * residualVertex p q h + 1 ∧
       t = residualVertex p q h + 1) := by
  unfold residualMinimizerList residualQuadratic
  rw [mem_minimizerList, quadratic_minimum_indices]
  change (t = residualVertex p q h ∨
    (residualVertex p q h < min h q ∧
     (h : Int) + q - p = 2 * (residualVertex p q h : Int) + 1 ∧
     t = residualVertex p q h + 1)) ↔ _
  omega
```

Lemma 12.19 (The parity and triangle criterion, with all boundary cases)**.**

Corresponding Lean declaration(s): `O77.Arithmetic.residualTie_iff`,`O77.Arithmetic.residualMultiplicity_formula`,`O77.Arithmetic.residualMultiplicity_one_or_two`.

For all natural p, q, h, including zeros,

$$\mu_0(p,q,h) = \begin{cases} 2, & p+q+h \text{ is odd and } p \le q+h,\ q \le p+h,\ h \le p+q, \\ 1, & \text{otherwise.} \end{cases}$$

In particular $1 \le \mu_0 \le 2$.

Proof. Write $D = h + q - p$ in $\mathbb{Z}$ and $K = \min(h, q)$. The tie condition from Lemma 12.14 is equivalent to

$$D \text{ odd}, \qquad 1 \le D \le 2K - 1. \tag{$*$}$$

Indeed, a tie has $D = 2v + 1$ with $0 \le v < K$, which proves these inequalities. Conversely $(*)$ gives $v = \lfloor D/2 \rfloor < K$ and $D = 2v + 1$. The parity of D equals that of $p + q + h$, since their difference is $2p$. The inequalities $0 < D < 2\min(h, q)$ are equivalent to $p < h + q$, $q < p + h$, and $h < p + q$. Under odd total parity, equality in any of the three weak triangle inequalities is impossible: for example $p = q + h$ makes $p + q + h = 2p$ even; the other cases are identical after relabeling. Thus strict and weak triangles agree in this parity class. This proves the criterion. The count is 2 in that case and 1 otherwise. If a dimension is zero, weak triangles force the other two to be equal, giving even total parity, so no zero-size exception is omitted. □

```
theorem residualTie_iff (p q h : Nat) :
    (residualVertex p q h < min h q ∧
     (h : Int) + q - p = 2 * (residualVertex p q h : Int) + 1) ↔
    ((p + q + h) % 2 = 1 ∧ p ≤ q + h ∧ q ≤ p + h ∧ h ≤ p + q) := by
  unfold residualVertex lowerVertex
  omega

theorem residualMultiplicity_formula (p q h : Nat) :
    residualMultiplicity p q h =
      if (p + q + h) % 2 = 1 ∧ p ≤ q + h ∧ q ≤ p + h ∧ h ≤ p + q
      then 2 else 1 := by
  unfold residualMultiplicity residualMinimizerList residualQuadratic
  rw [quadratic_minimizer_count]
  change (if residualVertex p q h < min h q ∧
    (h : Int) + q - p = 2 * (residualVertex p q h : Int) + 1 then 2 else 1) = _
  simp only [residualTie_iff]

theorem residualMultiplicity_one_or_two (p q h : Nat) :
    1 ≤ residualMultiplicity p q h ∧ residualMultiplicity p q h ≤ 2 := by
  rw [residualMultiplicity_formula]
  split <;> omega
```

Lemma 12.20 (The three dominated residual values)**.**

Corresponding Lean declaration(s): `O77.Arithmetic.residualMin_output_dominated`,`O77.Arithmetic.residualMin_input_dominated`,`O77.Arithmetic.residualMin_hidden_dominated`.

For arbitrary natural dimensions,

$$\begin{aligned} p > q + h &\implies d_0 = hq, \\ q > p + h &\implies d_0 = hp, \\ h > p + q &\implies d_0 = pq. \end{aligned}$$

Proof. If $p > q+h$, then $D < 0$, hence $v = 0$ and $c_N(v) = hq$. If $q > p+h$, then $q > h$, $K = h$, and $D = h + q - p > 2h$. Thus the vertex is clamped at $v = h$ and $c_N(h) = ph = hp$. If $h > p+q$, then $h > q$, $K = q$, and $D > 2q$; the index is $v = q$ and $c_N(q) = pq$. Each assertion follows from the already proved identity $d_0 = c_N(v)$. No positivity hypothesis on a dimension is needed for these implications. □

```
theorem residualMin_output_dominated (p q h : Nat) (hp : q + h < p) :
    residualMin p q h = h * q := by
  have hv : residualVertex p q h = 0 := by
    unfold residualVertex lowerVertex
    omega
  rw [residualMin_eq_cost_vertex, hv]
  simp [residualCost]

theorem residualMin_input_dominated (p q h : Nat) (hq : p + h < q) :
    residualMin p q h = h * p := by
  have hv : residualVertex p q h = h := by
    unfold residualVertex lowerVertex
    omega
  rw [residualMin_eq_cost_vertex, hv]
  simp [residualCost, Nat.mul_comm]

theorem residualMin_hidden_dominated (p q h : Nat) (hh : p + q < h) :
    residualMin p q h = p * q := by
  have hv : residualVertex p q h = q := by
    unfold residualVertex lowerVertex
    omega
  rw [residualMin_eq_cost_vertex, hv]
  simp [residualCost]
```

Lemma 12.21 (Balanced value with explicit parity correction)**.**

Corresponding Lean declaration(s): `O77.Arithmetic.residualVertex_balanced`,`O77.Arithmetic.residualMin_balanced`.

Suppose $p \le q+h$, $q \le p+h$, $h \le p+q$, and put $\chi = (p+q+h) \bmod 2 \in \{0,1\}$. Then in $\mathbb{Z}$,

$$2v = h + q - p - \chi, \qquad 4d_0 = 2h(p+q) - (p-q)^2 - h^2 + \chi.$$

Since the residual threshold candidate is $\lambda_0 = d_0/2$, the second equality is exactly the balanced numerator for $8\lambda_0$ in Proposition 8.2.

Proof. The triangle inequalities give $0 \le D = h + q - p \le 2\min(h,q) = 2K$. Consequently taking $\lfloor D/2 \rfloor$ does not require a strict clamp, and $D = 2v + \chi$ by Euclidean division and the parity identity $D \equiv p + q + h \pmod 2$. The guarded cost identity gives $d_0 = hq + v^2 - Dv$. Multiplying by four,

$$4d_0 = 4hq - D^2 + (2v - D)^2 = 4hq - D^2 + \chi^2.$$

Here $\chi^2 = \chi$, since χ is either zero or one. Expanding $4hq - (h + q - p)^2$ gives $2h(p+q) - (p-q)^2 - h^2$, proving the claim. All subtractions in this computation occur in $\mathbb{Z}$; the code does not perform a truncated natural division to represent a half-integer threshold. □

```
theorem residualVertex_balanced (p q h : Nat)
    (hp : p ≤ q + h) (hq : q ≤ p + h) (hh : h ≤ p + q) :
    2 * (residualVertex p q h : Int) =
      (h : Int) + q - p - ((p + q + h) % 2 : Nat) := by
  unfold residualVertex lowerVertex
  omega
```

```
/-- Multiplying the threshold formula by eight avoids division and rounding. -/
theorem residualMin_balanced (p q h : Nat)
    (hp : p ≤ q + h) (hq : q ≤ p + h) (hh : h ≤ p + q) :
    4 * (residualMin p q h : Int) =
      2 * (h : Int) * ((p : Int) + q) - ((p : Int) - q) ^ 2 - (h : Int) ^ 2 +
      (((p + q + h) % 2 : Nat) : Int) := by
  have hvc := residualVertex_balanced p q h hp hq hh
  have hcost := residualCost_cast p q h (residualVertex p q h)
    (residualVertex_bounds p q h).1
  rw ←[ residualMin_eq_cost_vertex] at hcost
  unfold residualQuadratic quadratic at hcost
  have hpar : (p + q + h) % 2 = 0 ∨ (p + q + h) % 2 = 1 := by omega
  rcases hpar with hpar | hpar <;> rw [hpar] at hvc ⊢ <;> grind only
```

Lemma 12.22 (Absent residual factors and strict positivity)**.**

Corresponding Lean declaration(s): `O77.Arithmetic.residualMin_zero`,`O77.Arithmetic.residualMultiplicity_zero`,`O77.Arithmetic.residualCost_positive`,`O77.Arithmetic.residualMin_positive`,`O77.Arithmetic.residualMin_eq_zero_iff`.

For all $p, q, h \in \mathbb{N}$,

$$d_0(p, q, h) = 0 \iff (p = 0 \text{ or } q = 0 \text{ or } h = 0).$$

In these zero-size cases $\mu_0 = 1$. If $p, q, h > 0$, then $c_N(t) > 0$ for every $t \in \mathbb{N}$, and in particular $d_0 > 0$.

Proof. When $p = 0$, one has $(h + q)/2 \geq \min(h, q)$, so $v = K$. At least one of $h - K, q - K$ is zero, and $c_N(K) = 0$. When $q = 0$ or $h = 0$, one has $K = v = 0$ and again $c_N(v) = 0$. The parity criterion in Lemma 12.19 cannot hold in any zero-size case, so the multiplicity is one. If all three dimensions are positive, $c_N(0) = hq > 0$. For $t > 0$, the summand pt is positive and the other natural summand is nonnegative, so $c_N(t) > 0$. Evaluate this at an attaining index from Lemma 12.17 to get $d_0 > 0$. Thus $d_0 = 0$ precludes all three dimensions being positive, completing both directions of the equivalence. □

```
theorem residualMin_zero (p q h : Nat) (hz : p = 0 ∨ q = 0 ∨ h = 0) :
    residualMin p q h = 0 := by
  rw [residualMin_eq_cost_vertex]
  rcases hz with hp | hq | hh
  · subst p
    have hv : residualVertex 0 q h = min h q := by
      unfold residualVertex lowerVertex
      omega
    rw [hv]
    unfold residualCost
    by_cases hle : h ≤ q <;> simp [Nat.min_def, hle]
  · subst q
    have hv : residualVertex p 0 h = 0 := by
      have := (residualVertex_bounds p 0 h).1
      omega
    simp [hv, residualCost]
  · subst h
    have hv : residualVertex p q 0 = 0 := by
      have := (residualVertex_bounds p q 0).1
      omega
    simp [hv, residualCost]

theorem residualMultiplicity_zero (p q h : Nat)
    (hz : p = 0 ∨ q = 0 ∨ h = 0) : residualMultiplicity p q h = 1 := by
```

```
  rw [residualMultiplicity_formula]
  have hn : ¬ ((p + q + h) % 2 = 1 ∧ p ≤ q + h ∧ q ≤ p + h ∧ h ≤ p + q) := by
    omega
  simp [hn]

theorem residualCost_positive (p q h t : Nat)
    (hp : 0 < p) (hq : 0 < q) (hh : 0 < h) : 0 < residualCost p q h t := by
  by_cases ht : t = 0
  · subst t
    simpa [residualCost] using Nat.mul_pos hh hq
  · have htpos : 0 < t := by omega
    have hpt := Nat.mul_pos hp htpos
    unfold residualCost
    omega

theorem residualMin_positive (p q h : Nat)
    (hp : 0 < p) (hq : 0 < q) (hh : 0 < h) : 0 < residualMin p q h := by
  rcases residualMin_attained p q h with ⟨t, _, ⟩hval
  rw ←[ hval]
  exact residualCost_positive p q h t hp hq hh

theorem residualMin_eq_zero_iff (p q h : Nat) :
    residualMin p q h = 0 ↔ p = 0 ∨ q = 0 ∨ h = 0 := by
  constructor
  · intro hz
    by_cases hp : p = 0
    · exact Or.inl hp
    · by_cases hq : q = 0
      · exact Or.inr (Or.inl hq)
      · by_cases hh : h = 0
        · exact Or.inr (Or.inr hh)
        · have hpos := residualMin_positive p q h (by omega) (by omega) (by omega)
          omega
  · exact residualMin_zero p q h
```

Lemma 12.23 (Transpose symmetry of the table)**.**

Corresponding Lean declaration(s): `O77.Arithmetic.residualMin_transpose`,`O77.Arithmetic.residualMultiplicity_transpose`.

For all $p, q, h \in \mathbb{N}$, $d_0(p, q, h) = d_0(q, p, h)$ and $\mu_0(p, q, h) = \mu_0(q, p, h)$.

Proof. If $p > q+h$, the output-dominated formula for (p, q, h) and input-dominated formula for (q, p, h) both give hq. If $q > p + h$, the roles are reversed and both values are hp. If $h > p + q$, both values are $pq = qp$. If none of these strict inequalities holds, both triples satisfy the weak triangle inequalities. Their balanced expressions agree: $(p - q)^2 = (q - p)^2$, $p + q = q + p$, and the parities coincide. Cancelling the nonzero integer factor 4 identifies d_0. These cases are exhaustive by integer order. The multiplicity criterion is symmetric term by term, with the first two triangle inequalities exchanged. This proves its equality as well. No bijection of the *entire* admissible index sets is asserted; those sets may have different cardinalities. □

```
theorem residualMin_transpose (p q h : Nat) :
    residualMin p q h = residualMin q p h := by
  by_cases hp : q + h < p
  · rw [residualMin_output_dominated p q h hp,
      residualMin_input_dominated q p h hp]
  · by_cases hq : p + h < q
    · rw [residualMin_input_dominated p q h hq,
        residualMin_output_dominated q p h hq]
    · by_cases hh : p + q < h
      · have hh' : q + p < h := by omega
```

```
        rw [residualMin_hidden_dominated p q h hh,
          residualMin_hidden_dominated q p h hh', Nat.mul_comm]
      · have hp' : p ≤ q + h := by omega
        have hq' : q ≤ p + h := by omega
        have hh' : h ≤ p + q := by omega
        have he := residualMin_balanced p q h hp' hq' hh'
        have he' := residualMin_balanced q p h hq' hp' (by omega)
        have hpar : (p + q + h) % 2 = (q + p + h) % 2 := by omega
        have hs : ((p : Int) - q) ^ 2 = ((q : Int) - p) ^ 2 := by grind
        grind

theorem residualMultiplicity_transpose (p q h : Nat) :
    residualMultiplicity p q h = residualMultiplicity q p h := by
  rw [residualMultiplicity_formula, residualMultiplicity_formula]
  have he : ((p + q + h) % 2 = 1 ∧ p ≤ q + h ∧ q ≤ p + h ∧ h ≤ p + q) ↔
      ((q + p + h) % 2 = 1 ∧ q ≤ p + h ∧ p ≤ q + h ∧ h ≤ q + p) := by omega
  simp only [he]
```

12.5 Connecting numerical outputs to the measured theorem

Definition 12.24 (Numerical candidates, not measured invariants)**.**

Corresponding Lean declaration(s): `077.candidateTwiceLambda`,`077.candidateMultiplicity`.

For natural I, O, H, r, a, b define the total numerical functions

$$N_\Lambda = Oa+b(I-a)+d_0(O-b, I-a, (H+r)-(a+b)), \qquad N_m = \mu_0(O-b, I-a, (H+r)-(a+b)).$$

The intended exponent in O77 is the real number $(N_\Lambda : \mathbb{R})/2$; it is not natural division by two. The expected multiplicity is N_m. Feasibility is a hypothesis in theorems interpreting the subtractions, not an argument choosing the values of these functions.

The later analytic endpoint must prove $\mathsf{BandPair}(L_\Phi, w; (N_\Lambda : \mathbb{R})/2, N_m)$ for actual matrix ranks. The code below contains no definition of a volume predicate and no theorem asserting that identification. Naming a number "candidate" cannot by itself turn it into a learning coefficient.

```
end Arithmetic

/-- Twice the candidate exponent; not a definition of a measured exponent. -/
def candidateTwiceLambda (I O H r a b : Nat) : Nat :=
  regularDim I O a b + residualMin (O - b) (I - a) (residualHidden H r a b)

def candidateMultiplicity (I O H r a b : Nat) : Nat :=
  residualMultiplicity (O - b) (I - a) (residualHidden H r a b)
```

Lemma 12.25 (Positive numerator in the O77 dimension range)**.**

Corresponding Lean declaration(s): `077.candidateTwiceLambda_positive`.

If $I, O, H > 0$ and $D : \mathsf{RankData}(I, O, H, r, a, b)$, then $N_\Lambda > 0$.

Proof. If $a > 0$, the summand Oa is positive, and all other summands are nonnegative natural numbers. If $a = 0$ and $b > 0$, then $I - a = I > 0$, so the summand bI is positive. If $a = b = 0$, feasibility gives $r \le a = 0$, hence $r = 0$. Then the residual dimensions are O, I, H, all positive, and the regular term is zero. Lemma 12.22 gives $d_0(O, I, H) > 0$. The cases exhaust the natural values of a, b. □

```
theorem candidateTwiceLambda_positive {I O H r a b : Nat}
    (hI : 0 < I) (hO : 0 < O) (hH : 0 < H) (D : RankData I O H r a b) :
    0 < candidateTwiceLambda I O H r a b := by
  by_cases ha : a = 0
  · subst a
    by_cases hb : b = 0
    · subst b
      have hr : r = 0 := by have := D.r_le_a; omega
      subst r
      simpa [candidateTwiceLambda, regularDim, residualHidden] using
        Arithmetic.residualMin_positive O I H hO hI hH
    · have hbpos : 0 < b := by omega
      have hp := Nat.mul_pos hbpos hI
      simp only [candidateTwiceLambda, regularDim, Nat.mul_zero, Nat.sub_zero,
        Nat.zero_add]
      omega
  · have hapos : 0 < a := by omega
    have hp := Nat.mul_pos hO hapos
    unfold candidateTwiceLambda regularDim
    omega
```

Lemma 12.26 (Parameter dimension and generator dimension are different identities)**.**

Corresponding Lean declaration(s): `077.shape_parameter_dimension`,`077.shape_generator_dimension`.

For a shape $d = (r, \alpha, \beta, h, p, q)$ *set* $a = r + \alpha$, $b = r + \beta$, $s = O_d a + bq$, *and*

$$g = a^2 + (\beta + h)a + \alpha q + O_d\beta + bh.$$

Then

$$s + ph + hq + g = H_d I_d + O_d H_d, \qquad s + pq = O_d I_d.$$

The first counts the chart's parameters*; the second counts its error* generators*.*

Proof. Use $H_d = a + \beta + h$, $I_d = a + q$, $O_d = b + p$. First $a^2 + (\beta + h)a = H_d a$, and $O_d a + O_d\beta = O_d(H_d - h)$. Adding $ph + bh = O_d h$ leaves $O_d H_d$. The remaining terms $bq + hq + \alpha q = (b + h + \alpha)q = H_d q$. Together with $H_d a$ this gives $H_d I_d + O_d H_d$. For the generator count, $s + pq = O_d a + (b + p)q = O_d(a + q) = O_d I_d$. Only distributivity and the displayed additive shape identities are used, so zero-size blocks remain valid. Neither identity claims that Y, Z can be recovered from their product YZ. □

```
/-- Arithmetic certificates are uniform in all six nonnegative block sizes. -/
theorem shape_parameter_dimension (d : BlockShape) :
    let a := d.r + d.alpha
    let b := d.r + d.beta
    let s := d.output * a + b * d.q
    let g := a*a + (d.beta+d.h)*a + d.alpha*d.q + d.output*d.beta + b*d.h
    s + d.p*d.h + d.h*d.q + g = d.hidden*d.input + d.output*d.hidden := by
  dsimp [BlockShape.input, BlockShape.output, BlockShape.hidden]
  grind

/-- The generator space has output times input coordinates, not parameter dimension. -/
theorem shape_generator_dimension (d : BlockShape) :
    d.output * (d.r + d.alpha) + (d.r + d.beta) * d.q + d.p * d.q =
      d.output * d.input := by
  dsimp [BlockShape.input, BlockShape.output]
  grind
```

12.6 Kernel-reduced calibration cases and a repaired edge statement

The examples below are exact finite evaluations, each accepted by ordinary `decide`. They complement the universal theorems above; they do not replace them. They include $(p, q, h) = (1, 1, 1)$, where $(d_0, \mu_0) = (1, 2)$; $(2, 2, 2)$, where the pair is $(3, 1)$; transpose symmetry when the index sets differ; and a zero-size residual. They also distinguish the three feasible rank pairs of $I = O = H = 2$, $r = 1$: all have numerator $N_\Lambda = 4$, whereas their multiplicities are $2, 1, 1$ at $(a, b) = (1, 1), (1, 2), (2, 1)$.

Correction to the earlier edge wording. An absent residual factor gives $d_0 = 0$ and $\mu_0 = 1$; it does *not* force zero excess over the global threshold. At $I = O = H = 2$, $r = 0$, the feasible pair $(a, b) = (2, 0)$ has $N_\Lambda = 4$, whereas $(0, 0)$ has $N_\Lambda = 3$. The sentence in Theorem 8.4 about zero residuals has been corrected accordingly. The finite formulas themselves are unchanged.

```
-- These are exact finite computations by kernel reduction, not floating-point tests.
example : residualMin 1 1 1 = 1 ∧ residualMultiplicity 1 1 1 = 2 := by decide
example : residualMin 2 2 2 = 3 ∧ residualMultiplicity 2 2 2 = 1 := by decide
example : residualMin 2 3 4 = residualMin 3 2 4 ∧
    residualMultiplicity 2 3 4 = 2 := by decide
example : residualMin 0 3 4 = 0 ∧ residualMultiplicity 0 3 4 = 1 := by decide
example : residualHidden 3 2 3 2 = 0 := by decide
example : candidateTwiceLambda 2 2 2 1 1 1 = 4 ∧
    candidateMultiplicity 2 2 2 1 1 1 = 2 := by decide
example : candidateTwiceLambda 2 2 2 1 1 2 = 4 ∧
    candidateMultiplicity 2 2 2 1 1 2 = 1 := by decide
example : candidateTwiceLambda 2 2 2 1 2 1 = 4 ∧
    candidateMultiplicity 2 2 2 1 2 1 = 1 := by decide

example : candidateTwiceLambda 2 2 2 0 2 0 = 4 ∧
    candidateTwiceLambda 2 2 2 0 0 0 = 3 := by decide

end O77
```

13 The complete numerical minimum locus

This section proves the complete finite minimum-locus assertion. It does not change the loss, the local band predicate, or the definition of a saddle rung. In particular, a numerical classification is not renamed as a theorem about Lebesgue measure.

Fix natural numbers I, O, H, r. When a feasible rank pair (a, b) is under consideration, write $D : \mathsf{RankData}(I, O, H, r, a, b)$ in the sense of Definition 12.1, and set

$$P = O - r, \quad Q = I - r, \quad J = H - r, \quad \alpha = a - r, \quad \beta = b - r,$$
$$p = O - b, \quad q = I - a, \quad h = (H + r) - (a + b).$$

The inequalities in D justify these natural differences and imply $P = p + \beta$, $Q = q + \alpha$, $J = h + \alpha + \beta$. For this section only, $N(a, b)$ denotes the value of `candidateTwiceLambdaIOHrab`, and $m(a, b)$ the value of `candidateMultiplicityIOHrab`. Thus $N(a, b)$ is twice a *candidate* exponent, not a measured exponent. Put $s = Oa + b(I - a)$ and $s_0 = Or + r(I - r)$. The fixed integer polynomial

$$\widetilde{C}(u) = JQ + u^2 - (J + Q - P)u$$

is evaluated on $0 \le u \le K_0 := \min(J, Q)$. On that interval it equals the nonnegative cost $(J - u)(Q - u) + Pu$.

After a constant shift, every rank pair minimizes the *same* polynomial on a smaller integer interval. Equality of minima is therefore an intersection statement about an actual minimizer set. The closed dimension cases are derived only afterwards. The conclusions agree with the minimizing-strata classification in [7, Corollary 11.2]; the interval restriction below supplies a uniform finite proof.

All `LeanCode` blocks in this section continue the already checked stream in Section 12. The following namespace reopening is part of the extraction; there is no second import and no dependence on an unprinted helper file.

```
namespace O77
namespace Arithmetic
```

13.1 A reusable restriction theorem for finite minima

Lemma 13.1 (Restriction and translation cannot decrease a minimum)**.**

Corresponding Lean declaration(s): `O77.Arithmetic.minimumUpTo_restrict_le`.

Let $f, g : \mathbb{N} \to \mathbb{Z}$, let $K, k, l \in \mathbb{N}$ satisfy $l+k \le K$, and let $C \in \mathbb{Z}$. Suppose $f(l+t) = C+g(t)$ for every $t \le k$. In the notation of Definition 12.4, $M_f(K) \le C + M_g(k)$.

Proof. Choose $t \le k$ with $g(t) = M_g(k)$ by Lemma 12.5. Then $l + t \le l + k \le K$. The lower-bound property of the minimum gives $M_f(K) \le f(l + t) = C + g(t) = C + M_g(k)$. The constant C need not be nonnegative; adding it preserves the order on $\mathbb{Z}$. □

```
/-- Restrict a finite minimization by an affine change of index and value. -/
theorem minimumUpTo_restrict_le (f g : Nat → Int) (K k l : Nat) (C : Int)
    (hdom : l + k ≤ K)
    (hshift : ∀ t : Nat, t ≤ k → f (l + t) = C + g t) :
    minimumUpTo f K ≤ C + minimumUpTo g k := by
  obtain ⟨t, ht, ⟩hval := minimumUpTo_attained g k
  have h := minimumUpTo_le f (show l + t ≤ K by omega)
  rw [hshift t ht, hval] at h
  exact h
```

Lemma 13.2 (Exact equality criterion for restriction)**.**

Corresponding Lean declaration(s): `O77.Arithmetic.minimumUpTo_restrict_eq_iff`.

Under all the hypotheses of Lemma 13.1,

$$C + M_g(k) = M_f(K) \quad \Longleftrightarrow \quad \exists u \in \mathbb{N} : \ \mathsf{IsMin}_K(f, u), \quad l \le u \le l + k.$$

In particular, equality requires an actual global minimizer in the restricted interval.

Proof. Suppose the two values agree. Choose an attaining $t \le k$ for g. Then $u = l + t$ is in the specified interval and $f(u) = C + M_g(k) = M_f(K)$; Lemma 12.6 makes u a global minimizer. Conversely let u satisfy the right side. Since $u \ge l$, set $t = u - l$; then $t \le k$ and $l + t = u$. Hence $C + M_g(k) \le C + g(t) = f(u) = M_f(K)$. Combine this upper bound with Lemma 13.1 and use antisymmetry. No equality is obtained from a comparison on an empty interval: both intervals here contain at least their left endpoints. □

```
theorem minimumUpTo_restrict_eq_iff (f g : Nat → Int)
    (K k l : Nat) (C : Int) (hdom : l + k ≤ K)
    (hshift : ∀ t : Nat, t ≤ k → f (l + t) = C + g t) :
    C + minimumUpTo g k = minimumUpTo f K ↔
      ∃ u : Nat, IsMinimumOn f K u ∧ l ≤ u ∧ u ≤ l + k := by
  constructor
  · intro heq
```

```
    obtain ⟨t, ht, ⟩hval := minimumUpTo_attained g k
    refine ⟨l + t, ?_, by omega, by ⟩omega
    rw [isMinimumOn_iff_value]
    exact ⟨by omega, by rw [hshift t ht, hval, heq⟩]
  · rintro ⟨u, hu, hlu, ⟩huk
    have ht : u - l ≤ k := by omega
    have hindex : l + (u - l) = u := by omega
    have hval := (isMinimumOn_iff_value f K u).mp hu
    have hs := hshift (u - l) ht
    rw [hindex] at hs
    have hupper := minimumUpTo_le g ht
    have hlower := minimumUpTo_restrict_le f g K k l C hdom hshift
    omega
```

Lemma 13.3 (Minimizer transport when the minima agree)**.**

Corresponding Lean declaration(s): `077.Arithmetic.isMinimumOn_restrict_iff`.

Retain f, g, K, k, l, C and their hypotheses from Lemma 13.1, and assume $C + M_g(k) = M_f(K)$. For every $t \in \mathbb{N}$,

$$\mathsf{IsMin}_k(g, t) \iff t \le k \text{ and } \mathsf{IsMin}_K(f, l + t).$$

Proof. By Lemma 12.6, the left side says $t \le k$ and $g(t) = M_g(k)$. For such t the domain bound gives $l + t \le K$, and the assumed identity gives $f(l + t) = M_f(K)$ if and only if $C + g(t) = C + M_g(k)$. Cancellation in $\mathbb{Z}$ turns this last equality into $g(t) = M_g(k)$. This proves both implications. The shift $t \mapsto l + t$ is injective and has inverse $u \mapsto u - l$ on its image; thus this is also the explicit bijection of the corresponding minimizer sets. □

```
theorem isMinimumOn_restrict_iff (f g : Nat → Int)
    (K k l : Nat) (C : Int) (hdom : l + k ≤ K)
    (hshift : ∀ t : Nat, t ≤ k → f (l + t) = C + g t)
    (heq : C + minimumUpTo g k = minimumUpTo f K) (t : Nat) :
    IsMinimumOn g k t ↔ t ≤ k ∧ IsMinimumOn f K (l + t) := by
  rw [isMinimumOn_iff_value, isMinimumOn_iff_value]
  constructor
  · rintro ⟨ht, ⟩hv
    exact ⟨ht, by omega, by rw [hshift t ht, hv, heq]⟩
  · rintro ⟨ht, _, ⟩hv
    rw [hshift t ht] at hv
    exact ⟨ht, by ⟩omega

end Arithmetic
```

13.2 The rank window and the single quadratic

Definition 13.4 (A baseline value and an independent numerical minimum predicate)**.**

Corresponding Lean declaration(s): `077.baselineTwiceLambda`,`077.IsNumericalThresholdMinimizer`.

Define $N_0 = N(r, r)$. This definition evaluates the pre-existing numerical formula at one rank pair; it does not assert that its value is minimal. A pair (a, b) is numerically threshold-minimizing *if it is feasible and*

$$\forall a', b' \in \mathbb{N}, \quad \mathsf{RankData}(I, O, H, r, a', b') \implies N(a, b) \le N(a', b').$$

This universal comparison does not mention the closed conditions later proposed for the minimum locus. It also remains distinct from the measured predicate `IsThresholdMinimizer` of Definition 2.4.

```
/-- Baseline value at numerical ranks (r,r), not a definition of the minimum. -/
def baselineTwiceLambda (I O H r : Nat) : Nat :=
  candidateTwiceLambda I O H r r r

/-- Minimality among numerical rank pairs; not a predicate about matrix volumes. -/
def IsNumericalThresholdMinimizer (I O H r a b : Nat) : Prop :=
  RankData I O H r a b ∧
    ∀ a' b' : Nat, RankData I O H r a' b' →
      candidateTwiceLambda I O H r a b ≤
        candidateTwiceLambda I O H r a' b'
```

Lemma 13.5 (The baseline ranks are genuinely feasible)**.**

Corresponding Lean declaration(s): `O77.RankData.base_bounds`,`O77.RankData.baseline`.

Every $D : \mathsf{RankData}(I, O, H, r, a, b)$ *implies* $r \le I, O, H$ *and* $\mathsf{RankData}(I, O, H, r, r, r)$.

Proof. The inequalities $r \le a \le I, H$ give the input and hidden bounds; $r \le b \le O$ gives the output bound. For the ranks (r, r), the two lower bounds are identities, the four upper bounds follow from these three dimension inequalities, and the final sum bound $2r \le H + r$ follows from $r \le H$. These are exactly the seven feasibility conditions. □

```
theorem RankData.base_bounds {I O H r a b : Nat}
    (D : RankData I O H r a b) : r ≤ I ∧ r ≤ O ∧ r ≤ H := by
  rcases D with (hra, hrb, haI, haH, hbO, hbH, )hab
  omega

theorem RankData.baseline {I O H r a b : Nat}
    (D : RankData I O H r a b) : RankData I O H r r r := by
  have h := D.base_bounds
  constructor <;> omega
```

Lemma 13.6 (Every residual interval is a nonempty baseline window)**.**

Corresponding Lean declaration(s): `O77.RankData.shift_domain`.

For feasible ranks, $\alpha + \min(h, q) \le K_0$*. For every* $u \in \mathbb{N}$*,*

$$\alpha \le u \le \alpha + \min(h, q) \quad \Longleftrightarrow \quad \alpha \le u, \quad u \le K_0, \quad u + \beta \le J.$$

Denote this interval by $W_{a,b}$*. It is nonempty, contains* α*, and is a subset of* $\{0, \dots, K_0\}$*.*

Proof. Use $Q = q + \alpha$ and $J = h + \alpha + \beta$. The upper endpoint $\alpha + \min(h, q)$ is both at most Q and at most J, proving the first assertion. If $\alpha \le u \le \alpha + \min(h, q)$, then $u \le Q$ and $u + \beta \le \alpha + h + \beta = J$, hence $u \le J$ as well. Conversely $u \le K_0$ implies $u \le Q = \alpha + q$, while $u + \beta \le J$ implies $u \le \alpha + h$. Subtracting α from these two inequalities, under $u \ge \alpha$, gives $u - \alpha \le \min(h, q)$, equivalently $u \le \alpha + \min(h, q)$. All substitutions are justified by the feasibility inequalities; no unguarded natural subtraction is expanded as an integer difference. □

```
theorem RankData.shift_domain {I O H r a b : Nat}
    (D : RankData I O H r a b) :
    (a - r) + min (residualHidden H r a b) (I - a) ≤ min (H - r) (I - r) ∧
    ∀ u : Nat, ((a - r) ≤ u ∧
      u ≤ (a - r) + min (residualHidden H r a b) (I - a)) ↔
      ((a - r) ≤ u ∧ u ≤ min (H - r) (I - r) ∧ u + (b - r) ≤ H - r) := by
  rcases D with (hra, hrb, haI, haH, hbO, hbH, )hab
  unfold residualHidden
  constructor
  · omega
  · intro u; omega
```

Lemma 13.7 (Exact translation of the cost polynomial)**.**

Corresponding Lean declaration(s): `077.rank_cost_shift`.

For every feasible pair and every natural t, as an identity in $\mathbb{Z}$,

$$s_0 + \widetilde{C}(\alpha + t) = s + \widetilde{c}_{p,q,h}(t), \qquad \widetilde{c}_{p,q,h}(t) = hq + t^2 - (h + q - p)t.$$

On $0 \le t \le \min(h, q)$ both polynomials agree with their guarded natural-valued costs.

Proof. First $s - s_0 = (O - r)\alpha + \beta q = (p + \beta)\alpha + \beta q$. For example, this follows directly from $s = O(r + \alpha) + (r + \beta)q$ and $s_0 = Or + r(\alpha + q)$, so the subtracted terms cancel as integers. Substitute $P = p + \beta$, $Q = q + \alpha$, and $J = h + \alpha + \beta$ into the factored polynomial:

$$\begin{aligned}\widetilde{C}(\alpha + t) &= (h + \beta - t)(q - t) + (p + \beta)(\alpha + t) \\ &= (h - t)(q - t) + pt + \beta(q - t) + \beta t + (p + \beta)\alpha \\ &= \widetilde{c}_{p,q,h}(t) + \beta q + (p + \beta)\alpha.\end{aligned}$$

The extra term is exactly $s - s_0$, giving the identity. This polynomial statement is valid for all natural t because its evaluation is in $\mathbb{Z}$; it does not assert an identity of truncated-subtraction costs outside their domains. The admitted-domain assertion is Lemma 12.16, applied to the two intervals using Lemma 13.6. □

```
/-- The polynomial identity is signed; dimension cancellations are guarded. -/
theorem rank_cost_shift {I O H r a b : Nat} (D : RankData I O H r a b)
    (t : Nat) :
    (regularDim I O r r : Int) +
      residualQuadratic (O-r) (I-r) (H-r) ((a-r)+t) =
    (regularDim I O a b : Int) +
      residualQuadratic (O-b) (I-a) (residualHidden H r a b) t := by
  rcases D with (hra, hrb, haI, haH, hbO, hbH, )hab
  have hrI : r ≤ I := by omega
  have hrO : r ≤ O := by omega
  have hrH : r ≤ H := by omega
  simp only [regularDim, residualQuadratic, Arithmetic.quadratic, residualHidden,
    Int.natCast_add, Int.natCast_mul, Int.natCast_sub hra,
    Int.natCast_sub haI, Int.natCast_sub hbO, Int.natCast_sub hrI,
    Int.natCast_sub hrO, Int.natCast_sub hrH, Int.natCast_sub hab]
  grind only
```

Lemma 13.8 (Coercing the numerical candidates without losing a sign)**.**

Corresponding Lean declaration(s): `077.baselineTwiceLambda_cast,077.candidateTwiceLambda_cast`.

If $r \le H$, then, in $\mathbb{Z}$,

$$N_0 = s_0 + M_{\widetilde{C}}(K_0).$$

For all natural I, O, H, r, a, b the definition of $N(a, b)$ similarly gives $N(a, b) = s + M_{\widetilde{c}_{p,q,h}}(\min(h, q))$ after coercion to $\mathbb{Z}$.

Proof. When $r \le H$, $(H + r) - 2r = H - r$ in $\mathbb{N}$. Substitute this into the baseline definition. Lemma 12.16 states that the nonnegative integer residual minimum is exactly the integer cast of `residualMin`; its preceding nonnegativity proof rules out loss through `Int.toNat`. Distributing the cast over the sum with s_0 proves the first identity. The same cast lemma and distribution over addition prove the general identity. Its residual dimensions are already natural, so that assertion does not need feasibility. □

```
theorem baselineTwiceLambda_cast {I O H r : Nat} (hr : r ≤ H) :
    (baselineTwiceLambda I O H r : Int) = (regularDim I O r r : Int) +
      Arithmetic.minimumUpTo (residualQuadratic (O-r) (I-r) (H-r))
        (min (H-r) (I-r)) := by
  have hh : residualHidden H r r r = H - r := by unfold residualHidden; omega
  simp only [baselineTwiceLambda, candidateTwiceLambda, hh, Int.natCast_add]
  rw [Arithmetic.residualMin_cast]

theorem candidateTwiceLambda_cast (I O H r a b : Nat) :
    (candidateTwiceLambda I O H r a b : Int) = (regularDim I O a b : Int) +
      Arithmetic.minimumUpTo
        (residualQuadratic (O-b) (I-a) (residualHidden H r a b))
        (min (residualHidden H r a b) (I-a)) := by
  simp only [candidateTwiceLambda, Int.natCast_add]
  rw [Arithmetic.residualMin_cast]
```

13.3 Lower bound, attainment, and complete equality cases

Theorem 13.9 (Global lower bound without a dimension case split)**.**

Corresponding Lean declaration(s): `077.baselineTwiceLambda_le`.

Every feasible rank pair satisfies $N_0 \le N(a,b)$.

Proof. In Lemma 13.1 take $f = \widetilde{C}$, $g = \widetilde{c}_{p,q,h}$, $K = K_0$, $k = \min(h,q)$, $l = \alpha$, and $C = s - s_0 \in \mathbb{Z}$. The interval inclusion is Lemma 13.6; the affine identity is Lemma 13.7. It follows that $M_{\widetilde{C}}(K_0) \le (s - s_0) + M_g(k)$. Add s_0 and use Lemma 13.8. This is the inequality between the integer casts of $N_0, N(a,b)$, and the natural-to-integer embedding reflects order. No residual analytic table or case formula is used in this proof. □

```
theorem baselineTwiceLambda_le {I O H r a b : Nat}
    (D : RankData I O H r a b) :
    baselineTwiceLambda I O H r ≤ candidateTwiceLambda I O H r a b := by
  have hs (t : Nat) (_ : t ≤ min (residualHidden H r a b) (I-a)) :
      residualQuadratic (O-r) (I-r) (H-r) ((a-r)+t) =
        ((regularDim I O a b : Int) - (regularDim I O r r : Int)) +
        residualQuadratic (O-b) (I-a) (residualHidden H r a b) t := by
    have := rank_cost_shift D t
    omega
  have hle := Arithmetic.minimumUpTo_restrict_le _ _ _ _ _ _
    D.shift_domain.1 hs
  have h0 := baselineTwiceLambda_cast (I := I) (O := O) D.base_bounds.2.2
  have h1 := candidateTwiceLambda_cast I O H r a b
  omega
```

Theorem 13.10 (The whole equality locus as an interval intersection)**.**

Corresponding Lean declaration(s): `077.candidate_eq_baseline_iff_interval`.

For every feasible pair,

$$N(a,b) = N_0 \quad \Longleftrightarrow \quad \exists u \in \mathcal{M}_{\widetilde{C}}(K_0) : \quad \alpha \le u, \quad u + \beta \le J.$$

Here $\mathcal{M}_{\widetilde{C}}(K_0)$ *is the independently defined minimizer list, interpreted by membership. The right side states that it meets* $W_{a,b}$.

Proof. Apply Lemma 13.2 with the data of the preceding proof. The equality of integer minimum expressions is equivalent to $N(a,b) = N_0$ by Lemma 13.8 and cancellation of s_0. A minimizing u automatically satisfies $u \le K_0$. For such u, the interval condition $l \le u \le l + k$ is equivalent to $\alpha \le u$ and $u + \beta \le J$ by Lemma 13.6. Finally, minimizer-list membership and

the semantic minimum predicate agree by Lemma 12.6. Every conversion is reversible, proving the equivalence rather than just sufficiency. □

```
theorem candidate_eq_baseline_iff_interval {I O H r a b : Nat}
    (D : RankData I O H r a b) :
    candidateTwiceLambda I O H r a b = baselineTwiceLambda I O H r ↔
      ∃ u : Nat, u ∈ residualMinimizerList (O-r) (I-r) (H-r) ∧
        a-r ≤ u ∧ u + (b-r) ≤ H-r := by
  have hs (t : Nat) (_ : t ≤ min (residualHidden H r a b) (I-a)) :
      residualQuadratic (O-r) (I-r) (H-r) ((a-r)+t) =
        ((regularDim I O a b : Int) - (regularDim I O r r : Int)) +
        residualQuadratic (O-b) (I-a) (residualHidden H r a b) t := by
    have := rank_cost_shift D t
    omega
  have he := Arithmetic.minimumUpTo_restrict_eq_iff _ _ _ _ _ _
    D.shift_domain.1 hs
  have h0 := baselineTwiceLambda_cast (I := I) (O := O) D.base_bounds.2.2
  have h1 := candidateTwiceLambda_cast I O H r a b
  constructor
  · intro hEq
    have hx : (regularDim I O a b : Int) - (regularDim I O r r : Int) +
        Arithmetic.minimumUpTo
          (residualQuadratic (O-b) (I-a) (residualHidden H r a b))
          (min (residualHidden H r a b) (I-a)) =
        Arithmetic.minimumUpTo (residualQuadratic (O-r) (I-r) (H-r))
          (min (H-r) (I-r)) := by omega
    obtain ⟨u, hu, hl, ⟩hh := he.mp hx
    refine ⟨u, ?_, hl, ?⟩_
    · exact (Arithmetic.mem_minimizerList _ _ _).mpr hu
    · exact ((D.shift_domain.2 u).mp ⟨hl, ⟩hh).2.2
  · rintro ⟨u, hu, hl, ⟩hh
    have hu' := (Arithmetic.mem_minimizerList _ _ _).mp hu
    have hinterval := (D.shift_domain.2 u).mpr ⟨hl, hu'.1, ⟩hh
    have hx := he.mpr ⟨u, hu', ⟩hinterval
    omega
```

Theorem 13.11 (Independent finite minimality is equality with the baseline)**.**

Corresponding Lean declaration(s): `077.numerical_minimizer_iff_baseline`.

For feasible (a, b)*, numerical threshold-minimality in Definition 13.4 is equivalent to* $N(a, b) = N_0$*.*

Proof. If (a, b) minimizes, compare it with the feasible baseline pair (r, r) from Lemma 13.5. This gives $N(a, b) \le N_0$, while Theorem 13.9 gives the opposite inequality. Conversely suppose $N(a, b) = N_0$. For every feasible (a', b'), Theorem 13.9 gives $N_0 \le N(a', b')$. Replace N_0 by $N(a, b)$ to obtain exactly the universal comparison in the definition. Thus existence and attainment of the numerical minimum have been proved, not inserted into a data structure. □

```
theorem numerical_minimizer_iff_baseline {I O H r a b : Nat}
    (D : RankData I O H r a b) :
    IsNumericalThresholdMinimizer I O H r a b ↔
      candidateTwiceLambda I O H r a b = baselineTwiceLambda I O H r := by
  constructor
  · rintro ⟨_, ⟩hmin
    have hupper := hmin r r D.baseline
    have hlower := baselineTwiceLambda_le D
    change candidateTwiceLambda I O H r a b ≤ baselineTwiceLambda I O H r at hupper
    omega
  · intro heq
    refine ⟨D, ?⟩_
```

```
    intro a' b' D'
    rw [heq]
    exact baselineTwiceLambda_le D'
```

13.4 Evaluating the interval condition in all dimension regimes

Lemma 13.12 (Closed condition using at most two vertex indices)**.**

Corresponding Lean declaration(s): `077.candidate_eq_baseline_iff_vertex`.

Let v be the value of `residualVertexPQJ`, and let T denote $v < K_0$ and $J+Q = P+2v+1$. For feasible ranks,

$$N(a,b) = N_0 \iff (\alpha \le v \text{ and } v+\beta \le J) \text{ or } (T \text{ and } \alpha \le v+1 \text{ and } v+1+\beta \le J).$$

Proof. Lemma 12.18 says precisely that the baseline minimizing indices are v, and also $v+1$ when T holds. Substitute this disjunction into Theorem 13.10. A witness for the first alternative is $u = v$; a witness for the second is $u = v+1$. Conversely every witness belongs to one of these two alternatives by that minimizer classification. This proves both directions, including a clamped vertex at either endpoint. □

```
theorem candidate_eq_baseline_iff_vertex {I O H r a b : Nat}
    (D : RankData I O H r a b) :
    let v := residualVertex (O-r) (I-r) (H-r)
    candidateTwiceLambda I O H r a b = baselineTwiceLambda I O H r ↔
      (a-r ≤ v ∧ v+(b-r) ≤ H-r) ∨
      (v < min (H-r) (I-r) ∧ (H-r)+(I-r) = (O-r)+2*v+1 ∧
        a-r ≤ v+1 ∧ v+1+(b-r) ≤ H-r) := by
  dsimp only
  rw [candidate_eq_baseline_iff_interval D]
  constructor
  · rintro ⟨u, hu, hl, ⟩hh
    have h := (Arithmetic.residualMinimizer_indices (O-r) (I-r) (H-r) u).mp hu
    omega
  · intro h
    rcases h with h | h
    · refine ⟨residualVertex (O-r) (I-r) (H-r), ?_, ⟩h
      exact (Arithmetic.residualMinimizer_indices _ _ _ _).mpr (Or.inl rfl)
    · refine ⟨residualVertex (O-r) (I-r) (H-r) + 1, ?_, h).2.2
      exact (Arithmetic.residualMinimizer_indices _ _ _ _).mpr
        (Or.inr ⟨h.1, h.2.1, ⟩rfl)
```

Lemma 13.13 (Output-dominated equality locus)**.**

Corresponding Lean declaration(s): `077.candidate_eq_baseline_output`.

For feasible ranks, if $I+H < O+r$, then $N(a,b) = N_0$ if and only if $a = r$.

Proof. The dimension inequality is $Q+J < P$. Hence the signed vertex parameter $J+Q-P$ is negative, its truncated natural value is zero, and $v = 0$. The tie equation $J+Q = P+2v+1$ is impossible. By Lemma 13.12, equality requires $\alpha \le 0$ and $\beta \le J$. The second inequality follows from $b \le H$; the first is equivalent to $a = r$ since $r \le a$. This reasoning remains valid when Q or J is zero. □

```
theorem candidate_eq_baseline_output {I O H r a b : Nat}
    (D : RankData I O H r a b) (hO : I+H < O+r) :
    candidateTwiceLambda I O H r a b = baselineTwiceLambda I O H r ↔ a = r := by
  have hc := candidate_eq_baseline_iff_vertex D
  dsimp only at hc
```

```
  rw [hc]
  have hv : residualVertex (O-r) (I-r) (H-r) = 0 := by
    have h := D.base_bounds
    unfold residualVertex Arithmetic.lowerVertex
    omega
  rw [hv]
  rcases D with ⟨hra, hrb, haI, haH, hbO, hbH, ⟩⟨hab⟩
  omega
```

Lemma 13.14 (Input-dominated equality locus)**.**

Corresponding Lean declaration(s): `077.candidate_eq_baseline_input`.

For feasible ranks, if $O + H < I + r$, then $N(a, b) = N_0$ if and only if $b = r$.

Proof. Now $P + J < Q$, so $K_0 = J$ and $J + Q - P > 2J$. Its halved lower integer vertex is at least J; clamping gives $v = J$. The condition $v < K_0$ is false, so there is no second vertex. The first alternative of Lemma 13.12 is $\alpha \le J$ and $J + \beta \le J$. The first follows from $a \le H$; the second is $\beta = 0$, equivalently $b = r$. □

```
theorem candidate_eq_baseline_input {I O H r a b : Nat}
    (D : RankData I O H r a b) (hI : O+H < I+r) :
    candidateTwiceLambda I O H r a b = baselineTwiceLambda I O H r ↔ b = r := by
  have hc := candidate_eq_baseline_iff_vertex D
  dsimp only at hc
  rw [hc]
  have hv : residualVertex (O-r) (I-r) (H-r) = H-r := by
    have h := D.base_bounds
    unfold residualVertex Arithmetic.lowerVertex
    omega
  rw [hv]
  rcases D with ⟨hra, hrb, haI, haH, hbO, hbH, ⟩⟨hab⟩
  omega
```

Lemma 13.15 (Hidden-dominated equality locus)**.**

Corresponding Lean declaration(s): `077.candidate_eq_baseline_hidden`.

For feasible ranks, if $I + O < H + r$, then $N(a, b) = N_0$ for every feasible (a, b).

Proof. In this regime $P + Q < J$, so $K_0 = Q$ and $J + Q - P > 2Q$. Thus $v = Q$ and there is no tie. We have $\alpha \le Q$ by $a \le I$, and $\beta \le P$ by $b \le O$. Consequently $Q + \beta \le Q + P < J$. The index $v = Q$ lies in every rank window, and Lemma 13.12 proves equality. □

```
theorem candidate_eq_baseline_hidden {I O H r a b : Nat}
    (D : RankData I O H r a b) (hH : I+O < H+r) :
    candidateTwiceLambda I O H r a b = baselineTwiceLambda I O H r := by
  have hc := candidate_eq_baseline_iff_vertex D
  dsimp only at hc
  rw [hc]
  have hv : residualVertex (O-r) (I-r) (H-r) = I-r := by
    have h := D.base_bounds
    unfold residualVertex Arithmetic.lowerVertex
    omega
  rw [hv]
  rcases D with ⟨hra, hrb, haI, haH, hbO, hbH, ⟩⟨hab⟩
  omega
```

Lemma 13.16 (Balanced equality locus, including the odd-parity boundary)**.**

Corresponding Lean declaration(s): `O77.candidate_eq_baseline_balanced`.

Suppose (a,b) *is feasible and* $O+r \le I+H$, $I+r \le O+H$, $H+r \le I+O$. *Put* $\chi = (I+O+H+r) \bmod 2$. *Then*

$$N(a,b) = N_0 \quad\Longleftrightarrow\quad 2a+O \le I+H+r+\chi, \qquad 2b+I \le O+H+r+\chi.$$

No natural subtraction occurs in the two exported inequalities.

Proof. The baseline triple (P,Q,J) satisfies all three weak triangle inequalities. Its total parity is χ, because $P+Q+J = I+O+H-3r$ differs from $I+O+H+r$ by $4r$. By Lemmas 12.21 and 12.19,

$$2v = J+Q-P-\chi, \qquad \mathcal{M}_{\widetilde{C}}(K_0) = \begin{cases} \{v\}, & \chi = 0, \\ \{v, v+1\}, & \chi = 1. \end{cases}$$

The set notation denotes the entries of the noduplicate list; in the second case both indices are in $\{0,\dots,K_0\}$.

We claim that this set meets $W_{a,b}$ exactly when $\alpha \le v+\chi$ and $v+\beta \le J$. Necessity follows from any witnessing index, which is at least v and at most $v+\chi$. For sufficiency, if $\alpha \le v$, choose $u = v$. Otherwise $\alpha > v$, so $\alpha \le v+\chi$ forces $\chi = 1$ and $\alpha = v+1$. Choose $u = v+1 = \alpha$. It is a baseline minimizer, and $u+\beta = \alpha+\beta \le J$ follows from feasibility. This proves the claim without confusing separate endpoint bounds with an arbitrary, potentially disconnected set of minimizers.

Multiplying the two integer bounds by 2 and substituting $2v = J+Q-P-\chi$ gives $2\alpha \le J+Q-P+\chi$ and $2\beta \le J-Q+P+\chi$. Substitute $\alpha = a-r$, $\beta = b-r$, $P = O-r$, $Q = I-r$, $J = H-r$; the resulting inequalities are exactly the two displayed statements. All substitutions are reversible under feasibility. In particular, the χ term is necessary: a window containing only one member of a tied minimum still attains the minimum. □

```
theorem candidate_eq_baseline_balanced {I O H r a b : Nat}
    (D : RankData I O H r a b)
    (hO : O+r ≤ I+H) (hI : I+r ≤ O+H) (hH : H+r ≤ I+O) :
    candidateTwiceLambda I O H r a b = baselineTwiceLambda I O H r ↔
      2*a+O ≤ I+H+r+(I+O+H+r)%2 ∧
      2*b+I ≤ O+H+r+(I+O+H+r)%2 := by
  have hc := candidate_eq_baseline_iff_vertex D
  dsimp only at hc
  rw [hc]
  have hb := D.base_bounds
  have hv := Arithmetic.residualVertex_balanced (O-r) (I-r) (H-r)
    (by omega) (by omega) (by omega)
  have hvdom := (Arithmetic.residualVertex_bounds (O-r) (I-r) (H-r)).1
  rcases D with (hra, hrb, haI, haH, hbO, hbH, )hab
  omega
```

Definition 13.17 (Closed numerical rank condition)**.**

Corresponding Lean declaration(s): `O77.MinimumRankCondition`,`O77.instDecidableMinimumRankCondition`.

For natural I,O,H,r,a,b, *define* MinRanks(a,b) *by the following decidable case distinction, in the displayed order:*

$I+H < O+r$	$a = r$
$O+H < I+r$	$b = r$
$I+O < H+r$	*true*
otherwise	$2a+O \le I+H+r+\chi$ *and* $2b+I \le O+H+r+\chi$.

Here $\chi = (I + O + H + r) \bmod 2$. *Feasibility is not part of this total predicate; it is an explicit hypothesis of the theorem identifying it with numerical minimality.*

For $r \le I, O, H$, at most one of the three strict regimes holds. Indeed, adding the output- and input-dominated inequalities forces $H < r$; adding output and hidden forces $I < r$; adding input and hidden forces $O < r$. If none holds, all three reversed weak inequalities hold. Thus the priority order changes no feasible case. The decision instance below computes only integer inequalities and is not an assumed classification theorem.

```
/-- Closed rank conditions, separate from the independent finite-minimality predicate. -/
def MinimumRankCondition (I O H r a b : Nat) : Prop :=
  if I+H < O+r then a = r else
  if O+H < I+r then b = r else
  if I+O < H+r then True else
    2*a+O ≤ I+H+r+(I+O+H+r)%2 ∧
    2*b+I ≤ O+H+r+(I+O+H+r)%2

instance instDecidableMinimumRankCondition (I O H r a b : Nat) : Decidable (MinimumRankCondition I O H
      r a b) :=
  inferInstanceAs (Decidable (if I+H < O+r then a = r else
    if O+H < I+r then b = r else if I+O < H+r then True else
      2*a+O ≤ I+H+r+(I+O+H+r)%2 ∧ 2*b+I ≤ O+H+r+(I+O+H+r)%2))
```

Theorem 13.18 (The entire numerical minimum-rank classification)**.**

Corresponding Lean declaration(s): `077.numerical_minimizer_iff_closed`.

For every feasible rank pair, numerical threshold-minimality is equivalent to MinRanks(a, b).

Proof. By Theorem 13.11, it suffices to characterize $N(a, b) = N_0$. When the first strict regime holds, Lemma 13.13 gives exactly the first branch. If it does not but the second holds, use Lemma 13.14. If neither holds but the third does, equality always holds by Lemma 13.15. In the remaining case the three weak inequalities are available, and Lemma 13.16 gives the last branch. These are the same branches used to define MinRanks, so both directions follow in every case. This is a theorem about the universal numerical-minimum predicate, not the reflexive assertion that a branch formula equals itself. □

```
theorem numerical_minimizer_iff_closed {I O H r a b : Nat}
    (D : RankData I O H r a b) :
    IsNumericalThresholdMinimizer I O H r a b ↔ MinimumRankCondition I O H r a b := by
  rw [numerical_minimizer_iff_baseline D]
  unfold MinimumRankCondition
  split
  · rename_i hO; exact candidate_eq_baseline_output D hO
  · split
    · rename_i _ hI; exact candidate_eq_baseline_input D hI
    · split
      · rename_i _ _ hH
        exact ⟨fun _ => True.intro, fun _ => candidate_eq_baseline_hidden D ⟩hH
      · rename_i hO hI hH
        exact candidate_eq_baseline_balanced D (by omega) (by omega) (by omega)
```

13.5 Multiplicity is a second, separately proved comparison

Lemma 13.19 (Baseline logarithmic-multiplicity candidate)**.**

Corresponding Lean declaration(s): `077.baselineMultiplicity_formula`.

For $r \le I, O, H$,

$$m(r,r) = \begin{cases} 2, & \chi = 1 \text{ and } O + r \le I + H,\ I + r \le O + H,\ H + r \le I + O, \\ 1, & \text{otherwise.} \end{cases}$$

Proof. The residual dimensions at (r, r) are P, Q, J. Their parity is χ, as in Lemma 13.16. Their three triangle inequalities are equivalent, respectively, to the three displayed dimension inequalities, by adding or subtracting the same $2r$ after writing $P = O - r$, $Q = I - r$, $J = H - r$. Substitute these equivalent conditions into the already proved formula for $\mu_0(P, Q, J)$ in Lemma 12.19. It includes zero residual dimensions, because an odd balanced triple cannot contain zero: if, for example, $P = 0$, the other two triangle inequalities force $Q = J$ and hence even parity. □

```
/-- Logarithmic multiplicity at the baseline is a maximum even before restricting to minimizers. -/
theorem baselineMultiplicity_formula {I O H r : Nat}
    (hrI : r ≤ I) (hrO : r ≤ O) (hrH : r ≤ H) :
    candidateMultiplicity I O H r r r =
      if (I+O+H+r)%2 = 1 ∧ O+r ≤ I+H ∧ I+r ≤ O+H ∧ H+r ≤ I+O
      then 2 else 1 := by
  unfold candidateMultiplicity
  rw [Arithmetic.residualMultiplicity_formula]
  have he :
      (((O-r)+(I-r)+residualHidden H r r r)%2 = 1 ∧
       O-r ≤ (I-r)+residualHidden H r r r ∧
       I-r ≤ (O-r)+residualHidden H r r r ∧
       residualHidden H r r r ≤ (O-r)+(I-r)) ↔
      ((I+O+H+r)%2 = 1 ∧ O+r ≤ I+H ∧ I+r ≤ O+H ∧ H+r ≤ I+O) := by
    unfold residualHidden
    omega
  simp only [he]
```

Lemma 13.20 (All feasible multiplicity candidates are bounded by the baseline)**.**

Corresponding Lean declaration(s): `077.candidateMultiplicity_le_baseline`.

For every feasible (a, b), $m(a, b) \le m(r, r)$. This bound does not require (a, b) to minimize the threshold.

Proof. Every residual multiplicity is 1 or 2 by Lemma 12.19. If the baseline value is 2, the bound follows immediately. Otherwise we show that a local value of 2 is impossible. Such a value would make (p, q, h) an odd balanced triple. Since $P = p + \beta$, $Q = q + \alpha$, $J = h + \alpha + \beta$, its inequalities imply

$$P \le Q + J, \qquad Q \le P + J, \qquad J \le P + Q.$$

For example, $p \le q + h$ gives $P = p + \beta \le q + h + \beta \le q + h + 2\alpha + \beta = Q + J$; the second implication exchanges p, q and α, β; the third follows by adding $\alpha + \beta$ to $h \le p + q$. Also $P + Q + J = p + q + h + 2(\alpha + \beta)$ has the same odd parity. Lemma 13.19 would therefore make $m(r, r) = 2$, a contradiction. Hence the local value is 1 when the baseline value is 1. □

```
theorem candidateMultiplicity_le_baseline {I O H r a b : Nat}
    (D : RankData I O H r a b) :
    candidateMultiplicity I O H r a b ≤ candidateMultiplicity I O H r r r := by
  have hb := D.base_bounds
  rw [baselineMultiplicity_formula hb.1 hb.2.1 hb.2.2]
  unfold candidateMultiplicity
  rw [Arithmetic.residualMultiplicity_formula]
  rcases D with ⟨hra, hrb, haI, haH, hbO, hbH, ⟩hab
  unfold residualHidden
  split <;> split <;> omega
```

Theorem 13.21 (Attainment of both stages of the numerical optimization)**.**

Corresponding Lean declaration(s): `077.baseline_minimizes_threshold_maximizes_multiplicity`.

If $r \le I, O, H$, the pair (r, r) is numerically threshold-minimizing. Among all such minimizers the largest multiplicity candidate is $m(r, r)$, with the value in Lemma 13.19.

Proof. The seven fields of $\mathsf{RankData}(I, O, H, r, r, r)$ follow from the three assumed bounds exactly as in Lemma 13.5. Its numerical value equals N_0 by definition, so Theorem 13.11 makes it a minimizer. Every other minimizing pair is feasible by definition, so Lemma 13.20 bounds its multiplicity by that at (r, r). The latter belongs to the minimizing set, giving attainment as well as an upper bound. This does not assert that multiplicity is constant on the minimizing set. □

```
theorem baseline_minimizes_threshold_maximizes_multiplicity {I O H r : Nat}
    (hrI : r ≤ I) (hrO : r ≤ O) (hrH : r ≤ H) :
    IsNumericalThresholdMinimizer I O H r r r ∧
    ∀ a b : Nat, IsNumericalThresholdMinimizer I O H r a b →
      candidateMultiplicity I O H r a b ≤ candidateMultiplicity I O H r r r := by
  have D : RankData I O H r r r := by constructor <;> omega
  refine ⟨(numerical_minimizer_iff_baseline D).mpr rfl, ?⟩_
  intro a b hmin
  exact candidateMultiplicity_le_baseline hmin.1
```

13.6 Two reusable consequences of the window interpretation

Lemma 13.22 (The candidate is the actual minimum on its window)**.**

Corresponding Lean declaration(s): `077.candidate_window_bound`,`077.candidate_window_attained`.

For a feasible pair and every $u \in W_{a,b}$, $N(a, b) \le s_0 + \widetilde{C}(u)$ in $\mathbb{Z}$. There exists $u \in W_{a,b}$ with equality.

Proof. For an arbitrary u in the window, put $t = u - \alpha$. Then $t \le \min(h, q)$ and $\alpha + t = u$. The residual-minimum inequality gives $M_{\widetilde{c}}(\min(h, q)) \le \widetilde{c}(t)$. Add s, use Lemma 13.8, and then use Lemma 13.7 to obtain the claimed bound. For attainment choose $t \le \min(h, q)$ attaining that residual minimum and put $u = \alpha + t$. Lemma 13.6 puts u in the window, and the two identities just used become equalities. Both statements concern this finite nonempty interval, not an unbounded minimum of the polynomial. □

```
/-- The baseline quadratic bounds every value in the rank window. -/
theorem candidate_window_bound {I O H r a b : Nat}
    (D : RankData I O H r a b) (u : Nat)
    (hl : a-r ≤ u) (hu : u ≤ min (H-r) (I-r)) (hb : u+(b-r) ≤ H-r) :
    (candidateTwiceLambda I O H r a b : Int) ≤
      (regularDim I O r r : Int) + residualQuadratic (O-r) (I-r) (H-r) u := by
  have hw := (D.shift_domain.2 u).mpr ⟨hl, hu, ⟩hb
  have ht : u-(a-r) ≤ min (residualHidden H r a b) (I-a) := by omega
```

```
  have he : (a-r)+(u-(a-r)) = u := by omega
  have hval := rank_cost_shift D (u-(a-r))
  rw [he] at hval
  have hle := Arithmetic.minimumUpTo_le
    (residualQuadratic (O-b) (I-a) (residualHidden H r a b)) ht
  rw [candidateTwiceLambda_cast]
  omega

theorem candidate_window_attained {I O H r a b : Nat}
    (D : RankData I O H r a b) :
    ∃ u : Nat, a-r ≤ u ∧ u ≤ min (H-r) (I-r) ∧ u+(b-r) ≤ H-r ∧
      (candidateTwiceLambda I O H r a b : Int) =
        (regularDim I O r r : Int) + residualQuadratic (O-r) (I-r) (H-r) u := by
  obtain ⟨t, ht, ⟩hv := Arithmetic.minimumUpTo_attained
    (residualQuadratic (O-b) (I-a) (residualHidden H r a b))
    (min (residualHidden H r a b) (I-a))
  have hw := (D.shift_domain.2 ((a-r)+t)).mp ⟨by omega, by ⟩omega
  refine ⟨(a-r)+t, hw.1, hw.2.1, hw.2.2, ?⟩_
  rw [candidateTwiceLambda_cast]
  have he := rank_cost_shift D t
  omega
```

Theorem 13.23 (Coordinatewise rank monotonicity of the candidate threshold)**.**

Corresponding Lean declaration(s): `077.candidateTwiceLambda_mono`.

Fix I, O, H, r and two feasible pairs (a, b) and (a', b'). If $a \le a'$ and $b \le b'$, then $N(a, b) \le N(a', b')$.

Proof. Their increments satisfy $\alpha \le \alpha'$ and $\beta \le \beta'$. If $u \in W_{a',b'}$, then $\alpha' \le u \le K_0$ and $u + \beta' \le J$. It follows that $\alpha \le u \le K_0$ and $u + \beta \le J$, so $u \in W_{a,b}$. Choose u attaining the primed candidate by Lemma 13.22. Its unprimed bound gives $N(a, b) \le s_0 + \widetilde{C}(u) = N(a', b')$. The final comparison of natural numbers follows because their integer embedding reflects order. Neither a strict inequality nor a geometric specialization relation is asserted. □

```
/-- Nested rank windows prove monotonicity before any closed-form case split. -/
theorem candidateTwiceLambda_mono {I O H r a b a' b' : Nat}
    (D : RankData I O H r a b) (D' : RankData I O H r a' b')
    (ha : a ≤ a') (hb : b ≤ b') :
    candidateTwiceLambda I O H r a b ≤ candidateTwiceLambda I O H r a' b' := by
  obtain ⟨u, hl, hu, hh, ⟩heq := candidate_window_attained D'
  have hle := candidate_window_bound D u (by omega) hu (by omega)
  omega
```

Lemma 13.24 (Exact minimizer correspondence on the equality locus)**.**

Corresponding Lean declaration(s): `077.residual_minimizers_shift_iff`.

Suppose (a, b) is feasible and $N(a, b) = N_0$. For every $t \in \mathbb{N}$,

$$t \in \mathcal{M}_{\widetilde{c}_{p,q,h}}(\min(h, q)) \quad \Longleftrightarrow \quad t \le \min(h, q) \text{ and } \alpha + t \in \mathcal{M}_{\widetilde{C}}(K_0).$$

Thus the local minimizer set is carried bijectively to the baseline minimizer set intersected with the rank window.

Proof. The equality of candidates and Lemma 13.8 give $(s - s_0) + M_{\widetilde{c}}(\min(h, q)) = M_{\widetilde{C}}(K_0)$. Together with Lemmas 13.6 and 13.7, these are exactly the hypotheses of Lemma 13.3. Apply that lemma and use the proved membership characterization of both minimizer lists. The inverse on indices in the rank window is $u \mapsto u - \alpha$, so injectivity and surjectivity are explicit. This explains why one tied global minimizer can survive while the other is removed: the threshold is unchanged but the local multiplicity candidate drops. □

```
/-- On a threshold-minimizing stratum, the shift gives the actual minimizer correspondence. -/
theorem residual_minimizers_shift_iff {I O H r a b : Nat}
    (D : RankData I O H r a b)
    (heq : candidateTwiceLambda I O H r a b = baselineTwiceLambda I O H r)
    (t : Nat) :
    t ∈ residualMinimizerList (O-b) (I-a) (residualHidden H r a b) ↔
      t ≤ min (residualHidden H r a b) (I-a) ∧
      (a-r)+t ∈ residualMinimizerList (O-r) (I-r) (H-r) := by
  have hs (u : Nat) (_ : u ≤ min (residualHidden H r a b) (I-a)) :
      residualQuadratic (O-r) (I-r) (H-r) ((a-r)+u) =
        ((regularDim I O a b : Int) - (regularDim I O r r : Int)) +
        residualQuadratic (O-b) (I-a) (residualHidden H r a b) u := by
    have := rank_cost_shift D u
    omega
  have h0 := baselineTwiceLambda_cast (I := I) (O := O) D.base_bounds.2.2
  have h1 := candidateTwiceLambda_cast I O H r a b
  have hc : (regularDim I O a b : Int) - (regularDim I O r r : Int) +
      Arithmetic.minimumUpTo
        (residualQuadratic (O-b) (I-a) (residualHidden H r a b))
        (min (residualHidden H r a b) (I-a)) =
      Arithmetic.minimumUpTo (residualQuadratic (O-r) (I-r) (H-r))
        (min (H-r) (I-r)) := by omega
  unfold residualMinimizerList
  simp only [Arithmetic.mem_minimizerList]
  exact Arithmetic.isMinimumOn_restrict_iff _ _ _ _ _ _ D.shift_domain.1 hs hc t
```

13.7 The attained numerical value, with division postponed

Theorem 13.25 (Four branches of the global value)**.**

Corresponding Lean declaration(s): `077.baselineTwiceLambda_output,077.baselineTwiceLambda_input,077.baselineTwiceLambda_hidden,077.baselineTwiceLambda_balanced`.

Suppose $r \le I, O, H$. The attained twice-threshold candidate N_0 is

$$\begin{array}{l|l} I+H<O+r & H(I-r)+Or \\ O+H<I+r & H(O-r)+Ir \\ I+O<H+r & OI. \end{array}$$

In the weak balanced regime it satisfies the following equality in $\mathbb{Z}$:

$$4N_0 = 2(H+r)(I+O) - (I-O)^2 - (H+r)^2 + \chi, \qquad \chi = (I+O+H+r) \bmod 2.$$

These are signed or natural integer identities; they do not use natural division to encode a half-integral exponent.

Proof. The baseline regular term is $s_0 = Or + r(I-r) = Ir + r(O-r)$. Its residual dimensions are P, Q, J. In the first strict regime $d_0(P,Q,J) = JQ$, so

$$N_0 = Or + r(I-r) + (H-r)(I-r) = Or + H(I-r).$$

In the second $d_0(P,Q,J) = JP$, giving $Ir + H(O-r)$ by the same distributive identity. In the third $d_0(P,Q,J) = PQ$, and $s_0 + PQ = Or + [r + (O-r)](I-r) = O[r + (I-r)] = OI$. Each subtraction cancels only under the three assumed rank bounds.

In the balanced case, the proved residual table gives $4d_0(P,Q,J) = 2J(P+Q) - (P-Q)^2 - J^2 + \chi$. Add $4s_0 = 4r(I+O-r)$ and use the integer identity

$$\begin{aligned} &4r(I+O-r) + 2(H-r)(I+O-2r) - (I-O)^2 - (H-r)^2 \\ &\quad = 2(H+r)(I+O) - (I-O)^2 - (H+r)^2. \end{aligned}$$

It follows by distributing the products and squares: the coefficient of $I+O$ is $4r+2(H-r) = 2(H+r)$, while the remaining H, r terms are $-4r^2-4r(H-r)-(H-r)^2 = -(H+r)^2$. This is exactly the displayed formula. Multiplication by 4 is used in Lean to avoid rounding; it corresponds to multiplication of the exponent by 8, not by 4. □

The arithmetic identities give all four candidate values printed in Theorem 8.4 after interpreting the exponent as $N_0/2$ in $\mathbb{Q}$ or $\mathbb{R}$. That real-valued wrapper belongs to the later Mathlib specification, not the checked standard-library code below.

```
theorem baselineTwiceLambda_output {I O H r : Nat}
    (hrI : r ≤ I) (hrO : r ≤ O) (hrH : r ≤ H) (hO : I+H < O+r) :
    baselineTwiceLambda I O H r = H*(I-r)+O*r := by
  have hh : residualHidden H r r r = H-r := by unfold residualHidden; omega
  have hp : (I-r)+(H-r) < O-r := by omega
  simp only [baselineTwiceLambda, candidateTwiceLambda, hh,
    Arithmetic.residualMin_output_dominated _ _ _ hp, regularDim]
  have he : r+(H-r) = H := by omega
  grind only

theorem baselineTwiceLambda_input {I O H r : Nat}
    (hrI : r ≤ I) (hrO : r ≤ O) (hrH : r ≤ H) (hI : O+H < I+r) :
    baselineTwiceLambda I O H r = H*(O-r)+I*r := by
  have hh : residualHidden H r r r = H-r := by unfold residualHidden; omega
  have hq : (O-r)+(H-r) < I-r := by omega
  simp only [baselineTwiceLambda, candidateTwiceLambda, hh,
    Arithmetic.residualMin_input_dominated _ _ _ hq, regularDim]
  have heI : r+(I-r) = I := by omega
  have heO : r+(O-r) = O := by omega
  have heH : r+(H-r) = H := by omega
  grind only

theorem baselineTwiceLambda_hidden {I O H r : Nat}
    (hrI : r ≤ I) (hrO : r ≤ O) (hrH : r ≤ H) (hH : I+O < H+r) :
    baselineTwiceLambda I O H r = O*I := by
  have hh : residualHidden H r r r = H-r := by unfold residualHidden; omega
  have hp : (O-r)+(I-r) < H-r := by omega
  simp only [baselineTwiceLambda, candidateTwiceLambda, hh,
    Arithmetic.residualMin_hidden_dominated _ _ _ hp]
  exact regularDim_add_residual hrI hrO

theorem baselineTwiceLambda_balanced {I O H r : Nat}
    (hrI : r ≤ I) (hrO : r ≤ O) (hrH : r ≤ H)
    (hO : O+r ≤ I+H) (hI : I+r ≤ O+H) (hH : H+r ≤ I+O) :
    4*(baselineTwiceLambda I O H r : Int) =
      2*((H : Int)+r)*((I : Int)+O) - ((I : Int)-O)^2 - ((H : Int)+r)^2 +
      (((I+O+H+r)%2 : Nat) : Int) := by
  have hh : residualHidden H r r r = H-r := by unfold residualHidden; omega
  have hres := Arithmetic.residualMin_balanced (O-r) (I-r) (H-r)
    (by omega) (by omega) (by omega)
  have hpar : ((O-r)+(I-r)+(H-r))%2 = (I+O+H+r)%2 := by omega
  rw [hpar] at hres
  simp only [Int.natCast_sub hrI, Int.natCast_sub hrO, Int.natCast_sub hrH] at hres
  simp only [baselineTwiceLambda, candidateTwiceLambda, hh, regularDim,
    Int.natCast_add, Int.natCast_mul, Int.natCast_sub hrI]
  grind only
```

13.8 Closed arithmetic controls

The first two examples below invoke the universal numerical-minimality theorem itself. They prove that $(a,b) = (2,1)$ minimizes at $(I,O,H,r) = (2,2,2,1)$, and that $(2,0)$ does not

minimize at $(2, 2, 2, 0)$. The first example is not a finite search hidden behind the word "minimum": its type contains the universal comparison with all feasible ranks. The remaining examples are exact computations accepted by ordinary `decide`. They test the balanced tie, its one-index truncation, an absent but non-minimizing residual, and each strict dimension regime. The two tied examples retain multiplicities 2 and 1 at the same threshold. No example is used in place of the universal theorem.

```
example : IsNumericalThresholdMinimizer 2 2 2 1 2 1 := by
  have D : RankData 2 2 2 1 2 1 := by constructor <;> decide
  exact (numerical_minimizer_iff_closed D).mpr (by decide)

example : ¬ IsNumericalThresholdMinimizer 2 2 2 0 2 0 := by
  have D : RankData 2 2 2 0 2 0 := by constructor <;> decide
  intro h
  have hclosed := (numerical_minimizer_iff_closed D).mp h
  have hfalse : ¬ MinimumRankCondition 2 2 2 0 2 0 := by decide
  exact hfalse hclosed

-- Small exact tests include ties, lost ties, and a non-minimal absent residual.
example : MinimumRankCondition 2 2 2 1 1 1 := by decide
example : MinimumRankCondition 2 2 2 1 2 1 := by decide
example : ¬ MinimumRankCondition 2 2 2 0 2 0 := by decide
example : baselineTwiceLambda 2 2 2 0 = 3 := by decide
example : baselineTwiceLambda 1 5 2 1 = 5 := by decide
example : baselineTwiceLambda 5 1 2 1 = 5 := by decide
example : baselineTwiceLambda 2 2 6 1 = 4 := by decide
example : candidateMultiplicity 2 2 2 1 1 1 = 2 ∧
    candidateMultiplicity 2 2 2 1 2 1 = 1 := by decide

end O77
```

From arithmetic to measured minima. The finite results compare all feasible ranks. Their measured application also requires actual rank necessity, realization over the fixed target, the local pair at every exact fit, and uniqueness. These additional arguments are proved in `O77FiberClassification` and `O77SemanticMinimum`; they are not assumptions in the final answer.

14 Rectangular matrix and chart algebra

This section moves from the finite rank formulas to the algebra inside the elimination chart. It proves the finite-sum matrix identities, the first gauge and both of its round trips, the row normalizer and both inverse products, the additive shear, and the exact reduced error vector. All sizes remain symbolic and may be zero. No conclusion about ranks, determinants, smoothness, a norm, or a volume is a field of a structure used by these proofs.

The dependency-independent core intentionally imports only `Std`. For this small algebraic part it uses a transparent entry representation of rectangular matrices and the ordinary unital-ring laws in `Lean.Grind.Ring`. This is *not* a substitute axiomatic model for the real numbers or for Lebesgue measure. The theorems quantify over every coefficient ring; their later O77 applications use Mathlib matrices and the pinned analytic libraries. Mathlib provides `Ring.toGrindRing`, so no fictitious real coefficient type is introduced. The core is limited to finite matrix operations and deliberately does not rebuild analysis or linear algebra.

Throughout this section R denotes an arbitrary associative unital ring, not necessarily commutative. Its additive group is commutative. Natural matrix sizes have no positivity hypothesis. For $n \in \mathbb{N}$, write $[n] = \{0, \ldots, n-1\}$; $[0]$ is empty. Code uses `Finn`. Products are always written in their stated order.

14.1 Finite sums and ordinary rectangular matrices

Definition 14.1 (The finite sum used by the entry representation)**.**

Corresponding Lean declaration(s): `O77.MatrixCore.sum`.

For a function $f : [n] \to R$, define $\Sigma_n f$ by $\Sigma_0 f = 0$ and $\Sigma_{n+1} f = \Sigma_n(f|_{[n]}) + f(n)$. This recursive definition is independent of every chart or asymptotic statement.

The restriction $[n] \hookrightarrow [n+1]$ is `Fin.castSucc`; its complementary final index is `Fin.lastn`. The following namespace is opened and later closed within the same extractable stream as the preceding arithmetic chapters.

```
namespace O77.MatrixCore
universe u
variable {R : Type u} [Lean.Grind.Ring R]

/-- Finite sum in increasing index order; the empty sum is zero. -/
def sum (n : Nat) (f : Fin n → R) : R :=
  match n with
  | 0 => 0
  | k + 1 => sum k (fun i => f i.castSucc) + f (Fin.last k)
```

Lemma 14.2 (Zero summands and pointwise replacement)**.**

Corresponding Lean declaration(s): `O77.MatrixCore.sum_zero`,`O77.MatrixCore.sum_congr`.

For every n, $\Sigma_n(i \mapsto 0) = 0$. If $f, g : [n] \to R$ satisfy $f(i) = g(i)$ for every i, then $\Sigma_n f = \Sigma_n g$.

Proof. For the first assertion, the $n = 0$ case is the definition. The induction step is $\Sigma_{n+1} 0 = \Sigma_n 0 + 0 = 0 + 0 = 0$. For the second assertion, function extensionality identifies f and g, so applying the function Σ_n gives equal results. Neither assertion requires a choice of an ordering beyond the fixed recursion. □

```
@[simp] theorem sum_zero (n : Nat) : sum n (fun _ => (0 : R)) = 0 := by
  induction n with
  | zero => rfl
  | succ n ih => simp only [sum, ih]; grind

theorem sum_congr (n : Nat) (f g : Fin n → R)
    (h : ∀ i, f i = g i) : sum n f = sum n g := by
  have : f = g := funext h
  rw [this]
```

Lemma 14.3 (Finite sums respect addition, subtraction, and multiplication by a constant)**.**

Corresponding Lean declaration(s): `O77.MatrixCore.sum_add`,`O77.MatrixCore.sum_sub`,`O77.MatrixCore.sum_mul_right`,`O77.MatrixCore.sum_mul_left`.

For every n, functions $f, g : [n] \to R$, and $a \in R$,

$$\Sigma_n(f+g) = \Sigma_n f + \Sigma_n g, \qquad \Sigma_n(f-g) = \Sigma_n f - \Sigma_n g,$$
$$\Sigma_n(i \mapsto f(i)a) = (\Sigma_n f)a, \qquad \Sigma_n(i \mapsto af(i)) = a(\Sigma_n f).$$

Here addition and subtraction of functions are pointwise.

Proof. Each assertion is proved by induction on n. For $n = 0$ the required identities are $0 = 0 + 0$, $0 = 0 - 0$, $0 = 0a$, and $0 = a0$, which are ring laws. For the addition step, the recursion and the induction hypothesis give $(\Sigma_n f + \Sigma_n g) + (f(n) + g(n))$; associativity and commutativity of addition change this to $(\Sigma_n f + f(n)) + (\Sigma_n g + g(n))$. For subtraction they

give $(\Sigma_n f - \Sigma_n g) + (f(n) - g(n))$; expansion of subtraction as addition of an additive inverse gives $(\Sigma_n f + f(n)) - (\Sigma_n g + g(n))$. For multiplication on the right they give $(\Sigma_n f)a + f(n)a = (\Sigma_n f + f(n))a$ by right distributivity. On the left they give $a(\Sigma_n f) + af(n) = a(\Sigma_n f + f(n))$ by left distributivity. No step exchanges two multiplicative factors. □

```
theorem sum_add (n : Nat) (f g : Fin n → R) :
    sum n (fun i => f i + g i) = sum n f + sum n g := by
  induction n with
  | zero => simp only [sum]; grind
  | succ n ih => simp only [sum, ih]; grind

theorem sum_sub (n : Nat) (f g : Fin n → R) :
    sum n (fun i => f i - g i) = sum n f - sum n g := by
  induction n with
  | zero => simp only [sum]; grind
  | succ n ih => simp only [sum, ih]; grind

theorem sum_mul_right (n : Nat) (f : Fin n → R) (a : R) :
    sum n (fun i => f i * a) = sum n f * a := by
  induction n with
  | zero => simp only [sum]; grind
  | succ n ih => simp only [sum, ih]; grind

theorem sum_mul_left (n : Nat) (a : R) (f : Fin n → R) :
    sum n (fun i => a * f i) = a * sum n f := by
  induction n with
  | zero => simp only [sum]; grind
  | succ n ih => simp only [sum, ih]; grind
```

Lemma 14.4 (Interchange of two finite sums)**.**

Corresponding Lean declaration(s): `077.MatrixCore.sum_swap`.

For $f : [m] \times [n] \to R$, $\Sigma_m(i \mapsto \Sigma_n(j \mapsto f(i,j))) = \Sigma_n(j \mapsto \Sigma_m(i \mapsto f(i,j)))$.

Proof. Induct on m. When $m = 0$, the left side is zero and the right side is a sum of zeros. For $m+1$, separate the final row in the left side. The induction hypothesis changes the first m rows into $\Sigma_n(j \mapsto \Sigma_m(i \mapsto f(i,j)))$. Adding the final-row sum and using additivity of Σ_n gives $\Sigma_n(j \mapsto \Sigma_m(i \mapsto f(i,j)) + f(m,j))$. The inner expression is precisely the recursion defining Σ_{m+1}. This is a finite algebraic identity, not a measure-theoretic Fubini assumption. □

```
theorem sum_swap (m n : Nat) (f : Fin m → Fin n → R) :
    sum m (fun i => sum n (fun j => f i j)) =
      sum n (fun j => sum m (fun i => f i j)) := by
  induction m with
  | zero => simp only [sum, sum_zero]
  | succ m ih => simp only [sum, sum_add, ih]
```

Lemma 14.5 (A single nonzero summand)**.**

Corresponding Lean declaration(s): `077.MatrixCore.sum_single`.

For $j \in [n]$ and $a \in R$, $\Sigma_n(i \mapsto [\,i = j\,]a) = a$, where the summand is a if $i = j$ and zero otherwise.

Proof. Induct on n. There is no $j \in [0]$, so the base case follows from the empty type. For $j \in [n+1]$, either $j = n$ or $j < n$. If $j = n$, every summand in the first n indices is zero and the final summand is a; the result is $0 + a = a$. If $j < n$, the final summand is zero and the induction hypothesis gives a for the first n summands; the result is $a + 0 = a$. Injectivity of the index inclusion and its disjointness from the final index follow from the inequalities defining `Finn`; the code proves those guards explicitly. □

```
theorem sum_single (n : Nat) (j : Fin n) (a : R) :
    sum n (fun i => if i = j then a else 0) = a := by
  induction n with
  | zero => exact Fin.elim0 j
  | succ n ih =>
    refine Fin.lastCases ?_ (fun j => ?_) j
    · have h : ∀ i : Fin n, i.castSucc ≠ Fin.last n := by
        intro i hi; have := congrArg Fin.val hi; simp at this; omega
      simp only [sum, h, ↓reduceIte, sum_zero]
      grind
    · have h : Fin.last n ≠ j.castSucc := by
        intro hi; have := congrArg Fin.val hi; simp at this; omega
      simp only [sum, Fin.castSucc_inj, h, ↓reduceIte, ih]
      grind
```

Definition 14.6 (Entry matrices and their algebraic operations)**.**

Corresponding Lean declaration(s): `O77.MatrixCore.Mat`,`O77.MatrixCore.add`,`O77.MatrixCore.sub`,`O77.MatrixCore.zero`,`O77.MatrixCore.one`,`O77.MatrixCore.mul`.

Set $M_{m,n}(R) = ([m] \to [n] \to R)$. Addition, subtraction, and the zero matrix are entrywise. The identity I_n has entry 1 when its two indices agree and zero otherwise. For $A \in M_{m,n}(R)$ and $B \in M_{n,k}(R)$ define $(AB)_{ij} = \Sigma_n(t \mapsto A_{it}B_{tj})$. These are concrete functions, not operations supplied by a chart-oracle structure.

The symbols $+$, $-$ and juxtaposition in the mathematical statements below refer to these operations. Code uses explicit `add`, `sub`, and `mul` so that a default pointwise function multiplication cannot be mistaken for matrix multiplication.

```
abbrev Mat (m n : Nat) (R : Type u) := Fin m → Fin n → R

def add {m n : Nat} (A B : Mat m n R) : Mat m n R := fun i j => A i j + B i j
def sub {m n : Nat} (A B : Mat m n R) : Mat m n R := fun i j => A i j - B i j
def zero (m n : Nat) : Mat m n R := fun _ _ => 0
def one (n : Nat) : Mat n n R := fun i j => if i = j then 1 else 0
def mul {m n k : Nat} (A : Mat m n R) (B : Mat n k R) : Mat m k R :=
  fun i j => sum n (fun t => A i t * B t j)
```

Lemma 14.7 (Zero products)**.**

Corresponding Lean declaration(s): `O77.MatrixCore.mul_zero`,`O77.MatrixCore.zero_mul`.

For compatible shapes, $A0 = 0$ and $0A = 0$.

Proof. At each output entry every summand is respectively $A_{it}0 = 0$ or $0A_{tj} = 0$. The sum of zeros is zero by Lemma 14.2; matrix equality is entrywise equality. When the summation index is empty the same proof uses the empty sum, not a nonempty-index assumption. □

```
@[simp] theorem mul_zero {m n k : Nat} (A : Mat m n R) :
    mul A (zero n k) = zero m k := by
  funext i j; simp only [mul, zero, Lean.Grind.Semiring.mul_zero, sum_zero]

@[simp] theorem zero_mul {m n k : Nat} (A : Mat n k R) :
    mul (zero m n) A = zero m k := by
  funext i j; simp only [mul, zero, Lean.Grind.Semiring.zero_mul, sum_zero]
```

Lemma 14.8 (The identity matrices are two-sided units)**.**

Corresponding Lean declaration(s): `O77.MatrixCore.mul_one`,`O77.MatrixCore.one_mul`.

For $A \in M_{m,n}(R)$, $AI_n = A$ and $I_mA = A$.

Proof. The (i, j) entry of AI_n is $\Sigma_n(t \mapsto A_{it}\delta_{tj})$. If $t = j$ the summand is $A_{ij}1 = A_{ij}$; otherwise it is zero. Lemma 14.5 gives A_{ij}. For $I_m A$, the summand is $\delta_{it}A_{tj}$, equal to $1A_{ij} = A_{ij}$ exactly when $t = i$ and zero otherwise; the same single-summand lemma, now on $[m]$, applies. If there are no output entries, function extensionality proves the equality without producing an index in an empty set. □

```
@[simp] theorem mul_one {m n : Nat} (A : Mat m n R) : mul A (one n) = A := by
  funext i j
  unfold mul one
  calc
    sum n (fun t => A i t * (if t = j then 1 else 0)) =
        sum n (fun t => if t = j then A i j else 0) := by
      apply sum_congr; intro t; split <;> grind
    _ = A i j := sum_single n j (A i j)

@[simp] theorem one_mul {m n : Nat} (A : Mat m n R) : mul (one m) A = A := by
  funext i j
  unfold mul one
  calc
    sum m (fun t => (if i = t then 1 else 0) * A t j) =
        sum m (fun t => if t = i then A i j else 0) := by
      apply sum_congr; intro t; split <;> split <;> grind
    _ = A i j := sum_single m i (A i j)
```

Lemma 14.9 (The four rectangular distributive laws)**.**

Corresponding Lean declaration(s): `O77.MatrixCore.mul_add`,`O77.MatrixCore.add_mul`,`O77.MatrixCore.mul_sub`,`O77.MatrixCore.sub_mul`.

With every sum or difference taken between matrices of equal shape, $A(B+C) = AB+AC$, $(A+B)C = AC+BC$, $A(B-C) = AB-AC$, *and* $(A-B)C = AC-BC$.

Proof. At each entry, left or right scalar distributivity expands a summand into the stated two summands in the same multiplicative order. Pointwise replacement in the finite sum is justified by Lemma 14.2. The addition or subtraction formula of Lemma 14.3 then separates the two finite sums. The resulting two sums are exactly the corresponding entries of the two matrix products on the right. □

```
theorem mul_add {m n k : Nat} (A : Mat m n R) (B C : Mat n k R) :
    mul A (add B C) = add (mul A B) (mul A C) := by
  funext i j
  simp only [mul, add, Lean.Grind.Semiring.left_distrib, sum_add]

theorem add_mul {m n k : Nat} (A B : Mat m n R) (C : Mat n k R) :
    mul (add A B) C = add (mul A C) (mul B C) := by
  funext i j
  simp only [mul, add, Lean.Grind.Semiring.right_distrib, sum_add]

theorem mul_sub {m n k : Nat} (A : Mat m n R) (B C : Mat n k R) :
    mul A (sub B C) = sub (mul A B) (mul A C) := by
  funext i j
  simp only [mul, sub]
  calc
    sum n (fun t => A i t * (B t j - C t j)) =
        sum n (fun t => A i t * B t j - A i t * C t j) := by
      apply sum_congr; intro t; grind
    _ = _ := sum_sub n _ _

theorem sub_mul {m n k : Nat} (A B : Mat m n R) (C : Mat n k R) :
    mul (sub A B) C = sub (mul A C) (mul B C) := by
```

```
  funext i j
  simp only [mul, sub]
  calc
    sum n (fun t => (A i t - B i t) * C t j) =
        sum n (fun t => A i t * C t j - B i t * C t j) := by
      apply sum_congr; intro t; grind
    _ = _ := sum_sub n _ _
```

Lemma 14.10 (Associativity with three independent rectangular shapes)**.**

Corresponding Lean declaration(s): `077.MatrixCore.mul_assoc`.

For $A \in M_{m,n}(R)$, $B \in M_{n,k}(R)$, and $C \in M_{k,l}(R)$, $(AB)C = A(BC)$.

Proof. Fix $i \in [m]$ and $j \in [l]$. The left-hand entry is

$$\Sigma_k\big(t \mapsto (\Sigma_n(s \mapsto A_{is}B_{st}))C_{tj}\big) = \Sigma_k\big(t \mapsto \Sigma_n(s \mapsto (A_{is}B_{st})C_{tj})\big).$$

Move C_{tj} into the inner finite sum using right multiplication in Lemma 14.3, then interchange the two sums by Lemma 14.4. Scalar associativity changes each summand to $A_{is}(B_{st}C_{tj})$. Pull A_{is} out of the inner sum using left multiplication. The result is $\Sigma_n(s \mapsto A_{is}\Sigma_k(t \mapsto B_{st}C_{tj}))$, the entry of $A(BC)$. Extensionality gives the matrix identity. At no point is matrix multiplication assumed commutative or are n and k identified. □

```
theorem mul_assoc {m n k l : Nat} (A : Mat m n R) (B : Mat n k R) (C : Mat k l R) :
    mul (mul A B) C = mul A (mul B C) := by
  funext i j
  simp only [mul]
  calc
    sum k (fun t => sum n (fun s => A i s * B s t) * C t j) =
        sum k (fun t => sum n (fun s => (A i s * B s t) * C t j)) := by
      apply sum_congr; intro t; exact (sum_mul_right n _ _).symm
    _ = sum n (fun s => sum k (fun t => (A i s * B s t) * C t j)) := sum_swap k n _
    _ = sum n (fun s => A i s * sum k (fun t => B s t * C t j)) := by
      apply sum_congr; intro s
      simp only [Lean.Grind.Semiring.mul_assoc, sum_mul_left]
```

Lemma 14.11 (Additive cancellation for equal-shaped matrices)**.**

Corresponding Lean declaration(s): `077.MatrixCore.add_sub_cancel`,`077.MatrixCore.sub_add_cancel`.

For $A, B \in M_{m,n}(R)$, $(A + B) - B = A$ and $(A - B) + B = A$.

Proof. For every entry, replace subtraction by addition of the additive inverse and apply the additive group cancellation law. Extensionality then gives each asserted matrix equality. The code's ring tactic constructs proofs of these scalar group identities; it is not supplied with either matrix identity as a premise. □

```
theorem add_sub_cancel {m n : Nat} (A B : Mat m n R) : sub (add A B) B = A := by
  funext i j; simp only [add, sub]; grind

theorem sub_add_cancel {m n : Nat} (A B : Mat m n R) : add (sub A B) B = A := by
  funext i j; simp only [add, sub]; grind
```

14.2 Block indexing without hidden shape equalities

Lemma 14.12 (Splitting the summation range into two consecutive blocks)**.**

Corresponding Lean declaration(s): `077.MatrixCore.sum_split`.

For $f : [m + n] \to R$, $\Sigma_{m+n} f = \Sigma_m(i \mapsto f(i)) + \Sigma_n(j \mapsto f(m + j))$, where the two maps into $[m + n]$ are the displayed inclusions.

Proof. Induct on n. For $n = 0$, the first inclusion is the identity and the second sum is zero. For $n + 1$, remove the last index $m + n$ from $\Sigma_{m+n+1} f$. The induction hypothesis splits the remaining sum into the first m indices and the next n indices. Adding back $f(m+n)$ completes the second sum to Σ_{n+1}. Associativity of addition proves the result. The inclusions into the enlarged range agree because their underlying natural values are respectively i and $m + j$; the `Fin` bounds are carried by their constructors. This argument covers $m = 0$ as well. □

```
theorem sum_split (m n : Nat) (f : Fin (m+n) → R) :
    sum (m+n) f = sum m (fun i => f (i.castAdd n)) +
      sum n (fun j => f (j.natAdd m)) := by
  induction n with
  | zero => simp [sum, Lean.Grind.Semiring.add_zero]
  | succ n ih =>
    change sum (m+n) (fun i => f i.castSucc) + f (Fin.last (m+n)) =
      sum m (fun i => f (i.castAdd (n+1))) +
        (sum n (fun j => f (j.castSucc.natAdd m)) + f ((Fin.last n).natAdd m))
    rw [ih]
    have hc : (fun i : Fin m => f ((i.castAdd n).castSucc)) =
        (fun i : Fin m => f (i.castAdd (n+1))) := rfl
    rw [hc]
    simp only [Fin.castSucc_natAdd, Fin.natAdd_last]
    grind
```

Definition 14.13 (Horizontal and vertical concatenation)**.**

Corresponding Lean declaration(s): `O77.MatrixCore.hcat`,`O77.MatrixCore.vcat`,`O77.MatrixCore.hcat_vcat`,`O77.MatrixCore.mul_hcat`,`O77.MatrixCore.vcat_mul`.

For $A \in M_{m,n}(R)$ and $B \in M_{m,k}(R)$, $[A \mid B]$ has entry A_{ij} for $j < n$ and $B_{i,j-n}$ for $n \le j < n + k$. For $A \in M_{m,k}(R)$ and $B \in M_{n,k}(R)$, $\binom{A}{B}$ has entry A_{ij} for $i < m$ and $B_{i-m,j}$ for $m \le i < m + n$. For compatible A, B, C, D the resulting operations satisfy

$$[A \mid B]\begin{pmatrix} C \\ D \end{pmatrix} = AC + BD, \qquad A[B \mid C] = [AB \mid AC], \qquad \begin{pmatrix} A \\ B \end{pmatrix} C = \begin{pmatrix} AC \\ BC \end{pmatrix}.$$

Proof. For the first formula the inner index range is $[n + k]$. Split it at n by Lemma 14.12; the two sums are the entries of AC and BD. For the second formula, each output column lies in exactly one of the two column blocks. In the first block its entries are the finite sums defining AB; in the second they are those defining AC. For the third formula, split the output row index in the same way: the entries are those of AC in the first row block and of BC in the second. The code uses `Fin.addCases`, not a cast equating differently shaped matrices. □

```
def hcat {m n k : Nat} (A : Mat m n R) (B : Mat m k R) : Mat m (n+k) R :=
  fun i => Fin.addCases (A i) (B i)
def vcat {m n k : Nat} (A : Mat m k R) (B : Mat n k R) : Mat (m+n) k R :=
  Fin.addCases A B

theorem hcat_vcat {m n k l : Nat} (A : Mat m n R) (B : Mat m k R)
    (C : Mat n l R) (D : Mat k l R) :
    mul (hcat A B) (vcat C D) = add (mul A C) (mul B D) := by
  funext i j
  unfold mul
  rw [sum_split]
  simp only [hcat, vcat, Fin.addCases_left, Fin.addCases_right, add]

theorem mul_hcat {m n k l : Nat} (A : Mat m n R)
    (B : Mat n k R) (C : Mat n l R) :
    mul A (hcat B C) = hcat (mul A B) (mul A C) := by
  funext i j
```

```
  refine Fin.addCases (fun j => ?_) (fun j => ?_) j <;>
    simp only [mul, hcat, Fin.addCases_left, Fin.addCases_right]

theorem vcat_mul {m n k l : Nat} (A : Mat m k R)
    (B : Mat n k R) (C : Mat k l R) :
    mul (vcat A B) C = vcat (mul A C) (mul B C) := by
  funext i j
  refine Fin.addCases (fun i => ?_) (fun i => ?_) i <;>
    simp only [mul, vcat, Fin.addCases_left, Fin.addCases_right]
```

Lemma 14.14 (Two-by-two block multiplication)**.**

Corresponding Lean declaration(s): `077.MatrixCore.block,077.MatrixCore.add_vcat,077.MatrixCore.block_mul_vcat,077.MatrixCore.block_mul_block`.

Define $\left(\begin{smallmatrix} A & B \\ C & D \end{smallmatrix}\right)$ *as* $[\binom{A}{C} \mid \binom{B}{D}]$*. For six arbitrary sizes and blocks of the indicated compatible shapes,*

$$\begin{pmatrix} A & B \\ C & D \end{pmatrix}\begin{pmatrix} U & V \\ X & Y \end{pmatrix} = \begin{pmatrix} AU+BX & AV+BY \\ CU+DX & CV+DY \end{pmatrix}.$$

For a single column block the corresponding formula is $\left(\begin{smallmatrix} A & B \\ C & D \end{smallmatrix}\right)\binom{U}{V} = \binom{AU+BV}{CU+DV}$.

Proof. Entrywise addition respects vertical concatenation, since each row belongs to one row block and both sides then have the same two summands. Applying the first and third identities of Definition 14.13 proves the single-column-block formula. For a two-column-block matrix, apply the second identity of that definition to separate its two column blocks, and then apply the single-column-block formula to each. Concatenating these two results is exactly the displayed two-by-two matrix. This proves the formula for independent rectangular sizes rather than for four equally sized square blocks. □

```
def block {m n k l : Nat} (A : Mat m k R) (B : Mat m l R)
    (C : Mat n k R) (D : Mat n l R) : Mat (m+n) (k+l) R :=
  hcat (vcat A C) (vcat B D)


theorem add_vcat {m n k : Nat} (A C : Mat m k R) (B D : Mat n k R) :
    add (vcat A B) (vcat C D) = vcat (add A C) (add B D) := by
  funext i j
  refine Fin.addCases (fun i => ?_) (fun i => ?_) i <;>
    simp only [vcat, add, Fin.addCases_left, Fin.addCases_right]

theorem block_mul_vcat {m n k l t : Nat} (A : Mat m k R) (B : Mat m l R)
    (C : Mat n k R) (D : Mat n l R) (U : Mat k t R) (V : Mat l t R) :
    mul (block A B C D) (vcat U V) =
      vcat (add (mul A U) (mul B V)) (add (mul C U) (mul D V)) := by
  simp only [block, hcat_vcat, vcat_mul, add_vcat]

theorem block_mul_block {m n k l p q : Nat} (A : Mat m k R) (B : Mat m l R)
    (C : Mat n k R) (D : Mat n l R) (U : Mat k p R) (V : Mat k q R)
    (X : Mat l p R) (Y : Mat l q R) :
    mul (block A B C D) (block U V X Y) =
      block (add (mul A U) (mul B X)) (add (mul A V) (mul B Y))
        (add (mul C U) (mul D X)) (add (mul C V) (mul D Y)) := by
  change mul (block A B C D) (hcat (vcat U X) (vcat V Y)) = _
  rw [mul_hcat, block_mul_vcat, block_mul_vcat]
  rfl
```

Lemma 14.15 (Remaining zero and cancellation identities)**.**

Corresponding Lean declaration(s): `O77.MatrixCore.add_zero`,`O77.MatrixCore.zero_add`,`O77.MatrixCore.sub_self`.

For any rectangular A, $A + 0 = 0 + A = A$ and $A - A = 0$.

Proof. The entries are respectively $A_{ij} + 0$, $0 + A_{ij}$, and $A_{ij} - A_{ij}$. The three scalar additive-group laws give the stated entries, hence the matrices are equal. □

```
@[simp] theorem add_zero {m n : Nat} (A : Mat m n R) : add A (zero m n) = A := by
  funext i j; simp only [add, zero]; grind
@[simp] theorem zero_add {m n : Nat} (A : Mat m n R) : add (zero m n) A = A := by
  funext i j; simp only [add, zero]; grind
@[simp] theorem sub_self {m n : Nat} (A : Mat m n R) : sub A A = zero m n := by
  funext i j; simp only [sub, zero]; grind
```

Lemma 14.16 (The block-diagonal identity is the ordinary identity)**.**

Corresponding Lean declaration(s): `O77.MatrixCore.block_diagonal_one`.

For every m, n, $\left(\begin{smallmatrix} I_m & 0 \\ 0 & I_n \end{smallmatrix}\right) = I_{m+n}$.

Proof. Split both indices into their two consecutive ranges. If both are in the first range, they are equal after inclusion exactly when they were equal before inclusion, so both matrices have the same Kronecker entry. The same argument applies if both are in the second range: $m + i = m + j$ is equivalent to $i = j$. If the indices are in different ranges, the first is less than m and the second is at least m, hence they cannot agree; both entries are zero. These four cases exhaust all entries, including when a range is empty. □

```
theorem block_diagonal_one (m n : Nat) :
    block (one m : Mat m m R) (zero m n) (zero n m) (one n) = one (m+n) := by
  funext i j
  refine Fin.addCases (fun i => ?_) (fun i => ?_) i <;>
    refine Fin.addCases (fun j => ?_) (fun j => ?_) j
  · have h : i.castAdd n = j.castAdd n ↔ i = j := by
      constructor
      · intro he; exact Fin.ext (congrArg (fun z : Fin (m+n) => z.val) he)
      · intro he; rw [he]
    simp [block, hcat, vcat, one, h]
  · have h : i.castAdd n ≠ j.natAdd m := by
      intro he; have := congrArg Fin.val he; simp at this; omega
    simp [block, hcat, vcat, one, zero, h]
  · have h : i.natAdd m ≠ j.castAdd n := by
      intro he; have := congrArg Fin.val he; simp at this; omega
    simp [block, hcat, vcat, one, zero, h]
  · have h : i.natAdd m = j.natAdd m ↔ i = j := by
      constructor
      · intro he
        apply Fin.ext
        have hv := congrArg Fin.val he
        simp only [Fin.val_natAdd] at hv
        omega
      · intro he; rw [he]
    simp [block, hcat, vcat, one, h]

end O77.MatrixCore
```

14.3 The first gauge: both round trips and product preservation

Definition 14.17 (Independent input and normalized coordinate tuples)**.**

Corresponding Lean declaration(s): `O77.MatrixCore.GaugeInput`,`O77.MatrixCore.GaugeCoordinates`.

Fix $a, k, q, o \in \mathbb{N}$. An input tuple is (U, V, B, D) with $U \in M_{a,q}(R)$, $V \in M_{k,q}(R)$, $B \in M_{o,a}(R)$, $D \in M_{o,k}(R)$. A normalized tuple is $(X, S, \bar{B}, \bar{D})$ of these same four respective shapes. The two record types have precisely these four matrix fields and no proof fields.

The square pivot $P \in M_{a,a}(R)$ and the lower-left block $C \in M_{k,a}(R)$ will be retained as parameters. In the original chart these are A_{11} and A_{21}; the tuple letters U, V here mean A_{12}, A_{22}, not the later error-coordinate matrix U. The scopes are separated in both the statement and the code.

```
namespace O77.MatrixCore
universe u
variable {R : Type u} [Lean.Grind.Ring R]

structure GaugeInput (a k q o : Nat) (R : Type u) where
  a12 : Mat a q R
  a22 : Mat k q R
  b1 : Mat o a R
  b2 : Mat o k R

structure GaugeCoordinates (a k q o : Nat) (R : Type u) where
  x : Mat a q R
  s : Mat k q R
  b1bar : Mat o a R
  b2bar : Mat o k R
```

Definition 14.18 (The explicit forward and backward gauge formulas)**.**

Corresponding Lean declaration(s): `O77.MatrixCore.gaugeForward`,`O77.MatrixCore.gaugeBackward`.

For $P, E \in M_{a,a}(R)$ and $C \in M_{k,a}(R)$, define

$$F_{P,E,C}(U, V, B, D) = (EU,\ V - CEU,\ BP + DC,\ D),$$
$$G_{P,E,C}(X, S, \bar{B}, \bar{D}) = (PX,\ S + CX,\ (\bar{B} - \bar{D}C)E,\ \bar{D}).$$

These are total expressions built from addition, subtraction and multiplication of their matrix entries. No inverse is taken by a totalized inverse operation: E is an actual matrix argument.

```
def gaugeForward {a k q o : Nat} (P E : Mat a a R) (C : Mat k a R)
    (z : GaugeInput a k q o R) : GaugeCoordinates a k q o R where
  x := mul E z.a12
  s := sub z.a22 (mul C (mul E z.a12))
  b1bar := add (mul z.b1 P) (mul z.b2 C)
  b2bar := z.b2

def gaugeBackward {a k q o : Nat} (P E : Mat a a R) (C : Mat k a R)
    (z : GaugeCoordinates a k q o R) : GaugeInput a k q o R where
  a12 := mul P z.x
  a22 := add z.s (mul C z.x)
  b1 := mul (sub z.b1bar (mul z.b2bar C)) E
  b2 := z.b2bar
```

Theorem 14.19 (The two exact gauge round trips)**.**

Corresponding Lean declaration(s): `O77.MatrixCore.gauge_backward_forward`,`O77.MatrixCore.gauge_forward_backward`.

If $PE = I_a$, then $G_{P,E,C}F_{P,E,C}$ is the identity on every input tuple. If $EP = I_a$, then

$F_{P,E,C}G_{P,E,C}$ is the identity on every normalized tuple. Consequently, if both inverse identities hold, the two maps are mutually inverse.

Proof. For the first composite, its four entries are PEU, $(V - CEU) + CEU$, $((BP + DC) - DC)E$, and D. The first is $(PE)U = U$; the second is V by additive cancellation; the third is $(BP)E = B(PE) = B$; the fourth is already D. Thus every input field is recovered. For the second composite, the four entries are EPX, $(S + CX) - CEPX$, $((\bar{B} - \bar{D}C)E)P + \bar{D}C$, and $\bar{D}$. They simplify respectively to X, $(S + CX) - CX = S$, $(\bar{B} - \bar{D}C)(EP) + \bar{D}C = \bar{B}$, and $\bar{D}$. Only the indicated one-sided inverse is used in each composite. Over a general ring we do not infer one inverse identity from the other. If P and C themselves vary, adjoining them unchanged to each tuple makes these identities apply at each of their values. □

```
theorem gauge_backward_forward {a k q o : Nat} (P E : Mat a a R) (C : Mat k a R)
    (hPE : mul P E = one a) (z : GaugeInput a k q o R) :
    gaugeBackward P E C (gaugeForward P E C z) = z := by
  cases z with
  | mk U V B D =>
    simp only [gaugeForward, gaugeBackward]
    rw ←[ mul_assoc P E U, hPE, one_mul, sub_add_cancel, add_sub_cancel,
      mul_assoc B P E, hPE, mul_one]

theorem gauge_forward_backward {a k q o : Nat} (P E : Mat a a R) (C : Mat k a R)
    (hEP : mul E P = one a) (z : GaugeCoordinates a k q o R) :
    gaugeForward P E C (gaugeBackward P E C z) = z := by
  cases z with
  | mk X S B D =>
    simp only [gaugeForward, gaugeBackward]
    rw ←[ mul_assoc E P X, hEP, one_mul, add_sub_cancel,
      mul_assoc (sub B (mul D C)) E P, hEP, mul_one, sub_add_cancel]
```

Theorem 14.20 (The gauge preserves the actual rectangular product)**.**

Corresponding Lean declaration(s): `077.MatrixCore.gauge_product_identity`.

Assume $PE = I_a$. Put $(X, S, \bar{B}, \bar{D}) = F_{P,E,C}(U, V, B, D)$. Then

$$[B \mid D]\begin{pmatrix} P & U \\ C & V \end{pmatrix} = [\bar{B} \mid \bar{D}]\begin{pmatrix} I_a & X \\ 0 & S \end{pmatrix}.$$

Proof. The two left and right output blocks on the first side are $BP + DC$ and $BU + DV$. On the second side they are $\bar{B}$ and $\bar{B}X + \bar{D}S$. The first blocks agree by definition. Expanding the second gives

$$(BP + DC)(EU) + D(V - CEU) = B(PE)U + DCEU + DV - DCEU = BU + DV.$$

All equalities use the proved rectangular associativity and distributive laws, and $PE = I_a$. Thus the two column blocks, and hence the full products, agree. This proves product preservation, *not* preservation of a Frobenius norm by a change of basis. □

```
theorem gauge_product_identity {a k q o : Nat} (P E : Mat a a R) (C : Mat k a R)
    (hPE : mul P E = one a) (z : GaugeInput a k q o R) :
    let w := gaugeForward P E C z
    mul (hcat z.b1 z.b2) (block P z.a12 C z.a22) =
      mul (hcat w.b1bar w.b2bar) (block (one a) w.x (zero k a) w.s) := by
  dsimp only
  simp only [block, mul_hcat, hcat_vcat, mul_one, mul_zero]
  have he : mul P (mul E z.a12) = z.a12 := by
    rw ←[ mul_assoc, hPE, one_mul]
  have hl : add (add (mul z.b1 P) (mul z.b2 C)) (zero o a) =
```

```
      add (mul z.b1 P) (mul z.b2 C) := by
    funext i j; simp only [add, zero]; grind
  have hr : add (mul (add (mul z.b1 P) (mul z.b2 C)) (mul E z.a12))
      (mul z.b2 (sub z.a22 (mul C (mul E z.a12)))) =
      add (mul z.b1 z.a12) (mul z.b2 z.a22) := by
    rw [add_mul, mul_sub, mul_assoc z.b1 P, he,
      mul_assoc z.b2 C]
    funext i j; simp only [add, sub]; grind
  simp only [gaugeForward]
  rw [hl, hr]
```

14.4 A separate invertible map on error generators

Definition 14.21 (A triangular generator transformation)**.**

Corresponding Lean declaration(s): `O77.MatrixCore.generatorForward`,`O77.MatrixCore.generatorBackward`.

Fix $X \in M_{a,q}(R)$ and $T, T^{-} \in M_{o,o}(R)$. On $M_{o,a}(R) \times M_{o,q}(R)$ define

$$\Gamma_{X,T}(U,V) = (U, T(V - UX)), \qquad \Delta_{X,T^{-}}(U,W) = (U, UX + T^{-}W).$$

The superscript $-$ names a supplied candidate inverse matrix, not an assertion that it is an inverse. The following theorem states precisely which identities are needed.

```
/-- A triangular change of error generators; no assertion about a loss or volume is a field. -/
def generatorForward {o a q : Nat} (X : Mat a q R) (T : Mat o o R)
    (z : Mat o a R × Mat o q R) : Mat o a R × Mat o q R :=
  (z.1, mul T (sub z.2 (mul z.1 X)))

def generatorBackward {o a q : Nat} (X : Mat a q R) (Tinv : Mat o o R)
    (z : Mat o a R × Mat o q R) : Mat o a R × Mat o q R :=
  (z.1, add (mul z.1 X) (mul Tinv z.2))
```

Lemma 14.22 (Both generator round trips)**.**

Corresponding Lean declaration(s): `O77.MatrixCore.generator_backward_forward`,`O77.MatrixCore.generator_forward_backward`.

If $T^{-}T = I_o$, then $\Delta_{X,T^{-}}\Gamma_{X,T}$ is the identity. If $TT^{-} = I_o$, then $\Gamma_{X,T}\Delta_{X,T^{-}}$ is the identity.

Proof. The first coordinate is unchanged in either direction. The second coordinate of the first composite is $UX + T^{-}T(V - UX) = UX + (V - UX) = V$. For the other order it is $T((UX + T^{-}W) - UX) = TT^{-}W = W$. Associativity justifies the products, and the additive group laws justify both cancellations. This is an invertible transformation of the *error vector*, not of the model parameter space. In the analytic application its coefficients X, T, T^{-} vary with the parameter; joint regularity and local bounds are proved in the analytic module before applying a measure comparison. □

```
theorem generator_backward_forward {o a q : Nat} (X : Mat a q R) (T Tinv : Mat o o R)
    (h : mul Tinv T = one o) (z : Mat o a R × Mat o q R) :
    generatorBackward X Tinv (generatorForward X T z) = z := by
  cases z with
  | mk U V =>
    simp only [generatorForward, generatorBackward]
    rw ←[ mul_assoc, h, one_mul]
    congr 1
    funext i j; simp only [add, sub]; grind
```

```
theorem generator_forward_backward {o a q : Nat} (X : Mat a q R) (T Tinv : Mat o o R)
    (h : mul T Tinv = one o) (z : Mat o a R × Mat o q R) :
    generatorForward X T (generatorBackward X Tinv z) = z := by
  cases z with
  | mk U V =>
    simp only [generatorForward, generatorBackward]
    have hh : sub (add (mul U X) (mul Tinv V)) (mul U X) = mul Tinv V := by
      funext i j; simp only [add, sub]; grind
    rw [hh, ← mul_assoc, h, one_mul]
```

Lemma 14.23 (The retained-factor shear is invertible)**.**

Corresponding Lean declaration(s): `O77.MatrixCore.shear`,`O77.MatrixCore.unshear`,`O77.MatrixCore.unshear_shear`,`O77.MatrixCore.shear_unshear`.

For every $Y \in M_{b,h}(R)$ and $Z \in M_{h,q}(R)$ the map $T \mapsto T + YZ$ on $M_{b,q}(R)$ has inverse $T' \mapsto T' - YZ$. Adjoining Y, Z unchanged gives mutually inverse maps on the three-factor parameter tuple.

Proof. At fixed Y, Z, the two composites are $(T + YZ) - YZ = T$ and $(T' - YZ) + YZ = T'$, by Lemma 14.11. When Y, Z are retained as coordinates the same equalities apply at their unchanged values. Replacing the pair (Y, Z) by its product is not part of this map and is not asserted invertible. □

```
/-- Shear in the regular-coordinate matrix, with the two residual factors retained. -/
def shear {b h q : Nat} (Y : Mat b h R) (Z : Mat h q R) (T : Mat b q R) :=
  add T (mul Y Z)
def unshear {b h q : Nat} (Y : Mat b h R) (Z : Mat h q R) (T : Mat b q R) :=
  sub T (mul Y Z)

theorem unshear_shear {b h q : Nat} (Y : Mat b h R) (Z : Mat h q R) (T : Mat b q R) :
    unshear Y Z (shear Y Z T) = T := add_sub_cancel T (mul Y Z)

theorem shear_unshear {b h q : Nat} (Y : Mat b h R) (Z : Mat h q R) (T : Mat b q R) :
    shear Y Z (unshear Y Z T) = T := sub_add_cancel T (mul Y Z)

end O77.MatrixCore
```

14.5 Row normalization and the residual generator

Definition 14.24 (The row normalizer and its candidate inverse)**.**

Corresponding Lean declaration(s): `O77.MatrixCore.normalizer`,`O77.MatrixCore.denormalizer`.

For $P, E \in M_{b,b}(R)$ and $C \in M_{p,b}(R)$, put

$$N(E, C) = \begin{pmatrix} E & 0 \\ -CE & I_p \end{pmatrix}, \qquad D(P, C) = \begin{pmatrix} P & 0 \\ C & I_p \end{pmatrix}.$$

The code writes $-CE$ as $0 - CE$. The letter C is local to this normalization and denotes the bottom block of the tall pivot matrix, not the target Φ.

```
namespace O77.MatrixCore
universe u
variable {R : Type u} [Lean.Grind.Ring R]

def normalizer {b p : Nat} (E : Mat b b R) (C : Mat p b R) : Mat (b+p) (b+p) R :=
  block E (zero b p) (sub (zero p b) (mul C E)) (one p)
def denormalizer {b p : Nat} (P : Mat b b R) (C : Mat p b R) : Mat (b+p) (b+p) R :=
  block P (zero b p) C (one p)
```

Lemma 14.25 (Both normalizer inverse products)**.**

Corresponding Lean declaration(s): `O77.MatrixCore.normalizer_denormalizer`,`O77.MatrixCore.denormalizer_normalizer`.

If $EP = I_b$, then $N(E,C)D(P,C) = I_{b+p}$. If $PE = I_b$, then $D(P,C)N(E,C) = I_{b+p}$.

Proof. In the first product, block multiplication gives diagonal blocks EP and I_p, upper-right block zero and lower-left block $(-CE)P + C = -C(EP) + C = 0$. In the second product the diagonal blocks are PE and I_p, and the lower-left block is $CE - CE = 0$; its upper-right block is again zero. The respective inverse hypotheses identify the diagonal blocks. Lemma 14.16 identifies the resulting block matrix with the actual identity on $[b+p]$. These are full matrix product equalities, including $b = 0$ or $p = 0$. □

```
theorem normalizer_denormalizer {b p : Nat} (P E : Mat b b R) (C : Mat p b R)
    (hEP : mul E P = one b) :
    mul (normalizer E C) (denormalizer P C) = one (b+p) := by
  simp only [normalizer, denormalizer, block_mul_block, mul_zero, zero_mul,
    mul_one, one_mul, add_zero, zero_add, sub_mul, mul_assoc, hEP]
  have hz : add (sub (zero p b) C) C = zero p b := sub_add_cancel _ _
  rw [hz, block_diagonal_one]

theorem denormalizer_normalizer {b p : Nat} (P E : Mat b b R) (C : Mat p b R)
    (hPE : mul P E = one b) :
    mul (denormalizer P C) (normalizer E C) = one (b+p) := by
  simp only [normalizer, denormalizer, block_mul_block, mul_zero, zero_mul,
    mul_one, one_mul, add_zero, zero_add, hPE]
  have hz : add (mul C E) (sub (zero p b) (mul C E)) = zero p b := by
    funext i j; simp only [add, sub, zero]; grind
  rw [hz, block_diagonal_one]
```

Lemma 14.26 (The action of the normalizer on an arbitrary tall matrix)**.**

Corresponding Lean declaration(s): `O77.MatrixCore.normalizer_action`.

For $U \in M_{b,t}(R)$ and $V \in M_{p,t}(R)$, $N(E,C)\binom{U}{V} = \binom{EU}{V - CEU}$. No inverse hypothesis is needed.

Proof. The upper block is $EU + 0V = EU$. The lower is $(-CE)U + I_pV = V - C(EU)$, using associativity and the additive group laws. Apply the single-column-block formula of Lemma 14.14; the two entry identities prove the full tall-matrix equality. □

```
theorem normalizer_action {b p t : Nat} (E : Mat b b R) (C : Mat p b R)
    (U : Mat b t R) (V : Mat p t R) :
    mul (normalizer E C) (vcat U V) =
      vcat (mul E U) (sub V (mul C (mul E U))) := by
  simp only [normalizer, block_mul_vcat, zero_mul, one_mul,
    add_zero, sub_mul, mul_assoc]
  congr 1
  funext i j; simp only [add, sub, zero]; grind
```

Lemma 14.27 (The tall pivot becomes the standard inclusion)**.**

Corresponding Lean declaration(s): `O77.MatrixCore.normalizer_pivot`.

If $EP = I_b$, then $N(E,C)\binom{P}{C} = \binom{I_b}{0}$.

Proof. Lemma 14.26 gives $\binom{EP}{C - CEP}$. Its top block is I_b and its bottom block is $C - CI_b = 0$. Thus the entire tall matrix is the standard inclusion, not merely a matrix of rank b. □

```
theorem normalizer_pivot {b p : Nat} (P E : Mat b b R) (C : Mat p b R)
    (hEP : mul E P = one b) :
    mul (normalizer E C) (vcat P C) = vcat (one b) (zero p b) := by
  rw [normalizer_action, hEP, mul_one, sub_self]
```

Definition 14.28 (Splitting and reconstructing rows)**.**

Corresponding Lean declaration(s): `077.MatrixCore.rowsTop,077.MatrixCore.rowsBot,077.MatrixCore.rowsTop_vcat,077.MatrixCore.rowsBot_vcat,077.MatrixCore.vcat_rows`.

For $Y \in M_{m+n,k}(R)$, let Y^{top} consist of the first m rows and Y^{bot} of the next n rows. Then $(\binom{A}{B})^{\mathrm{top}} = A$, $(\binom{A}{B})^{\mathrm{bot}} = B$, and $\binom{Y^{\mathrm{top}}}{Y^{\mathrm{bot}}} = Y$. These statements concern entries only; they need no ring laws.

Proof. The first two equations are evaluation on the corresponding inclusion of row indices. For the third, every row index is in exactly one of those ranges; evaluation gives the original row in either case. The code explicitly omits the unused ring hypothesis from these three reconstruction lemmas. □

```
def rowsTop {m n k : Nat} (A : Mat (m+n) k R) : Mat m k R := fun i j => A (i.castAdd n) j
def rowsBot {m n k : Nat} (A : Mat (m+n) k R) : Mat n k R := fun i j => A (i.natAdd m) j

omit [Lean.Grind.Ring R] in
@[simp] theorem rowsTop_vcat {m n k : Nat} (A : Mat m k R) (B : Mat n k R) :
    rowsTop (vcat A B) = A := by
  funext i j; simp only [rowsTop, vcat, Fin.addCases_left]

omit [Lean.Grind.Ring R] in
@[simp] theorem rowsBot_vcat {m n k : Nat} (A : Mat m k R) (B : Mat n k R) :
    rowsBot (vcat A B) = B := by
  funext i j; simp only [rowsBot, vcat, Fin.addCases_right]

omit [Lean.Grind.Ring R] in
theorem vcat_rows {m n k : Nat} (A : Mat (m+n) k R) :
    vcat (rowsTop A) (rowsBot A) = A := by
  funext i j
  refine Fin.addCases (fun i => ?_) (fun i => ?_) i <;>
    simp only [vcat, rowsTop, rowsBot, Fin.addCases_left, Fin.addCases_right]
```

Theorem 14.29 (The exact normalized error block)**.**

Corresponding Lean declaration(s): `077.MatrixCore.normalized_generator_identity`.

Let $U \in M_{b+p,a}(R)$, $X \in M_{a,q}(R)$, $P, E \in M_{b,b}(R)$, $C \in M_{p,b}(R)$, $T \in M_{b,q}(R)$, $Y \in M_{b+p,h}(R)$, and $Z \in M_{h,q}(R)$, with $EP = I_b$. Put

$$N = N(E, C), \quad NY = \binom{Y_1}{Y_0}, \quad V = UX + \binom{P}{C}T + YZ.$$

Here Y_1, Y_0 are the actual row projections of NY. Then $N(V - UX) = \binom{T+Y_1Z}{Y_0Z}$.

Proof. Subtracting UX from V leaves $\binom{P}{C}T + YZ$ by additive cancellation. Distribute N and reassociate the products to obtain $(N\binom{P}{C})T + (NY)Z$. The pivot lemma makes the first term $\binom{T}{0}$. By the row-splitting identity, NY is exactly $\binom{Y_1}{Y_0}$; vertical concatenation and multiplication make the second term $\binom{Y_1Z}{Y_0Z}$. Adding the two tall matrices gives the stated formula. With $T' = T + Y_1Z$, the full transformed error is $[U \mid \binom{T'}{Y_0Z}]$. The normalizer inverse and Lemma 14.22 separately establish invertibility of this error-generator transformation when both inverse identities hold. No assertion about its Jacobian, norm bounds or band volume is used here. □

```
/-- Exact generator reduction, with every size and normalization premise visible. -/
theorem normalized_generator_identity {a b p h q : Nat}
    (U : Mat (b+p) a R) (X : Mat a q R)
    (P E : Mat b b R) (C : Mat p b R)
    (T : Mat b q R) (Y : Mat (b+p) h R) (Z : Mat h q R)
    (hEP : mul E P = one b) :
    let N := normalizer E C
    let Y1 := rowsTop (m := b) (n := p) (mul N Y)
    let Y0 := rowsBot (m := b) (n := p) (mul N Y)
    let V := add (add (mul U X) (mul (vcat P C) T)) (mul Y Z)
    mul N (sub V (mul U X)) = vcat (add T (mul Y1 Z)) (mul Y0 Z) := by
  dsimp only
  have hc : sub (add (add (mul U X) (mul (vcat P C) T)) (mul Y Z))
      (mul U X) = add (mul (vcat P C) T) (mul Y Z) := by
    funext i j; simp only [add, sub]; grind
  rw [hc, mul_add, ← mul_assoc, normalizer_pivot P E C hEP,
    vcat_mul, one_mul, zero_mul, ← mul_assoc]
  have hy : mul (mul (normalizer E C) Y) Z =
      vcat (mul (rowsTop (m := b) (n := p) (mul (normalizer E C) Y)) Z)
        (mul (rowsBot (m := b) (n := p) (mul (normalizer E C) Y)) Z) := by
    calc
      _ = mul (vcat (rowsTop (m := b) (n := p) (mul (normalizer E C) Y))
          (rowsBot (m := b) (n := p) (mul (normalizer E C) Y))) Z := by
        rw [vcat_rows]
      _ = _ := vcat_mul _ _ Z
  rw [hy, add_vcat, zero_add]

end O77.MatrixCore
```

14.6 Connecting the reduction to the original product expression

Lemma 14.30 (Subtraction by column blocks)**.**

Corresponding Lean declaration(s): `O77.MatrixCore.sub_hcat`,`O77.MatrixCore.sub_zero`.

For equal-shaped column blocks, $[A \mid B] - [C \mid D] = [A - C \mid B - D]$*, and for any* A *one has* $A - 0 = A$*.*

Proof. If a column lies in the first range, the first equation has entry $A_{ij} - C_{ij}$ on both sides. If it lies in the second, both entries are $B_{ij} - D_{ij}$. The two ranges exhaust the columns. For the second equation each entry is the scalar identity $A_{ij} - 0 = A_{ij}$. Thus both statements follow by extensionality. □

```
namespace O77.MatrixCore
universe u
variable {R : Type u} [Lean.Grind.Ring R]

theorem sub_hcat {m n k : Nat} (A C : Mat m n R) (B D : Mat m k R) :
    sub (hcat A B) (hcat C D) = hcat (sub A C) (sub B D) := by
  funext i j
  refine Fin.addCases (fun j => ?_) (fun j => ?_) j <;>
    simp only [sub, hcat, Fin.addCases_left, Fin.addCases_right]

@[simp] theorem sub_zero {m n : Nat} (A : Mat m n R) : sub A (zero m n) = A := by
  funext i j; simp only [sub, zero]; grind
```

Theorem 14.31 (The error expansion after the gauge is a rectangular identity)**.**

Corresponding Lean declaration(s): `O77.MatrixCore.block_error_expansion`.

Fix independent sizes $r, \alpha, \beta, h, q, o$. Take

$$J \in M_{o,r}(R), \quad U \in M_{o,r+\alpha}(R), \quad X_R \in M_{r,q}(R), \quad X_P \in M_{\alpha,q}(R),$$
$$D \in M_{o,\beta}(R), \quad S_Q \in M_{\beta,q}(R), \quad Y \in M_{o,h}(R), \quad Z \in M_{h,q}(R).$$

Define $X = \binom{X_R}{X_P}$, $C_I = [J \mid 0]$, $B_I = U + C_I$, $S = \binom{S_Q}{Z}$, $\bar{B} = [B_I \mid [D \mid Y]]$, *and* $\bar{A} = \left(\begin{smallmatrix} I_{r+\alpha} & X \\ 0 & S \end{smallmatrix}\right)$. *Then*

$$\bar{B}\bar{A} - [C_I \mid 0] = \left[U \;\middle|\; UX + [J \mid D]\begin{pmatrix} X_R \\ S_Q \end{pmatrix} + YZ \right].$$

The hidden index has size $(r+\alpha)+(\beta+h)$ *and the input index size* $(r+\alpha)+q$*; these are actual typed shapes of the products in the formula.*

Proof. Block multiplication gives the first output block B_I and the second $B_I X + [D \mid Y]\binom{S_Q}{Z}$. Subtracting the target first block leaves $B_I - C_I = U$; the target second block is zero. The second output block expands to $UX + C_I X + DS_Q + YZ$. Because $C_I = [J \mid 0]$ and $X = \binom{X_R}{X_P}$, it satisfies $C_I X = JX_R$. Furthermore $JX_R + DS_Q = [J \mid D]\binom{X_R}{S_Q}$ by the block product formula. Associating additions gives the displayed expression. Thus the formula used in Theorem 14.29 is derived from an actual multiplication of rectangular matrices; it is not merely a name assigned to a proposed normal form. In the original chart J is the canonical inclusion, $[J \mid D]$ is split into its top square pivot and bottom block, and $Z = S_Z$. The adapted-section and numbered-coordinate modules establish these coordinate choices for every real factorization with the given ranks. □

```
/-- Expansion of the actual rectangular product after the first gauge. -/
theorem block_error_expansion {r alpha beta h q o : Nat}
    (J : Mat o r R) (U : Mat o (r+alpha) R)
    (XR : Mat r q R) (XP : Mat alpha q R)
    (D : Mat o beta R) (SQ : Mat beta q R)
    (Y : Mat o h R) (Z : Mat h q R) :
    let X := vcat XR XP
    let CI := hcat J (zero o alpha)
    let BI := add U CI
    let S := vcat SQ Z
    let Bbar := hcat BI (hcat D Y)
    let Abar := block (one (r+alpha)) X (zero (beta+h) (r+alpha)) S
    sub (mul Bbar Abar) (hcat CI (zero o q)) =
      hcat U (add (add (mul U X) (mul (hcat J D) (vcat XR SQ))) (mul Y Z)) := by
  dsimp only
  simp only [block, mul_hcat, hcat_vcat, mul_one, mul_zero,
    add_zero, sub_hcat, add_sub_cancel, sub_zero, add_mul, zero_mul]
  have h : add (add (mul U (vcat XR XP)) (mul J XR))
      (add (mul D SQ) (mul Y Z)) =
      add (add (mul U (vcat XR XP)) (add (mul J XR) (mul D SQ))) (mul Y Z) := by
    funext i j; simp only [add]; grind
  rw [h]

end O77.MatrixCore
```

14.7 Nonvacuity, boundary regressions, and the implementation boundary

Lemma 14.32 (A nontrivial invertible pivot over the integers)**.**

Corresponding Lean declaration(s): `O77.MatrixCore.Regression.pivot,O77.MatrixCore.Regression.pivotInv,O77.MatrixCore.Regression.pivot_mul_inverse,O77.MatrixCore.Regression.inverse_mul_pivot`.

The matrices $P = \left(\begin{smallmatrix} 1 & 2 \\ 0 & 1 \end{smallmatrix}\right)$ *and* $E = \left(\begin{smallmatrix} 1 & -2 \\ 0 & 1 \end{smallmatrix}\right)$ *over* $\mathbb{Z}$ *satisfy* $PE = EP = I_2$*. Consequently*

both gauge round trips apply to every tuple with $a = 2, k = 3, q = 1, o = 4$ *and every retained* $C \in M_{3,2}(\mathbb{Z})$.

Proof. In PE the upper-right entry is $-2+2=0$; in EP it is $2-2=0$. Both diagonal entries are 1, and both lower-left entries are 0. These four entry computations in each order are accepted by ordinary `decide`; function extensionality yields the two matrix equalities. The already-proved round-trip theorems then apply to the indicated genuinely rectangular shapes without inspecting the other entries. The subsequent examples check a Schur-complement sign ($2-3=-1$), the changed left output block ($4+5\cdot 3=19$), a zero inner dimension, a zero-sized pivot, the normalizer value $7-3\cdot 5=-8$, and the shear inverse $5+2\cdot 3-2\cdot 3=5$. They do not replace the universal proofs above. □

```
namespace O77.MatrixCore.Regression

def pivot : Mat 2 2 Int := fun i j => if i = j then 1 else if i.val = 0 then 2 else 0
def pivotInv : Mat 2 2 Int := fun i j => if i = j then 1 else if i.val = 0 then -2 else 0

theorem pivot_mul_inverse : mul pivot pivotInv = one 2 := by
  funext i j
  exact (by decide : ∀ i j : Fin 2, mul pivot pivotInv i j = one 2 i j) i j

theorem inverse_mul_pivot : mul pivotInv pivot = one 2 := by
  funext i j
  exact (by decide : ∀ i j : Fin 2, mul pivotInv pivot i j = one 2 i j) i j

example (C : Mat 3 2 Int) (z : GaugeInput 2 3 1 4 Int) :
    gaugeBackward pivot pivotInv C (gaugeForward pivot pivotInv C z) = z :=
  gauge_backward_forward pivot pivotInv C pivot_mul_inverse z

example (C : Mat 3 2 Int) (z : GaugeCoordinates 2 3 1 4 Int) :
    gaugeForward pivot pivotInv C (gaugeBackward pivot pivotInv C z) = z :=
  gauge_forward_backward pivot pivotInv C inverse_mul_pivot z

example : (gaugeForward (one 1 : Mat 1 1 Int) (one 1) (fun _ _ => 3)
    ⟨(fun _ _ => 1, fun _ _ => 2, fun _ _ => 4, fun _ _ => ⟩5 : GaugeInput 1 1 1 1 Int)).s
      0 0 = -1 := by decide

example : (gaugeForward (one 1 : Mat 1 1 Int) (one 1) (fun _ _ => 3)
    ⟨(fun _ _ => 1, fun _ _ => 2, fun _ _ => 4, fun _ _ => ⟩5 : GaugeInput 1 1 1 1 Int)).b1bar
      0 0 = 19 := by decide

example : ∀ i : Fin 2, ∀ j : Fin 3,
    mul (zero 2 0 : Mat 2 0 Int) (zero 0 3) i j = 0 := by decide

example : ∀ i j : Fin 2,
    normalizer (one 0 : Mat 0 0 Int) (zero 2 0) i j = one 2 i j := by decide

example : mul (normalizer (one 1 : Mat 1 1 Int) (fun _ _ => 3))
    (vcat (fun _ _ => 5 : Mat 1 1 Int) (fun _ _ => 7 : Mat 1 1 Int)) 1 0 = -8 := by decide

example : unshear (fun _ _ => 2 : Mat 1 1 Int) (fun _ _ => 3 : Mat 1 1 Int)
    (shear (fun _ _ => 2 : Mat 1 1 Int) (fun _ _ => 3 : Mat 1 1 Int)
      (fun _ _ => 5 : Mat 1 1 Int)) 0 0 = 5 := by decide

end O77.MatrixCore.Regression
```

15 The canonical multiplication derivative

This section verifies the algebraic geometry visible in the first-order part of the multiplication map. It constructs the canonical depth-two factors, proves their product, computes the image of the linearization by an explicit section, splits the target matrix space into regular and residual coordinates, and proves the exact quadratic perturbation formula. The coefficient object is still an arbitrary associative unital ring R; hence every identity below is independent of real analysis. When $R = \mathbb{R}$, the algebraic map called the "canonical derivative" is the ordinary derivative of $(A, B) \mapsto BA$ at the canonical point. The analytic and adapted-coordinate modules connect these entry identities with real calculus and arbitrary fiber points.

Fix six natural numbers $r, \alpha, \beta, h, p, q$ and abbreviate

$$a = r + \alpha, \qquad b = r + \beta, \qquad I = a + q, \qquad O = b + p, \qquad H = a + (\beta + h).$$

Thus $\alpha = a - r$, $\beta = b - r$, and $h = H - a - b + r$ in the notation of the fiber problem. Rows and columns are ordered exactly as

$$\mathbb{R}^I = (\mathbb{R}^r \oplus \mathbb{R}^\alpha) \oplus \mathbb{R}^q, \quad \mathbb{R}^H = (\mathbb{R}^r \oplus \mathbb{R}^\alpha) \oplus (\mathbb{R}^\beta \oplus \mathbb{R}^h), \quad \mathbb{R}^O = (\mathbb{R}^r \oplus \mathbb{R}^\beta) \oplus \mathbb{R}^p.$$

The parenthesization is part of the type discipline. No unproved reassociation of finite index types is used in the code.

15.1 Column restrictions and compatibility with the matrix operations

Definition 15.1 (Left and right column restrictions)**.**

Corresponding Lean declaration(s): `O77.MatrixCore.colsLeft`,`O77.MatrixCore.colsRight`.

Let $S \in M_{m,n+k}(R)$. Its left and right column restrictions are the matrices $S_{\mathrm{L}} \in M_{m,n}(R)$ and $S_{\mathrm{R}} \in M_{m,k}(R)$ defined by

$$(S_{\mathrm{L}})_{ij} = S_{i,j}, \qquad (S_{\mathrm{R}})_{ij} = S_{i,n+j}.$$

The additions in the subscripts are the canonical inclusions of $[n]$ and $[k]$ into $[n + k]$; no numerical subtraction is involved.

Lemma 15.2 (Column reconstruction and functoriality)**.**

Corresponding Lean declaration(s): `O77.MatrixCore.colsLeft_hcat`,`O77.MatrixCore.colsRight_hcat`,`O77.MatrixCore.hcat_cols`,`O77.MatrixCore.add_hcat`,`O77.MatrixCore.colsLeft_zero`,`O77.MatrixCore.colsRight_zero`,`O77.MatrixCore.rowsTop_zero`,`O77.MatrixCore.rowsBot_zero`,`O77.MatrixCore.colsLeft_add`,`O77.MatrixCore.colsRight_add`,`O77.MatrixCore.rowsTop_add`,`O77.MatrixCore.rowsBot_add`,`O77.MatrixCore.rowsTop_mul`,`O77.MatrixCore.rowsBot_mul`,`O77.MatrixCore.colsLeft_mul`,`O77.MatrixCore.colsRight_mul`.

For every matrix with a consecutive row or column decomposition, restriction is inverse to concatenation and commutes with the operations for which the restricted index is external to the summation. Concretely,

$$[S_{\mathrm{L}} \mid S_{\mathrm{R}}] = S, \quad ([A \mid B])_{\mathrm{L}} = A, \quad ([A \mid B])_{\mathrm{R}} = B,$$

and the analogous statements hold for the top and bottom row restrictions. Restrictions preserve zero and addition. Moreover, whenever the products are defined,

$$(XY)_{\mathrm{top}} = X_{\mathrm{top}}Y, \quad (XY)_{\mathrm{bot}} = X_{\mathrm{bot}}Y, \quad (XY)_{\mathrm{L}} = XY_{\mathrm{L}}, \quad (XY)_{\mathrm{R}} = XY_{\mathrm{R}}.$$

Finally, addition distributes over horizontal concatenation: $[A \mid B] + [C \mid D] = [A + C \mid B + D]$.

Proof. All assertions are equalities of functions. For reconstruction, fix a row i and a column $j \in [n+k]$. The eliminator for the disjoint decomposition $[n+k] = [n] \sqcup [k]$ gives exactly two cases. If j comes from $[n]$, horizontal concatenation selects the left matrix and the definition of S_{L} returns S_{ij}. If j comes from $[k]$, the right definition returns the same entry. The two one-sided round trips are the same computation without the final case split.

Zero and addition are entrywise, so restriction commutes with them by reflexivity after the definitions are unfolded. For row restriction of a product, fixing a selected row merely replaces the first row index in $(XY)_{ij} = \sum_t X_{it} Y_{tj}$; the finite summation is unchanged. Column restriction is identical with the roles of rows and columns reversed: fixing a selected output column merely replaces j in the second factor. Thus each multiplication identity is entrywise reflexive. The concatenation identity for addition follows from the same two column cases. These proofs use no commutativity of multiplication. □

```
namespace O77.MatrixCore
universe u
variable {R : Type u} [Lean.Grind.Ring R]

/-- Left and right column restrictions for a matrix with a concatenated column index. -/
def colsLeft {m n k : Nat} (A : Mat m (n+k) R) : Mat m n R :=
  fun i j => A i (j.castAdd k)

def colsRight {m n k : Nat} (A : Mat m (n+k) R) : Mat m k R :=
  fun i j => A i (j.natAdd n)

omit [Lean.Grind.Ring R] in
@[simp] theorem colsLeft_hcat {m n k : Nat} (A : Mat m n R) (B : Mat m k R) :
    colsLeft (hcat A B) = A := by
  funext i j
  simp only [colsLeft, hcat, Fin.addCases_left]

omit [Lean.Grind.Ring R] in
@[simp] theorem colsRight_hcat {m n k : Nat} (A : Mat m n R) (B : Mat m k R) :
    colsRight (hcat A B) = B := by
  funext i j
  simp only [colsRight, hcat, Fin.addCases_right]

omit [Lean.Grind.Ring R] in
theorem hcat_cols {m n k : Nat} (A : Mat m (n+k) R) :
    hcat (colsLeft A) (colsRight A) = A := by
  funext i j
  refine Fin.addCases (fun j => ?_) (fun j => ?_) j <;>
    simp only [hcat, colsLeft, colsRight, Fin.addCases_left, Fin.addCases_right]

theorem add_hcat {m n k : Nat} (A C : Mat m n R) (B D : Mat m k R) :
    add (hcat A B) (hcat C D) = hcat (add A C) (add B D) := by
  funext i j
  refine Fin.addCases (fun j => ?_) (fun j => ?_) j <;>
    simp only [add, hcat, Fin.addCases_left, Fin.addCases_right]

@[simp] theorem colsLeft_zero {m n k : Nat} :
    colsLeft (zero m (n+k) : Mat m (n+k) R) = zero m n := by
  funext i j
  rfl

@[simp] theorem colsRight_zero {m n k : Nat} :
    colsRight (zero m (n+k) : Mat m (n+k) R) = zero m k := by
  funext i j
  rfl
```

```
@[simp] theorem rowsTop_zero {m n k : Nat} :
    rowsTop (zero (m+n) k : Mat (m+n) k R) = zero m k := by
  funext i j
  rfl

@[simp] theorem rowsBot_zero {m n k : Nat} :
    rowsBot (zero (m+n) k : Mat (m+n) k R) = zero n k := by
  funext i j
  rfl

@[simp] theorem colsLeft_add {m n k : Nat} (A B : Mat m (n+k) R) :
    colsLeft (add A B) = add (colsLeft A) (colsLeft B) := by
  funext i j
  rfl

@[simp] theorem colsRight_add {m n k : Nat} (A B : Mat m (n+k) R) :
    colsRight (add A B) = add (colsRight A) (colsRight B) := by
  funext i j
  rfl

@[simp] theorem rowsTop_add {m n k : Nat} (A B : Mat (m+n) k R) :
    rowsTop (add A B) = add (rowsTop A) (rowsTop B) := by
  funext i j
  rfl

@[simp] theorem rowsBot_add {m n k : Nat} (A B : Mat (m+n) k R) :
    rowsBot (add A B) = add (rowsBot A) (rowsBot B) := by
  funext i j
  rfl

theorem rowsTop_mul {m n k l : Nat} (A : Mat (m+n) k R) (B : Mat k l R) :
    rowsTop (mul A B) = mul (rowsTop A) B := by
  funext i j
  rfl

theorem rowsBot_mul {m n k l : Nat} (A : Mat (m+n) k R) (B : Mat k l R) :
    rowsBot (mul A B) = mul (rowsBot A) B := by
  funext i j
  rfl

theorem colsLeft_mul {m n k l : Nat} (A : Mat m n R) (B : Mat n (k+l) R) :
    colsLeft (mul A B) = mul A (colsLeft B) := by
  funext i j
  rfl

theorem colsRight_mul {m n k l : Nat} (A : Mat m n R) (B : Mat n (k+l) R) :
    colsRight (mul A B) = mul A (colsRight B) := by
  funext i j
  rfl
```

15.2 The canonical exact factorization

Definition 15.3 (The standard pivot and the canonical factors)**.**

Corresponding Lean declaration(s): `O77.MatrixCore.canonicalPivot`,`O77.MatrixCore.canonicalPivot_mul`,`O77.MatrixCore.canonicalJ`,`O77.MatrixCore.canonicalD`,`O77.MatrixCore.canonicalJ_D_reconstruct`,`O77.MatrixCore.canonicalA`,`O77.MatrixCore.canonicalCI`,`O77.MatrixCore.canonicalB`,`O77.MatrixCore.canonicalTarget`.

Put

$$P_{b,p} = \begin{pmatrix} I_b \\ 0_{p\times b} \end{pmatrix} \in M_{b+p,b}(R).$$

Split its $b = r + \beta$ columns as $P_{b,p} = [J \mid D]$, where $J \in M_{b+p,r}(R)$ and $D \in M_{b+p,\beta}(R)$. Thus, in ordinary block notation,

$$J = \begin{pmatrix} I_r \\ 0_{\beta\times r} \\ 0_{p\times r} \end{pmatrix}, \qquad D = \begin{pmatrix} 0_{r\times\beta} \\ I_\beta \\ 0_{p\times\beta} \end{pmatrix}.$$

Define

$$C_I = [J \mid 0_{O\times\alpha}], \qquad A_0 = \begin{pmatrix} I_a & 0_{a\times q} \\ 0_{(\beta+h)\times a} & 0_{(\beta+h)\times q} \end{pmatrix},$$
$$B_0 = [C_I \mid D \mid 0_{O\times h}], \qquad C_0 = [C_I \mid 0_{O\times q}].$$

Over a field, A_0, B_0, C_0 have ranks a, b, r, respectively. The present checked statement does not use a rank function; it records the matrices and their exact product.

```
/-- The standard full-column-rank pivot `[I_b; 0]`. -/
def canonicalPivot (b p : Nat) : Mat (b+p) b R :=
  vcat (one b) (zero p b)


theorem canonicalPivot_mul (b p q : Nat) (T : Mat b q R) :
    mul (canonicalPivot (R := R) b p) T = vcat T (zero p q) := by
  simp only [canonicalPivot, vcat_mul, one_mul, zero_mul]

/-- The first `r` columns of the standard pivot, where `b = r + beta`. -/
def canonicalJ (r beta p : Nat) : Mat ((r+beta)+p) r R :=
  colsLeft (n := r) (k := beta) (canonicalPivot (R := R) (r+beta) p)

/-- The last `beta` columns of the standard pivot, where `b = r + beta`. -/
def canonicalD (r beta p : Nat) : Mat ((r+beta)+p) beta R :=
  colsRight (n := r) (k := beta) (canonicalPivot (R := R) (r+beta) p)

theorem canonicalJ_D_reconstruct (r beta p : Nat) :
    hcat (canonicalJ (R := R) r beta p) (canonicalD (R := R) r beta p) =
      canonicalPivot (R := R) (r+beta) p := by
  exact hcat_cols _

/-- The active first factor at the canonical base point. -/
def canonicalA (r alpha beta h q : Nat) :
    Mat ((r+alpha)+(beta+h)) ((r+alpha)+q) R :=
  block (one (r+alpha)) (zero (r+alpha) q)
    (zero (beta+h) (r+alpha)) (zero (beta+h) q)

/-- The shared-output block `C_I = [J | 0]`. -/
def canonicalCI (r alpha beta p : Nat) : Mat ((r+beta)+p) (r+alpha) R :=
  hcat (canonicalJ (R := R) r beta p) (zero ((r+beta)+p) alpha)

/-- The active second factor at the canonical base point. -/
def canonicalB (r alpha beta h p : Nat) :
    Mat ((r+beta)+p) ((r+alpha)+(beta+h)) R :=
  hcat (canonicalCI (R := R) r alpha beta p)
    (hcat (canonicalD (R := R) r beta p) (zero ((r+beta)+p) h))

/-- The canonical rank-`r` target. -/
def canonicalTarget (r alpha beta p q : Nat) :
    Mat ((r+beta)+p) ((r+alpha)+q) R :=
  hcat (canonicalCI (R := R) r alpha beta p) (zero ((r+beta)+p) q)
```

Theorem 15.4 (The canonical factors multiply to the canonical target)**.**

Corresponding Lean declaration(s): `O77.MatrixCore.canonical_product`.

For all $r, \alpha, \beta, h, p, q$ and every associative unital ring R,

$$B_0 A_0 = C_0.$$

This includes every zero-dimensional block case.

Proof. Regard A_0 as a 2×2 block matrix for the hidden-row split $a \mid (\beta + h)$ and the input-column split $a \mid q$. Regard B_0 as the horizontal concatenation of its first a columns C_I and its remaining $\beta + h$ columns $[D \mid 0_h]$. Block multiplication gives

$$B_0 A_0 = \left[C_I I_a + [D \mid 0_h] 0_{(\beta+h)\times a} \;\middle|\; C_I 0_{a\times q} + [D \mid 0_h] 0_{(\beta+h)\times q} \right] = [C_I \mid 0] = C_0.$$

Every equality uses the unit and zero laws already proved for the entry representation. No inverse or determinant hypothesis occurs. □

```
theorem canonical_product (r alpha beta h p q : Nat) :
    mul (canonicalB (R := R) r alpha beta h p)
      (canonicalA (R := R) r alpha beta h q) =
      canonicalTarget (R := R) r alpha beta p q := by
  simp only [canonicalB, canonicalA, canonicalTarget, block, mul_hcat,
    hcat_vcat, mul_one, mul_zero, add_zero]
```

15.3 A section of the active output coordinates

Definition 15.5 (Canonical active section)**.**

Corresponding Lean declaration(s): `O77.MatrixCore.canonicalSectionTop`,`O77.MatrixCore.canonicalSectionBot`,`O77.MatrixCore.canonicalSection`.

Define $Q \in M_{H,b}(R)$ by vertical concatenation $Q = \binom{Q_{\mathrm{top}}}{Q_{\mathrm{bot}}}$, where

$$Q_{\mathrm{top}} = \begin{pmatrix} I_r & 0_{r\times\beta} \\ 0_{\alpha\times r} & 0_{\alpha\times\beta} \end{pmatrix} \in M_{a,b}(R), \qquad Q_{\mathrm{bot}} = \begin{pmatrix} 0_{\beta\times r} & I_\beta \\ 0_{h\times r} & 0_{h\times\beta} \end{pmatrix} \in M_{\beta+h,b}(R).$$

The map Q sends the first r active output coordinates to the shared hidden coordinates and the next β active output coordinates to the β hidden summand.

```
/-- A right inverse to the active columns of the canonical second factor. -/
def canonicalSectionTop (r alpha beta : Nat) : Mat (r+alpha) (r+beta) R :=
  block (one r) (zero r beta) (zero alpha r) (zero alpha beta)

def canonicalSectionBot (r beta h : Nat) : Mat (beta+h) (r+beta) R :=
  block (zero beta r) (one beta) (zero h r) (zero h beta)

def canonicalSection (r alpha beta h : Nat) :
    Mat ((r+alpha)+(beta+h)) (r+beta) R :=
  vcat (canonicalSectionTop (R := R) r alpha beta)
    (canonicalSectionBot (R := R) r beta h)
```

Lemma 15.6 (The active section and the two vanishing boundary blocks)**.**

Corresponding Lean declaration(s): `O77.MatrixCore.canonicalB_section`,`O77.MatrixCore.rowsBot_canonicalB`,`O77.MatrixCore.colsRight_canonicalA`.

The canonical matrices satisfy

$$B_0 Q = P_{b,p}, \qquad (B_0)_{\mathrm{bot}} = 0_{p\times H}, \qquad (A_0)_{\mathrm{R}} = 0_{H\times q}.$$

Here "bottom" means the last p output rows and "right" the last q input columns. The first equality is a section only onto the active top b output coordinates; it does not assert that B_0 has a two-sided inverse.

Proof. Write $B_0 = [C_I \mid D \mid 0_h]$ and $Q = \binom{Q_{\mathrm{top}}}{Q_{\mathrm{bot}}}$. The first product is

$$B_0 Q = C_I Q_{\mathrm{top}} + [D \mid 0_h] Q_{\mathrm{bot}}.$$

The definition of Q_{top} gives $C_I Q_{\mathrm{top}} = [J \mid 0_\beta]$; the definition of Q_{bot} gives $[D \mid 0_h] Q_{\mathrm{bot}} = [0_r \mid D]$. Adding the two horizontal blocks yields $[J \mid D] = P_{b,p}$. The other two statements follow directly from the explicit zero rows of $P_{b,p}$ and the explicit zero right column block of A_0. The Lean proof performs the same block multiplication and then reduces the remaining entries to ring zero laws. □

```
theorem canonicalB_section (r alpha beta h p : Nat) :
    mul (canonicalB (R := R) r alpha beta h p)
      (canonicalSection (R := R) r alpha beta h) =
      canonicalPivot (R := R) (r+beta) p := by
  simp only [canonicalB, canonicalSection, canonicalCI,
    canonicalSectionTop, canonicalSectionBot, block, hcat_vcat, mul_hcat,
    mul_one, mul_zero, add_zero]
  rw [add_hcat]
  have hJ : add (canonicalJ (R := R) r beta p) (zero ((r+beta)+p) r) =
      canonicalJ (R := R) r beta p := by
    funext i j; simp only [add, zero]; grind
  have hD : add (zero ((r+beta)+p) beta) (canonicalD (R := R) r beta p) =
      canonicalD (R := R) r beta p := by
    funext i j; simp only [add, zero]; grind
  rw [hJ, hD, canonicalJ_D_reconstruct]

@[simp] theorem rowsBot_canonicalB (r alpha beta h p : Nat) :
    rowsBot (m := r+beta) (n := p)
      (canonicalB (R := R) r alpha beta h p) =
      zero p ((r+alpha)+(beta+h)) := by
  funext i j
  refine Fin.addCases (fun j => ?_) (fun j => ?_) j
  · refine Fin.addCases (fun j => ?_) (fun j => ?_) j <;>
      simp only [canonicalB, canonicalCI, canonicalJ, canonicalD,
        canonicalPivot, rowsBot, colsLeft, hcat, vcat,
        Fin.addCases_left, Fin.addCases_right, zero]
  · refine Fin.addCases (fun j => ?_) (fun j => ?_) j <;>
      simp only [canonicalB, canonicalCI, canonicalJ, canonicalD,
        canonicalPivot, rowsBot, colsRight, hcat, vcat,
        Fin.addCases_left, Fin.addCases_right, zero]

@[simp] theorem colsRight_canonicalA (r alpha beta h q : Nat) :
    colsRight (n := r+alpha) (k := q)
      (canonicalA (R := R) r alpha beta h q) =
      zero ((r+alpha)+(beta+h)) q := by
  funext i j
  refine Fin.addCases (fun i => ?_) (fun i => ?_) i <;>
    simp only [canonicalA, block, colsRight, hcat, vcat,
      Fin.addCases_left, Fin.addCases_right, zero]
```

15.4 The residual projection and the linearization

Definition 15.7 (Residual block and algebraic linearization)**.**

Corresponding Lean declaration(s): `O77.MatrixCore.bottomRight`,`O77.MatrixCore.bottomRight_add`,`O77.MatrixCore.canonicalDerivative`.

For $S \in M_{b+p,a+q}(R)$ define its residual block

$$\rho(S) = S_{\mathrm{bot,R}} \in M_{p,q}(R).$$

For perturbations $\dot{A} \in M_{H,I}(R)$ and $\dot{B} \in M_{O,H}(R)$ define

$$\mathcal{D}_0(\dot{A}, \dot{B}) = B_0\dot{A} + \dot{B}A_0.$$

When $R = \mathbb{R}$, this is the derivative of the multiplication map at (A_0, B_0). The code proves additivity of ρ as a definitional equality.

```
/-- The bottom-right `p × q` residual block. -/
def bottomRight {m n p q : Nat} (S : Mat (m+p) (n+q) R) : Mat p q R :=
  rowsBot (m := m) (n := p) (colsRight (n := n) (k := q) S)


theorem bottomRight_add {m n p q : Nat} (A B : Mat (m+p) (n+q) R) :
    bottomRight (m := m) (n := n) (add A B) =
      add (bottomRight (m := m) (n := n) A)
        (bottomRight (m := m) (n := n) B) := by
  rfl

/-- Derivative of matrix multiplication at the canonical base point. -/
def canonicalDerivative (r alpha beta h p q : Nat)
    (dA : Mat ((r+alpha)+(beta+h)) ((r+alpha)+q) R)
    (dB : Mat ((r+beta)+p) ((r+alpha)+(beta+h)) R) :
    Mat ((r+beta)+p) ((r+alpha)+q) R :=
  add (mul (canonicalB (R := R) r alpha beta h p) dA)
    (mul dB (canonicalA (R := R) r alpha beta h q))
```

Lemma 15.8 (Every linearized error has zero residual block)**.**

Corresponding Lean declaration(s): `O77.MatrixCore.canonicalDerivative_bottomRight_zero`.

For all perturbations $(\dot{A}, \dot{B})$,

$$\rho\big(\mathcal{D}_0(\dot{A}, \dot{B})\big) = 0.$$

Proof. For the first summand, selecting the bottom p rows after multiplication gives $(B_0)_{\mathrm{bot}}\dot{A} = 0$ by Lemma 15.6. For the second, selecting the right q columns gives $\dot{B}(A_0)_{\mathrm{R}} = 0$. Applying the remaining row or column restriction and then adding therefore gives zero. The order of restriction does not matter here because both select the same bottom-right entries. The proof does not use cancellation or a rank count. □

```
theorem canonicalDerivative_bottomRight_zero (r alpha beta h p q : Nat)
    (dA : Mat ((r+alpha)+(beta+h)) ((r+alpha)+q) R)
    (dB : Mat ((r+beta)+p) ((r+alpha)+(beta+h)) R) :
    bottomRight (m := r+beta) (n := r+alpha)
      (canonicalDerivative (R := R) r alpha beta h p q dA dB) =
      zero p q := by
  simp only [bottomRight, canonicalDerivative, colsRight_add,
    colsRight_mul, rowsBot_mul, rowsBot_canonicalB, colsRight_canonicalA,
    zero_mul, mul_zero, add_zero]
```

Definition 15.9 (A preimage of a non-residual matrix)**.**

Corresponding Lean declaration(s): `O77.MatrixCore.derivativePreimageA`,`O77.MatrixCore.derivativePreimageB`.

Let $S \in M_{b+p,a+q}(R)$. Write

$$L = S_{\mathrm{L}} \in M_{b+p,a}(R), \qquad T = (S_{\mathrm{R}})_{\mathrm{top}} \in M_{b,q}(R).$$

Define

$$\dot{A}_S = [0_{H\times a} \mid QT], \qquad \dot{B}_S = [L \mid 0_{O\times(\beta+h)}].$$

These formulas are total for every S; the condition $\rho(S) = 0$ is needed only to prove that they map back to S.

```
/-- A concrete preimage for a matrix whose residual block vanishes. -/
def derivativePreimageA (r alpha beta h p q : Nat)
    (S : Mat ((r+beta)+p) ((r+alpha)+q) R) :
    Mat ((r+alpha)+(beta+h)) ((r+alpha)+q) R :=
  let T := rowsTop (m := r+beta) (n := p)
    (colsRight (n := r+alpha) (k := q) S)
  hcat (zero ((r+alpha)+(beta+h)) (r+alpha))
    (mul (canonicalSection (R := R) r alpha beta h) T)

def derivativePreimageB (r alpha beta h p q : Nat)
    (S : Mat ((r+beta)+p) ((r+alpha)+q) R) :
    Mat ((r+beta)+p) ((r+alpha)+(beta+h)) R :=
  hcat (colsLeft (n := r+alpha) (k := q) S)
    (zero ((r+beta)+p) (beta+h))
```

Theorem 15.10 (Exact image of the multiplication derivative at the canonical point)**.**

Corresponding Lean declaration(s): `O77.MatrixCore.canonicalB_mul_preimageA`,`O77.MatrixCore.preimageB_mul_canonicalA`,`O77.MatrixCore.canonicalDerivative_preimage`,`O77.MatrixCore.exists_canonicalDerivative_eq_iff`.

For every $S \in M_{b+p,a+q}(R)$,

$$\big(\exists \dot{A}, \dot{B} :\ \mathcal{D}_0(\dot{A}, \dot{B}) = S\big) \quad \Longleftrightarrow \quad \rho(S) = 0.$$

More precisely, if $\rho(S) = 0$, the pair $(\dot{A}_S, \dot{B}_S)$ of Definition 15.9 is an explicit preimage.

Proof. The forward implication is Lemma 15.8. For the converse, assume $\rho(S) = 0$. Since S_{R} has top block T and bottom block $\rho(S)$, row reconstruction gives $S_{\mathrm{R}} = \binom{T}{0}$. By the section identity $B_0 Q = P_{b,p}$,

$$B_0 \dot{A}_S = [0 \mid B_0 QT] = [0 \mid P_{b,p} T] = [0 \mid \begin{pmatrix} T \\ 0 \end{pmatrix}].$$

By the form of A_0,

$$\dot{B}_S A_0 = [L \mid 0].$$

Adding gives $[L \mid \binom{T}{0}] = [S_{\mathrm{L}} \mid S_{\mathrm{R}}] = S$. Each reconstruction is an equality of matrices, including when one of the dimensions is zero. The Lean theorem exposes the existential quantifiers and does not package surjectivity as an uninspected structure field. □

```
theorem canonicalB_mul_preimageA (r alpha beta h p q : Nat)
    (S : Mat ((r+beta)+p) ((r+alpha)+q) R) :
    mul (canonicalB (R := R) r alpha beta h p)
      (derivativePreimageA (R := R) r alpha beta h p q S) =
      hcat (zero ((r+beta)+p) (r+alpha))
        (vcat (rowsTop (m := r+beta) (n := p)
          (colsRight (n := r+alpha) (k := q) S)) (zero p q)) := by
  simp only [derivativePreimageA, mul_hcat, mul_zero, ← mul_assoc,
    canonicalB_section, canonicalPivot_mul]

theorem preimageB_mul_canonicalA (r alpha beta h p q : Nat)
    (S : Mat ((r+beta)+p) ((r+alpha)+q) R) :
    mul (derivativePreimageB (R := R) r alpha beta h p q S)
      (canonicalA (R := R) r alpha beta h q) =
      hcat (colsLeft (n := r+alpha) (k := q) S)
        (zero ((r+beta)+p) q) := by
  simp only [derivativePreimageB, canonicalA, block, hcat_vcat, mul_hcat,
    mul_one, mul_zero, add_zero]

theorem canonicalDerivative_preimage (r alpha beta h p q : Nat)
    (S : Mat ((r+beta)+p) ((r+alpha)+q) R)
    (hS : bottomRight (m := r+beta) (n := r+alpha) S = zero p q) :
    canonicalDerivative (R := R) r alpha beta h p q
      (derivativePreimageA (R := R) r alpha beta h p q S)
      (derivativePreimageB (R := R) r alpha beta h p q S) = S := by
  rw [canonicalDerivative, canonicalB_mul_preimageA,
    preimageB_mul_canonicalA, add_hcat]
  simp only [add_zero, zero_add]
  have hright : colsRight (n := r+alpha) (k := q) S =
      vcat (rowsTop (m := r+beta) (n := p)
        (colsRight (n := r+alpha) (k := q) S)) (zero p q) := by
    calc
      _ = vcat (rowsTop (m := r+beta) (n := p)
            (colsRight (n := r+alpha) (k := q) S))
          (rowsBot (m := r+beta) (n := p)
            (colsRight (n := r+alpha) (k := q) S)) := by
              symm
              exact vcat_rows _
      _ = _ := by rw ←[ hS]; rfl
  rw ←[ hright, hcat_cols]

/-- Constructive characterization of the derivative image at the canonical point. -/
theorem exists_canonicalDerivative_eq_iff (r alpha beta h p q : Nat)
    (S : Mat ((r+beta)+p) ((r+alpha)+q) R) :
    ∃( dA dB, canonicalDerivative (R := R) r alpha beta h p q dA dB = S) ↔
      bottomRight (m := r+beta) (n := r+alpha) S = zero p q := by
  constructor
  · rintro ⟨dA, dB, ⟩rfl
    exact canonicalDerivative_bottomRight_zero r alpha beta h p q dA dB
  · intro hS
    exact ⟨derivativePreimageA (R := R) r alpha beta h p q S,
      derivativePreimageB (R := R) r alpha beta h p q S,
      canonicalDerivative_preimage r alpha beta h p q S ⟩hS
```

15.5 An explicit regular–residual decomposition of the target space

Definition 15.11 (Non-residual coordinates and residual injection)**.**

Corresponding Lean declaration(s): `O77.MatrixCore.NonResidualCoordinates`,`O77.MatrixCore.assembleNonResidual`,`O77.MatrixCore.splitNonResidual`,`O77.MatrixCore.injectResidual`.

Let

$$\mathcal{N}_{a,b,p,q} = M_{b+p,a}(R) \times M_{b,q}(R).$$

For $(L,T) \in \mathcal{N}_{a,b,p,q}$ *set*

$$\operatorname{asm}(L,T) = [L \mid \begin{pmatrix} T \\ 0_{p\times q} \end{pmatrix}].$$

For a full matrix S*, set* $\operatorname{split}(S) = (S_{\mathrm{L}}, (S_{\mathrm{R}})_{\mathrm{top}})$*. For a residual matrix* $U \in M_{p,q}(R)$ *define*

$$\iota(U) = [0_{(b+p)\times a} \mid \begin{pmatrix} 0_{b\times q} \\ U \end{pmatrix}].$$

Thus asm *fills all entries except the residual bottom-right block, while* ι *fills only that block.*

```
/-- Coordinates for the complement of the bottom-right residual block. -/
structure NonResidualCoordinates (a b p q : Nat) (R : Type u) where
  left : Mat (b+p) a R
  topRight : Mat b q R

def assembleNonResidual {a b p q : Nat}
    (z : NonResidualCoordinates a b p q R) : Mat (b+p) (a+q) R :=
  hcat z.left (vcat z.topRight (zero p q))

def splitNonResidual {a b p q : Nat} (S : Mat (b+p) (a+q) R) :
    NonResidualCoordinates a b p q R where
  left := colsLeft (n := a) (k := q) S
  topRight := rowsTop (m := b) (n := p) (colsRight (n := a) (k := q) S)
```

Theorem 15.12 (Direct coordinate decomposition of every target matrix)**.**

Corresponding Lean declaration(s): `O77.MatrixCore.split_assemble_nonResidual`,`O77.MatrixCore.bottomRight_assemble_nonResidual`,`O77.MatrixCore.bottomRight_injectResidual`,`O77.MatrixCore.target_decomposition`,`O77.MatrixCore.nonResidual_residual_decomposition_unique`,`O77.MatrixCore.assemble_split_of_bottomRight_zero`,`O77.MatrixCore.exists_assemble_nonResidual_eq_iff`.

The maps above satisfy

$$\operatorname{split}(\operatorname{asm}(L,T)) = (L,T), \qquad \rho(\operatorname{asm}(L,T)) = 0, \qquad \rho(\iota(U)) = U.$$

Every $S \in M_{b+p,a+q}(R)$ *has the exact decomposition*

$$\boxed{S = \operatorname{asm}(\operatorname{split} S) + \iota(\rho(S)).}$$

This decomposition is unique: if $S = \operatorname{asm}(z) + \iota(U)$*, then* $z = \operatorname{split} S$ *and* $U = \rho(S)$*. Consequently* S *lies in the image of* asm *if and only if* $\rho(S) = 0$*.*

Proof. The first three identities follow by selecting the indicated row and column blocks. For the boxed identity, write S as the four blocks

$$S = \begin{pmatrix} S_{11} & S_{12} \\ S_{21} & S_{22} \end{pmatrix} \quad \text{with row split } b \mid p \text{ and column split } a \mid q.$$

Then

$$\operatorname{asm}(\operatorname{split} S) = \begin{pmatrix} S_{11} & S_{12} \\ S_{21} & 0 \end{pmatrix}, \qquad \iota(\rho(S)) = \begin{pmatrix} 0 & 0 \\ 0 & S_{22} \end{pmatrix};$$

their sum is S entry by entry. For uniqueness, apply the left-column restriction and the top-right restriction to an equality $\mathrm{asm}(z) + \iota(U) = S$; the residual injection vanishes under both restrictions, so the two fields of z are forced to equal those of split S. Applying ρ to the same equality forces $U = \rho(S)$. If $\rho(S) = 0$, the residual summand vanishes and the first term alone equals S; conversely every assembled matrix has zero residual block. The implementation proves the boxed identity by the three exhaustive index cases: a left column, a top-right entry, and a bottom-right entry, and proves uniqueness by these three projection arguments. □

```
theorem split_assemble_nonResidual {a b p q : Nat}
    (z : NonResidualCoordinates a b p q R) :
    splitNonResidual (assembleNonResidual z) = z := by
  cases z
  simp only [splitNonResidual, assembleNonResidual, colsLeft_hcat,
    colsRight_hcat, rowsTop_vcat]

theorem bottomRight_assemble_nonResidual {a b p q : Nat}
    (z : NonResidualCoordinates a b p q R) :
    bottomRight (m := b) (n := a) (assembleNonResidual z) = zero p q := by
  simp only [bottomRight, assembleNonResidual, colsRight_hcat, rowsBot_vcat]

/-- Injection of the residual bottom-right block into the full target space. -/
def injectResidual {a b p q : Nat} (T : Mat p q R) : Mat (b+p) (a+q) R :=
  hcat (zero (b+p) a) (vcat (zero b q) T)

theorem bottomRight_injectResidual {a b p q : Nat} (T : Mat p q R) :
    bottomRight (m := b) (n := a) (injectResidual (a := a) (b := b) T) = T := by
  simp only [bottomRight, injectResidual, colsRight_hcat, rowsBot_vcat]

theorem target_decomposition {a b p q : Nat} (S : Mat (b+p) (a+q) R) :
    add (assembleNonResidual (splitNonResidual S))
      (injectResidual (a := a) (b := b)
        (bottomRight (m := b) (n := a) S)) = S := by
  funext i j
  refine Fin.addCases (fun j => ?_) (fun j => ?_) j
  · simp only [assembleNonResidual, splitNonResidual, injectResidual,
      bottomRight, add, hcat, colsLeft, Fin.addCases_left, zero]
    grind
  · refine Fin.addCases (fun i => ?_) (fun i => ?_) i
    · simp only [assembleNonResidual, splitNonResidual, injectResidual,
        bottomRight, add, hcat, vcat, colsRight, rowsTop,
        Fin.addCases_right, Fin.addCases_left, zero]
      grind
    · simp only [assembleNonResidual, splitNonResidual, injectResidual,
        bottomRight, add, hcat, vcat, colsRight, rowsBot,
        Fin.addCases_right, zero]
      grind

/-- Uniqueness of the non-residual plus residual coordinate decomposition. -/
theorem nonResidual_residual_decomposition_unique {a b p q : Nat}
    (S : Mat (b+p) (a+q) R)
    (z : NonResidualCoordinates a b p q R) (T : Mat p q R)
    (h : add (assembleNonResidual z)
      (injectResidual (a := a) (b := b) T) = S) :
    z = splitNonResidual S ∧
      T = bottomRight (m := b) (n := a) S := by
  constructor
  · cases z with
    | mk left topRight =>
      have hleft := congrArg (fun M : Mat (b+p) (a+q) R =>
          colsLeft (n := a) (k := q) M) h
      have hleft_eq : left = (splitNonResidual S).left := by
```

```
        simpa only [colsLeft_add, assembleNonResidual, injectResidual,
          colsLeft_hcat, splitNonResidual, add_zero] using hleft
      have htop := congrArg (fun M : Mat (b+p) (a+q) R =>
          rowsTop (m := b) (n := p) (colsRight (n := a) (k := q) M)) h
      have htop_eq : topRight = (splitNonResidual S).topRight := by
        simpa only [colsRight_add, rowsTop_add, assembleNonResidual,
          injectResidual, colsRight_hcat, rowsTop_vcat,
          splitNonResidual, add_zero] using htop
      cases hs : splitNonResidual S with
      | mk l t =>
        simp only [hs] at hleft_eq htop_eq ⊢
        cases hleft_eq
        cases htop_eq
        rfl
  · have hbot := congrArg (fun M : Mat (b+p) (a+q) R =>
      bottomRight (m := b) (n := a) M) h
    simpa only [bottomRight_add, bottomRight_assemble_nonResidual,
      bottomRight_injectResidual, zero_add] using hbot

theorem assemble_split_of_bottomRight_zero {a b p q : Nat}
    (S : Mat (b+p) (a+q) R)
    (hS : bottomRight (m := b) (n := a) S = zero p q) :
    assembleNonResidual (splitNonResidual S) = S := by
  have h := target_decomposition (R := R) S
  rw [hS] at h
  have hz : injectResidual (a := a) (b := b) (zero p q : Mat p q R) =
      zero (b+p) (a+q) := by
    funext i j
    refine Fin.addCases (fun j => ?_) (fun j => ?_) j
    · simp only [injectResidual, hcat, Fin.addCases_left, zero]
    · refine Fin.addCases (fun i => ?_) (fun i => ?_) i <;>
        simp only [injectResidual, hcat, vcat, Fin.addCases_left,
          Fin.addCases_right, zero]
  rw [hz] at h
  exact (by simpa only [add_zero] using h)

theorem exists_assemble_nonResidual_eq_iff {a b p q : Nat}
    (S : Mat (b+p) (a+q) R) :
    ∃( z : NonResidualCoordinates a b p q R, assembleNonResidual z = S) ↔
      bottomRight (m := b) (n := a) S = zero p q := by
  constructor
  · rintro ⟨z, ⟩rfl
    exact bottomRight_assemble_nonResidual z
  · intro hS
    exact ⟨splitNonResidual S, assemble_split_of_bottomRight_zero S ⟩hS
```

Corollary 15.13 (Derivative image coordinates and the regular count)**.**

Corresponding Lean declaration(s): `O77.MatrixCore.canonicalDerivative_image_iff_nonResidual`,`O77.MatrixCore.nonResidual_coordinate_count`.

The image of $\mathcal{D}_0$ is exactly the image of asm*. It is therefore coordinatized by*

$$M_{O,a}(R) \times M_{b,q}(R),$$

which has

$$Oa + bq = (b+p)a + bq = \mathrm{regularDim}(a+q, b+p, a, b)$$

scalar coordinates. Over a field, after identifying these free matrix spaces with finite free modules, this constructive parametrization yields $\operatorname{rank} \mathcal{D}_0 = Oa + b(I-a)$*. The last rank statement is the mathematical content of the range characterization; the checked code proves the*

image equivalence and the natural-number coordinate identity, not a theorem about Mathlib's rank API.

Proof. Theorem 15.10 and Theorem 15.12 both characterize their respective images by the same equation $\rho(S) = 0$, so the images coincide. The coordinate count is the sum of the numbers of entries of the two factors. Since $I = a + q$ and $O = b + p$, the displayed equality is obtained from $(a + q) - a = q$; no subtraction with an unproved inequality is used. □

```
theorem canonicalDerivative_image_iff_nonResidual (r alpha beta h p q : Nat)
    (S : Mat ((r+beta)+p) ((r+alpha)+q) R) :
    ∃( dA dB, canonicalDerivative (R := R) r alpha beta h p q dA dB = S) ↔
      ∃ z : NonResidualCoordinates (r+alpha) (r+beta) p q R,
        assembleNonResidual z = S := by
  rw [exists_canonicalDerivative_eq_iff, exists_assemble_nonResidual_eq_iff]

/-- The number of non-residual scalar entries agrees with the regular-dimension formula. -/
theorem nonResidual_coordinate_count (a b p q : Nat) :
    (b+p) * a + b * q = regularDim (a+q) (b+p) a b := by
  simp only [regularDim, Nat.add_sub_cancel_left]
```

15.6 The exact nonlinear remainder

Theorem 15.14 (Perturbation formula for matrix multiplication)**.**

Corresponding Lean declaration(s): `O77.MatrixCore.product_perturbation_expansion`,`O77.MatrixCore.canonical_error_expansion`,`O77.MatrixCore.canonical_error_bottomRight`.

Let $A_0 \in M_{H,I}(R)$, $B_0 \in M_{O,H}(R)$ and $C \in M_{O,I}(R)$ satisfy $B_0A_0 = C$. For all perturbations $\dot{A}, \dot{B}$,

$$(B_0 + \dot{B})(A_0 + \dot{A}) - C = B_0\dot{A} + \dot{B}A_0 + \dot{B}\dot{A}.$$

At the canonical point this becomes

$$(B_0 + \dot{B})(A_0 + \dot{A}) - C_0 = \mathcal{D}_0(\dot{A}, \dot{B}) + \dot{B}\dot{A}.$$

Applying the residual projection and using $\rho(\mathcal{D}_0(\dot{A}, \dot{B})) = 0$ gives the exact identity

$$\boxed{\rho\big((B_0 + \dot{B})(A_0 + \dot{A}) - C_0\big) = \rho(\dot{B}\dot{A}).}$$

Thus the residual block has no constant or linear term; its first possible contribution is bilinear. This statement is weaker than the full elimination chart: it does not yet separate the bilinear remainder into the final factors Y_0S_Z or prove a norm comparison.

Proof. Distributivity gives

$$(B_0 + \dot{B})(A_0 + \dot{A}) = B_0A_0 + B_0\dot{A} + \dot{B}A_0 + \dot{B}\dot{A}.$$

Subtracting $C = B_0A_0$ cancels the first term in the additive group of matrices. This proves the general identity. The canonical identity is its specialization using Theorem 15.4. Finally, ρ is additive and annihilates the linearized summand by Lemma 15.8; hence only $\rho(\dot{B}\dot{A})$ remains. No asymptotic truncation is involved: all three formulas are exact polynomial identities. □

```
/-- Bilinear expansion around an arbitrary exact product. -/
theorem product_perturbation_expansion {i h o : Nat}
    (A0 dA : Mat h i R) (B0 dB : Mat o h R) (C : Mat o i R)
    (h0 : mul B0 A0 = C) :
    sub (mul (add B0 dB) (add A0 dA)) C =
      add (add (mul B0 dA) (mul dB A0)) (mul dB dA) := by
```

```
  rw [add_mul, mul_add, mul_add, h0]
  funext x y
  simp only [add, sub]
  grind

theorem canonical_error_expansion (r alpha beta h p q : Nat)
    (dA : Mat ((r+alpha)+(beta+h)) ((r+alpha)+q) R)
    (dB : Mat ((r+beta)+p) ((r+alpha)+(beta+h)) R) :
    sub
      (mul (add (canonicalB (R := R) r alpha beta h p) dB)
        (add (canonicalA (R := R) r alpha beta h q) dA))
      (canonicalTarget (R := R) r alpha beta p q) =
    add (canonicalDerivative (R := R) r alpha beta h p q dA dB) (mul dB dA) := by
  exact product_perturbation_expansion
    (canonicalA (R := R) r alpha beta h q) dA
    (canonicalB (R := R) r alpha beta h p) dB
    (canonicalTarget (R := R) r alpha beta p q)
    (canonical_product r alpha beta h p q)

theorem canonical_error_bottomRight (r alpha beta h p q : Nat)
    (dA : Mat ((r+alpha)+(beta+h)) ((r+alpha)+q) R)
    (dB : Mat ((r+beta)+p) ((r+alpha)+(beta+h)) R) :
    bottomRight (m := r+beta) (n := r+alpha)
      (sub
        (mul (add (canonicalB (R := R) r alpha beta h p) dB)
          (add (canonicalA (R := R) r alpha beta h q) dA))
        (canonicalTarget (R := R) r alpha beta p q)) =
    bottomRight (m := r+beta) (n := r+alpha) (mul dB dA) := by
  rw [canonical_error_expansion, bottomRight_add,
    canonicalDerivative_bottomRight_zero]
  funext i j
  simp only [add, zero]
  grind


end O77.MatrixCore
```

15.7 Closed regression cases for the canonical derivative layer

Lemma 15.15 (A concrete 2×2 regression family)**.**

Corresponding Lean declaration(s): `O77.MatrixCore.CanonicalRegression.sampleNonResidual`,`O77.MatrixCore.CanonicalRegression.sampleResidual`.

For $r = 1$, $\alpha = \beta = 0$ and $h = p = q = 1$, the canonical factors and target are the 2×2 matrices with a single 1 in the upper-left corner. The matrices

$$S_0 = \begin{pmatrix} 1 & 2 \\ 3 & 0 \end{pmatrix}, \qquad S_1 = \begin{pmatrix} 1 & 2 \\ 3 & 7 \end{pmatrix}$$

have residual entries 0 and 7, respectively. The extracted file checks the canonical product, constructs an explicit derivative preimage of S_0, verifies the direct decomposition of S_1, and checks the coordinate count $2 \cdot 1 + 1 \cdot 1 = 3$.

Proof. All assertions are finite closed computations in $\mathbb{Z}$. They are regression tests only; the universal theorems above, rather than these examples, establish the mathematical claims. The function equalities are proved entrywise because the standard library does not supply a decision procedure for equality of all function-valued matrices automatically. □

```
namespace O77.MatrixCore.CanonicalRegression

open O77.MatrixCore

def sampleNonResidual : Mat 2 2 Int := fun i j =>
  if i.val = 0 then (if j.val = 0 then 1 else 2)
  else (if j.val = 0 then 3 else 0)

def sampleResidual : Mat 2 2 Int := fun i j =>
  if i.val = 0 then (if j.val = 0 then 1 else 2)
  else (if j.val = 0 then 3 else 7)

example : mul (canonicalB (R := Int) 1 0 0 1 1)
    (canonicalA (R := Int) 1 0 0 1 1) =
    canonicalTarget (R := Int) 1 0 0 1 1 :=
  canonical_product 1 0 0 1 1 1

example : bottomRight (R := Int) (m := 1) (n := 1) (p := 1) (q := 1) sampleNonResidual =
    (zero 1 1 : Mat 1 1 Int) := by
  funext i j
  exact (by decide : ∀ i j : Fin 1,
    bottomRight (R := Int) (m := 1) (n := 1) (p := 1) (q := 1) sampleNonResidual i j =
      (zero 1 1 : Mat 1 1 Int) i j) i j

example : canonicalDerivative (R := Int) 1 0 0 1 1 1
    (derivativePreimageA (R := Int) 1 0 0 1 1 1 sampleNonResidual)
    (derivativePreimageB (R := Int) 1 0 0 1 1 1 sampleNonResidual) =
    sampleNonResidual := by
  apply canonicalDerivative_preimage
  funext i j
  exact (by decide : ∀ i j : Fin 1,
    bottomRight (R := Int) (m := 1) (n := 1) (p := 1) (q := 1) sampleNonResidual i j =
      (zero 1 1 : Mat 1 1 Int) i j) i j

example : bottomRight (R := Int) (m := 1) (n := 1) (p := 1) (q := 1) sampleResidual 0 0 = 7 := by
  decide

example : add (assembleNonResidual (splitNonResidual (a := 1) (b := 1) (p := 1) (q := 1)
     sampleResidual))
    (injectResidual (R := Int) (a := 1) (b := 1)
      (bottomRight (R := Int) (m := 1) (n := 1) (p := 1) (q := 1) sampleResidual)) = sampleResidual :=
  target_decomposition (R := Int) (a := 1) (b := 1) (p := 1) (q := 1) sampleResidual

example : (1+1) * 1 + 1 * 1 = regularDim (1+1) (1+1) 1 1 :=
  nonResidual_coordinate_count 1 1 1 1

example : ∀ i : Fin 2, ∀ j : Fin 2,
    canonicalTarget (R := Int) 1 0 0 1 1 i j =
      (if i.val = 0 ∧ j.val = 0 then 1 else 0) := by
  decide

end O77.MatrixCore.CanonicalRegression
```

16 The exact algebraic elimination chart

This section closes the principal algebraic composition step in the fiber branch. The preceding checked chapters proved the first triangular gauge, the output normalizer, the shear, the canonical derivative calculation, and the relevant block identities separately. The present section composes those ingredients into one forward coordinate map, writes an explicit inverse,

checks both round trips, and proves the exact error formula from which Sneiderman's two-sided norm comparison is obtained.

No general-purpose linear algebra is rebuilt here. The coefficient object is an arbitrary associative unital ring R, represented by the already used `Lean.Grind.Ring` interface, and the matrix type is the checked entry-function type of Section 14. The purpose is to verify the O77-specific rational formulas. Topology, determinants, real-analyticity, Frobenius norms, and Lebesgue measure do not occur in this section.

Fix $r, \alpha, \beta, h, p, q \in \mathbb{N}$, and put

$$a = r + \alpha, \qquad b = r + \beta, \qquad I = a + q, \qquad O = b + p, \qquad H = a + (\beta + h).$$

The row and column parenthesizations are those of Section 15. All statements below include zero block sizes.

16.1 The proof-carrying pivot domain and the two coordinate records

Definition 16.1 (Invertible pivots and the varying output pivot)**.**

Corresponding Lean declaration(s): `O77.MatrixCore.InvMatrix`,`O77.MatrixCore.OutputPivotData`,`O77.MatrixCore.outputTall`,`O77.MatrixCore.outputTop`,`O77.MatrixCore.outputBottom`,`O77.MatrixCore.outputTop_inverse_mul`,`O77.MatrixCore.outputTop_mul_inverse`.

An input pivot datum *is a pair of matrices $P, E \in M_a(R)$ together with the two identities*

$$EP = I_a, \qquad PE = I_a.$$

An output pivot datum *consists of a matrix $D \in M_{O,\beta}(R)$ and a matrix $F \in M_b(R)$. Let*

$$J \in M_{O,r}(R), \qquad P(D) = [J \mid D] \in M_{O,b}(R),$$

where J is the fixed canonical inclusion used in the canonical factorization. Writing

$$P(D) = \begin{pmatrix} P_{\mathrm{top}}(D) \\ P_{\mathrm{bot}}(D) \end{pmatrix}, \qquad P_{\mathrm{top}}(D) \in M_b(R), \quad P_{\mathrm{bot}}(D) \in M_{p,b}(R),$$

the datum requires

$$FP_{\mathrm{top}}(D) = I_b, \qquad P_{\mathrm{top}}(D)F = I_b.$$

These records model the two principal open sets on which the rational chart is defined. At this stage invertibility is explicit data, not an assertion extracted from a determinant.

Definition 16.2 (Raw blocks and eliminated coordinates)**.**

Corresponding Lean declaration(s): `O77.MatrixCore.ElimInput`,`O77.MatrixCore.ElimCoordinates`,`O77.MatrixCore.ElimInput.aMatrix`,`O77.MatrixCore.ElimInput.bMatrix`,`O77.MatrixCore.elimTarget`.

A raw point of the chart domain is the tuple

$$z = (P, E, C, A_{12}, A_{22}, B_1, D, F, Y),$$

where the inverse equations of Definition 16.1 hold and

block	*shape*
C	$(\beta + h) \times a$
A_{12}	$a \times q$
A_{22}	$(\beta + h) \times q$
B_1	$O \times a$
D	$O \times \beta$
Y	$O \times h$.

It represents the full matrices

$$A(z) = \begin{pmatrix} P & A_{12} \\ C & A_{22} \end{pmatrix} \in M_{H,I}(R), \qquad B(z) = [B_1 \mid D \mid Y] \in M_{O,H}(R).$$

The fixed target $C_0 \in M_{O,I}(R)$ is the canonical rank-r matrix from Section 15.

The eliminated-coordinate record is

$$c = (U, T', Y_0, Z; P, E, C, X_P, D, F, Y_1),$$

with

$$U \in M_{O,a}(R), \quad T' \in M_{b,q}(R), \quad Y_0 \in M_{p,h}(R), \quad Z \in M_{h,q}(R),$$

and passive blocks

$$X_P \in M_{\alpha,q}(R), \qquad Y_1 \in M_{b,h}(R).$$

The pivot data and C are retained. The first four displayed matrices are precisely the regular and residual generators needed by the normal form; the remaining blocks are gauge coordinates. The words "regular" and "residual" name their later role and do not yet make an analytic claim.

```
namespace O77.MatrixCore

universe u
variable {R : Type u} [Lean.Grind.Ring R]

/-- A square matrix together with a specified two-sided inverse.  This is an
algebraic model of the open nonsingular locus; the inverse proof is not a
coordinate of the analytic chart. -/
structure InvMatrix (n : Nat) (R : Type u) [Lean.Grind.Ring R] where
  matrix : Mat n n R
  inverse : Mat n n R
  inverse_mul : mul inverse matrix = one n
  mul_inverse : mul matrix inverse = one n

/-- The varying `D` block and the inverse of the top square block of `[J|D]`. -/
structure OutputPivotData (r beta p : Nat) (R : Type u) [Lean.Grind.Ring R] where
  d : Mat ((r+beta)+p) beta R
  inverse : Mat (r+beta) (r+beta) R
  inverse_mul_top :
    mul inverse
      (rowsTop (m := r+beta) (n := p)
        (hcat (canonicalJ (R := R) r beta p) d)) = one (r+beta)
  top_mul_inverse :
    mul
      (rowsTop (m := r+beta) (n := p)
        (hcat (canonicalJ (R := R) r beta p) d)) inverse = one (r+beta)

/-- Raw parameter blocks on the proof-carrying pivot domain. -/
structure ElimInput (r alpha beta h p q : Nat) (R : Type u) [Lean.Grind.Ring R] where
  aPivot : InvMatrix (r+alpha) R
  aLower : Mat (beta+h) (r+alpha) R
  a12 : Mat (r+alpha) q R
  a22 : Mat (beta+h) q R
  b1 : Mat ((r+beta)+p) (r+alpha) R
  outPivot : OutputPivotData r beta p R
  bY : Mat ((r+beta)+p) h R

/-- Coordinates of the complete algebraic elimination map.  The first four
fields are the regular and residual generators; the last five are gauge
coordinates. -/
```

```
structure ElimCoordinates (r alpha beta h p q : Nat) (R : Type u) [Lean.Grind.Ring R] where
  regularLeft : Mat ((r+beta)+p) (r+alpha) R
  regularRight : Mat (r+beta) q R
  residualLeft : Mat p h R
  residualRight : Mat h q R
  aPivot : InvMatrix (r+alpha) R
  aLower : Mat (beta+h) (r+alpha) R
  xPassive : Mat alpha q R
  outPivot : OutputPivotData r beta p R
  yPassive : Mat (r+beta) h R

/-- The full `A` matrix represented by raw elimination input. -/
def ElimInput.aMatrix {r alpha beta h p q : Nat}
    (z : ElimInput r alpha beta h p q R) :
    Mat ((r+alpha)+(beta+h)) ((r+alpha)+q) R :=
  block z.aPivot.matrix z.a12 z.aLower z.a22

/-- The full `B` matrix represented by raw elimination input. -/
def ElimInput.bMatrix {r alpha beta h p q : Nat}
    (z : ElimInput r alpha beta h p q R) :
    Mat ((r+beta)+p) ((r+alpha)+(beta+h)) R :=
  hcat z.b1 (hcat z.outPivot.d z.bY)

/-- The fixed canonical target in the same coordinates. -/
def elimTarget (r alpha beta p q : Nat) :
    Mat ((r+beta)+p) ((r+alpha)+q) R :=
  canonicalTarget (R := R) r alpha beta p q

/-- The tall varying output pivot `[J|D]`. -/
def outputTall {r beta p : Nat} (z : OutputPivotData r beta p R) :
    Mat ((r+beta)+p) (r+beta) R :=
  hcat (canonicalJ (R := R) r beta p) z.d

/-- The top square block of the varying output pivot. -/
def outputTop {r beta p : Nat} (z : OutputPivotData r beta p R) :
    Mat (r+beta) (r+beta) R :=
  rowsTop (m := r+beta) (n := p) (outputTall z)

/-- The bottom block of the varying output pivot. -/
def outputBottom {r beta p : Nat} (z : OutputPivotData r beta p R) :
    Mat p (r+beta) R :=
  rowsBot (m := r+beta) (n := p) (outputTall z)

@[simp] theorem outputTop_inverse_mul {r beta p : Nat}
    (z : OutputPivotData r beta p R) :
    mul z.inverse (outputTop z) = one (r+beta) := z.inverse_mul_top

@[simp] theorem outputTop_mul_inverse {r beta p : Nat}
    (z : OutputPivotData r beta p R) :
    mul (outputTop z) z.inverse = one (r+beta) := z.top_mul_inverse
```

16.2 The forward and inverse formulas

Definition 16.3 (The complete forward and backward maps)**.**

Corresponding Lean declaration(s): `077.MatrixCore.elimForward`,`077.MatrixCore.elimBackward`.

For a raw point z, apply the first gauge to obtain

$$X = EA_{12}, \qquad S = A_{22} - CX, \qquad \bar{B}_1 = B_1P + [D \mid Y]C.$$

Split

$$X = \begin{pmatrix} X_R \\ X_P \end{pmatrix}, \qquad S = \begin{pmatrix} S_Q \\ Z \end{pmatrix}, \qquad X_R \in M_{r,q}(R), \quad S_Q \in M_{\beta,q}(R),$$

and put

$$U = \bar{B}_1 - C_I, \qquad T = \begin{pmatrix} X_R \\ S_Q \end{pmatrix} \in M_{b,q}(R).$$

Here $C_I \in M_{O,a}(R)$ is the left column block of the canonical target. Define the output normalizer and denormalizer

$$N(D) = \begin{pmatrix} F & 0 \\ -P_{\mathrm{bot}}(D)F & I_p \end{pmatrix}, \qquad Q(D) = \begin{pmatrix} P_{\mathrm{top}}(D) & 0 \\ P_{\mathrm{bot}}(D) & I_p \end{pmatrix}.$$

Write

$$N(D)Y = \begin{pmatrix} Y_1 \\ Y_0 \end{pmatrix}$$

and perform the final shear

$$T' = T + Y_1 Z.$$

The forward map sends z to

$$(U, T', Y_0, Z; P, E, C, X_P, D, F, Y_1).$$

Conversely, from such a coordinate tuple define

$$T = T' - Y_1 Z, \qquad X = \begin{pmatrix} T_{[1,r]} \\ X_P \end{pmatrix}, \qquad S = \begin{pmatrix} T_{[r+1,b]} \\ Z \end{pmatrix}, \qquad Y = Q(D)\begin{pmatrix} Y_1 \\ Y_0 \end{pmatrix}.$$

Then set

$$\bar{B}_1 = U + C_I, \qquad \bar{B}_2 = [D \mid Y],$$

and apply the inverse first gauge:

$$A_{12} = PX, \qquad A_{22} = S + CX, \qquad B_1 = (\bar{B}_1 - \bar{B}_2 C)E.$$

The matrix D is retained verbatim; the recovered Y is the last h columns of $\bar{B}_2$. These formulas define the backward map.

```
/-- The complete forward elimination map on its proof-carrying domain. -/
def elimForward {r alpha beta h p q : Nat}
    (z : ElimInput r alpha beta h p q R) :
    ElimCoordinates r alpha beta h p q R :=
  let gi : GaugeInput (r+alpha) (beta+h) q ((r+beta)+p) R :=
    { a12 := z.a12
      a22 := z.a22
      b1 := z.b1
      b2 := hcat z.outPivot.d z.bY }
  let gc := gaugeForward z.aPivot.matrix z.aPivot.inverse z.aLower gi
  let xR := rowsTop (m := r) (n := alpha) gc.x
  let xP := rowsBot (m := r) (n := alpha) gc.x
  let sQ := rowsTop (m := beta) (n := h) gc.s
  let sZ := rowsBot (m := beta) (n := h) gc.s
  let u := sub gc.b1bar (canonicalCI (R := R) r alpha beta p)
  let nrm := normalizer z.outPivot.inverse (outputBottom z.outPivot)
  let nY := mul nrm z.bY
  let y1 := rowsTop (m := r+beta) (n := p) nY
  let y0 := rowsBot (m := r+beta) (n := p) nY
  let t := vcat xR sQ
  { regularLeft := u
```

```
    regularRight := shear y1 sZ t
    residualLeft := y0
    residualRight := sZ
    aPivot := z.aPivot
    aLower := z.aLower
    xPassive := xP
    outPivot := z.outPivot
    yPassive := y1 }

/-- The complete inverse elimination formula. -/
def elimBackward {r alpha beta h p q : Nat}
    (z : ElimCoordinates r alpha beta h p q R) :
    ElimInput r alpha beta h p q R :=
  let t := unshear z.yPassive z.residualRight z.regularRight
  let xR := rowsTop (m := r) (n := beta) t
  let sQ := rowsBot (m := r) (n := beta) t
  let x := vcat xR z.xPassive
  let s := vcat sQ z.residualRight
  let y := mul (denormalizer (outputTop (p := p) z.outPivot) (outputBottom (r := r) z.outPivot))
    (vcat z.yPassive z.residualLeft)
  let gc : GaugeCoordinates (r+alpha) (beta+h) q ((r+beta)+p) R :=
    { x := x
      s := s
      b1bar := add z.regularLeft (canonicalCI (R := R) r alpha beta p)
      b2bar := hcat z.outPivot.d y }
  let gi := gaugeBackward z.aPivot.matrix z.aPivot.inverse z.aLower gc
  { aPivot := z.aPivot
    aLower := z.aLower
    a12 := gi.a12
    a22 := gi.a22
    b1 := gi.b1
    outPivot := z.outPivot
    bY := colsRight (n := beta) (k := h) gi.b2 }

end O77.MatrixCore
```

16.3 Output normalization is exactly invertible

Lemma 16.4 (The output normalization has both round trips)**.**

Corresponding Lean declaration(s): `O77.MatrixCore.output_normalizer_denormalizer`,`O77.MatrixCore.output_denormalizer_normalizer`,`O77.MatrixCore.outputY_backward_forward`,`O77.MatrixCore.outputY_forward_backward`.

For every output pivot datum,

$$N(D)Q(D) = I_O, \qquad Q(D)N(D) = I_O.$$

Consequently, for every $Y \in M_{O,h}(R)$*, if* $N(D)Y = \binom{Y_1}{Y_0}$ *then*

$$Q(D)\begin{pmatrix} Y_1 \\ Y_0 \end{pmatrix} = Y.$$

Conversely, normalizing $Q(D)\binom{Y_1}{Y_0}$ *recovers the prescribed pair* (Y_1, Y_0)*.*

Proof. The two matrix products are the already checked normalizer identities with $P = P_{\mathrm{top}}(D)$, $E = F$, and $C = P_{\mathrm{bot}}(D)$. For the first recovery, reassemble the top and bottom rows of $N(D)Y$ to obtain $N(D)Y$, multiply by $Q(D)$, and use $Q(D)N(D) = I_O$. For the reverse recovery, associate the product as $N(D)Q(D)\binom{Y_1}{Y_0}$ and use the other inverse identity. No cancellation law beyond the two recorded inverse equations is used. □

```
namespace O77.MatrixCore
universe u
variable {R : Type u} [Lean.Grind.Ring R]

@[simp] theorem output_normalizer_denormalizer {r beta p : Nat}
    (z : OutputPivotData r beta p R) :
    mul (normalizer z.inverse (outputBottom z))
      (denormalizer (outputTop z) (outputBottom z)) = one ((r+beta)+p) := by
  exact normalizer_denormalizer (outputTop z) z.inverse (outputBottom z)
    z.inverse_mul_top

@[simp] theorem output_denormalizer_normalizer {r beta p : Nat}
    (z : OutputPivotData r beta p R) :
    mul (denormalizer (outputTop z) (outputBottom z))
      (normalizer z.inverse (outputBottom z)) = one ((r+beta)+p) := by
  exact denormalizer_normalizer (outputTop z) z.inverse (outputBottom z)
    z.top_mul_inverse

/-- The output normalization recovers the original `Y` block. -/
theorem outputY_backward_forward {r beta p h : Nat}
    (z : OutputPivotData r beta p R) (Y : Mat ((r+beta)+p) h R) :
    mul (denormalizer (outputTop z) (outputBottom z))
      (vcat
        (rowsTop (m := r+beta) (n := p)
          (mul (normalizer z.inverse (outputBottom z)) Y))
        (rowsBot (m := r+beta) (n := p)
          (mul (normalizer z.inverse (outputBottom z)) Y))) = Y := by
  rw [vcat_rows, ← mul_assoc, output_denormalizer_normalizer, one_mul]

/-- Forward normalization recovers prescribed top and bottom normalized blocks. -/
theorem outputY_forward_backward {r beta p h : Nat}
    (z : OutputPivotData r beta p R)
    (Y1 : Mat (r+beta) h R) (Y0 : Mat p h R) :
    mul (normalizer z.inverse (outputBottom z))
      (mul (denormalizer (outputTop z) (outputBottom z)) (vcat Y1 Y0)) =
      vcat Y1 Y0 := by
  rw ←[ mul_assoc, output_normalizer_denormalizer, one_mul]

end O77.MatrixCore
```

16.4 The two chart round trips

Theorem 16.5 (Backward after forward)**.**

Corresponding Lean declaration(s): `O77.MatrixCore.gaugeInput_ext`,`O77.MatrixCore.gaugeCoordinates_ext`,`O77.MatrixCore.elimInput_ext`,`O77.MatrixCore.elimCoordinates_ext`,`O77.MatrixCore.elim_backward_forward`.

For every raw point z on the proof-carrying pivot domain,

$$\mathrm{elimBackward}(\mathrm{elimForward}(z)) = z.$$

Proof. The inverse shear first gives T back. Taking its first r and last β rows recovers X_R and S_Q, while X_P and Z were retained; row reconstruction therefore returns X and S. Lemma 16.4 recovers Y. The matrix $\bar{B}_2 = [D \mid Y]$ is consequently the original second gauge block. The already checked identity `gauge_backward_forward` then recovers A_{12}, A_{22}, B_1, and $[D \mid Y]$. Projecting the latter to its last h columns recovers the stored Y block. All pivot data and the block C were retained unchanged. Extensionality of the finite records concludes the equality. □

```
namespace O77.MatrixCore
universe u

/-- Extensionality for raw gauge-input records. -/
theorem gaugeInput_ext {S : Type u} {a k q o : Nat}
    {x y : GaugeInput a k q o S}
    (h1 : x.a12 = y.a12) (h2 : x.a22 = y.a22)
    (h3 : x.b1 = y.b1) (h4 : x.b2 = y.b2) : x = y := by
  cases x
  cases y
  simp_all

/-- Extensionality for gauge-coordinate records. -/
theorem gaugeCoordinates_ext {S : Type u} {a k q o : Nat}
    {x y : GaugeCoordinates a k q o S}
    (h1 : x.x = y.x) (h2 : x.s = y.s)
    (h3 : x.b1bar = y.b1bar) (h4 : x.b2bar = y.b2bar) : x = y := by
  cases x
  cases y
  simp_all

variable {R : Type u} [Lean.Grind.Ring R]

/-- Extensionality for the raw elimination records. -/
theorem elimInput_ext {r alpha beta h p q : Nat}
    {x y : ElimInput r alpha beta h p q R}
    (h1 : x.aPivot = y.aPivot) (h2 : x.aLower = y.aLower)
    (h3 : x.a12 = y.a12) (h4 : x.a22 = y.a22)
    (h5 : x.b1 = y.b1) (h6 : x.outPivot = y.outPivot)
    (h7 : x.bY = y.bY) : x = y := by
  cases x
  cases y
  simp_all

/-- Extensionality for the eliminated coordinate records. -/
theorem elimCoordinates_ext {r alpha beta h p q : Nat}
    {x y : ElimCoordinates r alpha beta h p q R}
    (h1 : x.regularLeft = y.regularLeft)
    (h2 : x.regularRight = y.regularRight)
    (h3 : x.residualLeft = y.residualLeft)
    (h4 : x.residualRight = y.residualRight)
    (h5 : x.aPivot = y.aPivot) (h6 : x.aLower = y.aLower)
    (h7 : x.xPassive = y.xPassive) (h8 : x.outPivot = y.outPivot)
    (h9 : x.yPassive = y.yPassive) : x = y := by
  cases x
  cases y
  simp_all

/-- The inverse formula recovers every raw block. -/
theorem elim_backward_forward {r alpha beta h p q : Nat}
    (z : ElimInput r alpha beta h p q R) :
    elimBackward (elimForward z) = z := by
  cases z with
  | mk aP C A12 A22 B1 oP BY =>
    simp only [elimForward, elimBackward]
    rw [unshear_shear, rowsTop_vcat, rowsBot_vcat, vcat_rows,
      outputY_backward_forward]
    simp only [vcat_rows, sub_add_cancel]
    let gi : GaugeInput (r+alpha) (beta+h) q ((r+beta)+p) R :=
      { a12 := A12, a22 := A22, b1 := B1, b2 := hcat oP.d BY }
    let gc := gaugeForward aP.matrix aP.inverse C gi
    have harg :
```

```
        ({ x := gc.x, s := gc.s, b1bar := gc.b1bar, b2bar := hcat oP.d BY } :
          GaugeCoordinates (r+alpha) (beta+h) q ((r+beta)+p) R) = gc := by
      apply gaugeCoordinates_ext <;> rfl
    rw [show
      ({ x := (gaugeForward aP.matrix aP.inverse C gi).x
         s := (gaugeForward aP.matrix aP.inverse C gi).s
         b1bar := (gaugeForward aP.matrix aP.inverse C gi).b1bar
         b2bar := hcat oP.d BY } :
        GaugeCoordinates (r+alpha) (beta+h) q ((r+beta)+p) R) =
        gaugeForward aP.matrix aP.inverse C gi by simpa [gc] using harg]
    have hround := gauge_backward_forward aP.matrix aP.inverse C aP.mul_inverse gi
    apply elimInput_ext
    · rfl
    · rfl
    · exact congrArg GaugeInput.a12 hround
    · exact congrArg GaugeInput.a22 hround
    · exact congrArg GaugeInput.b1 hround
    · rfl
    · have hb2 := congrArg GaugeInput.b2 hround
      simpa [gi] using congrArg (colsRight (n := beta) (k := h)) hb2

end O77.MatrixCore
```

Theorem 16.6 (Forward after backward)**.**

Corresponding Lean declaration(s): `O77.MatrixCore.gaugeInput_rebuild_after_backward`,`O77.MatrixCore.elim_forward_backward`.

For every eliminated-coordinate point c,

$$\mathrm{elimForward}(\mathrm{elimBackward}(c)) = c.$$

Proof. Start from $T = T' - Y_1 Z$ and reconstruct X, S, and Y as in Definition 16.3. The inverse first gauge produces a raw tuple. When the forward first gauge is applied again, the checked identity `gauge_forward_backward` returns exactly the prepared tuple $(X, S, U + C_I, [D \mid Y])$. The only bookkeeping subtlety is that the raw record stores D and Y separately: after the inverse gauge, recombining the retained first β columns with the recovered last h columns gives the whole block $[D \mid Y]$. Lemma 16.4 returns Y_1, Y_0 from Y, row splitting returns the four parts of X and S, and the shear identity gives $(T' - Y_1 Z) + Y_1 Z = T'$. Every field of c is thus recovered. □

```
namespace O77.MatrixCore
universe u
variable {R : Type u} [Lean.Grind.Ring R]

/-- Reassembling the retained `D` columns and the recovered `Y` columns
reproduces the entire second gauge-input block. -/
theorem gaugeInput_rebuild_after_backward {a beta h q o : Nat}
    (P E : Mat a a R) (C : Mat (beta+h) a R)
    (D : Mat o beta R) (Y : Mat o h R)
    (gc : GaugeCoordinates a (beta+h) q o R)
    (hb2 : gc.b2bar = hcat D Y) :
    ({ a12 := (gaugeBackward P E C gc).a12
       a22 := (gaugeBackward P E C gc).a22
       b1 := (gaugeBackward P E C gc).b1
       b2 := hcat D
         (colsRight (n := beta) (k := h) (gaugeBackward P E C gc).b2) } :
      GaugeInput a (beta+h) q o R) = gaugeBackward P E C gc := by
  apply gaugeInput_ext <;> try rfl
  change hcat D (colsRight (n := beta) (k := h) gc.b2bar) = gc.b2bar
```

```
  rw [hb2, colsRight_hcat]

/-- The forward formula recovers every eliminated coordinate. -/
theorem elim_forward_backward {r alpha beta h p q : Nat}
    (z : ElimCoordinates r alpha beta h p q R) :
    elimForward (elimBackward z) = z := by
  cases z with
  | mk U Tp Y0 Z aP C XP oP Y1 =>
    let T : Mat (r+beta) q R := unshear Y1 Z Tp
    let X : Mat (r+alpha) q R :=
      vcat (rowsTop (m := r) (n := beta) T) XP
    let S : Mat (beta+h) q R :=
      vcat (rowsBot (m := r) (n := beta) T) Z
    let Y : Mat ((r+beta)+p) h R :=
      mul (denormalizer (outputTop oP) (outputBottom oP)) (vcat Y1 Y0)
    let gc : GaugeCoordinates (r+alpha) (beta+h) q ((r+beta)+p) R :=
      { x := X
        s := S
        b1bar := add U (canonicalCI (R := R) r alpha beta p)
        b2bar := hcat oP.d Y }
    have hgi := gaugeInput_rebuild_after_backward
      aP.matrix aP.inverse C oP.d Y gc (by rfl)
    have hround := gauge_forward_backward aP.matrix aP.inverse C aP.inverse_mul gc
    simp only [elimBackward, elimForward]
    rw [show
      ({ a12 := (gaugeBackward aP.matrix aP.inverse C gc).a12
         a22 := (gaugeBackward aP.matrix aP.inverse C gc).a22
         b1 := (gaugeBackward aP.matrix aP.inverse C gc).b1
         b2 := hcat oP.d
           (colsRight (n := beta) (k := h)
             (gaugeBackward aP.matrix aP.inverse C gc).b2) } :
        GaugeInput (r+alpha) (beta+h) q ((r+beta)+p) R) =
        gaugeBackward aP.matrix aP.inverse C gc from hgi]
    rw [hround]
    simp only [gaugeBackward, colsRight_hcat]
    simp only [gc, X, S, T]
    rw [add_sub_cancel, outputY_forward_backward,
      rowsTop_vcat, rowsBot_vcat, rowsTop_vcat, rowsBot_vcat,
      rowsTop_vcat, rowsBot_vcat, vcat_rows, shear_unshear]

end O77.MatrixCore
```

16.5 The exact generator normal form

Lemma 16.7 (Reconstruction through the output denormalizer)**.**

Corresponding Lean declaration(s): `O77.MatrixCore.normalized_generator_reconstruction`,`O77.MatrixCore.ElimCoordinates.xMatrix`,`O77.MatrixCore.ElimCoordinates.tMatrix`,`O77.MatrixCore.ElimCoordinates.outputDenormalizer`.

Let

$$U \in M_{b+p,a}(R), \quad X \in M_{a,q}(R), \quad T \in M_{b,q}(R), \quad Y \in M_{b+p,h}(R), \quad Z \in M_{h,q}(R),$$

and write a tall pivot as $\binom{P}{C}$ *with* $P \in M_b(R)$ *invertible and* $C \in M_{p,b}(R)$*. Put*

$$N = \begin{pmatrix} P^{-1} & 0 \\ -CP^{-1} & I_p \end{pmatrix}, \qquad NY = \begin{pmatrix} Y_1 \\ Y_0 \end{pmatrix}.$$

Then the exact identity

$$UX + \begin{pmatrix} P \\ C \end{pmatrix} T + YZ = UX + \begin{pmatrix} P & 0 \\ C & I_p \end{pmatrix} \begin{pmatrix} T + Y_1 Z \\ Y_0 Z \end{pmatrix} \tag{16.7.1}$$

holds. For an eliminated-coordinate record c, the helper maps `xMatrix`*,* `tMatrix`*, and* `outputDenormalizer` *recover, respectively, X, $T = T' - Y_1 Z$, and the displayed denormalizer.*

Proof. The checked normalized-error identity gives

$$N\big((UX + \begin{pmatrix} P \\ C \end{pmatrix} T + YZ) - UX\big) = \begin{pmatrix} T + Y_1 Z \\ Y_0 Z \end{pmatrix}.$$

Multiply by the denormalizer on the left. Its product with N is the identity, so the parenthesized difference equals the denormalized column block. Finally add UX to both sides. The helper definitions merely undo the shear and concatenate the retained row blocks; they contain no additional theorem. □

```
namespace O77.MatrixCore
universe u
variable {R : Type u} [Lean.Grind.Ring R]

/-- Reconstruction form of the exact generator reduction.  It is the algebraic
identity used by the two-sided norm comparison. -/
theorem normalized_generator_reconstruction {a b p h q : Nat}
    (U : Mat (b+p) a R) (X : Mat a q R)
    (P E : Mat b b R) (C : Mat p b R)
    (T : Mat b q R) (Y : Mat (b+p) h R) (Z : Mat h q R)
    (hEP : mul E P = one b) (hPE : mul P E = one b) :
    let N := normalizer E C
    let Y1 := rowsTop (m := b) (n := p) (mul N Y)
    let Y0 := rowsBot (m := b) (n := p) (mul N Y)
    let V := add (add (mul U X) (mul (vcat P C) T)) (mul Y Z)
    V = add (mul U X)
      (mul (denormalizer P C)
        (vcat (add T (mul Y1 Z)) (mul Y0 Z))) := by
  dsimp only
  have hred := normalized_generator_identity U X P E C T Y Z hEP
  have hmul := congrArg (fun W => mul (denormalizer P C) W) hred
  have hsub :
      sub (add (add (mul U X) (mul (vcat P C) T)) (mul Y Z))
        (mul U X) =
      mul (denormalizer P C)
        (vcat (add T
          (mul (rowsTop (m := b) (n := p) (mul (normalizer E C) Y)) Z))
          (mul (rowsBot (m := b) (n := p) (mul (normalizer E C) Y)) Z)) := by
    simpa only ←[ mul_assoc, denormalizer_normalizer P E C hPE, one_mul] using hmul
  calc
    add (add (mul U X) (mul (vcat P C) T)) (mul Y Z) =
        add (mul U X)
          (sub (add (add (mul U X) (mul (vcat P C) T)) (mul Y Z))
            (mul U X)) := by
          funext i j; simp only [add, sub]; grind
    _ = _ := by rw [hsub]

/-- Recover the `X` matrix encoded in eliminated coordinates. -/
def ElimCoordinates.xMatrix {r alpha beta h p q : Nat}
    (z : ElimCoordinates r alpha beta h p q R) : Mat (r+alpha) q R :=
  vcat
    (rowsTop (m := r) (n := beta)
      (unshear z.yPassive z.residualRight z.regularRight))
    z.xPassive

/-- Recover the pre-shear regular matrix `T`. -/
def ElimCoordinates.tMatrix {r alpha beta h p q : Nat}
    (z : ElimCoordinates r alpha beta h p q R) : Mat (r+beta) q R :=
```

```
  unshear z.yPassive z.residualRight z.regularRight

/-- The denormalizing output matrix encoded by the varying `D` block. -/
def ElimCoordinates.outputDenormalizer {r alpha beta h p q : Nat}
    (z : ElimCoordinates r alpha beta h p q R) :
    Mat ((r+beta)+p) ((r+beta)+p) R :=
  denormalizer (outputTop z.outPivot) (outputBottom z.outPivot)
```

Theorem 16.8 (Complete exact error formula)**.**

Corresponding Lean declaration(s): `O77.MatrixCore.elim_forward_error_normal_form`.

For every raw point z, let $c = \mathrm{elimForward}(z)$, let $X = X(c)$ be the matrix reconstructed from (T', Y_1, Z, X_P), and let $Q = Q(c)$ be the output denormalizer reconstructed from D. Then

$$\boxed{B(z)A(z) - C_0 = \left[U \;\middle|\; UX + Q\begin{pmatrix} T' \\ Y_0 Z \end{pmatrix} \right].} \tag{16.8.1}$$

Equivalently, set

$$U_c = c.\mathrm{regularLeft}, \quad X_c = c.\mathrm{xMatrix}, \quad Q_c = c.\mathrm{outputDenormalizer}, \quad T'_c = c.\mathrm{regularRight},$$

$$Y_{0,c} = c.\mathrm{residualLeft}, \qquad Z_c = c.\mathrm{residualRight}.$$

Then the same identity reads

$$B(z)A(z) - C_0 = \left[U_c \;\middle|\; U_c X_c + Q_c \begin{pmatrix} T'_c \\ Y_{0,c} Z_c \end{pmatrix} \right].$$

This is an identity of matrices, not merely an equality modulo higher-order terms.

Proof. Apply the checked first-gauge product identity. In the gauged variables, the checked block expansion gives

$$\bar{B}\bar{A} - C_0 = [U \mid UX + P(D)T + YZ].$$

Decompose $P(D) = \binom{P_{\mathrm{top}}}{P_{\mathrm{bot}}}$, and apply Lemma 16.7 with $P = P_{\mathrm{top}}$, $C = P_{\mathrm{bot}}$, and the inverse stored in the output pivot datum. By definition of the forward map,

$$N(D)Y = \begin{pmatrix} Y_1 \\ Y_0 \end{pmatrix}, \qquad T' = T + Y_1 Z.$$

The right block is therefore exactly $UX + Q(D)\binom{T'}{Y_0 Z}$. The helper identity for $X(c)$ follows from $T = T' - Y_1 Z$ and row reconstruction. Substituting these definitional equalities yields (16.8.1).

The theorem deliberately stops before the norm estimate. The exact right block still contains UX and the varying invertible matrix $Q(D)$. Over $\mathbb{R}$, shrinking the chart so that X is small and $Q(D), Q(D)^{-1}$ are uniformly bounded turns this identity into the comparison with

$$\|U\|^2 + \|T'\|^2 + \|Y_0 Z\|_F^2.$$

Those analytic inequalities are proved in `O77Analytic`; they are not encoded into the algebraic statement. □

```
/-- The exact error normal form in the complete algebraic elimination coordinates. -/
theorem elim_forward_error_normal_form {r alpha beta h p q : Nat}
    (z : ElimInput r alpha beta h p q R) :
    let c := elimForward z
    sub (mul z.bMatrix z.aMatrix) (elimTarget (R := R) r alpha beta p q) =
```

```
      hcat c.regularLeft
        (add (mul c.regularLeft c.xMatrix)
          (mul c.outputDenormalizer
            (vcat c.regularRight
              (mul c.residualLeft c.residualRight)))) := by
  let gi : GaugeInput (r+alpha) (beta+h) q ((r+beta)+p) R :=
    { a12 := z.a12
      a22 := z.a22
      b1 := z.b1
      b2 := hcat z.outPivot.d z.bY }
  let gc := gaugeForward z.aPivot.matrix z.aPivot.inverse z.aLower gi
  let XR := rowsTop (m := r) (n := alpha) gc.x
  let XP := rowsBot (m := r) (n := alpha) gc.x
  let SQ := rowsTop (m := beta) (n := h) gc.s
  let Z := rowsBot (m := beta) (n := h) gc.s
  let U := sub gc.b1bar (canonicalCI (R := R) r alpha beta p)
  let Pfull := outputTall z.outPivot
  let Ptop := outputTop z.outPivot
  let Pbot := outputBottom z.outPivot
  let Y := z.bY
  let N := normalizer z.outPivot.inverse Pbot
  let Y1 := rowsTop (m := r+beta) (n := p) (mul N Y)
  let Y0 := rowsBot (m := r+beta) (n := p) (mul N Y)
  let T := vcat XR SQ
  have hproduct := gauge_product_identity z.aPivot.matrix z.aPivot.inverse z.aLower
    z.aPivot.mul_inverse gi
  have hblock := block_error_expansion
    (canonicalJ (R := R) r beta p) U XR XP z.outPivot.d SQ Y Z
  have hPfull : hcat (canonicalJ (R := R) r beta p) z.outPivot.d = Pfull := rfl
  have hPsplit : vcat Ptop Pbot = Pfull := by
    exact vcat_rows Pfull
  have hrec := normalized_generator_reconstruction U gc.x Ptop z.outPivot.inverse
    Pbot T Y Z z.outPivot.inverse_mul_top z.outPivot.top_mul_inverse
  dsimp only [ElimInput.aMatrix, ElimInput.bMatrix, elimTarget]
  rw [hproduct]
  have hx : vcat XR XP = gc.x := vcat_rows gc.x
  have hs : vcat SQ Z = gc.s := vcat_rows gc.s
  have hb1 : add U (canonicalCI (R := R) r alpha beta p) = gc.b1bar :=
    sub_add_cancel gc.b1bar (canonicalCI (R := R) r alpha beta p)
  have hb2 : hcat z.outPivot.d Y = gc.b2bar := by rfl
  have htgt : hcat (canonicalCI (R := R) r alpha beta p)
      (zero ((r+beta)+p) q) = canonicalTarget (R := R) r alpha beta p q := by rfl
  dsimp only at hblock
  simp only [canonicalCI] at hb1 htgt
  rw [hx, hs, hb1, hb2, htgt, hPfull] at hblock
  have herr :
      sub
        (mul
          (hcat gc.b1bar gc.b2bar)
          (block (one (r+alpha)) gc.x (zero (beta+h) (r+alpha)) gc.s))
        (canonicalTarget (R := R) r alpha beta p q) =
      hcat U
        (add (add (mul U gc.x) (mul Pfull T)) (mul Y Z)) := hblock
  rw [herr]
  have hcU : (elimForward z).regularLeft = U := by rfl
  have hcTp : (elimForward z).regularRight = shear Y1 Z T := by rfl
  have hcY0 : (elimForward z).residualLeft = Y0 := by rfl
  have hcZ : (elimForward z).residualRight = Z := by rfl
  have hcDn : (elimForward z).outputDenormalizer = denormalizer Ptop Pbot := by rfl
  have hcX : (elimForward z).xMatrix = gc.x := by
    simp only [ElimCoordinates.xMatrix, elimForward]
    rw [unshear_shear, rowsTop_vcat, vcat_rows]
```

```
  rw [hcU, hcX, hcDn, hcTp, hcY0, hcZ]
  have hrec' :
      add (add (mul U gc.x) (mul Pfull T)) (mul Y Z) =
        add (mul U gc.x)
          (mul (denormalizer Ptop Pbot)
            (vcat (shear Y1 Z T) (mul Y0 Z))) := by
    rw ←[ hPsplit]
    simpa only [shear] using hrec
  rw [hrec']

end O77.MatrixCore
```

16.6 Inverse witnesses do not create hidden coordinates

Lemma 16.9 (Uniqueness of the proof-carrying inverses)**.**

Corresponding Lean declaration(s): `O77.MatrixCore.twoSided_inverse_unique`,`O77.MatrixCore.invMatrix_ext_matrix`,`O77.MatrixCore.outputPivotData_ext_d`.

If E is a left inverse of P and F is a right inverse of P, then $E = F$. Consequently, two input-pivot records with the same matrix P are equal, and two output-pivot records with the same block D are equal.

Proof. Associativity and the unit laws give

$$E = EI = E(PF) = (EP)F = IF = F.$$

For the records, equality of the underlying pivot matrix reduces both inverse fields to two-sided inverses of the same matrix, so the preceding calculation identifies them. The remaining fields are propositions and hence are proof-irrelevant. For output pivots, equality of D gives equality of the top block $P_{\mathrm{top}}(D)$, after which the same argument applies. □

```
namespace O77.MatrixCore
universe u
variable {R : Type u} [Lean.Grind.Ring R]

/-- A two-sided inverse is unique; hence inverse witnesses do not add algebraic
coordinates to the proof-carrying chart domain. -/
theorem twoSided_inverse_unique {n : Nat} (P E F : Mat n n R)
    (hEP : mul E P = one n) (hPF : mul P F = one n) : E = F := by
  calc
    E = mul E (one n) := by rw [mul_one]
    _ = mul E (mul P F) := by rw [hPF]
    _ = mul (mul E P) F := by rw [mul_assoc]
    _ = mul (one n) F := by rw [hEP]
    _ = F := by rw [one_mul]

/-- Equality of inverse records is determined by their underlying matrix. -/
theorem invMatrix_ext_matrix {n : Nat} {x y : InvMatrix n R}
    (h : x.matrix = y.matrix) : x = y := by
  cases x with
  | mk Px Ex hExP hPxE =>
    cases y with
    | mk Py Ey hEyP hPyE =>
      simp only at h
      cases h
      have hE : Ex = Ey := twoSided_inverse_unique Px Ex Ey hExP hPyE
      cases hE
      rfl
```

```
/-- Equality of output-pivot records is determined by the varying `D` block. -/
theorem outputPivotData_ext_d {r beta p : Nat}
    {x y : OutputPivotData r beta p R} (h : x.d = y.d) : x = y := by
  cases x with
  | mk Dx Ex hEx hDx =>
    cases y with
    | mk Dy Ey hEy hDy =>
      simp only at h
      cases h
      have hE : Ex = Ey := twoSided_inverse_unique
        (rowsTop (m := r+beta) (n := p)
          (hcat (canonicalJ (R := R) r beta p) Dx)) Ex Ey hEx hDy
      cases hE
      rfl

end O77.MatrixCore
```

16.7 The analytic use of the algebraic chart

The forward map, inverse map, both round trips, and generator normal form are exact identities. The pivot inverse is a uniquely determined witness, so the proof-carrying record introduces no additional real coordinate. On the determinant-open pivot sets, `O77Analytic` identifies these formulas with analytic maps of ordinary matrix entries and supplies uniform Frobenius bounds. The subsequent transport modules prove the volume comparison, including its nonconstant Jacobian density and its all-small-radius quantifiers.

17 Direct saddle-band calculus

This section addresses the full-dimensional, all-small-radii estimate in O77(b), rather than extending the finite matrix library. An alternative approach first constructs a partial Morse normal form and then estimates its bands. The preferred argument here estimates the bands directly. It needs only two uniformly positive directions, the ordinary implicit-function theorem in those two variables, and the existence of lower values arbitrarily close to the base point. No parameter-dependent quadratic congruence, differentiability under a parameter integral, or three-dimensional Morse splitting is used.

The proof proceeds from a scalar secant bound to planar area and then to the full ambient volume. The centered-family premise in the reusable analytic theorem is constructed from actual discarded-mode derivatives in `O77SaddlePreparation` and `O77SaddleAttachment`.

17.1 The one-dimensional statement behind the uniform planar bound

Definition 17.1 (Bands, squared-radius rays, and radial control)**.**

Corresponding Lean declaration(s): `O77.SaddleDirect.Plane,O77.SaddleDirect.disk,O77.SaddleDirect.intervalBand,O77.SaddleDirect.planarBand,O77.SaddleDirect.ray,O77.SaddleDirect.RadialControl`.

For $R > 0$ let $D_R = \{x = (x_1, x_2) \in \mathbb{R}^2 : x_1^2 + x_2^2 < R^2\}$. For functions $h : \mathbb{R}^2 \to \mathbb{R}$ and $\varphi : \mathbb{R} \to \mathbb{R}$, a real level $a \in \mathbb{R}$, and positive real numbers L, ε, write

$$B(h; R, a, \varepsilon) = \{x \in D_R : |h(x) - a| < \varepsilon\}, \qquad E(\varphi; L, a, \varepsilon) = \{s \in (0, L) : |\varphi(s) - a| < \varepsilon\}.$$

For $s \geq 0$ and $\theta \in \mathbb{R}$ put

$$p_\theta(s) = (\sqrt{s}\cos\theta, \sqrt{s}\sin\theta), \qquad \varphi_\theta(s) = h(p_\theta(s)).$$

A radial control for h with constants (R, κ, K) consists of $R > 0$, $0 < \kappa \le K$, global continuity of h, $h(0) = 0$, and the following inequalities for every $\theta \in \mathbb{R}$ and every $0 \le s \le t \le R^2$:

$$\frac{\kappa}{2}(t - s) \le \varphi_\theta(t) - \varphi_\theta(s) \le \frac{K}{2}(t - s). \tag{13}$$

These are inequalities about the actual function, not assumptions about its band volumes.

Implementation convention. The compiled implementation represents the plane by `Real ×Real`, with coordinate Lebesgue measure, but defines its Euclidean disks by the displayed sum of squares. It never calls the default product-norm ball a Euclidean disk. The radial-control record includes *global* continuity of h. We include that hypothesis in the natural-language interfaces below as well; every fixed-tail translation of the polynomial O77 loss satisfies it. The integration estimates use only values on the displayed disk, but this fact is not used to weaken the stated record.

```
noncomputable section
open Real MeasureTheory Set
open scoped ENNReal
namespace O77.SaddleDirect

abbrev Plane := Real × Real

def disk (R : Real) : Set Plane :=
  {x | x.1 ^ 2 + x.2 ^ 2 < R ^ 2}

def intervalBand (phi : Real -> Real) (L a eps : Real) : Set Real :=
  {s | 0 < s ∧ s < L ∧ |phi s - a| < eps}

def planarBand (h : Plane -> Real) (R a eps : Real) : Set Plane :=
  {x | x ∈ disk R ∧ |h x - a| < eps}

def ray (h : Plane -> Real) (theta s : Real) : Real :=
  h (Real.sqrt s * Real.cos theta, Real.sqrt s * Real.sin theta)

structure RadialControl (h : Plane -> Real) (R kappa K : Real) : Prop where
  radius_pos : 0 < R
  lower_pos : 0 < kappa
  lower_le_upper : kappa ≤ K
  continuous_h : Continuous h
  at_zero : h (0, 0) = 0
  secants : ∀ theta s, s ∈ Icc (0 : Real) (R ^ 2) ->
    ∀ t, t ∈ Icc (0 : Real) (R ^ 2) -> s ≤ t ->
    (kappa / 2) * (t - s) ≤ ray h theta t - ray h theta s ∧
    ray h theta t - ray h theta s ≤ (K / 2) * (t - s)
```

Lemma 17.2 (An interval inverse controls every translated band)**.**

Corresponding Lean declaration(s): `O77.SaddleDirect.Library.interval_band_volume_bounds`.

Let $L > 0$, $0 < c \le C$, and let $\varphi : [0, L] \to \mathbb{R}$ be continuous, with $\varphi(0) = 0$. Suppose that for $0 \le s \le t \le L$,

$$c(t - s) \le \varphi(t) - \varphi(s) \le C(t - s).$$

For every $a \in \mathbb{R}$ and every $\varepsilon > 0$,

$$|E(\varphi; L, a, \varepsilon)| \le \frac{2\varepsilon}{c}. \tag{14}$$

If additionally $0 < a - \varepsilon$ and $a + \varepsilon < cL$, then

$$|E(\varphi; L, a, \varepsilon)| \ge \frac{2\varepsilon}{C}. \tag{15}$$

Here $|\cdot|$ denotes one-dimensional Lebesgue measure. No derivative of φ at zero is required.

Proof. The lower secant inequality implies strict increase: if $s < t$, then $\varphi(t)-\varphi(s) \geq c(t-s) > 0$. Put $b = \varphi(L)$; then $b \geq cL > 0$. The intermediate-value theorem and strict increase give a unique inverse $\chi : [0, b] \to [0, L]$. In particular $\chi(0) = 0$ and $\chi(b) = L$. The secant bounds, applied to $\chi(u) \leq \chi(v)$, give

$$\frac{v-u}{C} \leq \chi(v) - \chi(u) \leq \frac{v-u}{c} \qquad (0 \leq u \leq v \leq b).$$

Define the clipping map $q_b(x) = \min(b, \max(0, x))$ and put $u = q_b(a-\varepsilon)$, $v = q_b(a+\varepsilon)$. Both $\max(0,\cdot)$ and $\min(b,\cdot)$ are increasing and do not increase a nonnegative difference: this follows by separating the three possibilities that both arguments lie below, straddle, or lie above the clipping endpoint. Thus $0 \leq v-u \leq 2\varepsilon$.

The band E is exactly $(\chi(u), \chi(v))$. To check this even when clipping occurs, note that $0 < s < L$ implies $0 < \varphi(s) < b$. If $a-\varepsilon \leq 0$, its lower band condition is automatic and $\chi(u) = 0$; if $a-\varepsilon \geq b$, the band and the displayed interval are empty. Otherwise the lower band condition is $\chi(a-\varepsilon) < s$. The corresponding three cases at the upper endpoint give the upper interval condition. These verifications also cover equality at 0 or b. Consequently $|E| = \chi(v) - \chi(u) \leq (v-u)/c \leq 2\varepsilon/c$, proving (14).

Under the additional assumptions, $a-\varepsilon > 0$ and $a+\varepsilon < cL \leq b$, so neither endpoint is clipped. Now $v-u = 2\varepsilon$ and the other inverse inequality gives (15). All sets used here are intervals, hence measurable. □

Checked library-level translation. The implementation does not postulate an inverse or a volume oracle. For the upper bound it uses the lower secant inequality to bound the diameter of any two points in the band, then applies `Real.volume_le_diam`. For the lower bound, the strict interior assumptions and the intermediate-value theorem produce points mapped to $a-\varepsilon$ and $a+\varepsilon$; the open interval between them lies in the band, and the upper secant inequality gives its minimum length. The argument uses `Real.volume_Ioo` and is valid without constructing a global inverse function.

```
namespace Library

-- Ordinary interval inverse and length of an open interval.
-- Formalizes the complete proof at dir:interval-band.
theorem interval_band_volume_bounds
    (L c C : Real) (phi : Real -> Real)
    (hL : 0 < L) (hc : 0 < c) (hcC : c ≤ C)
    (hphi : ContinuousOn phi (Icc (0 : Real) L)) (hzero : phi 0 = 0)
    (hsec : ∀ s, s ∈ Icc (0 : Real) L ->
      ∀ t, t ∈ Icc (0 : Real) L -> s ≤ t ->
      c * (t - s) ≤ phi t - phi s ∧ phi t - phi s ≤ C * (t - s))
    (a eps : Real) (heps : 0 < eps) :
    volume (intervalBand phi L a eps) ≤ ENNReal.ofReal (2 * eps / c) ∧
    ((0 < a - eps ∧ a + eps < c * L) ->
      ENNReal.ofReal (2 * eps / C) ≤ volume (intervalBand phi L a eps)) := by
  have hL0 : (0 : Real) ≤ L := hL.le
  have hzeroMem : (0 : Real) ∈ Icc (0 : Real) L := ⟨le_rfl, ⟩hL0
  have hLMem : L ∈ Icc (0 : Real) L := ⟨hL0, ⟩le_rfl
  have hphiL : c * L ≤ phi L := by
    have hs := (hsec 0 hzeroMem L hLMem hL0).1
    simpa only [hzero, sub_zero] using hs
  constructor
  · refine (Real.volume_le_diam (intervalBand phi L a eps)).trans ?_
    apply Metric.ediam_le_of_forall_dist_le
    intro x hx y hy
    change 0 < x ∧ x < L ∧ |phi x - a| < eps at hx
```

```
    change 0 < y ∧ y < L ∧ |phi y - a| < eps at hy
    have hxI : x ∈ Icc (0 : Real) L := ⟨hx.1.le, hx.2.1.⟩le
    have hyI : y ∈ Icc (0 : Real) L := ⟨hy.1.le, hy.2.1.⟩le
    rcases (abs_lt.mp hx.2.2) with ⟨hxlo, ⟩hxhi
    rcases (abs_lt.mp hy.2.2) with ⟨hylo, ⟩hyhi
    by_cases hxy : x ≤ y
    · have hs := (hsec x hxI y hyI hxy).1
      rw [Real.dist_eq, abs_sub_comm, abs_of_nonneg (sub_nonneg.mpr hxy)]
      apply (₀le_div_iff hc).2
      nlinarith
    · have hyx : y ≤ x := le_of_not_ge hxy
      have hs := (hsec y hyI x hxI hyx).1
      rw [Real.dist_eq, abs_of_nonneg (sub_nonneg.mpr hyx)]
      apply (₀le_div_iff hc).2
      nlinarith
  · intro hinterior
    have hC : 0 < C := lt_of_lt_of_le hc hcC
    have hloRange : a - eps ∈ Icc (phi 0) (phi L) := by
      rw [hzero]
      constructor <;> linarith
    have hhiRange : a + eps ∈ Icc (phi 0) (phi L) := by
      rw [hzero]
      constructor <;> linarith
    obtain ⟨xlo, hxloI, ⟩hxloEq :=
      (intermediate_value_Icc hL0 hphi hloRange)
    obtain ⟨xhi, hxhiI, ⟩hxhiEq :=
      (intermediate_value_Icc hL0 hphi hhiRange)
    have hxloPos : 0 < xlo := by
      by_contra h
      have hxloZero : xlo = 0 := le_antisymm (le_of_not_gt h) hxloI.1
      rw [hxloZero, hzero] at hxloEq
      linarith
    have hxhiLt : xhi < L := by
      by_contra h
      have hxhiL : xhi = L := le_antisymm hxhiI.2 (le_of_not_gt h)
      rw [hxhiL] at hxhiEq
      linarith
    have hxloLt : xlo < xhi := by
      by_contra h
      have hge : xhi ≤ xlo := le_of_not_gt h
      have hs := (hsec xhi hxhiI xlo hxloI hge).1
      rw [hxloEq, hxhiEq] at hs
      have hmul : 0 ≤ c * (xlo - xhi) :=
        mul_nonneg hc.le (sub_nonneg.mpr hge)
      linarith
    have hsubset : Ioo xlo xhi ⊆ intervalBand phi L a eps := by
      intro x hx
      have hxI : x ∈ Icc (0 : Real) L :=
        ⟨(hxloPos.trans hx.1).le, (hx.2.trans hxhiLt).⟩le
      have hslo := (hsec xlo hxloI x hxI hx.1.le).1
      have hshi := (hsec x hxI xhi hxhiI hx.2.le).1
      have hmulLo : 0 < c * (x - xlo) := mul_pos hc (sub_pos.mpr hx.1)
      have hmulHi : 0 < c * (xhi - x) := mul_pos hc (sub_pos.mpr hx.2)
      rw [hxloEq] at hslo
      rw [hxhiEq] at hshi
      change 0 < x ∧ x < L ∧ |phi x - a| < eps
      refine ⟨hxloPos.trans hx.1, hx.2.trans hxhiLt, (abs_lt).2 ?⟩_
      constructor <;> linarith
    have hwidth : 2 * eps / C ≤ xhi - xlo := by
      have hs := (hsec xlo hxloI xhi hxhiI hxloLt.le).2
      rw [hxloEq, hxhiEq] at hs
      apply (₀div_le_iff hC).2
```

```
      nlinarith
    calc
      ENNReal.ofReal (2 * eps / C) ≤ ENNReal.ofReal (xhi - xlo) :=
        ENNReal.ofReal_le_ofReal hwidth
      _ = volume (Ioo xlo xhi) := Real.volume_Ioo.symm
      _ ≤ volume (intervalBand phi L a eps) := measure_mono hsubset
-- The ordinary polar formula, followed by the injective change s = r^2.
-- The helper statements expose the measurable-set and Jacobian obligations.
def radialBand (h : Plane -> Real) (theta R a eps : Real) : Set Real :=
  {r | 0 < r ∧ r < R ∧ |h (r * Real.cos theta, r * Real.sin theta) - a| < eps}

lemma radialBand_measurable (h : Plane -> Real) (hh : Continuous h)
    (theta R a eps : Real) : MeasurableSet (radialBand h theta R a eps) := by
  unfold radialBand
  exact (isOpen_lt continuous_const continuous_id).measurableSet.inter
    ((isOpen_lt continuous_id continuous_const).measurableSet.inter
      (isOpen_lt
        ((hh.comp ((continuous_id.mul continuous_const).prodMk
          (continuous_id.mul continuous_const))).sub continuous_const).abs
        continuous_const).measurableSet)

lemma sq_image_radialBand (h : Plane -> Real) (theta R a eps : Real) (hR : 0 < R) :
    (fun r : Real => r ^ 2) '' radialBand h theta R a eps =
      intervalBand (ray h theta) (R ^ 2) a eps := by
  ext s
  constructor
  · rintro ⟨r, hr, ⟩rfl
    rcases hr with ⟨hr0, hrR, ⟩hrband
    refine ⟨sq_pos_of_pos hr0, ?_, ?⟩_
    · exact (₀sq_lt_sq hr0.le hR.le).2 hrR
    · simpa [ray, Real.sqrt_sq_eq_abs, abs_of_pos hr0] using hrband
  · intro hs
    rcases hs with ⟨hs0, hsR, ⟩hsband
    have hsnonneg : 0 ≤ s := hs0.le
    have hsqrtpos : 0 < Real.sqrt s := Real.sqrt_pos.2 hs0
    have hsqrtsq : Real.sqrt s ^ 2 = s := Real.sq_sqrt hsnonneg
    have hsqrtR : Real.sqrt s < R := by
      apply (₀sq_lt_sq (Real.sqrt_nonneg s) hR.le).1
      simpa [hsqrtsq] using hsR
    refine ⟨Real.sqrt s, ⟨hsqrtpos, hsqrtR, ?⟩_, ⟩hsqrtsq
    simpa [ray] using hsband

lemma sq_injOn_radialBand (h : Plane -> Real) (theta R a eps : Real) :
    Set.InjOn (fun r : Real => r ^ 2) (radialBand h theta R a eps) := by
  intro x hx y hy hxy
  exact (₀pow_left_inj hx.1.le hy.1.le (by norm_num : (2:Nat) ≠ 0)).mp hxy


lemma radial_change_raw (h : Plane -> Real) (hh : Continuous h)
    (theta R a eps : Real) (hR : 0 < R) :
    volume (intervalBand (ray h theta) (R ^ 2) a eps) =
      ∫⁻ r in radialBand h theta R a eps, ENNReal.ofReal (2 * r) := by
  have hS := radialBand_measurable h hh theta R a eps
  have hderiv : ∀ r ∈ radialBand h theta R a eps,
      HasFDerivWithinAt (fun x : Real => x ^ 2)
        (ContinuousLinearMap.toSpanSingleton Real (2 * r))
        (radialBand h theta R a eps) r := by
    intro r hr
    simpa using (hasDerivAt_pow 2 r).hasFDerivAt.hasFDerivWithinAt
  have hcv := lintegral_image_eq_lintegral_abs_det_fderiv_mul
    volume hS hderiv (sq_injOn_radialBand h theta R a eps) (fun _ : Real => (1 : ENNReal))
  rw [sq_image_radialBand h theta R a eps hR] at hcv
```

```
  calc
    volume (intervalBand (ray h theta) (R ^ 2) a eps) =
        ∫⁻ x in intervalBand (ray h theta) (R ^ 2) a eps, (1 : ENNReal) :=
      (setLIntegral_one _).symm
    _ = ∫⁻ x in radialBand h theta R a eps,
        ENNReal.ofReal |(ContinuousLinearMap.toSpanSingleton Real (2 * x)).det| * 1 := hcv
    _ = ∫⁻ r in radialBand h theta R a eps, ENNReal.ofReal (2 * r) := by
      apply setLIntegral_congr_fun hS
      intro r hr
      simp [abs_of_pos hr.1]


lemma radial_integral_eq_half_volume (h : Plane -> Real) (hh : Continuous h)
    (theta R a eps : Real) (hR : 0 < R) :
    ∫⁻( r in radialBand h theta R a eps, ENNReal.ofReal r) =
      ENNReal.ofReal (1 / 2 : Real) *
        volume (intervalBand (ray h theta) (R ^ 2) a eps) := by
  have hS := radialBand_measurable h hh theta R a eps
  have hraw := radial_change_raw h hh theta R a eps hR
  have hfactor :
      ∫⁻( r in radialBand h theta R a eps, ENNReal.ofReal (2 * r)) =
        ENNReal.ofReal (2 : Real) *
          ∫⁻( r in radialBand h theta R a eps, ENNReal.ofReal r) := by
    calc
      ∫⁻( r in radialBand h theta R a eps, ENNReal.ofReal (2 * r)) =
          ∫⁻ r in radialBand h theta R a eps,
            ENNReal.ofReal (2 : Real) * ENNReal.ofReal r := by
        apply setLIntegral_congr_fun hS
        intro r hr
        exact ENNReal.ofReal_mul (by norm_num : (0 : Real) ≤ 2)
      _ = ENNReal.ofReal (2 : Real) *
          ∫⁻( r in radialBand h theta R a eps, ENNReal.ofReal r) := by
        apply lintegral_const_mul
        fun_prop
  rw [hraw, hfactor, ← mul_assoc]
  have hhalf : ENNReal.ofReal (1 / 2 : Real) * ENNReal.ofReal (2 : Real) = 1 := by
    rw [ENNReal.ofReal_div_of_pos (by norm_num : (0 : Real) < 2)]
    simp only [ENNReal.ofReal_one, ENNReal.ofReal_ofNat, one_div]
    exact ENNReal.inv_mul_cancel (a := (2 : ENNReal)) (by norm_num) (by norm_num)
  rw [hhalf, one_mul]


lemma polar_symm_mem_planarBand_iff (h : Plane -> Real)
    (theta R a eps r : Real) (hR : 0 < R) (hr : 0 < r) :
    polarCoord.symm (r, theta) ∈ planarBand h R a eps ↔
      r ∈ radialBand h theta R a eps := by
  have htrig :
      (r * Real.cos theta) ^ 2 + (r * Real.sin theta) ^ 2 = r ^ 2 := by
    calc
      (r * Real.cos theta) ^ 2 + (r * Real.sin theta) ^ 2 =
          r ^ 2 * (Real.cos theta ^ 2 + Real.sin theta ^ 2) := by ring
      _ = r ^ 2 := by rw [Real.cos_sq_add_sin_sq]; ring
  simp only [polarCoord_symm_apply, planarBand, disk, radialBand, mem_ofPred_eq]
  rw [htrig]
  constructor
  · rintro ⟨hrsq, ⟩hband
    exact ⟨hr, (₀sq_lt_sq hr.le hR.le).1 hrsq, ⟩hband
  · rintro ⟨_, hrR, ⟩hband
    exact ⟨(₀sq_lt_sq hr.le hR.le).2 hrR, ⟩hband


lemma polar_radial_integral (h : Plane -> Real) (hh : Continuous h)
```

```
    (theta R a eps : Real) (hR : 0 < R) :
    ∫⁻( r in Ioi (0 : Real), ENNReal.ofReal r •
        (planarBand h R a eps).indicator 1 (polarCoord.symm (r, theta))) =
      ∫⁻ r in radialBand h theta R a eps, ENNReal.ofReal r := by
  have hIoi : MeasurableSet (Ioi (0 : Real)) := measurableSet_Ioi
  have hS := radialBand_measurable h hh theta R a eps
  calc
    ∫⁻( r in Ioi (0 : Real), ENNReal.ofReal r •
        (planarBand h R a eps).indicator 1 (polarCoord.symm (r, theta))) =
        ∫⁻ r in Ioi (0 : Real),
          (radialBand h theta R a eps).indicator
            (fun r => ENNReal.ofReal r) r := by
      apply setLIntegral_congr_fun hIoi
      intro r hr
      have hr0 : 0 < r := hr
      have hiff := polar_symm_mem_planarBand_iff h theta R a eps r hR hr0
      by_cases hp : polarCoord.symm (r, theta) ∈ planarBand h R a eps
      · have hs : r ∈ radialBand h theta R a eps := hiff.mp hp
        have hp' : (r * Real.cos theta, r * Real.sin theta) ∈
            planarBand h R a eps := by simpa using hp
        change ENNReal.ofReal r •
            (planarBand h R a eps).indicator 1
              (r * Real.cos theta, r * Real.sin theta) = _
        rw [Set.indicator_of_mem hp', Set.indicator_of_mem hs]
        simp [smul_eq_mul]
      · have hs : Not (r ∈ radialBand h theta R a eps) := by
          intro hs
          exact hp (hiff.mpr hs)
        have hp' : Not ((r * Real.cos theta, r * Real.sin theta) ∈
            planarBand h R a eps) := by simpa using hp
        change ENNReal.ofReal r •
            (planarBand h R a eps).indicator 1
              (r * Real.cos theta, r * Real.sin theta) = _
        rw [Set.indicator_of_notMem hp', Set.indicator_of_notMem hs]
        simp
    _ = ∫⁻ r in radialBand h theta R a eps ∩ Ioi (0 : Real),
          ENNReal.ofReal r := setLIntegral_indicator hS _
    _ = ∫⁻ r in radialBand h theta R a eps, ENNReal.ofReal r := by
      rw [inter_eq_left.mpr]
      intro r hr
      exact hr.1
```

Lemma 17.3 (Squared-radius polar slicing)**.**

Corresponding Lean declaration(s): `O77.SaddleDirect.Library.radialBand`,`O77.SaddleDirect.Library.radialBand_measurable`,`O77.SaddleDirect.Library.sq_image_radialBand`,`O77.SaddleDirect.Library.sq_injOn_radialBand`,`O77.SaddleDirect.Library.radial_change_raw`,`O77.SaddleDirect.Library.radial_integral_eq_half_volume`.

Corresponding Lean declaration(s): `O77.SaddleDirect.Library.polar_symm_mem_planarBand_iff`,`O77.SaddleDirect.Library.polar_radial_integral`,`O77.SaddleDirect.Library.squarePolar_measure`,`O77.SaddleDirect.Library.angular_constant`.

For h continuous on $\overline{D}_R$, $R > 0$, $a \in \mathbb{R}$, and $\varepsilon > 0$,

$$\operatorname{vol}_2 B(h; R, a, \varepsilon) = \frac{1}{2} \int_{-\pi}^{\pi} |E(\varphi_\theta; R^2, a, \varepsilon)| \, d\theta. \tag{16}$$

All integrations may first be read as nonnegative Lebesgue integrals. In particular the formula does not require prior integrability of an unknown density of h.

Proof. The set whose area is measured is measurable and contained in D_R. Apply the ordinary polar-coordinate formula to its indicator; the excluded ray and the origin have planar measure zero. In coordinates (r, θ), the area integral is

$$\int_{-\pi}^{\pi} \int_{0}^{R} \mathbf{1}_{\{|h(r\cos\theta, r\sin\theta) - a| < \varepsilon\}}\, r\, dr\, d\theta.$$

For fixed θ, substitute $s = r^2$ on $(0, R)$, using $ds = 2r\, dr$. One can perform this substitution first on compact subintervals of $(0, R)$ and pass to the whole interval by monotone convergence; no differentiability of $\sqrt{s}$ at zero is needed. The inner integral becomes one half of the length of $E(\varphi_\theta; R^2, a, \varepsilon)$. Tonelli applies to the measurable nonnegative integrand. Finiteness follows alternatively from containment in the disk. No measurable choice of inverse endpoints depending on θ is used. □

The two coordinate measure identities below are checked against the pinned API. The implementation uses `lintegral_comp_polarCoord_symm` from the module `Mathlib.Analysis.SpecialFunctions.PolarCoord`. It is a polar formula on `Real×Real`, with radial Jacobian r, not the squared-radius formula above. The implementation performs the substitution $s = r^2$ with the general Jacobian theorem and proves the disk/radial-set identification explicitly. The existing product/Jacobian references cover the measure tools [14, 15]. The precise inspected source is recorded here without changing the bibliography:

https://github.com/leanprover-community/mathlib4/blob/664cbbe52927ce6acaa39ca35498336116e123b8/Mathlib/Analysis/SpecialFunctions/PolarCoord.lean

```
theorem squarePolar_measure (h : Plane -> Real) (hh : Continuous h)
    (R a eps : Real) (hR : 0 < R) :
    volume (planarBand h R a eps) =
      ∫⁻ theta in Ioo (-Real.pi) Real.pi,
        ENNReal.ofReal (1 / 2 : Real) *
          volume (intervalBand (ray h theta) (R ^ 2) a eps) := by
  have hmeas : MeasurableSet (planarBand h R a eps) := by
    unfold planarBand disk
    exact (isOpen_lt (by fun_prop) continuous_const).measurableSet.inter
      (isOpen_lt ((hh.sub continuous_const).abs) continuous_const).measurableSet
  rw ←[ lintegral_indicator_one hmeas]
  rw ←[ lintegral_comp_polarCoord_symm]
  rw [polarCoord_target]
  change ∫⁻( (p : Real × Real) in Ioi 0 ×ˢ Ioo (-Real.pi) Real.pi,
      ENNReal.ofReal p.1 •
        (planarBand h R a eps).indicator 1 (polarCoord.symm p)
      ∂(volume.prod volume)) = _
  rw [setLIntegral_prod_symm]
  · apply setLIntegral_congr_fun measurableSet_Ioo
    intro theta htheta
    change ∫⁻( r in Ioi (0 : Real), ENNReal.ofReal r •
        (planarBand h R a eps).indicator 1 (polarCoord.symm (r, theta))) = _
    rw [polar_radial_integral h hh theta R a eps hR]
    exact radial_integral_eq_half_volume h hh theta R a eps hR
  · fun_prop


-- Integrate a constant over the angle interval of length 2*pi.
theorem angular_constant (b : Real) (hb : 0 ≤ b) :
    ∫⁻( _theta in Ioo (-Real.pi) Real.pi,
      ENNReal.ofReal (1 / 2 : Real) * ENNReal.ofReal b) =
        ENNReal.ofReal (Real.pi * b) := by
  rw [setLIntegral_const, Real.volume_Ioo]
  rw ←[ ENNReal.ofReal_mul (by norm_num : (0 : Real) ≤ 1 / 2)]
```

```
  rw ←[ ENNReal.ofReal_mul (mul_nonneg (by norm_num) hb)]
  congr 1
  ring

end Library
```

17.2 The uniform planar estimate, with its lower-bound window

Theorem 17.4 (Every translated band has bounded area density)**.**

Corresponding Lean declaration(s): `077.SaddleDirect.planarBand_bounds`.

Assume radial control for h with constants (R, κ, K). For every *real shift a and every $\varepsilon > 0$,*

$$\operatorname{vol}_2 B(h; R, a, \varepsilon) \le \frac{4\pi}{\kappa}\varepsilon. \tag{17}$$

If also $0 < a - \varepsilon$ and $a + \varepsilon < \kappa R^2/2$, then

$$\operatorname{vol}_2 B(h; R, a, \varepsilon) \ge \frac{4\pi}{K}\varepsilon. \tag{18}$$

The upper constant is independent of a. The lower assertion is not made when the band misses the interior energy range of the disk.

Proof. Fix θ. The function φ_θ is continuous on $[0, R^2]$, vanishes at zero, and has lower and upper secant slopes $\kappa/2$ and $K/2$. Lemma 17.2 therefore gives a length at most $4\varepsilon/\kappa$. If the additional interior condition holds, it gives a length at least $4\varepsilon/K$. These bounds hold for every angle with the same constants. Substitute them in (16); the angle interval has length 2π and the radial substitution contributes $1/2$. This gives exactly (17) and (18). The integrands are nonnegative, so integral monotonicity applies before any conversion of an extended nonnegative measure to a real number. □

The following proof and all of its supporting adapter dependencies compile without `sorryAx`. Neither the measure conclusion nor its uniformity in a is a field of `RadialControl`; both are derived from the secant hypotheses and the checked measure identities.

```
theorem planarBand_bounds (h : Plane -> Real) (R kappa K : Real)
    (hc : RadialControl h R kappa K) (a eps : Real) (heps : 0 < eps) :
    volume (planarBand h R a eps) ≤
        ENNReal.ofReal (4 * Real.pi * eps / kappa) ∧
    ((0 < a - eps ∧ a + eps < (kappa / 2) * R ^ 2) ->
      ENNReal.ofReal (4 * Real.pi * eps / K) ≤
        volume (planarBand h R a eps)) := by
  have hK : 0 < K := lt_of_lt_of_le hc.lower_pos hc.lower_le_upper
  have hR2 : 0 < R ^ 2 := sq_pos_of_pos hc.radius_pos
  have hkne : kappa ≠ 0 := ne_of_gt hc.lower_pos
  have hKne : K ≠ 0 := ne_of_gt hK
  have hdiv1 : 2 * eps / (kappa / 2) = 4 * eps / kappa := by
    field_simp [hkne]
    ring
  have hdiv2 : 2 * eps / (K / 2) = 4 * eps / K := by
    field_simp [hKne]
    ring
  have hslices : ∀ theta : Real,
      volume (intervalBand (ray h theta) (R ^ 2) a eps) ≤
          ENNReal.ofReal (4 * eps / kappa) ∧
      ((0 < a - eps ∧ a + eps < (kappa / 2) * R ^ 2) ->
        ENNReal.ofReal (4 * eps / K) ≤
          volume (intervalBand (ray h theta) (R ^ 2) a eps)) := by
    intro theta
```

```
    have hpath : Continuous (fun s : Real =>
        (Real.sqrt s * Real.cos theta, Real.sqrt s * Real.sin theta)) :=
      (Real.continuous_sqrt.mul continuous_const).prodMk
        (Real.continuous_sqrt.mul continuous_const)
    have hcont : ContinuousOn (ray h theta) (Icc (0 : Real) (R ^ 2)) := by
      unfold ray
      exact (hc.continuous_h.comp hpath).continuousOn
    have hz : ray h theta 0 = 0 := by
      simpa [ray] using hc.at_zero
    have hb := Library.interval_band_volume_bounds
      (R ^ 2) (kappa / 2) (K / 2) (ray h theta)
      hR2 (div_pos hc.lower_pos (by norm_num)) (by linarith [hc.lower_le_upper])
      hcont hz (hc.secants theta) a eps heps
    simpa only [hdiv1, hdiv2] using hb
  constructor
  · calc
      volume (planarBand h R a eps) =
          ∫⁻ theta in Ioo (-Real.pi) Real.pi,
            ENNReal.ofReal (1 / 2 : Real) *
              volume (intervalBand (ray h theta) (R ^ 2) a eps) :=
        Library.squarePolar_measure h hc.continuous_h R a eps hc.radius_pos
      _ ≤ ∫⁻ _theta in Ioo (-Real.pi) Real.pi,
          ENNReal.ofReal (1 / 2 : Real) *
            ENNReal.ofReal (4 * eps / kappa) := by
        apply lintegral_mono
        intro theta
        exact mul_le_mul_of_nonneg_left (hslices theta).1 (by exact zero_le)
      _ = ENNReal.ofReal (4 * Real.pi * eps / kappa) := by
        rw [Library.angular_constant _ (le_of_lt (div_pos (mul_pos (by norm_num) heps) hc.lower_pos))]
        congr 1
        ring
  · intro hinterior
    calc
      ENNReal.ofReal (4 * Real.pi * eps / K) =
          ∫⁻ _theta in Ioo (-Real.pi) Real.pi,
            ENNReal.ofReal (1 / 2 : Real) *
              ENNReal.ofReal (4 * eps / K) := by
        rw [Library.angular_constant _ (le_of_lt (div_pos (mul_pos (by norm_num) heps) hK))]
        congr 1
        ring
      _ ≤ ∫⁻ theta in Ioo (-Real.pi) Real.pi,
          ENNReal.ofReal (1 / 2 : Real) *
            volume (intervalBand (ray h theta) (R ^ 2) a eps) := by
        apply lintegral_mono
        intro theta
        exact mul_le_mul_of_nonneg_left ((hslices theta).2 hinterior) (by exact zero_le)
      _ = volume (planarBand h R a eps) :=
        (Library.squarePolar_measure h hc.continuous_h
          R a eps hc.radius_pos).symm

end O77.SaddleDirect
end
```

17.3 Obtaining radial control from a positive Hessian plane

Lemma 17.5 (Uniform two-variable convexity and the critical center)**.** *Let $d \in \mathbb{N}$ and let $f : \mathbb{R}^2 \times \mathbb{R}^d \to \mathbb{R}$ be globally continuous and C^2 on an open neighbourhood of $(0,0)$. For $S > 0$, write $B_d(0,S) = \{z \in \mathbb{R}^d : \|z\| < S\}$. The notation D_y denotes differentiation in the first, planar variable. Assume $D_y f(0,0) = 0$ and that $D^2_{yy} f(0,0)$ is positive definite. There are numbers $R, S > 0$, $0 < \kappa \le K$, and a C^1 map $c : B_d(0,S) \to \mathbb{R}^2$, with $c(0) = 0$, such that the*

following hold:

1. *The closed product* $\overline{D}_{2R} \times \overline{B}_d(0,S)$ *lies in the domain of* f*, and throughout it,* $\kappa\|v\|^2 \le D^2_{yy} f(y,z)[v,v] \le K\|v\|^2$ *for every* $v \in \mathbb{R}^2$*.*
2. *For* $\|z\| < S$*,* $D_y f(c(z),z) = 0$ *and* $\|c(z)\| < R/2$*.*
3. *For each* $z \in B_d(0,S)$ *put* $g(z) = f(c(z),z) - f(0,0)$*, and define the function* $h_z : \mathbb{R}^2 \to \mathbb{R}$ *by* $h_z(x) = f(c(z)+x,z) - f(c(z),z)$*. Then* $g : B_d(0,S) \to \mathbb{R}$ *is continuous,* $g(0) = 0$*, and* h_z *has radial control with* (R,κ,K) *for every* $z \in B_d(0,S)$*. Global continuity of* h_z *follows from that of* f*.*

The choices of R, S, κ, K *are made once, before a radius used in the band-pair predicate.*

Proof. On the unit circle the continuous quadratic form $D^2_{yy} f(0,0)[v,v]$ has a positive minimum m and a finite maximum M. Positivity of the minimum follows from positive definiteness and compactness: if the minimum were zero, it would be attained at a nonzero unit vector. Continuity of the Hessian as a bilinear form allows a neighbourhood on which its operator-norm difference from the base Hessian is less than $m/2$. Hence the desired bounds hold with $\kappa = m/2$ and $K = M + m/2$. Choose a closed product neighbourhood inside that neighbourhood and inside the original open domain; shrinking it preserves the same constants.

Identify the partial differential $D_y f$ with its two gradient coordinates and apply the ordinary finite-dimensional C^1 implicit-function theorem to the equation $D_y f(y,z) = 0$. Its derivative in y at zero is the invertible positive definite Hessian. We obtain a C^1 critical center c, with $c(0) = 0$, on some open neighbourhood in the z variables. Shrink S so that this map is defined on a neighbourhood of $\overline{B}_d(0,S)$ and $\|c(z)\| < R/2$ there. Continuity of g and $g(0) = 0$ follow by composition.

Fix z and a unit vector $\theta \in \mathbb{R}^2$. For $0 \le r \le R$ let $u(r) = h_z(r\theta)$. The segment lies in the region where the Hessian bounds hold, because $\|c(z) + r\theta\| < 3R/2 < 2R$. The chain rule gives $u'(0) = 0$ and $\kappa \le u''(r) \le K$. The fundamental theorem of calculus therefore gives $\kappa r \le u'(r) \le Kr$, and, for $0 \le a \le b \le R$,

$$\frac{\kappa}{2}(b^2 - a^2) \le u(b) - u(a) \le \frac{K}{2}(b^2 - a^2).$$

Taking $a = \sqrt{s}$ and $b = \sqrt{t}$ proves (13). Only differentiation along this fixed line and two ordinary one-dimensional integrals have been used. There is no differentiation of a parameter-dependent remainder integral. The formulas also give $h_z(x) \ge \kappa\|x\|^2/2 \ge 0$ for $\|x\| \le R$. □

The implemented derivative-to-center interface. The local regularity hypotheses are used on one common neighborhood. `O77SaddlePreparation` applies the implicit-function API to the actual partial gradient and derives squared-radius secants from bounds on the first and second derivatives along each line. In Lean the scalar argument uses derivative bounds and monotonicity of the functions obtained by subtracting the bounding quadratic polynomials; the two integrations in the ordinary proof express the same inequalities. `O77SaddleAttachment` constructs these derivative hypotheses for the native matrix loss on a common product neighborhood. The resulting centered-family record stores the actual center, its bounds, and pointwise secants; it stores no band-volume conclusion. The relevant library interfaces are documented in [19, 23].

Lemma 17.6 (Lower values pass to the critical-center value)**.** *In the preceding setting, assume that for every neighbourhood of* $(0,0)$ *there is a point* (y,z) *with* $f(y,z) < f(0,0)$*. Then for every* $\eta \in (0,S)$ *there is* z *with* $\|z\| < \eta$ *and* $g(z) < 0$*.*

Proof. Choose a lower-value point satisfying $\|z\| < \eta$ and $\|y\| < R/2$; both conditions define a neighbourhood of zero. Since $\|c(z)\| < R/2$, we have $\|y - c(z)\| < R$. The last inequality in

the proof of Lemma 17.5 implies

$$f(c(z), z) \le f(y, z) < f(0, 0).$$

Thus $g(z) < 0$ as required. This uses no sign hypothesis on g away from the selected points. If $d = 0$ the hypotheses cannot all hold: the center is zero and the same convexity inequality prevents lower values. No zero-dimensional measure is nevertheless discarded in any of the surrounding lemmas. □

17.4 The full-dimensional estimate, with the radius fixed first

Theorem 17.7 (A positive plane and nearby lower values force pair $(1, 1)$)**.** *Let $d \in \mathbb{N}$ and let $f : \mathbb{R}^2 \times \mathbb{R}^d \to \mathbb{R}$ be globally continuous and C^2 near $(0, 0)$. Assume $D_y f(0, 0) = 0$ and $D^2_{yy} f(0, 0)[v, v] > 0$ for every $v \in \mathbb{R}^2 \setminus \{0\}$. If for every $\rho > 0$ there is (y, z) with $\|y\|^2 + \|z\|^2 < \rho^2$ and $f(y, z) < f(0, 0)$, then, with the* full Euclidean coordinate measure*,*

$$\mathsf{BandPair}(f, (0, 0); 1, 1).$$

More explicitly, there is $\delta_0 > 0$ such that, for every fixed $0 < \delta < \delta_0$, there are $0 < c_\delta \le C_\delta < \infty$ and $0 < \varepsilon_\delta < e^{-1}$ with

$$c_\delta \varepsilon \le \mathrm{vol}_{d+2}\{(y, z) \in B_{d+2}(0, \delta) : |f(y, z) - f(0, 0)| < \varepsilon\} \le C_\delta \varepsilon \qquad (0 < \varepsilon < \varepsilon_\delta).$$

Neither constant depends on ε. No measure on a transverse slice is substituted for the full-dimensional measure.

Proof. Take the data of Lemma 17.5 and set $\delta_0 = \min(R/2, S)/2 > 0$. Fix $0 < \delta < \delta_0$ for the remainder of the proof.

Upper bound. For a point of the ambient ball, $\|y\| < \delta$ and $\|z\| < \delta < S$. Thus $\|y - c(z)\| < \delta + R/2 < R$. At fixed z the planar section of the band is, after translation by $-c(z)$, contained in $B(h_z; R, -g(z), \varepsilon)$. Lebesgue translation invariance and (17) bound its area by $4\pi\varepsilon/\kappa$ for every $\varepsilon > 0$, independently of z and of the real number $-g(z)$. The original band is measurable, since f is continuous on a neighbourhood of the ball. Tonelli, applied to its indicator, now gives the upper bound with

$$C_\delta = \frac{4\pi}{\kappa} \mathrm{vol}_d B_d(0, \delta). \tag{19}$$

This is a finite positive constant, including the empty-product convention in dimension zero.

A fixed lower integration window. Put $r_\delta = \delta/4$. By continuity of c and g at zero, choose $0 < \eta < \min(S, \delta/4)$ so that for every $\|z\| < \eta$,

$$\|c(z)\| < \delta/4, \qquad |g(z)| < \kappa r_\delta^2/16.$$

Lemma 17.6 supplies z_0 with $\|z_0\| < \eta/2$ and $g(z_0) < 0$. Set $a_0 = -g(z_0) > 0$. In particular $a_0 < \kappa r_\delta^2/16$. Continuity at z_0 gives $\nu > 0$ so that the ball $\Omega = B_d(z_0, \nu)$ is contained in $B_d(0, \eta)$ and

$$a_0/2 < -g(z) < 3a_0/2 \qquad (z \in \Omega).$$

For example intersect the neighbourhood supplied by the last strict inequalities with $B_d(0, \eta)$ and choose a ball about z_0 contained in the intersection. Its measure is positive and finite. Crucially, z_0, a_0, ν, and Ω depend on δ but are all chosen before ε.

Lower bound. Let $0 < \varepsilon < a_0/4$. At each $z \in \Omega$ put $a = -g(z)$. Then

$$0 < a_0/4 < a - \varepsilon, \qquad a + \varepsilon < 7a_0/4 < \kappa r_\delta^2/2.$$

Radial control remains valid on the smaller disk of radius $r_\delta < R$. Equation (18) therefore gives

$$\mathrm{vol}_2\{y : \|y - c(z)\| < r_\delta, |f(y,z) - f(0,0)| < \varepsilon\} \geq 4\pi\varepsilon/K.$$

Every point of these sections lies in the ambient δ-ball: indeed $\|y\| < r_\delta + \|c(z)\| < \delta/2$ and $\|z\| < \eta < \delta/4$, whence $\|(y,z)\|^2 < \delta^2/4 + \delta^2/16 < \delta^2$. Integrating the section inequality over the fixed measurable set Ω yields

$$\mathrm{vol}_{d+2}(\text{ambient band}) \geq \widetilde{c}_\delta \varepsilon, \qquad \widetilde{c}_\delta = \frac{4\pi}{K}\,\mathrm{vol}_d\,\Omega > 0. \tag{20}$$

Only sectionwise translations and Tonelli have been used. There is no need to compute the Jacobian of a nonlinear map $(x,z) \mapsto (c(z)+x, z)$. Finally take $c_\delta = \min(\widetilde{c}_\delta, C_\delta) > 0$ and $\varepsilon_\delta = \min(a_0/4, e^{-1}/2) > 0$. The upper estimate holds for all $\varepsilon > 0$ and the lower one holds below this threshold. Since $G_{1,1}(\varepsilon) = \varepsilon$, these are exactly the quantified bounds in Definition 2.2. □

17.5 Application at every O77 saddle point

Lemma 17.8 (A negative Hessian direction produces nearby lower values)**.** *Let E be a finite-dimensional real Euclidean space, $w \in E$, and $f : E \to \mathbb{R}$ a function that is C^2 on an open neighbourhood of w. Assume $Df(w) = 0$ and let $v \in E$ satisfy $D^2 f(w)[v,v] < 0$. Then for every $\rho > 0$ there exists $x \in E$ with $\|x - w\| < \rho$ and $f(x) < f(w)$.*

Proof. The vector v is nonzero, since a bilinear form vanishes on $(0,0)$. Put $\gamma = -D^2 f(w)[v,v]/2 > 0$. Taylor's theorem along the line gives $f(w+tv) - f(w) = -\gamma t^2 + o(t^2)$ as $t \to 0$. By the definition of $o(t^2)$, for some $t_0 > 0$ and $0 < |t| < t_0$ the remainder has absolute value at most $\gamma t^2/2$. Then $f(w+tv) - f(w) \leq -\gamma t^2/2 < 0$. Given any ball of radius $\rho > 0$ about w, choose $0 < t < \min(t_0, \rho/\|v\|)$; the point $w + tv$ lies in that ball and has smaller value. □

Corollary 17.9 (A C^2 strict saddle with a positive plane)**.** *Let E be a real Euclidean space of finite dimension $n \geq 2$, let $w \in E$, and let $f : E \to \mathbb{R}$ be globally continuous and C^2 near w, with $Df(w) = 0$. Suppose there are a two-dimensional linear subspace $P \subset E$ and a vector $v \in E$ such that $D^2 f(w)[x,x] > 0$ for every $x \in P \setminus \{0\}$ and $D^2 f(w)[v,v] < 0$. Then* $\mathsf{BandPair}(f, w; 1, 1)$*, for the Euclidean norm and Lebesgue measure of E.*

Proof. Choose an orthonormal basis of the positive plane for the original Euclidean inner product, not for the Hessian, and extend it to an orthonormal basis of the ambient space. This gives a linear isometry from $\mathbb{R}^2 \times \mathbb{R}^{n-2}$ with its Euclidean product norm to the ambient space. Translate w to zero and precompose with that map. It preserves coordinate Lebesgue measure and Euclidean balls (or use its determinant of absolute value one). At zero the pulled-back function has zero partial gradient in the planar variable and a positive definite partial Hessian in that same variable. Lemma 17.8 supplies its nearby lower values. Apply Theorem 17.7 and transport the resulting band sets by the affine isometry. No Hessian-orthogonal complement and no nonlinear normal form are required. □

Theorem 17.10 (O77(b) by direct planar convexity)**.** *Let $I, O \geq 1$, let $H, r \in \mathbb{N}$ satisfy $r < H$, and let $\Phi \in \mathbb{R}^{O \times I}$, the singular vectors, and the positive singular values satisfy Definition 2.6. For every integer k with $0 \leq k < r$ and every $w \in C_k$,*

$$\mathsf{BandPair}(L_\Phi, w; 1, 1).$$

Every such rung is nonempty. This proof has no partial-Morse-splitting premise.

Proof. If $r = 0$ there is no $k < r$, so the quantified assertion is vacuous. Otherwise fix arbitrary $k < r$ and arbitrary $w = (A, B) \in C_k$. Then $H > r \geq 1$ implies $H \geq 2$. The loss is a polynomial, hence C^2, and w is critical by Lemma 10.1. Choose any discarded mode $j > k$ with coefficient $\sigma_j > 0$. For the Hessian restriction of Lemma 10.2, the subspace $(x, y) = (x, -x)$ is positive definite:

$$Q(x, -x) = \|Bx\|^2 + \|A^\mathsf{T} x\|^2 + 2\sigma_j \|x\|^2 \geq 2\sigma_j \|x\|^2.$$

Choose two orthonormal hidden vectors e_1, e_2. The corresponding perturbations

$$x \longmapsto \frac{1}{\sqrt{2}}(x v_j^\mathsf{T}, -u_j x^\mathsf{T}), \qquad x \in \operatorname{span}(e_1, e_2),$$

form a Euclidean two-plane in the full parameter space; its Hessian is bounded below by $\sigma_j \|x\|^2$ in these isometric coordinates. Lemma 10.3 supplies a negative vector, since $\operatorname{rank}(BA) = k < H$. That lemma uses the kernel alternative and an explicit small positive multiple; it does not assume balanced factors or a generic point. Corollary 17.9 therefore applies at this w. All choices were made after fixing an arbitrary point, and its conclusion has the all-small-radii quantifier. Thus the result holds at every point of every required rung. Nonemptiness is already established by the explicit factorization in Lemma 10.1. □

Composition with the fiber branch. The ordinary argument just given is implemented at arbitrary rung points in `O77SaddleAttachment` and assembled with rung inhabitation in `O77RungCompletion`. The six-clause contract in `O77Answer` combines this result with the measured fiber branch. The final module supplies the Wishart theorem to that intermediate conditional assembly.

17.6 Why the hypotheses and the constants matter

Proposition 17.11 (Two sharp failure controls)**.** *A single positive Hessian direction is insufficient for the pair* $(1, 1)$*, and a positive plane without nearby lower values is insufficient for its lower bound.*

Proof. For $f(x, y) = x^2 - y^2$, set $u = x - y$ and $v = x + y$. The inverse linear change has absolute determinant $1/2$ and $f = uv$. In the window corresponding to $|u|, |v| < R$, direct integration for $0 < \varepsilon < R^2$ gives

$$\operatorname{vol}\{|uv| < \varepsilon\} \text{ in the } (x, y) \text{ window} = \frac{1}{2}\, 4 \left(\int_0^{\varepsilon/R} R \, du + \int_{\varepsilon/R}^{R} \frac{\varepsilon}{u} \, du \right) = 2\varepsilon\big(1 + \log(R^2/\varepsilon)\big).$$

The windows are comparable to Euclidean balls, so the local pair is $(1, 2)$, not $(1, 1)$. For $f(x, y, z) = x^2 + y^2 + z^2$, the positive plane is present, but there are no lower values. For $0 < \varepsilon < \delta^2$ the band in the δ-ball is the ball of radius $\sqrt{\varepsilon}$; its volume is $(4\pi/3)\varepsilon^{3/2}$. It cannot be bounded below by $c_\delta \varepsilon$ with $c_\delta > 0$ for all small ε. These are actual volume computations, not merely mutations of proposed Lean definitions. □

A broader consequence, not needed for O77. Theorem 17.7 only needs nearby lower values, not a negative Hessian eigenvalue. Thus it also applies to $x^2 + y^2 - z^4$ at zero: the negative direction may first appear at fourth order. This example explains why proving just a nondegenerate quadratic normal form would be stronger than necessary for the volume conclusion.

18 Full-dimensional saddle-band assembly

This section proves the quantifier-sensitive implication from a centered family of planar functions to the band volume in the *whole* parameter space. The planar estimate of Theorem 17.4 is used, not reproved. Fubini–Tonelli, translation invariance of Lebesgue measure, positivity of open-set measure, and compact-set finiteness are used through their existing Mathlib declarations. No new integration theory, Morse lemma, or matrix library is built in this section.

The endpoint in this chapter has explicit pointwise center and radial hypotheses. This is a reusable theorem about planar families. Its O77 client constructs those hypotheses from the polynomial loss; the final answer does not assume their existence.

18.1 A concrete ambient measure and a pointwise geometric interface

Definition 18.1 (Ambient bands and centered functions)**.**

Corresponding Lean declaration(s): `O77.SaddleDirect.Tail`,`O77.SaddleDirect.planeSq`,`O77.SaddleDirect.ambientSq`,`O77.SaddleDirect.ambientBand`,`O77.SaddleDirect.centerValue`,`O77.SaddleDirect.centeredSlice`,`O77.SaddleDirect.translatedBand`,`O77.SaddleDirect.LowerValuesAccumulate`,`O77.SaddleDirect.CenterNegativeAccumulate`.

Fix $d \in \mathbb{N}$. Write $E_d = \mathbb{R}^d$ with its Euclidean norm and coordinate Lebesgue measure, and write $P = \mathbb{R} \times \mathbb{R}$ for the plane. The total space $P \times E_d$ has product coordinate Lebesgue measure. For a function $f : P \times E_d \to \mathbb{R}$, real numbers $\delta, \varepsilon > 0$, and a point $(y, z) \in P \times E_d$, define

$$q_P(y) = y_1^2 + y_2^2, \qquad q_W(y, z) = q_P(y) + \|z\|^2,$$
$$V_f(\delta, \varepsilon) = \mathrm{vol}\{w : q_W(w) < \delta^2,\ |f(w) - f(0,0)| < \varepsilon\}.$$

For $c : E_d \to P$, set

$$g(z) = f(c(z), z) - f(0,0), \qquad h_z(x) = f(c(z) + x, z) - f(c(z), z).$$

For each $z \in E_d$ and each $r, \varepsilon > 0$, the translated planar section is $T_z(r, \varepsilon) = \{y \in P : q_P(y - c(z)) < r^2,\ |f(y, z) - f(0,0)| < \varepsilon\}$. Let `LowerValuesAccumulate` mean that every $t > 0$ admits a point w with $q_W(w) < t^2$ and $f(w) < f(0,0)$. Let `CenterNegativeAccumulate` mean that every $\eta > 0$ admits z with $\|z\| < \eta$ and $g(z) < 0$.

The use of q_W is deliberate: the default Lean norm on a product need not be the Euclidean product norm. No theorem below equates those norms. The measurable space and measure are the actual standard products; flattening these coordinates into one `EuclideanSpace` is a later coordinate identification, not a change of invariant. In the formal statements f is globally continuous. This is a legitimate specialization for the polynomial O77 loss after linear coordinates. The local C^2 version remains Theorem 17.7; no arbitrary continuous extension of a local f is claimed here.

```
noncomputable section
open MeasureTheory Set Filter
open scoped ENNReal Topology
namespace O77.SaddleDirect

abbrev Tail (d : Nat) := EuclideanSpace Real (Fin d)

def planeSq (y : Plane) : Real := y.1 ^ 2 + y.2 ^ 2

def ambientSq {d : Nat} (w : Plane × Tail d) : Real :=
  planeSq w.1 + norm w.2 ^ 2

def ambientBand {d : Nat} (f : Plane × Tail d -> Real)
```

```
    (delta eps : Real) : Set (Plane × Tail d) :=
  {w | ambientSq w < delta ^ 2 ∧ |f w - f (0, 0)| < eps}

def centerValue {d : Nat} (f : Plane × Tail d -> Real)
    (c : Tail d -> Plane) (z : Tail d) : Real :=
  f (c z, z) - f (0, 0)

def centeredSlice {d : Nat} (f : Plane × Tail d -> Real)
    (c : Tail d -> Plane) (z : Tail d) (x : Plane) : Real :=
  f (c z + x, z) - f (c z, z)

def translatedBand {d : Nat} (f : Plane × Tail d -> Real)
    (c : Tail d -> Plane) (z : Tail d) (R eps : Real) : Set Plane :=
  {y | planeSq (y - c z) < R ^ 2 ∧ |f (y, z) - f (0, 0)| < eps}

def LowerValuesAccumulate {d : Nat}
    (f : Plane × Tail d -> Real) : Prop :=
  ∀ t : Real, 0 < t -> ∃ w, ambientSq w < t ^ 2 ∧ f w < f (0, 0)

def CenterNegativeAccumulate {d : Nat} (f : Plane × Tail d -> Real)
    (c : Tail d -> Plane) : Prop :=
  ∀ eta : Real, 0 < eta -> ∃ z, norm z < eta ∧ centerValue f c z < 0
```

Definition 18.2 (Centered family: no volume assertion among the fields)**.**

Corresponding Lean declaration(s): `O77.SaddleDirect.CenteredFamily`.

A centered family for $f : P \times E_d \to \mathbb{R}$ consists of $R, S > 0$, $0 < \kappa \le K$, a map $c : E_d \to P$ with $c(0) = 0$, continuous at every z with $\|z\| < S$, and the following pointwise assertions for every such z:

(i) $q_P(c(z)) < (R/2)^2$*;*

(ii) h_z *has radial control* (R, κ, K) *in Definition 17.1;*

(iii) $h_z(x) \ge 0$ *whenever* $q_P(x) \le R^2$*.*

The last assertion is a redundant, useful interface to the convexity calculation: the C^2 construction proves the stronger $h_z(x) \ge \kappa q_P(x)/2$. None of these fields mentions a measure, a learning coefficient, or an asymptotic conclusion. The map c may have an arbitrary total extension away from the tail ball; continuity there is neither assumed nor used.

The ordinary implicit-function argument constructs the centers from the partial gradient. `O77SaddlePreparation` implements that construction under uniform derivative hypotheses, and `O77SaddleAttachment` proves those hypotheses for the native loss. Merely defining the record would not establish either fact.

```
structure CenteredFamily {d : Nat} (f : Plane × Tail d -> Real) where
  radius : Real
  tailRadius : Real
  lowerBound : Real
  upperBound : Real
  radius_pos : 0 < radius
  tail_pos : 0 < tailRadius
  lower_pos : 0 < lowerBound
  lower_le_upper : lowerBound ≤ upperBound
  center : Tail d -> Plane
  center_zero : center 0 = 0
  center_cont : ∀ z, norm z < tailRadius -> ContinuousAt center z
  center_small : ∀ z, norm z < tailRadius -> planeSq (center z) < (radius / 2) ^ 2
  radial : ∀ z, norm z < tailRadius ->
    RadialControl (centeredSlice f center z) radius lowerBound upperBound
  minimum_on_disk : ∀ z, norm z < tailRadius ->
```

```
    ∀ x, planeSq x ≤ radius ^ 2 -> 0 ≤ centeredSlice f center z x

-- These are pointwise geometric data, not volume estimates.
-- The C^2 positive-plane construction uses uniformPartialHessian_of_positive and
     centeredFamily_of_uniformHessian.
```

Definition 18.3 (The ambient band-pair predicate)**.**

Corresponding Lean declaration(s): `077.SaddleDirect.bandGauge`,`077.SaddleDirect.HasAmbientBandPair`.

For $\lambda \in \mathbb{R}$ and $m \in \mathbb{N}$ let $G_{\lambda,m}(\varepsilon) = \varepsilon^{\lambda}(\log(1/\varepsilon))^{m-1}$, with the power taken over positive ε in all estimates. The predicate `HasAmbientBandPair` *requires $\lambda \geq 0$, $m \geq 1$, and*

$$\exists \delta_0 > 0\ \forall \delta \in (0, \delta_0)\ \exists c, C, \varepsilon_0 > 0 : \quad c \leq C,\ \varepsilon_0 < e^{-1},$$

followed by $cG_{\lambda,m}(\varepsilon) \leq V_f(\delta, \varepsilon) \leq CG_{\lambda,m}(\varepsilon)$ for every $0 < \varepsilon < \varepsilon_0$*. Real bounds are embedded into the extended nonnegative reals when compared with measure. This is Definition 2.2 in the explicitly regrouped Euclidean coordinates, not a definition in terms of the proposed answer.*

```
def bandGauge (lam : Real) (m : Nat) (eps : Real) : Real :=
  eps ^ lam * (Real.log (1 / eps)) ^ (m - 1)

def HasAmbientBandPair {d : Nat} (f : Plane × Tail d -> Real)
    (lam : Real) (m : Nat) : Prop :=
  0 ≤ lam ∧ 1 ≤ m ∧ ∃ delta0 : Real, 0 < delta0 ∧
    ∀ delta : Real, 0 < delta -> delta < delta0 ->
      ∃ c C eps0 : Real, 0 < c ∧ c ≤ C ∧ 0 < eps0 ∧
        eps0 < Real.exp (-1) ∧ ∀ eps : Real, 0 < eps -> eps < eps0 ->
          ENNReal.ofReal (c * bandGauge lam m eps) ≤
              volume (ambientBand f delta eps) ∧
          volume (ambientBand f delta eps) ≤
              ENNReal.ofReal (C * bandGauge lam m eps)
```

Small adapters, not additional foundational projects. The following private declarations express the displayed square sums, restriction to a smaller radius, and continuity of g at an interior tail point. The two inequalities $q_P(x \pm y) \leq 2q_P(x) + 2q_P(y)$ follow by adding $(x_i \mp y_i)^2 \geq 0$ for $i = 1, 2$. The proof of measurability uses openness of strict inequalities between continuous functions. Restriction of radial control uses only $0 < r \leq R \Rightarrow r^2 \leq R^2$. These are short adapters to existing algebra and topology; they are not advertised as independent new results or reasons to postpone the full-dimensional proof.

```
-- Short algebra/topology adapters, not new mathematical foundations.
private lemma planeSq_nonneg (y : Plane) : 0 ≤ planeSq y := by
  exact add_nonneg (sq_nonneg y.1) (sq_nonneg y.2)

private lemma planeSq_add_le (x y : Plane) :
    planeSq (x + y) ≤ 2 * planeSq x + 2 * planeSq y := by
  change (x.1 + y.1) ^ 2 + (x.2 + y.2) ^ 2 ≤
    2 * (x.1 ^ 2 + x.2 ^ 2) + 2 * (y.1 ^ 2 + y.2 ^ 2)
  nlinarith [sq_nonneg (x.1 - y.1), sq_nonneg (x.2 - y.2)]

private lemma planeSq_sub_le (x y : Plane) :
    planeSq (x - y) ≤ 2 * planeSq x + 2 * planeSq y := by
  change (x.1 - y.1) ^ 2 + (x.2 - y.2) ^ 2 ≤
    2 * (x.1 ^ 2 + x.2 ^ 2) + 2 * (y.1 ^ 2 + y.2 ^ 2)
  nlinarith [sq_nonneg (x.1 + y.1), sq_nonneg (x.2 + y.2)]

private lemma planeSq_continuous : Continuous planeSq := by
  unfold planeSq
```

```
  fun_prop

private lemma ambientBand_measurable {d : Nat}
    (f : Plane × Tail d -> Real) (hf : Continuous f) (delta eps : Real) :
    MeasurableSet (ambientBand f delta eps) := by
  have hq : Continuous (ambientSq (d := d)) := by
    unfold ambientSq planeSq
    fun_prop
  exact (isOpen_lt hq continuous_const).measurableSet.inter
    (isOpen_lt ((hf.sub continuous_const).abs) continuous_const).measurableSet

private lemma centerValue_continuousAt {d : Nat}
    (f : Plane × Tail d -> Real) (hf : Continuous f)
    (D : CenteredFamily f) (z : Tail d) (hz : norm z < D.tailRadius) :
    ContinuousAt (centerValue f D.center) z := by
  exact (hf.continuousAt.comp
    ((D.center_cont z hz).prodMk continuousAt_id)).sub continuousAt_const

private lemma radial_restrict (h : Plane -> Real) (R kappa K r : Real)
    (hc : RadialControl h R kappa K) (hr : 0 < r) (hrR : r ≤ R) :
    RadialControl h r kappa K := by
  have hsquares : r ^ 2 ≤ R ^ 2 := by
    nlinarith [mul_nonneg (sub_nonneg.mpr hrR)
      (show 0 ≤ R + r by linarith [hc.radius_pos])]
  exact {
    radius_pos := hr
    lower_pos := hc.lower_pos
    lower_le_upper := hc.lower_le_upper
    continuous_h := hc.continuous_h
    at_zero := hc.at_zero
    secants := fun theta s hs t ht hst =>
      hc.secants theta s (hs.1, hs.2.trans )hsquares
        t (ht.1, ht.2.trans )hsquares hst }
```

18.2 Passing lower values to the centers

Lemma 18.4 (The negative-center property follows from the original lower-value property)**.**

Corresponding Lean declaration(s): `077.SaddleDirect.negative_center_of_lower_values`.

Let D be a centered family for f. If `LowerValuesAccumulate` holds for f, then `CenterNegativeAccumulate` holds for f and D's center.

Proof. Fix $\eta > 0$ and put $t = \min(R/4, S, \eta) > 0$. Choose a lower-value point $w = (y, z)$ with $q_P(y) + \|z\|^2 < t^2$. Nonnegativity of both summands gives $q_P(y) < t^2$ and $\|z\| < t$, hence $\|z\| < S$ and $\|z\| < \eta$. The center bound and the difference inequality give

$$q_P(y - c(z)) \le 2q_P(y) + 2q_P(c(z)) < 2(R/4)^2 + 2(R/2)^2 = 5R^2/8 < R^2.$$

The pointwise minimum property applies to $x = y - c(z)$ and yields $f(c(z), z) \le f(y, z) < f(0, 0)$. Thus $g(z) < 0$, as required. No continuity or measure argument has entered this implication. When $d = 0$ its antecedent is impossible: there is only the center tail point and the same minimum inequality excludes a lower value. The statement does not artificially assume $d \ge 1$. □

```
theorem negative_center_of_lower_values {d : Nat}
    (f : Plane × Tail d -> Real) (D : CenteredFamily f)
    (hlower : LowerValuesAccumulate f) :
    CenterNegativeAccumulate f D.center := by
  intro eta heta
```

```
  let t : Real := min (D.radius / 4) (min D.tailRadius eta)
  have ht : 0 < t := by
    exact lt_min (by linarith [D.radius_pos]) (lt_min D.tail_pos heta)
  have htR : t ≤ D.radius / 4 := min_le_left _ _
  have htS : t ≤ D.tailRadius := (min_le_right _ _).trans (min_le_left _ _)
  have hteta : t ≤ eta := (min_le_right _ _).trans (min_le_right _ _)
  obtain ⟨w, hw, ⟩hfneg := hlower t ht
  have hz : norm w.2 < t := by
    have := planeSq_nonneg w.1
    dsimp [ambientSq] at hw
    nlinarith [norm_nonneg w.2]
  have hy : planeSq w.1 < t ^ 2 := by
    dsimp [ambientSq] at hw
    nlinarith [sq_nonneg (norm w.2)]
  have hzS : norm w.2 < D.tailRadius := lt_of_lt_of_le hz htS
  have hc := D.center_small w.2 hzS
  have ht2 : t ^ 2 ≤ (D.radius / 4) ^ 2 := by
    nlinarith [mul_nonneg (sub_nonneg.mpr htR)
      (show 0 ≤ D.radius / 4 + t by linarith [D.radius_pos])]
  have hdiff : planeSq (w.1 - D.center w.2) ≤ D.radius ^ 2 := by
    have hh := planeSq_sub_le w.1 (D.center w.2)
    nlinarith [sq_pos_of_pos D.radius_pos]
  have hm := D.minimum_on_disk w.2 hzS (w.1 - D.center w.2) hdiff
  have hcancel : D.center w.2 + (w.1 - D.center w.2) = w.1 := by
    ext <;> simp
  have hm' : f (D.center w.2, w.2) ≤ f w := by
    apply sub_nonneg.mp
    simpa only [centeredSlice, hcancel] using hm
  refine ⟨w.2, lt_of_lt_of_le hz hteta, ?⟩_
  unfold centerValue
  linarith
```

18.3 The positive-volume window is chosen before the band width

Definition 18.5 (A fixed lower window at radius δ)**.**

Corresponding Lean declaration(s): `077.SaddleDirect.FixedLowerWindow`.

Fix $\delta > 0$ and write $r_\delta = \delta/4$. A fixed lower window is data $z_0 \in E_d$, $\nu, a_0 > 0$ such that $a_0 < \kappa r_\delta^2/16$ and, for every $z \in \Omega := B_d(z_0, \nu)$,

$$\|z\| < S, \quad \|z\| < \delta/4, \quad q_P(c(z)) < r_\delta^2, \qquad a_0/2 < -g(z) < 3a_0/2.$$

There is no ε argument in this definition. In particular, the stored ball cannot change when the asymptotic band width subsequently varies.

Lemma 18.6 (Existence of a fixed lower window)**.**

Corresponding Lean declaration(s): `077.SaddleDirect.fixedLowerWindow_nonempty`.

Assume f is continuous, D is a centered family, and its center values are negative arbitrarily close to zero. Then for every $\delta > 0$ there is a fixed lower window.

Proof. Continuity of f and of c at zero implies continuity there of g and of $q_P \circ c$; both have value zero. The positive bounds $\min(S, \delta/4)$, r_δ^2, and $\kappa r_\delta^2/16$ therefore admit one $\eta > 0$ such that $\|z\| < \eta$ implies

$$\|z\| < \min(S, \delta/4), \quad q_P(c(z)) < r_\delta^2, \quad |g(z)| < \kappa r_\delta^2/16.$$

Choose z_0 in that ball with $g(z_0) < 0$ and set $a_0 = -g(z_0) > 0$. The displayed bound proves the required cap on a_0. Since $\|z_0\| < S$, the map c is continuous at z_0, so $-g$ is continuous

there. At z_0 one has the two strict inequalities $a_0/2 < -g(z_0) < 3a_0/2$. Their preimage is a neighbourhood of z_0. Intersect it with the open ball $B_d(0,\eta)$, which also contains z_0. The intersection contains some ball $B_d(z_0,\nu)$ with $\nu > 0$. Membership in that ball implies every required inequality. All selections have now been made for this fixed δ, without reference to ε. The ball is nonempty and open, so its Lebesgue measure is positive; it is bounded, so that measure is finite. □

The formal existence theorem returns `Nonempty(FixedLowerWindowfDdelta)`. It has no assumption asserting an $O(\varepsilon)$ or $\Omega(\varepsilon)$ bound. The occurrence of "window" in the structure name does not disguise an asymptotic premise.

```
structure FixedLowerWindow {d : Nat} (f : Plane × Tail d -> Real)
    (D : CenteredFamily f) (delta : Real) where
  z0 : Tail d
  nu : Real
  a0 : Real
  nu_pos : 0 < nu
  a0_pos : 0 < a0
  cap : a0 < D.lowerBound * (delta / 4) ^ 2 / 16
  tail : ∀ z ∈ Metric.ball z0 nu, norm z < D.tailRadius ∧ norm z < delta / 4
  center : ∀ z ∈ Metric.ball z0 nu,
    planeSq (D.center z) < (delta / 4) ^ 2
  energy : ∀ z ∈ Metric.ball z0 nu,
    a0 / 2 < -centerValue f D.center z ∧
      -centerValue f D.center z < 3 * a0 / 2

theorem fixedLowerWindow_nonempty {d : Nat}
    (f : Plane × Tail d -> Real) (hf : Continuous f)
    (D : CenteredFamily f) (hneg : CenterNegativeAccumulate f D.center)
    (delta : Real) (hdelta : 0 < delta) :
    Nonempty (FixedLowerWindow f D delta) := by
  let g := centerValue f D.center
  have hk : 0 < D.lowerBound := D.lower_pos
  have hzS : norm (0 : Tail d) < D.tailRadius := by simpa using D.tail_pos
  have hg0 : g 0 = 0 := by simp [g, centerValue, D.center_zero]
  have hc0 : planeSq (D.center 0) = 0 := by simp [D.center_zero, planeSq]
  have hgcont : ContinuousAt g 0 := centerValue_continuousAt f hf D 0 hzS
  have hccont : ContinuousAt (fun z => planeSq (D.center z)) 0 :=
    planeSq_continuous.continuousAt.comp (D.center_cont 0 hzS)
  have hnearN : ∀ᶠ z : Tail d in nhds 0,
      norm z < min D.tailRadius (delta / 4) :=
    continuous_norm.continuousAt.eventually
      (gt_mem_nhds (by simpa using lt_min D.tail_pos (by linarith : 0 < delta / 4)))
  have hnearC : ∀ᶠ z : Tail d in nhds 0,
      planeSq (D.center z) < (delta / 4) ^ 2 :=
    hccont.eventually (gt_mem_nhds (by
      change planeSq (D.center 0) < (delta / 4) ^ 2
      rw [hc0]
      positivity))
  have hnearG : ∀ᶠ z : Tail d in nhds 0,
      |g z| < D.lowerBound * (delta / 4) ^ 2 / 16 :=
    hgcont.abs.eventually (gt_mem_nhds (by
      change |g 0| < D.lowerBound * (delta / 4) ^ 2 / 16
      rw [hg0, abs_zero]
      positivity))
  have hnear := hnearN.and (hnearC.and hnearG)
  obtain ⟨eta, heta, ⟩hetaSet := Metric.mem_nhds_iff.mp hnear
  obtain ⟨z0, hz0, ⟩hnegative := hneg eta heta
  have hzBall : z0 ∈ Metric.ball (0 : Tail d) eta := by
    simpa only [Metric.mem_ball, dist_zero_right] using hz0
  have hdata0 := hetaSet hzBall
```

```
  let a0 : Real := -g z0
  have ha0 : 0 < a0 := neg_pos.mpr hnegative
  have hcap : a0 < D.lowerBound * (delta / 4) ^ 2 / 16 := by
    have hh := hdata0.2.2
    rw [abs_of_neg hnegative] at hh
    exact hh
  have hz0S : norm z0 < D.tailRadius := lt_of_lt_of_le hdata0.1 (min_le_left _ _)
  have hgcont0 : ContinuousAt (fun z => -g z) z0 :=
    (centerValue_continuousAt f hf D z0 hz0S).neg
  have henergy : ∀ᶠ z in nhds z0, a0 / 2 < -g z ∧ -g z < 3 * a0 / 2 :=
    hgcont0.eventually (Ioo_mem_nhds (by change a0 / 2 < a0; linarith)
      (by change a0 < 3 * a0 / 2; linarith))
  have hbase : ∀ᶠ z : Tail d in nhds z0,
      z ∈ Metric.ball (0 : Tail d) eta :=
    Metric.isOpen_ball.mem_nhds hzBall
  obtain ⟨nu, hnu, ⟩hnuSet := Metric.mem_nhds_iff.mp (hbase.and henergy)
  refine ⟨{
    z0 := z0
    nu := nu
    a0 := a0
    nu_pos := hnu
    a0_pos := ha0
    cap := hcap
    tail := ?_
    center := ?_
    energy := ?_ ⟩}
  · intro z hz
    have hzdata := hetaSet (hnuSet hz).1
    exact ⟨lt_of_lt_of_le hzdata.1 (min_le_left _ _),
      lt_of_lt_of_le hzdata.1 (min_le_right _ _)⟩
  · intro z hz
    exact (hetaSet (hnuSet hz).1).2.1
  · intro z hz
    exact (hnuSet hz).2
```

18.4 Section bounds inside the actual ambient ball

Lemma 18.7 (Translation identity and the upper and lower section inequalities)**.**

Corresponding Lean declaration(s): `O77.SaddleDirect.translatedBand_volume`,`O77.SaddleDirect.upper_section`,`O77.SaddleDirect.lower_section`.

Let $0 < \delta < R/4$, $\delta < S$, *and* $\varepsilon > 0$, *and let* $A_z = \{y : q_W(y,z) < \delta^2,\ |f(y,z) - f(0,0)| < \varepsilon\}$. *For* $\|z\| < \delta$ *one has* $\mathrm{vol}_2(A_z) \le 4\pi\varepsilon/\kappa$. *If* W *is a fixed lower window,* $z \in \Omega$, *and* $0 < \varepsilon < a_0/4$, *then* $\mathrm{vol}_2(A_z) \ge 4\pi\varepsilon/K$.

Proof. For any fixed z, r, ε, translation by $c(z)$ has

$$(c(z) + \cdot)^{-1} T_z(r,\varepsilon) = B(h_z; r, -g(z), \varepsilon),$$

because $h_z(x) - (-g(z)) = f(c(z)+x, z) - f(0,0)$. Translation preserves planar Lebesgue measure. This is a statement for a fixed z; no nonlinear change of variables in the whole space and no Jacobian depending on z is needed.

For the upper bound, $y \in A_z$ implies $q_P(y) < \delta^2$. Since $\|z\| < \delta < S$,

$$q_P(y - c(z)) \le 2q_P(y) + 2q_P(c(z)) < 2\delta^2 + R^2/2 < 5R^2/8 < R^2.$$

Consequently $A_z \subseteq T_z(R,\varepsilon)$. Translate and apply the uniform upper bound of Theorem 17.4. Its uniformity in the shift $-g(z)$ is essential.

For the lower bound let $z \in \Omega$ and set $a = -g(z)$. By the window inequalities and $\varepsilon < a_0/4$,

$$a - \varepsilon > a_0/4 > 0, \qquad a + \varepsilon < 7a_0/4 < 7\kappa r_\delta^2/64 < \kappa r_\delta^2/2.$$

Restrict radial control to $r_\delta < R$. The planar lower bound gives $\operatorname{vol}_2 T_z(r_\delta, \varepsilon) \geq 4\pi\varepsilon/K$. It remains to check that the whole translated disk, not only one circle in it, lies in the original ball. If $q_P(y - c(z)) < r_\delta^2$, then

$$q_P(y) \leq 2q_P(y - c(z)) + 2q_P(c(z)) < 4r_\delta^2 = \delta^2/4, \qquad \|z\|^2 < \delta^2/16.$$

Thus $q_W(y, z) < 5\delta^2/16 < \delta^2$. It follows that $T_z(r_\delta, \varepsilon) \subseteq A_z$, and measure monotonicity proves the assertion. □

```
-- Reuse Haar translation invariance: no nonlinear Jacobian is needed.
theorem translatedBand_volume {d : Nat}
    (f : Plane × Tail d -> Real) (c : Tail d -> Plane)
    (z : Tail d) (R eps : Real) :
    volume (translatedBand f c z R eps) =
      volume (planarBand (centeredSlice f c z) R (-centerValue f c z) eps) := by
  have heq : (fun x : Plane => c z + x) ⁻¹' translatedBand f c z R eps =
      planarBand (centeredSlice f c z) R (-centerValue f c z) eps := by
    ext x
    have hv : centeredSlice f c z x - (-centerValue f c z) =
        f (c z + x, z) - f (0, 0) := by
      unfold centeredSlice centerValue
      ring
    have hcancel : c z + x - c z = x := by
      ext <;> simp
    change (planeSq (c z + x - c z) < R ^ 2 ∧
        |f (c z + x, z) - f (0, 0)| < eps) ↔
      (planeSq x < R ^ 2 ∧
        |centeredSlice f c z x - (-centerValue f c z)| < eps)
    rw [hcancel, hv]
  rw ←[ measure_preimage_add (volume : Measure Plane) (c z)
    (translatedBand f c z R eps), heq]

theorem upper_section {d : Nat} (f : Plane × Tail d -> Real)
    (D : CenteredFamily f) (delta eps : Real)
    (hd : 0 < delta) (hdR : delta < D.radius / 4) (hdS : delta < D.tailRadius)
    (heps : 0 < eps) (z : Tail d) (hz : norm z < delta) :
    volume {y : Plane | (y, z) ∈ ambientBand f delta eps} ≤
      ENNReal.ofReal (4 * Real.pi * eps / D.lowerBound) := by
  have hzS : norm z < D.tailRadius := hz.trans hdS
  have hsub : {y : Plane | (y, z) ∈ ambientBand f delta eps} ⊆
      translatedBand f D.center z D.radius eps := by
    intro y hy
    change ambientSq (y, z) < delta ^ 2 ∧ _ at hy
    have hysq : planeSq y < delta ^ 2 := by
      dsimp [ambientSq] at hy
      nlinarith [sq_nonneg (norm z)]
    have hc := D.center_small z hzS
    have hdiff := planeSq_sub_le y (D.center z)
    have hd2 : delta ^ 2 < (D.radius / 4) ^ 2 := by
      nlinarith [mul_pos (sub_pos.mpr hdR)
        (show 0 < D.radius / 4 + delta by linarith [D.radius_pos])]
    refine (?_, hy).2
    nlinarith [sq_pos_of_pos D.radius_pos]
  calc
    _ ≤ volume (translatedBand f D.center z D.radius eps) := measure_mono hsub
    _ = volume (planarBand (centeredSlice f D.center z)
          D.radius (-centerValue f D.center z) eps) :=
```

```
      translatedBand_volume f D.center z D.radius eps
    _ ≤ _ := (planarBand_bounds _ D.radius D.lowerBound D.upperBound
      (D.radial z hzS) _ eps heps).1

theorem lower_section {d : Nat} (f : Plane × Tail d -> Real)
    (D : CenteredFamily f) (delta eps : Real)
    (hd : 0 < delta) (hdR : delta < D.radius / 4)
    (W : FixedLowerWindow f D delta)
    (heps : 0 < eps) (hesmall : eps < W.a0 / 4)
    (z : Tail d) (hz : z ∈ Metric.ball W.z0 W.nu) :
    ENNReal.ofReal (4 * Real.pi * eps / D.upperBound) ≤
      volume {y : Plane | (y, z) ∈ ambientBand f delta eps} := by
  have htail := W.tail z hz
  have henergy := W.energy z hz
  have hr : 0 < delta / 4 := by linarith
  have hrR : delta / 4 ≤ D.radius := by linarith [D.radius_pos]
  have hc := radial_restrict _ D.radius D.lowerBound D.upperBound (delta / 4)
    (D.radial z htail.1) hr hrR
  have hinterior : 0 < -centerValue f D.center z - eps ∧
      -centerValue f D.center z + eps < (D.lowerBound / 2) * (delta / 4) ^ 2 := by
    constructor
    · linarith [W.a0_pos]
    · nlinarith [W.cap, W.a0_pos]
  have hsub : translatedBand f D.center z (delta / 4) eps ⊆
      {y : Plane | (y, z) ∈ ambientBand f delta eps} := by
    intro y hy
    have hcsmall := W.center z hz
    have hysq : planeSq y ≤
        2 * planeSq (y - D.center z) + 2 * planeSq (D.center z) := by
      simpa only [sub_add_cancel] using planeSq_add_le (y - D.center z) (D.center z)
    have hzsq : norm z ^ 2 < (delta / 4) ^ 2 := by
      nlinarith [norm_nonneg z, mul_pos (sub_pos.mpr htail.2)
        (show 0 < delta / 4 + norm z by linarith [norm_nonneg z])]
    change planeSq (y - D.center z) < (delta / 4) ^ 2 ∧ _ at hy
    change ambientSq (y, z) < delta ^ 2 ∧ _
    refine ⟨?_, hy⟩.2
    dsimp [ambientSq]
    nlinarith [sq_pos_of_pos hd]
  calc
    _ ≤ volume (planarBand (centeredSlice f D.center z)
          (delta / 4) (-centerValue f D.center z) eps) :=
      (planarBand_bounds _ (delta / 4) D.lowerBound D.upperBound hc _ eps heps).2 hinterior
    _ = volume (translatedBand f D.center z (delta / 4) eps) :=
      (translatedBand_volume f D.center z (delta / 4) eps).symm
    _ ≤ _ := measure_mono hsub
```

18.5 Fubini is used from Mathlib, not rebuilt

Theorem 18.8 (Full-dimensional measured inequalities)**.**

Corresponding Lean declaration(s): `077.SaddleDirect.ambientBand_upper`,`077.SaddleDirect.ambientBand_lower`.

Assume f is continuous and $0 < \delta < \min(R/4, S)$. For every $\varepsilon > 0$,

$$V_f(\delta, \varepsilon) \le \frac{4\pi}{\kappa}\varepsilon \operatorname{vol}_d B_d(0, \delta).$$

For every fixed lower window W and every $0 < \varepsilon < a_0/4$,

$$V_f(\delta, \varepsilon) \ge \frac{4\pi}{K}\varepsilon \operatorname{vol}_d B_d(z_0, \nu).$$

These inequalities are initially comparisons in $[0, +\infty]$*, not inequalities involving a totalized real-valued integral.*

Proof. The full band is measurable because it is the intersection of two open sets defined by strict inequalities between continuous functions. Tonelli for its indicator gives $V_f(\delta, \varepsilon) = \int_{E_d} \mathrm{vol}_2(A_z)\, dz$. If $\|z\| \geq \delta$, then $\|z\|^2 \geq \delta^2$ and $q_P(y) \geq 0$, so A_z is empty. The upper section estimate therefore bounds the integrand by $(4\pi\varepsilon/\kappa)\mathbf{1}_{B_d(0,\delta)}(z)$ for all z. Integrating proves the upper bound. For the lower bound the section estimate applies on the fixed ball Ω, and the integrand is nonnegative elsewhere. It is therefore bounded below by $(4\pi\varepsilon/K)\mathbf{1}_{\Omega}(z)$. Integration gives the second inequality. Only the original band must be known measurable; the proof does not introduce a possibly nonmeasurable choice of planar inverse endpoints. □

The code calls `Measure.prod_apply_symm` for the actual product measure, `lintegral_mono`, `lintegral_indicator`, and `setLIntegral_const`. In particular, no placeholder named "Fubini" or "integrate the inequalities" is used. The product-measure statement and its measurable-set hypothesis were inspected in the pinned source. The use of `measure_preimage_add` above is likewise the existing Haar-measure translation theorem, not a new axiom about coordinate changes.

```
theorem ambientBand_upper {d : Nat}
    (f : Plane × Tail d -> Real) (hf : Continuous f)
    (D : CenteredFamily f) (delta eps : Real)
    (hd : 0 < delta) (hdR : delta < D.radius / 4) (hdS : delta < D.tailRadius)
    (heps : 0 < eps) :
    volume (ambientBand f delta eps) ≤
      ENNReal.ofReal (4 * Real.pi * eps / D.lowerBound) *
        volume (Metric.ball (0 : Tail d) delta) := by
  let T := Metric.ball (0 : Tail d) delta
  let b := ENNReal.ofReal (4 * Real.pi * eps / D.lowerBound)
  have hm := ambientBand_measurable f hf delta eps
  classical
  calc
    volume (ambientBand f delta eps) =
        ∫⁻ z : Tail d, volume {y : Plane | (y, z) ∈ ambientBand f delta eps} :=
      Measure.prod_apply_symm hm
    _ ≤ ∫⁻ z : Tail d, T.indicator (fun _ => b) z := by
      apply lintegral_mono
      intro z
      by_cases hz : z ∈ T
      · rw [Set.indicator_of_mem hz]
        exact upper_section f D delta eps hd hdR hdS heps z
          (by simpa only [T, Metric.mem_ball, dist_zero_right] using hz)
      · have hzero : {y : Plane | (y, z) ∈ ambientBand f delta eps} = ∅ := by
          ext y
          simp only [Set.mem_ofPred_eq, Set.mem_empty_iff_false, iff_false]
          intro hy
          have hzge : delta ≤ norm z := by
            simpa only [T, Metric.mem_ball, dist_zero_right, not_lt] using hz
          change ambientSq (y, z) < delta ^ 2 ∧ _ at hy
          dsimp [ambientSq] at hy
          nlinarith [planeSq_nonneg y, norm_nonneg z,
            mul_nonneg (sub_nonneg.mpr hzge)
              (show 0 ≤ norm z + delta by linarith [norm_nonneg z])]
        simp only [hzero, measure_empty, Set.indicator_of_notMem hz, le_refl]
    _ = b * volume T := by rw [lintegral_indicator Metric.isOpen_ball.measurableSet,
        setLIntegral_const]

theorem ambientBand_lower {d : Nat}
    (f : Plane × Tail d -> Real) (hf : Continuous f)
    (D : CenteredFamily f) (delta eps : Real)
```

```
    (hd : 0 < delta) (hdR : delta < D.radius / 4)
    (W : FixedLowerWindow f D delta)
    (heps : 0 < eps) (hesmall : eps < W.a0 / 4) :
    ENNReal.ofReal (4 * Real.pi * eps / D.upperBound) *
        volume (Metric.ball W.z0 W.nu) ≤ volume (ambientBand f delta eps) := by
  let T := Metric.ball W.z0 W.nu
  let b := ENNReal.ofReal (4 * Real.pi * eps / D.upperBound)
  have hm := ambientBand_measurable f hf delta eps
  classical
  calc
    b * volume T = ∫⁻ z : Tail d, T.indicator (fun _ => b) z := by
      rw [lintegral_indicator Metric.isOpen_ball.measurableSet, setLIntegral_const]
    _ ≤ ∫⁻ z : Tail d, volume {y : Plane | (y, z) ∈ ambientBand f delta eps} := by
      apply lintegral_mono
      intro z
      by_cases hz : z ∈ T
      · rw [Set.indicator_of_mem hz]
        exact lower_section f D delta eps hd hdR W heps hesmall z hz
      · simp only [Set.indicator_of_notMem hz, zero_le]
    _ = volume (ambientBand f delta eps) := (Measure.prod_apply_symm hm).symm
```

18.6 Assembly of every quantifier in the local pair

Theorem 18.9 (The pointwise centered-family data imply the ambient pair)**.**

Corresponding Lean declaration(s): `O77.SaddleDirect.ambientPair_of_centeredFamily`,`O77.SaddleDirect.ambientPair_of_lowerValues`.

Let $f : P \times E_d \to \mathbb{R}$ be continuous and let D be a centered family for f. If its center values are negative arbitrarily close to zero, then HasAmbientBandPair$(f; 1, 1)$*. In particular the same conclusion follows from* `LowerValuesAccumulate` *for the original function f.*

Proof. Set $\delta_0 = \min(R/4, S) > 0$ and fix an arbitrary $0 < \delta < \delta_0$. Choose W by Lemma 18.6 and let $m_U = \mathrm{vol}_d\, B_d(0, \delta)$ and $m_L = \mathrm{vol}_d\, \Omega$. Both numbers are strictly positive and finite: a positive-radius ball is a nonempty open set and is contained in its compact closed ball. These facts also hold in E_0, whose coordinate measure gives its single point mass one. Thus the use of `ENNReal.toReal` at precisely this step is justified, and converting either mass to a real number and back preserves it.

Put $C = (4\pi/\kappa)m_U > 0$, $c_L = (4\pi/K)m_L > 0$, $c = \min(c_L, C) > 0$, and $\varepsilon_0 = \min(a_0/4, e^{-1}/2) > 0$. Then $c \le C$ and $\varepsilon_0 < e^{-1}$. For every $0 < \varepsilon < \varepsilon_0$, Theorem 18.8 gives $c\varepsilon \le V_f(\delta, \varepsilon) \le C\varepsilon$. Since $G_{1,1}(\varepsilon) = \varepsilon$, this is the predicate of Definition 18.3 with all quantifiers in the required order. The arbitrary radius δ was fixed before W, c, C, ε_0 were selected. Finally apply Lemma 18.4 to obtain the second statement from the lower-value property of f. □

```
-- Routine use of positivity on open sets and finiteness on compact sets.
-- The proof is valid also for d = 0; no Nonempty (Fin d) is assumed.
private lemma tail_ball_mass {d : Nat} (z : Tail d) (nu : Real) (hnu : 0 < nu) :
    0 < (volume (Metric.ball z nu)).toReal ∧
    ENNReal.ofReal (volume (Metric.ball z nu)).toReal = volume (Metric.ball z nu) := by
  have hp : 0 < volume (Metric.ball z nu) :=
    Metric.isOpen_ball.measure_pos volume (z, Metric.mem_ball_self )hnu
  have ht : volume (Metric.ball z nu) < ⊤ :=
    lt_of_le_of_lt (measure_mono Metric.ball_subset_closedBall)
      (isCompact_closedBall z nu).measure_lt_top
  exact ⟨ENNReal.toReal_pos_iff.mpr ⟨hp, ⟩ht, ENNReal.ofReal_toReal (ne_of_lt ht)⟩

theorem ambientPair_of_centeredFamily {d : Nat}
    (f : Plane × Tail d -> Real) (hf : Continuous f)
```

```
    (D : CenteredFamily f) (hneg : CenterNegativeAccumulate f D.center) :
    HasAmbientBandPair f 1 1 := by
  have hK : 0 < D.upperBound := lt_of_lt_of_le D.lower_pos D.lower_le_upper
  have hpi : 0 < Real.pi := Real.pi_pos
  have hk : 0 < D.lowerBound := D.lower_pos
  refine ⟨zero_le_one, le_rfl, min (D.radius / 4) D.tailRadius,
    lt_min (by linarith [D.radius_pos]) D.tail_pos, ?⟩_
  intro delta hd hd0
  have hdR : delta < D.radius / 4 := lt_of_lt_of_le hd0 (min_le_left _ _)
  have hdS : delta < D.tailRadius := lt_of_lt_of_le hd0 (min_le_right _ _)
  obtain ⟨⟩W := fixedLowerWindow_nonempty f hf D hneg delta hd
  have hmU := tail_ball_mass (0 : Tail d) delta hd
  have hmL := tail_ball_mass W.z0 W.nu W.nu_pos
  let C : Real := (4 * Real.pi / D.lowerBound) * (volume (Metric.ball (0 : Tail d) delta)).toReal
  let cL : Real := (4 * Real.pi / D.upperBound) * (volume (Metric.ball W.z0 W.nu)).toReal
  have hC : 0 < C := mul_pos (div_pos (by positivity) D.lower_pos) hmU.1
  have hcL : 0 < cL := mul_pos (div_pos (by positivity) hK) hmL.1
  let eps0 := min (W.a0 / 4) (Real.exp (-1) / 2)
  have he0 : 0 < eps0 := lt_min (by linarith [W.a0_pos]) (by positivity)
  have he0exp : eps0 < Real.exp (-1) :=
    lt_of_le_of_lt (min_le_right _ _) (by linarith [Real.exp_pos (-1)])
  refine ⟨min cL C, C, eps0, lt_min hcL hC, min_le_right _ _, he0, he0exp, ?⟩_
  intro eps heps hsmall
  have heL : eps < W.a0 / 4 := lt_of_lt_of_le hsmall (min_le_left _ _)
  have hu := ambientBand_upper f hf D delta eps hd hdR hdS heps
  have hl := ambientBand_lower f hf D delta eps hd hdR W heps heL
  have hcastU : ENNReal.ofReal (4 * Real.pi * eps / D.lowerBound) *
      volume (Metric.ball (0 : Tail d) delta) = ENNReal.ofReal (C * eps) := by
    rw ←[ hmU.2, ← ENNReal.ofReal_mul (by positivity)]
    congr 1
    dsimp [C]
    ring
  have hcastL : ENNReal.ofReal (4 * Real.pi * eps / D.upperBound) *
      volume (Metric.ball W.z0 W.nu) = ENNReal.ofReal (cL * eps) := by
    rw ←[ hmL.2, ← ENNReal.ofReal_mul (by positivity)]
    congr 1
    dsimp [cL]
    ring
  rw [hcastU] at hu
  rw [hcastL] at hl
  have hgauge : bandGauge 1 1 eps = eps := by simp [bandGauge]
  simp only [hgauge]
  constructor
  · exact (ENNReal.ofReal_le_ofReal
      (mul_le_mul_of_nonneg_right (min_le_left _ _) (le_of_lt heps))).trans hl
  · exact hu

theorem ambientPair_of_lowerValues {d : Nat}
    (f : Plane × Tail d -> Real) (hf : Continuous f)
    (D : CenteredFamily f) (hlower : LowerValuesAccumulate f) :
    HasAmbientBandPair f 1 1 :=
  ambientPair_of_centeredFamily f hf D (negative_center_of_lower_values f D hlower)

end O77.SaddleDirect
end
```

18.7 Constructing the derivative-to-center interface

Proposition 18.10 (How the C^2 preparation feeds this endpoint)**.** *For a globally continuous $f : P \times E_d \to \mathbb{R}$ that is C^2 on a neighbourhood of zero, with zero partial differential and positive definite partial Hessian at zero, the data in Lemma 17.5 satisfy Definition 18.2. If zero is*

critical and some Hessian direction is negative, the lower-value hypothesis in Theorem 18.9 is satisfied. Hence the latter theorem gives exactly the measured conclusion of Theorem 17.7 for such f.

Proof. Take R, S, κ, K, c from Lemma 17.5. On its tail ball c is C^1, so it is continuous at every point there. Extend it arbitrarily outside a slightly larger domain of definition; every point in the smaller open tail ball retains its original neighbourhood, so its local continuity and all equations are unchanged. Its base value is zero, its center bound is the specified one, and the two line integrations in that lemma give the secant inequalities. For each fixed tail point, global continuity of f gives global continuity of $x \mapsto h_z(x)$, as required by the current `RadialControl`. The same line integration gives $h_z(x) \geq \kappa q_P(x)/2 \geq 0$ on the closed disk, supplying the pointwise minimum field without any volume hypothesis. A negative Hessian direction at a critical point gives $f(tv) - f(0) = \frac{1}{2}t^2 D^2 f(0)[v, v] + o(t^2) < 0$ for all sufficiently small nonzero t. Its Euclidean distance from zero can be made less than any prescribed positive radius, which is precisely `LowerValuesAccumulate`. This proves the compatibility of the mathematical interfaces. The corresponding constructions for the native loss are proved in `O77SaddlePreparation` and `O77SaddleAttachment`; they are separate proofs from the declaration of the centered-family record. □

Library facts used by the construction. The following inventory identifies the measure and calculus interfaces used in this argument and in its derivative-preparation clients [15, 14, 22, 19].

Existing declaration / module	Use, and what is not being re-proved
`Measure.prod_apply_symm`, `Mathlib.MeasureTheory.Measure.Prod`	Measure of a measurable set as an integral of its planar sections. This is the product measure theorem already in Mathlib, not an O77 lemma.
`measure_preimage_add`, `Mathlib.MeasureTheory.Group.Measure`	A fixed planar translation preserves Lebesgue measure. Its application is per tail point.
`IsOpen.measure_pos`, `IsCompact.measure_lt_top`	Positive finite mass of the fixed tail balls; no new ball-volume formula is needed.
`Metric.mem_nhds_iff`, `ContinuousAt.eventually`	Choose one ball in a finite intersection of neighbourhoods. The nontrivial assembly is the order of these choices relative to δ and ε.
`ContDiffAt.implicitFunction`, `ContDiffAt.contDiffAt_implicitFunction`	Solve the two-component partial-gradient equation and obtain its regularity.

The product-measure and translation signatures were fetched at commit `664cbbe52927ce6acaa39ca35498336116e123b8`; the corresponding source blobs are `209d083a9256edee952e080a22462d87c080ccb7` and `6b47bb8ce072504dfaf9080749c99b525d35c91b`. The pinned `Mathlib.Analysis.Convex.Strong` module was also inspected: it already provides `StrongConvexOn`. There is no reason to define a parallel general theory of strong convexity. The squared-radius secant statement used here is proved directly with its hypotheses explicit. The implicit-function API solves its *second* variable. For our function $f(y, z)$, apply it to the partial-gradient equation written as $G(z, y)$; do not invert a tail derivative by leaving the blocks in the wrong order.

The derivative-preparation module applies this implicit-function API to the actual partial gradient; the saddle-attachment module constructs a common product neighbourhood for the positive Hessian bounds.

18.8 Trust boundary and failure controls

The theorem in this section is a reusable centered-family-to-band implication: its centered-family and lower-value arguments remain visible in its type. They are constructed from actual discarded-mode derivatives in `O77SaddlePreparation` and `O77SaddleAttachment`, and the pointwise rung result is assembled in `O77RungCompletion`. Thus the intermediate implication does not assume a band-volume answer, and the completed O77 theorem does not retain its geometric arguments as hypotheses.

The upper and lower estimates deliberately use the original ambient band. Removing $\|z\| < \delta$ from the upper integration region would generally give infinite measure. Allowing the lower tail ball to depend on ε would destroy the asserted positive constant. Replacing the shift $-g(z)$ by $g(z)$ would make the lower band empty when the center value is negative and the centered slice is nonnegative. These are mathematical failures, not merely changes of proof style.

Concrete regression family. For $u, v \in \mathbb{R}$ and $m \geq 1$, the family

$$f_{u,v,m}(x, y, z) = (x - uz)^2 + (y - vz)^2 - z^{2m}$$

has $c(z) = (uz, vz)$, $g(z) = -z^{2m}$ and $h_z(x, y) = x^2 + y^2$. It tests nonconstant centers and, when $m > 1$, lower values without a negative quadratic tail direction. The centered disk-band area is exactly

$$\pi\big[\min(r^2, a + \varepsilon) - \max(0, a - \varepsilon)\big]_+.$$

On an interior energy interval this equals $2\pi\varepsilon$, agreeing with $4\pi\varepsilon/\kappa = 4\pi\varepsilon/K$ for $\kappa = K = 2$. The interval clipping formula also shows directly that the sign-mutated lower estimate fails. Rational tests of this family check the centering identity and the inequalities defining the fixed window, not the truth of the general integration theorem.

The reusable implication and its actual client. The theorem proved above has ordinary centered-family and nearby-lower-value arguments and concludes the full-dimensional radius-first pair. These arguments are constructed in the saddle modules, using actual first and second derivatives of the loss. The tail window is fixed before the band width throughout.

19 Spectral-prefix arithmetic

The residual argument uses explicit two-sided inequalities. First a finite-measure Laplace estimate is converted into a sublevel estimate. The Gaussian row integral and Gram spectrum reduce the matrix integral to an ordered eigenvalue integral. A separated-box argument supplies its power and logarithm, and quartic homogeneity localizes the answer to every fixed ball. The exact Wishart identity used in the intermediate reduction is proved in the Gram and spectral chapters of this part.

Lemma 19.1 (Recovering volume order from Laplace order)**.**

Corresponding Lean declaration(s): `O77.Residual.powerLogTopReal,O77.Residual.powerLogZeroReal,O77.Residual.powerLogTop,O77.Residual.powerLogZero,O77.Residual.powerLogTopReal_nonneg,O77.Residual.powerLogZeroReal_nonneg`.

Corresponding Lean declaration(s): `O77.Residual.rpow_div_neg_eq,O77.Residual.powerLogTopReal_half_le,O77.Residual.rpow_inv_neg_eq,O77.Residual.powerLogTopReal_inv,O77.Residual.rpow_scaled_inv_neg_eq,O77.Re`

sidual.powerLogTopReal_scaled_lower.

Corresponding Lean declaration(s): O77.Residual.powerLogTop_half_le,O77.Residual.powerLogTop_inv,O77.Residual.powerLogTop_scaled_lower,O77.Residual.powerLogZero_eq_bandGauge.

Corresponding Lean declaration(s): O77.Residual.LaplacePowerLogBounds,O77.Residual.SublevelPowerLogBounds,O77.Residual.EThetaZeroPowerLog.

Corresponding Lean declaration(s): O77.Residual.sublevelPowerLogBounds_of_laplacePowerLogBounds,O77.Residual.eThetaZeroPowerLog_of_sublevelPowerLogBounds,O77.Residual.volumeOrder_of_laplaceOrder.

Let X be a measurable space, let ν be a finite measure on X, and let $f : X \to \mathbb{R}$ be measurable with $f(x) \geq 0$ for every $x \in X$. For $\lambda \in \mathbb{R}$ with $\lambda > 0$ and $k \in \mathbb{N}$, put

$$H_{\lambda,k}(T) = T^{-\lambda}(\log T)^k, \qquad G_{\lambda,k}(\varepsilon) = \varepsilon^{\lambda}(\log(1/\varepsilon))^k.$$

Suppose there are $0 < c \leq C$ and $T_0 \geq 2$ such that, for every $T \geq T_0$,

$$cH_{\lambda,k}(T) \leq J(T) := \int e^{-Tf}\, d\nu \leq CH_{\lambda,k}(T).$$

Then there are $0 < c_V \leq C_V$ and $0 < \varepsilon_0 < e^{-1}$ such that, for every $0 < \varepsilon < \varepsilon_0$,

$$c_V G_{\lambda,k}(\varepsilon) \leq V(\varepsilon) := \nu\{f < \varepsilon\} \leq C_V G_{\lambda,k}(\varepsilon).$$

The checked statement is `volumeOrder_of_laplaceOrder`. It keeps the integrals and masses in $[0, \infty]$, embeds real constants by `ENNReal.ofReal`, and first proves a raw `SublevelPowerLogBounds` statement before shrinking its lower constant and threshold to the exact predicate `EThetaZeroPowerLog`. The identity

$$\texttt{powerLogZero}\ \lambda\ k\ \varepsilon = \mathrm{ofReal}(\texttt{bandGauge}\ \lambda\ (k+1)\ \varepsilon)$$

is checked, so logarithmic degree k means multiplicity $k + 1$.

Proof. The large-parameter gauge is formalized as $H_{\lambda,k}(T) = T^{-\lambda}(\log T)^k$. For $T \geq 2$, positivity and monotonicity of the logarithm give

$$H_{\lambda,k}(T/2) \leq 2^{\lambda} H_{\lambda,k}(T). \tag{19.1.1}$$

Lemma 20.2, applied with $R = 2^{\lambda}$, produces one fixed $a \geq 1$ and, for every

$$T \geq T_* := \max(2T_0, \max(2, 1)),$$

the lower estimate

$$\mathrm{ofReal}(c/2)H_{\lambda,k}(T) \leq V(a/T). \tag{19.1.2}$$

Set

$$\widetilde{\varepsilon}_0 = \frac{1}{2}\min\left(T_0^{-1}, \frac{a}{T_*}\right), \qquad c_0 = \frac{c}{2}a^{-\lambda}, \qquad C_0 = eC.$$

These three numbers are positive, and $\widetilde{\varepsilon}_0 < 1$. Fix $0 < \varepsilon < \widetilde{\varepsilon}_0$. Then $T_0 \leq 1/\varepsilon$, and the universal upper estimate from Lemma 20.1 gives

$$V(\varepsilon) \leq eJ(1/\varepsilon) \leq eCH_{\lambda,k}(1/\varepsilon) = C_0 G_{\lambda,k}(\varepsilon). \tag{19.1.3}$$

For the lower bound, $T_* \le a/\varepsilon$, so apply (19.1.2) at $T = a/\varepsilon$. Since $a \ge 1$, monotonicity of log and the exact real-power identity imply

$$a^{-\lambda} G_{\lambda,k}(\varepsilon) \le H_{\lambda,k}(a/\varepsilon).$$

Therefore

$$c_0 G_{\lambda,k}(\varepsilon) \le \frac{c}{2} H_{\lambda,k}(a/\varepsilon) \le V\left(\frac{a}{a/\varepsilon}\right) = V(\varepsilon). \tag{19.1.4}$$

The raw checked predicate records (19.1.3)–(19.1.4) with positive constants. To obtain the exact O77 convention, replace c_0 by $\min(c_0, C_0)$ and replace the threshold by $\min(\widetilde{\varepsilon}_0, e^{-1}/2)$. This enforces $0 < c_V \le C_V$ and $\varepsilon_0 < e^{-1}$ without changing either inequality.

All constants and thresholds are chosen before the quantified ε. The proof uses only monotone inequalities and the safe finite-tail cancellation of Lemma 20.1; it invokes neither an exact Tauberian theorem nor a pre-existing theorem asserting existence of local pairs.

Audit obligation. The finite-measure hypothesis is used to know that the common Laplace tail is not ∞. The fixed cutoff depends on c, C and the half-scale factor. The lower constant retains $a^{-\lambda}$, and logarithmic degree k is translated to multiplicity $k+1$.

Provenance: Blueprint reconstruction, replacing the source's existence-plus-Abelian-read-off interface at the two-sided level.

□

Lemma 19.2 (Localizing the homogeneous Gaussian Laplace integral)**.** *Let $f \ge 0$ be measurable on $\mathbb{R}^D$, $D \ge 1$, and homogeneous of degree four for positive scalars. For each fixed $R > 0$ and all $T \ge 0$, put*

$$J_G(T) = \int_{\mathbb{R}^D} e^{-Tf(w)} e^{-\|w\|^2}\, dw, \qquad J_R(T) = \int_{B(0,R)} e^{-Tf(w)}\, dw.$$

There exist positive finite constants depending on R, D such that $J_G(T) \asymp J_R(T)$ uniformly in T. Consequently a two-sided power-log order for J_G gives the same order for every fixed-ball integral J_R.

Proof. On $B(0,R)$ the Gaussian weight is at least e^{-R^2}, so

$$J_G(T) \ge e^{-R^2} J_R(T). \tag{19.2.1}$$

For the reverse inequality set $\delta = R/2$. Apart from boundaries of Lebesgue measure zero, $\mathbb{R}^D$ is the disjoint union of $B(0,\delta)$ and the shells

$$S_j = \{w : \delta 2^j \le \|w\| < \delta 2^{j+1}\}, \qquad j \ge 0.$$

The inner-ball contribution is at most $J_R(T)$. On S_j make the linear change of variables $w = 2^j z$. The new region is contained in $\{z : \delta \le \|z\| < R\} \subset B(0,R)$, the Jacobian is 2^{jD}, and homogeneity gives $f(2^j z) = 2^{4j} f(z)$. Hence the shell contribution is at most

$$2^{jD} e^{-\delta^2 4^j} \int_{B(0,R)} e^{-T 2^{4j} f(z)}\, dz \le 2^{jD} e^{-\delta^2 4^j} J_R(T),$$

because $f \ge 0$ and $2^{4j} T \ge T$. The numerical series $\sum_{j\ge 0} 2^{jD} e^{-\delta^2 4^j}$ converges: the ratio of consecutive terms is

$$2^D \exp(-3\delta^2 4^j) \longrightarrow 0.$$

If its sum is $K_{R,D} < \infty$, then $J_G(T) \le (1 + K_{R,D}) J_R(T)$ for every $T \ge 0$. Together with (19.2.1) this proves uniform comparability. Both integrals are finite because their integrands

are bounded respectively by the integrable Gaussian and by the indicator of a finite-volume ball; they are strictly positive because the integrands are positive.

Provenance: Homogeneous shell decomposition adapted from Sneiderman, Lemma 8.6; monotonicity removes the need to assume a local pair first.

□

Lemma 19.3 (Exact Gaussian integration in the left factor)**.** *For $p, q, h \geq 1$ and $T \geq 0$,*

$$\begin{aligned} J_G(T) &= \int e^{-T\|YZ\|_F^2 - \|Y\|_F^2 - \|Z\|_F^2}\, dY\, dZ \\ &= \pi^{ph/2} \int_{\mathbb{R}^{h\times q}} e^{-\|Z\|_F^2} \det(I_h + TZZ^{\mathsf{T}})^{-p/2}\, dZ. \end{aligned}$$

Both sides are finite and strictly positive.

Proof. Fix $Z \in \mathbb{R}^{h\times q}$ and put $M = I_h + TZZ^{\mathsf{T}}$. For every row vector $x \in \mathbb{R}^h$,

$$xMx^{\mathsf{T}} = \|x\|^2 + T\|xZ\|^2,$$

so M is symmetric positive definite. If $y_1, \ldots, y_p$ are the rows of Y, then

$$T\|YZ\|_F^2 + \|Y\|_F^2 = \sum_{a=1}^{p} y_a M y_a^{\mathsf{T}}.$$

By the real spectral theorem choose an orthogonal Q with $Q^{\mathsf{T}}MQ = \operatorname{diag}(\mu_1, \ldots, \mu_h)$ and $\mu_i > 0$. The orthogonal change of variables has Jacobian of absolute value one. Fubini and the standard one-dimensional Gaussian integral give

$$\int_{\mathbb{R}^h} e^{-yMy^{\mathsf{T}}}\, dy = \prod_{i=1}^{h} \int_{\mathbb{R}} e^{-\mu_i u_i^2}\, du_i = \pi^{h/2} \prod_{i=1}^{h} \mu_i^{-1/2} = \pi^{h/2} \det(M)^{-1/2}.$$

The original integrand is nonnegative and measurable, so Tonelli permits the Y-integration to be performed first and then row by row. Multiplying the preceding identity over the p rows gives the displayed determinant to the power $-p/2$, and the remaining factor is $e^{-\|Z\|_F^2}$.

Finiteness can be checked independently of the equality: the original integrand is bounded by $e^{-\|Y\|_F^2 - \|Z\|_F^2}$, whose integral is $\pi^{(ph+hq)/2}$; equivalently, $\det(I_h + TZZ^{\mathsf{T}}) \geq 1$ bounds the right-hand side by $\pi^{ph/2} \int e^{-\|Z\|_F^2} dZ$. Strict positivity follows because the integrands are positive everywhere.

Provenance: Sneiderman, Proposition 8.9, pp. 34–35.

□

The exact spectral identity used here. For $k \leq d$, the formal interface fixes a positive finite constant before both $\rho, T \geq 0$ and equates the Gaussian Gram determinant integral with the sorted full-orthant Laguerre integral. Its concrete definition is given beside its code below. It is a specialization of the classical real-Wishart identity [10]; the terminal spectral module proves this specialization. No arbitrary-test-function version is required as a hypothesis of O77.

Lemma 19.4 (One-dimensional model-integral order)**.**

Corresponding Lean declaration(s): `O77.Residual.Model.integrand`,`O77.Residual.Model.integral`,`O77.Residual.Model.gauge`,`O77.Residual.Model.integrand_pos`,`O77.Residual.Model.integrand_nonneg`.

Corresponding Lean declaration(s): `O77.Residual.Model.continuousOn_integrand_Ioi`,`O77.Residual.Model.integrand_measurableOn_Ioi`,`O77.Residual.Model.factor_le_one`,`O77.Residual.Model.integrand_le_gamma`,`O77.Residual.Model.integrableOn_integrand_Ioi`.

Corresponding Lean declaration(s): `O77.Residual.Model.integral_nonneg`.

Corresponding Lean declaration(s): `O77.Residual.Model.nearIntegral_nonneg`,`O77.Residual.Model.middleIntegral_nonneg`,`O77.Residual.Model.tailIntegral_nonneg`,`O77.Residual.Model.nearIntegral_le_integral`,`O77.Residual.Model.middleIntegral_le_integral`.

Corresponding Lean declaration(s): `O77.Residual.Model.rpow_neg_antitone_of_one_le`,`O77.Residual.Model.middleIntegral_upper_of_lt`,`O77.Residual.Model.lowerConstantLt`,`O77.Residual.Model.upperConstantLt`,`O77.Residual.Model.lowerConstantLt_pos`.

Corresponding Lean declaration(s): `O77.Residual.Model.upperConstantLt_pos`,`O77.Residual.Model.tailIntegral_upper_of_lt`,`O77.Residual.Model.integral_bounds_of_lt`,`O77.Residual.Model.nearIntegral_upper_of_gt`,`O77.Residual.Model.middleIntegral_upper_of_gt`.

Corresponding Lean declaration(s): `O77.Residual.Model.lowerConstantGt`,`O77.Residual.Model.upperConstantGt`,`O77.Residual.Model.lowerConstantGt_pos`,`O77.Residual.Model.upperConstantGt_pos`,`O77.Residual.Model.integral_bounds_of_gt`.

Corresponding Lean declaration(s): `O77.Residual.Model.one_le_log_of_exp_one_le`,`O77.Residual.Model.lowerConstantEq`,`O77.Residual.Model.upperConstantEq`,`O77.Residual.Model.lowerConstantEq_pos`,`O77.Residual.Model.upperConstantEq_pos`.

Corresponding Lean declaration(s): `O77.Residual.Model.nearIntegral_upper_resonant`,`O77.Residual.Model.tailIntegral_upper_resonant`,`O77.Residual.Model.integral_bounds_of_eq`,`O77.Residual.Model.HasPowerLogBounds`,`O77.Residual.Model.gauge_nonneg`.

Corresponding Lean declaration(s): `O77.Residual.Model.modelIntegral_powerLog_bounds`,`O77.Residual.Model.integral_rpow_zero_inv_example`,`O77.Residual.Model.integral_rpow_inv_one_example`,`O77.Residual.Model.integral_inv_inv_one_example`,`O77.Residual.Model.modelIntegral_powerLog_bounds_lt_example`.

Corresponding Lean declaration(s): `O77.Residual.Model.modelIntegral_powerLog_bounds_eq_example`,`O77.Residual.Model.modelIntegral_powerLog_bounds_gt_example`.

For $A > 0$ and $\rho > 0$, put

$$I_{A,\rho}(T) = \int_0^\infty e^{-s} s^{A-1} (1 + Ts)^{-\rho}\, ds, \qquad H_{A,\rho}(T) = T^{-\min(A,\rho)} \begin{cases} \log T, & A = \rho, \\ 1, & A \neq \rho. \end{cases}$$

There are $c, C, T_0 \in \mathbb{R}$ such that $0 < c \le C$, $2 \le T_0$, and for every $T \ge T_0$,

$$c H_{A,\rho}(T) \le I_{A,\rho}(T) \le C H_{A,\rho}(T).$$

Thus the one-dimensional factor has exactly one logarithm when $A = \rho$ and none otherwise.

Proof. The code first proves that the integrand is nonnegative, continuous on $(0, \infty)$, and dominated by the convergent Gamma integrand $e^{-s}s^{A-1}$. Hence every integral used below is genuine; no totalized value of a nonintegrable function enters.

Split the integral at $1/T$ and 1 as $I_0 + I_1 + I_2$. For $T \geq 1$, the checked pointwise bounds and elementary integrations give

$$e^{-1}2^{-\rho}\frac{T^{-A}}{A} \leq I_0 \leq \frac{T^{-A}}{A}, \tag{19.4.1}$$

and, if $A \neq \rho$,

$$e^{-1}2^{-\rho}T^{-\rho}\frac{1 - T^{-(A-\rho)}}{A - \rho} \leq I_1 \leq T^{-\rho}\frac{1 - T^{-(A-\rho)}}{A - \rho}. \tag{19.4.2}$$

When $A = \rho$, the corresponding estimate is

$$e^{-1}2^{-A}T^{-A}\log T \leq I_1 \leq T^{-A}\log T. \tag{19.4.3}$$

Finally, with

$$Q_A = \int_1^\infty e^{-s}s^{A-1}\,ds \geq 0,$$

the tail satisfies $0 \leq I_2 \leq Q_A T^{-\rho}$.

If $A < \rho$, use I_0 for the lower bound. In the middle estimate, $T^{-\rho}(T^{\rho-A} - 1)/(\rho - A) \leq T^{-A}/(\rho - A)$, and the tail obeys $T^{-\rho} \leq T^{-A}$. One may take the preliminary constants

$$\ell = e^{-1}2^{-\rho}/A, \qquad U = A^{-1} + (\rho - A)^{-1} + Q_A.$$

If $\rho < A$, the upper estimate is analogous, now using $T^{-A} \leq T^{-\rho}$. For the lower bound, the fixed interval $[1/2, 1] \subseteq [1/T, 1]$ for $T \geq 2$ gives

$$I_1 \geq e^{-1}2^{-\rho}T^{-\rho}\frac{1 - 2^{-(A-\rho)}}{A - \rho},$$

whose coefficient is positive. If $A = \rho$, (19.4.3) supplies the lower bound. For $T \geq e$, $\log T \geq 1$, so the near and tail terms are absorbed into the same $T^{-A}\log T$ upper gauge.

In each regime the implementation replaces the preliminary lower coefficient by its minimum with the upper coefficient, thereby enforcing $0 < c \leq C$. It takes $T_0 = 2$ in the nonresonant cases and $T_0 = \max(2, e)$ at resonance. This proves the quantified statement and fixes the logarithmic power without an appeal to an external asymptotic existence theorem.

Provenance: Sneiderman, Appendix A, Lemma A.2; the present proof is recast as explicit three-range inequalities and checked against the pinned Mathlib integral API.

□

```
noncomputable section
open Real MeasureTheory Set
open scoped ENNReal Interval

namespace O77.Residual.Model

def integrand (A rho T s : Real) : Real :=
  Real.exp (-s) * s ^ (A - 1) * (1 + T * s) ^ (-rho)

def integral (A rho T : Real) : Real :=
  ∫ s : Real in Ioi 0, integrand A rho T s
```

```
def gauge (A rho T : Real) : Real :=
  T ^ (- min A rho) * if A = rho then Real.log T else 1

lemma integrand_pos {A rho T s : Real} (hs : 0 < s) (hT : 0 ≤ T) :
    0 < integrand A rho T s := by
  unfold integrand
  have hbase : 0 < 1 + T * s := by positivity
  exact mul_pos (mul_pos (Real.exp_pos _) (Real.rpow_pos_of_pos hs _))
    (Real.rpow_pos_of_pos hbase _)

lemma integrand_nonneg {A rho T s : Real} (hs : 0 ≤ s) (hT : 0 ≤ T) :
    0 ≤ integrand A rho T s := by
  unfold integrand
  have hbase : 0 ≤ 1 + T * s := by
    nlinarith [mul_nonneg hT hs]
  exact mul_nonneg
    (mul_nonneg (Real.exp_pos _).le (Real.rpow_nonneg hs _))
    (Real.rpow_nonneg hbase _)

lemma continuousOn_integrand_Ioi {A rho T : Real} (hT : 0 ≤ T) :
    ContinuousOn (integrand A rho T) (Ioi 0) := by
  unfold integrand
  apply ContinuousOn.mul
  · apply ContinuousOn.mul
    · fun_prop
    · exact continuousOn_id.rpow_const (by
        intro s hs
        exact Or.inl (ne_of_gt hs))
  · apply ContinuousOn.rpow_const
    · fun_prop
    · intro s hs
      left
      have hs0 : 0 < s := hs
      have hmul : 0 ≤ T * s := mul_nonneg hT hs0.le
      exact ne_of_gt (by linarith)

lemma integrand_measurableOn_Ioi {A rho T : Real} (hT : 0 ≤ T) :
    AEStronglyMeasurable (integrand A rho T) (volume.restrict (Ioi 0)) :=
  (continuousOn_integrand_Ioi hT).aestronglyMeasurable measurableSet_Ioi

lemma factor_le_one {rho x : Real} (hrho : 0 ≤ rho) (hx : 1 ≤ x) :
    x ^ (-rho) ≤ 1 := by
  calc
    x ^ (-rho) ≤ x ^ (0 : Real) :=
      Real.rpow_le_rpow_of_exponent_le hx (neg_nonpos.mpr hrho)
    _ = 1 := Real.rpow_zero x

lemma integrand_le_gamma {A rho T s : Real}
    (hs : 0 < s) (hT : 0 ≤ T) (hrho : 0 ≤ rho) :
    integrand A rho T s ≤ Real.exp (-s) * s ^ (A - 1) := by
  have hbase : 1 ≤ 1 + T * s := by
    nlinarith [mul_nonneg hT hs.le]
  have hfac := factor_le_one hrho hbase
  unfold integrand
  have hnonneg : 0 ≤ Real.exp (-s) * s ^ (A - 1) :=
    mul_nonneg (Real.exp_pos _).le (Real.rpow_nonneg hs.le _)
  exact mul_le_of_le_one_right hnonneg hfac

lemma integrableOn_integrand_Ioi {A rho T : Real}
    (hA : 0 < A) (hT : 0 ≤ T) (hrho : 0 ≤ rho) :
    IntegrableOn (integrand A rho T) (Ioi 0) := by
```

```
  have hGamma := Real.GammaIntegral_convergent hA
  apply Integrable.mono' hGamma
  · exact integrand_measurableOn_Ioi hT
  · filter_upwards [self_mem_ae_restrict measurableSet_Ioi] with s hs
    rw [Real.norm_of_nonneg (integrand_nonneg hs.le hT)]
    exact integrand_le_gamma hs hT hrho

lemma integral_nonneg {A rho T : Real} (hT : 0 ≤ T) :
    0 ≤ integral A rho T := by
  unfold integral
  apply integral_nonneg_of_ae
  filter_upwards [self_mem_ae_restrict measurableSet_Ioi] with s hs
  exact integrand_nonneg hs.le hT

end O77.Residual.Model

namespace O77.Residual.Model

def nearIntegral (A rho T : Real) : Real :=
  ∫ s : Real in Ioc 0 (1 / T), integrand A rho T s

def middleIntegral (A rho T : Real) : Real :=
  ∫ s : Real in Ioc (1 / T) 1, integrand A rho T s

def tailIntegral (A rho T : Real) : Real :=
  ∫ s : Real in Ioi 1, integrand A rho T s

lemma zero_le_inv {T : Real} (hT : 0 < T) : (0 : Real) ≤ 1 / T := by positivity

lemma inv_le_one {T : Real} (hT : 1 ≤ T) : (1 / T : Real) ≤ 1 := by
  exact (div_le_one (by linarith : 0 < T)).2 hT

lemma integral_eq_three {A rho T : Real}
    (hA : 0 < A) (hrho : 0 ≤ rho) (hT : 1 ≤ T) :
    integral A rho T =
      nearIntegral A rho T + middleIntegral A rho T + tailIntegral A rho T := by
  have hT0 : 0 < T := lt_of_lt_of_le zero_lt_one hT
  have hInt := integrableOn_integrand_Ioi hA hT0.le hrho
  have h01 : Ioc (0 : Real) (1 / T) ∪ Ioc (1 / T) 1 = Ioc 0 1 :=
    Ioc_union_Ioc_eq_Ioc (zero_le_inv hT0) (inv_le_one hT)
  have h12 : Ioc (0 : Real) 1 ∪ Ioi 1 = Ioi 0 :=
    Ioc_union_Ioi_eq_Ioi (show (0 : Real) ≤ 1 by norm_num)
  have hdis01 : Disjoint (Ioc (0 : Real) (1 / T)) (Ioc (1 / T) 1) := by
    rw [Set.disjoint_left]
    intro x hx hy
    exact (not_lt_of_ge hx.2) hy.1
  have hdis12 : Disjoint (Ioc (0 : Real) 1) (Ioi 1) := by
    rw [Set.disjoint_left]
    intro x hx hy
    exact (not_lt_of_ge hx.2) hy
  have hI0 := hInt.mono_set (show Ioc (0 : Real) (1 / T) ⊆ Ioi 0 by
    intro x hx; exact hx.1)
  have hI1 := hInt.mono_set (show Ioc (1 / T) 1 ⊆ Ioi 0 by
    intro x hx; exact (zero_le_inv hT0).trans_lt hx.1)
  have hI01 := hInt.mono_set (show Ioc (0 : Real) 1 ⊆ Ioi 0 by
    intro x hx; exact hx.1)
  have hI2 := hInt.mono_set (show Ioi (1 : Real) ⊆ Ioi 0 by
    intro x hx; exact lt_trans zero_lt_one (show (1 : Real) < x from hx))
  have hsplit01 :
      ∫( s : Real in Ioc 0 1, integrand A rho T s) =
        ∫( s : Real in Ioc 0 (1 / T), integrand A rho T s) +
          ∫ s : Real in Ioc (1 / T) 1, integrand A rho T s := by
```

```
    rw ←[ h01]
    exact ₀setIntegral_union hdis01.aedisjoint
      measurableSet_Ioc.nullMeasurableSet hI0 hI1
  have hsplit12 :
      ∫( s : Real in Ioi 0, integrand A rho T s) =
        ∫( s : Real in Ioc 0 1, integrand A rho T s) +
          ∫ s : Real in Ioi 1, integrand A rho T s := by
    rw ←[ h12]
    exact ₀setIntegral_union hdis12.aedisjoint
      measurableSet_Ioi.nullMeasurableSet hI01 hI2
  unfold integral nearIntegral middleIntegral tailIntegral
  rw [hsplit12, hsplit01]


end O77.Residual.Model

namespace O77.Residual.Model

lemma exp_neg_one_le_exp_neg {s : Real} (hs : s ≤ 1) :
    Real.exp (-1) ≤ Real.exp (-s) := by
  exact Real.exp_le_exp.mpr (neg_le_neg hs)

lemma rpow_neg_anti {x y rho : Real} (hx : 0 < x) (hxy : x ≤ y)
    (hrho : 0 ≤ rho) : y ^ (-rho) ≤ x ^ (-rho) :=
  Real.rpow_le_rpow_of_nonpos hx hxy (neg_nonpos.mpr hrho)

lemma near_pointwise_lower {A rho T s : Real}
    (hrho : 0 ≤ rho) (hT : 1 ≤ T)
    (hs0 : 0 < s) (hsT : s ≤ 1 / T) :
    Real.exp (-1) * (2 : Real) ^ (-rho) * s ^ (A - 1) ≤
      integrand A rho T s := by
  have hT0 : 0 < T := lt_of_lt_of_le zero_lt_one hT
  have hs1 : s ≤ 1 := hsT.trans (inv_le_one hT)
  have hTs0 : 0 ≤ T * s := mul_nonneg hT0.le hs0.le
  have hTs1 : T * s ≤ 1 := by
    calc
      T * s ≤ T * (1 / T) := mul_le_mul_of_nonneg_left hsT hT0.le
      _ = 1 := by field_simp [ne_of_gt hT0]
  have hbasePos : 0 < 1 + T * s := by linarith
  have hbaseTwo : 1 + T * s ≤ 2 := by linarith
  have hexp : Real.exp (-1) ≤ Real.exp (-s) := exp_neg_one_le_exp_neg hs1
  have hfac : (2 : Real) ^ (-rho) ≤ (1 + T * s) ^ (-rho) :=
    rpow_neg_anti hbasePos hbaseTwo hrho
  have hsPow : 0 ≤ s ^ (A - 1) := Real.rpow_nonneg hs0.le _
  unfold integrand
  calc
    Real.exp (-1) * 2 ^ (-rho) * s ^ (A - 1) ≤
        Real.exp (-s) * (1 + T * s) ^ (-rho) * s ^ (A - 1) := by
      gcongr
    _ = Real.exp (-s) * s ^ (A - 1) * (1 + T * s) ^ (-rho) := by ring

lemma near_pointwise_upper {A rho T s : Real}
    (hrho : 0 ≤ rho) (hT : 0 ≤ T) (hs0 : 0 < s) :
    integrand A rho T s ≤ s ^ (A - 1) := by
  have hbase : 1 ≤ 1 + T * s := by
    nlinarith [mul_nonneg hT hs0.le]
  have hfac : (1 + T * s) ^ (-rho) ≤ 1 := factor_le_one hrho hbase
  have hexp : Real.exp (-s) ≤ 1 := by
    rw ←[ Real.exp_zero]
    exact Real.exp_le_exp.mpr (by linarith)
  have hsPow : 0 ≤ s ^ (A - 1) := Real.rpow_nonneg hs0.le _
  unfold integrand
```

```
  calc
    Real.exp (-s) * s ^ (A - 1) * (1 + T * s) ^ (-rho) ≤
        1 * s ^ (A - 1) * 1 := by gcongr
    _ = s ^ (A - 1) := by ring

lemma rpow_product_normalize {A rho T s : Real}
    (hT : 0 ≤ T) (hs : 0 < s) :
    s ^ (A - 1) * (T * s) ^ (-rho) =
      T ^ (-rho) * s ^ (A - rho - 1) := by
  calc
    s ^ (A - 1) * (T * s) ^ (-rho) =
        s ^ (A - 1) * (T ^ (-rho) * s ^ (-rho)) := by
      rw [Real.mul_rpow hT hs.le]
    _ = T ^ (-rho) * (s ^ (A - 1) * s ^ (-rho)) := by ring
    _ = T ^ (-rho) * s ^ ((A - 1) + (-rho)) := by
      rw [Real.rpow_add hs]
    _ = T ^ (-rho) * s ^ (A - rho - 1) := by ring_nf

lemma middle_pointwise_upper {A rho T s : Real}
    (hrho : 0 ≤ rho) (hT : 0 < T) (hsT : 1 / T ≤ s) (hs0 : 0 < s) :
    integrand A rho T s ≤
      T ^ (-rho) * (Real.exp (-s) * s ^ (A - rho - 1)) := by
  have hTs : 1 ≤ T * s := by
    calc
      (1 : Real) = T * (1 / T) := by field_simp [ne_of_gt hT]
      _ ≤ T * s := mul_le_mul_of_nonneg_left hsT hT.le
  have hbasePos : 0 < T * s := lt_of_lt_of_le zero_lt_one hTs
  have hbaseLe : T * s ≤ 1 + T * s := by linarith
  have hfac : (1 + T * s) ^ (-rho) ≤ (T * s) ^ (-rho) :=
    rpow_neg_anti hbasePos hbaseLe hrho
  have hfront : 0 ≤ Real.exp (-s) * s ^ (A - 1) :=
    mul_nonneg (Real.exp_pos _).le (Real.rpow_nonneg hs0.le _)
  unfold integrand
  calc
    Real.exp (-s) * s ^ (A - 1) * (1 + T * s) ^ (-rho) ≤
        Real.exp (-s) * s ^ (A - 1) * (T * s) ^ (-rho) :=
      mul_le_mul_of_nonneg_left hfac hfront
    _ = Real.exp (-s) * (s ^ (A - 1) * (T * s) ^ (-rho)) := by ring
    _ = Real.exp (-s) * (T ^ (-rho) * s ^ (A - rho - 1)) := by
      rw [rpow_product_normalize (A := A) (rho := rho) hT.le hs0]
    _ = T ^ (-rho) * (Real.exp (-s) * s ^ (A - rho - 1)) := by ring

lemma middle_pointwise_lower {A rho T s : Real}
    (hrho : 0 ≤ rho) (hT : 0 < T)
    (hsT : 1 / T ≤ s) (hs1 : s ≤ 1) :
    Real.exp (-1) * (2 : Real) ^ (-rho) * T ^ (-rho) *
        s ^ (A - rho - 1) ≤ integrand A rho T s := by
  have hs0 : 0 < s := lt_of_lt_of_le (by positivity : 0 < (1 / T : Real)) hsT
  have hTs : 1 ≤ T * s := by
    calc
      (1 : Real) = T * (1 / T) := by field_simp [ne_of_gt hT]
      _ ≤ T * s := mul_le_mul_of_nonneg_left hsT hT.le
  have hTsPos : 0 < T * s := lt_of_lt_of_le zero_lt_one hTs
  have hbaseLe : 1 + T * s ≤ 2 * (T * s) := by linarith
  have hbasePos : 0 < 1 + T * s := by linarith
  have hfac : (2 * (T * s)) ^ (-rho) ≤ (1 + T * s) ^ (-rho) :=
    rpow_neg_anti hbasePos hbaseLe hrho
  have hexp : Real.exp (-1) ≤ Real.exp (-s) := exp_neg_one_le_exp_neg hs1
  have hsPow : 0 ≤ s ^ (A - 1) := Real.rpow_nonneg hs0.le _
  have hnorm :
      s ^ (A - 1) * (2 * (T * s)) ^ (-rho) =
        2 ^ (-rho) * T ^ (-rho) * s ^ (A - rho - 1) := by
```

```
    rw [Real.mul_rpow (by norm_num : (0 : Real) ≤ 2) (mul_nonneg hT.le hs0.le)]
    calc
      s ^ (A - 1) * (2 ^ (-rho) * (T * s) ^ (-rho)) =
          2 ^ (-rho) * (s ^ (A - 1) * (T * s) ^ (-rho)) := by ring
      _ = 2 ^ (-rho) * (T ^ (-rho) * s ^ (A - rho - 1)) := by
        rw [rpow_product_normalize (A := A) (rho := rho) hT.le hs0]
      _ = 2 ^ (-rho) * T ^ (-rho) * s ^ (A - rho - 1) := by ring
  unfold integrand
  calc
    Real.exp (-1) * 2 ^ (-rho) * T ^ (-rho) * s ^ (A - rho - 1) =
        Real.exp (-1) * (s ^ (A - 1) * (2 * (T * s)) ^ (-rho)) := by
      rw [hnorm]
      ring
    _ ≤ Real.exp (-s) * (s ^ (A - 1) * (1 + T * s) ^ (-rho)) := by
      gcongr
    _ = Real.exp (-s) * s ^ (A - 1) * (1 + T * s) ^ (-rho) := by ring

lemma tail_pointwise_upper {A rho T s : Real}
    (hrho : 0 ≤ rho) (hT : 1 ≤ T) (hs : 1 ≤ s) :
    integrand A rho T s ≤
      T ^ (-rho) * (Real.exp (-s) * s ^ (A - 1)) := by
  have hT0 : 0 < T := lt_of_lt_of_le zero_lt_one hT
  have hs0 : 0 < s := lt_of_lt_of_le zero_lt_one hs
  have hmid := middle_pointwise_upper (A := A) (rho := rho)
    hrho hT0 (by
      have : (1 / T : Real) ≤ 1 := inv_le_one hT
      exact this.trans hs) hs0
  have hpow : s ^ (A - rho - 1) ≤ s ^ (A - 1) := by
    apply Real.rpow_le_rpow_of_exponent_le hs
    linarith
  have hexp0 : 0 ≤ Real.exp (-s) := (Real.exp_pos _).le
  have hinside := mul_le_mul_of_nonneg_left hpow hexp0
  have hTpow : 0 ≤ T ^ (-rho) := Real.rpow_nonneg hT0.le _
  exact hmid.trans (mul_le_mul_of_nonneg_left hinside hTpow)


lemma integral_rpow_zero_inv {A T : Real} (hA : 0 < A) (hT : 0 < T) :
    ∫( s : Real in Ioc 0 (1 / T), s ^ (A - 1)) = T ^ (-A) / A := by
  rw ←[ intervalIntegral.integral_of_le (show (0 : Real) ≤ 1 / T by positivity)]
  rw [integral_rpow (Or.inl (by linarith : -1 < A - 1))]
  have hA0 : A ≠ 0 := ne_of_gt hA
  rw [show A - 1 + 1 = A by ring, Real.zero_rpow hA0, sub_zero]
  rw [one_div, Real.inv_rpow hT.le, ← Real.rpow_neg hT.le]

lemma integral_rpow_inv_one {B T : Real} (hB : B ≠ 0) (hT : 1 ≤ T) :
    ∫( s : Real in Ioc (1 / T) 1, s ^ (B - 1)) =
      (1 - T ^ (-B)) / B := by
  have hT0 : 0 < T := lt_of_lt_of_le zero_lt_one hT
  have hInvPos : 0 < (1 / T : Real) := by positivity
  have hInvLe : (1 / T : Real) ≤ 1 := (div_le_one hT0).2 hT
  rw ←[ intervalIntegral.integral_of_le hInvLe]
  have hzero : (0 : Real) ∉ uIcc (1 / T) 1 := by
    rw [uIcc_of_le hInvLe]
    intro hz
    exact (not_le_of_gt hInvPos) hz.1
  have hexp : B - 1 ≠ (-1 : Real) := by
    intro heq
    apply hB
    linarith
  rw [integral_rpow (Or.inr ⟨hexp, ⟩hzero)]
  rw [show B - 1 + 1 = B by ring, Real.one_rpow]
  rw [one_div, Real.inv_rpow hT0.le, ← Real.rpow_neg hT0.le]
```

```
lemma integral_inv_inv_one {T : Real} (hT : 1 ≤ T) :
    ∫( s : Real in Ioc (1 / T) 1, 1 / s) = Real.log T := by
  have hT0 : 0 < T := lt_of_lt_of_le zero_lt_one hT
  have hInvPos : 0 < (1 / T : Real) := by positivity
  have hInvLe : (1 / T : Real) ≤ 1 := (div_le_one hT0).2 hT
  rw ←[ intervalIntegral.integral_of_le hInvLe]
  have hzero : (0 : Real) ∉ uIcc (1 / T) 1 := by
    rw [uIcc_of_le hInvLe]
    intro hz
    exact (not_le_of_gt hInvPos) hz.1
  rw [integral_one_div hzero]
  have hne : T ≠ 0 := ne_of_gt hT0
  rw [show (1 : Real) / (1 / T) = T by field_simp]

lemma nearIntegral_lower {A rho T : Real}
    (hA : 0 < A) (hrho : 0 ≤ rho) (hT : 1 ≤ T) :
    Real.exp (-1) * (2 : Real) ^ (-rho) * (T ^ (-A) / A) ≤
      nearIntegral A rho T := by
  have hT0 : 0 < T := lt_of_lt_of_le zero_lt_one hT
  have hInv0 : (0 : Real) ≤ 1 / T := by positivity
  have hInt := integrableOn_integrand_Ioi hA hT0.le hrho
  have hIntNear : IntegrableOn (integrand A rho T) (Ioc 0 (1 / T)) :=
    hInt.mono_set (fun s hs => hs.1)
  have hPowInt : IntervalIntegrable (fun s : Real => s ^ (A - 1)) volume 0 (1 / T) :=
    intervalIntegral.intervalIntegrable_rpow' (by linarith)
  have hPowSet : IntegrableOn (fun s : Real => s ^ (A - 1)) (Ioc 0 (1 / T)) :=
    (intervalIntegrable_iff_integrableOn_Ioc_of_le hInv0).1 hPowInt
  let c : Real := Real.exp (-1) * (2 : Real) ^ (-rho)
  have hComp : IntegrableOn (fun s : Real => c * s ^ (A - 1)) (Ioc 0 (1 / T)) :=
    hPowSet.const_mul c
  have hmono :
      ∫( s : Real in Ioc 0 (1 / T), c * s ^ (A - 1)) ≤
        ∫ s : Real in Ioc 0 (1 / T), integrand A rho T s := by
    apply MeasureTheory.integral_mono_ae hComp hIntNear
    filter_upwards [ae_restrict_mem measurableSet_Ioc] with s hs
    exact near_pointwise_lower hrho hT hs.1 hs.2
  rw [MeasureTheory.integral_const_mul, integral_rpow_zero_inv hA hT0] at hmono
  exact hmono

lemma nearIntegral_upper {A rho T : Real}
    (hA : 0 < A) (hrho : 0 ≤ rho) (hT : 1 ≤ T) :
    nearIntegral A rho T ≤ T ^ (-A) / A := by
  have hT0 : 0 < T := lt_of_lt_of_le zero_lt_one hT
  have hInv0 : (0 : Real) ≤ 1 / T := by positivity
  have hInt := integrableOn_integrand_Ioi hA hT0.le hrho
  have hIntNear : IntegrableOn (integrand A rho T) (Ioc 0 (1 / T)) :=
    hInt.mono_set (fun s hs => hs.1)
  have hPowInt : IntervalIntegrable (fun s : Real => s ^ (A - 1)) volume 0 (1 / T) :=
    intervalIntegral.intervalIntegrable_rpow' (by linarith)
  have hPowSet : IntegrableOn (fun s : Real => s ^ (A - 1)) (Ioc 0 (1 / T)) :=
    (intervalIntegrable_iff_integrableOn_Ioc_of_le hInv0).1 hPowInt
  have hmono :
      ∫( s : Real in Ioc 0 (1 / T), integrand A rho T s) ≤
        ∫ s : Real in Ioc 0 (1 / T), s ^ (A - 1) := by
    apply MeasureTheory.integral_mono_ae hIntNear hPowSet
    filter_upwards [ae_restrict_mem measurableSet_Ioc] with s hs
    exact near_pointwise_upper hrho hT0.le hs.1
  rw [integral_rpow_zero_inv hA hT0] at hmono
  exact hmono
```

```
lemma middle_pointwise_upper_power {A rho T s : Real}
    (hrho : 0 ≤ rho) (hT : 0 < T) (hsT : 1 / T ≤ s) (hs0 : 0 < s) :
    integrand A rho T s ≤ T ^ (-rho) * s ^ (A - rho - 1) := by
  have hmid := middle_pointwise_upper (A := A) (rho := rho) hrho hT hsT hs0
  have hexp : Real.exp (-s) ≤ 1 := by
    rw ←[ Real.exp_zero]
    exact Real.exp_le_exp.mpr (by linarith)
  have hpow : 0 ≤ s ^ (A - rho - 1) := Real.rpow_nonneg hs0.le _
  have hinside : Real.exp (-s) * s ^ (A - rho - 1) ≤ s ^ (A - rho - 1) := by
    nlinarith [mul_le_mul_of_nonneg_right hexp hpow]
  exact hmid.trans (mul_le_mul_of_nonneg_left hinside (Real.rpow_nonneg hT.le _))

lemma middlePower_integrableOn {B T : Real} (hT : 1 ≤ T) :
    IntegrableOn (fun s : Real => s ^ (B - 1)) (Ioc (1 / T) 1) := by
  have hT0 : 0 < T := lt_of_lt_of_le zero_lt_one hT
  have hInvPos : 0 < (1 / T : Real) := by positivity
  have hInvLe : (1 / T : Real) ≤ 1 := (div_le_one hT0).2 hT
  have hzero : (0 : Real) ∉ uIcc (1 / T) 1 := by
    rw [uIcc_of_le hInvLe]
    intro hz
    exact (not_le_of_gt hInvPos) hz.1
  have hInt : IntervalIntegrable (fun s : Real => s ^ (B - 1)) volume (1 / T) 1 :=
    intervalIntegral.intervalIntegrable_rpow (Or.inr hzero)
  exact (intervalIntegrable_iff_integrableOn_Ioc_of_le hInvLe).1 hInt

lemma middleIntegral_bounds_nonresonant {A rho T : Real}
    (hA : 0 < A) (hrho : 0 ≤ rho) (hT : 1 ≤ T)
    (hneq : A ≠ rho) :
    Real.exp (-1) * (2 : Real) ^ (-rho) * T ^ (-rho) *
          ((1 - T ^ (-(A - rho))) / (A - rho)) ≤
        middleIntegral A rho T ∧
      middleIntegral A rho T ≤
        T ^ (-rho) * ((1 - T ^ (-(A - rho))) / (A - rho)) := by
  have hT0 : 0 < T := lt_of_lt_of_le zero_lt_one hT
  have hInvLe : (1 / T : Real) ≤ 1 := inv_le_one hT
  have hInt := integrableOn_integrand_Ioi hA hT0.le hrho
  have hIntMid : IntegrableOn (integrand A rho T) (Ioc (1 / T) 1) :=
    hInt.mono_set (fun s hs => (show (0 : Real) < s by
      exact (show (0 : Real) ≤ 1 / T by positivity).trans_lt hs.1))
  have hPow := middlePower_integrableOn (B := A - rho) hT
  let c : Real := Real.exp (-1) * (2 : Real) ^ (-rho) * T ^ (-rho)
  let d : Real := T ^ (-rho)
  have hCompLow : IntegrableOn (fun s : Real => c * s ^ (A - rho - 1))
      (Ioc (1 / T) 1) := hPow.const_mul c
  have hCompUp : IntegrableOn (fun s : Real => d * s ^ (A - rho - 1))
      (Ioc (1 / T) 1) := hPow.const_mul d
  have hlower :
      ∫( s : Real in Ioc (1 / T) 1, c * s ^ (A - rho - 1)) ≤
        middleIntegral A rho T := by
    apply MeasureTheory.integral_mono_ae hCompLow hIntMid
    filter_upwards [ae_restrict_mem measurableSet_Ioc] with s hs
    exact middle_pointwise_lower hrho hT0 hs.1.le hs.2
  have hupper : middleIntegral A rho T ≤
      ∫ s : Real in Ioc (1 / T) 1, d * s ^ (A - rho - 1) := by
    apply MeasureTheory.integral_mono_ae hIntMid hCompUp
    filter_upwards [ae_restrict_mem measurableSet_Ioc] with s hs
    have hs0 : 0 < s := (show (0 : Real) ≤ 1 / T by positivity).trans_lt hs.1
    exact middle_pointwise_upper_power hrho hT0 hs.1.le hs0
  have hB : A - rho ≠ 0 := sub_ne_zero.mpr hneq
  rw [MeasureTheory.integral_const_mul,
    integral_rpow_inv_one hB hT] at hlower
  rw [MeasureTheory.integral_const_mul,
```

```
    integral_rpow_inv_one hB hT] at hupper
  exact ⟨hlower, ⟩hupper

lemma middleIntegral_bounds_resonant {A T : Real}
    (hA : 0 < A) (hT : 1 ≤ T) :
    Real.exp (-1) * (2 : Real) ^ (-A) * T ^ (-A) * Real.log T ≤
        middleIntegral A A T ∧
      middleIntegral A A T ≤ T ^ (-A) * Real.log T := by
  have hT0 : 0 < T := lt_of_lt_of_le zero_lt_one hT
  have hInt := integrableOn_integrand_Ioi hA hT0.le hA.le
  have hInv0 : (0 : Real) ≤ 1 / T := by positivity
  have hIntMid : IntegrableOn (integrand A A T) (Ioc (1 / T) 1) :=
    hInt.mono_set (fun s hs => hInv0.trans_lt hs.1)
  have hInvLe : (1 / T : Real) ≤ 1 := inv_le_one hT
  have hInvPos : 0 < (1 / T : Real) := by positivity
  have hzero : (0 : Real) ∉ uIcc (1 / T) 1 := by
    rw [uIcc_of_le hInvLe]
    intro hz
    exact (not_le_of_gt hInvPos) hz.1
  have hInvInt : IntervalIntegrable (fun s : Real => 1 / s) volume (1 / T) 1 :=
    intervalIntegral.intervalIntegrable_one_div
      (fun s hs => ne_of_mem_of_not_mem hs hzero) continuousOn_id
  have hInvSet : IntegrableOn (fun s : Real => 1 / s) (Ioc (1 / T) 1) :=
    (intervalIntegrable_iff_integrableOn_Ioc_of_le hInvLe).1 hInvInt
  let c : Real := Real.exp (-1) * (2 : Real) ^ (-A) * T ^ (-A)
  let d : Real := T ^ (-A)
  have hCompLow : IntegrableOn (fun s : Real => c * (1 / s)) (Ioc (1 / T) 1) :=
    hInvSet.const_mul c
  have hCompUp : IntegrableOn (fun s : Real => d * (1 / s)) (Ioc (1 / T) 1) :=
    hInvSet.const_mul d
  have hlower :
      ∫ s : Real in Ioc (1 / T) 1, c * (1 / s)) ≤ middleIntegral A A T := by
    apply MeasureTheory.integral_mono_ae hCompLow hIntMid
    filter_upwards [ae_restrict_mem measurableSet_Ioc] with s hs
    have hs0 : 0 < s := hInv0.trans_lt hs.1
    have hraw := middle_pointwise_lower (A := A) (rho := A) hA.le hT0 hs.1.le hs.2
    rw [show A - A - 1 = (-1 : Real) by ring, Real.rpow_neg_one, ← one_div] at hraw
    exact hraw
  have hupper : middleIntegral A A T ≤
      ∫ s : Real in Ioc (1 / T) 1, d * (1 / s) := by
    apply MeasureTheory.integral_mono_ae hIntMid hCompUp
    filter_upwards [ae_restrict_mem measurableSet_Ioc] with s hs
    have hs0 : 0 < s := hInv0.trans_lt hs.1
    have hraw := middle_pointwise_upper_power (A := A) (rho := A)
      hA.le hT0 hs.1.le hs0
    rw [show A - A - 1 = (-1 : Real) by ring, Real.rpow_neg_one, ← one_div] at hraw
    exact hraw
  rw [MeasureTheory.integral_const_mul, integral_inv_inv_one hT] at hlower
  rw [MeasureTheory.integral_const_mul, integral_inv_inv_one hT] at hupper
  exact ⟨hlower, ⟩hupper

/-- The fixed finite tail constant used in the model-integral upper estimate. -/
def tailConstant (A : Real) : Real :=
  ∫ s : Real in Ioi 1, Real.exp (-s) * s ^ (A - 1)

lemma tailConstant_nonneg (A : Real) : 0 ≤ tailConstant A := by
  unfold tailConstant
  apply integral_nonneg_of_ae
  filter_upwards [ae_restrict_mem measurableSet_Ioi] with s hs
  exact mul_nonneg (Real.exp_pos _).le (Real.rpow_nonneg (le_trans zero_le_one hs.le) _)

lemma tailIntegral_upper {A rho T : Real}
```

```
    (hA : 0 < A) (hrho : 0 ≤ rho) (hT : 1 ≤ T) :
    tailIntegral A rho T ≤ T ^ (-rho) * tailConstant A := by
  have hT0 : 0 < T := lt_of_lt_of_le zero_lt_one hT
  have hGamma := Real.GammaIntegral_convergent hA
  have hTailGamma : IntegrableOn (fun s : Real => Real.exp (-s) * s ^ (A - 1))
      (Ioi 1) := hGamma.mono_set (fun s hs => show (0 : Real) < s from lt_trans zero_lt_one hs)
  have hInt := integrableOn_integrand_Ioi hA hT0.le hrho
  have hIntTail : IntegrableOn (integrand A rho T) (Ioi 1) :=
    hInt.mono_set (fun s hs => show (0 : Real) < s from lt_trans zero_lt_one hs)
  have hTpow : 0 ≤ T ^ (-rho) := Real.rpow_nonneg hT0.le _
  have hComp : IntegrableOn
      (fun s : Real => T ^ (-rho) * (Real.exp (-s) * s ^ (A - 1))) (Ioi 1) :=
    hTailGamma.const_mul (T ^ (-rho))
  have hmono : tailIntegral A rho T ≤
      ∫ s : Real in Ioi 1, T ^ (-rho) * (Real.exp (-s) * s ^ (A - 1)) := by
    apply MeasureTheory.integral_mono_ae hIntTail hComp
    filter_upwards [ae_restrict_mem measurableSet_Ioi] with s hs
    exact tail_pointwise_upper hrho hT hs.le
  rw [MeasureTheory.integral_const_mul] at hmono
  simpa [tailConstant] using hmono


/-- The positive fixed-interval power mass used when `A > rho`. -/
def fixedPowerConstant (B : Real) : Real :=
  (1 - (1 / 2 : Real) ^ B) / B

lemma fixedPowerConstant_pos {B : Real} (hB : 0 < B) :
    0 < fixedPowerConstant B := by
  unfold fixedPowerConstant
  have hpow : (1 / 2 : Real) ^ B < 1 :=
    Real.rpow_lt_one (by norm_num) (by norm_num) hB
  exact div_pos (sub_pos.mpr hpow) hB

lemma integral_rpow_half_one {B : Real} (hB : B ≠ 0) :
    ∫( s : Real in Ioc (1 / 2) 1, s ^ (B - 1)) = fixedPowerConstant B := by
  have hpow : (1 / 2 : Real) ^ B = (2 : Real) ^ (-B) := by
    rw [one_div, Real.inv_rpow (by norm_num : (0 : Real) ≤ 2),
      ← Real.rpow_neg (by norm_num : (0 : Real) ≤ 2)]
  rw [integral_rpow_inv_one (B := B) (T := (2 : Real)) hB (by norm_num)]
  simp only [fixedPowerConstant, hpow]

lemma middleIntegral_fixed_lower {A rho T : Real}
    (hA : 0 < A) (hrho : 0 ≤ rho) (hArho : rho < A) (hT : 2 ≤ T) :
    Real.exp (-1) * (2 : Real) ^ (-rho) * fixedPowerConstant (A - rho) *
        T ^ (-rho) ≤ middleIntegral A rho T := by
  have hT0 : 0 < T := lt_of_lt_of_le zero_lt_two hT
  have hT1 : 1 ≤ T := by linarith
  have hInvHalf : (1 / T : Real) ≤ 1 / 2 := by
    apply (div_le_div_iff_of_pos_left zero_lt_one hT0 (by norm_num)).2
    linarith
  have hInv0 : (0 : Real) ≤ 1 / T := by positivity
  let S : Set Real := Ioc (1 / 2) 1
  let B : Set Real := Ioc (1 / T) 1
  have hSB : S ⊆ B := by
    intro s hs
    exact (hInvHalf.trans_lt hs.1, hs).2
  have hInt := integrableOn_integrand_Ioi hA hT0.le hrho
  have hIntB : IntegrableOn (integrand A rho T) B := by
    apply hInt.mono_set
    intro s hs
    exact hInv0.trans_lt hs.1
  have hIntS : IntegrableOn (integrand A rho T) S := hIntB.mono_set hSB
```

```
  have hPowInt : IntervalIntegrable (fun s : Real => s ^ (A - rho - 1))
      volume (1 / 2) 1 :=
    intervalIntegral.intervalIntegrable_rpow (Or.inr (notMem_uIcc_of_lt (by norm_num) zero_lt_one))
  have hPowS : IntegrableOn (fun s : Real => s ^ (A - rho - 1)) S := by
    exact (intervalIntegrable_iff_integrableOn_Ioc_of_le (by norm_num)).1 hPowInt
  let c : Real := Real.exp (-1) * (2 : Real) ^ (-rho) * T ^ (-rho)
  have hComp : IntegrableOn (fun s : Real => c * s ^ (A - rho - 1)) S :=
    hPowS.const_mul c
  have hcomp_le :
      ∫ s : Real in S, c * s ^ (A - rho - 1)) ≤
        ∫ s : Real in S, integrand A rho T s := by
    apply MeasureTheory.integral_mono_ae hComp hIntS
    filter_upwards [ae_restrict_mem measurableSet_Ioc] with s hs
    have hsT : 1 / T ≤ s := hInvHalf.trans hs.1.le
    exact middle_pointwise_lower hrho hT0 hsT hs.2
  have hnonneg : 0 ≤ᵐ[volume.restrict B] (integrand A rho T) := by
    filter_upwards [ae_restrict_mem measurableSet_Ioc] with s hs
    exact integrand_nonneg (hInv0.trans hs.1.le) hT0.le
  have hset_le :
      ∫ s : Real in S, integrand A rho T s) ≤
        ∫ s : Real in B, integrand A rho T s := by
    exact MeasureTheory.integral_mono_measure
      (MeasureTheory.Measure.restrict_mono hSB le_rfl) hnonneg hIntB
  have htotal := hcomp_le.trans hset_le
  rw [MeasureTheory.integral_const_mul,
    integral_rpow_half_one (sub_ne_zero.mpr hArho.ne')] at htotal
  dsimp [S, B, c] at htotal
  unfold middleIntegral
  calc
    Real.exp (-1) * (2 : Real) ^ (-rho) * fixedPowerConstant (A - rho) *
        T ^ (-rho) =
      (Real.exp (-1) * (2 : Real) ^ (-rho) * T ^ (-rho)) *
        fixedPowerConstant (A - rho) := by ring
    _ ≤ ∫ s : Real in Ioc (1 / T) 1, integrand A rho T s := htotal



lemma nearIntegral_nonneg {A rho T : Real} (hT : 0 ≤ T) :
    0 ≤ nearIntegral A rho T := by
  unfold nearIntegral
  apply integral_nonneg_of_ae
  filter_upwards [ae_restrict_mem measurableSet_Ioc] with s hs
  exact integrand_nonneg hs.1.le hT

lemma middleIntegral_nonneg {A rho T : Real} (hT : 0 < T) :
    0 ≤ middleIntegral A rho T := by
  unfold middleIntegral
  apply integral_nonneg_of_ae
  filter_upwards [ae_restrict_mem measurableSet_Ioc] with s hs
  have hs0 : 0 ≤ s := (show (0 : Real) ≤ 1 / T by positivity).trans hs.1.le
  exact integrand_nonneg hs0 hT.le

lemma tailIntegral_nonneg {A rho T : Real} (hT : 0 ≤ T) :
    0 ≤ tailIntegral A rho T := by
  unfold tailIntegral
  apply integral_nonneg_of_ae
  filter_upwards [ae_restrict_mem measurableSet_Ioi] with s hs
  exact integrand_nonneg (lt_trans zero_lt_one hs).le hT

lemma nearIntegral_le_integral {A rho T : Real}
    (hA : 0 < A) (hrho : 0 ≤ rho) (hT : 1 ≤ T) :
    nearIntegral A rho T ≤ integral A rho T := by
```

```
  rw [integral_eq_three hA hrho hT]
  have hT0 : 0 < T := lt_of_lt_of_le zero_lt_one hT
  nlinarith [middleIntegral_nonneg (A := A) (rho := rho) hT0,
    tailIntegral_nonneg (A := A) (rho := rho) hT0.le]

lemma middleIntegral_le_integral {A rho T : Real}
    (hA : 0 < A) (hrho : 0 ≤ rho) (hT : 1 ≤ T) :
    middleIntegral A rho T ≤ integral A rho T := by
  rw [integral_eq_three hA hrho hT]
  have hT0 : 0 < T := lt_of_lt_of_le zero_lt_one hT
  nlinarith [nearIntegral_nonneg (A := A) (rho := rho) hT0.le,
    tailIntegral_nonneg (A := A) (rho := rho) hT0.le]

lemma rpow_neg_antitone_of_one_le {T A rho : Real}
    (hT : 1 ≤ T) (hArho : A ≤ rho) :
    T ^ (-rho) ≤ T ^ (-A) :=
  Real.rpow_le_rpow_of_exponent_le hT (neg_le_neg hArho)

lemma middleIntegral_upper_of_lt {A rho T : Real}
    (hA : 0 < A) (hrho : 0 ≤ rho) (hArho : A < rho) (hT : 1 ≤ T) :
    middleIntegral A rho T ≤ T ^ (-A) / (rho - A) := by
  have hraw := (middleIntegral_bounds_nonresonant hA hrho hT hArho.ne).2
  have hT0 : 0 < T := lt_of_lt_of_le zero_lt_one hT
  have hD : 0 < rho - A := sub_pos.mpr hArho
  have hpow0 : 0 ≤ T ^ (rho - A) := Real.rpow_nonneg hT0.le _
  have hpow1 : 1 ≤ T ^ (rho - A) := by
    rw ←[ Real.rpow_zero T]
    exact Real.rpow_le_rpow_of_exponent_le hT hD.le
  have hfrac : (T ^ (rho - A) - 1) / (rho - A) ≤ T ^ (rho - A) / (rho - A) := by
    exact div_le_div_of_nonneg_right (sub_le_self _ zero_le_one) hD.le
  have hrewrite :
      (1 - T ^ (-(A - rho))) / (A - rho) =
        (T ^ (rho - A) - 1) / (rho - A) := by
    have hexp : -(A - rho) = rho - A := by ring
    have hden : A - rho = -(rho - A) := by ring
    rw [hexp, hden, div_neg, ← neg_div]
    congr 1
    ring
  rw [hrewrite] at hraw
  calc
    middleIntegral A rho T ≤
        T ^ (-rho) * ((T ^ (rho - A) - 1) / (rho - A)) := hraw
    _ ≤ T ^ (-rho) * (T ^ (rho - A) / (rho - A)) := by
      exact mul_le_mul_of_nonneg_left hfrac (Real.rpow_nonneg hT0.le _)
    _ = T ^ (-A) / (rho - A) := by
      rw [mul_div, ← Real.rpow_add hT0]
      congr 2
      ring



/-- Explicit lower constant in the regime `A < rho`. -/
def lowerConstantLt (A rho : Real) : Real :=
  Real.exp (-1) * (2 : Real) ^ (-rho) / A

/-- Explicit upper constant in the regime `A < rho`. -/
def upperConstantLt (A rho : Real) : Real :=
  1 / A + 1 / (rho - A) + tailConstant A

lemma lowerConstantLt_pos {A rho : Real} (hA : 0 < A) :
    0 < lowerConstantLt A rho := by
  unfold lowerConstantLt
```

```
  positivity

lemma upperConstantLt_pos {A rho : Real} (hA : 0 < A) (hArho : A < rho) :
    0 < upperConstantLt A rho := by
  unfold upperConstantLt
  have hD : 0 < rho - A := sub_pos.mpr hArho
  have htail : 0 ≤ tailConstant A := tailConstant_nonneg A
  positivity

lemma tailIntegral_upper_of_lt {A rho T : Real}
    (hA : 0 < A) (hrho : 0 ≤ rho) (hArho : A ≤ rho) (hT : 1 ≤ T) :
    tailIntegral A rho T ≤ T ^ (-A) * tailConstant A := by
  calc
    tailIntegral A rho T ≤ T ^ (-rho) * tailConstant A :=
      tailIntegral_upper hA hrho hT
    _ ≤ T ^ (-A) * tailConstant A := by
      exact mul_le_mul_of_nonneg_right
        (rpow_neg_antitone_of_one_le hT hArho) (tailConstant_nonneg A)

lemma integral_bounds_of_lt {A rho T : Real}
    (hA : 0 < A) (hrho : 0 ≤ rho) (hArho : A < rho) (hT : 2 ≤ T) :
    lowerConstantLt A rho * T ^ (-A) ≤ integral A rho T ∧
      integral A rho T ≤ upperConstantLt A rho * T ^ (-A) := by
  have hT1 : 1 ≤ T := by linarith
  have hnearLower := nearIntegral_lower hA hrho hT1
  have hnearLe := nearIntegral_le_integral hA hrho hT1
  have hlower : lowerConstantLt A rho * T ^ (-A) ≤ integral A rho T := by
    calc
      lowerConstantLt A rho * T ^ (-A) =
          Real.exp (-1) * (2 : Real) ^ (-rho) * (T ^ (-A) / A) := by
        unfold lowerConstantLt
        field_simp [ne_of_gt hA]
      _ ≤ nearIntegral A rho T := hnearLower
      _ ≤ integral A rho T := hnearLe
  have hnearUpper := nearIntegral_upper hA hrho hT1
  have hmiddleUpper := middleIntegral_upper_of_lt hA hrho hArho hT1
  have htailUpper := tailIntegral_upper_of_lt hA hrho hArho.le hT1
  have hupper : integral A rho T ≤ upperConstantLt A rho * T ^ (-A) := by
    rw [integral_eq_three hA hrho hT1]
    calc
      nearIntegral A rho T + middleIntegral A rho T + tailIntegral A rho T ≤
          T ^ (-A) / A + T ^ (-A) / (rho - A) +
            T ^ (-A) * tailConstant A := by
        gcongr
      _ = upperConstantLt A rho * T ^ (-A) := by
        unfold upperConstantLt
        ring
  exact ⟨hlower, ⟩hupper



lemma nearIntegral_upper_of_gt {A rho T : Real}
    (hA : 0 < A) (hrho : 0 ≤ rho) (hArho : rho ≤ A) (hT : 1 ≤ T) :
    nearIntegral A rho T ≤ T ^ (-rho) / A := by
  calc
    nearIntegral A rho T ≤ T ^ (-A) / A := nearIntegral_upper hA hrho hT
    _ ≤ T ^ (-rho) / A := by
      exact div_le_div_of_nonneg_right
        (Real.rpow_le_rpow_of_exponent_le hT (neg_le_neg hArho)) hA.le

lemma middleIntegral_upper_of_gt {A rho T : Real}
    (hA : 0 < A) (hrho : 0 ≤ rho) (hArho : rho < A) (hT : 1 ≤ T) :
```

```
    middleIntegral A rho T ≤ T ^ (-rho) / (A - rho) := by
  have hraw := (middleIntegral_bounds_nonresonant hA hrho hT hArho.ne').2
  have hT0 : 0 < T := lt_of_lt_of_le zero_lt_one hT
  have hD : 0 < A - rho := sub_pos.mpr hArho
  have hpow0 : 0 ≤ T ^ (-(A - rho)) := Real.rpow_nonneg hT0.le _
  have hnum : 1 - T ^ (-(A - rho)) ≤ 1 := sub_le_self _ hpow0
  have hfrac : (1 - T ^ (-(A - rho))) / (A - rho) ≤ 1 / (A - rho) := by
    exact div_le_div_of_nonneg_right hnum hD.le
  calc
    middleIntegral A rho T ≤
        T ^ (-rho) * ((1 - T ^ (-(A - rho))) / (A - rho)) := hraw
    _ ≤ T ^ (-rho) * (1 / (A - rho)) := by
      exact mul_le_mul_of_nonneg_left hfrac (Real.rpow_nonneg hT0.le _)
    _ = T ^ (-rho) / (A - rho) := by ring

/-- Explicit lower constant in the regime `rho < A`. -/
def lowerConstantGt (A rho : Real) : Real :=
  Real.exp (-1) * (2 : Real) ^ (-rho) * fixedPowerConstant (A - rho)

/-- Explicit upper constant in the regime `rho < A`. -/
def upperConstantGt (A rho : Real) : Real :=
  1 / A + 1 / (A - rho) + tailConstant A

lemma lowerConstantGt_pos {A rho : Real} (hrhoA : rho < A) :
    0 < lowerConstantGt A rho := by
  unfold lowerConstantGt
  have hD : 0 < A - rho := sub_pos.mpr hrhoA
  exact mul_pos (mul_pos (Real.exp_pos _) (Real.rpow_pos_of_pos (by norm_num) _))
    (fixedPowerConstant_pos hD)

lemma upperConstantGt_pos {A rho : Real} (hA : 0 < A) (hrhoA : rho < A) :
    0 < upperConstantGt A rho := by
  unfold upperConstantGt
  have hD : 0 < A - rho := sub_pos.mpr hrhoA
  have htail : 0 ≤ tailConstant A := tailConstant_nonneg A
  positivity

lemma integral_bounds_of_gt {A rho T : Real}
    (hA : 0 < A) (hrho : 0 ≤ rho) (hrhoA : rho < A) (hT : 2 ≤ T) :
    lowerConstantGt A rho * T ^ (-rho) ≤ integral A rho T ∧
      integral A rho T ≤ upperConstantGt A rho * T ^ (-rho) := by
  have hT1 : 1 ≤ T := by linarith
  have hmiddleLower := middleIntegral_fixed_lower hA hrho hrhoA hT
  have hmiddleLe := middleIntegral_le_integral hA hrho hT1
  have hlower : lowerConstantGt A rho * T ^ (-rho) ≤ integral A rho T := by
    calc
      lowerConstantGt A rho * T ^ (-rho) =
          Real.exp (-1) * (2 : Real) ^ (-rho) * fixedPowerConstant (A - rho) *
            T ^ (-rho) := rfl
      _ ≤ middleIntegral A rho T := hmiddleLower
      _ ≤ integral A rho T := hmiddleLe
  have hnearUpper := nearIntegral_upper_of_gt hA hrho hrhoA.le hT1
  have hmiddleUpper := middleIntegral_upper_of_gt hA hrho hrhoA hT1
  have htailUpper := tailIntegral_upper hA hrho hT1
  have hupper : integral A rho T ≤ upperConstantGt A rho * T ^ (-rho) := by
    rw [integral_eq_three hA hrho hT1]
    calc
      nearIntegral A rho T + middleIntegral A rho T + tailIntegral A rho T ≤
          T ^ (-rho) / A + T ^ (-rho) / (A - rho) +
            T ^ (-rho) * tailConstant A := by
        gcongr
      _ = upperConstantGt A rho * T ^ (-rho) := by
```

```
        unfold upperConstantGt
        ring
  exact ⟨hlower, ⟩hupper



lemma one_le_log_of_exp_one_le {T : Real} (hT : Real.exp 1 ≤ T) :
    1 ≤ Real.log T := by
  have hlog := Real.log_le_log (Real.exp_pos 1) hT
  simpa using hlog

/-- Explicit lower constant in the resonant regime `A = rho`. -/
def lowerConstantEq (A : Real) : Real :=
  Real.exp (-1) * (2 : Real) ^ (-A)

/-- Explicit upper constant in the resonant regime `A = rho`. -/
def upperConstantEq (A : Real) : Real :=
  1 / A + 1 + tailConstant A

lemma lowerConstantEq_pos (A : Real) : 0 < lowerConstantEq A := by
  unfold lowerConstantEq
  positivity

lemma upperConstantEq_pos {A : Real} (hA : 0 < A) : 0 < upperConstantEq A := by
  unfold upperConstantEq
  have htail : 0 ≤ tailConstant A := tailConstant_nonneg A
  positivity

lemma nearIntegral_upper_resonant {A T : Real}
    (hA : 0 < A) (hT : Real.exp 1 ≤ T) :
    nearIntegral A A T ≤ (T ^ (-A) / A) * Real.log T := by
  have hT1 : 1 ≤ T := by
    exact (show (1 : Real) ≤ Real.exp 1 by
      exact (Real.one_lt_exp_iff.mpr zero_lt_one).le).trans hT
  have hnear := nearIntegral_upper hA hA.le hT1
  have hpow : 0 ≤ T ^ (-A) / A := div_nonneg (Real.rpow_nonneg (le_trans zero_le_one hT1) _) hA.le
  exact hnear.trans (le_mul_of_one_le_right hpow (one_le_log_of_exp_one_le hT))

lemma tailIntegral_upper_resonant {A T : Real}
    (hA : 0 < A) (hT : Real.exp 1 ≤ T) :
    tailIntegral A A T ≤ (T ^ (-A) * tailConstant A) * Real.log T := by
  have hT1 : 1 ≤ T := by
    exact (show (1 : Real) ≤ Real.exp 1 by
      exact (Real.one_lt_exp_iff.mpr zero_lt_one).le).trans hT
  have htail := tailIntegral_upper hA hA.le hT1
  have hnonneg : 0 ≤ T ^ (-A) * tailConstant A :=
    mul_nonneg (Real.rpow_nonneg (le_trans zero_le_one hT1) _) (tailConstant_nonneg A)
  exact htail.trans (le_mul_of_one_le_right hnonneg (one_le_log_of_exp_one_le hT))

lemma integral_bounds_of_eq {A T : Real}
    (hA : 0 < A) (hT : Real.exp 1 ≤ T) :
    lowerConstantEq A * (T ^ (-A) * Real.log T) ≤ integral A A T ∧
      integral A A T ≤ upperConstantEq A * (T ^ (-A) * Real.log T) := by
  have hT1 : 1 ≤ T := by
    exact (show (1 : Real) ≤ Real.exp 1 by
      exact (Real.one_lt_exp_iff.mpr zero_lt_one).le).trans hT
  have hmiddle := middleIntegral_bounds_resonant hA hT1
  have hmiddleLe := middleIntegral_le_integral hA hA.le hT1
  have hlower : lowerConstantEq A * (T ^ (-A) * Real.log T) ≤ integral A A T := by
    calc
      lowerConstantEq A * (T ^ (-A) * Real.log T) =
          Real.exp (-1) * (2 : Real) ^ (-A) * T ^ (-A) * Real.log T := by
```

```
        unfold lowerConstantEq
        ring
      _ ≤ middleIntegral A A T := hmiddle.1
      _ ≤ integral A A T := hmiddleLe
  have hnear := nearIntegral_upper_resonant hA hT
  have htail := tailIntegral_upper_resonant hA hT
  have hupper : integral A A T ≤ upperConstantEq A * (T ^ (-A) * Real.log T) := by
    rw [integral_eq_three hA hA.le hT1]
    calc
      nearIntegral A A T + middleIntegral A A T + tailIntegral A A T ≤
          (T ^ (-A) / A) * Real.log T +
            T ^ (-A) * Real.log T +
            (T ^ (-A) * tailConstant A) * Real.log T := by
        exact add_le_add (add_le_add hnear hmiddle.2) htail
      _ = upperConstantEq A * (T ^ (-A) * Real.log T) := by
        unfold upperConstantEq
        ring
  exact ⟨hlower, ⟩hupper



/-- Uniform two-sided power-log bounds for the one-dimensional model integral. -/
def HasPowerLogBounds (A rho : Real) : Prop :=
  ∃ c C ₀T : Real,
    0 < c ∧ c ≤ C ∧ 2 ≤ ₀T ∧
      ∀ T : Real, ₀T ≤ T →
        c * gauge A rho T ≤ integral A rho T ∧
          integral A rho T ≤ C * gauge A rho T

lemma gauge_nonneg {A rho T : Real} (hT : 1 ≤ T) :
    0 ≤ gauge A rho T := by
  unfold gauge
  have hpow : 0 ≤ T ^ (-min A rho) :=
    Real.rpow_nonneg (le_trans zero_le_one hT) _
  split_ifs with hEq
  · exact mul_nonneg hpow (Real.log_nonneg hT)
  · simpa using hpow

/-- Complete one-dimensional model-integral estimate, including the unique logarithmic resonance. -/
theorem modelIntegral_powerLog_bounds {A rho : Real}
    (hA : 0 < A) (hrho : 0 < rho) : HasPowerLogBounds A rho := by
  rcases lt_trichotomy A rho with hlt | heq | hgt
  · let l : Real := lowerConstantLt A rho
    let u : Real := upperConstantLt A rho
    let c : Real := min l u
    refine ⟨c, u, 2, ?_, ?_, le_rfl, ?⟩_
    · dsimp [c, l, u]
      exact lt_min (lowerConstantLt_pos hA) (upperConstantLt_pos hA hlt)
    · dsimp [c]
      exact min_le_right _ _
    · intro T hT
      have hb := integral_bounds_of_lt hA hrho.le hlt hT
      have hg : gauge A rho T = T ^ (-A) := by
        simp [gauge, min_eq_left hlt.le, hlt.ne]
      rw [hg]
      have hpow : 0 ≤ T ^ (-A) :=
        Real.rpow_nonneg (le_trans zero_le_two hT) _
      constructor
      · calc
          c * T ^ (-A) ≤ l * T ^ (-A) := by
            exact mul_le_mul_of_nonneg_right (min_le_left l u) hpow
          _ ≤ integral A rho T := hb.1
```

```
      · exact hb.2
  · subst rho
    let l : Real := lowerConstantEq A
    let u : Real := upperConstantEq A
    let c : Real := min l u
    let ₀T : Real := max 2 (Real.exp 1)
    refine ⟨c, u, ₀T, ?_, ?_, ?_, ?⟩_
    · dsimp [c, l, u]
      exact lt_min (lowerConstantEq_pos A) (upperConstantEq_pos hA)
    · dsimp [c]
      exact min_le_right _ _
    · dsimp [₀T]
      exact le_max_left _ _
    · intro T hT
      have hExp : Real.exp 1 ≤ T := (le_max_right 2 (Real.exp 1)).trans hT
      have hb := integral_bounds_of_eq hA hExp
      have hg : gauge A A T = T ^ (-A) * Real.log T := by
        simp [gauge]
      rw [hg]
      have hT1 : 1 ≤ T := by
        exact (show (1 : Real) ≤ Real.exp 1 by
          exact (Real.one_lt_exp_iff.mpr zero_lt_one).le).trans hExp
      have hpow : 0 ≤ T ^ (-A) * Real.log T :=
        mul_nonneg (Real.rpow_nonneg (le_trans zero_le_one hT1) _)
          (Real.log_nonneg hT1)
      constructor
      · calc
          c * (T ^ (-A) * Real.log T) ≤
              l * (T ^ (-A) * Real.log T) := by
            exact mul_le_mul_of_nonneg_right (min_le_left l u) hpow
          _ ≤ integral A A T := hb.1
      · exact hb.2
  · let l : Real := lowerConstantGt A rho
    let u : Real := upperConstantGt A rho
    let c : Real := min l u
    refine ⟨c, u, 2, ?_, ?_, le_rfl, ?⟩_
    · dsimp [c, l, u]
      exact lt_min (lowerConstantGt_pos hgt) (upperConstantGt_pos hA hgt)
    · dsimp [c]
      exact min_le_right _ _
    · intro T hT
      have hb := integral_bounds_of_gt hA hrho.le hgt hT
      have hg : gauge A rho T = T ^ (-rho) := by
        simp [gauge, min_eq_right hgt.le, hgt.ne']
      rw [hg]
      have hpow : 0 ≤ T ^ (-rho) :=
        Real.rpow_nonneg (le_trans zero_le_two hT) _
      constructor
      · calc
          c * T ^ (-rho) ≤ l * T ^ (-rho) := by
            exact mul_le_mul_of_nonneg_right (min_le_left l u) hpow
          _ ≤ integral A rho T := hb.1
      · exact hb.2


/-- Exact regression for the near power integral. -/
theorem integral_rpow_zero_inv_example :
    ∫( s : Real in Ioc 0 (1 / 2), s ^ ((1 : Real) - 1)) = 1 / 2 := by
  rw [integral_rpow_zero_inv (A := (1 : Real)) (T := (2 : Real)) (by norm_num) (by norm_num)]
  norm_num

/-- Exact regression for the nonresonant middle power integral. -/
```

```
theorem integral_rpow_inv_one_example :
    ∫( s : Real in Ioc (1 / 2) 1, s ^ ((1 : Real) - 1)) = 1 / 2 := by
  rw [integral_rpow_inv_one (B := (1 : Real)) (T := (2 : Real)) (by norm_num) (by norm_num)]
  norm_num

/-- Exact regression for the resonant logarithmic integral. -/
theorem integral_inv_inv_one_example :
    ∫( s : Real in Ioc (1 / 2) 1, 1 / s) = Real.log 2 := by
  exact integral_inv_inv_one (T := (2 : Real)) (by norm_num)

/-- A concrete `A < rho` specialization of the complete theorem. -/
theorem modelIntegral_powerLog_bounds_lt_example :
    HasPowerLogBounds 1 2 := by
  exact modelIntegral_powerLog_bounds (by norm_num) (by norm_num)

/-- A concrete resonant specialization of the complete theorem. -/
theorem modelIntegral_powerLog_bounds_eq_example :
    HasPowerLogBounds 1 1 := by
  exact modelIntegral_powerLog_bounds (by norm_num) (by norm_num)

/-- A concrete `rho < A` specialization of the complete theorem. -/
theorem modelIntegral_powerLog_bounds_gt_example :
    HasPowerLogBounds 2 1 := by
  exact modelIntegral_powerLog_bounds (by norm_num) (by norm_num)

end O77.Residual.Model
```

Lemma 19.5 (Ordered chamber estimate)**.**

Corresponding Lean declaration(s): `O77.Residual.spectralA_strictMono`,`O77.Residual.HasSpectralPattern`,`O77.Residual.hasSpectralPattern`,`O77.Residual.ChamberProductGaugeBounds`,`O77.Residual.laguerreChamberIntegral_upper_chamberGauge`.

Corresponding Lean declaration(s): `O77.Residual.laguerreChamber_productGauge_bounds`,`O77.Residual.chamberExponent`,`O77.Residual.chamberResonanceCount`,`O77.Residual.chamberPowerLogGauge`,`O77.Residual.finset_prod_rpow`.

Corresponding Lean declaration(s): `O77.Residual.prod_ite_const_eq_pow_card_filter`,`O77.Residual.chamberGaugeProduct_eq_powerLog`,`O77.Residual.chamberResonanceCount_le_one`,`O77.Residual.ChamberPowerLogBounds`,`O77.Residual.laguerreChamber_powerLog_bounds`.

For $k \in \mathbb{N}$, $\eta > -1$ and $\rho > 0$, put $A_i = \eta + i$ for $1 \le i \le k$ and

$$E_* = \sum_{i=1}^{k} \min(A_i, \rho), \qquad m_* = 1 + \#\{i \in \{1, \dots, k\} : A_i = \rho\}.$$

With finite product Lebesgue measure on $\mathbb{R}^k$, define

$$I_{k,\eta,\rho}(T) = \int_{0<s_1\le\cdots\le s_k} e^{-\sum_i s_i} \prod_i s_i^{\eta} \prod_{i<j} (s_j - s_i) \prod_i (1 + T s_i)^{-\rho}\, ds.$$

There are $0 < c \le C$ and $T_0 \ge 16^{k+1}$ such that, for every $T \ge T_0$,

$$c T^{-E_*} (\log T)^{m_*-1} \le I_{k,\eta,\rho}(T) \le C T^{-E_*} (\log T)^{m_*-1}.$$

Moreover $m_* \in \{1, 2\}$. For $k = 0$, the integral and empty product are 1, $E_* = 0$, and $m_* = 1$.

Proof. The case $k = 0$ is the empty finite product: the function space $\mathbb{R}^0$ is a singleton, the ordered chamber is the whole space, and every integrand factor is an empty product. Assume only for the remaining discussion that $k > 0$.

On the ordered chamber, Lemma 20.8 gives

$$0 \le \prod_{i<j}(s_j - s_i) \le \prod_{j=1}^{k} s_j^{j-1}.$$

Extend each one-dimensional model kernel by zero outside $(0, \infty)$. The product of these extensions is integrable, and the chamber indicator is pointwise bounded by that product. Enlarging to the full orthant and applying the finite-product integral theorem gives

$$I_{k,\eta,\rho}(T) \le \prod_{j=1}^{k} \int_0^\infty e^{-s} s^{\eta+j-1}(1+Ts)^{-\rho}\, ds.$$

Lemma 19.4 bounds the jth factor by a constant times $T^{-\min(A_j,\rho)}(\log T)^{[A_j=\rho]}$. Taking a common finite threshold and multiplying the positive constants proves the upper estimate.

For the lower estimate, strict monotonicity of A_i gives exactly one of two patterns. Either there is a cut c such that $A_i < \rho$ for $i < c$ and $\rho < A_i$ for $i \ge c$, or there is a unique resonant index r with $A_r = \rho$, with strict inequalities on either side. Use the nonresonant or resonant box of Lemma 20.12. Formally the rectangles are taken half-open, $(\ell_i, u_i]$; replacing displayed closed endpoints by half-open ones changes neither the integral nor any pointwise estimate. For $T \ge 16^{k+1}$, every box point is positive and satisfies $2s_i \le s_j$ for $i < j$, hence

$$2^{-\binom{k}{2}} \prod_j s_j^{j-1} \le \prod_{i<j}(s_j - s_i).$$

The box is measurable and lies in the chamber. Restricting the nonnegative integral to it, using the pointwise comparison, and applying Tonelli gives exactly the product of the localized one-dimensional integrals. Lemma 20.11 supplies T^{-A_i} on each scaled coordinate, $T^{-\rho}$ on each fixed coordinate, and $T^{-\rho}\log T$ on the unique resonant coordinate. The finite product of the explicit positive constants is independent of T. This gives exponent E_* and precisely one logarithm if and only if resonance occurs. Since $A_{i+1} - A_i = 1$, there is at most one resonant index, so $m_* \in \{1, 2\}$.

Provenance: Sneiderman, Appendix A, especially the upper product estimate and the explicit separated vertex/edge sectors.

□

Lemma 19.6 (Checked reversal of chamber-prefix costs)**.**

Corresponding Lean declaration(s): `O77.Arithmetic.chamberK`,`O77.Arithmetic.chamberD`,`O77.Arithmetic.chamberTwiceCost`,`O77.Arithmetic.chamberTwiceMinimum`,`O77.Arithmetic.chamberMinimizerList`,`O77.Arithmetic.chamberMultiplicity`.

Corresponding Lean declaration(s): `O77.Arithmetic.chamberTwiceCost_eq_residualQuadratic`,`O77.Arithmetic.chamberTwiceMinimum_eq_residualMin_cast`,`O77.Arithmetic.mem_chamberMinimizerList_iff_reversed`,`O77.Arithmetic.map_chamberMinimizerList_reverse`,`O77.Arithmetic.chamberMultiplicity_eq_residualMultiplicity`,`O77.Arithmetic.chamberMultiplicity_formula`.

Let $p, q, h \in \mathbb{N}$, put $k = \min(h, q)$ and $d = \max(h, q)$, and define in signed arithmetic

$$C_{p,q,h}(\ell) = pk + \ell(d - k - p) + \ell^2, \qquad 0 \le \ell \le k.$$

Then, after coercion to $\mathbb{Z}$*,*

$$C_{p,q,h}(\ell) = hq + (k-\ell)(p-h-q) + (k-\ell)^2 = c_{p,q,h}(k-\ell).$$

Consequently the minimum of $C_{p,q,h}$ *on* $[0,k]$ *is* $d_0(p,q,h)$*. Moreover, mapping each chamber minimizer* ℓ *to* $k-\ell$ *gives the reverse of the increasing residual-minimizer list. In particular the two lists have the same length, and that length is*

$$\begin{cases} 2, & p+q+h \text{ odd and } p \le q+h,\ q \le p+h,\ h \le p+q, \\ 1, & \text{otherwise.} \end{cases}$$

These assertions include $k = 0$ *and all cases in which one residual dimension vanishes.*

Proof. If $h \le q$, then $(k,d) = (h,q)$; if $q \le h$, then $(k,d) = (q,h)$. Thus in either case $kd = hq$ and $k + d = h + q$. For $0 \le \ell \le k$ set $t = k - \ell$. Expanding only in $\mathbb{Z}$ gives

$$\begin{aligned} C_{p,q,h}(\ell) &= pk + (k-t)(d-k-p) + (k-t)^2 \\ &= kd + t(p-d-k) + t^2 \\ &= hq + t(p-h-q) + t^2. \end{aligned}$$

Since $t \le k \le h, q$, the last signed polynomial is the coercion of $(h-t)(q-t) + pt = c_{p,q,h}(t)$. The map $\ell \mapsto k-\ell$ is an involutive bijection of $\{0,\dots,k\}$, so it preserves the minimum and the complete minimizer set. Both executable lists enumerate their minimizers without repetition in increasing order. Reversal changes the increasing chamber list to the decreasing residual list, which is exactly the reverse of the residual enumeration. Taking lengths gives equality of multiplicities, and the displayed formula is Lemma 12.19. When $k = 0$ both intervals contain only 0; no separate cancellation or positivity argument is needed. □

```
namespace O77
namespace Arithmetic

/-- The smaller and larger Gram dimensions in the residual spectral reduction. -/
def chamberK (h q : Nat) : Nat := min h q

def chamberD (h q : Nat) : Nat := max h q

/-- Twice the prefix exponent obtained from the ordered Laguerre chamber.
The signed coefficient is formed in `Int`; no truncated subtraction occurs. -/
def chamberTwiceCost (p q h ell : Nat) : Int :=
  let k := chamberK h q
  let d := chamberD h q
  (p : Int) * (k : Int) +
    (ell : Int) * ((d : Int) - (k : Int) - (p : Int)) +
    (ell : Int) ^ 2

def chamberTwiceMinimum (p q h : Nat) : Int :=
  minimumUpTo (chamberTwiceCost p q h) (chamberK h q)

def chamberMinimizerList (p q h : Nat) : List Nat :=
  minimizerList (chamberTwiceCost p q h) (chamberK h q)

def chamberMultiplicity (p q h : Nat) : Nat :=
  (chamberMinimizerList p q h).length

private theorem min_mul_max (h q : Nat) : min h q * max h q = h * q := by
  by_cases hle : h ≤ q
  · simp [Nat.min_eq_left hle, Nat.max_eq_right hle]
```

```
  · have hge : q ≤ h := by omega
    simp [Nat.min_eq_right hge, Nat.max_eq_left hge, Nat.mul_comm]

private theorem min_add_max (h q : Nat) : min h q + max h q = h + q := by
  by_cases hle : h ≤ q
  · simp [Nat.min_eq_left hle, Nat.max_eq_right hle]
  · have hge : q ≤ h := by omega
    simp [Nat.min_eq_right hge, Nat.max_eq_left hge, Nat.add_comm]

/-- Reversing the chamber prefix index gives exactly the residual rank cost. -/
theorem chamberTwiceCost_eq_residualQuadratic (p q h ell : Nat)
    (hell : ell ≤ chamberK h q) :
    chamberTwiceCost p q h ell =
      residualQuadratic p q h (chamberK h q - ell) := by
  let k := chamberK h q
  let d := chamberD h q
  have hke : ell ≤ k := hell
  have hprod : k * d = h * q := by
    dsimp [k, d, chamberK, chamberD]
    exact min_mul_max h q
  have hsum : k + d = h + q := by
    dsimp [k, d, chamberK, chamberD]
    exact min_add_max h q
  simp only [chamberTwiceCost, residualQuadratic, quadratic]
  rw [Int.natCast_sub hke]
  have hprodI : (k : Int) * d = (h : Int) * q := by
    exact_mod_cast hprod
  have hsumI : (k : Int) + d = (h : Int) + q := by
    exact_mod_cast hsum
  grind

/-- The spectral prefix minimum is the residual minimum, before division by two. -/
theorem chamberTwiceMinimum_eq_residualMin_cast (p q h : Nat) :
    chamberTwiceMinimum p q h = (residualMin p q h : Int) := by
  let k := chamberK h q
  apply Int.le_antisymm
  · rcases minimumUpTo_attained (residualQuadratic p q h) k with
      ⟨t, ht, ⟩hval
    have hell : k - t ≤ k := by omega
    have hrev : k - (k - t) = t := by omega
    have hle := minimumUpTo_le (chamberTwiceCost p q h) hell
    rw [chamberTwiceCost_eq_residualQuadratic p q h (k - t) hell, hrev,
      hval] at hle
    simpa [chamberTwiceMinimum, k, chamberK, residualMin_cast] using hle
  · rcases minimumUpTo_attained (chamberTwiceCost p q h) k with
      ⟨ell, hell, ⟩hval
    have ht : k - ell ≤ k := by omega
    have hle := minimumUpTo_le (residualQuadratic p q h) ht
    rw ←[ chamberTwiceCost_eq_residualQuadratic p q h ell hell, hval] at hle
    simpa [chamberTwiceMinimum, k, chamberK, residualMin_cast] using hle

/-- On the finite interval, reversal identifies the complete minimizer predicates. -/
theorem mem_chamberMinimizerList_iff_reversed (p q h ell : Nat)
    (hell : ell ≤ chamberK h q) :
    ell ∈ chamberMinimizerList p q h ↔
      chamberK h q - ell ∈ residualMinimizerList p q h := by
  unfold chamberMinimizerList residualMinimizerList
  rw [mem_minimizerList, mem_minimizerList,
    isMinimumOn_iff_value, isMinimumOn_iff_value]
  have hmin :
      minimumUpTo (chamberTwiceCost p q h) (chamberK h q) =
        minimumUpTo (residualQuadratic p q h) (min h q) := by
```

```
    simpa [chamberTwiceMinimum, residualMin_cast] using
      chamberTwiceMinimum_eq_residualMin_cast p q h
  constructor
  · rintro ⟨_, ⟩hval
    refine ⟨by simp [chamberK], ?⟩_
    calc
      residualQuadratic p q h (chamberK h q - ell) =
          chamberTwiceCost p q h ell :=
        (chamberTwiceCost_eq_residualQuadratic p q h ell hell).symm
      _ = minimumUpTo (chamberTwiceCost p q h) (chamberK h q) := hval
      _ = minimumUpTo (residualQuadratic p q h) (min h q) := hmin
  · rintro ⟨_, ⟩hval
    refine ⟨hell, ?⟩_
    calc
      chamberTwiceCost p q h ell =
          residualQuadratic p q h (chamberK h q - ell) :=
        chamberTwiceCost_eq_residualQuadratic p q h ell hell
      _ = minimumUpTo (residualQuadratic p q h) (min h q) := hval
      _ = minimumUpTo (chamberTwiceCost p q h) (chamberK h q) := hmin.symm

private theorem map_reverseIndex_range (k : Nat) :
    (List.range (k + 1)).map (fun t => k - t) =
      (List.range (k + 1)).reverse := by
  apply List.ext_getElem
  · simp
  · intro i hi1 hi2
    simp only [List.length_map, List.length_range] at hi1
    simp only [List.getElem_map, List.getElem_range, List.getElem_reverse,
      List.length_range]
    omega

/-- Reversal sends the chamber minimizer list to the reverse residual list. -/
theorem map_chamberMinimizerList_reverse (p q h : Nat) :
    (chamberMinimizerList p q h).map
        (fun ell => chamberK h q - ell) =
      (residualMinimizerList p q h).reverse := by
  let k := chamberK h q
  let cp : Nat → Bool := fun ell =>
    decide (chamberTwiceCost p q h ell = chamberTwiceMinimum p q h)
  let rp : Nat → Bool := fun t =>
    decide (residualQuadratic p q h t =
      minimumUpTo (residualQuadratic p q h) k)
  have hfilter :
      (List.range (k + 1)).filter cp =
        (List.range (k + 1)).filter (rp ∘ fun ell => k - ell) := by
    apply List.filter_congr
    intro ell hell
    have hell' : ell ≤ k := by
      rw [List.mem_range] at hell
      omega
    have hprop :
        (chamberTwiceCost p q h ell = chamberTwiceMinimum p q h) ↔
          (residualQuadratic p q h (k - ell) =
            minimumUpTo (residualQuadratic p q h) k) := by
      rw [chamberTwiceCost_eq_residualQuadratic p q h ell hell',
        chamberTwiceMinimum_eq_residualMin_cast, residualMin_cast]
      simp [k, chamberK]
    dsimp only [cp, rp, Function.comp_apply]
    by_cases hc : chamberTwiceCost p q h ell = chamberTwiceMinimum p q h
    · have hr := hprop.mp hc
      simp [hc, hr]
    · have hr : ¬ residualQuadratic p q h (k - ell) =
```

```
          minimumUpTo (residualQuadratic p q h) k := by
        intro hr
        exact hc (hprop.mpr hr)
      simp [hc, hr]
  change
    (List.map (fun ell => k - ell)
      (List.filter cp (List.range (k + 1)))) =
    (List.filter rp (List.range (k + 1))).reverse
  rw [hfilter, ← List.filter_map, map_reverseIndex_range, List.filter_reverse]

/-- The chamber logarithmic multiplicity is exactly the residual multiplicity. -/
theorem chamberMultiplicity_eq_residualMultiplicity (p q h : Nat) :
    chamberMultiplicity p q h = residualMultiplicity p q h := by
  have h := congrArg List.length (map_chamberMinimizerList_reverse p q h)
  simpa [chamberMultiplicity, residualMultiplicity] using h

/-- The chamber multiplicity is one or two, with exactly the residual parity test. -/
theorem chamberMultiplicity_formula (p q h : Nat) :
    chamberMultiplicity p q h =
      if (p + q + h) % 2 = 1 ∧ p ≤ q + h ∧ q ≤ p + h ∧ h ≤ p + q
      then 2 else 1 := by
  rw [chamberMultiplicity_eq_residualMultiplicity, residualMultiplicity_formula]

-- Positive, rectangular, tied, and zero-dimensional regression cases.
example : chamberTwiceMinimum 1 1 1 = 1 ∧ chamberMultiplicity 1 1 1 = 2 := by decide
example : chamberTwiceMinimum 2 3 4 = 6 ∧ chamberMultiplicity 2 3 4 = 2 := by decide
example : chamberTwiceMinimum 3 2 4 = 6 ∧ chamberMultiplicity 3 2 4 = 2 := by decide
example : chamberTwiceMinimum 0 3 4 = 0 ∧ chamberMultiplicity 0 3 4 = 1 := by decide

end Arithmetic
end O77
```

19.1 Spectral increments and the unique resonance

Lemma 19.7 (Checked spectral increments and the unique resonance)**.**

Corresponding Lean declaration(s): `O77.Arithmetic.chamberResonant`,`O77.Arithmetic.chamberResonanceIndex`, `O77.Arithmetic.chamberResonanceMultiplicity`,`O77.Arithmetic.chamberTwiceIncrement`,`O77.Arithmetic.chamberTwicePrefixExponent`.

Corresponding Lean declaration(s): `O77.Arithmetic.exists_chamberResonant_iff`,`O77.Arithmetic.chamberResonant_unique`,`O77.Arithmetic.chamberResonanceIndex_spec`,`O77.Arithmetic.chamberResonant_iff_eq_index`,`O77.Arithmetic.chamberMultiplicity_eq_one_add_resonanceMultiplicity`.

Corresponding Lean declaration(s): `O77.Arithmetic.chamberTwiceIncrement_succ`,`O77.Arithmetic.chamberTwicePrefixExponent_eq_chamberTwiceCost`,`O77.Arithmetic.chamberResonant_iff_increment_eq_zero`,`O77.Arithmetic.chamberTwicePrefixMinimum_eq_residualMin_cast`,`O77.Arithmetic.chamberPrefixMinimizerList_eq_chamberMinimizerList`.

Put $k = \min(h, q)$ and $d = \max(h, q)$. Define the signed twice-increment

$$\Delta_i = d - k - 1 + 2i - p \in \mathbb{Z},$$

and recursively define the twice-prefix exponent by

$$P_0 = pk, \qquad P_{\ell+1} = P_\ell + \Delta_{\ell+1}.$$

Then for every $\ell \in \mathbb{N}$,

$$P_\ell = pk + \ell(d - k - p) + \ell^2 = C_{p,q,h}(\ell),$$

and $\Delta_{i+1} = \Delta_i + 2$. *A factor index* $1 \le i \le k$ *is* resonant *when* $\Delta_i = 0$, *equivalently*

$$d + 2i = p + k + 1.$$

There is at most one resonant index. It exists exactly when

$$(p + q + h) \bmod 2 = 1, \qquad p \le q + h, \qquad q \le p + h, \qquad h \le p + q,$$

and in that case it is

$$i_* = (p + k + 1 - d)/2.$$

Moreover the minimum and complete minimizer list of the recursively defined P_ℓ *agree with those of* $C_{p,q,h}$. *The index-reversal map* $\ell \mapsto k - \ell$ *sends that minimizer list to the reverse of the residual minimizer list from Lemma 19.6, and the two minimum values agree. Finally, the chamber logarithmic multiplicity is one plus the indicator of the resonance.*

Proof. The recurrence expands by induction. If $P_\ell = C(\ell)$, adding $d - k - 1 + 2(\ell + 1) - p$ gives $C(\ell + 1)$ after collecting the integer terms. The formula for consecutive increments is immediate, and resonance is exactly the displayed subtraction-free equation.

Suppose first that $1 \le i \le k$ satisfies the resonance equation. Then $p + k + d = 2d + 2i - 1$ is odd. The inequalities $i \le k$ and $i \ge 1$ give $p \le d + k$ and $d \le p + k$. Since $\{d, k\} = \{h, q\}$, these are exactly the three weak triangle inequalities. Conversely, those inequalities give $d \le p + k$ and $p \le d + k$. Oddness of $p + k + d$ implies that $n := p + k + 1 - d$ is even. The first inequality gives $n \ge 1$, hence in fact $n \ge 2$; the second gives $n \le 2k + 1$, hence, by evenness, $n \le 2k$. Thus $i = n/2$ lies in $\{1, \dots, k\}$ and satisfies $d + 2i = p + k + 1$. Uniqueness follows by cancelling $d, p, k, 1$ from two such equations and then cancelling the factor 2 in $\mathbb{N}$.

The recurrence identity identifies P pointwise with the checked chamber cost. The already checked minimum and minimizer-list transport of Lemma 19.6 therefore applies without a new optimization argument. Its closed multiplicity formula is 2 precisely in the odd balanced case and 1 otherwise; the existence-and-uniqueness result above rewrites this as one plus the resonance indicator.

Provenance: The increment mechanism is the finite arithmetic behind Sneiderman, Propositions 8.14 and 8.18; the subtraction-safe resonance equivalence and executable prefix statements are formalized here.

□

```
namespace O77
namespace Arithmetic

/-- Equality of one Laguerre factor's two competing exponents, written without
natural-number subtraction: `2 η(+i) = p`, where η`2 = d-k-1`. -/
def chamberResonant (p q h i : Nat) : Prop :=
  1 ≤ i ∧ i ≤ chamberK h q ∧
    chamberD h q + 2 * i = p + chamberK h q + 1

instance (p q h i : Nat) : Decidable (chamberResonant p q h i) := by
  unfold chamberResonant
  infer_instance

/-- A resonant one-dimensional factor exists exactly in the odd balanced case. -/
theorem exists_chamberResonant_iff (p q h : Nat) :
    ∃( i, chamberResonant p q h i) ↔
      (p + q + h) % 2 = 1 ∧
        p ≤ q + h ∧ q ≤ p + h ∧ h ≤ p + q := by
  by_cases hhq : h ≤ q
  · simp only [chamberResonant, chamberK, chamberD,
      Nat.min_eq_left hhq, Nat.max_eq_right hhq]
    constructor
```

```
    · rintro ⟨i, hi1, hih, ⟩heq
      omega
    · rintro ⟨hodd, hp, hq, ⟩hh
      refine ⟨(p + h + 1 - q) / 2, ?⟩_
      omega
  · have hqh : q ≤ h := by omega
    simp only [chamberResonant, chamberK, chamberD,
      Nat.min_eq_right hqh, Nat.max_eq_left hqh]
    constructor
    · rintro ⟨i, hi1, hiq, ⟩heq
      omega
    · rintro ⟨hodd, hp, hq, ⟩hh
      refine ⟨(p + q + 1 - h) / 2, ?⟩_
      omega

/-- At most one one-dimensional factor is resonant. -/
theorem chamberResonant_unique (p q h i j : Nat)
    (hi : chamberResonant p q h i) (hj : chamberResonant p q h j) : i = j := by
  unfold chamberResonant at hi hj
  omega

/-- The only possible resonant index. Outside the balanced parity case the
truncated numerator is harmless because the resonance predicate is false. -/
def chamberResonanceIndex (p q h : Nat) : Nat :=
  (p + chamberK h q + 1 - chamberD h q) / 2

/-- The explicit candidate is resonant exactly in the odd balanced case. -/
theorem chamberResonanceIndex_spec (p q h : Nat) :
    chamberResonant p q h (chamberResonanceIndex p q h) ↔
      (p + q + h) % 2 = 1 ∧
        p ≤ q + h ∧ q ≤ p + h ∧ h ≤ p + q := by
  constructor
  · intro hi
    exact (exists_chamberResonant_iff p q h).mp ⟨_, ⟩hi
  · intro hcond
    rcases (exists_chamberResonant_iff p q h).mpr hcond with ⟨i, ⟩hi
    have hieq : i = chamberResonanceIndex p q h := by
      unfold chamberResonant at hi
      unfold chamberResonanceIndex
      omega
    simpa [hieq] using hi

/-- Every resonant factor is the explicit candidate, and the balanced parity
condition is sufficient as well as necessary. -/
theorem chamberResonant_iff_eq_index (p q h i : Nat) :
    chamberResonant p q h i ↔
      i = chamberResonanceIndex p q h ∧
        (p + q + h) % 2 = 1 ∧
        p ≤ q + h ∧ q ≤ p + h ∧ h ≤ p + q := by
  constructor
  · intro hi
    have hcond := (exists_chamberResonant_iff p q h).mp ⟨i, ⟩hi
    have hspec := (chamberResonanceIndex_spec p q h).mpr hcond
    exact ⟨chamberResonant_unique p q h i _ hi hspec, ⟩hcond
  · rintro ⟨rfl, ⟩hcond
    exact (chamberResonanceIndex_spec p q h).mpr hcond

/-- The number of equality factors in the one-dimensional product. Uniqueness
shows that this indicator is their exact cardinality. -/
def chamberResonanceMultiplicity (p q h : Nat) : Nat :=
  if chamberResonant p q h (chamberResonanceIndex p q h) then 1 else 0
```

```
/-- The logarithmic multiplicity is one plus the number of resonant factors. -/
theorem chamberMultiplicity_eq_one_add_resonanceMultiplicity (p q h : Nat) :
    chamberMultiplicity p q h = 1 + chamberResonanceMultiplicity p q h := by
  rw [chamberMultiplicity_formula]
  by_cases hcond :
      (p + q + h) % 2 = 1 ∧ p ≤ q + h ∧ q ≤ p + h ∧ h ≤ p + q
  · have hres : chamberResonant p q h (chamberResonanceIndex p q h) :=
      (chamberResonanceIndex_spec p q h).2 hcond
    simp [chamberResonanceMultiplicity, hcond, hres]
  · have hres : ¬ chamberResonant p q h (chamberResonanceIndex p q h) := by
      intro hr
      exact hcond ((chamberResonanceIndex_spec p q h).1 hr)
    simp [chamberResonanceMultiplicity, hcond, hres]

/-- Twice the change in the ordered-chamber prefix exponent when factor `i`
is moved from the large-`T` regime to the small-`T` regime. -/
def chamberTwiceIncrement (p q h i : Nat) : Int :=
  (chamberD h q : Int) - (chamberK h q : Int) - 1 +
    2 * (i : Int) - (p : Int)

/-- Twice the prefix exponent, formed recursively from the one-dimensional
factor exponents. This is the integral-side formula before closed expansion. -/
def chamberTwicePrefixExponent (p q h : Nat) : Nat → Int
  | 0 => (p : Int) * (chamberK h q : Int)
  | ell + 1 =>
      chamberTwicePrefixExponent p q h ell +
        chamberTwiceIncrement p q h (ell + 1)

/-- Consecutive twice-increments differ by exactly two. -/
theorem chamberTwiceIncrement_succ (p q h i : Nat) :
    chamberTwiceIncrement p q h (i + 1) =
      chamberTwiceIncrement p q h i + 2 := by
  simp [chamberTwiceIncrement]
  grind

/-- The recursive spectral prefix exponent expands to the checked chamber cost. -/
theorem chamberTwicePrefixExponent_eq_chamberTwiceCost
    (p q h ell : Nat) :
    chamberTwicePrefixExponent p q h ell =
      chamberTwiceCost p q h ell := by
  induction ell with
  | zero =>
      simp [chamberTwicePrefixExponent, chamberTwiceCost]
  | succ ell ih =>
      simp only [chamberTwicePrefixExponent, ih]
      simp [chamberTwiceIncrement, chamberTwiceCost]
      grind

/-- A factor is resonant exactly when its twice-increment vanishes, together
with the required finite-index bounds. -/
theorem chamberResonant_iff_increment_eq_zero (p q h i : Nat) :
    chamberResonant p q h i ↔
      1 ≤ i ∧ i ≤ chamberK h q ∧
        chamberTwiceIncrement p q h i = 0 := by
  unfold chamberResonant chamberTwiceIncrement
  constructor <;> rintro ⟨hi1, hik, ⟩heq
  · exact ⟨hi1, hik, by ⟩omega
  · exact ⟨hi1, hik, by ⟩omega

/-- The minimum of the recursively defined spectral prefixes is the residual
minimum before division by two. -/
theorem chamberTwicePrefixMinimum_eq_residualMin_cast (p q h : Nat) :
```

```
    minimumUpTo (chamberTwicePrefixExponent p q h) (chamberK h q) =
      (residualMin p q h : Int) := by
  have hfun : chamberTwicePrefixExponent p q h = chamberTwiceCost p q h := by
    funext ell
    exact chamberTwicePrefixExponent_eq_chamberTwiceCost p q h ell
  rw [hfun]
  exact chamberTwiceMinimum_eq_residualMin_cast p q h

/-- The complete minimizer list of the recursive spectral prefix formula is the
same list as for the closed chamber cost. -/
theorem chamberPrefixMinimizerList_eq_chamberMinimizerList (p q h : Nat) :
    minimizerList (chamberTwicePrefixExponent p q h) (chamberK h q) =
      chamberMinimizerList p q h := by
  have hfun : chamberTwicePrefixExponent p q h = chamberTwiceCost p q h := by
    funext ell
    exact chamberTwicePrefixExponent_eq_chamberTwiceCost p q h ell
  simp [chamberMinimizerList, hfun]

-- Resonant, nonresonant, rectangular, and absent-factor regressions.
example : chamberResonant 1 1 1 1 := by decide
example : chamberResonanceIndex 2 3 4 = 1 := by decide
example : chamberResonanceMultiplicity 2 3 4 = 1 := by decide
example : chamberResonanceMultiplicity 2 2 2 = 0 := by decide
example : chamberResonanceMultiplicity 0 3 4 = 0 := by decide
example : chamberTwicePrefixExponent 2 3 4 0 = 6 := by decide
example : chamberTwicePrefixExponent 2 3 4 1 = 6 := by decide
example : chamberTwicePrefixExponent 2 3 4 2 = 8 := by decide

end Arithmetic
end O77
```

20 Ordered-Laguerre analysis

This section proves the Laplace/sublevel estimates and the ordered eigenvalue integral estimate used in the residual matrix-product analysis. Each statement concerns an explicitly defined integral or sublevel measure. The hypotheses of the transfer lemmas are quantitative bounds on those functions; the later Gram and spectral modules prove the identities needed to apply them.

Lemma 20.1 (Laplace/sublevel split with finite-mass cancellation)**.**

Corresponding Lean declaration(s): `O77.Residual.sublevelMass,O77.Residual.laplaceKernel,O77.Residual.laplaceIntegral,O77.Residual.measurable_laplaceKernel,O77.Residual.measurableSet_sublevel`.

Corresponding Lean declaration(s): `O77.Residual.sublevelMass_le_exp_mul_laplace,O77.Residual.laplaceIntegral_le_sublevel_add_half,O77.Residual.laplace_split_bounds`.

Corresponding Lean declaration(s): `O77.Residual.laplaceIntegral_le_measure_univ,O77.Residual.laplaceIntegral_ne_top,O77.Residual.sublevelMass_lower_of_laplace_gap,O77.Residual.exp_factor_cannot_be_dropped,O77.Residual.tail_term_cannot_be_dropped`.

Let X be a measurable space, let μ be a measure on X, and let $f : X \to \mathbb{R}$ be measurable. For $\epsilon, T \in \mathbb{R}$, define, in extended nonnegative reals,

$$V(\epsilon) = \mu\{f < \epsilon\}, \qquad J(T) = \int^{-} \mathrm{ofReal}(e^{-Tf})\, d\mu.$$

For every $\epsilon > 0$,

$$V(\epsilon) \le eJ(1/\epsilon). \tag{20.1.1}$$

If additionally $f \ge 0$ and $T > 0$, then for every real a,

$$J(T) \le V(a/T) + e^{-a/2}J(T/2). \tag{20.1.2}$$

Neither assertion requires μ to be finite. If μ is finite, $f \ge 0$, and $T \ge 0$, then $J(T) \le \mu(\mathrm{univ}) < \infty$. Consequently, for $T > 0$, $a \in \mathbb{R}$, and $L \in [0, \infty]$, the explicit gap inequality

$$L + e^{-a/2}J(T/2) \le J(T) \tag{20.1.3}$$

implies $L \le V(a/T)$. This last implication is a cancellation statement in $[0, \infty]$ and uses finiteness of the tail; it is not valid merely by formal subtraction.

Both terms guarded by the estimates are essential. On the one-point space with Dirac measure and $f \equiv \frac{1}{2}$, one has $V(1) = 1 > e^{-1/2} = J(1)$, so the factor e in (20.1.1) cannot be dropped. With $f \equiv 1$, one has $J(1) = e^{-1} > 0 = V(0)$, so the tail term in (20.1.2) cannot be dropped.

Proof. On $f < \epsilon$, one has $f/\epsilon < 1$, hence $1 \le e\, e^{-f/\epsilon}$. Integrating the indicator gives (20.1.1) directly in `ENNReal`; no subtraction from an infinite quantity occurs. For (20.1.2), split pointwise according to $f < a/T$. On that set, nonnegativity of f and positivity of T give $e^{-Tf} \le 1$. On the complement, $Tf \ge a$, so

$$-Tf \le -a/2 - (T/2)f$$

and therefore $e^{-Tf} \le e^{-a/2}e^{-(T/2)f}$. Integrate the pointwise sum, identify the indicator integral with $V(a/T)$, and pull out the nonnegative constant. Positivity of a is not used in this split; it enters only when one later chooses a decaying tail cutoff.

For finite μ and $T \ge 0$, the pointwise inequalities $0 \le e^{-Tf} \le 1$ give $J(T) \le \mu(\mathrm{univ}) < \infty$. Under (20.1.3), compose that gap with (20.1.2). The common tail $e^{-a/2}J(T/2)$ is not ∞, so the cancellative-order lemma for `ENNReal` yields $L \le V(a/T)$. This is the only cancellation used.

For the two controls, integration against a Dirac measure is evaluation. The displayed values follow immediately, and $e^{-1/2} < 1$ and $e^{-1} > 0$ prove the strict failures. Thus the regression theorems reject two tempting strengthenings of the universal inequalities; they are not numerical approximations. □

```
noncomputable section
open MeasureTheory Set
open scoped ENNReal
namespace O77.Residual

variable α{ : Type*} [MeasurableSpace α]

def sublevelMass μ( : Measure α) (f : α -> Real) (eps : Real) : ENNReal :=
  μ {x | f x < eps}

def laplaceKernel (f : α -> Real) (T : Real) (x : α) : ENNReal :=
  ENNReal.ofReal (Real.exp (-T * f x))

def laplaceIntegral μ( : Measure α) (f : α -> Real) (T : Real) : ENNReal :=
  ∫⁻ x, laplaceKernel f T x ∂μ

lemma measurable_laplaceKernel {f : α -> Real} (hf : Measurable f) (T : Real) :
    Measurable (laplaceKernel f T) := by
  unfold laplaceKernel
  exact ENNReal.measurable_ofReal.comp
```

```
    (Real.measurable_exp.comp ((hf.const_mul (-T))))

lemma measurableSet_sublevel {f : α -> Real} (hf : Measurable f) (eps : Real) :
    MeasurableSet {x | f x < eps} :=
  measurableSet_lt hf measurable_const


theorem sublevelMass_le_exp_mul_laplace μ( : Measure α) {f : α -> Real}
    (hf : Measurable f) {eps : Real} (heps : 0 < eps) :
    sublevelMass μ f eps ≤
      ENNReal.ofReal (Real.exp 1) * laplaceIntegral μ f (1 / eps) := by
  let S : Set α := {x | f x < eps}
  have hS : MeasurableSet S := measurableSet_sublevel hf eps
  have hpoint : ∀ x : α,
      S.indicator (1 : α → ENNReal) x ≤
        ENNReal.ofReal (Real.exp 1) * laplaceKernel f (1 / eps) x := by
    intro x
    by_cases hx : x ∈ S
    · rw [Set.indicator_of_mem hx]
      change (1 : ENNReal) ≤
        ENNReal.ofReal (Real.exp 1) *
          ENNReal.ofReal (Real.exp (-(1 / eps) * f x))
      have hquot : f x / eps < 1 := (div_lt_one heps).2 hx
      have hargEq :
          (1 : Real) + (-(1 / eps) * f x) = 1 - f x / eps := by
        field_simp [ne_of_gt heps]
        ring
      have harg : (0 : Real) ≤ 1 + (-(1 / eps) * f x) := by
        rw [hargEq]
        exact (sub_pos.mpr hquot).le
      have hreal : (1 : Real) ≤
          Real.exp 1 * Real.exp (-(1 / eps) * f x) := by
        calc
          (1 : Real) = Real.exp 0 := Real.exp_zero.symm
          _ ≤ Real.exp (1 + (-(1 / eps) * f x)) := Real.exp_le_exp.mpr harg
          _ = Real.exp 1 * Real.exp (-(1 / eps) * f x) := Real.exp_add _ _
      rw ←[ ENNReal.ofReal_one,
        ← ENNReal.ofReal_mul (Real.exp_pos 1).le]
      exact ENNReal.ofReal_le_ofReal hreal
    · rw [Set.indicator_of_notMem hx]
      exact bot_le
  calc
    sublevelMass μ f eps = ∫⁻ x, S.indicator (1 : α → ENNReal) x ∂μ := by
      change μ S = _
      exact (lintegral_indicator_one hS).symm
    _ ≤ ∫⁻ x, ENNReal.ofReal (Real.exp 1) * laplaceKernel f (1 / eps) x ∂μ :=
      lintegral_mono hpoint
    _ = ENNReal.ofReal (Real.exp 1) * ∫⁻ x, laplaceKernel f (1 / eps) x ∂μ := by
      apply lintegral_const_mul
      exact measurable_laplaceKernel hf _
    _ = ENNReal.ofReal (Real.exp 1) * laplaceIntegral μ f (1 / eps) := rfl


theorem laplaceIntegral_le_sublevel_add_half μ( : Measure α) {f : α -> Real}
    (hf : Measurable f) (hf0 : ∀ x, 0 ≤ f x)
    {T a : Real} (hT : 0 < T) :
    laplaceIntegral μ f T ≤ sublevelMass μ f (a / T) +
      ENNReal.ofReal (Real.exp (-a / 2)) * laplaceIntegral μ f (T / 2) := by
  let S : Set α := {x | f x < a / T}
  have hS : MeasurableSet S := measurableSet_sublevel hf (a / T)
  let c : ENNReal := ENNReal.ofReal (Real.exp (-a / 2))
  have hpoint : ∀ x : α,
```

```
      laplaceKernel f T x ≤
        S.indicator (1 : α → ENNReal) x + c * laplaceKernel f (T / 2) x := by
    intro x
    by_cases hx : x ∈ S
    · rw [Set.indicator_of_mem hx]
      have hexp_le : Real.exp (-T * f x) ≤ 1 := by
        calc
          Real.exp (-T * f x) ≤ Real.exp 0 := by
            apply Real.exp_le_exp.mpr
            have hprod : 0 ≤ T * f x := mul_nonneg hT.le (hf0 x)
            linarith
          _ = 1 := Real.exp_zero
      have hk_le : laplaceKernel f T x ≤ 1 := by
        unfold laplaceKernel
        rw ←[ ENNReal.ofReal_one]
        exact ENNReal.ofReal_le_ofReal hexp_le
      exact hk_le.trans (le_add_right le_rfl)
    · rw [Set.indicator_of_notMem hx, zero_add]
      have htail : a / T ≤ f x := le_of_not_gt hx
      have hhalf : a / 2 ≤ (T / 2) * f x := by
        apply (₀div_le_iff (by norm_num : (0 : Real) < 2)).2
        have hmul : a ≤ T * f x := by
          apply (₀div_le_iff hT).1 at htail
          simpa [mul_comm] using htail
        nlinarith
      have harg : -T * f x ≤ -a / 2 + (-(T / 2) * f x) := by
        nlinarith
      have hreal : Real.exp (-T * f x) ≤
          Real.exp (-a / 2) * Real.exp (-(T / 2) * f x) := by
        rw ←[ Real.exp_add]
        exact Real.exp_le_exp.mpr harg
      unfold laplaceKernel c
      rw ←[ ENNReal.ofReal_mul (Real.exp_pos (-a / 2)).le]
      exact ENNReal.ofReal_le_ofReal hreal
  calc
    laplaceIntegral μ f T = ∫⁻ x, laplaceKernel f T x ∂μ := rfl
    _ ≤ ∫⁻ x, S.indicator (1 : α → ENNReal) x +
        c * laplaceKernel f (T / 2) x ∂μ := lintegral_mono hpoint
    _ = ∫⁻( x, S.indicator (1 : α → ENNReal) x ∂μ) +
        ∫⁻ x, c * laplaceKernel f (T / 2) x ∂μ := by
      apply lintegral_add_left
      exact measurable_const.indicator hS
    _ = μ S + ∫⁻( x, c * laplaceKernel f (T / 2) x ∂μ) := by
      have hInd : ∫⁻( x, S.indicator (1 : α → ENNReal) x ∂μ) = μ S :=
        lintegral_indicator_one hS
      rw [hInd]
    _ = μ S + c * ∫⁻( x, laplaceKernel f (T / 2) x ∂μ) := by
      congr 1
      exact lintegral_const_mul c (measurable_laplaceKernel hf _)
    _ = sublevelMass μ f (a / T) +
        ENNReal.ofReal (Real.exp (-a / 2)) * laplaceIntegral μ f (T / 2) := rfl


/-- The two elementary inequalities relating a sublevel mass to its Laplace integral.
The second inequality is valid for every real cutoff `a`; positivity of `a` is only
needed in later asymptotic applications. -/
theorem laplace_split_bounds μ( : Measure α) {f : α -> Real}
    (hf : Measurable f) (hf0 : ∀ x, 0 ≤ f x)
    {eps T a : Real} (heps : 0 < eps) (hT : 0 < T) :
    sublevelMass μ f eps ≤
        ENNReal.ofReal (Real.exp 1) * laplaceIntegral μ f (1 / eps) ∧
      laplaceIntegral μ f T ≤ sublevelMass μ f (a / T) +
```

```
        ENNReal.ofReal (Real.exp (-a / 2)) * laplaceIntegral μ f (T / 2) := by
  exact ⟨sublevelMass_le_exp_mul_laplace μ hf heps,
    laplaceIntegral_le_sublevel_add_half μ hf hf0 ⟩hT


/-- A nonnegative Laplace kernel is bounded by the total mass whenever `T ≥ 0`. -/
theorem laplaceIntegral_le_measure_univ μ( : Measure α) {f : α -> Real}
    (hf0 : ∀ x, 0 ≤ f x) {T : Real} (hT : 0 ≤ T) :
    laplaceIntegral μ f T ≤ μ Set.univ := by
  calc
    laplaceIntegral μ f T = ∫⁻ x, laplaceKernel f T x ∂μ := rfl
    _ ≤ ∫⁻ _x : α, (1 : ENNReal) ∂μ := by
      apply lintegral_mono
      intro x
      unfold laplaceKernel
      rw ←[ ENNReal.ofReal_one]
      apply ENNReal.ofReal_le_ofReal
      calc
        Real.exp (-T * f x) ≤ Real.exp 0 := by
          apply Real.exp_le_exp.mpr
          exact mul_nonpos_of_nonpos_of_nonneg (neg_nonpos.mpr hT) (hf0 x)
        _ = 1 := Real.exp_zero
    _ = μ Set.univ := lintegral_one

/-- Under a finite measure, the Laplace integral is finite.  This lemma is used before
any cancellation in `ENNReal`; it prevents ∞`` from being silently cancelled. -/
theorem laplaceIntegral_ne_top μ( : Measure α) [IsFiniteMeasure μ] {f : α -> Real}
    (hf0 : ∀ x, 0 ≤ f x) {T : Real} (hT : 0 ≤ T) :
    laplaceIntegral μ f T ≠ ⊤ := by
  exact ne_of_lt ((laplaceIntegral_le_measure_univ μ hf0 hT).trans_lt
    (measure_lt_top μ Set.univ))

/-- Safe lower-bound extraction from the Laplace split.  The hypothesis is the explicit
Laplace gap; finite total mass supplies the non∞-`` side condition needed to cancel the tail. -/
theorem sublevelMass_lower_of_laplace_gap μ( : Measure α) [IsFiniteMeasure μ]
    {f : α -> Real} (hf : Measurable f) (hf0 : ∀ x, 0 ≤ f x)
    {T a : Real} {L : ENNReal} (hT : 0 < T)
    (hgap : L + ENNReal.ofReal (Real.exp (-a / 2)) *
        laplaceIntegral μ f (T / 2) ≤ laplaceIntegral μ f T) :
    L ≤ sublevelMass μ f (a / T) := by
  have htail : ENNReal.ofReal (Real.exp (-a / 2)) *
      laplaceIntegral μ f (T / 2) ≠ ⊤ :=
    ENNReal.mul_ne_top ENNReal.ofReal_ne_top
      (laplaceIntegral_ne_top μ hf0 (div_nonneg hT.le (by norm_num)))
  apply ENNReal.le_of_add_le_add_right htail
  exact hgap.trans (laplaceIntegral_le_sublevel_add_half μ hf hf0 hT)


-- Regression controls: both constants in the split inequalities are essential.
theorem exp_factor_cannot_be_dropped : ¬ sublevelMass (Measure.dirac ()) (fun _ : Unit => (1 / 2 :
     Real)) 1 ≤
    laplaceIntegral (Measure.dirac ()) (fun _ : Unit => (1 / 2 : Real)) 1 := by
  have hV : sublevelMass (Measure.dirac ()) (fun _ : Unit => (1 / 2 : Real)) 1 = 1 := by
    simp [sublevelMass]
    norm_num
  have hJ : laplaceIntegral (Measure.dirac ()) (fun _ : Unit => (1 / 2 : Real)) 1 =
      ENNReal.ofReal (Real.exp (-1 / 2)) := by
    simp [laplaceIntegral, laplaceKernel]
    norm_num
  rw [hV, hJ]
  exact not_le_of_gt (ENNReal.ofReal_lt_one.mpr
    (Real.exp_lt_one_iff.mpr (by norm_num)))
```

```
theorem tail_term_cannot_be_dropped : ¬ laplaceIntegral (Measure.dirac ()) (fun _ : Unit => (1 : Real
    )) 1 ≤
    sublevelMass (Measure.dirac ()) (fun _ : Unit => (1 : Real)) 0 := by
  have hJ : laplaceIntegral (Measure.dirac ()) (fun _ : Unit => (1 : Real)) 1 =
      ENNReal.ofReal (Real.exp (-1)) := by
    simp [laplaceIntegral, laplaceKernel]
  have hV : sublevelMass (Measure.dirac ()) (fun _ : Unit => (1 : Real)) 0 = 0 := by
    simp [sublevelMass]
  rw [hJ, hV]
  exact not_le_of_gt (ENNReal.ofReal_pos.mpr (Real.exp_pos (-1)))


-- Positive instantiations of the finite-measure interface.
example : laplaceIntegral (Measure.dirac ()) (fun _ : Unit => (1 : Real)) 1 ≠ ⊤ := by
  exact laplaceIntegral_ne_top (Measure.dirac ()) (fun _ => by norm_num) (by norm_num)

example : (0 : ENNReal) ≤
    sublevelMass (Measure.dirac ()) (fun _ : Unit => (0 : Real)) 0 := by
  have hgap : (0 : ENNReal) + ENNReal.ofReal (Real.exp (-(0 : Real) / 2)) *
      laplaceIntegral (Measure.dirac ()) (fun _ : Unit => (0 : Real)) ((1 : Real) / 2) ≤
      laplaceIntegral (Measure.dirac ()) (fun _ : Unit => (0 : Real)) 1 := by
    simp [laplaceIntegral, laplaceKernel]
  have h := sublevelMass_lower_of_laplace_gap (Measure.dirac ())
    (f := fun _ : Unit => (0 : Real)) (T := 1) (a := 0) (L := 0)
    (by fun_prop) (fun _ => by norm_num) (by norm_num) hgap
  simpa only [zero_div] using h

end O77.Residual
end
```

```
noncomputable section
open MeasureTheory Set
open scoped ENNReal
namespace O77.Residual

variable α{ : Type*} [MeasurableSpace α]

def powerLogTopReal (lam : Real) (k : Nat) (T : Real) : Real :=
  T ^ (-lam) * (Real.log T) ^ k

def powerLogZeroReal (lam : Real) (k : Nat) (eps : Real) : Real :=
  eps ^ lam * (Real.log (1 / eps)) ^ k

def powerLogTop (lam : Real) (k : Nat) (T : Real) : ENNReal :=
  ENNReal.ofReal (powerLogTopReal lam k T)

def powerLogZero (lam : Real) (k : Nat) (eps : Real) : ENNReal :=
  ENNReal.ofReal (powerLogZeroReal lam k eps)

lemma powerLogTopReal_nonneg {lam : Real} {k : Nat} {T : Real} (hT : 1 ≤ T) :
    0 ≤ powerLogTopReal lam k T := by
  unfold powerLogTopReal
  exact mul_nonneg (Real.rpow_nonneg (by linarith) _) (pow_nonneg (Real.log_nonneg hT) _)

lemma powerLogZeroReal_nonneg {lam : Real} {k : Nat} {eps : Real}
    (heps : 0 < eps) (heps1 : eps ≤ 1) :
    0 ≤ powerLogZeroReal lam k eps := by
  unfold powerLogZeroReal
  have hinv : 1 ≤ 1 / eps := by
```

```
    apply (₀le_div_iff heps).2
    simpa using heps1
  exact mul_nonneg (Real.rpow_nonneg heps.le _) (pow_nonneg (Real.log_nonneg hinv) _)

lemma rpow_div_neg_eq {lam T : Real} (hT : 0 ≤ T) :
    (T / 2) ^ (-lam) = (2 : Real) ^ lam * T ^ (-lam) := by
  rw [Real.div_rpow hT (by norm_num : (0 : Real) ≤ 2) (-lam)]
  rw [Real.rpow_neg (by norm_num : (0 : Real) ≤ 2) lam]
  simp only [div_eq_mul_inv, inv_inv]
  ring

lemma powerLogTopReal_half_le {lam : Real} (k : Nat)
    {T : Real} (hT : 2 ≤ T) :
    powerLogTopReal lam k (T / 2) ≤
      (2 : Real) ^ lam * powerLogTopReal lam k T := by
  have hT0 : 0 ≤ T := by linarith
  have hhalf1 : 1 ≤ T / 2 := by linarith
  have hhalf0 : 0 ≤ T / 2 := by linarith
  have hhalfT : T / 2 ≤ T := by linarith
  have hlog : Real.log (T / 2) ≤ Real.log T :=
    Real.log_le_log (by linarith) hhalfT
  have hpow : (Real.log (T / 2)) ^ k ≤ (Real.log T) ^ k :=
    ₀pow_le_pow_left (Real.log_nonneg hhalf1) hlog k
  unfold powerLogTopReal
  calc
    (T / 2) ^ (-lam) * Real.log (T / 2) ^ k ≤
        (T / 2) ^ (-lam) * Real.log T ^ k :=
      mul_le_mul_of_nonneg_left hpow (Real.rpow_nonneg hhalf0 _)
    _ = (2 : Real) ^ lam * (T ^ (-lam) * Real.log T ^ k) := by
      rw [rpow_div_neg_eq hT0]
      ring

lemma rpow_inv_neg_eq {lam eps : Real} (heps : 0 < eps) :
    (1 / eps) ^ (-lam) = eps ^ lam := by
  rw [one_div, Real.inv_rpow heps.le (-lam)]
  rw [Real.rpow_neg heps.le lam]
  simp

lemma powerLogTopReal_inv {lam : Real} (k : Nat) {eps : Real} (heps : 0 < eps) :
    powerLogTopReal lam k (1 / eps) = powerLogZeroReal lam k eps := by
  unfold powerLogTopReal powerLogZeroReal
  rw [rpow_inv_neg_eq heps]

lemma rpow_scaled_inv_neg_eq {lam a eps : Real} (ha : 0 ≤ a) (heps : 0 < eps) :
    (a / eps) ^ (-lam) = a ^ (-lam) * eps ^ lam := by
  rw [Real.div_rpow ha heps.le (-lam)]
  rw [Real.rpow_neg heps.le lam]
  simp only [div_eq_mul_inv, inv_inv]

lemma powerLogTopReal_scaled_lower {lam : Real} (k : Nat) {a eps : Real}
    (ha : 1 ≤ a) (heps : 0 < eps) (heps1 : eps ≤ 1) :
    a ^ (-lam) * powerLogZeroReal lam k eps ≤
      powerLogTopReal lam k (a / eps) := by
  have ha0 : 0 ≤ a := by linarith
  have hbasePos : 0 < 1 / eps := one_div_pos.mpr heps
  have hbaseLe : 1 / eps ≤ a / eps := by
    exact (div_le_div_iff_of_pos_right heps).2 ha
  have hinv1 : 1 ≤ 1 / eps := by
    apply (₀le_div_iff heps).2
    simpa using heps1
  have hlog0 : 0 ≤ Real.log (1 / eps) := Real.log_nonneg hinv1
  have hlogle : Real.log (1 / eps) ≤ Real.log (a / eps) :=
```

```
    Real.log_le_log hbasePos hbaseLe
  have hpowlog : Real.log (1 / eps) ^ k ≤ Real.log (a / eps) ^ k :=
    ₀pow_le_pow_left hlog0 hlogle k
  unfold powerLogTopReal powerLogZeroReal
  rw [rpow_scaled_inv_neg_eq ha0 heps]
  have harpow : 0 ≤ a ^ (-lam) := Real.rpow_nonneg ha0 _
  have hepsrpow : 0 ≤ eps ^ lam := Real.rpow_nonneg heps.le _
  calc
    a ^ (-lam) * (eps ^ lam * Real.log (1 / eps) ^ k) =
        (a ^ (-lam) * eps ^ lam) * Real.log (1 / eps) ^ k := by ring
    _ ≤ (a ^ (-lam) * eps ^ lam) * Real.log (a / eps) ^ k :=
      mul_le_mul_of_nonneg_left hpowlog (mul_nonneg harpow hepsrpow)

lemma exists_exp_tail_cutoff (c C R : Real) (hc : 0 < c) (hC : 0 ≤ C) (hR : 0 ≤ R) :
    ∃ a : Real, 1 ≤ a ∧ Real.exp (-a / 2) * C * R ≤ c / 2 := by
  let B : Real := C * R
  have hB : 0 ≤ B := mul_nonneg hC hR
  let x : Real := 1 + 2 * B / c
  have hx1 : 1 ≤ x := by
    dsimp [x]
    have : 0 ≤ 2 * B / c := div_nonneg (mul_nonneg (by norm_num) hB) hc.le
    linarith
  have hx : 0 < x := lt_of_lt_of_le (by norm_num) hx1
  let a : Real := 1 + 2 * Real.log x
  have hloga : 0 ≤ Real.log x := Real.log_nonneg hx1
  have ha : 1 ≤ a := by
    dsimp [a]
    linarith
  have hexpneg : Real.exp (-a / 2) ≤ 1 / x := by
    have harg : -a / 2 = (-1 / 2 : Real) - Real.log x := by
      dsimp [a]
      ring
    rw [harg, Real.exp_sub, Real.exp_log hx]
    have hsmall : Real.exp (-1 / 2 : Real) ≤ 1 := by
      rw ←[ Real.exp_zero]
      exact Real.exp_le_exp.mpr (by norm_num)
    have hx0 : 0 ≤ x := hx.le
    exact div_le_div_of_nonneg_right hsmall hx0
  have hBx : (1 / x) * B ≤ c / 2 := by
    rw [one_div, inv_mul_eq_div]
    apply (₀div_le_iff hx).2
    dsimp [x]
    field_simp [ne_of_gt hc]
    nlinarith
  refine ⟨a, ha, ?_⟩
  simpa [B, mul_assoc] using (mul_le_mul_of_nonneg_right hexpneg hB).trans hBx


/-- A fixed tail cutoff for any nonnegative gauge with an eventual half-scale bound.
The cutoff is independent of `T`; all cancellation remains in `ENNReal` and is
performed only through `sublevelMass_lower_of_laplace_gap`. -/
theorem laplace_tail_gap μ( : Measure α) [IsFiniteMeasure μ]
    {f : α -> Real} (hf : Measurable f) (hf0 : ∀ x, 0 ≤ f x)
    (H : Real -> ENNReal) {c C R T0 T1 : Real}
    (hc : 0 < c) (hC : 0 ≤ C) (hR : 0 ≤ R)
    (hlower : ∀ T, T0 ≤ T ->
      ENNReal.ofReal c * H T ≤ laplaceIntegral μ f T)
    (hupper : ∀ T, T0 ≤ T ->
      laplaceIntegral μ f T ≤ ENNReal.ofReal C * H T)
    (hhalf : ∀ T, T1 ≤ T ->
      H (T / 2) ≤ ENNReal.ofReal R * H T) :
    ∃ a : Real, 1 ≤ a ∧ ∀ T : Real,
```

```
      max (2 * T0) (max T1 1) ≤ T ->
        ENNReal.ofReal (c / 2) * H T ≤ sublevelMass μ f (a / T) := by
  obtain ⟨a, ha, ⟩hcut := exists_exp_tail_cutoff c C R hc hC hR
  refine ⟨a, ha, ?⟩_
  intro T hTbound
  have h2T0 : 2 * T0 ≤ T := (le_max_left _ _).trans hTbound
  have hright : max T1 1 ≤ T := (le_max_right _ _).trans hTbound
  have hT1 : T1 ≤ T := (le_max_left _ _).trans hright
  have hTone : 1 ≤ T := (le_max_right _ _).trans hright
  have hTpos : 0 < T := lt_of_lt_of_le (by norm_num) hTone
  have hT0 : T0 ≤ T := by linarith
  have hT0half : T0 ≤ T / 2 := by linarith
  have hcoeff :
      ENNReal.ofReal (Real.exp (-a / 2)) * ENNReal.ofReal C * ENNReal.ofReal R ≤
        ENNReal.ofReal (c / 2) := by
    rw ←[ ENNReal.ofReal_mul (Real.exp_pos (-a / 2)).le]
    rw ←[ ENNReal.ofReal_mul (mul_nonneg (Real.exp_pos (-a / 2)).le hC)]
    exact ENNReal.ofReal_le_ofReal hcut
  have htail :
      ENNReal.ofReal (Real.exp (-a / 2)) * laplaceIntegral μ f (T / 2) ≤
        ENNReal.ofReal (c / 2) * H T := by
    calc
      ENNReal.ofReal (Real.exp (-a / 2)) * laplaceIntegral μ f (T / 2) ≤
          ENNReal.ofReal (Real.exp (-a / 2)) *
            (ENNReal.ofReal C * H (T / 2)) :=
        mul_le_mul_of_nonneg_left (hupper (T / 2) hT0half) bot_le
      _ ≤ ENNReal.ofReal (Real.exp (-a / 2)) *
            (ENNReal.ofReal C * (ENNReal.ofReal R * H T)) := by
        exact mul_le_mul_of_nonneg_left
          (mul_le_mul_of_nonneg_left (hhalf T hT1) bot_le) bot_le
      _ = (ENNReal.ofReal (Real.exp (-a / 2)) * ENNReal.ofReal C *
            ENNReal.ofReal R) * H T := by ac_rfl
      _ ≤ ENNReal.ofReal (c / 2) * H T := mul_le_mul_of_nonneg_right hcoeff bot_le
  let L : ENNReal := ENNReal.ofReal (c / 2) * H T
  have hc2 : 0 ≤ c / 2 := by linarith
  have hconst : ENNReal.ofReal (c / 2) + ENNReal.ofReal (c / 2) =
      ENNReal.ofReal c := by
    rw ←[ ENNReal.ofReal_add hc2 hc2]
    congr 1
    ring
  have hdouble : L + L = ENNReal.ofReal c * H T := by
    dsimp [L]
    rw ←[ add_mul, hconst]
  have hgap : L + ENNReal.ofReal (Real.exp (-a / 2)) *
        laplaceIntegral μ f (T / 2) ≤ laplaceIntegral μ f T := by
    calc
      L + ENNReal.ofReal (Real.exp (-a / 2)) * laplaceIntegral μ f (T / 2) ≤
          L + L := by simpa [add_comm] using add_le_add_left htail L
      _ = ENNReal.ofReal c * H T := hdouble
      _ ≤ laplaceIntegral μ f T := hlower T hT0
  exact sublevelMass_lower_of_laplace_gap μ hf hf0 hTpos hgap

end O77.Residual
end

noncomputable section
open MeasureTheory Set
open scoped ENNReal
namespace O77.Residual

variable β{ : Type*} [MeasurableSpace β]
```

```
lemma powerLogTop_half_le {lam : Real} (k : Nat) {T : Real} (hT : 2 ≤ T) :
    powerLogTop lam k (T / 2) ≤
      ENNReal.ofReal ((2 : Real) ^ lam) * powerLogTop lam k T := by
  unfold powerLogTop
  rw ←[ ENNReal.ofReal_mul (Real.rpow_nonneg (by norm_num : (0 : Real) ≤ 2) lam)]
  exact ENNReal.ofReal_le_ofReal (powerLogTopReal_half_le k hT)

lemma powerLogTop_inv {lam : Real} (k : Nat) {eps : Real} (heps : 0 < eps) :
    powerLogTop lam k (1 / eps) = powerLogZero lam k eps := by
  unfold powerLogTop powerLogZero
  rw [powerLogTopReal_inv k heps]

lemma powerLogTop_scaled_lower {lam : Real} (k : Nat) {a eps : Real}
    (ha : 1 ≤ a) (heps : 0 < eps) (heps1 : eps ≤ 1) :
    ENNReal.ofReal (a ^ (-lam)) * powerLogZero lam k eps ≤
      powerLogTop lam k (a / eps) := by
  unfold powerLogTop powerLogZero
  rw ←[ ENNReal.ofReal_mul (Real.rpow_nonneg (by linarith : 0 ≤ a) (-lam))]
  exact ENNReal.ofReal_le_ofReal (powerLogTopReal_scaled_lower k ha heps heps1)

/-- The zero-parameter gauge is exactly the gauge already used by the checked
saddle-band predicate, with logarithmic degree `k` corresponding to multiplicity
`k + 1`. -/
lemma powerLogZero_eq_bandGauge (lam : Real) (k : Nat) (eps : Real) :
    powerLogZero lam k eps =
      ENNReal.ofReal (O77.SaddleDirect.bandGauge lam (k + 1) eps) := by
  simp [powerLogZero, powerLogZeroReal, O77.SaddleDirect.bandGauge]

/-- Explicit two-sided power-log bounds for a Laplace transform at infinity. -/
def LaplacePowerLogBounds (J : Real -> ENNReal) (lam : Real) (k : Nat) : Prop :=
  ∃ c C T0 : Real,
    0 < c ∧ c ≤ C ∧ 2 ≤ T0 ∧
      ∀ T : Real, T0 ≤ T ->
        ENNReal.ofReal c * powerLogTop lam k T ≤ J T ∧
          J T ≤ ENNReal.ofReal C * powerLogTop lam k T

/-- Explicit two-sided power-log bounds for a sublevel mass at zero. -/
def SublevelPowerLogBounds (V : Real -> ENNReal) (lam : Real) (k : Nat) : Prop :=
  ∃ c C eps0 : Real,
    0 < c ∧ 0 < C ∧ 0 < eps0 ∧ eps0 < 1 ∧
      ∀ eps : Real, 0 < eps -> eps < eps0 ->
        ENNReal.ofReal c * powerLogZero lam k eps ≤ V eps ∧
          V eps ≤ ENNReal.ofReal C * powerLogZero lam k eps

/-- The elementary converse Laplace bridge for power-log orders.  It uses one fixed
cutoff selected from the two Laplace constants and the half-scale factor; no exact
Tauberian asymptotics and no subtraction in `ENNReal` are used. -/
theorem sublevelPowerLogBounds_of_laplacePowerLogBounds μ( : Measure β) [IsFiniteMeasure μ]
    {f : β -> Real} (hf : Measurable f) (hf0 : ∀ x, 0 ≤ f x)
    {lam : Real} (hlam : 0 < lam) (k : Nat)
    (hJ : LaplacePowerLogBounds (laplaceIntegral μ f) lam k) :
    SublevelPowerLogBounds (sublevelMass μ f) lam k := by
  rcases hJ with ⟨c, C, T0, hc, hcC, hT0, ⟩hbounds
  have hC : 0 < C := hc.trans_le hcC
  have hRone : 1 ≤ (2 : Real) ^ lam :=
    Real.one_le_rpow (by norm_num) hlam.le
  have hR : 0 ≤ (2 : Real) ^ lam := (by norm_num : (0 : Real) ≤ 1).trans hRone
  obtain ⟨a, ha, ⟩hlower := laplace_tail_gap μ hf hf0
    (powerLogTop lam k) (c := c) (C := C) (R := (2 : Real) ^ lam)
    (T0 := T0) (T1 := 2) hc hC.le hR
    (fun T hT => (hbounds T hT).1)
    (fun T hT => (hbounds T hT).2)
```

```
    (fun T hT => powerLogTop_half_le k (by linarith))
  let Tstar : Real := max (2 * T0) (max 2 1)
  have hT0pos : 0 < T0 := lt_of_lt_of_le (by norm_num) hT0
  have hTstar_pos : 0 < Tstar := by
    dsimp [Tstar]
    have : (1 : Real) ≤ max (2 * T0) (max 2 1) :=
      (le_max_right (2 : Real) 1).trans (le_max_right (2 * T0) (max 2 1))
    linarith
  have ha_pos : 0 < a := lt_of_lt_of_le (by norm_num) ha
  let eps0 : Real := min (1 / T0) (a / Tstar) / 2
  have hfirst_pos : 0 < 1 / T0 := one_div_pos.mpr hT0pos
  have hsecond_pos : 0 < a / Tstar := div_pos ha_pos hTstar_pos
  have hmin_pos : 0 < min (1 / T0) (a / Tstar) :=
    lt_min hfirst_pos hsecond_pos
  have heps0_pos : 0 < eps0 := by
    dsimp [eps0]
    positivity
  have hT0_two : (1 / T0 : Real) ≤ 1 / 2 := by
    exact one_div_le_one_div_of_le (by norm_num) hT0
  have heps0_lt_one : eps0 < 1 := by
    have hmin_le : min (1 / T0) (a / Tstar) ≤ 1 / T0 := min_le_left _ _
    dsimp [eps0]
    nlinarith
  let cV : Real := (c / 2) * a ^ (-lam)
  let CV : Real := Real.exp 1 * C
  have hcV : 0 < cV := by
    dsimp [cV]
    exact mul_pos (half_pos hc) (Real.rpow_pos_of_pos ha_pos (-lam))
  have hCV : 0 < CV := by
    dsimp [CV]
    exact mul_pos (Real.exp_pos 1) hC
  refine ⟨cV, CV, eps0, hcV, hCV, heps0_pos, heps0_lt_one, ?_⟩
  intro eps heps heps0
  have heps_min : eps < min (1 / T0) (a / Tstar) := by
    dsimp [eps0] at heps0
    linarith
  have heps_first : eps < 1 / T0 := heps_min.trans_le (min_le_left _ _)
  have heps_second : eps < a / Tstar := heps_min.trans_le (min_le_right _ _)
  have hTupper : T0 ≤ 1 / eps := by
    apply (₀le_div_iff heps).2
    have hmul : eps * T0 < 1 := (₀lt_div_iff hT0pos).1 heps_first
    nlinarith
  have hTlower : Tstar ≤ a / eps := by
    apply (₀le_div_iff heps).2
    have hmul : eps * Tstar < a := (₀lt_div_iff hTstar_pos).1 heps_second
    nlinarith
  have heps_le_one : eps ≤ 1 := by
    exact (le_of_lt (heps0.trans heps0_lt_one))
  have hquot : a / (a / eps) = eps := by
    field_simp [ne_of_gt ha_pos, ne_of_gt heps]
  have hscale := powerLogTop_scaled_lower (lam := lam) k ha heps heps_le_one
  have hc2 : 0 ≤ c / 2 := (half_pos hc).le
  have haR : 0 ≤ a ^ (-lam) := Real.rpow_nonneg ha_pos.le _
  have hcV_ofReal : ENNReal.ofReal cV =
      ENNReal.ofReal (c / 2) * ENNReal.ofReal (a ^ (-lam)) := by
    dsimp [cV]
    exact ENNReal.ofReal_mul hc2
  have hlower_eps :
      ENNReal.ofReal cV * powerLogZero lam k eps ≤ sublevelMass μ f eps := by
    calc
      ENNReal.ofReal cV * powerLogZero lam k eps =
          ENNReal.ofReal (c / 2) *
```

```
            (ENNReal.ofReal (a ^ (-lam)) * powerLogZero lam k eps) := by
        rw [hcV_ofReal]
        ac_rfl
      _ ≤ ENNReal.ofReal (c / 2) * powerLogTop lam k (a / eps) :=
        mul_le_mul_of_nonneg_left hscale bot_le
      _ ≤ sublevelMass μ f (a / (a / eps)) := hlower (a / eps) hTlower
      _ = sublevelMass μ f eps := by rw [hquot]
  have hupper_raw := sublevelMass_le_exp_mul_laplace μ hf heps
  have hJupper := (hbounds (1 / eps) hTupper).2
  have hupper_eps : sublevelMass μ f eps ≤
      ENNReal.ofReal CV * powerLogZero lam k eps := by
    calc
      sublevelMass μ f eps ≤
          ENNReal.ofReal (Real.exp 1) * laplaceIntegral μ f (1 / eps) := hupper_raw
      _ ≤ ENNReal.ofReal (Real.exp 1) *
          (ENNReal.ofReal C * powerLogTop lam k (1 / eps)) :=
        mul_le_mul_of_nonneg_left hJupper bot_le
      _ = ENNReal.ofReal CV * powerLogZero lam k eps := by
        rw [powerLogTop_inv k heps]
        dsimp [CV]
        rw [ENNReal.ofReal_mul (Real.exp_pos 1).le]
        ac_rfl
  exact ⟨hlower_eps, ⟩hupper_eps

/-- The radius-fixed small-parameter predicate with the exact O77 threshold
convention.  The constants are selected before the small-parameter threshold. -/
def EThetaZeroPowerLog (V : Real -> ENNReal) (lam : Real) (k : Nat) : Prop :=
  ∃ c C eps0 : Real,
    0 < c ∧ c ≤ C ∧ 0 < eps0 ∧ eps0 < Real.exp (-1) ∧
      ∀ eps : Real, 0 < eps -> eps < eps0 ->
        ENNReal.ofReal c * powerLogZero lam k eps ≤ V eps ∧
          V eps ≤ ENNReal.ofReal C * powerLogZero lam k eps

/-- Shrinking the lower constant and the threshold converts the raw two-sided
bounds to the exact O77 convention `0 < c ≤ C` and `eps0 < exp (-1)`. -/
theorem eThetaZeroPowerLog_of_sublevelPowerLogBounds
    {V : Real -> ENNReal} {lam : Real} {k : Nat}
    (hV : SublevelPowerLogBounds V lam k) :
    EThetaZeroPowerLog V lam k := by
  rcases hV with ⟨c, C, eps0, hc, hC, heps0, heps0_one, ⟩hbounds
  let c' : Real := min c C
  let eps0' : Real := min eps0 (Real.exp (-1) / 2)
  have hc' : 0 < c' := by
    dsimp [c']
    exact lt_min hc hC
  have hc'C : c' ≤ C := by
    dsimp [c']
    exact min_le_right _ _
  have hexp : 0 < Real.exp (-1) := Real.exp_pos _
  have heps0' : 0 < eps0' := by
    dsimp [eps0']
    exact lt_min heps0 (half_pos hexp)
  have heps0'_exp : eps0' < Real.exp (-1) := by
    have hle : eps0' ≤ Real.exp (-1) / 2 := by
      dsimp [eps0']
      exact min_le_right _ _
    nlinarith
  refine ⟨c', C, eps0', hc', hc'C, heps0', heps0'_exp, ?⟩_
  intro eps heps heps'
  have heps_eps0 : eps < eps0 := heps'.trans_le (by
    dsimp [eps0']
    exact min_le_left _ _)
```

```
  obtain ⟨hlower, ⟩hupper := hbounds eps heps heps_eps0
  constructor
  · exact (mul_le_mul_of_nonneg_right
      (ENNReal.ofReal_le_ofReal (by
        dsimp [c']
        exact min_le_left _ _)) bot_le).trans hlower
  · exact hupper

/-- The complete elementary converse Laplace bridge in the exact radius-fixed
O77 small-parameter format. -/
theorem volumeOrder_of_laplaceOrder μ( : Measure β) [IsFiniteMeasure μ]
    {f : β -> Real} (hf : Measurable f) (hf0 : ∀ x, 0 ≤ f x)
    {lam : Real} (hlam : 0 < lam) (k : Nat)
    (hJ : LaplacePowerLogBounds (laplaceIntegral μ f) lam k) :
    EThetaZeroPowerLog (sublevelMass μ f) lam k :=
  eThetaZeroPowerLog_of_sublevelPowerLogBounds
    (sublevelPowerLogBounds_of_laplacePowerLogBounds μ hf hf0 hlam k hJ)

-- Closed positive controls for the gauge identities and both fixed factors.
example : powerLogTopReal 1 0 8 = (1 / 8 : Real) := by
  norm_num [powerLogTopReal]

example : powerLogZeroReal 1 0 (1 / 2 : Real) = (1 / 2 : Real) := by
  norm_num [powerLogZeroReal]

example : powerLogTop 1 0 ((8 : Real) / 2) ≤
    ENNReal.ofReal ((2 : Real) ^ (1 : Real)) * powerLogTop 1 0 8 := by
  exact powerLogTop_half_le 0 (by norm_num)

example : ENNReal.ofReal ((2 : Real) ^ (-(1 : Real))) *
    powerLogZero 1 0 (1 / 2 : Real) ≤
      powerLogTop 1 0 ((2 : Real) / (1 / 2 : Real)) := by
  exact powerLogTop_scaled_lower 0 (by norm_num) (by norm_num) (by norm_num)

-- A fixed cutoff chosen independently of the comparison constants can fail.
theorem unit_tail_cutoff_can_fail :
    ¬ Real.exp (-(1 : Real) / 2) * 4 * 1 ≤ (1 : Real) / 2 := by
  have h := Real.add_one_le_exp (-(1 : Real) / 2)
  norm_num at h ⊢
  nlinarith

-- The half-scale factor cannot be replaced by one even in the pure-power case.
theorem half_scale_factor_cannot_be_one :
    ¬ powerLogTop 1 0 ((2 : Real) / 2) ≤ powerLogTop 1 0 2 := by
  norm_num [powerLogTop, powerLogTopReal]

-- The fixed rescaling factor `a ^ (-lam)` in the lower small-parameter
-- constant is genuine: without it the scaled gauge inequality already fails
-- for `lam = 1`, `k = 0`, `a = 2`, and `eps = 1/4`.
theorem scale_factor_cannot_be_one :
    ¬ powerLogZero 1 0 (1 / 4 : Real) ≤
      powerLogTop 1 0 ((2 : Real) / (1 / 4 : Real)) := by
  norm_num [powerLogZero, powerLogZeroReal, powerLogTop, powerLogTopReal]

/-- Without any hypothesis making the common Laplace tail finite, the formal
`ENNReal` gap can hold as `∞ ≤ ∞` while the claimed lower sublevel bound is false.
Thus finite total mass is a sufficient cancellation hypothesis, not cosmetic. -/
theorem tail_finiteness_cannot_be_dropped :
    let μ : Measure Real := volume
    let f : Real -> Real := fun _ => 0
    ((1 : ENNReal) + ENNReal.ofReal (Real.exp (-(0 : Real) / 2)) *
        laplaceIntegral μ f ((1 : Real) / 2) ≤ laplaceIntegral μ f 1) ∧
```

```
      ¬ ((1 : ENNReal) ≤ sublevelMass μ f 0) := by
  simp [laplaceIntegral, laplaceKernel, sublevelMass]

-- Packaging controls independent of any analytic Laplace computation.
example : SublevelPowerLogBounds (powerLogZero 1 0) 1 0 := by
  refine ⟨1, 1, (1 / 2 : Real), by norm_num, by norm_num, by norm_num, by norm_num, ?_⟩
  intro eps heps heps0
  constructor <;> simp

example : EThetaZeroPowerLog (powerLogZero 1 0) 1 0 :=
  eThetaZeroPowerLog_of_sublevelPowerLogBounds (by
    refine ⟨1, 1, (1 / 2 : Real), by norm_num, by norm_num, by norm_num, by norm_num, ?_⟩
    intro eps heps heps0
    constructor <;> simp)

end O77.Residual
end
```

Lemma 20.2 (Choosing one fixed tail cutoff)**.**

Corresponding Lean declaration(s): `O77.Residual.exists_exp_tail_cutoff`,`O77.Residual.laplace_tail_gap`.

Corresponding Lean declaration(s): `O77.Residual.unit_tail_cutoff_can_fail`,`O77.Residual.half_scale_factor_cannot_be_one`,`O77.Residual.scale_factor_cannot_be_one`.

Corresponding Lean declaration(s): `O77.Residual.tail_finiteness_cannot_be_dropped`.

Let μ be finite, let $f \geq 0$ be measurable, and write $J(T) = \int e^{-Tf}\, d\mu$ and $V(\varepsilon) = \mu\{f < \varepsilon\}$. Let $H : \mathbb{R} \to [0, \infty]$, and suppose that for constants $c > 0$, $C, R \geq 0$ and thresholds T_0, T_1 one has

$$\mathrm{ofReal}(c)H(T) \leq J(T) \leq \mathrm{ofReal}(C)H(T) \quad (T \geq T_0), \tag{20.2.1}$$

and

$$H(T/2) \leq \mathrm{ofReal}(R)H(T) \quad (T \geq T_1). \tag{20.2.2}$$

Then there is one $a \geq 1$, independent of T, such that

$$\mathrm{ofReal}(c/2)H(T) \leq V(a/T)$$

for every $T \geq \max(2T_0, \max(T_1, 1))$.

For the power–log gauge, (20.2.2) holds with $T_1 = 2$ and $R = 2^\lambda$. Thus this generic theorem is exactly the fixed-tail step used in Lemma 19.1.

Proof. Put $B = CR \geq 0$, $x = 1 + 2B/c$, and

$$a = 1 + 2\log x.$$

Then $x \geq 1$, so $a \geq 1$. Moreover

$$e^{-a/2} = e^{-1/2}x^{-1} \leq x^{-1} \quad \text{and} \quad \frac{B}{x} = \frac{Bc}{c + 2B} \leq \frac{c}{2}.$$

Consequently

$$e^{-a/2}CR \leq c/2. \tag{20.2.3}$$

This is an explicit choice; no appeal to a nonconstructive asymptotic selection is hidden.

Now fix $T \geq \max(2T_0, \max(T_1, 1))$. Then $T > 0$, both T and $T/2$ are in the range of (20.2.1), and (20.2.2) is available. Combining the upper bound at $T/2$ with (20.2.2)–(20.2.3) gives, in $[0, \infty]$,

$$e^{-a/2} J(T/2) \leq \mathrm{ofReal}(c/2) H(T). \tag{20.2.4}$$

Write $L = \mathrm{ofReal}(c/2) H(T)$. Since $2\,\mathrm{ofReal}(c/2) = \mathrm{ofReal}(c)$, (20.2.1) and (20.2.4) imply

$$L + e^{-a/2} J(T/2) \leq L + L = \mathrm{ofReal}(c) H(T) \leq J(T).$$

This is precisely the explicit gap required by the checked cancellation theorem `sublevelMass_lower_of_laplace_gap`. That theorem first proves the common tail is finite from the finite-measure hypothesis and only then cancels it; no subtraction in $[0, \infty]$ is performed.

The named negative controls show four distinct obstructions: $a = 1$ need not work for arbitrary comparison constants; $H(T/2) \leq H(T)$ is already false for the pure power $H(T) = T^{-1}$; the lower rescaling inequality fails if the factor $a^{-\lambda}$ is deleted; and an infinite common tail can make the formal gap true while the desired lower sublevel bound is false. Thus finite total mass is a sufficient way to justify cancellation, not merely a convenience. □

Lemma 20.3 (Dilation of the bounded-window Laplace integral)**.** *Let $f : \mathbb{R}^D \to [0, \infty)$ be measurable and homogeneous of degree $\nu > 0$ under positive dilations. For $c, R > 0$,*

$$\int_{B(0,cR)} e^{-Tf(w)}\, dw = c^D \int_{B(0,R)} e^{-Tc^\nu f(z)}\, dz.$$

For fixed R, the latter integral is nonincreasing in $T \geq 0$.

Natural-language proof / implementation plan. The linear change of variables $w = cz$ maps the two balls bijectively, has absolute determinant c^D, and changes $f(cz)$ to $c^\nu f(z)$. Nonnegativity of f makes the integrand pointwise nonincreasing in T. The statement also covers $D = 0$ with determinant 1 and a one-point ball, although the nonzero residual application has $D > 0$. □

Lemma 20.4 (A summable shell majorant independent of the Laplace parameter)**.** *For fixed $D \geq 0$, $\delta > 0$, the series $\sum_{j \geq 0} 2^{jD} e^{-\delta^2 4^j}$ converges. If $f \geq 0$ is homogeneous of positive degree, the Gaussian-weighted contribution of the shell $\delta 2^j \leq \|w\| < \delta 2^{j+1}$ is at most*

$$2^{jD} e^{-\delta^2 4^j}\, J_{2\delta}(T).$$

Natural-language proof / implementation plan. The ratio of consecutive coefficients is $2^D e^{-3\delta^2 4^j}$, which tends to zero. Bound the tail by a geometric series after the ratio becomes at most $1/2$; the finitely many earlier terms are harmless. On the shell, the Gaussian is at most $e^{-\delta^2 4^j}$. Substitute $w = 2^j z$; the rescaled shell is contained in $B(0, 2\delta)$, and homogeneity changes T to $T 2^{j\nu} \geq T$. Apply monotonicity from Lemma 20.3. The inner ball and these half-open shells form a measurable partition, so countable additivity/Tonelli sums the bounds. This proves the Gaussian/local comparison without assuming any local exponent beforehand. □

Lemma 20.5 (The determinant in the Gaussian reduction)**.** *For a real $h \times q$ matrix Z and $T \geq 0$, $M = I_h + TZZ^{\mathsf{T}}$ is symmetric positive definite. With the $k = \min(h, q)$ Gram eigenvalues s_i from Proposition 19,*

$$\|Z\|_F^2 = \sum_i s_i, \qquad \det M = \prod_{i=1}^{k} (1 + T s_i) > 0.$$

Natural-language proof / implementation plan. For nonzero v, $v^{\mathsf{T}}Mv = \|v\|^2 + T\|Z^{\mathsf{T}}v\|^2 > 0$. The trace of either Gram matrix is the sum of the squared entries of Z. For a nonzero eigenvalue, the maps $v \mapsto Z^{\mathsf{T}}v$ and $u \mapsto Zu$ give inverse eigenspace isomorphisms up to multiplication by that eigenvalue. The finite-dimensional spectral theorem for real symmetric matrices identifies algebraic multiplicities with eigenspace dimensions. The remaining eigenvalues of the larger Gram matrix are zero, contributing factors 1 to $\det M$. This lemma supplies the exact polynomial determinant link to the eigenvalue-measure input; it is not itself the Wishart change of measure.

□

Lemma 20.6 (One positive quadratic Gaussian and finitely many rows)**.** *For a real positive definite symmetric $h \times h$ matrix M,*

$$\int_{\mathbb{R}^h} e^{-y^{\mathsf{T}}My}\,dy = \pi^{h/2}(\det M)^{-1/2}.$$

Applying this independently to p rows of Y gives Lemma 19.3, including finiteness.

Natural-language proof / implementation plan. Choose an orthogonal diagonalization with positive eigenvalues d_i. Orthogonal change of coordinates preserves Lebesgue measure. Tonelli turns the nonnegative exponential of the sum into the product of one-dimensional Gaussian integrals. Each is $\sqrt{\pi/d_i}$, and their product is the displayed determinant expression. The row space of a matrix is a regrouping of its independent Euclidean entry coordinates; use the exact coordinate regrouping of `coordinateLebesgue_eq_volume` before applying product integration. In the actual residual integral the whole integrand is bounded above by $e^{-\|Y\|_F^2-\|Z\|_F^2}$, giving absolute integrability without relying on a formal totalized integral at a divergent function.

□

Lemma 20.7 (Symmetrization with ties removed explicitly)**.** *For a nonnegative measurable symmetric function F on $(0,\infty)^k$, its integral over that orthant is $k!$ times its integral over $0 < s_1 < \cdots < s_k$. Replacing strict order by weak order changes nothing for Lebesgue integration.*

Natural-language proof / implementation plan. The set of coordinate ties is a finite union of hyperplanes $s_i = s_j$. For $i \neq j$, integration first in s_i gives a singleton section and hence zero measure; localize to bounded boxes and take a countable union if desired to avoid multiplying zero by an infinite volume. Off the tie set, each point has a unique sorting permutation. The corresponding strict chambers are pairwise disjoint and cover the orthant minus a null set. A coordinate permutation preserves the finite product Lebesgue measure, and symmetry leaves the integrand unchanged. Sum the equal nonnegative integrals over the $k!$ permutations. The case $k = 1$ has an empty tie set and is immediate. □

Lemma 20.8 (The Vandermonde majorant and its separated lower bound)**.**

Corresponding Lean declaration(s): `O77.Residual.vandermondeList,O77.Residual.chamberMonomialList,O77.Residual.separatedConstantList,O77.Residual.prod_map_nonneg,O77.Residual.prod_map_le_prod_map`.

Corresponding Lean declaration(s): `O77.Residual.chamberMonomialList_nonneg,O77.Residual.vandermondeList_nonneg,O77.Residual.vandermondeList_le_chamberMonomialList,O77.Residual.prod_half_mul,O77.Residual.separatedConstantList_pos`.

Corresponding Lean declaration(s): `O77.Residual.separatedConstant_mul_chamberMonomialList_le_vandermondeList,O77.Residual.separatedConstantList_eq_choose,O77.Residual.chamberMonomialList_ofFn,O77.Residual.`

vandermondeFin,O77.Residual.chamberMonomialFin.

Corresponding Lean declaration(s): O77.Residual.vandermondeFin_nonneg_le,O77.Residual.half_pow_choose_mul_chamberMonomialFin_le_vandermondeFin,O77.Residual.half_pow_choose_eq_pair_count,O77.Residual.vandermondeList_example,O77.Residual.chamberMonomialList_example.

Corresponding Lean declaration(s): O77.Residual.separated_vandermonde_example,O77.Residual.vandermonde_upper_without_nonnegativity_false,O77.Residual.vandermonde_lower_without_separation_false.

For $0 < s_1 \le \cdots \le s_k$,

$$0 \le \prod_{i<j}(s_j - s_i) \le \prod_{j=1}^{k} s_j^{j-1}.$$

If additionally $2s_i \le s_j$ *for* $i < j$, *then*

$$2^{-\binom{k}{2}} \prod_{j=1}^{k} s_j^{j-1} \le \prod_{i<j}(s_j - s_i).$$

The result includes $k = 0$ *and* $k = 1$, *where both empty products are* 1.

Proof. The implementation defines the Vandermonde recursively on coordinate lists. Removing the first coordinate x contributes the factors $y - x$ over the remaining coordinates. On a nonnegative ordered list, $0 \le y - x \le y$; on a two-separated list, $y/2 \le y - x$. Multiplication preserves both inequalities because every factor involved is nonnegative. Induction gives the list result.

The recursive chamber monomial is then identified with $\prod_{j:\mathrm{Fin}\,k} s_j^j$, and the recursive lower constant with $(1/2)^{\binom{k}{2}}$. Passing through `List.ofFn` yields the finite-function statement displayed above. The code also proves $\binom{k}{2} = k(k-1)/2$ in the natural-number convention used by the blueprint. The negative-coordinate example $(-2, -1)$ shows that nonnegativity is essential for the upper bound, while $(1, 1)$ shows that two-separation is essential for the lower bound.

□

```
noncomputable section
open Real MeasureTheory Set
open scoped ENNReal BigOperators
namespace O77.Residual
/-- The Vandermonde product of a list, with the list order giving the chamber order. -/
def vandermondeList : List Real -> Real
  | [] => 1
  | x :: xs => vandermondeList xs * (xs.map fun y => y - x).prod

/-- The monomial obtained by replacing each factor `s_j - s_i` by `s_j`. -/
def chamberMonomialList : List Real -> Real
  | [] => 1
  | _ :: xs => chamberMonomialList xs * xs.prod

/-- The product of one factor `1/2` for every unordered pair of list positions. -/
def separatedConstantList : List Real -> Real
  | [] => 1
  | _ :: xs => separatedConstantList xs * (1 / 2 : Real) ^ xs.length

lemma prod_map_nonneg α{ : Type*} {l : List α} {f : α -> Real}
    (h : ∀ x ∈ l, 0 ≤ f x) : 0 ≤ (l.map f).prod := by
  apply List.prod_nonneg
```

```
  intro y hy
  rw [List.mem_map] at hy
  rcases hy with ⟨x, hx, ⟩rfl
  exact h x hx

lemma prod_map_le_prod_map α{ : Type*} {l : List α} {f g : α -> Real}
    (hf : ∀ x ∈ l, 0 ≤ f x) (hfg : ∀ x ∈ l, f x ≤ g x) :
    (l.map f).prod ≤ (l.map g).prod := by
  induction l with
  | nil => simp
  | cons x xs ih =>
      have hfx : 0 ≤ f x := hf x (by simp)
      have hxle : f x ≤ g x := hfg x (by simp)
      have htail0 : ∀ y ∈ xs, 0 ≤ f y := by
        intro y hy
        exact hf y (by simp [hy])
      have htail : ∀ y ∈ xs, f y ≤ g y := by
        intro y hy
        exact hfg y (by simp [hy])
      simp only [List.map_cons, List.prod_cons]
      have hgx : 0 ≤ g x := hfx.trans hxle
      exact mul_le_mul hxle (ih htail0 htail) (prod_map_nonneg htail0) hgx

lemma chamberMonomialList_nonneg {l : List Real}
    (h : ∀ x ∈ l, 0 ≤ x) : 0 ≤ chamberMonomialList l := by
  induction l with
  | nil => simp [chamberMonomialList]
  | cons x xs ih =>
      have htail : ∀ y ∈ xs, 0 ≤ y := by
        intro y hy
        exact h y (by simp [hy])
      simp only [chamberMonomialList]
      exact mul_nonneg (ih htail) (List.prod_nonneg htail)

lemma vandermondeList_nonneg {l : List Real}
    (hsorted : l.Pairwise (· ≤ ·)) : 0 ≤ vandermondeList l := by
  induction l with
  | nil => simp [vandermondeList]
  | cons x xs ih =>
      rw [List.pairwise_cons] at hsorted
      have hfac : ∀ y ∈ xs, 0 ≤ y - x := by
        intro y hy
        exact sub_nonneg.mpr (hsorted.1 y hy)
      simp only [vandermondeList]
      exact mul_nonneg (ih hsorted.2) (prod_map_nonneg hfac)

/-- On a nonnegative weakly ordered chamber, the Vandermonde is bounded above
by the chamber monomial. -/
theorem vandermondeList_le_chamberMonomialList {l : List Real}
    (hpos : ∀ x ∈ l, 0 ≤ x) (hsorted : l.Pairwise (· ≤ ·)) :
    vandermondeList l ≤ chamberMonomialList l := by
  induction l with
  | nil => simp [vandermondeList, chamberMonomialList]
  | cons x xs ih =>
      rw [List.pairwise_cons] at hsorted
      have htailPos : ∀ y ∈ xs, 0 ≤ y := by
        intro y hy
        exact hpos y (by simp [hy])
      have hfac0 : ∀ y ∈ xs, 0 ≤ y - x := by
        intro y hy
        exact sub_nonneg.mpr (hsorted.1 y hy)
      have hfacLe : ∀ y ∈ xs, y - x ≤ y := by
```

```
        intro y hy
        have hx0 : 0 ≤ x := hpos x (by simp)
        linarith
      have hprodLe : (xs.map fun y => y - x).prod ≤ xs.prod := by
        simpa using (prod_map_le_prod_map (l := xs) hfac0 hfacLe)
      have hiv := ih htailPos hsorted.2
      have hv0 := vandermondeList_nonneg hsorted.2
      have hmono0 := chamberMonomialList_nonneg htailPos
      have hprod0 := prod_map_nonneg hfac0
      simp only [vandermondeList, chamberMonomialList]
      exact mul_le_mul hiv hprodLe hprod0 hmono0

lemma prod_half_mul (xs : List Real) :
    (xs.map fun y => y / 2).prod = (1 / 2 : Real) ^ xs.length * xs.prod := by
  have h := List.prod_map_mul (l := xs) (f := fun _ : Real => (1 / 2 : Real))
    (g := fun y : Real => y)
  simp [div_eq_mul_inv, mul_comm] at h ⊢

lemma separatedConstantList_pos (l : List Real) : 0 < separatedConstantList l := by
  induction l with
  | nil => simp [separatedConstantList]
  | cons x xs ih =>
      simp only [separatedConstantList]
      exact mul_pos ih (pow_pos (by norm_num) _)

/-- If every earlier coordinate is at most half of every later coordinate,
then the Vandermonde has the matching separated lower bound. -/
theorem separatedConstant_mul_chamberMonomialList_le_vandermondeList
    {l : List Real}
    (hpos : ∀ x ∈ l, 0 ≤ x)
    (hsep : l.Pairwise (fun x y => 2 * x ≤ y)) :
    separatedConstantList l * chamberMonomialList l ≤ vandermondeList l := by
  induction l with
  | nil => simp [separatedConstantList, chamberMonomialList, vandermondeList]
  | cons x xs ih =>
      rw [List.pairwise_cons] at hsep
      have htailPos : ∀ y ∈ xs, 0 ≤ y := by
        intro y hy
        exact hpos y (by simp [hy])
      have hfac0 : ∀ y ∈ xs, 0 ≤ y - x := by
        intro y hy
        have hx0 : 0 ≤ x := hpos x (by simp)
        have hxy := hsep.1 y hy
        linarith
      have hhalf0 : ∀ y ∈ xs, 0 ≤ y / 2 := by
        intro y hy
        exact div_nonneg (htailPos y hy) (by norm_num)
      have hhalfLe : ∀ y ∈ xs, y / 2 ≤ y - x := by
        intro y hy
        have hxy := hsep.1 y hy
        linarith
      have hprodLower : (1 / 2 : Real) ^ xs.length * xs.prod ≤
          (xs.map fun y => y - x).prod := by
        rw ←[ prod_half_mul xs]
        exact prod_map_le_prod_map hhalf0 hhalfLe
      have hi := ih htailPos hsep.2
      have hleft0 : 0 ≤ separatedConstantList xs * chamberMonomialList xs :=
        mul_nonneg (separatedConstantList_pos xs).le
          (chamberMonomialList_nonneg htailPos)
      have hright0 : 0 ≤ (1 / 2 : Real) ^ xs.length * xs.prod :=
        mul_nonneg (pow_nonneg (by norm_num) _) (List.prod_nonneg htailPos)
      have hv0 : 0 ≤ vandermondeList xs := hleft0.trans hi
```

```
      simp only [separatedConstantList, chamberMonomialList, vandermondeList]
      calc
        (separatedConstantList xs * (1 / 2 : Real) ^ xs.length) *
            (chamberMonomialList xs * xs.prod) =
            (separatedConstantList xs * chamberMonomialList xs) *
              ((1 / 2 : Real) ^ xs.length * xs.prod) := by ring
        _ ≤ vandermondeList xs * (xs.map fun y => y - x).prod :=
          mul_le_mul hi hprodLower hright0 hv0


/-- The recursive separated constant is exactly one factor `1/2` per pair. -/
theorem separatedConstantList_eq_choose (l : List Real) :
    separatedConstantList l = (1 / 2 : Real) ^ Nat.choose l.length 2 := by
  induction l with
  | nil => simp [separatedConstantList]
  | cons x xs ih =>
      have hchoose : Nat.choose (xs.length + 1) 2 =
          Nat.choose xs.length 2 + xs.length := by
        rw [Nat.choose_succ_succ]
        simp [Nat.choose_one_right, add_comm]
      simp only [separatedConstantList, List.length_cons]
      rw [ih, hchoose, pow_add]

/-- The recursively defined chamber monomial is the indexed product∏
` j, s_j ^ j`, for zero-based indices `j : Fin k`. -/
theorem chamberMonomialList_ofFn {k : Nat} (s : Fin k -> Real) :
    chamberMonomialList (List.ofFn s) = ∏ j, s j ^ j.val := by
  induction k with
  | zero => simp [chamberMonomialList]
  | succ k ih =>
      rw [List.ofFn_succ]
      simp only [chamberMonomialList]
      rw [ih (fun i : Fin k => s i.succ), List.prod_ofFn, Fin.prod_univ_succ]
      simp only [Fin.val_zero, pow_zero, one_mul, Fin.val_succ]
      rw ←[ Finset.prod_mul_distrib]
      apply Finset.prod_congr rfl
      intro i hi
      rw [pow_succ]

/-- The list-based Vandermonde applied to the coordinate list of a finite vector. -/
def vandermondeFin {k : Nat} (s : Fin k -> Real) : Real :=
  vandermondeList (List.ofFn s)

/-- The chamber monomial in its usual indexed form. -/
def chamberMonomialFin {k : Nat} (s : Fin k -> Real) : Real :=
  ∏ j, s j ^ j.val

/-- The Vandermonde comparison on a finite weakly ordered nonnegative chamber. -/
theorem vandermondeFin_nonneg_le {k : Nat} (s : Fin k -> Real)
    (hpos : ∀ i, 0 ≤ s i)
    (hsorted : ∀ {i j : Fin k}, i < j -> s i ≤ s j) :
    0 ≤ vandermondeFin s ∧ vandermondeFin s ≤ chamberMonomialFin s := by
  have hposList : ∀ x ∈ List.ofFn s, 0 ≤ x := by
    intro x hx
    rw [List.mem_ofFn] at hx
    rcases hx with ⟨i, ⟩rfl
    exact hpos i
  have hsortedList : (List.ofFn s).Pairwise (· ≤ ·) :=
    List.pairwise_ofFn.2 hsorted
  constructor
  · exact vandermondeList_nonneg hsortedList
  · rw [chamberMonomialFin, ← chamberMonomialList_ofFn]
```

```
    exact vandermondeList_le_chamberMonomialList hposList hsortedList

/-- On a two-separated finite chamber, the lower Vandermonde constant is
`(1/2)^(k choose 2)`. -/
theorem half_pow_choose_mul_chamberMonomialFin_le_vandermondeFin
    {k : Nat} (s : Fin k -> Real)
    (hpos : ∀ i, 0 ≤ s i)
    (hsep : ∀ {i j : Fin k}, i < j -> 2 * s i ≤ s j) :
    (1 / 2 : Real) ^ Nat.choose k 2 * chamberMonomialFin s ≤ vandermondeFin s := by
  have hposList : ∀ x ∈ List.ofFn s, 0 ≤ x := by
    intro x hx
    rw [List.mem_ofFn] at hx
    rcases hx with ⟨i, ⟩rfl
    exact hpos i
  have hsepList : (List.ofFn s).Pairwise (fun x y => 2 * x ≤ y) :=
    List.pairwise_ofFn.2 hsep
  have hmain := separatedConstant_mul_chamberMonomialList_le_vandermondeList
    hposList hsepList
  rw [separatedConstantList_eq_choose] at hmain
  unfold vandermondeFin chamberMonomialFin
  rw ←[ chamberMonomialList_ofFn]
  simpa using hmain

/-- The closed exponent is the standard `k(k-1)/2` pair count. -/
theorem half_pow_choose_eq_pair_count (k : Nat) :
    (1 / 2 : Real) ^ Nat.choose k 2 =
      (1 / 2 : Real) ^ (k * (k - 1) / 2) := by
  rw [Nat.choose_two_right]



end O77.Residual
end
```

Lemma 20.9 (Pointwise Laguerre product comparison)**.**

Corresponding Lean declaration(s): `O77.Residual.commonRpowProduct`,`O77.Residual.absorbedRpowProduct`,`O77.Residual.commonRpowProduct_mul_chamberMonomialFin`,`O77.Residual.laguerreChamberCore`,`O77.Residual.commonRpowProduct_pos`.

Corresponding Lean declaration(s): `O77.Residual.laguerreChamberCore_nonneg_le_absorbed`,`O77.Residual.half_pow_choose_mul_absorbed_le_laguerreChamberCore`,`O77.Residual.laguerreWeightProduct`,`O77.Residual.laguerreModelProduct`,`O77.Residual.laguerreChamberIntegrand`.

Corresponding Lean declaration(s): `O77.Residual.laguerreWeightProduct_pos`,`O77.Residual.absorbed_mul_weight_eq_model`,`O77.Residual.laguerreChamberIntegrand_nonneg_le_model`,`O77.Residual.half_pow_choose_mul_model_le_laguerreChamberIntegrand`,`O77.Residual.spectralLaguerreIntegrand`.

Corresponding Lean declaration(s): `O77.Residual.laguerreWeightProduct_eq_exp_sum_mul`,`O77.Residual.laguerreChamberIntegrand_eq_spectral`.

For zero-based coordinates $s : \mathrm{Fin}\, k \to \mathbb{R}$, define

$$\mathcal{L}_{\eta,\rho,T}(s) = e^{-\sum_i s_i}\Big(\prod_i s_i^{\eta}\Big)\Big(\prod_{i<j}(s_j - s_i)\Big)\prod_i (1+Ts_i)^{-\rho},$$

and

$$\mathcal{M}_{\eta,\rho,T}(s) = \prod_i e^{-s_i} s_i^{\eta+i}(1+Ts_i)^{-\rho}.$$

In this display the index in the exponent is zero-based: it means $i.\mathrm{val}$ *for* $i : \mathrm{Fin}\, k$*. Thus the corresponding one-dimensional model parameter is*

$$A_i = \eta + i.\mathrm{val} + 1, \qquad s_i^{A_i - 1} = s_i^{\eta + i.\mathrm{val}}.$$

Equivalently, with the paper's one-based indices $j = 1, \dots, k$*, this is the familiar notation* $A_j = \eta + j$*. If* $T \geq 0$*, every* $s_i > 0$*, and the coordinates are weakly increasing, then*

$$0 \leq \mathcal{L}_{\eta,\rho,T}(s) \leq \mathcal{M}_{\eta,\rho,T}(s).$$

If they are two-separated, then

$$2^{-\binom{k}{2}} \mathcal{M}_{\eta,\rho,T}(s) \leq \mathcal{L}_{\eta,\rho,T}(s).$$

The factored definition of $\mathcal{L}$ *agrees exactly with the conventional spectral formula above.*

Proof. Positivity of the coordinates permits the real-power identity $s_i^{\eta} s_i^{i.\mathrm{val}} = s_i^{\eta + i.\mathrm{val}}$. In the model-integral notation this exponent is $A_i - 1$, not A_i; the preceding $+1$ is therefore mandatory. Multiplying this identity over all coordinates absorbs the chamber monomial from Lemma 20.8 into the power product. The remaining weight $\prod_i e^{-s_i}(1 + T s_i)^{-\rho}$ is strictly positive for $T \geq 0$. Multiplication by that weight transports the Vandermonde upper and lower inequalities. Finally, $\prod_i e^{-s_i} = e^{-\sum_i s_i}$ proves equality with the conventional spectral expression. No integration or change of variables occurs in this lemma.

□

```
noncomputable section
open Real MeasureTheory Set
open scoped ENNReal BigOperators
namespace O77.Residual
/-- The product of the common Laguerre power before the Vandermonde is absorbed. -/
def commonRpowProduct {k : Nat} (eta : Real) (s : Fin k -> Real) : Real :=
  ∏ i, s i ^ eta

/-- The coordinatewise powers after absorbing the chamber monomial. -/
def absorbedRpowProduct {k : Nat} (eta : Real) (s : Fin k -> Real) : Real :=
  ∏ i, s i ^ (eta + i.val)

/-- On the positive chamber, multiplying the common real powers by the chamber
monomial adds the zero-based index to each real exponent. -/
theorem commonRpowProduct_mul_chamberMonomialFin {k : Nat}
    (eta : Real) (s : Fin k -> Real) (hpos : ∀ i, 0 < s i) :
    commonRpowProduct eta s * chamberMonomialFin s = absorbedRpowProduct eta s := by
  unfold commonRpowProduct chamberMonomialFin absorbedRpowProduct
  rw ←[ Finset.prod_mul_distrib]
  apply Finset.prod_congr rfl
  intro i hi
  rw ←[ Real.rpow_natCast]
  rw ←[ Real.rpow_add (hpos i)]


/-- The Laguerre power times the Vandermonde on the ordered chamber. -/
def laguerreChamberCore {k : Nat} (eta : Real) (s : Fin k -> Real) : Real :=
  commonRpowProduct eta s * vandermondeFin s

lemma commonRpowProduct_pos {k : Nat} (eta : Real) (s : Fin k -> Real)
    (hpos : ∀ i, 0 < s i) : 0 < commonRpowProduct eta s := by
  unfold commonRpowProduct
  exact Finset.prod_pos fun i _ => Real.rpow_pos_of_pos (hpos i) eta
```

```
/-- The absorbed coordinate product is an upper majorant of the Laguerre
power-Vandermonde core on the positive ordered chamber. -/
theorem laguerreChamberCore_nonneg_le_absorbed {k : Nat}
    (eta : Real) (s : Fin k -> Real)
    (hpos : ∀ i, 0 < s i)
    (hsorted : ∀ {i j : Fin k}, i < j -> s i ≤ s j) :
    0 ≤ laguerreChamberCore eta s ∧
      laguerreChamberCore eta s ≤ absorbedRpowProduct eta s := by
  have hV := vandermondeFin_nonneg_le s (fun i => (hpos i).le) hsorted
  have hcommon := (commonRpowProduct_pos eta s hpos).le
  constructor
  · exact mul_nonneg hcommon hV.1
  · rw [laguerreChamberCore, ← commonRpowProduct_mul_chamberMonomialFin eta s hpos]
    exact mul_le_mul_of_nonneg_left hV.2 hcommon

/-- On a two-separated positive chamber, the absorbed coordinate product,
with one factor `1/2` per pair, is a lower minorant of the Laguerre core. -/
theorem half_pow_choose_mul_absorbed_le_laguerreChamberCore {k : Nat}
    (eta : Real) (s : Fin k -> Real)
    (hpos : ∀ i, 0 < s i)
    (hsep : ∀ {i j : Fin k}, i < j -> 2 * s i ≤ s j) :
    (1 / 2 : Real) ^ Nat.choose k 2 * absorbedRpowProduct eta s ≤
      laguerreChamberCore eta s := by
  have hV := half_pow_choose_mul_chamberMonomialFin_le_vandermondeFin s
    (fun i => (hpos i).le) hsep
  have hcommon := (commonRpowProduct_pos eta s hpos).le
  rw [laguerreChamberCore, ← commonRpowProduct_mul_chamberMonomialFin eta s hpos]
  calc
    (1 / 2 : Real) ^ Nat.choose k 2 *
        (commonRpowProduct eta s * chamberMonomialFin s) =
        commonRpowProduct eta s *
          ((1 / 2 : Real) ^ Nat.choose k 2 * chamberMonomialFin s) := by ring
    _ ≤ commonRpowProduct eta s * vandermondeFin s :=
      mul_le_mul_of_nonneg_left hV hcommon


/-- The remaining positive coordinate weights in the Laguerre integrand. -/
def laguerreWeightProduct {k : Nat} (rho T : Real) (s : Fin k -> Real) : Real :=
  ∏ i, Real.exp (-s i) * (1 + T * s i) ^ (-rho)

/-- The completely separated product of the one-dimensional model kernels. -/
def laguerreModelProduct {k : Nat} (eta rho T : Real) (s : Fin k -> Real) : Real :=
  ∏ i, Real.exp (-s i) * s i ^ (eta + i.val) * (1 + T * s i) ^ (-rho)

/-- The ordered-chamber Laguerre integrand, written as its power-Vandermonde
core times the remaining coordinate weights. -/
def laguerreChamberIntegrand {k : Nat} (eta rho T : Real)
    (s : Fin k -> Real) : Real :=
  laguerreChamberCore eta s * laguerreWeightProduct rho T s

lemma laguerreWeightProduct_pos {k : Nat} (rho T : Real) (s : Fin k -> Real)
    (hT : 0 ≤ T) (hpos : ∀ i, 0 < s i) : 0 < laguerreWeightProduct rho T s := by
  unfold laguerreWeightProduct
  apply Finset.prod_pos
  intro i hi
  have hmul : 0 ≤ T * s i := mul_nonneg hT (hpos i).le
  have hbase : 0 < 1 + T * s i := by linarith
  exact mul_pos (Real.exp_pos _) (Real.rpow_pos_of_pos hbase _)

lemma absorbed_mul_weight_eq_model {k : Nat} (eta rho T : Real)
    (s : Fin k -> Real) :
    absorbedRpowProduct eta s * laguerreWeightProduct rho T s =
```

```
      laguerreModelProduct eta rho T s := by
  unfold absorbedRpowProduct laguerreWeightProduct laguerreModelProduct
  rw ←[ Finset.prod_mul_distrib]
  apply Finset.prod_congr rfl
  intro i hi
  ring

/-- The full ordered-chamber Laguerre integrand is bounded above by the
separated product of one-dimensional model kernels. -/
theorem laguerreChamberIntegrand_nonneg_le_model {k : Nat}
    (eta rho T : Real) (s : Fin k -> Real)
    (hT : 0 ≤ T) (hpos : ∀ i, 0 < s i)
    (hsorted : ∀ {i j : Fin k}, i < j -> s i ≤ s j) :
    0 ≤ laguerreChamberIntegrand eta rho T s ∧
      laguerreChamberIntegrand eta rho T s ≤ laguerreModelProduct eta rho T s := by
  have hcore := laguerreChamberCore_nonneg_le_absorbed eta s hpos hsorted
  have hw := (laguerreWeightProduct_pos rho T s hT hpos).le
  constructor
  · exact mul_nonneg hcore.1 hw
  · rw [laguerreChamberIntegrand, ← absorbed_mul_weight_eq_model]
    exact mul_le_mul_of_nonneg_right hcore.2 hw

/-- On a two-separated chamber, the full Laguerre integrand has the matching
one-factor-per-pair lower minorant by the separated model product. -/
theorem half_pow_choose_mul_model_le_laguerreChamberIntegrand {k : Nat}
    (eta rho T : Real) (s : Fin k -> Real)
    (hT : 0 ≤ T) (hpos : ∀ i, 0 < s i)
    (hsep : ∀ {i j : Fin k}, i < j -> 2 * s i ≤ s j) :
    (1 / 2 : Real) ^ Nat.choose k 2 * laguerreModelProduct eta rho T s ≤
      laguerreChamberIntegrand eta rho T s := by
  have hcore := half_pow_choose_mul_absorbed_le_laguerreChamberCore eta s hpos hsep
  have hw := (laguerreWeightProduct_pos rho T s hT hpos).le
  rw [laguerreChamberIntegrand, ← absorbed_mul_weight_eq_model]
  calc
    (1 / 2 : Real) ^ Nat.choose k 2 *
        (absorbedRpowProduct eta s * laguerreWeightProduct rho T s) =
        ((1 / 2 : Real) ^ Nat.choose k 2 * absorbedRpowProduct eta s) *
          laguerreWeightProduct rho T s := by ring
    _ ≤ laguerreChamberCore eta s * laguerreWeightProduct rho T s :=
      mul_le_mul_of_nonneg_right hcore hw


/-- The same Laguerre integrand in the conventional spectral formula. -/
def spectralLaguerreIntegrand {k : Nat} (eta rho T : Real)
    (s : Fin k -> Real) : Real :=
  Real.exp ∑(-( i, s i)) * commonRpowProduct eta s * vandermondeFin s *
    ∏( i, (1 + T * s i) ^ (-rho))

lemma laguerreWeightProduct_eq_exp_sum_mul {k : Nat} (rho T : Real)
    (s : Fin k -> Real) :
    laguerreWeightProduct rho T s =
      Real.exp ∑(-( i, s i)) * ∏( i, (1 + T * s i) ^ (-rho)) := by
  unfold laguerreWeightProduct
  rw [Finset.prod_mul_distrib, ← Real.exp_sum]
  congr 2
  rw [Finset.sum_neg_distrib]

/-- The factored and conventional spectral expressions agree exactly. -/
theorem laguerreChamberIntegrand_eq_spectral {k : Nat}
    (eta rho T : Real) (s : Fin k -> Real) :
    laguerreChamberIntegrand eta rho T s =
      spectralLaguerreIntegrand eta rho T s := by
```

```
  rw [laguerreChamberIntegrand, laguerreWeightProduct_eq_exp_sum_mul]
  unfold laguerreChamberCore spectralLaguerreIntegrand
  ring


end O77.Residual
end
```

Lemma 20.10 (Checked three-range bounds and elementary integrals)**.**

Corresponding Lean declaration(s): `O77.Residual.Model.nearIntegral,O77.Residual.Model.middleIntegral,O77.Residual.Model.tailIntegral,O77.Residual.Model.zero_le_inv,O77.Residual.Model.inv_le_one`.

Corresponding Lean declaration(s): `O77.Residual.Model.integral_eq_three,O77.Residual.Model.exp_neg_one_le_exp_neg,O77.Residual.Model.rpow_neg_anti,O77.Residual.Model.near_pointwise_lower,O77.Residual.Model.near_pointwise_upper`.

Corresponding Lean declaration(s): `O77.Residual.Model.rpow_product_normalize,O77.Residual.Model.middle_pointwise_upper,O77.Residual.Model.middle_pointwise_lower,O77.Residual.Model.tail_pointwise_upper,O77.Residual.Model.integral_rpow_zero_inv`.

Corresponding Lean declaration(s): `O77.Residual.Model.integral_rpow_inv_one,O77.Residual.Model.integral_inv_inv_one,O77.Residual.Model.nearIntegral_lower,O77.Residual.Model.nearIntegral_upper,O77.Residual.Model.middle_pointwise_upper_power`.

Corresponding Lean declaration(s): `O77.Residual.Model.middlePower_integrableOn,O77.Residual.Model.middleIntegral_bounds_nonresonant,O77.Residual.Model.middleIntegral_bounds_resonant,O77.Residual.Model.tailConstant,O77.Residual.Model.tailConstant_nonneg`.

Corresponding Lean declaration(s): `O77.Residual.Model.tailIntegral_upper,O77.Residual.Model.fixedPowerConstant,O77.Residual.Model.fixedPowerConstant_pos,O77.Residual.Model.integral_rpow_half_one,O77.Residual.Model.middleIntegral_fixed_lower`.

For $A > 0$, $\rho \geq 0$, and $T \geq 1$, the model integral is the exact sum of its restrictions to $(0, 1/T]$, $(1/T, 1]$, and $(1, \infty)$. The first restriction satisfies (19.4.1). The middle restriction satisfies (19.4.2) when $A \neq \rho$ and (19.4.3) at resonance. Its elementary factors are proved exactly:

$$\int_0^{1/T} s^{A-1}\,ds = \frac{T^{-A}}{A}, \qquad \int_{1/T}^{1} s^{B-1}\,ds = \frac{1 - T^{-B}}{B} \quad (B \neq 0),$$

$$\int_{1/T}^{1} \frac{ds}{s} = \log T.$$

The tail is bounded by $Q_A T^{-\rho}$, with Q_A as in the proof of Lemma 19.4. When $\rho < A$ and $T \geq 2$, the restriction to $[1/2, 1]$ has the explicit positive lower constant used there.

Proof. All statements are the auxiliary declarations used in the checked proof of Lemma 19.4. The set identity

$$(0, 1/T] \sqcup (1/T, 1] \sqcup (1, \infty) = (0, \infty)$$

is proved with disjoint measurable intervals before additivity of the integral is applied. For the first power integral, $A > 0$ gives integrability at the endpoint zero. The middle power integral is on the interval $[1/T, 1]$, which is bounded away from zero. These facts discharge the relevant

hypotheses of Mathlib's real-power integral formulas before those formulas are applied. The resonant case is separated before division by $B = A - \rho$. The tail is dominated by a Gamma integrand, and the fixed-interval coefficient $(1 - 2^{-(A-\rho)})/(A - \rho)$ is proved strictly positive when $\rho < A$.

□

Lemma 20.11 (Localized lower factors on the spectral boxes)**.**

Corresponding Lean declaration(s): `O77.Residual.Model.localizedIntegral,O77.Residual.Model.endpointRpowMin,O77.Residual.Model.endpointRpowMin_pos,O77.Residual.Model.endpointRpowMin_le,O77.Residual.Model.rpow_scaled_identity`.

Corresponding Lean declaration(s): `O77.Residual.Model.rpow_one_sub_mul_inv,O77.Residual.Model.integral_const_Ioc,O77.Residual.Model.scaledIntervalConstant,O77.Residual.Model.scaledIntervalConstant_pos,O77.Residual.Model.scaled_interval_pointwise_lower`.

Corresponding Lean declaration(s): `O77.Residual.Model.localized_scaled_lower,O77.Residual.Model.fixedIntervalConstant,O77.Residual.Model.fixedIntervalConstant_pos,O77.Residual.Model.fixed_interval_pointwise_lower,O77.Residual.Model.localized_fixed_lower`.

Corresponding Lean declaration(s): `O77.Residual.Model.integral_inv_cdiv_one,O77.Residual.Model.half_log_le_log_div,O77.Residual.Model.localized_resonant_lower`.

Let $A > 0$, $\rho > 0$ and $c \geq 1$. There are explicit positive constants, independent of T, with the following properties:

(i) if $A < \rho$ and $2c \leq T$, then

$$c_< T^{-A} \leq \int_{c/T}^{2c/T} e^{-s} s^{A-1} (1 + Ts)^{-\rho}\, ds;$$

(ii) if $\rho < A$ and $1 \leq T$, then

$$c_> T^{-\rho} \leq \int_{c}^{2c} e^{-s} s^{A-1} (1 + Ts)^{-\rho}\, ds;$$

(iii) if $A = \rho$ and $c^2 \leq T$, then

$$c_= T^{-A} \log T \leq \int_{c/T}^{1} e^{-s} s^{A-1} (1 + Ts)^{-A}\, ds.$$

The same statements hold for the formal half-open intervals $(c/T, 2c/T]$, $(c, 2c]$ and $(c/T, 1]$ used in the product boxes.

Proof. For the scaled interval, write $u = Ts$ only at the level of the elementary factors. On $c/T < s \leq 2c/T$ one has $s \leq 1$ when $2c \leq T$, $1 + Ts \leq 1 + 2c$, and

$$s^{A-1} \geq m_{A,c} T^{1-A}, \qquad m_{A,c} = \min\{c^{A-1}, (2c)^{A-1}\} > 0.$$

The interval has length c/T. Multiplying these inequalities gives (i) with $c_< = ce^{-1}(1 + 2c)^{-\rho} m_{A,c} > 0$. The checked proof derives the same estimate directly by integral monotonicity, so it needs no separate global change-of-variables theorem.

On the fixed interval, $e^{-s} \geq e^{-2c}$, $s^{A-1} \geq m_{A,c}$, and $1 + Ts \leq T(1 + 2c)$ because $T, c \geq 1$. Since the interval length is c, this gives (ii) with $c_> = ce^{-2c}(1 + 2c)^{-\rho} m_{A,c} > 0$.

At resonance, $c/T < s \le 1$ lies inside $1/T < s \le 1$. Hence

$$e^{-s} s^{A-1} (1 + Ts)^{-A} \ge e^{-1} 2^{-A} T^{-A} s^{-1}.$$

The exact endpoint integral is $\log(T/c)$. From $c \ge 1$ and $c^2 \le T$, $\log(T/c) \ge \frac{1}{2} \log T$, which proves (iii) with $c_= = e^{-1} 2^{-A}/2 > 0$. Integrability is established before each monotonicity argument. No constant depends on T.

Provenance: Sneiderman, Appendix A, localized factors in the proof of the explicit vertex and edge lower bounds. □

```
noncomputable section
open Real MeasureTheory Set
open scoped ENNReal BigOperators

namespace O77.Residual.Model

/-- The model kernel restricted to a half-open interval. -/
def localizedIntegral (A rho T l u : Real) : Real :=
  ∫ s : Real in Ioc l u, integrand A rho T s

/-- A positive endpoint lower bound for the power `u^(A-1)` on `[c,2c]`. -/
def endpointRpowMin (A c : Real) : Real :=
  min (c ^ (A - 1)) ((2 * c) ^ (A - 1))

lemma endpointRpowMin_pos {A c : Real} (hc : 0 < c) :
    0 < endpointRpowMin A c := by
  unfold endpointRpowMin
  exact lt_min (Real.rpow_pos_of_pos hc _) (Real.rpow_pos_of_pos (by positivity) _)

lemma endpointRpowMin_le {A c u : Real} (hc : 0 < c)
    (hcu : c ≤ u) (hu : u ≤ 2 * c) :
    endpointRpowMin A c ≤ u ^ (A - 1) := by
  unfold endpointRpowMin
  rcases le_total 0 (A - 1) with hnonneg | hnonpos
  · exact (min_le_left _ _).trans (Real.rpow_le_rpow hc.le hcu hnonneg)
  · exact (min_le_right _ _).trans
      (Real.rpow_le_rpow_of_nonpos (lt_of_lt_of_le hc hcu) hu hnonpos)

lemma rpow_scaled_identity {A T s : Real} (hT : 0 < T) (hs : 0 < s) :
    s ^ (A - 1) = T ^ (1 - A) * (T * s) ^ (A - 1) := by
  rw [Real.mul_rpow hT.le hs.le]
  calc
    s ^ (A - 1) = 1 * s ^ (A - 1) := by ring
    _ = (T ^ (1 - A) * T ^ (A - 1)) * s ^ (A - 1) := by
      rw ←[ Real.rpow_add hT]
      norm_num
    _ = T ^ (1 - A) * (T ^ (A - 1) * s ^ (A - 1)) := by ring

lemma rpow_one_sub_mul_inv {A T : Real} (hT : 0 < T) :
    T ^ (1 - A) * (1 / T) = T ^ (-A) := by
  rw [one_div, ← Real.rpow_neg_one]
  rw ←[ Real.rpow_add hT]
  congr 1
  ring

lemma integral_const_Ioc {l u c : Real} (hlu : l ≤ u) :
    ∫( _s : Real in Ioc l u, c) = (u - l) * c := by
  rw ←[ intervalIntegral.integral_of_le hlu, intervalIntegral.integral_const]
  simp [smul_eq_mul]

/-- Crude but uniform positive constant for a scaled interval. -/
```

```
def scaledIntervalConstant (A rho c : Real) : Real :=
  c * (Real.exp (-1) * (1 + 2 * c) ^ (-rho) * endpointRpowMin A c)

lemma scaledIntervalConstant_pos {A rho c : Real} (hc : 0 < c) :
    0 < scaledIntervalConstant A rho c := by
  unfold scaledIntervalConstant
  have hb : 0 < 1 + 2 * c := by positivity
  exact mul_pos hc (mul_pos (mul_pos (Real.exp_pos _) (Real.rpow_pos_of_pos hb _))
    (endpointRpowMin_pos hc))

lemma scaled_interval_pointwise_lower {A rho c T s : Real}
    (hc : 0 < c) (hrho : 0 ≤ rho) (hT : 2 * c ≤ T)
    (hs : s ∈ Ioc (c / T) (2 * c / T)) :
    Real.exp (-1) * (1 + 2 * c) ^ (-rho) * endpointRpowMin A c *
        T ^ (1 - A) ≤ integrand A rho T s := by
  have hT0 : 0 < T := lt_of_lt_of_le (by positivity : 0 < 2 * c) hT
  have hs0 : 0 < s := by
    exact (div_pos hc hT0).trans hs.1
  have hTsLower : c ≤ T * s := by
    have := mul_le_mul_of_nonneg_left hs.1.le hT0.le
    field_simp [ne_of_gt hT0] at this
    simpa [mul_assoc] using this
  have hTsUpper : T * s ≤ 2 * c := by
    have := mul_le_mul_of_nonneg_left hs.2 hT0.le
    field_simp [ne_of_gt hT0] at this
    simpa [mul_assoc] using this
  have hsOne : s ≤ 1 := by
    calc
      s ≤ 2 * c / T := hs.2
      _ ≤ 1 := (div_le_one hT0).2 hT
  have hexp : Real.exp (-1) ≤ Real.exp (-s) := exp_neg_one_le_exp_neg hsOne
  have hbase0 : 0 < 1 + T * s := by positivity
  have hbase : 1 + T * s ≤ 1 + 2 * c := by linarith
  have hfac : (1 + 2 * c) ^ (-rho) ≤ (1 + T * s) ^ (-rho) :=
    Real.rpow_le_rpow_of_nonpos hbase0 hbase (neg_nonpos.mpr hrho)
  have hpowScaled : endpointRpowMin A c * T ^ (1 - A) ≤ s ^ (A - 1) := by
    rw [rpow_scaled_identity hT0 hs0]
    have hmin := endpointRpowMin_le (A := A) hc hTsLower hTsUpper
    have htPow : 0 ≤ T ^ (1 - A) := Real.rpow_nonneg hT0.le _
    nlinarith [mul_le_mul_of_nonneg_left hmin htPow]
  unfold integrand
  have hconst0 : 0 ≤ endpointRpowMin A c * T ^ (1 - A) :=
    mul_nonneg (endpointRpowMin_pos hc).le (Real.rpow_nonneg hT0.le _)
  calc
    Real.exp (-1) * (1 + 2 * c) ^ (-rho) * endpointRpowMin A c * T ^ (1 - A)
        = Real.exp (-1) * (1 + 2 * c) ^ (-rho) *
            (endpointRpowMin A c * T ^ (1 - A)) := by ring
    _ ≤ Real.exp (-s) * (1 + T * s) ^ (-rho) * s ^ (A - 1) := by
      gcongr
    _ = Real.exp (-s) * s ^ (A - 1) * (1 + T * s) ^ (-rho) := by ring

/-- Localized scaled-interval lower bound. -/
theorem localized_scaled_lower {A rho c T : Real}
    (hA : 0 < A) (hc : 0 < c) (hrho : 0 ≤ rho) (hT : 2 * c ≤ T) :
    scaledIntervalConstant A rho c * T ^ (-A) ≤
      localizedIntegral A rho T (c / T) (2 * c / T) := by
  have hT0 : 0 < T := lt_of_lt_of_le (by positivity : 0 < 2 * c) hT
  have hlu : c / T ≤ 2 * c / T := by
    exact div_le_div_of_nonneg_right (by linarith) hT0.le
  have hInt := integrableOn_integrand_Ioi hA hT0.le hrho
  have hIntLoc : IntegrableOn (integrand A rho T) (Ioc (c / T) (2 * c / T)) :=
    hInt.mono_set (fun s hs => (div_pos hc hT0).trans hs.1)
```

```
  let d : Real := Real.exp (-1) * (1 + 2 * c) ^ (-rho) * endpointRpowMin A c *
      T ^ (1 - A)
  have hvol : volume (Ioc (c / T) (2 * c / T)) ≠ ∞ := by
    rw [Real.volume_Ioc]
    simp
  have hdInt : IntegrableOn (fun _s : Real => d) (Ioc (c / T) (2 * c / T)) :=
    integrableOn_const hvol
  have hmono :
      ∫( _s : Real in Ioc (c / T) (2 * c / T), d) ≤
        localizedIntegral A rho T (c / T) (2 * c / T) := by
    apply MeasureTheory.integral_mono_ae hdInt hIntLoc
    filter_upwards [ae_restrict_mem measurableSet_Ioc] with s hs
    exact scaled_interval_pointwise_lower hc hrho hT hs
  rw [integral_const_Ioc hlu] at hmono
  have hwidth : 2 * c / T - c / T = c * (1 / T) := by field_simp [ne_of_gt hT0]; ring
  rw [hwidth] at hmono
  have hscale := rpow_one_sub_mul_inv (A := A) hT0
  unfold scaledIntervalConstant localizedIntegral
  dsimp [d] at hmono
  calc
    c * (Real.exp (-1) * (1 + 2 * c) ^ (-rho) * endpointRpowMin A c) * T ^ (-A)
        = (c * (1 / T)) *
            (Real.exp (-1) * (1 + 2 * c) ^ (-rho) * endpointRpowMin A c *
              T ^ (1 - A)) := by
          rw ←[ hscale]
          ring
    _ ≤ _ := hmono

/-- Crude but uniform positive constant for a fixed interval. -/
def fixedIntervalConstant (A rho c : Real) : Real :=
  c * (Real.exp (-(2 * c)) * (1 + 2 * c) ^ (-rho) * endpointRpowMin A c)

lemma fixedIntervalConstant_pos {A rho c : Real} (hc : 0 < c) :
    0 < fixedIntervalConstant A rho c := by
  unfold fixedIntervalConstant
  have hb : 0 < 1 + 2 * c := by positivity
  exact mul_pos hc (mul_pos (mul_pos (Real.exp_pos _) (Real.rpow_pos_of_pos hb _))
    (endpointRpowMin_pos hc))

lemma fixed_interval_pointwise_lower {A rho c T s : Real}
    (hc : 1 ≤ c) (hrho : 0 ≤ rho) (hT : 1 ≤ T)
    (hs : s ∈ Ioc c (2 * c)) :
    Real.exp (-(2 * c)) * (1 + 2 * c) ^ (-rho) * endpointRpowMin A c *
        T ^ (-rho) ≤ integrand A rho T s := by
  have hc0 : 0 < c := lt_of_lt_of_le zero_lt_one hc
  have hT0 : 0 < T := lt_of_lt_of_le zero_lt_one hT
  have hs0 : 0 < s := hc0.trans hs.1
  have hexp : Real.exp (-(2 * c)) ≤ Real.exp (-s) :=
    Real.exp_le_exp.mpr (neg_le_neg hs.2)
  have hpow : endpointRpowMin A c ≤ s ^ (A - 1) :=
    endpointRpowMin_le hc0 hs.1.le hs.2
  have hbase0 : 0 < 1 + T * s := by positivity
  have hbase : 1 + T * s ≤ T * (1 + 2 * c) := by
    nlinarith [mul_le_mul_of_nonneg_left hs.2 hT0.le]
  have hbig0 : 0 < T * (1 + 2 * c) := by positivity
  have hfac : (T * (1 + 2 * c)) ^ (-rho) ≤ (1 + T * s) ^ (-rho) :=
    Real.rpow_le_rpow_of_nonpos hbase0 hbase (neg_nonpos.mpr hrho)
  have hsplit : (T * (1 + 2 * c)) ^ (-rho) =
      T ^ (-rho) * (1 + 2 * c) ^ (-rho) := by
    rw [Real.mul_rpow hT0.le (by positivity : 0 ≤ 1 + 2 * c)]
  rw [hsplit] at hfac
  unfold integrand
```

```
  have hnonneg : 0 ≤ endpointRpowMin A c := (endpointRpowMin_pos hc0).le
  calc
    Real.exp (-(2 * c)) * (1 + 2 * c) ^ (-rho) * endpointRpowMin A c * T ^ (-rho)
        = Real.exp (-(2 * c)) * endpointRpowMin A c *
            (T ^ (-rho) * (1 + 2 * c) ^ (-rho)) := by ring
    _ ≤ Real.exp (-s) * s ^ (A - 1) * (1 + T * s) ^ (-rho) := by
      gcongr

/-- Localized fixed-interval lower bound. -/
theorem localized_fixed_lower {A rho c T : Real}
    (hA : 0 < A) (hc : 1 ≤ c) (hrho : 0 ≤ rho) (hT : 1 ≤ T) :
    fixedIntervalConstant A rho c * T ^ (-rho) ≤
      localizedIntegral A rho T c (2 * c) := by
  have hc0 : 0 < c := lt_of_lt_of_le zero_lt_one hc
  have hT0 : 0 < T := lt_of_lt_of_le zero_lt_one hT
  have hInt := integrableOn_integrand_Ioi hA hT0.le hrho
  have hIntLoc : IntegrableOn (integrand A rho T) (Ioc c (2 * c)) :=
    hInt.mono_set (fun s hs => hc0.trans hs.1)
  let d : Real := Real.exp (-(2 * c)) * (1 + 2 * c) ^ (-rho) *
      endpointRpowMin A c * T ^ (-rho)
  have hvol : volume (Ioc c (2 * c)) ≠ ∞ := by
    rw [Real.volume_Ioc]
    simp
  have hdInt : IntegrableOn (fun _s : Real => d) (Ioc c (2 * c)) :=
    integrableOn_const hvol
  have hmono : ∫ _s : Real in Ioc c (2 * c), d) ≤
      localizedIntegral A rho T c (2 * c) := by
    apply MeasureTheory.integral_mono_ae hdInt hIntLoc
    filter_upwards [ae_restrict_mem measurableSet_Ioc] with s hs
    exact fixed_interval_pointwise_lower hc hrho hT hs
  have hlu : c ≤ 2 * c := by linarith
  rw [integral_const_Ioc hlu] at hmono
  unfold fixedIntervalConstant
  dsimp [d] at hmono
  calc
    c * (Real.exp (-(2 * c)) * (1 + 2 * c) ^ (-rho) * endpointRpowMin A c) *
        T ^ (-rho) =
      (2 * c - c) *
        (Real.exp (-(2 * c)) * (1 + 2 * c) ^ (-rho) * endpointRpowMin A c *
          T ^ (-rho)) := by ring
    _ ≤ localizedIntegral A rho T c (2 * c) := hmono

lemma integral_inv_cdiv_one {c T : Real} (hc : 0 < c) (hT : c ≤ T) :
    ∫ s : Real in Ioc (c / T) 1, 1 / s) = Real.log (T / c) := by
  have hT0 : 0 < T := hc.trans_le hT
  have hlo0 : 0 < c / T := div_pos hc hT0
  have hlo1 : c / T ≤ 1 := (div_le_one hT0).2 hT
  rw ←[ intervalIntegral.integral_of_le hlo1]
  have hzero : (0 : Real) ∉ uIcc (c / T) 1 := by
    rw [uIcc_of_le hlo1]
    intro hz
    exact (not_le_of_gt hlo0) hz.1
  rw [integral_one_div hzero]
  have hcne : c ≠ 0 := ne_of_gt hc
  have hTne : T ≠ 0 := ne_of_gt hT0
  congr 1
  field_simp

lemma half_log_le_log_div {c T : Real} (hc : 1 ≤ c) (hT : c ^ 2 ≤ T) :
    (1 / 2 : Real) * Real.log T ≤ Real.log (T / c) := by
  have hc0 : 0 < c := lt_of_lt_of_le zero_lt_one hc
  have hT0 : 0 < T := lt_of_lt_of_le (sq_pos_of_pos hc0) hT
```

```
  have hratio0 : 0 < T / c := div_pos hT0 hc0
  have hcRatio : c ≤ T / c := by
    apply (₀le_div_iff hc0).2
    nlinarith [hT]
  have hlogc : Real.log c ≤ Real.log (T / c) := Real.log_le_log hc0 hcRatio
  have hprod : (T / c) * c = T := by field_simp
  have hlogprod := Real.log_mul (ne_of_gt hratio0) (ne_of_gt hc0)
  rw [hprod] at hlogprod
  nlinarith [hlogc]

/-- Localized resonant lower bound retaining the complete logarithm. -/
theorem localized_resonant_lower {A c T : Real}
    (hA : 0 < A) (hc : 1 ≤ c) (hT : c ^ 2 ≤ T) :
    (Real.exp (-1) * (2 : Real) ^ (-A) / 2) *
        (T ^ (-A) * Real.log T) ≤
      localizedIntegral A A T (c / T) 1 := by
  have hc0 : 0 < c := lt_of_lt_of_le zero_lt_one hc
  have hT0 : 0 < T := lt_of_lt_of_le (sq_pos_of_pos hc0) hT
  have hcT : c ≤ T := by nlinarith [sq_nonneg (c - 1), hT]
  have hlo1 : c / T ≤ 1 := (div_le_one hT0).2 hcT
  have hInt := integrableOn_integrand_Ioi hA hT0.le hA.le
  have hIntLoc : IntegrableOn (integrand A A T) (Ioc (c / T) 1) :=
    hInt.mono_set (fun s hs => (div_pos hc0 hT0).trans hs.1)
  have hInvInt : IntegrableOn (fun s : Real => 1 / s) (Ioc (c / T) 1) := by
    have hzero : (0 : Real) ∉ uIcc (c / T) 1 := by
      rw [uIcc_of_le hlo1]
      intro hz
      exact (not_le_of_gt (div_pos hc0 hT0)) hz.1
    have hi : IntervalIntegrable (fun s : Real => 1 / s) volume (c / T) 1 :=
      intervalIntegral.intervalIntegrable_one_div
        (fun s hs => ne_of_mem_of_not_mem hs hzero) continuousOn_id
    exact (intervalIntegrable_iff_integrableOn_Ioc_of_le hlo1).1 hi
  let d : Real := Real.exp (-1) * (2 : Real) ^ (-A) * T ^ (-A)
  have hdInt : IntegrableOn (fun s : Real => d * (1 / s)) (Ioc (c / T) 1) :=
    hInvInt.const_mul d
  have hmono : ∫( s : Real in Ioc (c / T) 1, d * (1 / s)) ≤
      localizedIntegral A A T (c / T) 1 := by
    apply MeasureTheory.integral_mono_ae hdInt hIntLoc
    filter_upwards [ae_restrict_mem measurableSet_Ioc] with s hs
    have hs0 : 0 < s := (div_pos hc0 hT0).trans hs.1
    have hmid := middle_pointwise_lower (A := A) (rho := A) hA.le hT0
      (by
        have : 1 / T ≤ c / T := div_le_div_of_nonneg_right hc hT0.le
        exact this.trans hs.1.le) hs.2
    rw [show A - A - 1 = (-1 : Real) by ring, Real.rpow_neg_one, inv_eq_one_div] at hmid
    exact hmid
  rw [MeasureTheory.integral_const_mul, integral_inv_cdiv_one hc0 hcT] at hmono
  have hlog := half_log_le_log_div hc hT
  have hcoef : 0 ≤ Real.exp (-1) * 2 ^ (-A) * T ^ (-A) := by positivity
  have hscaled := mul_le_mul_of_nonneg_left hlog hcoef
  unfold localizedIntegral
  dsimp [d] at hmono
  calc
    (Real.exp (-1) * 2 ^ (-A) / 2) * (T ^ (-A) * Real.log T)
        = (Real.exp (-1) * 2 ^ (-A) * T ^ (-A)) *
            ((1 / 2 : Real) * Real.log T) := by ring
    _ ≤ (Real.exp (-1) * 2 ^ (-A) * T ^ (-A)) * Real.log (T / c) := hscaled
    _ ≤ _ := hmono

end O77.Residual.Model
end
```

Lemma 20.12 (Checked nonresonant and resonant box geometry)**.**

Corresponding Lean declaration(s): `O77.Residual.fourScale,O77.Residual.fourScale_pos,O77.Residual.four_mul_fourScale_le,O77.Residual.nonresonantBoxLower,O77.Residual.nonresonantBoxUpper`.

Corresponding Lean declaration(s): `O77.Residual.MemNonresonantBox,O77.Residual.nonresonantBox_endpoint_separation,O77.Residual.nonresonantBox_positive,O77.Residual.nonresonantBox_two_separated,O77.Residual.nonresonantBox_vandermonde_lower`.

Corresponding Lean declaration(s): `O77.Residual.resonantBoxLower,O77.Residual.resonantBoxUpper,O77.Residual.MemResonantBox,O77.Residual.resonantBox_endpoint_separation,O77.Residual.resonantBox_nonempty_at_resonance`.

Corresponding Lean declaration(s): `O77.Residual.resonantBox_positive,O77.Residual.resonantBox_two_separated,O77.Residual.resonantBox_vandermonde_lower,O77.Residual.chamberBoxThreshold,O77.Residual.chamberBoxThreshold_pos`.

Corresponding Lean declaration(s): `O77.Residual.chamberBoxThreshold_one_le,O77.Residual.fourScale_lt_chamberBoxThreshold,O77.Residual.nonresonantBox_lower_lt_upper,O77.Residual.resonantBox_lower_lt_upper,O77.Residual.nonresonantBox_nonempty`.

Corresponding Lean declaration(s): `O77.Residual.resonantBox_nonempty,O77.Residual.nonresonantBox_chamber_package,O77.Residual.resonantBox_chamber_package,O77.Residual.nonresonantBoxWidth,O77.Residual.nonresonantBoxVolumeFactor`.

Corresponding Lean declaration(s): `O77.Residual.resonantBoxWidth,O77.Residual.resonantBoxVolumeFactor,O77.Residual.nonresonantBoxWidth_pos,O77.Residual.resonantBoxWidth_pos,O77.Residual.nonresonantBoxVolumeFactor_pos`.

Corresponding Lean declaration(s): `O77.Residual.resonantBoxVolumeFactor_pos,O77.Residual.nonresonantBox_example,O77.Residual.resonantBox_example,O77.Residual.nonresonant_endpoint_separation_below_one_false,O77.Residual.resonantBox_below_scale_empty`.

Let $k \in \mathbb{N}$ and put $q_i = 4^{i+1}$ for $i : \operatorname{Fin} k$. For a cut index $c \in \mathbb{N}$, the nonresonant box uses

$$[q_i/T, 2q_i/T] \quad (i < c), \qquad [q_i, 2q_i] \quad (c \le i).$$

For a resonant index $r : \operatorname{Fin} k$, use the scaled intervals for $i < r$, $[q_r/T, 1]$ at r, and the fixed intervals for $r < i$. If $T \ge T_k := 16^{k+1}$, then every coordinate interval is strictly nonempty, each box has a point, and every point s of either box satisfies

$$s_i > 0, \qquad 2s_i \le s_j \quad (i < j), \qquad 2^{-\binom{k}{2}} \prod_j s_j^j \le \prod_{i<j} (s_j - s_i).$$

The product of the coordinate widths is strictly positive. For $k = 0$ the nonresonant box is the unique empty-coordinate box; no resonant index exists.

Proof. The powers satisfy $4q_i \le q_j$ whenever $i < j$. Within a scaled or fixed group this makes twice an earlier upper endpoint no larger than a later lower endpoint. At a scaled–fixed transition the same inequality follows from $T \ge 1$. At a resonant coordinate, the lower endpoint

is separated from all preceding scaled intervals, while the upper endpoint 1 is separated from every following fixed interval.

The uniform threshold T_k is positive and dominates every q_i. It therefore makes all interval widths positive, including $1-q_r/T$ in the resonant coordinate. Midpoints give explicit witnesses. The positivity and two-separation just proved allow direct application of Lemma 20.8. Finite products of the positive widths are positive. This proves geometry and algebra of the lower boxes but not the equality between Lebesgue measure of a product box and its width product. □

```
noncomputable section
open Real MeasureTheory Set
open scoped ENNReal BigOperators
namespace O77.Residual
def fourScale {k : Nat} (i : Fin k) : Real := (4 : Real) ^ (i.val + 1)

lemma fourScale_pos {k : Nat} (i : Fin k) : 0 < fourScale i := by
  exact pow_pos (by norm_num) _

lemma four_mul_fourScale_le {k : Nat} {i j : Fin k} (hij : i < j) :
    4 * fourScale i ≤ fourScale j := by
  have hexp : i.val + 2 ≤ j.val + 1 := by omega
  calc
    4 * fourScale i = (4 : Real) ^ (i.val + 2) := by
      simp [fourScale, pow_succ, mul_comm]
    _ ≤ (4 : Real) ^ (j.val + 1) := ₀pow_le_pow_right (by norm_num) hexp
    _ = fourScale j := rfl

/-- A nonresonant box has a scaled initial segment and a fixed final segment. -/
def nonresonantBoxLower {k : Nat} (cut : Nat) (T : Real) (i : Fin k) : Real :=
  if i.val < cut then fourScale i / T else fourScale i

def nonresonantBoxUpper {k : Nat} (cut : Nat) (T : Real) (i : Fin k) : Real :=
  if i.val < cut then 2 * fourScale i / T else 2 * fourScale i

def MemNonresonantBox {k : Nat} (cut : Nat) (T : Real) (s : Fin k -> Real) : Prop :=
  ∀ i, nonresonantBoxLower cut T i ≤ s i ∧
    s i ≤ nonresonantBoxUpper cut T i

lemma nonresonantBox_endpoint_separation {k cut : Nat} {T : Real}
    (hT : 1 ≤ T) {i j : Fin k} (hij : i < j) :
    2 * nonresonantBoxUpper cut T i ≤ nonresonantBoxLower cut T j := by
  have hTpos : 0 < T := lt_of_lt_of_le (by norm_num) hT
  have hpow := four_mul_fourScale_le hij
  have hjpos : 0 ≤ fourScale j := (fourScale_pos j).le
  by_cases hi : i.val < cut
  · by_cases hj : j.val < cut
    · simp [nonresonantBoxUpper, nonresonantBoxLower, hi, hj]
      rw [show 2 * (2 * fourScale i / T) = 4 * fourScale i / T by ring]
      exact div_le_div_of_nonneg_right hpow hTpos.le
    · simp [nonresonantBoxUpper, nonresonantBoxLower, hi, hj]
      rw [show 2 * (2 * fourScale i / T) = 4 * fourScale i / T by ring]
      apply (₀div_le_iff hTpos).2
      calc
        4 * fourScale i ≤ fourScale j := hpow
        _ ≤ fourScale j * T := by nlinarith
  · have hj : ¬ j.val < cut := by
      intro hjcut
      have : i.val < cut := lt_of_lt_of_le hij hjcut.le
      exact hi this
    simp [nonresonantBoxUpper, nonresonantBoxLower, hi, hj]
    rw [show 2 * (2 * fourScale i) = 4 * fourScale i by ring]
```

```
    exact hpow

lemma nonresonantBox_positive {k cut : Nat} {T : Real}
    (hT : 1 ≤ T) {s : Fin k -> Real} (hs : MemNonresonantBox cut T s) :
    ∀ i, 0 < s i := by
  intro i
  have hTpos : 0 < T := lt_of_lt_of_le (by norm_num) hT
  have hlo : 0 < nonresonantBoxLower cut T i := by
    unfold nonresonantBoxLower
    split
    · exact div_pos (fourScale_pos i) hTpos
    · exact fourScale_pos i
  exact hlo.trans_le (hs i).1

/-- Every point of a nonresonant box is two-separated. -/
theorem nonresonantBox_two_separated {k cut : Nat} {T : Real}
    (hT : 1 ≤ T) {s : Fin k -> Real} (hs : MemNonresonantBox cut T s) :
    ∀ {i j : Fin k}, i < j -> 2 * s i ≤ s j := by
  intro i j hij
  calc
    2 * s i ≤ 2 * nonresonantBoxUpper cut T i := by
      exact mul_le_mul_of_nonneg_left (hs i).2 (by norm_num)
    _ ≤ nonresonantBoxLower cut T j := nonresonantBox_endpoint_separation hT hij
    _ ≤ s j := (hs j).1

/-- The full separated Vandermonde lower bound on a nonresonant box. -/
theorem nonresonantBox_vandermonde_lower {k cut : Nat} {T : Real}
    (hT : 1 ≤ T) {s : Fin k -> Real} (hs : MemNonresonantBox cut T s) :
    (1 / 2 : Real) ^ Nat.choose k 2 * chamberMonomialFin s ≤ vandermondeFin s := by
  exact half_pow_choose_mul_chamberMonomialFin_le_vandermondeFin s
    (fun i => (nonresonantBox_positive hT hs i).le)
    (nonresonantBox_two_separated hT hs)

/-- A resonant box has scaled coordinates before `r`, the interval
`[4^(r+1)/T, 1]` at `r`, and fixed coordinates after `r`. -/
def resonantBoxLower {k : Nat} (r : Fin k) (T : Real) (i : Fin k) : Real :=
  if i < r then fourScale i / T else if i = r then fourScale i / T else fourScale i

def resonantBoxUpper {k : Nat} (r : Fin k) (T : Real) (i : Fin k) : Real :=
  if i < r then 2 * fourScale i / T else if i = r then 1 else 2 * fourScale i

def MemResonantBox {k : Nat} (r : Fin k) (T : Real) (s : Fin k -> Real) : Prop :=
  ∀ i, resonantBoxLower r T i ≤ s i ∧ s i ≤ resonantBoxUpper r T i

lemma resonantBox_endpoint_separation {k : Nat} {r : Fin k} {T : Real}
    (hT : 1 ≤ T) {i j : Fin k} (hij : i < j) :
    2 * resonantBoxUpper r T i ≤ resonantBoxLower r T j := by
  have hTpos : 0 < T := lt_of_lt_of_le (by norm_num) hT
  have hpow := four_mul_fourScale_le hij
  have hjpos : 0 ≤ fourScale j := (fourScale_pos j).le
  by_cases hir : i < r
  · by_cases hjr : j < r
    · simp [resonantBoxUpper, resonantBoxLower, hir, hjr]
      rw [show 2 * (2 * fourScale i / T) = 4 * fourScale i / T by ring]
      exact div_le_div_of_nonneg_right hpow hTpos.le
    · by_cases hjEq : j = r
      · subst j
        simp [resonantBoxUpper, resonantBoxLower, hir]
        rw [show 2 * (2 * fourScale i / T) = 4 * fourScale i / T by ring]
        exact div_le_div_of_nonneg_right hpow hTpos.le
      · simp [resonantBoxUpper, resonantBoxLower, hir, hjr, hjEq]
        rw [show 2 * (2 * fourScale i / T) = 4 * fourScale i / T by ring]
```

```
        apply (₀div_le_iff hTpos).2
        calc
          4 * fourScale i ≤ fourScale j := hpow
          _ ≤ fourScale j * T := by nlinarith
  · by_cases hiEq : i = r
    · subst i
      have hjAfter : ¬ j < r := not_lt_of_ge hij.le
      have hjNe : j ≠ r := ne_of_gt hij
      simp [resonantBoxUpper, resonantBoxLower, hjAfter, hjNe]
      have hjpow : (2 : Real) ≤ fourScale j := by
        unfold fourScale
        have h4 : (4 : Real) ≤ 4 ^ (j.val + 1) := by
          have hpow' := ₀pow_le_pow_right (show (1 : Real) ≤ 4 by norm_num)
            (show 1 ≤ j.val + 1 by omega)
          simpa using hpow'
        linarith
      simpa using hjpow
    · have hiAfter : r < i := lt_of_le_of_ne (not_lt.mp hir) (Ne.symm hiEq)
      have hjAfter : ¬ j < r := not_lt_of_ge (le_trans hiAfter.le hij.le)
      have hjNe : j ≠ r := ne_of_gt (lt_of_lt_of_le hiAfter hij.le)
      simp [resonantBoxUpper, resonantBoxLower, hir, hiEq, hjAfter, hjNe]
      rw [show 2 * (2 * fourScale i) = 4 * fourScale i by ring]
      exact hpow

lemma resonantBox_nonempty_at_resonance {k : Nat} {r : Fin k} {T : Real}
    (hT : fourScale r ≤ T) : resonantBoxLower r T r ≤ resonantBoxUpper r T r := by
  have hTpos : 0 < T := (fourScale_pos r).trans_le hT
  simp [resonantBoxLower, resonantBoxUpper]
  exact (₀div_le_iff hTpos).2 (by simpa using hT)

lemma resonantBox_positive {k : Nat} {r : Fin k} {T : Real}
    (hT : 1 ≤ T) {s : Fin k -> Real} (hs : MemResonantBox r T s) :
    ∀ i, 0 < s i := by
  intro i
  have hTpos : 0 < T := lt_of_lt_of_le (by norm_num) hT
  have hlo : 0 < resonantBoxLower r T i := by
    unfold resonantBoxLower
    split
    · exact div_pos (fourScale_pos i) hTpos
    · split
      · exact div_pos (fourScale_pos i) hTpos
      · exact fourScale_pos i
  exact hlo.trans_le (hs i).1

/-- Every point of a resonant box is two-separated. -/
theorem resonantBox_two_separated {k : Nat} {r : Fin k} {T : Real}
    (hT : 1 ≤ T) {s : Fin k -> Real} (hs : MemResonantBox r T s) :
    ∀ {i j : Fin k}, i < j -> 2 * s i ≤ s j := by
  intro i j hij
  calc
    2 * s i ≤ 2 * resonantBoxUpper r T i := by
      exact mul_le_mul_of_nonneg_left (hs i).2 (by norm_num)
    _ ≤ resonantBoxLower r T j := resonantBox_endpoint_separation hT hij
    _ ≤ s j := (hs j).1

/-- The full separated Vandermonde lower bound on a resonant box. -/
theorem resonantBox_vandermonde_lower {k : Nat} {r : Fin k} {T : Real}
    (hT : 1 ≤ T) {s : Fin k -> Real} (hs : MemResonantBox r T s) :
    (1 / 2 : Real) ^ Nat.choose k 2 * chamberMonomialFin s ≤ vandermondeFin s := by
  exact half_pow_choose_mul_chamberMonomialFin_le_vandermondeFin s
    (fun i => (resonantBox_positive hT hs i).le)
    (resonantBox_two_separated hT hs)
```

```
/-- The uniform large-parameter threshold used by every separated box. -/
def chamberBoxThreshold (k : Nat) : Real := (16 : Real) ^ (k + 1)

lemma chamberBoxThreshold_pos (k : Nat) : 0 < chamberBoxThreshold k := by
  exact pow_pos (by norm_num) _

lemma chamberBoxThreshold_one_le (k : Nat) : 1 ≤ chamberBoxThreshold k := by
  exact ₀one_le_pow (by norm_num)

lemma fourScale_lt_chamberBoxThreshold {k : Nat} (i : Fin k) :
    fourScale i < chamberBoxThreshold k := by
  have hexp : i.val + 1 < 2 * (k + 1) := by omega
  calc
    fourScale i = (4 : Real) ^ (i.val + 1) := rfl
    _ < (4 : Real) ^ (2 * (k + 1)) := ₀pow_lt_pow_right (by norm_num) hexp
    _ = (16 : Real) ^ (k + 1) := by rw [pow_mul]; norm_num
    _ = chamberBoxThreshold k := rfl

lemma nonresonantBox_lower_lt_upper {k cut : Nat} {T : Real}
    (hT : chamberBoxThreshold k ≤ T) (i : Fin k) :
    nonresonantBoxLower cut T i < nonresonantBoxUpper cut T i := by
  have hTpos : 0 < T := (chamberBoxThreshold_pos k).trans_le hT
  by_cases hi : i.val < cut
  · simp [nonresonantBoxLower, nonresonantBoxUpper, hi]
    have hq : 0 < fourScale i / T := div_pos (fourScale_pos i) hTpos
    rw [show 2 * fourScale i / T = 2 * (fourScale i / T) by ring]
    linarith
  · simp [nonresonantBoxLower, nonresonantBoxUpper, hi]
    linarith [fourScale_pos i]

lemma resonantBox_lower_lt_upper {k : Nat} {r : Fin k} {T : Real}
    (hT : chamberBoxThreshold k ≤ T) (i : Fin k) :
    resonantBoxLower r T i < resonantBoxUpper r T i := by
  have hTpos : 0 < T := (chamberBoxThreshold_pos k).trans_le hT
  by_cases hir : i < r
  · simp [resonantBoxLower, resonantBoxUpper, hir]
    have hq : 0 < fourScale i / T := div_pos (fourScale_pos i) hTpos
    rw [show 2 * fourScale i / T = 2 * (fourScale i / T) by ring]
    linarith
  · by_cases hiEq : i = r
    · subst i
      simp [resonantBoxLower, resonantBoxUpper]
      apply (div_lt_one hTpos).2
      exact (fourScale_lt_chamberBoxThreshold r).trans_le hT
    · simp [resonantBoxLower, resonantBoxUpper, hir, hiEq]
      linarith [fourScale_pos i]

/-- The coordinatewise midpoint gives a point of every nonresonant box. -/
theorem nonresonantBox_nonempty {k cut : Nat} {T : Real}
    (hT : chamberBoxThreshold k ≤ T) :
    ∃ s : Fin k -> Real, MemNonresonantBox cut T s := by
  let s : Fin k -> Real := fun i =>
    (nonresonantBoxLower cut T i + nonresonantBoxUpper cut T i) / 2
  refine ⟨s, ?⟩
  intro i
  have hle := (nonresonantBox_lower_lt_upper (cut := cut) hT i).le
  constructor <;> dsimp [s] <;> linarith

/-- The coordinatewise midpoint gives a point of every resonant box. -/
```

```
theorem resonantBox_nonempty {k : Nat} {r : Fin k} {T : Real}
    (hT : chamberBoxThreshold k ≤ T) :
    ∃ s : Fin k -> Real, MemResonantBox r T s := by
  let s : Fin k -> Real := fun i =>
    (resonantBoxLower r T i + resonantBoxUpper r T i) / 2
  refine ⟨s, ?⟩_
  intro i
  have hle := (resonantBox_lower_lt_upper (r := r) hT i).le
  constructor <;> dsimp [s] <;> linarith

/-- At the uniform threshold, both kinds of boxes lie in the positive strict
ordered chamber and carry the separated Vandermonde lower bound. -/
theorem nonresonantBox_chamber_package {k cut : Nat} {T : Real}
    (hT : chamberBoxThreshold k ≤ T) {s : Fin k -> Real}
    (hs : MemNonresonantBox cut T s) :
    ∀( i, 0 < s i) ∧
    ∀( {i j : Fin k}, i < j -> 2 * s i ≤ s j) ∧
    ((1 / 2 : Real) ^ Nat.choose k 2 * chamberMonomialFin s ≤ vandermondeFin s) := by
  have hTone : 1 ≤ T := (chamberBoxThreshold_one_le k).trans hT
  exact ⟨nonresonantBox_positive hTone hs,
    nonresonantBox_two_separated hTone hs,
    nonresonantBox_vandermonde_lower hTone ⟩hs

theorem resonantBox_chamber_package {k : Nat} {r : Fin k} {T : Real}
    (hT : chamberBoxThreshold k ≤ T) {s : Fin k -> Real}
    (hs : MemResonantBox r T s) :
    ∀( i, 0 < s i) ∧
    ∀( {i j : Fin k}, i < j -> 2 * s i ≤ s j) ∧
    ((1 / 2 : Real) ^ Nat.choose k 2 * chamberMonomialFin s ≤ vandermondeFin s) := by
  have hTone : 1 ≤ T := (chamberBoxThreshold_one_le k).trans hT
  exact ⟨resonantBox_positive hTone hs,
    resonantBox_two_separated hTone hs,
    resonantBox_vandermonde_lower hTone ⟩hs


/-- Coordinate length of a nonresonant separated interval. -/
def nonresonantBoxWidth {k : Nat} (cut : Nat) (T : Real) (i : Fin k) : Real :=
  nonresonantBoxUpper cut T i - nonresonantBoxLower cut T i

/-- Product of the coordinate lengths of a nonresonant separated box. -/
def nonresonantBoxVolumeFactor {k : Nat} (cut : Nat) (T : Real) : Real :=
  ∏ i : Fin k, nonresonantBoxWidth cut T i

/-- Coordinate length of a resonant separated interval. -/
def resonantBoxWidth {k : Nat} (r : Fin k) (T : Real) (i : Fin k) : Real :=
  resonantBoxUpper r T i - resonantBoxLower r T i

/-- Product of the coordinate lengths of a resonant separated box. -/
def resonantBoxVolumeFactor {k : Nat} (r : Fin k) (T : Real) : Real :=
  ∏ i : Fin k, resonantBoxWidth r T i

lemma nonresonantBoxWidth_pos {k cut : Nat} {T : Real}
    (hT : chamberBoxThreshold k ≤ T) (i : Fin k) :
    0 < nonresonantBoxWidth cut T i := by
  unfold nonresonantBoxWidth
  exact sub_pos.mpr (nonresonantBox_lower_lt_upper (cut := cut) hT i)

lemma resonantBoxWidth_pos {k : Nat} {r : Fin k} {T : Real}
    (hT : chamberBoxThreshold k ≤ T) (i : Fin k) :
    0 < resonantBoxWidth r T i := by
  unfold resonantBoxWidth
  exact sub_pos.mpr (resonantBox_lower_lt_upper (r := r) hT i)
```

```
/-- The finite product of coordinate lengths of every nonresonant box is positive. -/
theorem nonresonantBoxVolumeFactor_pos {k cut : Nat} {T : Real}
    (hT : chamberBoxThreshold k ≤ T) :
    0 < nonresonantBoxVolumeFactor (k := k) cut T := by
  unfold nonresonantBoxVolumeFactor
  exact Finset.prod_pos fun i _ => nonresonantBoxWidth_pos (cut := cut) hT i

/-- The finite product of coordinate lengths of every resonant box is positive. -/
theorem resonantBoxVolumeFactor_pos {k : Nat} {r : Fin k} {T : Real}
    (hT : chamberBoxThreshold k ≤ T) :
    0 < resonantBoxVolumeFactor r T := by
  unfold resonantBoxVolumeFactor
  exact Finset.prod_pos fun i _ => resonantBoxWidth_pos (r := r) hT i


-- Small exact regressions for orientation, pair count, and empty dimensions.
example : vandermondeFin (k := 0) (fun i : Fin 0 => nomatch i) = 1 := by
  rfl

example : chamberMonomialFin (k := 0) (fun i : Fin 0 => nomatch i) = 1 := by
  simp [chamberMonomialFin]

example : vandermondeFin ![(1 : Real), 2, 4] = 6 := by
  norm_num [vandermondeFin, vandermondeList]

example : chamberMonomialFin ![(1 : Real), 2, 4] = 32 := by
  norm_num [chamberMonomialFin, Fin.prod_univ_succ]

example : (1 / 2 : Real) ^ Nat.choose 3 2 *
    chamberMonomialFin ![(1 : Real), 2, 4] = 4 := by
  norm_num [chamberMonomialFin, Fin.prod_univ_succ]

example : MemNonresonantBox (k := 3) 2 4096
    ![(1 / 4096 : Real) * 6, (1 / 4096 : Real) * 24, 96] := by
  intro i
  fin_cases i <;> norm_num [MemNonresonantBox, nonresonantBoxLower,
    nonresonantBoxUpper, fourScale]

example : MemResonantBox (k := 3) (1 : Fin 3) 4096
    ![(1 / 4096 : Real) * 6, (1 / 2 : Real), 96] := by
  intro i
  fin_cases i <;> norm_num [MemResonantBox, resonantBoxLower,
    resonantBoxUpper, fourScale]


/-- A nontrivial exact Vandermonde regression. -/
theorem vandermondeList_example :
    vandermondeList [1, 2, 4] = 6 := by
  norm_num [vandermondeList]

/-- The matching chamber monomial is `1^0 * 2^1 * 4^2 = 32`. -/
theorem chamberMonomialList_example :
    chamberMonomialList [1, 2, 4] = 32 := by
  norm_num [chamberMonomialList]

/-- The exact separated lower bound is nontrivial on `(1,2,4)`. -/
theorem separated_vandermonde_example :
    separatedConstantList [1, 2, 4] * chamberMonomialList [1, 2, 4]
      ≤ vandermondeList [1, 2, 4] := by
  norm_num [separatedConstantList, chamberMonomialList, vandermondeList]
```

```
/-- A concrete point in a three-coordinate nonresonant box. -/
theorem nonresonantBox_example :
    MemNonresonantBox 2 64 ![(1 / 16 : Real), 1 / 4, 64] := by
  intro i
  fin_cases i <;>
    norm_num [nonresonantBoxLower, nonresonantBoxUpper, fourScale]

/-- A concrete point in a three-coordinate resonant box. -/
theorem resonantBox_example :
    MemResonantBox (1 : Fin 3) 256 ![(1 / 64 : Real), 1 / 16, 64] := by
  intro i
  fin_cases i <;>
    norm_num [resonantBoxLower, resonantBoxUpper, fourScale]

/-- Nonnegativity is genuinely needed in the Vandermonde upper estimate. -/
theorem vandermonde_upper_without_nonnegativity_false :
    ¬ vandermondeList [-2, -1] ≤ chamberMonomialList [-2, -1] := by
  norm_num [vandermondeList, chamberMonomialList]

/-- Two-separation is genuinely needed in the stated Vandermonde lower estimate. -/
theorem vandermonde_lower_without_separation_false :
    ¬ separatedConstantList [1, 1] * chamberMonomialList [1, 1]
        ≤ vandermondeList [1, 1] := by
  norm_num [separatedConstantList, chamberMonomialList, vandermondeList]

/-- The hypothesis `1 ≤ T` in the box-separation lemma cannot be deleted. -/
theorem nonresonant_endpoint_separation_below_one_false :
    ¬ 2 * nonresonantBoxUpper (k := 2) 1 (1 / 2) (0 : Fin 2)
        ≤ nonresonantBoxLower (k := 2) 1 (1 / 2) (1 : Fin 2) := by
  norm_num [nonresonantBoxLower, nonresonantBoxUpper, fourScale]

/-- A resonant interval can be empty when `T` lies below its scale. -/
theorem resonantBox_below_scale_empty :
    ¬ ∃ s : Fin 1 -> Real, MemResonantBox (0 : Fin 1) 1 s := by
  rintro ⟨s, ⟩hs
  have h := hs (0 : Fin 1)
  norm_num [resonantBoxLower, resonantBoxUpper, fourScale] at h
  linarith [h.1, h.2]

end O77.Residual
end
```

Lemma 20.13 (Integrated separated-box and orthant estimates)**.**

Corresponding Lean declaration(s): `O77.Residual.spectralA,O77.Residual.spectralA_pos,O77.Residual.orderedPositiveChamber,O77.Residual.mem_orderedPositiveChamber_iff,O77.Residual.measurableSet_orderedPositiveChamber`.

Corresponding Lean declaration(s): `O77.Residual.laguerreChamberIntegral,O77.Residual.positiveModelFactor,O77.Residual.positiveModelProduct,O77.Residual.positiveModelFactor_nonneg,O77.Residual.positiveModelProduct_nonneg`.

Corresponding Lean declaration(s): `O77.Residual.positiveModelFactor_integrable,O77.Residual.positiveModelFactor_integral,O77.Residual.positiveModelProduct_integrable,O77.Residual.positiveModelProduct_integral,O77.Residual.positiveModelProduct_eq_model`.

Corresponding Lean declaration(s): `O77.Residual.continuous_vandermondeFin,O77.Residual.measurable_vandermondeFin,O77.Residual.measurable_rpow_const,O77.Residual.measurable_laguerreChamberIntegrand,O77.Resi`

dual.chamberIndicator.

Corresponding Lean declaration(s): O77.Residual.chamberIndicator_nonneg,O77.Residual.chamberIndicator_le_positiveModelProduct,O77.Residual.chamberIndicator_integrable,O77.Residual.laguerreChamberIntegral_eq_integral_indicator,O77.Residual.laguerreChamberIntegral_le_modelProduct.

Corresponding Lean declaration(s): O77.Residual.coordinateBox,O77.Residual.mem_coordinateBox_iff,O77.Residual.measurableSet_coordinateBox,O77.Residual.laguerreModelProduct_eq_coordinateProduct,O77.Residual.coordinateBox_model_integral.

Corresponding Lean declaration(s): O77.Residual.nonresonantBoxSet,O77.Residual.resonantBoxSet,O77.Residual.measurableSet_nonresonantBoxSet,O77.Residual.measurableSet_resonantBoxSet,O77.Residual.memNonresonantBox_of_mem_set.

Corresponding Lean declaration(s): O77.Residual.memResonantBox_of_mem_set,O77.Residual.nonresonantBoxSet_subset_chamber,O77.Residual.resonantBoxSet_subset_chamber,O77.Residual.laguerreChamberIntegrand_integrableOn_chamber,O77.Residual.laguerreChamberIntegrand_nonnegOn_chamber.

Corresponding Lean declaration(s): O77.Residual.nonresonantBox_model_integral_le_chamber,O77.Residual.resonantBox_model_integral_le_chamber,O77.Residual.chamberGaugeProduct,O77.Residual.one_le_fourScale,O77.Residual.two_mul_fourScale_le_chamberBoxThreshold.

Corresponding Lean declaration(s): O77.Residual.fourScale_sq_le_chamberBoxThreshold,O77.Residual.modelGauge_eq_left,O77.Residual.modelGauge_eq_right,O77.Residual.modelGauge_eq_resonant,O77.Residual.nonresonantCoordinateConstant.

Corresponding Lean declaration(s): O77.Residual.nonresonantCoordinateConstant_pos,O77.Residual.nonresonantBoxConstant,O77.Residual.nonresonantBoxConstant_pos,O77.Residual.nonresonant_localized_factor_lower,O77.Residual.nonresonant_product_localized_lower.

Corresponding Lean declaration(s): O77.Residual.nonresonantBox_chamberGauge_lower,O77.Residual.resonantCoordinateConstant,O77.Residual.resonantCoordinateConstant_pos,O77.Residual.resonantBoxConstant,O77.Residual.resonantBoxConstant_pos.

Corresponding Lean declaration(s): O77.Residual.resonant_localized_factor_lower,O77.Residual.resonant_product_localized_lower,O77.Residual.resonantBox_chamberGauge_lower.

For zero-based $i : \operatorname{Fin} k$, put $A_i = \eta + i.\mathrm{val} + 1$; in one-based notation this is $A_j = \eta + j$. Assume $\eta > -1$ and $\rho > 0$. Let Δ_k be the positive weakly ordered chamber and let $\mu_k = \bigotimes_{i:\operatorname{Fin} k} \mathrm{vol}$. Then:

(i) the full chamber integrand is measurable and integrable on Δ_k;

(ii) its chamber integral is bounded above by the product of the k one-dimensional model integrals;

(iii) for the unique strict-cut or resonant sign pattern of the $A_i - \rho$, the corresponding half-open separated box is measurable, contained in Δ_k, and its model-product integral is exactly the product of the localized coordinate integrals;

(iv) restricting the chamber integral to that box gives a positive constant times

$$T^{-\sum_i \min(A_i,\rho)}(\log T)^{\#\{i:A_i=\rho\}}.$$

All constants are independent of T once $T \geq 16^{k+1}$. These assertions include $k = 0$.

Proof. Extend each model factor by zero outside $(0,\infty)$. Its integrability is the one-dimensional result. The finite product is integrable for the explicit product measure, and its integral is the product of the coordinate integrals. The ordered chamber is a measurable finite intersection of positivity and coordinate-order inequalities. The Vandermonde is continuous, every real-power factor is measurable, and the chamber integrand is nonnegative. The pointwise upper estimate of Lemma 20.9 therefore yields (i) and (ii) by integral monotonicity.

For lower bounds, define the rectangle by the formal half-open intervals $(\ell_i(T), u_i(T)]$. Mathlib's restriction-of-product-measures identity changes the restricted product measure into the product of the restricted one-dimensional Lebesgue measures. The finite-product integral theorem then proves exactly

$$\int_{\prod_i(\ell_i,u_i]} \prod_i f_i(s_i)\, d\mu_k = \prod_i \int_{(\ell_i,u_i]} f_i(s_i)\, ds_i.$$

The box geometry gives positivity, chamber containment and the pointwise lower factor $2^{-\binom{k}{2}}$. Restricting a nonnegative integrable function to a measurable subset and using the pointwise lower comparison proves (iii).

The sequence A_i is strictly increasing. If no equality $A_i = \rho$ occurs, the indices with $A_i < \rho$ form an initial segment and those with $\rho < A_i$ form the complementary final segment. If equality occurs, it occurs at one unique index. Apply the three localized bounds of Lemma 20.11 coordinatewise and multiply their positive constants. Scaled coordinates contribute T^{-A_i}, fixed coordinates contribute $T^{-\rho}$, and the resonant coordinate contributes $T^{-\rho}\log T$. This is (iv). For $k = 0$, every product is empty, the unique rectangle and chamber are the singleton function space, and the same identities reduce to $1 = 1$.

□

```
noncomputable section
open Real MeasureTheory Set
open scoped ENNReal BigOperators

namespace O77.Residual

/-- The one-based Laguerre exponent attached to a zero-based spectral coordinate. -/
def spectralA {k : Nat} (eta : Real) (i : Fin k) : Real := eta + i.val + 1

lemma spectralA_pos {k : Nat} {eta : Real} (heta : -1 < eta) (i : Fin k) :
    0 < spectralA eta i := by
  unfold spectralA
  have hi : 0 ≤ (i.val : Real) := by positivity
  linarith

/-- The open positive ordered chamber used by the real Laguerre integral. -/
def orderedPositiveChamber (k : Nat) : Set (Fin k -> Real) :=
  (Set.univ.pi fun _ : Fin k => Ioi (0 : Real)) ∩
    ⋂ i : Fin k, ⋂ j : Fin k, {s | i < j -> s i ≤ s j}

lemma mem_orderedPositiveChamber_iff {k : Nat} {s : Fin k -> Real} :
    s ∈ orderedPositiveChamber k ↔
      ∀( i, 0 < s i) ∧ ∀( {i j : Fin k}, i < j -> s i ≤ s j) := by
  simp [orderedPositiveChamber]
```

```
lemma measurableSet_orderedPositiveChamber (k : Nat) :
    MeasurableSet (orderedPositiveChamber k) := by
  unfold orderedPositiveChamber
  apply MeasurableSet.inter
  · exact MeasurableSet.univ_pi (fun _ => measurableSet_Ioi)
  · apply MeasurableSet.iInter
    intro i
    apply MeasurableSet.iInter
    intro j
    by_cases hij : i < j
    · simpa [hij] using measurableSet_le (measurable_pi_apply i) (measurable_pi_apply j)
    · simp [hij]

/-- The actual ordered-chamber integral, with the explicit product Lebesgue measure. -/
def laguerreChamberIntegral {k : Nat} (eta rho T : Real) : Real :=
  ∫ s : Fin k -> Real in orderedPositiveChamber k,
    laguerreChamberIntegrand eta rho T s ∂(Measure.pi fun _ : Fin k => volume)

/-- A one-dimensional model factor extended by zero outside the positive half-line. -/
def positiveModelFactor (A rho T : Real) : Real -> Real :=
  (Ioi (0 : Real)).indicator (Model.integrand A rho T)

/-- Product of the positive one-dimensional model factors. -/
def positiveModelProduct {k : Nat} (eta rho T : Real) (s : Fin k -> Real) : Real :=
  ∏ i, positiveModelFactor (spectralA eta i) rho T (s i)

lemma positiveModelFactor_nonneg {A rho T s : Real} (hT : 0 ≤ T) :
    0 ≤ positiveModelFactor A rho T s := by
  by_cases hs : s ∈ Ioi (0 : Real)
  · simp only [positiveModelFactor, indicator_of_mem hs]
    exact Model.integrand_nonneg hs.le hT
  · simp [positiveModelFactor, hs]

lemma positiveModelProduct_nonneg {k : Nat} (eta rho T : Real)
    (hT : 0 ≤ T) (s : Fin k -> Real) :
    0 ≤ positiveModelProduct eta rho T s := by
  unfold positiveModelProduct
  exact Finset.prod_nonneg fun i _ => positiveModelFactor_nonneg hT

lemma positiveModelFactor_integrable {A rho T : Real}
    (hA : 0 < A) (hrho : 0 ≤ rho) (hT : 0 ≤ T) :
    Integrable (positiveModelFactor A rho T) volume := by
  exact (Model.integrableOn_integrand_Ioi hA hT hrho).integrable_indicator measurableSet_Ioi

lemma positiveModelFactor_integral {A rho T : Real} :
    ∫ s : Real, positiveModelFactor A rho T s = Model.integral A rho T := by
  rw [positiveModelFactor, MeasureTheory.integral_indicator measurableSet_Ioi]
  rfl

lemma positiveModelProduct_integrable {k : Nat} {eta rho T : Real}
    (heta : -1 < eta) (hrho : 0 ≤ rho) (hT : 0 ≤ T) :
    Integrable (positiveModelProduct (k := k) eta rho T)
      (Measure.pi fun _ : Fin k => volume) := by
  unfold positiveModelProduct
  apply MeasureTheory.Integrable.fin_nat_prod
  intro i
  exact positiveModelFactor_integrable (spectralA_pos heta i) hrho hT

lemma positiveModelProduct_integral {k : Nat} (eta rho T : Real) :
    ∫ s : Fin k -> Real, positiveModelProduct eta rho T s
        ∂(Measure.pi fun _ : Fin k => volume)) =
      ∏ i : Fin k, Model.integral (spectralA eta i) rho T := by
```

```
  unfold positiveModelProduct
  rw [MeasureTheory.integral_fintype_prod_eq_prod]
  apply Finset.prod_congr rfl
  intro i hi
  exact positiveModelFactor_integral

lemma positiveModelProduct_eq_model {k : Nat} {eta rho T : Real}
    {s : Fin k -> Real} (hpos : ∀ i, 0 < s i) :
    positiveModelProduct eta rho T s = laguerreModelProduct eta rho T s := by
  unfold positiveModelProduct laguerreModelProduct spectralA
  apply Finset.prod_congr rfl
  intro i hi
  simp [positiveModelFactor, hpos i, Model.integrand, mul_comm, mul_left_comm]

lemma continuous_vandermondeFin (k : Nat) :
    Continuous (fun s : Fin k -> Real => vandermondeFin s) := by
  induction k with
  | zero =>
      change Continuous (fun _s : Fin 0 -> Real => (1 : Real))
      exact continuous_const
  | succ k ih =>
      have htailMap : Continuous
          (fun s : Fin (k + 1) -> Real => fun i : Fin k => s i.succ) := by
        exact continuous_pi (fun i => continuous_apply i.succ)
      have htail : Continuous
          (fun s : Fin (k + 1) -> Real => vandermondeFin (fun i : Fin k => s i.succ)) :=
        ih.comp htailMap
      have hfac : Continuous
          (fun s : Fin (k + 1) -> Real => ∏ i : Fin k, (s i.succ - s 0)) := by
        apply continuous_finsetProd
        intro i hi
        exact
          (continuous_apply i.succ :
            Continuous (fun s : Fin (k + 1) -> Real => s i.succ)).sub
          (continuous_apply (0 : Fin (k + 1)) :
            Continuous (fun s : Fin (k + 1) -> Real => s 0))
      change Continuous
        (fun s : Fin (k + 1) -> Real => vandermondeList (List.ofFn s))
      have heq : (fun s : Fin (k + 1) -> Real => vandermondeList (List.ofFn s)) =
          (fun s => vandermondeFin (fun i : Fin k => s i.succ) *
            ∏ i : Fin k, (s i.succ - s 0)) := by
        funext s
        rw [List.ofFn_succ]
        simp only [vandermondeList]
        rw [List.map_ofFn, List.prod_ofFn]
        rfl
      rw [heq]
      exact htail.mul hfac

lemma measurable_vandermondeFin (k : Nat) :
    Measurable (fun s : Fin k -> Real => vandermondeFin s) :=
  (continuous_vandermondeFin k).measurable

lemma measurable_rpow_const (a : Real) :
    Measurable (fun x : Real => x ^ a) := by
  refine measurable_of_continuousOn_compl_singleton (0 : Real) ?_
  exact continuousOn_id.rpow_const (fun x hx => Or.inl hx)

lemma measurable_laguerreChamberIntegrand {k : Nat} (eta rho T : Real) :
    Measurable (laguerreChamberIntegrand (k := k) eta rho T) := by
  unfold laguerreChamberIntegrand laguerreChamberCore commonRpowProduct
    laguerreWeightProduct
```

```
  apply Measurable.mul
  · apply Measurable.mul
    · apply Finset.measurable_prod
      intro i hi
      exact (measurable_rpow_const eta).comp (measurable_pi_apply i)
    · exact measurable_vandermondeFin k
  · apply Finset.measurable_prod
    intro i hi
    apply Measurable.mul
    · exact Real.measurable_exp.comp (measurable_pi_apply i).neg
    · exact (measurable_rpow_const (-rho)).comp
        (measurable_const.add (measurable_const.mul (measurable_pi_apply i)))

/-- The chamber integrand, extended by zero to the whole coordinate space. -/
def chamberIndicator {k : Nat} (eta rho T : Real) : (Fin k -> Real) -> Real :=
  (orderedPositiveChamber k).indicator (laguerreChamberIntegrand eta rho T)

lemma chamberIndicator_nonneg {k : Nat} (eta rho T : Real) (hT : 0 ≤ T)
    (s : Fin k -> Real) : 0 ≤ chamberIndicator eta rho T s := by
  by_cases hs : s ∈ orderedPositiveChamber k
  · rw [chamberIndicator, indicator_of_mem hs]
    have hmem := mem_orderedPositiveChamber_iff.mp hs
    exact (laguerreChamberIntegrand_nonneg_le_model eta rho T s hT hmem.1 hmem.2).1
  · simp [chamberIndicator, hs]

lemma chamberIndicator_le_positiveModelProduct {k : Nat}
    (eta rho T : Real) (hT : 0 ≤ T) (s : Fin k -> Real) :
    chamberIndicator eta rho T s ≤ positiveModelProduct eta rho T s := by
  by_cases hs : s ∈ orderedPositiveChamber k
  · rw [chamberIndicator, indicator_of_mem hs]
    have hmem := mem_orderedPositiveChamber_iff.mp hs
    rw [positiveModelProduct_eq_model hmem.1]
    exact (laguerreChamberIntegrand_nonneg_le_model eta rho T s hT hmem.1 hmem.2).2
  · simp [chamberIndicator, hs]
    exact positiveModelProduct_nonneg eta rho T hT s

lemma chamberIndicator_integrable {k : Nat} {eta rho T : Real}
    (heta : -1 < eta) (hrho : 0 ≤ rho) (hT : 0 ≤ T) :
    Integrable (chamberIndicator (k := k) eta rho T)
      (Measure.pi fun _ : Fin k => volume) := by
  have hmodel := positiveModelProduct_integrable (k := k) heta hrho hT
  apply hmodel.mono'
  · exact (measurable_laguerreChamberIntegrand eta rho T).indicator
      (measurableSet_orderedPositiveChamber k) |>.aestronglyMeasurable
  · filter_upwards with s
    rw [Real.norm_of_nonneg (chamberIndicator_nonneg eta rho T hT s)]
    exact chamberIndicator_le_positiveModelProduct eta rho T hT s

lemma laguerreChamberIntegral_eq_integral_indicator {k : Nat} (eta rho T : Real) :
    laguerreChamberIntegral (k := k) eta rho T =
      ∫ s : Fin k -> Real, chamberIndicator eta rho T s
        ∂(Measure.pi fun _ : Fin k => volume) := by
  unfold laguerreChamberIntegral chamberIndicator
  rw [MeasureTheory.integral_indicator (measurableSet_orderedPositiveChamber k)]

/-- Integrated upper bound by the product of the one-dimensional model integrals. -/
theorem laguerreChamberIntegral_le_modelProduct {k : Nat} {eta rho T : Real}
    (heta : -1 < eta) (hrho : 0 ≤ rho) (hT : 0 ≤ T) :
    laguerreChamberIntegral (k := k) eta rho T ≤
      ∏ i : Fin k, Model.integral (spectralA eta i) rho T := by
  rw [laguerreChamberIntegral_eq_integral_indicator,
    ← positiveModelProduct_integral eta rho T]
```

```
  exact MeasureTheory.integral_mono
    (chamberIndicator_integrable heta hrho hT)
    (positiveModelProduct_integrable heta hrho hT)
    (chamberIndicator_le_positiveModelProduct eta rho T hT)

end O77.Residual
end
```

```
noncomputable section
open Real MeasureTheory Set
open scoped ENNReal BigOperators

namespace O77.Residual

/-- A finite coordinate rectangle with half-open intervals `(l_i,u_i]`. -/
def coordinateBox {k : Nat} (l u : Fin k -> Real) : Set (Fin k -> Real) :=
  Set.univ.pi fun i => Ioc (l i) (u i)

lemma mem_coordinateBox_iff {k : Nat} {l u s : Fin k -> Real} :
    s ∈ coordinateBox l u ↔ ∀ i, l i < s i ∧ s i ≤ u i := by
  simp [coordinateBox]

lemma measurableSet_coordinateBox {k : Nat} (l u : Fin k -> Real) :
    MeasurableSet (coordinateBox l u) := by
  exact MeasurableSet.univ_pi fun i => measurableSet_Ioc

lemma laguerreModelProduct_eq_coordinateProduct {k : Nat}
    (eta rho T : Real) (s : Fin k -> Real) :
    laguerreModelProduct eta rho T s =
      ∏ i : Fin k, Model.integrand (spectralA eta i) rho T (s i) := by
  unfold laguerreModelProduct Model.integrand spectralA
  apply Finset.prod_congr rfl
  intro i hi
  congr 1
  ring_nf

/-- The model product integrates exactly as the product of its localized factors. -/
theorem coordinateBox_model_integral {k : Nat} (eta rho T : Real)
    (l u : Fin k -> Real) :
    ∫( s : Fin k -> Real in coordinateBox l u,
      laguerreModelProduct eta rho T s ∂(Measure.pi fun _ : Fin k => volume)) =
      ∏ i : Fin k, Model.localizedIntegral (spectralA eta i) rho T (l i) (u i) := by
  simp_rw [laguerreModelProduct_eq_coordinateProduct]
  unfold coordinateBox Model.localizedIntegral
  rw [Measure.restrict_pi_pi]
  simpa using
    (MeasureTheory.integral_fintype_prod_eq_prod (𝕂 := Real)
      μ( := fun i : Fin k => volume.restrict (Ioc (l i) (u i)))
      (fun i : Fin k => Model.integrand (spectralA eta i) rho T))

/-- The concrete nonresonant rectangle used by the lower chamber estimate. -/
def nonresonantBoxSet {k : Nat} (cut : Nat) (T : Real) : Set (Fin k -> Real) :=
  coordinateBox (nonresonantBoxLower cut T) (nonresonantBoxUpper cut T)

/-- The concrete resonant rectangle used by the lower chamber estimate. -/
def resonantBoxSet {k : Nat} (r : Fin k) (T : Real) : Set (Fin k -> Real) :=
  coordinateBox (resonantBoxLower r T) (resonantBoxUpper r T)

lemma measurableSet_nonresonantBoxSet {k cut : Nat} (T : Real) :
    MeasurableSet (nonresonantBoxSet (k := k) cut T) :=
  measurableSet_coordinateBox _ _
```

```
lemma measurableSet_resonantBoxSet {k : Nat} (r : Fin k) (T : Real) :
    MeasurableSet (resonantBoxSet r T) :=
  measurableSet_coordinateBox _ _

lemma memNonresonantBox_of_mem_set {k cut : Nat} {T : Real} {s : Fin k -> Real}
    (hs : s ∈ nonresonantBoxSet (k := k) cut T) : MemNonresonantBox cut T s := by
  intro i
  have hi := (mem_coordinateBox_iff.mp hs) i
  exact ⟨hi.1.le, hi⟩.2

lemma memResonantBox_of_mem_set {k : Nat} {r : Fin k} {T : Real}
    {s : Fin k -> Real} (hs : s ∈ resonantBoxSet r T) : MemResonantBox r T s := by
  intro i
  have hi := (mem_coordinateBox_iff.mp hs) i
  exact ⟨hi.1.le, hi⟩.2

lemma nonresonantBoxSet_subset_chamber {k cut : Nat} {T : Real}
    (hT : chamberBoxThreshold k ≤ T) :
    nonresonantBoxSet (k := k) cut T ⊆ orderedPositiveChamber k := by
  intro s hs
  have hmem := nonresonantBox_chamber_package hT (memNonresonantBox_of_mem_set hs)
  apply mem_orderedPositiveChamber_iff.mpr
  exact ⟨hmem.1, fun i j hij => by
    linarith [hmem.1 i, hmem.2.1 hij]⟩

lemma resonantBoxSet_subset_chamber {k : Nat} {r : Fin k} {T : Real}
    (hT : chamberBoxThreshold k ≤ T) :
    resonantBoxSet r T ⊆ orderedPositiveChamber k := by
  intro s hs
  have hmem := resonantBox_chamber_package hT (memResonantBox_of_mem_set hs)
  apply mem_orderedPositiveChamber_iff.mpr
  exact ⟨hmem.1, fun i j hij => by
    linarith [hmem.1 i, hmem.2.1 hij]⟩

lemma laguerreChamberIntegrand_integrableOn_chamber {k : Nat} {eta rho T : Real}
    (heta : -1 < eta) (hrho : 0 ≤ rho) (hT : 0 ≤ T) :
    IntegrableOn (laguerreChamberIntegrand (k := k) eta rho T)
      (orderedPositiveChamber k) (Measure.pi fun _ : Fin k => volume) := by
  have hmodel := positiveModelProduct_integrable (k := k) heta hrho hT
  apply hmodel.mono_measure Measure.restrict_le_self |>.mono'
  · exact (measurable_laguerreChamberIntegrand eta rho T).aestronglyMeasurable.restrict
  · filter_upwards [ae_restrict_mem (measurableSet_orderedPositiveChamber k)] with s hs
    have hmem := mem_orderedPositiveChamber_iff.mp hs
    rw [Real.norm_of_nonneg
      (laguerreChamberIntegrand_nonneg_le_model eta rho T s hT hmem.1 hmem.2).1]
    rw [positiveModelProduct_eq_model hmem.1]
    exact (laguerreChamberIntegrand_nonneg_le_model eta rho T s hT hmem.1 hmem.2).2

lemma laguerreChamberIntegrand_nonnegOn_chamber {k : Nat}
    (eta rho T : Real) (hT : 0 ≤ T) :
    0 ≤ᵐ[(Measure.pi fun _ : Fin k => volume).restrict (orderedPositiveChamber k)]
      laguerreChamberIntegrand eta rho T := by
  filter_upwards [ae_restrict_mem (measurableSet_orderedPositiveChamber k)] with s hs
  have hmem := mem_orderedPositiveChamber_iff.mp hs
  exact (laguerreChamberIntegrand_nonneg_le_model eta rho T s hT hmem.1 hmem.2).1

/-- The pointwise separated lower comparison integrated over a nonresonant box. -/
theorem nonresonantBox_model_integral_le_chamber {k cut : Nat}
    {eta rho T : Real} (heta : -1 < eta) (hrho : 0 ≤ rho)
    (hT : chamberBoxThreshold k ≤ T) :
    (1 / 2 : Real) ^ Nat.choose k 2 *
        ∏( i : Fin k, Model.localizedIntegral (spectralA eta i) rho T
```

```
          (nonresonantBoxLower cut T i) (nonresonantBoxUpper cut T i)) ≤
      laguerreChamberIntegral (k := k) eta rho T := by
  let μ : Measure (Fin k -> Real) := Measure.pi fun _ : Fin k => volume
  let box : Set (Fin k -> Real) := nonresonantBoxSet (k := k) cut T
  have hT0 : 0 ≤ T := (chamberBoxThreshold_pos k).le.trans hT
  have hIntChamber := laguerreChamberIntegrand_integrableOn_chamber (k := k) heta hrho hT0
  have hsubset : box ⊆ orderedPositiveChamber k := nonresonantBoxSet_subset_chamber hT
  have hIntBox : IntegrableOn (laguerreChamberIntegrand (k := k) eta rho T) box μ :=
    hIntChamber.mono_set hsubset
  have hModelInt := positiveModelProduct_integrable (k := k) heta hrho hT0
  have hModelBox : IntegrableOn (laguerreModelProduct (k := k) eta rho T) box μ := by
    have hp : IntegrableOn (positiveModelProduct (k := k) eta rho T) box μ :=
      hModelInt.integrableOn
    apply hp.congr_fun
    · intro s hs
      have hmem := nonresonantBox_chamber_package hT (memNonresonantBox_of_mem_set hs)
      exact positiveModelProduct_eq_model hmem.1
    · exact measurableSet_nonresonantBoxSet T
  have hLowerInt : IntegrableOn
      (fun s : Fin k -> Real => (1 / 2 : Real) ^ Nat.choose k 2 *
        laguerreModelProduct eta rho T s) box μ := by
    exact hModelBox.const_mul _
  have hPointwise :
      (fun s : Fin k -> Real => (1 / 2 : Real) ^ Nat.choose k 2 *
        laguerreModelProduct eta rho T s) ≤ᵐ[μ.restrict box]
        laguerreChamberIntegrand eta rho T := by
    filter_upwards [ae_restrict_mem (measurableSet_nonresonantBoxSet T)] with s hs
    have hmem := nonresonantBox_chamber_package hT (memNonresonantBox_of_mem_set hs)
    exact half_pow_choose_mul_model_le_laguerreChamberIntegrand
      eta rho T s hT0 hmem.1 hmem.2.1
  have hBoxLower :
      ∫ s : Fin k -> Real in box,
        (1 / 2 : Real) ^ Nat.choose k 2 * laguerreModelProduct eta rho T s ∂μ) ≤
      ∫ s : Fin k -> Real in box, laguerreChamberIntegrand eta rho T s ∂μ :=
    MeasureTheory.setIntegral_mono_ae_restrict hLowerInt hIntBox hPointwise
  have hBoxChamber :
      ∫ s : Fin k -> Real in box, laguerreChamberIntegrand eta rho T s ∂μ) ≤
        ∫ s : Fin k -> Real in orderedPositiveChamber k,
          laguerreChamberIntegrand eta rho T s ∂μ := by
    exact MeasureTheory.setIntegral_mono_set hIntChamber
      (laguerreChamberIntegrand_nonnegOn_chamber eta rho T hT0)
      (Filter.Eventually.of_forall hsubset)
  calc
    (1 / 2 : Real) ^ Nat.choose k 2 *
        ∏( i : Fin k, Model.localizedIntegral (spectralA eta i) rho T
          (nonresonantBoxLower cut T i) (nonresonantBoxUpper cut T i)) =
        ∫ s : Fin k -> Real in box,
          (1 / 2 : Real) ^ Nat.choose k 2 *
            laguerreModelProduct eta rho T s ∂μ := by
      rw [MeasureTheory.integral_const_mul]
      simp only [box, μ, nonresonantBoxSet]
      rw [coordinateBox_model_integral]
    _ ≤ ∫ s : Fin k -> Real in box,
          laguerreChamberIntegrand eta rho T s ∂μ := hBoxLower
    _ ≤ laguerreChamberIntegral (k := k) eta rho T := hBoxChamber

/-- The pointwise separated lower comparison integrated over a resonant box. -/
theorem resonantBox_model_integral_le_chamber {k : Nat} {r : Fin k}
    {eta rho T : Real} (heta : -1 < eta) (hrho : 0 ≤ rho)
    (hT : chamberBoxThreshold k ≤ T) :
    (1 / 2 : Real) ^ Nat.choose k 2 *
        ∏( i : Fin k, Model.localizedIntegral (spectralA eta i) rho T
```

```
          (resonantBoxLower r T i) (resonantBoxUpper r T i)) ≤
      laguerreChamberIntegral (k := k) eta rho T := by
  let μ : Measure (Fin k -> Real) := Measure.pi fun _ : Fin k => volume
  let box : Set (Fin k -> Real) := resonantBoxSet r T
  have hT0 : 0 ≤ T := (chamberBoxThreshold_pos k).le.trans hT
  have hIntChamber := laguerreChamberIntegrand_integrableOn_chamber (k := k) heta hrho hT0
  have hsubset : box ⊆ orderedPositiveChamber k := resonantBoxSet_subset_chamber hT
  have hIntBox : IntegrableOn (laguerreChamberIntegrand (k := k) eta rho T) box μ :=
    hIntChamber.mono_set hsubset
  have hModelInt := positiveModelProduct_integrable (k := k) heta hrho hT0
  have hModelBox : IntegrableOn (laguerreModelProduct (k := k) eta rho T) box μ := by
    have hp : IntegrableOn (positiveModelProduct (k := k) eta rho T) box μ :=
      hModelInt.integrableOn
    apply hp.congr_fun
    · intro s hs
      have hmem := resonantBox_chamber_package hT (memResonantBox_of_mem_set hs)
      exact positiveModelProduct_eq_model hmem.1
    · exact measurableSet_resonantBoxSet r T
  have hLowerInt : IntegrableOn
      (fun s : Fin k -> Real => (1 / 2 : Real) ^ Nat.choose k 2 *
        laguerreModelProduct eta rho T s) box μ := by
    exact hModelBox.const_mul _
  have hPointwise :
      (fun s : Fin k -> Real => (1 / 2 : Real) ^ Nat.choose k 2 *
        laguerreModelProduct eta rho T s) ≤ᵐ[μ.restrict box]
        laguerreChamberIntegrand eta rho T := by
    filter_upwards [ae_restrict_mem (measurableSet_resonantBoxSet r T)] with s hs
    have hmem := resonantBox_chamber_package hT (memResonantBox_of_mem_set hs)
    exact half_pow_choose_mul_model_le_laguerreChamberIntegrand
      eta rho T s hT0 hmem.1 hmem.2.1
  have hBoxLower :
      ∫ s : Fin k -> Real in box,
        (1 / 2 : Real) ^ Nat.choose k 2 * laguerreModelProduct eta rho T s ∂μ) ≤
      ∫ s : Fin k -> Real in box, laguerreChamberIntegrand eta rho T s ∂μ :=
    MeasureTheory.setIntegral_mono_ae_restrict hLowerInt hIntBox hPointwise
  have hBoxChamber :
      ∫ s : Fin k -> Real in box, laguerreChamberIntegrand eta rho T s ∂μ) ≤
        ∫ s : Fin k -> Real in orderedPositiveChamber k,
          laguerreChamberIntegrand eta rho T s ∂μ := by
    exact MeasureTheory.setIntegral_mono_set hIntChamber
      (laguerreChamberIntegrand_nonnegOn_chamber eta rho T hT0)
      (Filter.Eventually.of_forall hsubset)
  calc
    (1 / 2 : Real) ^ Nat.choose k 2 *
        ∏( i : Fin k, Model.localizedIntegral (spectralA eta i) rho T
          (resonantBoxLower r T i) (resonantBoxUpper r T i)) =
        ∫ s : Fin k -> Real in box,
          (1 / 2 : Real) ^ Nat.choose k 2 *
            laguerreModelProduct eta rho T s ∂μ := by
      rw [MeasureTheory.integral_const_mul]
      simp only [box, μ, resonantBoxSet]
      rw [coordinateBox_model_integral]
    _ ≤ ∫ s : Fin k -> Real in box,
          laguerreChamberIntegrand eta rho T s ∂μ := hBoxLower
    _ ≤ laguerreChamberIntegral (k := k) eta rho T := hBoxChamber

end O77.Residual
end
```

```
noncomputable section
open Real MeasureTheory Set
open scoped ENNReal BigOperators
```

```
namespace O77.Residual

/-- Product of the one-dimensional power-log gauges on the spectral coordinates. -/
def chamberGaugeProduct {k : Nat} (eta rho T : Real) : Real :=
  ∏ i : Fin k, Model.gauge (spectralA eta i) rho T

lemma one_le_fourScale {k : Nat} (i : Fin k) : 1 ≤ fourScale i := by
  unfold fourScale
  exact ₀one_le_pow (by norm_num)

lemma two_mul_fourScale_le_chamberBoxThreshold {k : Nat} (i : Fin k) :
    2 * fourScale i ≤ chamberBoxThreshold k := by
  have h4 : 2 * fourScale i ≤ 4 * fourScale i := by
    nlinarith [fourScale_pos i]
  have hexp : i.val + 2 ≤ 2 * (k + 1) := by omega
  calc
    2 * fourScale i ≤ 4 * fourScale i := h4
    _ = (4 : Real) ^ (i.val + 2) := by simp [fourScale, pow_succ, mul_comm]
    _ ≤ (4 : Real) ^ (2 * (k + 1)) := ₀pow_le_pow_right (by norm_num) hexp
    _ = chamberBoxThreshold k := by
      rw [pow_mul]
      norm_num [chamberBoxThreshold]

lemma fourScale_sq_le_chamberBoxThreshold {k : Nat} (i : Fin k) :
    (fourScale i) ^ 2 ≤ chamberBoxThreshold k := by
  have hexp : 2 * (i.val + 1) ≤ 2 * (k + 1) := by omega
  calc
    (fourScale i) ^ 2 = (4 : Real) ^ (2 * (i.val + 1)) := by
      simp [fourScale, ← pow_mul, mul_comm]
    _ ≤ (4 : Real) ^ (2 * (k + 1)) := ₀pow_le_pow_right (by norm_num) hexp
    _ = chamberBoxThreshold k := by
      rw [pow_mul]
      norm_num [chamberBoxThreshold]

lemma modelGauge_eq_left {A rho T : Real} (hArho : A < rho) :
    Model.gauge A rho T = T ^ (-A) := by
  simp [Model.gauge, min_eq_left hArho.le, hArho.ne]

lemma modelGauge_eq_right {A rho T : Real} (hArho : rho < A) :
    Model.gauge A rho T = T ^ (-rho) := by
  simp [Model.gauge, min_eq_right hArho.le, hArho.ne.symm]

lemma modelGauge_eq_resonant (A T : Real) :
    Model.gauge A A T = T ^ (-A) * Real.log T := by
  simp [Model.gauge]

/-- Coordinate lower constant for a nonresonant separated box. -/
def nonresonantCoordinateConstant {k : Nat}
    (eta rho : Real) (cut : Nat) (i : Fin k) : Real :=
  if i.val < cut then
    Model.scaledIntervalConstant (spectralA eta i) rho (fourScale i)
  else
    Model.fixedIntervalConstant (spectralA eta i) rho (fourScale i)

lemma nonresonantCoordinateConstant_pos {k cut : Nat} {eta rho : Real}
    (i : Fin k) : 0 < nonresonantCoordinateConstant eta rho cut i := by
  unfold nonresonantCoordinateConstant
  split
  · exact Model.scaledIntervalConstant_pos (fourScale_pos i)
  · exact Model.fixedIntervalConstant_pos (fourScale_pos i)
```

```
/-- Product lower constant for a nonresonant separated box. -/
def nonresonantBoxConstant {k : Nat} (eta rho : Real) (cut : Nat) : Real :=
  (1 / 2 : Real) ^ Nat.choose k 2 *
    ∏ i : Fin k, nonresonantCoordinateConstant eta rho cut i

lemma nonresonantBoxConstant_pos {k cut : Nat} {eta rho : Real} :
    0 < nonresonantBoxConstant (k := k) eta rho cut := by
  unfold nonresonantBoxConstant
  exact mul_pos (pow_pos (by norm_num) _)
    (Finset.prod_pos fun i _ => nonresonantCoordinateConstant_pos i)

lemma nonresonant_localized_factor_lower {k cut : Nat} {eta rho T : Real}
    (heta : -1 < eta) (hrho : 0 < rho)
    (hT : chamberBoxThreshold k ≤ T)
    (hlo : ∀ i : Fin k, i.val < cut -> spectralA eta i < rho)
    (hhi : ∀ i : Fin k, cut ≤ i.val -> rho < spectralA eta i)
    (i : Fin k) :
    nonresonantCoordinateConstant eta rho cut i *
        Model.gauge (spectralA eta i) rho T ≤
      Model.localizedIntegral (spectralA eta i) rho T
        (nonresonantBoxLower cut T i) (nonresonantBoxUpper cut T i) := by
  have hA : 0 < spectralA eta i := spectralA_pos heta i
  have hTone : 1 ≤ T := (chamberBoxThreshold_one_le k).trans hT
  by_cases hi : i.val < cut
  · have hscale : 2 * fourScale i ≤ T :=
      (two_mul_fourScale_le_chamberBoxThreshold i).trans hT
    have hloc := Model.localized_scaled_lower hA (fourScale_pos i) hrho.le hscale
    rw [modelGauge_eq_left (hlo i hi)]
    simpa [nonresonantCoordinateConstant, nonresonantBoxLower,
      nonresonantBoxUpper, hi] using hloc
  · have hicut : cut ≤ i.val := by omega
    have hloc := Model.localized_fixed_lower hA (one_le_fourScale i) hrho.le hTone
    rw [modelGauge_eq_right (hhi i hicut)]
    simpa [nonresonantCoordinateConstant, nonresonantBoxLower,
      nonresonantBoxUpper, hi] using hloc

lemma nonresonant_product_localized_lower {k cut : Nat} {eta rho T : Real}
    (heta : -1 < eta) (hrho : 0 < rho)
    (hT : chamberBoxThreshold k ≤ T)
    (hlo : ∀ i : Fin k, i.val < cut -> spectralA eta i < rho)
    (hhi : ∀ i : Fin k, cut ≤ i.val -> rho < spectralA eta i) :
    ∏( i : Fin k, nonresonantCoordinateConstant eta rho cut i) *
        chamberGaugeProduct (k := k) eta rho T ≤
      ∏ i : Fin k, Model.localizedIntegral (spectralA eta i) rho T
        (nonresonantBoxLower cut T i) (nonresonantBoxUpper cut T i) := by
  have hTone : 1 ≤ T := (chamberBoxThreshold_one_le k).trans hT
  rw [chamberGaugeProduct, ← Finset.prod_mul_distrib]
  apply Finset.prod_le_prod
  · intro i hi
    exact mul_nonneg (nonresonantCoordinateConstant_pos i).le
      (Model.gauge_nonneg hTone)
  · intro i hi
    exact nonresonant_localized_factor_lower heta hrho hT hlo hhi i

/-- Integrated nonresonant lower bound in the same product gauge as the upper estimate. -/
theorem nonresonantBox_chamberGauge_lower {k cut : Nat} {eta rho T : Real}
    (heta : -1 < eta) (hrho : 0 < rho)
    (hT : chamberBoxThreshold k ≤ T)
    (hlo : ∀ i : Fin k, i.val < cut -> spectralA eta i < rho)
    (hhi : ∀ i : Fin k, cut ≤ i.val -> rho < spectralA eta i) :
    nonresonantBoxConstant (k := k) eta rho cut *
        chamberGaugeProduct (k := k) eta rho T ≤
```

```
      laguerreChamberIntegral (k := k) eta rho T := by
  have hp := nonresonant_product_localized_lower heta hrho hT hlo hhi
  have hhalf : 0 ≤ (1 / 2 : Real) ^ Nat.choose k 2 := pow_nonneg (by norm_num) _
  have hbox := nonresonantBox_model_integral_le_chamber (k := k) (cut := cut)
    heta hrho.le hT
  unfold nonresonantBoxConstant
  calc
    ((1 / 2 : Real) ^ Nat.choose k 2 *
        ∏ i : Fin k, nonresonantCoordinateConstant eta rho cut i) *
        chamberGaugeProduct eta rho T =
      (1 / 2 : Real) ^ Nat.choose k 2 *
        ∏(( i : Fin k, nonresonantCoordinateConstant eta rho cut i) *
          chamberGaugeProduct eta rho T) := by ring
    _ ≤ (1 / 2 : Real) ^ Nat.choose k 2 *
        ∏( i : Fin k, Model.localizedIntegral (spectralA eta i) rho T
          (nonresonantBoxLower cut T i) (nonresonantBoxUpper cut T i)) :=
      mul_le_mul_of_nonneg_left hp hhalf
    _ ≤ laguerreChamberIntegral eta rho T := hbox

/-- Coordinate lower constant for a box with one resonant coordinate. -/
def resonantCoordinateConstant {k : Nat}
    (eta rho : Real) (r i : Fin k) : Real :=
  if i < r then
    Model.scaledIntervalConstant (spectralA eta i) rho (fourScale i)
  else if i = r then
    Real.exp (-1) * (2 : Real) ^ (-(spectralA eta i)) / 2
  else
    Model.fixedIntervalConstant (spectralA eta i) rho (fourScale i)

lemma resonantCoordinateConstant_pos {k : Nat} {eta rho : Real}
    (r i : Fin k) : 0 < resonantCoordinateConstant eta rho r i := by
  unfold resonantCoordinateConstant
  split
  · exact Model.scaledIntervalConstant_pos (fourScale_pos i)
  · split
    · positivity
    · exact Model.fixedIntervalConstant_pos (fourScale_pos i)

/-- Product lower constant for a resonant separated box. -/
def resonantBoxConstant {k : Nat} (eta rho : Real) (r : Fin k) : Real :=
  (1 / 2 : Real) ^ Nat.choose k 2 *
    ∏ i : Fin k, resonantCoordinateConstant eta rho r i

lemma resonantBoxConstant_pos {k : Nat} {eta rho : Real} (r : Fin k) :
    0 < resonantBoxConstant eta rho r := by
  unfold resonantBoxConstant
  exact mul_pos (pow_pos (by norm_num) _)
    (Finset.prod_pos fun i _ => resonantCoordinateConstant_pos r i)

lemma resonant_localized_factor_lower {k : Nat} {eta rho T : Real}
    (heta : -1 < eta) (hrho : 0 < rho) {r : Fin k}
    (hT : chamberBoxThreshold k ≤ T)
    (hlo : ∀ i : Fin k, i < r -> spectralA eta i < rho)
    (heq : spectralA eta r = rho)
    (hhi : ∀ i : Fin k, r < i -> rho < spectralA eta i)
    (i : Fin k) :
    resonantCoordinateConstant eta rho r i *
        Model.gauge (spectralA eta i) rho T ≤
      Model.localizedIntegral (spectralA eta i) rho T
        (resonantBoxLower r T i) (resonantBoxUpper r T i) := by
  have hA : 0 < spectralA eta i := spectralA_pos heta i
  have hTone : 1 ≤ T := (chamberBoxThreshold_one_le k).trans hT
```

```
  by_cases hir : i < r
  · have hscale : 2 * fourScale i ≤ T :=
      (two_mul_fourScale_le_chamberBoxThreshold i).trans hT
    have hloc := Model.localized_scaled_lower hA (fourScale_pos i) hrho.le hscale
    rw [modelGauge_eq_left (hlo i hir)]
    simpa [resonantCoordinateConstant, resonantBoxLower, resonantBoxUpper, hir] using hloc
  · by_cases hieq : i = r
    · subst i
      subst rho
      have hsq : (fourScale r) ^ 2 ≤ T :=
        (fourScale_sq_le_chamberBoxThreshold r).trans hT
      have hloc := Model.localized_resonant_lower (spectralA_pos heta r)
        (one_le_fourScale r) hsq
      rw [modelGauge_eq_resonant]
      simpa [resonantCoordinateConstant, resonantBoxLower, resonantBoxUpper] using hloc
    · have hri : r < i := lt_of_le_of_ne (not_lt.mp hir) (Ne.symm hieq)
      have hloc := Model.localized_fixed_lower hA (one_le_fourScale i) hrho.le hTone
      rw [modelGauge_eq_right (hhi i hri)]
      simpa [resonantCoordinateConstant, resonantBoxLower, resonantBoxUpper,
        hir, hieq] using hloc

lemma resonant_product_localized_lower {k : Nat} {eta rho T : Real}
    (heta : -1 < eta) (hrho : 0 < rho) {r : Fin k}
    (hT : chamberBoxThreshold k ≤ T)
    (hlo : ∀ i : Fin k, i < r -> spectralA eta i < rho)
    (heq : spectralA eta r = rho)
    (hhi : ∀ i : Fin k, r < i -> rho < spectralA eta i) :
    ∏( i : Fin k, resonantCoordinateConstant eta rho r i) *
        chamberGaugeProduct (k := k) eta rho T ≤
      ∏ i : Fin k, Model.localizedIntegral (spectralA eta i) rho T
        (resonantBoxLower r T i) (resonantBoxUpper r T i) := by
  have hTone : 1 ≤ T := (chamberBoxThreshold_one_le k).trans hT
  rw [chamberGaugeProduct, ← Finset.prod_mul_distrib]
  apply Finset.prod_le_prod
  · intro i hi
    exact mul_nonneg (resonantCoordinateConstant_pos r i).le
      (Model.gauge_nonneg hTone)
  · intro i hi
    exact resonant_localized_factor_lower heta hrho hT hlo heq hhi i

/-- Integrated resonant lower bound, including the full logarithmic factor. -/
theorem resonantBox_chamberGauge_lower {k : Nat} {eta rho T : Real}
    (heta : -1 < eta) (hrho : 0 < rho) {r : Fin k}
    (hT : chamberBoxThreshold k ≤ T)
    (hlo : ∀ i : Fin k, i < r -> spectralA eta i < rho)
    (heq : spectralA eta r = rho)
    (hhi : ∀ i : Fin k, r < i -> rho < spectralA eta i) :
    resonantBoxConstant (k := k) eta rho r *
        chamberGaugeProduct (k := k) eta rho T ≤
      laguerreChamberIntegral (k := k) eta rho T := by
  have hp := resonant_product_localized_lower heta hrho hT hlo heq hhi
  have hhalf : 0 ≤ (1 / 2 : Real) ^ Nat.choose k 2 := pow_nonneg (by norm_num) _
  have hbox := resonantBox_model_integral_le_chamber (k := k) (r := r)
    heta hrho.le hT
  unfold resonantBoxConstant
  calc
    ((1 / 2 : Real) ^ Nat.choose k 2 *
        ∏ i : Fin k, resonantCoordinateConstant eta rho r i) *
        chamberGaugeProduct eta rho T =
      (1 / 2 : Real) ^ Nat.choose k 2 *
        ∏(( i : Fin k, resonantCoordinateConstant eta rho r i) *
          chamberGaugeProduct eta rho T) := by ring
```

```
    _ ≤ (1 / 2 : Real) ^ Nat.choose k 2 *
        ∏( i : Fin k, Model.localizedIntegral (spectralA eta i) rho T
          (resonantBoxLower r T i) (resonantBoxUpper r T i)) :=
      mul_le_mul_of_nonneg_left hp hhalf
    _ ≤ laguerreChamberIntegral eta rho T := hbox

end O77.Residual
end
```

```
noncomputable section
open Real MeasureTheory Set
open scoped ENNReal BigOperators

namespace O77.Residual

lemma spectralA_strictMono {k : Nat} (eta : Real) :
    StrictMono (spectralA (k := k) eta) := by
  intro i j hij
  have hval : (i.val : Real) < j.val := by exact_mod_cast hij
  unfold spectralA
  linarith

/-- Every finite spectral sequence has either a strict cut or one unique resonant coordinate. -/
def HasSpectralPattern {k : Nat} (eta rho : Real) : Prop :=
  ∃( cut : Nat,
      cut ≤ k ∧
      ∀( i : Fin k, i.val < cut -> spectralA eta i < rho) ∧
      ∀( i : Fin k, cut ≤ i.val -> rho < spectralA eta i)) ∨
  ∃( r : Fin k,
      ∀( i : Fin k, i < r -> spectralA eta i < rho) ∧
      spectralA eta r = rho ∧
      ∀( i : Fin k, r < i -> rho < spectralA eta i))

/-- The strict monotonicity of `eta+i+1` gives the complete cut/resonance alternative. -/
theorem hasSpectralPattern (k : Nat) (eta rho : Real) :
    HasSpectralPattern (k := k) eta rho := by
  classical
  by_cases heq : ∃ r : Fin k, spectralA eta r = rho
  · right
    rcases heq with ⟨r, ⟩hr
    refine ⟨r, ?_, hr, ?⟩_
    · intro i hir
      have hlt := spectralA_strictMono (k := k) eta hir
      linarith
    · intro i hri
      have hlt := spectralA_strictMono (k := k) eta hri
      linarith
  · by_cases hhigh : ∃ i : Fin k, rho < spectralA eta i
    · let S : Finset (Fin k) := Finset.univ.filter fun i => rho < spectralA eta i
      have hS : S.Nonempty := by
        rcases hhigh with ⟨i, ⟩hi
        exact ⟨i, by simp [S, hi⟩]
      let r : Fin k := S.min' hS
      have hrmem : r ∈ S := Finset.min'_mem S hS
      have hrhigh : rho < spectralA eta r := by
        simpa [S] using hrmem
      left
      refine ⟨r.val, r.isLt.le, ?_, ?⟩_
      · intro i hi
        have hir : i < r := hi
        have hnothigh : ¬ rho < spectralA eta i := by
          intro hihigh
```

```
          have himem : i ∈ S := by simp [S, hihigh]
          have hri : r ≤ i := Finset.min'_le S i himem
          exact (not_le_of_gt hir) hri
        rcases lt_trichotomy (spectralA eta i) rho with hlow | hequal | hgreater
        · exact hlow
        · exact False.elim (heq ⟨i, ⟩hequal)
        · exact False.elim (hnothigh hgreater)
      · intro i hriVal
        have hri : r ≤ i := hriVal
        have hmono : spectralA eta r ≤ spectralA eta i :=
          (spectralA_strictMono (k := k) eta).monotone hri
        exact hrhigh.trans_le hmono
    · left
      refine ⟨k, le_rfl, ?_, ?⟩_
      · intro i hi
        have hnothigh : ¬ rho < spectralA eta i := by
          intro h
          exact hhigh ⟨i, ⟩h
        rcases lt_trichotomy (spectralA eta i) rho with hlow | hequal | hgreater
        · exact hlow
        · exact False.elim (heq ⟨i, ⟩hequal)
        · exact False.elim (hnothigh hgreater)
      · intro i hi
        omega

/-- A convenient two-sided product-gauge predicate for the ordered chamber. -/
def ChamberProductGaugeBounds {k : Nat} (eta rho : Real) : Prop :=
  ∃ c C T0 : Real,
    0 < c ∧ c ≤ C ∧ chamberBoxThreshold k ≤ T0 ∧
      ∀ T : Real, T0 ≤ T ->
        c * chamberGaugeProduct (k := k) eta rho T ≤
            laguerreChamberIntegral (k := k) eta rho T ∧
          laguerreChamberIntegral (k := k) eta rho T ≤
            C * chamberGaugeProduct (k := k) eta rho T

/-- The product of the one-dimensional upper bounds controls the chamber integral. -/
theorem laguerreChamberIntegral_upper_chamberGauge {k : Nat} {eta rho : Real}
    (heta : -1 < eta) (hrho : 0 < rho) :
    ∃ C T0 : Real, 0 < C ∧ chamberBoxThreshold k ≤ T0 ∧
      ∀ T : Real, T0 ≤ T ->
        laguerreChamberIntegral (k := k) eta rho T ≤
          C * chamberGaugeProduct (k := k) eta rho T := by
  classical
  have hdata : ∀ i : Fin k, Model.HasPowerLogBounds (spectralA eta i) rho :=
    fun i => Model.modelIntegral_powerLog_bounds (spectralA_pos heta i) hrho
  choose c C T0 hc hcC htwo hbounds using hdata
  let Cstar : Real := ∏ i : Fin k, C i
  let Tstar : Real := chamberBoxThreshold k + ∑ i : Fin k, T0 i
  have hCpos : 0 < Cstar := by
    dsimp [Cstar]
    exact Finset.prod_pos fun i hi => (hc i).trans_le (hcC i)
  have hthreshold : chamberBoxThreshold k ≤ Tstar := by
    dsimp [Tstar]
    have hsum0 : 0 ≤ ∑ i : Fin k, T0 i := by
      exact Finset.sum_nonneg fun i hi => by linarith [htwo i]
    linarith
  refine ⟨Cstar, Tstar, hCpos, hthreshold, ?⟩_
  intro T hT
  have hTchamber : chamberBoxThreshold k ≤ T := hthreshold.trans hT
  have hTone : 1 ≤ T := (chamberBoxThreshold_one_le k).trans hTchamber
  have hT0 : 0 ≤ T := zero_le_one.trans hTone
  have hcoordThreshold : ∀ i : Fin k, T0 i ≤ T := by
```

```
    intro i
    have hsingle : T0 i ≤ ∑ j : Fin k, T0 j := by
      apply Finset.single_le_sum
      · intro j hj
        linarith [htwo j]
      · simp
    dsimp [Tstar] at hT
    linarith [chamberBoxThreshold_pos k]
  have hprod :
      ∏( i : Fin k, Model.integral (spectralA eta i) rho T) ≤
        ∏ i : Fin k, C i * Model.gauge (spectralA eta i) rho T := by
    apply Finset.prod_le_prod
    · intro i hi
      exact Model.integral_nonneg hT0
    · intro i hi
      exact (hbounds i T (hcoordThreshold i)).2
  calc
    laguerreChamberIntegral (k := k) eta rho T ≤
        ∏ i : Fin k, Model.integral (spectralA eta i) rho T :=
      laguerreChamberIntegral_le_modelProduct heta hrho.le hT0
    _ ≤ ∏ i : Fin k, C i * Model.gauge (spectralA eta i) rho T := hprod
    _ = Cstar * chamberGaugeProduct eta rho T := by
      rw [Finset.prod_mul_distrib]
      rfl

/-- The separated-box lower bound and the product upper bound give a complete
ordered-chamber estimate in the product gauge. -/
theorem laguerreChamber_productGauge_bounds {k : Nat} {eta rho : Real}
    (heta : -1 < eta) (hrho : 0 < rho) :
    ChamberProductGaugeBounds (k := k) eta rho := by
  classical
  obtain ⟨C, Tupper, hC, hboxUpper, ⟩hupper :=
    laguerreChamberIntegral_upper_chamberGauge (k := k) heta hrho
  rcases hasSpectralPattern k eta rho with hnonres | hres
  · rcases hnonres with ⟨cut, hcut, hlo, ⟩hhi
    let c0 : Real := nonresonantBoxConstant (k := k) eta rho cut
    let c : Real := min c0 C
    let Tstar : Real := max Tupper (chamberBoxThreshold k)
    have hc0 : 0 < c0 := nonresonantBoxConstant_pos
    have hc : 0 < c := lt_min hc0 hC
    have hcC : c ≤ C := min_le_right _ _
    have hbox : chamberBoxThreshold k ≤ Tstar := le_max_right _ _
    refine ⟨c, C, Tstar, hc, hcC, hbox, ?⟩_
    intro T hT
    have hTu : Tupper ≤ T := (le_max_left _ _).trans hT
    have hTb : chamberBoxThreshold k ≤ T := hbox.trans hT
    have hG : 0 ≤ chamberGaugeProduct (k := k) eta rho T := by
      unfold chamberGaugeProduct
      exact Finset.prod_nonneg fun i hi =>
        Model.gauge_nonneg ((chamberBoxThreshold_one_le k).trans hTb)
    constructor
    · calc
        c * chamberGaugeProduct eta rho T ≤
            c0 * chamberGaugeProduct eta rho T :=
          mul_le_mul_of_nonneg_right (min_le_left _ _) hG
        _ ≤ laguerreChamberIntegral eta rho T :=
          nonresonantBox_chamberGauge_lower heta hrho hTb hlo hhi
    · exact hupper T hTu
  · rcases hres with ⟨r, hlo, heq, ⟩hhi
    let c0 : Real := resonantBoxConstant (k := k) eta rho r
    let c : Real := min c0 C
    let Tstar : Real := max Tupper (chamberBoxThreshold k)
```

```
    have hc0 : 0 < c0 := resonantBoxConstant_pos r
    have hc : 0 < c := lt_min hc0 hC
    have hcC : c ≤ C := min_le_right _ _
    have hbox : chamberBoxThreshold k ≤ Tstar := le_max_right _ _
    refine ⟨c, C, Tstar, hc, hcC, hbox, ?_⟩
    intro T hT
    have hTu : Tupper ≤ T := (le_max_left _ _).trans hT
    have hTb : chamberBoxThreshold k ≤ T := hbox.trans hT
    have hG : 0 ≤ chamberGaugeProduct (k := k) eta rho T := by
      unfold chamberGaugeProduct
      exact Finset.prod_nonneg fun i hi =>
        Model.gauge_nonneg ((chamberBoxThreshold_one_le k).trans hTb)
    constructor
    · calc
        c * chamberGaugeProduct eta rho T ≤
            c0 * chamberGaugeProduct eta rho T :=
          mul_le_mul_of_nonneg_right (min_le_left _ _) hG
        _ ≤ laguerreChamberIntegral eta rho T :=
          resonantBox_chamberGauge_lower heta hrho hTb hlo heq hhi
    · exact hupper T hTu

end O77.Residual
end
```

```
noncomputable section
open Real MeasureTheory Set
open scoped ENNReal BigOperators

namespace O77.Residual

/-- Sum of the one-dimensional exponents in the ordered-chamber product. -/
def chamberExponent {k : Nat} (eta rho : Real) : Real :=
  ∑ i : Fin k, min (spectralA eta i) rho

/-- Number of resonant one-dimensional factors. -/
def chamberResonanceCount {k : Nat} (eta rho : Real) : Nat :=
  (Finset.univ.filter fun i : Fin k => spectralA eta i = rho).card

/-- The collapsed power-log gauge of the ordered chamber. -/
def chamberPowerLogGauge {k : Nat} (eta rho T : Real) : Real :=
  T ^ (-chamberExponent (k := k) eta rho) *
    (Real.log T) ^ chamberResonanceCount (k := k) eta rho

lemma finset_prod_rpow ι{ : Type*} [Fintype ι] (T : Real) (hT : 0 < T)
    (e : ι -> Real) :
    ∏( i : ι, T ^ e i) = T ^ ∑( i : ι, e i) := by
  classical
  induction (Finset.univ : Finset ι) using Finset.induction_on with
  | empty => simp
  | @insert a s ha ih =>
      rw [Finset.prod_insert ha, Finset.sum_insert ha, ih, Real.rpow_add hT]

lemma prod_ite_const_eq_pow_card_filter ι{ : Type*} [Fintype ι]
    (P : ι -> Prop) [DecidablePred P] (x : Real) :
    ∏( i : ι, if P i then x else 1) =
      x ^ (Finset.univ.filter P).card := by
  classical
  rw ←[ Finset.prod_filter]
  simp

/-- The product gauge is exactly one power with one logarithm per resonance. -/
theorem chamberGaugeProduct_eq_powerLog {k : Nat} (eta rho T : Real)
```

```
    (hT : 0 < T) :
    chamberGaugeProduct (k := k) eta rho T =
      chamberPowerLogGauge (k := k) eta rho T := by
  classical
  unfold chamberGaugeProduct chamberPowerLogGauge chamberExponent chamberResonanceCount
  simp only [Model.gauge]
  rw [Finset.prod_mul_distrib]
  rw [finset_prod_rpow T hT (fun i : Fin k => -min (spectralA eta i) rho)]
  have hneg : ∏( i : Fin k, -min (spectralA eta i) rho) =
      ∏-( i : Fin k, min (spectralA eta i) rho) := by
    rw [Finset.sum_neg_distrib]
  rw [hneg]
  congr 1
  exact prod_ite_const_eq_pow_card_filter
    ι( := Fin k) (fun i : Fin k => spectralA eta i = rho) (Real.log T)

lemma chamberResonanceCount_le_one {k : Nat} (eta rho : Real) :
    chamberResonanceCount (k := k) eta rho ≤ 1 := by
  classical
  unfold chamberResonanceCount
  apply Finset.card_le_one.mpr
  intro i hi j hj
  simp only [Finset.mem_filter, Finset.mem_univ, true_and] at hi hj
  by_contra hij
  rcases lt_or_gt_of_ne hij with hijlt | hijgt
  · have hstrict := spectralA_strictMono (k := k) eta hijlt
    linarith
  · have hstrict := spectralA_strictMono (k := k) eta hijgt
    linarith

/-- Concrete two-sided power-log bounds for the ordered Laguerre chamber. -/
def ChamberPowerLogBounds {k : Nat} (eta rho : Real) : Prop :=
  ∃ c C T0 : Real,
    0 < c ∧ c ≤ C ∧ chamberBoxThreshold k ≤ T0 ∧
      ∀ T : Real, T0 ≤ T ->
        c * chamberPowerLogGauge (k := k) eta rho T ≤
            laguerreChamberIntegral (k := k) eta rho T ∧
          laguerreChamberIntegral (k := k) eta rho T ≤
            C * chamberPowerLogGauge (k := k) eta rho T

/-- Full integrated chamber estimate: the exponent is the sum of the coordinate
minima and the logarithmic degree is the number of resonant coordinates. -/
theorem laguerreChamber_powerLog_bounds {k : Nat} {eta rho : Real}
    (heta : -1 < eta) (hrho : 0 < rho) :
    ChamberPowerLogBounds (k := k) eta rho := by
  rcases laguerreChamber_productGauge_bounds (k := k) heta hrho with
    ⟨c, C, T0, hc, hcC, hbox, ⟩hbounds
  refine ⟨c, C, T0, hc, hcC, hbox, ?⟩_
  intro T hT
  have hTpos : 0 < T :=
    (chamberBoxThreshold_pos k).trans_le (hbox.trans hT)
  simpa [chamberGaugeProduct_eq_powerLog eta rho T hTpos] using hbounds T hT

-- Exact closed controls, including the empty product.
example : chamberExponent (k := 0) 3 4 = 0 := by simp [chamberExponent]
example : chamberResonanceCount (k := 0) 3 4 = 0 := by simp [chamberResonanceCount]
example : chamberPowerLogGauge (k := 0) 3 4 7 = 1 := by
  simp [chamberPowerLogGauge, chamberExponent, chamberResonanceCount]

end O77.Residual
end
```

Lemma 20.14 (O77 specialization of the integrated chamber)**.**

Corresponding Lean declaration(s): `O77.Residual.prefixGaugeExponent,O77.Residual.chamberExponent_eq_sum_range,O77.Residual.chamberExponent_le_prefixGaugeExponent,O77.Residual.chamberExponent_eq_prefixGaugeExponent_of_strictCut,O77.Residual.chamberExponent_eq_prefixGaugeExponent_of_resonance`.

Corresponding Lean declaration(s): `O77.Residual.prefixGaugeExponent_succ,O77.Residual.residualSpectralEta,O77.Residual.residualSpectralRho,O77.Residual.residualSpectralEta_gt_neg_one,O77.Residual.residualSpectralRho_pos`.

Corresponding Lean declaration(s): `O77.Residual.two_mul_residualSpectral_difference_eq_increment,O77.Residual.two_mul_prefixGaugeExponent_eq_chamberTwicePrefixExponent,O77.Residual.two_mul_residual_chamberExponent_eq_residualMin,O77.Residual.residual_spectralA_eq_rho_iff_chamberResonant,O77.Residual.residual_spectral_resonance_exists_iff`.

Corresponding Lean declaration(s): `O77.Residual.chamberResonanceCount_eq_one_iff,O77.Residual.residual_chamberResonanceCount_eq_resonanceMultiplicity,O77.Residual.residual_laguerreChamber_powerLog_bounds`.

For $p, q, h \in \mathbb{N}$, set

$$k = \min(h, q), \quad d = \max(h, q), \quad \eta = \frac{d - k - 1}{2}, \quad \rho = \frac{p}{2},$$

where the subtraction defining η is in $\mathbb{R}$. Let

$$E_* = \sum_{i=1}^{k} \min(\eta + i, \rho), \qquad \nu_* = \#\{i \in \{1, \dots, k\} : \eta + i = \rho\}.$$

Then

$$2E_* = d_0(p, q, h), \qquad \nu_* = \mu_0(p, q, h) - 1.$$

If $p > 0$, there exist $0 < c \le C$ and $T_0 \ge 16^{k+1}$ such that for all $T \ge T_0$,

$$cT^{-d_0(p,q,h)/2}(\log T)^{\mu_0(p,q,h)-1} \le I_{k,\eta,\rho}(T) \le CT^{-d_0(p,q,h)/2}(\log T)^{\mu_0(p,q,h)-1}.$$

The theorem includes $q = 0$ or $h = 0$, for which $k = 0$ and the chamber integral is the empty product. The case $p = 0$ is not obtained by applying a theorem requiring $\rho > 0$; it is the separate absent residual-factor case.

Proof. For $0 \le \ell \le k$, define the prefix exponent

$$E_\ell = \sum_{i \le \ell} (\eta + i) + \sum_{i > \ell} \rho.$$

Its successive difference is $\eta + \ell - \rho$. After multiplication by 2, this is the checked integer increment

$$d - k - 1 + 2\ell - p.$$

Induction on ℓ, carried out after coercion to $\mathbb{R}$, gives

$$2E_\ell = P_\ell,$$

where P_ℓ is the checked recursive prefix cost. Since the spectral sequence is strictly increasing, E_* equals one of the prefix exponents: at a strict cut it is the corresponding prefix, and at resonance either adjacent prefix gives the same value. Conversely, the coordinatewise inequality

$\min(\eta + i, \rho) \le \eta + i$ and $\min(\eta + i, \rho) \le \rho$ shows $E_* \le E_\ell$ for every prefix. Thus the checked finite minimum of the P_ℓ gives $2E_* = d_0(p, q, h)$.

Equality $\eta + i = \rho$ is equivalent, after multiplication by 2, to vanishing of the same checked integer increment. The existing resonance theorem says that at most one such index exists and identifies its indicator with the residual resonance multiplicity. The checked identity $\mu_0 = 1 +$ resonance multiplicity yields $\nu_* = \mu_0 - 1$. Substituting these two identities into Lemma 19.5 proves the displayed two-sided estimate when $p > 0$. The empty-index case uses no subtraction or cancellation.

Provenance: Sneiderman, Proposition 8.14 and Appendix A; the formal proof uses the previously checked prefix-cost reversal and resonance arithmetic.

□

```
noncomputable section
open Real MeasureTheory Set
open scoped ENNReal BigOperators

namespace O77.Residual

/-- Prefix exponent in which the first `ell` ordered coordinates use their
small-scale exponents and all later coordinates use `rho`. -/
def prefixGaugeExponent {k : Nat} (eta rho : Real) (ell : Nat) : Real :=
  ∑ n ∈ Finset.range k, if n < ell then eta + (n : Real) + 1 else rho

lemma chamberExponent_eq_sum_range {k : Nat} (eta rho : Real) :
    chamberExponent (k := k) eta rho =
      ∑ n ∈ Finset.range k, min (eta + (n : Real) + 1) rho := by
  unfold chamberExponent
  simpa [spectralA] using
    (Fin.sum_univ_eq_sum_range
      (fun n : Nat => min (eta + (n : Real) + 1) rho) k)

lemma chamberExponent_le_prefixGaugeExponent {k : Nat} (eta rho : Real) (ell : Nat) :
    chamberExponent (k := k) eta rho ≤ prefixGaugeExponent (k := k) eta rho ell := by
  unfold prefixGaugeExponent
  rw [chamberExponent_eq_sum_range]
  apply Finset.sum_le_sum
  intro n hn
  by_cases h : n < ell
  · simp [h]
  · simp [h]

lemma chamberExponent_eq_prefixGaugeExponent_of_strictCut {k cut : Nat}
    {eta rho : Real} (_hcut : cut ≤ k)
    (hlo : ∀ i : Fin k, i.val < cut -> spectralA eta i < rho)
    (hhi : ∀ i : Fin k, cut ≤ i.val -> rho < spectralA eta i) :
    chamberExponent (k := k) eta rho = prefixGaugeExponent (k := k) eta rho cut := by
  unfold prefixGaugeExponent
  rw [chamberExponent_eq_sum_range]
  apply Finset.sum_congr rfl
  intro n hn
  have hnk : n < k := Finset.mem_range.mp hn
  let i : Fin k := ⟨n, ⟩hnk
  by_cases hnc : n < cut
  · have h := hlo i hnc
    have h' : eta + (n : Real) + 1 < rho := by
      simpa [i, spectralA] using h
    simp [hnc]
    exact h'.le
  · have hcn : cut ≤ n := Nat.le_of_not_gt hnc
    have h := hhi i hcn
```

```
    have h' : rho < eta + (n : Real) + 1 := by
      simpa [i, spectralA] using h
    simp [hnc]
    exact h'.le

lemma chamberExponent_eq_prefixGaugeExponent_of_resonance {k : Nat} {r : Fin k}
    {eta rho : Real}
    (hlo : ∀ i : Fin k, i < r -> spectralA eta i < rho)
    (heq : spectralA eta r = rho)
    (hhi : ∀ i : Fin k, r < i -> rho < spectralA eta i) :
    chamberExponent (k := k) eta rho =
      prefixGaugeExponent (k := k) eta rho r.val := by
  unfold prefixGaugeExponent
  rw [chamberExponent_eq_sum_range]
  apply Finset.sum_congr rfl
  intro n hn
  have hnk : n < k := Finset.mem_range.mp hn
  let i : Fin k := ⟨n, hnk⟩
  by_cases hnr : n < r.val
  · have hir : i < r := hnr
    have h := hlo i hir
    have h' : eta + (n : Real) + 1 < rho := by
      simpa [i, spectralA] using h
    simp [hnr]
    exact h'.le
  · have hrn : r.val ≤ n := Nat.le_of_not_gt hnr
    by_cases hne : n = r.val
    · subst n
      have heq' : eta + (r.val : Real) + 1 = rho := by
        simpa [spectralA] using heq
      simp [heq']
    · have hrn' : r.val < n := lt_of_le_of_ne hrn (Ne.symm hne)
      have hri : r < i := hrn'
      have h := hhi i hri
      have h' : rho < eta + (n : Real) + 1 := by
        simpa [i, spectralA] using h
      simp [hnr]
      exact h'.le

lemma prefixGaugeExponent_succ {k ell : Nat} {eta rho : Real}
    (hell : ell < k) :
    prefixGaugeExponent (k := k) eta rho (ell + 1) =
      prefixGaugeExponent (k := k) eta rho ell +
        (eta + (ell : Real) + 1 - rho) := by
  unfold prefixGaugeExponent
  calc
    ∑( n ∈ Finset.range k, if n < ell + 1 then eta + (n : Real) + 1 else rho) =
        ∑ n ∈ Finset.range k,
          ((if n < ell then eta + (n : Real) + 1 else rho) +
            if n = ell then (eta + (ell : Real) + 1 - rho) else 0) := by
      apply Finset.sum_congr rfl
      intro n hn
      by_cases hlt : n < ell
      · have hsucc : n < ell + 1 := by omega
        simp [hlt, hsucc, ne_of_lt hlt]
      · by_cases heq : n = ell
        · subst n
          simp
        · have hnot : ¬ n < ell + 1 := by omega
          simp [hlt, heq, hnot]
    _ = ∑( n ∈ Finset.range k, if n < ell then eta + (n : Real) + 1 else rho) +
        ∑ n ∈ Finset.range k,
```

```
          if n = ell then (eta + (ell : Real) + 1 - rho) else 0 := by
      rw [Finset.sum_add_distrib]
    _ = ∑( n ∈ Finset.range k, if n < ell then eta + (n : Real) + 1 else rho) +
        (eta + (ell : Real) + 1 - rho) := by
      have hellmem : ell ∈ Finset.range k := Finset.mem_range.mpr hell
      simp [hellmem]

/-- O77's real Laguerre parameter `eta=(d-k-1)/2`. -/
def residualSpectralEta (q h : Nat) : Real :=
  ((O77.Arithmetic.chamberD h q : Real) -
    (O77.Arithmetic.chamberK h q : Real) - 1) / 2

/-- O77's determinant exponent `rho=p/2`. -/
def residualSpectralRho (p : Nat) : Real := (p : Real) / 2

lemma residualSpectralEta_gt_neg_one (q h : Nat) :
    -1 < residualSpectralEta q h := by
  have hkd : O77.Arithmetic.chamberK h q ≤ O77.Arithmetic.chamberD h q := min_le_max
  have hkdR : (O77.Arithmetic.chamberK h q : Real) ≤
      O77.Arithmetic.chamberD h q := by exact_mod_cast hkd
  unfold residualSpectralEta
  linarith

lemma residualSpectralRho_pos {p : Nat} (hp : 0 < p) :
    0 < residualSpectralRho p := by
  unfold residualSpectralRho
  positivity

/-- Twice the difference between a spectral factor exponent and `rho` is
exactly the checked integer increment at the corresponding one-based index. -/
theorem two_mul_residualSpectral_difference_eq_increment
    (p q h : Nat) {i : Fin (O77.Arithmetic.chamberK h q)} :
    2 * (spectralA (residualSpectralEta q h) i - residualSpectralRho p) =
      (O77.Arithmetic.chamberTwiceIncrement p q h (i.val + 1) : Real) := by
  unfold spectralA residualSpectralEta residualSpectralRho
    O77.Arithmetic.chamberTwiceIncrement
  push_cast
  ring

/-- The real prefix exponent is exactly half the checked integer prefix cost. -/
theorem two_mul_prefixGaugeExponent_eq_chamberTwicePrefixExponent
    (p q h ell : Nat) (hell : ell ≤ O77.Arithmetic.chamberK h q) :
    2 * prefixGaugeExponent
        (k := O77.Arithmetic.chamberK h q)
        (residualSpectralEta q h) (residualSpectralRho p) ell =
      (O77.Arithmetic.chamberTwicePrefixExponent p q h ell : Real) := by
  induction ell with
  | zero =>
      unfold prefixGaugeExponent residualSpectralRho
      simp [O77.Arithmetic.chamberTwicePrefixExponent]
      ring
  | succ ell ih =>
      have hellt : ell < O77.Arithmetic.chamberK h q := by omega
      rw [prefixGaugeExponent_succ hellt]
      rw [O77.Arithmetic.chamberTwicePrefixExponent, Int.cast_add]
      rw ←[ ih (by omega)]
      have hinc := two_mul_residualSpectral_difference_eq_increment p q h
        (i := ⟨ell, ⟩hellt)
      simp only [spectralA] at hinc
      rw ←[ hinc]
      ring
```

```
/-- The real spectral exponent is half the residual minimum. This is the
coercion-safe analytic/arithmetic seam needed by O77. -/
theorem two_mul_residual_chamberExponent_eq_residualMin
    (p q h : Nat) :
    2 * chamberExponent
        (k := O77.Arithmetic.chamberK h q)
        (residualSpectralEta q h) (residualSpectralRho p) =
      (O77.residualMin p q h : Real) := by
  let k := O77.Arithmetic.chamberK h q
  let eta := residualSpectralEta q h
  let rho := residualSpectralRho p
  have hpattern := hasSpectralPattern k eta rho
  obtain ⟨ell, hell, ⟩hEq :
      ∃ ell : Nat, ell ≤ k ∧
        chamberExponent (k := k) eta rho = prefixGaugeExponent (k := k) eta rho ell := by
    rcases hpattern with hstrict | hres
    · rcases hstrict with ⟨cut, hcut, hlo, ⟩hhi
      exact ⟨cut, hcut,
        chamberExponent_eq_prefixGaugeExponent_of_strictCut hcut hlo ⟩hhi
    · rcases hres with ⟨r, hlo, heq, ⟩hhi
      exact ⟨r.val, r.isLt.le,
        chamberExponent_eq_prefixGaugeExponent_of_resonance hlo heq ⟩hhi
  let P : Nat -> Int := O77.Arithmetic.chamberTwicePrefixExponent p q h
  have hselected :
      2 * chamberExponent (k := k) eta rho = (P ell : Real) := by
    rw [hEq]
    simpa [P, k, eta, rho] using
      two_mul_prefixGaugeExponent_eq_chamberTwicePrefixExponent p q h ell hell
  have hleft :
      (O77.Arithmetic.minimumUpTo P k : Real) ≤
        2 * chamberExponent (k := k) eta rho := by
    have hle := O77.Arithmetic.minimumUpTo_le P hell
    calc
      (O77.Arithmetic.minimumUpTo P k : Real) ≤ (P ell : Real) := by exact_mod_cast hle
      _ = 2 * chamberExponent (k := k) eta rho := hselected.symm
  have hright :
      2 * chamberExponent (k := k) eta rho ≤
        (O77.Arithmetic.minimumUpTo P k : Real) := by
    rcases O77.Arithmetic.minimumUpTo_attained P k with ⟨j, hj, ⟩hval
    have hbase := chamberExponent_le_prefixGaugeExponent (k := k) eta rho j
    have hmul :
        2 * chamberExponent (k := k) eta rho ≤
          2 * prefixGaugeExponent (k := k) eta rho j :=
      mul_le_mul_of_nonneg_left hbase (by norm_num)
    calc
      2 * chamberExponent (k := k) eta rho ≤
          2 * prefixGaugeExponent (k := k) eta rho j := hmul
      _ = (P j : Real) := by
        simpa [P, k, eta, rho] using
          two_mul_prefixGaugeExponent_eq_chamberTwicePrefixExponent p q h j hj
      _ = (O77.Arithmetic.minimumUpTo P k : Real) := by rw [hval]
  have hminimum :
      (O77.Arithmetic.minimumUpTo P k : Real) =
        2 * chamberExponent (k := k) eta rho := le_antisymm hleft hright
  have hcore := O77.Arithmetic.chamberTwicePrefixMinimum_eq_residualMin_cast p q h
  have hrewrite :
      O77.Arithmetic.minimumUpTo P k =
        (O77.residualMin p q h : Int) := by
    simpa [P, k] using hcore
  rw ←[ hminimum, hrewrite]
  norm_num
```

```
/-- Equality of a real spectral exponent with `rho` is exactly the checked
one-based resonance predicate. -/
theorem residual_spectralA_eq_rho_iff_chamberResonant
    (p q h : Nat) (i : Fin (O77.Arithmetic.chamberK h q)) :
    spectralA (residualSpectralEta q h) i = residualSpectralRho p ↔
      O77.Arithmetic.chamberResonant p q h (i.val + 1) := by
  rw [O77.Arithmetic.chamberResonant_iff_increment_eq_zero]
  have hbound : i.val + 1 ≤ O77.Arithmetic.chamberK h q := by omega
  refine ⟨?_, ?⟩
  · intro heq
    refine ⟨by omega, hbound, ?⟩
    have hinc := two_mul_residualSpectral_difference_eq_increment p q h (i := i)
    rw [heq, sub_self, mul_zero] at hinc
    exact_mod_cast hinc.symm
  · rintro ⟨_, _, ⟩hzero
    have hinc := two_mul_residualSpectral_difference_eq_increment p q h (i := i)
    rw [hzero, Int.cast_zero] at hinc
    linarith

lemma residual_spectral_resonance_exists_iff (p q h : Nat) :
    ∃( i : Fin (O77.Arithmetic.chamberK h q),
        spectralA (residualSpectralEta q h) i = residualSpectralRho p) ↔
      ∃ j : Nat, O77.Arithmetic.chamberResonant p q h j := by
  constructor
  · rintro ⟨i, ⟩hi
    exact ⟨i.val + 1,
      (residual_spectralA_eq_rho_iff_chamberResonant p q h i).mp ⟩hi
  · rintro ⟨j, ⟩hj
    have hj' := (O77.Arithmetic.chamberResonant_iff_increment_eq_zero p q h j).mp hj
    let i : Fin (O77.Arithmetic.chamberK h q) := ⟨j - 1, by ⟩omega
    have hval : i.val + 1 = j := by dsimp [i]; omega
    refine ⟨i, ?⟩
    apply (residual_spectralA_eq_rho_iff_chamberResonant p q h i).mpr
    simpa [hval] using hj

lemma chamberResonanceCount_eq_one_iff {k : Nat} (eta rho : Real) :
    chamberResonanceCount (k := k) eta rho = 1 ↔
      ∃ i : Fin k, spectralA eta i = rho := by
  classical
  constructor
  · intro hcard
    unfold chamberResonanceCount at hcard
    have hnonempty : (Finset.univ.filter fun i : Fin k => spectralA eta i = rho).Nonempty :=
      Finset.card_pos.mp (by omega)
    rcases hnonempty with ⟨i, ⟩hi
    exact ⟨i, by simpa using ⟩hi
  · rintro ⟨i, ⟩hi
    have hpos : 0 < chamberResonanceCount (k := k) eta rho := by
      unfold chamberResonanceCount
      apply Finset.card_pos.mpr
      exact ⟨i, by simp [hi]⟩
    have hle := chamberResonanceCount_le_one (k := k) eta rho
    omega

/-- The number of resonant spectral factors is the checked residual resonance
indicator, hence one less than the residual multiplicity. -/
theorem residual_chamberResonanceCount_eq_resonanceMultiplicity
    (p q h : Nat) :
    chamberResonanceCount
        (k := O77.Arithmetic.chamberK h q)
        (residualSpectralEta q h) (residualSpectralRho p) =
      O77.Arithmetic.chamberResonanceMultiplicity p q h := by
```

```
  let k := O77.Arithmetic.chamberK h q
  let eta := residualSpectralEta q h
  let rho := residualSpectralRho p
  by_cases hex : ∃ i : Fin k, spectralA eta i = rho
  · have hcount : chamberResonanceCount (k := k) eta rho = 1 :=
      (chamberResonanceCount_eq_one_iff eta rho).mpr hex
    have hnat : ∃ j : Nat, O77.Arithmetic.chamberResonant p q h j := by
      simpa [k, eta, rho] using
        (residual_spectral_resonance_exists_iff p q h).mp (by simpa [k, eta, rho] using hex)
    have hcond := (O77.Arithmetic.exists_chamberResonant_iff p q h).mp hnat
    have hcand := (O77.Arithmetic.chamberResonanceIndex_spec p q h).mpr hcond
    simp [O77.Arithmetic.chamberResonanceMultiplicity, hcand, hcount, k, eta, rho]
  · have hcount : chamberResonanceCount (k := k) eta rho = 0 := by
      have hle := chamberResonanceCount_le_one (k := k) eta rho
      by_contra hne
      have hone : chamberResonanceCount (k := k) eta rho = 1 := by omega
      exact hex ((chamberResonanceCount_eq_one_iff eta rho).mp hone)
    have hnotnat : ¬ ∃ j : Nat, O77.Arithmetic.chamberResonant p q h j := by
      intro hj
      have : ∃ i : Fin (O77.Arithmetic.chamberK h q),
          spectralA (residualSpectralEta q h) i = residualSpectralRho p :=
        (residual_spectral_resonance_exists_iff p q h).mpr hj
      exact hex (by simpa [k, eta, rho] using this)
    have hnotcand : ¬ O77.Arithmetic.chamberResonant p q h
        (O77.Arithmetic.chamberResonanceIndex p q h) := by
      intro hc
      exact hnotnat ⟨_, ⟩hc
    simp [O77.Arithmetic.chamberResonanceMultiplicity, hnotcand, hcount, k, eta, rho]

/-- The generic ordered-chamber power-log theorem specializes exactly to the
O77 residual exponent `₀d/2` and logarithmic degree μ₀`-1`. -/
theorem residual_laguerreChamber_powerLog_bounds
    {p q h : Nat} (hp : 0 < p) :
    ∃ c C T0 : Real,
      0 < c ∧ c ≤ C ∧
      chamberBoxThreshold (O77.Arithmetic.chamberK h q) ≤ T0 ∧
      ∀ T : Real, T0 ≤ T ->
        c * (T ^ (-(O77.residualMin p q h : Real) / 2) *
          (Real.log T) ^ (O77.residualMultiplicity p q h - 1)) ≤
            laguerreChamberIntegral
              (k := O77.Arithmetic.chamberK h q)
              (residualSpectralEta q h) (residualSpectralRho p) T ∧
        laguerreChamberIntegral
              (k := O77.Arithmetic.chamberK h q)
              (residualSpectralEta q h) (residualSpectralRho p) T ≤
          C * (T ^ (-(O77.residualMin p q h : Real) / 2) *
            (Real.log T) ^ (O77.residualMultiplicity p q h - 1)) := by
  rcases laguerreChamber_powerLog_bounds
      (k := O77.Arithmetic.chamberK h q)
      (residualSpectralEta_gt_neg_one q h) (residualSpectralRho_pos hp) with
    ⟨c, C, T0, hc, hcC, hT0, ⟩hbounds
  refine ⟨c, C, T0, hc, hcC, hT0, ?⟩_
  intro T hT
  have hexp := two_mul_residual_chamberExponent_eq_residualMin p q h
  have hres := residual_chamberResonanceCount_eq_resonanceMultiplicity p q h
  have hmult := O77.Arithmetic.chamberMultiplicity_eq_one_add_resonanceMultiplicity p q h
  have hdegree :
      O77.residualMultiplicity p q h - 1 =
        chamberResonanceCount
          (k := O77.Arithmetic.chamberK h q)
          (residualSpectralEta q h) (residualSpectralRho p) := by
    rw ←[ O77.Arithmetic.chamberMultiplicity_eq_residualMultiplicity,
```

```
      hmult, Nat.add_sub_cancel_left, ← hres]
  have hexponent :
      -chamberExponent
          (k := O77.Arithmetic.chamberK h q)
          (residualSpectralEta q h) (residualSpectralRho p) =
        -(O77.residualMin p q h : Real) / 2 := by
    linarith
  simpa [chamberPowerLogGauge, hexponent, hdegree] using hbounds T hT

-- Exact seam regressions.
example : residualSpectralEta 1 1 = -(1 : Real) / 2 := by
  norm_num [residualSpectralEta, O77.Arithmetic.chamberD, O77.Arithmetic.chamberK]

example : residualSpectralRho 2 = 1 := by norm_num [residualSpectralRho]

example : chamberResonanceCount (k := 3) (residualSpectralEta 3 4)
    (residualSpectralRho 2) = 1 := by
  calc
    chamberResonanceCount (k := 3) (residualSpectralEta 3 4)
        (residualSpectralRho 2) =
      O77.Arithmetic.chamberResonanceMultiplicity 2 3 4 :=
        residual_chamberResonanceCount_eq_resonanceMultiplicity 2 3 4
    _ = 1 := by decide

example : 2 * chamberExponent (k := 3) (residualSpectralEta 3 4)
    (residualSpectralRho 2) = 6 := by
  calc
    2 * chamberExponent (k := 3) (residualSpectralEta 3 4)
        (residualSpectralRho 2) = (O77.residualMin 2 3 4 : Real) :=
      two_mul_residual_chamberExponent_eq_residualMin 2 3 4
    _ = 6 := by exact_mod_cast (show O77.residualMin 2 3 4 = 6 by decide)

end O77.Residual
end
```

Residual branch assembly, with interfaces visible. The chamber upper bound, separated-box lower bound, finite-product integration, and exact spectral exponent/multiplicity specialization are checked in Lemmas 19.5 and 20.14. The subsequent sections instantiate the fixed-Z Gaussian integral, smaller-Gram determinant, almost-everywhere chamber partition, exact $k!$ factor, conditional Wishart reduction, Gaussian matrix-Laplace estimate, exact dilation law, dyadic shell summation, and Laplace-to-sublevel transfer. The resulting residual-origin pair is checked in all dimensions, conditional only in the positive-dimensional branch on the explicit real-Wishart premise. The geometric application is supplied by the analytic-chart, coordinate-measure, and all-point classification modules.

21 Abstract residual interfaces: reusable conditional assembly

For audit purposes, this section records the original five finite assembly interfaces: `O77.Residual.gaussian_rows`, `O77.Residual.gram_determinant_spectral`, `O77.Residual.symmetric_integral_chambers`, `O77.Residual.gaussianLaplace_powerLog_bounds_of_wishart`, and `O77.Residual.homogeneous_gaussianLaplace_comparable_ball`. The first three names are abstract interfaces for Gaussian, spectral, and chamber facts; the last two encode the Wishart and homogeneous-localization assembly. The contract code proves only finite composition of the stated fields. The concrete residual sections following this one prove the matrix, measure, chamber, and localization identities; the Gram and spectral chapters prove the Wishart proposition. The structures distinguish finite assembly from its analytic ingredients.

Intermediate origin-pair declarations in this module retain their explicit Wishart argument for reuse; the completed answer supplies that argument with `O77.GlobalSpectralIntegral.realWishartEigenvalueFormula`.

21.1 Gaussian integration row by row

Let

$$Y \in \mathbb{R}^{p\times h}, \qquad Z \in \mathbb{R}^{h\times q}, \qquad T \geq 0,$$

and write the rows of Y as $y_1, \dots, y_p \in \mathbb{R}^h$. Put

$$M_Z(T) = I_h + TZZ^{\mathsf{T}}.$$

Then $M_Z(T)$ is positive definite, since for every $x \in \mathbb{R}^h$,

$$xM_Z(T)x^{\mathsf{T}} = \|x\|^2 + T\|xZ\|^2,$$

which is strictly positive when $x \neq 0$. Moreover,

$$T\|YZ\|_F^2 + \|Y\|_F^2 = \sum_{i=1}^{p} y_i M_Z(T) y_i^{\mathsf{T}}.$$

Tonelli's theorem applies because the integrand is nonnegative. If one inserts the standard positive-definite Gaussian formula

$$\int_{\mathbb{R}^h} e^{-xMx^{\mathsf{T}}}\,dx = \pi^{h/2}\det(M)^{-1/2},$$

one obtains

$$\int_{\mathbb{R}^{p\times h}} e^{-T\|YZ\|_F^2 - \|Y\|_F^2}\,dY = \pi^{ph/2}\det(M_Z(T))^{-p/2}. \tag{21}$$

For $p = 0$ both sides are the empty product 1. The checked theorem `gaussian_rows` below is the exact finite-row assembly: it takes the Tonelli factorization and the one-row formula as explicit premises and proves the matrix-row product conclusion. The concrete Gaussian section below discharges those premises in the exact coordinate measure, while retaining the conditional assembly theorem under an explicit renamed declaration.

Proposition 21.1 (Gaussian rows, relative to the one-row quadratic Gaussian formula)**.** *Assume the Tonelli factorization into p row integrals and the displayed one-row formula for the positive-definite matrix $M_Z(T)$. Then (21) holds.*

Proof. The exponent is the sum of the p row quadratic forms. Tonelli converts the integral of the product into the product of the row integrals. Replacing every row integral by the same positive-definite Gaussian value gives the p-th power, namely $\pi^{ph/2}\det(M_Z(T))^{-p/2}$. No determinant sign ambiguity occurs because $M_Z(T)$ is positive definite. □

21.2 The Gram determinant and the nonzero spectrum

Put $k = \min(h, q)$. If $h \leq q$, let $G_Z = ZZ^{\mathsf{T}} \in \mathbb{R}^{h\times h}$; if $q < h$, let $G_Z = Z^{\mathsf{T}}Z \in \mathbb{R}^{q\times q}$. In either case G_Z is a $k \times k$ positive-semidefinite Gram matrix. Let $s_1, \dots, s_k \geq 0$ be its eigenvalues, with repetitions.

Proposition 21.2 (Gram determinant in the small spectral variables)**.** *For every real T,*

$$\det(I_h + TZZ^{\mathsf{T}}) = \prod_{i=1}^{k}(1 + Ts_i).$$

Proof. If $h \le q$, apply the real spectral theorem directly to ZZ^{T} and use invariance of the determinant under orthogonal conjugation. If $q < h$, first use Sylvester's determinant identity

$$\det(I_h + TZZ^{\mathsf{T}}) = \det(I_q + TZ^{\mathsf{T}}Z),$$

and then apply the spectral theorem to $Z^{\mathsf{T}}Z$. The two cases use all eigenvalues of the smaller Gram matrix, so no informal deletion of zero eigenvalues is required. At $k = 0$ both determinants and the product are 1. □

The checked declaration `gram_determinant_spectral` records the exact logical seam: Sylvester's identity and the determinant-as-product-of-eigenvalues statement are separate premises, and the conclusion is their composition. The subsequent concrete section supplies the matrix-specific instantiation with the correct dimensions, both cardinality branches, nonnegative eigenvalues, and factor lower bounds at nonnegative scale.

21.3 Symmetric orthant integrals and ordered chambers

Let $F : (0,\infty)^k \to [0,\infty]$ be measurable and invariant under coordinate permutations. Assume that the union of the tie hyperplanes $\{s_i = s_j\}$ is null. The complement is the disjoint union, indexed by S_k, of the strict order chambers. Coordinate permutations preserve Lebesgue measure, and symmetry of F makes all chamber integrals equal.

Proposition 21.3 (Symmetric integral as a factorial times one chamber)**.** *Under the preceding hypotheses,*

$$\int_{(0,\infty)^k} F(s)\,ds = k! \int_{0<s_1<\cdots<s_k} F(s)\,ds.$$

The strict chamber may be replaced by the weak chamber. For the Laguerre density this replacement is especially harmless because the Vandermonde factor itself vanishes on every tie hyperplane.

Proof. Delete the finite union of tie hyperplanes; this does not change the integral. Every remaining point has a unique sorting permutation, so the remaining orthant is the disjoint union of $k!$ permuted strict chambers. Finite additivity of the nonnegative integral gives the sum of the chamber integrals. For every permutation, change coordinates by its permutation matrix. The Jacobian has absolute determinant 1, and the integrand is unchanged. Thus every term equals the integral on the reference chamber. When $k = 0$, there is one empty permutation and the singleton chamber has mass one. Both sides therefore equal $F(())$; the value is 1 only when the integrand at the empty tuple is 1, as it is for the Laguerre product. □

The checked theorem `symmetric_integral_chambers` proves the finite-additive part from an explicit partition equality and equality of the chamber contributions. The concrete chamber section instantiates the almost-everywhere partition and the exact measure-preserving permutation facts, including the empty and two-coordinate controls.

21.4 Conditional Gaussian Laplace order from the Wishart adapter

Assume the exact real Wishart/Gram-eigenvalue formula of Proposition 19, with a positive normalization constant. Apply it to

$$\psi_T(s_1,\dots,s_k) = \prod_{i=1}^{k} (1 + Ts_i)^{-p/2}.$$

Propositions 21.1–21.3 give

$$J_G(T) = K_{p,h,q}\, I_{k,\eta,p/2}(T), \qquad K_{p,h,q} > 0,$$

where

$$\eta = \frac{\max(h,q) - \min(h,q) - 1}{2}$$

and $I_{k,\eta,p/2}$ is the ordered Laguerre chamber integral already treated in the preceding section. Multiplication by a fixed positive constant changes neither exponent nor logarithmic power. The checked declaration `gaussianLaplace_powerLog_bounds_of_wishart` formalizes precisely this last transfer from a positive exact reduction and a two-sided chamber estimate.

Consequently, conditional on the exact Wishart measure identity,

$$J_G(T) \asymp T^{-d_0(p,q,h)/2} (\log T)^{\mu_0(p,q,h)-1}$$

for positive residual dimensions. The quadratic-form Gaussian integral, smaller-Gram spectral determinant, chamber symmetrization, and outer matrix-integral assembly are concrete declarations. The intermediate theorem keeps the exact Wishart proposition as an ordinary argument. The terminal spectral module proves that proposition and hence discharges this argument in the final answer.

21.5 Homogeneous Gaussian localization to a fixed ball

Let $D = ph + hq$ and

$$K(Y,Z) = \|YZ\|_F^2.$$

Then $K(cY, cZ) = c^4 K(Y,Z)$. For $R > 0$ define

$$J_G(T) = \int_{\mathbb{R}^D} e^{-TK(w)} e^{-\|w\|^2}\, dw, \qquad J_R(T) = \int_{\|w\|<R} e^{-TK(w)}\, dw.$$

Proposition 21.4 (Homogeneous Gaussian Laplace integral is comparable with a fixed ball)**.** *For every $R > 0$ there are constants $0 < c_R \le C_R < \infty$, independent of $T \ge 0$, such that*

$$c_R J_R(T) \le J_G(T) \le C_R J_R(T).$$

One may take $c_R = e^{-R^2}$ and

$$C_R = 1 + \sum_{n=0}^{\infty} 2^{(n+1)D} e^{-4^n R^2}.$$

Proof. On the ball of radius R, the Gaussian density is at least e^{-R^2}, proving the lower bound. For the upper bound, retain the contribution of the ball and decompose its complement into shells

$$S_n = \{2^n R \le \|w\| < 2^{n+1} R\}, \qquad n \ge 0.$$

On S_n the Gaussian density is at most $e^{-4^n R^2}$. Substitute $w = 2^{n+1}u$. The Jacobian is $2^{(n+1)D}$, the rescaled shell lies in the ball of radius R, and quartic homogeneity gives

$$K(2^{n+1}u) = 2^{4(n+1)} K(u).$$

Since $T \ge 0$ and $K \ge 0$,

$$e^{-T 2^{4(n+1)} K(u)} \le e^{-TK(u)}.$$

The n-th shell contribution is therefore bounded by

$$2^{(n+1)D} e^{-4^n R^2} J_R(T).$$

The displayed numerical series converges because its ratio tends to zero; summing the shell bounds gives the upper estimate. The constants are selected after R and before T. □

The checked theorem `homogeneous_gaussianLaplace_comparable_ball` is the finite assembly from an inner-ball estimate and a summed shell estimate. The later concrete localization sections instantiate those premises using finite-dimensional volume scaling, quartic homogeneity, the exact shell partition, and summability of the displayed series. This early interface is retained only to expose the logical seam; the concrete theorem, not this contract, is the residual result used downstream.

21.6 Checked contract layer for the five bridge steps

The following historical block is deliberately a contract layer. It certifies finite logical assembly while keeping the Gaussian formula, spectral identity, chamber partition, Wishart formula, and homogeneous shell estimate visible as premises. All deterministic premises are instantiated in the concrete sections below, with the Wishart formula proved in the later Gram and spectral chapters. This block is a reusable implication from specified ingredients; its proof alone is not the proof of those ingredients.

```
namespace O77.Residual

/-- A concrete two-sided order at large parameter values. -/
def AtTopTwoSidedOrder (F G : ℝ → ℝ) : Prop :=
  ∃ c C T0 : ℝ, 0 < c ∧ c ≤ C ∧
    ∀ T : ℝ, T0 ≤ T → c * G T ≤ F T ∧ F T ≤ C * G T

/-- Multiplication by a fixed positive constant preserves a two-sided large-parameter order. -/
theorem atTopTwoSidedOrder_const_mul {F G : ℝ → ℝ} {K : ℝ}
    (hK : 0 < K) (h : AtTopTwoSidedOrder F G) :
    AtTopTwoSidedOrder (fun T => K * F T) G := by
  rcases h with ⟨c, C, T0, hc, hcC, ⟩hFC
  refine ⟨K * c, K * C, T0, mul_pos hK hc,
    mul_le_mul_of_nonneg_left hcC hK.le, ?⟩_
  intro T hT
  rcases hFC T hT with ⟨hl, ⟩hu
  constructor
  · calc
      (K * c) * G T = K * (c * G T) := by ring
      _ ≤ K * F T := mul_le_mul_of_nonneg_left hl hK.le
  · calc
      K * F T ≤ K * (C * G T) := mul_le_mul_of_nonneg_left hu hK.le
      _ = (K * C) * G T := by ring

/-- Exact data needed to assemble independent equal row integrals. -/
structure GaussianRowsContract (p : ℕ) where
  matrixIntegral : ℝ
  rowIntegral : Fin p → ℝ
  rowFactor : Fin p → ℝ
  tonelliRows : matrixIntegral = ∏ i, rowIntegral i
  oneRow : ∀ i, rowIntegral i = rowFactor i

/-- The finite row-product conclusion after Tonelli and the one-row Gaussian formula. -/
theorem gaussian_rows {p : ℕ} (D : GaussianRowsContract p) :
    D.matrixIntegral = ∏ i, D.rowFactor i := by
  rw [D.tonelliRows]
  apply Finset.prod_congr rfl
  intro i hi
  exact D.oneRow i

/-- Exact algebraic data separating Sylvester's identity from the small Gram spectral formula. -/
structure GramDeterminantContract (k : ℕ) where
  largeDet : ℝ
  smallDet : ℝ
```

```
  T : ℝ
  spectrum : Fin k → ℝ
  sylvester : largeDet = smallDet
  smallSpectral : smallDet = ∏ i, (1 + T * spectrum i)

/-- Composition of Sylvester's identity with the determinant spectral formula. -/
theorem gram_determinant_spectral {k : ℕ} (D : GramDeterminantContract k) :
    D.largeDet = ∏ i, (1 + D.T * D.spectrum i) :=
  D.sylvester.trans D.smallSpectral

/-- Finite data remaining after the null tie set has been removed from a symmetric integral. -/
structure SymmetricChamberContract ι( : Type*) [Fintype ι] where
  fullIntegral : ℝ
  chamberIntegral : ℝ
  pieceIntegral : ι → ℝ
  finitePartition : fullIntegral = ∑ i, pieceIntegral i
  symmetricPieces : ∀ i, pieceIntegral i = chamberIntegral

/-- A symmetric finite chamber partition contributes the cardinality times one chamber. -/
theorem symmetric_integral_chambers ι{ : Type*} [Fintype ι]
    (D : SymmetricChamberContract ι) :
    D.fullIntegral = (Fintype.card ι : ℝ) * D.chamberIntegral := by
  classical
  rw [D.finitePartition]
  calc
    ∑( i, D.pieceIntegral i) = ∑ _i : ι, D.chamberIntegral := by
      apply Finset.sum_congr rfl
      intro i hi
      exact D.symmetricPieces i
    _ = (Fintype.card ι : ℝ) * D.chamberIntegral := by simp

/-- An exact positive Wishart reduction, without any asymptotic conclusion stored in the data. -/
structure WishartReduction (gaussianLaplace chamberIntegral : ℝ → ℝ) where
  constant : ℝ
  constant_pos : 0 < constant
  identity : ∀ T, gaussianLaplace T = constant * chamberIntegral T

/-- A positive exact Wishart reduction transports the checked chamber power--log order. -/
theorem gaussianLaplace_powerLog_bounds_of_wishart
    {gaussianLaplace chamberIntegral gauge : ℝ → ℝ}
    (W : WishartReduction gaussianLaplace chamberIntegral)
    (hchamber : AtTopTwoSidedOrder chamberIntegral gauge) :
    AtTopTwoSidedOrder gaussianLaplace gauge := by
  have hmul := atTopTwoSidedOrder_const_mul W.constant_pos hchamber
  have hfun : gaussianLaplace = fun T => W.constant * chamberIntegral T := by
    funext T
    exact W.identity T
  rw [hfun]
  exact hmul

/-- Algebraic data produced after the homogeneous dyadic-shell estimate has been summed. -/
structure HomogeneousGaussianShellContract
    (gaussianLaplace ballLaplace : ℝ → ℝ) where
  inner : ℝ → ℝ
  outer : ℝ → ℝ
  lowerConstant : ℝ
  shellConstant : ℝ
  lowerConstant_pos : 0 < lowerConstant
  shellConstant_nonneg : 0 ≤ shellConstant
  ball_nonneg : ∀ T, 0 ≤ ballLaplace T
  outer_nonneg : ∀ T, 0 ≤ outer T
  decomposition : ∀ T, gaussianLaplace T = inner T + outer T
```

```
  lower_inner : ∀ T, lowerConstant * ballLaplace T ≤ inner T
  upper_inner : ∀ T, inner T ≤ ballLaplace T
  upper_outer : ∀ T, outer T ≤ shellConstant * ballLaplace T

/-- The fixed-ball and summed-shell estimates give uniform two-sided comparability. -/
theorem homogeneous_gaussianLaplace_comparable_ball
    {gaussianLaplace ballLaplace : ℝ → ℝ}
    (D : HomogeneousGaussianShellContract gaussianLaplace ballLaplace) :
    ∃ c C : ℝ, 0 < c ∧ c ≤ C ∧
      ∀ T, c * ballLaplace T ≤ gaussianLaplace T ∧
        gaussianLaplace T ≤ C * ballLaplace T := by
  let C : ℝ := 1 + D.shellConstant
  let c : ℝ := min D.lowerConstant C
  have hC : 0 < C := by
    dsimp [C]
    linarith [D.shellConstant_nonneg]
  have hc : 0 < c := by
    dsimp [c]
    exact lt_min D.lowerConstant_pos hC
  refine ⟨c, C, hc, min_le_right _ _, ?_⟩
  intro T
  constructor
  · calc
      c * ballLaplace T ≤ D.lowerConstant * ballLaplace T :=
        mul_le_mul_of_nonneg_right (min_le_left _ _) (D.ball_nonneg T)
      _ ≤ D.inner T := D.lower_inner T
      _ ≤ D.inner T + D.outer T := le_add_of_nonneg_right (D.outer_nonneg T)
      _ = gaussianLaplace T := (D.decomposition T).symm
  · rw [D.decomposition T]
    calc
      D.inner T + D.outer T ≤
          ballLaplace T + D.shellConstant * ballLaplace T :=
        add_le_add (D.upper_inner T) (D.upper_outer T)
      _ = C * ballLaplace T := by
        dsimp [C]
        ring

/-- A zero reduction constant cannot transport a positive lower order. -/
theorem positive_reduction_constant_is_necessary :
    ¬ AtTopTwoSidedOrder (fun _ : ℝ => 0) (fun _ : ℝ => 1) := by
  intro h
  rcases h with ⟨c, C, T0, hc, hcC, ⟩hbounds
  have hl := (hbounds T0 le_rfl).1
  norm_num at hl
  linarith

/-- Two equal chamber contributions sum to two copies, not one. -/
theorem factorial_factor_is_visible :
    ∑ _i : Bool, (1 : ℝ)) = 2 := by norm_num

end O77.Residual
```

Corresponding Lean declaration(s): `O77.Residual.AtTopTwoSidedOrder`.

Corresponding Lean declaration(s): `O77.Residual.atTopTwoSidedOrder_const_mul`.

Corresponding Lean declaration(s): `O77.Residual.GaussianRowsContract`.

Corresponding Lean declaration(s): `O77.Residual.gaussian_rows`.

Corresponding Lean declaration(s): `O77.Residual.GramDeterminantContract`.

Corresponding Lean declaration(s): `O77.Residual.gram_determinant_spectral`.

Corresponding Lean declaration(s): `O77.Residual.SymmetricChamberContract`.

Corresponding Lean declaration(s): `O77.Residual.symmetric_integral_chambers`.

Corresponding Lean declaration(s): `O77.Residual.WishartReduction`.

Corresponding Lean declaration(s): `O77.Residual.gaussianLaplace_powerLog_bounds_of_wishart`.

Corresponding Lean declaration(s): `O77.Residual.HomogeneousGaussianShellContract`.

Corresponding Lean declaration(s): `O77.Residual.homogeneous_gaussianLaplace_comparable_ball`.

Corresponding Lean declaration(s): `O77.Residual.positive_reduction_constant_is_necessary`.

Corresponding Lean declaration(s): `O77.Residual.factorial_factor_is_visible`.

Status of the abstract contract layer. The declarations above remain valid finite assembly lemmas, but they are no longer the strongest residual results in the document. The concrete sections that follow instantiate the row-energy identity, positive-definite Gaussian integral, smaller-Gram determinant, almost-everywhere chamber partition, quartic dilation, shell summation, and Laplace-to-sublevel transfer in the actual matrix and measure spaces. On the preferred route, the only uninhabited analytic proposition is the real-Wishart eigenvalue transformation. The contract structures are retained solely as an audit trail and must not be cited as the concrete residual theorem.

22 Concrete Gaussian rows and Sylvester identity

22.1 Explicit Frobenius square and row energy

For finite index types p, h, q, define the squared Frobenius norm directly by

$$\|A\|^2_{F,\mathrm{entry}} = \sum_i \sum_j A_{ij}^2.$$

This avoids importing any claim that Mathlib's default norm on a function-valued matrix is the Frobenius norm required by O77. For $Y \in \mathbb{R}^{p\times h}$ and $Z \in \mathbb{R}^{h\times q}$, put

$$M_Z(T) = I_h + TZZ^{\mathsf{T}}, \qquad Q_{T,Z}(y) = yM_Z(T)y^{\mathsf{T}}.$$

Proposition 22.1 (Concrete row identity and positivity)**.** *For every real T,*

$$T\|YZ\|^2_{F,\mathrm{entry}} + \|Y\|^2_{F,\mathrm{entry}} = \sum_{i\in p} Q_{T,Z}(Y_i).$$

If $T \geq 0$, then $M_Z(T)$ is positive definite, including when h is empty.

Proof. The ith row of YZ is the vector–matrix product Y_iZ. Associativity of matrix–vector multiplication and the dot-product identity give

$$\langle Y_iZ, Y_iZ\rangle = \langle Y_i, (ZZ^{\mathsf{T}})Y_i\rangle.$$

Summing over rows and distributing the scalar T yields the stated equality. Over $\mathbb{R}$, conjugate transpose equals transpose. Hence $ZZ^{\mathsf{T}} = ZZ^*$ is positive semidefinite by Mathlib's Gram theorem. Nonnegative scalar multiplication preserves positive semidefiniteness, and adding the positive-definite identity matrix gives the result. No nonemptiness of h is needed: positive definiteness is vacuous on the unique zero vector when $h = 0$. □

The negative control in the code is load-bearing: when $h = q = 1$, $Z = 1$, and $T = -1$, the shifted Gram matrix is zero and is not positive definite. Thus the hypothesis $T \geq 0$ cannot be deleted. A separate 2×1 by 1×3 specialization checks that the row identity is genuinely rectangular rather than accidentally square.

22.2 Actual coordinate measures and finite-row integration

Let μ_h be the finite product of one-dimensional Lebesgue measure on the coordinate function space $h \to \mathbb{R}$, and let $\mu_{p,h} = \prod_{i\in p} \mu_h$ on the function-valued matrix space $\mathrm{Matrix}(p, h, \mathbb{R})$. These are the actual measures in the following declarations; there is no numeric placeholder for an integral.

The row identity rewrites the nonnegative Gaussian integrand as

$$\exp\bigl[-T\|YZ\|^2_F - \|Y\|^2_F\bigr] = \prod_{i\in p} e^{-Q_{T,Z}(Y_i)}.$$

Mathlib's finite-product integral theorem gives the product of row integrals. The checked `gaussian_rows_tonelli` theorem also assumes the row function is integrable and returns both integrability of the matrix integrand and the integral identity. This packaging is intentional: Mathlib's real Bochner integral has a default value on nonintegrable functions, and that convention must not be confused with a finite Gaussian integral.

For a positive-definite $h \times h$ matrix M, set

$$\mathcal{G}_h(M) = \left(\frac{\pi^{\#h}}{\det M}\right)^{1/2}.$$

The strongest theorem in this concrete Gaussian-row section has the following exact mathematical content.

Proposition 22.2 (Concrete Gaussian rows relative to one standard adapter)**.** *Assume, for the fixed finite row index type h, that every positive-definite M satisfies*

$$e^{-\langle y,My\rangle} \in L^1(\mu_h), \qquad \int e^{-\langle y,My\rangle}\, d\mu_h(y) = \mathcal{G}_h(M).$$

Then for every finite row type p, every $T \geq 0$, and every $Z \in \mathbb{R}^{h\times q}$, the matrix integrand is integrable and

$$\int_{\mathbb{R}^{p\times h}} e^{-T\|YZ\|_F^2 - \|Y\|_F^2}\, dY = \mathcal{G}_h(M_Z(T))^{\#p}.$$

Proof. Proposition 22.1 makes $M_Z(T)$ positive definite. Apply the assumed one-row theorem to this matrix, then apply the integrability-aware finite-row factorization. Every row integral is the same constant, so the finite product is its $\#p$th power. The assumption is an ordinary theorem parameter in Lean and remains visible in both `gaussian_rows_integrable_of_quadratic_gaussian` and `gaussian_rows_instantiated_of_quadratic_gaussian`; it is not stored in a contract structure and does not mention the residual power–log conclusion. □

22.3 Dimensions zero and one

The pinned library contains the isotropic finite-dimensional Gaussian formula

$$\int_V e^{-b\|v\|^2}\, dv = (\pi/b)^{\dim(V)/2} \qquad (b > 0).$$

For $h = 1$, a positive-definite matrix is the scalar $a = M_{00} > 0$, so the general quadratic form is ay^2. The code reduces to the library theorem, proves integrability separately, and identifies the determinant normalization exactly.

Proposition 22.3 (Unconditional hidden-dimension-one row integration)**.** *For $h = 1$, Proposition 22.2 holds without an external theorem parameter. It also holds for zero rows as the empty product* 1*. For $h = 0$, the one-row quadratic Gaussian integral is $1 = \mathcal{G}_0(M)$.*

Proof. In dimension one, expand the dot product and matrix–vector product to obtain $\langle y, My\rangle = M_{00}y_0^2$. Positive definiteness gives $M_{00} > 0$, so the scalar Gaussian formula applies. Product Lebesgue measure over the singleton index is the one-dimensional measure, and the determinant of a 1×1 matrix is its unique entry. When the index type is empty, the function space and matrix space are singletons, the product measure is the corresponding Dirac measure of total mass one, the determinant and empty product are one, and the exponent is zero. □

22.4 Sylvester in the exact Gram dimensions

Proposition 22.4 (Instantiated rectangular determinant identity)**.** *For every real T and every $Z \in \mathbb{R}^{h\times q}$,*

$$\det(I_h + TZZ^{\mathsf{T}}) = \det(I_q + TZ^{\mathsf{T}}Z).$$

Proof. Apply Mathlib's `Matrix.det_one_add_mul_comm` to the rectangular matrices Z and TZ^{T}, then normalize scalar multiplication through matrix multiplication. This proves the exact Sylvester seam. It does not yet prove that the determinant on the smaller side is $\prod_i(1 + Ts_i)$; that remaining statement requires the real symmetric spectral theorem in the chosen representation and nonnegativity of the Gram eigenvalues. □

22.5 Compiled concrete development

The following code was compiled first as a standalone Mathlib file with only the six direct imports listed in Section 1, and then in the complete document-order stream.

```
set_option autoImplicit false

noncomputable section

namespace O77.Residual

open scoped BigOperators

/-- Explicit squared Frobenius norm, independent of matrix norm instances. -/
def frobeniusSq {m n : Type*} [Fintype m] [Fintype n]
    (A : Matrix m n ℝ) : ℝ :=
  ∑ i, ∑ j, (A i j) ^ 2

lemma frobeniusSq_eq_sum_row_dot {m n : Type*} [Fintype m] [Fintype n]
    (A : Matrix m n ℝ) :
    frobeniusSq A = ∑ i, dotProduct (A i) (A i) := by
  simp [frobeniusSq, dotProduct, pow_two]

lemma row_mul_eq_vecMul {p h q : Type*} [Fintype h]
    (Y : Matrix p h ℝ) (Z : Matrix h q ℝ) (i : p) :
    (Y * Z) i = Matrix.vecMul (Y i) Z := by
  funext j
  simp [Matrix.mul_apply, Matrix.vecMul, dotProduct]

lemma transpose_mulVec_eq_vecMul {h q : Type*} [Fintype h] [Fintype q]
    (Z : Matrix h q ℝ) (y : h → ℝ) :
    Z.transpose.mulVec y = Matrix.vecMul y Z := by
  simpa using (Matrix.vecMul_transpose Z.transpose y).symm

lemma row_mul_dot_self_eq_gram {p h q : Type*} [Fintype h] [Fintype q]
    (Y : Matrix p h ℝ) (Z : Matrix h q ℝ) (i : p) :
    dotProduct ((Y * Z) i) ((Y * Z) i) =
      dotProduct (Y i) ((Z * Z.transpose).mulVec (Y i)) := by
  rw [row_mul_eq_vecMul]
  rw ←[ Matrix.mulVec_mulVec (Y i) Z Z.transpose]
  rw [Matrix.dotProduct_mulVec]
  rw [transpose_mulVec_eq_vecMul]

lemma frobeniusSq_mul_eq_sum_row_gram {p h q : Type*}
    [Fintype p] [Fintype h] [Fintype q]
    (Y : Matrix p h ℝ) (Z : Matrix h q ℝ) :
    frobeniusSq (Y * Z) =
      ∑ i, dotProduct (Y i) ((Z * Z.transpose).mulVec (Y i)) := by
  rw [frobeniusSq_eq_sum_row_dot]
  apply Finset.sum_congr rfl
  intro i hi
  exact row_mul_dot_self_eq_gram Y Z i

lemma dotProduct_gramShift_mulVec {h q : Type*}
    [Fintype h] [DecidableEq h] [Fintype q]
    (T : ℝ) (Z : Matrix h q ℝ) (y : h → ℝ) :
    dotProduct y ((1 + T • (Z * Z.transpose)).mulVec y) =
      dotProduct y y + T * dotProduct y ((Z * Z.transpose).mulVec y) := by
  rw [Matrix.add_mulVec, Matrix.one_mulVec, Matrix.smul_mulVec]
  unfold dotProduct
  simp only [Pi.add_apply, Pi.smul_apply, smul_eq_mul]
  calc
    ∑( x, y x * (y x + T * (Z * Z.transpose).mulVec y x)) =
```

```
        ∑ x, (y x * y x + T * (y x * (Z * Z.transpose).mulVec y x)) := by
      apply Finset.sum_congr rfl
      intro x hx
      ring
    _ = ∑( x, y x * y x) +
        ∑ x, T * (y x * (Z * Z.transpose).mulVec y x) := by
      rw [Finset.sum_add_distrib]
    _ = ∑( x, y x * y x) +
        T * ∑ x, y x * (Z * Z.transpose).mulVec y x := by
      rw [Finset.mul_sum]

/-- Concrete row-energy identity for the O77 Gaussian reduction. -/
theorem matrix_row_energy_identity {p h q : Type*}
    [Fintype p] [Fintype h] [DecidableEq h] [Fintype q]
    (T : ℝ) (Y : Matrix p h ℝ) (Z : Matrix h q ℝ) :
    T * frobeniusSq (Y * Z) + frobeniusSq Y =
      ∑ i, dotProduct (Y i)
        ((1 + T • (Z * Z.transpose)).mulVec (Y i)) := by
  rw [frobeniusSq_mul_eq_sum_row_gram, frobeniusSq_eq_sum_row_dot]
  simp_rw [dotProduct_gramShift_mulVec]
  rw [Finset.sum_add_distrib, Finset.mul_sum]
  ring

/-- For nonnegative scale, the shifted Gram matrix is positive definite. -/
theorem gram_posDef {h q : Type*} [Fintype h] [DecidableEq h] [Fintype q]
    (T : ℝ) (hT : 0 ≤ T) (Z : Matrix h q ℝ) :
    Matrix.PosDef (1 + T • (Z * Z.transpose)) := by
  have htranspose : Z.conjTranspose = Z.transpose := by
    ext i j
    simp [Matrix.conjTranspose_apply, Matrix.transpose_apply]
  have hgram : Matrix.PosSemidef (Z * Z.transpose) := by
    rw ←[ htranspose]
    exact Matrix.posSemidef_self_mul_conjTranspose Z
  exact Matrix.PosDef.one.add_posSemidef (hgram.smul hT)

/-- Sylvester's determinant identity in the exact rectangular Gram dimensions. -/
theorem sylvester_gram_determinant {h q : Type*}
    [Fintype h] [Fintype q] [DecidableEq h] [DecidableEq q]
    (T : ℝ) (Z : Matrix h q ℝ) :
    Matrix.det (1 + T • (Z * Z.transpose)) =
      Matrix.det (1 + T • (Z.transpose * Z)) := by
  simpa [Matrix.mul_smul, Matrix.smul_mul] using
    (Matrix.det_one_add_mul_comm Z (T • Z.transpose))

/-- The hypothesis `0 ≤ T` cannot be dropped: in dimension one, `T = -1`
and `Z = 1` make the shifted Gram matrix zero. -/
theorem gram_posDef_requires_nonnegative_scale :
    ¬ Matrix.PosDef
      ((1 : Matrix (Fin 1) (Fin 1) ℝ) +
        (-1 : ℝ) •
          ((1 : Matrix (Fin 1) (Fin 1) ℝ) *
            (1 : Matrix (Fin 1) (Fin 1) ℝ).transpose)) := by
  have hzero :
      ((1 : Matrix (Fin 1) (Fin 1) ℝ) +
        (-1 : ℝ) •
          ((1 : Matrix (Fin 1) (Fin 1) ℝ) *
            (1 : Matrix (Fin 1) (Fin 1) ℝ).transpose)) = 0 := by
    ext i j
    fin_cases i
    fin_cases j
    norm_num [Matrix.mul_apply]
  rw [hzero]
```

```
  intro h
  have hx : (Finsupp.single (0 : Fin 1) (1 : ℝ)) ≠ 0 := by simp
  have hpos := h.2 hx
  norm_num at hpos

/-- A rectangular compile-time control for the exact row-energy identity. -/
theorem matrix_row_energy_rectangular_test
    (T : ℝ) (Y : Matrix (Fin 2) (Fin 1) ℝ)
    (Z : Matrix (Fin 1) (Fin 3) ℝ) :
    T * frobeniusSq (Y * Z) + frobeniusSq Y =
      ∑ i, dotProduct (Y i)
        ((1 + T • (Z * Z.transpose)).mulVec (Y i)) :=
  matrix_row_energy_identity T Y Z

/-- The shifted empty Gram matrix is positive definite by the empty-vector convention. -/
theorem gram_posDef_empty_hidden_test {q : Type*} [Fintype q]
    (T : ℝ) (hT : 0 ≤ T) (Z : Matrix (Fin 0) q ℝ) :
    Matrix.PosDef (1 + T • (Z * Z.transpose)) :=
  gram_posDef T hT Z

/-- Coordinate Lebesgue measure on a finite real function space. -/
noncomputable def coordinateLebesgue ι( : Type*) [Fintype ι] :
    MeasureTheory.Measure ι( → ℝ) :=
  MeasureTheory.Measure.pi fun _ : ι => MeasureTheory.volume

noncomputable instance coordinateLebesgue.instSigmaFinite
    ι( : Type*) [Fintype ι] :
    MeasureTheory.SigmaFinite (coordinateLebesgue ι) := by
  unfold coordinateLebesgue
  infer_instance

/-- Product coordinate Lebesgue measure on a rectangular real matrix space. -/
noncomputable def matrixLebesgue (p h : Type*) [Fintype p] [Fintype h] :
    MeasureTheory.Measure (Matrix p h ℝ) :=
  MeasureTheory.Measure.pi fun _ : p => coordinateLebesgue h

/-- One row's quadratic form for the shifted Gram matrix. -/
def gramRowQuadratic {h q : Type*} [Fintype h] [DecidableEq h] [Fintype q]
    (T : ℝ) (Z : Matrix h q ℝ) (y : h → ℝ) : ℝ :=
  dotProduct y ((1 + T • (Z * Z.transpose)).mulVec y)

/-- The Gaussian integrand before separating the rows of `Y`. -/
def gaussianRowsIntegrand {p h q : Type*}
    [Fintype p] [Fintype h] [Fintype q]
    (T : ℝ) (Z : Matrix h q ℝ) (Y : Matrix p h ℝ) : ℝ :=
  Real.exp (-(T * frobeniusSq (Y * Z) + frobeniusSq Y))

lemma gaussianRowsIntegrand_eq_prod {p h q : Type*}
    [Fintype p] [Fintype h] [DecidableEq h] [Fintype q]
    (T : ℝ) (Z : Matrix h q ℝ) (Y : Matrix p h ℝ) :
    gaussianRowsIntegrand T Z Y =
      ∏ i, Real.exp (-(gramRowQuadratic T Z (Y i))) := by
  unfold gaussianRowsIntegrand gramRowQuadratic
  rw [matrix_row_energy_identity]
  rw ←[ Finset.sum_neg_distrib]
  exact Real.exp_sum _ _

lemma gaussianRowsIntegral_eq_prod {p h q : Type*}
    [Fintype p] [Fintype h] [DecidableEq h] [Fintype q]
    (T : ℝ) (Z : Matrix h q ℝ) :
    ∫( Y : Matrix p h ℝ, gaussianRowsIntegrand T Z Y ∂matrixLebesgue p h) =
      ∏ _i : p,
```

```
        ∫ y : h → ℝ, Real.exp (-(gramRowQuadratic T Z y)) ∂coordinateLebesgue h := by
  rw [show matrixLebesgue p h =
      MeasureTheory.Measure.pi (fun _ : p => coordinateLebesgue h) from rfl]
  calc
    ∫( Y : Matrix p h ℝ, gaussianRowsIntegrand T Z Y
        ∂MeasureTheory.Measure.pi (fun _ : p => coordinateLebesgue h)) =
      ∫ Y : Matrix p h ℝ,
        ∏ i, Real.exp (-(gramRowQuadratic T Z (Y i)))
        ∂MeasureTheory.Measure.pi (fun _ : p => coordinateLebesgue h) := by
      apply MeasureTheory.integral_congr_ae
      filter_upwards with Y
      exact gaussianRowsIntegrand_eq_prod T Z Y
    _ = ∏ _i : p,
        ∫ y : h → ℝ, Real.exp (-(gramRowQuadratic T Z y)) ∂coordinateLebesgue h := by
      exact MeasureTheory.integral_fintype_prod_eq_prod
        (f := fun _i : p => fun y : h → ℝ =>
          Real.exp (-(gramRowQuadratic T Z y)))

/-- Concrete finite-row Tonelli/Fubini package in the actual O77 matrix space.
The explicit row-integrability premise prevents Lean's default-zero value for a
nonintegrable real integral from being mistaken for a finite Gaussian identity. -/
theorem gaussian_rows_tonelli {p h q : Type*}
    [Fintype p] [Fintype h] [DecidableEq h] [Fintype q]
    (T : ℝ) (Z : Matrix h q ℝ)
    (hrow : MeasureTheory.Integrable
      (fun y : h → ℝ => Real.exp (-(gramRowQuadratic T Z y)))
      (coordinateLebesgue h)) :
    MeasureTheory.Integrable (gaussianRowsIntegrand (p := p) T Z)
        (matrixLebesgue p h) ∧
      ∫( Y : Matrix p h ℝ, gaussianRowsIntegrand T Z Y ∂matrixLebesgue p h) =
        ∏ _i : p,
          ∫ y : h → ℝ, Real.exp (-(gramRowQuadratic T Z y))
            ∂coordinateLebesgue h := by
  constructor
  · have hprod : MeasureTheory.Integrable
        (fun Y : Matrix p h ℝ =>
          ∏ i, Real.exp (-(gramRowQuadratic T Z (Y i))))
        (MeasureTheory.Measure.pi fun _ : p => coordinateLebesgue h) := by
      exact MeasureTheory.Integrable.fintype_prod (fun _i : p => hrow)
    rw [show matrixLebesgue p h =
        MeasureTheory.Measure.pi (fun _ : p => coordinateLebesgue h) from rfl]
    exact hprod.congr <| Filter.Eventually.of_forall fun Y =>
      (gaussianRowsIntegrand_eq_prod T Z Y).symm
  · exact gaussianRowsIntegral_eq_prod T Z

/-- Determinant-normalized value of a positive-definite quadratic Gaussian integral. -/
def quadraticGaussianValue {h : Type*} [Fintype h] [DecidableEq h]
    (M : Matrix h h ℝ) : ℝ :=
  (Real.pi ^ Fintype.card h / Matrix.det M) ^ (1 / 2 : ℝ)

lemma fin_one_quadratic_eq
    (M : Matrix (Fin 1) (Fin 1) ℝ) (y : Fin 1 → ℝ) :
    dotProduct y (M.mulVec y) = M 0 0 * (y 0) ^ 2 := by
  simp [dotProduct, Matrix.mulVec_apply, pow_two]
  ring

/-- Mathlib's scalar Gaussian integral in the normalization used by O77. -/
theorem scalar_quadratic_gaussian_integral {a : ℝ} (ha : 0 < a) :
    ∫( x : ℝ, Real.exp (-a * x ^ 2)) =
      (Real.pi / a) ^ (1 / 2 : ℝ) := by
  simpa [Real.norm_eq_abs, sq_abs, Module.finrank_self] using
    (GaussianFourier.integral_rexp_neg_mul_sq_norm (V := ℝ) ha)
```

```
/-- Integrability of a one-dimensional positive-definite quadratic Gaussian. -/
theorem quadratic_gaussian_integrable_fin_one
    (M : Matrix (Fin 1) (Fin 1) ℝ) (hM : Matrix.PosDef M) :
    MeasureTheory.Integrable
      (fun y : Fin 1 → ℝ => Real.exp (-(dotProduct y (M.mulVec y))))
      (coordinateLebesgue (Fin 1)) := by
  let a : ℝ := M 0 0
  have ha : 0 < a := hM.diag_pos
  have hscalar : MeasureTheory.Integrable
      (fun x : ℝ => Real.exp (-a * x ^ 2)) MeasureTheory.volume :=
    integrable_exp_neg_mul_sq ha
  have hprod : MeasureTheory.Integrable
      (fun y : Fin 1 → ℝ => ∏ _i : Fin 1, Real.exp (-a * (y 0) ^ 2))
      (MeasureTheory.Measure.pi fun _ : Fin 1 => MeasureTheory.volume) := by
    have h := MeasureTheory.Integrable.fintype_prod
      μ( := fun _ : Fin 1 => MeasureTheory.volume)
      (f := fun _i : Fin 1 => fun x : ℝ => Real.exp (-a * x ^ 2))
      (fun _i : Fin 1 => hscalar)
    simpa using h
  rw [show coordinateLebesgue (Fin 1) =
      MeasureTheory.Measure.pi (fun _ : Fin 1 => MeasureTheory.volume) from rfl]
  exact hprod.congr <| Filter.Eventually.of_forall fun y => by
    change ∏( _i : Fin 1, Real.exp (-a * (y 0) ^ 2)) =
      Real.exp (-(dotProduct y (M.mulVec y)))
    rw [fin_one_quadratic_eq]
    change ∏( _i : Fin 1, Real.exp (-a * (y 0) ^ 2)) =
      Real.exp (-(a * (y 0) ^ 2))
    simp

/-- The determinant-normalized quadratic Gaussian formula for a one-dimensional matrix. -/
theorem quadratic_gaussian_integral_fin_one
    (M : Matrix (Fin 1) (Fin 1) ℝ) (hM : Matrix.PosDef M) :
    ∫( y : Fin 1 → ℝ, Real.exp (-(dotProduct y (M.mulVec y)))
      ∂coordinateLebesgue (Fin 1)) = quadraticGaussianValue M := by
  let a : ℝ := M 0 0
  have ha : 0 < a := hM.diag_pos
  rw [show coordinateLebesgue (Fin 1) =
      MeasureTheory.Measure.pi (fun _ : Fin 1 => MeasureTheory.volume) from rfl]
  calc
    ∫( y : Fin 1 → ℝ, Real.exp (-(dotProduct y (M.mulVec y)))
        ∂MeasureTheory.Measure.pi (fun _ : Fin 1 => MeasureTheory.volume)) =
      ∫ y : Fin 1 → ℝ, ∏ i : Fin 1, Real.exp (-a * (y i) ^ 2)
        ∂MeasureTheory.Measure.pi (fun _ : Fin 1 => MeasureTheory.volume) := by
      apply MeasureTheory.integral_congr_ae
      filter_upwards with y
      rw [fin_one_quadratic_eq]
      change Real.exp (-(a * (y 0) ^ 2)) = _
      simp
    _ = ∏ _i : Fin 1, ∫ x : ℝ, Real.exp (-a * x ^ 2) := by
      exact MeasureTheory.integral_fintype_prod_eq_prod
        (f := fun _i : Fin 1 => fun x : ℝ => Real.exp (-a * x ^ 2))
    _ = (Real.pi / a) ^ (1 / 2 : ℝ) := by
      rw [Fintype.prod_unique]
      exact scalar_quadratic_gaussian_integral ha
    _ = quadraticGaussianValue M := by
      simp [quadraticGaussianValue, a]

/-- The positive-definite quadratic Gaussian formula in the empty row space. -/
theorem quadratic_gaussian_integral_empty
    (M : Matrix (Fin 0) (Fin 0) ℝ) :
    ∫( y : Fin 0 → ℝ, Real.exp (-(dotProduct y (M.mulVec y)))
```

```
      ∂coordinateLebesgue (Fin 0)) = quadraticGaussianValue M := by
  simp only [coordinateLebesgue, quadraticGaussianValue, dotProduct,
    Finset.univ_eq_empty, Finset.sum_empty, Matrix.det_isEmpty,
    Fintype.card_eq_zero, pow_zero, div_one, Real.one_rpow,
    neg_zero, Real.exp_zero, MeasureTheory.integral_const]
  rw [MeasureTheory.Measure.real_def, MeasureTheory.Measure.pi_empty_univ,
    ENNReal.toReal_one]
  norm_num

/-- The complete one-dimensional adapter: integrability and exact determinant normalization. -/
theorem quadratic_gaussian_package_fin_one
    (M : Matrix (Fin 1) (Fin 1) ℝ) (hM : Matrix.PosDef M) :
    MeasureTheory.Integrable
      (fun y : Fin 1 → ℝ => Real.exp (-(dotProduct y (M.mulVec y))))
      (coordinateLebesgue (Fin 1)) ∧
    ∫( y : Fin 1 → ℝ, Real.exp (-(dotProduct y (M.mulVec y)))
      ∂coordinateLebesgue (Fin 1)) = quadraticGaussianValue M :=
  ⟨quadratic_gaussian_integrable_fin_one M hM,
    quadratic_gaussian_integral_fin_one M ⟩hM

/-- Integrability of the concrete O77 matrix Gaussian, conditional only on the
standard positive-definite quadratic Gaussian theorem in one row space. -/
theorem gaussian_rows_integrable_of_quadratic_gaussian {p h q : Type*}
    [Fintype p] [Fintype h] [DecidableEq h] [Fintype q]
    (quadratic_gaussian_integral :
      ∀ M : Matrix h h ℝ, Matrix.PosDef M →
        MeasureTheory.Integrable
          (fun y : h → ℝ => Real.exp (-(dotProduct y (M.mulVec y))))
          (coordinateLebesgue h) ∧
        ∫( y : h → ℝ, Real.exp (-(dotProduct y (M.mulVec y)))
          ∂coordinateLebesgue h) = quadraticGaussianValue M)
    (T : ℝ) (hT : 0 ≤ T) (Z : Matrix h q ℝ) :
    MeasureTheory.Integrable (gaussianRowsIntegrand (p := p) T Z)
      (matrixLebesgue p h) := by
  let M : Matrix h h ℝ := 1 + T • (Z * Z.transpose)
  have hrow : MeasureTheory.Integrable
      (fun y : h → ℝ => Real.exp (-(gramRowQuadratic T Z y)))
      (coordinateLebesgue h) := by
    simpa [M, gramRowQuadratic] using
      (quadratic_gaussian_integral M (gram_posDef T hT Z)).1
  exact (gaussian_rows_tonelli T Z hrow).1

/-- Concrete Gaussian-row formula in the O77 matrix space, conditional only on the
standard positive-definite quadratic Gaussian integral in the row space. -/
theorem gaussian_rows_instantiated_of_quadratic_gaussian {p h q : Type*}
    [Fintype p] [Fintype h] [DecidableEq h] [Fintype q]
    (quadratic_gaussian_integral :
      ∀ M : Matrix h h ℝ, Matrix.PosDef M →
        MeasureTheory.Integrable
          (fun y : h → ℝ => Real.exp (-(dotProduct y (M.mulVec y))))
          (coordinateLebesgue h) ∧
        ∫( y : h → ℝ, Real.exp (-(dotProduct y (M.mulVec y)))
          ∂coordinateLebesgue h) = quadraticGaussianValue M)
    (T : ℝ) (hT : 0 ≤ T) (Z : Matrix h q ℝ) :
    ∫( Y : Matrix p h ℝ, gaussianRowsIntegrand T Z Y ∂matrixLebesgue p h) =
      (quadraticGaussianValue (1 + T • (Z * Z.transpose))) ^ Fintype.card p := by
  have hpackage := quadratic_gaussian_integral
    (1 + T • (Z * Z.transpose)) (gram_posDef T hT Z)
  rw [(gaussian_rows_tonelli T Z (by
    simpa [gramRowQuadratic] using hpackage.1)).2]
  have hrow := hpackage.2
  simp only [gramRowQuadratic]
```

```
  simp_rw [hrow]
  simp

/-- Unconditional Gaussian row integration for residual hidden dimension one. -/
theorem gaussian_rows_instantiated_fin_one {p q : Type*}
    [Fintype p] [Fintype q]
    (T : ℝ) (hT : 0 ≤ T) (Z : Matrix (Fin 1) q ℝ) :
    ∫( Y : Matrix p (Fin 1) ℝ, gaussianRowsIntegrand T Z Y
      ∂matrixLebesgue p (Fin 1)) =
      (quadraticGaussianValue (1 + T • (Z * Z.transpose))) ^ Fintype.card p := by
  exact gaussian_rows_instantiated_of_quadratic_gaussian
    (fun M hM => quadratic_gaussian_package_fin_one M hM) T hT Z

/-- The corresponding matrix Gaussian is integrable in hidden dimension one. -/
theorem gaussian_rows_integrable_fin_one {p q : Type*}
    [Fintype p] [Fintype q]
    (T : ℝ) (hT : 0 ≤ T) (Z : Matrix (Fin 1) q ℝ) :
    MeasureTheory.Integrable (gaussianRowsIntegrand (p := p) T Z)
      (matrixLebesgue p (Fin 1)) := by
  exact gaussian_rows_integrable_of_quadratic_gaussian
    (fun M hM => quadratic_gaussian_package_fin_one M hM) T hT Z

/-- The zero-row Gaussian integral is the empty product `1`. -/
theorem gaussian_rows_zero_rows {h q : Type*} [Fintype h] [DecidableEq h] [Fintype q]
    (T : ℝ) (Z : Matrix h q ℝ) :
    ∫( Y : Matrix (Fin 0) h ℝ, gaussianRowsIntegrand T Z Y
      ∂matrixLebesgue (Fin 0) h) = 1 := by
  simpa using gaussianRowsIntegral_eq_prod (p := Fin 0) T Z

end O77.Residual
```

Corresponding Lean declaration(s): `O77.Residual.frobeniusSq,O77.Residual.frobeniusSq_eq_sum_row_dot,O77.Residual.row_mul_eq_vecMul,O77.Residual.transpose_mulVec_eq_vecMul,O77.Residual.row_mul_dot_self_eq_gram,O77.Residual.frobeniusSq_mul_eq_sum_row_gram,O77.Residual.dotProduct_gramShift_mulVec,O77.Residual.matrix_row_energy_identity,O77.Residual.gram_posDef,O77.Residual.sylvester_gram_determinant,O77.Residual.gram_posDef_requires_nonnegative_scale,O77.Residual.matrix_row_energy_rectangular_test,O77.Residual.gram_posDef_empty_hidden_test,O77.Residual.coordinateLebesgue,O77.Residual.coordinateLebesgue.instSigmaFinite,O77.Residual.matrixLebesgue,O77.Residual.gramRowQuadratic,O77.Residual.gaussianRowsIntegrand,O77.Residual.gaussianRowsIntegrand_eq_prod,O77.Residual.gaussianRowsIntegral_eq_prod,O77.Residual.gaussian_rows_tonelli,O77.Residual.quadraticGaussianValue,O77.Residual.fin_one_quadratic_eq,O77.Residual.scalar_quadratic_gaussian_integral,O77.Residual.quadratic_gaussian_integrable_fin_one,O77.Residual.quadratic_gaussian_integral_fin_one,O77.Residual.quadratic_gaussian_integral_empty,O77.Residual.quadratic_gaussian_package_fin_one,O77.Residual.gaussian_rows_integrable_of_quadratic_gaussian,O77.Residual.gaussian_rows_instantiated_of_quadratic_gaussian,O77.Residual.gaussian_rows_instantiated_fin_one,O77.Residual.gaussian_rows_integrable_fin_one,O77.Residual.gaussian_rows_zero_rows`.

23 Arbitrary quadratic Gaussians and smaller-Gram spectra

23.1 The finite-dimensional positive-definite Gaussian integral

The abstract Gaussian-row interface isolates the following standard statement: for a positive-definite real matrix M on the coordinate space $h \to \mathbb{R}$, the function $y \mapsto e^{-y^{\mathsf{T}} M y}$ is integrable and has integral $(\pi^{\#h} / \det M)^{1/2}$. This section proves that statement without assuming an O77-specific conclusion.

The normalization must first be fixed. Mathlib's theorem `MeasureTheory.volume_pi` identifies its Euclidean volume on $h \to \mathbb{R}$ with the finite product of one-dimensional volumes.

Consequently the previously defined `coordinateLebesgueh` is exactly Mathlib volume, not merely a positive scalar multiple of it.

Proposition 23.1 (Arbitrary positive-definite quadratic Gaussian)**.** *Let h be any finite index type and let $M \in \mathbb{R}^{h\times h}$ be positive definite. Then $y \mapsto e^{-y^{\mathsf{T}}My}$ is integrable with respect to coordinate Lebesgue measure and*

$$\int_{\mathbb{R}^h} e^{-y^{\mathsf{T}}My}\,dy = \left(\frac{\pi^{\#h}}{\det M}\right)^{1/2}.$$

The statement includes the empty index type.

Proof. First suppose $M = \operatorname{diag}(d_i)$ with every $d_i > 0$. Expanding the matrix–vector product gives $y^{\mathsf{T}}My = \sum_i d_i y_i^2$, hence the integrand is the finite product $\prod_i e^{-d_i y_i^2}$. Each factor is integrable by the checked scalar Gaussian theorem. Finite-product Fubini gives

$$\int \prod_i e^{-d_i y_i^2}\,dy = \prod_i \left(\frac{\pi}{d_i}\right)^{1/2} = \left(\frac{\pi^{\#h}}{\prod_i d_i}\right)^{1/2}.$$

The last equality is proved in Lean from positivity using the finite-product rule for real powers; $\prod_i d_i = \det(\operatorname{diag} d)$.

For general M, use the real Hermitian spectral theorem. Let U be Mathlib's unitary eigenvector matrix and D the diagonal matrix of eigenvalues, so $M = UDU^{\mathsf{T}}$ and every diagonal entry of D is positive. The proof establishes directly that

$$(Ux)^{\mathsf{T}}M(Ux) = x^{\mathsf{T}}Dx.$$

It remains to justify the substitution $y = Ux$ in the selected measure normalization. Over $\mathbb{R}$, unitary means orthogonal. Taking determinants in $UU^{\mathsf{T}} = I$ shows $|\det U| = 1$. Mathlib's exact formula for the pushforward of coordinate volume by a linear matrix map therefore yields $U_*\,dy = dy$. Applying `MeasureTheory.integral_map` reduces the integral to the diagonal case. The determinant normalization is unchanged because Mathlib's Hermitian spectral theorem also gives $\det M = \prod_i d_i$. Integrability is transported by the same measure-preserving map rather than inferred from the numerical value of a possibly nonintegrable Bochner integral. □

The proof supplies a packaged theorem containing both integrability and the exact integral. Feeding that package to the abstract row assembly removes its only ordinary parameter.

Corollary 23.2 (Unconditional Gaussian rows in all finite dimensions)**.** *For finite p, h, q, $T \geq 0$, and $Z \in \mathbb{R}^{h\times q}$,*

$$\int_{\mathbb{R}^{p\times h}} e^{-T\|YZ\|_F^2 - \|Y\|_F^2}\,dY = \left[\left(\frac{\pi^{\#h}}{\det(I_h + TZZ^{\mathsf{T}})}\right)^{1/2}\right]^{\#p},$$

which is the exact real-power representation `quadraticGaussianValue` used in the code; the integrand is also proved integrable.

Proof. The shifted Gram matrix is positive definite by Proposition 22.1. Apply Proposition 23.1 to each row and invoke the checked finite-row Tonelli/Fubini theorem. No theorem parameter remains. □

23.2 The smaller-Gram spectral determinant

Both Gram matrices ZZ^{T} and $Z^{\mathsf{T}}Z$ are proved positive semidefinite by the corresponding Mathlib conjugate-transpose Gram theorems, after checking that real conjugate transpose is ordinary transpose. Their Hermitian eigenvalues are therefore nonnegative.

Proposition 23.3 (Hermitian shift determinant)**.** *If $A \in \mathbb{R}^{n\times n}$ is Hermitian and $T \in \mathbb{R}$, then*

$$\det(I_n + TA) = \prod_{i\in n}(1 + T\lambda_i(A)),$$

where `Matrix.IsHermitian.eigenvalues` *is Mathlib's eigenvalue function.*

Proof. Diagonalize $A = UDU^{\mathsf{T}}$. The unitary conjugation is an algebra automorphism, so it sends $I+TD$ to $I+TA$. Expanding the determinant of $U(I+TD)U^{\mathsf{T}}$ and taking determinants in $UU^{\mathsf{T}} = I$ cancels the two unitary determinants. The determinant of the diagonal matrix is the displayed product. This proof does not need $T \geq 0$. □

Corollary 23.4 (Instantiated smaller-Gram formula)**.** *Let $k = \min(\#h, \#q)$. If $\#h \leq \#q$, use the h nonnegative eigenvalues of ZZ^{T}; if $\#q \leq \#h$, use the q nonnegative eigenvalues of $Z^{\mathsf{T}}Z$. In the corresponding branch,*

$$\det(I_h + TZZ^{\mathsf{T}}) = \prod_{i=1}^{k}(1 + Ts_i), \qquad s_i \geq 0.$$

For $T \geq 0$, every factor is at least one; this inequality is checked separately for both Gram orientations.

Proof. In the first branch apply Proposition 23.3 directly to ZZ^{T}. In the second branch first apply the already checked rectangular Sylvester identity and then apply the same proposition to $Z^{\mathsf{T}}Z$. This is a literal case split on the two finite cardinalities; it never asserts that one may simply delete zero eigenvalues from the larger matrix. □

The controls cover a genuinely rectangular 2×3 matrix and an empty hidden index. The negative control proves that Gram eigenvalues are only nonnegative: the zero 1×1 matrix has eigenvalue zero, so a strengthening to strict positivity is false. A separate 2-dimensional identity-matrix example exercises the arbitrary Gaussian theorem, and a 4×2 by 2×3 example exercises unconditional row integration beyond the scalar case.

23.3 Positive-definite Gaussian and smaller-Gram spectrum: Lean code

The following block compiles after the concrete Gaussian-row block and in the full document-order profile.

```
set_option autoImplicit false
noncomputable section
namespace O77.Residual

open scoped BigOperators
open MeasureTheory

lemma coordinateLebesgue_eq_volume ι( : Type*) [Fintype ι] :
    coordinateLebesgue ι = volume := by
  exact MeasureTheory.volume_pi.symm

lemma diagonal_mulVec_apply {h : Type*} [Fintype h] [DecidableEq h]
    (d : h → ℝ) (y : h → ℝ) (i : h) :
```

```
    (Matrix.diagonal d).mulVec y i = d i * y i := by
  simp [Matrix.mulVec_apply]

lemma diagonal_quadratic_eq_sum {h : Type*} [Fintype h] [DecidableEq h]
    (d : h → ℝ) (y : h → ℝ) :
    dotProduct y ((Matrix.diagonal d).mulVec y) =
      ∑ i, d i * (y i) ^ 2 := by
  unfold dotProduct
  apply Finset.sum_congr rfl
  intro i hi
  rw [diagonal_mulVec_apply]
  ring

lemma diagonal_gaussian_eq_prod {h : Type*} [Fintype h] [DecidableEq h]
    (d : h → ℝ) (y : h → ℝ) :
    Real.exp (-(dotProduct y ((Matrix.diagonal d).mulVec y))) =
      ∏ i, Real.exp (-d i * (y i) ^ 2) := by
  rw [diagonal_quadratic_eq_sum]
  rw ←[ Finset.sum_neg_distrib]
  rw [Real.exp_sum]
  apply Finset.prod_congr rfl
  intro i hi
  congr 1
  ring

lemma finset_prod_rpow_nonneg ι{ : Type*} (s : Finset ι) (f : ι → ℝ)
    (hf : ∀ i ∈ s, 0 ≤ f i) (z : ℝ) :
    ∏( i ∈ s, f i) ^ z = ∏ i ∈ s, (f i) ^ z := by
  classical
  induction s using Finset.induction_on with
  | empty => simp
  | @insert a s ha ih =>
      rw [Finset.prod_insert ha, Finset.prod_insert ha]
      rw [Real.mul_rpow (hf a (Finset.mem_insert_self a s))
        (Finset.prod_nonneg fun i hi => hf i (Finset.mem_insert_of_mem hi))]
      rw [ih (fun i hi => hf i (Finset.mem_insert_of_mem hi))]

lemma diagonal_gaussian_integrable {h : Type*} [Fintype h] [DecidableEq h]
    (d : h → ℝ) (hd : ∀ i, 0 < d i) :
    Integrable
      (fun y : h → ℝ =>
        Real.exp (-(dotProduct y ((Matrix.diagonal d).mulVec y))))
      (coordinateLebesgue h) := by
  have hprod : Integrable
      (fun y : h → ℝ => ∏ i, Real.exp (-d i * (y i) ^ 2))
      (Measure.pi fun _ : h => volume) := by
    exact Integrable.fintype_prod (fun i => integrable_exp_neg_mul_sq (hd i))
  rw [show coordinateLebesgue h = Measure.pi (fun _ : h => volume) from rfl]
  exact hprod.congr <| Filter.Eventually.of_forall fun y =>
    (diagonal_gaussian_eq_prod d y).symm

lemma prod_scalar_gaussian_eq_value {h : Type*} [Fintype h] [DecidableEq h]
    (d : h → ℝ) (hd : ∀ i, 0 < d i) :
    ∏( i, (Real.pi / d i) ^ (1 / 2 : ℝ)) =
      quadraticGaussianValue (Matrix.diagonal d) := by
  have hnonneg : ∀ i ∈ Finset.univ, 0 ≤ Real.pi / d i := by
    intro i hi
    exact div_nonneg Real.pi_pos.le (hd i).le
  rw ←[ finset_prod_rpow_nonneg Finset.univ (fun i => Real.pi / d i) hnonneg]
  rw [Finset.prod_div_distrib, Finset.prod_const]
  simp only [Finset.card_univ]
  unfold quadraticGaussianValue
```

```
  rw [Matrix.det_diagonal]

/-- Positive diagonal quadratic Gaussian in every finite dimension. -/
theorem diagonal_quadratic_gaussian_integral {h : Type*}
    [Fintype h] [DecidableEq h]
    (d : h → ℝ) (hd : ∀ i, 0 < d i) :
    ∫( y : h → ℝ,
        Real.exp (-(dotProduct y ((Matrix.diagonal d).mulVec y)))
        ∂coordinateLebesgue h) =
      quadraticGaussianValue (Matrix.diagonal d) := by
  rw [show coordinateLebesgue h = Measure.pi (fun _ : h => volume) from rfl]
  calc
    ∫( y : h → ℝ,
        Real.exp (-(dotProduct y ((Matrix.diagonal d).mulVec y)))
        ∂Measure.pi (fun _ : h => volume)) =
      ∫ y : h → ℝ, ∏ i, Real.exp (-d i * (y i) ^ 2)
        ∂Measure.pi (fun _ : h => volume) := by
      apply integral_congr_ae
      filter_upwards with y
      exact diagonal_gaussian_eq_prod d y
    _ = ∏ i, ∫ x : ℝ, Real.exp (-d i * x ^ 2) := by
      exact integral_fintype_prod_eq_prod
        (f := fun i : h => fun x : ℝ => Real.exp (-d i * x ^ 2))
    _ = ∏ i, (Real.pi / d i) ^ (1 / 2 : ℝ) := by
      apply Finset.prod_congr rfl
      intro i hi
      exact scalar_quadratic_gaussian_integral (hd i)
    _ = quadraticGaussianValue (Matrix.diagonal d) :=
      prod_scalar_gaussian_eq_value d hd

lemma unitary_star_eq_transpose {n : Type*} [Fintype n] [DecidableEq n]
    (U : Matrix.unitaryGroup n ℝ) :
    star (U : Matrix n n ℝ) = (U : Matrix n n ℝ).transpose := by
  ext i j
  simp [Matrix.transpose_apply]

lemma unitary_transpose_mul_self {n : Type*} [Fintype n] [DecidableEq n]
    (U : Matrix.unitaryGroup n ℝ) :
    (U : Matrix n n ℝ).transpose * (U : Matrix n n ℝ) = 1 := by
  rw ←[ unitary_star_eq_transpose U]
  exact Unitary.coe_star_mul_self U

lemma unitary_mulVec_dotProduct {n : Type*} [Fintype n] [DecidableEq n]
    (U : Matrix.unitaryGroup n ℝ) (x z : n → ℝ) :
    dotProduct ((U : Matrix n n ℝ).mulVec x)
      ((U : Matrix n n ℝ).mulVec z) = dotProduct x z := by
  rw [Matrix.dotProduct_mulVec]
  rw ←[ transpose_mulVec_eq_vecMul (U : Matrix n n ℝ)]
  rw [Matrix.mulVec_mulVec]
  rw [unitary_transpose_mul_self U, Matrix.one_mulVec]

lemma unitary_det_abs {n : Type*} [Fintype n] [DecidableEq n]
    (U : Matrix.unitaryGroup n ℝ) : |Matrix.det (U : Matrix n n ℝ)| = 1 := by
  have hdetstar : Matrix.det (star (U : Matrix n n ℝ)) = Matrix.det (U : Matrix n n ℝ) := by
    calc
      Matrix.det (star (U : Matrix n n ℝ)) = star (Matrix.det (U : Matrix n n ℝ)) :=
        Matrix.det_conjTranspose (U : Matrix n n ℝ)
      _ = Matrix.det (U : Matrix n n ℝ) := by simp
  have hunitMat :
      (U : Matrix n n ℝ) * star (U : Matrix n n ℝ) = 1 :=
    Unitary.coe_mul_star_self U
  have hunitDet := congrArg Matrix.det hunitMat
```

```
  rw [Matrix.det_mul, Matrix.det_one, hdetstar] at hunitDet
  have habs_sq : |Matrix.det (U : Matrix n n ℝ)| ^ 2 = 1 := by
    simpa [sq_abs, pow_two] using hunitDet
  nlinarith [abs_nonneg (Matrix.det (U : Matrix n n ℝ))]

lemma unitary_det_ne_zero {n : Type*} [Fintype n] [DecidableEq n]
    (U : Matrix.unitaryGroup n ℝ) : Matrix.det (U : Matrix n n ℝ) ≠ 0 := by
  intro h
  have habs := unitary_det_abs U
  simp [h] at habs

lemma unitary_mulVec_map_volume {n : Type*} [Fintype n] [DecidableEq n]
    (U : Matrix.unitaryGroup n ℝ) :
    Measure.map (fun x : n → ℝ => (U : Matrix n n ℝ).mulVec x) volume = volume := by
  rw [show (fun x : n → ℝ => (U : Matrix n n ℝ).mulVec x) =
      ⇑(Matrix.toLin' (U : Matrix n n ℝ)) by
        funext x
        exact (Matrix.toLin'_apply _ _).symm]
  rw [Real.map_matrix_volume_pi_eq_smul_volume_pi (unitary_det_ne_zero U)]
  have hinvabs : |(Matrix.det (U : Matrix n n ℝ))⁻¹| = 1 := by
    rw [abs_inv, unitary_det_abs U, inv_one]
  rw [hinvabs, ENNReal.ofReal_one, one_smul]

lemma unitary_mulVec_measurePreserving {n : Type*} [Fintype n] [DecidableEq n]
    (U : Matrix.unitaryGroup n ℝ) :
    MeasurePreserving (fun x : n → ℝ => (U : Matrix n n ℝ).mulVec x)
      volume volume := by
  refine ⟨?_, unitary_mulVec_map_volume ⟩U
  exact (Matrix.toLin' (U : Matrix n n ℝ)).continuous_of_finiteDimensional.measurable

lemma unitary_mulVec_measurePreserving_coordinate {n : Type*}
    [Fintype n] [DecidableEq n]
    (U : Matrix.unitaryGroup n ℝ) :
    MeasurePreserving (fun x : n → ℝ => (U : Matrix n n ℝ).mulVec x)
      (coordinateLebesgue n) (coordinateLebesgue n) := by
  simpa [coordinateLebesgue_eq_volume] using unitary_mulVec_measurePreserving U

lemma spectral_mulVec_mulVec {n : Type*} [Fintype n] [DecidableEq n]
    (M : Matrix n n ℝ) (hM : Matrix.PosDef M) (x : n → ℝ) :
    M.mulVec ((hM.isHermitian.eigenvectorUnitary : Matrix n n ℝ).mulVec x) =
      (hM.isHermitian.eigenvectorUnitary : Matrix n n ℝ).mulVec
        ((Matrix.diagonal hM.isHermitian.eigenvalues).mulVec x) := by
  let U := hM.isHermitian.eigenvectorUnitary
  let D := Matrix.diagonal hM.isHermitian.eigenvalues
  have hspec : M = (U : Matrix n n ℝ) * D * star (U : Matrix n n ℝ) := by
    simpa [U, D, Unitary.conjStarAlgAut_apply] using hM.isHermitian.spectral_theorem
  have hstarself : star (U : Matrix n n ℝ) * (U : Matrix n n ℝ) = 1 :=
    Unitary.coe_star_mul_self U
  rw [show (hM.isHermitian.eigenvectorUnitary : Matrix n n ℝ) = (U : Matrix n n ℝ) from rfl]
  rw [show Matrix.diagonal hM.isHermitian.eigenvalues = D from rfl]
  rw [Matrix.mulVec_mulVec, Matrix.mulVec_mulVec]
  congr 1
  rw [hspec, Matrix.mul_assoc, hstarself, Matrix.mul_one]

lemma spectral_quadratic_change {n : Type*} [Fintype n] [DecidableEq n]
    (M : Matrix n n ℝ) (hM : Matrix.PosDef M) (x : n → ℝ) :
    dotProduct
        ((hM.isHermitian.eigenvectorUnitary : Matrix n n ℝ).mulVec x)
        (M.mulVec ((hM.isHermitian.eigenvectorUnitary : Matrix n n ℝ).mulVec x)) =
      dotProduct x ((Matrix.diagonal hM.isHermitian.eigenvalues).mulVec x) := by
  rw [spectral_mulVec_mulVec M hM x]
  exact unitary_mulVec_dotProduct hM.isHermitian.eigenvectorUnitary x
```

```
    ((Matrix.diagonal hM.isHermitian.eigenvalues).mulVec x)

lemma quadraticGaussianValue_diagonal_eigenvalues {n : Type*}
    [Fintype n] [DecidableEq n]
    (M : Matrix n n ℝ) (hM : Matrix.PosDef M) :
    quadraticGaussianValue (Matrix.diagonal hM.isHermitian.eigenvalues) =
      quadraticGaussianValue M := by
  have hdet : Matrix.det (Matrix.diagonal hM.isHermitian.eigenvalues) = Matrix.det M := by
    rw [Matrix.det_diagonal]
    exact hM.isHermitian.det_eq_prod_eigenvalues.symm
  unfold quadraticGaussianValue
  rw [hdet]

/-- Integrability of every finite-dimensional real positive-definite quadratic Gaussian. -/
theorem quadratic_gaussian_integrable {n : Type*}
    [Fintype n] [DecidableEq n]
    (M : Matrix n n ℝ) (hM : Matrix.PosDef M) :
    Integrable
      (fun y : n → ℝ => Real.exp (-(dotProduct y (M.mulVec y))))
      (coordinateLebesgue n) := by
  let U := hM.isHermitian.eigenvectorUnitary
  let fM : (n → ℝ) → ℝ := fun y => Real.exp (-(dotProduct y (M.mulVec y)))
  let fD : (n → ℝ) → ℝ := fun x =>
    Real.exp (-(dotProduct x ((Matrix.diagonal hM.isHermitian.eigenvalues).mulVec x)))
  have hD : Integrable fD (coordinateLebesgue n) := by
    exact diagonal_gaussian_integrable hM.isHermitian.eigenvalues hM.eigenvalues_pos
  have hcomp : Integrable (fM ∘ fun x : n → ℝ => (U : Matrix n n ℝ).mulVec x)
      (coordinateLebesgue n) := by
    exact hD.congr <| Filter.Eventually.of_forall fun x => by
      dsimp [fD, fM, U, Function.comp_def]
      rw [spectral_quadratic_change M hM x]
  have hfM : AEStronglyMeasurable fM (coordinateLebesgue n) := by
    exact (by fun_prop : Continuous fM).aestronglyMeasurable
  exact ((unitary_mulVec_measurePreserving_coordinate U).integrable_comp hfM).mp hcomp

/-- The arbitrary-dimensional determinant-normalized quadratic Gaussian formula. -/
theorem quadratic_gaussian_integral {n : Type*}
    [Fintype n] [DecidableEq n]
    (M : Matrix n n ℝ) (hM : Matrix.PosDef M) :
    ∫ y : n → ℝ, Real.exp (-(dotProduct y (M.mulVec y)))
      ∂coordinateLebesgue n) = quadraticGaussianValue M := by
  let U := hM.isHermitian.eigenvectorUnitary
  let fM : (n → ℝ) → ℝ := fun y => Real.exp (-(dotProduct y (M.mulVec y)))
  have hmap : Measure.map (fun x : n → ℝ => (U : Matrix n n ℝ).mulVec x) volume = volume :=
    unitary_mulVec_map_volume U
  have hUmeas : AEMeasurable (fun x : n → ℝ => (U : Matrix n n ℝ).mulVec x) volume :=
    (Matrix.toLin' (U : Matrix n n ℝ)).continuous_of_finiteDimensional.measurable.aemeasurable
  have hfmap : AEStronglyMeasurable fM
      (Measure.map (fun x : n → ℝ => (U : Matrix n n ℝ).mulVec x) volume) := by
    rw [hmap]
    exact (by fun_prop : Continuous fM).aestronglyMeasurable
  rw [coordinateLebesgue_eq_volume]
  calc
    ∫ y : n → ℝ, fM y ∂volume) =
        ∫ y : n → ℝ, fM y
          ∂Measure.map (fun x : n → ℝ => (U : Matrix n n ℝ).mulVec x) volume := by
      rw [hmap]
    _ = ∫ x : n → ℝ, fM ((U : Matrix n n ℝ).mulVec x) ∂volume := by
      exact integral_map hUmeas hfmap
    _ = ∫ x : n → ℝ,
        Real.exp (-(dotProduct x
          ((Matrix.diagonal hM.isHermitian.eigenvalues).mulVec x))) ∂volume := by
```

```
      apply integral_congr_ae
      filter_upwards with x
      dsimp [fM, U]
      rw [spectral_quadratic_change M hM x]
    _ = quadraticGaussianValue (Matrix.diagonal hM.isHermitian.eigenvalues) := by
      simpa [coordinateLebesgue_eq_volume] using
        diagonal_quadratic_gaussian_integral hM.isHermitian.eigenvalues hM.eigenvalues_pos
    _ = quadraticGaussianValue M :=
      quadraticGaussianValue_diagonal_eigenvalues M hM

/-- Complete arbitrary-dimensional package. -/
theorem quadratic_gaussian_package {n : Type*}
    [Fintype n] [DecidableEq n]
    (M : Matrix n n ℝ) (hM : Matrix.PosDef M) :
    Integrable
      (fun y : n → ℝ => Real.exp (-(dotProduct y (M.mulVec y))))
      (coordinateLebesgue n) ∧
    ∫( y : n → ℝ, Real.exp (-(dotProduct y (M.mulVec y)))
      ∂coordinateLebesgue n) = quadraticGaussianValue M :=
  ⟨quadratic_gaussian_integrable M hM, quadratic_gaussian_integral M ⟩hM



/-- The concrete O77 matrix Gaussian is integrable in every finite hidden dimension. -/
theorem gaussian_rows_integrable {p h q : Type*}
    [Fintype p] [Fintype h] [DecidableEq h] [Fintype q]
    (T : ℝ) (hT : 0 ≤ T) (Z : Matrix h q ℝ) :
    MeasureTheory.Integrable (gaussianRowsIntegrand (p := p) T Z)
      (matrixLebesgue p h) := by
  exact gaussian_rows_integrable_of_quadratic_gaussian
    (fun M hM => quadratic_gaussian_package M hM) T hT Z

/-- Concrete Gaussian-row formula in the O77 matrix space, now unconditional. -/
theorem gaussian_rows_instantiated {p h q : Type*}
    [Fintype p] [Fintype h] [DecidableEq h] [Fintype q]
    (T : ℝ) (hT : 0 ≤ T) (Z : Matrix h q ℝ) :
    ∫( Y : Matrix p h ℝ, gaussianRowsIntegrand T Z Y ∂matrixLebesgue p h) =
      (quadraticGaussianValue (1 + T • (Z * Z.transpose))) ^ Fintype.card p := by
  exact gaussian_rows_instantiated_of_quadratic_gaussian
    (fun M hM => quadratic_gaussian_package M hM) T hT Z

/-- Two-dimensional diagonal control for the full quadratic Gaussian theorem. -/
theorem quadratic_gaussian_integral_fin_two_test :
    ∫( y : Fin 2 → ℝ,
        Real.exp (-(dotProduct y ((1 : Matrix (Fin 2) (Fin 2) ℝ).mulVec y)))
        ∂coordinateLebesgue (Fin 2)) =
      quadraticGaussianValue (1 : Matrix (Fin 2) (Fin 2) ℝ) := by
  exact quadratic_gaussian_integral 1 Matrix.PosDef.one

/-- Rectangular arbitrary-hidden-dimension control for Gaussian row integration. -/
theorem gaussian_rows_instantiated_rectangular_test
    (T : ℝ) (hT : 0 ≤ T) (Z : Matrix (Fin 2) (Fin 3) ℝ) :
    ∫( Y : Matrix (Fin 4) (Fin 2) ℝ, gaussianRowsIntegrand T Z Y
      ∂matrixLebesgue (Fin 4) (Fin 2)) =
      (quadraticGaussianValue (1 + T • (Z * Z.transpose))) ^ 4 := by
  simpa using gaussian_rows_instantiated (p := Fin 4) T hT Z

/-- Left Gram matrices are positive semidefinite. -/
theorem leftGram_posSemidef {h q : Type*}
    [Fintype h] [Fintype q]
    (Z : Matrix h q ℝ) : Matrix.PosSemidef (Z * Z.transpose) := by
  have hstar : Z.conjTranspose = Z.transpose := by
```

```
    ext i j
    simp [Matrix.conjTranspose_apply, Matrix.transpose_apply]
  rw ←[ hstar]
  exact Matrix.posSemidef_self_mul_conjTranspose Z

/-- Right Gram matrices are positive semidefinite. -/
theorem rightGram_posSemidef {h q : Type*}
    [Fintype h] [Fintype q]
    (Z : Matrix h q ℝ) : Matrix.PosSemidef (Z.transpose * Z) := by
  have hstar : Z.conjTranspose = Z.transpose := by
    ext i j
    simp [Matrix.conjTranspose_apply, Matrix.transpose_apply]
  rw ←[ hstar]
  exact Matrix.posSemidef_conjTranspose_mul_self Z

/-- A Hermitian shift has the expected determinant product over the eigenvalues. -/
theorem det_one_add_smul_hermitian {n : Type*}
    [Fintype n] [DecidableEq n]
    (A : Matrix n n ℝ) (hA : A.IsHermitian) (T : ℝ) :
    Matrix.det (1 + T • A) = ∏ i, (1 + T * hA.eigenvalues i) := by
  let U := hA.eigenvectorUnitary
  let d : n → ℝ := hA.eigenvalues
  have hspec : A =
      ((Unitary.conjStarAlgAut ℝ (Matrix n n ℝ)) U) (Matrix.diagonal d) := by
    simpa [U, d] using hA.spectral_theorem
  conv_lhs => rw [hspec]
  have hdiag :
      (1 : Matrix n n ℝ) + T • Matrix.diagonal d =
        Matrix.diagonal (fun i => 1 + T * d i) := by
    ext i j
    by_cases hij : i = j
    · subst j
      simp [Matrix.diagonal]
    · simp [Matrix.diagonal, hij]
  have hshift :
      (1 : Matrix n n ℝ) +
          T • (((Unitary.conjStarAlgAut ℝ (Matrix n n ℝ)) U) (Matrix.diagonal d)) =
        ((Unitary.conjStarAlgAut ℝ (Matrix n n ℝ)) U)
          (Matrix.diagonal (fun i => 1 + T * d i)) := by
    rw ←[ hdiag]
    simp only [map_add, map_one, map_smul]
  rw [hshift, Unitary.conjStarAlgAut_apply]
  have hdetU : Matrix.det (U : Matrix n n ℝ) * Matrix.det (star (U : Matrix n n ℝ)) = 1 := by
    have hunitMat : (U : Matrix n n ℝ) * star (U : Matrix n n ℝ) = 1 :=
      Unitary.coe_mul_star_self U
    have hunitDet := congrArg Matrix.det hunitMat
    rw [Matrix.det_mul, Matrix.det_one] at hunitDet
    exact hunitDet
  rw [Matrix.det_mul, Matrix.det_mul, Matrix.det_diagonal]
  calc
    Matrix.det (U : Matrix n n ℝ) *
          ∏( i, (1 + T * d i)) *
          Matrix.det (star (U : Matrix n n ℝ)) =
        ∏( i, (1 + T * d i)) *
          (Matrix.det (U : Matrix n n ℝ) * Matrix.det (star (U : Matrix n n ℝ))) := by ring
    _ = ∏ i, (1 + T * d i) := by rw [hdetU, mul_one]
    _ = ∏ i, (1 + T * hA.eigenvalues i) := by rfl

/-- Spectral determinant formula through the left Gram matrix. -/
theorem leftGram_shift_det {h q : Type*}
    [Fintype h] [Fintype q] [DecidableEq h]
    (T : ℝ) (Z : Matrix h q ℝ) :
```

```
    Matrix.det (1 + T • (Z * Z.transpose)) =
      ∏ i, (1 + T * (leftGram_posSemidef Z).isHermitian.eigenvalues i) := by
  exact det_one_add_smul_hermitian _ (leftGram_posSemidef Z).isHermitian T

/-- Spectral determinant formula through the right Gram matrix. -/
theorem rightGram_shift_det {h q : Type*}
    [Fintype h] [Fintype q] [DecidableEq q]
    (T : ℝ) (Z : Matrix h q ℝ) :
    Matrix.det (1 + T • (Z.transpose * Z)) =
      ∏ i, (1 + T * (rightGram_posSemidef Z).isHermitian.eigenvalues i) := by
  exact det_one_add_smul_hermitian _ (rightGram_posSemidef Z).isHermitian T

/-- Sylvester followed by the right-Gram spectral formula. -/
theorem largeGram_shift_det_via_right {h q : Type*}
    [Fintype h] [Fintype q] [DecidableEq h] [DecidableEq q]
    (T : ℝ) (Z : Matrix h q ℝ) :
    Matrix.det (1 + T • (Z * Z.transpose)) =
      ∏ i, (1 + T * (rightGram_posSemidef Z).isHermitian.eigenvalues i) := by
  calc
    Matrix.det (1 + T • (Z * Z.transpose)) =
        Matrix.det (1 + T • (Z.transpose * Z)) := by
      simpa [Matrix.mul_smul, Matrix.smul_mul] using
        (Matrix.det_one_add_mul_comm Z (T • Z.transpose))
    _ = ∏ i, (1 + T * (rightGram_posSemidef Z).isHermitian.eigenvalues i) :=
      rightGram_shift_det T Z

/-- Every left-Gram eigenvalue is nonnegative. -/
theorem leftGram_eigenvalues_nonneg {h q : Type*}
    [Fintype h] [Fintype q] [DecidableEq h]
    (Z : Matrix h q ℝ) (i : h) :
    0 ≤ (leftGram_posSemidef Z).isHermitian.eigenvalues i :=
  (leftGram_posSemidef Z).eigenvalues_nonneg i

/-- Every right-Gram eigenvalue is nonnegative. -/
theorem rightGram_eigenvalues_nonneg {h q : Type*}
    [Fintype h] [Fintype q] [DecidableEq q]
    (Z : Matrix h q ℝ) (i : q) :
    0 ≤ (rightGram_posSemidef Z).isHermitian.eigenvalues i :=
  (rightGram_posSemidef Z).eigenvalues_nonneg i

/-- Concrete smaller-side Gram spectral formula, split by the two possible cardinal comparisons. -/
theorem gram_determinant_spectral_instantiated {h q : Type*}
    [Fintype h] [Fintype q] [DecidableEq h] [DecidableEq q]
    (T : ℝ) (Z : Matrix h q ℝ) :
    (Fintype.card h ≤ Fintype.card q ∧
      Matrix.det (1 + T • (Z * Z.transpose)) =
        ∏ i : h, (1 + T * (leftGram_posSemidef Z).isHermitian.eigenvalues i) ∧
      ∀ i : h, 0 ≤ (leftGram_posSemidef Z).isHermitian.eigenvalues i) ∨
    (Fintype.card q ≤ Fintype.card h ∧
      Matrix.det (1 + T • (Z * Z.transpose)) =
        ∏ i : q, (1 + T * (rightGram_posSemidef Z).isHermitian.eigenvalues i) ∧
      ∀ i : q, 0 ≤ (rightGram_posSemidef Z).isHermitian.eigenvalues i) := by
  rcases le_total (Fintype.card h) (Fintype.card q) with hhq | hqh
  · exact Or.inl ⟨hhq, leftGram_shift_det T Z, leftGram_eigenvalues_nonneg ⟩Z
  · exact Or.inr ⟨hqh, largeGram_shift_det_via_right T Z, rightGram_eigenvalues_nonneg ⟩Z

/-- In a nonnegative scale, every left-Gram spectral factor is at least one. -/
theorem leftGram_shift_factor_one_le {h q : Type*}
    [Fintype h] [Fintype q] [DecidableEq h]
    (T : ℝ) (hT : 0 ≤ T) (Z : Matrix h q ℝ) (i : h) :
    1 ≤ 1 + T * (leftGram_posSemidef Z).isHermitian.eigenvalues i := by
  nlinarith [leftGram_eigenvalues_nonneg Z i]
```

```
/-- In a nonnegative scale, every right-Gram spectral factor is at least one. -/
theorem rightGram_shift_factor_one_le {h q : Type*}
    [Fintype h] [Fintype q] [DecidableEq q]
    (T : ℝ) (hT : 0 ≤ T) (Z : Matrix h q ℝ) (i : q) :
    1 ≤ 1 + T * (rightGram_posSemidef Z).isHermitian.eigenvalues i := by
  nlinarith [rightGram_eigenvalues_nonneg Z i]

/-- Rectangular control: for a `2 × 3` matrix the left side is the smaller spectral product. -/
theorem gram_determinant_spectral_rectangular_test
    (T : ℝ) (Z : Matrix (Fin 2) (Fin 3) ℝ) :
    Matrix.det (1 + T • (Z * Z.transpose)) =
      ∏ i : Fin 2,
        (1 + T * (leftGram_posSemidef Z).isHermitian.eigenvalues i) :=
  leftGram_shift_det T Z

/-- Empty hidden dimension gives the empty product. -/
theorem gram_determinant_spectral_empty_hidden_test {q : Type*}
    [Fintype q] [DecidableEq q]
    (T : ℝ) (Z : Matrix (Fin 0) q ℝ) :
    Matrix.det (1 + T • (Z * Z.transpose)) = 1 := by
  rw [leftGram_shift_det T Z]
  simp

/-- Positive semidefinite Gram spectra need not be strictly positive. -/
theorem gram_eigenvalues_need_not_be_positive :
    ¬ 0 <
      (rightGram_posSemidef (0 : Matrix (Fin 1) (Fin 1) ℝ)).isHermitian.eigenvalues 0 := by
  let hzero := (rightGram_posSemidef (0 : Matrix (Fin 1) (Fin 1) ℝ)).isHermitian
  have hmat :
      (0 : Matrix (Fin 1) (Fin 1) ℝ).transpose *
          (0 : Matrix (Fin 1) (Fin 1) ℝ) = 0 := by
    simp
  have heigs : hzero.eigenvalues = 0 := hzero.eigenvalues_eq_zero_iff.mpr hmat
  have h0 : hzero.eigenvalues (0 : Fin 1) = 0 := congrFun heigs 0
  change ¬ 0 < hzero.eigenvalues 0
  rw [h0]
  norm_num


end O77.Residual
```

Corresponding Lean declaration(s): O77.Residual.coordinateLebesgue_eq_volume,O77.Residual.diagonal_mulVec_apply,O77.Residual.diagonal_quadratic_eq_sum,O77.Residual.diagonal_gaussian_eq_prod,O77.Residual.finset_prod_rpow_nonneg,O77.Residual.diagonal_gaussian_integrable,O77.Residual.prod_scalar_gaussian_eq_value,O77.Residual.diagonal_quadratic_gaussian_integral,O77.Residual.unitary_star_eq_transpose,O77.Residual.unitary_transpose_mul_self,O77.Residual.unitary_mulVec_dotProduct,O77.Residual.unitary_det_abs,O77.Residual.unitary_det_ne_zero,O77.Residual.unitary_mulVec_map_volume,O77.Residual.unitary_mulVec_measurePreserving,O77.Residual.unitary_mulVec_measurePreserving_coordinate,O77.Residual.spectral_mulVec_mulVec,O77.Residual.spectral_quadratic_change,O77.Residual.quadraticGaussianValue_diagonal_eigenvalues,O77.Residual.quadratic_gaussian_integrable,O77.Residual.quadratic_gaussian_integral,O77.Residual.quadratic_gaussian_package,O77.Residual.gaussian_rows_integrable,O77.Residual.gaussian_rows_instantiated,O77.Residual.quadratic_gaussian_integral_fin_two_test,O77.Residual.gaussian_rows_instantiated_rectangular_test,O77.Residual.leftGram_posSemidef,O77.Residual.rightGram_posSemidef,O77.Residual.det_one_add_smul_hermitian,O77.Residual.leftGram_shift_det,O77.Residual.rightGram_shift_det,O77.Residual.largeGram_shift_det_via_right,O77.Residual.leftGram_eigenvalues_nonneg,O77.Residual.rightGram_eigenvalues_nonneg,O77.Residual.gram_determinant_spectral_instantiated,O77.Residual.leftGram_shift_factor_one_le,O77.Residual.rightGram_shift_factor_one_le,O77.Residual.gram_determinant_spectral_rectangular_test,O77.Residual.gram_det

`erminant_spectral_empty_hidden_test,O77.Residual.gram_eigenvalues_need_not_be_positive`.

24 Permutation chambers and sorted spectra

24.1 Null boundaries and the almost-everywhere chamber partition

For $k \geq 0$, put

$$\mathcal{O}_k = \{x \in \mathbb{R}^k : 0 \leq x_i \text{ for every } i\}, \qquad \mathcal{O}_k^\circ = \{x \in \mathbb{R}^k : 0 < x_i \text{ for every } i\}.$$

For distinct coordinates $i \neq j$, let $H_{ij} = \{x : x_i = x_j\}$, and let $Z_i = \{x : x_i = 0\}$. The finite unions of the H_{ij} and Z_i are the tie and coordinate-boundary loci. For a permutation $\sigma \in \mathfrak{S}_k$, define

$$C_\sigma = \{x \in \mathcal{O}_k^\circ : x_{\sigma(0)} < x_{\sigma(1)} < \cdots < x_{\sigma(k-1)}\}.$$

In the Lean statement this chain is expressed as `StrictMono(x∘sigma)`; it includes the empty and one-coordinate cases without separate conventions.

Proposition 24.1 (Permutation chambers form an almost-everywhere partition)**.** *Every H_{ij} and every Z_i has coordinate Lebesgue measure zero. The sets C_σ are measurable and pairwise disjoint, and*

$$\mathcal{O}_k = \bigcup_{\sigma \in \mathfrak{S}_k} C_\sigma \quad \textit{almost everywhere.}$$

The strict reference chamber C_{id} and the weak chamber

$$\overline{C} = \{x : 0 \leq x_i \textit{ for all } i,\ x_i \leq x_j \textit{ whenever } i \leq j\}$$

are also equal almost everywhere.

Proof. The hyperplane H_{ij} is the kernel of the nonzero linear functional $x \mapsto x_i - x_j$; nonzeroness is witnessed by the coordinate vector e_i. Likewise, Z_i is the kernel of the nonzero coordinate projection. Mathlib's theorem that a proper finite-dimensional real submodule has additive Haar measure zero applies after the already checked identification of coordinate product measure with Euclidean volume. Finite unions of these hyperplanes are therefore null.

A point $x \in \mathcal{O}_k^\circ$ lies in a strict chamber exactly when its coordinate map is injective. If it is injective, `Tuple.sortx` gives a permutation for which the reordered tuple is monotone; monotonicity plus injectivity makes it strictly monotone. Conversely, strict monotonicity after a permutation forces injectivity. Thus the union of all chambers is exactly $\mathcal{O}_k^\circ$ minus the tie locus. The zero-coordinate locus identifies $\mathcal{O}_k^\circ$ with $\mathcal{O}_k$ almost everywhere.

If x lay in both C_σ and C_τ, the two reordered tuples would be monotone. Mathlib's uniqueness theorem for monotone rearrangements gives $x \circ \sigma = x \circ \tau$; injectivity of x then gives $\sigma = \tau$. This proves pairwise disjointness. Finally, the difference between the weak chamber and the strict chamber lies in the union of the coordinate-zero and tie loci, hence is null. □

24.2 The exact factorial and the ordered Laguerre density

The coordinate permutation is implemented by Mathlib's measurable equivalence `MeasurableEquiv.piCongrLeft`. Its product-measure theorem shows that this equivalence preserves `coordinateLebesgue(Fink)` exactly; no unspecified positive Jacobian constant is introduced.

Theorem 24.2 (Exact symmetric orthant-to-chamber identity)**.** *Let $F : (\mathrm{Fin}\, k \to \mathbb{R}) \to [0, \infty]$ be invariant under every coordinate permutation. Then, as an equality of extended nonnegative integrals,*

$$\int_{\mathcal{O}_k} F(x)\, dx = k! \int_{C_{\mathrm{id}}} F(x)\, dx = k! \int_{\overline{C}} F(x)\, dx.$$

This includes $k = 0$, when the permutation type is a singleton and $0! = 1$.

Proof. Replace $\mathcal{O}_k$ by $\mathcal{O}_k^\circ$ and then by the almost-everywhere disjoint union of Proposition 24.1. The nonnegative integral over a finite disjoint union is the sum of the chamber integrals. The measure-preserving coordinate permutation carries C_{id} onto C_σ, and permutation invariance of F makes all these integrals equal. Mathlib's cardinality theorem $\#\mathfrak{S}_k = k!$ supplies the visible coefficient. Replacing the strict reference chamber by the weak one uses their almost-everywhere equality. The Lean theorem is stated for an arbitrary extended-nonnegative function in Mathlib's `lintegral` semantics; the Wishart application must separately supply the Borel measurability required by its change-of-variables theorem. □

Let `sortedCoordinatess` be the nondecreasing rearrangement obtained by composing s with `Tuple.sorts`. Sorting is unchanged by any coordinate permutation. The checked ordered-Laguerre density is therefore extended symmetrically to the full orthant by evaluating it at `sortedCoordinatess` and applying `ENNReal.ofReal`.

Corollary 24.3 (Sorted Laguerre bridge)**.** *For $\eta > -1$, $\rho \geq 0$, and $T \geq 0$, the symmetrically sorted Laguerre density $\widetilde{L}_{k,\eta,\rho,T}$ satisfies*

$$\int_{\mathcal{O}_k} \widetilde{L}_{k,\eta,\rho,T}(s)\, ds = k!\ \mathrm{ofReal}(I_{k,\eta,\rho}(T))\,,$$

where $I_{k,\eta,\rho}(T)$ is the previously checked real integral on the positive weakly ordered chamber. For $k = 2$, the coefficient specializes definitionally to 2.

Proof. The sorted density is permutation invariant by uniqueness of the nondecreasing rearrangement. On the ordered chamber, sorting fixes the tuple. The strict and weak positive ordered chambers differ only on the tie locus. The existing integrability and nonnegativity theorems for the Laguerre density convert the real integral to its extended-nonnegative integral, after which Theorem 24.2 applies. □

24.3 Pointwise sorted smaller-Gram spectra

For the two explicit finite-dimensional orientations, define `sortedLeftGramEigenvaluesZ` by sorting the eigenvalues of ZZ^{T} when $Z \in \mathbb{R}^{k\times d}$, and define `sortedRightGramEigenvaluesZ` by sorting those of $Z^{\mathsf{T}}Z$ when $Z \in \mathbb{R}^{d\times k}$. The prior positive-semidefinite Gram theorem and permutation invariance of finite products give the following pointwise package.

Proposition 24.4 (Sorted smaller-Gram spectral packages)**.** *For either orientation, the sorted tuple $s_0 \leq \cdots \leq s_{k-1}$ is nonnegative and*

$$\det(I + TZZ^{\mathsf{T}}) = \prod_{i<k}(1 + Ts_i),$$

using Sylvester's identity in the right-Gram orientation. These statements hold for arbitrary real T.

Proof. The sorting permutation makes the eigenvalue tuple monotone. Every sorted entry is one of the original Gram eigenvalues and is therefore nonnegative. Reindexing a finite product along the sorting permutation does not change it, so the determinant formulas of Section 23 immediately yield the displayed products. □

This proposition is deliberately pointwise. It does not prove that the map from the matrix Z to the selected sorted eigenvalue tuple is Borel measurable, nor does it identify the pushforward of matrix Lebesgue measure. The pointwise calculation is used as an algebraic adapter. The Gram and spectral modules separately prove the required integral identity.

24.4 Controls and rejected strengthenings

The empty chamber, one-coordinate chamber, and two-coordinate chamber are all exercised. The $k = 2$ test makes omission of the factorial visible. Two counterexamples prevent an accidental strengthening from an almost-everywhere statement to a set equality. The nonnegative vector $(2, 1)$ does not lie in the identity strict chamber, so one chamber does not cover the orthant. The tied vector $(1, 1)$ lies in the orthant but in no strict permutation chamber, so even the union of all strict chambers is not literally the orthant. The existing zero-Gram example continues to show that the sorted spectral entries can only be claimed nonnegative, not positive.

24.5 Permutation chambers and sorted Gram spectra: Lean code

The following block compiles over the preceding Gaussian and spectral declarations and in the full document-order profile.

```
set_option autoImplicit false
noncomputable section
open MeasureTheory Set Function
open scoped BigOperators ENNReal

namespace O77.Residual

def coordinateDifference {k : ℕ} (i j : Fin k) :
    (Fin k → ℝ) →ₗ[ℝ] ℝ :=
  (LinearMap.proj i : (Fin k → ℝ) →ₗ[ℝ] ℝ) -
    (LinearMap.proj j : (Fin k → ℝ) →ₗ[ℝ] ℝ)

def tieHyperplane {k : ℕ} (i j : Fin k) : Set (Fin k → ℝ) :=
  {x | x i = x j}

lemma tieHyperplane_eq_ker {k : ℕ} (i j : Fin k) :
    tieHyperplane i j = (LinearMap.ker (coordinateDifference i j) : Set (Fin k → ℝ)) := by
  ext x
  simp [tieHyperplane, coordinateDifference, sub_eq_zero]

lemma coordinateDifference_ne_zero {k : ℕ} {i j : Fin k} (hij : i ≠ j) :
    coordinateDifference i j ≠ 0 := by
  intro h
  have hx := DFunLike.congr_fun h (Pi.single i (1 : ℝ))
  simp [coordinateDifference, hij] at hx

lemma tieHyperplane_null {k : ℕ} {i j : Fin k} (hij : i ≠ j) :
    coordinateLebesgue (Fin k) (tieHyperplane i j) = 0 := by
  rw [coordinateLebesgue_eq_volume, tieHyperplane_eq_ker]
  apply MeasureTheory.Measure.addHaar_submodule
  rw [ne_eq, LinearMap.ker_eq_top]
  exact coordinateDifference_ne_zero hij

abbrev DistinctPair (k : ℕ) := {p : Fin k × Fin k // p.1 ≠ p.2}

def tieHyperplanes (k : ℕ) : Set (Fin k → ℝ) :=
  ⋃ p : DistinctPair k, tieHyperplane p.1.1 p.1.2

lemma tieHyperplanes_null (k : ℕ) :
    coordinateLebesgue (Fin k) (tieHyperplanes k) = 0 := by
  apply measure_iUnion_null
  intro p
  exact tieHyperplane_null p.2

lemma mem_tieHyperplanes_iff_not_injective {k : ℕ} (x : Fin k → ℝ) :
```

```
    x ∈ tieHyperplanes k ↔ ¬ Function.Injective x := by
  constructor
  · intro hx hinj
    simp only [tieHyperplanes, Set.mem_iUnion, tieHyperplane] at hx
    rcases hx with ⟨p, ⟩hp
    exact p.2 (hinj hp)
  · intro h
    rw [Function.not_injective_iff] at h
    rcases h with ⟨i, j, hx, ⟩hij
    simp only [tieHyperplanes, Set.mem_iUnion, tieHyperplane]
    exact ⟨⟨(i, j), ⟩hij, ⟩hx


def nonnegativeOrthant (k : ℕ) : Set (Fin k → ℝ) :=
  {x | ∀ i, 0 ≤ x i}

def positiveOrthant (k : ℕ) : Set (Fin k → ℝ) :=
  {x | ∀ i, 0 < x i}

def strictOrderedPositive (k : ℕ) : Set (Fin k → ℝ) :=
  {x | ∀( i, 0 < x i) ∧ StrictMono x}

def weakOrderedNonnegative (k : ℕ) : Set (Fin k → ℝ) :=
  {x | ∀( i, 0 ≤ x i) ∧ Monotone x}

lemma measurableSet_nonnegativeOrthant (k : ℕ) : MeasurableSet (nonnegativeOrthant k) := by
  have hset : nonnegativeOrthant k = ⋂ i : Fin k, {x : Fin k → ℝ | 0 ≤ x i} := by
    ext x
    simp [nonnegativeOrthant]
  rw [hset]
  apply MeasurableSet.iInter
  intro i
  exact measurableSet_le measurable_const (measurable_pi_apply i)

lemma measurableSet_positiveOrthant (k : ℕ) : MeasurableSet (positiveOrthant k) := by
  have hset : positiveOrthant k = ⋂ i : Fin k, {x : Fin k → ℝ | 0 < x i} := by
    ext x
    simp [positiveOrthant]
  rw [hset]
  apply MeasurableSet.iInter
  intro i
  exact measurableSet_lt measurable_const (measurable_pi_apply i)

lemma measurableSet_strictOrder (k : ℕ) :
    MeasurableSet {x : Fin k → ℝ | StrictMono x} := by
  have hset : {x : Fin k → ℝ | StrictMono x} = ⋂ i : Fin k, ⋂ j : Fin k,
      {x : Fin k → ℝ | i < j → x i < x j} := by
    ext x
    simp [StrictMono]
  rw [hset]
  apply MeasurableSet.iInter
  intro i
  apply MeasurableSet.iInter
  intro j
  by_cases hij : i < j
  · simpa [hij] using measurableSet_lt (measurable_pi_apply i) (measurable_pi_apply j)
  · simp [hij]

lemma measurableSet_monotone (k : ℕ) :
    MeasurableSet {x : Fin k → ℝ | Monotone x} := by
  have hset : {x : Fin k → ℝ | Monotone x} = ⋂ i : Fin k, ⋂ j : Fin k,
      {x : Fin k → ℝ | i ≤ j → x i ≤ x j} := by
```

```
    ext x
    simp [Monotone]
  rw [hset]
  apply MeasurableSet.iInter
  intro i
  apply MeasurableSet.iInter
  intro j
  by_cases hij : i ≤ j
  · simpa [hij] using measurableSet_le (measurable_pi_apply i) (measurable_pi_apply j)
  · simp [hij]

lemma measurableSet_strictOrderedPositive (k : ℕ) :
    MeasurableSet (strictOrderedPositive k) := by
  exact (measurableSet_positiveOrthant k).inter (measurableSet_strictOrder k)

lemma measurableSet_weakOrderedNonnegative (k : ℕ) :
    MeasurableSet (weakOrderedNonnegative k) := by
  exact (measurableSet_nonnegativeOrthant k).inter (measurableSet_monotone k)


def coordinateZeroHyperplane {k : ℕ} (i : Fin k) : Set (Fin k → ℝ) :=
  {x | x i = 0}

lemma coordinateZeroHyperplane_eq_ker {k : ℕ} (i : Fin k) :
    coordinateZeroHyperplane i =
      (LinearMap.ker (LinearMap.proj i : (Fin k → ℝ) →ₗ[] ℝ) : Set (Fin k → ℝ)) := by
  ext x
  simp [coordinateZeroHyperplane]

lemma coordinateProjection_ne_zero {k : ℕ} (i : Fin k) :
    (LinearMap.proj i : (Fin k → ℝ) →ₗ[] ℝ) ≠ 0 := by
  intro h
  have hx := DFunLike.congr_fun h (Pi.single i (1 : ℝ))
  simp at hx

lemma coordinateZeroHyperplane_null {k : ℕ} (i : Fin k) :
    coordinateLebesgue (Fin k) (coordinateZeroHyperplane i) = 0 := by
  rw [coordinateLebesgue_eq_volume, coordinateZeroHyperplane_eq_ker]
  apply MeasureTheory.Measure.addHaar_submodule
  rw [ne_eq, LinearMap.ker_eq_top]
  exact coordinateProjection_ne_zero i

def coordinateZeroHyperplanes (k : ℕ) : Set (Fin k → ℝ) :=
  ⋃ i : Fin k, coordinateZeroHyperplane i

lemma coordinateZeroHyperplanes_null (k : ℕ) :
    coordinateLebesgue (Fin k) (coordinateZeroHyperplanes k) = 0 := by
  apply MeasureTheory.measure_iUnion_null
  intro i
  exact coordinateZeroHyperplane_null i

lemma mem_coordinateZeroHyperplanes_iff {k : ℕ} (x : Fin k → ℝ) :
    x ∈ coordinateZeroHyperplanes k ↔ ∃ i, x i = 0 := by
  simp [coordinateZeroHyperplanes, coordinateZeroHyperplane]

lemma positiveOrthant_subset_nonnegativeOrthant (k : ℕ) :
    positiveOrthant k ⊆ nonnegativeOrthant k := by
  intro x hx i
  exact (hx i).le

lemma nonnegative_diff_positive_subset_zeroHyperplanes (k : ℕ) :
    nonnegativeOrthant k \ positiveOrthant k ⊆ coordinateZeroHyperplanes k := by
```

```
  intro x hx
  have hnot : ¬ ∀ i, 0 < x i := hx.2
  push Not at hnot
  rcases hnot with ⟨i, ⟩hi
  apply (mem_coordinateZeroHyperplanes_iff x).2
  exact ⟨i, le_antisymm hi (hx.1 i)⟩

lemma positive_nonnegative_ae_eq (k : ℕ) :
    positiveOrthant k =ᵐ[coordinateLebesgue (Fin k)] nonnegativeOrthant k := by
  apply MeasureTheory.ae_eq_set.mpr
  constructor
  · have hsub := positiveOrthant_subset_nonnegativeOrthant k
    simp [Set.sdiff_eq_empty.2 hsub]
  · exact MeasureTheory.measure_mono_null
      (nonnegative_diff_positive_subset_zeroHyperplanes k)
      (coordinateZeroHyperplanes_null k)

noncomputable def coordinatePermutation {k : ℕ} σ( : Equiv.Perm (Fin k)) :
    (Fin k → ℝ) ≃ᵐ (Fin k → ℝ) :=
  MeasurableEquiv.piCongrLeft (fun _ : Fin k => ℝ) σ

@[simp] lemma coordinatePermutation_apply_apply {k : ℕ}
    σ( : Equiv.Perm (Fin k)) (x : Fin k → ℝ) (i : Fin k) :
    coordinatePermutation σ x σ( i) = x i := by
  simp [coordinatePermutation, MeasurableEquiv.coe_piCongrLeft]

@[simp] lemma coordinatePermutation_symm_apply_apply {k : ℕ}
    σ( : Equiv.Perm (Fin k)) (x : Fin k → ℝ) (i : Fin k) :
    (coordinatePermutation σ).symm x i = x σ( i) := by
  have h := congrFun ((coordinatePermutation σ).apply_symm_apply x) σ( i)
  simpa using h

def permutationChamber {k : ℕ} σ( : Equiv.Perm (Fin k)) : Set (Fin k → ℝ) :=
  {x | ∀( i, 0 < x i) ∧ StrictMono (x ∘ σ)}

lemma permutationChamber_eq_image {k : ℕ} σ( : Equiv.Perm (Fin k)) :
    permutationChamber σ = coordinatePermutation σ '' strictOrderedPositive k := by
  ext y
  constructor
  · intro hy
    refine ⟨(coordinatePermutation σ).symm y, ?_, by ⟩simp
    constructor
    · intro i
      simpa using hy.1 σ( i)
    · have heq : (coordinatePermutation σ).symm y = y ∘ σ := by
        funext i
        simp
      rw [heq]
      exact hy.2
  · rintro ⟨x, hx, ⟩rfl
    constructor
    · intro j
      obtain ⟨i, ⟩rfl := σ.surjective j
      simpa using hx.1 i
    · have heq : coordinatePermutation σ x ∘ σ = x := by
        funext i
        simp
      rw [heq]
      exact hx.2

lemma measurableSet_permutationChamber {k : ℕ} σ( : Equiv.Perm (Fin k)) :
    MeasurableSet (permutationChamber σ) := by
```

```
  rw [permutationChamber_eq_image]
  exact (coordinatePermutation σ).measurableSet_image.mpr
    (measurableSet_strictOrderedPositive k)


lemma injective_of_mem_permutationChamber {k : ℕ} σ{ : Equiv.Perm (Fin k)}
    {x : Fin k → ℝ} (hx : x ∈ permutationChamber σ) : Function.Injective x := by
  intro a b hab
  obtain ⟨i, ⟩rfl := σ.surjective a
  obtain ⟨j, ⟩rfl := σ.surjective b
  exact congrArg σ (hx.2.injective hab)

lemma permutationChambers_pairwise_disjoint (k : ℕ) :
    Pairwise (Disjoint on fun σ : Equiv.Perm (Fin k) => permutationChamber σ) := by
  intro σ τ στh
  change Disjoint (permutationChamber σ) (permutationChamber τ)
  rw [Set.disjoint_left]
  intro x σhx τhx
  apply στh
  apply Equiv.ext
  intro i
  apply injective_of_mem_permutationChamber σhx
  exact congrFun (Tuple.unique_monotone σhx.2.monotone τhx.2.monotone) i

lemma mem_iUnion_permutationChamber_iff {k : ℕ} (x : Fin k → ℝ) :
    x ∈ ⋃( σ : Equiv.Perm (Fin k), permutationChamber σ) ↔
      x ∈ positiveOrthant k ∧ Function.Injective x := by
  constructor
  · intro hx
    simp only [Set.mem_iUnion] at hx
    rcases hx with ⟨σ, σ⟩hx
    exact ⟨σhx.1, injective_of_mem_permutationChamber σ⟩hx
  · rintro ⟨hxpos, ⟩hxinj
    apply Set.mem_iUnion.2
    refine ⟨Tuple.sort x, hxpos, ?⟩_
    exact (Tuple.monotone_sort x).strictMono_of_injective
      (hxinj.comp (Tuple.sort x).injective)

lemma iUnion_permutationChamber_eq (k : ℕ) :
    ⋃( σ : Equiv.Perm (Fin k), permutationChamber σ) =
      positiveOrthant k \ tieHyperplanes k := by
  ext x
  rw [mem_iUnion_permutationChamber_iff]
  simp only [Set.mem_sdiff, mem_tieHyperplanes_iff_not_injective]
  tauto

lemma permutationChambers_ae_partition (k : ℕ) :
    positiveOrthant k ᵐ=[coordinateLebesgue (Fin k)]
      ⋃ σ : Equiv.Perm (Fin k), permutationChamber σ := by
  rw [iUnion_permutationChamber_eq]
  apply MeasureTheory.ae_eq_set.mpr
  constructor
  · apply MeasureTheory.measure_mono_null _ (tieHyperplanes_null k)
    intro x hx
    by_contra hxtie
    exact hx.2 ⟨hx.1, ⟩hxtie
  · simp

lemma strictOrderedPositive_subset_weakOrderedNonnegative (k : ℕ) :
    strictOrderedPositive k ⊆ weakOrderedNonnegative k := by
  intro x hx
  exact ⟨fun i => (hx.1 i).le, hx.2.⟩monotone
```

```
lemma weakOrderedNonnegative_diff_strict_subset_boundary (k : ℕ) :
    weakOrderedNonnegative k \ strictOrderedPositive k ⊆
      coordinateZeroHyperplanes k ∪ tieHyperplanes k := by
  intro x hx
  by_cases hpos : ∀ i, 0 < x i
  · right
    apply (mem_tieHyperplanes_iff_not_injective x).2
    intro hinj
    exact hx.2 (hpos, hx.1.2.strictMono_of_injective )hinj
  · left
    push Not at hpos
    rcases hpos with ⟨i, ⟩hi
    apply (mem_coordinateZeroHyperplanes_iff x).2
    exact ⟨i, le_antisymm hi (hx.1.1 i)⟩

lemma strict_weak_ae_eq (k : ℕ) :
    strictOrderedPositive k ᵐ[coordinateLebesgue (Fin k)]
      weakOrderedNonnegative k := by
  apply MeasureTheory.ae_eq_set.mpr
  constructor
  · have hsub := strictOrderedPositive_subset_weakOrderedNonnegative k
    simp [Set.sdiff_eq_empty.2 hsub]
  · apply MeasureTheory.measure_mono_null
      (weakOrderedNonnegative_diff_strict_subset_boundary k)
    exact MeasureTheory.measure_union_null
      (coordinateZeroHyperplanes_null k) (tieHyperplanes_null k)


lemma coordinatePermutation_measurePreserving {k : ℕ} σ( : Equiv.Perm (Fin k)) :
    MeasurePreserving (coordinatePermutation σ)
      (coordinateLebesgue (Fin k)) (coordinateLebesgue (Fin k)) := by
  simpa [coordinateLebesgue, coordinatePermutation] using
    (MeasureTheory.measurePreserving_piCongrLeft
      (fun _ : Fin k => volume) σ)

def PermutationInvariant {k : ℕ} (F : (Fin k → ℝ) → ℝ≥∞0) : Prop :=
  ∀ σ( : Equiv.Perm (Fin k)) (x : Fin k → ℝ), F (coordinatePermutation σ x) = F x

lemma setLIntegral_permutationChamber {k : ℕ}
    (F : (Fin k → ℝ) → ℝ≥∞0) (hF : PermutationInvariant F)
    σ( : Equiv.Perm (Fin k)) :
    ∫⁻( x in permutationChamber σ, F x ∂coordinateLebesgue (Fin k)) =
      ∫⁻ x in strictOrderedPositive k, F x ∂coordinateLebesgue (Fin k) := by
  rw [permutationChamber_eq_image]
  rw ←[ (coordinatePermutation_measurePreserving σ).setLIntegral_comp_emb
    (coordinatePermutation σ).measurableEmbedding F (strictOrderedPositive k)]
  apply MeasureTheory.setLIntegral_congr_fun (measurableSet_strictOrderedPositive k)
  intro x hx
  exact hF σ x


theorem symmetric_integral_chambers_instantiated {k : ℕ}
    (F : (Fin k → ℝ) → ℝ≥∞0) (hF : PermutationInvariant F) :
    ∫⁻( x in nonnegativeOrthant k, F x ∂coordinateLebesgue (Fin k)) =
      (k.factorial : ℝ≥∞0) *
        ∫⁻ x in strictOrderedPositive k, F x ∂coordinateLebesgue (Fin k) := by
  calc
    ∫⁻( x in nonnegativeOrthant k, F x ∂coordinateLebesgue (Fin k)) =
        ∫⁻ x in positiveOrthant k, F x ∂coordinateLebesgue (Fin k) := by
      rw [MeasureTheory.Measure.restrict_congr_set
        (positive_nonnegative_ae_eq k).symm]
```

```
    _ = ∫⁻ x in ⋃( σ : Equiv.Perm (Fin k), permutationChamber σ),
          F x ∂coordinateLebesgue (Fin k) := by
      rw [MeasureTheory.Measure.restrict_congr_set (permutationChambers_ae_partition k)]
    _ = ∑' σ : Equiv.Perm (Fin k),
          ∫⁻ x in permutationChamber σ, F x ∂coordinateLebesgue (Fin k) := by
      exact MeasureTheory.lintegral_iUnion
        (fun σ => measurableSet_permutationChamber σ)
        (permutationChambers_pairwise_disjoint k) F
    _ = ∑' σ_ : Equiv.Perm (Fin k),
          ∫⁻ x in strictOrderedPositive k, F x ∂coordinateLebesgue (Fin k) := by
      apply tsum_congr
      intro σ
      exact setLIntegral_permutationChamber F hF σ
    _ = (k.factorial : ℝ≥∞0) *
          ∫⁻ x in strictOrderedPositive k, F x ∂coordinateLebesgue (Fin k) := by
      rw [tsum_fintype]
      simp [Fintype.card_perm, nsmul_eq_mul]


theorem strict_weak_chamber_lintegral_eq {k : ℕ}
    (F : (Fin k → ℝ) → ℝ≥∞0) :
    ∫⁻( x in strictOrderedPositive k, F x ∂coordinateLebesgue (Fin k)) =
      ∫⁻ x in weakOrderedNonnegative k, F x ∂coordinateLebesgue (Fin k) := by
  rw [MeasureTheory.Measure.restrict_congr_set (strict_weak_ae_eq k)]

theorem symmetric_integral_weak_chamber_instantiated {k : ℕ}
    (F : (Fin k → ℝ) → ℝ≥∞0) (hF : PermutationInvariant F) :
    ∫⁻( x in nonnegativeOrthant k, F x ∂coordinateLebesgue (Fin k)) =
      (k.factorial : ℝ≥∞0) *
        ∫⁻ x in weakOrderedNonnegative k, F x ∂coordinateLebesgue (Fin k) := by
  rw [symmetric_integral_chambers_instantiated F hF,
    strict_weak_chamber_lintegral_eq]

lemma nonnegativeOrthant_zero : nonnegativeOrthant 0 = Set.univ := by
  ext x
  simp [nonnegativeOrthant]

lemma strictOrderedPositive_zero : strictOrderedPositive 0 = Set.univ := by
  ext x
  simp [strictOrderedPositive, StrictMono]

lemma weakOrderedNonnegative_zero : weakOrderedNonnegative 0 = Set.univ := by
  ext x
  simp [weakOrderedNonnegative, Monotone]

theorem symmetric_integral_chambers_empty_test
    (F : (Fin 0 → ℝ) → ℝ≥∞0) (hF : PermutationInvariant F) :
    ∫⁻( x in nonnegativeOrthant 0, F x ∂coordinateLebesgue (Fin 0)) =
      ∫⁻ x in strictOrderedPositive 0, F x ∂coordinateLebesgue (Fin 0) := by
  simpa using symmetric_integral_chambers_instantiated F hF

theorem symmetric_integral_chambers_fin_one_test
    (F : (Fin 1 → ℝ) → ℝ≥∞0) (hF : PermutationInvariant F) :
    ∫⁻( x in nonnegativeOrthant 1, F x ∂coordinateLebesgue (Fin 1)) =
      ∫⁻ x in strictOrderedPositive 1, F x ∂coordinateLebesgue (Fin 1) := by
  simpa using symmetric_integral_chambers_instantiated F hF

theorem symmetric_integral_chambers_fin_two_test
    (F : (Fin 2 → ℝ) → ℝ≥∞0) (hF : PermutationInvariant F) :
    ∫⁻( x in nonnegativeOrthant 2, F x ∂coordinateLebesgue (Fin 2)) =
      2 * ∫⁻ x in strictOrderedPositive 2, F x ∂coordinateLebesgue (Fin 2) := by
  simpa using symmetric_integral_chambers_instantiated F hF
```

```
def descendingPair : Fin 2 → ℝ := fun i => if i = 0 then 2 else 1

lemma descendingPair_mem_nonnegativeOrthant : descendingPair ∈ nonnegativeOrthant 2 := by
  intro i
  fin_cases i <;> norm_num [descendingPair]

lemma descendingPair_not_mem_strictOrderedPositive :
    descendingPair ∉ strictOrderedPositive 2 := by
  intro h
  have h01 := h.2 (show (0 : Fin 2) < 1 by decide)
  norm_num [descendingPair] at h01

theorem one_chamber_does_not_cover_fin_two :
    nonnegativeOrthant 2 ≠ strictOrderedPositive 2 := by
  intro h
  have hx : descendingPair ∈ strictOrderedPositive 2 := h ▸ descendingPair_mem_nonnegativeOrthant
  exact descendingPair_not_mem_strictOrderedPositive hx


def tiedPair : Fin 2 → ℝ := fun _ => 1

lemma tiedPair_mem_nonnegativeOrthant : tiedPair ∈ nonnegativeOrthant 2 := by
  intro i
  norm_num [tiedPair]

lemma tiedPair_not_mem_iUnion_permutationChamber :
    tiedPair ∉ ⋃ σ : Equiv.Perm (Fin 2), permutationChamber σ := by
  rw [mem_iUnion_permutationChamber_iff]
  intro h
  have h01 : tiedPair (0 : Fin 2) = tiedPair 1 := by rfl
  have heq := h.2 h01
  norm_num at heq

/-- The chamber decomposition is not a literal partition: points with tied
coordinates lie in the orthant but in no strict chamber. -/
theorem permutationChambers_not_exact_partition_fin_two :
    nonnegativeOrthant 2 ≠ ⋃ σ : Equiv.Perm (Fin 2), permutationChamber σ := by
  intro h
  have hx : tiedPair ∈ ⋃ σ : Equiv.Perm (Fin 2), permutationChamber σ :=
    h ▸ tiedPair_mem_nonnegativeOrthant
  exact tiedPair_not_mem_iUnion_permutationChamber hx

/-- The nondecreasing rearrangement of a finite tuple. -/
def sortedCoordinates {k : ℕ} (s : Fin k → ℝ) : Fin k → ℝ :=
  s ∘ Tuple.sort s

lemma sortedCoordinates_monotone {k : ℕ} (s : Fin k → ℝ) :
    Monotone (sortedCoordinates s) :=
  Tuple.monotone_sort s

lemma sortedCoordinates_eq_self_of_monotone {k : ℕ} {s : Fin k → ℝ}
    (hs : Monotone s) : sortedCoordinates s = s := by
  have h := Tuple.unique_monotone (Tuple.monotone_sort s)
    σ( := Tuple.sort s) τ( := (1 : Equiv.Perm (Fin k))) hs
  simpa [sortedCoordinates, Function.comp_def] using h

lemma coordinatePermutation_eq_comp_symm {k : ℕ}
    σ( : Equiv.Perm (Fin k)) (s : Fin k → ℝ) :
    coordinatePermutation σ s = s ∘ σ.symm := by
  funext i
  obtain ⟨j, ⟩rfl := σ.surjective i
```

```
  simp

lemma sortedCoordinates_coordinatePermutation {k : ℕ}
    σ( : Equiv.Perm (Fin k)) (s : Fin k → ℝ) :
    sortedCoordinates (coordinatePermutation σ s) = sortedCoordinates s := by
  let y : Fin k → ℝ := coordinatePermutation σ s
  let τ : Equiv.Perm (Fin k) := (Tuple.sort y).trans σ.symm
  have hy : y = s ∘ σ.symm := coordinatePermutation_eq_comp_symm σ s
  have τh : s ∘ τ = y ∘ Tuple.sort y := by
    funext i
    simp τ[, hy, Function.comp_def]
  have τhmono : Monotone (s ∘ τ) := by
    rw [τh]
    exact Tuple.monotone_sort y
  have heq : s ∘ Tuple.sort s = s ∘ τ :=
    Tuple.unique_monotone (Tuple.monotone_sort s) τhmono
  unfold sortedCoordinates
  change y ∘ Tuple.sort y = s ∘ Tuple.sort s
  exact τh.symm.trans heq.symm

/-- The ordered Laguerre density extended symmetrically to the full orthant by
sorting its coordinates. -/
def sortedLaguerreIntegrand {k : ℕ} (eta rho T : ℝ)
    (s : Fin k → ℝ) : ℝ≥∞0 :=
  ENNReal.ofReal (laguerreChamberIntegrand eta rho T (sortedCoordinates s))

lemma sortedLaguerreIntegrand_permutationInvariant {k : ℕ}
    (eta rho T : ℝ) : PermutationInvariant (sortedLaguerreIntegrand (k := k) eta rho T) := by
  intro σ s
  simp [sortedLaguerreIntegrand, sortedCoordinates_coordinatePermutation]

lemma strictOrderedPositive_subset_orderedPositiveChamber (k : ℕ) :
    strictOrderedPositive k ⊆ orderedPositiveChamber k := by
  intro s hs
  exact mem_orderedPositiveChamber_iff.mpr
    ⟨hs.1, fun i j hij => (hs.2 hij).⟩le

lemma orderedPositiveChamber_diff_strict_subset_ties (k : ℕ) :
    orderedPositiveChamber k \ strictOrderedPositive k ⊆ tieHyperplanes k := by
  intro s hs
  apply (mem_tieHyperplanes_iff_not_injective s).2
  intro hinj
  have hord := (mem_orderedPositiveChamber_iff.mp hs.1).2
  have hstrict : StrictMono s := by
    intro i j hij
    exact lt_of_le_of_ne (hord hij) (fun h => hij.ne (hinj h))
  exact hs.2 ⟨(mem_orderedPositiveChamber_iff.mp hs.1).1, ⟩hstrict

lemma strict_orderedPositiveChamber_ae_eq (k : ℕ) :
    strictOrderedPositive k ᵐ=[coordinateLebesgue (Fin k)] orderedPositiveChamber k := by
  apply MeasureTheory.ae_eq_set.mpr
  constructor
  · have hsub := strictOrderedPositive_subset_orderedPositiveChamber k
    simp [Set.sdiff_eq_empty.2 hsub]
  · exact MeasureTheory.measure_mono_null
      (orderedPositiveChamber_diff_strict_subset_ties k)
      (tieHyperplanes_null k)

lemma sortedCoordinates_eq_self_of_mem_orderedPositiveChamber {k : ℕ}
    {s : Fin k → ℝ} (hs : s ∈ orderedPositiveChamber k) :
    sortedCoordinates s = s := by
  apply sortedCoordinates_eq_self_of_monotone
```

```
  intro i j hij
  obtain rfl | hlt := hij.eq_or_lt
  · exact le_rfl
  · exact (mem_orderedPositiveChamber_iff.mp hs).2 hlt

lemma sortedLaguerreIntegrand_eq_on_orderedPositiveChamber {k : ℕ}
    (eta rho T : ℝ) {s : Fin k → ℝ} (hs : s ∈ orderedPositiveChamber k) :
    sortedLaguerreIntegrand eta rho T s =
      ENNReal.ofReal (laguerreChamberIntegrand eta rho T s) := by
  simp [sortedLaguerreIntegrand,
    sortedCoordinates_eq_self_of_mem_orderedPositiveChamber hs]

lemma sortedLaguerreIntegrand_lintegral_strict_eq_chamber {k : ℕ}
    {eta rho T : ℝ} (heta : -1 < eta) (hrho : 0 ≤ rho) (hT : 0 ≤ T) :
    ∫⁻ s in strictOrderedPositive k,
        sortedLaguerreIntegrand eta rho T s
        ∂coordinateLebesgue (Fin k)) =
      ENNReal.ofReal (laguerreChamberIntegral (k := k) eta rho T) := by
  rw [Measure.restrict_congr_set (strict_orderedPositiveChamber_ae_eq k)]
  have hfun :
      sortedLaguerreIntegrand (k := k) eta rho T
          =ᵐ[(coordinateLebesgue (Fin k)).restrict (orderedPositiveChamber k)]
        fun s => ENNReal.ofReal (laguerreChamberIntegrand eta rho T s) := by
    filter_upwards [ae_restrict_mem (measurableSet_orderedPositiveChamber k)] with s hs
    exact sortedLaguerreIntegrand_eq_on_orderedPositiveChamber eta rho T hs
  rw [lintegral_congr_ae hfun]
  have hInt : Integrable
      (laguerreChamberIntegrand (k := k) eta rho T)
      ((coordinateLebesgue (Fin k)).restrict (orderedPositiveChamber k)) := by
    change IntegrableOn (laguerreChamberIntegrand (k := k) eta rho T)
      (orderedPositiveChamber k) (Measure.pi fun _ : Fin k => volume)
    exact laguerreChamberIntegrand_integrableOn_chamber (k := k) heta hrho hT
  have hNonneg :
      0 ≤ᵐ[(coordinateLebesgue (Fin k)).restrict (orderedPositiveChamber k)]
        laguerreChamberIntegrand (k := k) eta rho T := by
    change 0 ≤ᵐ[(Measure.pi fun _ : Fin k => volume).restrict (orderedPositiveChamber k)]
      laguerreChamberIntegrand (k := k) eta rho T
    exact laguerreChamberIntegrand_nonnegOn_chamber (k := k) eta rho T hT
  rw ←[ ofReal_integral_eq_lintegral_ofReal hInt hNonneg]
  rfl

/-- The full nonnegative orthant integral of the symmetrically sorted Laguerre
density is exactly `k!` times the already checked ordered-chamber integral. -/
theorem sortedLaguerre_lintegral_orthant {k : ℕ}
    {eta rho T : ℝ} (heta : -1 < eta) (hrho : 0 ≤ rho) (hT : 0 ≤ T) :
    ∫⁻ s in nonnegativeOrthant k,
        sortedLaguerreIntegrand eta rho T s
        ∂coordinateLebesgue (Fin k)) =
      (k.factorial : ℝ≥∞0) *
        ENNReal.ofReal (laguerreChamberIntegral (k := k) eta rho T) := by
  rw [symmetric_integral_chambers_instantiated
    (sortedLaguerreIntegrand (k := k) eta rho T)
    (sortedLaguerreIntegrand_permutationInvariant eta rho T),
    sortedLaguerreIntegrand_lintegral_strict_eq_chamber heta hrho hT]

/-- In two eigenvalue variables the chamber coefficient is visibly `2`. -/
theorem sortedLaguerre_lintegral_fin_two
    {eta rho T : ℝ} (heta : -1 < eta) (hrho : 0 ≤ rho) (hT : 0 ≤ T) :
    ∫⁻ s in nonnegativeOrthant 2,
        sortedLaguerreIntegrand eta rho T s
        ∂coordinateLebesgue (Fin 2)) =
      2 * ENNReal.ofReal (laguerreChamberIntegral (k := 2) eta rho T) := by
```

```
  simpa using sortedLaguerre_lintegral_orthant (k := 2) heta hrho hT


/-- The left-Gram eigenvalue tuple, sorted into nondecreasing order. -/
def sortedLeftGramEigenvalues {k d : ℕ}
    (Z : Matrix (Fin k) (Fin d) ℝ) : Fin k → ℝ :=
  sortedCoordinates ((leftGram_posSemidef Z).isHermitian.eigenvalues)

/-- The right-Gram eigenvalue tuple, sorted into nondecreasing order. -/
def sortedRightGramEigenvalues {d k : ℕ}
    (Z : Matrix (Fin d) (Fin k) ℝ) : Fin k → ℝ :=
  sortedCoordinates ((rightGram_posSemidef Z).isHermitian.eigenvalues)

lemma sortedLeftGramEigenvalues_monotone {k d : ℕ}
    (Z : Matrix (Fin k) (Fin d) ℝ) : Monotone (sortedLeftGramEigenvalues Z) :=
  sortedCoordinates_monotone _

lemma sortedRightGramEigenvalues_monotone {d k : ℕ}
    (Z : Matrix (Fin d) (Fin k) ℝ) : Monotone (sortedRightGramEigenvalues Z) :=
  sortedCoordinates_monotone _

lemma sortedLeftGramEigenvalues_nonneg {k d : ℕ}
    (Z : Matrix (Fin k) (Fin d) ℝ) (i : Fin k) :
    0 ≤ sortedLeftGramEigenvalues Z i := by
  exact leftGram_eigenvalues_nonneg Z (Tuple.sort _ i)

lemma sortedRightGramEigenvalues_nonneg {d k : ℕ}
    (Z : Matrix (Fin d) (Fin k) ℝ) (i : Fin k) :
    0 ≤ sortedRightGramEigenvalues Z i := by
  exact rightGram_eigenvalues_nonneg Z (Tuple.sort _ i)

lemma leftGram_shift_det_sorted {k d : ℕ}
    (T : ℝ) (Z : Matrix (Fin k) (Fin d) ℝ) :
    Matrix.det (1 + T • (Z * Z.transpose)) =
      ∏ i : Fin k, (1 + T * sortedLeftGramEigenvalues Z i) := by
  rw [leftGram_shift_det]
  unfold sortedLeftGramEigenvalues sortedCoordinates
  exact (Equiv.prod_comp
    (Tuple.sort ((leftGram_posSemidef Z).isHermitian.eigenvalues))
    (fun i : Fin k => 1 + T * (leftGram_posSemidef Z).isHermitian.eigenvalues i)).symm

lemma largeGram_shift_det_via_sortedRight {d k : ℕ}
    (T : ℝ) (Z : Matrix (Fin d) (Fin k) ℝ) :
    Matrix.det (1 + T • (Z * Z.transpose)) =
      ∏ i : Fin k, (1 + T * sortedRightGramEigenvalues Z i) := by
  rw [largeGram_shift_det_via_right]
  unfold sortedRightGramEigenvalues sortedCoordinates
  exact (Equiv.prod_comp
    (Tuple.sort ((rightGram_posSemidef Z).isHermitian.eigenvalues))
    (fun i : Fin k => 1 + T * (rightGram_posSemidef Z).isHermitian.eigenvalues i)).symm

/-- The concrete sorted spectral package when the left Gram matrix is the smaller one. -/
theorem sortedLeftGram_spectral_package {k d : ℕ}
    (T : ℝ) (Z : Matrix (Fin k) (Fin d) ℝ) :
    Monotone (sortedLeftGramEigenvalues Z) ∧
      ∀( i, 0 ≤ sortedLeftGramEigenvalues Z i) ∧
      Matrix.det (1 + T • (Z * Z.transpose)) =
        ∏ i : Fin k, (1 + T * sortedLeftGramEigenvalues Z i) :=
  ⟨sortedLeftGramEigenvalues_monotone Z,
    sortedLeftGramEigenvalues_nonneg Z,
    leftGram_shift_det_sorted T ⟩Z
```

```
/-- The concrete sorted spectral package when the right Gram matrix is the smaller one. -/
theorem sortedRightGram_spectral_package {d k : ℕ}
    (T : ℝ) (Z : Matrix (Fin d) (Fin k) ℝ) :
    Monotone (sortedRightGramEigenvalues Z) ∧
      ∀( i, 0 ≤ sortedRightGramEigenvalues Z i) ∧
      Matrix.det (1 + T • (Z * Z.transpose)) =
        ∏ i : Fin k, (1 + T * sortedRightGramEigenvalues Z i) :=
  ⟨sortedRightGramEigenvalues_monotone Z,
    sortedRightGramEigenvalues_nonneg Z,
    largeGram_shift_det_via_sortedRight T ⟩Z

end O77.Residual
```

Corresponding Lean declaration(s): O77.Residual.coordinateDifference,O77.Residual.tieHyperplane,O77.Residual.tieHyperplane_eq_ker,O77.Residual.coordinateDifference_ne_zero,O77.Residual.tieHyperplane_null,O77.Residual.DistinctPair,O77.Residual.tieHyperplanes,O77.Residual.tieHyperplanes_null,O77.Residual.mem_tieHyperplanes_iff_not_injective,O77.Residual.nonnegativeOrthant,O77.Residual.positiveOrthant,O77.Residual.strictOrderedPositive,O77.Residual.weakOrderedNonnegative,O77.Residual.measurableSet_nonnegativeOrthant,O77.Residual.measurableSet_positiveOrthant,O77.Residual.measurableSet_strictOrder,O77.Residual.measurableSet_monotone,O77.Residual.measurableSet_strictOrderedPositive,O77.Residual.measurableSet_weakOrderedNonnegative,O77.Residual.coordinateZeroHyperplane,O77.Residual.coordinateZeroHyperplane_eq_ker,O77.Residual.coordinateProjection_ne_zero,O77.Residual.coordinateZeroHyperplane_null,O77.Residual.coordinateZeroHyperplanes,O77.Residual.coordinateZeroHyperplanes_null,O77.Residual.mem_coordinateZeroHyperplanes_iff,O77.Residual.positiveOrthant_subset_nonnegativeOrthant,O77.Residual.nonnegative_diff_positive_subset_zeroHyperplanes,O77.Residual.positive_nonnegative_ae_eq,O77.Residual.coordinatePermutation,O77.Residual.coordinatePermutation_apply_apply,O77.Residual.coordinatePermutation_symm_apply_apply,O77.Residual.permutationChamber,O77.Residual.permutationChamber_eq_image,O77.Residual.measurableSet_permutationChamber,O77.Residual.injective_of_mem_permutationChamber,O77.Residual.permutationChambers_pairwise_disjoint,O77.Residual.mem_iUnion_permutationChamber_iff,O77.Residual.iUnion_permutationChamber_eq,O77.Residual.permutationChambers_ae_partition,O77.Residual.strictOrderedPositive_subset_weakOrderedNonnegative,O77.Residual.weakOrderedNonnegative_diff_strict_subset_boundary,O77.Residual.strict_weak_ae_eq,O77.Residual.coordinatePermutation_measurePreserving,O77.Residual.PermutationInvariant,O77.Residual.setLIntegral_permutationChamber,O77.Residual.symmetric_integral_chambers_instantiated,O77.Residual.strict_weak_chamber_lintegral_eq,O77.Residual.symmetric_integral_weak_chamber_instantiated,O77.Residual.nonnegativeOrthant_zero,O77.Residual.strictOrderedPositive_zero,O77.Residual.weakOrderedNonnegative_zero,O77.Residual.symmetric_integral_chambers_empty_test,O77.Residual.symmetric_integral_chambers_fin_one_test,O77.Residual.symmetric_integral_chambers_fin_two_test,O77.Residual.descendingPair,O77.Residual.descendingPair_mem_nonnegativeOrthant,O77.Residual.descendingPair_not_mem_strictOrderedPositive,O77.Residual.one_chamber_does_not_cover_fin_two,O77.Residual.tiedPair,O77.Residual.tiedPair_mem_nonnegativeOrthant,O77.Residual.tiedPair_not_mem_iUnion_permutationChamber,O77.Residual.permutationChambers_not_exact_partition_fin_two,O77.Residual.sortedCoordinates,O77.Residual.sortedCoordinates_monotone,O77.Residual.sortedCoordinates_eq_self_of_monotone,O77.Residual.coordinatePermutation_eq_comp_symm,O77.Residual.sortedCoordinates_coordinatePermutation,O77.Residual.sortedLaguerreIntegrand,O77.Residual.sortedLaguerreIntegrand_permutationInvariant,O77.Residual.strictOrderedPositive_subset_orderedPositiveChamber,O77.Residual.orderedPositiveChamber_diff_strict_subset_ties,O77.Residual.strict_orderedPositiveChamber_ae_eq,O77.Residual.sortedCoordinates_eq_self_of_mem_orderedPositiveChamber,O77.Residual.sortedLaguerreIntegrand_eq_on_orderedPositiveChamber,O77.Residual.sortedLaguerreIntegrand_lintegral_strict_eq_chamber,O77.Residual.sortedLaguerre_lintegral_orthant,O77.Residual.sortedLaguerre_lintegral_fin_two,O77.Residual.sortedLeftGramEigenvalues,O77.Residual.sortedRightGramEigenvalues,O77.Residual.sortedLeftGramEigenvalues_monotone,O77.Residual.sortedRightGramEigenvalues_monotone,O77.Residual.sortedLeftGramEigenvalues_nonneg,O77.Residual.sortedRightGramEigenvalues_nonneg,O77.Residual.leftGram_shift_det_sorted,O77.Residual.largeGram_shift_det_via_sortedRight,O77.Residual.sortedLeftGram_spectral_package,O77.Residual.sortedRightG

`ram_spectral_package`.

25 The concrete real-Wishart interface: exact intermediate spectral interface

This section does not prove the real Wishart change of variables. Its purpose is narrower and load-bearing: it replaces an informal external-node name by an exact proposition in the matrix, measure, and eigenvalue conventions already fixed in the checked development. It then proves every deterministic step from that proposition to the ordered Laguerre chamber integral. The resulting ordinary theorem parameter is visible in the strongest theorem type; no field of `Answers077` and no residual `BandPair` conclusion occurs in the external proposition.

The normalization is the one used throughout the residual route. Matrix entries are integrated against ordinary coordinate Lebesgue measure and carry the unnormalized Gaussian weight $e^{-\|Z\|_F^2}$. After normalization, one entry has law $N(0,1/2)$, not $N(0,1)$. Consequently, for a $k \times d$ matrix with $k \le d$, the Gram eigenvalue density has exponential factor $e^{-\sum_i s_i}$ and determinant exponent

$$\eta_{k,d} = \frac{d-k-1}{2}.$$

The exponent is a real number obtained after coercion; it is not a truncated natural subtraction. The real Vandermonde has power one. These are precisely the scale and powers stated in Proposition 19.

25.1 The boundary exponent and its exact admissible range

Definition 25.1 (Real Wishart determinant exponent)**.** *For $k, d \in \mathbb{N}$, define*

$$\eta_{k,d} := \frac{(d:\mathbb{R}) - (k:\mathbb{R}) - 1}{2}.$$

The Lean definition is `077.Residual.wishartEta`.

Lemma 25.2 (Wishart exponent bookkeeping)**.** *If $k \le d$, then $-1 < \eta_{k,d}$. More precisely,*

$$\eta_{k,k} = -\frac{1}{2}, \qquad \eta_{k,k+1} = 0, \qquad k+1 \le d \Longrightarrow 0 \le \eta_{k,d}.$$

The implication $k \le d \Rightarrow 0 \le \eta_{k,d}$ is false.

Proof. Coercing $k \le d$ to $\mathbb{R}$ gives $d - k \ge 0$, hence $d - k - 1 \ge -1 > -2$ and division by the positive number 2 gives $\eta_{k,d} > -1$. Substitution gives the two displayed boundary values. Under $k+1 \le d$ one has $d-k-1 \ge 0$, giving nonnegativity. For the rejected strengthening take $k = d = 1$: then $\eta_{1,1} = -1/2 < 0$. □

This distinction matters analytically. The ordered-Laguerre theorem needs only $\eta > -1$, and therefore includes square residual matrices. Replacing that hypothesis by $\eta \ge 0$ would discard the square case and would not be a harmless simplification.

25.2 Concrete matrix and orthant integrals

For $Z \in \mathbb{R}^{k \times d}$ define the nonnegative extended-real integrand

$$\mathcal{G}^{\mathrm{L}}_{k,d;\rho,T}(Z) := \mathrm{ofReal}\Big(e^{-\|Z\|_F^2} \det(I_k + TZZ^{\mathsf{T}})^{-\rho}\Big).$$

For $Z \in \mathbb{R}^{d\times k}$ define analogously

$$\mathcal{G}^{\mathrm{R}}_{d,k;\rho,T}(Z) := \mathrm{ofReal}\Big(e^{-\|Z\|_F^2}\det(I_d + TZZ^{\mathsf{T}})^{-\rho}\Big).$$

The definitions use `ENNReal.ofReal` so that the external identity and all subsequent applications are stated as nonnegative integrals. When $T \geq 0$, the checked Gram positivity implies that the displayed determinants are positive, so this coercion does not alter the intended real integrand. The associated matrix-space integrals are

$$\mathcal{J}^{\mathrm{L}}_{k,d}(\rho,T) = \int_{\mathbb{R}^{k\times d}} \mathcal{G}^{\mathrm{L}}_{k,d;\rho,T}(Z)\,dZ, \qquad \mathcal{J}^{\mathrm{R}}_{d,k}(\rho,T) = \int_{\mathbb{R}^{d\times k}} \mathcal{G}^{\mathrm{R}}_{d,k;\rho,T}(Z)\,dZ,$$

where dZ is the already checked `matrixLebesgue` product measure.

The spectral-side integral is not introduced through a new eigenvalue-measurability assertion. Instead it uses the symmetric sorted extension of the Laguerre density defined in the preceding chamber section:

$$\mathcal{L}_k(\eta,\rho,T) := \int_{[0,\infty)^k} \mathrm{ofReal}\big(\mathcal{I}_{k,\eta,\rho,T}(\mathrm{sort}(s))\big)\,ds.$$

This is `O77.Residual.laguerreOrthantLIntegral`. The already checked chamber theorem gives, for $-1 < \eta$, $0 \leq \rho$, and $0 \leq T$,

$$\mathcal{L}_k(\eta,\rho,T) = k!\ \mathrm{ofReal}\big(I_{k,\eta,\rho}(T)\big),$$

including $k = 0$.

Two pointwise compatibility lemmas connect the matrix integrands to the previously checked smaller-Gram spectra. In the left orientation,

$$\det(I_k + TZZ^{\mathsf{T}}) = \prod_{i=0}^{k-1}(1 + Ts_i^{\mathrm{L}}(Z));$$

in the right orientation, Sylvester's identity first replaces the $d \times d$ determinant by the $k \times k$ right-Gram determinant and then gives the analogous product in the sorted right-Gram eigenvalues. The formal declarations `leftGramLaplaceIntegrand_eq_sorted` and `rightGramLaplaceIntegrand_eq_sorted` are exact rewrites of these identities. Furthermore,

$$\|Z^{\mathsf{T}}\|_F^2 = \|Z\|_F^2$$

and the rectangular Sylvester identity imply the pointwise transpose compatibility

$$\mathcal{G}^{\mathrm{R}}_{d,k;\rho,T}(Z^{\mathsf{T}}) = \mathcal{G}^{\mathrm{L}}_{k,d;\rho,T}(Z).$$

This does not by itself prove equality of the two matrix integrals: that last step would require a separate measure-preserving theorem for matrix transposition. The external interface below instead requires one common normalization for both orientations, which is the mathematically correct normalization and prevents the two cases from drifting apart.

25.3 The specialized external proposition

Definition 25.3 (Concrete real-Wishart formula)**.** *Fix $k, d \in \mathbb{N}$ and $C \in [0,\infty]$. The proposition `O77.Residual.HasConcreteRealWishartFormulakdC` means:*

1. *$k \leq d$;*
2. *$0 < C < \infty$;*

3. for every $\rho, T \in \mathbb{R}$ with $0 \le \rho$ and $0 \le T$,

$$\mathcal{J}^{\mathrm{L}}_{k,d}(\rho, T) = C\,\mathcal{L}_k(\eta_{k,d}, \rho, T)$$

and

$$\mathcal{J}^{\mathrm{R}}_{d,k}(\rho, T) = C\,\mathcal{L}_k(\eta_{k,d}, \rho, T).$$

The existential proposition `O77.Residual.ConcreteRealWishartEigenvalueFormulakd` is $\exists C\,\mathsf{HasConcreteRealWishartFormula}(k, d, C)$*. Finally, `O77.External.RealWishartEigenvalueFormula` asserts this existential statement for every $k \le d$.*

The quantifier order is deliberate. The constant $C = C_{k,d}$ is chosen before ρ and T, so it cannot absorb a parameter-dependent power of T or a logarithm. The same C is used in both rectangular orientations. Its strict positivity is used for the subsequent lower estimate, and its finiteness is needed to avoid an identically infinite right-hand side. The checked negative controls `zero_is_not_concreteWishart_normalization` and `top_is_not_concreteWishart_normalization` show that neither 0 nor ∞ satisfies the interface, without making any claim about the values of the integrals.

The interface is intentionally specialized to the two-parameter family of test functions required by O77, namely determinant powers with $\rho, T \ge 0$. It is weaker than the generic Borel-function formula in Proposition 19. Therefore a proof of the generic published formula would imply this interface, but the converse would not recover the full Wishart pushforward measure. This specialization is acceptable for the residual theorem because no downstream O77 argument uses another test function. It also avoids falsely claiming that this source has proved Borel measurability of the sorted eigenvalue map as a function of Z.

Theorem 25.4 (Conditional reduction to the ordered Laguerre chamber)**.** *Assume `O77.Residual.HasConcreteRealWishartFormulakdC`. For $\rho, T \ge 0$,*

$$\mathcal{J}^{\mathrm{L}}_{k,d}(\rho, T) = C\,k!\,\mathrm{ofReal}\big(I_{k,\eta_{k,d},\rho}(T)\big)$$

and

$$\mathcal{J}^{\mathrm{R}}_{d,k}(\rho, T) = C\,k!\,\mathrm{ofReal}\big(I_{k,\eta_{k,d},\rho}(T)\big).$$

Equivalently, the global proposition `O77.External.RealWishartEigenvalueFormula` implies the existence of one positive finite C for which both equalities hold.

Proof. The external premise first identifies each matrix integral with $C\mathcal{L}_k(\eta_{k,d}, \rho, T)$. Its rank field gives $k \le d$, so Lemma 25.2 yields $-1 < \eta_{k,d}$. The checked theorem `sortedLaguerre_integral_orthant` is therefore applicable under the stated hypotheses $\rho, T \ge 0$ and gives

$$\mathcal{L}_k(\eta_{k,d}, \rho, T) = k!\,\mathrm{ofReal}\big(I_{k,\eta_{k,d},\rho}(T)\big).$$

Substitution and associativity of multiplication in $\mathbb{R}^{\infty}_{\ge 0}$ prove both identities. No cancellation by C is used. The existential form unpacks the global external proposition, retains its witness C, and applies the two preceding identities. □

For $d = k$ the formal specialization rewrites $\eta_{k,k}$ to $-1/2$. This control is important: the square case is precisely where a natural-subtraction encoding of $d-k-1$ would silently produce the wrong exponent 0.

25.4 Concrete Wishart interface: Lean code

The following block adds no import and uses only declarations already present in the pinned profile.

```
set_option autoImplicit false

noncomputable section

open Real MeasureTheory Set
open scoped ENNReal BigOperators

namespace O77.Residual

/-- The determinant exponent in the real Wishart eigenvalue density for a
`k × d` matrix with `k ≤ d` and Gaussian weight `exp ‖(-‖Z_F²)`. -/
def wishartEta (k d : ℕ) : ℝ :=
  ((d : ℝ) - (k : ℝ) - 1) / 2

lemma wishartEta_gt_neg_one {k d : ℕ} (hkd : k ≤ d) :
    -1 < wishartEta k d := by
  unfold wishartEta
  have hcast : (k : ℝ) ≤ (d : ℝ) := by exact_mod_cast hkd
  linarith

@[simp] lemma wishartEta_square (k : ℕ) :
    wishartEta k k = ((-1 : ℝ) / 2) := by
  simp [wishartEta]

@[simp] lemma wishartEta_successor (k : ℕ) :
    wishartEta k (k + 1) = 0 := by
  simp [wishartEta]

lemma wishartEta_nonneg {k d : ℕ} (hkd : k + 1 ≤ d) :
    0 ≤ wishartEta k d := by
  unfold wishartEta
  have hcast : ((k + 1 : ℕ) : ℝ) ≤ (d : ℝ) := by exact_mod_cast hkd
  norm_num at hcast ⊢
  linarith

/-- The exponent is not always nonnegative in the legitimate range `k ≤ d`:
the square case has exponent `-1/2`. -/
theorem wishartEta_not_always_nonneg :
    ¬ 0 ≤ wishartEta 1 1 := by
  norm_num [wishartEta]

/-- The concrete left-Gram matrix integrand after the Gaussian rows have been
integrated out. The nonnegative-real codomain keeps the later Tonelli/Wishart
step free of hidden integrability assumptions. -/
def leftGramLaplaceIntegrand {k d : ℕ} (rho T : ℝ)
    (Z : Matrix (Fin k) (Fin d) ℝ) : ℝ≥∞0 :=
  ENNReal.ofReal
    (Real.exp (-frobeniusSq Z) *
      Matrix.det (1 + T • (Z * Z.transpose)) ^ (-rho))

/-- The same concrete matrix integrand in the transposed rectangular orientation. -/
def rightGramLaplaceIntegrand {d k : ℕ} (rho T : ℝ)
    (Z : Matrix (Fin d) (Fin k) ℝ) : ℝ≥∞0 :=
  ENNReal.ofReal
    (Real.exp (-frobeniusSq Z) *
      Matrix.det (1 + T • (Z * Z.transpose)) ^ (-rho))
```

```
lemma leftGramLaplaceIntegrand_eq_sorted {k d : ℕ} (rho T : ℝ)
    (Z : Matrix (Fin k) (Fin d) ℝ) :
    leftGramLaplaceIntegrand rho T Z =
      ENNReal.ofReal
        (Real.exp (-frobeniusSq Z) *
          ∏( i : Fin k, (1 + T * sortedLeftGramEigenvalues Z i)) ^ (-rho)) := by
  rw [leftGramLaplaceIntegrand, leftGram_shift_det_sorted]

lemma rightGramLaplaceIntegrand_eq_sorted {d k : ℕ} (rho T : ℝ)
    (Z : Matrix (Fin d) (Fin k) ℝ) :
    rightGramLaplaceIntegrand rho T Z =
      ENNReal.ofReal
        (Real.exp (-frobeniusSq Z) *
          ∏( i : Fin k, (1 + T * sortedRightGramEigenvalues Z i)) ^ (-rho)) := by
  rw [rightGramLaplaceIntegrand, largeGram_shift_det_via_sortedRight]

lemma frobeniusSq_transpose {m n : Type*} [Fintype m] [Fintype n]
    (Z : Matrix m n ℝ) : frobeniusSq Z.transpose = frobeniusSq Z := by
  unfold frobeniusSq
  rw [Finset.sum_comm]
  rfl

lemma rightGramLaplaceIntegrand_transpose {k d : ℕ} (rho T : ℝ)
    (Z : Matrix (Fin k) (Fin d) ℝ) :
    rightGramLaplaceIntegrand rho T Z.transpose =
      leftGramLaplaceIntegrand rho T Z := by
  unfold rightGramLaplaceIntegrand leftGramLaplaceIntegrand
  rw [frobeniusSq_transpose, Matrix.transpose_transpose,
    sylvester_gram_determinant]

/-- Full matrix-space nonnegative integral in the orientation in which the
left Gram matrix has size `k`. -/
def leftGramLaplaceLIntegral (k d : ℕ) (rho T : ℝ) : ℝ≥∞0 :=
  ∫⁻ Z : Matrix (Fin k) (Fin d) ℝ,
    leftGramLaplaceIntegrand rho T Z ∂matrixLebesgue (Fin k) (Fin d)

/-- Full matrix-space nonnegative integral in the transposed orientation. -/
def rightGramLaplaceLIntegral (d k : ℕ) (rho T : ℝ) : ℝ≥∞0 :=
  ∫⁻ Z : Matrix (Fin d) (Fin k) ℝ,
    rightGramLaplaceIntegrand rho T Z ∂matrixLebesgue (Fin d) (Fin k)

/-- The full nonnegative-orthant Laguerre integral with the symmetric sorted
extension from the chamber theorem. -/
def laguerreOrthantLIntegral (k : ℕ) (eta rho T : ℝ) : ℝ≥∞0 :=
  ∫⁻ s in nonnegativeOrthant k,
    sortedLaguerreIntegrand eta rho T s ∂coordinateLebesgue (Fin k)

/-- A narrow real-Wishart interface in the actual O77 matrix and measure spaces.
The same positive finite normalization constant must work in both rectangular
orientations and for every nonnegative `rho` and `T`; in particular the constant
cannot absorb any parameter-dependent power-log behaviour. -/
def HasConcreteRealWishartFormula (k d : ℕ) (C : ℝ≥∞0) : Prop :=
  k ≤ d ∧
  0 < C ∧ C < ∞ ∧
  ∀ rho T : ℝ, 0 ≤ rho → 0 ≤ T →
    leftGramLaplaceLIntegral k d rho T =
        C * laguerreOrthantLIntegral k (wishartEta k d) rho T ∧
      rightGramLaplaceLIntegral d k rho T =
        C * laguerreOrthantLIntegral k (wishartEta k d) rho T

/-- Existence of a positive finite normalizer for the exact matrix-to-spectrum
identity. The global theorem `O77.GlobalSpectralIntegral.realWishartEigenvalueFormula`
```

```
proves this proposition for every `k ≤ d`. -/
def ConcreteRealWishartEigenvalueFormula (k d : ℕ) : Prop :=
  ∃ C : ℝ≥∞0, HasConcreteRealWishartFormula k d C

end O77.Residual

namespace O77.External

/-- Global Wishart identity used by the residual calculation. Its proof is
`O77.GlobalSpectralIntegral.realWishartEigenvalueFormula`; the namespace name
`External` is retained for compatibility with the conditional intermediate lemmas. -/
def RealWishartEigenvalueFormula : Prop :=
  ∀ k d : ℕ, k ≤ d → O77.Residual.ConcreteRealWishartEigenvalueFormula k d

end O77.External

namespace O77.Residual
namespace HasConcreteRealWishartFormula

lemma rank_le {k d : ℕ} {C : ℝ≥∞0}
    (h : HasConcreteRealWishartFormula k d C) : k ≤ d := h.1

lemma constant_pos {k d : ℕ} {C : ℝ≥∞0}
    (h : HasConcreteRealWishartFormula k d C) : 0 < C := h.2.1

lemma constant_lt_top {k d : ℕ} {C : ℝ≥∞0}
    (h : HasConcreteRealWishartFormula k d C) : C < ∞ := h.2.2.1

lemma constant_ne_zero {k d : ℕ} {C : ℝ≥∞0}
    (h : HasConcreteRealWishartFormula k d C) : C ≠ 0 :=
  ne_of_gt h.constant_pos

lemma constant_ne_top {k d : ℕ} {C : ℝ≥∞0}
    (h : HasConcreteRealWishartFormula k d C) : C ≠ ∞ :=
  ne_of_lt h.constant_lt_top

lemma left_formula {k d : ℕ} {C : ℝ≥∞0}
    (h : HasConcreteRealWishartFormula k d C)
    {rho T : ℝ} (hrho : 0 ≤ rho) (hT : 0 ≤ T) :
    leftGramLaplaceLIntegral k d rho T =
      C * laguerreOrthantLIntegral k (wishartEta k d) rho T :=
  (h.2.2.2 rho T hrho hT).1

lemma right_formula {k d : ℕ} {C : ℝ≥∞0}
    (h : HasConcreteRealWishartFormula k d C)
    {rho T : ℝ} (hrho : 0 ≤ rho) (hT : 0 ≤ T) :
    rightGramLaplaceLIntegral d k rho T =
      C * laguerreOrthantLIntegral k (wishartEta k d) rho T :=
  (h.2.2.2 rho T hrho hT).2

end HasConcreteRealWishartFormula

/-- A zero normalization is rejected at the proposition boundary, independently
of any integral evaluation. -/
theorem zero_is_not_concreteWishart_normalization (k d : ℕ) :
    ¬ HasConcreteRealWishartFormula k d 0 := by
  intro h
  exact (h.constant_ne_zero rfl)

/-- An infinite normalization is likewise excluded. -/
theorem top_is_not_concreteWishart_normalization (k d : ℕ) :
    ¬ HasConcreteRealWishartFormula k d ∞ := by
```

```
  intro h
  exact (h.constant_ne_top rfl)

/-- Conditional left-orientation reduction all the way to the already checked
ordered Laguerre chamber integral, with the chamber factor `k!` still visible. -/
theorem concreteWishart_left_to_orderedLaguerre {k d : ℕ} {C : ℝ≥∞0}
    (hW : HasConcreteRealWishartFormula k d C)
    {rho T : ℝ} (hrho : 0 ≤ rho) (hT : 0 ≤ T) :
    leftGramLaplaceLIntegral k d rho T =
      C * (k.factorial : ℝ≥∞0) *
        ENNReal.ofReal
          (laguerreChamberIntegral (k := k) (wishartEta k d) rho T) := by
  rw [hW.left_formula hrho hT]
  unfold laguerreOrthantLIntegral
  rw [sortedLaguerre_lintegral_orthant
    (wishartEta_gt_neg_one hW.rank_le) hrho hT]
  simp [mul_assoc]

/-- Conditional transposed-orientation reduction to the same ordered chamber
and the same Wishart normalization constant. -/
theorem concreteWishart_right_to_orderedLaguerre {k d : ℕ} {C : ℝ≥∞0}
    (hW : HasConcreteRealWishartFormula k d C)
    {rho T : ℝ} (hrho : 0 ≤ rho) (hT : 0 ≤ T) :
    rightGramLaplaceLIntegral d k rho T =
      C * (k.factorial : ℝ≥∞0) *
        ENNReal.ofReal
          (laguerreChamberIntegral (k := k) (wishartEta k d) rho T) := by
  rw [hW.right_formula hrho hT]
  unfold laguerreOrthantLIntegral
  rw [sortedLaguerre_lintegral_orthant
    (wishartEta_gt_neg_one hW.rank_le) hrho hT]
  simp [mul_assoc]

/-- The global external Wishart proposition implies both concrete rectangular
reductions with one common positive finite normalization. The conclusion remains
existential in that normalization and therefore cannot silently absorb a chosen
parameter-dependent factor. -/
theorem externalWishart_to_orderedLaguerre
    (hW : O77.External.RealWishartEigenvalueFormula)
    {k d : ℕ} (hkd : k ≤ d) {rho T : ℝ}
    (hrho : 0 ≤ rho) (hT : 0 ≤ T) :
    ∃ C : ℝ≥∞0,
      0 < C ∧ C < ∞ ∧
      leftGramLaplaceLIntegral k d rho T =
        C * (k.factorial : ℝ≥∞0) *
          ENNReal.ofReal
            (laguerreChamberIntegral (k := k) (wishartEta k d) rho T) ∧
      rightGramLaplaceLIntegral d k rho T =
        C * (k.factorial : ℝ≥∞0) *
          ENNReal.ofReal
            (laguerreChamberIntegral (k := k) (wishartEta k d) rho T) := by
  rcases hW k d hkd with ⟨C, ⟩hC
  exact ⟨C, hC.constant_pos, hC.constant_lt_top,
    concreteWishart_left_to_orderedLaguerre hC hrho hT,
    concreteWishart_right_to_orderedLaguerre hC hrho ⟩hT

/-- A specialization exposing the square-case boundary exponent `-1/2`. -/
theorem concreteWishart_square_to_orderedLaguerre {k : ℕ} {C : ℝ≥∞0}
    (hW : HasConcreteRealWishartFormula k k C)
    {rho T : ℝ} (hrho : 0 ≤ rho) (hT : 0 ≤ T) :
    leftGramLaplaceLIntegral k k rho T =
      C * (k.factorial : ℝ≥∞0) *
```

```
        ENNReal.ofReal
          (laguerreChamberIntegral (k := k) ((-1 : ℝ) / 2) rho T) := by
  simpa using concreteWishart_left_to_orderedLaguerre hW hrho hT

end O77.Residual
```

Corresponding Lean declaration(s): O77.Residual.wishartEta,O77.Residual.wishartEta_gt_neg_one,O77.Residual.wishartEta_square,O77.Residual.wishartEta_successor,O77.Residual.wishartEta_nonneg,O77.Residual.wishartEta_not_always_nonneg,O77.Residual.leftGramLaplaceIntegrand,O77.Residual.rightGramLaplaceIntegrand,O77.Residual.leftGramLaplaceIntegrand_eq_sorted,O77.Residual.rightGramLaplaceIntegrand_eq_sorted,O77.Residual.frobeniusSq_transpose,O77.Residual.rightGramLaplaceIntegrand_transpose,O77.Residual.leftGramLaplaceLIntegral,O77.Residual.rightGramLaplaceLIntegral,O77.Residual.laguerreOrthantLIntegral,O77.Residual.HasConcreteRealWishartFormula,O77.Residual.ConcreteRealWishartEigenvalueFormula,O77.External.RealWishartEigenvalueFormula,O77.Residual.HasConcreteRealWishartFormula.rank_le,O77.Residual.HasConcreteRealWishartFormula.constant_pos,O77.Residual.HasConcreteRealWishartFormula.constant_lt_top,O77.Residual.HasConcreteRealWishartFormula.constant_ne_zero,O77.Residual.HasConcreteRealWishartFormula.constant_ne_top,O77.Residual.HasConcreteRealWishartFormula.left_formula,O77.Residual.HasConcreteRealWishartFormula.right_formula,O77.Residual.zero_is_not_concreteWishart_normalization,O77.Residual.top_is_not_concreteWishart_normalization,O77.Residual.concreteWishart_left_to_orderedLaguerre,O77.Residual.concreteWishart_right_to_orderedLaguerre,O77.Residual.externalWishart_to_orderedLaguerre,O77.Residual.concreteWishart_square_to_orderedLaguerre.

26 The conditional Gaussian Laplace estimate

26.1 Target, conventions, and theorem boundary

This section completes the deterministic residual substep R5 from the proof spine. It does not prove the real-Wishart change of variables. Rather, it starts from the precise proposition `O77.External.RealWishartEigenvalueFormula`, defined in the concrete Wishart-interface section, constructs the actual Gaussian-weighted Laplace integral of the residual matrix-product germ, performs the entire Y-integration in the chosen matrix Lebesgue spaces, branches correctly according to which Gram matrix is smaller, and invokes the already checked ordered-Laguerre estimate. The resulting power–log theorem has the Wishart proposition as an ordinary argument in its complete Lean type.

For natural numbers p, q, h, put

$$\mathcal{K}_{p,q,h}(Y,Z) = \|YZ\|_F^2, \qquad \mathcal{N}_{p,q,h}(Y,Z) = \|Y\|_F^2 + \|Z\|_F^2,$$

where $Y \in \mathbb{R}^{p\times h}$ and $Z \in \mathbb{R}^{h\times q}$. The Gaussian residual Laplace integral is represented in extended nonnegative reals by

$$\mathcal{J}^G_{p,q,h}(T) = \int_{\mathbb{R}^{p\times h}\times\mathbb{R}^{h\times q}} \exp(-T\mathcal{K}_{p,q,h}(Y,Z) - \mathcal{N}_{p,q,h}(Y,Z))\, dY\, dZ. \tag{R5.1}$$

The measure is the product of the explicit finite-coordinate Lebesgue measures constructed in the Gaussian-row section. The codomain is $\mathbb{R}_{\geq 0} \cup \{\infty\}$ until finiteness is supplied by the later estimates; no conversion of ∞ to a real number occurs.

The source defines `residualKernel` and `residualAmbientNormSq` by the two formulas above, and `residualGaussianLaplaceIntegrand` initially as the product of the outer Z-Gaussian weight with the already checked row integrand. The lemma `residualGaussianLaplaceIntegrand_eq_exp` verifies pointwise that this product is exactly the single exponential in (R5.1). This prevents the later use of the row theorem from silently changing the Gaussian normalization.

26.2 Measurability in the actual product space

The integrand is measurable for the product measurable structure on the two matrix spaces. The proof is deliberately entrywise. Matrix transposition and multiplication are reduced to coordinate projections and finite sums; the Frobenius square is reduced to a finite double sum of squares; and these measurable real functions are composed with addition, scalar multiplication, negation, exp, and `ENNReal.ofReal`. The supporting declarations are `Measurable.o77Matrix Transpose`, `Measurable.o77MatrixMul`, and `Measurable.o77FrobeniusSq`. This direct argument avoids relying on an accidental synthesis of a Borel-space instance for the product of two matrix types.

For the one-matrix outer integrand, continuity is proved as well. If $T \geq 0$, then $M_T(Z) = I + TZZ^{\mathsf{T}}$ is positive definite by the checked Gram-positivity theorem, hence $\det M_T(Z) > 0$. Continuity of matrix multiplication, transpose, determinant, and real powers on this positive determinant locus gives continuity, and therefore measurability, of `leftGramLaplaceIntegrand`. The finite matrix product measures are supplied with explicit σ-finiteness instances, so Tonelli's theorem applies without an implicit finite-measure assumption.

26.3 Exact integration of all Y-rows

Define the positive real Gaussian constant

$$\gamma_{p,h} = \left(\left(\pi^h\right)^{1/2}\right)^p = \pi^{ph/2}. \tag{R5.2}$$

The first expression is used in Lean because it matches the output of the previously checked finite-product row theorem without introducing a separate real-exponent normalization lemma.

Fix $Z \in \mathbb{R}^{h\times q}$ and $T \geq 0$. The row-energy identity gives

$$T\|YZ\|_F^2 + \|Y\|_F^2 = \sum_{i=1}^{p} y_i M_T(Z) y_i^{\mathsf{T}}, \qquad M_T(Z) = I + TZZ^{\mathsf{T}}.$$

Since $M_T(Z)$ is positive definite, the checked arbitrary-dimensional Gaussian theorem applies to every row. Tonelli and finite-product Fubini therefore give

$$\int_{\mathbb{R}^{p\times h}} e^{-T\|YZ\|_F^2 - \|Y\|_F^2}\, dY = \left(\frac{\pi^h}{\det M_T(Z)}\right)^{p/2} = \gamma_{p,h}\, \det M_T(Z)^{-p/2}. \tag{R5.3}$$

The last equality is not delegated to automation. The proof uses positivity of $\det M_T(Z)$, `Real.div_rpow`, the natural-power rule, multiplication of real exponents, and `Real.rpow_neg`. It is formalized in `quadraticGaussianValue_pow_eq_gaussianRowsConstant_mul_det`.

The real integral is then converted to a nonnegative integral using both the proved integrability and pointwise positivity. Multiplying by $e^{-\|Z\|_F^2}$ and applying Tonelli gives the exact identity

$$\mathcal{J}^G_{p,q,h}(T) = \gamma_{p,h} \int_{\mathbb{R}^{h\times q}} e^{-\|Z\|_F^2} \det(I + TZZ^{\mathsf{T}})^{-p/2}\, dZ. \tag{R5.4}$$

This is the theorem `residualGaussianLaplaceLIntegral_eq_leftGram`. It is unconditional apart from the load-bearing hypothesis $T \geq 0$; no Wishart statement occurs in its type.

26.4 The rectangular Gram split and the exact Wishart reduction

Put

$$k = \min(h, q), \qquad d = \max(h, q), \qquad \eta = \frac{d - k - 1}{2}, \qquad \rho = \frac{p}{2}.$$

When $h \le q$, (R5.4) is in the left-Gram orientation of the concrete Wishart proposition. When $q \le h$, the same matrix integral is definitionally the right-oriented integral with $d = h$ and $k = q$; this is not an informal deletion of zero eigenvalues. The previously checked Sylvester and smaller-Gram theorems are already built into the two orientations.

The two fixed-normalization lemmas `residualGaussianLaplace_eq_orderedLaguerre_left` and `residualGaussianLaplace_eq_orderedLaguerre_right` prove, respectively in the two branches,

$$\mathcal{J}^{G}_{p,q,h}(T) = \gamma_{p,h}\, C_{k,d}\, k!\, \mathrm{ofReal}(I_{k,\eta,\rho}(T))\,, \tag{R5.5}$$

where $I_{k,\eta,\rho}$ is the checked ordered Laguerre chamber integral. The identities

$$\mathsf{wishartEta}(k,d) = \mathsf{residualSpectralEta}(q,h), \qquad p/2 = \mathsf{residualSpectralRho}(p)$$

are proved in the exact coercion conventions of the source. The factor $k!$ is the one produced by the checked almost-everywhere chamber partition and remains visible in (R5.5).

The branch-free theorem `residualGaussianLaplace_to_orderedLaguerre` has the ordinary parameter

$$h_W : \mathsf{RealWishartEigenvalueFormula}.$$

For every $T \ge 0$ it returns one $C \in (0,\infty)$ satisfying (R5.5). It does not return a normalization depending on T or ρ: the quantifier order inside the external proposition chooses $C_{k,d}$ first.

26.5 Conversion of the ordered estimate to the full Gaussian estimate

The existing theorem `residual_laguerreChamber_powerLog_bounds` is a real-valued statement: for $p > 0$ it supplies $0 < c \le C$ and large T such that

$$c\,T^{-d_0(p,q,h)/2}(\log T)^{\mu_0(p,q,h)-1} \le I_{k,\eta,p/2}(T) \le C\,T^{-d_0(p,q,h)/2}(\log T)^{\mu_0(p,q,h)-1}. \tag{R5.6}$$

The theorem `residual_laguerreChamber_laplacePowerLogBounds` transports (R5.6) to the extended-nonnegative predicate `LaplacePowerLogBounds`. Its proof raises the threshold to the maximum of the old threshold and 2, proves the power–log gauge nonnegative there, and applies monotonicity of `ENNReal.ofReal` to both real inequalities.

Two small transport lemmas then close the bookkeeping. First, `LaplacePowerLogBounds.const_mul` proves that multiplication by a fixed nonzero finite $K \in \mathbb{R}_{\ge 0} \cup \{\infty\}$ preserves the pair. It converts K to the positive real `ENNReal.toRealK` only after proving $K \ne 0$ and $K \ne \infty$. Second, `LaplacePowerLogBounds.congr_of_eq_of_ge_two` transports a bound across pointwise equality for all $T \ge 2$. This is exactly the range in which the definition of `LaplacePowerLogBounds` asks for estimates.

The strongest theorem in this section is therefore:

Theorem 26.1 (Conditional Gaussian residual Laplace order)**.** *Assume the explicit proposition* $h_W : \mathsf{RealWishartEigenvalueFormula}$*, and let* p, q, h *be positive natural numbers. Then*

$$\mathsf{LaplacePowerLogBounds}\left(\mathcal{J}^{G}_{p,q,h}, \frac{d_0(p,q,h)}{2}, \mu_0(p,q,h) - 1\right).$$

Equivalently, for some positive constants $c \le C$ *and all sufficiently large* T*,*

$$c\,T^{-d_0/2}(\log T)^{\mu_0-1} \le \mathcal{J}^{G}_{p,q,h}(T) \le C\,T^{-d_0/2}(\log T)^{\mu_0-1}.$$

The Lean name is `O77.Residual.gaussianLaplace_powerLog_bounds_of_wishart_instantiated`. Its complete ordinary parameters are h_W, $0 < p$, $0 < q$, and $0 < h$. Its signature is conditional: an axiom audit does not discharge an ordinary argument. The final answer instead supplies h_W using the proved terminal spectral theorem.

26.6 Boundary tests and a homogeneity control

The zero-row theorem `residualGaussianLaplaceLIntegral_zero_rows` verifies the row-reduction identity at $p = 0$: the Gaussian row factor is 1 and the remaining determinant exponent is $\rho = 0$. The separate negative theorem `residualSpectralRho_zero_not_pos` proves that $\rho = 0$ is not positive, so the positive-ρ Laguerre asymptotic cannot simply be invoked in this neutral case. This is why the Gaussian power–log theorem is stated only for positive residual dimensions and why zero residual factors must be packaged separately by the neutral pair $(0, 1)$.

Three explicit conditional regression instances compile:

- $(p, q, h) = (1, 1, 1)$ gives exponent $1/2$ and logarithmic degree 1, hence order $T^{-1/2} \log T$; this tests the resonant multiplicity-two case;
- $(p, q, h) = (2, 3, 1)$ gives exponent 1 and degree 0 in the left-Gram branch;
- $(p, q, h) = (2, 1, 3)$ gives the same order in the right-Gram branch.

The last two examples test both rectangular orientations rather than only square matrices.

The final declarations in this section record the homogeneity needed by the localization argument. It proves

$$\mathcal{K}_{p,q,h}(cY, cZ) = c^4 \mathcal{K}_{p,q,h}(Y, Z), \qquad \mathcal{N}_{p,q,h}(cY, cZ) = c^2 \mathcal{N}_{p,q,h}(Y, Z). \tag{R5.7}$$

The first identity uses $(cY)(cZ) = c^2(YZ)$ and the quadratic scaling of the entrywise Frobenius square. The theorem `residualKernel_not_quadratic` evaluates the 1×1 identity matrices at $c = 2$ and refutes the plausible but false quadratic replacement: the residual kernel scales by 16, not by 4. The shell measure comparison, dilation Jacobian, and summability theorem are proved in the following two residual sections.

26.7 Conditional Gaussian Laplace estimate: Lean code

The following block is part of the pinned document-order profile. It imports no precompiled local module; all dependencies occur earlier in this source.

```
set_option autoImplicit false

noncomputable section

open Real MeasureTheory Set
open scoped ENNReal BigOperators

namespace O77.Residual

/-- The scalar factor obtained by integrating `p` rows of dimension `h`
against the weight `exp ‖(-‖y²)`. -/
def gaussianRowsConstant (p h : ℕ) : ℝ :=
  (((Real.pi ^ h : ℝ) ^ (1 / 2 : ℝ)) ^ p)

lemma gaussianRowsConstant_pos (p h : ℕ) : 0 < gaussianRowsConstant p h := by
  unfold gaussianRowsConstant
  positivity

lemma quadraticGaussianValue_pow_eq_gaussianRowsConstant_mul_det
    {p h : ℕ} (M : Matrix (Fin h) (Fin h) ℝ) (hM : Matrix.PosDef M) :
    (quadraticGaussianValue M) ^ p =
      gaussianRowsConstant p h * Matrix.det M ^ (-(p : ℝ) / 2) := by
  have hdet : 0 < Matrix.det M := hM.det_pos
  have hpi : 0 ≤ (Real.pi ^ h : ℝ) := by positivity
  unfold quadraticGaussianValue gaussianRowsConstant
```

```
  simp only [Fintype.card_fin]
  rw [Real.div_rpow hpi hdet.le]
  rw [div_pow]
  have hden : ((Matrix.det M) ^ (1 / 2 : ℝ)) ^ p =
      Matrix.det M ^ ((p : ℝ) / 2) := by
    rw ←[ Real.rpow_natCast]
    rw ←[ Real.rpow_mul hdet.le]
    have hexp : (1 / 2 : ℝ) * (p : ℝ) = (p : ℝ) / 2 := by ring
    rw [hexp]
  rw [hden, div_eq_mul_inv, ← Real.rpow_neg hdet.le]
  have hexp : -((p : ℝ) / 2) = -(p : ℝ) / 2 := by ring
  rw [hexp]

end O77.Residual

noncomputable section

open Real MeasureTheory Set
open scoped ENNReal BigOperators

namespace O77.Residual

/-- The rowwise real Gaussian formula, converted to an extended-nonnegative
integral and split into its constant and determinant factors. -/
theorem gaussianRows_lintegral_instantiated {p h q : ℕ}
    (T : ℝ) (hT : 0 ≤ T) (Z : Matrix (Fin h) (Fin q) ℝ) :
    ∫⁻ Y : Matrix (Fin p) (Fin h) ℝ,
        ENNReal.ofReal (gaussianRowsIntegrand T Z Y)
        ∂matrixLebesgue (Fin p) (Fin h)) =
      ENNReal.ofReal (gaussianRowsConstant p h) *
        ENNReal.ofReal
          (Matrix.det (1 + T • (Z * Z.transpose)) ^ (-(p : ℝ) / 2)) := by
  have hInt := gaussian_rows_integrable (p := Fin p) T hT Z
  have hnonneg : 0 ≤ᵐ[matrixLebesgue (Fin p) (Fin h)]
      (fun Y : Matrix (Fin p) (Fin h) ℝ => gaussianRowsIntegrand T Z Y) :=
    Filter.Eventually.of_forall fun Y => (Real.exp_pos _).le
  rw ←[ ofReal_integral_eq_lintegral_ofReal hInt hnonneg]
  rw [gaussian_rows_instantiated (p := Fin p) T hT Z]
  simp only [Fintype.card_fin]
  rw [quadraticGaussianValue_pow_eq_gaussianRowsConstant_mul_det
    (1 + T • (Z * Z.transpose)) (gram_posDef T hT Z)]
  rw [ENNReal.ofReal_mul (gaussianRowsConstant_pos p h).le]

end O77.Residual

noncomputable section

open Real MeasureTheory Set
open scoped ENNReal BigOperators

namespace O77.Residual


noncomputable instance matrixLebesgue.instSigmaFiniteFin
    (p h : ℕ) : SigmaFinite (matrixLebesgue (Fin p) (Fin h)) := by
  unfold matrixLebesgue
  exact Measure.pi.sigmaFinite (fun _ : Fin p => coordinateLebesgue (Fin h))

lemma continuous_frobeniusSq {m n : Type*} [Fintype m] [Fintype n] :
    Continuous (frobeniusSq : Matrix m n ℝ → ℝ) := by
  unfold frobeniusSq
  apply continuous_finsetSum Finset.univ
```

```
  intro i hi
  apply continuous_finsetSum Finset.univ
  intro j hj
  exact ((continuous_apply j).comp (continuous_apply i)).pow 2

lemma Measurable.o77MatrixTranspose {X m n : Type*}
    [MeasurableSpace X] [MeasurableSpace ℝ]
    {A : X → Matrix m n ℝ} (hA : Measurable A) :
    Measurable (fun x => (A x).transpose) := by
  apply Measurable.of_eval_matrix
  intro i j
  exact hA.eval_matrix

lemma Measurable.o77MatrixMul {X m n p : Type*}
    [MeasurableSpace X] [Fintype n]
    {A : X → Matrix m n ℝ} {B : X → Matrix n p ℝ}
    (hA : Measurable A) (hB : Measurable B) :
    Measurable (fun x => A x * B x) := by
  apply Measurable.of_eval_matrix
  intro i j
  simp only [Matrix.mul_apply]
  apply Finset.measurable_sum Finset.univ
  intro k hk
  exact hA.eval_matrix.mul hB.eval_matrix

lemma Measurable.o77FrobeniusSq {X m n : Type*}
    [MeasurableSpace X] [Fintype m] [Fintype n]
    {A : X → Matrix m n ℝ} (hA : Measurable A) :
    Measurable (fun x => frobeniusSq (A x)) := by
  unfold frobeniusSq
  apply Finset.measurable_sum Finset.univ
  intro i hi
  apply Finset.measurable_sum Finset.univ
  intro j hj
  exact hA.eval_matrix.pow measurable_const

lemma gaussianRowsIntegrand_continuous_fixed {p h q : Type*}
    [Fintype p] [Fintype h] [Fintype q]
    (T : ℝ) (Z : Matrix h q ℝ) :
    Continuous (gaussianRowsIntegrand (p := p) T Z) := by
  unfold gaussianRowsIntegrand
  have hmul : Continuous (fun Y : Matrix p h ℝ => Y * Z) :=
    continuous_id.matrix_mul continuous_const
  have hprod : Continuous (fun Y : Matrix p h ℝ => frobeniusSq (Y * Z)) :=
    continuous_frobeniusSq.comp hmul
  have hY : Continuous (frobeniusSq : Matrix p h ℝ → ℝ) :=
    continuous_frobeniusSq
  exact ((hprod.const_mul T).add hY).neg.rexp

lemma leftGramLaplaceIntegrand_continuous {k d : ℕ} (rho T : ℝ)
    (hT : 0 ≤ T) :
    Continuous (leftGramLaplaceIntegrand (k := k) (d := d) rho T) := by
  unfold leftGramLaplaceIntegrand
  have hZ : Continuous (fun Z : Matrix (Fin k) (Fin d) ℝ => Z) := continuous_id
  have hgram : Continuous
      (fun Z : Matrix (Fin k) (Fin d) ℝ => Z * Z.transpose) :=
    hZ.matrix_mul hZ.matrix_transpose
  have hshift : Continuous
      (fun Z : Matrix (Fin k) (Fin d) ℝ =>
        (1 : Matrix (Fin k) (Fin k) ℝ) + T • (Z * Z.transpose)) :=
    continuous_const.add (hgram.const_smul T)
  have hdet : Continuous
```

```
      (fun Z : Matrix (Fin k) (Fin d) ℝ =>
        Matrix.det (1 + T • (Z * Z.transpose))) :=
    hshift.matrix_det
  have hpow : Continuous
      (fun Z : Matrix (Fin k) (Fin d) ℝ =>
        Matrix.det (1 + T • (Z * Z.transpose)) ^ (-rho)) :=
    hdet.rpow_const (fun Z => Or.inl (ne_of_gt (gram_posDef T hT Z).det_pos))
  have hweight : Continuous
      (fun Z : Matrix (Fin k) (Fin d) ℝ => Real.exp (-frobeniusSq Z)) :=
    continuous_frobeniusSq.neg.rexp
  exact ENNReal.continuous_ofReal.comp (hweight.mul hpow)

lemma leftGramLaplaceIntegrand_measurable {k d : ℕ} (rho T : ℝ)
    (hT : 0 ≤ T) :
    Measurable (leftGramLaplaceIntegrand (k := k) (d := d) rho T) :=
  (leftGramLaplaceIntegrand_continuous rho T hT).measurable

/-- The Gaussian-weighted residual Laplace integrand on the actual pair of
matrix spaces. The first coordinate is `Y` and the second is `Z`. -/
def residualGaussianLaplaceIntegrand {p h q : ℕ} (T : ℝ)
    (w : Matrix (Fin p) (Fin h) ℝ × Matrix (Fin h) (Fin q) ℝ) : ℝ≥∞0 :=
  ENNReal.ofReal (Real.exp (-frobeniusSq w.2)) *
    ENNReal.ofReal (gaussianRowsIntegrand T w.2 w.1)

lemma residualGaussianLaplaceIntegrand_measurable {p h q : ℕ} (T : ℝ) :
    Measurable (residualGaussianLaplaceIntegrand (p := p) (h := h) (q := q) T) := by
  have hY : Measurable
      (fun w : Matrix (Fin p) (Fin h) ℝ × Matrix (Fin h) (Fin q) ℝ => w.1) :=
    measurable_fst
  have hZ : Measurable
      (fun w : Matrix (Fin p) (Fin h) ℝ × Matrix (Fin h) (Fin q) ℝ => w.2) :=
    measurable_snd
  have hmul : Measurable
      (fun w : Matrix (Fin p) (Fin h) ℝ × Matrix (Fin h) (Fin q) ℝ => w.1 * w.2) :=
    Measurable.o77MatrixMul hY hZ
  have hprod : Measurable
      (fun w : Matrix (Fin p) (Fin h) ℝ × Matrix (Fin h) (Fin q) ℝ =>
        frobeniusSq (w.1 * w.2)) := Measurable.o77FrobeniusSq hmul
  have hfy : Measurable
      (fun w : Matrix (Fin p) (Fin h) ℝ × Matrix (Fin h) (Fin q) ℝ =>
        frobeniusSq w.1) := Measurable.o77FrobeniusSq hY
  have hfz : Measurable
      (fun w : Matrix (Fin p) (Fin h) ℝ × Matrix (Fin h) (Fin q) ℝ =>
        frobeniusSq w.2) := Measurable.o77FrobeniusSq hZ
  have hrows : Measurable
      (fun w : Matrix (Fin p) (Fin h) ℝ × Matrix (Fin h) (Fin q) ℝ =>
        gaussianRowsIntegrand T w.2 w.1) := by
    unfold gaussianRowsIntegrand
    exact Real.continuous_exp.measurable.comp ((hprod.const_mul T).add hfy).neg
  unfold residualGaussianLaplaceIntegrand
  exact (ENNReal.continuous_ofReal.measurable.comp
      (Real.continuous_exp.measurable.comp hfz.neg)).mul
    (ENNReal.continuous_ofReal.measurable.comp hrows)


/-- The full Gaussian-weighted residual Laplace integral. -/
def residualGaussianLaplaceLIntegral (p q h : ℕ) (T : ℝ) : ℝ≥∞0 :=
  ∫⁻ w : Matrix (Fin p) (Fin h) ℝ × Matrix (Fin h) (Fin q) ℝ,
    residualGaussianLaplaceIntegrand T w
      ∂(matrixLebesgue (Fin p) (Fin h)).prod
        (matrixLebesgue (Fin h) (Fin q))
```

```
lemma residualGaussianLaplace_inner_eq_leftGram {p h q : ℕ}
    (T : ℝ) (hT : 0 ≤ T) (Z : Matrix (Fin h) (Fin q) ℝ) :
    ∫⁻( Y : Matrix (Fin p) (Fin h) ℝ,
        residualGaussianLaplaceIntegrand T (Y, Z)
        ∂matrixLebesgue (Fin p) (Fin h)) =
      ENNReal.ofReal (gaussianRowsConstant p h) *
        leftGramLaplaceIntegrand ((p : ℝ) / 2) T Z := by
  unfold residualGaussianLaplaceIntegrand
  change ∫⁻( Y : Matrix (Fin p) (Fin h) ℝ,
      ENNReal.ofReal (Real.exp (-frobeniusSq Z)) *
        ENNReal.ofReal (gaussianRowsIntegrand T Z Y)
        ∂matrixLebesgue (Fin p) (Fin h)) = _
  have hmeas : Measurable
      (fun Y : Matrix (Fin p) (Fin h) ℝ =>
        ENNReal.ofReal (gaussianRowsIntegrand T Z Y)) :=
    ENNReal.continuous_ofReal.measurable.comp
      (gaussianRowsIntegrand_continuous_fixed T Z).measurable
  calc
    ∫⁻( Y : Matrix (Fin p) (Fin h) ℝ,
        ENNReal.ofReal (Real.exp (-frobeniusSq Z)) *
          ENNReal.ofReal (gaussianRowsIntegrand T Z Y)
          ∂matrixLebesgue (Fin p) (Fin h)) =
        ENNReal.ofReal (Real.exp (-frobeniusSq Z)) *
          ∫⁻ Y : Matrix (Fin p) (Fin h) ℝ,
            ENNReal.ofReal (gaussianRowsIntegrand T Z Y)
            ∂matrixLebesgue (Fin p) (Fin h) :=
      lintegral_const_mul _ hmeas
    _ = ENNReal.ofReal (gaussianRowsConstant p h) *
        leftGramLaplaceIntegrand ((p : ℝ) / 2) T Z := by
      rw [gaussianRows_lintegral_instantiated T hT Z]
      rw [leftGramLaplaceIntegrand]
      rw [ENNReal.ofReal_mul (Real.exp_pos _).le]
      have hexp : -(p : ℝ) / 2 = -((p : ℝ) / 2) := by ring
      rw [hexp]
      ac_rfl


/-- Exact integration of the `Y` rows: the remaining matrix integral is the
left-Gram determinant integral with exponent `p/2`. -/
theorem residualGaussianLaplaceLIntegral_eq_leftGram {p h q : ℕ}
    (T : ℝ) (hT : 0 ≤ T) :
    residualGaussianLaplaceLIntegral p q h T =
      ENNReal.ofReal (gaussianRowsConstant p h) *
        leftGramLaplaceLIntegral h q ((p : ℝ) / 2) T := by
  unfold residualGaussianLaplaceLIntegral
  rw [lintegral_prod_symm _
    (residualGaussianLaplaceIntegrand_measurable T).aemeasurable]
  simp_rw [residualGaussianLaplace_inner_eq_leftGram T hT]
  rw [lintegral_const_mul _
    (leftGramLaplaceIntegrand_measurable ((p : ℝ) / 2) T hT)]
  rfl

end O77.Residual

noncomputable section

open Real MeasureTheory Set
open scoped ENNReal BigOperators

namespace O77.Residual

/-- The checked real ordered-Laguerre estimate, repackaged in the
```

```
extended-nonnegative Laplace-bound interface. -/
theorem residual_laguerreChamber_laplacePowerLogBounds
    {p q h : ℕ} (hp : 0 < p) :
    LaplacePowerLogBounds
      (fun T => ENNReal.ofReal
        (laguerreChamberIntegral
          (k := O77.Arithmetic.chamberK h q)
          (residualSpectralEta q h) (residualSpectralRho p) T))
      ((O77.residualMin p q h : ℝ) / 2)
      (O77.residualMultiplicity p q h - 1) := by
  rcases residual_laguerreChamber_powerLog_bounds (p := p) (q := q) (h := h) hp with
    ⟨c, C, T0, hc, hcC, hbox, ⟩hbounds
  let T1 : ℝ := max T0 2
  refine ⟨c, C, T1, hc, hcC, ?_, ?⟩_
  · exact le_max_right _ _
  · intro T hT
    have hT0 : T0 ≤ T := (le_max_left T0 2).trans hT
    have hTtwo : (2 : ℝ) ≤ T := (le_max_right T0 2).trans hT
    have hTone : (1 : ℝ) ≤ T := by linarith
    have hb := hbounds T hT0
    have hgauge :
        powerLogTopReal ((O77.residualMin p q h : ℝ) / 2)
            (O77.residualMultiplicity p q h - 1) T =
          T ^ (-(O77.residualMin p q h : ℝ) / 2) *
            (Real.log T) ^ (O77.residualMultiplicity p q h - 1) := by
      unfold powerLogTopReal
      congr 2
      ring
    constructor
    · unfold powerLogTop
      rw ←[ ENNReal.ofReal_mul hc.le]
      apply ENNReal.ofReal_le_ofReal
      simpa [hgauge] using hb.1
    · unfold powerLogTop
      have hC0 : 0 ≤ C := hc.le.trans hcC
      rw ←[ ENNReal.ofReal_mul hC0]
      apply ENNReal.ofReal_le_ofReal
      simpa [hgauge] using hb.2

end O77.Residual

noncomputable section

open Real MeasureTheory Set
open scoped ENNReal BigOperators

namespace O77.Residual
namespace LaplacePowerLogBounds

/-- Multiplication by a fixed strictly positive finite extended-nonnegative
constant preserves a two-sided power-log order. -/
theorem const_mul {J : ℝ → ℝ≥∞0} {lam : ℝ} {degree : ℕ}
    {K : ℝ≥∞0} (hK0 : K ≠ 0) (hKtop : K ≠ ∞)
    (hJ : O77.Residual.LaplacePowerLogBounds J lam degree) :
    O77.Residual.LaplacePowerLogBounds (fun T => K * J T) lam degree := by
  rcases hJ with ⟨c, C, T0, hc, hcC, hT0, ⟩hbounds
  let kappa : ℝ := K.toReal
  have hkappa : 0 < kappa := ENNReal.toReal_pos hK0 hKtop
  have hKcoe : ENNReal.ofReal kappa = K := ENNReal.ofReal_toReal hKtop
  refine ⟨kappa * c, kappa * C, T0, mul_pos hkappa hc,
    mul_le_mul_of_nonneg_left hcC hkappa.le, hT0, ?⟩_
  intro T hT
```

```
  have hb := hbounds T hT
  constructor
  · calc
      ENNReal.ofReal (kappa * c) * powerLogTop lam degree T =
          K * (ENNReal.ofReal c * powerLogTop lam degree T) := by
            rw [ENNReal.ofReal_mul hkappa.le, hKcoe]
            ac_rfl
      _ ≤ K * J T := mul_le_mul_of_nonneg_left hb.1 (by positivity)
  · calc
      K * J T ≤ K * (ENNReal.ofReal C * powerLogTop lam degree T) :=
        mul_le_mul_of_nonneg_left hb.2 (by positivity)
      _ = ENNReal.ofReal (kappa * C) * powerLogTop lam degree T := by
        rw [ENNReal.ofReal_mul hkappa.le, hKcoe]
        ac_rfl

end LaplacePowerLogBounds
end O77.Residual

noncomputable section

open Real MeasureTheory Set
open scoped ENNReal BigOperators

namespace O77.Residual

lemma wishartEta_chamber (h q : ℕ) :
    wishartEta (O77.Arithmetic.chamberK h q)
        (O77.Arithmetic.chamberD h q) =
      residualSpectralEta q h := by
  rfl

lemma leftGramLaplaceLIntegral_eq_rightGramLaplaceLIntegral
    (d k : ℕ) (rho T : ℝ) :
    leftGramLaplaceLIntegral d k rho T =
      rightGramLaplaceLIntegral d k rho T := by
  rfl

/-- Conditional exact reduction of the complete Gaussian residual matrix
integral to the already checked ordered Laguerre chamber integral. The one
remaining ordinary premise is the global real-Wishart proposition. -/
theorem residualGaussianLaplace_to_orderedLaguerre
    (hW : O77.External.RealWishartEigenvalueFormula)
    {p q h : ℕ} (T : ℝ) (hT : 0 ≤ T) :
    ∃ C : ℝ≥∞0,
      0 < C ∧ C < ∞ ∧
      residualGaussianLaplaceLIntegral p q h T =
        ENNReal.ofReal (gaussianRowsConstant p h) * C *
          (O77.Arithmetic.chamberK h q).factorial *
          ENNReal.ofReal
            (laguerreChamberIntegral
              (k := O77.Arithmetic.chamberK h q)
              (residualSpectralEta q h) (residualSpectralRho p) T) := by
  have hrho : 0 ≤ (p : ℝ) / 2 := by positivity
  rcases le_total h q with hhq | hqh
  · rcases hW h q hhq with ⟨C, ⟩hC
    refine ⟨C, hC.constant_pos, hC.constant_lt_top, ?⟩_
    rw [residualGaussianLaplaceLIntegral_eq_leftGram T hT]
    rw [concreteWishart_left_to_orderedLaguerre hC hrho hT]
    have hrhoeq : (p : ℝ) / 2 = residualSpectralRho p := rfl
    rw [hrhoeq]
    have heta : wishartEta h q = residualSpectralEta q h := by
      simpa [O77.Arithmetic.chamberK, O77.Arithmetic.chamberD,
```

```
        min_eq_left hhq, max_eq_right hhq] using wishartEta_chamber h q
    rw [heta]
    simp [O77.Arithmetic.chamberK, min_eq_left hhq, mul_assoc]
  · rcases hW q h hqh with ⟨C, ⟩hC
    refine ⟨C, hC.constant_pos, hC.constant_lt_top, ?⟩_
    rw [residualGaussianLaplaceLIntegral_eq_leftGram T hT]
    rw [leftGramLaplaceLIntegral_eq_rightGramLaplaceLIntegral]
    rw [concreteWishart_right_to_orderedLaguerre hC hrho hT]
    have hrhoeq : (p : ℝ) / 2 = residualSpectralRho p := rfl
    rw [hrhoeq]
    have heta : wishartEta q h = residualSpectralEta q h := by
      simpa [O77.Arithmetic.chamberK, O77.Arithmetic.chamberD,
        min_eq_right hqh, max_eq_left hqh] using wishartEta_chamber h q
    rw [heta]
    simp [O77.Arithmetic.chamberK, min_eq_right hqh, mul_assoc]

end O77.Residual

noncomputable section

open Real MeasureTheory Set
open scoped ENNReal BigOperators

namespace O77.Residual

/-- Fixed-normalization form of the exact reduction when the left Gram matrix
is the smaller one. -/
theorem residualGaussianLaplace_eq_orderedLaguerre_left
    {p h q : ℕ} {C : ℝ≥∞0} (hhq : h ≤ q)
    (hW : HasConcreteRealWishartFormula h q C)
    (T : ℝ) (hT : 0 ≤ T) :
    residualGaussianLaplaceLIntegral p q h T =
      ENNReal.ofReal (gaussianRowsConstant p h) * C *
        (O77.Arithmetic.chamberK h q).factorial *
        ENNReal.ofReal
          (laguerreChamberIntegral
            (k := O77.Arithmetic.chamberK h q)
            (residualSpectralEta q h) (residualSpectralRho p) T) := by
  have hrho : 0 ≤ (p : ℝ) / 2 := by positivity
  rw [residualGaussianLaplaceLIntegral_eq_leftGram T hT]
  rw [concreteWishart_left_to_orderedLaguerre hW hrho hT]
  have hrhoeq : (p : ℝ) / 2 = residualSpectralRho p := rfl
  rw [hrhoeq]
  have heta : wishartEta h q = residualSpectralEta q h := by
    simpa [O77.Arithmetic.chamberK, O77.Arithmetic.chamberD,
      min_eq_left hhq, max_eq_right hhq] using wishartEta_chamber h q
  rw [heta]
  simp [O77.Arithmetic.chamberK, min_eq_left hhq, mul_assoc]

/-- Fixed-normalization form of the exact reduction when the right Gram
matrix is the smaller one. -/
theorem residualGaussianLaplace_eq_orderedLaguerre_right
    {p h q : ℕ} {C : ℝ≥∞0} (hqh : q ≤ h)
    (hW : HasConcreteRealWishartFormula q h C)
    (T : ℝ) (hT : 0 ≤ T) :
    residualGaussianLaplaceLIntegral p q h T =
      ENNReal.ofReal (gaussianRowsConstant p h) * C *
        (O77.Arithmetic.chamberK h q).factorial *
        ENNReal.ofReal
          (laguerreChamberIntegral
            (k := O77.Arithmetic.chamberK h q)
            (residualSpectralEta q h) (residualSpectralRho p) T) := by
```

```
  have hrho : 0 ≤ (p : ℝ) / 2 := by positivity
  rw [residualGaussianLaplaceLIntegral_eq_leftGram T hT]
  rw [leftGramLaplaceLIntegral_eq_rightGramLaplaceLIntegral]
  rw [concreteWishart_right_to_orderedLaguerre hW hrho hT]
  have hrhoeq : (p : ℝ) / 2 = residualSpectralRho p := rfl
  rw [hrhoeq]
  have heta : wishartEta q h = residualSpectralEta q h := by
    simpa [O77.Arithmetic.chamberK, O77.Arithmetic.chamberD,
      min_eq_right hqh, max_eq_left hqh] using wishartEta_chamber h q
  rw [heta]
  simp [O77.Arithmetic.chamberK, min_eq_right hqh, mul_assoc]

namespace LaplacePowerLogBounds

/-- Eventual pointwise equality above `2` transports a power-log bound. -/
theorem congr_of_eq_of_ge_two {J J' : ℝ → ℝ≥∞0} {lam : ℝ} {degree : ℕ}
    (hJ : O77.Residual.LaplacePowerLogBounds J lam degree)
    (hEq : ∀ T : ℝ, 2 ≤ T → J T = J' T) :
    O77.Residual.LaplacePowerLogBounds J' lam degree := by
  rcases hJ with ⟨c, C, T0, hc, hcC, hT0, ⟩hbounds
  refine ⟨c, C, T0, hc, hcC, hT0, ?⟩_
  intro T hT
  have hTtwo : (2 : ℝ) ≤ T := hT0.trans hT
  simpa ←[ hEq T hTtwo] using hbounds T hT

end LaplacePowerLogBounds
end O77.Residual

noncomputable section

open Real MeasureTheory Set
open scoped ENNReal BigOperators

namespace O77.Residual

/-- The concrete Gaussian residual Laplace transform has the exact O77
power-log order, conditional only on the explicitly stated real-Wishart
change-of-variables proposition. The positivity hypotheses delimit the genuine
positive-dimensional residual case; zero residual factors are handled
separately by the neutral pair. -/
theorem gaussianLaplace_powerLog_bounds_of_wishart_instantiated
    (hW : O77.External.RealWishartEigenvalueFormula)
    {p q h : ℕ} (hp : 0 < p) (_hq : 0 < q) (_hh : 0 < h) :
    LaplacePowerLogBounds
      (residualGaussianLaplaceLIntegral p q h)
      ((O77.residualMin p q h : ℝ) / 2)
      (O77.residualMultiplicity p q h - 1) := by
  have hLag := residual_laguerreChamber_laplacePowerLogBounds
    (p := p) (q := q) (h := h) hp
  rcases le_total h q with hhq | hqh
  · rcases hW h q hhq with ⟨C, ⟩hC
    let K : ℝ≥∞0 :=
      ENNReal.ofReal (gaussianRowsConstant p h) * C *
        (O77.Arithmetic.chamberK h q).factorial
    have hgauss0 : ENNReal.ofReal (gaussianRowsConstant p h) ≠ 0 :=
      ne_of_gt (ENNReal.ofReal_pos.mpr (gaussianRowsConstant_pos p h))
    have hfactorial0 :
        ((O77.Arithmetic.chamberK h q).factorial : ℝ≥∞0) ≠ 0 := by
      exact_mod_cast Nat.factorial_ne_zero (O77.Arithmetic.chamberK h q)
    have hK0 : K ≠ 0 := by
      dsimp [K]
      exact mul_ne_zero (mul_ne_zero hgauss0 hC.constant_ne_zero) hfactorial0
```

```
    have hKtop : K ≠ ∞ := by
      dsimp [K]
      exact ENNReal.mul_ne_top
        (ENNReal.mul_ne_top ENNReal.ofReal_ne_top hC.constant_ne_top)
        (ENNReal.natCast_ne_top _)
    have hScaled := LaplacePowerLogBounds.const_mul hK0 hKtop hLag
    apply LaplacePowerLogBounds.congr_of_eq_of_ge_two hScaled
    intro T hT
    have hT0 : 0 ≤ T := by linarith
    simpa [K, mul_assoc] using
      (residualGaussianLaplace_eq_orderedLaguerre_left
        (p := p) hhq hC T hT0).symm
  · rcases hW q h hqh with ⟨C, ⟩hC
    let K : ℝ≥∞0 :=
      ENNReal.ofReal (gaussianRowsConstant p h) * C *
        (O77.Arithmetic.chamberK h q).factorial
    have hgauss0 : ENNReal.ofReal (gaussianRowsConstant p h) ≠ 0 :=
      ne_of_gt (ENNReal.ofReal_pos.mpr (gaussianRowsConstant_pos p h))
    have hfactorial0 :
        ((O77.Arithmetic.chamberK h q).factorial : ℝ≥∞0) ≠ 0 := by
      exact_mod_cast Nat.factorial_ne_zero (O77.Arithmetic.chamberK h q)
    have hK0 : K ≠ 0 := by
      dsimp [K]
      exact mul_ne_zero (mul_ne_zero hgauss0 hC.constant_ne_zero) hfactorial0
    have hKtop : K ≠ ∞ := by
      dsimp [K]
      exact ENNReal.mul_ne_top
        (ENNReal.mul_ne_top ENNReal.ofReal_ne_top hC.constant_ne_top)
        (ENNReal.natCast_ne_top _)
    have hScaled := LaplacePowerLogBounds.const_mul hK0 hKtop hLag
    apply LaplacePowerLogBounds.congr_of_eq_of_ge_two hScaled
    intro T hT
    have hT0 : 0 ≤ T := by linarith
    simpa [K, mul_assoc] using
      (residualGaussianLaplace_eq_orderedLaguerre_right
        (p := p) hqh hC T hT0).symm

end O77.Residual

noncomputable section

open Real MeasureTheory Set
open scoped ENNReal BigOperators

namespace O77.Residual

/-- Zero row dimension: exact row integration remains meaningful, but the
Laguerre exponent `rho=0` lies outside the positive-`rho` asymptotic theorem. -/
theorem residualGaussianLaplaceLIntegral_zero_rows
    (q h : ℕ) (T : ℝ) (hT : 0 ≤ T) :
    residualGaussianLaplaceLIntegral 0 q h T =
      leftGramLaplaceLIntegral h q 0 T := by
  simpa [gaussianRowsConstant] using
    residualGaussianLaplaceLIntegral_eq_leftGram (p := 0) (h := h) (q := q) T hT

/-- The condition `p>0` in the power-log assembly cannot be dropped by
blindly invoking the positive-`rho` Laguerre theorem. -/
theorem residualSpectralRho_zero_not_pos :
    ¬ 0 < residualSpectralRho 0 := by
  norm_num [residualSpectralRho]

/-- Scalar residual control: the conditional order is
```

```
`T^{-1/2} log T`, since the two prefix costs tie. -/
theorem gaussianLaplace_powerLog_bounds_scalar_test
    (hW : O77.External.RealWishartEigenvalueFormula) :
    LaplacePowerLogBounds
      (residualGaussianLaplaceLIntegral 1 1 1)
      (1 / 2 : ℝ) 1 := by
  have hmin : O77.residualMin 1 1 1 = 1 := by decide
  have hmult : O77.residualMultiplicity 1 1 1 = 2 := by decide
  simpa [hmin, hmult] using
    gaussianLaplace_powerLog_bounds_of_wishart_instantiated
      hW (p := 1) (q := 1) (h := 1) (by decide) (by decide) (by decide)

/-- Rectangular control with the left Gram matrix smaller. -/
theorem gaussianLaplace_powerLog_bounds_wide_test
    (hW : O77.External.RealWishartEigenvalueFormula) :
    LaplacePowerLogBounds
      (residualGaussianLaplaceLIntegral 2 3 1)
      1 0 := by
  have hmin : O77.residualMin 2 3 1 = 2 := by decide
  have hmult : O77.residualMultiplicity 2 3 1 = 1 := by decide
  simpa [hmin, hmult] using
    gaussianLaplace_powerLog_bounds_of_wishart_instantiated
      hW (p := 2) (q := 3) (h := 1) (by decide) (by decide) (by decide)

/-- Rectangular control with the right Gram matrix smaller. -/
theorem gaussianLaplace_powerLog_bounds_tall_test
    (hW : O77.External.RealWishartEigenvalueFormula) :
    LaplacePowerLogBounds
      (residualGaussianLaplaceLIntegral 2 1 3)
      1 0 := by
  have hmin : O77.residualMin 2 1 3 = 2 := by decide
  have hmult : O77.residualMultiplicity 2 1 3 = 1 := by decide
  simpa [hmin, hmult] using
    gaussianLaplace_powerLog_bounds_of_wishart_instantiated
      hW (p := 2) (q := 1) (h := 3) (by decide) (by decide) (by decide)

end O77.Residual

noncomputable section

open Real MeasureTheory Set
open scoped ENNReal BigOperators

namespace O77.Residual

/-- The actual quartic residual kernel on the product of the two matrix
spaces. -/
def residualKernel {p q h : ℕ}
    (w : Matrix (Fin p) (Fin h) ℝ × Matrix (Fin h) (Fin q) ℝ) : ℝ :=
  frobeniusSq (w.1 * w.2)

/-- The squared Euclidean norm on the ambient pair of matrix spaces, written
entrywise to avoid dependence on product-norm conventions. -/
def residualAmbientNormSq {p q h : ℕ}
    (w : Matrix (Fin p) (Fin h) ℝ × Matrix (Fin h) (Fin q) ℝ) : ℝ :=
  frobeniusSq w.1 + frobeniusSq w.2

/-- The full residual Gaussian integrand is exactly the exponential of the
quartic residual kernel plus the ambient quadratic Gaussian weight. -/
lemma residualGaussianLaplaceIntegrand_eq_exp {p q h : ℕ}
    (T : ℝ)
    (w : Matrix (Fin p) (Fin h) ℝ × Matrix (Fin h) (Fin q) ℝ) :
```

```
    residualGaussianLaplaceIntegrand T w =
      ENNReal.ofReal
        (Real.exp (-(T * residualKernel w + residualAmbientNormSq w))) := by
  unfold residualGaussianLaplaceIntegrand gaussianRowsIntegrand
    residualKernel residualAmbientNormSq
  rw ←[ ENNReal.ofReal_mul (Real.exp_pos _).le]
  congr 1
  rw ←[ Real.exp_add]
  congr 1
  ring

lemma frobeniusSq_smul {m n : Type*} [Fintype m] [Fintype n]
    (c : ℝ) (A : Matrix m n ℝ) :
    frobeniusSq (c • A) = c ^ 2 * frobeniusSq A := by
  unfold frobeniusSq
  simp_rw [Matrix.smul_apply, smul_eq_mul, mul_pow]
  calc
    ∑( i, ∑ j, c ^ 2 * A i j ^ 2) =
        ∑ i, c ^ 2 * ∑ j, A i j ^ 2 := by
      apply Finset.sum_congr rfl
      intro i hi
      rw [Finset.mul_sum]
    _ = c ^ 2 * ∑ i, ∑ j, A i j ^ 2 := by rw [Finset.mul_sum]

lemma smul_mul_smul {p h q : Type*} [Fintype h]
    (c : ℝ) (Y : Matrix p h ℝ) (Z : Matrix h q ℝ) :
    (c • Y) * (c • Z) = (c ^ 2) • (Y * Z) := by
  rw [Matrix.smul_mul, Matrix.mul_smul, smul_smul]
  simp [pow_two]

/-- Simultaneous radial scaling of both residual factors scales the squared
product norm by the fourth power. -/
theorem residualKernel_quartic {p q h : ℕ}
    (c : ℝ) (Y : Matrix (Fin p) (Fin h) ℝ)
    (Z : Matrix (Fin h) (Fin q) ℝ) :
    residualKernel (c • Y, c • Z) = c ^ 4 * residualKernel (Y, Z) := by
  unfold residualKernel
  rw [smul_mul_smul, frobeniusSq_smul]
  ring

/-- The ambient Gaussian norm, in contrast, scales quadratically. -/
theorem residualAmbientNormSq_quadratic {p q h : ℕ}
    (c : ℝ) (Y : Matrix (Fin p) (Fin h) ℝ)
    (Z : Matrix (Fin h) (Fin q) ℝ) :
    residualAmbientNormSq (c • Y, c • Z) =
      c ^ 2 * residualAmbientNormSq (Y, Z) := by
  unfold residualAmbientNormSq
  rw [frobeniusSq_smul, frobeniusSq_smul]
  ring

/-- Adversarial control: the quartic residual kernel cannot be replaced by a
quadratically homogeneous one. -/
theorem residualKernel_not_quadratic :
    residualKernel
        ((2 : ℝ) • (1 : Matrix (Fin 1) (Fin 1) ℝ),
          (2 : ℝ) • (1 : Matrix (Fin 1) (Fin 1) ℝ)) ≠
      (2 : ℝ) ^ 2 *
        residualKernel
          ((1 : Matrix (Fin 1) (Fin 1) ℝ),
            (1 : Matrix (Fin 1) (Fin 1) ℝ)) := by
  rw [residualKernel_quartic]
  norm_num [residualKernel, frobeniusSq, Matrix.mul_apply]
```

```
end O77.Residual
```

Corresponding Lean declaration(s): O77.Residual.gaussianRowsConstant,O77.Residual.gaussianRowsConstant_pos,O77.Residual.quadraticGaussianValue_pow_eq_gaussianRowsConstant_mul_det,O77.Residual.gaussianRows_lintegral_instantiated,O77.Residual.matrixLebesgue.instSigmaFiniteFin,O77.Residual.continuous_froboniusSq,O77.Residual.Measurable.o77MatrixTranspose,O77.Residual.Measurable.o77MatrixMul,O77.Residual.Measurable.o77FrobeniusSq,O77.Residual.gaussianRowsIntegrand_continuous_fixed,O77.Residual.leftGramLaplaceIntegrand_continuous,O77.Residual.leftGramLaplaceIntegrand_measurable,O77.Residual.residualGaussianLaplaceIntegrand,O77.Residual.residualGaussianLaplaceIntegrand_measurable,O77.Residual.residualGaussianLaplaceLIntegral,O77.Residual.residualGaussianLaplace_inner_eq_leftGram,O77.Residual.residualGaussianLaplaceLIntegral_eq_leftGram,O77.Residual.residual_laguerreChamber_laplacePowerLogBounds,O77.Residual.LaplacePowerLogBounds.const_mul,O77.Residual.wishartEta_chamber,O77.Residual.leftGramLaplaceLIntegral_eq_rightGramLaplaceLIntegral,O77.Residual.residualGaussianLaplace_to_orderedLaguerre,O77.Residual.residualGaussianLaplace_eq_orderedLaguerre_left,O77.Residual.residualGaussianLaplace_eq_orderedLaguerre_right,O77.Residual.LaplacePowerLogBounds.congr_of_eq_of_ge_two,O77.Residual.gaussianLaplace_powerLog_bounds_of_wishart_instantiated,O77.Residual.residualGaussianLaplaceLIntegral_zero_rows,O77.Residual.residualSpectralRho_zero_not_pos,O77.Residual.gaussianLaplace_powerLog_bounds_scalar_test,O77.Residual.gaussianLaplace_powerLog_bounds_wide_test,O77.Residual.gaussianLaplace_powerLog_bounds_tall_test,O77.Residual.residualKernel,O77.Residual.residualAmbientNormSq,O77.Residual.residualGaussianLaplaceIntegrand_eq_exp,O77.Residual.frobeniusSq_smul,O77.Residual.smul_mul_smul,O77.Residual.residualKernel_quartic,O77.Residual.residualAmbientNormSq_quadratic,O77.Residual.residualKernel_not_quadratic.

27 Exact dilation and the one-shell estimate

27.1 Dilation before summation

The preceding Gaussian analysis proves the large-T power–log order of the residual transform, conditional only on the explicit real-Wishart proposition. This section begins the homogeneous localization step R6. It proves the exact measure-scaling law for simultaneous residual dilation, a set-restricted nonnegative change-of-variables formula, the elementary lower comparison from a fixed ball to the Gaussian transform, and the exact estimate for each individual dyadic shell. The next section proves the exact shell partition and summability, then deduces the global Gaussian–ball comparison and the residual pair.

27.2 The residual ambient space and its explicit measure

For natural numbers p, q, h, write

$$E_{p,q,h} = \mathbb{R}^{p\times h} \times \mathbb{R}^{h\times q}, \qquad \lambda_{p,q,h} = \lambda_{p,h} \otimes \lambda_{h,q}, \tag{R6.1}$$

where each matrix measure is the finite product of one-dimensional Lebesgue measures already constructed in the concrete Gaussian-row section. The Lean abbreviations are `ResidualSpace` and `residualMatrixLebesgue`. No default matrix norm is used in the definition of either object.

The auxiliary definitions `matrixCoordinateVolume` and `residualCoordinateVolume` fix the coordinate-product normalization explicitly. Theorems `matrixLebesgue_fin_eq_volume` and `residualMatrixLebesgue_eq_volume` identify the selected measures with these coordinate volumes. The name "volume" here always refers to this specified coordinate normalization; it is not an assertion that Mathlib has selected a canonical matrix norm.

The real dimension is

$$D(p, q, h) = ph + hq. \tag{R6.2}$$

The theorem `finrank_residualSpace` proves that (R6.2) is exactly $\operatorname{finrank}_{\mathbb{R}} E_{p,q,h}$. The proof unfolds matrices to finite iterated function spaces and uses the finite-Π and product finrank formulas. Rectangular and empty-dimensional specializations are compiled controls.

27.3 Simultaneous dilation and the exact Jacobian

For $c \in \mathbb{R}$, define

$$\delta_c(Y, Z) = (cY, cZ). \tag{R6.3}$$

This map is continuous and measurable. The checked algebraic scaling identities become

$$\mathcal{K}(\delta_c w) = c^4 \mathcal{K}(w), \qquad \mathcal{N}(\delta_c w) = c^2 \mathcal{N}(w), \tag{R6.4}$$

where $\mathcal{K}(Y, Z) = \|YZ\|_F^2$ and $\mathcal{N}(Y, Z) = \|Y\|_F^2 + \|Z\|_F^2$. The quartic exponent in the first identity is essential; the retained scalar counterexample `residualKernel_not_quadratic` refutes a quadratic replacement.

The Jacobian factor is defined in extended nonnegative reals by

$$J_{p,q,h}(c) = \operatorname{ofReal}(c)^{D(p,q,h)}. \tag{R6.5}$$

For $c \geq 0$ the checked set-level scaling theorem is

$$\lambda_{p,q,h}(cS) = J_{p,q,h}(c)\, \lambda_{p,q,h}(S). \tag{R6.6}$$

Its proof uses the additive-Haar scaling theorem in finite dimension. A local Frobenius norm instance is activated only to provide the finite-dimensional normed topology required by that library theorem; the measurable structure is explicitly identified with the existing coordinate Π-space structure, and all balls used below remain defined by the explicit function $\mathcal{N}$. Thus no hidden max norm or operator norm enters the geometric statements.

For $c > 0$, `residualDilationMeasurableEquiv` bundles δ_c with inverse $\delta_{c^{-1}}$. The source proves

$$(\delta_c)_* \lambda_{p,q,h} = J_{p,q,h}(c)^{-1} \lambda_{p,q,h} \tag{R6.7}$$

and both the unrestricted and set-restricted nonnegative change-of-variables formulas. The latter is

$$\int^{-}_{\delta_c(S)} f(w)\, d\lambda(w) = J_{p,q,h}(c) \int^{-}_{S} f(\delta_c u)\, d\lambda(u), \tag{R6.8}$$

for measurable S and measurable $f : E_{p,q,h} \to \mathbb{R}_{\geq 0} \cup \{\infty\}$. The proof is not an informal Jacobian argument. It rewrites a restricted integral as an indicator integral, uses (R6.7) for the inverse dilation, proves the corresponding indicator identity pointwise, and cancels only after proving $J_{p,q,h}(c) \neq 0$.

The strict hypothesis $c > 0$ in the equivalence and integral transport is load-bearing. The control `residualDilation_zero_not_injective` proves that zero dilation is not injective already on $E_{1,1,1}$.

27.4 Fixed quadratic balls and the lower Gaussian comparison

Define the closed quadratic ball and its unweighted Laplace transform by

$$B_R = \{w : \mathcal{N}(w) \leq R^2\}, \qquad J_R(T) = \int^{-}_{B_R} e^{-T\mathcal{K}(w)}\, d\lambda(w). \tag{R6.9}$$

The source proves measurability of B_R and of the integrand directly from continuity of $\mathcal{N}$ and $\mathcal{K}$. Since $\mathcal{K} \geq 0$, $T \mapsto J_R(T)$ is antitone.

For $w \in B_R$ one has $\mathcal{N}(w) \le R^2$, hence

$$e^{-R^2} e^{-T\mathcal{K}(w)} \le e^{-T\mathcal{K}(w)-\mathcal{N}(w)} \le e^{-T\mathcal{K}(w)}. \tag{R6.10}$$

After integration, the left inequality gives the unconditional comparison

$$\mathrm{ofReal}(e^{-R^2}) J_R(T) \le \mathcal{J}^G_{p,q,h}(T). \tag{R6.11}$$

The right inequality bounds only the Gaussian contribution from B_R; it does not control the complement and is therefore not yet a global upper comparison.

The image identity $\delta_c(B_R) = B_{cR}$ for $c > 0$, together with (R6.4) and (R6.8), gives

$$J_{cR}(T) = J_{p,q,h}(c)\, J_R(c^4 T). \tag{R6.12}$$

This is the exact formal replacement for the substitution $w = cu$.

27.5 One dyadic shell

For $n \in \mathbb{N}$, put

$$c_n = 2^{n+1}, \qquad r_n = 2^n R, \qquad S_{R,n} = B_{c_n R} \setminus B_{r_n}. \tag{R6.13}$$

Membership in this shell gives the strict/weak bounds

$$r_n^2 < \mathcal{N}(w) \le (c_n R)^2. \tag{R6.14}$$

The strict lower inequality is obtained from nonmembership in the closed inner ball. In particular,

$$e^{-T\mathcal{K}(w)-\mathcal{N}(w)} \le e^{-r_n^2} e^{-T\mathcal{K}(w)}. \tag{R6.15}$$

The outer inequality in (R6.14) and (R6.12) show $S_{R,n} \subseteq \delta_{c_n}(B_R)$. Therefore, for $T \ge 0$,

$$\begin{aligned} \int_{S_{R,n}} e^{-T\mathcal{K}-\mathcal{N}}\, d\lambda &\le e^{-r_n^2} \int_{\delta_{c_n}(B_R)} e^{-T\mathcal{K}}\, d\lambda \\ &= e^{-r_n^2} J_{p,q,h}(c_n) J_R(c_n^4 T) \\ &\le e^{-r_n^2} J_{p,q,h}(c_n) J_R(T). \end{aligned} \tag{R6.16}$$

The last inequality uses $c_n^4 \ge 1$ and antitonicity of J_R. Since $J_{p,q,h}(c_n) = 2^{(n+1)D}$ and $r_n^2 = 4^n R^2$, (R6.16) is precisely

$$\boxed{\int_{S_{R,n}} e^{-T\mathcal{K}-\mathcal{N}}\, d\lambda \le 2^{(n+1)D} e^{-4^n R^2} J_R(T).} \tag{R6.17}$$

The checked theorem `residualGaussianShellLIntegral_le` retains the coefficient in factored ENNReal form, thereby avoiding any conversion of an infinite integral through `ENNReal.toReal`. Its ordinary parameter is only the hypothesis $0 \le T$; it has no Wishart premise.

27.6 Role of the single-shell theorem

The single-shell estimate is the local input to the global localization theorem in the next residual section. There the half-open shells are proved to partition the complement of a positive-radius ball, their exact coefficients are shown summable by a ratio test, and the resulting tail estimate is combined with the inner-ball lower bound. Keeping the single-shell result separate makes the quartic homogeneity and the D-dimensional Jacobian independently auditable.

27.7 Dilation and single-shell localization: Lean code

The following block compiles in document order after all preceding core and Mathlib-dependent blocks. The narrow import `Mathlib.Analysis.Matrix.Normed` in the initial header supplies the scoped Frobenius normed-space instances used by the Haar-scaling argument.

```
set_option autoImplicit false
noncomputable section

open Real MeasureTheory Set
open scoped ENNReal BigOperators

namespace O77.Residual

/-- The residual ambient space for dimensions `(p,q,h)`. -/
abbrev ResidualSpace (p q h : ℕ) :=
  Matrix (Fin p) (Fin h) ℝ × Matrix (Fin h) (Fin q) ℝ

/-- The explicit product matrix Lebesgue measure on the residual ambient space. -/
def residualMatrixLebesgue (p q h : ℕ) : Measure (ResidualSpace p q h) :=
  (matrixLebesgue (Fin p) (Fin h)).prod
    (matrixLebesgue (Fin h) (Fin q))

lemma residualMatrixLebesgue_eq (p q h : ℕ) :
    residualMatrixLebesgue p q h =
      (matrixLebesgue (Fin p) (Fin h)).prod
        (matrixLebesgue (Fin h) (Fin q)) := rfl

lemma continuous_residualKernel {p q h : ℕ} :
    Continuous (residualKernel : ResidualSpace p q h → ℝ) := by
  unfold residualKernel
  exact continuous_frobeniusSq.comp
    (continuous_fst.matrix_mul continuous_snd)

lemma continuous_residualAmbientNormSq {p q h : ℕ} :
    Continuous (residualAmbientNormSq : ResidualSpace p q h → ℝ) := by
  unfold residualAmbientNormSq
  exact (continuous_frobeniusSq.comp continuous_fst).add
    (continuous_frobeniusSq.comp continuous_snd)

lemma residualKernel_nonneg {p q h : ℕ} (w : ResidualSpace p q h) :
    0 ≤ residualKernel w := by
  unfold residualKernel frobeniusSq
  positivity

lemma residualAmbientNormSq_nonneg {p q h : ℕ} (w : ResidualSpace p q h) :
    0 ≤ residualAmbientNormSq w := by
  unfold residualAmbientNormSq frobeniusSq
  positivity

lemma measurable_residualKernel {p q h : ℕ} :
    Measurable (residualKernel : ResidualSpace p q h → ℝ) := by
  unfold residualKernel
  exact Measurable.o77FrobeniusSq
    (Measurable.o77MatrixMul measurable_fst measurable_snd)

lemma measurable_residualAmbientNormSq {p q h : ℕ} :
    Measurable (residualAmbientNormSq : ResidualSpace p q h → ℝ) := by
  unfold residualAmbientNormSq
  exact (Measurable.o77FrobeniusSq measurable_fst).add
    (Measurable.o77FrobeniusSq measurable_snd)
```

```
/-- The quadratic ball for the explicit residual ambient square norm. -/
def residualQuadraticBall (p q h : ℕ) (R : ℝ) : Set (ResidualSpace p q h) :=
  {w | residualAmbientNormSq w ≤ R ^ 2}

lemma measurableSet_residualQuadraticBall (p q h : ℕ) (R : ℝ) :
    MeasurableSet (residualQuadraticBall p q h R) := by
  exact measurableSet_le measurable_residualAmbientNormSq measurable_const

/-- The dyadic annulus between consecutive quadratic balls. -/
def residualQuadraticShell (p q h : ℕ) (R : ℝ) (n : ℕ) :
    Set (ResidualSpace p q h) :=
  residualQuadraticBall p q h ((2 : ℝ) ^ (n + 1) * R) \
    residualQuadraticBall p q h ((2 : ℝ) ^ n * R)

lemma measurableSet_residualQuadraticShell (p q h : ℕ) (R : ℝ) (n : ℕ) :
    MeasurableSet (residualQuadraticShell p q h R n) := by
  exact (measurableSet_residualQuadraticBall p q h _).diff
    (measurableSet_residualQuadraticBall p q h _)

/-- The unweighted fixed-ball Laplace integrand. -/
def residualBallLaplaceIntegrand {p q h : ℕ} (T : ℝ)
    (w : ResidualSpace p q h) : ℝ≥0∞ :=
  ENNReal.ofReal (Real.exp (-T * residualKernel w))

lemma residualBallLaplaceIntegrand_measurable {p q h : ℕ} (T : ℝ) :
    Measurable (residualBallLaplaceIntegrand (p := p) (q := q) (h := h) T) := by
  unfold residualBallLaplaceIntegrand
  exact ENNReal.continuous_ofReal.measurable.comp
    (Real.continuous_exp.measurable.comp
      (measurable_residualKernel.const_mul (-T)))

/-- The unweighted residual Laplace integral over a fixed quadratic ball. -/
def residualBallLaplaceLIntegral (p q h : ℕ) (R T : ℝ) : ℝ≥0∞ :=
  ∫⁻ w in residualQuadraticBall p q h R,
    residualBallLaplaceIntegrand T w ∂residualMatrixLebesgue p q h

/-- Canonical coordinate volume on a finite matrix space, with the
measurable-space instance fixed to the function-valued matrix instance. -/
noncomputable def matrixCoordinateVolume (p h : ℕ) :
    Measure (Matrix (Fin p) (Fin h) ℝ) := by
  change Measure (Fin p → Fin h → ℝ)
  exact volume

lemma matrixLebesgue_fin_eq_volume (p h : ℕ) :
    matrixLebesgue (Fin p) (Fin h) = matrixCoordinateVolume p h := by
  unfold matrixLebesgue matrixCoordinateVolume
  simp_rw [coordinateLebesgue_eq_volume]
  exact MeasureTheory.volume_pi.symm

/-- Canonical product coordinate volume on the residual pair. -/
noncomputable def residualCoordinateVolume (p q h : ℕ) :
    Measure (ResidualSpace p q h) := by
  change Measure ((Fin p → Fin h → ℝ) × (Fin h → Fin q → ℝ))
  exact volume

lemma residualMatrixLebesgue_eq_volume (p q h : ℕ) :
    residualMatrixLebesgue p q h = residualCoordinateVolume p q h := by
  unfold residualMatrixLebesgue residualCoordinateVolume
  rw [matrixLebesgue_fin_eq_volume, matrixLebesgue_fin_eq_volume]
  change (volume : Measure (Fin p → Fin h → ℝ)).prod
      (volume : Measure (Fin h → Fin q → ℝ)) =
    (volume : Measure ((Fin p → Fin h → ℝ) × (Fin h → Fin q → ℝ)))
```

```
  exact (Measure.volume_eq_prod _ _).symm

/-- Real dimension of the residual pair of matrix spaces. -/
def residualDimension (p q h : ℕ) : ℕ := p * h + h * q

lemma finrank_residualSpace (p q h : ℕ) :
    Module.finrank ℝ (ResidualSpace p q h) = residualDimension p q h := by
  change Module.finrank ℝ
      ((Fin p → Fin h → ℝ) × (Fin h → Fin q → ℝ)) = _
  simp [residualDimension, Module.finrank_pi_fintype, Nat.mul_comm]

end O77.Residual

set_option autoImplicit false
set_option linter.style.haveILetI false
noncomputable section

open Real MeasureTheory Set
open scoped ENNReal BigOperators Pointwise

namespace O77.Residual

/-- Simultaneous scalar dilation in the two residual factors. -/
def residualDilation {p q h : ℕ} (c : ℝ) (w : ResidualSpace p q h) :
    ResidualSpace p q h :=
  (c • w.1, c • w.2)

@[simp] lemma residualDilation_apply {p q h : ℕ} (c : ℝ)
    (w : ResidualSpace p q h) :
    residualDilation c w = (c • w.1, c • w.2) := rfl

lemma residualDilation_eq_smul {p q h : ℕ} (c : ℝ)
    (w : ResidualSpace p q h) : residualDilation c w = c • w := rfl

lemma residualDilation_continuous {p q h : ℕ} (c : ℝ) :
    Continuous (residualDilation (p := p) (q := q) (h := h) c) := by
  exact (continuous_fst.const_smul c).prodMk (continuous_snd.const_smul c)

lemma residualDilation_measurable {p q h : ℕ} (c : ℝ) :
    Measurable (residualDilation (p := p) (q := q) (h := h) c) := by
  exact (measurable_fst.const_smul c).prodMk (measurable_snd.const_smul c)

lemma residualDilation_kernel {p q h : ℕ} (c : ℝ)
    (w : ResidualSpace p q h) :
    residualKernel (residualDilation c w) = c ^ 4 * residualKernel w := by
  rcases w with ⟨Y, Z⟩
  exact residualKernel_quartic c Y Z

lemma residualDilation_ambient {p q h : ℕ} (c : ℝ)
    (w : ResidualSpace p q h) :
    residualAmbientNormSq (residualDilation c w) =
      c ^ 2 * residualAmbientNormSq w := by
  rcases w with ⟨Y, Z⟩
  exact residualAmbientNormSq_quadratic c Y Z

/-- The positive-coordinate Jacobian factor of simultaneous dilation. -/
def residualDilationJacobian (p q h : ℕ) (c : ℝ) : ℝ≥∞0 :=
  ENNReal.ofReal c ^ residualDimension p q h

lemma residualDilationJacobian_eq_ofReal_pow (p q h : ℕ) {c : ℝ}
    (hc : 0 ≤ c) :
    residualDilationJacobian p q h c =
```

```
      ENNReal.ofReal (c ^ residualDimension p q h) := by
  simp [residualDilationJacobian, ENNReal.ofReal_pow hc]

noncomputable instance coordinateLebesgue.instIsAddHaarMeasure
    ι( : Type*) [Fintype ι] :
    (coordinateLebesgue ι).IsAddHaarMeasure := by
  rw [coordinateLebesgue_eq_volume]
  exact MeasureTheory.isAddHaarMeasure_volume_pi ι

noncomputable instance matrixLebesgue.instIsAddHaarMeasureFin
    (p h : ℕ) :
    (matrixLebesgue (Fin p) (Fin h)).IsAddHaarMeasure := by
  unfold matrixLebesgue
  exact Measure.pi.isAddHaarMeasure
    (fun _ : Fin p => coordinateLebesgue (Fin h))

noncomputable instance residualMatrixLebesgue.instIsAddHaarMeasure
    (p q h : ℕ) :
    (residualMatrixLebesgue p q h).IsAddHaarMeasure := by
  unfold residualMatrixLebesgue
  exact Measure.prod.instIsAddHaarMeasure _ _

section DilationMeasure

attribute [local instance]
  Matrix.frobeniusNormedAddCommGroup Matrix.frobeniusNormedSpace

noncomputable local instance residualSpaceBorelSpace (p q h : ℕ) :
    BorelSpace (ResidualSpace p q h) :=
  inferInstanceAs (BorelSpace
    ((Fin p → Fin h → ℝ) × (Fin h → Fin q → ℝ)))

/-- The chosen residual product matrix measure scales by the exact ambient
Jacobian under nonnegative scalar multiplication of sets. -/
theorem residualMatrixLebesgue_smul_of_nonneg
    (p q h : ℕ) {c : ℝ} (hc : 0 ≤ c) (s : Set (ResidualSpace p q h)) :
    residualMatrixLebesgue p q h (c • s) =
      residualDilationJacobian p q h c * residualMatrixLebesgue p q h s := by
  rw [@Measure.addHaar_smul_of_nonneg
    (ResidualSpace p q h) _ _ _ (residualSpaceBorelSpace p q h) _
    (residualMatrixLebesgue p q h)
    (residualMatrixLebesgue.instIsAddHaarMeasure p q h) c hc s]
  rw [finrank_residualSpace]
  simp [residualDilationJacobian, ENNReal.ofReal_pow hc]

end DilationMeasure

/-- Simultaneous dilation by a nonzero scalar as a measurable equivalence. -/
def residualDilationMeasurableEquiv {p q h : ℕ} {c : ℝ} (hc : c ≠ 0) :
    ResidualSpace p q h ≃ᵐ ResidualSpace p q h where
  toFun := residualDilation c
  invFun := residualDilation ⁻c¹
  left_inv w := by
    ext <;> simp [residualDilation, smul_smul, hc]
  right_inv w := by
    ext <;> simp [residualDilation, smul_smul, hc]
  measurable_toFun := residualDilation_measurable c
  measurable_invFun := residualDilation_measurable ⁻c¹

@[simp] lemma residualDilationMeasurableEquiv_apply {p q h : ℕ}
    {c : ℝ} (hc : c ≠ 0) (w : ResidualSpace p q h) :
    residualDilationMeasurableEquiv hc w = residualDilation c w := rfl
```

```
@[simp] lemma residualDilationMeasurableEquiv_symm_apply {p q h : ℕ}
    {c : ℝ} (hc : c ≠ 0) (w : ResidualSpace p q h) :
    (residualDilationMeasurableEquiv hc).symm w = residualDilation c⁻¹ w := rfl

lemma residualDilation_image_eq_smul {p q h : ℕ} (c : ℝ)
    (s : Set (ResidualSpace p q h)) :
    residualDilation c '' s = c • s := by
  rfl

lemma residualDilation_preimage_eq_inv_smul {p q h : ℕ}
    {c : ℝ} (hc : c ≠ 0) (s : Set (ResidualSpace p q h)) :
    residualDilation c ⁻¹' s = c⁻¹ • s := by
  ext w
  constructor
  · intro hw
    refine ⟨residualDilation c w, hw, ?_⟩
    ext <;> simp [residualDilation, smul_smul, hc]
  · rintro ⟨u, hu, ⟩rfl
    simpa [residualDilation, smul_smul, hc] using hu

lemma residualDilationJacobian_inv (p q h : ℕ) {c : ℝ} (hc : 0 < c) :
    residualDilationJacobian p q h c⁻¹ =
      (residualDilationJacobian p q h c)⁻¹ := by
  unfold residualDilationJacobian
  rw [ENNReal.ofReal_inv_of_pos hc]
  simp [ENNReal.inv_pow]

lemma residualDilationJacobian_ne_zero (p q h : ℕ) {c : ℝ} (hc : 0 < c) :
    residualDilationJacobian p q h c ≠ 0 := by
  unfold residualDilationJacobian
  exact pow_ne_zero _ (ne_of_gt (ENNReal.ofReal_pos.mpr hc))

end O77.Residual

set_option autoImplicit false
set_option linter.style.haveILetI false
noncomputable section

open Real MeasureTheory Set
open scoped ENNReal BigOperators Pointwise

namespace O77.Residual

/-- Pushforward of the residual coordinate measure under positive dilation. -/
theorem residualDilation_map_measure (p q h : ℕ) {c : ℝ} (hc : 0 < c) :
    Measure.map (residualDilation (p := p) (q := q) (h := h) c)
        (residualMatrixLebesgue p q h) =
      (residualDilationJacobian p q h c)⁻¹ •
        residualMatrixLebesgue p q h := by
  apply Measure.ext
  intro s hs
  rw [Measure.map_apply (residualDilation_measurable c) hs]
  rw [residualDilation_preimage_eq_inv_smul hc.ne' s]
  rw [residualMatrixLebesgue_smul_of_nonneg p q h (inv_nonneg.mpr hc.le) s]
  rw [residualDilationJacobian_inv p q h hc]
  simp [Measure.smul_apply, smul_eq_mul]

/-- Unrestricted nonnegative change of variables under positive dilation. -/
theorem lintegral_residualDilation {p q h : ℕ} {c : ℝ} (hc : 0 < c)
    {f : ResidualSpace p q h → ℝ≥0∞} (hf : Measurable f) :
    ∫⁻ w, f (residualDilation c w) ∂residualMatrixLebesgue p q h) =
```

```
      (residualDilationJacobian p q h c)⁻¹ *
        ∫⁻ w, f w ∂residualMatrixLebesgue p q h := by
  rw ←[ MeasureTheory.lintegral_map hf (residualDilation_measurable c)]
  rw [residualDilation_map_measure p q h hc]
  rw [MeasureTheory.lintegral_smul_measure]
  rfl

lemma mem_residualDilation_image_iff {p q h : ℕ} {c : ℝ} (hc : c ≠ 0)
    {s : Set (ResidualSpace p q h)} {w : ResidualSpace p q h} :
    w ∈ residualDilation c '' s ↔ residualDilation ⁻c¹ w ∈ s := by
  constructor
  · rintro ⟨u, hu, ⟩rfl
    simpa [residualDilation, smul_smul, hc] using hu
  · intro hw
    refine ⟨residualDilation ⁻c¹ w, hw, ?⟩_
    ext <;> simp [residualDilation, smul_smul, hc]

lemma measurableSet_residualDilation_image {p q h : ℕ} {c : ℝ}
    (hc : c ≠ 0) {s : Set (ResidualSpace p q h)} (hs : MeasurableSet s) :
    MeasurableSet (residualDilation c '' s) := by
  exact (residualDilationMeasurableEquiv hc).measurableSet_image.mpr hs

lemma indicator_residualDilation_inv {p q h : ℕ} {c : ℝ} (hc : c ≠ 0)
    (s : Set (ResidualSpace p q h)) (f : ResidualSpace p q h → ℝ≥∞0)
    (w : ResidualSpace p q h) :
    (residualDilation c '' s).indicator f w =
      s.indicator (fun u => f (residualDilation c u))
        (residualDilation ⁻c¹ w) := by
  by_cases hw : w ∈ residualDilation c '' s
  · have hi : residualDilation ⁻c¹ w ∈ s :=
      (mem_residualDilation_image_iff hc).mp hw
    rw [Set.indicator_of_mem hw, Set.indicator_of_mem hi]
    congr 1
    ext <;> simp [residualDilation, smul_smul, hc]
  · have hi : residualDilation ⁻c¹ w ∉ s := by
      intro hi
      exact hw ((mem_residualDilation_image_iff hc).mpr hi)
    have hi' : (⁻c¹ • w.1, ⁻c¹ • w.2) ∉ s := by
      simpa [residualDilation] using hi
    simp [Set.indicator, hw, hi']

/-- Set-restricted nonnegative change of variables. -/
theorem residualDilation_setLIntegral {p q h : ℕ} {c : ℝ} (hc : 0 < c)
    {s : Set (ResidualSpace p q h)} (hs : MeasurableSet s)
    {f : ResidualSpace p q h → ℝ≥∞0} (hf : Measurable f) :
    ∫⁻( w in residualDilation c '' s, f w ∂residualMatrixLebesgue p q h) =
      residualDilationJacobian p q h c *
        ∫⁻ u in s, f (residualDilation c u) ∂residualMatrixLebesgue p q h := by
  have hc_inv : 0 < ⁻c¹ := inv_pos.mpr hc
  have himage := measurableSet_residualDilation_image hc.ne' hs
  rw ←[ MeasureTheory.lintegral_indicator himage]
  rw [show (residualDilation c '' s).indicator f =
      fun w => s.indicator (fun u => f (residualDilation c u))
        (residualDilation ⁻c¹ w) by
      funext w
      exact indicator_residualDilation_inv hc.ne' s f w]
  let g : ResidualSpace p q h → ℝ≥∞0 :=
    s.indicator (fun u => f (residualDilation c u))
  have hg : Measurable g :=
    (hf.comp (residualDilation_measurable c)).indicator hs
  change ∫⁻( w, g (residualDilation ⁻c¹ w)
      ∂residualMatrixLebesgue p q h) = _
```

```
  calc
    ∫⁻( w, g (residualDilation ⁻c¹ w)
        ∂residualMatrixLebesgue p q h) =
        (residualDilationJacobian p q h ⁻c¹)⁻¹ *
          ∫⁻ u, g u ∂residualMatrixLebesgue p q h :=
      lintegral_residualDilation hc_inv hg
    _ = residualDilationJacobian p q h c *
          ∫⁻ u, g u ∂residualMatrixLebesgue p q h := by
      rw [residualDilationJacobian_inv p q h hc, inv_inv]
    _ = residualDilationJacobian p q h c *
          ∫⁻ u in s, f (residualDilation c u)
            ∂residualMatrixLebesgue p q h := by
      rw [MeasureTheory.lintegral_indicator hs]

end O77.Residual

set_option autoImplicit false
noncomputable section

open Real MeasureTheory Set
open scoped ENNReal BigOperators Pointwise

namespace O77.Residual

lemma residualGaussianLaplaceIntegrand_lower_on_ball
    {p q h : ℕ} {R T : ℝ} {w : ResidualSpace p q h}
    (hw : w ∈ residualQuadraticBall p q h R) :
    ENNReal.ofReal (Real.exp (-R ^ 2)) *
        residualBallLaplaceIntegrand T w ≤
      residualGaussianLaplaceIntegrand T w := by
  rw [residualGaussianLaplaceIntegrand_eq_exp]
  unfold residualBallLaplaceIntegrand
  rw ←[ ENNReal.ofReal_mul (Real.exp_pos _).le]
  apply ENNReal.ofReal_le_ofReal
  rw ←[ Real.exp_add]
  apply Real.exp_le_exp.mpr
  have hN : residualAmbientNormSq w ≤ R ^ 2 := by
    simpa [residualQuadraticBall] using hw
  linarith

lemma residualGaussianLaplaceIntegrand_upper_on_ball
    {p q h : ℕ} {T : ℝ} (w : ResidualSpace p q h) :
    residualGaussianLaplaceIntegrand T w ≤
      residualBallLaplaceIntegrand T w := by
  rw [residualGaussianLaplaceIntegrand_eq_exp]
  unfold residualBallLaplaceIntegrand
  apply ENNReal.ofReal_le_ofReal
  apply Real.exp_le_exp.mpr
  have hN := residualAmbientNormSq_nonneg w
  linarith

lemma residualBallLaplaceIntegrand_dilation {p q h : ℕ}
    (c T : ℝ) (w : ResidualSpace p q h) :
    residualBallLaplaceIntegrand T (residualDilation c w) =
      residualBallLaplaceIntegrand (c ^ 4 * T) w := by
  unfold residualBallLaplaceIntegrand
  rw [residualDilation_kernel]
  congr 2
  ring

lemma residualBallLaplaceIntegrand_antitone {p q h : ℕ}
    {₁T ₂T : ℝ} (hT : ₁T ≤ ₂T) (w : ResidualSpace p q h) :
```

```
    residualBallLaplaceIntegrand ₂T w ≤ residualBallLaplaceIntegrand ₁T w := by
  unfold residualBallLaplaceIntegrand
  apply ENNReal.ofReal_le_ofReal
  apply Real.exp_le_exp.mpr
  have hK := residualKernel_nonneg w
  nlinarith

lemma residualBallLaplaceLIntegral_antitone (p q h : ℕ) (R : ℝ)
    {₁T ₂T : ℝ} (hT : ₁T ≤ ₂T) :
    residualBallLaplaceLIntegral p q h R ₂T ≤
      residualBallLaplaceLIntegral p q h R ₁T := by
  unfold residualBallLaplaceLIntegral
  exact MeasureTheory.setLIntegral_mono'
    (measurableSet_residualQuadraticBall p q h R)
    (fun w hw => residualBallLaplaceIntegrand_antitone hT w)

lemma residualDilation_image_ball {p q h : ℕ} {c : ℝ} (hc : 0 < c)
    (R : ℝ) :
    residualDilation c '' residualQuadraticBall p q h R =
      residualQuadraticBall p q h (c * R) := by
  ext w
  constructor
  · rintro ⟨u, hu, ⟩rfl
    have hu' : residualAmbientNormSq u ≤ R ^ 2 := by
      simpa [residualQuadraticBall] using hu
    show residualAmbientNormSq (residualDilation c u) ≤ (c * R) ^ 2
    calc
      residualAmbientNormSq (residualDilation c u) =
          c ^ 2 * residualAmbientNormSq u := residualDilation_ambient c u
      _ ≤ c ^ 2 * R ^ 2 := mul_le_mul_of_nonneg_left hu' (sq_nonneg c)
      _ = (c * R) ^ 2 := by ring
  · intro hw
    have hw' : residualAmbientNormSq w ≤ (c * R) ^ 2 := by
      simpa [residualQuadraticBall] using hw
    refine ⟨residualDilation ⁻c¹ w, ?_, ?⟩_
    · show residualAmbientNormSq (residualDilation ⁻c¹ w) ≤ R ^ 2
      have hc2 : 0 < c ^ 2 := sq_pos_of_pos hc
      calc
        residualAmbientNormSq (residualDilation ⁻c¹ w) =
            ⁻c¹ ^ 2 * residualAmbientNormSq w := residualDilation_ambient ⁻c¹ w
        _ = residualAmbientNormSq w / c ^ 2 := by field_simp
        _ ≤ R ^ 2 := by
          rw [₀div_le_iff hc2]
          calc
            residualAmbientNormSq w ≤ (c * R) ^ 2 := hw'
            _ = R ^ 2 * c ^ 2 := by ring
    · ext <;> simp [residualDilation, smul_smul, hc.ne']

/-- Exact rescaling of the fixed-ball residual Laplace integral. -/
theorem residualDilation_ball_setLIntegral {p q h : ℕ}
    {c : ℝ} (hc : 0 < c) (R T : ℝ) :
    residualBallLaplaceLIntegral p q h (c * R) T =
      residualDilationJacobian p q h c *
        residualBallLaplaceLIntegral p q h R (c ^ 4 * T) := by
  unfold residualBallLaplaceLIntegral
  rw ←[ residualDilation_image_ball hc R]
  rw [residualDilation_setLIntegral hc
    (measurableSet_residualQuadraticBall p q h R)
    (residualBallLaplaceIntegrand_measurable T)]
  apply congrArg (fun x => residualDilationJacobian p q h c * x)
  exact MeasureTheory.setLIntegral_congr_fun
    (measurableSet_residualQuadraticBall p q h R)
```

```
    (fun w hw => residualBallLaplaceIntegrand_dilation c T w)

/-- On a fixed quadratic ball the Gaussian weight is bounded below by
`exp (-R²)`, giving the lower half of Gaussian-to-ball comparison. -/
theorem residualBallLaplace_lower_gaussian (p q h : ℕ) (R T : ℝ) :
    ENNReal.ofReal (Real.exp (-R ^ 2)) *
        residualBallLaplaceLIntegral p q h R T ≤
      residualGaussianLaplaceLIntegral p q h T := by
  unfold residualBallLaplaceLIntegral residualGaussianLaplaceLIntegral
  change ENNReal.ofReal (Real.exp (-R ^ 2)) *
      ∫⁻( w in residualQuadraticBall p q h R,
        residualBallLaplaceIntegrand T w ∂residualMatrixLebesgue p q h) ≤
    ∫⁻ w, residualGaussianLaplaceIntegrand T w
      ∂residualMatrixLebesgue p q h
  calc
    ENNReal.ofReal (Real.exp (-R ^ 2)) *
        ∫⁻( w in residualQuadraticBall p q h R,
          residualBallLaplaceIntegrand T w ∂residualMatrixLebesgue p q h) =
      ∫⁻ w in residualQuadraticBall p q h R,
        ENNReal.ofReal (Real.exp (-R ^ 2)) *
          residualBallLaplaceIntegrand T w
          ∂residualMatrixLebesgue p q h := by
      rw [MeasureTheory.lintegral_const_mul]
      exact (residualBallLaplaceIntegrand_measurable T)
    _ ≤ ∫⁻ w in residualQuadraticBall p q h R,
        residualGaussianLaplaceIntegrand T w
          ∂residualMatrixLebesgue p q h :=
      MeasureTheory.setLIntegral_mono'
        (measurableSet_residualQuadraticBall p q h R)
        (fun w hw => residualGaussianLaplaceIntegrand_lower_on_ball hw)
    _ ≤ ∫⁻ w, residualGaussianLaplaceIntegrand T w
          ∂residualMatrixLebesgue p q h := by
      simpa only [Measure.restrict_univ] using
        (MeasureTheory.lintegral_mono_set
          (show residualQuadraticBall p q h R ⊆ Set.univ from Set.subset_univ _)
          (f := residualGaussianLaplaceIntegrand T)
          μ( := residualMatrixLebesgue p q h))

/-- The Gaussian contribution from the inner ball is bounded by the
unweighted fixed-ball transform. -/
theorem residualGaussian_innerBall_le_ballLaplace
    (p q h : ℕ) (R T : ℝ) :
    ∫⁻( w in residualQuadraticBall p q h R,
        residualGaussianLaplaceIntegrand T w
        ∂residualMatrixLebesgue p q h) ≤
      residualBallLaplaceLIntegral p q h R T := by
  unfold residualBallLaplaceLIntegral
  exact MeasureTheory.setLIntegral_mono'
    (measurableSet_residualQuadraticBall p q h R)
    (fun w hw => residualGaussianLaplaceIntegrand_upper_on_ball w)

end O77.Residual

set_option autoImplicit false
noncomputable section

open Real MeasureTheory Set
open scoped ENNReal BigOperators Pointwise

namespace O77.Residual

/-- Outer radial scale of the `n`-th dyadic shell. -/
```

```
def dyadicScale (n : ℕ) : ℝ := (2 : ℝ) ^ (n + 1)

/-- Inner radial scale of the `n`-th dyadic shell. -/
def dyadicInnerScale (n : ℕ) : ℝ := (2 : ℝ) ^ n

lemma dyadicScale_pos (n : ℕ) : 0 < dyadicScale n := by
  unfold dyadicScale
  exact pow_pos (by norm_num) _

lemma dyadicInnerScale_pos (n : ℕ) : 0 < dyadicInnerScale n := by
  unfold dyadicInnerScale
  exact pow_pos (by norm_num) _

lemma one_le_dyadicScale (n : ℕ) : 1 ≤ dyadicScale n := by
  unfold dyadicScale
  exact ₀one_le_pow (by norm_num)

lemma one_le_dyadicScale_pow_four (n : ℕ) : 1 ≤ dyadicScale n ^ 4 := by
  exact ₀one_le_pow (one_le_dyadicScale n)

lemma residualQuadraticShell_outer {p q h : ℕ} {R : ℝ} {n : ℕ}
    {w : ResidualSpace p q h} (hw : w ∈ residualQuadraticShell p q h R n) :
    residualAmbientNormSq w ≤ (dyadicScale n * R) ^ 2 := by
  simpa [residualQuadraticShell, residualQuadraticBall, dyadicScale] using hw.1

lemma residualQuadraticShell_inner {p q h : ℕ} {R : ℝ} {n : ℕ}
    {w : ResidualSpace p q h} (hw : w ∈ residualQuadraticShell p q h R n) :
    (dyadicInnerScale n * R) ^ 2 < residualAmbientNormSq w := by
  have hnot : ¬ residualAmbientNormSq w ≤ (dyadicInnerScale n * R) ^ 2 := by
    simpa [residualQuadraticShell, residualQuadraticBall, dyadicInnerScale] using hw.2
  exact lt_of_not_ge hnot

/-- Gaussian radial suppression on the `n`-th shell. -/
def residualShellGaussianCoefficient (R : ℝ) (n : ℕ) : ℝ≥∞0 :=
  ENNReal.ofReal (Real.exp (-((dyadicInnerScale n * R) ^ 2)))

lemma residualGaussianLaplaceIntegrand_shell_le
    {p q h : ℕ} {R T : ℝ} {n : ℕ} {w : ResidualSpace p q h}
    (hw : w ∈ residualQuadraticShell p q h R n) :
    residualGaussianLaplaceIntegrand T w ≤
      residualShellGaussianCoefficient R n * residualBallLaplaceIntegrand T w := by
  rw [residualGaussianLaplaceIntegrand_eq_exp]
  unfold residualShellGaussianCoefficient residualBallLaplaceIntegrand
  rw ←[ ENNReal.ofReal_mul (Real.exp_pos _).le]
  apply ENNReal.ofReal_le_ofReal
  rw ←[ Real.exp_add]
  apply Real.exp_le_exp.mpr
  have hinner := residualQuadraticShell_inner hw
  linarith

lemma residualQuadraticShell_subset_dilation_ball
    (p q h : ℕ) (R : ℝ) (n : ℕ) :
    residualQuadraticShell p q h R n ⊆
      residualDilation (dyadicScale n) '' residualQuadraticBall p q h R := by
  intro w hw
  rw [residualDilation_image_ball (dyadicScale_pos n) R]
  exact residualQuadraticShell_outer hw

/-- Gaussian contribution of one dyadic residual shell. -/
def residualGaussianShellLIntegral
    (p q h : ℕ) (R T : ℝ) (n : ℕ) : ℝ≥∞0 :=
  ∫⁻ w in residualQuadraticShell p q h R n,
```

```
    residualGaussianLaplaceIntegrand T w ∂residualMatrixLebesgue p q h

/-- The exact coefficient in the one-shell localization estimate. -/
def residualShellCoefficient
    (p q h : ℕ) (R : ℝ) (n : ℕ) : ℝ≥∞0 :=
  residualShellGaussianCoefficient R n *
    residualDilationJacobian p q h (dyadicScale n)

/-- Each dyadic Gaussian shell is bounded by its Gaussian radial decay,
its exact `D`-dimensional dilation Jacobian, and the fixed-ball transform. -/
theorem residualGaussianShellLIntegral_le
    (p q h : ℕ) (R T : ℝ) (n : ℕ) (hT : 0 ≤ T) :
    residualGaussianShellLIntegral p q h R T n ≤
      residualShellCoefficient p q h R n *
        residualBallLaplaceLIntegral p q h R T := by
  let a := residualShellGaussianCoefficient R n
  let c := dyadicScale n
  have hc : 0 < c := dyadicScale_pos n
  have hscale : T ≤ c ^ 4 * T := by
    have hpow : 1 ≤ c ^ 4 := one_le_dyadicScale_pow_four n
    nlinarith
  unfold residualGaussianShellLIntegral residualShellCoefficient
  change ∫⁻ w in residualQuadraticShell p q h R n,
      residualGaussianLaplaceIntegrand T w ∂residualMatrixLebesgue p q h) ≤
    a * residualDilationJacobian p q h c *
      residualBallLaplaceLIntegral p q h R T
  calc
    ∫⁻ w in residualQuadraticShell p q h R n,
        residualGaussianLaplaceIntegrand T w ∂residualMatrixLebesgue p q h) ≤
      ∫⁻ w in residualQuadraticShell p q h R n,
        a * residualBallLaplaceIntegrand T w ∂residualMatrixLebesgue p q h :=
      MeasureTheory.setLIntegral_mono'
        (measurableSet_residualQuadraticShell p q h R n)
        (fun w hw => residualGaussianLaplaceIntegrand_shell_le hw)
    _ = a * ∫⁻ w in residualQuadraticShell p q h R n,
        residualBallLaplaceIntegrand T w ∂residualMatrixLebesgue p q h) := by
      rw [MeasureTheory.lintegral_const_mul]
      exact residualBallLaplaceIntegrand_measurable T
    _ ≤ a * ∫⁻ w in residualDilation c '' residualQuadraticBall p q h R,
        residualBallLaplaceIntegrand T w ∂residualMatrixLebesgue p q h) := by
      have hmono :
          ∫⁻ w in residualQuadraticShell p q h R n,
            residualBallLaplaceIntegrand T w ∂residualMatrixLebesgue p q h) ≤
          ∫⁻ w in residualDilation c '' residualQuadraticBall p q h R,
            residualBallLaplaceIntegrand T w ∂residualMatrixLebesgue p q h := by
        exact MeasureTheory.lintegral_mono_set
          (residualQuadraticShell_subset_dilation_ball p q h R n)
      exact mul_le_mul_right hmono a
    _ = a * (residualDilationJacobian p q h c *
        residualBallLaplaceLIntegral p q h R (c ^ 4 * T)) := by
      rw [residualDilation_setLIntegral hc
        (measurableSet_residualQuadraticBall p q h R)
        (residualBallLaplaceIntegrand_measurable T)]
      congr 2
      exact MeasureTheory.setLIntegral_congr_fun
        (measurableSet_residualQuadraticBall p q h R)
        (fun w hw => residualBallLaplaceIntegrand_dilation c T w)
    _ ≤ a * (residualDilationJacobian p q h c *
        residualBallLaplaceLIntegral p q h R T) := by
      gcongr
      exact residualBallLaplaceLIntegral_antitone p q h R hscale
    _ = a * residualDilationJacobian p q h c *
```

```
        residualBallLaplaceLIntegral p q h R T := by
      ac_rfl

end O77.Residual

set_option autoImplicit false
noncomputable section

open Real MeasureTheory Set
open scoped ENNReal BigOperators Pointwise

namespace O77.Residual

lemma residualDimension_rectangular_test : residualDimension 2 3 1 = 5 := by
  norm_num [residualDimension]

lemma residualDimension_empty_test : residualDimension 0 0 0 = 0 := by
  norm_num [residualDimension]

lemma residualDilationJacobian_empty_test (c : ℝ) :
    residualDilationJacobian 0 0 0 c = 1 := by
  simp [residualDilationJacobian, residualDimension]

/-- Positivity/nonzeroness cannot be weakened to nonnegativity in the
measurable-equivalence construction: zero dilation is not injective already
on the scalar residual space. -/
lemma residualDilation_zero_not_injective :
    ¬ Function.Injective
      (residualDilation (p := 1) (q := 1) (h := 1) 0) := by
  intro hinj
  let ₀w : ResidualSpace 1 1 1 := (0, 0)
  let ₁w : ResidualSpace 1 1 1 := (1, 0)
  have hmap : residualDilation 0 ₀w = residualDilation 0 ₁w := by
    simp [₀w, ₁w, residualDilation]
  have heq := hinj hmap
  have hentry := congrArg (fun w : ResidualSpace 1 1 1 => w.1 0 0) heq
  norm_num [₀w, ₁w] at hentry

lemma residualGaussianShellLIntegral_scalar_test
    (R T : ℝ) (n : ℕ) (hT : 0 ≤ T) :
    residualGaussianShellLIntegral 1 1 1 R T n ≤
      residualShellCoefficient 1 1 1 R n *
        residualBallLaplaceLIntegral 1 1 1 R T :=
  residualGaussianShellLIntegral_le 1 1 1 R T n hT

end O77.Residual
```

Corresponding Lean declaration(s): `O77.Residual.ResidualSpace`,`O77.Residual.residualMatrixLebesgue`,`O77.Residual.residualMatrixLebesgue_eq`,`O77.Residual.continuous_residualKernel`,`O77.Residual.continuous_residualAmbientNormSq`,`O77.Residual.residualKernel_nonneg`,`O77.Residual.residualAmbientNormSq_nonneg`,`O77.Residual.measurable_residualKernel`,`O77.Residual.measurable_residualAmbientNormSq`,`O77.Residual.residualQuadraticBall`,`O77.Residual.measurableSet_residualQuadraticBall`,`O77.Residual.residualQuadraticShell`,`O77.Residual.measurableSet_residualQuadraticShell`,`O77.Residual.residualBallLaplaceIntegrand`,`O77.Residual.residualBallLaplaceIntegrand_measurable`,`O77.Residual.residualBallLaplaceLIntegral`.

Corresponding Lean declaration(s): `O77.Residual.matrixCoordinateVolume`,`O77.Residual.matrixLebesgue_fin_eq_volume`,`O77.Residual.residualCoordinateVolume`,`O77.Residual.residualMatrixLebesgue_eq_volume`,`O77.Resi

dual.residualDimension,O77.Residual.finrank_residualSpace.

Corresponding Lean declaration(s): O77.Residual.residualDilation,O77.Residual.residualDilation_apply,O77.Residual.residualDilation_eq_smul,O77.Residual.residualDilation_continuous,O77.Residual.residualDilation_measurable,O77.Residual.residualDilation_kernel,O77.Residual.residualDilation_ambient,O77.Residual.residualDilationJacobian,O77.Residual.residualDilationJacobian_eq_ofReal_pow,O77.Residual.coordinateLebesgue.instIsAddHaarMeasure,O77.Residual.matrixLebesgue.instIsAddHaarMeasureFin,O77.Residual.residualMatrixLebesgue.instIsAddHaarMeasure,O77.Residual.residualMatrixLebesgue_smul_of_nonneg,O77.Residual.residualDilationMeasurableEquiv,O77.Residual.residualDilationMeasurableEquiv_apply,O77.Residual.residualDilationMeasurableEquiv_symm_apply,O77.Residual.residualDilation_image_eq_smul,O77.Residual.residualDilation_preimage_eq_inv_smul,O77.Residual.residualDilationJacobian_inv,O77.Residual.residualDilationJacobian_ne_zero.

Corresponding Lean declaration(s): O77.Residual.residualDilation_map_measure,O77.Residual.lintegral_residualDilation,O77.Residual.mem_residualDilation_image_iff,O77.Residual.measurableSet_residualDilation_image,O77.Residual.indicator_residualDilation_inv,O77.Residual.residualDilation_setLIntegral.

Corresponding Lean declaration(s): O77.Residual.residualGaussianLaplaceIntegrand_lower_on_ball,O77.Residual.residualGaussianLaplaceIntegrand_upper_on_ball,O77.Residual.residualBallLaplaceIntegrand_dilation,O77.Residual.residualBallLaplaceIntegrand_antitone,O77.Residual.residualBallLaplaceLIntegral_antitone,O77.Residual.residualDilation_image_ball,O77.Residual.residualDilation_ball_setLIntegral,O77.Residual.residualBallLaplace_lower_gaussian,O77.Residual.residualGaussian_innerBall_le_ballLaplace.

Corresponding Lean declaration(s): O77.Residual.dyadicScale,O77.Residual.dyadicInnerScale,O77.Residual.dyadicScale_pos,O77.Residual.dyadicInnerScale_pos,O77.Residual.one_le_dyadicScale,O77.Residual.one_le_dyadicScale_pow_four,O77.Residual.residualQuadraticShell_outer,O77.Residual.residualQuadraticShell_inner,O77.Residual.residualShellGaussianCoefficient,O77.Residual.residualGaussianLaplaceIntegrand_shell_le,O77.Residual.residualQuadraticShell_subset_dilation_ball,O77.Residual.residualGaussianShellLIntegral,O77.Residual.residualShellCoefficient,O77.Residual.residualGaussianShellLIntegral_le.

Corresponding Lean declaration(s): O77.Residual.residualDimension_rectangular_test,O77.Residual.residualDimension_empty_test,O77.Residual.residualDilationJacobian_empty_test,O77.Residual.residualDilation_zero_not_injective,O77.Residual.residualGaussianShellLIntegral_scalar_test.

28 Global localization and the residual-origin pair

28.1 Summing the localization estimates

The preceding dilation section establishes the exact simultaneous-dilation Jacobian in the residual matrix space and bounds each individual dyadic-shell contribution by

$$2^{(n+1)(ph+hq)}e^{-4^nR^2}J_R(T).$$

We now prove that the half-open shells form an exact measurable partition of the complement of the initial ball, establish summability of the coefficient sequence without sending an infinite extended-real quantity through `ENNReal.toReal`, sum the shell integrals, and then reconcile closed balls (convenient for compactness) with the strict open balls appearing in the radius-first local-volume specification. This section checks all of those steps.

The strongest result of the section is an all-dimensional residual-origin theorem. In positive residual dimensions it is conditional on the single explicit ordinary premise `O77.External.RealWishartEigenvalueFormula`; if one of p, q, h is zero, the residual kernel is identically zero and the neutral pair $(0, 1)$ is proved directly, without that analytic premise. The theorem is instantiated in the actual spaces $\mathbb{R}^{p\times h} \times \mathbb{R}^{h\times q}$ and the coordinate-product Lebesgue measure. The full fiber theorem also uses the determinant-open chart, uniform norm comparison, free-coordinate transport, and adapted-basis transport proved in the later geometric modules.

28.2 Exact dyadic partition, including boundary points

Retain the closed quadratic ball and the half-open shells introduced above:

$$B_R = \{w : \mathcal{N}(w) \le R^2\}, \qquad S_{R,n} = B_{2^{n+1}R} \setminus B_{2^n R}, \tag{R6.18}$$

where $\mathcal{N}(Y, Z) = \|Y\|_F^2 + \|Z\|_F^2$. The choice "closed outer boundary, open inner boundary" is load-bearing. For $R > 0$ and $w \notin B_R$, put

$$x = \frac{\mathcal{N}(w)}{R^2} > 1.$$

Mathlib's Archimedean lemma `exists_nat_pow_near`, with base 4, gives an n such that $4^n \le x < 4^{n+1}$. There are two cases.

If $4^n < x$, then

$$(2^n R)^2 < \mathcal{N}(w) < (2^{n+1}R)^2,$$

so $w \in S_{R,n}$. If $4^n = x$, then necessarily $n > 0$; writing $n = m + 1$, the point lies on the outer boundary of $S_{R,m}$, and hence belongs to that preceding shell. Assigning an equality point to $S_{R,n}$ instead would fail because its inner boundary is excluded. This proves existence without any unspoken generic-radius assumption.

The declarations `iUnion_residualQuadraticShell_eq_compl_ball` and `residualBall_union_iUnion_shells` therefore prove

$$\bigcup_{n\ge 0} S_{R,n} = B_R^c, \qquad B_R \cup \bigcup_{n\ge 0} S_{R,n} = E_{p,q,h}. \tag{R6.19}$$

The shells are pairwise disjoint. If $m < n$, then $2^{m+1}R \le 2^n R$ after squaring, so the outer bound for $S_{R,m}$ is incompatible with the strict inner bound for $S_{R,n}$; the opposite order follows symmetrically.

The hypothesis $R > 0$ cannot be deleted. At $R = 0$ every dyadic radius is zero. In the scalar residual space the point $(Y, Z) = (1, 0)$ lies neither in B_0 nor in any shell. The theorem `residualBall_shell_partition_fails_at_zero` records this counterexample.

28.3 Summability of the exact shell coefficients

Let $D = ph + hq$ and define the real coefficient

$$a_n = e^{-(2^n R)^2} (2^{n+1})^D. \tag{R6.20}$$

The source first proves that the extended-nonnegative coefficient used in the shell estimate is exactly `ENNReal.ofReal` of (R6.20), not merely bounded by it. Direct algebra gives the recurrence

$$a_{n+1} = r_n a_n, \qquad r_n = 2^D e^{-3(2^n R)^2}, \tag{R6.21}$$

and hence $a_{n+1}/a_n = r_n$, since every a_n is strictly positive.

For $R > 0$, one has $(2^n R)^2 = 4^n R^2 \to +\infty$. Therefore $r_n \to 0$. The checked proof invokes the ratio test in its limit form, `summable_of_ratio_test_tendsto_lt_one`, with limiting ratio $0 < 1$. It concludes

$$\sum_{n\geq 0} a_n < \infty. \tag{R6.22}$$

Only after proving real summability and nonnegativity does the code apply `ENNReal.ofReal_tsum_of_nonneg`. Thus

$$\sum_{n\geq 0}^{\mathrm{ENNReal}} \mathrm{residualShellCoefficient}(p,q,h,R,n) < \infty. \tag{R6.23}$$

No use is made of the identity `ENNReal.toReal_top`; in particular an infinite sum cannot be silently converted to zero.

28.4 Summing the shell estimates

By (R6.19), measurability of every shell, and pairwise disjointness, the nonnegative integral decomposes exactly as

$$J_G(T) = \int_{B_R} e^{-T\mathcal{K}(w)-\mathcal{N}(w)}\, dw + \sum_{n\geq 0} G_{R,n}(T), \tag{R6.24}$$

where $G_{R,n}(T)$ is the Gaussian contribution of $S_{R,n}$. The proof uses `lintegral_union` and `lintegral_iUnion`; it is not an exchange of an unsigned series with an ordinary real integral.

For $T \geq 0$, summing the checked one-shell estimates gives

$$\sum_{n\geq 0} G_{R,n}(T) \leq \left(\sum_{n\geq 0} a_n\right) J_R(T). \tag{R6.25}$$

The inner-ball Gaussian contribution is at most $J_R(T)$. Define

$$A_{p,q,h,R} = 1 + \sum_{n\geq 0} a_n. \tag{R6.26}$$

By (R6.23), $0 < A_{p,q,h,R} < \infty$, and consequently

$$J_G(T) \leq A_{p,q,h,R} J_R(T). \tag{R6.27}$$

Together with the earlier lower comparison,

$$e^{-R^2} J_R(T) \leq J_G(T), \tag{R6.28}$$

this proves the unconditional localization theorem

$$0 < a_R, A_R < \infty, \qquad a_R J_R(T) \leq J_G(T) \leq A_R J_R(T) \quad (T \geq 0), \tag{R6.29}$$

for every $R > 0$. The complete Lean type is `O77.Residual.homogeneous_gaussianLaplace_comparable_ball_instantiated`. It has no Wishart parameter and no abstract contract argument.

28.5 Transporting the large-parameter power–log order

The general lemma `O77.Residual.LaplacePowerLogBounds.of_two_sided_comparison` formalizes the following safe cancellation statement. Suppose $a, A \in \mathbb{R}_{\ge 0} \cup \{\infty\}$ satisfy $0 < a, A < \infty$ and

$$aJ'(T) \le J(T) \le AJ'(T) \quad (T \ge 0).$$

If J has a two-sided large-T power–log order, then so does J'. The proof converts a and A to real numbers only after separately proving that they are nonzero and not top. The lower and upper constants are then multiplied by A^{-1} and a^{-1} respectively; the upper constant is enlarged by a maximum to preserve the convention $c \le C$.

Combining this lemma with (R6.29) and the conditional Gaussian estimate gives, for $p, q, h > 0$ and $R > 0$,

$$J_R(T) \asymp T^{-d_0(p,q,h)/2} (\log T)^{\mu_0(p,q,h)-1}, \tag{R7.1}$$

conditional only on the ordinary theorem parameter `hW:O77.External.RealWishartEigenvalueFormula`. The corresponding declaration is `residualBallLaplace_powerLog_bounds_of_wishart_instantiated`. Nothing in the localization argument supplies or weakens the Wishart premise.

28.6 Finite closed-ball measure and Laplace-to-sublevel transfer

To apply the previously checked finite-measure Laplace-to-sublevel theorem, the source proves that B_R has finite measure. This proof does not identify B_R with a ball for an unspecified matrix norm. Instead, if $w = (Y, Z) \in B_R$, then every coordinate square is bounded by its matrix Frobenius square, hence by $\mathcal{N}(w) \le R^2$. Therefore every matrix entry lies in the compact interval $[-|R|, |R|]$. The set B_R is closed by continuity of $\mathcal{N}$, so it is a closed subset of a finite product of compact intervals and is compact. Additive Haar measure is finite on compact sets.

The restricted measure

$$\mu_R = \lambda_{p,q,h}|_{B_R}$$

is defined as `residualQuadraticBallMeasure` and furnished with an `IsFiniteMeasure` instance. Its Laplace transform is definitionally J_R. The checked Laplace-to-sublevel theorem therefore gives, for positive residual dimensions,

$$\mu_R\{w : \mathcal{K}(w) < \varepsilon\} \asymp \varepsilon^{d_0/2} \left(\log \frac{1}{\varepsilon}\right)^{\mu_0 - 1} \quad (\varepsilon \downarrow 0), \tag{R7.2}$$

again conditional only on hW. The declaration `residualClosedBallSublevel_order_of_wishart_instantiated` keeps the strict kernel sublevel and the closed ambient window distinct.

28.7 Strict open balls and the radius-first quantifiers

The local O77 specification uses a strict ambient ball. Rather than prove that the sphere $\{\mathcal{N} = R^2\}$ is null, the source uses the elementary sandwich

$$B_{R/2} \subset B_R^\circ \subset B_R \qquad (R > 0), \tag{R7.3}$$

where $B_R^\circ = \{\mathcal{N} < R^2\}$. The lower power–log bound for the closed half-ball and the upper bound for the closed full ball imply the same pair for the open ball. Constants are chosen only after the radius has been fixed, and the two epsilon thresholds are replaced by their minimum. This proves `residualOpenBallSublevel_order_of_wishart_instantiated` without a boundary-null lemma.

The residual-specific predicate

$$\mathsf{HasResidualBandPairAtOrigin}(p,q,h;\lambda,m) \\ :\Longleftrightarrow \quad 0\le\lambda,\ 1\le m,\ \exists R_0>0\ \forall R\in(0,R_0)\ \mathsf{EThetaZeroPowerLog}\big(V_R,\lambda,m-1\big) \tag{R7.4}$$

uses exactly the required radius-first order: first choose R_0, then take an arbitrary smaller R, and only inside `EThetaZeroPowerLog` choose the two comparison constants and the small-ε threshold. Here

$$V_R(\varepsilon)=\lambda_{p,q,h}\{w:\mathcal{N}(w)<R^2,\ \mathcal{K}(w)<\varepsilon\}.$$

Since $\mathcal{K}\ge 0$ and $\mathcal{K}(0)=0$, no absolute value is needed at the residual origin. For $p,q,h>0$, the theorem `residualBandPair_of_wishart_instantiated` proves

$$\mathsf{HasResidualBandPairAtOrigin}\left(p,q,h;\frac{d_0(p,q,h)}{2},\mu_0(p,q,h)\right) \tag{R7.5}$$

under the single explicit Wishart premise.

28.8 Neutral residual factors and the all-dimensional endpoint

If $p=0$, $q=0$, or $h=0$, matrix multiplication has a zero outer or inner dimension and $\mathcal{K}\equiv 0$. For every $R>0$, the open quadratic ball is an open nonempty set, hence has strictly positive additive-Haar measure; it is contained in the compact closed ball, hence has finite measure. For every $\varepsilon>0$ its strict kernel sublevel is the whole open ball. The volume is therefore a positive constant in ε, and

$$\mathrm{powerLogZero}(0,0;\varepsilon)=1.$$

This directly proves the neutral pair $(0,1)$ in `residualBandPair_neutral_of_zero_dimension`. It does not invoke the positive-ρ Laguerre theorem, which is inapplicable when $p=0$.

The strongest theorem of this section is

```
theorem residualBandPair_all_dimensions_of_wishart_instantiated
    (hW : O77.External.RealWishartEigenvalueFormula)
    (p q h : ℕ) :
    HasResidualBandPairAtOrigin p q h
      ((O77.residualMin p q h : ℝ) / 2)
      (O77.residualMultiplicity p q h)
```

It splits on the three zero-dimensional alternatives. In those branches, the external parameter is unused and the neutral theorem applies. In the positive branch it invokes (R7.5). Thus the residual matrix-product origin theorem in this module exposes one ordinary Wishart argument, and all empty cases are internal. Its signature remains conditional for reuse. In the completed development, `O77.GlobalSpectralIntegral.realWishartEigenvalueFormula` supplies that argument, and the final theorem has no Wishart premise.

28.9 Adversarial controls and what this does not prove

The compiled controls include the nontrivial scalar case $(p,q,h)=(1,1,1)$, where the conditional pair is $(1/2,2)$, and the empty-output case $(0,3,2)$, where the unconditional neutral pair is $(0,1)$. Two tempting strengthenings are explicitly rejected: radius zero does not yield a shell partition, and simultaneous dilation of the residual kernel is quartic rather than quadratic.

The predicate (R7.4) is deliberately residual-specific. It does not identify its window with the arbitrary-point parameter-space ball in the public theorem, and it does not add the regular quadratic variables or free gauge variables produced by the elimination chart. Accordingly, this section completes R6 and the residual-origin content of R7, but it does not prove the measured

formula at an arbitrary exact factorization, the semantic minimum locus, any saddle-rung theorem, or `Answers077`.

28.10 Global localization and residual origin pair: Lean code

The following block compiles only after all component lemmas in the exact document-order stream.

```
set_option autoImplicit false
noncomputable section

open Real MeasureTheory Set Filter
open scoped ENNReal BigOperators Pointwise Topology

namespace O77.Residual

lemma dyadicInnerScale_mul_sq (n : ℕ) (R : ℝ) :
    (dyadicInnerScale n * R) ^ 2 = (4 : ℝ) ^ n * R ^ 2 := by
  unfold dyadicInnerScale
  rw [mul_pow]
  congr 1
  calc
    ((2 : ℝ) ^ n) ^ 2 = (2 : ℝ) ^ (n * 2) :=
      (pow_mul (2 : ℝ) n 2).symm
    _ = (2 : ℝ) ^ (2 * n) := by congr 1; omega
    _ = ((2 : ℝ) ^ 2) ^ n := pow_mul (2 : ℝ) 2 n
    _ = (4 : ℝ) ^ n := by norm_num

lemma dyadicScale_mul_sq (n : ℕ) (R : ℝ) :
    (dyadicScale n * R) ^ 2 = (4 : ℝ) ^ (n + 1) * R ^ 2 := by
  unfold dyadicScale
  rw [mul_pow]
  congr 1
  calc
    ((2 : ℝ) ^ (n + 1)) ^ 2 = (2 : ℝ) ^ ((n + 1) * 2) :=
      (pow_mul (2 : ℝ) (n + 1) 2).symm
    _ = (2 : ℝ) ^ (2 * (n + 1)) := by congr 1; omega
    _ = ((2 : ℝ) ^ 2) ^ (n + 1) := pow_mul (2 : ℝ) 2 (n + 1)
    _ = (4 : ℝ) ^ (n + 1) := by norm_num

/-- Every point outside a positive-radius residual quadratic ball belongs to
one of the half-open dyadic shells. Boundary points at radius `2^n R` are
placed in the preceding shell. -/
theorem exists_mem_residualQuadraticShell_of_not_mem_ball
    (p q h : ℕ) {R : ℝ} (hR : 0 < R) {w : ResidualSpace p q h}
    (hw : w ∉ residualQuadraticBall p q h R) :
    ∃ n : ℕ, w ∈ residualQuadraticShell p q h R n := by
  let N : ℝ := residualAmbientNormSq w
  have hR2 : 0 < R ^ 2 := sq_pos_of_pos hR
  have hNR : R ^ 2 < N := by
    have hnot : ¬ N ≤ R ^ 2 := by
      simpa [N, residualQuadraticBall] using hw
    exact lt_of_not_ge hnot
  let x : ℝ := N / R ^ 2
  have hx : 1 < x := by
    rw [show x = N / R ^ 2 by rfl]
    exact (₀lt_div_iff hR2).2 (by simpa using hNR)
  obtain ⟨n, hnle, ⟩hnlt :=
    exists_nat_pow_near (show (1 : ℝ) ≤ x from hx.le)
      (by norm_num : (1 : ℝ) < 4)
  by_cases heq : (4 : ℝ) ^ n = x
```

```
  · have hn0 : n ≠ 0 := by
      intro hn
      subst n
      norm_num at heq
      linarith
    obtain ⟨m, ⟩rfl := Nat.exists_eq_succ_of_ne_zero hn0
    refine ⟨m, ?⟩_
    change w ∈ residualQuadraticBall p q h (dyadicScale m * R) ∧
      w ∉ residualQuadraticBall p q h (dyadicInnerScale m * R)
    have hN : N = (4 : ℝ) ^ (m + 1) * R ^ 2 := by
      calc
        N = x * R ^ 2 := by
          dsimp [x]
          field_simp [ne_of_gt hR2]
        _ = (4 : ℝ) ^ (m + 1) * R ^ 2 := by rw ←[ heq]
    constructor
    · change residualAmbientNormSq w ≤ (dyadicScale m * R) ^ 2
      rw [dyadicScale_mul_sq, ← hN]
    · change ¬ residualAmbientNormSq w ≤ (dyadicInnerScale m * R) ^ 2
      rw [dyadicInnerScale_mul_sq]
      have hpow : (4 : ℝ) ^ m < (4 : ℝ) ^ (m + 1) := by
        rw [pow_succ]
        have hp : 0 < (4 : ℝ) ^ m := pow_pos (by norm_num) _
        nlinarith
      have hlt : (4 : ℝ) ^ m * R ^ 2 < N := by
        rw [hN]
        exact mul_lt_mul_of_pos_right hpow hR2
      exact not_le_of_gt hlt
  · have hnstrict : (4 : ℝ) ^ n < x := lt_of_le_of_ne hnle heq
    refine ⟨n, ?⟩_
    change w ∈ residualQuadraticBall p q h (dyadicScale n * R) ∧
      w ∉ residualQuadraticBall p q h (dyadicInnerScale n * R)
    constructor
    · change residualAmbientNormSq w ≤ (dyadicScale n * R) ^ 2
      rw [dyadicScale_mul_sq]
      have hout : N < (4 : ℝ) ^ (n + 1) * R ^ 2 := by
        rw [show x = N / R ^ 2 by rfl] at hnlt
        exact (₀div_lt_iff hR2).1 hnlt
      exact le_of_lt hout
    · change ¬ residualAmbientNormSq w ≤ (dyadicInnerScale n * R) ^ 2
      rw [dyadicInnerScale_mul_sq]
      have hin : (4 : ℝ) ^ n * R ^ 2 < N := by
        rw [show x = N / R ^ 2 by rfl] at hnstrict
        exact (₀lt_div_iff hR2).1 hnstrict
      exact not_le_of_gt hin

end O77.Residual

set_option autoImplicit false
noncomputable section

open Real MeasureTheory Set Filter
open scoped ENNReal BigOperators Pointwise Topology

namespace O77.Residual

lemma residualQuadraticBall_subset_dyadicInnerBall
    (p q h : ℕ) (R : ℝ) (n : ℕ) :
    residualQuadraticBall p q h R ⊆
      residualQuadraticBall p q h (dyadicInnerScale n * R) := by
  intro w hw
  change residualAmbientNormSq w ≤ (dyadicInnerScale n * R) ^ 2
```

```
  have hbase : (1 : ℝ) ≤ (4 : ℝ) ^ n :=
    ₀pow_le_pow_right (by norm_num : (1 : ℝ) ≤ 4) (Nat.zero_le n)
  have hR2 : 0 ≤ R ^ 2 := sq_nonneg R
  have hrad : R ^ 2 ≤ (4 : ℝ) ^ n * R ^ 2 := by
    simpa only [one_mul] using mul_le_mul_of_nonneg_right hbase hR2
  rw [dyadicInnerScale_mul_sq]
  exact hw.trans hrad

/-- For positive radius, the union of the half-open dyadic shells is exactly
the complement of the initial closed quadratic ball. -/
theorem iUnion_residualQuadraticShell_eq_compl_ball
    (p q h : ℕ) {R : ℝ} (hR : 0 < R) :
    ⋃( n : ℕ, residualQuadraticShell p q h R n) =
      (residualQuadraticBall p q h R)ᶜ := by
  ext w
  constructor
  · intro hw
    simp only [mem_iUnion] at hw
    obtain ⟨n, ⟩hn := hw
    have hsub := residualQuadraticBall_subset_dyadicInnerBall p q h R n
    exact fun hball => hn.2 (hsub hball)
  · intro hw
    have hnot : w ∉ residualQuadraticBall p q h R := by
      simpa only [mem_compl_iff] using hw
    obtain ⟨n, ⟩hn :=
      exists_mem_residualQuadraticShell_of_not_mem_ball p q h hR hnot
    exact mem_iUnion.2 ⟨n, ⟩hn

/-- The initial ball together with all half-open dyadic shells covers the
entire residual ambient space. -/
theorem residualBall_union_iUnion_shells
    (p q h : ℕ) {R : ℝ} (hR : 0 < R) :
    residualQuadraticBall p q h R ∪
        ⋃( n : ℕ, residualQuadraticShell p q h R n) = Set.univ := by
  rw [iUnion_residualQuadraticShell_eq_compl_ball p q h hR]
  exact union_compl_self _

/-- Distinct half-open dyadic shells are disjoint. -/
theorem residualQuadraticShell_pairwiseDisjoint
    (p q h : ℕ) (R : ℝ) :
    Pairwise (Function.onFun Disjoint
      (residualQuadraticShell p q h R)) := by
  intro m n hmn
  rcases lt_or_gt_of_ne hmn with hmnlt | hmnlt
  · change Disjoint (residualQuadraticShell p q h R m)
        (residualQuadraticShell p q h R n)
    rw [Set.disjoint_left]
    intro w hwm hwn
    have hmout := residualQuadraticShell_outer hwm
    have hninner := residualQuadraticShell_inner hwn
    have hpow : (4 : ℝ) ^ (m + 1) ≤ (4 : ℝ) ^ n := by
      exact ₀pow_le_pow_right (by norm_num : (1 : ℝ) ≤ 4) (by omega)
    have hR2 : 0 ≤ R ^ 2 := sq_nonneg R
    have hrad : (dyadicScale m * R) ^ 2 ≤
        (dyadicInnerScale n * R) ^ 2 := by
      rw [dyadicScale_mul_sq, dyadicInnerScale_mul_sq]
      exact mul_le_mul_of_nonneg_right hpow hR2
    exact (not_lt_of_ge (hmout.trans hrad)) hninner
  · change Disjoint (residualQuadraticShell p q h R m)
        (residualQuadraticShell p q h R n)
    exact Disjoint.symm (by
      rw [Set.disjoint_left]
```

```
      intro w hwn hwm
      have hnout := residualQuadraticShell_outer hwn
      have hminner := residualQuadraticShell_inner hwm
      have hpow : (4 : ℝ) ^ (n + 1) ≤ (4 : ℝ) ^ m := by
        exact pow_le_pow_right (by norm_num : (1 : ℝ) ≤ 4) (by omega)
      have hR2 : 0 ≤ R ^ 2 := sq_nonneg R
      have hrad : (dyadicScale n * R) ^ 2 ≤
          (dyadicInnerScale m * R) ^ 2 := by
        rw [dyadicScale_mul_sq, dyadicInnerScale_mul_sq]
        exact mul_le_mul_of_nonneg_right hpow hR2
      exact (not_lt_of_ge (hnout.trans hrad)) hminner

end O77.Residual

set_option autoImplicit false
noncomputable section

open Real MeasureTheory Set Filter
open scoped ENNReal BigOperators Pointwise Topology

namespace O77.Residual

/-- Real-valued form of the dyadic shell coefficient. -/
def residualShellCoefficientReal
    (p q h : ℕ) (R : ℝ) (n : ℕ) : ℝ :=
  Real.exp (-((((2 : ℝ) ^ n) * R) ^ 2)) *
    (((2 : ℝ) ^ (n + 1)) ^ residualDimension p q h)

lemma residualShellCoefficientReal_pos
    (p q h : ℕ) (R : ℝ) (n : ℕ) :
    0 < residualShellCoefficientReal p q h R n := by
  unfold residualShellCoefficientReal
  positivity

lemma residualShellCoefficient_eq_ofReal
    (p q h : ℕ) (R : ℝ) (n : ℕ) :
    residualShellCoefficient p q h R n =
      ENNReal.ofReal (residualShellCoefficientReal p q h R n) := by
  unfold residualShellCoefficient residualShellGaussianCoefficient
    residualDilationJacobian residualShellCoefficientReal
    dyadicInnerScale dyadicScale
  rw [ENNReal.ofReal_mul (Real.exp_pos _).le,
    ENNReal.ofReal_pow (by positivity : (0 : ℝ) ≤ (2 : ℝ) ^ (n + 1))]

/-- Exact ratio factor between consecutive real shell coefficients. -/
def residualShellRatioReal
    (p q h : ℕ) (R : ℝ) (n : ℕ) : ℝ :=
  ((2 : ℝ) ^ residualDimension p q h) *
    Real.exp (-3 * ((((2 : ℝ) ^ n) * R) ^ 2))

lemma residualShellCoefficientReal_succ
    (p q h : ℕ) (R : ℝ) (n : ℕ) :
    residualShellCoefficientReal p q h R (n + 1) =
      residualShellRatioReal p q h R n *
        residualShellCoefficientReal p q h R n := by
  unfold residualShellCoefficientReal residualShellRatioReal
  rw [show (((2 : ℝ) ^ (n + 1) * R) ^ 2) =
      4 * (((2 : ℝ) ^ n * R) ^ 2) by
    rw [pow_succ]
    ring]
  rw [show ((2 : ℝ) ^ (n + 1 + 1)) ^ residualDimension p q h =
      (2 : ℝ) ^ residualDimension p q h *
```

```
        (((2 : ℝ) ^ (n + 1)) ^ residualDimension p q h) by
    rw [show n + 1 + 1 = (n + 1) + 1 by omega, pow_succ, mul_pow]
    ring]
  rw [show -(4 * (((2 : ℝ) ^ n * R) ^ 2)) =
      (-3 * (((2 : ℝ) ^ n * R) ^ 2)) +
        (-(((2 : ℝ) ^ n * R) ^ 2)) by ring]
  rw [Real.exp_add]
  ring

lemma residualShellCoefficientReal_ratio
    (p q h : ℕ) (R : ℝ) (n : ℕ) :
    residualShellCoefficientReal p q h R (n + 1) /
        residualShellCoefficientReal p q h R n =
      residualShellRatioReal p q h R n := by
  rw [residualShellCoefficientReal_succ]
  field_simp [ne_of_gt (residualShellCoefficientReal_pos p q h R n)]

lemma tendsto_dyadic_radius_sq_atTop
    (R : ℝ) (hR : 0 < R) :
    Tendsto (fun n : ℕ => (((2 : ℝ) ^ n * R) ^ 2)) atTop atTop := by
  have hp : Tendsto (fun n : ℕ => (4 : ℝ) ^ n) atTop atTop :=
    tendsto_pow_atTop_atTop_of_one_lt (by norm_num)
  have hR2 : 0 < R ^ 2 := sq_pos_of_pos hR
  have heq : (fun n : ℕ => (((2 : ℝ) ^ n * R) ^ 2)) =
      (fun n : ℕ => (4 : ℝ) ^ n * R ^ 2) := by
    funext n
    exact dyadicInnerScale_mul_sq n R
  rw [heq]
  exact hp.atTop_mul_const hR2

lemma residualShellRatioReal_tendsto_zero
    (p q h : ℕ) (R : ℝ) (hR : 0 < R) :
    Tendsto (residualShellRatioReal p q h R) atTop (nhds 0) := by
  unfold residualShellRatioReal
  have hsq := tendsto_dyadic_radius_sq_atTop R hR
  have hthree : Tendsto
      (fun n : ℕ => 3 * (((2 : ℝ) ^ n * R) ^ 2)) atTop atTop :=
    hsq.const_mul_atTop (by norm_num)
  have hexp : Tendsto
      (fun n : ℕ => Real.exp (-(3 * (((2 : ℝ) ^ n * R) ^ 2))))
      atTop (nhds 0) :=
    Real.tendsto_exp_neg_atTop_nhds_zero.comp hthree
  have hc : Tendsto
      (fun _ : ℕ => (2 : ℝ) ^ residualDimension p q h) atTop
      (nhds ((2 : ℝ) ^ residualDimension p q h)) := tendsto_const_nhds
  simpa only [neg_mul, mul_zero] using hc.mul hexp

/-- The real dyadic shell coefficient series is summable for every positive
radius. The proof is the ratio test, with limiting ratio zero. -/
theorem gaussian_shell_series_summable_real
    (p q h : ℕ) {R : ℝ} (hR : 0 < R) :
    Summable (residualShellCoefficientReal p q h R) := by
  apply summable_of_ratio_test_tendsto_lt_one (l := 0)
  · norm_num
  · exact Filter.Eventually.of_forall
      (fun n => ne_of_gt (residualShellCoefficientReal_pos p q h R n))
  · have hfun :
        (fun n : ℕ =>
          ‖residualShellCoefficientReal p q h R (n + 1)‖ /
            ‖residualShellCoefficientReal p q h R ‖n) =
          residualShellRatioReal p q h R := by
      funext n
```

```
      rw [Real.norm_eq_abs,
        abs_of_pos (residualShellCoefficientReal_pos p q h R (n + 1)),
        Real.norm_eq_abs,
        abs_of_pos (residualShellCoefficientReal_pos p q h R n)]
      exact residualShellCoefficientReal_ratio p q h R n
    rw [hfun]
    exact residualShellRatioReal_tendsto_zero p q h R hR

/-- The actual ENNReal shell coefficient series has finite total mass. -/
theorem gaussian_shell_series_tsum_lt_top
    (p q h : ℕ) {R : ℝ} (hR : 0 < R) :
    ∑(' n : ℕ, residualShellCoefficient p q h R n) < ∞ := by
  have hs := gaussian_shell_series_summable_real p q h hR
  have hnonneg : ∀ n : ℕ, 0 ≤ residualShellCoefficientReal p q h R n :=
    fun n => (residualShellCoefficientReal_pos p q h R n).le
  have heq :
      ENNReal.ofReal ∑(' n : ℕ, residualShellCoefficientReal p q h R n) =
        ∑' n : ℕ, residualShellCoefficient p q h R n := by
    rw [ENNReal.ofReal_tsum_of_nonneg hnonneg hs]
    congr 1
    funext n
    exact (residualShellCoefficient_eq_ofReal p q h R n).symm
  rw ←[ heq]
  exact lt_top_iff_ne_top.mpr ENNReal.ofReal_ne_top

end O77.Residual

set_option autoImplicit false
noncomputable section

open Real MeasureTheory Set Filter
open scoped ENNReal BigOperators Pointwise Topology

namespace O77.Residual

/-- The Gaussian residual integral is exactly the contribution of the initial
quadratic ball plus the sum of its pairwise-disjoint half-open dyadic shells. -/
theorem residualGaussianLaplaceLIntegral_eq_ball_add_shells
    (p q h : ℕ) {R : ℝ} (hR : 0 < R) (T : ℝ) :
    residualGaussianLaplaceLIntegral p q h T =
      ∫⁻( w in residualQuadraticBall p q h R,
        residualGaussianLaplaceIntegrand T w
          ∂residualMatrixLebesgue p q h) +
      ∑' n : ℕ, residualGaussianShellLIntegral p q h R T n := by
  unfold residualGaussianLaplaceLIntegral
  change ∫⁻( w, residualGaussianLaplaceIntegrand T w
      ∂residualMatrixLebesgue p q h) = _
  let B := residualQuadraticBall p q h R
  let S := fun n : ℕ => residualQuadraticShell p q h R n
  have hcover : B ∪ ⋃( n : ℕ, S n) = Set.univ := by
    simpa [B, S] using residualBall_union_iUnion_shells p q h hR
  have hmeasS : ∀ n : ℕ, MeasurableSet (S n) := by
    intro n
    simpa [S] using measurableSet_residualQuadraticShell p q h R n
  have hmeasUnion : MeasurableSet ⋃( n : ℕ, S n) :=
    MeasurableSet.iUnion hmeasS
  have hdisj : Disjoint B ⋃( n : ℕ, S n) := by
    rw [Set.disjoint_left]
    intro w hwB hwS
    have heq : ⋃( n : ℕ, S n) = ᶜB := by
      simpa [B, S] using iUnion_residualQuadraticShell_eq_compl_ball p q h hR
    rw [heq] at hwS
```

```
    exact hwS hwB
  calc
    ∫⁻( w, residualGaussianLaplaceIntegrand T w
        ∂residualMatrixLebesgue p q h) =
      ∫⁻ w in B ∪ ⋃( n : ℕ, S n),
        residualGaussianLaplaceIntegrand T w
          ∂residualMatrixLebesgue p q h := by
      rw [hcover]
      simp
    _ = ∫⁻( w in B, residualGaussianLaplaceIntegrand T w
          ∂residualMatrixLebesgue p q h) +
        ∫⁻ w in ⋃ n : ℕ, S n, residualGaussianLaplaceIntegrand T w
          ∂residualMatrixLebesgue p q h :=
      MeasureTheory.lintegral_union hmeasUnion hdisj
    _ = ∫⁻( w in residualQuadraticBall p q h R,
          residualGaussianLaplaceIntegrand T w
            ∂residualMatrixLebesgue p q h) +
        ∑' n : ℕ, residualGaussianShellLIntegral p q h R T n := by
      rw [MeasureTheory.lintegral_iUnion hmeasS
        (residualQuadraticShell_pairwiseDisjoint p q h R)]
      rfl

/-- Summing the checked one-shell estimates bounds the entire Gaussian tail
by the total shell coefficient times the fixed-ball transform. -/
theorem residualGaussianShells_tsum_le
    (p q h : ℕ) (R T : ℝ) (hT : 0 ≤ T) :
    ∑(' n : ℕ, residualGaussianShellLIntegral p q h R T n) ≤
      ∑(' n : ℕ, residualShellCoefficient p q h R n) *
        residualBallLaplaceLIntegral p q h R T := by
  calc
    ∑(' n : ℕ, residualGaussianShellLIntegral p q h R T n) ≤
        ∑' n : ℕ, residualShellCoefficient p q h R n *
          residualBallLaplaceLIntegral p q h R T :=
      ENNReal.tsum_le_tsum
        (fun n => residualGaussianShellLIntegral_le p q h R T n hT)
    _ = ∑(' n : ℕ, residualShellCoefficient p q h R n) *
          residualBallLaplaceLIntegral p q h R T :=
      ENNReal.tsum_mul_right

/-- Explicit finite coefficient in the global Gaussian-to-ball comparison. -/
def residualGaussianBallComparisonConstant
    (p q h : ℕ) (R : ℝ) : ℝ≥∞0 :=
  1 + ∑' n : ℕ, residualShellCoefficient p q h R n

lemma residualGaussianBallComparisonConstant_pos
    (p q h : ℕ) (R : ℝ) :
    0 < residualGaussianBallComparisonConstant p q h R := by
  unfold residualGaussianBallComparisonConstant
  positivity

lemma residualGaussianBallComparisonConstant_lt_top
    (p q h : ℕ) {R : ℝ} (hR : 0 < R) :
    residualGaussianBallComparisonConstant p q h R < ∞ := by
  unfold residualGaussianBallComparisonConstant
  exact ENNReal.add_lt_top.mpr
    ⟨by simp, gaussian_shell_series_tsum_lt_top p q h ⟩hR

/-- The complete Gaussian transform is bounded above by one explicit finite
constant times the transform on a fixed positive-radius quadratic ball. -/
theorem residualGaussianLaplace_upper_ball
    (p q h : ℕ) {R T : ℝ} (hR : 0 < R) (hT : 0 ≤ T) :
    residualGaussianLaplaceLIntegral p q h T ≤
```

```
      residualGaussianBallComparisonConstant p q h R *
        residualBallLaplaceLIntegral p q h R T := by
  rw [residualGaussianLaplaceLIntegral_eq_ball_add_shells p q h hR T]
  calc
    ∫⁻( w in residualQuadraticBall p q h R,
        residualGaussianLaplaceIntegrand T w
          ∂residualMatrixLebesgue p q h) +
        ∑' n : ℕ, residualGaussianShellLIntegral p q h R T n ≤
      residualBallLaplaceLIntegral p q h R T +
        ∑(' n : ℕ, residualShellCoefficient p q h R n) *
          residualBallLaplaceLIntegral p q h R T :=
      add_le_add
        (residualGaussian_innerBall_le_ballLaplace p q h R T)
        (residualGaussianShells_tsum_le p q h R T hT)
    _ = residualGaussianBallComparisonConstant p q h R *
        residualBallLaplaceLIntegral p q h R T := by
      unfold residualGaussianBallComparisonConstant
      rw [add_mul, one_mul]

/-- Fully instantiated homogeneous localization: for every positive fixed radius,
the Gaussian-weighted and fixed-ball residual transforms are comparable by
strictly positive finite constants, uniformly for all nonnegative Laplace
parameters. No Wishart input is used. -/
theorem homogeneous_gaussianLaplace_comparable_ball_instantiated
    (p q h : ℕ) {R : ℝ} (hR : 0 < R) :
    ∃ a A : ℝ≥∞0,
      0 < a ∧ a < ∞ ∧ 0 < A ∧ A < ∞ ∧
      ∀ T : ℝ, 0 ≤ T →
        a * residualBallLaplaceLIntegral p q h R T ≤
          residualGaussianLaplaceLIntegral p q h T ∧
        residualGaussianLaplaceLIntegral p q h T ≤
          A * residualBallLaplaceLIntegral p q h R T := by
  let a : ℝ≥∞0 := ENNReal.ofReal (Real.exp (-R ^ 2))
  let A : ℝ≥∞0 := residualGaussianBallComparisonConstant p q h R
  refine ⟨a, A, ?_, ?_, ?_, ?_, ?⟩_
  · dsimp [a]
    exact ENNReal.ofReal_pos.mpr (Real.exp_pos _)
  · dsimp [a]
    exact lt_top_iff_ne_top.mpr ENNReal.ofReal_ne_top
  · dsimp [A]
    exact residualGaussianBallComparisonConstant_pos p q h R
  · dsimp [A]
    exact residualGaussianBallComparisonConstant_lt_top p q h hR
  · intro T hT
    constructor
    · simpa [a] using residualBallLaplace_lower_gaussian p q h R T
    · simpa [A] using residualGaussianLaplace_upper_ball p q h hR hT

/-- The global comparison specializes nontrivially to the scalar residual
space. -/
lemma homogeneous_gaussianLaplace_comparable_ball_scalar_test
    {R : ℝ} (hR : 0 < R) :
    ∃ a A : ℝ≥∞0,
      0 < a ∧ a < ∞ ∧ 0 < A ∧ A < ∞ ∧
      ∀ T : ℝ, 0 ≤ T →
        a * residualBallLaplaceLIntegral 1 1 1 R T ≤
          residualGaussianLaplaceLIntegral 1 1 1 T ∧
        residualGaussianLaplaceLIntegral 1 1 1 T ≤
          A * residualBallLaplaceLIntegral 1 1 1 R T :=
  homogeneous_gaussianLaplace_comparable_ball_instantiated 1 1 1 hR

/-- Positivity of the radius is load-bearing in the shell partition: at radius
```

```
zero, the ball together with all shells does not cover the scalar residual
space. -/
lemma residualBall_shell_partition_fails_at_zero :
    residualQuadraticBall 1 1 1 0 ∪
        ⋃( n : ℕ, residualQuadraticShell 1 1 1 0 n) ≠ Set.univ := by
  intro hcover
  let w : ResidualSpace 1 1 1 := (1, 0)
  have hw : w ∈ residualQuadraticBall 1 1 1 0 ∪
      ⋃( n : ℕ, residualQuadraticShell 1 1 1 0 n) := by
    rw [hcover]
    exact Set.mem_univ w
  rcases hw with hwB | hwS
  · have hentry := hwB
    norm_num [w, residualQuadraticBall, residualAmbientNormSq,
      frobeniusSq] at hentry
  · simp only [Set.mem_iUnion] at hwS
    obtain ⟨n, ⟩hn := hwS
    have hout := residualQuadraticShell_outer hn
    norm_num [w, dyadicScale, residualAmbientNormSq, frobeniusSq] at hout

end O77.Residual

set_option autoImplicit false
noncomputable section

open Real MeasureTheory Set Filter
open scoped ENNReal BigOperators Pointwise Topology

namespace O77.Residual
namespace LaplacePowerLogBounds

/-- Two-sided comparison by fixed strictly positive finite `ENNReal` constants
transports a power-log order. The comparison need only hold for nonnegative
Laplace parameters, because the power-log predicate starts at a threshold at
least two. -/
theorem of_two_sided_comparison
    {J J' : ℝ → ℝ≥∞0} {lam : ℝ} {degree : ℕ}
    {a A : ℝ≥∞0}
    (ha0 : a ≠ 0) (hatop : a ≠ ∞)
    (hA0 : A ≠ 0) (hAtop : A ≠ ∞)
    (hcomp : ∀ T : ℝ, 0 ≤ T →
      a * J' T ≤ J T ∧ J T ≤ A * J' T)
    (hJ : O77.Residual.LaplacePowerLogBounds J lam degree) :
    O77.Residual.LaplacePowerLogBounds J' lam degree := by
  rcases hJ with ⟨c, C, T0, hc, hcC, hT0, ⟩hbounds
  let alpha : ℝ := a.toReal
  let Alpha : ℝ := A.toReal
  have halpha : 0 < alpha := ENNReal.toReal_pos ha0 hatop
  have hAlpha : 0 < Alpha := ENNReal.toReal_pos hA0 hAtop
  have hainv : ENNReal.ofReal ⁻alpha¹ = ⁻a¹ := by
    dsimp [alpha]
    rw [ENNReal.ofReal_inv_of_pos halpha, ENNReal.ofReal_toReal hatop]
  have hAinv : ENNReal.ofReal ⁻Alpha¹ = ⁻A¹ := by
    dsimp [Alpha]
    rw [ENNReal.ofReal_inv_of_pos hAlpha, ENNReal.ofReal_toReal hAtop]
  let c' : ℝ := ⁻Alpha¹ * c
  let Cbase : ℝ := ⁻alpha¹ * C
  let C' : ℝ := max c' Cbase
  have hc' : 0 < c' := by
    dsimp [c']
    exact mul_pos (inv_pos.mpr hAlpha) hc
  refine ⟨c', C', T0, hc', le_max_left _ _, hT0, ?⟩_
```

```
  intro T hT
  have hTnonneg : 0 ≤ T := by linarith
  have hb := hbounds T hT
  have hcmp := hcomp T hTnonneg
  have hc'coe : ENNReal.ofReal c' = ⁻A¹ * ENNReal.ofReal c := by
    dsimp [c']
    rw [ENNReal.ofReal_mul (inv_nonneg.mpr hAlpha.le), hAinv]
  have hCbasecoe : ENNReal.ofReal Cbase = ⁻a¹ * ENNReal.ofReal C := by
    dsimp [Cbase]
    rw [ENNReal.ofReal_mul (inv_nonneg.mpr halpha.le), hainv]
  constructor
  · calc
      ENNReal.ofReal c' * powerLogTop lam degree T =
          ⁻A¹ * (ENNReal.ofReal c * powerLogTop lam degree T) := by
        rw [hc'coe]
        ac_rfl
      _ ≤ ⁻A¹ * J T := mul_le_mul_of_nonneg_left hb.1 (by positivity)
      _ ≤ J' T := (ENNReal.inv_mul_le_iff hA0 hAtop).2 hcmp.2
  · have hJ' : J' T ≤ ⁻a¹ * J T :=
      (ENNReal.mul_le_iff_le_inv ha0 hatop).1 hcmp.1
    calc
      J' T ≤ ⁻a¹ * J T := hJ'
      _ ≤ ⁻a¹ * (ENNReal.ofReal C * powerLogTop lam degree T) :=
        mul_le_mul_of_nonneg_left hb.2 (by positivity)
      _ = ENNReal.ofReal Cbase * powerLogTop lam degree T := by
        rw [hCbasecoe]
        ac_rfl
      _ ≤ ENNReal.ofReal C' * powerLogTop lam degree T := by
        exact mul_le_mul_of_nonneg_right
          (ENNReal.ofReal_le_ofReal (by
            dsimp [C']
            exact le_max_right _ _)) (by positivity)

end LaplacePowerLogBounds

/-- Conditional on the precise real-Wishart input, every positive-radius
fixed residual quadratic ball has the same power-log Laplace order as the
Gaussian transform. -/
theorem residualBallLaplace_powerLog_bounds_of_wishart_instantiated
    (hW : O77.External.RealWishartEigenvalueFormula)
    {p q h : ℕ} (hp : 0 < p) (hq : 0 < q) (hh : 0 < h)
    {R : ℝ} (hR : 0 < R) :
    LaplacePowerLogBounds
      (residualBallLaplaceLIntegral p q h R)
      ((O77.residualMin p q h : ℝ) / 2)
      (O77.residualMultiplicity p q h - 1) := by
  have hG := gaussianLaplace_powerLog_bounds_of_wishart_instantiated
    hW hp hq hh
  rcases homogeneous_gaussianLaplace_comparable_ball_instantiated p q h hR with
    ⟨a, A, ha, hatop, hA, hAtop, ⟩hcomp
  exact LaplacePowerLogBounds.of_two_sided_comparison
    (ne_of_gt ha) (ne_of_lt hatop) (ne_of_gt hA) (ne_of_lt hAtop)
    hcomp hG

/-- Scalar positive control for the conditional fixed-ball transfer. -/
lemma residualBallLaplace_powerLog_bounds_scalar_test
    (hW : O77.External.RealWishartEigenvalueFormula)
    {R : ℝ} (hR : 0 < R) :
    LaplacePowerLogBounds
      (residualBallLaplaceLIntegral 1 1 1 R)
      (1 / 2 : ℝ) 1 := by
  have hmin : O77.residualMin 1 1 1 = 1 := by decide
```

```
  have hmult : O77.residualMultiplicity 1 1 1 = 2 := by decide
  simpa [hmin, hmult] using
    residualBallLaplace_powerLog_bounds_of_wishart_instantiated
      hW (p := 1) (q := 1) (h := 1) (by decide) (by decide)
        (by decide) hR

end O77.Residual

set_option autoImplicit false
noncomputable section

open Real MeasureTheory Set Filter
open scoped ENNReal BigOperators Pointwise Topology

namespace O77.Residual

lemma matrix_entry_sq_le_frobeniusSq
    {m n : ℕ} (A : Matrix (Fin m) (Fin n) ℝ) (i : Fin m) (j : Fin n) :
    (A i j) ^ 2 ≤ frobeniusSq A := by
  unfold frobeniusSq
  have hrow : (A i j) ^ 2 ≤ ∑ j' : Fin n, (A i j') ^ 2 :=
    Finset.single_le_sum (fun j' _ => sq_nonneg (A i j')) (Finset.mem_univ j)
  have htotal : ∑( j' : Fin n, (A i j') ^ 2) ≤
      ∑ i' : Fin m, ∑ j' : Fin n, (A i' j') ^ 2 :=
    Finset.single_le_sum
      (fun i' _ => Finset.sum_nonneg (fun j' _ => sq_nonneg (A i' j')))
      (Finset.mem_univ i)
  exact hrow.trans htotal

lemma frobeniusSq_nonneg_fin
    {m n : ℕ} (A : Matrix (Fin m) (Fin n) ℝ) :
    0 ≤ frobeniusSq A := by
  unfold frobeniusSq
  positivity

/-- The explicit residual quadratic ball is compact in the coordinate
function-space topology. This avoids choosing a matrix norm to prove
finiteness of its coordinate Lebesgue measure. -/
theorem residualQuadraticBall_isCompact
    (p q h : ℕ) (R : ℝ) :
    IsCompact (residualQuadraticBall p q h R) := by
  let I : Set ℝ := Set.Icc (-|R|) |R|
  let KY : Set (Matrix (Fin p) (Fin h) ℝ) :=
    {Y | ∀ i j, Y i j ∈ I}
  let KZ : Set (Matrix (Fin h) (Fin q) ℝ) :=
    {Z | ∀ i j, Z i j ∈ I}
  have hKY : IsCompact KY := by
    dsimp [KY]
    exact isCompact_pi_infinite (fun _ =>
      isCompact_pi_infinite (fun _ => isCompact_Icc))
  have hKZ : IsCompact KZ := by
    dsimp [KZ]
    exact isCompact_pi_infinite (fun _ =>
      isCompact_pi_infinite (fun _ => isCompact_Icc))
  have hK : IsCompact (KY ×ˢ KZ) := hKY.prod hKZ
  have hclosed : IsClosed (residualQuadraticBall p q h R) := by
    unfold residualQuadraticBall
    exact isClosed_le continuous_residualAmbientNormSq continuous_const
  apply hK.of_isClosed_subset hclosed
  intro w hw
  have hsum : residualAmbientNormSq w ≤ R ^ 2 := by
    simpa [residualQuadraticBall] using hw
```

```
  have hY : frobeniusSq w.1 ≤ R ^ 2 := by
    unfold residualAmbientNormSq at hsum
    nlinarith [frobeniusSq_nonneg_fin w.2]
  have hZ : frobeniusSq w.2 ≤ R ^ 2 := by
    unfold residualAmbientNormSq at hsum
    nlinarith [frobeniusSq_nonneg_fin w.1]
  rw [Set.mem_prod]
  constructor
  · dsimp [KY]
    intro i j
    have hsquare := (matrix_entry_sq_le_frobeniusSq w.1 i j).trans hY
    have habs : |w.1 i j| ≤ |R| := sq_le_sq.mp hsquare
    exact (abs_le.mp habs)
  · dsimp [KZ]
    intro i j
    have hsquare := (matrix_entry_sq_le_frobeniusSq w.2 i j).trans hZ
    have habs : |w.2 i j| ≤ |R| := sq_le_sq.mp hsquare
    exact (abs_le.mp habs)

lemma residualQuadraticBall_measure_lt_top
    (p q h : ℕ) (R : ℝ) :
    residualMatrixLebesgue p q h
        (residualQuadraticBall p q h R) < ∞ := by
  exact (residualQuadraticBall_isCompact p q h R).measure_lt_top

end O77.Residual

set_option autoImplicit false
noncomputable section

open Real MeasureTheory Set Filter
open scoped ENNReal BigOperators Pointwise Topology

namespace O77.Residual

/-- The finite measure obtained by restricting residual coordinate Lebesgue
measure to one explicit closed quadratic ball. -/
def residualQuadraticBallMeasure
    (p q h : ℕ) (R : ℝ) : Measure (ResidualSpace p q h) :=
  (residualMatrixLebesgue p q h).restrict
    (residualQuadraticBall p q h R)

noncomputable instance residualQuadraticBallMeasure.instIsFiniteMeasure
    (p q h : ℕ) (R : ℝ) :
    IsFiniteMeasure (residualQuadraticBallMeasure p q h R) := by
  refine ⟨?_⟩
  rw [residualQuadraticBallMeasure, Measure.restrict_apply_univ]
  exact residualQuadraticBall_measure_lt_top p q h R

lemma residualBallLaplaceLIntegral_eq_laplaceIntegral
    (p q h : ℕ) (R T : ℝ) :
    residualBallLaplaceLIntegral p q h R T =
      laplaceIntegral (residualQuadraticBallMeasure p q h R)
        residualKernel T := by
  rfl

/-- Conditional on the exact real-Wishart input, the strict residual-kernel
sublevel mass inside every positive-radius closed quadratic ball has the exact
O77 radius-fixed two-sided power-log order on closed balls. The open-ball
version follows by sandwiching an open ball between closed balls of radii
R/2 and R; no null-boundary assertion is needed. -/
theorem residualClosedBallSublevel_order_of_wishart_instantiated
```

```
    (hW : O77.External.RealWishartEigenvalueFormula)
    {p q h : ℕ} (hp : 0 < p) (hq : 0 < q) (hh : 0 < h)
    {R : ℝ} (hR : 0 < R) :
    EThetaZeroPowerLog
      (sublevelMass (residualQuadraticBallMeasure p q h R) residualKernel)
      ((O77.residualMin p q h : ℝ) / 2)
      (O77.residualMultiplicity p q h - 1) := by
  have hminNat := O77.Arithmetic.residualMin_positive p q h hp hq hh
  have hminReal : 0 < (O77.residualMin p q h : ℝ) := by
    exact_mod_cast hminNat
  have hlam : 0 < (O77.residualMin p q h : ℝ) / 2 := by
    linarith
  have hJball := residualBallLaplace_powerLog_bounds_of_wishart_instantiated
    hW hp hq hh hR
  have hJ : LaplacePowerLogBounds
      (laplaceIntegral (residualQuadraticBallMeasure p q h R) residualKernel)
      ((O77.residualMin p q h : ℝ) / 2)
      (O77.residualMultiplicity p q h - 1) := by
    apply LaplacePowerLogBounds.congr_of_eq_of_ge_two hJball
    intro T hT
    exact residualBallLaplaceLIntegral_eq_laplaceIntegral p q h R T
  exact volumeOrder_of_laplaceOrder
    (residualQuadraticBallMeasure p q h R)
    measurable_residualKernel residualKernel_nonneg hlam
    (O77.residualMultiplicity p q h - 1) hJ

/-- Scalar control: inside every positive-radius closed quadratic ball, the
sublevel mass of ‖`‖YZ_F²` has order `epsilon^(1/2) log(1/epsilon)`, conditional
on the Wishart input. -/
lemma residualClosedBallSublevel_order_scalar_test
    (hW : O77.External.RealWishartEigenvalueFormula)
    {R : ℝ} (hR : 0 < R) :
    EThetaZeroPowerLog
      (sublevelMass (residualQuadraticBallMeasure 1 1 1 R) residualKernel)
      (1 / 2 : ℝ) 1 := by
  have hmin : O77.residualMin 1 1 1 = 1 := by decide
  have hmult : O77.residualMultiplicity 1 1 1 = 2 := by decide
  simpa [hmin, hmult] using
    residualClosedBallSublevel_order_of_wishart_instantiated
      hW (p := 1) (q := 1) (h := 1)
        (by decide) (by decide) (by decide) hR

end O77.Residual

set_option autoImplicit false
noncomputable section

open Real MeasureTheory Set Filter
open scoped ENNReal BigOperators Pointwise Topology

namespace O77.Residual

/-- Open quadratic ball for the canonical Euclidean/Frobenius coordinates on
the residual matrix pair. -/
def residualOpenQuadraticBall
    (p q h : ℕ) (R : ℝ) : Set (ResidualSpace p q h) :=
  {w | residualAmbientNormSq w < R ^ 2}

lemma measurableSet_residualOpenQuadraticBall
    (p q h : ℕ) (R : ℝ) :
    MeasurableSet (residualOpenQuadraticBall p q h R) := by
  unfold residualOpenQuadraticBall
```

```
  exact measurableSet_lt measurable_residualAmbientNormSq measurable_const

/-- Residual coordinate Lebesgue measure restricted to the open quadratic
ball. -/
def residualOpenQuadraticBallMeasure
    (p q h : ℕ) (R : ℝ) : Measure (ResidualSpace p q h) :=
  (residualMatrixLebesgue p q h).restrict
    (residualOpenQuadraticBall p q h R)

lemma residualClosedHalfBall_subset_openBall
    (p q h : ℕ) {R : ℝ} (hR : 0 < R) :
    residualQuadraticBall p q h (R / 2) ⊆
      residualOpenQuadraticBall p q h R := by
  intro w hw
  have hw' : residualAmbientNormSq w ≤ (R / 2) ^ 2 := by
    simpa [residualQuadraticBall] using hw
  unfold residualOpenQuadraticBall
  have hrad : (R / 2) ^ 2 < R ^ 2 := by
    nlinarith [sq_pos_of_pos hR]
  exact hw'.trans_lt hrad

lemma residualOpenBall_subset_closedBall
    (p q h : ℕ) (R : ℝ) :
    residualOpenQuadraticBall p q h R ⊆
      residualQuadraticBall p q h R := by
  intro w hw
  change residualAmbientNormSq w < R ^ 2 at hw
  change residualAmbientNormSq w ≤ R ^ 2
  exact le_of_lt hw

lemma residualClosedHalfBall_sublevel_le_openBall
    (p q h : ℕ) {R : ℝ} (hR : 0 < R) (eps : ℝ) :
    sublevelMass (residualQuadraticBallMeasure p q h (R / 2))
        residualKernel eps ≤
      sublevelMass (residualOpenQuadraticBallMeasure p q h R)
        residualKernel eps := by
  unfold residualQuadraticBallMeasure residualOpenQuadraticBallMeasure
  exact (Measure.restrict_mono_set (residualMatrixLebesgue p q h)
    (residualClosedHalfBall_subset_openBall p q h hR)) _

lemma residualOpenBall_sublevel_le_closedBall
    (p q h : ℕ) (R eps : ℝ) :
    sublevelMass (residualOpenQuadraticBallMeasure p q h R)
        residualKernel eps ≤
      sublevelMass (residualQuadraticBallMeasure p q h R)
        residualKernel eps := by
  unfold residualQuadraticBallMeasure residualOpenQuadraticBallMeasure
  exact (Measure.restrict_mono_set (residualMatrixLebesgue p q h)
    (residualOpenBall_subset_closedBall p q h R)) _

/-- The strict residual-kernel sublevel volume in every positive-radius open
quadratic ball has the exact radius-fixed O77 power-log order, conditional
only on the explicit real-Wishart premise. The proof sandwiches the open ball
between closed balls of radii `R/2` and `R`; no null-boundary assertion is
needed. -/
theorem residualOpenBallSublevel_order_of_wishart_instantiated
    (hW : O77.External.RealWishartEigenvalueFormula)
    {p q h : ℕ} (hp : 0 < p) (hq : 0 < q) (hh : 0 < h)
    {R : ℝ} (hR : 0 < R) :
    EThetaZeroPowerLog
      (sublevelMass (residualOpenQuadraticBallMeasure p q h R) residualKernel)
      ((O77.residualMin p q h : ℝ) / 2)
```

```
      (O77.residualMultiplicity p q h - 1) := by
  have hRhalf : 0 < R / 2 := by linarith
  have hinner := residualClosedBallSublevel_order_of_wishart_instantiated
    hW hp hq hh hRhalf
  have houter := residualClosedBallSublevel_order_of_wishart_instantiated
    hW hp hq hh hR
  rcases hinner with ⟨ci, Ci, ei, hci, hciCi, hei, heiExp, ⟩hi
  rcases houter with ⟨co, Co, eo, hco, hcoCo, heo, heoExp, ⟩ho
  let c : ℝ := min ci Co
  let e : ℝ := min ei eo
  have hCo : 0 < Co := hco.trans_le hcoCo
  have hc : 0 < c := by
    dsimp [c]
    exact lt_min hci hCo
  have hcCo : c ≤ Co := by
    dsimp [c]
    exact min_le_right _ _
  have he : 0 < e := by
    dsimp [e]
    exact lt_min hei heo
  have heExp : e < Real.exp (-1) := by
    exact (min_le_left ei eo).trans_lt heiExp
  refine ⟨c, Co, e, hc, hcCo, he, heExp, ?⟩_
  intro eps heps hepsE
  have hepsi : eps < ei := hepsE.trans_le (min_le_left ei eo)
  have hepso : eps < eo := hepsE.trans_le (min_le_right ei eo)
  have hib := hi eps heps hepsi
  have hob := ho eps heps hepso
  constructor
  · calc
      ENNReal.ofReal c * powerLogZero
          ((O77.residualMin p q h : ℝ) / 2)
          (O77.residualMultiplicity p q h - 1) eps ≤
        ENNReal.ofReal ci * powerLogZero
          ((O77.residualMin p q h : ℝ) / 2)
          (O77.residualMultiplicity p q h - 1) eps := by
        exact mul_le_mul_of_nonneg_right
          (ENNReal.ofReal_le_ofReal (by
            dsimp [c]
            exact min_le_left _ _)) (by positivity)
      _ ≤ sublevelMass (residualQuadraticBallMeasure p q h (R / 2))
          residualKernel eps := hib.1
      _ ≤ sublevelMass (residualOpenQuadraticBallMeasure p q h R)
          residualKernel eps :=
        residualClosedHalfBall_sublevel_le_openBall p q h hR eps
  · calc
      sublevelMass (residualOpenQuadraticBallMeasure p q h R)
          residualKernel eps ≤
        sublevelMass (residualQuadraticBallMeasure p q h R)
          residualKernel eps :=
        residualOpenBall_sublevel_le_closedBall p q h R eps
      _ ≤ ENNReal.ofReal Co * powerLogZero
          ((O77.residualMin p q h : ℝ) / 2)
          (O77.residualMultiplicity p q h - 1) eps := hob.2

/-- Residual-specific origin band-pair predicate in the actual matrix and
coordinate-Lebesgue spaces. Its quantifier order is the public O77 order:
choose an admissible outer radius, then an arbitrary smaller radius, and only
then the comparison constants and epsilon threshold. -/
def HasResidualBandPairAtOrigin
    (p q h : ℕ) (lam : ℝ) (m : ℕ) : Prop :=
  0 ≤ lam ∧ 1 ≤ m ∧
```

```
    ∃ R0 : ℝ, 0 < R0 ∧
      ∀ R : ℝ, 0 < R → R < R0 →
        EThetaZeroPowerLog
          (sublevelMass (residualOpenQuadraticBallMeasure p q h R)
            residualKernel)
          lam (m - 1)

/-- Conditional residual volume theorem for positive residual dimensions. It
is instantiated in the actual matrix spaces and reduces the residual branch
to the single explicit real-Wishart change-of-variables premise. -/
theorem residualBandPair_of_wishart_instantiated
    (hW : O77.External.RealWishartEigenvalueFormula)
    {p q h : ℕ} (hp : 0 < p) (hq : 0 < q) (hh : 0 < h) :
    HasResidualBandPairAtOrigin p q h
      ((O77.residualMin p q h : ℝ) / 2)
      (O77.residualMultiplicity p q h) := by
  have hminNat := O77.Arithmetic.residualMin_positive p q h hp hq hh
  have hminReal : 0 < (O77.residualMin p q h : ℝ) := by
    exact_mod_cast hminNat
  refine ⟨by positivity,
    (O77.Arithmetic.residualMultiplicity_one_or_two p q h).1,
    1, by norm_num, ?_⟩
  intro R hR hRone
  exact residualOpenBallSublevel_order_of_wishart_instantiated
    hW hp hq hh hR

/-- Scalar positive control for the complete conditional residual origin
band-pair. -/
lemma residualBandPair_scalar_test
    (hW : O77.External.RealWishartEigenvalueFormula) :
    HasResidualBandPairAtOrigin 1 1 1 (1 / 2 : ℝ) 2 := by
  have hmin : O77.residualMin 1 1 1 = 1 := by decide
  have hmult : O77.residualMultiplicity 1 1 1 = 2 := by decide
  simpa [hmin, hmult] using
    residualBandPair_of_wishart_instantiated hW
      (p := 1) (q := 1) (h := 1) (by decide) (by decide) (by decide)

end O77.Residual

set_option autoImplicit false
noncomputable section

open Real MeasureTheory Set Filter
open scoped ENNReal BigOperators Pointwise Topology

namespace O77.Residual

lemma isOpen_residualOpenQuadraticBall
    (p q h : ℕ) (R : ℝ) :
    IsOpen (residualOpenQuadraticBall p q h R) := by
  unfold residualOpenQuadraticBall
  exact isOpen_lt continuous_residualAmbientNormSq continuous_const

lemma zero_mem_residualOpenQuadraticBall
    (p q h : ℕ) {R : ℝ} (hR : 0 < R) :
    (0 : ResidualSpace p q h) ∈ residualOpenQuadraticBall p q h R := by
  change residualAmbientNormSq (0 : ResidualSpace p q h) < R ^ 2
  simp [residualAmbientNormSq, frobeniusSq]
  positivity

lemma residualOpenQuadraticBall_measure_pos
    (p q h : ℕ) {R : ℝ} (hR : 0 < R) :
```

```
    0 < residualMatrixLebesgue p q h
      (residualOpenQuadraticBall p q h R) := by
  exact (isOpen_residualOpenQuadraticBall p q h R).measure_pos
    (residualMatrixLebesgue p q h)
    ⟨0, zero_mem_residualOpenQuadraticBall p q h ⟩hR

lemma residualOpenQuadraticBall_measure_lt_top
    (p q h : ℕ) (R : ℝ) :
    residualMatrixLebesgue p q h
      (residualOpenQuadraticBall p q h R) < ∞ := by
  exact (measure_mono (residualOpenBall_subset_closedBall p q h R)).trans_lt
    (residualQuadraticBall_measure_lt_top p q h R)

lemma residualKernel_eq_zero_of_zero_dimension
    {p q h : ℕ} (hz : p = 0 ∨ q = 0 ∨ h = 0)
    (w : ResidualSpace p q h) :
    residualKernel w = 0 := by
  rcases hz with rfl | rfl | rfl <;>
    simp [residualKernel, frobeniusSq, Matrix.mul_apply]

lemma powerLogZero_zero_zero (eps : ℝ) :
    powerLogZero 0 0 eps = 1 := by
  simp [powerLogZero, powerLogZeroReal]

/-- If one residual factor has zero dimension, the residual kernel vanishes
identically and the radius-first residual band pair is the neutral pair `(0,1)`.
This is proved directly from the finite positive volume of every positive-radius
open ball, not by invoking the positive-exponent Laguerre theorem. -/
theorem residualBandPair_neutral_of_zero_dimension
    {p q h : ℕ} (hz : p = 0 ∨ q = 0 ∨ h = 0) :
    HasResidualBandPairAtOrigin p q h 0 1 := by
  refine ⟨by norm_num, by norm_num, 1, by norm_num, ?⟩_
  intro R hR hRone
  let M : ℝ≥∞0 := residualMatrixLebesgue p q h
    (residualOpenQuadraticBall p q h R)
  have hM0 : M ≠ 0 := by
    exact ne_of_gt (residualOpenQuadraticBall_measure_pos p q h hR)
  have hMtop : M ≠ ∞ := by
    exact ne_of_lt (residualOpenQuadraticBall_measure_lt_top p q h R)
  let c : ℝ := M.toReal
  have hc : 0 < c := ENNReal.toReal_pos hM0 hMtop
  have hMcoe : ENNReal.ofReal c = M := by
    exact ENNReal.ofReal_toReal hMtop
  let eps0 : ℝ := Real.exp (-1) / 2
  have heps0 : 0 < eps0 := by
    dsimp [eps0]
    positivity
  have heps0Exp : eps0 < Real.exp (-1) := by
    dsimp [eps0]
    have hexp := Real.exp_pos (-1)
    linarith
  refine ⟨c, c, eps0, hc, le_rfl, heps0, heps0Exp, ?⟩_
  intro eps heps heps0'
  have hset : {w : ResidualSpace p q h | residualKernel w < eps} = Set.univ := by
    ext w
    simp [residualKernel_eq_zero_of_zero_dimension hz w, heps]
  have hV :
      sublevelMass (residualOpenQuadraticBallMeasure p q h R)
          residualKernel eps = M := by
    unfold sublevelMass residualOpenQuadraticBallMeasure
    rw [Measure.restrict_apply
      (measurableSet_sublevel measurable_residualKernel eps), hset]
```

```
    simp [M]
  have hgauge : powerLogZero 0 0 eps = 1 :=
    powerLogZero_zero_zero eps
  constructor <;> rw [hgauge, mul_one, hV, hMcoe]

/-- Complete residual origin theorem in all dimensions, conditional only on the
explicit real-Wishart adapter in the positive-dimensional branch. The external
premise is unused when a residual factor is absent. -/
theorem residualBandPair_all_dimensions_of_wishart_instantiated
    (hW : O77.External.RealWishartEigenvalueFormula)
    (p q h : ℕ) :
    HasResidualBandPairAtOrigin p q h
      ((O77.residualMin p q h : ℝ) / 2)
      (O77.residualMultiplicity p q h) := by
  by_cases hp0 : p = 0
  · have hz : p = 0 ∨ q = 0 ∨ h = 0 := Or.inl hp0
    have hmin := O77.Arithmetic.residualMin_zero p q h hz
    have hmult := O77.Arithmetic.residualMultiplicity_zero p q h hz
    simpa [hmin, hmult] using
      (residualBandPair_neutral_of_zero_dimension hz)
  by_cases hq0 : q = 0
  · have hz : p = 0 ∨ q = 0 ∨ h = 0 := Or.inr (Or.inl hq0)
    have hmin := O77.Arithmetic.residualMin_zero p q h hz
    have hmult := O77.Arithmetic.residualMultiplicity_zero p q h hz
    simpa [hmin, hmult] using
      (residualBandPair_neutral_of_zero_dimension hz)
  by_cases hh0 : h = 0
  · have hz : p = 0 ∨ q = 0 ∨ h = 0 := Or.inr (Or.inr hh0)
    have hmin := O77.Arithmetic.residualMin_zero p q h hz
    have hmult := O77.Arithmetic.residualMultiplicity_zero p q h hz
    simpa [hmin, hmult] using
      (residualBandPair_neutral_of_zero_dimension hz)
  · exact residualBandPair_of_wishart_instantiated hW
      (Nat.pos_of_ne_zero hp0) (Nat.pos_of_ne_zero hq0)
      (Nat.pos_of_ne_zero hh0)

/-- Empty-factor control for the neutral residual branch. -/
lemma residualBandPair_zero_output_test :
    HasResidualBandPairAtOrigin 0 3 2 0 1 := by
  exact residualBandPair_neutral_of_zero_dimension (Or.inl rfl)

end O77.Residual
```

Corresponding Lean declaration(s): O77.Residual.dyadicInnerScale_mul_sq,O77.Residual.dyadicScale_mul_sq ,O77.Residual.exists_mem_residualQuadraticShell_of_not_mem_ball,O77.Residual.residualQuadraticBall_sub set_dyadicInnerBall,O77.Residual.iUnion_residualQuadraticShell_eq_compl_ball,O77.Residual.residualBall _union_iUnion_shells,O77.Residual.residualQuadraticShell_pairwiseDisjoint,O77.Residual.residualShellCoef ficientReal,O77.Residual.residualShellCoefficientReal_pos,O77.Residual.residualShellCoefficient_eq_ofR eal,O77.Residual.residualShellRatioReal,O77.Residual.residualShellCoefficientReal_succ,O77.Residual.re sidualShellCoefficientReal_ratio,O77.Residual.tendsto_dyadic_radius_sq_atTop,O77.Residual.residualShel lRatioReal_tendsto_zero,O77.Residual.gaussian_shell_series_summable_real,O77.Residual.gaussian_shell_s eries_tsum_lt_top.

Corresponding Lean declaration(s): O77.Residual.residualGaussianLaplaceLIntegral_eq_ball_add_shells,O77 .Residual.residualGaussianShells_tsum_le,O77.Residual.residualGaussianBallComparisonConstant,O77.Resid ual.residualGaussianBallComparisonConstant_pos,O77.Residual.residualGaussianBallComparisonConstant_lt_ top,O77.Residual.residualGaussianLaplace_upper_ball,O77.Residual.homogeneous_gaussianLaplace_comparable_

ball_instantiated,O77.Residual.homogeneous_gaussianLaplace_comparable_ball_scalar_test,O77.Residual.residualBall_shell_partition_fails_at_zero.

Corresponding Lean declaration(s): O77.Residual.LaplacePowerLogBounds.of_two_sided_comparison,O77.Residual.residualBallLaplace_powerLog_bounds_of_wishart_instantiated,O77.Residual.residualBallLaplace_powerLog_bounds_scalar_test.

Corresponding Lean declaration(s): O77.Residual.matrix_entry_sq_le_frobeniusSq,O77.Residual.frobeniusSq_nonneg_fin,O77.Residual.residualQuadraticBall_isCompact,O77.Residual.residualQuadraticBall_measure_lt_top,O77.Residual.residualQuadraticBallMeasure,O77.Residual.residualQuadraticBallMeasure.instIsFiniteMeasure,O77.Residual.residualBallLaplaceLIntegral_eq_laplaceIntegral,O77.Residual.residualClosedBallSublevel_order_of_wishart_instantiated,O77.Residual.residualClosedBallSublevel_order_scalar_test.

Corresponding Lean declaration(s): O77.Residual.residualOpenQuadraticBall,O77.Residual.measurableSet_residualOpenQuadraticBall,O77.Residual.residualOpenQuadraticBallMeasure,O77.Residual.residualClosedHalfBall_subset_openBall,O77.Residual.residualOpenBall_subset_closedBall,O77.Residual.residualClosedHalfBall_sublevel_le_openBall,O77.Residual.residualOpenBall_sublevel_le_closedBall,O77.Residual.residualOpenBallSublevel_order_of_wishart_instantiated,O77.Residual.HasResidualBandPairAtOrigin,O77.Residual.residualBandPair_of_wishart_instantiated,O77.Residual.residualBandPair_scalar_test.

Corresponding Lean declaration(s): O77.Residual.isOpen_residualOpenQuadraticBall,O77.Residual.zero_mem_residualOpenQuadraticBall,O77.Residual.residualOpenQuadraticBall_measure_pos,O77.Residual.residualOpenQuadraticBall_measure_lt_top,O77.Residual.residualKernel_eq_zero_of_zero_dimension,O77.Residual.powerLogZero_zero_zero,O77.Residual.residualBandPair_neutral_of_zero_dimension,O77.Residual.residualBandPair_all_dimensions_of_wishart_instantiated,O77.Residual.residualBandPair_zero_output_test.

29 Module `O77AdaptedSections`

29.1 Actual images, kernels, and compatible sections

Let $V \xrightarrow{A} W \xrightarrow{B} U$ be real linear maps. In this section $R = \operatorname{im} A$, $S = \operatorname{im} B$, and $K = R \cap \ker B$ always mean actual subspaces. Finite dimensionality of W is imposed only for the dimension statements; the section constructions use the general splitting theory of vector spaces. The first two identifications are

$$\operatorname{im}(B|_R) = \operatorname{im}(BA), \qquad \ker(B|_R) \simeq R \cap \ker B.$$

For the image equality, a vector in R is written Av and the witness is substituted. The kernel equivalence simply removes or adds the nested subtype wrappers; its underlying vector is unchanged. Applying `LinearMap.finrank_range_add_finrank_ker` to $B|_R$ gives $\dim K + \operatorname{rank}(BA) = \operatorname{rank} A$. Since $K \subseteq \ker B$, the same theorem for B and `Submodule.finrank_mono` imply $\operatorname{rank} A + \operatorname{rank} B \le \dim W + \operatorname{rank}(BA)$. Thus the rank inequality precedes, and does not assume, the normal form.

```
import Mathlib
import O77

set_option autoImplicit false
noncomputable section
open Set Module
namespace O77.Adapted0906
```

```
section LinearMaps
variable {V W U : Type*} [AddCommGroup V] [Module ℝ V]
  [AddCommGroup W] [Module ℝ W] [AddCommGroup U] [Module ℝ U]

/-- The restriction uses actual images of maps, not assigned rank labels. -/
lemma range_on_image (A : V →ₗ[ℝ] W) (B : W →ₗ[ℝ] U) :
    LinearMap.range (B.domRestrict (LinearMap.range A)) = LinearMap.range (B.comp A) := by
  ext u
  constructor
  · rintro ⟨w, ⟩hw
    obtain ⟨v, ⟩hv := w.property
    refine ⟨v, ?⟩_
    change B (A v) = u
    rw [hv]
    exact hw
  · rintro ⟨v, ⟩hv
    exact ⟨⟨A v, ⟨v, ⟩⟩rfl, ⟩hv

def kernel_on_image (A : V →ₗ[ℝ] W) (B : W →ₗ[ℝ] U) :
    LinearMap.ker (B.domRestrict (LinearMap.range A)) ≃ₗ[ℝ]
      (LinearMap.range A ⊓ LinearMap.ker B : Submodule ℝ W) where
  toFun x := ⟨x.1.1, x.1.2, x⟩.2
  invFun x := ⟨⟨x.1, x⟩.2.1, x⟩.2.2
  left_inv _ := rfl
  right_inv _ := rfl
  map_add' _ _ := rfl
  map_smul' _ _ := rfl

/-- The image/kernel dimension identity needed before choosing any basis. -/
theorem image_kernel_finrank [FiniteDimensional ℝ W]
    (A : V →ₗ[ℝ] W) (B : W →ₗ[ℝ] U) :
    finrank ℝ (LinearMap.range A ⊓ LinearMap.ker B : Submodule ℝ W) +
      finrank ℝ (LinearMap.range (B.comp A)) = finrank ℝ (LinearMap.range A) := by
  have hn := (B.domRestrict (LinearMap.range A)).finrank_range_add_finrank_ker
  rw [range_on_image, (kernel_on_image A B).finrank_eq] at hn
  simpa only [Nat.add_comm] using hn

/-- Sylvester's inequality is derived from the same actual restriction. -/
theorem rank_sum_bound [FiniteDimensional ℝ W]
    (A : V →ₗ[ℝ] W) (B : W →ₗ[ℝ] U) :
    finrank ℝ (LinearMap.range A) + finrank ℝ (LinearMap.range B) ≤
      finrank ℝ W + finrank ℝ (LinearMap.range (B.comp A)) := by
  have h1 := image_kernel_finrank A B
  have h2 := B.finrank_range_add_finrank_ker
  have h3 := Submodule.finrank_mono
    (show LinearMap.range A ⊓ LinearMap.ker B ≤ LinearMap.ker B from inf_le_right)
  omega
```

Extending a prescribed section. Suppose $T : W \to U$ is onto and $s_0 : C \to W$ satisfies $Ts_0 = \iota_C$. Choose a complement to C, let $\pi : U \to C$ be its projection, and choose any right inverse t to T. Define

$$s = s_0\pi + t(1 - \iota_C\pi).$$

Then $Ts = \iota_C\pi + (1 - \iota_C\pi) = 1$, while $s\iota_C = s_0$ because $\pi\iota_C = 1$. These are the two conclusions of `section_extending_subspace`; the first is surjectivity-compatible, the second preserves the already chosen values. The actual library inputs are `exists_isCompl`, `Submodule.projectionOnto_apply_left`, and `LinearMap.exists_rightInverse_of_surjective`.

```
/-- Extend a section on a subspace without changing any of its values.
```

```
Mathlib supplies the complement, projection and unrestricted right inverse.
The only new step is the correction s0*pi + t*(id-inclusion*pi). -/
theorem section_extending_subspace (T : W →ₗ[ℝ] U)
    (hT : LinearMap.range T = ⊤) (C : Submodule ℝ U)
    (s0 : C →ₗ[ℝ] W) (h0 : T.comp s0 = C.subtype) :
    ∃ s : U →ₗ[ℝ] W, T.comp s = LinearMap.id ∧ s.comp C.subtype = s0 := by
  classical
  obtain ⟨D, ⟩hD := exists_isCompl C
  let pi := C.projectionOnto D hD
  obtain ⟨t, ⟩ht := T.exists_rightInverse_of_surjective hT
  let s : U →ₗ[ℝ] W := s0.comp pi + t.comp (LinearMap.id - C.subtype.comp pi)
  have h0v (c : C) : T (s0 c) = (c : U) := LinearMap.congr_fun h0 c
  have htv (u : U) : T (t u) = u := LinearMap.congr_fun ht u
  refine ⟨s, ?_, ?⟩_
  · ext u
    change T (s0 (pi u) + t (u-(pi u : U))) = u
    rw [map_add, h0v, htv]
    abel
  · ext c
    change s0 (pi (c : U)) + t ((c : U)-(pi (c : U) : U)) = s0 c
    have hp : pi (c : U) = c := Submodule.projectionOnto_apply_left hD c
    rw [hp, sub_self, map_zero, add_zero]
```

The compatibility needed by both arrows. Apply the preceding construction to $B : W \to S$ and the subspace $C = \mathrm{im}(B|_R) \subseteq S$. First choose a section $C \to R$ of $B|_R$; then extend its composite $C \to R \hookrightarrow W$ to a section $s : S \to W$. Consequently $s(Bx) \in R$ whenever $x \in R$. For such x, both x and $s(Bx)$ lie in R, and $B(x - s(Bx)) = Bx - Bx = 0$. Hence

$$x - s(Bx) \in R \cap \ker B.$$

This containment is the reason the input and hidden decompositions can share the same summand. Independently chosen unrelated complements would not give this conclusion automatically.

```
/-- A section of B onto its image that sends B(image A) back into image A.
No normal-form witness or dimension label is assumed. This compatibility is
what an arbitrary independently chosen section need not satisfy. -/
theorem compatible_image_section (A : V →ₗ[ℝ] W) (B : W →ₗ[ℝ] U) :
    ∃ s : LinearMap.range B →ₗ[ℝ] W,
      B.rangeRestrict.comp s = LinearMap.id ∧
      ∀ x : LinearMap.range A, s (B.rangeRestrict (x : W)) ∈ LinearMap.range A := by
  let F : LinearMap.range A →ₗ[ℝ] LinearMap.range B :=
    B.rangeRestrict.domRestrict (LinearMap.range A)
  let C := LinearMap.range F
  obtain ⟨t, ⟩ht := F.rangeRestrict.exists_rightInverse_of_surjective F.range_rangeRestrict
  let s0 : C →ₗ[ℝ] W := (LinearMap.range A).subtype.comp t
  have h0 : B.rangeRestrict.comp s0 = C.subtype := by
    apply LinearMap.ext; intro c
    exact congrArg Subtype.val (LinearMap.congr_fun ht c)
  obtain ⟨s, hs, ⟩hsC := section_extending_subspace
    B.rangeRestrict B.range_rangeRestrict C s0 h0
  refine ⟨s, hs, ?⟩_
  intro x
  let c : C := ⟨F x, ⟨x, ⟩⟩rfl
  have hv : s (B.rangeRestrict (x : W)) = s0 c := LinearMap.congr_fun hsC c
  rw [hv]
  exact (t c).property

/-- With the compatible section, the kernel remainder stays inside image A.
This is the key containment behind the simultaneous decomposition of the pair. -/
```

```
theorem compatible_kernel_remainder (A : V →ₗ[ℝ] W) (B : W →ₗ[ℝ] U)
    (s : LinearMap.range B →ₗ[ℝ] W)
    (hs : B.rangeRestrict.comp s = LinearMap.id)
    (hAs : ∀ x : LinearMap.range A, s (B.rangeRestrict (x : W)) ∈ LinearMap.range A)
    (x : LinearMap.range A) :
    (x : W)-s (B.rangeRestrict (x : W)) ∈ LinearMap.range A ⊓ LinearMap.ker B := by
  refine ⟨(LinearMap.range A).sub_mem x.property (hAs x), ?_⟩
  change B ((x : W)-s (B.rangeRestrict (x : W))) = 0
  have he : B (s (B.rangeRestrict (x : W))) = B (x : W) :=
    congrArg Subtype.val (LinearMap.congr_fun hs (B.rangeRestrict (x : W)))
  rw [map_sub, he, sub_self]
end LinearMaps
```

Translation to matrix rank. For $A \in \mathbb{R}^{H\times I}$ and $B \in \mathbb{R}^{O\times H}$, use their actual `Matrix.mulVecLin` maps. The equality `Matrix.mulVecLin_mul` identifies the composite and $\dim(\mathrm{Fin}(H) \to \mathbb{R}) = H$ fixes the middle dimension. Together with the library inequalities for the rank of a product and rank bounded by matrix width or image dimension, this proves all seven fields of `O77.RankData`: $r \le a, b$, $a \le I, H$, $b \le O, H$, and $a + b \le H + r$, with $(r, a, b) = (\operatorname{rank} BA, \operatorname{rank} A, \operatorname{rank} B)$. At an exact fit, $BA = \Phi$ is used only to replace the composite rank by $\operatorname{rank}\Phi$. The final examples include the zero-dimensional case; no nonempty-index assumption entered the argument.

```
/-- Numerical feasibility obtained from the actual Mathlib matrix ranks.
This is necessity, not the separate realization of every feasible rank pair. -/
theorem actual_rankData {I O H : ℕ} (A : Matrix (Fin H) (Fin I) ℝ) (B : Matrix (Fin O) (Fin H) ℝ) :
    O77.RankData I O H (B*A).rank A.rank B.rank := by
  have hs := rank_sum_bound A.mulVecLin B.mulVecLin
  have hdim : finrank ℝ (Fin H → ℝ) = H := by simp
  rw ←[ Matrix.mulVecLin_mul, hdim] at hs
  refine ⟨Matrix.rank_mul_le_right B A, Matrix.rank_mul_le_left B A,
    Matrix.rank_le_width A, ?_, ?_, Matrix.rank_le_width B, ⟩hs
  · simpa [Matrix.rank] using (LinearMap.range A.mulVecLin).finrank_le
  · simpa [Matrix.rank] using (LinearMap.range B.mulVecLin).finrank_le

theorem rankData_of_exact_fit {I O H : ℕ} {Phi : Matrix (Fin O) (Fin I) ℝ} {A : Matrix (Fin H) (Fin I)
     ℝ} {B : Matrix (Fin O) (Fin H) ℝ}
    (hw : B*A = Phi) : O77.RankData I O H Phi.rank A.rank B.rank := by
  simpa only [hw] using actual_rankData A B

-- Regression statements.
example {I O H : ℕ} (A : Matrix (Fin H) (Fin I) ℝ) (B : Matrix (Fin O) (Fin H) ℝ) :
    A.rank + B.rank ≤ H + (B*A).rank := (actual_rankData A B).sum_le

example (A : Matrix (Fin 0) (Fin 0) ℝ) (B : Matrix (Fin 0) (Fin 0) ℝ) : O77.RankData 0 0 0 (B*A).rank
     A.rank B.rank :=
  actual_rankData A B


end O77.Adapted0906
end
```

30 Module `O77NumberedCoordinates`

30.1 One decomposition for the two-arrow diagram

All spaces in this module are real vector spaces; no norm, measure, or loss is involved. For a map $f : V \to W$ and a section $s : \operatorname{im} f \to V$, the map

$$\operatorname{im} f \times \ker f \longrightarrow V, \qquad (x, z) \longmapsto s(x) + z$$

is a linear equivalence. Its inverse is $v \mapsto (f(v), v - s(f(v)))$, where $f(v)$ is regarded as a vector in $\operatorname{im} f$. The implementation `sectionProd` uses a projection onto $\operatorname{im} s$, proves that its kernel is $\ker f$, and applies the library complementary-submodule equivalence `Submodule.prodEquivOfIsCompl`. The following evaluation lemmas retain the crucial equation $f(s(x) + z) = x$, rather than merely the dimensions.

```
import Mathlib
import O77AdaptedSections

/-! Simultaneous coordinates for two composable real linear maps.
No norm, measure, loss, or candidate local invariant occurs in this module. -/
set_option autoImplicit false
noncomputable section
open Module
namespace O77.NumberedCoordinates

section Splitting
variable {V W U : Type*}
  [AddCommGroup V] [Module ℝ V] [AddCommGroup W] [Module ℝ W]
  [AddCommGroup U] [Module ℝ U]

/-- A range section gives coordinates with the prescribed section as the
first summand. The complement and inverse come from Mathlib's projection
and complementary-submodule equivalences, not a second splitting library. -/
def sectionProd (f : V →ₗ[ℝ] W) (s : LinearMap.range f →ₗ[ℝ] V)
    (hs : f.rangeRestrict.comp s = LinearMap.id) :
    (LinearMap.range f × LinearMap.ker f) ≃ₗ[ℝ] V := by
  have hleft : Function.LeftInverse f.rangeRestrict s :=
    fun x => LinearMap.congr_fun hs x
  let e := LinearEquiv.ofInjective s hleft.injective
  let p : V →ₗ[ℝ] LinearMap.range s := e.toLinearMap.comp f.rangeRestrict
  have hp (x : LinearMap.range s) : p (x : V) = x := by
    obtain ⟨y, ⟩rfl := e.surjective x
    change e (f.rangeRestrict (s y)) = e y
    rw [hleft]
  have hker : LinearMap.ker p = LinearMap.ker f := by
    ext x
    change e (f.rangeRestrict x) = 0 ↔ f x = 0
    rw [e.map_eq_zero_iff]
    exact Subtype.ext_iff
  have hc : IsCompl (LinearMap.range s) (LinearMap.ker f) := by
    rw ←[ hker]
    exact LinearMap.isCompl_of_proj hp
  exact (e.prodCongr (LinearEquiv.refl ℝ (LinearMap.ker f))).trans
    ((LinearMap.range s).prodEquivOfIsCompl (LinearMap.ker f) hc)

@[simp] lemma sectionProd_apply (f : V →ₗ[ℝ] W)
    (s : LinearMap.range f →ₗ[ℝ] V)
    (hs : f.rangeRestrict.comp s = LinearMap.id)
    (x : LinearMap.range f) (z : LinearMap.ker f) :
    sectionProd f s hs (x, z) = s x + (z : V) := rfl

/-- Unlike an arbitrary direct-sum isomorphism, this one remembers f. -/
```

```
@[simp] lemma sectionProd_map (f : V →ₗ[ℝ] W)
    (s : LinearMap.range f →ₗ[ℝ] V)
    (hs : f.rangeRestrict.comp s = LinearMap.id)
    (x : LinearMap.range f) (z : LinearMap.ker f) :
    f (sectionProd f s hs (x, z)) = (x : W) := by
  rw [sectionProd_apply, map_add]
  have hx := congrArg Subtype.val (LinearMap.congr_fun hs x)
  change f (s x) = (x : W) at hx
  rw [hx, show f z = 0 from z.property, add_zero]
```

The subspaces shared by the diagram. Put $R = \operatorname{im} A$, $S = \operatorname{im} B$, and $F = B|_R : R \to S$. Write $C = \operatorname{im} F$ and $K = \ker F$. The equivalence `compositeRangeEquiv` identifies C with $\operatorname{im}(BA)$ by retaining the underlying vector in U; `kernelEmbedding` embeds K in $\ker B$ by retaining its vector in W. Both proofs explicitly preserve the relevant image or kernel witnesses. The compatible section $s_B : S \to W$ from the preceding module restricts to $s_C : C \to R$: its membership proof is precisely the compatibility property, and $Fs_C = 1$ follows from $Bs_B = 1$. Thus the section used to split R is the restriction of the section used to split W.

```
/-- The middle map, with both its domain and codomain restricted to images. -/
def imageMap (A : V →ₗ[ℝ] W) (B : W →ₗ[ℝ] U) :
    LinearMap.range A →ₗ[ℝ] LinearMap.range B :=
  B.rangeRestrict.domRestrict (LinearMap.range A)

/-- Removing the two subtype wrappers identifies the actual composite image.
The two inverses only repackage witnesses; they do not choose a new basis. -/
def compositeRangeEquiv (A : V →ₗ[ℝ] W) (B : W →ₗ[ℝ] U) :
    LinearMap.range (imageMap A B) ≃ₗ[ℝ] LinearMap.range (B.comp A) where
  toFun c := ⟨c.1.1, by
    obtain ⟨x, ⟩hx := c.2
    obtain ⟨v, ⟩hv := x.2
    refine ⟨v, ?⟩_
    change B (A v) = c.1.1
    rw [hv]
    exact congrArg Subtype.val ⟩hx
  invFun c := ⟨⟨c.1, by
    obtain ⟨v, ⟩hv := c.2
    exact ⟨A v, ⟩⟩hv, by
    obtain ⟨v, ⟩hv := c.2
    refine ⟨⟨A v, ⟨v, ⟩⟩rfl, ?⟩_
    exact Subtype.ext ⟩hv
  left_inv _ := by apply Subtype.ext; apply Subtype.ext; rfl
  right_inv _ := by apply Subtype.ext; rfl
  map_add' _ _ := by apply Subtype.ext; rfl
  map_smul' _ _ := by apply Subtype.ext; rfl

/-- The kernel of the image restriction, embedded in the full kernel of B. -/
def kernelEmbedding (A : V →ₗ[ℝ] W) (B : W →ₗ[ℝ] U) :
    LinearMap.ker (imageMap A B) →ₗ[ℝ] LinearMap.ker B where
  toFun x := ⟨x.1.1, congrArg Subtype.val x).2
  map_add' _ _ := rfl
  map_smul' _ _ := rfl

lemma kernelEmbedding_injective (A : V →ₗ[ℝ] W) (B : W →ₗ[ℝ] U) :
    Function.Injective (kernelEmbedding A B) := by
  intro x y h
  apply Subtype.ext
  apply Subtype.ext
  exact congrArg (fun z : LinearMap.ker B => (z : W)) h

/-- The compatible section already constructed in O77AdaptedSections
```

```
restricts to a section of the image-to-image map. -/
def restrictedSection (A : V →ₗ[ℝ] W) (B : W →ₗ[ℝ] U)
    (s : LinearMap.range B →ₗ[ℝ] W)
    (hAs : ∀ x : LinearMap.range A,
      s (B.rangeRestrict (x : W)) ∈ LinearMap.range A) :
    LinearMap.range (imageMap A B) →ₗ[ℝ] LinearMap.range A where
  toFun c := ⟨s c.1, by
    obtain ⟨x, ⟩hx := c.2
    rw ←[ hx]
    exact hAs ⟩x
  map_add' x y := Subtype.ext (s.map_add x.1 y.1)
  map_smul' a x := Subtype.ext (s.map_smul a x.1)

lemma restrictedSection_rightInverse (A : V →ₗ[ℝ] W) (B : W →ₗ[ℝ] U)
    (s : LinearMap.range B →ₗ[ℝ] W)
    (hs : B.rangeRestrict.comp s = LinearMap.id)
    (hAs : ∀ x : LinearMap.range A,
      s (B.rangeRestrict (x : W)) ∈ LinearMap.range A) :
    (imageMap A B).rangeRestrict.comp (restrictedSection A B s hAs) =
      LinearMap.id := by
  apply LinearMap.ext
  intro c
  apply Subtype.ext
  exact LinearMap.congr_fun hs c.1

end Splitting
```

What the coordinate structure asserts. Write $\mathbb{R}^n = \mathrm{Fin}(n) \to \mathbb{R}$. For six natural numbers $r, \alpha, \beta, h, p, q$, the coordinate spaces are

$$V_0 = (\mathbb{R}^r \times \mathbb{R}^\alpha) \times \mathbb{R}^q, \quad W_0 = (\mathbb{R}^r \times \mathbb{R}^\alpha) \times (\mathbb{R}^\beta \times \mathbb{R}^h), \quad U_0 = (\mathbb{R}^r \times \mathbb{R}^\beta) \times \mathbb{R}^p.$$

A `PairCoordinates` consists of three actual linear equivalences to V, W, U and two commuting-square equations. In these coordinates $A(c,k,q) = ((c,k),(0,0))$ and $B((c,k),(d,z)) = ((c,d),0)$. Therefore $BA(c,k,q) = ((c,0),0)$. The structure stores neither an asserted learning coefficient nor assigned rank values. Its dimension equations follow from the three equivalences using `LinearEquiv.finrank_eq` and the dimension of a product.

```
abbrev Vec (n : ℕ) := Fin n → ℝ
abbrev InputBlock (r alpha q : ℕ) := (Vec r × Vec alpha) × Vec q
abbrev HiddenBlock (r alpha beta h : ℕ) :=
  (Vec r × Vec alpha) × (Vec beta × Vec h)
abbrev OutputBlock (r beta p : ℕ) := (Vec r × Vec beta) × Vec p

section Coordinates
variable {V W U : Type*}
  [AddCommGroup V] [Module ℝ V] [AddCommGroup W] [Module ℝ W]
  [AddCommGroup U] [Module ℝ U]

/-- An isomorphism of the actual two-arrow diagram with the displayed
identity/zero-block diagram. Only coordinate maps and their commuting
squares are data; no rank formula or measured conclusion is a field. -/
structure PairCoordinates (A : V →ₗ[ℝ] W) (B : W →ₗ[ℝ] U)
    (r alpha beta h p q : ℕ) where
  input : InputBlock r alpha q ≃ₗ[ℝ] V
  hidden : HiddenBlock r alpha beta h ≃ₗ[ℝ] W
  output : OutputBlock r beta p ≃ₗ[ℝ] U
  mapA : ∀ x, A (input x) = hidden (x.1, (0, 0))
  mapB : ∀ x, B (hidden x) = output ((x.1.1, x.2.1), 0)
```

```
lemma PairCoordinates.mapBA {A : V →ₗ[ℝ] W} {B : W →ₗ[ℝ] U}
    {r alpha beta h p q : ℕ} (c : PairCoordinates A B r alpha beta h p q)
    (x : InputBlock r alpha q) :
    B (A (c.input x)) = c.output ((x.1.1, 0), 0) := by
  rw [c.mapA, c.mapB]

/-- Dimension bookkeeping follows from the actual equivalences. -/
lemma PairCoordinates.dimensions [FiniteDimensional ℝ V] [FiniteDimensional ℝ W]
    [FiniteDimensional ℝ U] {A : V →ₗ[ℝ] W} {B : W →ₗ[ℝ] U}
    {r alpha beta h p q : ℕ} (c : PairCoordinates A B r alpha beta h p q) :
    (r + alpha) + q = finrank ℝ V ∧
    (r + alpha) + (beta + h) = finrank ℝ W ∧
    (r + beta) + p = finrank ℝ U := by
  refine ⟨?_, ?_, ?⟩
  · simpa [InputBlock, Vec] using c.input.finrank_eq
  · simpa [HiddenBlock, Vec] using c.hidden.finrank_eq
  · simpa [OutputBlock, Vec] using c.output.finrank_eq

end Coordinates
```

Construction and coherent numbering. Choose a complement D to C in S, a complement Z to the embedded K in $\ker B$, a complement P to S in U, and put $Q = \ker A$. The section equivalences give

$$R \simeq C \times K, \quad S \simeq C \times D, \quad V \simeq (C \times K) \times Q, \quad W \simeq (C \times K) \times (D \times Z), \quad U \simeq (C \times D) \times P.$$

For example, the hidden coordinate vector $((c, k), (d, z))$ maps to $s_B(c + d) + k + z$; the compatible section identifies $s_C(c)$ with $s_B(c)$. Applying A or B now proves the two displayed coordinate formulas: A kills Q, and B kills K and Z. Only after these equations are established does the proof choose a basis of each of C, K, D, Z, P, Q, using `LinearEquiv.ofFinrankEq`. Each choice is reused at every occurrence of that summand. Consequently numbering does not alter either commuting square. The final rank equations are $r = \operatorname{rank} BA$, $r + \alpha = \operatorname{rank} A$, and $r + \beta = \operatorname{rank} B$.

```
/-- A coordinate choice is made once per summand, then reused at every
vertex where that summand occurs. No topology is used to number a basis. -/
private def number (E : Type*) [AddCommGroup E] [Module ℝ E]
    [FiniteDimensional ℝ E] : Vec (finrank ℝ E) ≃ₗ[ℝ] E :=
  LinearEquiv.ofFinrankEq _ _ (by simp [Vec])

section Existence
variable {V W U : Type*}
  [AddCommGroup V] [Module ℝ V] [FiniteDimensional ℝ V]
  [AddCommGroup W] [Module ℝ W] [FiniteDimensional ℝ W]
  [AddCommGroup U] [Module ℝ U] [FiniteDimensional ℝ U]

/-- Simultaneous normal form with actual ranks, before passing from block
products to the manuscript's particular row/column enumerations.
The coordinate choices are witnesses, not assumptions of this theorem. -/
theorem exists_numbered_coordinates (A : V →ₗ[ℝ] W) (B : W →ₗ[ℝ] U) :
    ∃ (r alpha beta h p q : ℕ) (_c : PairCoordinates A B r alpha beta h p q),
      r = finrank ℝ (LinearMap.range (B.comp A)) ∧
      r + alpha = finrank ℝ (LinearMap.range A) ∧
      r + beta = finrank ℝ (LinearMap.range B) := by
  classical
  let R := LinearMap.range A
  let S := LinearMap.range B
  let F := imageMap A B
  let C := LinearMap.range F
  let K := LinearMap.ker F
```

```
  obtain ⟨sB, hsB, ⟩hAs := O77.Adapted0906.compatible_image_section A B
  let sC := restrictedSection A B sB hAs
  have hsC : F.rangeRestrict.comp sC = LinearMap.id :=
    restrictedSection_rightInverse A B sB hsB hAs
  let eR : (C × K) ≃ₗ[ℝ] R := sectionProd F sC hsC
  obtain ⟨sA, ⟩hsA :=
    A.rangeRestrict.exists_rightInverse_of_surjective A.range_rangeRestrict
  let eA := sectionProd A sA hsA
  let eB := sectionProd B sB hsB
  obtain ⟨D, ⟩hD := exists_isCompl C
  let eS : (C × D) ≃ₗ[ℝ] S := C.prodEquivOfIsCompl D hD
  let jK := kernelEmbedding A B
  let eK : K ≃ₗ[ℝ] LinearMap.range jK :=
    LinearEquiv.ofInjective jK (kernelEmbedding_injective A B)
  obtain ⟨Z, ⟩hZ := exists_isCompl (LinearMap.range jK)
  let eKer : (K × Z) ≃ₗ[ℝ] LinearMap.ker B :=
    (eK.prodCongr (LinearEquiv.refl ℝ Z)).trans
      ((LinearMap.range jK).prodEquivOfIsCompl Z hZ)
  obtain ⟨P, ⟩hP := exists_isCompl S
  let eU : (S × P) ≃ₗ[ℝ] U := S.prodEquivOfIsCompl P hP

  -- First construct the diagram isomorphism on the actual summand types.
  let eI : ((C × K) × LinearMap.ker A) ≃ₗ[ℝ] V :=
    (eR.prodCongr (LinearEquiv.refl ℝ (LinearMap.ker A))).trans eA
  let eH : ((C × K) × (D × Z)) ≃ₗ[ℝ] W :=
    (LinearEquiv.prodProdProdComm ℝ C K D Z).trans
      ((eS.prodCongr eKer).trans eB)
  let eO : ((C × D) × P) ≃ₗ[ℝ] U :=
    (eS.prodCongr (LinearEquiv.refl ℝ P)).trans eU

  have hAsection (x : R) : A (sA x) = (x : W) :=
    congrArg Subtype.val (LinearMap.congr_fun hsA x)
  have hBsection (x : S) : B (sB x) = (x : U) :=
    congrArg Subtype.val (LinearMap.congr_fun hsB x)
  have heA (x : (C × K) × LinearMap.ker A) :
      A (eI x) = eH (x.1, (0, 0)) := by
    change A (sA (eR x.1) + (x.2 : V)) =
      sB ((x.1.1 : S) + 0) + ((jK x.1.2 + 0 : LinearMap.ker B) : W)
    rw [map_add, hAsection, show A (x.2 : V) = 0 from x.2.property]
    simp only [add_zero]
    rfl
  have heB (x : (C × K) × (D × Z)) :
      B (eH x) = eO ((x.1.1, x.2.1), 0) := by
    change B (sB ((x.1.1 : S) + (x.2.1 : S)) +
        ((jK x.1.2 + (x.2.2 : LinearMap.ker B) : LinearMap.ker B) : W)) =
      (((x.1.1 : S) + (x.2.1 : S) : S) : U) + 0
    rw [map_add, hBsection]
    congr 1
    exact (jK x.1.2 + (x.2.2 : LinearMap.ker B)).property

  -- Use each numbering at every occurrence of the same summand.
  let r := finrank ℝ C
  let alpha := finrank ℝ K
  let beta := finrank ℝ D
  let h := finrank ℝ Z
  let p := finrank ℝ P
  let q := finrank ℝ (LinearMap.ker A)
  let nC := number C
  let nK := number K
  let nD := number D
  let nZ := number Z
  let nP := number P
```

```
  let nQ := number (LinearMap.ker A)
  let nI : InputBlock r alpha q ≃ₗ[ℝ] (C × K) × LinearMap.ker A :=
    (nC.prodCongr nK).prodCongr nQ
  let nH : HiddenBlock r alpha beta h ≃ₗ[ℝ] (C × K) × (D × Z) :=
    (nC.prodCongr nK).prodCongr (nD.prodCongr nZ)
  let nO : OutputBlock r beta p ≃ₗ[ℝ] (C × D) × P :=
    (nC.prodCongr nD).prodCongr nP
  let c : PairCoordinates A B r alpha beta h p q := {
    input := nI.trans eI
    hidden := nH.trans eH
    output := nO.trans eO
    mapA := by
      intro x
      change A (eI (nI x)) = eH (nH (x.1, (0, 0)))
      rw [heA]
      congr 1
      simp [nI, nH]
    mapB := by
      intro x
      change B (eH (nH x)) = eO (nO ((x.1.1, x.2.1), 0))
      rw [heB]
      congr 1
      simp [nH, nO] }
  refine ⟨r, alpha, beta, h, p, q, c, ?_, ?_, ?⟩_
  · exact (compositeRangeEquiv A B).finrank_eq
  · simpa only [Module.finrank_prod] using eR.finrank_eq
  · simpa only [Module.finrank_prod] using eS.finrank_eq
```

Recovering the six numerical dimensions. For native matrices put $a = \operatorname{rank} A$, $b = \operatorname{rank} B$, $r = \operatorname{rank} BA$. The proved additive dimension equations force

$$\alpha = a - r, \quad \beta = b - r, \quad q = I - a, \quad p = O - b, \quad h = H + r - a - b.$$

The natural subtractions in the Lean type occur only after those additive equalities and rank bounds are available. In particular the hidden quantity is not obtained by first subtracting $a + b$ from H. The exact-fit specialization replaces $\operatorname{rank} BA$ with $\operatorname{rank} \Phi$. The two examples test the empty diagram and the retained composite map; they do not introduce additional hypotheses into the existence theorem.

```
/-- At actual matrix ranks all six dimensions are forced. Natural
subtractions occur only after the corresponding additive equalities have
been proved, including the hidden residual dimension. -/
theorem native_numbered_coordinates {I O H : ℕ}
    (A : Matrix (Fin H) (Fin I) ℝ) (B : Matrix (Fin O) (Fin H) ℝ) :
    Nonempty (PairCoordinates A.mulVecLin B.mulVecLin
      (B * A).rank (A.rank - (B * A).rank) (B.rank - (B * A).rank)
      (H + (B * A).rank - A.rank - B.rank) (O - B.rank) (I - A.rank)) := by
  obtain ⟨r, alpha, beta, h, p, q, c, hr, ha, ⟩hb :=
    exists_numbered_coordinates A.mulVecLin B.mulVecLin
  have hd := c.dimensions
  have hi : (r + alpha) + q = I := by simpa using hd.1
  have hh : (r + alpha) + (beta + h) = H := by simpa using hd.2.1
  have ho : (r + beta) + p = O := by simpa using hd.2.2
  have er : r = (B * A).rank := by
    change r = finrank ℝ (LinearMap.range (B * A).mulVecLin)
    rw [Matrix.mulVecLin_mul]
    exact hr
  change r + alpha = A.rank at ha
  change r + beta = B.rank at hb
  have ea : alpha = A.rank - (B * A).rank := by omega
```

```
  have eb : beta = B.rank - (B * A).rank := by omega
  have eh : h = H + (B * A).rank - A.rank - B.rank := by omega
  have ep : p = O - B.rank := by omega
  have eq : q = I - A.rank := by omega
  rw [er, ea, eb, eh, ep, eq] at c
  exact ⟨⟩c

/-- Exact-fit specialization; the target hypothesis is used only to
identify the already computed composite rank. No singular data are needed. -/
theorem exact_fit_numbered_coordinates {I O H : ℕ}
    {A : Matrix (Fin H) (Fin I) ℝ} {B : Matrix (Fin O) (Fin H) ℝ}
    {Phi : Matrix (Fin O) (Fin I) ℝ} (hfit : B * A = Phi) :
    Nonempty (PairCoordinates A.mulVecLin B.mulVecLin
      Phi.rank (A.rank - Phi.rank) (B.rank - Phi.rank)
      (H + Phi.rank - A.rank - B.rank) (O - B.rank) (I - A.rank)) := by
  simpa only [hfit] using native_numbered_coordinates A B

end Existence

-- Boundary regression declarations.
example (A : Matrix (Fin 0) (Fin 0) ℝ) (B : Matrix (Fin 0) (Fin 0) ℝ) :
    Nonempty (PairCoordinates A.mulVecLin B.mulVecLin 0 0 0 0 0 0) := by
  have hA : A = 0 := Subsingleton.elim _ _
  have hB : B = 0 := Subsingleton.elim _ _
  simpa [hA, hB] using native_numbered_coordinates A B

example {I O H : ℕ} {A : Matrix (Fin H) (Fin I) ℝ}
    {B : Matrix (Fin O) (Fin H) ℝ} {Phi : Matrix (Fin O) (Fin I) ℝ}
    (hfit : B * A = Phi) :
    ∃ (c : PairCoordinates A.mulVecLin B.mulVecLin
      Phi.rank (A.rank - Phi.rank) (B.rank - Phi.rank)
      (H + Phi.rank - A.rank - B.rank) (O - B.rank) (I - A.rank)),
      ∀ x, B.mulVecLin (A.mulVecLin (c.input x)) =
        c.output ((x.1.1, 0), 0) := by
  obtain ⟨⟩c := exact_fit_numbered_coordinates hfit
  exact ⟨c, c.⟩mapBA
```

Actual orbit classification. If two matrix pairs have equal ranks of A, B, BA, their six dimensions agree. Compose one input coordinate inverse with the other input coordinates, and similarly at the hidden and output vertices. The shared block formulas then prove $A'e_I = e_H A$ and $B'e_H = e_O B$. To convert these linear equivalences into invertible matrices, use `Matrix.GeneralLinearGroup.toLin` and `LinearMap.GeneralLinearGroup.ofLinearEquiv`. The generic helper `matrixUnit_intertwines` turns $M'e_D = e_C M$ into $G_C M G_D^{-1} = M'$ by multiplying the commuting-square equation on the right by G_D^{-1}. Applying it to both arrows supplies actual witnesses G_I, G_H, G_O, including when some coordinate spaces are zero-dimensional.

```
section Orbits
/-- Equal actual ranks give commuting linear equivalences at all three
vertices. The same numbered intermediate summands are reused in both squares. -/
theorem pair_equivalences_of_ranks {I O H : ℕ}
    (A A' : Matrix (Fin H) (Fin I) ℝ) (B B' : Matrix (Fin O) (Fin H) ℝ)
    (hA : A.rank = A'.rank) (hB : B.rank = B'.rank)
    (hBA : (B * A).rank = (B' * A').rank) :
    ∃ (eI : Vec I ≃ₗ[ℝ] Vec I) (eH : Vec H ≃ₗ[ℝ] Vec H)
      (eO : Vec O ≃ₗ[ℝ] Vec O),
      ∀( x, A'.mulVecLin (eI x) = eH (A.mulVecLin x)) ∧
      ∀( y, B'.mulVecLin (eH y) = eO (B.mulVecLin y)) := by
  obtain ⟨⟩c := native_numbered_coordinates A B
  obtain ⟨c⟩' := native_numbered_coordinates A' B'
```

```
  rw ←[ hA, ← hB, ← hBA] at c'
  let eI := c.input.symm.trans c'.input
  let eH := c.hidden.symm.trans c'.hidden
  let eO := c.output.symm.trans c'.output
  refine ⟨eI, eH, eO, ?_, ?⟩_
  · intro x
    obtain ⟨v, ⟩rfl := c.input.surjective x
    change A'.mulVecLin (c'.input (c.input.symm (c.input v))) =
      c'.hidden (c.hidden.symm (A.mulVecLin (c.input v)))
    rw [c.input.symm_apply_apply, c.mapA, c'.mapA, c.hidden.symm_apply_apply]
  · intro y
    obtain ⟨v, ⟩rfl := c.hidden.surjective y
    change B'.mulVecLin (c'.hidden (c.hidden.symm (c.hidden v))) =
      c'.output (c.output.symm (B.mulVecLin (c.hidden v)))
    rw [c.hidden.symm_apply_apply, c.mapB, c'.mapB, c.output.symm_apply_apply]

/-- Use the library's equivalence of the two general linear groups. -/
def matrixUnit {n : ℕ} (e : Vec n ≃ₗ[ℝ] Vec n) :
    (Matrix (Fin n) (Fin n) ℝ)ˣ :=
  Matrix.GeneralLinearGroup.toLin.symm
    (LinearMap.GeneralLinearGroup.ofLinearEquiv e)

@[simp] lemma matrixUnit_mulVecLin {n : ℕ} (e : Vec n ≃ₗ[ℝ] Vec n) :
    (matrixUnit e : Matrix (Fin n) (Fin n) ℝ).mulVecLin = e.toLinearMap := by
  exact congrArg Units.val
    (Matrix.GeneralLinearGroup.toLin.apply_symm_apply
      (LinearMap.GeneralLinearGroup.ofLinearEquiv e))

/-- The same commuting-square conversion is used for A and B. -/
lemma matrixUnit_intertwines {n m : ℕ}
    (eD : Vec n ≃ₗ[ℝ] Vec n) (eC : Vec m ≃ₗ[ℝ] Vec m)
    (M M' : Matrix (Fin m) (Fin n) ℝ)
    (h : ∀ x, M'.mulVecLin (eD x) = eC (M.mulVecLin x)) :
    (matrixUnit eC : Matrix (Fin m) (Fin m) ℝ) * M *
      ↑(((matrixUnit eD)⁻¹) : Matrix (Fin n) (Fin n) ℝ) = M' := by
  have hc : M' * (matrixUnit eD : Matrix (Fin n) (Fin n) ℝ) =
      (matrixUnit eC : Matrix (Fin m) (Fin m) ℝ) * M := by
    apply Matrix.toLin'.injective
    apply LinearMap.ext
    intro x
    change (M' * (matrixUnit eD : Matrix (Fin n) (Fin n) ℝ)).mulVecLin x =
      ((matrixUnit eC : Matrix (Fin m) (Fin m) ℝ) * M).mulVecLin x
    simp only [Matrix.mulVecLin_mul, LinearMap.comp_apply, matrixUnit_mulVecLin]
    exact h x
  rw ←[ hc, Matrix.mul_assoc]
  simp only ←[ Units.val_mul, mul_inv_cancel, Units.val_one, Matrix.mul_one]

/-- Witness-producing orbit classification, in the original Fin-indexed
matrix spaces. This is purely algebraic and includes all empty dimensions. -/
theorem basis_matrices_of_ranks {I O H : ℕ}
    (A A' : Matrix (Fin H) (Fin I) ℝ) (B B' : Matrix (Fin O) (Fin H) ℝ)
    (hA : A.rank = A'.rank) (hB : B.rank = B'.rank)
    (hBA : (B * A).rank = (B' * A').rank) :
    ∃ (GI : (Matrix (Fin I) (Fin I) ℝ)ˣ)
      (GO : (Matrix (Fin O) (Fin O) ℝ)ˣ)
      (GH : (Matrix (Fin H) (Fin H) ℝ)ˣ),
      (GH : Matrix (Fin H) (Fin H) ℝ) * A *
        ↑((↑GI⁻¹) : Matrix (Fin I) (Fin I) ℝ) = A' ∧
      (GO : Matrix (Fin O) (Fin O) ℝ) * B *
        ↑((↑GH⁻¹) : Matrix (Fin H) (Fin H) ℝ) = B' := by
  obtain ⟨eI, eH, eO, heA, ⟩heB := pair_equivalences_of_ranks A A' B B' hA hB hBA
  exact ⟨matrixUnit eI, matrixUnit eO, matrixUnit eH,
```

```
    matrixUnit_intertwines eI eH A A' heA,
    matrixUnit_intertwines eH eO B B' ⟩heB

end Orbits

end O77.NumberedCoordinates
end
```

31 Module `O77Analytic`

31.1 The explicit analytic elimination chart

Here `RMmn` means a real $m \times n$ matrix. The first lemma proves analyticity of the genuine square-matrix inverse at every matrix with nonzero determinant: `Matrix.isUnit_iff_isUnit_det` turns the hypothesis into a unit, `analyticAt_inverse` supplies analytic inversion, and `Matrix.nonsing_inv_eq_ringInverse` identifies the library formulas. The family version is composition with an analytic matrix-valued map. The Frobenius scope in this small argument supplies a submultiplicative matrix norm; it does not identify the default norm of a nested product with a Euclidean norm. The three subsequent bridges identify the recursive core's finite sum, multiplication, and identity matrix with the corresponding Mathlib operations. They transport algebraic equalities only.

```
import Mathlib.Analysis.Analytic.Constructions
import Mathlib.Analysis.Matrix.Normed
import Mathlib.LinearAlgebra.Matrix.NonsingularInverse
import Mathlib.Tactic
import O77

set_option autoImplicit false
noncomputable section
open scoped BigOperators

namespace O77.Reconstruction0905

abbrev RM (m n : Nat) := Matrix (Fin m) (Fin n) ℝ

section AnalyticInverse
-- This local scope supplies a submultiplicative matrix norm.
-- It is NOT a claim that an arbitrary product norm is Euclidean.
open scoped Matrix.Norms.Frobenius

lemma matrix_inverse_analyticAt {n : Nat} (P : RM n n)
    (hP : P.det ≠ 0) :
    AnalyticAt ℝ (fun Q : RM n n => ⁻Q¹) P := by
  have hunit : IsUnit P :=
    (Matrix.isUnit_iff_isUnit_det P).2
      (isUnit_iff_ne_zero.mpr hP)
  rcases hunit with ⟨u, ⟩rfl
  simpa only [Matrix.nonsing_inv_eq_ringInverse] using
    (analyticAt_inverse (𝕂 := ℝ) u)

lemma family_inverse_analyticAt
    {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E]
    {n : Nat} {f : E → RM n n} {x : E}
    (hf : AnalyticAt ℝ f x) (hx : (f x).det ≠ 0) :
    AnalyticAt ℝ (fun y => (f y)⁻¹) x :=
  (matrix_inverse_analyticAt (f x) hx).comp hf

end AnalyticInverse
```

```
-- Bridge only the two matrix operations needed to attach inverse
-- witnesses.  No norm or topology is transported by this bridge.
lemma core_sum_eq {n : Nat} (f : Fin n → ℝ) :
    MatrixCore.sum n f = ∑ i, f i := by
  induction n with
  | zero => simp [MatrixCore.sum]
  | succ n ih =>
      simpa only [MatrixCore.sum, Fin.sum_univ_castSucc] using
        congrArg (fun z => z + f (Fin.last n))
          (ih (fun i => f i.castSucc))

lemma core_mul_eq {m n k : Nat} (A : RM m n) (B : RM n k) :
    MatrixCore.mul A B = A * B := by
  funext i j
  exact core_sum_eq (fun t => A i t * B t j)

lemma core_one_eq (n : Nat) :
    MatrixCore.one (R := ℝ) n = (1 : RM n n) := by
  ext i j
  simp [MatrixCore.one, Matrix.one_apply]
```

The seven raw and nine new blocks. Fix $r, \alpha, \beta, h, p, q \in \mathbb{N}$, and abbreviate $a = r + \alpha$, $b = r + \beta$, $I = a + q$, $O = b + p$, and $H = a + \beta + h$. The original pair is assembled from

$$A = \begin{pmatrix} P & A_{12} \\ C & A_{22} \end{pmatrix}, \qquad B = (B_1 \; D \; Y),$$

where the seven raw blocks have sizes $a \times a$, $(\beta + h) \times a$, $a \times q$, $(\beta + h) \times q$, $O \times a$, $O \times \beta$, and $O \times h$. The nine new blocks are $(U, T', Y_0, Z, P, C, X_P, D, Y_1)$, of respective sizes $O \times a$, $b \times q$, $p \times h$, $h \times q$, $a \times a$, $(\beta + h) \times a$, $\alpha \times q$, $O \times \beta$, and $b \times h$. Let J be the first r canonical output columns and let $C_I = (J \; 0)$ be the $O \times a$ left target block. Set $E = \mathrm{top}_b(J \; D)$ and $F = \mathrm{bottom}_p(J \; D)$. Both chart domains require $\det P \neq 0$ and $\det E \neq 0$. These are open conditions on ordinary matrix entries; inverse witnesses are not extra ambient coordinates.

```
abbrev Raw (r alpha beta h p q : Nat) :=
  RM (r+alpha) (r+alpha) × RM (beta+h) (r+alpha) ×
  RM (r+alpha) q × RM (beta+h) q ×
  RM ((r+beta)+p) (r+alpha) ×
  RM ((r+beta)+p) beta × RM ((r+beta)+p) h

abbrev Coordinates (r alpha beta h p q : Nat) :=
  RM ((r+beta)+p) (r+alpha) × RM (r+beta) q ×
  RM p h × RM h q × RM (r+alpha) (r+alpha) ×
  RM (beta+h) (r+alpha) × RM alpha q ×
  RM ((r+beta)+p) beta × RM (r+beta) h

variable {r alpha beta h p q : Nat}

def pivotTop (D : RM ((r+beta)+p) beta) : RM (r+beta) (r+beta) :=
  MatrixCore.rowsTop (m := r+beta) (n := p)
    (MatrixCore.hcat (MatrixCore.canonicalJ (R := ℝ) r beta p) D)

def pivotBottom (D : RM ((r+beta)+p) beta) : RM p (r+beta) :=
  MatrixCore.rowsBot (m := r+beta) (n := p)
    (MatrixCore.hcat (MatrixCore.canonicalJ (R := ℝ) r beta p) D)

def rawGuard (w : Raw r alpha beta h p q) : Prop :=
  let (P, _, _, _, _, D, _) := w
  P.det ≠ 0 ∧ (pivotTop D).det ≠ 0
```

```
def coordinateGuard (c : Coordinates r alpha beta h p q) : Prop :=
  let (_, _, _, _, P, _, _, D, _) := c
  P.det ≠ 0 ∧ (pivotTop D).det ≠ 0

def sourceDomain : Set (Raw r alpha beta h p q) := {w | rawGuard w}
def targetDomain : Set (Coordinates r alpha beta h p q) :=
  {c | coordinateGuard c}
```

Forward and inverse formulas. Compute $X = P^{-1}A_{12}$, $S = A_{22} - CX$, and $U = B_1P + (D\ Y)C - C_I$. Split $X = \binom{X_R}{X_P}$ and $S = \binom{S_Q}{Z}$. The output normalizer and its inverse are

$$R = \begin{pmatrix} E^{-1} & 0 \\ -FE^{-1} & 1 \end{pmatrix}, \qquad Q = \begin{pmatrix} E & 0 \\ F & 1 \end{pmatrix}.$$

Write $RY = \binom{Y_1}{Y_0}$ and define $T' = \binom{X_R}{S_Q} + Y_1Z$. For the inverse, first undo this shear, recover X, S, put $Y = Q\binom{Y_1}{Y_0}$, and then set

$$A_{12} = PX, \quad A_{22} = S + CX, \quad B_1 = (U + C_I - (D\ Y)C)P^{-1}.$$

These are exactly `encode` and `decode`. Inverse notation is explicitly typed as a square matrix throughout, avoiding the pointwise reciprocal operation on a raw function type. The determinant guards are preserved because P and D themselves are retained by the chart.

```
-- Total formulas are convenient; the inverse theorems are guarded.
def encode (w : Raw r alpha beta h p q) :
    Coordinates r alpha beta h p q :=
  let (P, C, A12, A22, B1, D, Y) := w
  let X := MatrixCore.mul (⁻P¹ : RM (r+alpha) (r+alpha)) A12
  let S := MatrixCore.sub A22 (MatrixCore.mul C X)
  let U := MatrixCore.sub
    (MatrixCore.add (MatrixCore.mul B1 P)
      (MatrixCore.mul (MatrixCore.hcat D Y) C))
    (MatrixCore.canonicalCI (R := ℝ) r alpha beta p)
  let XR := MatrixCore.rowsTop (m := r) (n := alpha) X
  let XP := MatrixCore.rowsBot (m := r) (n := alpha) X
  let SQ := MatrixCore.rowsTop (m := beta) (n := h) S
  let Z := MatrixCore.rowsBot (m := beta) (n := h) S
  let R := MatrixCore.normalizer ((pivotTop D)⁻¹ : RM (r+beta) (r+beta)) (pivotBottom D)
  let RY := MatrixCore.mul R Y
  let Y1 := MatrixCore.rowsTop (m := r+beta) (n := p) RY
  let Y0 := MatrixCore.rowsBot (m := r+beta) (n := p) RY
  let Tprime := MatrixCore.shear Y1 Z (MatrixCore.vcat XR SQ)
  (U, Tprime, Y0, Z, P, C, XP, D, Y1)

def decode (c : Coordinates r alpha beta h p q) :
    Raw r alpha beta h p q :=
  let (U, Tprime, Y0, Z, P, C, XP, D, Y1) := c
  let T := MatrixCore.unshear Y1 Z Tprime
  let X := MatrixCore.vcat
    (MatrixCore.rowsTop (m := r) (n := beta) T) XP
  let S := MatrixCore.vcat
    (MatrixCore.rowsBot (m := r) (n := beta) T) Z
  let Y := MatrixCore.mul
    (MatrixCore.denormalizer (pivotTop D) (pivotBottom D))
    (MatrixCore.vcat Y1 Y0)
  let A12 := MatrixCore.mul P X
  let A22 := MatrixCore.add S (MatrixCore.mul C X)
  let B1 := MatrixCore.mul
    (MatrixCore.sub
```

```
      (MatrixCore.add U (MatrixCore.canonicalCI (R := ℝ) r alpha beta p))
      (MatrixCore.mul (MatrixCore.hcat D Y) C)) (⁻P¹ : RM (r+alpha) (r+alpha))
  (P, C, A12, A22, B1, D, Y)

lemma encode_guard (w : Raw r alpha beta h p q) :
    coordinateGuard (encode w) ↔ rawGuard w := by
  rcases w with ⟨P, C, A12, A22, B1, D, ⟩Y
  rfl

lemma decode_guard (c : Coordinates r alpha beta h p q) :
    rawGuard (decode c) ↔ coordinateGuard c := by
  rcases c with ⟨U, Tprime, Y0, Z, P, C, XP, D, ⟩Y1
  rfl
```

Reusing the proved algebraic inverse identities. On the guarded domains, `liftPivot` and `liftOutput` equip the actual matrices with the two genuine inverse identities from `Matrix.nonsing_inv_mul` and `Matrix.mul_nonsing_inv`. The lift/forget maps leave all seven or nine matrix blocks unchanged. Their compatibility with `MatrixCore.elimForward` and `MatrixCore.elimBackward` therefore transports the already proved algebraic identities `MatrixCore.elim_backward_forward` and `MatrixCore.elim_forward_backward` to the actual maps: $\mathrm{decode} \circ \mathrm{encode} = 1$ on the source and $\mathrm{encode} \circ \mathrm{decode} = 1$ on the target. No equality outside those domains is used.

```
def liftPivot {n : Nat} (P : RM n n) (hP : P.det ≠ 0) :
    MatrixCore.InvMatrix n ℝ where
  matrix := P
  inverse := (⁻P¹ : RM n n)
  inverse_mul :=
    (core_mul_eq (⁻P¹ : RM n n) P).trans
      ((Matrix.nonsing_inv_mul P (isUnit_iff_ne_zero.mpr hP)).trans
        (core_one_eq n).symm)
  mul_inverse :=
    (core_mul_eq P (⁻P¹ : RM n n)).trans
      ((Matrix.mul_nonsing_inv P (isUnit_iff_ne_zero.mpr hP)).trans
        (core_one_eq n).symm)

def liftOutput (D : RM ((r+beta)+p) beta)
    (hD : (pivotTop D).det ≠ 0) :
    MatrixCore.OutputPivotData r beta p ℝ where
  d := D
  inverse := ((pivotTop D)⁻¹ : RM (r+beta) (r+beta))
  inverse_mul_top :=
    (core_mul_eq ((pivotTop D)⁻¹ : RM (r+beta) (r+beta)) (pivotTop D)).trans
      ((Matrix.nonsing_inv_mul _ (isUnit_iff_ne_zero.mpr hD)).trans
        (core_one_eq (r+beta)).symm)
  top_mul_inverse :=
    (core_mul_eq (pivotTop D) ((pivotTop D)⁻¹ : RM (r+beta) (r+beta))).trans
      ((Matrix.mul_nonsing_inv _ (isUnit_iff_ne_zero.mpr hD)).trans
        (core_one_eq (r+beta)).symm)

def liftRaw (w : Raw r alpha beta h p q) (hw : rawGuard w) :
    MatrixCore.ElimInput r alpha beta h p q ℝ :=
  { aPivot := liftPivot w.1 hw.1
    aLower := w.2.1
    a12 := w.2.2.1
    a22 := w.2.2.2.1
    b1 := w.2.2.2.2.1
    outPivot := liftOutput w.2.2.2.2.2.1 hw.2
    bY := w.2.2.2.2.2.2 }
```

```
def liftCoordinates (c : Coordinates r alpha beta h p q)
    (hc : coordinateGuard c) :
    MatrixCore.ElimCoordinates r alpha beta h p q ℝ :=
  { regularLeft := c.1
    regularRight := c.2.1
    residualLeft := c.2.2.1
    residualRight := c.2.2.2.1
    aPivot := liftPivot c.2.2.2.2.1 hc.1
    aLower := c.2.2.2.2.2.1
    xPassive := c.2.2.2.2.2.2.1
    outPivot := liftOutput c.2.2.2.2.2.2.2.1 hc.2
    yPassive := c.2.2.2.2.2.2.2.2 }

def forgetRaw (z : MatrixCore.ElimInput r alpha beta h p q ℝ) :
    Raw r alpha beta h p q :=
  (z.aPivot.matrix, z.aLower, z.a12, z.a22,
    z.b1, z.outPivot.d, z.bY)

def forgetCoordinates
    (z : MatrixCore.ElimCoordinates r alpha beta h p q ℝ) :
    Coordinates r alpha beta h p q :=
  (z.regularLeft, z.regularRight, z.residualLeft, z.residualRight,
    z.aPivot.matrix, z.aLower, z.xPassive, z.outPivot.d, z.yPassive)

lemma forget_liftRaw (w : Raw r alpha beta h p q) (hw : rawGuard w) :
    forgetRaw (liftRaw w hw) = w := by
  rcases w with ⟨P, C, A12, A22, B1, D, ⟩Y
  rfl

lemma forget_liftCoordinates (c : Coordinates r alpha beta h p q)
    (hc : coordinateGuard c) :
    forgetCoordinates (liftCoordinates c hc) = c := by
  rcases c with ⟨U, Tprime, Y0, Z, P, C, XP, D, ⟩Y1
  rfl

lemma lift_encode (w : Raw r alpha beta h p q) (hw : rawGuard w) :
    liftCoordinates (encode w) ((encode_guard w).2 hw) =
      MatrixCore.elimForward (liftRaw w hw) := by
  rcases w with ⟨P, C, A12, A22, B1, D, ⟩Y
  rfl

lemma lift_decode (c : Coordinates r alpha beta h p q)
    (hc : coordinateGuard c) :
    liftRaw (decode c) ((decode_guard c).2 hc) =
      MatrixCore.elimBackward (liftCoordinates c hc) := by
  rcases c with ⟨U, Tprime, Y0, Z, P, C, XP, D, ⟩Y1
  simp only [liftRaw, liftCoordinates, decode, MatrixCore.elimBackward,
    MatrixCore.gaugeBackward, MatrixCore.colsRight_hcat]
  rfl

lemma decode_encode (w : Raw r alpha beta h p q) (hw : rawGuard w) :
    decode (encode w) = w := by
  have hc := (encode_guard w).2 hw
  calc
    decode (encode w) =
        forgetRaw (liftRaw (decode (encode w))
          ((decode_guard (encode w)).2 hc)) :=
      (forget_liftRaw _ _).symm
    _ = forgetRaw (MatrixCore.elimBackward
          (MatrixCore.elimForward (liftRaw w hw))) := by
      rw [lift_decode _ hc, lift_encode _ hw]
    _ = w := by rw [MatrixCore.elim_backward_forward, forget_liftRaw]
```

```
lemma encode_decode (c : Coordinates r alpha beta h p q)
    (hc : coordinateGuard c) : encode (decode c) = c := by
  have hw := (decode_guard c).2 hc
  calc
    encode (decode c) =
        forgetCoordinates (liftCoordinates (encode (decode c))
          ((encode_guard (decode c)).2 hw)) :=
      (forget_liftCoordinates _ _).symm
    _ = forgetCoordinates (MatrixCore.elimForward
          (MatrixCore.elimBackward (liftCoordinates c hc))) := by
      rw [lift_encode _ hw, lift_decode _ hc]
    _ = c := by
      rw [MatrixCore.elim_forward_backward, forget_liftCoordinates]

end O77.Reconstruction0905
namespace O77.Reconstruction0905
```

Comparing finite lists of generators. For any rectangular matrix M, write $|M|_F^2 = \sum_{ij} M_{ij}^2$. Its equality with the square of the explicitly selected Frobenius norm follows from `Matrix.frobenius_norm_def`. The library inequality `Matrix.frobenius_norm_mul` yields $|MN|_F^2 \le |M|_F^2 |N|_F^2$. If two column lists satisfy $g = Tf$ and $f = Sg$, and $|T|_F^2, |S|_F^2 \le B$ with $B > 0$, then $|g|_F^2/B \le |f|_F^2 \le B|g|_F^2$. The two lists need not have equal lengths: only the displayed rectangular products and both reconstruction identities are required.

```
section FrobeniusComparison
open scoped Matrix.Norms.Frobenius
open O77.Residual (frobeniusSq)

lemma frobeniusSq_nonneg {m n : Nat} (A : RM m n) :
    0 ≤ frobeniusSq A := by
  exact Finset.sum_nonneg (fun i _ =>
    Finset.sum_nonneg (fun j _ => sq_nonneg (A i j)))

-- Both sides now use the explicitly selected Frobenius matrix norm.
lemma frobeniusSq_eq_norm_sq {m n : Nat} (A : RM m n) :
    frobeniusSq A = ‖A‖ ^ 2 := by
  rw [Matrix.frobenius_norm_def]
  simp only [Real.rpow_two, Real.norm_eq_abs, sq_abs]
  rw ←[ Real.sqrt_eq_rpow]
  exact (Real.sq_sqrt (frobeniusSq_nonneg A)).symm

lemma frobeniusSq_mul_le {m n k : Nat} (A : RM m n) (B : RM n k) :
    frobeniusSq (A * B) ≤ frobeniusSq A * frobeniusSq B := by
  rw [frobeniusSq_eq_norm_sq, frobeniusSq_eq_norm_sq,
    frobeniusSq_eq_norm_sq]
  calc
    ‖A * B‖ ^ 2 ≤ ‖(‖A‖ * ‖B‖) ^ 2 := by
      gcongr
      exact Matrix.frobenius_norm_mul A B
    _ = ‖A‖ ^ 2 * ‖B‖ ^ 2 := mul_pow _ _ _

-- Column matrices flatten finite lists of generators without using
-- the max norm of a product.  Rectangular redundant lists are allowed.
lemma bounded_generator_comparison {m n : Nat}
    (T : RM m n) (S : RM n m) (f : RM n 1) (g : RM m 1)
    (hg : g = T * f) (hf : f = S * g)
    {B : ℝ} (hB : 0 < B)
    (hT : frobeniusSq T ≤ B) (hS : frobeniusSq S ≤ B) :
    frobeniusSq g / B ≤ frobeniusSq f ∧
```

```
      frobeniusSq f ≤ B * frobeniusSq g := by
  have hgf : frobeniusSq g ≤ B * frobeniusSq f := by
    calc
      frobeniusSq g = frobeniusSq (T * f) := congrArg frobeniusSq hg
      _ ≤ frobeniusSq T * frobeniusSq f := frobeniusSq_mul_le T f
      _ ≤ B * frobeniusSq f :=
        mul_le_mul_of_nonneg_right hT (frobeniusSq_nonneg f)
  have hfg : frobeniusSq f ≤ B * frobeniusSq g := by
    calc
      frobeniusSq f = frobeniusSq (S * g) := congrArg frobeniusSq hf
      _ ≤ frobeniusSq S * frobeniusSq g := frobeniusSq_mul_le S g
      _ ≤ B * frobeniusSq g :=
        mul_le_mul_of_nonneg_right hS (frobeniusSq_nonneg g)
  constructor
  · apply (₀div_le_iff hB).2
    simpa only [mul_comm] using hgf
  · exact hfg

end FrobeniusComparison
end O77.Reconstruction0905
end
set_option autoImplicit false
noncomputable section
open scoped BigOperators
namespace O77.Reconstruction0905
```

Analyticity in the actual entry topology. The remaining chart proof works in the elementwise matrix topology. A determinant is a finite sum of signed products of entries; its analyticity is proved using `Finset.analyticAt_prod` and `Finset.analyticAt_sum`. An adjugate entry is a determinant after a row replacement. The formula $M^{-1} = (\det M)^{-1}\operatorname{adj}(M)$ then proves analytic matrix inversion under $\det M \neq 0$ in this topology. The subsequent helpers treat addition, subtraction, multiplication, concatenation, projections, and the shear as finite sums, products, or entry selections. Thus they provide the precise closure properties used by `fun_prop`; they do not assume that the nonlinear chart is analytic.

```
section AnalyticHelpers
open scoped Matrix.Norms.Elementwise
attribute [local fun_prop] analyticAt_fst analyticAt_snd
variable {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E]
variable {x : E}
@[fun_prop] lemma entrywise_det_analyticAt {n : Nat} {f : E → RM n n}
    (hf : AnalyticAt ℝ f x) :
    AnalyticAt ℝ (fun y => (f y).det) x := by
  have hp (sigma : Equiv.Perm (Fin n)) :
      AnalyticAt ℝ (fun y => ∏ i : Fin n, f y (sigma i) i) x := by
    have hi (i : Fin n) : AnalyticAt ℝ (fun y => f y (sigma i) i) x :=
      analyticAt_pi_iff.mp (analyticAt_pi_iff.mp hf (sigma i)) i
    convert! Finset.analyticAt_prod (𝕂 := ℝ) (A := ℝ) Finset.univ (fun i _ => hi i) using 1
    first | rfl | (ext y; simp)
  have hs := Finset.analyticAt_sum Finset.univ
    (fun sigma _ =>
      (analyticAt_const (v := ↑↑((Equiv.Perm.sign sigma) : ℝ))).mul (hp sigma))
  convert! hs using 1
  first | rfl | (ext y; simp [Matrix.det_apply'])
@[fun_prop] lemma entrywise_adjugate_analyticAt {n : Nat} {f : E → RM n n}
    (hf : AnalyticAt ℝ f x) :
    AnalyticAt ℝ (fun y => (f y).adjugate) x := by
  apply AnalyticAt.pi
  intro i
  apply AnalyticAt.pi
```

```
  intro j
  simp only [Matrix.adjugate_apply]
  apply entrywise_det_analyticAt
  apply AnalyticAt.pi
  intro k
  apply AnalyticAt.pi
  intro l
  simp only [Matrix.updateRow_apply]
  split_ifs
  · exact analyticAt_const
  · exact (analyticAt_pi_iff.mp (analyticAt_pi_iff.mp hf k) l)
@[fun_prop] lemma entrywise_inverse_analyticAt {n : Nat} {f : E → RM n n}
    (hf : AnalyticAt ℝ f x) (h : (f x).det ≠ 0) :
    AnalyticAt ℝ (fun y => (f y)⁻¹) x := by
  have hs := ((entrywise_det_analyticAt hf).inv h).smul (entrywise_adjugate_analyticAt hf)
  convert! hs using 1
  try rfl
  all_goals ext y i j; simp [Matrix.inv_def, Ring.inverse_eq_inv']

lemma entry_analyticAt {m n : Nat} {f : E → RM m n}
    (hf : AnalyticAt ℝ f x) (i : Fin m) (j : Fin n) :
    AnalyticAt ℝ (fun y => f y i j) x :=
  analyticAt_pi_iff.mp (analyticAt_pi_iff.mp hf i) j

@[fun_prop] lemma core_add_analyticAt {m n : Nat} {f g : E → RM m n}
    (hf : AnalyticAt ℝ f x) (hg : AnalyticAt ℝ g x) :
    AnalyticAt ℝ (fun y => (MatrixCore.add (f y) (g y) : RM m n)) x := by
  exact hf.add hg

@[fun_prop] lemma core_sub_analyticAt {m n : Nat} {f g : E → RM m n}
    (hf : AnalyticAt ℝ f x) (hg : AnalyticAt ℝ g x) :
    AnalyticAt ℝ (fun y => (MatrixCore.sub (f y) (g y) : RM m n)) x := by
  exact hf.sub hg

@[fun_prop] lemma core_mul_analyticAt {m n k : Nat}
    {f : E → RM m n} {g : E → RM n k}
    (hf : AnalyticAt ℝ f x) (hg : AnalyticAt ℝ g x) :
    AnalyticAt ℝ (fun y => (MatrixCore.mul (f y) (g y) : RM m k)) x := by
  apply AnalyticAt.pi
  intro i
  apply AnalyticAt.pi
  intro j
  change AnalyticAt ℝ (fun y => MatrixCore.sum n
    (fun t => f y i t * g y t j)) x
  simp only [core_sum_eq]
  have hs := Finset.analyticAt_sum Finset.univ
    (fun t _ => (entry_analyticAt hf i t).mul (entry_analyticAt hg t j))
  convert! hs using 1
  first | rfl | (ext y; simp)

@[fun_prop] lemma core_vcat_analyticAt {m n k : Nat}
    {f : E → RM m k} {g : E → RM n k}
    (hf : AnalyticAt ℝ f x) (hg : AnalyticAt ℝ g x) :
    AnalyticAt ℝ (fun y => (MatrixCore.vcat (f y) (g y) : RM (m+n) k)) x := by
  apply AnalyticAt.pi
  intro i
  apply AnalyticAt.pi
  intro j
  refine Fin.addCases (fun i => ?_) (fun i => ?_) i
  · convert! entry_analyticAt hf i j using 1
    funext y
    exact congrArg (fun M : RM m k => M i j) (MatrixCore.rowsTop_vcat (f y) (g y))
```

```
  · convert! entry_analyticAt hg i j using 1
    funext y
    exact congrArg (fun M : RM n k => M i j) (MatrixCore.rowsBot_vcat (f y) (g y))

@[fun_prop] lemma core_hcat_analyticAt {m n k : Nat}
    {f : E → RM m n} {g : E → RM m k}
    (hf : AnalyticAt ℝ f x) (hg : AnalyticAt ℝ g x) :
    AnalyticAt ℝ (fun y => (MatrixCore.hcat (f y) (g y) : RM m (n+k))) x := by
  apply AnalyticAt.pi
  intro i
  apply AnalyticAt.pi
  intro j
  refine Fin.addCases (fun j => ?_) (fun j => ?_) j
  · simpa only [MatrixCore.hcat, Fin.addCases_left, Fin.addCases_right] using entry_analyticAt hf i j
  · simpa only [MatrixCore.hcat, Fin.addCases_left, Fin.addCases_right] using entry_analyticAt hg i j

@[fun_prop] lemma core_rowsTop_analyticAt {m n k : Nat}
    {f : E → RM (m+n) k} (hf : AnalyticAt ℝ f x) :
    AnalyticAt ℝ (fun y =>
      (MatrixCore.rowsTop (m := m) (n := n) (f y) : RM m k)) x := by
  apply AnalyticAt.pi
  intro i
  apply AnalyticAt.pi
  intro j
  exact entry_analyticAt hf (Fin.castAdd n i) j

@[fun_prop] lemma core_rowsBot_analyticAt {m n k : Nat}
    {f : E → RM (m+n) k} (hf : AnalyticAt ℝ f x) :
    AnalyticAt ℝ (fun y =>
      (MatrixCore.rowsBot (m := m) (n := n) (f y) : RM n k)) x := by
  apply AnalyticAt.pi
  intro i
  apply AnalyticAt.pi
  intro j
  exact entry_analyticAt hf (Fin.natAdd m i) j

@[fun_prop] lemma core_block_analyticAt {m n a b : Nat}
    {f : E → RM m a} {g : E → RM m b}
    {h : E → RM n a} {k : E → RM n b}
    (hf : AnalyticAt ℝ f x) (hg : AnalyticAt ℝ g x)
    (hh : AnalyticAt ℝ h x) (hk : AnalyticAt ℝ k x) :
    AnalyticAt ℝ (fun y =>
      (MatrixCore.block (f y) (g y) (h y) (k y) : RM (m+n) (a+b))) x := by
  exact core_hcat_analyticAt (core_vcat_analyticAt hf hh)
    (core_vcat_analyticAt hg hk)

@[fun_prop] lemma core_normalizer_analyticAt {b p : Nat}
    {f : E → RM b b} {g : E → RM p b}
    (hf : AnalyticAt ℝ f x) (hg : AnalyticAt ℝ g x) :
    AnalyticAt ℝ (fun y =>
      (MatrixCore.normalizer (f y) (g y) : RM (b+p) (b+p))) x := by
  exact core_block_analyticAt hf analyticAt_const
    (core_sub_analyticAt analyticAt_const (core_mul_analyticAt hg hf))
    analyticAt_const

@[fun_prop] lemma core_denormalizer_analyticAt {b p : Nat}
    {f : E → RM b b} {g : E → RM p b}
    (hf : AnalyticAt ℝ f x) (hg : AnalyticAt ℝ g x) :
    AnalyticAt ℝ (fun y =>
      (MatrixCore.denormalizer (f y) (g y) : RM (b+p) (b+p))) x := by
  exact core_block_analyticAt hf analyticAt_const hg analyticAt_const
```

```
@[fun_prop] lemma core_shear_analyticAt {b h q : Nat}
    {f : E → RM b h} {g : E → RM h q} {t : E → RM b q}
    (hf : AnalyticAt ℝ f x) (hg : AnalyticAt ℝ g x)
    (ht : AnalyticAt ℝ t x) :
    AnalyticAt ℝ (fun y => (MatrixCore.shear (f y) (g y) (t y) : RM b q)) x :=
  core_add_analyticAt ht (core_mul_analyticAt hf hg)

@[fun_prop] lemma core_unshear_analyticAt {b h q : Nat}
    {f : E → RM b h} {g : E → RM h q} {t : E → RM b q}
    (hf : AnalyticAt ℝ f x) (hg : AnalyticAt ℝ g x)
    (ht : AnalyticAt ℝ t x) :
    AnalyticAt ℝ (fun y => (MatrixCore.unshear (f y) (g y) (t y) : RM b q)) x :=
  core_sub_analyticAt ht (core_mul_analyticAt hf hg)

@[fun_prop] lemma pivotTop_analyticAt {r beta p : Nat}
    {f : E → RM ((r+beta)+p) beta} (hf : AnalyticAt ℝ f x) :
    AnalyticAt ℝ (fun y => pivotTop (f y)) x :=
  core_rowsTop_analyticAt (core_hcat_analyticAt analyticAt_const hf)

@[fun_prop] lemma pivotBottom_analyticAt {r beta p : Nat}
    {f : E → RM ((r+beta)+p) beta} (hf : AnalyticAt ℝ f x) :
    AnalyticAt ℝ (fun y => pivotBottom (f y)) x :=
  core_rowsBot_analyticAt (core_hcat_analyticAt analyticAt_const hf)

end AnalyticHelpers
```

Open domains and analytic inverse maps. Continuity of the two determinant functions makes their nonvanishing loci open. Every operation in the forward and backward formulas is covered by the preceding analytic helpers; the only denominators are discharged by the guards. Therefore both formulas are analytic on neighborhoods of points of their respective domains. The record `eliminationOpenPartialHomeomorph` combines these proved facts with both inverse identities and the domain-preservation equalities. Its fields are thus an assembled result about explicit maps, not an assumed chart.

```
section AnalyticChart
open scoped Matrix.Norms.Elementwise
attribute [local fun_prop] analyticAt_fst analyticAt_snd
variable {r alpha beta h p q : Nat}

lemma sourceDomain_isOpen :
    IsOpen (sourceDomain (r := r) (alpha := alpha) (beta := beta)
      (h := h) (p := p) (q := q)) := by
  have hP : Continuous (fun w : Raw r alpha beta h p q => w.1.det) := by
    apply continuous_iff_continuousAt.mpr
    intro w
    exact (entrywise_det_analyticAt (by fun_prop)).continuousAt
  have hD : Continuous (fun w : Raw r alpha beta h p q =>
      (pivotTop w.2.2.2.2.1).det) := by
    apply continuous_iff_continuousAt.mpr
    intro w
    exact (entrywise_det_analyticAt (pivotTop_analyticAt (by fun_prop))).continuousAt
  exact (isOpen_ne.preimage hP).inter (isOpen_ne.preimage hD)

lemma targetDomain_isOpen :
    IsOpen (targetDomain (r := r) (alpha := alpha) (beta := beta)
      (h := h) (p := p) (q := q)) := by
  have hP : Continuous (fun c : Coordinates r alpha beta h p q =>
      c.2.2.2.2.1.det) := by
    apply continuous_iff_continuousAt.mpr
    intro c
```

```
    exact (entrywise_det_analyticAt (by fun_prop)).continuousAt
  have hD : Continuous (fun c : Coordinates r alpha beta h p q =>
      (pivotTop c.2.2.2.2.2.2.1).det) := by
    apply continuous_iff_continuousAt.mpr
    intro c
    exact (entrywise_det_analyticAt (pivotTop_analyticAt (by fun_prop))).continuousAt
  exact (isOpen_ne.preimage hP).inter (isOpen_ne.preimage hD)

lemma encode_analyticAt (w : Raw r alpha beta h p q) (hw : rawGuard w) :
    AnalyticAt ℝ encode w := by
  change w.1.det ≠ 0 ∧ (pivotTop w.2.2.2.2.2.1).det ≠ 0 at hw
  have hP := hw.1
  have hD := hw.2
  unfold encode
  fun_prop (disch := assumption)

lemma decode_analyticAt (c : Coordinates r alpha beta h p q)
    (hc : coordinateGuard c) : AnalyticAt ℝ decode c := by
  change c.2.2.2.2.1.det ≠ 0 ∧
    (pivotTop c.2.2.2.2.2.2.1).det ≠ 0 at hc
  have hP := hc.1
  unfold decode
  fun_prop (disch := assumption)

lemma encode_analyticOnNhd : AnalyticOnNhd ℝ
    (encode (r := r) (alpha := alpha) (beta := beta) (h := h) (p := p) (q := q))
    sourceDomain := fun w hw => encode_analyticAt w hw

lemma decode_analyticOnNhd : AnalyticOnNhd ℝ
    (decode (r := r) (alpha := alpha) (beta := beta) (h := h) (p := p) (q := q))
    targetDomain := fun c hc => decode_analyticAt c hc

/-- Actual open matrix-coordinate spaces; no inverse witnesses are ambient coordinates. -/
def eliminationOpenPartialHomeomorph :
    OpenPartialHomeomorph (Raw r alpha beta h p q) (Coordinates r alpha beta h p q) where
  toFun := encode
  invFun := decode
  source := sourceDomain
  target := targetDomain
  map_source' := fun w hw => (encode_guard w).2 hw
  map_target' := fun c hc => (decode_guard c).2 hc
  left_inv' := fun w hw => decode_encode w hw
  right_inv' := fun c hc => encode_decode c hc
  open_source := sourceDomain_isOpen
  open_target := targetDomain_isOpen
  continuousOn_toFun := encode_analyticOnNhd.continuousOn
  continuousOn_invFun := decode_analyticOnNhd.continuousOn

end AnalyticChart
end O77.Reconstruction0905
end
set_option autoImplicit false
noncomputable section
open scoped BigOperators
namespace O77.Reconstruction0905
open O77.Residual (frobeniusSq)
variable {r alpha beta h p q : Nat}
```

The original product error in the new variables. The definitions below assemble A, B from the seven raw blocks and define $L = |BA - \Phi_0|_F^2/2$, with $\Phi_0 = (C_I\ 0)$. For a coordinate

tuple c, let $X(c)$ be the recovered matrix above, let $R(c), Q(c)$ be the output normalizer and its inverse, and put

$$V(c) = \begin{pmatrix} T' \\ Y_0 Z \end{pmatrix}, \qquad W(c) = UX(c) + Q(c)V(c).$$

The decisive identity is

$$BA - \Phi_0 = (U\ W(c)).$$

It is obtained from `MatrixCore.elim_forward_error_normal_form` after substituting the explicit inverse chart, and then transporting core multiplication into Mathlib multiplication. The equation $RQ = 1$ also gives $V = R(W - UX)$. Thus the entries of (U, W) and (U, V) generate one another with controlled matrix coefficients.

```
def parameterA (w : Raw r alpha beta h p q) :
    RM ((r+alpha)+(beta+h)) ((r+alpha)+q) :=
  MatrixCore.block w.1 w.2.2.1 w.2.1 w.2.2.2.1

def parameterB (w : Raw r alpha beta h p q) :
    RM ((r+beta)+p) ((r+alpha)+(beta+h)) :=
  MatrixCore.hcat w.2.2.2.2.1
    (MatrixCore.hcat w.2.2.2.2.2.1 w.2.2.2.2.2.2)

def targetMatrix (r alpha beta p q : Nat) : RM ((r+beta)+p) ((r+alpha)+q) :=
  MatrixCore.elimTarget (R := ℝ) r alpha beta p q

def parameterError (w : Raw r alpha beta h p q) :
    RM ((r+beta)+p) ((r+alpha)+q) :=
  parameterB w * parameterA w - targetMatrix r alpha beta p q

def parameterLoss (w : Raw r alpha beta h p q) : ℝ :=
  frobeniusSq (parameterError w) / 2

def coefficientX (c : Coordinates r alpha beta h p q) : RM (r+alpha) q :=
  MatrixCore.vcat
    (MatrixCore.rowsTop (m := r) (n := beta)
      (MatrixCore.unshear c.2.2.2.2.2.2.2.2 c.2.2.2.1 c.2.1))
    c.2.2.2.2.2.1

def coefficientR (c : Coordinates r alpha beta h p q) :
    RM ((r+beta)+p) ((r+beta)+p) :=
  MatrixCore.normalizer
    ((pivotTop c.2.2.2.2.2.2.2.1)⁻¹ : RM (r+beta) (r+beta))
    (pivotBottom c.2.2.2.2.2.2.1)

def coefficientQ (c : Coordinates r alpha beta h p q) :
    RM ((r+beta)+p) ((r+beta)+p) :=
  MatrixCore.denormalizer (pivotTop c.2.2.2.2.2.2.2.1)
    (pivotBottom c.2.2.2.2.2.2.1)

def reducedColumn (c : Coordinates r alpha beta h p q) : RM ((r+beta)+p) q :=
  MatrixCore.vcat c.2.1 (c.2.2.1 * c.2.2.2.1)

def reconstructedRight (c : Coordinates r alpha beta h p q) : RM ((r+beta)+p) q :=
  c.1 * coefficientX c + coefficientQ c * reducedColumn c

def coordinateEnergy (c : Coordinates r alpha beta h p q) : ℝ :=
  frobeniusSq c.1 + frobeniusSq c.2.1 + frobeniusSq (c.2.2.1 * c.2.2.2.1)

lemma parameterError_decode (c : Coordinates r alpha beta h p q)
    (hc : coordinateGuard c) :
    parameterError (decode c) = MatrixCore.hcat c.1 (reconstructedRight c) := by
```

```
  have he := MatrixCore.elim_forward_error_normal_form
    (MatrixCore.elimBackward (liftCoordinates c hc))
  dsimp only at he
  rw [MatrixCore.elim_forward_backward] at he
  rw ←[ lift_decode c hc] at he
  let Vc : RM ((r+beta)+p) q :=
    MatrixCore.vcat c.2.1 (MatrixCore.mul c.2.2.1 c.2.2.2.1)
  change MatrixCore.sub
    (MatrixCore.mul (parameterB (decode c)) (parameterA (decode c)))
    (targetMatrix r alpha beta p q) =
      MatrixCore.hcat c.1 (MatrixCore.add
        (MatrixCore.mul c.1 (coefficientX c))
        (MatrixCore.mul (coefficientQ c) Vc)) at he
  have hV : Vc = reducedColumn c :=
    congrArg (fun Z : RM p q => MatrixCore.vcat c.2.1 Z)
      (core_mul_eq c.2.2.1 c.2.2.2.1)
  have hleft : MatrixCore.sub
      (MatrixCore.mul (parameterB (decode c)) (parameterA (decode c)))
      (targetMatrix r alpha beta p q) = parameterError (decode c) :=
    congrArg (fun X : RM ((r+beta)+p) ((r+alpha)+q) =>
      X - targetMatrix r alpha beta p q)
      (core_mul_eq (parameterB (decode c)) (parameterA (decode c)))
  have hright : MatrixCore.add
      (MatrixCore.mul c.1 (coefficientX c))
      (MatrixCore.mul (coefficientQ c) Vc) = reconstructedRight c :=
    ₂congrArg (fun X Y : RM ((r+beta)+p) q => X+Y)
      (core_mul_eq c.1 (coefficientX c))
      ((core_mul_eq (coefficientQ c) Vc).trans
        (congrArg (fun V : RM ((r+beta)+p) q => coefficientQ c * V) hV))
  exact hleft.symm.trans
    (he.trans (congrArg (fun V : RM ((r+beta)+p) q => MatrixCore.hcat c.1 V) hright))

lemma coefficientR_mul_Q (c : Coordinates r alpha beta h p q)
    (hc : coordinateGuard c) : coefficientR c * coefficientQ c = 1 := by
  let d := c.2.2.2.2.2.2.1
  have h := MatrixCore.normalizer_denormalizer (pivotTop d)
    ((pivotTop d)⁻¹ : RM (r+beta) (r+beta)) (pivotBottom d)
    (liftOutput d hc.2).inverse_mul_top
  exact (core_mul_eq (coefficientR c) (coefficientQ c)).symm.trans
    (h.trans (core_one_eq ((r+beta)+p)))

lemma reducedColumn_reconstruct (c : Coordinates r alpha beta h p q)
    (hc : coordinateGuard c) :
    reducedColumn c = coefficientR c *
      (reconstructedRight c - c.1 * coefficientX c) := by
  rw [reconstructedRight, add_sub_cancel_left, ← Matrix.mul_assoc,
    coefficientR_mul_Q c hc, Matrix.one_mul]
```

Uniform comparison of the actual loss and model. Concatenation partitions the finite entry sum, so $|\binom{T'}{Y_0 Z}|_F^2 = |T'|_F^2 + |Y_0 Z|_F^2$ and $L = (|U|_F^2 + |W|_F^2)/2$. Set $N = |U|_F^2 + |T'|_F^2 + |Y_0 Z|_F^2$. If $M \geq 1$ bounds $|X|_F^2, |R|_F^2, |Q|_F^2$, the inequalities $|E+F|_F^2, |E-F|_F^2 \leq 2|E|_F^2 + 2|F|_F^2$ and matrix submultiplicativity give, with $u = |U|_F^2$, $v = |V|_F^2$, $w = |W|_F^2$,

$$w \leq 2Mu + 2Mv, \qquad v \leq 2Mw + 2M^2 u.$$

Hence for $K = 1 + 2M + 2M^2 > 0$,

$$\frac{N}{2K} \leq L \circ \mathrm{decode} \leq \frac{K}{2} N.$$

The factor $1/2$ from the original loss is retained on both sides.

```
lemma frobeniusSq_hcat {m n k : Nat} (A : RM m n) (B : RM m k) :
    frobeniusSq (MatrixCore.hcat A B : RM m (n+k)) =
      frobeniusSq A + frobeniusSq B := by
  simp only [frobeniusSq, MatrixCore.hcat, Fin.sum_univ_add,
    Fin.addCases_left, Fin.addCases_right, Finset.sum_add_distrib]

lemma frobeniusSq_vcat {m n k : Nat} (A : RM m k) (B : RM n k) :
    frobeniusSq (MatrixCore.vcat A B : RM (m+n) k) =
      frobeniusSq A + frobeniusSq B := by
  unfold frobeniusSq
  rw [Fin.sum_univ_add]
  change ∑ i, ∑ j, (MatrixCore.rowsTop (MatrixCore.vcat A B) i j)^2) +
    ∑ i, ∑ j, (MatrixCore.rowsBot (MatrixCore.vcat A B) i j)^2) = _
  exact congrArg₂ (fun x y : ℝ => x+y)
    (congrArg (fun M : RM m k => ∑ i, ∑ j, (M i j)^2)
      (MatrixCore.rowsTop_vcat A B))
    (congrArg (fun M : RM n k => ∑ i, ∑ j, (M i j)^2)
      (MatrixCore.rowsBot_vcat A B))

lemma coordinateEnergy_eq (c : Coordinates r alpha beta h p q) :
    coordinateEnergy c = frobeniusSq c.1 + frobeniusSq (reducedColumn c) := by
  simp only [coordinateEnergy, reducedColumn, frobeniusSq_vcat, add_assoc]

lemma parameterLoss_decode (c : Coordinates r alpha beta h p q)
    (hc : coordinateGuard c) :
    parameterLoss (decode c) =
      (frobeniusSq c.1 + frobeniusSq (reconstructedRight c)) / 2 := by
  rw [parameterLoss, parameterError_decode c hc, frobeniusSq_hcat]

lemma frobeniusSq_add_le {m n : Nat} (A B : RM m n) :
    frobeniusSq (A+B) ≤ 2*frobeniusSq A + 2*frobeniusSq B := by
  calc
    frobeniusSq (A+B) ≤ ∑ i, ∑ j,
        (2*(A i j)^2 + 2*(B i j)^2) := by
      apply Finset.sum_le_sum
      intro i _
      apply Finset.sum_le_sum
      intro j _
      change (A i j+B i j)^2 ≤ _
      nlinarith [sq_nonneg (A i j-B i j)]
    _ = _ := by
      simp [frobeniusSq, Finset.sum_add_distrib, Finset.mul_sum]

lemma frobeniusSq_sub_le {m n : Nat} (A B : RM m n) :
    frobeniusSq (A-B) ≤ 2*frobeniusSq A + 2*frobeniusSq B := by
  calc
    frobeniusSq (A-B) ≤ ∑ i, ∑ j,
        (2*(A i j)^2 + 2*(B i j)^2) := by
      apply Finset.sum_le_sum
      intro i _
      apply Finset.sum_le_sum
      intro j _
      change (A i j-B i j)^2 ≤ _
      nlinarith [sq_nonneg (A i j+B i j)]
    _ = _ := by
      simp [frobeniusSq, Finset.sum_add_distrib, Finset.mul_sum]

def comparisonConstant (M : ℝ) : ℝ := 1+2*M+2*M^2

lemma comparisonConstant_pos {M : ℝ} (hM : 1 ≤ M) : 0 < comparisonConstant M := by
  dsimp [comparisonConstant]
  positivity
```

```
lemma loss_comparison_of_bounds (c : Coordinates r alpha beta h p q)
    (hc : coordinateGuard c) {M : ℝ} (hM : 1 ≤ M)
    (hX : frobeniusSq (coefficientX c) ≤ M)
    (hR : frobeniusSq (coefficientR c) ≤ M)
    (hQ : frobeniusSq (coefficientQ c) ≤ M) :
    coordinateEnergy c / (2*comparisonConstant M) ≤ parameterLoss (decode c) ∧
      parameterLoss (decode c) ≤ (comparisonConstant M / 2) * coordinateEnergy c := by
  let u := frobeniusSq c.1
  let w := frobeniusSq (reconstructedRight c)
  let v := frobeniusSq (reducedColumn c)
  have hu : 0 ≤ u := frobeniusSq_nonneg _
  have hw : 0 ≤ w := frobeniusSq_nonneg _
  have hv : 0 ≤ v := frobeniusSq_nonneg _
  have hM0 : 0 ≤ M := le_trans zero_le_one hM
  have hUX : frobeniusSq (c.1 * coefficientX c) ≤ u*M :=
    (frobeniusSq_mul_le _ _).trans (mul_le_mul_of_nonneg_left hX hu)
  have hQV : frobeniusSq (coefficientQ c * reducedColumn c) ≤ M*v :=
    (frobeniusSq_mul_le _ _).trans (mul_le_mul_of_nonneg_right hQ hv)
  have hwBound : w ≤ 2*M*u+2*M*v := by
    have hh := frobeniusSq_add_le (c.1 * coefficientX c)
      (coefficientQ c * reducedColumn c)
    change w ≤ 2 * frobeniusSq (c.1 * coefficientX c) +
      2 * frobeniusSq (coefficientQ c * reducedColumn c) at hh
    nlinarith
  have hvBound : v ≤ 2*M*w+2*M^2*u := by
    calc
      v = frobeniusSq (coefficientR c *
          (reconstructedRight c - c.1 * coefficientX c)) :=
        congrArg frobeniusSq (reducedColumn_reconstruct c hc)
      _ ≤ frobeniusSq (coefficientR c) *
          frobeniusSq (reconstructedRight c - c.1 * coefficientX c) :=
        frobeniusSq_mul_le _ _
      _ ≤ M * (2*w + 2*(u*M)) := by
        apply mul_le_mul hR
        · exact (frobeniusSq_sub_le _ _).trans (by dsimp [w]; linarith [hUX])
        · exact frobeniusSq_nonneg _
        · exact hM0
      _ = 2*M*w+2*M^2*u := by ring
  have hforward : u+v ≤ comparisonConstant M * (u+w) := by
    dsimp [comparisonConstant]
    nlinarith [mul_nonneg hM0 hu, mul_nonneg (sq_nonneg M) hw]
  have hbackward : u+w ≤ comparisonConstant M * (u+v) := by
    dsimp [comparisonConstant]
    nlinarith [mul_nonneg (sq_nonneg M) hu, mul_nonneg (sq_nonneg M) hv]
  rw [coordinateEnergy_eq, parameterLoss_decode c hc]
  change (u+v)/(2*comparisonConstant M) ≤ (u+w)/2 ∧
    (u+w)/2 ≤ (comparisonConstant M/2)*(u+v)
  constructor
  · apply (₀div_le_iff (mul_pos (by norm_num) (comparisonConstant_pos hM))).2
    nlinarith [hforward]
  · nlinarith [hbackward]
```

Fixing the neighborhood before the band width. At a guarded point c_0, take $M = 1 + |X(c_0)|_F^2 + |R(c_0)|_F^2 + |Q(c_0)|_F^2$. Each of the three values is strictly below M. Continuity provides three neighborhood bounds; intersect them with the determinant-open target and choose an open neighborhood contained in the intersection. On this single neighborhood the preceding comparison holds with one fixed positive K. The theorem `local_loss_comparison` chooses this neighborhood and constant before any radius or ε appears.

```
section LocalBounds
open scoped Matrix.Norms.Elementwise
attribute [local fun_prop] analyticAt_fst analyticAt_snd

lemma coefficientX_analyticAt (c : Coordinates r alpha beta h p q) :
    AnalyticAt ℝ coefficientX c := by
  unfold coefficientX
  fun_prop

lemma coefficientQ_analyticAt (c : Coordinates r alpha beta h p q) :
    AnalyticAt ℝ coefficientQ c := by
  unfold coefficientQ
  fun_prop

lemma coefficientR_analyticAt (c : Coordinates r alpha beta h p q)
    (hc : coordinateGuard c) : AnalyticAt ℝ coefficientR c := by
  have hD : (pivotTop c.2.2.2.2.2.2.1).det ≠ 0 := hc.2
  unfold coefficientR
  fun_prop (disch := assumption)

lemma frobeniusSq_continuous : Continuous (frobeniusSq (m := Fin (r+alpha))
    (n := Fin q)) := by
  unfold frobeniusSq
  fun_prop

lemma continuousAt_frobeniusSq {E : Type*} [TopologicalSpace E]
    {m n : Nat} {f : E → RM m n} {x : E} (hf : ContinuousAt f x) :
    ContinuousAt (fun y => frobeniusSq (f y)) x := by
  have hs : Continuous (frobeniusSq (m := Fin m) (n := Fin n)) := by
    unfold frobeniusSq
    fun_prop
  exact hs.continuousAt.comp hf

/-- One open neighbourhood and one bound are fixed before any band width is introduced. -/
theorem local_loss_comparison (c0 : Coordinates r alpha beta h p q)
    (hc0 : coordinateGuard c0) :
    ∃ (N : Set (Coordinates r alpha beta h p q)) (K : ℝ),
      IsOpen N ∧ c0 ∈ N ∧ N ⊆ targetDomain ∧ 0 < K ∧
        ∀ c ∈ N, coordinateEnergy c / (2*K) ≤ parameterLoss (decode c) ∧
          parameterLoss (decode c) ≤ (K/2) * coordinateEnergy c := by
  let M := 1 + frobeniusSq (coefficientX c0) +
    frobeniusSq (coefficientR c0) + frobeniusSq (coefficientQ c0)
  have hx0 := frobeniusSq_nonneg (coefficientX c0)
  have hr0 := frobeniusSq_nonneg (coefficientR c0)
  have hq0 := frobeniusSq_nonneg (coefficientQ c0)
  have hM : 1 ≤ M := by dsimp [M]; linarith
  have hxM : frobeniusSq (coefficientX c0) < M := by dsimp [M]; linarith
  have hrM : frobeniusSq (coefficientR c0) < M := by dsimp [M]; linarith
  have hqM : frobeniusSq (coefficientQ c0) < M := by dsimp [M]; linarith
  have hx := (continuousAt_frobeniusSq
    (coefficientX_analyticAt c0).continuousAt).eventually_lt continuousAt_const hxM
  have hr := (continuousAt_frobeniusSq
    (coefficientR_analyticAt c0 hc0).continuousAt).eventually_lt continuousAt_const hrM
  have hq := (continuousAt_frobeniusSq
    (coefficientQ_analyticAt c0).continuousAt).eventually_lt continuousAt_const hqM
  have hguard : targetDomain ∈ nhds c0 := targetDomain_isOpen.mem_nhds hc0
  have hall := Filter.Eventually.and hguard (hx.and (hr.and hq))
  rcases mem_nhds_iff.mp hall with ⟨N, hN, hOpen, ⟩hcN
  refine ⟨N, comparisonConstant M, hOpen, hcN, ?_, comparisonConstant_pos hM, ?⟩_
  · intro c hc
    exact (hN hc).1
  · intro c hc
```

```
    have hh := hN hc
    exact loss_comparison_of_bounds c hh.1 hM hh.2.1.le hh.2.2.1.le hh.2.2.2.le

end LocalBounds
end O77.Reconstruction0905
end
set_option autoImplicit false
noncomputable section
open scoped BigOperators Matrix.Norms.Elementwise
namespace O77.Reconstruction0905
variable {r alpha beta h p q : Nat}
```

Centering without changing the model energy. Translate the coordinate origin by c_0: the centered encoder is $w \mapsto \mathrm{encode}(w) - c_0$ and the decoder is $z \mapsto \mathrm{decode}(z + c_0)$. Their inverse and analytic properties follow by composition with translations. For the model energy to remain $N(z)$, the first four blocks of c_0 must vanish individually: $U_0 = T_0' = Y_{0,0} = Z_0 = 0$. The theorem assumes and uses these four equations; merely assuming $Y_{0,0}Z_0 = 0$ would not justify the translation identity. The canonical tuple $(0, 0, 0, 0, 1, 0, 0, D_0, 0)$ satisfies these equations, and its output pivot is exactly the identity, so both determinants are nonzero even in empty dimensions. This proves the canonical centered chart with its uniform loss bound. Volume transport is established in the following modules.

```
def centeredEncode (c0 : Coordinates r alpha beta h p q)
    (w : Raw r alpha beta h p q) : Coordinates r alpha beta h p q := encode w - c0

def centeredDecode (c0 z : Coordinates r alpha beta h p q) : Raw r alpha beta h p q :=
  decode (z+c0)

def centeredTarget (c0 : Coordinates r alpha beta h p q) :
    Set (Coordinates r alpha beta h p q) := {z | z+c0 ∈ targetDomain}

lemma centeredDecode_encode (c0 : Coordinates r alpha beta h p q)
    (w : Raw r alpha beta h p q) (hw : rawGuard w) :
    centeredDecode c0 (centeredEncode c0 w) = w := by
  simpa only [centeredDecode, centeredEncode, sub_add_cancel] using decode_encode w hw

lemma centeredEncode_decode (c0 z : Coordinates r alpha beta h p q)
    (hz : z ∈ centeredTarget c0) : centeredEncode c0 (centeredDecode c0 z) = z := by
  change encode (decode (z+c0)) - c0 = z
  rw [encode_decode (z+c0) hz, add_sub_cancel_right]

lemma centeredTarget_isOpen (c0 : Coordinates r alpha beta h p q) :
    IsOpen (centeredTarget c0) :=
  targetDomain_isOpen.preimage (continuous_id.add continuous_const)

lemma centeredEncode_analyticAt (c0 : Coordinates r alpha beta h p q)
    (w : Raw r alpha beta h p q) (hw : rawGuard w) :
    AnalyticAt ℝ (centeredEncode c0) w :=
  (encode_analyticAt w hw).sub analyticAt_const

lemma centeredDecode_analyticAt (c0 z : Coordinates r alpha beta h p q)
    (hz : z ∈ centeredTarget c0) : AnalyticAt ℝ (centeredDecode c0) z := by
  have ht : AnalyticAt ℝ (fun y : Coordinates r alpha beta h p q => y+c0) z :=
    analyticAt_id.add analyticAt_const
  exact AnalyticAt.comp (f := fun y : Coordinates r alpha beta h p q => y+c0)
    (x := z) (decode_analyticAt (z+c0) hz) ht

def centeredElimination (c0 : Coordinates r alpha beta h p q) :
    OpenPartialHomeomorph (Raw r alpha beta h p q) (Coordinates r alpha beta h p q) where
  toFun := centeredEncode c0
```

```
  invFun := centeredDecode c0
  source := sourceDomain
  target := centeredTarget c0
  map_source' := by
    intro w hw
    change coordinateGuard (encode w - c0 + c0)
    simpa only [sub_add_cancel] using (encode_guard w).2 hw
  map_target' := fun z hz => (decode_guard (z+c0)).2 hz
  left_inv' := centeredDecode_encode c0
  right_inv' := centeredEncode_decode c0
  open_source := sourceDomain_isOpen
  open_target := centeredTarget_isOpen c0
  continuousOn_toFun := fun w hw =>
    (centeredEncode_analyticAt c0 w hw).continuousAt.continuousWithinAt
  continuousOn_invFun := fun z hz =>
    (centeredDecode_analyticAt c0 z hz).continuousAt.continuousWithinAt

lemma centered_base (c0 : Coordinates r alpha beta h p q) (hc0 : coordinateGuard c0) :
    centeredEncode c0 (decode c0) = 0 ∧ centeredDecode c0 0 = decode c0 := by
  simp only [centeredEncode, centeredDecode, encode_decode c0 hc0, sub_self, zero_add,
    and_self]

lemma coordinateEnergy_translate (c0 : Coordinates r alpha beta h p q)
    (hU : c0.1 = 0) (hT : c0.2.1 = 0)
    (hY : c0.2.2.1 = 0) (hZ : c0.2.2.2.1 = 0)
    (z : Coordinates r alpha beta h p q) :
    coordinateEnergy (z+c0) = coordinateEnergy z := by
  simp only [coordinateEnergy, Prod.fst_add, Prod.snd_add, hU, hT, hY, hZ, add_zero]

/-- The actual canonical-chart acceptance gate: analytic inverse maps and one fixed
open neighbourhood carrying the Frobenius comparison. The first four base blocks
are explicitly zero; no claim about volume transport is hidden in this theorem. -/
theorem centered_chart_with_uniform_bound
    (c0 : Coordinates r alpha beta h p q) (hc0 : coordinateGuard c0)
    (hU : c0.1 = 0) (hT : c0.2.1 = 0)
    (hY : c0.2.2.1 = 0) (hZ : c0.2.2.2.1 = 0) :
    ∃ (N : Set (Coordinates r alpha beta h p q)) (K : ℝ),
      IsOpen N ∧ (0 : Coordinates r alpha beta h p q) ∈ N ∧
      N ⊆ centeredTarget c0 ∧ 0 < K ∧
      ∀ z ∈ N,
        AnalyticAt ℝ (centeredDecode c0) z ∧
        AnalyticAt ℝ (centeredEncode c0) (centeredDecode c0 z) ∧
        centeredEncode c0 (centeredDecode c0 z) = z ∧
        coordinateEnergy z / (2*K) ≤ parameterLoss (centeredDecode c0 z) ∧
        parameterLoss (centeredDecode c0 z) ≤ (K/2)*coordinateEnergy z := by
  rcases local_loss_comparison c0 hc0 with ⟨W, K, hW, hcW, hWT, hK, ⟩hBound
  refine ⟨{z | z+c0 ∈ W}, K, hW.preimage (continuous_id.add continuous_const), ?_, ?_,
    hK, ?⟩_
  · simpa only [Set.mem_ofPred_eq, zero_add] using hcW
  · intro z hz
    exact hWT hz
  · intro z hz
    have hzt : z ∈ centeredTarget c0 := hWT hz
    have hb := hBound (z+c0) hz
    rw [coordinateEnergy_translate c0 hU hT hY hZ z] at hb
    exact ⟨centeredDecode_analyticAt c0 z hzt,
      centeredEncode_analyticAt c0 (centeredDecode c0 z) ((decode_guard (z+c0)).2 hzt),
      centeredEncode_decode c0 z hzt, ⟩hb

lemma pivotTop_canonicalD :
    pivotTop (MatrixCore.canonicalD (R := ℝ) r beta p) = (1 : RM (r+beta) (r+beta)) := by
  have ht := congrArg (MatrixCore.rowsTop (m := r+beta) (n := p))
```

```
    (MatrixCore.canonicalJ_D_reconstruct (R := ℝ) r beta p)
  exact ht.trans ((MatrixCore.rowsTop_vcat _ _).trans (core_one_eq (r+beta)))

def canonicalCoordinates (r alpha beta h p q : Nat) : Coordinates r alpha beta h p q :=
  (0, 0, 0, 0, 1, 0, 0, MatrixCore.canonicalD (R := ℝ) r beta p, 0)

lemma canonicalCoordinates_guard : coordinateGuard (canonicalCoordinates r alpha beta h p q) := by
  change (1 : RM (r+alpha) (r+alpha)).det ≠ 0 ∧
    (pivotTop (MatrixCore.canonicalD (R := ℝ) r beta p)).det ≠ 0
  rw [pivotTop_canonicalD, Matrix.det_one, Matrix.det_one]
  exact ⟨one_ne_zero, ⟩one_ne_zero

/-- Empty dimension tuples are included because all six dimensions are arbitrary naturals. -/
theorem canonical_centered_chart_with_uniform_bound :
    let c0 := canonicalCoordinates r alpha beta h p q
    ∃ (N : Set (Coordinates r alpha beta h p q)) (K : ℝ),
      IsOpen N ∧ (0 : Coordinates r alpha beta h p q) ∈ N ∧
      N ⊆ centeredTarget c0 ∧ 0 < K ∧
      ∀ z ∈ N,
        AnalyticAt ℝ (centeredDecode c0) z ∧
        AnalyticAt ℝ (centeredEncode c0) (centeredDecode c0 z) ∧
        centeredEncode c0 (centeredDecode c0 z) = z ∧
        coordinateEnergy z / (2*K) ≤ parameterLoss (centeredDecode c0 z) ∧
        parameterLoss (centeredDecode c0 z) ≤ (K/2)*coordinateEnergy z :=
  centered_chart_with_uniform_bound _ canonicalCoordinates_guard rfl rfl rfl rfl

end O77.Reconstruction0905
end
set_option autoImplicit false
noncomputable section
open scoped Matrix.Norms.Elementwise
namespace O77.Reconstruction0905
```

Reading the boundary declarations. The final declarations distinguish the pointwise reciprocal on a raw function type from the matrix inverse used above, reject a singular source pivot, and instantiate the chart in empty and rectangular dimensions. The reciprocal example is deliberately about a different expected type; it is not an equation used by the encoder or decoder.

```
/-- Deliberately ambiguous notation for regression only: the expected function type
selects pointwise reciprocal. The encoder and decoder never use this expression. -/
lemma expected_core_inverse_is_entrywise {n : Nat} (P : RM n n) (i j : Fin n) :
    (show MatrixCore.Mat n n ℝ from ⁻P¹) i j = (P i j)⁻¹ := rfl

lemma singular_source_rejected {r alpha beta h p q : Nat}
    (w : Raw r alpha beta h p q) (hP : w.1.det = 0) : ¬ rawGuard w :=
  fun hw => hw.1 hP

lemma empty_canonical_guard : coordinateGuard (canonicalCoordinates 0 0 0 0 0 0) :=
  canonicalCoordinates_guard

lemma empty_inverse_analytic (P : RM 0 0) :
    AnalyticAt ℝ (fun Q : RM 0 0 => ⁻Q¹) P :=
  entrywise_inverse_analyticAt analyticAt_id (by simp)

lemma rectangular_decode_analytic :
    AnalyticAt ℝ decode (canonicalCoordinates 1 2 1 3 2 4) :=
  decode_analyticAt _ canonicalCoordinates_guard

end O77.Reconstruction0905
end
```

32 Module `O77CanonicalRanks`

32.1 Ranks of the actual canonical matrices

The canonical entries come from the algebraic core, whereas the rank is Mathlib's dimension of the actual matrix image. The helper `core_rank_congr` transports equality of raw entry functions through `Matrix.of` before taking rank. For a vertical zero padding $V = \binom{A}{0}$, the inequality $\operatorname{rank} A \le \operatorname{rank} V$ follows by taking the upper submatrix and applying `Matrix.rank_submatrix_le`. The reverse inequality follows from $V = PA$, where P is the standard inclusion, and `Matrix.rank_mul_le_right`. Transposition proves the corresponding horizontal-padding statement using `Matrix.rank_transpose`. These arguments include empty rows or columns.

```
import Mathlib
import O77Analytic

/-! Actual ranks of the inherited rectangular canonical matrices.
Core matrices are raw functions, whereas Mathlib's Matrix is a semireducible
type. Transport core equalities explicitly through Matrix.of and rank; do not
rewrite mixed-carrier expressions at a weaker transparency level. -/
set_option autoImplicit false
noncomputable section
open O77.Reconstruction0905 (RM core_mul_eq core_one_eq)
namespace O77.CanonicalRanks

/-- Transport a raw core equality to the rank of the same matrix entries. -/
private lemma core_rank_congr {m n : ℕ} {A B : MatrixCore.Mat m n ℝ}
    (h : A = B) : Matrix.rank (Matrix.of A) = Matrix.rank (Matrix.of B) :=
  congrArg (fun M : MatrixCore.Mat m n ℝ => Matrix.rank (Matrix.of M)) h

/-- A normalization rule for the core's finite row concatenation. -/
@[simp] lemma vcat_zero (m n k : ℕ) :
    MatrixCore.vcat (0 : RM m k) (0 : RM n k) = 0 := by
  exact MatrixCore.vcat_rows (m := m) (n := n) (0 : RM (m+n) k)

/-- Padding adds no image dimension. The lower bound uses a submatrix;
the upper bound factors through the unpadded matrix. -/
lemma rank_vcat_zero {m n : ℕ} (A : RM m n) (k : ℕ) :
    Matrix.rank (MatrixCore.vcat A (0 : RM k n)) = Matrix.rank A := by
  let P : RM (m+k) m := Matrix.of (MatrixCore.canonicalPivot (R := ℝ) m k)
  let V : RM (m+k) n := Matrix.of
    (MatrixCore.vcat (fun i j => A i j) (fun _ _ => (0 : ℝ)))
  have hf : P * A = V :=
    (core_mul_eq P A).symm.trans (MatrixCore.canonicalPivot_mul m k n A)
  have hl : Matrix.rank A ≤ Matrix.rank V := by
    calc
      Matrix.rank A = Matrix.rank (V.submatrix (Fin.castAdd k) id) :=
        (core_rank_congr (MatrixCore.rowsTop_vcat (m := m) (n := k)
          (fun i j => A i j) (fun _ _ => (0 : ℝ)))).symm
      _ ≤ Matrix.rank V := Matrix.rank_submatrix_le V (Fin.castAdd k) id
  refine le_antisymm ?_ hl
  calc
    Matrix.rank V = Matrix.rank (P * A) :=
      congrArg (fun M : RM (m+k) n => Matrix.rank M) hf.symm
    _ ≤ Matrix.rank A := Matrix.rank_mul_le_right P A

/-- Transposition and the inherited projection/reconstruction identities
give the column version without a second rank argument or Fin reduction. -/
lemma rank_hcat_zero {m n : ℕ} (A : RM m n) (k : ℕ) :
    Matrix.rank (MatrixCore.hcat A (0 : RM m k)) = Matrix.rank A := by
  let C : RM m (n+k) := Matrix.of
    (MatrixCore.hcat (fun i j => A i j) (fun _ _ => (0 : ℝ)))
```

```
  have ht : Matrix.transpose C = Matrix.of
      (MatrixCore.vcat (fun i j => A j i) (fun _ _ => (0 : ℝ))) := by
    have hL : MatrixCore.rowsTop (m := n) (n := k) (fun i j => C j i) =
        (fun i j => A j i) :=
      congrArg (fun M : MatrixCore.Mat m n ℝ => fun i j => M j i)
        (MatrixCore.colsLeft_hcat (m := m) (n := n) (k := k)
          (fun i j => A i j) (fun _ _ => (0 : ℝ)))
    have hR : MatrixCore.rowsBot (m := n) (n := k) (fun i j => C j i) =
        (fun _ _ => (0 : ℝ)) :=
      congrArg (fun M : MatrixCore.Mat m k ℝ => fun i j => M j i)
        (MatrixCore.colsRight_hcat (m := m) (n := n) (k := k)
          (fun i j => A i j) (fun _ _ => (0 : ℝ)))
    exact (MatrixCore.vcat_rows (m := n) (n := k) (fun i j => C j i)).symm.trans
      (congrArg (fun U : MatrixCore.Mat n m ℝ =>
        fun V : MatrixCore.Mat k m ℝ => MatrixCore.vcat U V) hL hR)
  calc
    Matrix.rank C = Matrix.rank (Matrix.transpose C) := (Matrix.rank_transpose C).symm
    _ = Matrix.rank (MatrixCore.vcat (Matrix.transpose A) (0 : RM k m)) :=
      congrArg (fun M : RM (n+k) m => Matrix.rank M) ht
    _ = Matrix.rank (Matrix.transpose A) := rank_vcat_zero (Matrix.transpose A) k
    _ = Matrix.rank A := Matrix.rank_transpose A
```

Identity minors and the canonical ranks. The padded identity $P_b = \binom{1_b}{0}$ has rank b by the padding lemma and `Matrix.rank_one`. The canonical A_0 is an identity of size $r+\alpha$ with zero rows and columns appended, hence $\operatorname{rank} A_0 = r+\alpha$. For J, the first r rows form an actual identity minor and there are exactly r columns, giving $\operatorname{rank} J = r$. For B_0, an explicit core section S satisfies $B_0 S = P_{r+\beta}$, so $\operatorname{rank} B_0 \geq r+\beta$; its last p rows are zero, so the padding result gives the reverse inequality. Finally $\Phi_0 = (J\ 0\ 0)$ has rank r. Thus the ranks used by the measured canonical theorem refer to its actual matrices, not to labels chosen to match the proposed formula.

```
lemma rank_canonicalPivot (b p : ℕ) :
    Matrix.rank (MatrixCore.canonicalPivot (R := ℝ) b p) = b := by
  calc
    Matrix.rank (MatrixCore.canonicalPivot (R := ℝ) b p) =
        Matrix.rank (Matrix.of (MatrixCore.one (R := ℝ) b)) :=
      rank_vcat_zero (Matrix.of (MatrixCore.one (R := ℝ) b)) p
    _ = Matrix.rank (1 : RM b b) := core_rank_congr (core_one_eq b)
    _ = b := by simp only [Matrix.rank_one, Fintype.card_fin]

lemma rank_canonicalA (r alpha beta h q : ℕ) :
    Matrix.rank (MatrixCore.canonicalA (R := ℝ) r alpha beta h q) = r + alpha := by
  have ha := congrArg
    (fun Z : MatrixCore.Mat ((r+alpha)+(beta+h)) q ℝ =>
      MatrixCore.hcat (MatrixCore.canonicalPivot (R := ℝ) (r+alpha) (beta+h)) Z)
    (vcat_zero (r+alpha) (beta+h) q)
  calc
    Matrix.rank (MatrixCore.canonicalA (R := ℝ) r alpha beta h q) =
        Matrix.rank (MatrixCore.hcat
          (MatrixCore.canonicalPivot (R := ℝ) (r+alpha) (beta+h))
          (MatrixCore.zero ((r+alpha)+(beta+h)) q)) := core_rank_congr ha
    _ = Matrix.rank (MatrixCore.canonicalPivot (R := ℝ) (r+alpha) (beta+h)) :=
      rank_hcat_zero (Matrix.of (MatrixCore.canonicalPivot (R := ℝ)
        (r+alpha) (beta+h))) q
    _ = r+alpha := rank_canonicalPivot (r+alpha) (beta+h)

/-- The first r rows of J form an actual identity minor, even when r=0. -/
lemma rank_canonicalJ (r beta p : ℕ) :
    Matrix.rank (MatrixCore.canonicalJ (R := ℝ) r beta p) = r := by
  let J : RM ((r+beta)+p) r := Matrix.of (MatrixCore.canonicalJ r beta p)
```

```
  let rows : Fin r → Fin ((r+beta)+p) := fun i => (i.castAdd beta).castAdd p
  have hm : J.submatrix rows id = (1 : RM r r) := by
    funext i j
    have he := congrFun (congrFun
      (MatrixCore.rowsTop_vcat (MatrixCore.one (R := ℝ) (r+beta))
        (MatrixCore.zero p (r+beta))) (i.castAdd beta)) (j.castAdd beta)
    have hij : i.castAdd beta = j.castAdd beta ↔ i = j := by
      constructor
      · intro hij
        exact Fin.ext (congrArg (fun t : Fin (r+beta) => t.val) hij)
      · intro hij
        exact congrArg (Fin.castAdd beta) hij
    have hdiag : MatrixCore.one (R := ℝ) (r+beta) (i.castAdd beta) (j.castAdd beta) =
        (1 : RM r r) i j := by
      change (if i.castAdd beta = j.castAdd beta then (1 : ℝ) else 0) =
        (if i = j then 1 else 0)
      simp only [hij]
    exact he.trans hdiag
  have hl := Matrix.rank_submatrix_le J rows id
  rw [hm, Matrix.rank_one, Fintype.card_fin] at hl
  exact le_antisymm (Matrix.rank_le_width J) hl

lemma rank_canonicalB (r alpha beta h p : ℕ) :
    Matrix.rank (MatrixCore.canonicalB (R := ℝ) r alpha beta h p) = r + beta := by
  let B : RM ((r+beta)+p) ((r+alpha)+(beta+h)) :=
    Matrix.of (MatrixCore.canonicalB r alpha beta h p)
  let S : RM ((r+alpha)+(beta+h)) (r+beta) :=
    Matrix.of (MatrixCore.canonicalSection r alpha beta h)
  have hs : B * S = MatrixCore.canonicalPivot (R := ℝ) (r+beta) p :=
    (core_mul_eq B S).symm.trans
      (MatrixCore.canonicalB_section r alpha beta h p)
  have hl : r+beta ≤ Matrix.rank B := by
    calc
      r+beta = Matrix.rank (MatrixCore.canonicalPivot (R := ℝ) (r+beta) p) :=
        (rank_canonicalPivot (r+beta) p).symm
      _ = Matrix.rank (B * S) :=
        congrArg (fun M : RM ((r+beta)+p) (r+beta) => Matrix.rank M) hs.symm
      _ ≤ Matrix.rank B := Matrix.rank_mul_le_left B S
  let U : RM (r+beta) ((r+alpha)+(beta+h)) :=
    Matrix.of (MatrixCore.rowsTop (m := r+beta) (n := p) (fun i j => B i j))
  have hb : B = Matrix.of
      (MatrixCore.vcat (fun i j => U i j) (fun _ _ => (0 : ℝ))) := by
    exact (MatrixCore.vcat_rows (m := r+beta) (n := p) (fun i j => B i j)).symm.trans
      (congrArg (fun Z : MatrixCore.Mat p ((r+alpha)+(beta+h)) ℝ =>
        MatrixCore.vcat (fun i j => U i j) Z)
        (MatrixCore.rowsBot_canonicalB r alpha beta h p))
  refine le_antisymm ?_ hl
  calc
    Matrix.rank B = Matrix.rank (MatrixCore.vcat U (0 : RM p ((r+alpha)+(beta+h)))) :=
      congrArg (fun M : RM ((r+beta)+p) ((r+alpha)+(beta+h)) => Matrix.rank M) hb
    _ = Matrix.rank U := rank_vcat_zero U p
    _ ≤ r+beta := Matrix.rank_le_height U

lemma rank_canonicalTarget (r alpha beta p q : ℕ) :
    Matrix.rank (MatrixCore.canonicalTarget (R := ℝ) r alpha beta p q) = r := by
  exact (rank_hcat_zero (Matrix.of (MatrixCore.canonicalCI (R := ℝ) r alpha beta p)) q).trans
    ((rank_hcat_zero (Matrix.of (MatrixCore.canonicalJ (R := ℝ) r beta p)) alpha).trans
      (rank_canonicalJ r beta p))

end O77.CanonicalRanks
end
```

33 Module `O77Transport`

33.1 From entries and Jacobians to measured bands

For a matrix indexed by finite types m, n, the flattening $M \mapsto ((i,j) \mapsto M_{ij})$ simply enumerates entries. Its sum of squared coordinates is $|M|_F^2$. To prove exact measure preservation, evaluate the pushforward of the nested product of one-dimensional Lebesgue measures on a measurable rectangle $\prod_{(i,j)} S_{ij}$. Its preimage is $\prod_i \prod_j S_{ij}$, whose measure is the identical finite product. The uniqueness theorem `Measure.pi_eq`, together with `Measure.pi_pi`, therefore identifies the measures exactly. Concatenating two entry lists and relabeling a finite index set use `measurePreserving_sumPiEquivProdPi_symm` and `measurePreserving_piCongrLeft`. In particular, the empty entry space has total mass one. None of these maps is a nonorthogonal basis change.

```
import O77
import O77Analytic
import Mathlib.MeasureTheory.Function.Jacobian
import Mathlib.MeasureTheory.Constructions.Pi

set_option autoImplicit false
noncomputable section
open Set Filter MeasureTheory
open scoped Topology ENNReal BigOperators

namespace O77.Transport0906

section EntryMeasures
variable {m n : Type*} [Fintype m] [Fintype n]

-- Flattening is entry reindexing, not a matrix multiplication.
def matrixEntries : Matrix m n ℝ ≃ₗ[ℝ] ((m × n) → ℝ) where
  toFun A ij := A ij.1 ij.2
  invFun x i j := x (i,j)
  left_inv A := rfl
  right_inv x := by funext ij; cases ij; rfl
  map_add' A B := rfl
  map_smul' c A := rfl

lemma matrixEntries_sq (A : Matrix m n ℝ) :
    ∑( ij : m × n, (matrixEntries A ij)^2) =
      O77.Residual.frobeniusSq A := by
  change ∑( ij : m × n, (A ij.1 ij.2)^2) = ∑ i, ∑ j, (A i j)^2
  exact Fintype.sum_prod_type _

def matrixEntriesMeasurable : Matrix m n ℝ ≃ᵐ ((m × n) → ℝ) where
  toEquiv := matrixEntries.toEquiv
  measurable_toFun := by
    apply Measurable.of_eval
    intro ij
    exact (measurable_pi_apply ij.2).comp (measurable_pi_apply ij.1)
  measurable_invFun := by
    apply Measurable.of_eval
    intro i
    apply Measurable.of_eval
    intro j
    exact measurable_pi_apply (i,j)

-- Rectangle uniqueness proves the normalization, not just proportionality.
theorem matrixEntries_measurePreserving :
    MeasurePreserving (matrixEntriesMeasurable (m := m) (n := n))
      (O77.Residual.matrixLebesgue m n)
```

```
      (O77.Residual.coordinateLebesgue (m × n)) := by
  refine (matrixEntriesMeasurable.measurable, ?)_
  unfold O77.Residual.coordinateLebesgue
  refine (Measure.pi_eq fun s hs => ?_).symm
  rw [Measure.map_apply matrixEntriesMeasurable.measurable
    (MeasurableSet.univ_pi hs)]
  have hpre : matrixEntriesMeasurable ⁻¹' Set.univ.pi s =
      Set.univ.pi (fun i => Set.univ.pi (fun j => s (i,j))) := by
    ext A
    change (fun ij : m × n => A ij.1 ij.2) ∈ Set.univ.pi s ↔
      A ∈ Set.univ.pi (fun i => Set.univ.pi (fun j => s (i,j)))
    constructor
    · intro h i _ j _
      exact h (i,j) (Set.mem_univ _)
    · intro h ij _
      exact h ij.1 (Set.mem_univ _) ij.2 (Set.mem_univ _)
  rw [hpre]
  change (Measure.pi fun _ : m => Measure.pi fun _ : n => (volume : Measure ℝ))
    (Set.univ.pi (fun i => Set.univ.pi (fun j => s (i,j)))) = _
  simp_rw [Measure.pi_pi]
  exact (Fintype.prod_prod_type (fun ij : m × n => volume (s ij))).symm

variable ι{ κ : Type*} [Fintype ι] [Fintype κ]

-- The inverse sum equivalence concatenates two coordinate vectors.
lemma joinEntries_measurePreserving :
    MeasurePreserving
      (MeasurableEquiv.sumPiEquivProdPi (fun _ : ι ⊕ κ => ℝ)).symm
      ((O77.Residual.coordinateLebesgue ι).prod
        (O77.Residual.coordinateLebesgue κ))
      (O77.Residual.coordinateLebesgue ι( ⊕ κ)) := by
  exact measurePreserving_sumPiEquivProdPi_symm (fun _ => volume)

lemma reindexEntries_measurePreserving (e : ι ≃ κ) :
    MeasurePreserving (MeasurableEquiv.piCongrLeft (fun _ : κ => ℝ) e)
      (O77.Residual.coordinateLebesgue ι)
      (O77.Residual.coordinateLebesgue κ) := by
  exact measurePreserving_piCongrLeft (fun _ => volume) e

lemma emptyEntryMass :
    O77.Residual.coordinateLebesgue (Fin 0) Set.univ = 1 := by
  simp [O77.Residual.coordinateLebesgue]
end EntryMeasures
```

Differentiating genuine local inverse identities. Let E, F be real normed vector spaces, let $x \in E$, and let $f : E \to F$, $g : F \to E$ have Fréchet derivatives $A : E \to F$ and $B : F \to E$ at x and $f(x)$, respectively; these derivatives are continuous linear maps. Assume that $g(f(u)) = u$ for every u in a neighborhood of x. The chain rule gives derivative BA for $g \circ f$, while its agreement with the identity on that neighborhood gives derivative 1. Uniqueness of derivatives proves $BA = 1$. The concrete encoder/decoder instances obtain the needed eventual equality from their open domains; equality merely at the center would not suffice. The maps here can have different source types, so no determinant is taken until the two coordinate spaces have been flattened into one common Euclidean space.

```
section InverseDerivatives
variable {E F : Type*}
  [NormedAddCommGroup E] [NormedSpace ℝ E]
  [NormedAddCommGroup F] [NormedSpace ℝ F]
  {f : E → F} {g : F → E} {x : E}
  {A : E →L[ℝ] F} {B : F →L[ℝ] E}
```

```
-- The identity is required on a neighbourhood, not merely at one point.
theorem derivative_left_inverse
    (hf : HasFDerivAt f A x) (hg : HasFDerivAt g B (f x))
    (hgf : ∀ᶠ u in nhds x, g (f u) = u) :
    B.comp A = ContinuousLinearMap.id ℝ E := by
  have hid : HasFDerivAt (g ∘ f) (ContinuousLinearMap.id ℝ E) x :=
    (hasFDerivAt_id x).congr_of_eventuallyEq hgf
  exact (hg.comp x hf).unique hid
end InverseDerivatives

section ConcreteChartDerivatives
open scoped Matrix.Norms.Elementwise
open O77.Reconstruction0905
variable {r alpha beta h p q : ℕ}

-- These maps have different source types; take no determinant here.
theorem decode_encode_derivative
    (w : Raw r alpha beta h p q) (hw : rawGuard w) :
    (fderiv ℝ decode (encode w)).comp (fderiv ℝ encode w) =
      ContinuousLinearMap.id ℝ (Raw r alpha beta h p q) := by
  apply derivative_left_inverse
    (encode_analyticAt w hw).differentiableAt.hasFDerivAt
    (decode_analyticAt (encode w)
      ((encode_guard w).2 hw)).differentiableAt.hasFDerivAt
  filter_upwards [sourceDomain_isOpen.mem_nhds hw] with u hu
  exact decode_encode u hu

theorem encode_decode_derivative
    (c : Coordinates r alpha beta h p q) (hc : coordinateGuard c) :
    (fderiv ℝ encode (decode c)).comp (fderiv ℝ decode c) =
      ContinuousLinearMap.id ℝ (Coordinates r alpha beta h p q) := by
  apply derivative_left_inverse
    (decode_analyticAt c hc).differentiableAt.hasFDerivAt
    (encode_analyticAt (decode c)
      ((decode_guard c).2 hc)).differentiableAt.hasFDerivAt
  filter_upwards [targetDomain_isOpen.mem_nhds hc] with z hz
  exact encode_decode z hz
end ConcreteChartDerivatives
```

Positive upper and lower Jacobian bounds. For continuous linear endomorphisms A, B of the same finite-dimensional real normed space, $BA = 1$ implies $\det B \det A = 1$ by `LinearMap.det_comp`, and hence $\det A \neq 0$. For the neighborhood assertion, let $A(y)$ be a field of continuous linear endomorphisms, continuous at a fixed point x, and suppose $\det A(x) \neq 0$. Continuity of $y \mapsto |\det A(y)|$ provides an open neighborhood of x on which

$$\tfrac{1}{2}|\det A(x)| \leq |\det A(y)| \leq 2|\det A(x)|.$$

The concrete chart later supplies both hypotheses; this lemma itself is only a reusable continuity argument.

```
section SquareDerivatives
variable {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E]

-- To apply this, conjugate the two chart types into the same entry space.
theorem det_ne_zero_of_left_inverse (A B : E →L[ℝ] E)
    (hBA : B.comp A = ContinuousLinearMap.id ℝ E) : A.det ≠ 0 := by
  have hdet := congrArg
    (fun T : E →L[ℝ] E => T.det) hBA
  have hmul : B.det * A.det = 1 := by
```

```
    simpa only [ContinuousLinearMap.det,
      ContinuousLinearMap.toLinearMap_comp, LinearMap.det_comp,
      ContinuousLinearMap.coe_id, LinearMap.det_id] using hdet
  intro hzero
  rw [hzero, mul_zero] at hmul
  exact zero_ne_one hmul

-- This neighbourhood bound needs continuity only at the base point.
theorem local_abs_det_bounds {A : E → E →L[ℝ] E} {x : E}
    (hA : ContinuousAt A x) (hdet : (A x).det ≠ 0) :
    ∃ U : Set E, IsOpen U ∧ x ∈ U ∧
      ∀ y ∈ U, |(A x).det|/2 ≤ |(A y).det| ∧
        |(A y).det| ≤ 2*|(A x).det| := by
  have hpos : 0 < |(A x).det| := abs_pos.mpr hdet
  have hJ : ContinuousAt (fun y => |(A y).det|) x :=
    (ContinuousLinearMap.continuous_det.continuousAt.comp hA).abs
  have hlo := continuousAt_const.eventually_lt hJ (by linarith :
    |(A x).det|/2 < |(A x).det|)
  have hhi := hJ.eventually_lt continuousAt_const (by linarith :
    |(A x).det| < 2*|(A x).det|)
  rcases mem_nhds_iff.mp (hlo.and hhi) with ⟨U, hU, hopen, ⟩hxU
  exact ⟨U, hopen, hxU, fun y hy => ⟨(hU hy).1.le, (hU hy).2.⟩⟩le
end SquareDerivatives
end O77.Transport0906
end
noncomputable section
open Set Filter MeasureTheory
open scoped Topology ENNReal
namespace O77.Transport0906
```

The actual change-of-variables inequality. Let E be a finite-dimensional real normed space with its Borel measurable structure, let μ be additive Haar measure on E, and let $V \subseteq E$. Let $g : E \to E$ be Fréchet differentiable at every point of V and injective on V. For real constants ℓ, u, assume $\ell \le |\det Dg(z)| \le u$ for every $z \in V$. For a measurable $S \subseteq V$, the library theorems `measurable_image_of_fderivWithin` and `lintegral_abs_det_fderiv_eq_addHaar_image` give

$$\mu(gS) = \int_S |\det Dg(z)|\, d\mu(z), \qquad \ell\,\mu(S) \le \mu(gS) \le u\,\mu(S),$$

with the displayed coefficients interpreted through `ENNReal.ofReal` in the code. Integrating constant bounds proves the inequalities directly in the nonnegative extended reals. Separately, any subset of a bounded ball has finite measure because a containing closed ball is compact. This finiteness fact is available before later conversion to real-valued volume.

```
section ImageMeasure
variable {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E]
  [FiniteDimensional ℝ E] [MeasurableSpace E] [BorelSpace E]
  μ( : Measure E) [Measure.IsAddHaarMeasure μ]
  {g : E → E} {Dg : E → E →L[ℝ] E} {V S : Set E}

-- No `toReal` occurs in this measure comparison.
theorem image_measure_bounds
    (hS : MeasurableSet S) (hSV : S ⊆ V)
    (hg : ∀ z ∈ V, HasFDerivAt g (Dg z) z) (hinj : InjOn g V)
    {lo hi : ℝ}
    (hJ : ∀ z ∈ V, lo ≤ |(Dg z).det| ∧ |(Dg z).det| ≤ hi) :
    MeasurableSet (g '' S) ∧
      ENNReal.ofReal lo * μ S ≤ μ (g '' S) ∧
      μ (g '' S) ≤ ENNReal.ofReal hi * μ S := by
```

```
  have hder : ∀ z ∈ S, HasFDerivWithinAt g (Dg z) S z :=
    fun z hz => (hg z (hSV hz)).hasFDerivWithinAt
  have hginj : InjOn g S := hinj.mono hSV
  have hmeas := measurable_image_of_fderivWithin hS hder hginj
  have hchange := lintegral_abs_det_fderiv_eq_addHaar_image μ hS hder hginj
  refine ⟨hmeas, ?_, ?⟩_
  · calc
      ENNReal.ofReal lo * μ S =
          ∫⁻ z in S, ENNReal.ofReal lo ∂μ := by simp
      _ ≤ ∫⁻ z in S, ENNReal.ofReal |(Dg z).det| ∂μ := by
        apply lintegral_mono_ae
        filter_upwards [ae_restrict_mem hS] with z hz
        exact ENNReal.ofReal_le_ofReal (hJ z (hSV hz)).1
      _ = μ (g '' S) := hchange
  · calc
      μ (g '' S) =
          ∫⁻ z in S, ENNReal.ofReal |(Dg z).det| ∂μ := hchange.symm
      _ ≤ ∫⁻ z in S, ENNReal.ofReal hi ∂μ := by
        apply lintegral_mono_ae
        filter_upwards [ae_restrict_mem hS] with z hz
        exact ENNReal.ofReal_le_ofReal (hJ z (hSV hz)).2
      _ = ENNReal.ofReal hi * μ S := by simp

omit [BorelSpace E] in
lemma bounded_window_mass_ne_top {x : E} {R : ℝ}
    (hS : S ⊆ Metric.ball x R) : μ S ≠ ⊤ := by
  apply ne_of_lt
  exact lt_of_le_of_lt
    (measure_mono (hS.trans Metric.ball_subset_closedBall))
    (isCompact_closedBall x R).measure_lt_top
end ImageMeasure
```

Which band set is transported. Let E, F now be arbitrary sets, let $f : E \to F$, $g : F \to E$, let $\Omega \subseteq E$, and let $L : E \to \mathbb{R}$. For any $c, \epsilon \in \mathbb{R}$, the identity $g \circ f = 1$ on Ω implies the exact set equality

$$g\{z \in f(\Omega) : |L(gz) - c| < \epsilon\} = \{x \in \Omega : |L(x) - c| < \epsilon\}.$$

For the left-to-right inclusion write $z = f(u)$ with $u \in \Omega$; then $g(z) = u$ and the asserted band inequality is unchanged. For the converse choose $z = f(x)$ and use $g(f(x)) = x$. If $V \subseteq F$ and $f \circ g = 1$ on V, applying f to $g(z) = g(z')$ for $z, z' \in V$ gives $z = z'$, proving injectivity there. These are set-theoretic consequences of the stated inverse identities.

```
section SetTransport
variable {E F : Type*} {f : E → F} {g : F → E}
  Ω{ : Set E} {loss : E → ℝ}

-- Exactly the set whose inverse Jacobian must be integrated.
lemma inverse_image_band (hgf : ∀ x ∈ Ω, g (f x) = x)
    (level eps : ℝ) :
    g '' {z | z ∈ f '' Ω ∧ |loss (g z)-level| < eps} =
      {x | x ∈ Ω ∧ |loss x-level| < eps} := by
  ext x
  constructor
  · rintro ⟨z, ⟨⟨u, hu, ⟩rfl, ⟩hz, ⟩rfl
    change g (f u) ∈ Ω ∧ |loss (g (f u))-level| < eps
    exact ⟨(hgf u hu).symm ▸ hu, ⟩hz
  · rintro ⟨hx, ⟩hband
    refine ⟨f x, ⟨⟨x, hx, ⟩rfl, ?⟩_, hgf x ⟩hx
    simpa only [hgf x hx] using hband
```

```
lemma inverse_injOn {V : Set F}
    (hfg : ∀ z ∈ V, f (g z) = z) : InjOn g V := by
  intro z hz z' hz' heq
  calc
    z = f (g z) := (hfg z hz).symm
    _ = f (g z') := congrArg f heq
    _ = z' := hfg z' hz'
end SetTransport
```

Inner and outer windows. Return to the finite-dimensional measure setting, retaining the Fréchet derivative and Jacobian bounds for $g : E \to E$ on $V \subseteq E$ from the preceding assertion. Let $U \subseteq E$ be the original window and let $f : E \to E$ be its coordinate map. Let $L, N : E \to \mathbb{R}$ be scalar functions, with N measurable. Let $W_-, W_+ \subseteq V$ be measurable coordinate windows such that $g(W_-) \subseteq U$ and $f(U) \subseteq W_+$. Assume $g \circ f = 1$ on U and $f \circ g = 1$ on V. For constants $a, b > 0$, assume $L(gz) \le bN(z)$ on W_- and $aN(z) \le L(gz)$ on W_+. Then, for every $\varepsilon \in \mathbb{R}$,

$$g\{z \in W_- : N(z) < \varepsilon/b\} \subseteq \{x \in U : L(x) < \varepsilon\} \subseteq g\{z \in W_+ : N(z) < \varepsilon/a\}.$$

The right inclusion uses the specific preimage $f(x)$ and its inverse identity. Apply the preceding Jacobian bounds to the two measurable coordinate sublevels to obtain `sublevel_window_sandwich`. The strict inequalities are preserved by division only because a, b are strictly positive.

```
section SublevelTransport
variable {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E]
  [FiniteDimensional ℝ E] [MeasurableSpace E] [BorelSpace E]
  μ( : Measure E) [Measure.IsAddHaarMeasure μ]
  {f g : E → E} {Dg : E → E →L[ℝ] E}
  {U V inner outer : Set E} {loss model : E → ℝ}

-- This is a concrete measure inequality, not a pair supplied as a field.
theorem sublevel_window_sandwich
    (hinner : MeasurableSet inner) (houter : MeasurableSet outer)
    (hmodel : Measurable model) (hiV : inner ⊆ V) (hoV : outer ⊆ V)
    (hg : ∀ z ∈ V, HasFDerivAt g (Dg z) z)
    (hfg : ∀ z ∈ V, f (g z) = z)
    (hgf : ∀ x ∈ U, g (f x) = x)
    (hgi : MapsTo g inner U) (hfo : MapsTo f U outer)
    {lo hi a b : ℝ}
    (ha : 0 < a) (hb : 0 < b)
    (hJ : ∀ z ∈ V, lo ≤ |(Dg z).det| ∧ |(Dg z).det| ≤ hi)
    (hupper : ∀ z ∈ inner, loss (g z) ≤ b * model z)
    (hlower : ∀ z ∈ outer, a * model z ≤ loss (g z))
    (eps : ℝ) :
    ENNReal.ofReal lo * μ {z | z ∈ inner ∧ model z < eps/b} ≤
      μ {x | x ∈ U ∧ loss x < eps} ∧
    μ {x | x ∈ U ∧ loss x < eps} ≤
      ENNReal.ofReal hi * μ {z | z ∈ outer ∧ model z < eps/a} := by
  let Sin : Set E := {z | z ∈ inner ∧ model z < eps/b}
  let Sout : Set E := {z | z ∈ outer ∧ model z < eps/a}
  have hSin : MeasurableSet Sin :=
    hinner.inter (hmodel measurableSet_Iio)
  have hSout : MeasurableSet Sout :=
    houter.inter (hmodel measurableSet_Iio)
  have hSinV : Sin ⊆ V := fun z hz => hiV hz.1
  have hSoutV : Sout ⊆ V := fun z hz => hoV hz.1
  have hinj : InjOn g V := inverse_injOn hfg
  have hIn := image_measure_bounds μ hSin hSinV hg hinj hJ
  have hOut := image_measure_bounds μ hSout hSoutV hg hinj hJ
  have hlowset : g '' Sin ⊆ {x | x ∈ U ∧ loss x < eps} := by
```

```
    rintro x ⟨z, hz, ⟩rfl
    refine ⟨hgi hz.1, (hupper z hz.1).trans_lt ?⟩_
    have hh : model z * b < eps := (₀lt_div_iff hb).1 hz.2
    simpa only [mul_comm] using hh
  have huppset : {x | x ∈ U ∧ loss x < eps} ⊆ g '' Sout := by
    intro x hx
    refine ⟨f x, ⟨hfo hx.1, ?⟩_, hgf x hx⟩.1
    apply (₀lt_div_iff ha).2
    have hh := hlower (f x) (hfo hx.1)
    rw [hgf x hx.1] at hh
    simpa only [mul_comm] using hh.trans_lt hx.2
  exact ⟨hIn.2.1.trans (measure_mono hlowset),
    (measure_mono huppset).trans hOut⟩.2.2
end SublevelTransport
end O77.Transport0906
end
noncomputable section
open Set Filter MeasureTheory
open scoped Topology ENNReal
namespace O77.Transport0906
open O77.Residual (EThetaZeroPowerLog powerLogZero)
```

Preserving a power–log order and its radius quantifier. Fix $\lambda \in \mathbb{R}$ and $k \in \mathbb{N}$. Write $P_{\lambda,k}(\varepsilon) = \varepsilon^{\lambda}(\log(1/\varepsilon))^{k}$ for $0 < \varepsilon < e^{-1}$. The predicate `EThetaZeroPowerLog` means bounds between positive constant multiples of this function for every sufficiently small positive ε. If $\ell A(\varepsilon) \le B(\varepsilon) \le uC(\varepsilon)$, with $\ell, u > 0$ and both A, C of order $P_{\lambda,k}$, intersect their cutoffs and multiply their constants. Taking the minimum of the proposed lower and upper constants enforces the convention $c \le C$. The definition `RadiusOrder` adds $\lambda \ge 0$, $m \ge 1$, and $\exists \rho > 0\ \forall \delta \in (0, \rho)$ with logarithmic degree $m - 1$. For radius distortion $\delta/L, L\delta$ with $L \ge 1$, choose the new radius bound $\min(\rho, r/L)$ before fixing δ. Both auxiliary radii then lie below the old bound r, so the two fixed-radius estimates apply. The next module also treats distortion of ε.

```
-- Same-epsilon sandwich, useful for the exact pullback before loss comparison.
theorem order_of_two_window_bounds
    {A B C : ℝ → ENNReal} {lam lo hi : ℝ} {k : ℕ}
    (hA : EThetaZeroPowerLog A lam k)
    (hC : EThetaZeroPowerLog C lam k)
    (hlo : 0 < lo) (hhi : 0 < hi)
    (hB : ∀ eps, 0 < eps →
      ENNReal.ofReal lo * A eps ≤ B eps ∧
      B eps ≤ ENNReal.ofReal hi * C eps) :
    EThetaZeroPowerLog B lam k := by
  rcases hA with ⟨a, A0, ea, ha, haA, hea, heaexp, ⟩hAb
  rcases hC with ⟨c, C0, ec, hc, hcC, hec, hecexp, ⟩hCb
  let low := min (lo*a) (hi*C0)
  let high := hi*C0
  let e0 := min ea ec
  have hC0 : 0 < C0 := hc.trans_le hcC
  have hlow : 0 < low := lt_min (mul_pos hlo ha) (mul_pos hhi hC0)
  have hlowhigh : low ≤ high := min_le_right _ _
  have he0 : 0 < e0 := lt_min hea hec
  have he0exp : e0 < Real.exp (-1) :=
    (min_le_left ea ec).trans_lt heaexp
  refine ⟨low, high, e0, hlow, hlowhigh, he0, he0exp, ?⟩_
  intro eps heps heps0
  have hpa := hAb eps heps (heps0.trans_le (min_le_left ea ec))
  have hpc := hCb eps heps (heps0.trans_le (min_le_right ea ec))
  constructor
```

```
  · calc
      ENNReal.ofReal low * powerLogZero lam k eps ≤
          ENNReal.ofReal (lo*a) * powerLogZero lam k eps :=
        mul_le_mul' (ENNReal.ofReal_le_ofReal (min_le_left _ _)) le_rfl
      _ = ENNReal.ofReal lo *
          (ENNReal.ofReal a * powerLogZero lam k eps) := by
        rw [ENNReal.ofReal_mul hlo.le, mul_assoc]
      _ ≤ ENNReal.ofReal lo * A eps := mul_le_mul' le_rfl hpa.1
      _ ≤ B eps := (hB eps heps).1
  · calc
      B eps ≤ ENNReal.ofReal hi * C eps := (hB eps heps).2
      _ ≤ ENNReal.ofReal hi *
          (ENNReal.ofReal C0 * powerLogZero lam k eps) :=
        mul_le_mul' le_rfl hpc.2
      _ = ENNReal.ofReal high * powerLogZero lam k eps := by
        dsimp [high]
        rw [ENNReal.ofReal_mul hhi.le, mul_assoc]

-- All-small-radius order; this is an internal adapter, not Answers077.
def RadiusOrder (V : ℝ → ℝ → ENNReal) (lam : ℝ) (m : ℕ) : Prop :=
  0 ≤ lam ∧ 1 ≤ m ∧ ∃ rho : ℝ, 0 < rho ∧
    ∀ delta, 0 < delta → delta < rho →
      EThetaZeroPowerLog (V delta) lam (m-1)

theorem radiusOrder_of_fixed_distortion
    {VN VW : ℝ → ℝ → ENNReal} {lam lo hi L rho : ℝ} {m : ℕ}
    (hN : RadiusOrder VN lam m)
    (hlo : 0 < lo) (hhi : 0 < hi) (hL : 1 ≤ L) (hrho : 0 < rho)
    (hW : ∀ delta, 0 < delta → delta < rho →
      ∀ eps, 0 < eps →
        ENNReal.ofReal lo * VN (delta/L) eps ≤ VW delta eps ∧
        VW delta eps ≤ ENNReal.ofReal hi * VN (L*delta) eps) :
    RadiusOrder VW lam m := by
  rcases hN with ⟨hlam, hm, r, hr, ⟩hNr
  have hLp : 0 < L := lt_of_lt_of_le zero_lt_one hL
  let r0 := min rho (r/L)
  have hr0 : 0 < r0 := lt_min hrho (div_pos hr hLp)
  refine ⟨hlam, hm, r0, hr0, ?⟩_
  intro delta hd hd0
  have hdrho : delta < rho := hd0.trans_le (min_le_left _ _)
  have hds : delta < r/L := hd0.trans_le (min_le_right _ _)
  have hlarge : L*delta < r := by
    simpa only [mul_comm] using (₀lt_div_iff hLp).1 hds
  have hsmall : delta/L < r := by
    exact (div_le_self hd.le hL).trans_lt
      ((le_mul_of_one_le_left hd.le hL).trans_lt hlarge)
  exact order_of_two_window_bounds
    (hNr (delta/L) (div_pos hd hLp) hsmall)
    (hNr (L*delta) (mul_pos hLp hd) hlarge) hlo hhi
    (hW delta hd hdrho)
end O77.Transport0906
end
```

34 Module `O77NormalForm`

34.1 Adding regular variables and retaining the gauge volume

Throughout this module, a power–log degree k corresponds to multiplicity $m = k+1$. First we justify every positive rescaling of the small parameter. For $a > 0$ and $0 < \varepsilon < \exp(-(2|\log a| + 1))$, set $L = \log(1/\varepsilon)$. Then $\log(1/(a\varepsilon)) = L - \log a$ lies between $L/2$ and $2L$. Using

`Real.mul_rpow`,

$$a^{\lambda}2^{-k}P_{\lambda,k}(\varepsilon) \le P_{\lambda,k}(a\varepsilon) \le a^{\lambda}2^{k}P_{\lambda,k}(\varepsilon).$$

The proof checks nonnegativity before raising the logarithms to natural powers. Thus `rescale_order` retains both threshold and multiplicity; its cutoff is reduced also to ensure $a\varepsilon$ lies in the original range. No volume is converted to a real number in this argument.

```
import Mathlib
import O77Transport

set_option autoImplicit false
noncomputable section
open Set MeasureTheory
open scoped ENNReal BigOperators Topology
namespace O77.NormalForm0906
open O77.Residual
open O77.Transport0906 (order_of_two_window_bounds RadiusOrder)

/-- Quantitative rescaling of the existing real power--log gauge.
The cutoff depends on the scale, not on a radius or a measured function. -/
lemma powerLogZeroReal_scale_bounds {a eps lam : ℝ} (k : ℕ)
    (ha : 0 < a) (heps : 0 < eps)
    (hsmall : eps < Real.exp (-(2 * |Real.log a| + 1))) :
    (a ^ lam * (1 / 2 : ℝ) ^ k) * powerLogZeroReal lam k eps ≤
      powerLogZeroReal lam k (a * eps) ∧
    powerLogZeroReal lam k (a * eps) ≤
      (a ^ lam * (2 : ℝ) ^ k) * powerLogZeroReal lam k eps := by
  have hlogeps : Real.log eps < -(2 * |Real.log a| + 1) := by
    have h := Real.log_lt_log heps hsmall
    simpa only [Real.log_exp] using h
  have hlog : Real.log (1 / eps) = -Real.log eps := by
    simp only [one_div, Real.log_inv]
  have hscaled : Real.log (1 / (a * eps)) =
      Real.log (1 / eps) - Real.log a := by
    rw [one_div, Real.log_inv, Real.log_mul ha.ne' heps.ne', hlog]
    ring
  have hL : 0 ≤ Real.log (1 / eps) := by
    rw [hlog]
    nlinarith [abs_nonneg (Real.log a)]
  have hlow : (1 / 2 : ℝ) * Real.log (1 / eps) ≤
      Real.log (1 / (a * eps)) := by
    rw [hscaled, hlog]
    linarith [le_abs_self (Real.log a)]
  have hupp : Real.log (1 / (a * eps)) ≤
      2 * Real.log (1 / eps) := by
    rw [hscaled, hlog]
    linarith [neg_abs_le (Real.log a), abs_nonneg (Real.log a)]
  have hscaled0 : 0 ≤ Real.log (1 / (a * eps)) :=
    (mul_nonneg (by norm_num) hL).trans hlow
  have hloPow := ₀pow_le_pow_left (mul_nonneg (by norm_num) hL) hlow k
  have hhiPow := ₀pow_le_pow_left hscaled0 hupp k
  rw [mul_pow] at hloPow hhiPow
  have hp : 0 ≤ a ^ lam * eps ^ lam :=
    mul_nonneg (Real.rpow_nonneg ha.le _) (Real.rpow_nonneg heps.le _)
  constructor
  · unfold powerLogZeroReal
    rw [Real.mul_rpow ha.le heps.le]
    calc
      (a ^ lam * (1 / 2 : ℝ) ^ k) *
          (eps ^ lam * Real.log (1 / eps) ^ k) =
          (a ^ lam * eps ^ lam) *
            ((1 / 2 : ℝ) ^ k * Real.log (1 / eps) ^ k) := by ring
      _ ≤ (a ^ lam * eps ^ lam) * Real.log (1 / (a * eps)) ^ k :=
```

```
        mul_le_mul_of_nonneg_left hloPow hp
  · unfold powerLogZeroReal
    rw [Real.mul_rpow ha.le heps.le]
    calc
      (a ^ lam * eps ^ lam) * Real.log (1 / (a * eps)) ^ k ≤
          (a ^ lam * eps ^ lam) *
            ((2 : ℝ) ^ k * Real.log (1 / eps) ^ k) :=
        mul_le_mul_of_nonneg_left hhiPow hp
      _ = (a ^ lam * (2 : ℝ) ^ k) *
          (eps ^ lam * Real.log (1 / eps) ^ k) := by ring

/-- A rescaling adapter for the inherited ENNReal predicate, not a new
asymptotic definition. In particular no measure is converted to a real. -/
theorem rescale_order {V : ℝ → ENNReal} {lam a : ℝ} {k : ℕ}
    (hV : EThetaZeroPowerLog V lam k) (ha : 0 < a) :
    EThetaZeroPowerLog (fun eps => V (a * eps)) lam k := by
  rcases hV with ⟨c, C, e, hc, hcC, he, heExp, ⟩hV
  let low : ℝ := a ^ lam * (1 / 2 : ℝ) ^ k
  let high : ℝ := a ^ lam * (2 : ℝ) ^ k
  let cutoff : ℝ := Real.exp (-(2 * |Real.log a| + 1))
  have hl : 0 < low := mul_pos (Real.rpow_pos_of_pos ha _) (by positivity)
  have hh : 0 < high := mul_pos (Real.rpow_pos_of_pos ha _) (by positivity)
  have hcut : 0 < cutoff := Real.exp_pos _
  have hcut1 : cutoff < 1 := by
    have hneg : -(2 * |Real.log a| + 1) < 0 := by
      linarith [abs_nonneg (Real.log a)]
    simpa only [Real.exp_zero] using Real.exp_lt_exp.mpr hneg
  apply eThetaZeroPowerLog_of_sublevelPowerLogBounds
  refine ⟨c * low, C * high, min (e / a) cutoff,
    mul_pos hc hl, mul_pos (hc.trans_le hcC) hh,
    lt_min (div_pos he ha) hcut,
    (min_le_right _ _).trans_lt hcut1, ?⟩_
  intro eps heps heps0
  have hscaled : a * eps < e := by
    have h := (₀lt_div_iff ha).1
      (heps0.trans_le (min_le_left _ _))
    simpa only [mul_comm] using h
  have hb := hV (a * eps) (mul_pos ha heps) hscaled
  have hg := powerLogZeroReal_scale_bounds (lam := lam) k ha heps
    (heps0.trans_le (min_le_right _ _))
  have hglo : ENNReal.ofReal low * powerLogZero lam k eps ≤
      powerLogZero lam k (a * eps) := by
    change ENNReal.ofReal low * ENNReal.ofReal (powerLogZeroReal lam k eps) ≤
      ENNReal.ofReal (powerLogZeroReal lam k (a * eps))
    rw ←[ ENNReal.ofReal_mul hl.le]
    exact ENNReal.ofReal_le_ofReal hg.1
  have hghi : powerLogZero lam k (a * eps) ≤
      ENNReal.ofReal high * powerLogZero lam k eps := by
    change ENNReal.ofReal (powerLogZeroReal lam k (a * eps)) ≤
      ENNReal.ofReal high * ENNReal.ofReal (powerLogZeroReal lam k eps)
    rw ←[ ENNReal.ofReal_mul hh.le]
    exact ENNReal.ofReal_le_ofReal hg.2
  constructor
  · calc
      ENNReal.ofReal (c * low) * powerLogZero lam k eps =
          ENNReal.ofReal c * (ENNReal.ofReal low * powerLogZero lam k eps) := by
        rw [ENNReal.ofReal_mul hc.le, mul_assoc]
      _ ≤ ENNReal.ofReal c * powerLogZero lam k (a * eps) :=
        mul_le_mul' le_rfl hglo
      _ ≤ V (a * eps) := hb.1
  · calc
      V (a * eps) ≤ ENNReal.ofReal C * powerLogZero lam k (a * eps) := hb.2
```

```
      _ ≤ ENNReal.ofReal C * (ENNReal.ofReal high * powerLogZero lam k eps) :=
        mul_le_mul' le_rfl hghi
      _ = ENNReal.ofReal (C * high) * powerLogZero lam k eps := by
        rw [ENNReal.ofReal_mul (hc.trans_le hcC).le, mul_assoc]
```

Products and two simultaneous distortions. For positive $\varepsilon \le 1$, $P_{a,k}(\varepsilon)P_{b,l}(\varepsilon) = P_{a+b,k+l}(\varepsilon)$. This identity, positivity of constants, and intersection of cutoffs prove `product_order`. The next radius lemma combines this section's rescaling result with the preceding two-window argument:

$$\ell V_N(\delta/L, a\varepsilon) \le V_W(\delta,\varepsilon) \le u V_N(L\delta, b\varepsilon), \qquad a,b,\ell,u>0,$$

preserves the radius-first pair. In the proof the allowable radius is chosen first; for each fixed radius the two rescaled orders then provide cutoffs which may depend on that radius. No uniform-in-radius cutoff is required or claimed.

```
/-- Disjoint-variable products add logarithmic degrees. -/
lemma powerLogZero_product {a b eps : ℝ} {k l : ℕ}
    (heps : 0 < eps) (heps1 : eps ≤ 1) :
    powerLogZero a k eps * powerLogZero b l eps =
      powerLogZero (a + b) (k + l) eps := by
  rw [powerLogZero, powerLogZero, powerLogZero,
    ← ENNReal.ofReal_mul (powerLogZeroReal_nonneg heps heps1)]
  congr 1
  unfold powerLogZeroReal
  rw [Real.rpow_add heps, pow_add]
  ring

theorem product_order {V W : ℝ → ENNReal} {a b : ℝ} {k l : ℕ}
    (hV : EThetaZeroPowerLog V a k) (hW : EThetaZeroPowerLog W b l) :
    EThetaZeroPowerLog (fun eps => V eps * W eps) (a + b) (k + l) := by
  rcases hV with ⟨c, C, e, hc, hcC, he, heExp, ⟩hV
  rcases hW with ⟨d, D, f, hd, hdD, hf, hfExp, ⟩hW
  have he1 : e < 1 := heExp.trans (Real.exp_lt_one_iff.mpr (by norm_num))
  apply eThetaZeroPowerLog_of_sublevelPowerLogBounds
  refine ⟨c*d, C*D, min e f, mul_pos hc hd,
    mul_pos (hc.trans_le hcC) (hd.trans_le hdD), lt_min he hf,
    (min_le_left _ _).trans_lt he1, ?⟩_
  intro eps heps heps0
  have hev := heps0.trans_le (min_le_left e f)
  have hew := heps0.trans_le (min_le_right e f)
  have heps1 : eps ≤ 1 := (hev.trans he1).le
  have hv := hV eps heps hev
  have hw := hW eps heps hew
  constructor
  · calc
      ENNReal.ofReal (c*d) * powerLogZero (a+b) (k+l) eps =
          (ENNReal.ofReal c * powerLogZero a k eps) *
            (ENNReal.ofReal d * powerLogZero b l eps) := by
        rw [ENNReal.ofReal_mul hc.le, ← powerLogZero_product heps heps1]
        ac_rfl
      _ ≤ V eps * W eps := mul_le_mul' hv.1 hw.1
  · calc
      V eps * W eps ≤
          (ENNReal.ofReal C * powerLogZero a k eps) *
            (ENNReal.ofReal D * powerLogZero b l eps) := mul_le_mul' hv.2 hw.2
      _ = ENNReal.ofReal (C*D) * powerLogZero (a+b) (k+l) eps := by
        rw [ENNReal.ofReal_mul (hc.trans_le hcC).le,
          ← powerLogZero_product heps heps1]
        ac_rfl
```

```
/-- Completes the previous radius adapter by retaining the two different
small-parameter scales. All radii are fixed before either epsilon cutoff. -/
theorem radiusOrder_of_scaled_distortion
    {VN VW : ℝ → ℝ → ENNReal} {lam lo hi a b L rho : ℝ} {m : ℕ}
    (hN : RadiusOrder VN lam m)
    (hlo : 0 < lo) (hhi : 0 < hi) (ha : 0 < a) (hb : 0 < b)
    (hL : 1 ≤ L) (hrho : 0 < rho)
    (hW : ∀ delta, 0 < delta → delta < rho → ∀ eps, 0 < eps →
      ENNReal.ofReal lo * VN (delta/L) (a*eps) ≤ VW delta eps ∧
      VW delta eps ≤ ENNReal.ofReal hi * VN (L*delta) (b*eps)) :
    RadiusOrder VW lam m := by
  rcases hN with ⟨hlam, hm, r, hr, ⟩hNr
  have hLp : 0 < L := lt_of_lt_of_le zero_lt_one hL
  refine ⟨hlam, hm, min rho (r/L), lt_min hrho (div_pos hr hLp), ?⟩_
  intro delta hd hd0
  have hdrho := hd0.trans_le (min_le_left rho (r/L))
  have hlarge : L*delta < r := by
    simpa only [mul_comm] using
      (₀lt_div_iff hLp).1 (hd0.trans_le (min_le_right rho (r/L)))
  have hsmall : delta/L < r := by
    exact (div_le_self hd.le hL).trans_lt
      ((le_mul_of_one_le_left hd.le hL).trans_lt hlarge)
  exact order_of_two_window_bounds
    (rescale_order (hNr (delta/L) (div_pos hd hLp) hsmall) ha)
    (rescale_order (hNr (L*delta) (mul_pos hLp hd) hlarge) hb)
    hlo hhi (hW delta hd hdrho)
end O77.NormalForm0906
end
noncomputable section
open Set MeasureTheory
open scoped ENNReal BigOperators Topology
namespace O77.NormalForm0906
open O77.Residual
open O77.Transport0906 (order_of_two_window_bounds)
```

Independent nonnegative summands. For $f, g \geq 0$ in separate variables, under their actual product measure,

$$\{f < \varepsilon/2\} \times \{g < \varepsilon/2\} \subseteq \{f + g < \varepsilon\} \subseteq \{f < \varepsilon\} \times \{g < \varepsilon\}.$$

The product-measure rectangle formula `Measure.prod_prod` converts these inclusions into lower and upper products of sublevel masses. Combining product order, the scale $1/2$, and the two-window sandwich proves that thresholds and logarithmic degrees add. This proof needs neither an assumed convolution asymptotic nor a beta-integral calculation.

```
/-- The independent nonnegative-sum sandwich uses the library product
measure, not a postulated convolution formula or a beta integral. -/
theorem sublevelMass_prod_sum_bounds
    {E F : Type*} [MeasurableSpace E] [MeasurableSpace F]
    (mu : Measure E) (nu : Measure F) [SigmaFinite mu] [SigmaFinite nu]
    {f : E → ℝ} {g : F → ℝ}
    (hf0 : ∀ x, 0 ≤ f x) (hg0 : ∀ y, 0 ≤ g y) (eps : ℝ) :
    sublevelMass mu f (eps/2) * sublevelMass nu g (eps/2) ≤
      sublevelMass (mu.prod nu) (fun z => f z.1 + g z.2) eps ∧
    sublevelMass (mu.prod nu) (fun z => f z.1 + g z.2) eps ≤
      sublevelMass mu f eps * sublevelMass nu g eps := by
  have hlow : {x | f x < eps/2} ×ˢ {y | g y < eps/2} ⊆
      {z | f z.1 + g z.2 < eps} := by
    rintro ⟨x,⟩y ⟨hx,⟩hy
    change f x < eps/2 at hx
```

```
    change g y < eps/2 at hy
    change f x + g y < eps
    linarith
  have hupp : {z | f z.1 + g z.2 < eps} ⊆
      {x | f x < eps} ×ˢ {y | g y < eps} := by
    rintro ⟨x,⟩y hz
    change f x + g y < eps at hz
    change f x < eps ∧ g y < eps
    exact ⟨(le_add_of_nonneg_right (hg0 y)).trans_lt hz,
      (le_add_of_nonneg_left (hf0 x)).trans_lt ⟩hz
  constructor
  · have hm : (mu.prod nu) ({x | f x < eps/2} ×ˢ {y | g y < eps/2}) ≤
        (mu.prod nu) {z | f z.1 + g z.2 < eps} := measure_mono hlow
    simpa only [sublevelMass, Measure.prod_prod] using hm
  · have hm : (mu.prod nu) {z | f z.1 + g z.2 < eps} ≤
        (mu.prod nu) ({x | f x < eps} ×ˢ {y | g y < eps}) := measure_mono hupp
    simpa only [sublevelMass, Measure.prod_prod] using hm

/-- A genuine statement about the sum in its actual product measure. -/
theorem independent_sum_order
    {E F : Type*} [MeasurableSpace E] [MeasurableSpace F]
    {mu : Measure E} {nu : Measure F} [SigmaFinite mu] [SigmaFinite nu]
    {f : E → ℝ} {g : F → ℝ} {a b : ℝ} {k l : ℕ}
    (hf0 : ∀ x, 0 ≤ f x) (hg0 : ∀ y, 0 ≤ g y)
    (hf : EThetaZeroPowerLog (sublevelMass mu f) a k)
    (hg : EThetaZeroPowerLog (sublevelMass nu g) b l) :
    EThetaZeroPowerLog
      (sublevelMass (mu.prod nu) (fun z => f z.1 + g z.2))
      (a+b) (k+l) := by
  have hp := product_order hf hg
  have hlower : EThetaZeroPowerLog
      (fun eps => sublevelMass mu f (eps/2) * sublevelMass nu g (eps/2))
      (a+b) (k+l) := by
    simpa only [div_eq_mul_inv, one_mul, mul_comm] using
      rescale_order hp (show (0 : ℝ) < 1/2 by norm_num)
  apply order_of_two_window_bounds hlower hp
    (show (0 : ℝ) < 1 by norm_num) (show (0 : ℝ) < 1 by norm_num)
  intro eps heps
  simpa only [ENNReal.ofReal_one, one_mul] using
    sublevelMass_prod_sum_bounds mu nu hf0 hg0 eps
```

Constant directions and quadratic directions. If $V(\varepsilon) = cP_{\lambda,k}(\varepsilon)$ on a small interval and $0 < c < \infty$, its two-sided order follows with both real constants equal to c.toReal. Positivity and finiteness are hypotheses before this conversion is made. In particular, the constant zero function on a window of positive finite mass has pair $(0, 1)$. For a positive-radius ball in a finite-dimensional normed space E, $\|x\|^2 < \varepsilon < R^2$ cuts out exactly the ball of radius $\sqrt{\varepsilon}$. `Measure.addHaar_ball_of_pos` gives its mass as $\varepsilon^{\dim(E)/2}$ times the unit-ball mass. The latter is positive by openness and finite by compact containment. This also works for $\dim E = 0$: every positive-radius ball consists of the unique point and has its positive Haar mass. For the standard entry measure that mass is one.

```
/-- Exact mass formulas share one finite-normalizer and cutoff argument.
The equality hypothesis is about the actual function V, not its asymptotic pair. -/
theorem order_of_exact_powerLog {V : ℝ → ENNReal} {lam e : ℝ} {k : ℕ}
    {c : ENNReal} (hc0 : 0 < c) (hctop : c < ⊤) (he : 0 < e)
    (hV : ∀ eps, 0 < eps → eps < e → V eps = c * powerLogZero lam k eps) :
    EThetaZeroPowerLog V lam k := by
  have hc : 0 < c.toReal := ENNReal.toReal_pos hc0.ne' hctop.ne
  have hc_eq : ENNReal.ofReal c.toReal = c := ENNReal.ofReal_toReal hctop.ne
  refine ⟨c.toReal, c.toReal, min e (Real.exp (-1)/2), hc, le_rfl,
```

```
    lt_min he (half_pos (Real.exp_pos _)), ?_, ?⟩_
  · exact (min_le_right _ _).trans_lt (half_lt_self (Real.exp_pos _))
  · intro eps heps heps0
    simp only [hV eps heps (heps0.trans_le (min_le_left _ _)), hc_eq, le_refl, and_self]

/-- The constant-germ case keeps the finite, positive window mass.
No conversion of top to zero is used. -/
theorem zero_window_order {E : Type*} [MeasurableSpace E]
    (mu : Measure E) (S : Set E) (hS0 : 0 < mu S) (hStop : mu S < ⊤) :
    EThetaZeroPowerLog (sublevelMass (mu.restrict S) (fun _ => 0)) 0 0 := by
  apply order_of_exact_powerLog hS0 hStop (show (0 : ℝ) < 1 from zero_lt_one)
  intro eps heps _
  simp only [sublevelMass, heps, Set.ofPred_true, Measure.restrict_apply_univ,
    powerLogZero_zero_zero, mul_one]

section Quadratic
variable {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E]
  [FiniteDimensional ℝ E] [MeasurableSpace E] [BorelSpace E]
  (mu : Measure E) [Measure.IsAddHaarMeasure mu]

/-- The library Haar-ball scaling theorem works also in dimension zero
because the radius is positive. It avoids a nonempty-index gamma formula. -/
theorem quadratic_ball_sublevel {R eps : ℝ}
    (hR : 0 < R) (heps : 0 < eps) (hepsR : eps < R^2) :
    sublevelMass (mu.restrict (Metric.ball (0 : E) R))
        (fun x => ‖‖x^2) eps =
      ENNReal.ofReal (eps ^ ((Module.finrank ℝ E : ℝ)/2)) *
        mu (Metric.ball (0 : E) 1) := by
  have hs : 0 < Real.sqrt eps := Real.sqrt_pos.2 heps
  have hsq : (Real.sqrt eps)^2 = eps := Real.sq_sqrt heps.le
  have hsR : Real.sqrt eps < R :=
    (₀sq_lt_sq hs.le hR.le).1 (by simpa only [hsq] using hepsR)
  have hset : {x : E | ‖‖x^2 < eps} ∩ Metric.ball 0 R =
      Metric.ball 0 (Real.sqrt eps) := by
    ext x
    change ‖(‖x^2 < eps ∧ dist x 0 < R) ↔ dist x 0 < Real.sqrt eps
    simp only [dist_zero_right]
    have hnorm : ‖‖x^2 < eps ↔ ‖‖x < Real.sqrt eps := by
      simpa only [hsq] using ₀sq_lt_sq (norm_nonneg x) hs.le
    rw [hnorm]
    exact ⟨And.left, fun hx => ⟨hx, hx.trans ⟩⟩hsR
  unfold sublevelMass
  rw [Measure.restrict_apply (measurableSet_lt (by fun_prop) measurable_const),
    hset, Measure.addHaar_ball_of_pos mu 0 hs]
  have hp : (Real.sqrt eps) ^ Module.finrank ℝ E =
      eps ^ ((Module.finrank ℝ E : ℝ)/2) := by
    rw [Real.sqrt_eq_rpow, ← Real.rpow_natCast, ← Real.rpow_mul heps.le]
    congr 1
    ring
  rw [hp]

theorem quadratic_ball_order {R : ℝ} (hR : 0 < R) :
    EThetaZeroPowerLog
      (sublevelMass (mu.restrict (Metric.ball (0 : E) R)) (fun x => ‖‖x^2))
      ((Module.finrank ℝ E : ℝ)/2) 0 := by
  have hunit0 : 0 < mu (Metric.ball (0 : E) 1) :=
    Metric.isOpen_ball.measure_pos mu ⟨0, Metric.mem_ball_self ⟩zero_lt_one
  have hunittop : mu (Metric.ball (0 : E) 1) < ⊤ :=
    (measure_mono Metric.ball_subset_closedBall).trans_lt
      (isCompact_closedBall (0 : E) 1).measure_lt_top
  apply order_of_exact_powerLog hunit0 hunittop (sq_pos_of_pos hR)
  intro eps heps hepsR
```

```
  rw [quadratic_ball_sublevel mu hR heps hepsR]
  simp only [powerLogZero, powerLogZeroReal, pow_zero, mul_one]
  exact mul_comm _ _
end Quadratic
end O77.NormalForm0906
end
noncomputable section
open Set MeasureTheory
open scoped ENNReal BigOperators Topology
namespace O77.NormalForm0906
open O77.Residual
open O77.Transport0906 (order_of_two_window_bounds RadiusOrder)
```

The model and its exact ambient measure. Let $u \in \mathbb{R}^s$, $Y \in \mathbb{R}^{p\times h}$, $Z \in \mathbb{R}^{h\times q}$, and $v \in \mathbb{R}^g$. The code uses the associated product $\mathbb{R}^s \times ((\mathbb{R}^{p\times h} \times \mathbb{R}^{h\times q}) \times \mathbb{R}^g)$ with the product of standard Euclidean volume and the already fixed matrix entry measures. The equation `PiLp.volume_preserving_toLp` identifies the Euclidean synonym's volume with coordinate-product volume. Define

$$N(u,Y,Z,v) = \|u\|^2 + |YZ|_F^2, \qquad Q(u,Y,Z,v) = \|u\|^2 + |Y|_F^2 + |Z|_F^2 + \|v\|^2.$$

The model ball is $Q < R^2$ and its sublevel mass is `modelVolume`. The gauge variable v occurs in the window, although it does not occur in N. The residual function $|YZ|_F^2$ is jointly homogeneous of degree four; the regular term is quadratic. Its residual exponent is supplied by the proved residual analysis, rather than inferred from quadratic ball scaling. These definitions explicitly avoid the maximum norm on a nested product. The polynomial functions are measurable, nonnegative, and vanish at the origin; only these facts justify replacing the absolute band by a sublevel there.

```
abbrev Block (d : ℕ) := EuclideanSpace ℝ (Fin d)
abbrev ModelSpace (s g p q h : ℕ) :=
  Block s × (ResidualSpace p q h × Block g)

/-- Expose the product of the two inherited sigma-finite matrix measures.
This is an explicit adapter, so clients need not search through the named
measure definition or register a second global instance for it. -/
lemma residualMatrixLebesgue_sigmaFinite (p q h : ℕ) :
    SigmaFinite (residualMatrixLebesgue p q h) := by
  unfold residualMatrixLebesgue
  exact Measure.prod.instSigmaFinite

/-- Fixed coordinate Lebesgue measure, with the stated association. -/
def modelMeasure (s g p q h : ℕ) : Measure (ModelSpace s g p q h) :=
  (volume : Measure (Block s)).prod
    ((residualMatrixLebesgue p q h).prod (volume : Measure (Block g)))

instance modelMeasure_sigmaFinite (s g p q h : ℕ) :
    SigmaFinite (modelMeasure s g p q h) := by
  let := residualMatrixLebesgue_sigmaFinite p q h
  unfold modelMeasure
  exact Measure.prod.instSigmaFinite

/-- The standard Euclidean block measure is the entry-product measure,
not an arbitrary Haar normalization. This is a library theorem. -/
lemma euclidean_entry_measure (d : ℕ) :
    MeasurePreserving
      (WithLp.toLp 2 : (Fin d → ℝ) → Block d) volume volume :=
  PiLp.volume_preserving_toLp (Fin d)
```

```
def model {s g p q h : ℕ} (w : ModelSpace s g p q h) : ℝ :=
  ‖w‖.1^2 + residualKernel w.2.1

def modelAmbientSq {s g p q h : ℕ} (w : ModelSpace s g p q h) : ℝ :=
  ‖w‖.1^2 + residualAmbientNormSq w.2.1 + ‖w‖.2.2^2

def modelBall (s g p q h : ℕ) (R : ℝ) : Set (ModelSpace s g p q h) :=
  {w | modelAmbientSq w < R^2}

def modelBox (s g p q h : ℕ) (R : ℝ) : Set (ModelSpace s g p q h) :=
  Metric.ball 0 R ×ˢ
    (residualOpenQuadraticBall p q h R ×ˢ Metric.ball 0 R)

def modelVolume (s g p q h : ℕ) (R eps : ℝ) : ENNReal :=
  sublevelMass ((modelMeasure s g p q h).restrict (modelBall s g p q h R))
    model eps

lemma measurable_model {s g p q h : ℕ} :
    Measurable (model : ModelSpace s g p q h → ℝ) := by
  unfold model
  have hu : Measurable (fun w : ModelSpace s g p q h => ‖w‖.1^2) := by fun_prop
  exact hu.add
    (measurable_residualKernel.comp (measurable_fst.comp measurable_snd))

lemma measurable_modelAmbientSq {s g p q h : ℕ} :
    Measurable (modelAmbientSq : ModelSpace s g p q h → ℝ) := by
  unfold modelAmbientSq
  have hu : Measurable (fun w : ModelSpace s g p q h => ‖w‖.1^2) := by fun_prop
  have hv : Measurable (fun w : ModelSpace s g p q h => ‖w‖.2.2^2) := by fun_prop
  exact (hu.add (measurable_residualAmbientNormSq.comp
    (measurable_fst.comp measurable_snd))).add hv

lemma modelBand_eq_sublevel {s g p q h : ℕ} (eps : ℝ) :
    {w : ModelSpace s g p q h |
      |model w - model (0 : ModelSpace s g p q h)| < eps} =
      {w | model w < eps} := by
  have hz : model (0 : ModelSpace s g p q h) = 0 := by
    simp [model, residualKernel, frobeniusSq]
  have hn : ∀ w : ModelSpace s g p q h, 0 ≤ model w :=
    fun w => add_nonneg (sq_nonneg _) (residualKernel_nonneg _)
  ext w
  change (|model w - model (0 : ModelSpace s g p q h)| < eps) ↔ model w < eps
  rw [hz, sub_zero, abs_of_nonneg (hn w)]
```

A fixed product window inside and outside each ball. Let $B_{\mathrm{box}}(R)$ impose radius R separately on u, on the residual pair (Y, Z) with its combined squared-entry norm, and on v. For $R > 0$,

$$B_{\mathrm{box}}(R/2) \subseteq \{Q < R^2\} \subseteq B_{\mathrm{box}}(R).$$

The first inclusion follows because three summands less than $R^2/4$ have sum less than R^2; the second follows from their nonnegativity. For the product window, use `Measure.prod_restrict` to identify the restricted measure with the product of the three restricted measures. The zero gauge germ contributes exactly a positive finite window factor; the regular quadratic germ contributes $s/2$ and degree zero; the residual contributes its given pair. Applying the independent-sum theorem twice therefore gives threshold $s/2 + \lambda$ and unchanged residual multiplicity. The gauge radius is fixed with R, before ε tends to zero.

```
/-- Three independent Euclidean factors: the residual pair is one factor.
The tuple's default maximum norm is not used in modelBall. -/
lemma modelBox_half_subset_ball (s g p q h : ℕ) {R : ℝ} (hR : 0 < R) :
```

```
    modelBox s g p q h (R/2) ⊆ modelBall s g p q h R := by
  rintro w ⟨hu, hy, ⟩hv
  have hu' : ‖w‖.1 < R/2 := by simpa [Metric.mem_ball] using hu
  have hv' : ‖w‖.2.2 < R/2 := by simpa [Metric.mem_ball] using hv
  change residualAmbientNormSq w.2.1 < (R/2)^2 at hy
  have hu2 : ‖w‖.1^2 < (R/2)^2 :=
    (₀sq_lt_sq (norm_nonneg _) (half_pos hR).le).2 hu'
  have hv2 : ‖w‖.2.2^2 < (R/2)^2 :=
    (₀sq_lt_sq (norm_nonneg _) (half_pos hR).le).2 hv'
  change ‖w‖.1^2 + residualAmbientNormSq w.2.1 + ‖w‖.2.2^2 < R^2
  nlinarith [sq_pos_of_pos hR]

lemma modelBall_subset_box (s g p q h : ℕ) {R : ℝ} (hR : 0 < R) :
    modelBall s g p q h R ⊆ modelBox s g p q h R := by
  intro w hw
  change ‖w‖.1^2 + residualAmbientNormSq w.2.1 + ‖w‖.2.2^2 < R^2 at hw
  have hy0 := residualAmbientNormSq_nonneg w.2.1
  refine ⟨?_, ?_, ?⟩_
  · change dist w.1 0 < R
    rw [dist_zero_right]
    nlinarith [sq_nonneg ‖w‖.2.2, norm_nonneg w.1]
  · change residualAmbientNormSq w.2.1 < R^2
    nlinarith [sq_nonneg ‖w‖.1, sq_nonneg ‖w‖.2.2]
  · change dist w.2.2 0 < R
    rw [dist_zero_right]
    nlinarith [sq_nonneg ‖w‖.1, norm_nonneg w.2.2]

/-- First assemble the true restricted product measure in a fixed box.
The residual coefficient is an input here only; it is instantiated below. -/
theorem modelBox_order (s g : ℕ) {p q h : ℕ} {R lam : ℝ} {k : ℕ}
    (hR : 0 < R)
    (hres : EThetaZeroPowerLog
      (sublevelMass (residualOpenQuadraticBallMeasure p q h R) residualKernel) lam k) :
    EThetaZeroPowerLog
      (sublevelMass ((modelMeasure s g p q h).restrict (modelBox s g p q h R)) model)
      ((s : ℝ)/2 + lam) k := by
  let := residualMatrixLebesgue_sigmaFinite p q h
  let : SigmaFinite (residualOpenQuadraticBallMeasure p q h R) := by
    unfold residualOpenQuadraticBallMeasure
    infer_instance
  let nu : Measure (Block g) := volume.restrict (Metric.ball 0 R)
  let : SigmaFinite nu := by
    dsimp only [nu]
    infer_instance
  have hfree : EThetaZeroPowerLog (sublevelMass nu (fun _ => 0)) 0 0 := by
    apply zero_window_order
    · exact Metric.isOpen_ball.measure_pos volume ⟨0, Metric.mem_ball_self ⟩hR
    · exact (measure_mono Metric.ball_subset_closedBall).trans_lt
        (isCompact_closedBall (0 : Block g) R).measure_lt_top
  have hresfree := independent_sum_order residualKernel_nonneg
    (fun _ : Block g => le_rfl) hres hfree
  have hquad := quadratic_ball_order
    (volume : Measure (Block s)) hR
  have hall := independent_sum_order
    (fun u : Block s => sq_nonneg |‖u)
    (fun z : ResidualSpace p q h × Block g =>
      add_nonneg (residualKernel_nonneg z.1) le_rfl) hquad hresfree
  have hmeasure :
      ((volume : Measure (Block s)).restrict (Metric.ball 0 R)).prod
        ((residualOpenQuadraticBallMeasure p q h R).prod nu) =
      (modelMeasure s g p q h).restrict (modelBox s g p q h R) := by
    dsimp only [nu, residualOpenQuadraticBallMeasure, modelMeasure, modelBox]
```

```
    rw [Measure.prod_restrict, Measure.prod_restrict]
  rw [hmeasure] at hall
  have hmodel :
      (fun z : ModelSpace s g p q h => ‖z‖.1^2 + (residualKernel z.2.1 + 0)) =
        (O77.NormalForm0906.model : ModelSpace s g p q h → ℝ) := by
    funext z
    change ‖z‖.1^2 + (residualKernel z.2.1 + 0) =
      ‖z‖.1^2 + residualKernel z.2.1
    rw [add_zero]
  change EThetaZeroPowerLog
    (sublevelMass ((modelMeasure s g p q h).restrict (modelBox s g p q h R))
      (fun z : ModelSpace s g p q h => ‖z‖.1^2 + (residualKernel z.2.1 + 0))) _ _ at hall
  rw [hmodel] at hall
  simpa only [finrank_euclideanSpace, Fintype.card_fin,
    add_zero, zero_add, Nat.cast_zero, Nat.cast_add] using hall
```

From product windows to every sufficiently small ball. The enclosing product window has finite mass: both Euclidean balls and the residual quadratic ball do. Hence every model sublevel in that ball has finite measure. Given a radius-first residual pair, fix any allowed $R > 0$ and apply its estimate at both $R/2$ and R. The preceding inclusions and product-window order give the same pair for the actual model ball. This proves `normalForm_pair_of_residual`. The Wishart wrapper supplies the residual pair $(d_0(p,q,h)/2, \mu_0(p,q,h))$ from the preceding residual analysis. The neutral wrapper instead supplies $(0,1)$ whenever $pqh = 0$. Consequently the model pair is $((s + d_0)/2, \mu_0)$ in general and $(s/2, 1)$ when residual multiplication vanishes. A vanished residual does not imply that $s = 0$ or that the point is a global threshold minimizer.

```
/-- Finiteness is established before any real-valued volume adapter. -/
lemma modelVolume_ne_top (s g p q h : ℕ) {R : ℝ} (hR : 0 < R) (eps : ℝ) :
    modelVolume s g p q h R eps ≠ ⊤ := by
  let := residualMatrixLebesgue_sigmaFinite p q h
  have hu : (volume : Measure (Block s)) (Metric.ball 0 R) < ⊤ :=
    (measure_mono Metric.ball_subset_closedBall).trans_lt
      (isCompact_closedBall (0 : Block s) R).measure_lt_top
  have hv : (volume : Measure (Block g)) (Metric.ball 0 R) < ⊤ :=
    (measure_mono Metric.ball_subset_closedBall).trans_lt
      (isCompact_closedBall (0 : Block g) R).measure_lt_top
  have hbox : modelMeasure s g p q h (modelBox s g p q h R) < ⊤ := by
    unfold modelMeasure modelBox
    rw [Measure.prod_prod, Measure.prod_prod]
    exact ENNReal.mul_lt_top hu
      (ENNReal.mul_lt_top (residualOpenQuadraticBall_measure_lt_top p q h R) hv)
  have hball : modelMeasure s g p q h (modelBall s g p q h R) < ⊤ :=
    (measure_mono (modelBall_subset_box s g p q h hR)).trans_lt hbox
  unfold modelVolume sublevelMass
  have hbound : ((modelMeasure s g p q h).restrict
      (modelBall s g p q h R)) Set.univ < ⊤ := by
    simpa only [Measure.restrict_apply_univ] using hball
  exact ne_of_lt ((measure_mono (Set.subset_univ _)).trans_lt hbound)

/-- The residual origin predicate is used with its actual coordinate measure.
Both auxiliary radii are fixed before the two epsilon estimates are chosen. -/
theorem normalForm_pair_of_residual (s g : ℕ) {p q h : ℕ} {lam : ℝ} {m : ℕ}
    (hres : HasResidualBandPairAtOrigin p q h lam m) :
    RadiusOrder (modelVolume s g p q h) ((s : ℝ)/2 + lam) m := by
  rcases hres with ⟨hlam, hm, R0, hR0, ⟩hres
  refine ⟨by positivity, hm, R0, hR0, ?⟩_
  intro R hR hRR
  have hhalf : R/2 < R0 := (half_lt_self hR).trans hRR
  have hlow := modelBox_order s g (half_pos hR)
    (hres (R/2) (half_pos hR) hhalf)
```

```
  have hupp := modelBox_order s g hR (hres R hR hRR)
  apply order_of_two_window_bounds hlow hupp
    (show (0 : ℝ) < 1 by norm_num) (show (0 : ℝ) < 1 by norm_num)
  intro eps heps
  simp only [ENNReal.ofReal_one, one_mul]
  exact ⟨(Measure.restrict_mono_set (modelMeasure s g p q h)
      (modelBox_half_subset_ball s g p q h hR)) _,
    (Measure.restrict_mono_set (modelMeasure s g p q h)
      (modelBall_subset_box s g p q h hR)) ⟩_

/-- Measured normal-form pair in every regular, gauge and residual dimension,
conditional on the exact Wishart identity. -/
theorem normalForm_pair_of_wishart
    (hW : O77.External.RealWishartEigenvalueFormula) (s g p q h : ℕ) :
    RadiusOrder (modelVolume s g p q h)
      (((s : ℝ) + (O77.residualMin p q h : ℝ))/2)
      (O77.residualMultiplicity p q h) := by
  have ha : (s : ℝ)/2 + (O77.residualMin p q h : ℝ)/2 =
      ((s : ℝ) + (O77.residualMin p q h : ℝ))/2 := by ring
  simpa only [ha] using normalForm_pair_of_residual s g
    (residualBandPair_all_dimensions_of_wishart_instantiated hW p q h)

/-- No external premise is needed when the residual multiplication vanishes. -/
theorem normalForm_pair_neutral (s g : ℕ) {p q h : ℕ}
    (hz : p = 0 ∨ q = 0 ∨ h = 0) :
    RadiusOrder (modelVolume s g p q h) ((s : ℝ)/2) 1 := by
  simpa only [add_zero] using normalForm_pair_of_residual s g
    (residualBandPair_neutral_of_zero_dimension hz)

-- Regression consequences.
example (hW : O77.External.RealWishartEigenvalueFormula) :
    RadiusOrder (modelVolume 2 3 1 1 1) (3/2 : ℝ) 2 := by
  have h := normalForm_pair_of_residual 2 3 (residualBandPair_scalar_test hW)
  norm_num at h ⊢
  exact h

example : RadiusOrder (modelVolume 4 7 0 3 2) 2 1 := by
  have h := normalForm_pair_neutral 4 7
    (p := 0) (q := 3) (h := 2) (Or.inl rfl)
  norm_num at h
  exact h

example : RadiusOrder (modelVolume 0 0 0 0 0) 0 1 := by
  simpa using normalForm_pair_neutral 0 0
    (p := 0) (q := 0) (h := 0) (Or.inl rfl)
end O77.NormalForm0906
end
```

35 Module `O77Coordinates`

35.1 Exact entry layouts and the nonlinear Jacobian

An `EntryLayout` of a measured real vector space (E, μ) is a linear enumeration $e : E \simeq \mathbb{R}^{\iota}$ of its actual entries together with an exact measure-preservation proof. Its squared-entry function is $\sum_i e(x)_i^2$. Product layouts concatenate index sets by disjoint sum, so their squared-entry functions add and their measures are the corresponding product measures. Relabeling an index set preserves the finite sum and product measure. Numbering it by $\mathrm{Fin}(d)$ changes no entry. Finally `WithLp.linearEquiv` supplies the Euclidean carrier, and `EuclideanSpace.norm_sq_eq` identifies its squared norm with the same entry sum. No norm assertion about an arbitrary

original product space is included in the layout structure.

```
import Mathlib
import O77NormalForm

set_option autoImplicit false
noncomputable section
open Set Filter MeasureTheory
open scoped BigOperators Topology ENNReal NNReal Matrix.Norms.Elementwise
namespace O77.Coordinates0906
open O77.Residual
open O77.Reconstruction0905
open O77.Transport0906
open O77.NormalForm0906

/-- A linear enumeration of actual entries, together with its exact
measure normalization. No norm on E is asserted to be Euclidean. -/
structure EntryLayout (E ι : Type*) [AddCommGroup E] [Module ℝ E]
    [MeasurableSpace E] [Fintype ι] (mu : Measure E) where
  equiv : E ≃ₗ[ℝ] ι → ℝ
  preserves : MeasurePreserving equiv mu (coordinateLebesgue ι)

namespace EntryLayout
variable {E F ι κ : Type*}
  [AddCommGroup E] [Module ℝ E] [MeasurableSpace E]
  [AddCommGroup F] [Module ℝ F] [MeasurableSpace F]
  [Fintype ι] [Fintype κ] {mu : Measure E} {nu : Measure F}

def sq (e : EntryLayout E ι mu) (x : E) : ℝ :=
  ∑ i, (e.equiv x i)^2

/-- Library product and sum-Pi equivalences do all measure transport. -/
def prod [SigmaFinite mu] [SigmaFinite nu]
    (e : EntryLayout E ι mu) (f : EntryLayout F κ nu) :
    EntryLayout (E × F) (ι ⊕ κ) (mu.prod nu) where
  equiv := (e.equiv.prodCongr f.equiv).trans
    (LinearEquiv.sumArrowLequivProdArrow ι κ ℝ ℝ).symm
  preserves := joinEntries_measurePreserving.comp
    (e.preserves.prod f.preserves)

lemma sq_prod [SigmaFinite mu] [SigmaFinite nu]
    (e : EntryLayout E ι mu) (f : EntryLayout F κ nu) (x : E) (y : F) :
    (e.prod f).sq (x,y) = e.sq x + f.sq y := by
  change ∑ i : ι ⊕ κ, (Sum.elim (e.equiv x) (f.equiv y) i)^2) = _
  simp only [Fintype.sum_sum_type, Sum.elim_inl, Sum.elim_inr, sq]

/-- The projection form also rewrites a tuple variable, without requiring
the simplifier to eta-expand that variable into a pair. -/
lemma sq_prod_apply [SigmaFinite mu] [SigmaFinite nu]
    (e : EntryLayout E ι mu) (f : EntryLayout F κ nu) (w : E × F) :
    (e.prod f).sq w = e.sq w.1 + f.sq w.2 := by
  rcases w with ⟨x, y⟩
  exact sq_prod e f x y

/-- Compose already identified component energies without asking the
simplifier to reconstruct the product's typeclass arguments. -/
lemma sq_prod_of_eq [SigmaFinite mu] [SigmaFinite nu]
    (e : EntryLayout E ι mu) (f : EntryLayout F κ nu) (w : E × F)
    {a b : ℝ} (he : e.sq w.1 = a) (hf : f.sq w.2 = b) :
    (e.prod f).sq w = a + b :=
  (sq_prod_apply e f w).trans (congrArg₂ (fun x y : ℝ => x+y) he hf)

/-- Only entry labels change; there is no arbitrary linear basis change. -/
```

```
def reindex (e : EntryLayout E ι mu) (b : ι ≃ κ) :
    EntryLayout E κ mu where
  equiv := e.equiv.trans (LinearEquiv.piCongrLeft ℝ (fun _ : κ => ℝ) b)
  preserves := (reindexEntries_measurePreserving b).comp e.preserves

lemma sq_reindex (e : EntryLayout E ι mu) (b : ι ≃ κ) (x : E) :
    (e.reindex b).sq x = e.sq x := by
  unfold sq
  calc
    ∑( j : κ, ((e.reindex b).equiv x j)^2) =
        ∑ i : ι, ((e.reindex b).equiv x (b i))^2 := (b.sum_comp _).symm
    _ = ∑ i : ι, (e.equiv x i)^2 := by
      apply Finset.sum_congr rfl
      intro i _
      change ((Equiv.piCongrLeft (fun _ : κ => ℝ) b) (e.equiv x) (b i))^2 = _
      rw [Equiv.piCongrLeft_apply_apply]

def numbered (e : EntryLayout E ι mu) {d : ℕ}
    (hd : Fintype.card ι = d) : EntryLayout E (Fin d) mu :=
  e.reindex ((Fintype.equivFin ι).trans (finCongr hd))

lemma sq_numbered (e : EntryLayout E ι mu) {d : ℕ}
    (hd : Fintype.card ι = d) (x : E) :
    (e.numbered hd).sq x = e.sq x := sq_reindex e _ x

/-- The library's L2 synonym equips the enumerated entries with the
Euclidean norm; the source retains its original topology and measure. -/
def euclidean {d : ℕ} (e : EntryLayout E (Fin d) mu) :
    E ≃ₗ[ℝ] Block d :=
  e.equiv.trans (WithLp.linearEquiv 2 ℝ (Fin d → ℝ)).symm

lemma euclidean_preserves {d : ℕ} (e : EntryLayout E (Fin d) mu) :
    MeasurePreserving e.euclidean mu volume := by
  have hfun : (e.euclidean : E → Block d) = WithLp.toLp 2 ∘ e.equiv := rfl
  rw [hfun]
  exact (euclidean_entry_measure d).comp e.preserves

lemma euclidean_norm_sq {d : ℕ}
    (e : EntryLayout E (Fin d) mu) (x : E) :
    ‖e.euclidean ‖x^2 = e.sq x := by
  rw [EuclideanSpace.norm_sq_eq]
  change ∑( i : Fin d, ‖e.equiv x ‖i^2) = ∑ i : Fin d, (e.equiv x i)^2
  simp only [Real.norm_eq_abs, sq_abs]
end EntryLayout
```

Counting the coordinate blocks. Put $a = r + \alpha$, $b = r + \beta$, $O = b + p$, $I = a + q$, $H = a + \beta + h$. The regular variables (U, T') have

$$s = Oa + bq$$

entries. The five gauge blocks (P, C, X_P, D, Y_1) have

$$g = a^2 + (\beta + h)a + \alpha q + O\beta + bh$$

entries. The residual blocks have $ph + hq$ entries. The raw seven-block and new nine-block index sets are explicit disjoint sums of matrix entry indices, and both have cardinality

$$s + g + ph + hq = H(I + O).$$

The identities are proved by finite-type cardinality rules and distributive arithmetic. Each measure follows the same association as its tuple type. The regular and gauge blocks are

separately packed into Euclidean spaces; the residual matrices are kept as matrices so their multiplication is unchanged.

```
def matrixLayout (m n : ℕ) :
    EntryLayout (RM m n) (Fin m × Fin n) (matrixLebesgue (Fin m) (Fin n)) :=
  ⟨matrixEntries, ⟩matrixEntries_measurePreserving

lemma matrixLayout_sq (m n : ℕ) (A : RM m n) :
    (matrixLayout m n).sq A = frobeniusSq A := matrixEntries_sq A

/-- All dimensions are subtraction-free, including the empty cases. -/
def regularDim (r alpha beta p q : ℕ) : ℕ :=
  ((r+beta)+p)*(r+alpha) + (r+beta)*q

def gaugeDim (r alpha beta h p q : ℕ) : ℕ :=
  (r+alpha)*(r+alpha) + (beta+h)*(r+alpha) + alpha*q +
    ((r+beta)+p)*beta + (r+beta)*h

def ambientDim (r alpha beta h p q : ℕ) : ℕ :=
  ((r+alpha)+(beta+h))*(((r+alpha)+q)+((r+beta)+p))

abbrev Regular (r alpha beta p q : ℕ) :=
  RM ((r+beta)+p) (r+alpha) × RM (r+beta) q
abbrev Gauge (r alpha beta h p q : ℕ) :=
  RM (r+alpha) (r+alpha) × RM (beta+h) (r+alpha) ×
  RM alpha q × RM ((r+beta)+p) beta × RM (r+beta) h
abbrev RegularIndex (r alpha beta p q : ℕ) :=
  (Fin ((r+beta)+p) × Fin (r+alpha)) ⊕ (Fin (r+beta) × Fin q)
abbrev GaugeIndex (r alpha beta h p q : ℕ) :=
  (Fin (r+alpha) × Fin (r+alpha)) ⊕ (Fin (beta+h) × Fin (r+alpha)) ⊕
  (Fin alpha × Fin q) ⊕ (Fin ((r+beta)+p) × Fin beta) ⊕
  (Fin (r+beta) × Fin h)
abbrev RawIndex (r alpha beta h p q : ℕ) :=
  (Fin (r+alpha) × Fin (r+alpha)) ⊕ (Fin (beta+h) × Fin (r+alpha)) ⊕
  (Fin (r+alpha) × Fin q) ⊕ (Fin (beta+h) × Fin q) ⊕
  (Fin ((r+beta)+p) × Fin (r+alpha)) ⊕
  (Fin ((r+beta)+p) × Fin beta) ⊕ (Fin ((r+beta)+p) × Fin h)
abbrev CoordinateIndex (r alpha beta h p q : ℕ) :=
  (Fin ((r+beta)+p) × Fin (r+alpha)) ⊕ (Fin (r+beta) × Fin q) ⊕
  (Fin p × Fin h) ⊕ (Fin h × Fin q) ⊕ GaugeIndex r alpha beta h p q

local notation "ml" => fun m n => matrixLebesgue (Fin m) (Fin n)
def regularMeasure (r alpha beta p q : ℕ) : Measure (Regular r alpha beta p q) :=
  (ml ((r+beta)+p) (r+alpha)).prod (ml (r+beta) q)
def gaugeMeasure (r alpha beta h p q : ℕ) : Measure (Gauge r alpha beta h p q) :=
  (ml (r+alpha) (r+alpha)).prod ((ml (beta+h) (r+alpha)).prod
    ((ml alpha q).prod ((ml ((r+beta)+p) beta).prod (ml (r+beta) h))))
def rawMeasure (r alpha beta h p q : ℕ) : Measure (Raw r alpha beta h p q) :=
  (ml (r+alpha) (r+alpha)).prod ((ml (beta+h) (r+alpha)).prod
    ((ml (r+alpha) q).prod ((ml (beta+h) q).prod
      ((ml ((r+beta)+p) (r+alpha)).prod
        ((ml ((r+beta)+p) beta).prod (ml ((r+beta)+p) h))))))
def coordinateMeasure (r alpha beta h p q : ℕ) :
    Measure (Coordinates r alpha beta h p q) :=
  (ml ((r+beta)+p) (r+alpha)).prod ((ml (r+beta) q).prod
    ((ml p h).prod ((ml h q).prod (gaugeMeasure r alpha beta h p q))))

instance gaugeMeasure_sigmaFinite (r alpha beta h p q : ℕ) :
    SigmaFinite (gaugeMeasure r alpha beta h p q) := by
  unfold gaugeMeasure
  infer_instance

lemma regularMeasure_sigmaFinite (r alpha beta p q : ℕ) :
```

```
    SigmaFinite (regularMeasure r alpha beta p q) := by
  unfold regularMeasure
  exact Measure.prod.instSigmaFinite

variable (r alpha beta h p q : ℕ)

def regularLayout : EntryLayout (Regular r alpha beta p q)
    (RegularIndex r alpha beta p q) (regularMeasure r alpha beta p q) :=
  (matrixLayout ((r+beta)+p) (r+alpha)).prod (matrixLayout (r+beta) q)
def gaugeLayout : EntryLayout (Gauge r alpha beta h p q)
    (GaugeIndex r alpha beta h p q) (gaugeMeasure r alpha beta h p q) :=
  (matrixLayout (r+alpha) (r+alpha)).prod
    ((matrixLayout (beta+h) (r+alpha)).prod ((matrixLayout alpha q).prod
      ((matrixLayout ((r+beta)+p) beta).prod (matrixLayout (r+beta) h))))
def rawLayout : EntryLayout (Raw r alpha beta h p q)
    (RawIndex r alpha beta h p q) (rawMeasure r alpha beta h p q) :=
  (matrixLayout (r+alpha) (r+alpha)).prod
    ((matrixLayout (beta+h) (r+alpha)).prod ((matrixLayout (r+alpha) q).prod
      ((matrixLayout (beta+h) q).prod ((matrixLayout ((r+beta)+p) (r+alpha)).prod
        ((matrixLayout ((r+beta)+p) beta).prod (matrixLayout ((r+beta)+p) h))))))
def coordinateLayout : EntryLayout (Coordinates r alpha beta h p q)
    (CoordinateIndex r alpha beta h p q) (coordinateMeasure r alpha beta h p q) :=
  (matrixLayout ((r+beta)+p) (r+alpha)).prod ((matrixLayout (r+beta) q).prod
    ((matrixLayout p h).prod ((matrixLayout h q).prod (gaugeLayout r alpha beta h p q))))

lemma regularIndex_card : Fintype.card (RegularIndex r alpha beta p q) =
    regularDim r alpha beta p q := by
  simp [RegularIndex, regularDim]
lemma gaugeIndex_card : Fintype.card (GaugeIndex r alpha beta h p q) =
    gaugeDim r alpha beta h p q := by
  simp [GaugeIndex, gaugeDim, add_assoc]
lemma rawIndex_card : Fintype.card (RawIndex r alpha beta h p q) =
    ambientDim r alpha beta h p q := by
  simp only [RawIndex, Fintype.card_sum, Fintype.card_prod, Fintype.card_fin,
    ambientDim]
  ring
lemma coordinateIndex_card : Fintype.card (CoordinateIndex r alpha beta h p q) =
    ambientDim r alpha beta h p q := by
  simp only [CoordinateIndex, GaugeIndex, Fintype.card_sum, Fintype.card_prod,
    Fintype.card_fin, ambientDim]
  ring

/-- The first two and last five blocks are enumerated independently.
The residual matrices are never reindexed across their multiplication. -/
def regularPack : Regular r alpha beta p q ≃ₗᵢℝ[] Block (regularDim r alpha beta p q) :=
  ((regularLayout r alpha beta p q).numbered (regularIndex_card r alpha beta p q)).euclidean
def gaugePack : Gauge r alpha beta h p q ≃ₗᵢℝ[] Block (gaugeDim r alpha beta h p q) :=
  ((gaugeLayout r alpha beta h p q).numbered (gaugeIndex_card r alpha beta h p q)).euclidean

end O77.Coordinates0906
end
noncomputable section
open Set Filter MeasureTheory
open scoped BigOperators Topology ENNReal NNReal Matrix.Norms.Elementwise
namespace O77.Coordinates0906
open O77.Residual
open O77.Reconstruction0905
open O77.Transport0906
open O77.NormalForm0906
variable (r alpha beta h p q : ℕ)
```

Regrouping and attachment to the measured model. The map `regroup` changes only parentheses: $(U, T', Y_0, Z, \text{gauge})$ becomes $((U, T'), ((Y_0, Z), \text{gauge}))$. `measurePreserving_prodAssoc` proves its exact product-measure identity. Compose with the separately normalized regular and gauge packs to obtain `coordinateToModel`; its inverse restores the same blocks. Its measure-preservation theorem uses the product of the three component measure-preservation theorems, with the identity on (Y_0, Z).

```
/-- Adjacent pairs and the gauge tail are regrouped, not mixed. -/
def regroup : Coordinates r alpha beta h p q ≃ₗ[ℝ]
    (Regular r alpha beta p q × (ResidualSpace p q h × Gauge r alpha beta h p q)) where
  toFun c := ((c.1, c.2.1), ((c.2.2.1, c.2.2.2.1), c.2.2.2.2))
  invFun w := (w.1.1, w.1.2, w.2.1.1, w.2.1.2, w.2.2)
  left_inv _ := rfl
  right_inv _ := rfl
  map_add' _ _ := rfl
  map_smul' _ _ := rfl

lemma regroup_preserves : MeasurePreserving (regroup r alpha beta h p q)
    (coordinateMeasure r alpha beta h p q)
    ((regularMeasure r alpha beta p q).prod
      ((residualMatrixLebesgue p q h).prod (gaugeMeasure r alpha beta h p q))) := by
  let u := matrixLebesgue (Fin ((r+beta)+p)) (Fin (r+alpha))
  let t := matrixLebesgue (Fin (r+beta)) (Fin q)
  let y := matrixLebesgue (Fin p) (Fin h)
  let z := matrixLebesgue (Fin h) (Fin q)
  let v := gaugeMeasure r alpha beta h p q
  have h1 := (measurePreserving_prodAssoc u t (y.prod (z.prod v))).symm
  have h2 := (MeasurePreserving.id (u.prod t)).prod
    (measurePreserving_prodAssoc y z v).symm
  exact h2.comp h1

/-- Exact attachment to the preceding measured model, not an abstract
pair-preserving map supplied as an assumption. -/
def coordinateToModel : Coordinates r alpha beta h p q ≃ₗ[ℝ]
    ModelSpace (regularDim r alpha beta p q) (gaugeDim r alpha beta h p q) p q h :=
  (regroup r alpha beta h p q).trans
    ((regularPack r alpha beta p q).prodCongr
      ((LinearEquiv.refl ℝ (ResidualSpace p q h)).prodCongr
        (gaugePack r alpha beta h p q)))

lemma coordinateToModel_preserves : MeasurePreserving
    (coordinateToModel r alpha beta h p q) (coordinateMeasure r alpha beta h p q)
    (modelMeasure (regularDim r alpha beta p q) (gaugeDim r alpha beta h p q) p q h) := by
  let := regularMeasure_sigmaFinite r alpha beta p q
  let := residualMatrixLebesgue_sigmaFinite p q h
  have hu := ((regularLayout r alpha beta p q).numbered
    (regularIndex_card r alpha beta p q)).euclidean_preserves
  have hv := ((gaugeLayout r alpha beta h p q).numbered
    (gaugeIndex_card r alpha beta h p q)).euclidean_preserves
  exact (hu.prod ((MeasurePreserving.id (residualMatrixLebesgue p q h)).prod hv)).comp
    (regroup_preserves r alpha beta h p q)
```

Energy and norm identities are separate checks. `rawAmbientSq` sums all seven raw blocks' squared entries, whereas `coordinateAmbientSq` sums all nine coordinate blocks' squared entries. The nested layout identities reduce these to the corresponding finite entry sums. Concatenation also proves that the first sum equals $|A|_F^2 + |B|_F^2$ for the assembled raw matrices.

For the model map one proves both

$$N(\mathrm{coordinateToModel}(c)) = \mathrm{coordinateEnergy}(c),$$
$$Q(\mathrm{coordinateToModel}(c)) = \mathrm{coordinateAmbientSq}(c).$$

The first identity controls the scalar germ; the second controls the ball. They do not assert that the nonlinear elimination itself preserves squared norms. The band/sublevel identity is pulled back from the already proved zero-valued nonnegative model, and measurability follows from this same map.

```
def rawAmbientSq (w : Raw r alpha beta h p q) : ℝ :=
  frobeniusSq w.1 + frobeniusSq w.2.1 + frobeniusSq w.2.2.1 +
  frobeniusSq w.2.2.2.1 + frobeniusSq w.2.2.2.2.1 +
  frobeniusSq w.2.2.2.2.2.1 + frobeniusSq w.2.2.2.2.2.2

def coordinateAmbientSq (c : Coordinates r alpha beta h p q) : ℝ :=
  frobeniusSq c.1 + frobeniusSq c.2.1 + frobeniusSq c.2.2.1 +
  frobeniusSq c.2.2.2.1 + frobeniusSq c.2.2.2.2.1 +
  frobeniusSq c.2.2.2.2.2.1 + frobeniusSq c.2.2.2.2.2.2.1 +
  frobeniusSq c.2.2.2.2.2.2.1 + frobeniusSq c.2.2.2.2.2.2.2

lemma rawLayout_sq (w : Raw r alpha beta h p q) :
    (rawLayout r alpha beta h p q).sq w = rawAmbientSq r alpha beta h p q w := by
  unfold rawLayout rawAmbientSq
  exact (EntryLayout.sq_prod_of_eq _ _ w
    (matrixLayout_sq _ _ w.1)
    (EntryLayout.sq_prod_of_eq _ _ w.2
      (matrixLayout_sq _ _ w.2.1)
      (EntryLayout.sq_prod_of_eq _ _ w.2.2
        (matrixLayout_sq _ _ w.2.2.1)
        (EntryLayout.sq_prod_of_eq _ _ w.2.2.2
          (matrixLayout_sq _ _ w.2.2.2.1)
          (EntryLayout.sq_prod_of_eq _ _ w.2.2.2.2
            (matrixLayout_sq _ _ w.2.2.2.2.1)
            (EntryLayout.sq_prod_of_eq _ _ w.2.2.2.2.2
              (matrixLayout_sq _ _ w.2.2.2.2.2.1)
              (matrixLayout_sq _ _ w.2.2.2.2.2.2)))))))).trans
    (by simp only [add_assoc])
lemma coordinateLayout_sq (c : Coordinates r alpha beta h p q) :
    (coordinateLayout r alpha beta h p q).sq c =
      coordinateAmbientSq r alpha beta h p q c := by
  unfold coordinateLayout gaugeLayout coordinateAmbientSq
  exact (EntryLayout.sq_prod_of_eq _ _ c
    (matrixLayout_sq _ _ c.1)
    (EntryLayout.sq_prod_of_eq _ _ c.2
      (matrixLayout_sq _ _ c.2.1)
      (EntryLayout.sq_prod_of_eq _ _ c.2.2
        (matrixLayout_sq _ _ c.2.2.1)
        (EntryLayout.sq_prod_of_eq _ _ c.2.2.2
          (matrixLayout_sq _ _ c.2.2.2.1)
          (EntryLayout.sq_prod_of_eq _ _ c.2.2.2.2
            (matrixLayout_sq _ _ c.2.2.2.2.1)
            (EntryLayout.sq_prod_of_eq _ _ c.2.2.2.2.2
              (matrixLayout_sq _ _ c.2.2.2.2.2.1)
              (EntryLayout.sq_prod_of_eq _ _ c.2.2.2.2.2.2
                (matrixLayout_sq _ _ c.2.2.2.2.2.2.1)
                (matrixLayout_sq _ _ c.2.2.2.2.2.2.2)))))))))).trans
    (by simp only [add_assoc])

lemma rawAmbientSq_parameters (w : Raw r alpha beta h p q) :
```

```
    rawAmbientSq r alpha beta h p q w =
      frobeniusSq (parameterA w) + frobeniusSq (parameterB w) := by
  have hA : frobeniusSq (parameterA w) =
      (frobeniusSq w.1 + frobeniusSq w.2.1) +
        (frobeniusSq w.2.2.1 + frobeniusSq w.2.2.2.1) :=
    (frobeniusSq_hcat
      (MatrixCore.vcat w.1 w.2.1 : RM ((r+alpha)+(beta+h)) (r+alpha))
      (MatrixCore.vcat w.2.2.1 w.2.2.2.1 : RM ((r+alpha)+(beta+h)) q)).trans
      (congrArg (fun x y : ℝ => x+y)
        (frobeniusSq_vcat w.1 w.2.1)
        (frobeniusSq_vcat w.2.2.1 w.2.2.2.1))
  have hB : frobeniusSq (parameterB w) =
      frobeniusSq w.2.2.2.2.1 +
        (frobeniusSq w.2.2.2.2.2.1 + frobeniusSq w.2.2.2.2.2.2) :=
    (frobeniusSq_hcat w.2.2.2.2.1
      (MatrixCore.hcat w.2.2.2.2.2.1 w.2.2.2.2.2.2 : RM ((r+beta)+p) (beta+h))).trans
      (congrArg (fun x : ℝ => frobeniusSq w.2.2.2.2.1 + x)
        (frobeniusSq_hcat w.2.2.2.2.2.1 w.2.2.2.2.2.2))
  rw [hA, hB]
  simp only [rawAmbientSq, add_assoc]

lemma regularPack_norm_sq (u : Regular r alpha beta p q) :
    ‖regularPack r alpha beta p q ‖u^2 = frobeniusSq u.1 + frobeniusSq u.2 := by
  rw [regularPack, EntryLayout.euclidean_norm_sq, EntryLayout.sq_numbered]
  unfold regularLayout
  exact EntryLayout.sq_prod_of_eq _ _ u
    (matrixLayout_sq _ _ u.1) (matrixLayout_sq _ _ u.2)

lemma gaugePack_norm_sq (v : Gauge r alpha beta h p q) :
    ‖gaugePack r alpha beta h p q ‖v^2 = frobeniusSq v.1 + frobeniusSq v.2.1 +
      frobeniusSq v.2.2.1 + frobeniusSq v.2.2.2.1 + frobeniusSq v.2.2.2.2 := by
  rw [gaugePack, EntryLayout.euclidean_norm_sq, EntryLayout.sq_numbered]
  unfold gaugeLayout
  exact (EntryLayout.sq_prod_of_eq _ _ v
    (matrixLayout_sq _ _ v.1)
    (EntryLayout.sq_prod_of_eq _ _ v.2
      (matrixLayout_sq _ _ v.2.1)
      (EntryLayout.sq_prod_of_eq _ _ v.2.2
        (matrixLayout_sq _ _ v.2.2.1)
        (EntryLayout.sq_prod_of_eq _ _ v.2.2.2
          (matrixLayout_sq _ _ v.2.2.2.1)
          (matrixLayout_sq _ _ v.2.2.2.2))))).trans
    (by simp only [add_assoc])

lemma coordinateToModel_energy (c : Coordinates r alpha beta h p q) :
    model (coordinateToModel r alpha beta h p q c) = coordinateEnergy c := by
  change ‖regularPack r alpha beta p q (c.1,c.2.1)‖^2 +
    residualKernel (c.2.2.1,c.2.2.2.1) = coordinateEnergy c
  rw [regularPack_norm_sq]
  rfl

lemma coordinateToModel_ambient (c : Coordinates r alpha beta h p q) :
    modelAmbientSq (coordinateToModel r alpha beta h p q c) =
      coordinateAmbientSq r alpha beta h p q c := by
  change ‖regularPack r alpha beta p q (c.1,c.2.1)‖^2 +
    residualAmbientNormSq (c.2.2.1,c.2.2.2.1) +
    ‖gaugePack r alpha beta h p q c‖.2.2.2.2^2 = _
  rw [regularPack_norm_sq, gaugePack_norm_sq]
  simp only [residualAmbientNormSq, coordinateAmbientSq]
  ring

/-- The band/sublevel identification is transported from the same model,
```

```
not justified by changing the definition of the local invariant. -/
lemma coordinateBand_eq_sublevel (eps : ℝ) :
    {c : Coordinates r alpha beta h p q |
      |coordinateEnergy c - coordinateEnergy (0 : Coordinates r alpha beta h p q)| < eps} =
    {c | coordinateEnergy c < eps} := by
  let e := coordinateToModel r alpha beta h p q
  have hz : model (0 : ModelSpace (regularDim r alpha beta p q)
      (gaugeDim r alpha beta h p q) p q h) =
      coordinateEnergy (0 : Coordinates r alpha beta h p q) := by
    simpa only [map_zero] using coordinateToModel_energy r alpha beta h p q 0
  have he := congrArg (fun S => e ⁻¹' S)
    (modelBand_eq_sublevel (s := regularDim r alpha beta p q)
      (g := gaugeDim r alpha beta h p q) (p := p) (q := q) (h := h) eps)
  simpa only [e, preimage_ofPred_eq, coordinateToModel_energy, hz] using he

lemma measurable_coordinateEnergy :
    Measurable (coordinateEnergy : Coordinates r alpha beta h p q → ℝ) := by
  have hm := measurable_model.comp (coordinateToModel_preserves r alpha beta h p q).measurable
  simpa only [Function.comp_def, coordinateToModel_energy] using hm

lemma measurable_coordinateAmbientSq :
    Measurable (coordinateAmbientSq r alpha beta h p q) := by
  have hm := measurable_modelAmbientSq.comp
    (coordinateToModel_preserves r alpha beta h p q).measurable
  simpa only [Function.comp_def, coordinateToModel_ambient] using hm
```

Exact equality of coordinate and model volumes. The coordinate ball uses the sum of all nine blocks' squared entries. The two preceding identities show that the preimage of a model sublevel intersected with its ball is exactly the coordinate sublevel intersected with its ball. Both are measurable. Taking preimages under the proved measure-preserving equivalence gives

$$V_{\mathrm{coord}}(R,\varepsilon) = V_{\mathrm{model}}(R,\varepsilon)$$

for every R, ε, before any asymptotic limit. Therefore any residual pair supplies the coordinate pair with regular shift $s/2$. The Wishart and neutral wrappers instantiate it, and model-volume finiteness supplies coordinate-volume finiteness.

```
/-- The radius is measured in all nine blocks' entry squares. -/
def coordinateBall (R : ℝ) : Set (Coordinates r alpha beta h p q) :=
  {c | coordinateAmbientSq r alpha beta h p q c < R^2}
def coordinateVolume (R eps : ℝ) : ENNReal :=
  sublevelMass ((coordinateMeasure r alpha beta h p q).restrict
    (coordinateBall r alpha beta h p q R)) coordinateEnergy eps

/-- Equality holds for every R and eps, before any asymptotics. In particular
there is no Haar scalar, gamma factor, or epsilon-dependent gauge window. -/
theorem coordinateVolume_eq_modelVolume (R eps : ℝ) :
    coordinateVolume r alpha beta h p q R eps =
      modelVolume (regularDim r alpha beta p q) (gaugeDim r alpha beta h p q) p q h R eps := by
  let s := regularDim r alpha beta p q
  let g := gaugeDim r alpha beta h p q
  have hset : MeasurableSet ({w : ModelSpace s g p q h | model w < eps} ∩
      modelBall s g p q h R) :=
    (measurable_model measurableSet_Iio).inter
      (measurable_modelAmbientSq measurableSet_Iio)
  have he := (coordinateToModel_preserves r alpha beta h p q).measure_preimage
    hset.nullMeasurableSet
  have hpre : (coordinateToModel r alpha beta h p q) ⁻¹'
      ({w : ModelSpace s g p q h | model w < eps} ∩ modelBall s g p q h R) =
      {c | coordinateEnergy c < eps} ∩ coordinateBall r alpha beta h p q R := by
```

```
    ext c
    change (model (coordinateToModel r alpha beta h p q c) < eps ∧
      modelAmbientSq (coordinateToModel r alpha beta h p q c) < R^2) ↔
      coordinateEnergy c < eps ∧ coordinateAmbientSq r alpha beta h p q c < R^2
    rw [coordinateToModel_energy, coordinateToModel_ambient]
  have hcset : MeasurableSet
      {c : Coordinates r alpha beta h p q | coordinateEnergy c < eps} :=
    measurable_coordinateEnergy r alpha beta h p q measurableSet_Iio
  have hmset : MeasurableSet {w : ModelSpace s g p q h | model w < eps} :=
    measurable_model measurableSet_Iio
  unfold coordinateVolume modelVolume sublevelMass
  rw [Measure.restrict_apply hcset, Measure.restrict_apply hmset]
  rw [hpre] at he
  exact he

/-- Exact entry transport of any measured residual pair. The premise
concerns only the residual dimensions used by this coordinate model. -/
theorem coordinate_pair_of_residual {lam : ℝ} {m : ℕ}
    (hres : HasResidualBandPairAtOrigin p q h lam m) :
    RadiusOrder (coordinateVolume r alpha beta h p q)
      ((regularDim r alpha beta p q : ℝ)/2 + lam) m := by
  have hv : coordinateVolume r alpha beta h p q =
      modelVolume (regularDim r alpha beta p q) (gaugeDim r alpha beta h p q) p q h := by
    funext R eps
    exact coordinateVolume_eq_modelVolume r alpha beta h p q R eps
  rw [hv]
  exact normalForm_pair_of_residual (regularDim r alpha beta p q)
    (gaugeDim r alpha beta h p q) hres

theorem coordinate_pair_of_wishart (hW : O77.External.RealWishartEigenvalueFormula) :
    RadiusOrder (coordinateVolume r alpha beta h p q)
      (((regularDim r alpha beta p q : ℝ) + (O77.residualMin p q h : ℝ))/2)
      (O77.residualMultiplicity p q h) := by
  simpa only [add_div] using coordinate_pair_of_residual r alpha beta h p q
    (residualBandPair_all_dimensions_of_wishart_instantiated hW p q h)

theorem coordinate_pair_neutral (hz : p = 0 ∨ q = 0 ∨ h = 0) :
    RadiusOrder (coordinateVolume r alpha beta h p q)
      ((regularDim r alpha beta p q : ℝ)/2) 1 := by
  simpa only [add_zero] using coordinate_pair_of_residual r alpha beta h p q
    (residualBandPair_neutral_of_zero_dimension hz)

lemma coordinateVolume_ne_top {R : ℝ} (hR : 0 < R) (eps : ℝ) :
    coordinateVolume r alpha beta h p q R eps ≠ ⊤ := by
  rw [coordinateVolume_eq_modelVolume]
  exact modelVolume_ne_top _ _ _ _ _ hR eps

end O77.Coordinates0906
end
noncomputable section
open Set Filter MeasureTheory
open scoped BigOperators Topology ENNReal NNReal Matrix.Norms.Elementwise
namespace O77.Coordinates0906
open O77.Residual
open O77.Reconstruction0905
open O77.Transport0906
open O77.NormalForm0906
variable (r alpha beta h p q : ℕ)
```

One Euclidean carrier for differentiating the chart. Let $E_0 = \mathbb{R}^{H(I+O)}$ with its Euclidean norm and volume. The next finite product helpers prove the continuous scalar

multiplication and Borel-space instances for the already chosen entry topology; they introduce no new norm or measure. Using the cardinality identities above, flatten the raw and coordinate tuples separately to E_0, by maps E_R, E_C. Both are continuous linear equivalences, both preserve the specified measures exactly, and their squared norms are respectively `rawAmbientSq` and `coordinateAmbientSq`. Automatic continuity is invoked only for finite-dimensional linear maps.

```
abbrev Flat := Block (ambientDim r alpha beta h p q)

/-- Build the product evidence from the library's matrix-entry and product
instances explicitly, avoiding a search through nine nested factors. -/
private lemma continuousSMul_pair {E F : Type*}
    [TopologicalSpace E] [TopologicalSpace F] [SMul ℝ E] [SMul ℝ F]
    (hE : ContinuousSMul ℝ E) (hF : ContinuousSMul ℝ F) :
    ContinuousSMul ℝ (E × F) := by
  let := hE
  let := hF
  exact Prod.continuousSMul

private lemma matrixContinuousSMul (m n : ℕ) : ContinuousSMul ℝ (RM m n) :=
  inferInstanceAs (ContinuousSMul ℝ (Fin m → Fin n → ℝ))

private lemma rawContinuousSMul : ContinuousSMul ℝ (Raw r alpha beta h p q) :=
  continuousSMul_pair (matrixContinuousSMul (r+alpha) (r+alpha))
    (continuousSMul_pair (matrixContinuousSMul (beta+h) (r+alpha))
      (continuousSMul_pair (matrixContinuousSMul (r+alpha) q)
        (continuousSMul_pair (matrixContinuousSMul (beta+h) q)
          (continuousSMul_pair (matrixContinuousSMul ((r+beta)+p) (r+alpha))
            (continuousSMul_pair (matrixContinuousSMul ((r+beta)+p) beta)
              (matrixContinuousSMul ((r+beta)+p) h))))))

private lemma coordinateContinuousSMul :
    ContinuousSMul ℝ (Coordinates r alpha beta h p q) :=
  continuousSMul_pair (matrixContinuousSMul ((r+beta)+p) (r+alpha))
    (continuousSMul_pair (matrixContinuousSMul (r+beta) q)
      (continuousSMul_pair (matrixContinuousSMul p h)
        (continuousSMul_pair (matrixContinuousSMul h q)
          (continuousSMul_pair (matrixContinuousSMul (r+alpha) (r+alpha))
            (continuousSMul_pair (matrixContinuousSMul (beta+h) (r+alpha))
              (continuousSMul_pair (matrixContinuousSMul alpha q)
                (continuousSMul_pair (matrixContinuousSMul ((r+beta)+p) beta)
                  (matrixContinuousSMul (r+beta) h)))))))

/-- These are proofs for the existing matrix-entry topology and sigma
algebra. The outer Pi and product instances are applied explicitly. -/
lemma matrix_borelSpace (m n : ℕ) : BorelSpace (RM m n) := by
  let : BorelSpace (Fin n → ℝ) := Pi.borelSpace (X := fun _ : Fin n => ℝ)
  let : SecondCountableTopology (Fin n → ℝ) := inferInstance
  exact Pi.borelSpace (X := fun _ : Fin m => Fin n → ℝ)

/-- Second countability of the left matrix factor is enough; no nested
search through the right-hand tuple is needed. -/
lemma matrix_prod_borelSpace (m n : ℕ) {F : Type*}
    [TopologicalSpace F] [MeasurableSpace F] (hF : BorelSpace F) :
    BorelSpace (RM m n × F) := by
  let := matrix_borelSpace m n
  let := hF
  let : SecondCountableTopology (RM m n) :=
    inferInstanceAs (SecondCountableTopology (Fin m → Fin n → ℝ))
  let : SecondCountableTopologyEither (RM m n) F := (Or.inl )inferInstance
  exact Prod.borelSpace α( := RM m n) β( := F)
```

```
local instance rawBorelSpace : BorelSpace (Raw r alpha beta h p q) :=
  matrix_prod_borelSpace (r+alpha) (r+alpha)
    (matrix_prod_borelSpace (beta+h) (r+alpha)
      (matrix_prod_borelSpace (r+alpha) q
        (matrix_prod_borelSpace (beta+h) q
          (matrix_prod_borelSpace ((r+beta)+p) (r+alpha)
            (matrix_prod_borelSpace ((r+beta)+p) beta
              (matrix_borelSpace ((r+beta)+p) h))))))

local instance coordinateBorelSpace : BorelSpace (Coordinates r alpha beta h p q) :=
  matrix_prod_borelSpace ((r+beta)+p) (r+alpha)
    (matrix_prod_borelSpace (r+beta) q
      (matrix_prod_borelSpace p h
        (matrix_prod_borelSpace h q
          (matrix_prod_borelSpace (r+alpha) (r+alpha)
            (matrix_prod_borelSpace (beta+h) (r+alpha)
              (matrix_prod_borelSpace alpha q
                (matrix_prod_borelSpace ((r+beta)+p) beta
                  (matrix_borelSpace (r+beta) h))))))))

/-- Both matrix tuples are put on exactly the same Euclidean carrier.
Automatic continuity is used only for finite-dimensional linear maps. -/
def rawFlat : Raw r alpha beta h p q ≃L[ℝ] Flat r alpha beta h p q := by
  letI := rawContinuousSMul r alpha beta h p q
  exact (((rawLayout r alpha beta h p q).numbered
    (rawIndex_card r alpha beta h p q)).euclidean).toContinuousLinearEquiv

def coordinateFlat : Coordinates r alpha beta h p q ≃L[ℝ] Flat r alpha beta h p q := by
  letI := coordinateContinuousSMul r alpha beta h p q
  exact (((coordinateLayout r alpha beta h p q).numbered
    (coordinateIndex_card r alpha beta h p q)).euclidean).toContinuousLinearEquiv

lemma rawFlat_preserves : MeasurePreserving (rawFlat r alpha beta h p q)
    (rawMeasure r alpha beta h p q) volume :=
  ((rawLayout r alpha beta h p q).numbered
    (rawIndex_card r alpha beta h p q)).euclidean_preserves

lemma coordinateFlat_preserves : MeasurePreserving (coordinateFlat r alpha beta h p q)
    (coordinateMeasure r alpha beta h p q) volume :=
  ((coordinateLayout r alpha beta h p q).numbered
    (coordinateIndex_card r alpha beta h p q)).euclidean_preserves

lemma rawFlat_norm_sq (w : Raw r alpha beta h p q) :
    ‖rawFlat r alpha beta h p q ‖w^2 = rawAmbientSq r alpha beta h p q w := by
  change ‖((rawLayout r alpha beta h p q).numbered
    (rawIndex_card r alpha beta h p q)).euclidean ‖w^2 = _
  rw [EntryLayout.euclidean_norm_sq, EntryLayout.sq_numbered, rawLayout_sq]

lemma coordinateFlat_norm_sq (c : Coordinates r alpha beta h p q) :
    ‖coordinateFlat r alpha beta h p q ‖c^2 = coordinateAmbientSq r alpha beta h p q c := by
  change ‖((coordinateLayout r alpha beta h p q).numbered
    (coordinateIndex_card r alpha beta h p q)).euclidean ‖c^2 = _
  rw [EntryLayout.euclidean_norm_sq, EntryLayout.sq_numbered, coordinateLayout_sq]
```

The flattened nonlinear maps and their domains. Let c_0 be the canonical coordinate tuple. Define

$$f = E_C \circ (\mathrm{encode} - c_0) \circ E_R^{-1}, \qquad g = E_R \circ (z \mapsto \mathrm{decode}(z + c_0)) \circ E_C^{-1}.$$

The flattened source and target are the preimages of the corresponding open chart domains; $x_0 = E_R(\mathrm{decode}(c_0))$. Substitution of the inverse identities shows $f(x_0) = 0$, $g(0) = x_0$, $gf = 1$

on the source, and $fg = 1$ on the target. Analyticity of the original maps and continuity of the linear equivalences give C^1 regularity. Thus the derivative of g is an endomorphism of exactly the Euclidean carrier whose measure is being used.

```
local notation "ER" => rawFlat r alpha beta h p q
local notation "EC" => coordinateFlat r alpha beta h p q
local notation "C0" => canonicalCoordinates r alpha beta h p q
local notation "E0" => Flat r alpha beta h p q

def flatEncode (x : E0) : E0 := EC (centeredEncode C0 ((ER).symm x))
def flatDecode (z : E0) : E0 := ER (centeredDecode C0 ((EC).symm z))
def flatSource : Set E0 := (ER).symm ⁻¹' sourceDomain
def flatTarget : Set E0 := (EC).symm ⁻¹' centeredTarget C0
def flatBase : E0 := ER (decode C0)

lemma flatSource_isOpen : IsOpen (flatSource r alpha beta h p q) :=
  sourceDomain_isOpen.preimage (ER).symm.continuous
lemma flatTarget_isOpen : IsOpen (flatTarget r alpha beta h p q) :=
  (centeredTarget_isOpen C0).preimage (EC).symm.continuous

lemma flatEncode_mem (x : E0) (hx : x ∈ flatSource r alpha beta h p q) :
    flatEncode r alpha beta h p q x ∈ flatTarget r alpha beta h p q := by
  change (EC).symm (EC (centeredEncode C0 ((ER).symm x))) ∈ centeredTarget C0
  rw [ContinuousLinearEquiv.symm_apply_apply]
  exact (centeredElimination C0).map_source hx

lemma flatDecode_mem (z : E0) (hz : z ∈ flatTarget r alpha beta h p q) :
    flatDecode r alpha beta h p q z ∈ flatSource r alpha beta h p q := by
  change (ER).symm (ER (centeredDecode C0 ((EC).symm z))) ∈ sourceDomain
  rw [ContinuousLinearEquiv.symm_apply_apply]
  exact (centeredElimination C0).map_target hz

lemma flat_decode_encode (x : E0) (hx : x ∈ flatSource r alpha beta h p q) :
    flatDecode r alpha beta h p q (flatEncode r alpha beta h p q x) = x := by
  simp only [flatDecode, flatEncode, ContinuousLinearEquiv.symm_apply_apply,
    centeredDecode_encode C0 ((ER).symm x) hx, ContinuousLinearEquiv.apply_symm_apply]

lemma flat_encode_decode (z : E0) (hz : z ∈ flatTarget r alpha beta h p q) :
    flatEncode r alpha beta h p q (flatDecode r alpha beta h p q z) = z := by
  simp only [flatEncode, flatDecode, ContinuousLinearEquiv.symm_apply_apply,
    centeredEncode_decode C0 ((EC).symm z) hz, ContinuousLinearEquiv.apply_symm_apply]

lemma flat_base_points :
    (0 : E0) ∈ flatTarget r alpha beta h p q ∧
    flatBase r alpha beta h p q ∈ flatSource r alpha beta h p q ∧
    flatEncode r alpha beta h p q (flatBase r alpha beta h p q) = 0 ∧
    flatDecode r alpha beta h p q 0 = flatBase r alpha beta h p q := by
  have hc : coordinateGuard C0 := canonicalCoordinates_guard
  have hz : (0 : E0) ∈ flatTarget r alpha beta h p q := by
    change coordinateGuard ((EC).symm 0 + C0)
    simpa only [map_zero, zero_add] using hc
  have hbase : flatBase r alpha beta h p q ∈ flatSource r alpha beta h p q := by
    change rawGuard ((ER).symm (ER (decode C0)))
    simpa only [ContinuousLinearEquiv.symm_apply_apply] using (decode_guard C0).2 hc
  refine ⟨hz, hbase, ?_, ?⟩_
  · simp only [flatEncode, flatBase, ContinuousLinearEquiv.symm_apply_apply,
      centeredEncode, encode_decode C0 hc, sub_self, map_zero]
  · simp only [flatDecode, centeredDecode, map_zero, zero_add, flatBase]

/-- C1 is all the change-of-variables theorem needs. Analyticity of the
original nonlinear maps comes from the inherited checked module. -/
lemma flatEncode_C1 (x : E0) (hx : x ∈ flatSource r alpha beta h p q) :
    ContDiffAt ℝ 1 (flatEncode r alpha beta h p q) x := by
```

```
  have hc : ContDiffAt ℝ 1 (centeredEncode C0) ((ER).symm x) :=
    (centeredEncode_analyticAt C0 ((ER).symm x) hx).contDiffAt
  exact (EC).contDiff.contDiffAt.comp x (hc.comp x (ER).symm.contDiff.contDiffAt)

lemma flatDecode_C1 (z : E0) (hz : z ∈ flatTarget r alpha beta h p q) :
    ContDiffAt ℝ 1 (flatDecode r alpha beta h p q) z := by
  have hc : ContDiffAt ℝ 1 (centeredDecode C0) ((EC).symm z) :=
    (centeredDecode_analyticAt C0 ((EC).symm z) hz).contDiffAt
  exact (ER).contDiff.contDiffAt.comp z (hc.comp z (EC).symm.contDiff.contDiffAt)
```

The inverse density and its uniform neighborhood. Differentiating $fg = 1$ on the open target gives $Df(g(z))Dg(z) = 1$. Taking determinants shows $\det Dg(z) \neq 0$. At zero, C^1 regularity makes Dg continuous, so a fixed open neighborhood V has positive constants ℓ, u with $\ell \leq |\det Dg(z)| \leq u$ for every $z \in V$. The library consequence `ContDiffAt.exists_lipschitzOnWith` also provides local Lipschitz bounds for both maps, used later to choose compatible balls. The change-of-variables result now gives

$$\ell \operatorname{vol}(S) \leq \operatorname{vol}(gS) \leq u \operatorname{vol}(S) \quad (S \subseteq V \text{ measurable}).$$

These are actual bounds for the explicit decoder's Jacobian; the decoder has not been declared measure preserving.

```
/-- The chain rule is applied to an actual identity on an open set.
The derivative is an endomorphism only after the two enumerations. -/
theorem flat_inverse_derivative (z : E0) (hz : z ∈ flatTarget r alpha beta h p q) :
    (fderiv ℝ (flatEncode r alpha beta h p q) (flatDecode r alpha beta h p q z)).comp
      (fderiv ℝ (flatDecode r alpha beta h p q) z) = ContinuousLinearMap.id ℝ E0 := by
  apply derivative_left_inverse
    ((flatDecode_C1 r alpha beta h p q z hz).differentiableAt one_ne_zero).hasFDerivAt
    ((flatEncode_C1 r alpha beta h p q _ (flatDecode_mem r alpha beta h p q z hz)).differentiableAt
     one_ne_zero).hasFDerivAt
  filter_upwards [(flatTarget_isOpen r alpha beta h p q).mem_nhds hz] with y hy
  exact flat_encode_decode r alpha beta h p q y hy

lemma flat_inverse_det_ne_zero (z : E0) (hz : z ∈ flatTarget r alpha beta h p q) :
    (fderiv ℝ (flatDecode r alpha beta h p q) z).det ≠ 0 :=
  det_ne_zero_of_left_inverse _ _ (flat_inverse_derivative r alpha beta h p q z hz)

/-- A positive bounded inverse density on a fixed open neighborhood of
zero, obtained without postulating a Jacobian or a loss-band estimate. -/
theorem flat_local_jacobian_bounds :
    ∃ (V : Set E0) (lo hi : ℝ), IsOpen V ∧ (0 : E0) ∈ V ∧
      V ⊆ flatTarget r alpha beta h p q ∧ 0 < lo ∧ 0 < hi ∧
      ∀ z ∈ V, lo ≤ |(fderiv ℝ (flatDecode r alpha beta h p q) z).det| ∧
        |(fderiv ℝ (flatDecode r alpha beta h p q) z).det| ≤ hi := by
  have hzero := (flat_base_points r alpha beta h p q).1
  have hdet := flat_inverse_det_ne_zero r alpha beta h p q 0 hzero
  have hcont := (flatDecode_C1 r alpha beta h p q 0 hzero).continuousAt_fderiv one_ne_zero
  obtain ⟨V, hV, h0, ⟩hbound := local_abs_det_bounds hcont hdet
  let J := |(fderiv ℝ (flatDecode r alpha beta h p q) (0 : E0)).det|
  have hJ : 0 < J := abs_pos.mpr hdet
  refine ⟨V ∩ flatTarget r alpha beta h p q, J/2, 2*J,
    hV.inter (flatTarget_isOpen r alpha beta h p q), ⟨h0,⟩hzero,
    inter_subset_right, half_pos hJ, mul_pos (by norm_num) hJ, ?⟩_
  exact fun z hz => hbound z hz.1

/-- Direct library reuse: no compact-ball mean-value proof is rebuilt. -/
theorem flat_local_lipschitz :
    ∃( C : ℝ≥0, ∃ U ∈ nhds (flatBase r alpha beta h p q),
      LipschitzOnWith C (flatEncode r alpha beta h p q) U) ∧
```

```
    ∃( C : ℝ≥0, ∃ V ∈ nhds (0 : E0),
      LipschitzOnWith C (flatDecode r alpha beta h p q) V) :=
  ⟨(flatEncode_C1 r alpha beta h p q _
      (flat_base_points r alpha beta h p q).2.1).exists_lipschitzOnWith,
    (flatDecode_C1 r alpha beta h p q 0
      (flat_base_points r alpha beta h p q).1).⟩exists_lipschitzOnWith

/-- The previous abstract Jacobian theorem is now instantiated with the
actual flattened nonlinear decoder, not with an assumed transport field. -/
theorem flat_local_image_bounds :
    ∃ (V : Set E0) (lo hi : ℝ), IsOpen V ∧ (0 : E0) ∈ V ∧
      V ⊆ flatTarget r alpha beta h p q ∧ 0 < lo ∧ 0 < hi ∧
      ∀ S : Set E0, MeasurableSet S → S ⊆ V →
        MeasurableSet (flatDecode r alpha beta h p q '' S) ∧
        ENNReal.ofReal lo * volume S ≤ volume (flatDecode r alpha beta h p q '' S) ∧
        volume (flatDecode r alpha beta h p q '' S) ≤ ENNReal.ofReal hi * volume S := by
  obtain ⟨V, lo, hi, hV, h0, hVT, hlo, hhi, ⟩hJ :=
    flat_local_jacobian_bounds r alpha beta h p q
  refine ⟨V, lo, hi, hV, h0, hVT, hlo, hhi, ?⟩_
  intro S hS hSV
  apply image_measure_bounds (volume : Measure E0) hS hSV
    (fun z hz => ((flatDecode_C1 r alpha beta h p q z (hVT hz)).differentiableAt one_ne_zero).
     hasFDerivAt)
    (inverse_injOn (fun z hz => flat_encode_decode r alpha beta h p q z (hVT hz)))
    hJ
```

Returning the image estimate to tuple measures. For a coordinate set S, the identity $g(E_C S) = E_R(\mathrm{centeredDecode}(S))$ follows directly from the definition of g. The homeomorphism E_C preserves measurability of images, so the flat estimate applies to $E_C S$. Exact measure preservation of E_R and E_C then returns the same bounds to the raw and coordinate entry measures. This proves `centered_decode_image_bounds` with one open neighborhood and constants for every measurable subset. The final numerical examples check the dimension partition $77 = 20 + 39 + 18$ and the zero-dimensional coordinate pair.

```
/-- Actual matrix-entry measures on both sides of the nonlinear chart.
The hypotheses quantify over measurable sets, not over local-pair data. -/
theorem centered_decode_image_bounds :
    ∃ (V : Set (Coordinates r alpha beta h p q)) (lo hi : ℝ),
      IsOpen V ∧ (0 : Coordinates r alpha beta h p q) ∈ V ∧
      V ⊆ centeredTarget C0 ∧ 0 < lo ∧ 0 < hi ∧
      ∀ S : Set (Coordinates r alpha beta h p q), MeasurableSet S → S ⊆ V →
        MeasurableSet (centeredDecode C0 '' S) ∧
        ENNReal.ofReal lo * coordinateMeasure r alpha beta h p q S ≤
          rawMeasure r alpha beta h p q (centeredDecode C0 '' S) ∧
        rawMeasure r alpha beta h p q (centeredDecode C0 '' S) ≤
          ENNReal.ofReal hi * coordinateMeasure r alpha beta h p q S := by
  obtain ⟨V, lo, hi, hV, h0, hVT, hlo, hhi, ⟩hbound :=
    flat_local_image_bounds r alpha beta h p q
  refine ⟨(EC ⁻¹' V), lo, hi, hV.preimage (EC).continuous, ?_, ?_, hlo, hhi, ?⟩_
  · simpa only [mem_preimage, map_zero] using h0
  · intro c hc
    have ht := hVT hc
    change (EC).symm (EC c) ∈ centeredTarget C0 at ht
    simpa only [ContinuousLinearEquiv.symm_apply_apply] using ht
  · intro S hS hSV
    have hCS : MeasurableSet (EC '' S) :=
      (EC).toHomeomorph.toMeasurableEquiv.measurableEmbedding.measurableSet_image.mpr hS
    have hCSV : EC '' S ⊆ V := by
      rintro z ⟨c, hc, ⟩rfl
      exact hSV hc
```

```
    have hb := hbound (EC '' S) hCS hCSV
    have hset : flatDecode r alpha beta h p q '' (EC '' S) =
        ER '' (centeredDecode C0 '' S) := by
      simp only [image_image, flatDecode, ContinuousLinearEquiv.symm_apply_apply]
    have hCmass := (coordinateFlat_preserves r alpha beta h p q).measure_preimage
      hCS.nullMeasurableSet
    rw [preimage_image_eq _ (EC).injective] at hCmass
    have hRmass := (rawFlat_preserves r alpha beta h p q).measure_preimage
      hb.1.nullMeasurableSet
    rw [hset, preimage_image_eq _ (ER).injective] at hRmass
    have hmeas := hb.1.preimage (ER).continuous.measurable
    rw [hset, preimage_image_eq _ (ER).injective] at hmeas
    refine ⟨hmeas, ?_, ?⟩_
    · simpa only [hset, ← hCmass, ← hRmass] using hb.2.1
    · simpa only [hset, ← hCmass, ← hRmass] using hb.2.2

-- All-dimensional regression consequences.
example : ambientDim 1 2 1 3 2 4 = 77 := by norm_num [ambientDim]
example : regularDim 1 2 1 2 4 = 20 := by norm_num [regularDim]
example : gaugeDim 1 2 1 3 2 4 = 39 := by norm_num [gaugeDim]
example : RadiusOrder (coordinateVolume 0 0 0 0 0 0) 0 1 := by
  simpa [regularDim] using coordinate_pair_neutral 0 0 0 0 0 0 (Or.inl rfl)

end O77.Coordinates0906
end
```

36 Module `O77Canonical`

36.1 The native measured predicate and canonical attachment

For fixed $I, O, H \in \mathbb{N}$, the native parameter space consists of $w = (A, B) \in \mathbb{R}^{H\times I} \times \mathbb{R}^{O\times H}$. Its measure is the product Lebesgue measure in these original entries, and its specified norm is

$$\|w\|_{\mathrm{entry}} = \sqrt{|A|_F^2 + |B|_F^2}.$$

For a target Φ, define $L_\Phi(A, B) = |BA - \Phi|_F^2/2$ and

$$V_\Phi(w; R, \varepsilon) = \mathrm{vol}\{v : \|v - w\|_{\mathrm{entry}} < R,\ |L_\Phi(v) - L_\Phi(w)| < \varepsilon\}.$$

The following `HasLocalBandPair` definition applies `RadiusOrder` to this actual function V_Φ. Hence it means $\lambda \geq 0$, $m \geq 1$ and, for every sufficiently small positive fixed radius, two-sided bounds by positive constant multiples of $\varepsilon^\lambda \log(1/\varepsilon)^{m-1}$ for all sufficiently small positive ε. Constants and the cutoff may depend on the radius. The candidate rank formula, a chart, and Wishart do not occur in this definition.

```
import Mathlib
import O77Coordinates

set_option autoImplicit false
noncomputable section
open Set Filter MeasureTheory
open scoped BigOperators Topology ENNReal NNReal Matrix.Norms.Elementwise

namespace O77.Native
open O77.Residual
open O77.Reconstruction0905 (RM)
open O77.Coordinates0906 (EntryLayout matrixLayout matrixLayout_sq)
open O77.NormalForm0906 (Block)
open O77.Transport0906 (RadiusOrder)
```

```
/-- Native matrices, in the input/output/hidden order of the specification. -/
abbrev Parameters (I O H : ℕ) := RM H I × RM O H

/-- The measurable structure is the product of the actual matrix entries.
The norm scopes already use this same product topology. -/
local instance parametersBorelSpace (I O H : ℕ) :
    BorelSpace (Parameters I O H) :=
  O77.Coordinates0906.matrix_prod_borelSpace H I
    (O77.Coordinates0906.matrix_borelSpace O H)

def entryMeasure (I O H : ℕ) : Measure (Parameters I O H) :=
  (matrixLebesgue (Fin H) (Fin I)).prod (matrixLebesgue (Fin O) (Fin H))

def ambientSq {I O H : ℕ} (w : Parameters I O H) : ℝ :=
  frobeniusSq w.1 + frobeniusSq w.2

def entryNorm {I O H : ℕ} (w : Parameters I O H) : ℝ :=
  Real.sqrt (ambientSq w)

def loss {I O H : ℕ} (Phi : RM O I) (w : Parameters I O H) : ℝ :=
  frobeniusSq (w.2 * w.1 - Phi) / 2

/-- Unlike a bare squared-radius test, this is empty for nonpositive R. -/
def entryBall {I O H : ℕ} (w : Parameters I O H) (R : ℝ) : Set (Parameters I O H) :=
  {v | entryNorm (v-w) < R}

def bandSet {I O H : ℕ} (Phi : RM O I) (w : Parameters I O H) (R eps : ℝ) :
    Set (Parameters I O H) :=
  {v | v ∈ entryBall w R ∧ |loss Phi v - loss Phi w| < eps}

def bandVolume {I O H : ℕ} (Phi : RM O I) (w : Parameters I O H) (R eps : ℝ) : ENNReal :=
  entryMeasure I O H (bandSet Phi w R eps)

/-- The actual signed, radius-first predicate; no geometric witness or
Wishart premise is part of its definition. -/
def HasLocalBandPair {I O H : ℕ} (Phi : RM O I) (w : Parameters I O H)
    (lam : ℝ) (m : ℕ) : Prop :=
  RadiusOrder (bandVolume Phi w) lam m
```

Verifying the intended metric and real-valued reading. The native layout enumerates all $HI + OH = H(I + O)$ entries exactly once. Its flattening E_N preserves the original measure and satisfies $\|E_N w\|^2 = |A|_F^2 + |B|_F^2$. Thus the specified entry ball is precisely the preimage of the Euclidean ball centered at $E_N w$ with the same radius, even for nonpositive radii (where both are empty). The loss is continuous, so bands are measurable. Every band lies in a finite-volume ball, proving `bandVolume_ne_top` before `toReal` appears. The equivalence `hasLocalBandPair_iff_real` spells out the same inequalities with ordinary real volumes. It uses `ENNReal.toReal_le_toReal` only after checking finiteness of every quantity; the positive small-parameter cutoff also guarantees that the power–log factor is nonnegative. This is an explicit interpretation of the formal contract, not a second definition of it.

```
variable (I O H : ℕ)

def layout : EntryLayout (Parameters I O H)
    ((Fin H × Fin I) ⊕ (Fin O × Fin H)) (entryMeasure I O H) :=
  (matrixLayout H I).prod (matrixLayout O H)

lemma layout_card : Fintype.card ((Fin H × Fin I) ⊕ (Fin O × Fin H)) = H*(I+O) := by
  simp only [Fintype.card_sum, Fintype.card_prod, Fintype.card_fin]
```

```
  ring

def flat : Parameters I O H ≃L[ℝ] Block (H*(I+O)) :=
  (((layout I O H).numbered (layout_card I O H)).euclidean).toContinuousLinearEquiv

lemma flat_preserves : MeasurePreserving (flat I O H) (entryMeasure I O H) volume :=
  ((layout I O H).numbered (layout_card I O H)).euclidean_preserves

lemma flat_norm_sq (w : Parameters I O H) : ‖flat I O H ‖w^2 = ambientSq w := by
  change ‖((layout I O H).numbered (layout_card I O H)).euclidean ‖w^2 = _
  rw [EntryLayout.euclidean_norm_sq, EntryLayout.sq_numbered]
  exact EntryLayout.sq_prod_of_eq (matrixLayout H I) (matrixLayout O H) w
    (matrixLayout_sq H I w.1) (matrixLayout_sq O H w.2)

lemma entryNorm_eq (w : Parameters I O H) : entryNorm w = ‖flat I O H ‖w := by
  rw [entryNorm, ← flat_norm_sq I O H w, Real.sqrt_sq (norm_nonneg _)]

lemma entryBall_eq (w : Parameters I O H) (R : ℝ) :
    entryBall w R = (flat I O H) ⁻¹' Metric.ball (flat I O H w) R := by
  ext v
  simp only [entryBall, mem_ofPred_eq, mem_preimage, Metric.mem_ball,
    dist_eq_norm, entryNorm_eq, map_sub]

lemma entryBall_mass (w : Parameters I O H) (R : ℝ) :
    entryMeasure I O H (entryBall w R) = volume (Metric.ball (flat I O H w) R) := by
  rw [entryBall_eq]
  exact (flat_preserves I O H).measure_preimage measurableSet_ball.nullMeasurableSet

lemma continuous_loss (Phi : RM O I) : Continuous (loss (H := H) Phi) := by
  unfold loss frobeniusSq
  fun_prop

lemma measurable_bandSet (Phi : RM O I) (w : Parameters I O H) (R eps : ℝ) :
    MeasurableSet (bandSet Phi w R eps) := by
  have hball : MeasurableSet (entryBall w R) := by
    rw [entryBall_eq]
    exact measurableSet_ball.preimage (flat I O H).continuous.measurable
  exact hball.inter
    ((((continuous_loss I O H Phi).sub continuous_const).abs).measurable measurableSet_Iio)

/-- Finiteness is proved before any real-valued interpretation. -/
lemma bandVolume_ne_top (Phi : RM O I) (w : Parameters I O H) (R eps : ℝ) :
    bandVolume Phi w R eps ≠ ⊤ := by
  apply ne_of_lt
  calc
    bandVolume Phi w R eps ≤ entryMeasure I O H (entryBall w R) :=
      measure_mono (fun _ hv => hv.1)
    _ = volume (Metric.ball (flat I O H w) R) := entryBall_mass I O H w R
    _ < ⊤ := measure_ball_lt_top

/-- This is the real-valued reading of the same normalized inequalities,
not a second asymptotic convention. The proof uses library order conversion. -/
theorem hasLocalBandPair_iff_real (Phi : RM O I) (w : Parameters I O H)
    (lam : ℝ) (m : ℕ) :
    HasLocalBandPair Phi w lam m ↔
      0 ≤ lam ∧ 1 ≤ m ∧ ∃ rho : ℝ, 0 < rho ∧
        ∀ R, 0 < R → R < rho → ∃ c C e0 : ℝ,
          0 < c ∧ c ≤ C ∧ 0 < e0 ∧ e0 < Real.exp (-1) ∧
          ∀ eps, 0 < eps → eps < e0 →
            c * powerLogZeroReal lam (m-1) eps ≤ (bandVolume Phi w R eps).toReal ∧
            (bandVolume Phi w R eps).toReal ≤ C * powerLogZeroReal lam (m-1) eps := by
  have hbound (R eps c C : ℝ) (hc : 0 < c) (hcC : c ≤ C)
```

```
      (heps : 0 < eps) (heps1 : eps ≤ 1) :
      (ENNReal.ofReal c * powerLogZero lam (m-1) eps ≤ bandVolume Phi w R eps ∧
        bandVolume Phi w R eps ≤ ENNReal.ofReal C * powerLogZero lam (m-1) eps) ↔
      (c * powerLogZeroReal lam (m-1) eps ≤ (bandVolume Phi w R eps).toReal ∧
        (bandVolume Phi w R eps).toReal ≤ C * powerLogZeroReal lam (m-1) eps) := by
    have ht := powerLogZeroReal_nonneg (lam := lam) (k := m-1) heps heps1
    have hfin := bandVolume_ne_top I O H Phi w R eps
    have hcfin : ENNReal.ofReal c * powerLogZero lam (m-1) eps ≠ ⊤ := by
      exact ENNReal.mul_ne_top ENNReal.ofReal_ne_top ENNReal.ofReal_ne_top
    have hCfin : ENNReal.ofReal C * powerLogZero lam (m-1) eps ≠ ⊤ := by
      exact ENNReal.mul_ne_top ENNReal.ofReal_ne_top ENNReal.ofReal_ne_top
    rw ←[ ENNReal.toReal_le_toReal hcfin hfin,
      ← ENNReal.toReal_le_toReal hfin hCfin]
    simp only [ENNReal.toReal_mul, powerLogZero,
      ENNReal.toReal_ofReal hc.le, ENNReal.toReal_ofReal (hc.le.trans hcC),
      ENNReal.toReal_ofReal ht]
  constructor <;> rintro ⟨hlam, hm, rho, hrho, ⟩hR <;>
    refine ⟨hlam, hm, rho, hrho, ?⟩_ <;> intro R hRp hRrho <;>
    obtain ⟨c, C, e0, hc, hcC, he0, he0exp, ⟩hb := hR R hRp hRrho <;>
    refine ⟨c, C, e0, hc, hcC, he0, he0exp, ?⟩_ <;> intro eps heps heps0
  · exact (hbound R eps c C hc hcC heps
      ((heps0.trans he0exp).trans (Real.exp_lt_one_iff.mpr (by norm_num))).le).1
        (hb eps heps heps0)
  · exact (hbound R eps c C hc hcC heps
      ((heps0.trans he0exp).trans (Real.exp_lt_one_iff.mpr (by norm_num))).le).2
        (hb eps heps heps0)
end O77.Native
end
noncomputable section
open Set Filter MeasureTheory
open scoped BigOperators Topology ENNReal NNReal Matrix.Norms.Elementwise
namespace O77.Canonical0906
attribute [local instance] O77.Native.parametersBorelSpace
  O77.Coordinates0906.rawBorelSpace O77.Coordinates0906.coordinateBorelSpace
open O77.Residual
open O77.Reconstruction0905
open O77.Coordinates0906
open O77.Transport0906
open O77.NormalForm0906
```

Choosing compatible balls once. Suppose local Lipschitz maps f, g send basepoints x_0, y_0 to one another, with neighborhoods U, V. Take local Lipschitz constants C_f, C_g and choose $L = \max(1, C_f, C_g)$. Choose radii $r_x, r_y > 0$ whose balls lie inside the needed neighborhoods and Lipschitz domains, and set $\rho = \min(r_x, r_y/L)$. For every $0 < \delta < \rho$ one then has

$$B(x_0, \delta) \subseteq U, \quad B(y_0, L\delta) \subseteq V, \quad g(B(y_0, \delta/L)) \subseteq B(x_0, \delta), \quad f(B(x_0, \delta)) \subseteq B(y_0, L\delta).$$

The proof is just the two Lipschitz inequalities and the basepoint equations. The same L, ρ work for all these radii. This lemma assumes no Jacobian or scalar-loss comparison, so those conditions must later be put on the same chosen neighborhoods explicitly.

```
section CompatibleBalls
variable {E F : Type*} [PseudoMetricSpace E] [PseudoMetricSpace F]
  {f : E → F} {g : F → E} {x0 : E} {y0 : F} {U : Set E} {V : Set F}

/-- Radius arithmetic for two local Lipschitz maps. Inverse identities,
Jacobian estimates and loss comparison are not smuggled into this lemma. -/
theorem compatible_balls
    (hU : U ∈ nhds x0) (hV : V ∈ nhds y0)
    (hf0 : f x0 = y0) (hg0 : g y0 = x0)
```

```
    (hf : ∃ C : ℝ≥0, ∃ S ∈ nhds x0, LipschitzOnWith C f S)
    (hg : ∃ C : ℝ≥0, ∃ S ∈ nhds y0, LipschitzOnWith C g S) :
    ∃ L rho : ℝ, 1 ≤ L ∧ 0 < rho ∧
      ∀ delta, 0 < delta → delta < rho →
        Metric.ball x0 delta ⊆ U ∧ Metric.ball y0 (L*delta) ⊆ V ∧
        MapsTo g (Metric.ball y0 (delta/L)) (Metric.ball x0 delta) ∧
        MapsTo f (Metric.ball x0 delta) (Metric.ball y0 (L*delta)) := by
  obtain ⟨Cf, Sf, hSf, ⟩hfLip := hf
  obtain ⟨Cg, Sg, hSg, ⟩hgLip := hg
  obtain ⟨rx, hrx, ⟩hxball := Metric.mem_nhds_iff.1 (inter_mem hU hSf)
  obtain ⟨rz, hrz, ⟩hzball := Metric.mem_nhds_iff.1 (inter_mem hV hSg)
  let L : ℝ := max 1 (max (Cf : ℝ) (Cg : ℝ))
  have hL : 1 ≤ L := le_max_left _ _
  have hLp : 0 < L := zero_lt_one.trans_le hL
  have hCf : (Cf : ℝ) ≤ L := (le_max_left _ _).trans (le_max_right _ _)
  have hCg : (Cg : ℝ) ≤ L := (le_max_right _ _).trans (le_max_right _ _)
  refine ⟨L, min rx (rz/L), hL, lt_min hrx (div_pos hrz hLp), ?⟩_
  intro delta hd hd0
  have hdx : delta < rx := hd0.trans_le (min_le_left _ _)
  have hdz : L*delta < rz := by
    simpa only [mul_comm] using
      (₀lt_div_iff hLp).1 (hd0.trans_le (min_le_right _ _))
  have hsmall : delta/L ≤ L*delta :=
    (div_le_self hd.le hL).trans (le_mul_of_one_le_left hd.le hL)
  have hsx : Metric.ball x0 delta ⊆ U ∩ Sf :=
    fun x hx => hxball (lt_trans hx hdx)
  have hsz : Metric.ball y0 (L*delta) ⊆ V ∩ Sg :=
    fun z hz => hzball (lt_trans hz hdz)
  refine ⟨fun x hx => (hsx hx).1, fun z hz => (hsz hz).1, ?_, ?⟩_
  · intro z hz
    have hzS : z ∈ Sg := (hsz (lt_of_lt_of_le hz hsmall)).2
    change dist (g z) x0 < delta
    calc
      dist (g z) x0 = dist (g z) (g y0) := by rw [hg0]
      _ ≤ (Cg : ℝ) * dist z y0 := hgLip.dist_le_mul z hzS y0 (mem_of_mem_nhds hSg)
      _ ≤ L * dist z y0 := mul_le_mul_of_nonneg_right hCg dist_nonneg
      _ < L * (delta/L) := mul_lt_mul_of_pos_left hz hLp
      _ = delta := ₀mul_div_cancel delta hLp.ne'
  · intro x hx
    change dist (f x) y0 < L*delta
    calc
      dist (f x) y0 = dist (f x) (f x0) := by rw [hf0]
      _ ≤ (Cf : ℝ) * dist x x0 := hfLip.dist_le_mul x (hsx hx).2 x0 (mem_of_mem_nhds hSf)
      _ ≤ L * dist x x0 := mul_le_mul_of_nonneg_right hCf dist_nonneg
      _ < L*delta := mul_lt_mul_of_pos_left hx hLp
end CompatibleBalls
```

Seven raw blocks are exactly the original matrices. Again put $a = r + \alpha$, $b = r + \beta$, $I = a + q$, $O = b + p$, $H = a + \beta + h$ and let c_0 be the canonical coordinate tuple. The map `splitNative` cuts the first factor into the four blocks P, C, A_{12}, A_{22} and the second into the three blocks B_1, D, Y. Its inverse `assembleNative` restores the rectangular matrices by concatenation. Both inverse proofs are entrywise consequences of the core's row/column extraction and reconstruction identities. Concatenation proves equality of the native and raw sums of squared entries, and the assembled native loss equals `parameterLoss` by definition. After their separately normalized flattenings, assembly is therefore an actual Euclidean linear isometry. Its measure-preservation theorem transports back through the two exact entry layouts, showing that assembly preserves the original product measure with no extra scalar.

```
variable (r alpha beta h p q : ℕ)
```

```
local notation "C0" => canonicalCoordinates r alpha beta h p q
local notation "ER" => rawFlat r alpha beta h p q
local notation "II" => (r+alpha)+q
local notation "OO" => (r+beta)+p
local notation "HH" => (r+alpha)+(beta+h)
local notation "NW" => O77.Native.Parameters II OO HH
local notation "EN" => O77.Native.flat II OO HH

/-- Split the original matrices into the seven raw blocks. Linearity is
entrywise; the two inverse proofs reuse the inherited block identities. -/
def splitNative : NW ≃ₗ[ℝ] Raw r alpha beta h p q where
  toFun v :=
    (MatrixCore.rowsTop (MatrixCore.colsLeft v.1),
     MatrixCore.rowsBot (MatrixCore.colsLeft v.1),
     MatrixCore.rowsTop (MatrixCore.colsRight v.1),
     MatrixCore.rowsBot (MatrixCore.colsRight v.1),
     MatrixCore.colsLeft v.2,
     MatrixCore.colsLeft (MatrixCore.colsRight v.2),
     MatrixCore.colsRight (MatrixCore.colsRight v.2))
  invFun w := (parameterA w, parameterB w)
  left_inv v := by
    apply Prod.ext
    · change MatrixCore.hcat
        (MatrixCore.vcat (MatrixCore.rowsTop (MatrixCore.colsLeft v.1))
          (MatrixCore.rowsBot (MatrixCore.colsLeft v.1)))
        (MatrixCore.vcat (MatrixCore.rowsTop (MatrixCore.colsRight v.1))
          (MatrixCore.rowsBot (MatrixCore.colsRight v.1))) = v.1
      exact (congrArg₂
        (fun A : MatrixCore.Mat HH (r+alpha) ℝ =>
          fun B : MatrixCore.Mat HH q ℝ => MatrixCore.hcat A B)
        (MatrixCore.vcat_rows (m := r+alpha) (n := beta+h)
          (fun i j => MatrixCore.colsLeft v.1 i j))
        (MatrixCore.vcat_rows (m := r+alpha) (n := beta+h)
          (fun i j => MatrixCore.colsRight v.1 i j))).trans
          (MatrixCore.hcat_cols (n := r+alpha) (k := q) (fun i j => v.1 i j))
    · change MatrixCore.hcat (MatrixCore.colsLeft v.2)
        (MatrixCore.hcat (MatrixCore.colsLeft (MatrixCore.colsRight v.2))
          (MatrixCore.colsRight (MatrixCore.colsRight v.2))) = v.2
      exact (congrArg
        (fun B : MatrixCore.Mat OO (beta+h) ℝ =>
          MatrixCore.hcat (fun i j => MatrixCore.colsLeft v.2 i j) B)
        (MatrixCore.hcat_cols (n := beta) (k := h)
          (fun i j => MatrixCore.colsRight v.2 i j))).trans
          (MatrixCore.hcat_cols (n := r+alpha) (k := beta+h) (fun i j => v.2 i j))
  right_inv w := by
    rcases w with ⟨P, Q, M, N, U, D, ⟩Y
    refine Prod.ext ?_ (Prod.ext ?_ (Prod.ext ?_
      (Prod.ext ?_ (Prod.ext ?_ (Prod.ext ?_ ?_)))))
    all_goals
      funext i j
      simp only [parameterA, parameterB, MatrixCore.block,
        MatrixCore.colsLeft, MatrixCore.colsRight,
        MatrixCore.rowsTop, MatrixCore.rowsBot, MatrixCore.hcat,
        MatrixCore.vcat, Fin.addCases_left, Fin.addCases_right]
    · exact congrFun (Fin.addCases_left
        (motive := fun _ : Fin ((r+alpha)+(beta+h)) => Fin (r+alpha) → ℝ)
        (left := fun i j => P i j) (right := fun i j => Q i j) i) j
    · exact congrFun (Fin.addCases_right
        (motive := fun _ : Fin ((r+alpha)+(beta+h)) => Fin (r+alpha) → ℝ)
        (left := fun i j => P i j) (right := fun i j => Q i j) i) j
    · exact congrFun (Fin.addCases_left
        (motive := fun _ : Fin ((r+alpha)+(beta+h)) => Fin q → ℝ)
```

```
        (left := fun i j => M i j) (right := fun i j => N i j) i) j
    · exact congrFun (Fin.addCases_right
        (motive := fun _ : Fin ((r+alpha)+(beta+h)) => Fin q → ℝ)
        (left := fun i j => M i j) (right := fun i j => N i j) i) j
  map_add' _ _ := rfl
  map_smul' _ _ := rfl

def assembleNative : Raw r alpha beta h p q ≃ₗ[ℝ] NW :=
  (splitNative r alpha beta h p q).symm

lemma assembleNative_apply (w : Raw r alpha beta h p q) :
    assembleNative r alpha beta h p q w = (parameterA w, parameterB w) := rfl

lemma assembleNative_sq (w : Raw r alpha beta h p q) :
    O77.Native.ambientSq (assembleNative r alpha beta h p q w) =
      rawAmbientSq r alpha beta h p q w :=
  (rawAmbientSq_parameters r alpha beta h p q w).symm

lemma native_loss_assemble (w : Raw r alpha beta h p q) :
    O77.Native.loss (targetMatrix r alpha beta p q) (assembleNative r alpha beta h p q w) =
      parameterLoss w := rfl

/-- The source and target are Euclidean only after their separately
normalized enumerations. This is not an isometry for tuple maximum norms. -/
def flatAssembly : Flat r alpha beta h p q ≃ₗᵢ[ℝ] Flat r alpha beta h p q where
  __ := (ER).symm.toLinearEquiv.trans
    ((assembleNative r alpha beta h p q).trans (EN).toLinearEquiv)
  norm_map' x := by
    change ‖EN (assembleNative r alpha beta h p q ((ER).symm x))‖ = ‖‖x
    apply (₀sq_eq_sq (norm_nonneg _) (norm_nonneg _)).1
    rw [O77.Native.flat_norm_sq, assembleNative_sq,
      ← rawFlat_norm_sq r alpha beta h p q ((ER).symm x), (ER).apply_symm_apply]

/-- Exact native entry-product measure, obtained from the library theorem
for Euclidean linear isometries. No Haar normalization is chosen anew. -/
theorem assembleNative_preserves : MeasurePreserving (assembleNative r alpha beta h p q)
    (rawMeasure r alpha beta h p q) (O77.Native.entryMeasure II OO HH) := by
  have hn : MeasurePreserving (EN).toHomeomorph.toMeasurableEquiv
      (O77.Native.entryMeasure II OO HH) volume := O77.Native.flat_preserves II OO HH
  have hc := hn.symm.comp
    ((flatAssembly r alpha beta h p q).measurePreserving.comp
      (rawFlat_preserves r alpha beta h p q))
  convert hc using 1
  funext w
  change assembleNative r alpha beta h p q w =
    (EN).symm (EN (assembleNative r alpha beta h p q ((ER).symm (ER w))))
  simp only [ContinuousLinearEquiv.symm_apply_apply]

lemma continuous_parameterLoss : Continuous (parameterLoss : Raw r alpha beta h p q → ℝ) := by
  have hc := (O77.Native.continuous_loss II OO HH (targetMatrix r alpha beta p q)).comp
    (assembleNative r alpha beta h p q).toContinuousLinearEquiv.continuous
  change Continuous (parameterLoss : Raw r alpha beta h p q → ℝ) at hc
  exact hc
```

The precise canonical center and its bands. The canonical native center is the assembly of decode(c_0); it is not another matrix pair chosen only for its ranks. Define the raw band using the sum of the seven raw blocks' squared entries and the original centered loss. Linearity of assembly carries $w - \mathrm{decode}(c_0)$ to the native parameter difference, and its exact measure preservation gives `rawBandVolume_eq_native` for every radius and band width. This identity preserves the absolute centered band and the factor $1/2$ in the loss. The fact that the center

is an exact fit is proved next, before replacing bands with sublevels.

```
def canonicalParameter : NW := assembleNative r alpha beta h p q (decode C0)

def rawBandVolume (R eps : ℝ) : ENNReal :=
  rawMeasure r alpha beta h p q {w |
    Real.sqrt (rawAmbientSq r alpha beta h p q (w-decode C0)) < R ∧
      |parameterLoss w - parameterLoss (decode C0)| < eps}

/-- Native matrices and the seven raw blocks give exactly the same bands,
with the original loss factor 1/2 and the original Euclidean radius. -/
theorem rawBandVolume_eq_native (R eps : ℝ) :
    rawBandVolume r alpha beta h p q R eps =
      O77.Native.bandVolume (targetMatrix r alpha beta p q)
        (canonicalParameter r alpha beta h p q) R eps := by
  have hm := O77.Native.measurable_bandSet II OO HH (targetMatrix r alpha beta p q)
    (canonicalParameter r alpha beta h p q) R eps
  have he := (assembleNative_preserves r alpha beta h p q).measure_preimage hm.nullMeasurableSet
  have hpre : (assembleNative r alpha beta h p q) ⁻¹'
      O77.Native.bandSet (targetMatrix r alpha beta p q)
        (canonicalParameter r alpha beta h p q) R eps =
      {w | Real.sqrt (rawAmbientSq r alpha beta h p q (w-decode C0)) < R ∧
        |parameterLoss w - parameterLoss (decode C0)| < eps} := by
    ext w
    change (Real.sqrt (O77.Native.ambientSq
        (assembleNative r alpha beta h p q w - assembleNative r alpha beta h p q (decode C0))) < R ∧
      |O77.Native.loss (targetMatrix r alpha beta p q) (assembleNative r alpha beta h p q w) -
        O77.Native.loss (targetMatrix r alpha beta p q)
          (assembleNative r alpha beta h p q (decode C0))| < eps) ↔ _
    rw ←[ map_sub, assembleNative_sq, native_loss_assemble, native_loss_assemble]
    rfl
  rw [hpre] at he
  exact he
end O77.Canonical0906
end
noncomputable section
open Set Filter MeasureTheory
open scoped BigOperators Topology ENNReal NNReal Matrix.Norms.Elementwise
namespace O77.Canonical0906
attribute [local instance] O77.Native.parametersBorelSpace
  O77.Coordinates0906.rawBorelSpace O77.Coordinates0906.coordinateBorelSpace
open O77.Residual
open O77.Reconstruction0905
open O77.Coordinates0906
open O77.Transport0906
open O77.NormalForm0906
variable (r alpha beta h p q : ℕ)
local notation "C0" => canonicalCoordinates r alpha beta h p q
local notation "ER" => rawFlat r alpha beta h p q
local notation "EC" => coordinateFlat r alpha beta h p q
local notation "E0" => Flat r alpha beta h p q
local notation "Fchart" => flatEncode r alpha beta h p q
local notation "Gchart" => flatDecode r alpha beta h p q
local notation "x0" => flatBase r alpha beta h p q
```

Proving that the canonical base is an exact fit. Flatten the original loss by E_R and the model energy by E_C, obtaining L on the raw Euclidean carrier and N on the coordinate carrier. The previously proved chart comparison becomes $N(z)/(2K) \le L(g(z)) \le KN(z)/2$ on a fixed open neighborhood. At $z = 0$, $N(0) = 0$, $g(0) = x_0$, and $L \ge 0$, so $L(x_0) = 0$. A zero sum of squared real entries implies that each entry of $BA - \Phi_0$ is zero; formally the proof

uses the explicit Frobenius norm identity and `norm_eq_zero`. Thus `canonical_exact_fit` proves $B_0A_0 = \Phi_0$ without Wishart or an assumed local pair.

```
def flatLoss (x : E0) : ℝ := parameterLoss ((ER).symm x)
def flatNormal (z : E0) : ℝ := coordinateEnergy ((EC).symm z)

def flatNormalVolume (R eps : ℝ) : ENNReal :=
  volume {z : E0 | z ∈ Metric.ball 0 R ∧ flatNormal r alpha beta h p q z < eps}
def flatSublevelVolume (R eps : ℝ) : ENNReal :=
  volume {x : E0 | x ∈ Metric.ball x0 R ∧ flatLoss r alpha beta h p q x < eps}

lemma continuous_flatLoss : Continuous (flatLoss r alpha beta h p q) :=
  (continuous_parameterLoss r alpha beta h p q).comp (ER).symm.continuous

lemma measurable_flatNormal : Measurable (flatNormal r alpha beta h p q) :=
  (measurable_coordinateEnergy r alpha beta h p q).comp (EC).symm.continuous.measurable

lemma flatLoss_nonneg (x : E0) : 0 ≤ flatLoss r alpha beta h p q x :=
  div_nonneg (O77.Reconstruction0905.frobeniusSq_nonneg _) (by norm_num)

lemma flatNormal_zero : flatNormal r alpha beta h p q (0 : E0) = 0 := by
  simp [flatNormal, coordinateEnergy, frobeniusSq]

/-- The checked Frobenius comparison is pulled back along the very same
coordinate enumeration used for the inverse derivative. -/
theorem flat_loss_comparison :
    ∃ (V : Set E0) (K : ℝ), IsOpen V ∧ (0 : E0) ∈ V ∧
      V ⊆ flatTarget r alpha beta h p q ∧ 0 < K ∧
      ∀ z ∈ V, flatNormal r alpha beta h p q z / (2*K) ≤ flatLoss r alpha beta h p q (Gchart z) ∧
        flatLoss r alpha beta h p q (Gchart z) ≤ (K/2)*flatNormal r alpha beta h p q z := by
  obtain ⟨N, K, hN, h0, hNT, hK, ⟩hb :=
    canonical_centered_chart_with_uniform_bound (r := r) (alpha := alpha)
      (beta := beta) (h := h) (p := p) (q := q)
  refine ⟨(EC).symm ⁻¹' N, K, hN.preimage (EC).symm.continuous, ?_, ?_, hK, ?⟩_
  · simpa only [mem_preimage, map_zero] using h0
  · exact fun z hz => hNT hz
  · intro z hz
    simpa only [flatLoss, flatNormal, flatDecode, ContinuousLinearEquiv.symm_apply_apply]
      using (hb ((EC).symm z) hz).2.2.2

lemma flatLoss_at_base : flatLoss r alpha beta h p q x0 = 0 := by
  obtain ⟨V, K, hV, h0, hVT, hK, ⟩hb := flat_loss_comparison r alpha beta h p q
  have hh := (hb 0 h0).2
  rw [(flat_base_points r alpha beta h p q).2.2.2, flatNormal_zero, mul_zero] at hh
  exact le_antisymm hh (flatLoss_nonneg r alpha beta h p q x0)

lemma parameterLoss_at_base : parameterLoss (decode C0) = 0 := by
  simpa only [flatLoss, flatBase, ContinuousLinearEquiv.symm_apply_apply] using
    flatLoss_at_base r alpha beta h p q

open scoped Matrix.Norms.Frobenius in
/-- The named native base really lies in the exact-fit fiber. This conclusion
has no Wishart or local-pair premise. -/
theorem canonical_exact_fit :
    (canonicalParameter r alpha beta h p q).2 * (canonicalParameter r alpha beta h p q).1 =
      targetMatrix r alpha beta p q := by
  have hl := parameterLoss_at_base r alpha beta h p q
  have hs : frobeniusSq (parameterError (decode C0)) = 0 := by
    unfold parameterLoss at hl
    exact (div_eq_zero_iff.1 hl).resolve_right two_ne_zero
  rw [O77.Reconstruction0905.frobeniusSq_eq_norm_sq] at hs
  have hz : parameterError (decode C0) = 0 := norm_eq_zero.1 (sq_eq_zero_iff.1 hs)
  exact sub_eq_zero.1 hz
```

Two exact volume identifications. For $R > 0$, $\|E_C c\| < R$ is equivalent to the coordinate squared-entry condition $Q(c) < R^2$. This and exact measure preservation identify the flat normal sublevel mass with `coordinateVolume`. For the original loss, the just-proved base value $L(x_0) = 0$ and global nonnegativity give $|L(x) - L(x_0)| = L(x)$. The analogous pullback under E_R identifies the flat original sublevel mass with `rawBandVolume`. The use of $R > 0$ for squaring the radius and the use of the zero base value are explicit and serve different roles.

```
lemma flatNormalVolume_eq_coordinate {R : ℝ} (hR : 0 < R) (eps : ℝ) :
    flatNormalVolume r alpha beta h p q R eps = coordinateVolume r alpha beta h p q R eps := by
  have hm : MeasurableSet {z : E0 | z ∈ Metric.ball 0 R ∧
      flatNormal r alpha beta h p q z < eps} :=
    measurableSet_ball.inter (measurable_flatNormal r alpha beta h p q measurableSet_Iio)
  have he := (coordinateFlat_preserves r alpha beta h p q).measure_preimage hm.nullMeasurableSet
  have hball (c : Coordinates r alpha beta h p q) :
      EC c ∈ Metric.ball 0 R ↔ coordinateAmbientSq r alpha beta h p q c < R^2 := by
    rw [Metric.mem_ball, dist_zero_right, ← coordinateFlat_norm_sq r alpha beta h p q c]
    exact (₀sq_lt_sq (norm_nonneg _) hR.le).symm
  have hpre : EC ⁻¹' {z : E0 | z ∈ Metric.ball 0 R ∧ flatNormal r alpha beta h p q z < eps} =
      {c | coordinateEnergy c < eps} ∩ coordinateBall r alpha beta h p q R := by
    ext c
    simp only [mem_preimage, mem_ofPred_eq, mem_inter_iff, hball, coordinateBall,
      flatNormal, ContinuousLinearEquiv.symm_apply_apply, and_comm]
  rw [hpre] at he
  unfold flatNormalVolume coordinateVolume sublevelMass
  have hsub : MeasurableSet
      {c : Coordinates r alpha beta h p q | coordinateEnergy c < eps} :=
    measurable_coordinateEnergy r alpha beta h p q measurableSet_Iio
  rw [Measure.restrict_apply hsub]
  exact he.symm

/-- Absolute bands become sublevels only after the base value and global
nonnegativity have been proved. Both measures are kept in ENNReal. -/
lemma flatSublevelVolume_eq_raw (R eps : ℝ) :
    flatSublevelVolume r alpha beta h p q R eps = rawBandVolume r alpha beta h p q R eps := by
  have hm : MeasurableSet {x : E0 | x ∈ Metric.ball x0 R ∧ flatLoss r alpha beta h p q x < eps} :=
    measurableSet_ball.inter ((continuous_flatLoss r alpha beta h p q).measurable measurableSet_Iio)
  have he := (rawFlat_preserves r alpha beta h p q).measure_preimage hm.nullMeasurableSet
  have hpre : ER ⁻¹' {x : E0 | x ∈ Metric.ball x0 R ∧ flatLoss r alpha beta h p q x < eps} =
      {w | Real.sqrt (rawAmbientSq r alpha beta h p q (w-decode C0)) < R ∧
        |parameterLoss w - parameterLoss (decode C0)| < eps} := by
    ext w
    have hnonneg : 0 ≤ parameterLoss w :=
      div_nonneg (O77.Reconstruction0905.frobeniusSq_nonneg _) (by norm_num)
    have hsqrt : Real.sqrt (rawAmbientSq r alpha beta h p q (w-decode C0)) =
        ‖ER w - ‖x0 := by
      rw ←[ rawFlat_norm_sq r alpha beta h p q (w-decode C0),
        Real.sqrt_sq (norm_nonneg _), map_sub]
      rfl
    simp only [mem_preimage, mem_ofPred_eq, Metric.mem_ball, dist_eq_norm,
      flatLoss, ContinuousLinearEquiv.symm_apply_apply, hsqrt,
      parameterLoss_at_base, sub_zero, abs_of_nonneg hnonneg]
  rw [hpre] at he
  exact he.symm
```

The radius-first canonical volume sandwich. Intersect the inverse-Jacobian neighborhood and the loss-comparison neighborhood at zero. Apply `compatible_balls` to this intersection and the source chart, using the proved local Lipschitz bounds of both maps. Only then fix $0 < \delta < \rho$. The inner and outer coordinate balls have radii $\delta/L, L\delta$, lie in the common neighborhood, and satisfy the required mapping properties. The general sublevel transport

theorem with $a = 1/(2K)$ and $b = K/2$ gives

$$\ell V_{\mathrm{coord}}(\delta/L, (2/K)\varepsilon) \le V_{\mathrm{raw}}(\delta, \varepsilon) \le u V_{\mathrm{coord}}(L\delta, (2K)\varepsilon).$$

The constants L, ρ, K, ℓ, u were all chosen before δ, ε. The two preceding exact volume identities identify these as the required original-entry volumes. Parameter and radius rescaling now preserve the pair by `radiusOrder_of_scaled_distortion`.

```
/-- All neighborhoods are intersected before delta or epsilon is chosen.
This is the actual original-measure estimate, not a field in a proof package. -/
theorem canonical_volume_sandwich :
    ∃ L rho K lo hi : ℝ, 1 ≤ L ∧ 0 < rho ∧ 0 < K ∧ 0 < lo ∧ 0 < hi ∧
      ∀ delta, 0 < delta → delta < rho → ∀ eps, 0 < eps →
        ENNReal.ofReal lo * coordinateVolume r alpha beta h p q (delta/L) ((2/K)*eps) ≤
          rawBandVolume r alpha beta h p q delta eps ∧
        rawBandVolume r alpha beta h p q delta eps ≤
          ENNReal.ofReal hi * coordinateVolume r alpha beta h p q (L*delta) ((2*K)*eps) := by
  obtain ⟨VJ, lo, hi, hVJ, h0J, hJT, hlo, hhi, ⟩hJ :=
    flat_local_jacobian_bounds r alpha beta h p q
  obtain ⟨VC, K, hVC, h0C, hCT, hK, ⟩hcomp := flat_loss_comparison r alpha beta h p q
  let V := VJ ∩ VC
  have hV : V ∈ nhds (0 : E0) := (hVJ.inter hVC).mem_nhds ⟨h0J,⟩h0C
  have hU : flatSource r alpha beta h p q ∈ nhds x0 :=
    (flatSource_isOpen r alpha beta h p q).mem_nhds (flat_base_points r alpha beta h p q).2.1
  obtain ⟨hLipF,⟩hLipG := flat_local_lipschitz r alpha beta h p q
  obtain ⟨L, rho, hL, hrho, ⟩hballs := compatible_balls hU hV
    (flat_base_points r alpha beta h p q).2.2.1
    (flat_base_points r alpha beta h p q).2.2.2 hLipF hLipG
  have hLp : 0 < L := zero_lt_one.trans_le hL
  refine ⟨L, rho, K, lo, hi, hL, hrho, hK, hlo, hhi, ?⟩_
  intro delta hd hdrho eps heps
  obtain ⟨hsource, houter, hgi, ⟩hfo := hballs delta hd hdrho
  have hsmall : delta/L ≤ L*delta :=
    (div_le_self hd.le hL).trans (le_mul_of_one_le_left hd.le hL)
  have hinner : Metric.ball (0 : E0) (delta/L) ⊆ V :=
    fun z hz => houter (lt_of_lt_of_le hz hsmall)
  have hb := sublevel_window_sandwich (volume : Measure E0)
    (f := Fchart) (g := Gchart) (Dg := fderiv ℝ Gchart)
    (U := Metric.ball x0 delta) (V := V)
    (inner := Metric.ball 0 (delta/L)) (outer := Metric.ball 0 (L*delta))
    (loss := flatLoss r alpha beta h p q) (model := flatNormal r alpha beta h p q)
    measurableSet_ball measurableSet_ball (measurable_flatNormal r alpha beta h p q)
    hinner houter
    (fun z hz => ((flatDecode_C1 r alpha beta h p q z (hJT hz.1)).differentiableAt one_ne_zero).
     hasFDerivAt)
    (fun z hz => flat_encode_decode r alpha beta h p q z (hJT hz.1))
    (fun x hx => flat_decode_encode r alpha beta h p q x (hsource hx))
    hgi hfo (a := 1/(2*K)) (b := K/2)
    (one_div_pos.mpr (mul_pos (by norm_num) hK)) (half_pos hK)
    (fun z hz => hJ z hz.1)
    (fun z hz => (hcomp z (hinner hz).2).2)
    (fun z hz => by
      simpa only [div_eq_mul_inv, one_mul, mul_comm] using (hcomp z (houter hz).2).1)
    eps
  change ENNReal.ofReal lo * flatNormalVolume r alpha beta h p q (delta/L) (eps/(K/2)) ≤
      flatSublevelVolume r alpha beta h p q delta eps ∧
    flatSublevelVolume r alpha beta h p q delta eps ≤ ENNReal.ofReal hi *
      flatNormalVolume r alpha beta h p q (L*delta) (eps/(1/(2*K))) at hb
  rw [flatNormalVolume_eq_coordinate r alpha beta h p q (div_pos hd hLp),
    flatNormalVolume_eq_coordinate r alpha beta h p q (mul_pos hLp hd),
    flatSublevelVolume_eq_raw] at hb
  have hscale1 : eps/(K/2) = (2/K)*eps := by
    field_simp [ne_of_gt hK]
```

```
  have hscale2 : eps/(1/(2*K)) = (2*K)*eps := by
    rw [div_div_eq_mul_div, div_one, mul_comm]
  simpa only [hscale1,hscale2] using hb
```

The canonical measured pair. The reusable theorem `raw_pair_of_coordinate` transports any proved coordinate order through the concrete sandwich. Applying it to the normal-form order and then using the exact raw/native volume identity proves the native canonical pair $(s/2+\lambda, m)$ from a residual pair (λ, m) in the specific (p, q, h) dimensions. The Wishart wrapper supplies $(\lambda, m) = (d_0/2, \mu_0)$; the neutral wrapper uses $(0, 1)$. Thus all chart domains, loss comparisons, measures, and windows have been constructed by the proof. The retained conditional wrappers are reusable intermediate theorems; their Wishart input is discharged in the final unconditional assembly elsewhere in the development. The concluding examples retain the scalar logarithmic resonance, a nonzero regular contribution with absent residual, and the empty model.

```
/-- Reusable conclusion of the concrete chart, with only the already measured
coordinate order as input. The public corollaries below discharge that input. -/
theorem raw_pair_of_coordinate {lam : ℝ} {m : ℕ}
    (hN : RadiusOrder (coordinateVolume r alpha beta h p q) lam m) :
    RadiusOrder (rawBandVolume r alpha beta h p q) lam m := by
  obtain ⟨L, rho, K, lo, hi, hL, hrho, hK, hlo, hhi, ⟩hb :=
    canonical_volume_sandwich r alpha beta h p q
  exact radiusOrder_of_scaled_distortion hN hlo hhi
    (div_pos (by norm_num) hK) (mul_pos (by norm_num) hK) hL hrho hb

/-- The concrete canonical chart transports a pair proved for its actual
residual dimensions. No Wishart premise or unrelated residual instance is
required by this interface. -/
theorem canonical_pair_of_residual {lam : ℝ} {m : ℕ}
    (hres : HasResidualBandPairAtOrigin p q h lam m) :
    O77.Native.HasLocalBandPair (targetMatrix r alpha beta p q)
      (canonicalParameter r alpha beta h p q)
      ((O77.Coordinates0906.regularDim r alpha beta p q : ℝ)/2 + lam) m := by
  have hv : rawBandVolume r alpha beta h p q =
      O77.Native.bandVolume (targetMatrix r alpha beta p q) (canonicalParameter r alpha beta h p q) :=
      by
    funext R eps
    exact rawBandVolume_eq_native r alpha beta h p q R eps
  unfold O77.Native.HasLocalBandPair
  rw ←[ hv]
  exact raw_pair_of_coordinate r alpha beta h p q
    (coordinate_pair_of_residual r alpha beta h p q hres)

/-- Native canonical endpoint. Its only external mathematical premise is the
inherited Wishart proposition; the loss, measures, inverse maps and windows
have been supplied above, rather than assumed as local-pair fields. -/
theorem canonical_pair_of_wishart (hW : O77.External.RealWishartEigenvalueFormula) :
    O77.Native.HasLocalBandPair (targetMatrix r alpha beta p q)
      (canonicalParameter r alpha beta h p q)
      (((O77.Coordinates0906.regularDim r alpha beta p q : ℝ) + (O77.residualMin p q h : ℝ))/2)
      (O77.residualMultiplicity p q h) := by
  simpa only [add_div] using canonical_pair_of_residual r alpha beta h p q
    (residualBandPair_all_dimensions_of_wishart_instantiated hW p q h)

theorem canonical_pair_neutral (hz : p = 0 ∨ q = 0 ∨ h = 0) :
    O77.Native.HasLocalBandPair (targetMatrix r alpha beta p q)
      (canonicalParameter r alpha beta h p q)
      ((O77.Coordinates0906.regularDim r alpha beta p q : ℝ)/2) 1 := by
  simpa only [add_zero] using canonical_pair_of_residual r alpha beta h p q
```

```
    (residualBandPair_neutral_of_zero_dimension hz)

-- Coordinate regression declarations.
example (hW : O77.External.RealWishartEigenvalueFormula) :
    O77.Native.HasLocalBandPair (targetMatrix 0 0 0 1 1)
      (canonicalParameter 0 0 0 1 1 1) (1/2 : ℝ) 2 := by
  have hd : O77.residualMin 1 1 1 = 1 := by decide
  have hm : O77.residualMultiplicity 1 1 1 = 2 := by decide
  simpa [O77.Coordinates0906.regularDim, hd, hm] using
    canonical_pair_of_wishart 0 0 0 1 1 1 hW

example : O77.Native.HasLocalBandPair (targetMatrix 1 0 0 1 1)
    (canonicalParameter 1 0 0 0 1 1) (3/2 : ℝ) 1 := by
  simpa [O77.Coordinates0906.regularDim] using
    canonical_pair_neutral 1 0 0 0 1 1 (Or.inr (Or.inr rfl))

example : O77.Native.HasLocalBandPair (targetMatrix 0 0 0 0 0)
    (canonicalParameter 0 0 0 0 0 0) 0 1 := by
  simpa [O77.Coordinates0906.regularDim] using
    canonical_pair_neutral 0 0 0 0 0 0 (Or.inl rfl)

end O77.Canonical0906
end
```

37 Module `O77BasisChange`

37.1 Nonorthogonal changes of basis at exact fits

For invertible matrices G_I, G_H, G_O, define

$$A' = G_H A G_I^{-1}, \qquad B' = G_O B G_H^{-1}, \qquad \Phi' = G_O \Phi G_I^{-1}.$$

These are actual linear equivalences of rectangular matrix spaces, with inverse obtained from the unit inverses. Multiplying out cancels $G_H^{-1} G_H$ and gives

$$B'A' - \Phi' = G_O (BA - \Phi) G_I^{-1}.$$

It follows in particular that exact fits map to exact fits. `Matrix.rank_mul_eq_left_of_isUnit_det` and `Matrix.rank_mul_eq_right_of_isUnit_det` prove preservation of the three actual ranks. No orthogonality is assumed, and preservation of rank alone is not yet preservation of the measured invariant.

```
import Mathlib
import O77Canonical
import O77AdaptedSections

set_option autoImplicit false
noncomputable section
open Set MeasureTheory
open scoped BigOperators Topology ENNReal Matrix.Norms.Elementwise
namespace O77.BasisChange0906
open O77.Residual (frobeniusSq)
open O77.Reconstruction0905
open O77.Native

/-- An actual rectangular change of bases. The inverse uses unit inverses,
not the totalized matrix inverse on an unguarded matrix. -/
def matrixChange {m n : ℕ} (L : (RM m m)ˣ) (R : (RM n n)ˣ) :
    RM m n ≃ₗ[ℝ] RM m n where
```

```
  toFun M := (L : RM m m) * M * ↑((-R¹) : RM n n)
  invFun M := ↑((-L¹) : RM m m) * M * (R : RM n n)
  left_inv M := by
    change ↑((-L¹) : RM m m) * ((L : RM m m) * M * ↑((-R¹) : RM n n)) *
      (R : RM n n) = M
    calc
      _ = ↑(((-L¹) : RM m m) * (L : RM m m)) * M *
          ↑(((-R¹) : RM n n) * (R : RM n n)) := by
        simp only [Matrix.mul_assoc]
      _ = M := by simp only ←[ Units.val_mul, inv_mul_cancel, Units.val_one,
        Matrix.one_mul, Matrix.mul_one]
  right_inv M := by
    change (L : RM m m) * ↑(((-L¹) : RM m m) * M * (R : RM n n)) *
      ↑((-R¹) : RM n n) = M
    calc
      _ = ((L : RM m m) * ↑((-L¹) : RM m m)) * M *
          ((R : RM n n) * ↑((-R¹) : RM n n)) := by
        simp only [Matrix.mul_assoc]
      _ = M := by simp only ←[ Units.val_mul, mul_inv_cancel, Units.val_one,
        Matrix.one_mul, Matrix.mul_one]
  map_add' M N := by simp only [Matrix.mul_add, Matrix.add_mul]
  map_smul' c M := by
    simp only [Matrix.mul_smul, Matrix.smul_mul, RingHom.id_apply]

@[simp] lemma matrixChange_inverse {m n : ℕ} (L : (RM m m)ˣ) (R : (RM n n)ˣ) :
    matrixChange (-L¹) (-R¹) = (matrixChange L R).symm := by
  apply LinearEquiv.ext
  intro M
  change ↑((-L¹) : RM m m) * M * ↑(((-R¹)⁻¹) : RM n n) =
    ↑((-L¹) : RM m m) * M * (R : RM n n)
  simp only [inv_inv]

/-- Input, output, hidden order, as in the native parameter definition. -/
def change {I O H : ℕ} (GI : (RM I I)ˣ) (GO : (RM O O)ˣ) (GH : (RM H H)ˣ) :
    Parameters I O H ≃ₗ[ℝ] Parameters I O H :=
  (matrixChange GH GI).prodCongr (matrixChange GO GH)

def changeTarget {I O : ℕ} (GI : (RM I I)ˣ) (GO : (RM O O)ˣ) (Phi : RM O I) : RM O I :=
  matrixChange GO GI Phi

@[simp] lemma change_inverse {I O H : ℕ}
    (GI : (RM I I)ˣ) (GO : (RM O O)ˣ) (GH : (RM H H)ˣ) :
    change (-GI¹) (-GO¹) (-GH¹) = (change GI GO GH).symm := by
  apply LinearEquiv.ext
  intro w
  change (matrixChange (-GH¹) (-GI¹) w.1, matrixChange (-GO¹) (-GH¹) w.2) =
    ((matrixChange GH GI).symm w.1, (matrixChange GO GH).symm w.2)
  rw [matrixChange_inverse, matrixChange_inverse]

lemma error_change {I O H : ℕ} (GI : (RM I I)ˣ) (GO : (RM O O)ˣ)
    (GH : (RM H H)ˣ) (Phi : RM O I) (w : Parameters I O H) :
    (change GI GO GH w).2 * (change GI GO GH w).1 - changeTarget GI GO Phi =
      matrixChange GO GI (w.2*w.1-Phi) := by
  change ((GO : RM O O) * w.2 * ↑((-GH¹) : RM H H)) *
      ((GH : RM H H) * w.1 * ↑((-GI¹) : RM I I)) -
    (GO : RM O O) * Phi * ↑((-GI¹) : RM I I) =
      (GO : RM O O) * (w.2*w.1-Phi) * ↑((-GI¹) : RM I I)
  have hc : ↑((-GH¹) : RM H H) * (GH : RM H H) = 1 := by
    rw ←[ Units.val_mul, inv_mul_cancel, Units.val_one]
  calc
    _ = (GO : RM O O) * w.2 * ↑(((-GH¹) : RM H H) * (GH : RM H H)) *
        w.1 * ↑((-GI¹) : RM I I) -
```

```
        (GO : RM O O) * Phi * ↑((-GI¹) : RM I I) := by
      simp only [Matrix.mul_assoc]
    _ = _ := by rw [hc]; simp only [Matrix.mul_one, Matrix.mul_sub,
      Matrix.sub_mul, Matrix.mul_assoc]

lemma exact_fit_change {I O H : ℕ} (GI : (RM I I)ˣ) (GO : (RM O O)ˣ)
    (GH : (RM H H)ˣ) {Phi : RM O I} {w : Parameters I O H}
    (hw : w.2*w.1 = Phi) :
    (change GI GO GH w).2 * (change GI GO GH w).1 = changeTarget GI GO Phi := by
  apply sub_eq_zero.1
  rw [error_change, hw, sub_self, map_zero]

/-- Rank invariance is imported from Mathlib, not proved by new elimination. -/
lemma matrixChange_rank {m n : ℕ} (L : (RM m m)ˣ) (R : (RM n n)ˣ) (M : RM m n) :
    (matrixChange L R M).rank = M.rank := by
  change ((L : RM m m) * M * ↑((-R¹) : RM n n)).rank = M.rank
  rw [Matrix.rank_mul_eq_left_of_isUnit_det _ _
      ((-R¹).isUnit.map Matrix.detMonoidHom),
    Matrix.rank_mul_eq_right_of_isUnit_det _ _ (L.isUnit.map Matrix.detMonoidHom)]

lemma change_ranks {I O H : ℕ} (GI : (RM I I)ˣ) (GO : (RM O O)ˣ)
    (GH : (RM H H)ˣ) (w : Parameters I O H) (Phi : RM O I) :
    (change GI GO GH w).1.rank = w.1.rank ∧
    (change GI GO GH w).2.rank = w.2.rank ∧
    (changeTarget GI GO Phi).rank = Phi.rank :=
  ⟨matrixChange_rank GH GI w.1, matrixChange_rank GO GH w.2,
    matrixChange_rank GO GI ⟩Phi
```

Two-sided comparison of the uncentered losses. Applying Frobenius submultiplicativity twice gives

$$|G_O M G_I^{-1}|_F^2 \le |G_O|_F^2 |G_I^{-1}|_F^2 |M|_F^2.$$

Apply the same bound to the inverse transformation. A common constant is

$$C = 1 + |G_O|_F^2 |G_I^{-1}|_F^2 + |G_O^{-1}|_F^2 |G_I|_F^2 \ge 1.$$

It is positive even when a matrix space is empty. Substitution of the product-error identity yields $L_{\Phi'}(w') \le C L_\Phi(w)$ and $L_\Phi(w) \le C L_{\Phi'}(w')$ for every parameter. These are inequalities for nonnegative uncentered losses. They do not by themselves compare absolute bands around positive-loss centers.

```
/-- Reuse the already checked Frobenius multiplication inequality twice. -/
lemma matrixChange_frobenius_le {m n : ℕ}
    (L : (RM m m)ˣ) (R : (RM n n)ˣ) (M : RM m n) :
    frobeniusSq (matrixChange L R M) ≤
      (frobeniusSq (L : RM m m) * frobeniusSq ↑((-R¹) : RM n n)) * frobeniusSq M := by
  calc
    _ ≤ frobeniusSq ((L : RM m m)*M) * frobeniusSq ↑((-R¹) : RM n n) :=
      frobeniusSq_mul_le _ _
    _ ≤ (frobeniusSq (L : RM m m) * frobeniusSq M) *
        frobeniusSq ↑((-R¹) : RM n n) :=
      mul_le_mul_of_nonneg_right (frobeniusSq_mul_le _ _) (frobeniusSq_nonneg _)
    _ = _ := by ring

/-- A positive symmetric comparison constant, also valid for empty matrices. -/
def lossConstant {I O : ℕ} (GI : (RM I I)ˣ) (GO : (RM O O)ˣ) : ℝ :=
  1 + frobeniusSq (GO : RM O O) * frobeniusSq ↑((-GI¹) : RM I I) +
    frobeniusSq ↑((-GO¹) : RM O O) * frobeniusSq (GI : RM I I)

lemma lossConstant_ge_one {I O : ℕ} (GI : (RM I I)ˣ) (GO : (RM O O)ˣ) :
```

```
    1 ≤ lossConstant GI GO := by
  have h1 := mul_nonneg (frobeniusSq_nonneg (GO : RM O O))
    (frobeniusSq_nonneg ↑((-GI⁻¹) : RM I I))
  have h2 := mul_nonneg (frobeniusSq_nonneg ↑((-GO⁻¹) : RM O O))
    (frobeniusSq_nonneg (GI : RM I I))
  exact (le_add_of_nonneg_right h1).trans (le_add_of_nonneg_right h2)

/-- The loss comparison holds globally, but comparison of centered bands
will additionally require that the center is an exact fit. -/
theorem loss_comparison {I O H : ℕ} (GI : (RM I I)ˣ) (GO : (RM O O)ˣ)
    (GH : (RM H H)ˣ) (Phi : RM O I) (w : Parameters I O H) :
    loss (changeTarget GI GO Phi) (change GI GO GH w) ≤ lossConstant GI GO * loss Phi w ∧
    loss Phi w ≤ lossConstant GI GO * loss (changeTarget GI GO Phi) (change GI GO GH w) := by
  let M := w.2*w.1-Phi
  let N := matrixChange GO GI M
  have h1 := matrixChange_frobenius_le GO GI M
  have h2 := matrixChange_frobenius_le (-GO⁻¹) (-GI⁻¹) N
  rw [matrixChange_inverse] at h2
  have hround : (matrixChange GO GI).symm N = M := (matrixChange GO GI).symm_apply_apply M
  rw [hround] at h2
  have hM := frobeniusSq_nonneg M
  have hN := frobeniusSq_nonneg N
  have hF := mul_nonneg (frobeniusSq_nonneg (GO : RM O O))
    (frobeniusSq_nonneg ↑((-GI⁻¹) : RM I I))
  have hB := mul_nonneg (frobeniusSq_nonneg ↑((-GO⁻¹) : RM O O))
    (frobeniusSq_nonneg (GI : RM I I))
  simp only [inv_inv] at h2
  have hFle : frobeniusSq (GO : RM O O) * frobeniusSq ↑((-GI⁻¹) : RM I I) ≤
      lossConstant GI GO :=
    (le_add_of_nonneg_left zero_le_one).trans (le_add_of_nonneg_right hB)
  have hBle : frobeniusSq ↑((-GO⁻¹) : RM O O) * frobeniusSq (GI : RM I I) ≤
      lossConstant GI GO := le_add_of_nonneg_left (add_nonneg zero_le_one hF)
  have hupper := h1.trans (mul_le_mul_of_nonneg_right hFle hM)
  have hlower := h2.trans (mul_le_mul_of_nonneg_right hBle hN)
  unfold loss
  rw [error_change]
  constructor
  · simpa only [mul_div_assoc] using
      div_le_div_of_nonneg_right hupper (show (0 : ℝ) ≤ 2 by norm_num)
  · simpa only [mul_div_assoc] using
      div_le_div_of_nonneg_right hlower (show (0 : ℝ) ≤ 2 by norm_num)

end O77.BasisChange0906
end
noncomputable section
open Set MeasureTheory
open scoped BigOperators Topology ENNReal Matrix.Norms.Elementwise
namespace O77.BasisChange0906
open O77.Reconstruction0905
open O77.Native
open O77.NormalForm0906
```

The measure scalar and the norm bounds. For any linear equivalence e of the native parameter space, conjugate it by the fixed entry flattening E_N to a Euclidean linear equivalence T. Define $J = |\det T| > 0$. The exact library formula `Measure.addHaar_image_continuousLinearEquiv`, transported through the entry layout, gives

$$\mu(eS) = J\mu(S)$$

for measurable S. This is the forward image Jacobian; it is generally not one. Let L be the maximum of one and the operator norms of T and T^{-1}. The library operator-norm

bound proves $\|ev\|_{\mathrm{entry}}, \|e^{-1}v\|_{\mathrm{entry}} \le L\|v\|_{\mathrm{entry}}$. Thus both scalar and metric distortions are quantified in the same original-entry geometry.

```
section NativeLinearMaps
variable {I O H : ℕ}
attribute [local instance] O77.Native.parametersBorelSpace
local notation "P" => Parameters I O H
local notation "E" => Block (H*(I+O))
local notation "EN" => O77.Native.flat I O H

def flattenChange (e : P ≃ₗ[ℝ] P) : E ≃L[ℝ] E :=
  ((EN).symm.trans e.toContinuousLinearEquiv).trans EN

def changeJacobian (e : P ≃ₗ[ℝ] P) : ℝ :=
  |LinearMap.det (flattenChange e).toLinearEquiv.toLinearMap|

lemma changeJacobian_pos (e : P ≃ₗ[ℝ] P) : 0 < changeJacobian e :=
  abs_pos.mpr (LinearEquiv.isUnit_det' (flattenChange e).toLinearEquiv).ne_zero

/-- Exact image scaling in the fixed native entry measure. The determinant
is on the common normalized Euclidean carrier, with no arbitrary Haar scalar. -/
theorem native_linear_image_measure (e : P ≃ₗ[ℝ] P) {S : Set P} (hS : MeasurableSet S) :
    entryMeasure I O H (e '' S) = ENNReal.ofReal (changeJacobian e) * entryMeasure I O H S := by
  have hflat {T : Set P} (hT : MeasurableSet T) :
      entryMeasure I O H T = volume (EN '' T) := by
    have him : MeasurableSet (EN '' T) := by
      change MeasurableSet ((EN).toHomeomorph.toMeasurableEquiv '' T)
      exact (EN).toHomeomorph.toMeasurableEquiv.measurableEmbedding.measurableSet_image.mpr hT
    have hm := (O77.Native.flat_preserves I O H).measure_preimage him.nullMeasurableSet
    simpa only [preimage_image_eq _ (EN).injective] using hm
  have heS : MeasurableSet (e '' S) := by
    change MeasurableSet (e.toContinuousLinearEquiv.toHomeomorph.toMeasurableEquiv '' S)
    exact e.toContinuousLinearEquiv.toHomeomorph.toMeasurableEquiv.measurableEmbedding.
     measurableSet_image.mpr hS
  have hcomp : (flattenChange e) ∘ EN = EN ∘ e := by
    funext v
    change EN (e ((EN).symm (EN v))) = EN (e v)
    rw [(EN).symm_apply_apply]
  have himage : EN '' (e '' S) = flattenChange e '' (EN '' S) := by
    simp only [image_image]
    exact congrArg (fun f : P → E => f '' S) hcomp.symm
  calc
    entryMeasure I O H (e '' S) = volume (EN '' (e '' S)) := hflat heS
    _ = volume (flattenChange e '' (EN '' S)) := congrArg volume himage
    _ = ENNReal.ofReal (changeJacobian e) * volume (EN '' S) :=
      Measure.addHaar_image_continuousLinearEquiv volume (flattenChange e) (EN '' S)
    _ = ENNReal.ofReal (changeJacobian e) * entryMeasure I O H S := by rw ←[ hflat hS]

/-- A global pair of Euclidean bounds. No default product or matrix norm is
identified with the entry norm: all operator norms are taken after flattening. -/
theorem native_linear_norm_bounds (e : P ≃ₗ[ℝ] P) :
    ∃ L : ℝ, 1 ≤ L ∧ ∀ v : P,
      entryNorm (e v) ≤ L * entryNorm v ∧ entryNorm (e.symm v) ≤ L * entryNorm v := by
  let T := flattenChange e
  let L : ℝ := max 1 (max ‖T.‖toContinuousLinearMap ‖T.symm.‖toContinuousLinearMap)
  have hL : 1 ≤ L := le_max_left _ _
  have hf : ‖T.‖toContinuousLinearMap ≤ L := (le_max_left _ _).trans (le_max_right _ _)
  have hg : ‖T.symm.‖toContinuousLinearMap ≤ L := (le_max_right _ _).trans (le_max_right _ _)
  refine ⟨L, hL, ?⟩
  intro v
  have hforward := (T.toContinuousLinearMap.le_opNorm (EN v)).trans
    (mul_le_mul_of_nonneg_right hf (norm_nonneg _))
  have hbackward := (T.symm.toContinuousLinearMap.le_opNorm (EN v)).trans
```

```
    (mul_le_mul_of_nonneg_right hg (norm_nonneg _))
  change ‖T (EN v)‖ ≤ L * ‖EN ‖v at hforward
  change ‖T.symm (EN v)‖ ≤ L * ‖EN ‖v at hbackward
  have hv : T (EN v) = EN (e v) := by
    change EN (e ((EN).symm (EN v))) = EN (e v)
    rw [(EN).symm_apply_apply]
  have hv' : T.symm (EN v) = EN (e.symm v) := by
    change EN (e.symm ((EN).symm (EN v))) = EN (e.symm v)
    rw [(EN).symm_apply_apply]
  rw [hv] at hforward
  rw [hv'] at hbackward
  constructor
  · simpa only [entryNorm_eq] using hforward
  · simpa only [entryNorm_eq] using hbackward
end NativeLinearMaps
```

Why the exact-fit hypothesis is essential. If $BA = \Phi$, then $L_\Phi(w) = 0$, and since $L_\Phi \geq 0$ the centered band is precisely $L_\Phi(v) < \varepsilon$ inside the entry ball. The transformed center is also an exact fit. Therefore the loss and norm bounds give the set inclusions

$$e\,\mathcal{B}_\Phi(w;\delta/L,\varepsilon/C) \subseteq \mathcal{B}_{\Phi'}(ew;\delta,\varepsilon) \subseteq e\,\mathcal{B}_\Phi(w;L\delta,C\varepsilon),$$

where $\mathcal{B}$ denotes the just-defined native band set. For the upper inclusion, take the concrete preimage $e^{-1}z$. Apply the exact image formula to both outer sets. The resulting volume sandwich has the same positive factor J on both sides, and the stated radius and bandwidth distortions. All constants are independent of δ, ε. This argument does not extend to positive-loss centers, where comparison of L, L' does not compare their centered differences.

```
lemma mem_bandSet_of_exact_fit {I O H : ℕ} {Phi : RM O I} {w : Parameters I O H}
    (hw : w.2*w.1 = Phi) (v : Parameters I O H) (R eps : ℝ) :
    v ∈ bandSet Phi w R eps ↔ entryNorm (v-w) < R ∧ loss Phi v < eps := by
  have hw0 : loss Phi w = 0 := by simp [loss, hw, O77.Residual.frobeniusSq]
  have hv0 : 0 ≤ loss Phi v := div_nonneg (frobeniusSq_nonneg _) (by norm_num)
  simp only [bandSet, entryBall, mem_ofPred_eq, hw0, sub_zero, abs_of_nonneg hv0]

/-- The lower image uses epsilon/C and the upper image uses C*epsilon.
The coefficient is the forward image Jacobian, not its reciprocal. -/
theorem basis_change_volume_sandwich {I O H : ℕ}
    (GI : (RM I I)ˣ) (GO : (RM O O)ˣ) (GH : (RM H H)ˣ)
    {Phi : RM O I} {w : Parameters I O H} (hw : w.2*w.1 = Phi) :
    ∃ L C J : ℝ, 1 ≤ L ∧ 0 < C ∧ 0 < J ∧
      ∀ delta, 0 < delta → ∀ eps, 0 < eps →
        ENNReal.ofReal J * bandVolume Phi w (delta/L) ((1/C)*eps) ≤
          bandVolume (changeTarget GI GO Phi) (change GI GO GH w) delta eps ∧
        bandVolume (changeTarget GI GO Phi) (change GI GO GH w) delta eps ≤
          ENNReal.ofReal J * bandVolume Phi w (L*delta) (C*eps) := by
  let e := change GI GO GH
  let Phi' := changeTarget GI GO Phi
  let C := lossConstant GI GO
  have hC : 0 < C := zero_lt_one.trans_le (lossConstant_ge_one GI GO)
  obtain ⟨L, hL, ⟩hnorm := native_linear_norm_bounds e
  have hLp : 0 < L := zero_lt_one.trans_le hL
  have hw' : (e w).2 * (e w).1 = Phi' := exact_fit_change GI GO GH hw
  refine ⟨L, C, changeJacobian e, hL, hC, changeJacobian_pos e, ?⟩_
  intro delta hd eps heps
  let Slo := bandSet Phi w (delta/L) ((1/C)*eps)
  let Shi := bandSet Phi w (L*delta) (C*eps)
  let Snew := bandSet Phi' (e w) delta eps
  have hlo : e '' Slo ⊆ Snew := by
    rintro _ ⟨v, hv, ⟩rfl
```

```
    obtain ⟨hvr, ⟩hve := (mem_bandSet_of_exact_fit hw v _ _).1 hv
    apply (mem_bandSet_of_exact_fit hw' (e v) delta eps).2
    constructor
    · calc
        entryNorm (e v-e w) = entryNorm (e (v-w)) := by rw [map_sub]
        _ ≤ L*entryNorm (v-w) := (hnorm (v-w)).1
        _ < L*(delta/L) := mul_lt_mul_of_pos_left hvr hLp
        _ = delta := by field_simp [ne_of_gt hLp]
    · have hcomp := (loss_comparison GI GO GH Phi v).1
      change loss Phi' (e v) ≤ C*loss Phi v at hcomp
      calc
        loss Phi' (e v) ≤ C*loss Phi v := hcomp
        _ < C*((1/C)*eps) := mul_lt_mul_of_pos_left hve hC
        _ = eps := by field_simp [ne_of_gt hC]
  have hhi : Snew ⊆ e '' Shi := by
    intro z hz
    obtain ⟨hzr, ⟩hze := (mem_bandSet_of_exact_fit hw' z delta eps).1 hz
    refine ⟨e.symm z, ?_, e.apply_symm_apply ⟩z
    apply (mem_bandSet_of_exact_fit hw (e.symm z) _ _).2
    constructor
    · calc
        entryNorm (e.symm z-w) = entryNorm (e.symm (z-e w)) := by
          rw [map_sub, e.symm_apply_apply]
        _ ≤ L*entryNorm (z-e w) := (hnorm (z-e w)).2
        _ < L*delta := mul_lt_mul_of_pos_left hzr hLp
    · have hc := (loss_comparison GI GO GH Phi (e.symm z)).2
      change loss Phi (e.symm z) ≤ C*loss Phi' (e (e.symm z)) at hc
      rw [e.apply_symm_apply] at hc
      exact hc.trans_lt (mul_lt_mul_of_pos_left hze hC)
  have hmlo := native_linear_image_measure e
    (O77.Native.measurable_bandSet I O H Phi w (delta/L) ((1/C)*eps))
  have hmhi := native_linear_image_measure e (O77.Native.measurable_bandSet I O H Phi w (L*delta) (C*
     eps))
  constructor
  · change ENNReal.ofReal (changeJacobian e) * entryMeasure I O H Slo ≤ entryMeasure I O H Snew
    rw ←[ hmlo]
    exact measure_mono hlo
  · change entryMeasure I O H Snew ≤ ENNReal.ofReal (changeJacobian e) * entryMeasure I O H Shi
    rw ←[ hmhi]
    exact measure_mono hhi
```

Transport of the complete pair. The preceding sandwich and the quantitative power–log rescaling lemma preserve both λ and m at any exact fit. Apply the same theorem to the inverse basis matrices to obtain the equivalence `hasLocalBandPair_change_iff`. The canonical-orbit corollaries simply apply this transport to the concrete canonical pair, including the neutral residual case. They give the pair for every specified orbit witness; the existence of such witnesses for all equal-rank matrix pairs was proved algebraically in the numbered coordinate module. The following fiber modules compose the two facts.

```
/-- Full multiplicity and radius-first transport for the actual native
loss, at any exact fit and for arbitrary invertible basis matrices. -/
theorem hasLocalBandPair_change {I O H : ℕ}
    (GI : (RM I I)ˣ) (GO : (RM O O)ˣ) (GH : (RM H H)ˣ)
    {Phi : RM O I} {w : Parameters I O H} (hw : w.2*w.1 = Phi)
    {lam : ℝ} {m : ℕ} (hpair : HasLocalBandPair Phi w lam m) :
    HasLocalBandPair (changeTarget GI GO Phi) (change GI GO GH w) lam m := by
  obtain ⟨L, C, J, hL, hC, hJ, ⟩hb := basis_change_volume_sandwich GI GO GH hw
  exact radiusOrder_of_scaled_distortion hpair hJ hJ (one_div_pos.mpr hC) hC hL
    (show (0 : ℝ) < 1 by norm_num) (fun delta hd _ eps heps => hb delta hd eps heps)
```

```
theorem hasLocalBandPair_change_iff {I O H : ℕ}
    (GI : (RM I I)ˣ) (GO : (RM O O)ˣ) (GH : (RM H H)ˣ)
    {Phi : RM O I} {w : Parameters I O H} (hw : w.2*w.1 = Phi)
    (lam : ℝ) (m : ℕ) :
    HasLocalBandPair (changeTarget GI GO Phi) (change GI GO GH w) lam m ↔
      HasLocalBandPair Phi w lam m := by
  constructor
  · intro hp
    have hh := hasLocalBandPair_change (⁻GI¹) (⁻GO¹) (⁻GH¹)
      (exact_fit_change GI GO GH hw) hp
    simpa only [change_inverse, changeTarget, matrixChange_inverse,
      LinearEquiv.symm_apply_apply] using hh
  · exact hasLocalBandPair_change GI GO GH hw

/-- Transport the canonical measured pair along arbitrary invertible basis
changes. The all-points attachment is `O77.FiberClassification.exact_fit_pair_of_wishart`. -/
theorem canonical_orbit_pair (hW : O77.External.RealWishartEigenvalueFormula)
    (r alpha beta h p q : ℕ)
    (GI : (RM ((r+alpha)+q) ((r+alpha)+q))ˣ)
    (GO : (RM ((r+beta)+p) ((r+beta)+p))ˣ)
    (GH : (RM ((r+alpha)+(beta+h)) ((r+alpha)+(beta+h)))ˣ) :
    HasLocalBandPair
      (changeTarget GI GO (targetMatrix r alpha beta p q))
      (change GI GO GH (O77.Canonical0906.canonicalParameter r alpha beta h p q))
      (((O77.Coordinates0906.regularDim r alpha beta p q : ℝ) +
        (O77.residualMin p q h : ℝ)) / 2) (O77.residualMultiplicity p q h) := by
  have hp := O77.Canonical0906.canonical_pair_of_wishart r alpha beta h p q hW
  have hfit := O77.Canonical0906.canonical_exact_fit r alpha beta h p q
  exact hasLocalBandPair_change GI GO GH hfit hp

/-- The neutral-residual orbit endpoint has no Wishart premise. -/
theorem canonical_orbit_pair_neutral (r alpha beta h p q : ℕ)
    (GI : (RM ((r+alpha)+q) ((r+alpha)+q))ˣ)
    (GO : (RM ((r+beta)+p) ((r+beta)+p))ˣ)
    (GH : (RM ((r+alpha)+(beta+h)) ((r+alpha)+(beta+h)))ˣ)
    (hz : p = 0 ∨ q = 0 ∨ h = 0) :
    HasLocalBandPair
      (changeTarget GI GO (targetMatrix r alpha beta p q))
      (change GI GO GH (O77.Canonical0906.canonicalParameter r alpha beta h p q))
      ((O77.Coordinates0906.regularDim r alpha beta p q : ℝ)/2) 1 :=
  hasLocalBandPair_change GI GO GH
    (O77.Canonical0906.canonical_exact_fit r alpha beta h p q)
    (O77.Canonical0906.canonical_pair_neutral r alpha beta h p q hz)

example {I O H : ℕ} (GI : (RM I I)ˣ) (GO : (RM O O)ˣ) (GH : (RM H H)ˣ)
    {Phi : RM O I} {w : O77.Native.Parameters I O H} (hw : w.2*w.1 = Phi) :
    O77.Native.HasLocalBandPair (O77.BasisChange0906.changeTarget GI GO Phi)
        (O77.BasisChange0906.change GI GO GH w) (1/2 : ℝ) 2 ↔
      O77.Native.HasLocalBandPair Phi w (1/2 : ℝ) 2 :=
  O77.BasisChange0906.hasLocalBandPair_change_iff GI GO GH hw _ _

end O77.BasisChange0906
end
```

38 Module `O77FiberRanks`

38.1 Rank invariance of the measured pair on exact fits

Take two native matrix pairs $w = (A, B)$ and $w' = (A', B')$ of the same ambient dimensions with the same three actual ranks $\operatorname{rank} A = \operatorname{rank} A'$, $\operatorname{rank} B = \operatorname{rank} B'$, and $\operatorname{rank} BA = \operatorname{rank} B'A'$.

The algebraic theorem `O77.NumberedCoordinates.basis_matrices_of_ranks` supplies invertible input, hidden, and output matrices carrying w to w'. The product identity forces the same change of bases to carry BA to $B'A'$. Treating each pair as an exact fit for its own product, the proved basis-change equivalence therefore gives, for every λ, m,

$$\mathrm{HasLocalBandPair}(BA, w, \lambda, m) \iff \mathrm{HasLocalBandPair}(B'A', w', \lambda, m).$$

This is an invariance assertion, not yet an existence assertion for a pair. On a single fixed fiber $BA = B'A' = \Phi$, equality of the two factor ranks already guarantees equality of the product ranks, giving the second theorem. The statement requires neither positive target rank nor distinct singular values; those belong only to the later saddle data.

```
import Mathlib
import O77NumberedCoordinates
import O77BasisChange

/-! The measured equivalence below is obtained from actual base matrices,
not from a record whose fields assume rank dependence of the local pair. -/
set_option autoImplicit false
noncomputable section
open O77.Native O77.BasisChange0906
open scoped Matrix.Norms.Elementwise
namespace O77.FiberRanks

/-- Rank-only invariance of the native measured predicate on exact fits,
allowing the two targets to differ. This does not assert pair existence. -/
theorem pair_iff_of_ranks {I O H : ℕ} (w w' : Parameters I O H)
    (hA : w.1.rank = w'.1.rank) (hB : w.2.rank = w'.2.rank)
    (hBA : (w.2 * w.1).rank = (w'.2 * w'.1).rank) (lam : ℝ) (m : ℕ) :
    HasLocalBandPair (w.2 * w.1) w lam m ↔
      HasLocalBandPair (w'.2 * w'.1) w' lam m := by
  obtain ⟨GI, GO, GH, heA, ⟩heB :=
    O77.NumberedCoordinates.basis_matrices_of_ranks w.1 w'.1 w.2 w'.2 hA hB hBA
  have hw : change GI GO GH w = w' := Prod.ext heA heB
  have ht : changeTarget GI GO (w.2 * w.1) = w'.2 * w'.1 := by
    have hf := exact_fit_change GI GO GH (Phi := w.2 * w.1) (w := w) rfl
    rw [hw] at hf
    exact hf.symm
  simpa only [hw, ht] using
    (hasLocalBandPair_change_iff GI GO GH
      (Phi := w.2 * w.1) (w := w) rfl lam m).symm

/-- At a fixed target only the two factor ranks need be compared.
No distinct-singular-value or positive-target-rank hypothesis occurs. -/
theorem same_fiber_same_ranks_pair {I O H : ℕ}
    {Phi : Matrix (Fin O) (Fin I) ℝ} {w w' : Parameters I O H}
    (hw : w.2 * w.1 = Phi) (hw' : w'.2 * w'.1 = Phi)
    (hA : w.1.rank = w'.1.rank) (hB : w.2.rank = w'.2.rank)
    (lam : ℝ) (m : ℕ) :
    HasLocalBandPair Phi w lam m ↔ HasLocalBandPair Phi w' lam m := by
  have hBA : (w.2 * w.1).rank = (w'.2 * w'.1).rank := by rw [hw, hw']
  simpa only [hw, hw'] using pair_iff_of_ranks w w' hA hB hBA lam m

end O77.FiberRanks
end
```

39 Module `O77FiberClassification`

39.1 All exact fits and every feasible rank stratum

The measured canonical point was defined as the actual decoder output followed by assembly. To use its algebraic rank calculation, first evaluate that decoder at $c_0 = (0, 0, 0, 0, 1, 0, 0, D_0, 0)$. Undoing the shear gives $T = X = S = 0$; the output reconstruction multiplies zero and gives $Y = 0$; hence $A_{12} = A_{22} = 0$, and $B_1 = C_I 1^{-1} = C_I$. This proves `decode_canonical` without an unnecessary evaluation of the output denormalizer. Assembly then gives exactly the core pair (A_0, B_0), not merely a point with equivalent ranks. The canonical-rank module now supplies $\operatorname{rank} A_0 = r + \alpha$, $\operatorname{rank} B_0 = r + \beta$, and $\operatorname{rank} \Phi_0 = r$.

```
import Mathlib
import O77CanonicalRanks
import O77FiberRanks

/-! Attaching the measured canonical point to every actual exact fit.
The canonical point is the decoder output already used by the volume theorem.
Dimension equalities are eliminated together, so matrices, norms and measures
are transported by equality, not by an unspecified change of basis. -/
set_option autoImplicit false
noncomputable section
open O77.Native
open O77.Reconstruction0905
open O77.Canonical0906 (canonicalParameter canonical_exact_fit
  canonical_pair_of_wishart canonical_pair_neutral assembleNative_apply)
open O77.CanonicalRanks
open scoped Matrix.Norms.Elementwise
namespace O77.FiberClassification

/-- Evaluate the actual decoder at its uncentered canonical base point.
The output normalizer need not be evaluated: it multiplies the zero Y block. -/
lemma decode_canonical (r alpha beta h p q : ℕ) :
    decode (canonicalCoordinates r alpha beta h p q) =
      ((1 : RM (r+alpha) (r+alpha)), 0, 0, 0,
        MatrixCore.canonicalCI (R := ℝ) r alpha beta p,
        MatrixCore.canonicalD (R := ℝ) r beta p, 0) := by
  -- All arithmetic in the component calculation stays in the raw core.
  -- Only the final Prod.ext transports these equalities to the Matrix tuple.
  let D : MatrixCore.Mat ((r+beta)+p) beta ℝ := MatrixCore.canonicalD r beta p
  let CI : MatrixCore.Mat ((r+beta)+p) (r+alpha) ℝ :=
    MatrixCore.canonicalCI r alpha beta p
  let P : MatrixCore.Mat (r+alpha) (r+alpha) ℝ :=
    fun i j => (1 : RM (r+alpha) (r+alpha)) i j
  let Pinv : MatrixCore.Mat (r+alpha) (r+alpha) ℝ :=
    fun i j => ((1 : RM (r+alpha) (r+alpha))⁻¹) i j
  let T := MatrixCore.unshear (MatrixCore.zero (R := ℝ) (r+beta) h)
    (MatrixCore.zero h q) (MatrixCore.zero (r+beta) q)
  let X := MatrixCore.vcat (MatrixCore.rowsTop (m := r) (n := beta) T)
    (MatrixCore.zero alpha q)
  let S := MatrixCore.vcat (MatrixCore.rowsBot (m := r) (n := beta) T)
    (MatrixCore.zero h q)
  let R0 := MatrixCore.denormalizer
    (fun i j => pivotTop (r := r) (p := p) (Matrix.of D) i j)
    (fun i j => pivotBottom (r := r) (p := p) (Matrix.of D) i j)
  let Y := MatrixCore.mul R0
    (MatrixCore.vcat (MatrixCore.zero (R := ℝ) (r+beta) h) (MatrixCore.zero p h))
  have hT : T = MatrixCore.zero (r+beta) q := by
    simp only [T, MatrixCore.unshear, MatrixCore.mul_zero, MatrixCore.sub_self]
  have hX : X = MatrixCore.zero (r+alpha) q := by
    dsimp only [X]
```

```
    rw [hT]
    exact MatrixCore.vcat_rows (m := r) (n := alpha)
      (MatrixCore.zero (R := ℝ) (r+alpha) q)
  have hS : S = MatrixCore.zero (beta+h) q := by
    dsimp only [S]
    rw [hT]
    exact MatrixCore.vcat_rows (m := beta) (n := h)
      (MatrixCore.zero (R := ℝ) (beta+h) q)
  have hY : Y = MatrixCore.zero ((r+beta)+p) h :=
    (congrArg (MatrixCore.mul R0)
      (MatrixCore.vcat_rows (m := r+beta) (n := p)
        (MatrixCore.zero (R := ℝ) ((r+beta)+p) h))).trans (MatrixCore.mul_zero R0)
  have hA12 : MatrixCore.mul P X = MatrixCore.zero (r+alpha) q :=
    (congrArg (MatrixCore.mul P) hX).trans (MatrixCore.mul_zero P)
  have hA22 : MatrixCore.add S
      (MatrixCore.mul (MatrixCore.zero (R := ℝ) (beta+h) (r+alpha)) X) =
      MatrixCore.zero (beta+h) q := by
    rw [hS, MatrixCore.zero_mul, MatrixCore.add_zero]
  have hinv : (1 : RM (r+alpha) (r+alpha))⁻¹ = 1 := by simp
  have hPinv : Pinv = MatrixCore.one (R := ℝ) (r+alpha) :=
    (congrArg (fun M : RM (r+alpha) (r+alpha) => fun i j => M i j) hinv).trans
      (core_one_eq (r+alpha)).symm
  have hB1 : MatrixCore.mul
      (MatrixCore.sub (MatrixCore.add (MatrixCore.zero ((r+beta)+p) (r+alpha)) CI)
        (MatrixCore.mul (MatrixCore.hcat D Y) (MatrixCore.zero (beta+h) (r+alpha))))
      Pinv = CI := by
    rw [MatrixCore.mul_zero, MatrixCore.zero_add, MatrixCore.sub_zero,
      hPinv, MatrixCore.mul_one]
  exact Prod.ext rfl (Prod.ext rfl (Prod.ext hA12 (Prod.ext hA22
    (Prod.ext hB1 (Prod.ext rfl hY)))))

/-- This equality identifies the measured point with the algebraic core point;
we do not replace it by another representative having the desired ranks. -/
lemma canonicalParameter_eq (r alpha beta h p q : ℕ) :
    canonicalParameter r alpha beta h p q =
      (MatrixCore.canonicalA (R := ℝ) r alpha beta h q,
       MatrixCore.canonicalB (R := ℝ) r alpha beta h p) := by
  let w0 : Raw r alpha beta h p q :=
    (1, 0, 0, 0, Matrix.of (MatrixCore.canonicalCI (R := ℝ) r alpha beta p),
      Matrix.of (MatrixCore.canonicalD (R := ℝ) r beta p), 0)
  have hd : decode (canonicalCoordinates r alpha beta h p q) = w0 :=
    decode_canonical r alpha beta h p q
  have hA : parameterA w0 = MatrixCore.canonicalA (R := ℝ) r alpha beta h q :=
    congrArg (fun Q : MatrixCore.Mat (r+alpha) (r+alpha) ℝ =>
      MatrixCore.block Q (MatrixCore.zero (r+alpha) q)
        (MatrixCore.zero (beta+h) (r+alpha)) (MatrixCore.zero (beta+h) q))
      (core_one_eq (r+alpha)).symm
  calc
    canonicalParameter r alpha beta h p q =
        O77.Canonical0906.assembleNative r alpha beta h p q w0 :=
      congrArg (O77.Canonical0906.assembleNative r alpha beta h p q) hd
    _ = (parameterA w0, parameterB w0) := assembleNative_apply r alpha beta h p q w0
    _ = _ := Prod.ext hA rfl

lemma canonicalParameter_ranks (r alpha beta h p q : ℕ) :
    (canonicalParameter r alpha beta h p q).1.rank = r + alpha ∧
    (canonicalParameter r alpha beta h p q).2.rank = r + beta ∧
    (targetMatrix r alpha beta p q).rank = r := by
  rw [canonicalParameter_eq]
  exact ⟨rank_canonicalA r alpha beta h q, rank_canonicalB r alpha beta h p,
    rank_canonicalTarget r alpha beta p ⟩q
```

Dimension transport and the general point. Suppose $(r+\alpha)+q=I$, $(r+\beta)+p=O$, and $(r+\alpha)+(\beta+h)=H$. Substitute all three equalities simultaneously in the dependent matrix types. The canonical point and a given exact fit with the corresponding three ranks now lie in the same native space, so rank-invariance transports its measured pair. This use of equality changes neither entries nor measure and requires no separate norm-equivalence assertion. For feasible (r,a,b), the rank-shape dimension equalities give these identifications with $\alpha=a-r$, $\beta=b-r$, $p=O-b$, $q=I-a$, $h=H+r-(a+b)$. The inequalities $r\le a,b$ and $b\le O$ justify the cancellations $r+(a-r)=a$, $r+(b-r)=b$, and $b+(O-b)=O$. Consequently the canonical regular count becomes exactly $Oa+b(I-a)$, as required by the public formula.

```
/-- Equality transport changes every dependent occurrence of a dimension at
once. All three equalities are necessary; none asserts norm equivalence. -/
theorem transfer_canonical_pair {I O H r alpha beta h p q : ℕ}
    (hI : (r+alpha)+q = I) (hO : (r+beta)+p = O)
    (hH : (r+alpha)+(beta+h) = H)
    {Phi : RM O I} {w : Parameters I O H}
    (hfit : w.2 * w.1 = Phi) (hr : Phi.rank = r)
    (hA : w.1.rank = r+alpha) (hB : w.2.rank = r+beta)
    {lam : ℝ} {m : ℕ}
    (hc : HasLocalBandPair (targetMatrix r alpha beta p q)
      (canonicalParameter r alpha beta h p q) lam m) :
    HasLocalBandPair Phi w lam m := by
  subst I
  subst O
  subst H
  let w0 := canonicalParameter r alpha beta h p q
  have h0 := canonicalParameter_ranks r alpha beta h p q
  have hf0 := canonical_exact_fit r alpha beta h p q
  have hprod : (w0.2 * w0.1).rank = (w.2 * w.1).rank := by
    change ((canonicalParameter r alpha beta h p q).2 *
      (canonicalParameter r alpha beta h p q).1).rank = _
    rw [hf0, hfit, h0.2.2, hr]
  have hc0 : HasLocalBandPair (w0.2 * w0.1) w0 lam m := by
    simpa only [w0, hf0] using hc
  have hp := (O77.FiberRanks.pair_iff_of_ranks w0 w
    (h0.1.trans hA.symm) (h0.2.1.trans hB.symm) hprod lam m).mp hc0
  simpa only [hfit] using hp

section RankShape
variable {I O H r a b : ℕ}

/-- Only parentheses differ from the checked rankShape_dimensions theorem. -/
private lemma shape_dimensions (D : O77.RankData I O H r a b) :
    (r+(a-r))+(I-a) = I ∧ (r+(b-r))+(O-b) = O ∧
      (r+(a-r))+((b-r)+O77.residualHidden H r a b) = H := by
  have hd := O77.rankShape_dimensions D
  refine ⟨?_, ?_, ?⟩_
  · simpa only [O77.rankShape, O77.BlockShape.input] using hd.1
  · simpa only [O77.rankShape, O77.BlockShape.output] using hd.2.1
  · simpa only [O77.rankShape, O77.BlockShape.hidden, Nat.add_assoc] using hd.2.2

/-- One arithmetic identification serves the conditional and neutral endpoints. -/
lemma canonical_regularDim (D : O77.RankData I O H r a b) :
    O77.Coordinates0906.regularDim r (a-r) (b-r) (O-b) (I-a) =
      O77.regularDim I O a b := by
  have ha : r+(a-r) = a := Nat.add_sub_of_le D.r_le_a
  have hb : r+(b-r) = b := Nat.add_sub_of_le D.r_le_b
  have ho : b+(O-b) = O := Nat.add_sub_of_le D.b_le_O
  dsimp only [O77.Coordinates0906.regularDim, O77.regularDim]
  rw [ha, hb, ho]
```

```
/-- A reusable transfer theorem, not the final existence theorem: its canonical
pair premise is discharged below by the concrete measured canonical results. -/
theorem transfer_rankData_pair (D : O77.RankData I O H r a b)
    {Phi : RM O I} {w : Parameters I O H}
    (hfit : w.2 * w.1 = Phi) (hr : Phi.rank = r)
    (hA : w.1.rank = a) (hB : w.2.rank = b) {lam : ℝ} {m : ℕ}
    (hc : HasLocalBandPair (targetMatrix r (a-r) (b-r) (O-b) (I-a))
      (canonicalParameter r (a-r) (b-r) (O77.residualHidden H r a b) (O-b) (I-a)) lam m) :
    HasLocalBandPair Phi w lam m := by
  obtain ⟨hi, ho, ⟩hh := shape_dimensions D
  exact transfer_canonical_pair hi ho hh hfit hr
    (hA.trans (Nat.add_sub_of_le D.r_le_a).symm)
    (hB.trans (Nat.add_sub_of_le D.r_le_b).symm) hc
end RankShape
```

The fiber formula at every given exact fit. Fix $w = (A, B)$ with $BA = \Phi$ and set $a = \operatorname{rank} A$, $b = \operatorname{rank} B$, $r = \operatorname{rank} \Phi$, $p = O - b$, $q = I - a$, $h = H + r - (a + b)$. Actual-rank feasibility is already proved. Apply the canonical measured result in these six dimensions, then the preceding dimension/rank transport. The resulting pair is

$$\left(\frac{Oa + b(I - a) + d_0(p, q, h)}{2},\ \mu_0(p, q, h) \right).$$

The dimension-local theorem assumes only the residual pair in these actual residual dimensions and proves the complete attachment to the original loss. The Wishart wrapper discharges that assumption through the residual analysis; the final unconditional theorem later supplies the proved Wishart identity itself. When $p = 0$, $q = 0$, or $h = 0$, the neutral residual result instead gives $(0, 1)$, and the proved arithmetic identifications $d_0 = 0$, $\mu_0 = 1$ put it in the same public formula. On $I, O, H > 0$, positivity of the full threshold is the already proved `O77.candidateTwiceLambda_positive` applied to actual-rank feasibility and then cast into $\mathbb{R}$. Auxiliary empty-dimensional models remain valid without this strict positivity conclusion.

```
/-- A dimension-local analytic input suffices at this actual exact fit.
The canonical chart and nonorthogonal basis transport are already proved;
no global Wishart statement is needed by this attachment theorem. -/
theorem exact_fit_pair_of_residual {I O H : ℕ}
    {Phi : RM O I} {w : Parameters I O H} (hfit : w.2 * w.1 = Phi)
    (hres : O77.Residual.HasResidualBandPairAtOrigin (O-w.2.rank) (I-w.1.rank)
      (O77.residualHidden H Phi.rank w.1.rank w.2.rank)
      ((O77.residualMin (O-w.2.rank) (I-w.1.rank)
        (O77.residualHidden H Phi.rank w.1.rank w.2.rank) : ℝ)/2)
      (O77.residualMultiplicity (O-w.2.rank) (I-w.1.rank)
        (O77.residualHidden H Phi.rank w.1.rank w.2.rank))) :
    HasLocalBandPair Phi w
      ((O77.candidateTwiceLambda I O H Phi.rank w.1.rank w.2.rank : ℝ)/2)
      (O77.candidateMultiplicity I O H Phi.rank w.1.rank w.2.rank) := by
  let D : O77.RankData I O H Phi.rank w.1.rank w.2.rank :=
    O77.Adapted0906.rankData_of_exact_fit hfit
  apply transfer_rankData_pair D hfit rfl rfl rfl
  have hc := O77.Canonical0906.canonical_pair_of_residual
    Phi.rank (w.1.rank-Phi.rank) (w.2.rank-Phi.rank)
    (O77.residualHidden H Phi.rank w.1.rank w.2.rank)
    (O-w.2.rank) (I-w.1.rank) hres
  simpa only [canonical_regularDim D, O77.candidateTwiceLambda,
    O77.candidateMultiplicity, Nat.cast_add, add_div] using hc

/-- The inherited public conditional signature is unchanged. -/
theorem exact_fit_pair_of_wishart
```

```
    (hW : O77.External.RealWishartEigenvalueFormula) {I O H : ℕ}
    {Phi : RM O I} {w : Parameters I O H} (hfit : w.2 * w.1 = Phi) :
    HasLocalBandPair Phi w
      ((O77.candidateTwiceLambda I O H Phi.rank w.1.rank w.2.rank : ℝ)/2)
      (O77.candidateMultiplicity I O H Phi.rank w.1.rank w.2.rank) := by
  exact exact_fit_pair_of_residual hfit
    (O77.Residual.residualBandPair_all_dimensions_of_wishart_instantiated hW _ _ _)

/-- No Wishart input is needed when one actual residual dimension vanishes. -/
theorem exact_fit_pair_neutral {I O H : ℕ}
    {Phi : RM O I} {w : Parameters I O H} (hfit : w.2 * w.1 = Phi)
    (hz : O-w.2.rank = 0 ∨ I-w.1.rank = 0 ∨
      O77.residualHidden H Phi.rank w.1.rank w.2.rank = 0) :
    HasLocalBandPair Phi w
      ((O77.candidateTwiceLambda I O H Phi.rank w.1.rank w.2.rank : ℝ)/2)
      (O77.candidateMultiplicity I O H Phi.rank w.1.rank w.2.rank) := by
  apply exact_fit_pair_of_residual hfit
  have hd := O77.Arithmetic.residualMin_zero (O-w.2.rank) (I-w.1.rank)
    (O77.residualHidden H Phi.rank w.1.rank w.2.rank) hz
  have hm := O77.Arithmetic.residualMultiplicity_zero (O-w.2.rank) (I-w.1.rank)
    (O77.residualHidden H Phi.rank w.1.rank w.2.rank) hz
  simpa only [hd, hm, Nat.cast_zero, zero_div] using
    O77.Residual.residualBandPair_neutral_of_zero_dimension hz

/-- Positivity on the original nonempty-dimensional O77 domain uses the checked
finite arithmetic. The wider formula also covers constant auxiliary models. -/
lemma exact_fit_threshold_positive {I O H : ℕ}
    (hI : 0 < I) (hO : 0 < O) (hH : 0 < H)
    {Phi : RM O I} {w : Parameters I O H} (hfit : w.2 * w.1 = Phi) :
    0 < (O77.candidateTwiceLambda I O H Phi.rank w.1.rank w.2.rank : ℝ)/2 := by
  apply half_pos
  exact_mod_cast O77.candidateTwiceLambda_positive hI hO hH
    (O77.Adapted0906.rankData_of_exact_fit hfit)
```

Realizing ranks over the prescribed target. It remains to prove sufficiency of feasibility for *each fixed* Φ, not merely for some target of the same rank. First, equal-rank single matrices M, N are handled by applying the two-arrow orbit theorem to the pairs $(M, 1)$ and $(N, 1)$. It produces invertible G_I, G_O with $G_O M G_I^{-1} = N$. Second, the canonical matrices in the dimensions forced by a feasible rank datum give an actual pair w_0 of ranks a, b whose product has rank r. Apply the single-matrix result to $B_0 A_0$ and the prescribed Φ. Change only the input and output bases of w_0, taking the hidden unit to be 1. The product becomes exactly Φ, while both factor ranks are preserved. This constructs the witness in

$$\mathrm{RankData}(I, O, H, \operatorname{rank}\Phi, a, b) \iff \exists (A, B) : BA = \Phi,\ \operatorname{rank} A = a,\ \operatorname{rank} B = b.$$

The reverse implication is the actual-rank necessity theorem. The whole realization argument is algebraic and independent of Wishart or any assumed measured pair.

```
section Realization
/-- A single matrix is treated as the first arrow of M followed by id.
Thus target transport reuses the simultaneous-pair theorem, including O=0. -/
theorem target_matrices_of_rank {I O : ℕ} (M N : RM O I) (hr : M.rank = N.rank) :
    ∃ (GI : (RM I I)ˣ) (GO : (RM O O)ˣ),
      (GO : RM O O) * M * ↑((⁻GI¹) : RM I I) = N := by
  obtain ⟨GI, _, GO, h, ⟩_ := O77.NumberedCoordinates.basis_matrices_of_ranks
    M N (1 : RM O O) (1 : RM O O) hr rfl (by simpa using hr)
  exact ⟨GI, GO, ⟩h

private lemma canonical_rank_witness_of_dimensions {I O H r alpha beta h p q : ℕ}
    (hi : (r+alpha)+q = I) (ho : (r+beta)+p = O)
```

```
    (hh : (r+alpha)+(beta+h) = H) :
    ∃ w : Parameters I O H,
      w.1.rank = r+alpha ∧ w.2.rank = r+beta ∧ (w.2*w.1).rank = r := by
  subst I
  subst O
  subst H
  have hr := canonicalParameter_ranks r alpha beta h p q
  refine ⟨canonicalParameter r alpha beta h p q, hr.1, hr.2.1, ?⟩_
  rw [canonical_exact_fit]
  exact hr.2.2

/-- Construct the rank triple before seeing any target matrix.
This does not assume that the prescribed rank stratum is nonempty. -/
lemma canonical_rank_witness {I O H r a b : ℕ} (D : O77.RankData I O H r a b) :
    ∃ w : Parameters I O H, w.1.rank = a ∧ w.2.rank = b ∧ (w.2*w.1).rank = r := by
  obtain ⟨hi, ho, ⟩hh := shape_dimensions D
  obtain ⟨w, hA, hB, ⟩hBA := canonical_rank_witness_of_dimensions hi ho hh
  exact ⟨w, hA.trans (Nat.add_sub_of_le D.r_le_a),
    hB.trans (Nat.add_sub_of_le D.r_le_b), ⟩hBA

/-- Necessity and sufficiency in the actual fiber of this fixed target.
The construction changes input/output bases only; the hidden unit is 1. -/
theorem feasible_iff_realized {I O H : ℕ} (Phi : RM O I) (a b : ℕ) :
    O77.RankData I O H Phi.rank a b ↔
      ∃ w : Parameters I O H, w.2*w.1 = Phi ∧ w.1.rank = a ∧ w.2.rank = b := by
  constructor
  · intro D
    obtain ⟨w0, ha, hb, ⟩hr := canonical_rank_witness D
    obtain ⟨GI, GO, ⟩ht := target_matrices_of_rank (w0.2*w0.1) Phi hr
    let w := O77.BasisChange0906.change GI GO (1 : (RM H H)ˣ) w0
    have hfit := O77.BasisChange0906.exact_fit_change GI GO (1 : (RM H H)ˣ)
      (Phi := w0.2*w0.1) (w := w0) rfl
    have hranks := O77.BasisChange0906.change_ranks GI GO (1 : (RM H H)ˣ)
      w0 (w0.2*w0.1)
    exact ⟨w, hfit.trans ht, hranks.1.trans ha, hranks.2.1.trans ⟩hb
  · rintro ⟨w, hfit, ha, ⟩hb
    have D := O77.Adapted0906.rankData_of_exact_fit hfit
    simpa only [ha, hb] using D
end Realization
```

Concrete consequences that distinguish the contract. At the 2×2 zero target, $(A, B) = (1, 0)$ has pair $(2, 1)$ by the neutral theorem: absence of residual variables does not remove the regular quadratic contribution. The scalar zero pair has pair $(1/2, 2)$, retaining the logarithmic resonance. The rank-realization example prescribes an arbitrary 3×4 target of rank one and constructs ranks $(2, 3)$ with hidden dimension four. The last example has hidden dimension zero and pair $(0, 1)$, as expected for the constant auxiliary germ. These are applications of the general statements, not substitutes for the quantified statements proved above.

```
-- Endpoint regressions; no numerical rank labels are assumed.
example : HasLocalBandPair (0 : RM 2 2) ((1 : RM 2 2), (0 : RM 2 2)) 2 1 := by
  have hf : (0 : RM 2 2) * (1 : RM 2 2) = 0 := by simp
  have hz : 2 - (1 : RM 2 2).rank = 0 := by simp
  have hp := exact_fit_pair_neutral (w := ((1 : RM 2 2), (0 : RM 2 2)))
    hf (Or.inr (Or.inl hz))
  have ht : O77.candidateTwiceLambda 2 2 2 0 2 0 = 4 := by decide
  have hm : O77.candidateMultiplicity 2 2 2 0 2 0 = 1 := by decide
  norm_num only [Matrix.rank_one, Matrix.rank_zero, Fintype.card_fin, ht, hm] at hp
  exact hp
```

```
example (hW : O77.External.RealWishartEigenvalueFormula) :
    HasLocalBandPair (0 : RM 1 1) ((0 : RM 1 1), (0 : RM 1 1)) (1/2 : ℝ) 2 := by
  have hp := exact_fit_pair_of_wishart hW (Phi := 0)
    (w := ((0 : RM 1 1), (0 : RM 1 1))) (by simp)
  have ht : O77.candidateTwiceLambda 1 1 1 0 0 0 = 1 := by decide
  have hm : O77.candidateMultiplicity 1 1 1 0 0 0 = 2 := by decide
  simpa only [Matrix.rank_zero, ht, hm, Nat.cast_one] using hp

example {Phi : RM 3 4} (hr : Phi.rank = 1) :
    ∃ w : Parameters 4 3 4, w.2*w.1 = Phi ∧ w.1.rank = 2 ∧ w.2.rank = 3 := by
  apply (feasible_iff_realized (H := 4) Phi 2 3).mp
  rw [hr]
  exact ⟨by decide, by decide, by decide, by decide, by decide, by decide, by ⟩decide

example : HasLocalBandPair (0 : RM 3 2) ((0 : RM 0 2), (0 : RM 3 0)) 0 1 := by
  have hp := exact_fit_pair_neutral (Phi := 0) (w := ((0 : RM 0 2), (0 : RM 3 0)))
    (by simp) (Or.inr (Or.inr (by simp [O77.residualHidden])))
  have ht : O77.candidateTwiceLambda 2 3 0 0 0 0 = 0 := by decide
  have hm : O77.candidateMultiplicity 2 3 0 0 0 0 = 1 := by decide
  simpa only [Matrix.rank_zero, ht, hm, Nat.cast_zero, zero_div] using hp

end O77.FiberClassification
end
```

40 Module `O77SaddlePreparation`

This module turns bounds on actual derivatives into the moving planar centers needed for a full-dimensional band estimate. Throughout, $P = \mathbb{R}^2$, $q_P(x) = x_1^2 + x_2^2$, and $E = \mathbb{R}^d$; in the code these are `Plane`, `planeSq`, and `Taild`. The default norm on `Plane` is the maximum norm, so every disk below is specified by q_P. The tail has its Euclidean norm and the ambient squared radius is $q_P(y) + \|z\|^2$. All derivatives are Fréchet derivatives of the named function. The scalar comparison is proved before applying it to these derivatives; this avoids imposing radial estimates as unexplained data.

```
import Mathlib
import O77

set_option autoImplicit false
noncomputable section
open MeasureTheory Set Filter
open scoped Topology ContDiff
namespace O77.SaddlePreparation0906
open O77.SaddleDirect
```

40.1 Two derivative comparisons give quadratic secants

Let $h, d, e : \mathbb{R} \to \mathbb{R}$ be functions and let $R, k, K \in \mathbb{R}$ with $R \geq 0$. Here d is a scalar derivative function, not the tail dimension. Assume that for every $t \in [0, R]$ the ordinary derivatives satisfy $h'(t) = d(t)$ and $d'(t) = e(t)$, including derivatives at the two endpoints, and that $d(0) = 0$ and $k \leq e(t) \leq K$ for every such t. Then, for every $a, b \in \mathbb{R}$ with $0 \leq a \leq b \leq R$,

$$\frac{k}{2}(b^2 - a^2) \leq h(b) - h(a) \leq \frac{K}{2}(b^2 - a^2).$$

Here k need not be positive. First apply the mean-value inequalities to d between 0 and r to get $kr \leq d(r) \leq Kr$. The derivatives of $h(r) - kr^2/2$ and $h(r) - Kr^2/2$ are respectively nonnegative and nonpositive; monotonicity on the closed interval gives the result. The formal proof uses

`Convex.mul_sub_le_image_sub_of_le_deriv`, `Convex.image_sub_le_mul_sub_of_deriv_le`, `monotoneOn_of_deriv_nonneg`, and `antitoneOn_of_deriv_nonpos`. Continuity on the closed interval and differentiability on its interior are obtained from the stated derivatives, including the endpoint case $R = 0$.

```
/-- Two mean-value comparisons, with no differentiation of sqrt at zero. -/
theorem quadratic_secants
    {h d e : ℝ → ℝ} {R k K : ℝ} (hR : 0 ≤ R)
    (hh : ∀ r ∈ Icc (0 : ℝ) R, HasDerivAt h (d r) r)
    (hd : ∀ r ∈ Icc (0 : ℝ) R, HasDerivAt d (e r) r)
    (hd0 : d 0 = 0)
    (he : ∀ r ∈ Icc (0 : ℝ) R, k ≤ e r ∧ e r ≤ K)
    {a b : ℝ} (ha : a ∈ Icc (0 : ℝ) R)
    (hb : b ∈ Icc (0 : ℝ) R) (hab : a ≤ b) :
    (k / 2) * (b ^ 2 - a ^ 2) ≤ h b - h a ∧
      h b - h a ≤ (K / 2) * (b ^ 2 - a ^ 2) := by
  have hc : ContinuousOn d (Icc (0 : ℝ) R) :=
    fun r hr => (hd r hr).continuousAt.continuousWithinAt
  have hdi : DifferentiableOn ℝ d (interior (Icc (0 : ℝ) R)) :=
    fun r hr => (hd r (interior_subset hr)).differentiableAt.differentiableWithinAt
  have hv (r : ℝ) (hr : r ∈ Icc (0 : ℝ) R) :
      k * r ≤ d r ∧ d r ≤ K * r := by
    constructor
    · have h := (convex_Icc (0 : ℝ) R).mul_sub_le_image_sub_of_le_deriv
        hc hdi (fun x hx => by
          rw [(hd x (interior_subset hx)).deriv]
          exact (he x (interior_subset hx)).1)
        0 ⟨le_rfl, ⟩hR r hr hr.1
      simpa [hd0] using h
    · have h := (convex_Icc (0 : ℝ) R).image_sub_le_mul_sub_of_deriv_le
        hc hdi (fun x hx => by
          rw [(hd x (interior_subset hx)).deriv]
          exact (he x (interior_subset hx)).2)
        0 ⟨le_rfl, ⟩hR r hr hr.1
      simpa [hd0] using h
  have hshift (c r : ℝ) (hr : r ∈ Icc (0 : ℝ) R) :
      HasDerivAt (fun t => h t - (c / 2) * t ^ 2) (d r - c * r) r := by
    have hsquare : HasDerivAt (fun t : ℝ => t ^ 2) (2 * r) r := by
      exact (hasDerivAt_pow 2 r).congr_deriv (by norm_num)
    exact ((hh r hr).sub (hsquare.const_mul (c / 2))).congr_deriv (by ring)
  have hmono : MonotoneOn (fun t => h t - (k / 2) * t ^ 2) (Icc (0 : ℝ) R) := by
    apply monotoneOn_of_deriv_nonneg (convex_Icc (0 : ℝ) R)
    · exact fun r hr => (hshift k r hr).continuousAt.continuousWithinAt
    · exact fun r hr => (hshift k r (interior_subset hr)).differentiableAt.differentiableWithinAt
    · intro r hr
      rw [(hshift k r (interior_subset hr)).deriv]
      exact sub_nonneg.mpr (hv r (interior_subset hr)).1
  have hanti : AntitoneOn (fun t => h t - (K / 2) * t ^ 2) (Icc (0 : ℝ) R) := by
    apply antitoneOn_of_deriv_nonpos (convex_Icc (0 : ℝ) R)
    · exact fun r hr => (hshift K r hr).continuousAt.continuousWithinAt
    · exact fun r hr => (hshift K r (interior_subset hr)).differentiableAt.differentiableWithinAt
    · intro r hr
      rw [(hshift K r (interior_subset hr)).deriv]
      exact sub_nonpos.mpr (hv r (interior_subset hr)).2
  constructor
  · have h := hmono ha hb hab
    nlinarith
  · have h := hanti ha hb hab
    nlinarith
```

Applying the comparison to an actual line. For $h : P \to \mathbb{R}$, fix a center c with $Dh(c) = 0$ and direction v. The chain rule on $r \mapsto c + rv$ gives

$$(h(c + rv))' = Dh(c + rv)[v], \qquad (Dh(c + rv)[v])' = D(Dh)(c + rv)[v, v].$$

Differentiating evaluation at the fixed vector v introduces a second term evaluated on the zero derivative of v; linearity makes that term zero. Thus bounds k, K on the displayed Hessian evaluations give the preceding secants. `HasDerivAt.clm_apply` supplies this evaluation rule. The following square identity $q_P(tv) = t^2 q_P(v)$ is an entrywise ring identity.

```
/-- Both derivatives are derivatives of the actual scalar function along the line. -/
theorem line_secants {h : Plane → ℝ} {c v : Plane} {R k K : ℝ}
    (hR : 0 ≤ R) (h0 : fderiv ℝ h c = 0)
    (hD : ∀ t ∈ Icc (0 : ℝ) R, DifferentiableAt ℝ h (c + t • v))
    (hD2 : ∀ t ∈ Icc (0 : ℝ) R,
      DifferentiableAt ℝ (fderiv ℝ h) (c + t • v))
    (hB : ∀ t ∈ Icc (0 : ℝ) R,
      k ≤ (fderiv ℝ (fderiv ℝ h) (c + t • v) v) v ∧
      (fderiv ℝ (fderiv ℝ h) (c + t • v) v) v ≤ K)
    {a b : ℝ} (ha : a ∈ Icc (0 : ℝ) R)
    (hb : b ∈ Icc (0 : ℝ) R) (hab : a ≤ b) :
    (k / 2) * (b ^ 2 - a ^ 2) ≤ h (c + b • v) - h (c + a • v) ∧
      h (c + b • v) - h (c + a • v) ≤ (K / 2) * (b ^ 2 - a ^ 2) := by
  let d : ℝ → ℝ := fun t => fderiv ℝ h (c + t • v) v
  let e : ℝ → ℝ := fun t => (fderiv ℝ (fderiv ℝ h) (c + t • v) v) v
  have hpath (t : ℝ) : HasDerivAt (fun s : ℝ => c + s • v) v t := by
    simpa using ((hasDerivAt_id' t).smul_const v).const_add c
  have hh (t : ℝ) (ht : t ∈ Icc (0 : ℝ) R) :
      HasDerivAt (fun s => h (c + s • v)) (d t) t :=
    (hD t ht).hasFDerivAt.comp_hasDerivAt t (hpath t)
  have hd (t : ℝ) (ht : t ∈ Icc (0 : ℝ) R) : HasDerivAt d (e t) t := by
    have hline : HasDerivAt (fun s : ℝ => fderiv ℝ h (c + s • v))
        (fderiv ℝ (fderiv ℝ h) (c + t • v) v) t :=
      (hD2 t ht).hasFDerivAt.comp_hasDerivAt t (hpath t)
    have heval : HasDerivAt d
        (e t + fderiv ℝ h (c + t • v) (0 : Plane)) t :=
      HasDerivAt.clm_apply hline (hasDerivAt_const t v)
    simpa only [map_zero, add_zero] using heval
  exact quadratic_secants hR hh hd (by simp [d, h0]) hB ha hb hab

private lemma planeSq_smul (t : ℝ) (v : Plane) :
    planeSq (t • v) = t ^ 2 * planeSq v := by
  rcases v with ⟨x, y⟩
  change (t * x) ^ 2 + (t * y) ^ 2 = t ^ 2 * (x ^ 2 + y ^ 2)
  ring
```

Squared-radius parametrization, including the origin. Suppose $R > 0$, $0 < k \le K$, h is continuous, $Dh(c) = 0$, and both required derivatives exist on $c + \{q_P \le R^2\}$, with $kq_P(v) \le D^2h(c + x)[v, v] \le Kq_P(v)$ there. For $v = (\cos\theta, \sin\theta)$ one has $q_P(v) = 1$. Apply the line secants at $a = \sqrt{s}$, $b = \sqrt{t}$ for $0 \le s \le t \le R^2$. The result is a two-sided bound by $k(t - s)/2$ and $K(t - s)/2$ for the increments of $h(c + \sqrt{t}\, v) - h(c)$. This proves precisely `RadialControl`; no derivative of $\sqrt{s}$ at zero is taken. The square-root inequalities use $R > 0$, and `Real.sq_sqrt` is invoked only for nonnegative arguments.

```
/-- Uniform actual Hessian bounds on a translated disk imply radial secants. -/
theorem radialControl_of_hessian_disk {h : Plane → ℝ} {c : Plane}
    {R k K : ℝ} (hR : 0 < R) (hk : 0 < k) (hkK : k ≤ K)
    (hc : Continuous h) (h0 : fderiv ℝ h c = 0)
    (hD : ∀ x, planeSq x ≤ R ^ 2 → DifferentiableAt ℝ h (c + x))
    (hD2 : ∀ x, planeSq x ≤ R ^ 2 →
```

```
      DifferentiableAt ℝ (fderiv ℝ h) (c + x))
    (hB : ∀ x, planeSq x ≤ R ^ 2 → ∀ v : Plane,
      k * planeSq v ≤ (fderiv ℝ (fderiv ℝ h) (c + x) v) v ∧
      (fderiv ℝ (fderiv ℝ h) (c + x) v) v ≤ K * planeSq v) :
    RadialControl (fun x => h (c + x) - h c) R k K := by
  refine ⟨hR, hk, hkK, by fun_prop, ?_, ?⟩_
  · change h (c + (0 : Plane)) - h c = 0
    rw [add_zero, sub_self]
  · intro theta s hs t ht hst
    let v : Plane := (Real.cos theta, Real.sin theta)
    have hv : planeSq v = 1 := by
      dsimp [planeSq, v]
      nlinarith [Real.sin_sq_add_cos_sq theta]
    have hp (r : ℝ) (hr : r ∈ Icc (0 : ℝ) R) : planeSq (r • v) ≤ R ^ 2 := by
      rw [planeSq_smul, hv, mul_one]
      nlinarith [mul_nonneg (sub_nonneg.mpr hr.2) (show 0 ≤ R + r by linarith [hr.1])]
    have hsqrt (u : ℝ) (hu : u ∈ Icc (0 : ℝ) (R ^ 2)) :
        Real.sqrt u ∈ Icc (0 : ℝ) R := by
      exact ⟨Real.sqrt_nonneg u, (Real.sqrt_le_left hR.le).mpr hu⟩.2
    have hsec := line_secants hR.le h0
      (fun r hr => hD _ (hp r hr))
      (fun r hr => hD2 _ (hp r hr))
      (fun r hr => by simpa [hv] using hB _ (hp r hr) v)
      (hsqrt s hs) (hsqrt t ht) (Real.sqrt_le_sqrt hst)
    change (k / 2) * (t - s) ≤
        (h (c + Real.sqrt t • v) - h c) - (h (c + Real.sqrt s • v) - h c) ∧
      (h (c + Real.sqrt t • v) - h c) - (h (c + Real.sqrt s • v) - h c) ≤
        (K / 2) * (t - s)
    simpa only [Real.sq_sqrt hs.1, Real.sq_sqrt ht.1,
      sub_sub_sub_cancel_right] using hsec
```

The center is a minimum on the disk. For any x with $q_P(x) \le R^2$, the segment tx, $0 \le t \le 1$, remains in the disk because $q_P(tx) = t^2 q_P(x)$. Apply the same line comparison with Hessian bounds $kq_P(x)$ and $Kq_P(x)$ to obtain

$$h(c+x) - h(c) \ge \frac{k}{2} q_P(x).$$

This helper itself allows arbitrary real k and R; its later use with $k > 0$ gives nonnegativity. The minimum assertion is therefore proved, not an additional field assumed about the function.

```
/-- The redundant minimum field is proved, not added as an extra hypothesis. -/
theorem minimum_on_disk_of_hessian {h : Plane → ℝ} {c : Plane}
    {R k K : ℝ} (h0 : fderiv ℝ h c = 0)
    (hD : ∀ x, planeSq x ≤ R ^ 2 → DifferentiableAt ℝ h (c + x))
    (hD2 : ∀ x, planeSq x ≤ R ^ 2 →
      DifferentiableAt ℝ (fderiv ℝ h) (c + x))
    (hB : ∀ x, planeSq x ≤ R ^ 2 → ∀ v : Plane,
      k * planeSq v ≤ (fderiv ℝ (fderiv ℝ h) (c + x) v) v ∧
      (fderiv ℝ (fderiv ℝ h) (c + x) v) v ≤ K * planeSq v)
    (x : Plane) (hx : planeSq x ≤ R ^ 2) :
    (k / 2) * planeSq x ≤ h (c + x) - h c := by
  have hq : 0 ≤ planeSq x := add_nonneg (sq_nonneg _) (sq_nonneg _)
  have hp (t : ℝ) (ht : t ∈ Icc (0 : ℝ) 1) : planeSq (t • x) ≤ R ^ 2 := by
    rw [planeSq_smul]
    exact (mul_le_mul_of_nonneg_right (by nlinarith [ht.1, ht.2, mul_nonneg ht.1 (sub_nonneg.mpr ht.2)
     ] : t ^ 2 ≤ 1)
      hq).trans (by simpa using hx)
  have hsec := line_secants (h := h) (c := c) (v := x) (R := 1) zero_le_one h0
    (fun t ht => hD _ (hp t ht))
    (fun t ht => hD2 _ (hp t ht)) (fun t ht => hB _ (hp t ht) x)
```

```
    (a := 0) (b := 1) ⟨le_rfl, ⟩zero_le_one ⟨zero_le_one, ⟩le_rfl zero_le_one
  have heq : (k * planeSq x) / 2 = (k / 2) * planeSq x := by ring
  simpa [heq] using hsec.1
```

40.2 Constructing moving critical centers

For a scalar function $f : P \times E \to \mathbb{R}$, write $f_z(y) = f(y, z)$. Define $G(z, y) = Df_z(y) \in P^*$. Its variables are ordered as (z, y) because the implicit-function theorem solves the second variable. This is the actual differential, with no choice of a gradient or Riesz map.

```
/-- The implicit equation is the actual partial differential; the plane is second. -/
def partialDifferential {d : ℕ} (f : Plane × Tail d → ℝ)
    (zy : Tail d × Plane) : Plane →L[ℝ] ℝ :=
  fderiv ℝ (fun y : Plane => f (y, zy.1)) zy.2
```

Assume G is C^1 at $(0, 0)$, $G(0, 0) = 0$, and $D_yG(0, 0) : P \to P^*$ is invertible. Given $R, S_0 > 0$, there are a function $c : E \to P$ and $0 < S < S_0$ such that $c(0) = 0$ and, for $\|z\| < S$,

$$c \text{ is continuous at } z, \qquad G(z, c(z)) = 0, \qquad q_P(c(z)) < (R/2)^2.$$

Use `ContDiffAt.implicitFunction`, its value at the base point, `ContDiffAt.contDiffAt_implicitFunction`, and `ContDiffAt.eventually_apply_implicitFunction`. Continuity and the strict square bound hold on neighborhoods by continuity at zero. The finite-order theorem `ContDiffAt.eventually` supplies continuity at every point of a common neighborhood. Intersect these neighborhoods and choose an open ball inside them, shrinking its radius below S_0. The total function c outside this neighborhood has no role.

```
/-- Apply the existing IFT and shrink once, before any band radius or width. -/
theorem implicit_center {d : ℕ} (f : Plane × Tail d → ℝ)
    {R S0 : ℝ} (hR : 0 < R) (hS0 : 0 < S0)
    (hG : ContDiffAt ℝ 1 (partialDifferential f) (0, 0))
    (hG0 : partialDifferential f (0, 0) = 0)
    (hInv : ((fderiv ℝ (partialDifferential f) (0, 0)).comp
      (ContinuousLinearMap.inr ℝ (Tail d) Plane)).IsInvertible) :
    ∃ (c : Tail d → Plane) (S : ℝ), 0 < S ∧ S < S0 ∧ c 0 = 0 ∧
      ∀ z, ‖‖z < S → ContinuousAt c z ∧
        partialDifferential f (z, c z) = 0 ∧ planeSq (c z) < (R / 2) ^ 2 := by
  let c : Tail d → Plane := hG.implicitFunction (by norm_num) hInv
  have hc0 : c 0 = 0 := hG.implicitFunction_apply_self (by norm_num) hInv
  have hc1 : ContDiffAt ℝ 1 c 0 := hG.contDiffAt_implicitFunction (by norm_num) hInv
  have hEq : ∀ᶠ z in nhds (0 : Tail d), partialDifferential f (z, c z) = 0 := by
    simpa only [hG0] using hG.eventually_apply_implicitFunction (by norm_num) hInv
  have hcont : ∀ᶠ z in nhds (0 : Tail d), ContinuousAt c z :=
    (hc1.eventually (by norm_num)).mono fun _ h => h.continuousAt
  have hqcont : Continuous (fun y : Plane => planeSq y) := by
    unfold planeSq
    fun_prop
  have hq0 : planeSq (c 0) < (R / 2) ^ 2 := by
    rw [hc0]
    change (0 : ℝ) ^ 2 + 0 ^ 2 < (R / 2) ^ 2
    simpa only [zero_pow (by decide : 2 ≠ 0), zero_add] using
      (sq_pos_of_pos (half_pos hR))
  have hsmall : ∀ᶠ z in nhds (0 : Tail d), planeSq (c z) < (R / 2) ^ 2 :=
    (hqcont.continuousAt.comp hc1.continuousAt).eventually
      (Iio_mem_nhds hq0)
  have hall : ∀ᶠ z in nhds (0 : Tail d), ContinuousAt c z ∧
      partialDifferential f (z, c z) = 0 ∧ planeSq (c z) < (R / 2) ^ 2 := by
    filter_upwards [hcont, hEq, hsmall] with z hz he hs
    exact ⟨hz, he, ⟩hs
  obtain ⟨s, hs, ⟩hsub := Metric.mem_nhds_iff.mp hall
```

```
  refine ⟨c, min s (S0 / 2), lt_min hs (half_pos hS0),
    (min_le_right _ _).trans_lt (half_lt_self hS0), hc0, ?⟩_
  intro z hz
  have hzs : ‖z‖ < s := hz.trans_le (min_le_left s (S0 / 2))
  have hzball : z ∈ Metric.ball (0 : Tail d) s := by
    simpa only [Metric.mem_ball, dist_zero_right] using hzs
  exact hsub hzball
```

The exact derivative-data interface. `UniformPartialHessianf` records $R, S_0 > 0$ and $0 < k \le K$, the preceding C^1, zero, and invertibility assertions for G, and differentiability of f_z and Df_z with

$$kq_P(v) \le D^2 f_z(y)[v,v] \le Kq_P(v) \quad (\|z\| < S_0,\ q_P(y) \le (2R)^2).$$

It contains neither a critical-center function nor a band-volume conclusion. The doubled planar radius is deliberate: the next proof must fit the translated radius-R disks into this common domain.

```
/-- Pointwise derivative data; no center, radial bound or volume pair is a field. -/
structure UniformPartialHessian {d : ℕ} (f : Plane × Tail d → ℝ) where
  radius : ℝ
  tailRadius : ℝ
  lower : ℝ
  upper : ℝ
  radius_pos : 0 < radius
  tail_pos : 0 < tailRadius
  lower_pos : 0 < lower
  lower_le_upper : lower ≤ upper
  partial_c1 : ContDiffAt ℝ 1 (partialDifferential f) (0, 0)
  partial_zero : partialDifferential f (0, 0) = 0
  partial_invertible : ((fderiv ℝ (partialDifferential f) (0, 0)).comp
    (ContinuousLinearMap.inr ℝ (Tail d) Plane)).IsInvertible
  diff : ∀ z, ‖z‖ < tailRadius → ∀ y, planeSq y ≤ (2 * radius) ^ 2 →
    DifferentiableAt ℝ (fun x : Plane => f (x, z)) y
  diff2 : ∀ z, ‖z‖ < tailRadius → ∀ y, planeSq y ≤ (2 * radius) ^ 2 →
    DifferentiableAt ℝ (fderiv ℝ (fun x : Plane => f (x, z))) y
  bounds : ∀ z, ‖z‖ < tailRadius → ∀ y, planeSq y ≤ (2 * radius) ^ 2 →
    ∀ v : Plane,
      lower * planeSq v ≤ (fderiv ℝ (fderiv ℝ (fun x : Plane => f (x, z))) y v) v ∧
      (fderiv ℝ (fderiv ℝ (fun x : Plane => f (x, z))) y v) v ≤ upper * planeSq v
```

Checking all translated disks on one domain. For continuous f and such data, construct c, S as above. If $q_P(x) \le R^2$, then

$$q_P(c(z) + x) \le 2q_P(c(z)) + 2q_P(x) < \tfrac{5}{2}R^2 < 4R^2.$$

Thus every point used by either derivative comparison lies in the original Hessian cylinder. Apply the radial and minimum lemmas to f_z centered at $c(z)$. Together with the continuity, zero, and smallness properties already proved, these furnish every field of `CenteredFamilyf`. In particular the disk-minimum field follows from $k > 0$ and $q_P(x) \ge 0$. All these choices precede the band radius and the band width.

```
/-- The derivative-to-record construction, including all disk/domain compatibility. -/
theorem centeredFamily_of_uniformHessian {d : ℕ}
    (f : Plane × Tail d → ℝ) (hf : Continuous f) (D : UniformPartialHessian f) :
    Nonempty (CenteredFamily f) := by
  obtain ⟨c, S, hS, hSS, hc0, ⟩hc := implicit_center f D.radius_pos D.tail_pos
    D.partial_c1 D.partial_zero D.partial_invertible
```

```
  have hshift (z : Tail d) (hz : |||z < S) (x : Plane)
      (hx : planeSq x ≤ D.radius ^ 2) : planeSq (c z + x) ≤ (2 * D.radius) ^ 2 := by
    have hs := (hc z hz).2.2
    have hadd : planeSq (c z + x) ≤ 2 * planeSq (c z) + 2 * planeSq x := by
      dsimp [planeSq]
      nlinarith [sq_nonneg ((c z).1 - x.1), sq_nonneg ((c z).2 - x.2)]
    nlinarith [sq_nonneg D.radius]
  have hcrit (z : Tail d) (hz : |||z < S) :
      fderiv ℝ (fun y : Plane => f (y, z)) (c z) = 0 := (hc z hz).2.1
  refine ({
    radius := D.radius
    tailRadius := S
    lowerBound := D.lower
    upperBound := D.upper
    radius_pos := D.radius_pos
    tail_pos := hS
    lower_pos := D.lower_pos
    lower_le_upper := D.lower_le_upper
    center := c
    center_zero := hc0
    center_cont := fun z hz => (hc z hz).1
    center_small := fun z hz => (hc z hz).2.2
    radial := ?_
    minimum_on_disk := ?_ )}
  · intro z hz
    exact radialControl_of_hessian_disk D.radius_pos D.lower_pos D.lower_le_upper
      (by fun_prop) (hcrit z hz)
      (fun x hx => D.diff z (hz.trans hSS) _ (hshift z hz x hx))
      (fun x hx => D.diff2 z (hz.trans hSS) _ (hshift z hz x hx))
      (fun x hx v => D.bounds z (hz.trans hSS) _ (hshift z hz x hx) v)
  · intro z hz x hx
    have h := minimum_on_disk_of_hessian (hcrit z hz)
      (fun x hx => D.diff z (hz.trans hSS) _ (hshift z hz x hx))
      (fun x hx => D.diff2 z (hz.trans hSS) _ (hshift z hz x hx))
      (fun x hx v => D.bounds z (hz.trans hSS) _ (hshift z hz x hx) v) x hx
    have hq : 0 ≤ planeSq x := add_nonneg (sq_nonneg _) (sq_nonneg _)
    exact (mul_nonneg (div_nonneg D.lower_pos.le (by norm_num)) hq).trans h
```

40.3 Why the conclusion has full ambient volume

Add `LowerValuesAccumulatef`: every positive ambient radius contains a point with $f < f(0)$. The previously established theorem `O77.SaddleDirect.ambientPair_of_lowerValues` now yields the pair $(1, 1)$. Its measure argument is as follows. Put $g(z) = f(c(z), z) - f(0)$. The disk-minimum property converts nearby lower values into negative values of g arbitrarily close to zero. For each fixed sufficiently small $\delta > 0$, choose such a z_0 and $a_0 = -g(z_0) > 0$ small enough for the inner planar disk. Continuity gives a fixed tail ball $B(z_0, \nu)$ on which $a_0/2 < -g(z) < 3a_0/2$, the centers and tails are small, and translated disks of radius $\delta/4$ lie inside the ambient δ-ball. This tail ball is chosen before ϵ.

The radial secants bound the planar signed-band areas above by $4\pi\epsilon/k$ and, on that fixed tail ball for $0 < \epsilon < a_0/4$, below by $4\pi\epsilon/K$. These constants arise because a squared-radius interval has planar area π times its length. Tonelli, implemented by `Measure.prod_apply_symm`, multiplies the lower bound by the strictly positive finite tail-ball volume; the upper bound integrates over the radius-δ tail ball. Positivity is proved on the open balls and finiteness on containing compact closed balls before any `ENNReal.toReal` conversion. A zero-dimensional tail has mass one. Finally shrink ϵ_0 below both $a_0/4$ and e^{-1} and reduce the lower constant if necessary to ensure $c_\delta \le C_\delta$. This is the ambient-volume proof, not an argument that assigns ambient mass to the positive two-plane alone.

The polynomial example following the theorem checks the secant bounds for $h(t) = t^2 + t^4$ on $[0, 1]$: its second derivative lies in $[2, 14]$, so the quadratic secant constants are 1 and 7.

```
/-- Reuse the checked full-dimensional theorem, not a new Fubini proof. -/
theorem ambientPair_of_uniformHessian {d : ℕ}
    (f : Plane × Tail d → ℝ) (hf : Continuous f) (D : UniformPartialHessian f)
    (hlower : LowerValuesAccumulate f) : HasAmbientBandPair f 1 1 := by
  obtain ⟨C⟩ := centeredFamily_of_uniformHessian f hf D
  exact ambientPair_of_lowerValues f hf C hlower

-- A nonconstant-Hessian regression on [0,1].
example {a b : ℝ} (ha : a ∈ Icc (0 : ℝ) 1) (hb : b ∈ Icc (0 : ℝ) 1)
    (hab : a ≤ b) :
    b ^ 2 - a ^ 2 ≤ (b ^ 2 + b ^ 4) - (a ^ 2 + a ^ 4) ∧
      (b ^ 2 + b ^ 4) - (a ^ 2 + a ^ 4) ≤ 7 * (b ^ 2 - a ^ 2) := by
  have hsec := quadratic_secants (h := fun t : ℝ => t ^ 2 + t ^ 4)
    (d := fun t => 2 * t + 4 * t ^ 3) (e := fun t => 2 + 12 * t ^ 2)
    (R := 1) (k := 2) (K := 14) zero_le_one
    (fun t _ => by
      have hpoly : HasDerivAt (fun t : ℝ => t ^ 2 + t ^ 4)
          (2 * t + 4 * t ^ 3) t := by
        exact ((hasDerivAt_pow 2 t).add (hasDerivAt_pow 4 t)).congr_deriv
          (by norm_num)
      exact hpoly)
    (fun t _ => by
      have hlinear : HasDerivAt (fun t : ℝ => 2 * t) 2 t := by
        exact ((hasDerivAt_id' t).const_mul 2).congr_deriv (by norm_num)
      have hcubic : HasDerivAt (fun t : ℝ => t ^ 3) (3 * t ^ 2) t := by
        exact (hasDerivAt_pow 3 t).congr_deriv (by norm_num)
      exact (hlinear.add (hcubic.const_mul 4)).congr_deriv (by ring))
    (by simp) (fun t ht => by
      have ht2 : t ^ 2 ≤ 1 := by
        nlinarith [ht.1, ht.2, mul_nonneg ht.1 (sub_nonneg.mpr ht.2)]
      constructor <;> nlinarith [sq_nonneg t]) ha hb hab
  simpa only [show (2 : ℝ) / 2 = 1 by norm_num,
    show (14 : ℝ) / 2 = 7 by norm_num, one_mul] using hsec

end O77.SaddlePreparation0906
end
```

41 Module `O77MatrixFrames`

The common notation in this module is rectangular synthesis $C \mapsto UCV^{\mathsf{T}}$. The columns of U and V are orthonormal when $U^{\mathsf{T}}U = 1$ and $V^{\mathsf{T}}V = 1$. All measures and norms used elsewhere retain their original definitions: the identities below concern entry-square sums and ranks, and introduce no new inner-product instance. Arbitrary finite index types include empty shapes.

```
import Mathlib
import O77

set_option autoImplicit false
noncomputable section
open scoped BigOperators Matrix

namespace O77.MatrixFrames
open O77.Residual (frobeniusSq)
```

41.1 Trace and rectangular compression

Expanding the diagonal of $M^{\mathsf{T}}N$ and interchanging two finite sums proves $\operatorname{tr}(M^{\mathsf{T}}N) = \sum_{i,j} M_{ij}N_{ij}$. Taking $N = M$ identifies this trace with the actual Frobenius square. For a row family V whose dot products are Kronecker deltas, entrywise matrix multiplication gives $VV^{\mathsf{T}} = 1$. These three bridges let later proofs use the existing trace and matrix APIs without silently switching norm conventions.

```
/-- Entry pairing expressed through Mathlib's trace, without a second inner-product instance. -/
lemma trace_transpose_mul_eq_sum {m n : Type*} [Fintype m] [Fintype n]
    (M N : Matrix m n ℝ) :
    Matrix.trace (M.transpose * N) = ∑ i, ∑ j, M i j * N i j := by
  simp only [Matrix.trace, Matrix.diag_apply, Matrix.mul_apply, Matrix.transpose_apply]
  exact Finset.sum_comm

/-- A bridge to the existing trace API; no matrix norm instance is changed. -/
lemma frobeniusSq_eq_trace {m n : Type*} [Fintype m] [Fintype n]
    (M : Matrix m n ℝ) : frobeniusSq M = Matrix.trace (M.transpose * M) := by
  rw [trace_transpose_mul_eq_sum]
  simp only [frobeniusSq, pow_two]


/-- An orthonormal row family is a rectangular coisometry. -/
lemma gram_of_rows {r n : Type*} [Fintype n] [DecidableEq r]
    (V : Matrix r n ℝ)
    (hV : ∀ i j, dotProduct (V i) (V j) = if i = j then 1 else 0) :
    V * V.transpose = 1 := by
  ext i j
  simpa only [Matrix.mul_apply, Matrix.transpose_apply, Matrix.one_apply,
    dotProduct] using hV i j

section Frames
variable {m n r s : Type*}
variable [Fintype m] [Fintype n] [Fintype r] [Fintype s]
variable [DecidableEq r] [DecidableEq s]
```

If $U^{\mathsf{T}}U = 1$ and $V^{\mathsf{T}}V = 1$, associativity gives $U^{\mathsf{T}}(UCV^{\mathsf{T}})V = C$. This compression is a left inverse to synthesis. Consequently $\operatorname{rank}(UCV^{\mathsf{T}})$ equals $\operatorname{rank} C$: multiplication bounds rank from above, and compression gives the reverse inequality. The proof invokes `Matrix.rank_mul_le_left` and `Matrix.rank_mul_le_right` in the corresponding directions.

```
/-- Compression is a left inverse to rectangular synthesis. -/
lemma compress_sandwich (U : Matrix m r ℝ) (V : Matrix n s ℝ)
    (hU : U.transpose * U = 1) (hV : V.transpose * V = 1)
    (C : Matrix r s ℝ) :
    U.transpose * (U * C * V.transpose) * V = C := by
  calc
    U.transpose * (U * C * V.transpose) * V =
        (U.transpose * U) * C * (V.transpose * V) := by
          simp only [Matrix.mul_assoc]
    _ = C := by rw [hU, hV, Matrix.one_mul, Matrix.mul_one]

/-- Rectangular synthesis preserves the actual rank, including empty shapes. -/
lemma rank_sandwich (U : Matrix m r ℝ) (V : Matrix n s ℝ)
    (hU : U.transpose * U = 1) (hV : V.transpose * V = 1)
    (C : Matrix r s ℝ) : (U * C * V.transpose).rank = C.rank := by
  apply le_antisymm
  · exact (Matrix.rank_mul_le_left _ _).trans (Matrix.rank_mul_le_right _ _)
  · calc
      C.rank = (U.transpose * (U * C * V.transpose) * V).rank :=
        congrArg Matrix.rank (compress_sandwich U V hU hV C).symm
      _ ≤ (U.transpose * (U * C * V.transpose)).rank := Matrix.rank_mul_le_left _ _
```

```
      _ ≤ (U * C * V.transpose).rank := Matrix.rank_mul_le_right _ _
```

For the same frames,

$$(UCV^{\mathsf{T}})^{\mathsf{T}}(UCV^{\mathsf{T}}) = VC^{\mathsf{T}}CV^{\mathsf{T}}.$$

The rectangular cyclic trace identity `Matrix.trace_mul_cycle` contracts $V^{\mathsf{T}}V$ and proves $\|UCV^{\mathsf{T}}\|_F^2 = \|C\|_F^2$. This is an isometry onto a subspace, not an assertion about ambient volumes in different dimensions.

```
/-- The two-sided embedding is an isometry for entry-square norms. -/
lemma frobeniusSq_sandwich (U : Matrix m r ℝ) (V : Matrix n s ℝ)
    (hU : U.transpose * U = 1) (hV : V.transpose * V = 1)
    (C : Matrix r s ℝ) : frobeniusSq (U * C * V.transpose) = frobeniusSq C := by
  rw [frobeniusSq_eq_trace, frobeniusSq_eq_trace]
  have hgram : (U * C * V.transpose).transpose * (U * C * V.transpose) =
      V * (C.transpose * C) * V.transpose := by
    simp only [Matrix.transpose_mul, Matrix.transpose_transpose]
    calc
      V * (C.transpose * U.transpose) * (U * C * V.transpose) =
          V * C.transpose * (U.transpose * U) * C * V.transpose := by
            simp only [Matrix.mul_assoc]
      _ = V * (C.transpose * C) * V.transpose := by
        rw [hU, Matrix.mul_one]
        simp only [Matrix.mul_assoc]
  rw [hgram, Matrix.trace_mul_cycle, hV, Matrix.one_mul]
end Frames
```

Composing through a shared hidden frame. If $E^{\mathsf{T}}E = 1$, then $(UCE^{\mathsf{T}})(EDV^{\mathsf{T}}) = U(CD)V^{\mathsf{T}}$. Only the hidden frame E needs orthonormal columns; U and V are arbitrary. Reassociation places $E^{\mathsf{T}}E$ between C and D, where it cancels. This is the common product calculation for rung witnesses.

```
/-- Composition through an orthonormal hidden frame contracts that frame.
No orthogonality assumptions on the outer matrices are needed. -/
lemma sandwich_mul {m n h r s t : Type*}
    [Fintype h] [Fintype r] [Fintype s] [Fintype t] [DecidableEq s]
    (U : Matrix m r ℝ) (E : Matrix h s ℝ) (V : Matrix n t ℝ)
    (hE : E.transpose * E = 1) (C : Matrix r s ℝ) (D : Matrix s t ℝ) :
    (U * C * E.transpose) * (E * D * V.transpose) = U * (C * D) * V.transpose := by
  calc
    (U * C * E.transpose) * (E * D * V.transpose) =
        U * C * (E.transpose * E) * D * V.transpose := by
          simp only [Matrix.mul_assoc]
    _ = U * (C * D) * V.transpose := by
      rw [hE, Matrix.mul_one]
      simp only [Matrix.mul_assoc]
```

41.2 Weighted singular expansions

For arbitrary row families u_j, v_j and real coefficients c_j, entrywise expansion gives

$$\sum_j c_j u_j v_j^{\mathsf{T}} = u^{\mathsf{T}} \operatorname{diag}(c) v.$$

There is no orthogonality or sign restriction in this identity. If V has orthonormal columns, then

$$(U \operatorname{diag}(c) V^{\mathsf{T}})V_j = c_j U_j.$$

Indeed $V^{\mathsf{T}}V_j$ is the jth coordinate vector, so the diagonal matrix multiplies it by c_j. Only the right frame is needed for this contraction; this avoids redundant hypotheses in all later clients.

```
/-- Weighted rank-one expansion as a diagonal coefficient matrix. The row
families need not be orthonormal, and the coefficients may have either sign. -/
lemma weighted_outer_eq {m n r : Type*} [Fintype r] [DecidableEq r]
    (u : Matrix r m ℝ) (v : Matrix r n ℝ) (c : r → ℝ) :
    ∑( j, c j • Matrix.vecMulVec (u j) (v j)) =
      u.transpose * Matrix.diagonal c * v := by
  ext i j
  rw [Matrix.sum_apply, Matrix.mul_apply]
  simp only [Matrix.smul_apply, Matrix.vecMulVec_apply, smul_eq_mul,
    Matrix.mul_diagonal, Matrix.transpose_apply]
  apply Finset.sum_congr rfl
  intro a _
  ring

/-- All spectral-vector contractions use this one lemma. Only the right frame
must be orthonormal; no order, positivity, target, loss or left Gram data enter. -/
lemma diagonal_sandwich_mulVec {m n r : Type*}
    [Fintype n] [Fintype r] [DecidableEq r]
    (U : Matrix m r ℝ) (V : Matrix n r ℝ)
    (hV : V.transpose * V = 1) (c : r → ℝ) (j : r) :
    (U * Matrix.diagonal c * V.transpose).mulVec (V.col j) = c j • U.col j := by
  have hc : V.transpose.mulVec (V.col j) = Pi.single j 1 := by
    funext i
    have h := congrFun (congrFun hV i) j
    simpa [Matrix.mul_apply, Matrix.mulVec, Matrix.col, Matrix.transpose_apply,
      dotProduct, Matrix.one_apply, Pi.single_apply, eq_comm] using h
  rw ←[ Matrix.mulVec_mulVec, hc, Matrix.mulVec_single_one]
  funext i
  simp [Matrix.col, Matrix.mul_diagonal, mul_comm]
```

If both row families are orthonormal, the rectangular norm identity reduces the Frobenius square of the weighted expansion to that of its diagonal coefficient matrix, namely $\sum_j c_j^2$. In particular this applies to negative coefficients in a residual singular expansion.

```
/-- This is the entry-square norm of the diagonal coefficient vector, derived
from the shared rectangular isometry rather than a second orthogonality sum. -/
lemma weighted_outer_frobeniusSq {m n r : Type*}
    [Fintype m] [Fintype n] [Fintype r] [DecidableEq r]
    (u : Matrix r m ℝ) (v : Matrix r n ℝ)
    (hu : u * u.transpose = 1) (hv : v * v.transpose = 1) (c : r → ℝ) :
    frobeniusSq (u.transpose * Matrix.diagonal c * v) = ∑ j, (c j)^2 := by
  have h := frobeniusSq_sandwich u.transpose v.transpose
    (by simpa using hu) (by simpa using hv) (Matrix.diagonal c)
  simpa [frobeniusSq, Matrix.diagonal_apply, eq_comm] using h
```

Annihilation depends only on the support. If for every j either $c_j = 0$ or $Bu_j = 0$, then $B\sum_j c_j u_j v_j^{\mathsf{T}} = 0$. Distribute the product over the finite sum and scalar coefficients, and use `Matrix.mul_vecMulVec`. Zero coefficients impose no unnecessary condition on Bu_j. Transposition supplies the right-hand critical equation from the same lemma.

```
/-- A single support-annihilation lemma handles left and, by transpose, right
critical equations. Zero coefficients impose no annihilation requirement. -/
lemma mul_weighted_outer_eq_zero {m n r h : Type*} [Fintype m] [Fintype r]
    (B : Matrix h m ℝ) (u : Matrix r m ℝ) (v : Matrix r n ℝ) (c : r → ℝ)
    (hzero : ∀ j, c j = 0 ∨ B.mulVec (u j) = 0) :
    B * ∑( j, c j • Matrix.vecMulVec (u j) (v j)) = 0 := by
  rw [Matrix.mul_sum]
  apply Finset.sum_eq_zero
  intro j _
  rcases hzero j with hc | hu
  · simp [hc]
```

```
  · rw [Matrix.mul_smul, Matrix.mul_vecMulVec, hu, Matrix.zero_vecMulVec, smul_zero]
```

Counting retained diagonal entries. If each $c_j \neq 0$ and $k \leq r$, the diagonal matrix with entries c_j for $j < k$ and zero otherwise has rank k. By `Matrix.rank_diagonal` its rank is the cardinality of the nonzero-entry subtype. The displayed equivalence maps that subtype to `Fink` by retaining the value of the index; its inverse uses $k \leq r$. This includes $k = 0$ and $r = 0$.

```
/-- The only cutoff-cardinality argument: reused for target and both factors. -/
lemma rank_cutoff_diagonal {r k : ℕ} (c : Fin r → ℝ)
    (hc : ∀ j, c j ≠ 0) (hk : k ≤ r) :
    (Matrix.diagonal (fun j : Fin r => if j.val < k then c j else 0)).rank = k := by
  classical
  let a : Fin r → ℝ := fun j => if j.val < k then c j else 0
  have ha (j : Fin r) : a j ≠ 0 ↔ j.val < k := by
    by_cases hj : j.val < k <;> simp [a, hj, hc j]
  let e : {j : Fin r // a j ≠ 0} ≃ Fin k := {
    toFun := fun j => ⟨j.1.val, (ha j.1).mp j⟩.2
    invFun := fun j => ⟨⟨j.val, j.isLt.trans_le ‹hk›, (ha _).mpr j.⟩isLt
    left_inv := by intro j; apply Subtype.ext; apply Fin.ext; rfl
    right_inv := by intro j; apply Fin.ext; rfl }
  change (Matrix.diagonal a).rank = k
  rw [Matrix.rank_diagonal, Fintype.card_congr e, Fintype.card_fin]

end O77.MatrixFrames
end
```

42 Module `O77RungGeometry`

Write $w = (A, B)$, with $A \in \mathbb{R}^{H \times I}$ and $B \in \mathbb{R}^{O \times H}$, and $L_\Phi(w) = \|BA - \Phi\|_F^2/2$. This module starts from an arbitrary point satisfying the actual rung equations and constructs two geometric facts there: an entry-orthonormal positive plane for the genuine Hessian, and lower-loss points in every ambient ball. Neither a balanced representative nor a local volume conclusion is assumed. In this module $\langle x, y\rangle$ is the entrywise dot product and $q(x) = \sum_i x_i^2$.

```
import Mathlib
import O77Canonical
import O77MatrixFrames

set_option autoImplicit false
noncomputable section
open Set Filter MeasureTheory
open scoped BigOperators Topology ContDiff Matrix.Norms.Elementwise
namespace O77.RungGeometry0906
open O77.Reconstruction0905 (RM)
open O77.Residual (frobeniusSq)
open O77.Native (Parameters loss entryNorm entryBall)
```

42.1 The singular data and the rung predicate

`vectorSq` is $q(x)$, not the square of the default supremum norm on a function type. Its equality with $\langle x, x\rangle$, nonnegativity, strict positivity when $x \neq 0$, and homogeneity $q(tx) = t^2 q(x)$ follow from the finite sums. Strict positivity selects a nonzero entry and uses its positive square.

```
/-- The entry-square convention, not the default supremum norm on vectors. -/
def vectorSq {n : ℕ} (x : Fin n → ℝ) : ℝ := ∑ i, (x i)^2

private lemma vectorSq_dot {n : ℕ} (x : Fin n → ℝ) :
```

```
    vectorSq x = dotProduct x x := by simp [vectorSq, dotProduct, pow_two]

private lemma vectorSq_nonneg {n : ℕ} (x : Fin n → ℝ) : 0 ≤ vectorSq x :=
  Finset.sum_nonneg (fun _ _ => sq_nonneg _)

private lemma vectorSq_pos {n : ℕ} {x : Fin n → ℝ} (hx : x ≠ 0) :
    0 < vectorSq x := by
  obtain ⟨i, ⟩hi := Function.ne_iff.mp hx
  exact Finset.sum_pos' (fun j _ => sq_nonneg (x j))
    ⟨i, Finset.mem_univ _, sq_pos_of_ne_zero ⟩hi

private lemma vectorSq_smul {n : ℕ} (t : ℝ) (x : Fin n → ℝ) :
    vectorSq (t • x) = t^2 * vectorSq x := by
  simp [vectorSq, mul_pow, Finset.mul_sum]
```

`SingularDataIOr` contains positive coefficients $\sigma_0 > \cdots > \sigma_{r-1}$ and two orthonormal families $u_j \in \mathbb{R}^O$, $v_j \in \mathbb{R}^I$. Its fields contain no assertion about the loss or its derivatives. Define

$$\Phi_k = \sum_{j<k} \sigma_j u_j v_j^{\mathsf{T}}, \qquad \Phi = \Phi_r.$$

The mathematical singular index $j+1$ is Lean's zero-based index j. The predicate `OnRungkw` is exactly

$$BA = \Phi_k, \qquad Av_j = 0, \quad B^{\mathsf{T}} u_j = 0 \quad (k \le j < r).$$

The annihilation conditions are indispensable. The final saddle statement is conditional on supplied distinct singular data of this kind; it does not claim a repeated-singular-value extension. The fiber statement does not have this restriction.

```
/-- Supplied singular witnesses; no loss, Hessian or volume assertion is a field. -/
structure SingularData (I O r : ℕ) where
  sigma : Fin r → ℝ
  sigma_pos : ∀ j, 0 < sigma j
  decreasing : StrictAnti sigma
  u : Fin r → Fin O → ℝ
  v : Fin r → Fin I → ℝ
  orth_u : ∀ i j, dotProduct (u i) (u j) = if i = j then 1 else 0
  orth_v : ∀ i j, dotProduct (v i) (v j) = if i = j then 1 else 0

namespace SingularData
variable {I O r : ℕ} (D : SingularData I O r)

/-- Index j represents the source's singular mode j+1. -/
def «prefix» (k : ℕ) : RM O I :=
  ∑ j : Fin r, if j.val < k then D.sigma j • Matrix.vecMulVec (D.u j) (D.v j) else 0

def target : RM O I := D.«prefix» r

/-- Includes both annihilation conditions, not only a product equation. -/
def OnRung {H : ℕ} (k : ℕ) (w : Parameters I O H) : Prop :=
  w.2 * w.1 = D.«prefix» k ∧ ∀ j : Fin r, k ≤ j.val →
    w.1.mulVec (D.v j) = 0 ∧ w.2.transpose.mulVec (D.u j) = 0
```

One coefficient vector for all contractions. Set $c_j^{(k)} = \sigma_j$ for $j < k$ and 0 otherwise. With u, v regarded as matrices of rows, orthonormality gives $uu^{\mathsf{T}} = vv^{\mathsf{T}} = 1$. The frame expansion is $\Phi_k = u^{\mathsf{T}} \operatorname{diag}(c^{(k)}) v$. Multiplying by v_j, or transposing and multiplying by u_j, gives $\sigma_j u_j$, respectively $\sigma_j v_j$, for retained indices and zero for discarded indices. Each proof instantiates the preceding common frame lemmas. Rank multiplication bounds give $\operatorname{rank} \Phi_k \le r$, so at any

rung point $r < H$ implies $\operatorname{rank}(BA) < H$. The exact equality with k is supplied later when $k \le r$.

```
/-- The diagonal coefficients of a prefix, isolated once for all contractions,
rank computations and loss-level calculations. -/
def prefixCoefficients (k : ℕ) (j : Fin r) : ℝ :=
  if j.val < k then D.sigma j else 0

lemma gram_u : (Matrix.of D.u) * (Matrix.of D.u).transpose = 1 :=
  O77.MatrixFrames.gram_of_rows (Matrix.of D.u) D.orth_u

lemma gram_v : (Matrix.of D.v) * (Matrix.of D.v).transpose = 1 :=
  O77.MatrixFrames.gram_of_rows (Matrix.of D.v) D.orth_v

lemma prefix_eq_sandwich (k : ℕ) :
    D.«prefix» k = (Matrix.of D.u).transpose * Matrix.diagonal (D.prefixCoefficients k) * (Matrix.of D
     .v) := by
  calc
    D.«prefix» k =
        ∑ j : Fin r, D.prefixCoefficients k j • Matrix.vecMulVec (D.u j) (D.v j) := by
      change ∑( j : Fin r, if j.val < k then
        D.sigma j • Matrix.vecMulVec (D.u j) (D.v j) else 0) = _
      apply Finset.sum_congr rfl
      intro j _
      by_cases hj : j.val < k <;> simp [prefixCoefficients, hj]
    _ = _ := O77.MatrixFrames.weighted_outer_eq
      (Matrix.of D.u) (Matrix.of D.v) (D.prefixCoefficients k)

lemma prefix_mul_v (k : ℕ) (j : Fin r) :
    (D.«prefix» k).mulVec (D.v j) =
      if j.val < k then D.sigma j • D.u j else 0 := by
  rw [D.prefix_eq_sandwich]
  simpa only [Matrix.transpose_transpose, Matrix.of_col, prefixCoefficients,
    ite_smul, zero_smul] using
    O77.MatrixFrames.diagonal_sandwich_mulVec (Matrix.of D.u).transpose (Matrix.of D.v).transpose
      (by simpa only [Matrix.transpose_transpose] using D.gram_v)
      (D.prefixCoefficients k) j

lemma prefix_transpose_mul_u (k : ℕ) (j : Fin r) :
    (D.«prefix» k).transpose.mulVec (D.u j) =
      if j.val < k then D.sigma j • D.v j else 0 := by
  rw [D.prefix_eq_sandwich]
  simp only [Matrix.transpose_mul, Matrix.transpose_transpose, Matrix.diagonal_transpose]
  rw ←[ Matrix.mul_assoc]
  simpa only [Matrix.transpose_transpose, Matrix.of_col, prefixCoefficients,
    ite_smul, zero_smul] using
    O77.MatrixFrames.diagonal_sandwich_mulVec (Matrix.of D.v).transpose (Matrix.of D.u).transpose
      (by simpa only [Matrix.transpose_transpose] using D.gram_u)
      (D.prefixCoefficients k) j

/-- No exact-rank or nonzero-coefficient hypothesis is needed for this bound. -/
lemma prefix_rank_le (k : ℕ) : (D.«prefix» k).rank ≤ r := by
  rw [D.prefix_eq_sandwich]
  exact (Matrix.rank_mul_le_left _ _).trans (Matrix.rank_le_width _)

lemma rung_rank_lt {H k : ℕ} {w : Parameters I O H}
    (hH : r < H) (hw : D.OnRung k w) : (w.2 * w.1).rank < H := by
  rw [hw.1]
  exact (D.prefix_rank_le k).trans_lt hH
end SingularData
```

What one discarded mode supplies. Fix dimensions $I, O, H \in \mathbb{N}$, a matrix $\Phi \in \mathbb{R}^{O\times I}$, and a parameter $w = (A, B)$ with $A \in \mathbb{R}^{H\times I}$ and $B \in \mathbb{R}^{O\times H}$. A `DiscardedModePhiw` records a real number $\sigma > 0$, unit vectors $u \in \mathbb{R}^O$ and $v \in \mathbb{R}^I$, and the four identities

$$Av = 0, \quad B^{\mathsf{T}}u = 0, \quad (BA - \Phi)v = -\sigma u, \quad (BA - \Phi)^{\mathsf{T}}u = -\sigma v.$$

At a point of the kth nonoptimal rung, choose the actual index $j = k$; the preceding contractions and the rung equations prove all these fields. This construction does not require $r < H$; that inequality is used for the later rank-defect and two-plane arguments.

```
/-- Exact identities for one discarded mode. These will be constructed from OnRung. -/
structure DiscardedMode {I O H : ℕ} (Phi : RM O I) (w : Parameters I O H) where
  sigma : ℝ
  sigma_pos : 0 < sigma
  u : Fin O → ℝ
  v : Fin I → ℝ
  unit_u : dotProduct u u = 1
  unit_v : dotProduct v v = 1
  kill_v : w.1.mulVec v = 0
  kill_u : w.2.transpose.mulVec u = 0
  error_v : (w.2 * w.1 - Phi).mulVec v = (-sigma) • u
  error_u : (w.2 * w.1 - Phi).transpose.mulVec u = (-sigma) • v

/-- Choosing j=k in zero-based Fin r gives an actual discarded mode at every rung point. -/
def discardedMode_of_rung {I O H r k : ℕ} (D : SingularData I O r)
    (hkr : k < r) (w : Parameters I O H) (hw : D.OnRung k w) :
    DiscardedMode D.target w := by
  let j : Fin r := ⟨k, hkr⟩
  have hj : ¬ j.val < k := by simp [j]
  have hunitu : dotProduct (D.u j) (D.u j) = 1 := by simpa using D.orth_u j j
  have hunitv : dotProduct (D.v j) (D.v j) = 1 := by simpa using D.orth_v j j
  refine ⟨D.sigma j, D.sigma_pos j, D.u j, D.v j, hunitu, hunitv,
    (hw.2 j le_rfl).1, (hw.2 j le_rfl).2, ?_, ?⟩_
  · rw [hw.1, Matrix.sub_mulVec, D.prefix_mul_v,
      SingularData.target, D.prefix_mul_v]
    simp [hj, j.isLt]
  · rw [hw.1, Matrix.transpose_sub, Matrix.sub_mulVec,
      D.prefix_transpose_mul_u, SingularData.target, D.prefix_transpose_mul_u]
    simp [hj, j.isLt]
```

42.2 Expanding the loss in the original entries

For any $m, n \in \mathbb{N}$ and matrices $M, N \in \mathbb{R}^{m\times n}$, the private pairing is $\langle M, N\rangle_F = \sum_{i=1}^{m}\sum_{j=1}^{n} M_{ij}N_{ij}$. In the identities below take $x, x' \in \mathbb{R}^m$ and $y, y' \in \mathbb{R}^n$; write $q(v) = \sum_i v_i^2$ for a vector of any finite dimension. Distributing finite sums proves the pairing's symmetry and linearity, its equality with the Frobenius square on the diagonal, and

$$\langle M, xy^{\mathsf{T}}\rangle_F = \langle x, My\rangle, \qquad \langle xy^{\mathsf{T}}, x'y'^{\mathsf{T}}\rangle_F = \langle x, x'\rangle\langle y, y'\rangle.$$

Thus $\|xy^{\mathsf{T}}\|_F^2 = q(x)q(y)$. Moving a matrix across a dot product transposes it. The four-term-square lemma then lists all four diagonal squares and twice all six cross-pairings; it is ordinary bilinear expansion, not an asymptotic estimate.

```
/-- Only bridges for the inherited sum-of-squares representation are introduced here. -/
private def pairing {m n : ℕ} (M N : RM m n) : ℝ := ∑ i, ∑ j, M i j * N i j

private lemma pairing_self {m n : ℕ} (M : RM m n) : pairing M M = frobeniusSq M := by
  simp [pairing, frobeniusSq, pow_two]

private lemma pairing_comm {m n : ℕ} (M N : RM m n) : pairing M N = pairing N M := by
```

```
  simp [pairing, mul_comm]

private lemma pairing_add_left {m n : ℕ} (M N P : RM m n) :
    pairing (M + N) P = pairing M P + pairing N P := by
  simp [pairing, add_mul, Finset.sum_add_distrib]

private lemma pairing_add_right {m n : ℕ} (M N P : RM m n) :
    pairing M (N + P) = pairing M N + pairing M P := by
  simp [pairing, mul_add, Finset.sum_add_distrib]

private lemma pairing_smul_left {m n : ℕ} (t : ℝ) (M N : RM m n) :
    pairing (t • M) N = t * pairing M N := by
  simp [pairing, Finset.mul_sum, mul_assoc]

private lemma pairing_smul_right {m n : ℕ} (t : ℝ) (M N : RM m n) :
    pairing M (t • N) = t * pairing M N := by
  simp [pairing, Finset.mul_sum, mul_left_comm]

private lemma pairing_outer {m n : ℕ} (M : RM m n) (x : Fin m → ℝ) (y : Fin n → ℝ) :
    pairing M (Matrix.vecMulVec x y) = dotProduct x (M.mulVec y) := by
  simp only [pairing, Matrix.vecMulVec_apply, Matrix.mulVec, dotProduct,
    Finset.mul_sum]
  apply Finset.sum_congr rfl
  intro i _
  apply Finset.sum_congr rfl
  intro j _
  ring

private lemma pairing_outer_outer {m n : ℕ}
    (x x' : Fin m → ℝ) (y y' : Fin n → ℝ) :
    pairing (Matrix.vecMulVec x y) (Matrix.vecMulVec x' y') =
      dotProduct x x' * dotProduct y y' := by
  change ∑ i, ∑ j, (x i * y j) * (x' i * y' j)) =
    ∑ i, x i * x' i) * ∑ j, y j * y' j)
  calc
    _ = ∑ i, (x i * x' i) * ∑ j, y j * y' j) := by
      apply Finset.sum_congr rfl
      intro i _
      rw [Finset.mul_sum]
      apply Finset.sum_congr rfl
      intro j _
      ring
    _ = _ := by rw [Finset.sum_mul]

private lemma outer_sq {m n : ℕ} (x : Fin m → ℝ) (y : Fin n → ℝ) :
    frobeniusSq (Matrix.vecMulVec x y) = vectorSq x * vectorSq y := by
  rw ←[ pairing_self, pairing_outer_outer, vectorSq_dot, vectorSq_dot]

private lemma dot_mulVec_transpose {m n : ℕ} (M : RM m n)
    (x : Fin m → ℝ) (y : Fin n → ℝ) :
    dotProduct x (M.mulVec y) = dotProduct (M.transpose.mulVec x) y := by
  rw [Matrix.dotProduct_mulVec, O77.Residual.transpose_mulVec_eq_vecMul]

private lemma four_term_square {m n : ℕ} (E X Y R : RM m n) (c : ℝ) :
    frobeniusSq (E + X + Y + c • R) =
      frobeniusSq E + frobeniusSq X + frobeniusSq Y + c^2 * frobeniusSq R +
        2 * (pairing E X + pairing E Y + c * pairing E R +
          pairing X Y + c * pairing X R + c * pairing Y R) := by
  calc
    frobeniusSq (E + X + Y + c • R) =
        pairing (E + X + Y + c • R) (E + X + Y + c • R) :=
      (pairing_self _).symm
```

```
    _ = _ := by
      simp only [pairing_add_left, pairing_add_right, pairing_smul_left,
        pairing_smul_right]
      rw [pairing_self E, pairing_self X, pairing_self Y, pairing_self R,
        pairing_comm X E, pairing_comm Y E, pairing_comm R E,
        pairing_comm Y X, pairing_comm R X, pairing_comm R Y]
      ring

variable {I O H : ℕ} {Phi : RM O I} {w : Parameters I O H}
```

For hidden vectors $x, y \in \mathbb{R}^H$ define $D(x,y) = (xv^\mathsf{T}, uy^\mathsf{T})$. The map is jointly homogeneous, and unit length of u, v gives the exact ambient entry-square identity $\|D(x,y)\|^2_{\mathrm{ent}} = q(x) + q(y)$.

```
/-- The actual two-factor perturbation, linear in its two hidden vectors. -/
def modeDelta (M : DiscardedMode Phi w) (x y : Fin H → ℝ) : Parameters I O H :=
  (Matrix.vecMulVec x M.v, Matrix.vecMulVec M.u y)

lemma modeDelta_smul (M : DiscardedMode Phi w) (t : ℝ) (x y : Fin H → ℝ) :
    modeDelta M (t • x) (t • y) = t • modeDelta M x y := by
  ext <;> simp [modeDelta]

lemma modeDelta_sq (M : DiscardedMode Phi w) (x y : Fin H → ℝ) :
    O77.Native.ambientSq (modeDelta M x y) = vectorSq x + vectorSq y := by
  simp only [O77.Native.ambientSq, modeDelta, outer_sq, vectorSq_dot,
    M.unit_u, M.unit_v, mul_one, one_mul]
```

Put $E = BA - \Phi$, $b = Bx$, $a = A^\mathsf{T}y$, and $X = bv^\mathsf{T}$, $Y = ua^\mathsf{T}$, $R = uv^\mathsf{T}$. The perturbed error is exactly $E + X + Y + \langle x, y\rangle R$. The annihilation identities imply $\langle b, u\rangle = \langle v, a\rangle = 0$. Consequently all cross-pairings in its square vanish except $\langle E, R\rangle_F = -\sigma$. The three new diagonal squares are $q(b)$, $q(a)$, and 1. Subtracting $\|E\|_F^2$ and dividing by two proves

$$L_\Phi(w + D(x,y)) - L_\Phi(w) = \tfrac{1}{2}(q(Bx) + q(A^\mathsf{T}y)) - \sigma\langle x, y\rangle + \tfrac{1}{2}\langle x, y\rangle^2.$$

Every cancellation is exposed separately in the code. Matrix products retain their order; scalar ring normalization is used only after the matrix identity and Frobenius pairings are established.

```
/-- Exact finite polynomial identity, not a quadratic approximation. -/
theorem discarded_loss_expansion (M : DiscardedMode Phi w) (x y : Fin H → ℝ) :
    loss Phi (w + modeDelta M x y) - loss Phi w =
      (vectorSq (w.2.mulVec x) + vectorSq (w.1.transpose.mulVec y)) / 2 -
        M.sigma * dotProduct x y + (dotProduct x y)^2 / 2 := by
  let E := w.2 * w.1 - Phi
  let b := w.2.mulVec x
  let a := w.1.transpose.mulVec y
  let X := Matrix.vecMulVec b M.v
  let Y := Matrix.vecMulVec M.u a
  let R := Matrix.vecMulVec M.u M.v
  have hbu : dotProduct b M.u = 0 := by
    rw [dotProduct_comm]
    change dotProduct M.u (w.2.mulVec x) = 0
    rw [dot_mulVec_transpose, M.kill_u, zero_dotProduct]
  have hva : dotProduct M.v a = 0 := by
    change dotProduct M.v (w.1.transpose.mulVec y) = 0
    rw [dot_mulVec_transpose, Matrix.transpose_transpose, M.kill_v, zero_dotProduct]
  have hex : pairing E X = 0 := by
    change pairing (w.2 * w.1 - Phi) (Matrix.vecMulVec b M.v) = 0
    rw [pairing_outer, M.error_v]
    simp [dotProduct_smul, smul_eq_mul, hbu]
  have hey : pairing E Y = 0 := by
    change pairing (w.2 * w.1 - Phi) (Matrix.vecMulVec M.u a) = 0
    rw [pairing_outer, dot_mulVec_transpose, M.error_u]
    simp [smul_dotProduct, smul_eq_mul, hva]
```

```
  have her : pairing E R = -M.sigma := by
    change pairing (w.2 * w.1 - Phi) (Matrix.vecMulVec M.u M.v) = _
    rw [pairing_outer, M.error_v]
    simp [dotProduct_smul, smul_eq_mul, M.unit_u]
  have hxy : pairing X Y = 0 := by
    simp [X, Y, pairing_outer_outer, hbu]
  have hxr : pairing X R = 0 := by
    simp [X, R, pairing_outer_outer, hbu]
  have hyr : pairing Y R = 0 := by
    simp [Y, R, pairing_outer_outer, dotProduct_comm a M.v, hva]
  have herror : (w + modeDelta M x y).2 * (w + modeDelta M x y).1 - Phi =
      E + X + Y + dotProduct x y • R := by
    change (w.2 + Matrix.vecMulVec M.u y) * (w.1 + Matrix.vecMulVec x M.v) - Phi = _
    rw [Matrix.add_mul, Matrix.mul_add, Matrix.mul_add,
      Matrix.mul_vecMulVec, Matrix.vecMulVec_mul,
      Matrix.vecMulVec_mul_vecMulVec, Matrix.vecMulVec_smul]
    rw ←[ O77.Residual.transpose_mulVec_eq_vecMul, dotProduct_comm y x]
    dsimp [E, X, Y, R, b, a]
    abel
  have hnorm := four_term_square E X Y R (dotProduct x y)
  rw [hex, hey, her, hxy, hxr, hyr] at hnorm
  have hnX : frobeniusSq X = vectorSq b := by simp [X, outer_sq, vectorSq_dot, M.unit_v]
  have hnY : frobeniusSq Y = vectorSq a := by simp [Y, outer_sq, vectorSq_dot, M.unit_u]
  have hnR : frobeniusSq R = 1 := by simp [R, outer_sq, vectorSq_dot, M.unit_u, M.unit_v]
  rw [hnX, hnY, hnR] at hnorm
  change frobeniusSq ((w + modeDelta M x y).2 * (w + modeDelta M x y).1 - Phi) / 2 -
    frobeniusSq E / 2 = _
  rw [herror, hnorm]
  dsimp [b, a]
  ring
```

The Hessian is the actual second derivative. `hessianValuePhiwd` is literally $D(DL_\Phi)(w)[d,d]$. Define the scalar form $Q(x,y) = q(Bx) + q(A^{\mathsf{T}}y) - 2\sigma\langle x, y\rangle$ separately. The native loss is a polynomial in finitely many entries, so `contDiff_native_loss` proves it is C^2. Substituting (tx, ty) into the preceding exact expansion gives

$$L_\Phi(w + tD(x,y)) - L_\Phi(w) = \tfrac{1}{2}t^2 Q(x,y) + \tfrac{1}{2}t^4\langle x, y\rangle^2.$$

```
/-- The Hessian diagonal, distinguished from the quadratic Taylor coefficient. -/
def hessianValue (Phi : RM O I) (w d : Parameters I O H) : ℝ :=
  (fderiv ℝ (fderiv ℝ (loss Phi)) w d) d

def modeQuadratic (M : DiscardedMode Phi w) (x y : Fin H → ℝ) : ℝ :=
  vectorSq (w.2.mulVec x) + vectorSq (w.1.transpose.mulVec y) -
    2 * M.sigma * dotProduct x y

lemma contDiff_native_loss (Phi : RM O I) : ContDiff ℝ 2 (loss (H := H) Phi) := by
  unfold loss frobeniusSq
  simp only [Matrix.mul_apply, Matrix.sub_apply]
  fun_prop

lemma discarded_line_identity (M : DiscardedMode Phi w) (x y : Fin H → ℝ) (t : ℝ) :
    loss Phi (w + t • modeDelta M x y) - loss Phi w =
      t^2 * modeQuadratic M x y / 2 + t^4 * (dotProduct x y)^2 / 2 := by
  have h := discarded_loss_expansion M (t • x) (t • y)
  rw [modeDelta_smul] at h
  simp only [Matrix.mulVec_smul, vectorSq_smul, smul_dotProduct,
    dotProduct_smul, smul_eq_mul] at h
  rw [h]
  dsimp [modeQuadratic]
```

```
  ring
```

For any C^2 function whose restriction to $w + td$ has the displayed even quartic form with coefficients q, c, differentiating that polynomial gives first derivative $tq+2t^3c$. Independently, the chain rule gives $Df(w+td)[d]$. Uniqueness of derivatives equates the two functions. Differentiate this equality once more at zero and use the chain rule and evaluation of continuous linear maps. The results are $Df(w)[d] = 0$ and $D^2f(w)[d, d] = q$. Thus the following specialization identifies the Hessian with Q, not $Q/2$. The factor $1/2$ belongs to the Taylor coefficient, not to the Hessian itself.

```
/-- Identify actual first and second derivatives from an exact even quartic on a line.
The differentiability hypotheses are discharged for the native polynomial loss below. -/
private lemma derivatives_of_even_line
    {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E]
    (f : E → ℝ) (hf : ContDiff ℝ 2 f) (w d : E) (q c : ℝ)
    (hline : ∀ t : ℝ, f (w + t • d) - f w = t^2 * q / 2 + t^4 * c / 2) :
    fderiv ℝ f w d = 0 ∧ (fderiv ℝ (fderiv ℝ f) w d) d = q := by
  let p : ℝ → ℝ := fun t => f w + t^2 * q / 2 + t^4 * c / 2
  let dp : ℝ → ℝ := fun t => t * q + 2 * t^3 * c
  have hfun : (fun t : ℝ => f (w + t • d)) = p := by
    funext t
    dsimp [p]
    linarith [hline t]
  have hp (t : ℝ) : HasDerivAt p (dp t) t := by
    have h2 : HasDerivAt (fun s : ℝ => s^2) (2 * t) t := by
      exact (hasDerivAt_pow 2 t).congr_deriv (by norm_num)
    have h4 : HasDerivAt (fun s : ℝ => s^4) (4 * t^3) t := by
      exact (hasDerivAt_pow 4 t).congr_deriv (by norm_num)
    exact ((((h2.mul_const q).div_const 2).const_add (f w)).add
      ((h4.mul_const c).div_const 2)).congr_deriv (by dsimp [dp]; ring)
  have hpath (t : ℝ) : HasDerivAt (fun s : ℝ => w + s • d) d t := by
    simpa using ((hasDerivAt_id' t).smul_const d).const_add w
  have hd : (fun t : ℝ => fderiv ℝ f (w + t • d) d) = dp := by
    funext t
    have h := ((hf.differentiable (by norm_num)) _).hasFDerivAt.comp_hasDerivAt t (hpath t)
    change HasDerivAt (fun s : ℝ => f (w + s • d))
      (fderiv ℝ f (w + t • d) d) t at h
    rw [hfun] at h
    exact h.unique (hp t)
  have hdf : ContDiff ℝ 1 (fderiv ℝ f) := hf.fderiv_right (by norm_num)
  have hsecond : HasDerivAt (fun t : ℝ => fderiv ℝ f (w + t • d) d)
      ((fderiv ℝ (fderiv ℝ f) w d) d) 0 := by
    have hline' : HasDerivAt (fun t : ℝ => fderiv ℝ f (w + t • d))
        (fderiv ℝ (fderiv ℝ f) w d) 0 := by
      simpa only [zero_smul, add_zero, Function.comp_def] using
        (((hdf.differentiable (by norm_num)) (w + (0 : ℝ) • d)).hasFDerivAt.comp_hasDerivAt
          0 (hpath 0))
    have heval : HasDerivAt (fun t : ℝ => fderiv ℝ f (w + t • d) d)
        (((fderiv ℝ (fderiv ℝ f) w d) d) +
          fderiv ℝ f (w + (0 : ℝ) • d) (0 : E)) 0 :=
      HasDerivAt.clm_apply hline' (hasDerivAt_const 0 d)
    simpa only [map_zero, add_zero] using heval
  have hdp : HasDerivAt dp q 0 := by
    have hlin : HasDerivAt (fun t : ℝ => t * q) q 0 := by
      simpa using (hasDerivAt_id' (0 : ℝ)).mul_const q
    have h3 : HasDerivAt (fun t : ℝ => t^3) 0 0 := by
      exact (hasDerivAt_pow 3 (0 : ℝ)).congr_deriv (by norm_num)
    exact (hlin.add ((h3.const_mul 2).mul_const c)).congr_deriv (by ring)
  constructor
  · have h := congrFun hd 0
    simpa [dp] using h
```

```
  · rw [hd] at hsecond
    exact hsecond.unique hdp

/-- Genuine Frechet derivatives of the original loss, not a renamed quadratic form. -/
theorem discarded_first_second (M : DiscardedMode Phi w) (x y : Fin H → ℝ) :
    fderiv ℝ (loss Phi) w (modeDelta M x y) = 0 ∧
      hessianValue Phi w (modeDelta M x y) = modeQuadratic M x y :=
  derivatives_of_even_line (loss Phi) (contDiff_native_loss Phi) w
    (modeDelta M x y) (modeQuadratic M x y) ((dotProduct x y)^2)
    (discarded_line_identity M x y)
```

On $y = -x$, the cross-term contributes $2\sigma q(x)$, while the two squared terms are nonnegative. Since $\|D(x,-x)\|_{\mathrm{ent}}^2 = 2q(x)$,

$$D^2 L_\Phi(w)[D(x,-x), D(x,-x)] \geq \sigma \|D(x,-x)\|_{\mathrm{ent}}^2.$$

This provides a positive hidden subspace with its exact ambient scale.

```
/-- Coercivity is relative to entry-Euclidean squared length, with its exact factor. -/
theorem antiDiagonal_coercive (M : DiscardedMode Phi w) (x : Fin H → ℝ) :
    M.sigma * O77.Native.ambientSq (modeDelta M x (-x)) ≤
      hessianValue Phi w (modeDelta M x (-x)) := by
  rw [(discarded_first_second M x (-x)).2, modeDelta_sq]
  have hneg : vectorSq (-x) = vectorSq x := by simp [vectorSq]
  rw [hneg]
  dsimp [modeQuadratic]
  rw [dotProduct_neg, ← vectorSq_dot]
  nlinarith [vectorSq_nonneg (w.2.mulVec x), vectorSq_nonneg (w.1.transpose.mulVec (-x))]
```

42.3 A negative direction and actual lower values

If $\mathrm{rank}(BA) < H$, at least one of B and A^T has a nonzero kernel vector. Otherwise both maps are injective. `LinearMap.finrank_range_of_inj` and `Matrix.rank_transpose` then give $\mathrm{rank}\, A = H$; injectivity of B preserves the dimension of $\mathrm{im}\, A$, using `Submodule.equivMapOfInjective`. Hence $\mathrm{rank}(BA) = H$, a contradiction. No singular decomposition of the factors and no balance assumption enters this argument.

```
/-- Rank deficiency forces one actual hidden kernel vector. No SVD or balanced
factorization is assumed. The rank comparison reuses Mathlib's range API. -/
theorem kernel_alternative (A : RM H I) (B : RM O H)
    (hrank : (B * A).rank < H) :
    ∃( z : Fin H → ℝ, z ≠ 0 ∧ B.mulVec z = 0) ∨
      ∃( z : Fin H → ℝ, z ≠ 0 ∧ A.transpose.mulVec z = 0) := by
  by_contra hn
  have hB0 : ∀ z, B.mulVec z = 0 → z = 0 := by
    intro z hz
    by_contra hz0
    exact hn (Or.inl ⟨z, hz0, ⟩hz)
  have hA0 : ∀ z, A.transpose.mulVec z = 0 → z = 0 := by
    intro z hz
    by_contra hz0
    exact hn (Or.inr ⟨z, hz0, ⟩hz)
  have hBi : Function.Injective B.mulVecLin :=
    LinearMap.ker_eq_bot.mp (LinearMap.ker_eq_bot'.mpr hB0)
  have hAi : Function.Injective A.transpose.mulVecLin :=
    LinearMap.ker_eq_bot.mp (LinearMap.ker_eq_bot'.mpr hA0)
  have hAt : A.transpose.rank = H := by
    change Module.finrank ℝ A.transpose.mulVecLin.range = H
    simpa using (LinearMap.finrank_range_of_inj (f := A.transpose.mulVecLin) hAi)
  have hA : A.rank = H := by simpa only [Matrix.rank_transpose] using hAt
  have hBA : (B * A).rank = A.rank := by
```

```
    change Module.finrank ℝ (B * A).mulVecLin.range = Module.finrank ℝ A.mulVecLin.range
    rw [Matrix.mulVecLin_mul, LinearMap.range_comp]
    exact (Submodule.equivMapOfInjective B.mulVecLin hBi A.mulVecLin.range).finrank_eq.symm
  omega
```

For $D \geq 0$ and $C > 0$, choose $t = C/(D+1) > 0$. The identity $t(D+1) = C$ gives $Dt^2 - 2Ct < 0$. If $Lz = 0$ with $z \neq 0$, apply this to $D = q(Rz)$ and $C = \sigma q(z)$: the vectors $x = z$, $y = tz$ satisfy

$$q(Lx) + q(Ry) - 2\sigma\langle x, y\rangle < 0.$$

Apply this common calculation to $L = B, R = A^{\mathsf{T}}$ or to the reversed pair from the kernel alternative, and interchange x, y in the second case. This constructs an actual negative value of Q.

```
private lemma negative_cross_term {D C : ℝ} (hD : 0 ≤ D) (hC : 0 < C) :
    let t := C / (D + 1)
    0 < t ∧ D * t^2 - 2 * C * t < 0 := by
  dsimp
  have hden : 0 < D + 1 := by linarith
  have ht : 0 < C / (D + 1) := div_pos hC hden
  have heq : (C / (D + 1)) * (D + 1) = C := ₀div_mul_cancel C hden.ne'
  constructor
  · exact ht
  · have heq2 := congrArg (fun s : ℝ => s * (C / (D + 1))) heq
    nlinarith [mul_pos hC ht, sq_nonneg (C / (D + 1))]

/-- One kernel calculation serves both orders of the two squared terms. -/
private lemma negative_quadratic_of_kernel {m n : ℕ}
    (L : RM m H) (R : RM n H) {sigma : ℝ} (hs : 0 < sigma)
    {z : Fin H → ℝ} (hz : z ≠ 0) (hLz : L.mulVec z = 0) :
    ∃ x y : Fin H → ℝ,
      vectorSq (L.mulVec x) + vectorSq (R.mulVec y) -
        2 * sigma * dotProduct x y < 0 := by
  let Z := vectorSq z
  let D := vectorSq (R.mulVec z)
  let t := (sigma * Z) / (D + 1)
  have ht := negative_cross_term (vectorSq_nonneg (R.mulVec z))
    (mul_pos hs (vectorSq_pos hz))
  refine ⟨z, t • z, ?⟩_
  have heq : vectorSq (L.mulVec z) + vectorSq (R.mulVec (t • z)) -
      2 * sigma * dotProduct z (t • z) = D * t^2 - 2 * (sigma * Z) * t := by
    simp only [hLz, Matrix.mulVec_smul, vectorSq_smul,
      dotProduct_smul, smul_eq_mul, ← vectorSq_dot]
    have hzero : vectorSq (0 : Fin m → ℝ) = 0 := by simp [vectorSq]
    rw [hzero]
    dsimp [D, Z]
    ring
  rw [heq]
  exact ht.2

/-- A negative Hessian vector is explicitly built in the discarded-mode image. -/
theorem discarded_negative_direction (M : DiscardedMode Phi w)
    (hrank : (w.2 * w.1).rank < H) :
    ∃ x y : Fin H → ℝ, modeQuadratic M x y < 0 := by
  rcases kernel_alternative w.1 w.2 hrank with ⟨z, hz, ⟩hBz | ⟨z, hz, ⟩hAz
  · exact negative_quadratic_of_kernel w.2 w.1.transpose M.sigma_pos hz hBz
  · obtain ⟨x, y, ⟩hxy :=
      negative_quadratic_of_kernel w.1.transpose w.2 M.sigma_pos hz hAz
    exact ⟨y, x, by simpa only [modeQuadratic, add_comm,
      dotProduct_comm y x] using ⟩hxy
```

For $q < 0$ and $c \geq 0$, choose $\rho = \min(1, -q/(c+1)) > 0$. If $0 < t < \rho$, then $t^2 < t$ and

$q + t^2 c < 0$, so $(t^2/2)(q + t^2 c) < 0$. This explicitly controls the quartic remainder; a negative Hessian value alone does not justify an arbitrary step size.

```
/-- Control the quartic remainder explicitly, rather than assuming Hessian sign
alone makes the loss smaller at an arbitrary step size. -/
private lemma quartic_negative_near {q c : ℝ} (hq : q < 0) (hc : 0 ≤ c) :
    ∃ rho : ℝ, 0 < rho ∧ ∀ t, 0 < t → t < rho →
      t^2 * q / 2 + t^4 * c / 2 < 0 := by
  have hden : 0 < c + 1 := by linarith
  refine ⟨min 1 (-q / (c + 1)), lt_min zero_lt_one (div_pos (neg_pos.mpr hq) hden), ?_⟩
  intro t ht htr
  have ht1 : t < 1 := htr.trans_le (min_le_left _ _)
  have htc : t * (c + 1) < -q := (₀lt_div_iff hden).mp (htr.trans_le (min_le_right _ _))
  have ht2 : t^2 < t := by nlinarith
  have hcoef : q + t^2 * c < 0 := by
    nlinarith [mul_nonneg (sub_nonneg.mpr ht2.le) hc]
  calc
    t^2 * q / 2 + t^4 * c / 2 = (t^2 / 2) * (q + t^2 * c) := by ring
    _ < 0 := mul_neg_of_pos_of_neg (div_pos (sq_pos_of_pos ht) (by norm_num)) hcoef
```

For a chosen negative direction $d = D(x, y)$ put $N = \|d\|_{\mathrm{ent}}$. Given any $\delta > 0$, take $t = \min(\rho/2, \delta/(2(N+1)))$. Then $t > 0$, $t < \rho$, and $tN < \delta$. Homogeneity of the actual entry norm places $w + td$ in its open δ-ball, while the exact quartic identity gives strictly smaller loss. Combining this with the constructed discarded mode and the rung rank bound proves `rung_lower_in_every_ball` at each given rung point.

```
/-- Lower values occur in every actual native entry-Euclidean ball. -/
theorem discarded_lower_in_every_ball (M : DiscardedMode Phi w)
    (hrank : (w.2 * w.1).rank < H) (delta : ℝ) (hdelta : 0 < delta) :
    ∃ v ∈ entryBall w delta, loss Phi v < loss Phi w := by
  obtain ⟨x, y, ⟩hq := discarded_negative_direction M hrank
  obtain ⟨rho, hrho, ⟩hsmall := quartic_negative_near hq (sq_nonneg (dotProduct x y))
  let d := modeDelta M x y
  let N := entryNorm d
  have hN : 0 ≤ N := Real.sqrt_nonneg _
  let t := min (rho / 2) (delta / (2 * (N + 1)))
  have hden : 0 < 2 * (N + 1) := by positivity
  have ht : 0 < t := lt_min (half_pos hrho) (div_pos hdelta hden)
  have htr : t < rho := (min_le_left _ _).trans_lt (half_lt_self hrho)
  have htn : t * (2 * (N + 1)) ≤ delta :=
    (₀le_div_iff hden).mp (min_le_right _ _)
  have hnorm : entryNorm (t • d) = t * N := by
    change entryNorm (t • d) = t * entryNorm d
    simp only [O77.Native.entryNorm_eq, map_smul, norm_smul, Real.norm_eq_abs, abs_of_pos ht]
  refine ⟨w + t • d, ?_, ?⟩_
  · change entryNorm ((w + t • d) - w) < delta
    rw [add_sub_cancel_left, hnorm]
    nlinarith [mul_nonneg ht.le hN]
  · have h := discarded_line_identity M x y t
    have hneg := hsmall t ht htr
    change loss Phi (w + t • modeDelta M x y) < loss Phi w
    linarith

/-- No balance, representative choice, Wishart premise or band-pair input. -/
theorem rung_lower_in_every_ball {I O H r k : ℕ} (D : SingularData I O r)
    (hH : r < H) (hkr : k < r) (w : Parameters I O H) (hw : D.OnRung k w) :
    ∀ delta : ℝ, 0 < delta → ∃ v ∈ entryBall w delta,
      loss D.target v < loss D.target w :=
  discarded_lower_in_every_ball (discardedMode_of_rung D hkr w hw)
    (D.rung_rank_lt hH hw)
```

42.4 Normalizing a two-plane

For distinct hidden indices i, j, set $h(p) = p_1 e_i + p_2 e_j$. Orthogonality of these coordinate vectors gives $q(h(p)) = p_1^2 + p_2^2$. Let $s = (\sqrt{2})^{-1}$, so $s^2 = 1/2$. The proposed plane is

$$P(p) = D(sh(p), -sh(p)).$$

Its linearity follows entrywise from the outer-product formulas. The factor s is essential: before normalization the ambient squared norm of the anti-diagonal perturbation is twice the hidden squared norm.

```
private def hiddenPair (i j : Fin H) (p : O77.SaddleDirect.Plane) : Fin H → ℝ :=
  p.1 • Pi.single i 1 + p.2 • Pi.single j 1

private lemma hiddenPair_sq (i j : Fin H) (hij : i ≠ j) (p : O77.SaddleDirect.Plane) :
    vectorSq (hiddenPair i j p) = O77.SaddleDirect.planeSq p := by
  rw [vectorSq_dot]
  simp [hiddenPair, dotProduct_add, dotProduct_smul, dotProduct_single,
    hij, Ne.symm hij, O77.SaddleDirect.planeSq, pow_two]

/-- The coefficient gives an entry-Euclidean orthonormal plane, not a max-norm isometry. -/
private def planeScale : ℝ := (Real.sqrt 2)⁻¹

private lemma planeScale_sq : planeScale^2 = 1 / 2 := by
  dsimp [planeScale]
  rw [inv_pow, Real.sq_sqrt (by norm_num : (0 : ℝ) ≤ 2)]
  norm_num

/-- A concrete linear plane in the original matrix entries. -/
def positivePlane (M : DiscardedMode Phi w) (i j : Fin H) :
    O77.SaddleDirect.Plane →ₗ[ℝ] Parameters I O H where
  toFun p := modeDelta M (planeScale • hiddenPair i j p)
    (-(planeScale • hiddenPair i j p))
  map_add' p q := by
    ext <;> simp [modeDelta, hiddenPair, Matrix.vecMulVec, smul_eq_mul,
      add_mul, mul_add, mul_assoc] <;> ring
  map_smul' t p := by
    ext <;> simp [modeDelta, hiddenPair, Matrix.vecMulVec, smul_eq_mul,
      mul_add, mul_assoc, mul_left_comm]
    ring
```

The ambient square identity for D now gives $\|P(p)\|_{\mathrm{ent}}^2 = q_P(p)$ exactly. If $P(p) = P(q)$, apply this identity to $p - q$; the sum of its two nonnegative squares vanishes, hence $p = q$. The genuine derivative identities and anti-diagonal coercivity then give $DL_\Phi(w)[P(p)] = 0$ and $D^2 L_\Phi(w)[P(p), P(p)] \geq \sigma q_P(p)$.

```
lemma positivePlane_sq (M : DiscardedMode Phi w) (i j : Fin H) (hij : i ≠ j)
    (p : O77.SaddleDirect.Plane) :
    O77.Native.ambientSq (positivePlane M i j p) = O77.SaddleDirect.planeSq p := by
  change O77.Native.ambientSq (modeDelta M (planeScale • hiddenPair i j p)
    (-(planeScale • hiddenPair i j p))) = _
  rw [modeDelta_sq]
  have hneg : vectorSq (-(planeScale • hiddenPair i j p)) =
      vectorSq (planeScale • hiddenPair i j p) := by simp [vectorSq]
  rw [hneg, vectorSq_smul, hiddenPair_sq i j hij, planeScale_sq]
  ring

lemma positivePlane_injective (M : DiscardedMode Phi w) (i j : Fin H) (hij : i ≠ j) :
    Function.Injective (positivePlane M i j) := by
  intro p q hpq
  have hz : positivePlane M i j (p - q) = 0 := by rw [map_sub, hpq, sub_self]
  have hs := congrArg O77.Native.ambientSq hz
```

```
  rw [positivePlane_sq M i j hij] at hs
  have hs0 : O77.SaddleDirect.planeSq (p - q) = 0 := by
    simpa [O77.Native.ambientSq, frobeniusSq] using hs
  dsimp [O77.SaddleDirect.planeSq] at hs0
  apply Prod.ext
  · have : p.1 - q.1 = 0 := by nlinarith [sq_nonneg (p.2 - q.2)]
    exact sub_eq_zero.mp this
  · have : p.2 - q.2 = 0 := by nlinarith [sq_nonneg (p.1 - q.1)]
    exact sub_eq_zero.mp this

lemma positivePlane_derivatives (M : DiscardedMode Phi w) (i j : Fin H)
    (hij : i ≠ j) (p : O77.SaddleDirect.Plane) :
    fderiv ℝ (loss Phi) w (positivePlane M i j p) = 0 ∧
      M.sigma * O77.SaddleDirect.planeSq p ≤
        hessianValue Phi w (positivePlane M i j p) := by
  constructor
  · exact (discarded_first_second M (planeScale • hiddenPair i j p)
      (-(planeScale • hiddenPair i j p))).1
  · have h := antiDiagonal_coercive M (planeScale • hiddenPair i j p)
    change M.sigma * O77.Native.ambientSq (positivePlane M i j p) ≤ _ at h
    rw [positivePlane_sq M i j hij] at h
    exact h
```

For $k < r < H$ one has $H \geq 2$, so choose the hidden indices $0, 1$. The theorem packages the normalized plane, its injectivity, its actual first and second derivative properties, and the nearby lower-loss points. This is pointwise at every $w \in C_k$ and contains no volume assertion. When $r = 0$ there is no nonoptimal index, as required.

The final two examples check the scalar-target quartic $-t^2 + t^4/2$: it is negative at $t = 1$ and not at $t = 2$. They retain a concrete reminder that step-size control is necessary.

```
/-- Every actual nonoptimal rung point supplies an explicitly normalized positive
plane and arbitrarily nearby lower values, without any balancedness hypothesis. -/
theorem rung_sign_geometry {I O H r k : ℕ} (D : SingularData I O r)
    (hH : r < H) (hkr : k < r) (w : Parameters I O H) (hw : D.OnRung k w) :
    ∃ (P : O77.SaddleDirect.Plane →ₗ[ℝ] Parameters I O H) (sigma : ℝ),
      0 < sigma ∧ Function.Injective P ∧
      ∀( p, O77.Native.ambientSq (P p) = O77.SaddleDirect.planeSq p) ∧
      ∀( p, fderiv ℝ (loss D.target) w (P p) = 0 ∧
        sigma * O77.SaddleDirect.planeSq p ≤ hessianValue D.target w (P p)) ∧
      ∀( delta : ℝ, 0 < delta → ∃ v ∈ entryBall w delta,
        loss D.target v < loss D.target w) := by
  have h2 : 2 ≤ H := by omega
  let i : Fin H := ⟨0, by ⟩omega
  let j : Fin H := ⟨1, by ⟩omega
  have hij : i ≠ j := by
    intro h
    have := congrArg Fin.val h
    norm_num [i, j] at this
  let M := discardedMode_of_rung D hkr w hw
  exact ⟨positivePlane M i j, M.sigma, M.sigma_pos,
    positivePlane_injective M i j hij, positivePlane_sq M i j hij,
    positivePlane_derivatives M i j hij, rung_lower_in_every_ball D hH hkr w ⟩hw

-- Regression examples.
example :
    (-(1 : ℝ)^2 + (1 : ℝ)^4 / 2 < 0) ∧
    ¬ (-(2 : ℝ)^2 + (2 : ℝ)^4 / 2 < 0) := by norm_num

example (t : ℝ) :
    loss (1 : RM 1 1)
      ((Matrix.of fun i _ => if i = (0 : Fin 2) then t else 0 : RM 2 1),
       (Matrix.of fun _ i => if i = (0 : Fin 2) then t else 0 : RM 1 2)) -
```

```
      loss (1 : RM 1 1) (0 : Parameters 1 1 2) = -t^2 + t^4 / 2 := by
  simp [loss, frobeniusSq, Matrix.mul_apply, Matrix.one_apply]
  ring

end O77.RungGeometry0906
end
```

43 Module `O77SaddleAttachment`

Let $P = \mathbb{R}^2$ with $q_P(y) = y_1^2 + y_2^2$, and let $E = \mathbb{R}^d$ have its Euclidean norm. This module attaches the geometric data at a native matrix point to the full-dimensional band theorem. It constructs the uniform Hessian cylinder and a measure-preserving completion of the *same* positive plane. The preceding saddle preparation is then applied to the pullback of the original scalar loss.

```
import Mathlib
import O77RungGeometry
import O77SaddlePreparation

set_option autoImplicit false
noncomputable section
open Set Filter MeasureTheory
open scoped Topology ContDiff ENNReal Matrix.Norms.Elementwise
namespace O77.SaddleAttachment0906
open O77.SaddleDirect
open O77.SaddlePreparation0906
attribute [local instance] O77.Native.parametersBorelSpace
```

43.1 From pointwise coercivity to a uniform cylinder

For $f : P \times E \to \mathbb{R}$, put $G(z, y) = D(f(\cdot, z))(y)$ and $T(z, y) = D_y G(z, y)$. The code defines `planeHessian` by composing the derivative of G with the injection of the plane variable. If f is C^2, the chain rule identifies $G(z, y) = Df(y, z) \circ \mathrm{inl}$, proves G is C^1, and gives

$$T(z, y) = D(D(f(\cdot, z)))(y).$$

In particular T is continuous in both variables. These adapters verify that the subsequent operator estimates concern the actual slice Hessian.

```
/-- The operator in the actual plane variable, not an independently supplied form. -/
def planeHessian {d : ℕ} (f : Plane × Tail d → ℝ) (zy : Tail d × Plane) :
    Plane →L[ℝ] Plane →L[ℝ] ℝ :=
  (fderiv ℝ (partialDifferential f) zy).comp
    (ContinuousLinearMap.inr ℝ (Tail d) Plane)

lemma partialDifferential_eq {d : ℕ} {f : Plane × Tail d → ℝ}
    (hf : ContDiff ℝ 2 f) (zy : Tail d × Plane) :
    partialDifferential f zy = (fderiv ℝ f (zy.2, zy.1)).comp
      (ContinuousLinearMap.inl ℝ Plane (Tail d)) := by
  have hi : HasFDerivAt (fun y : Plane => (y, zy.1))
      (ContinuousLinearMap.inl ℝ Plane (Tail d)) zy.2 := by
    exact hasFDerivAt_prodMk_left zy.2 zy.1
  exact (((hf.differentiable (by norm_num)) _).hasFDerivAt.comp zy.2 hi).fderiv

lemma partialDifferential_contDiff {d : ℕ} {f : Plane × Tail d → ℝ}
    (hf : ContDiff ℝ 2 f) : ContDiff ℝ 1 (partialDifferential f) := by
  have hd : ContDiff ℝ 1 (fderiv ℝ f) := hf.fderiv_right (by norm_num)
  have he : partialDifferential f = fun zy : Tail d × Plane =>
```

```
      (fderiv ℝ f (zy.2, zy.1)).comp
        (ContinuousLinearMap.inl ℝ Plane (Tail d)) := funext (partialDifferential_eq hf)
  rw [he]
  fun_prop

lemma planeHessian_eq_slice {d : ℕ} {f : Plane × Tail d → ℝ}
    (hf : ContDiff ℝ 2 f) (z : Tail d) (y : Plane) :
    planeHessian f (z, y) = fderiv ℝ (fderiv ℝ (fun x : Plane => f (x, z))) y := by
  have hi : HasFDerivAt (fun x : Plane => (z, x))
      (ContinuousLinearMap.inr ℝ (Tail d) Plane) y := by
    exact hasFDerivAt_prodMk_right z y
  have h := (((partialDifferential_contDiff hf).differentiable
    (by norm_num)) (z, y)).hasFDerivAt.comp y hi
  exact h.fderiv.symm

lemma planeHessian_continuous {d : ℕ} {f : Plane × Tail d → ℝ}
    (hf : ContDiff ℝ 2 f) : Continuous (planeHessian f) := by
  have hd : Continuous (fderiv ℝ (partialDifferential f)) :=
    (partialDifferential_contDiff hf).continuous_fderiv (by norm_num)
  unfold planeHessian
  fun_prop

-- Coordinate adapters: the plane has the product maximum norm, not the L2 norm.
```

Maximum-norm and operator-norm bookkeeping. The library's product norm satisfies $\|v\|_{\max}^2 \le q_P(v)$. Therefore, for a continuous bilinear form T represented as a nested continuous linear map, $|T(v,v)| \le \|T\|\,\|v\|_{\max}^2 \le \|T\| q_P(v)$. Two uses of `ContinuousLinearMap.le_opNorm` prove the first bound. No maximum-norm ball is silently identified with a Euclidean disk.

```
private lemma plane_norm_sq_le (v : Plane) : ‖v‖ ^ 2 ≤ planeSq v := by
  rcases v with ⟨x, y⟩
  simp only [Prod.norm_def, Real.norm_eq_abs, planeSq]
  rcases le_total |x| |y| with h | h
  · rw [max_eq_right h, sq_abs]
    nlinarith [sq_nonneg x]
  · rw [max_eq_left h, sq_abs]
    nlinarith [sq_nonneg y]

private lemma plane_form_bound (T : Plane →L[ℝ] Plane →L[ℝ] ℝ) (v : Plane) :
    |T v v| ≤ ‖T‖ * planeSq v := by
  calc
    |T v v| = ‖T v v‖ := (Real.norm_eq_abs _).symm
    _ ≤ ‖T v‖ * ‖v‖ := (T v).le_opNorm v
    _ ≤ (‖T‖ * ‖v‖) * ‖v‖ :=
      mul_le_mul_of_nonneg_right (T.le_opNorm v) (norm_nonneg v)
    _ = ‖T‖ * ‖v‖ ^ 2 := by ring
    _ ≤ ‖T‖ * planeSq v :=
      mul_le_mul_of_nonneg_left (plane_norm_sq_le v) (norm_nonneg T)
```

If $T(v,v) \ge \sigma q_P(v)$ for all v and $\sigma > 0$, then $T : P \to P^*$ is injective: $T(v-u) = 0$ forces $q_P(v-u) = 0$. The plane and its continuous dual have equal finite dimension; hence `LinearMap.linearEquivOfInjective` supplies a linear equivalence, which is continuous in finite dimensions. This proves the invertibility needed by the implicit-function theorem. Symmetry of T is not needed for this implication. The following continuity helper simply expresses operator-norm closeness to a continuous function's value at a point.

```
/-- Coercivity supplies the missing IFT inverse; finite-dimensional linear algebra
is imported, not added as a hypothesis about the loss. Symmetry is not needed. -/
theorem coercive_plane_isInvertible (T : Plane →L[ℝ] Plane →L[ℝ] ℝ)
    {sigma : ℝ} (hs : 0 < sigma)
```

```
    (hpos : ∀ v : Plane, sigma * planeSq v ≤ T v v) : T.IsInvertible := by
  have hinj : Function.Injective T := by
    intro v u h
    have hz : T (v - u) = 0 := by rw [map_sub, h, sub_self]
    have hp := hpos (v - u)
    rw [hz, zero_apply] at hp
    have hq : planeSq (v - u) ≤ 0 := by nlinarith [hs]
    have hn := (plane_norm_sq_le (v - u)).trans hq
    have hn0 : ‖v - ‖u = 0 := by nlinarith [norm_nonneg (v - u)]
    exact sub_eq_zero.mp (norm_eq_zero.mp hn0)
  have hdim : Module.finrank ℝ Plane = Module.finrank ℝ (Plane →L[] ℝ) := by
    simpa only [Module.finrank_linearMap, Module.finrank_self, mul_one] using
      (LinearMap.toContinuousLinearMap : (Plane →ₗ[ℝ] ℝ) ≃ₗ[ℝ] (Plane →L[] ℝ)).finrank_eq
  let e := (T.toLinearMap.linearEquivOfInjective hinj hdim).toContinuousLinearEquiv
  refine ⟨e, ?⟩_
  apply ContinuousLinearMap.ext
  intro v
  exact LinearMap.linearEquivOfInjective_apply hinj hdim v

/-- Fix the normed additive carrier before forming the difference. This avoids
asking a concrete nested-operator expression to choose its group operations. -/
private lemma eventually_norm_sub_lt {E F : Type*} [TopologicalSpace E]
    [SeminormedAddCommGroup F] {g : E → F} {x : E}
    (hg : ContinuousAt g x) {epsilon : ℝ} (hepsilon : 0 < epsilon) :
    ∀ᶠ y in nhds x, ‖g y - g ‖x < epsilon := by
  exact (tendsto_iff_norm_sub_tendsto_zero.mp hg).eventually_lt_const hepsilon
```

Assume f is C^2, $D(f(\cdot,0))(0) = 0$, and $T(0,0)[v,v] \geq \sigma q_P(v)$ with $\sigma > 0$. Continuity supplies $\eta > 0$ such that $\|T(z,y) - T(0,0)\| < \sigma/2$ whenever $\|(z,y)\|_{\max} < \eta$. Choose $R = \eta/4$, $S = \eta/2$, $k = \sigma/2$, and $K = \|T(0,0)\| + \sigma + 1$. For $\|z\| < S$ and $q_P(y) \leq (2R)^2$, $\|y\|_{\max} \leq \eta/2$, so the whole cylinder lies in that neighborhood. The preceding form estimate yields

$$|T(z,y)[v,v] - T(0,0)[v,v]| \leq (\sigma/2) q_P(v).$$

Combining it with coercivity and the operator bound at zero gives $k q_P(v) \leq T(z,y)[v,v] \leq K q_P(v)$. The invertibility proof and C^2 regularity supply every remaining field of `UniformPartialHessian`. These constants are fixed before all band radii and widths.

```
/-- Positive actual partial Hessian at the origin supplies every field of the
uniform-cylinder record. The radii are chosen before all band radii and widths. -/
theorem uniformPartialHessian_of_positive {d : ℕ} (f : Plane × Tail d → ℝ)
    (hf : ContDiff ℝ 2 f) {sigma : ℝ} (hs : 0 < sigma)
    (hzero : fderiv ℝ (fun y : Plane => f (y, 0)) 0 = 0)
    (hpos : ∀ v : Plane, sigma * planeSq v ≤
      (fderiv ℝ (fderiv ℝ (fun y : Plane => f (y, 0))) 0 v) v) :
    Nonempty (UniformPartialHessian f) := by
  let T := planeHessian f (0, 0)
  have hT (v : Plane) : sigma * planeSq v ≤ T v v := by
    simpa only [T, planeHessian_eq_slice hf] using hpos v
  have hclose : ∀ᶠ zy in nhds (0 : Tail d × Plane),
      ‖planeHessian f zy - ‖T < sigma / 2 := by
    exact eventually_norm_sub_lt (F := Plane →L[] Plane →L[] ℝ)
      ((planeHessian_continuous hf).continuousAt :
        ContinuousAt (planeHessian f) (0 : Tail d × Plane)) (half_pos hs)
  obtain ⟨eta, heta, ⟩he := Metric.mem_nhds_iff.mp hclose
  let R := eta / 4
  let S := eta / 2
  let K := |||T + sigma + 1
  have hdomain (z : Tail d) (hz : |||z < S) (y : Plane)
      (hy : planeSq y ≤ (2 * R) ^ 2) : ‖planeHessian f (z, y) - ‖T < sigma / 2 := by
    have hyN : |||y ≤ eta / 2 := by
      have hq := (plane_norm_sq_le y).trans hy
```

```
      dsimp [R] at hq
      nlinarith [norm_nonneg y]
    apply he
    rw [Metric.mem_ball, dist_zero_right, Prod.norm_def]
    exact max_lt (hz.trans (by dsimp [S]; linarith)) (by linarith)
  have hfz (z : Tail d) : ContDiff ℝ 2 (fun x : Plane => f (x, z)) := by fun_prop
  refine ⟨{
    radius := R
    tailRadius := S
    lower := sigma / 2
    upper := K
    radius_pos := by dsimp [R]; positivity
    tail_pos := by dsimp [S]; positivity
    lower_pos := half_pos hs
    lower_le_upper := by dsimp [K]; linarith [norm_nonneg T]
    partial_c1 := (partialDifferential_contDiff hf).contDiffAt
    partial_zero := hzero
    partial_invertible := coercive_plane_isInvertible T hs hT
    diff := ?_
    diff2 := ?_
    bounds := ?_ ⟩}
  · intro z _ y _
    exact (hfz z).differentiable (by norm_num) y
  · intro z _ y _
    exact ((hfz z).fderiv_right (m := 1) (by norm_num)).differentiable (by norm_num) y
  · intro z hz y hy v
    simp only ←[ planeHessian_eq_slice hf]
    have hq : 0 ≤ planeSq v := by dsimp [planeSq]; positivity
    have hd := plane_form_bound (planeHessian f (z, y) - T) v
    have hd' : |planeHessian f (z, y) v v - T v v| ≤ (sigma / 2) * planeSq v := by
      simpa only [sub_apply] using
        hd.trans (mul_le_mul_of_nonneg_right (hdomain z hz y hy).le hq)
    have ht := plane_form_bound T v
    have ht' := hT v
    have hlow := (abs_le.mp hd').1
    have hupp := (abs_le.mp hd').2
    have htUpper := (le_abs_self (T v v)).trans ht
    constructor
    · linarith
    · dsimp [K]
      nlinarith [mul_nonneg (show 0 ≤ sigma / 2 + 1 by positivity) hq]
```

For a C^2 function on this product carrier, the displayed zero and positive-plane hypotheses, together with actual lower values accumulating at zero, therefore imply the ambient pair $(1, 1)$. The proof constructs the uniform data, then the moving centers, then uses the full-dimensional tail-window theorem explained in Section 40.3.

```
/-- A reusable C2 saddle criterion in the checked product carrier. -/
theorem ambientPair_of_positive_plane {d : ℕ} (f : Plane × Tail d → ℝ)
    (hf : ContDiff ℝ 2 f) {sigma : ℝ} (hs : 0 < sigma)
    (hzero : fderiv ℝ (fun y : Plane => f (y, 0)) 0 = 0)
    (hpos : ∀ v : Plane, sigma * planeSq v ≤
      (fderiv ℝ (fderiv ℝ (fun y : Plane => f (y, 0))) 0 v) v)
    (hlower : LowerValuesAccumulate f) : HasAmbientBandPair f 1 1 := by
  obtain ⟨D := uniformPartialHessian_of_positive f hf hs hzero hpos
  exact ambientPair_of_uniformHessian f hf.continuous D hlower
```

43.2 Completing the specified plane with exact volume

Suppose a linear map $P : \mathbb{R}^2 \to \mathbb{R}^n$ satisfies $\|P(y)\|_2^2 = q_P(y)$. Regard its domain as `WithLp2Plane`; then P becomes a linear isometry J. Let $K = \operatorname{im} J$, let $d = \dim K^{\perp}$, and

choose an orthonormal basis of $K^{\perp}$. Use the range isometry and this basis to identify the two factors with K and $K^{\perp}$, and combine them by the orthogonal decomposition. The resulting continuous linear equivalence $e : P \times \mathbb{R}^d \to \mathbb{R}^n$ satisfies

$$2 + d = n, \quad e(y, 0) = P(y), \quad \|e(y, z)\|_2^2 = q_P(y) + \|z\|_2^2.$$

The sum-of-squares identity is Pythagoras in `Submodule.orthogonalDecomposition`. The dimension equality follows from the resulting linear equivalence, not from an assumed ambient dimension or truncated subtraction. Exact volume preservation follows by composing the isometries' measure-preserving statements and `WithLp.volume_preserving_toLp`; the latter is essential for the product/L2 coordinate adapters. The case $d = 0$ uses its singleton coordinate space of mass one.

```
/-- Extend the specified entry-orthonormal plane, not an unrelated two-plane.
The total map preserves the exact product measure and the sum of all squares. -/
theorem orthogonal_completion {n : ℕ}
    (P : Plane →ₗ[ℝ] EuclideanSpace ℝ (Fin n))
    (hP : ∀ y, ‖P ‖y ^ 2 = planeSq y) :
    ∃ (d : ℕ) (e : (Plane × Tail d) ≃L[ℝ] EuclideanSpace ℝ (Fin n)),
      2 + d = n ∧ ∀( y : Plane, e (y, 0) = P y) ∧
      ∀( x, ‖e ‖x ^ 2 = O77.SaddleDirect.ambientSq x) ∧
      MeasurePreserving e volume volume := by
  let LP := WithLp 2 Plane
  let J : LP →ₗᵢ[ℝ] EuclideanSpace ℝ (Fin n) := {
    toLinearMap := P.comp (WithLp.linearEquiv 2 ℝ Plane).toLinearMap
    norm_map' := by
      intro y
      apply (₀sq_eq_sq (norm_nonneg _) (norm_nonneg _)).mp
      change ‖P (WithLp.ofLp y)‖ ^ 2 = ‖‖y ^ 2
      rw [hP, WithLp.prod_norm_sq_eq_of_L2]
      simp [planeSq] }
  let K : Submodule ℝ (EuclideanSpace ℝ (Fin n)) := J.toLinearMap.range
  let d := Module.finrank ℝ (K.orthogonal)
  let b : OrthonormalBasis (Fin d) ℝ (K.orthogonal) := stdOrthonormalBasis ℝ (K.orthogonal)
  let eP : Plane ≃L[ℝ] K :=
    (WithLp.prodContinuousLinearEquiv 2 ℝ ℝ ℝ).symm.trans
      J.equivRange.toContinuousLinearEquiv
  let eT : Tail d ≃L[ℝ] (K.orthogonal) := b.repr.symm.toContinuousLinearEquiv
  let eK : (K × (K.orthogonal)) ≃L[ℝ] EuclideanSpace ℝ (Fin n) :=
    (WithLp.prodContinuousLinearEquiv 2 ℝ K (K.orthogonal)).symm.trans
      K.orthogonalDecomposition.symm.toContinuousLinearEquiv
  let e := (eP.prodCongr eT).trans eK
  have heApply (x : Plane × Tail d) :
      e x = (eP x.1 : EuclideanSpace ℝ (Fin n)) +
        (eT x.2 : EuclideanSpace ℝ (Fin n)) := by rfl
  have heP (y : Plane) : e (y, 0) = P y := by
    rw [heApply]
    simp only [map_zero, Submodule.coe_zero, add_zero]
    rfl
  have hpSq (y : Plane) : ‖eP ‖y ^ 2 = planeSq y := by
    change ‖J.equivRange (WithLp.toLp 2 y)‖ ^ 2 = planeSq y
    rw [J.equivRange.norm_map, WithLp.prod_norm_sq_eq_of_L2]
    simp [planeSq]
  have heSq (x : Plane × Tail d) : ‖e ‖x ^ 2 = O77.SaddleDirect.ambientSq x := by
    change ‖K.orthogonalDecomposition.symm (WithLp.toLp 2 (eP x.1, eT x.2))‖ ^ 2 = _
    rw [K.orthogonalDecomposition.symm.norm_map, WithLp.prod_norm_sq_eq_of_L2]
    change ‖eP x‖.1 ^ 2 + ‖eT x‖.2 ^ 2 = _
    rw [hpSq]
    change planeSq x.1 + ‖b.repr.symm x‖.2 ^ 2 = _
    rw [b.repr.symm.norm_map]
    rfl
  have hmP : MeasurePreserving eP volume volume :=
```

```
    J.equivRange.measurePreserving.comp (WithLp.volume_preserving_toLp ℝ ℝ)
  have hmT : MeasurePreserving eT volume volume := b.repr.symm.measurePreserving
  have hmK : MeasurePreserving eK volume volume :=
    K.orthogonalDecomposition.symm.measurePreserving.comp
      (WithLp.volume_preserving_toLp K (K.orthogonal))
  refine ⟨d, e, ?_, heP, heSq, hmK.comp (hmP.prod hmT)⟩
  simpa [Plane, Tail, Module.finrank_prod] using e.toLinearEquiv.finrank_eq
```

Derivatives after affine pullback. Let E, F be real normed vector spaces, let $f : E \to \mathbb{R}$ be of class C^2, let $w \in E$, and let $A : F \to E$ be continuous and linear. Define $g : F \to \mathbb{R}$ by $g(y) = f(w + Ay)$. For every $y \in F$, the chain rule gives $Dg(y) = Df(w + Ay) \circ A$. In the displayed identities below v is an arbitrary vector of F. Differentiate this equality, using `HasFDerivAt.clm_comp`, and evaluate twice on v at zero. Since A is constant,

$$Dg(0)[v] = Df(w)[Av], \qquad D^2g(0)[v, v] = D^2f(w)[Av, Av].$$

This computes the derivative of the actual pullback; it does not compare unrelated Hessian definitions.

```
/-- Two applications of the actual chain rule under an affine linear map.
No equality of unrelated Hessian specifications is assumed. -/
theorem affine_pullback_derivatives
    {E F : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E]
    [NormedAddCommGroup F] [NormedSpace ℝ F]
    (f : E → ℝ) (hf : ContDiff ℝ 2 f) (w : E) (A : F →L[ℝ] E) (v : F) :
    fderiv ℝ (fun y : F => f (w + A y)) 0 v = fderiv ℝ f w (A v) ∧
    (fderiv ℝ (fderiv ℝ (fun y : F => f (w + A y))) 0 v) v =
      (fderiv ℝ (fderiv ℝ f) w (A v)) (A v) := by
  let g : F → ℝ := fun y => f (w + A y)
  have hpath (y : F) : HasFDerivAt (fun x : F => w + A x) A y := by
    simpa using (A.hasFDerivAt (x := y)).const_add w
  have hD (y : F) : fderiv ℝ g y = (fderiv ℝ f (w + A y)).comp A :=
    (((hf.differentiable (by norm_num)) _).hasFDerivAt.comp y (hpath y)).fderiv
  have hDfun : fderiv ℝ g = fun y => (fderiv ℝ f (w + A y)).comp A := funext hD
  have hfD : ContDiff ℝ 1 (fderiv ℝ f) := hf.fderiv_right (by norm_num)
  have hsecond := ((((hfD.differentiable (by norm_num)) _).hasFDerivAt.comp
    (0 : F) (hpath 0)).clm_comp (hasFDerivAt_const A (0 : F))).fderiv
  constructor
  · change fderiv ℝ g 0 v = _
    simp only [hD, map_zero, add_zero, ContinuousLinearMap.comp_apply]
  · change (fderiv ℝ (fderiv ℝ g) 0 v) v = _
    rw [hDfun]
    have hev := congrArg (fun L : F →L[ℝ] F →L[ℝ] ℝ => L v v) hsecond
    simpa only [Function.comp_apply, ContinuousLinearMap.comp_apply,
      add_apply, zero_apply,
      ContinuousLinearMap.flip_apply, ContinuousLinearMap.compL_apply,
      map_zero, add_zero, zero_add] using hev
```

Returning to native matrix entries. The established enumeration `O77.Native.flat` sends matrix entries to Euclidean coordinates with exact squared norm and exact measure. Its inverse is measure preserving by `MeasurePreserving.symm`. Apply the preceding orthogonal completion to the enumerated plane and compose its result with this inverse. Thus the native equivalence e has $2 + d = H(I + O)$, retains the prescribed plane, preserves the sum of all entry squares, and sends product volume to `O77.Native.entryMeasureIOH`.

```
/-- The inverse entry enumeration preserves measure in both actual consumers. -/
private lemma native_flat_symm_preserves (I O H : ℕ) :
    MeasurePreserving (O77.Native.flat I O H).symm volume
```

```
      (O77.Native.entryMeasure I O H) := by
  let n := (O77.Native.flat I O H).toHomeomorph.toMeasurableEquiv
  exact (show MeasurePreserving n (O77.Native.entryMeasure I O H) volume from
    O77.Native.flat_preserves I O H).symm

/-- Pull back a normalized native plane to the exact product carrier.
This is the coordinate construction; no local-pair field appears in the witnesses. -/
theorem native_orthogonal_completion {I O H : ℕ}
    (P : Plane →ₗ[ℝ] O77.Native.Parameters I O H)
    (hP : ∀ y, O77.Native.ambientSq (P y) = planeSq y) :
    ∃ (d : ℕ) (e : (Plane × Tail d) ≃L[ℝ] O77.Native.Parameters I O H),
      2 + d = H * (I + O) ∧ ∀( y : Plane, e (y, 0) = P y) ∧
      ∀( x, O77.Native.ambientSq (e x) = O77.SaddleDirect.ambientSq x) ∧
      MeasurePreserving e volume (O77.Native.entryMeasure I O H) := by
  let N := O77.Native.flat I O H
  let Q := N.toLinearMap.comp P
  have hQ (y : Plane) : ‖Q ‖y ^ 2 = planeSq y := by
    change ‖N (P y)‖ ^ 2 = _
    rw [O77.Native.flat_norm_sq, hP]
  obtain ⟨d, e, hdim, heP, heSq, ⟩hme := orthogonal_completion Q hQ
  let E := e.trans N.symm
  refine ⟨d, E, hdim, ?_, ?_, (native_flat_symm_preserves I O H).comp ⟩hme
  · intro y
    change N.symm (e (y, 0)) = P y
    rw [heP]
    exact N.symm_apply_apply (P y)
  · intro x
    rw ←[ O77.Native.flat_norm_sq I O H]
    change ‖N (N.symm (e x))‖ ^ 2 = _
    rw [N.apply_symm_apply]
    exact heSq x

-- Only normalized entry coordinates are used to invoke Haar translation invariance.
```

Conjugating Euclidean translation by the same enumeration proves native translation invariance. The invoked theorem is `measurePreserving_add_left`; no unnormalized Haar scalar is introduced. Consequently the affine map $C(x) = w + e(x)$ also preserves the exact measures.

```
private lemma native_translation_preserves {I O H : ℕ}
    (w : O77.Native.Parameters I O H) :
    MeasurePreserving (fun v : O77.Native.Parameters I O H => w + v)
      (O77.Native.entryMeasure I O H) (O77.Native.entryMeasure I O H) := by
  let N := O77.Native.flat I O H
  have hm := (native_flat_symm_preserves I O H).comp
    ((measurePreserving_add_left volume (N w)).comp (O77.Native.flat_preserves I O H))
  change MeasurePreserving
    (fun v => N.symm (N w + N v))
    (O77.Native.entryMeasure I O H) (O77.Native.entryMeasure I O H) at hm
  simpa only ←[ map_add, ContinuousLinearEquiv.symm_apply_apply] using hm
```

43.3 An exact equality of signed bands

Let $f(x) = L_\Phi(w + e(x))$. For every $R > 0$ and every real ϵ,

$$C^{-1}\{v : \|v-w\|_{\mathrm{ent}} < R,\ |L_\Phi(v)-L_\Phi(w)| < \epsilon\} = \{x : q_P(x_P)+\|x_E\|^2 < R^2,\ |f(x)-f(0)| < \epsilon\}.$$

Indeed $C(0) = w$, the squared norm identity is exact, and $R > 0$ permits replacing the square-root ball test by its squared version. Both sets are measurable by continuity. Apply `MeasurePreserving.measure_preimage` to obtain equality of their volumes. This holds at positive-loss centers as well: the same scalar function is pulled back, and its absolute signed-band condition is retained.

```
/-- Exact signed band equality; only positive radii identify the squared-ball test. -/
theorem native_band_eq_ambient {I O H d : ℕ}
    (e : (Plane × Tail d) ≃LR[] O77.Native.Parameters I O H)
    (heSq : ∀ x, O77.Native.ambientSq (e x) = O77.SaddleDirect.ambientSq x)
    (hme : MeasurePreserving e volume (O77.Native.entryMeasure I O H))
    (Phi : O77.Reconstruction0905.RM O I) (w : O77.Native.Parameters I O H)
    {R : ℝ} (hR : 0 < R) (eps : ℝ) :
    O77.Native.bandVolume Phi w R eps =
      volume (ambientBand (fun x => O77.Native.loss Phi (w + e x)) R eps) := by
  let C : Plane × Tail d → O77.Native.Parameters I O H := fun x => w + e x
  have hmC : MeasurePreserving C volume (O77.Native.entryMeasure I O H) :=
    (native_translation_preserves w).comp hme
  have hq (x : Plane × Tail d) : 0 ≤ O77.SaddleDirect.ambientSq x := by
    dsimp [O77.SaddleDirect.ambientSq, planeSq]
    positivity
  have hset : C ⁻¹' O77.Native.bandSet Phi w R eps =
      ambientBand (fun x => O77.Native.loss Phi (w + e x)) R eps := by
    ext x
    have hball : O77.Native.entryNorm (C x - w) < R ↔
        O77.SaddleDirect.ambientSq x < R ^ 2 := by
      change Real.sqrt (O77.Native.ambientSq ((w + e x) - w)) < R ↔ _
      rw [add_sub_cancel_left, heSq]
      constructor <;> intro hx <;>
        nlinarith [Real.sq_sqrt (hq x), Real.sqrt_nonneg (O77.SaddleDirect.ambientSq x)]
    change (O77.Native.entryNorm (C x - w) < R ∧
        |O77.Native.loss Phi (C x) - O77.Native.loss Phi w| < eps) ↔
      (O77.SaddleDirect.ambientSq x < R ^ 2 ∧
        |O77.Native.loss Phi (C x) - O77.Native.loss Phi (C 0)| < eps)
    have hC0 : C 0 = w := by simp [C]
    rw [hC0, hball]
  have hm := hmC.measure_preimage
    (O77.Native.measurable_bandSet I O H Phi w R eps).nullMeasurableSet
  rw [hset] at hm
  exact hm.symm
```

For any λ, m, exact band equality transfers the ambient pair to the native pair by copying the radius witness and all constants at each radius. The two notational definitions of the power-log gauge agree; `ENNReal.ofReal_mul` handles the placement of finite positive constants. There is no radius distortion or rescaling of ϵ.

```
/-- The multiplicity and public radius-first convention are unchanged by exact
band equality. There is no positive-level comparison of two different losses. -/
theorem native_pair_of_ambient {I O H d : ℕ} {lam : ℝ} {m : ℕ}
    (e : (Plane × Tail d) ≃LR[] O77.Native.Parameters I O H)
    (heSq : ∀ x, O77.Native.ambientSq (e x) = O77.SaddleDirect.ambientSq x)
    (hme : MeasurePreserving e volume (O77.Native.entryMeasure I O H))
    (Phi : O77.Reconstruction0905.RM O I) (w : O77.Native.Parameters I O H)
    (hf : HasAmbientBandPair (fun x => O77.Native.loss Phi (w + e x)) lam m) :
    O77.Native.HasLocalBandPair Phi w lam m := by
  rcases hf with ⟨hlam, hm, r, hr, ⟩h
  refine ⟨hlam, hm, r, hr, ?⟩_
  intro R hR hRr
  obtain ⟨c, C, eps0, hc, hcC, he0, he1, ⟩hb := h R hR hRr
  refine ⟨c, C, eps0, hc, hcC, he0, he1, ?⟩_
  intro eps heps heps0
  have h := hb eps heps heps0
  rw [native_band_eq_ambient e heSq hme Phi w hR eps]
  simpa [bandGauge, O77.Residual.powerLogZero, O77.Residual.powerLogZeroReal,
    ENNReal.ofReal_mul hc.le,
    ENNReal.ofReal_mul (hc.le.trans hcC)] using h
```

Assembling the native criterion. Given a normalized native plane P, a zero first derivative along it, a positive Hessian bound with $\sigma > 0$, and lower values in every native ball, choose the preceding completion e and put $f(x) = L_\Phi(w + e(x))$. The loss is C^2, so its affine pullback is C^2. The chain-rule lemma transports the derivative hypotheses to the plane slice of f. For a lower-loss point v put $x = e^{-1}(v - w)$; the exact squared norm identity puts x in the corresponding ambient ball and gives $f(x) < f(0)$. The product-carrier positive-plane theorem therefore proves its ambient pair, and exact band transport proves the native pair $(1, 1)$. All geometric inputs are facts about actual derivatives and actual nearby values; neither a local pair nor a choice of centers is assumed.

```
/-- C2 positive-plane criterion directly in the original native matrix measure.
The geometric hypotheses are pointwise derivative facts and actual nearby values. -/
theorem native_pair_of_positive_plane {I O H : ℕ}
    (Phi : O77.Reconstruction0905.RM O I) (w : O77.Native.Parameters I O H)
    (P : Plane →L[ℝ] O77.Native.Parameters I O H) {sigma : ℝ} (hs : 0 < sigma)
    (hP : ∀ y, O77.Native.ambientSq (P y) = planeSq y)
    (hfirst : ∀ y, fderiv ℝ (O77.Native.loss Phi) w (P y) = 0)
    (hsecond : ∀ y, sigma * planeSq y ≤
      O77.RungGeometry0906.hessianValue Phi w (P y))
    (hlower : ∀ R : ℝ, 0 < R → ∃ v ∈ O77.Native.entryBall w R,
      O77.Native.loss Phi v < O77.Native.loss Phi w) :
    O77.Native.HasLocalBandPair Phi w 1 1 := by
  obtain ⟨d, e, hdim, heP, heSq, ⟩hme := native_orthogonal_completion P hP
  let f : Plane × Tail d → ℝ := fun x => O77.Native.loss Phi (w + e x)
  have hLoss : ContDiff ℝ 2 (O77.Native.loss (H := H) Phi) :=
    O77.RungGeometry0906.contDiff_native_loss Phi
  have hf : ContDiff ℝ 2 f := by dsimp [f]; fun_prop
  let A : Plane →L[ℝ] O77.Native.Parameters I O H :=
    e.toContinuousLinearMap.comp (ContinuousLinearMap.inl ℝ Plane (Tail d))
  have hA (y : Plane) : A y = P y := heP y
  have h0 : fderiv ℝ (fun y : Plane => f (y, 0)) 0 = 0 := by
    apply ContinuousLinearMap.ext
    intro y
    change fderiv ℝ (fun y : Plane => f (y, 0)) 0 y = 0
    have hd := (affine_pullback_derivatives (O77.Native.loss Phi) hLoss w A y).1
    have hd' : fderiv ℝ (fun y : Plane => f (y, 0)) 0 y =
        fderiv ℝ (O77.Native.loss Phi) w (P y) :=
      hd.trans (congrArg (fderiv ℝ (O77.Native.loss Phi) w) (hA y))
    exact hd'.trans (hfirst y)
  have hpos (y : Plane) : sigma * planeSq y ≤
      (fderiv ℝ (fderiv ℝ (fun p : Plane => f (p, 0))) 0 y) y := by
    have hd := (affine_pullback_derivatives (O77.Native.loss Phi) hLoss w A y).2
    have hd' : (fderiv ℝ (fderiv ℝ (fun p : Plane => f (p, 0))) 0 y) y =
        O77.RungGeometry0906.hessianValue Phi w (P y) :=
      hd.trans (congrArg (O77.RungGeometry0906.hessianValue Phi w) (hA y))
    exact hd'.symm ▸ hsecond y
  have hl : LowerValuesAccumulate f := by
    intro R hR
    obtain ⟨v, hv, ⟩hvl := hlower R hR
    let x := e.symm (v - w)
    have hx : w + e x = v := by simp [x]
    have hxs : O77.SaddleDirect.ambientSq x = O77.Native.ambientSq (v - w) := by
      rw ←[ heSq, ContinuousLinearEquiv.apply_symm_apply]
    have hn : Real.sqrt (O77.Native.ambientSq (v - w)) < R := hv
    have hnon : 0 ≤ O77.Native.ambientSq (v - w) := by
      rw ←[ O77.Native.flat_norm_sq I O H]
      positivity
    refine ⟨x, ?_, ?⟩_
    · rw [hxs]
      nlinarith [Real.sq_sqrt hnon, Real.sqrt_nonneg (O77.Native.ambientSq (v - w))]
    · change O77.Native.loss Phi (w + e x) <
```

```
        O77.Native.loss Phi (w + e (0 : Plane × Tail d))
      simpa only [hx, map_zero, add_zero] using hvl
  exact native_pair_of_ambient e heSq hme Phi w
    (ambientPair_of_positive_plane f hf hs h0 hpos hl)
```

The theorem `O77.RungGeometry0906.rung_sign_geometry` supplies every hypothesis of this criterion at each $w \in C_k$, $k < r < H$. Hence every such point has native pair $(1, 1)$, independently of Wishart. The next theorem spells out the equivalent real-valued bounds $c_R\epsilon \le V(R, \epsilon) \le C_R\epsilon$ at every sufficiently small fixed radius. It uses the established `O77.Native.hasLocalBandPair_iff_real`, whose finiteness result for native balls justifies conversion from extended-real volume.

The final example is nonvacuous: for $I = O = 1, H = 2, \Phi = 1$, the zero factors lie on the initial rung and have pair $(1, 1)$. The singleton singular families and both annihilation equations are constructed explicitly.

```
/-- Every point, not only a balanced representative. No Wishart premise remains
on this saddle branch. -/
theorem rung_hasLocalBandPair {I O H r k : ℕ}
    (D : O77.RungGeometry0906.SingularData I O r)
    (hH : r < H) (hkr : k < r) (w : O77.Native.Parameters I O H)
    (hw : D.OnRung k w) : O77.Native.HasLocalBandPair D.target w 1 1 := by
  obtain ⟨P, sigma, hs, _, hP, hder, ⟩hlower :=
    O77.RungGeometry0906.rung_sign_geometry D hH hkr w hw
  exact native_pair_of_positive_plane D.target w P hs hP
    (fun y => (hder y).1) (fun y => (hder y).2) hlower

/-- A direct endpoint in real-valued native volumes, with all quantifiers exposed. -/
theorem rung_real_band_bounds {I O H r k : ℕ}
    (D : O77.RungGeometry0906.SingularData I O r)
    (hH : r < H) (hkr : k < r) (w : O77.Native.Parameters I O H)
    (hw : D.OnRung k w) :
    ∃ r0 : ℝ, 0 < r0 ∧ ∀ R : ℝ, 0 < R → R < r0 →
      ∃ c C eps0 : ℝ, 0 < c ∧ c ≤ C ∧ 0 < eps0 ∧ eps0 < Real.exp (-1) ∧
        ∀ eps : ℝ, 0 < eps → eps < eps0 →
          c * eps ≤ (O77.Native.bandVolume D.target w R eps).toReal ∧
          (O77.Native.bandVolume D.target w R eps).toReal ≤ C * eps := by
  have h := (O77.Native.hasLocalBandPair_iff_real I O H D.target w 1 1).mp
    (rung_hasLocalBandPair D hH hkr w hw)
  simpa [O77.Residual.powerLogZeroReal] using h.2.2

-- Nonvacuous regression: the four-dimensional scalar-target saddle.
example : O77.Native.HasLocalBandPair (1 : O77.Reconstruction0905.RM 1 1)
    (0 : O77.Native.Parameters 1 1 2) 1 1 := by
  let D : O77.RungGeometry0906.SingularData 1 1 1 := {
    sigma := fun _ => 1
    sigma_pos := fun _ => zero_lt_one
    decreasing := by intro i j hij; omega
    u := fun _ _ => 1
    v := fun _ _ => 1
    orth_u := by intro i j; have h : i = j := Subsingleton.elim _ _; simp [h, dotProduct]
    orth_v := by intro i j; have h : i = j := Subsingleton.elim _ _; simp [h, dotProduct] }
  have ht : D.target = (1 : O77.Reconstruction0905.RM 1 1) := by
    ext i j
    fin_cases i
    fin_cases j
    norm_num [D, O77.RungGeometry0906.SingularData.target,
      O77.RungGeometry0906.SingularData.«prefix», Matrix.vecMulVec, Matrix.one_apply]
  have hw : D.OnRung 0 (0 : O77.Native.Parameters 1 1 2) := by
    simp [O77.RungGeometry0906.SingularData.OnRung,
      O77.RungGeometry0906.SingularData.«prefix»]
  simpa only [ht] using
```

```
    rung_hasLocalBandPair D (by norm_num) (by norm_num) (0 : O77.Native.Parameters 1 1 2) hw

end O77.SaddleAttachment0906
end
```

44 Module `O77RungCompletion`

The preceding module proved the pair at every given rung point. Here we construct a point on each rung and identify its product rank, loss level, and genuine criticality. Let D be singular data of rank r, U and V the matrices with columns u_j and v_j, and $c_j^{(k)} = \sigma_j$ for $j < k$ and zero otherwise. Thus $\Phi_k = U\,\mathrm{diag}(c^{(k)})V^{\mathsf{T}}$ and $\Phi = \Phi_r$.

```
import Mathlib
import O77SaddleAttachment

set_option autoImplicit false
noncomputable section
open scoped BigOperators Matrix Matrix.Norms.Elementwise

open O77.MatrixFrames O77.RungGeometry0906
open O77.Native (Parameters loss)
open O77.Residual (frobeniusSq)
open O77.Reconstruction0905 (RM)

namespace O77.RungGeometry0906.SingularData
variable {I O r : ℕ} (D : O77.RungGeometry0906.SingularData I O r)
```

44.1 Actual ranks, levels, and critical equations

For $k \le r$, rectangular synthesis preserves rank and the retained coefficients are nonzero. The cutoff-diagonal rank theorem therefore gives $\operatorname{rank}\Phi_k = k$, in particular $\operatorname{rank}\Phi = r$. Define the scalar level $\ell_k = \frac{1}{2}\sum_{j\ge k}\sigma_j^2$. This is a numerical expression; rung membership remains the product and annihilation predicate.

```
lemma prefix_rank_eq {k : ℕ} (hk : k ≤ r) : (D.«prefix» k).rank = k := by
  rw [D.prefix_eq_sandwich]
  have h := rank_sandwich (Matrix.of D.u).transpose (Matrix.of D.v).transpose
    (by simpa using D.gram_u) (by simpa using D.gram_v)
    (Matrix.diagonal (D.prefixCoefficients k))
  simp only [Matrix.transpose_transpose] at h
  rw [h]
  exact rank_cutoff_diagonal D.sigma (fun j => (D.sigma_pos j).ne') hk

lemma target_rank : D.target.rank = r := D.prefix_rank_eq le_rfl

/-- The level is derived from the error norm, not used to define rung membership. -/
def rungLevel (k : ℕ) : ℝ :=
  ∑ j : Fin r, if k ≤ j.val then (D.sigma j)^2 else 0) / 2
```

At any rung point $w = (A, B)$, the product equation gives

$$BA - \Phi = U\,\mathrm{diag}(c^{(k)} - \sigma)V^{\mathsf{T}}.$$

Subtract the two coefficient matrices, using linearity of the diagonal map and matrix multiplication. Frame isometry then shows that its Frobenius square is $\sum_{j\ge k}\sigma_j^2$, proving $L_\Phi(w) = \ell_k$ directly from the original loss.

```
lemma error_eq_sandwich {H k : ℕ} {w : Parameters I O H} (hw : D.OnRung k w) :
    w.2 * w.1 - D.target =
      (Matrix.of D.u).transpose * Matrix.diagonal (D.prefixCoefficients k - D.sigma) * (Matrix.of D.v)
      := by
  have hfull : D.prefixCoefficients r = D.sigma := by
    funext j
    simp [O77.RungGeometry0906.SingularData.prefixCoefficients, j.isLt]
  rw [hw.1, O77.RungGeometry0906.SingularData.target,
    D.prefix_eq_sandwich, D.prefix_eq_sandwich, hfull]
  have hd := (Matrix.diagonalLinearMap (Fin r) ℝ ℝ).map_sub
    (D.prefixCoefficients k) D.sigma
  change Matrix.diagonal (D.prefixCoefficients k - D.sigma) =
    Matrix.diagonal (D.prefixCoefficients k) - Matrix.diagonal D.sigma at hd
  rw [hd, Matrix.mul_sub, Matrix.sub_mul]

lemma loss_onRung {H k : ℕ} {w : Parameters I O H} (hw : D.OnRung k w) :
    loss D.target w = D.rungLevel k := by
  unfold loss
  rw [D.error_eq_sandwich hw,
    weighted_outer_frobeniusSq (Matrix.of D.u) (Matrix.of D.v) D.gram_u D.gram_v]
  unfold rungLevel
  congr 1
  apply Finset.sum_congr rfl
  intro j _
  by_cases hj : j.val < k
  · simp [O77.RungGeometry0906.SingularData.prefixCoefficients, hj,
      Nat.not_le.mpr hj]
  · simp [O77.RungGeometry0906.SingularData.prefixCoefficients, hj,
      Nat.le_of_not_gt hj]
```

The error coefficients vanish for $j < k$ and equal $-\sigma_j$ otherwise. For each nonzero coefficient the rung equations give $B^{\mathsf{T}} u_j = 0$ and $A v_j = 0$. Apply the support-annihilation lemma from `O77.MatrixFrames`, and transpose its second application, to obtain

$$B^{\mathsf{T}}(BA - \Phi) = 0, \qquad (BA - \Phi)A^{\mathsf{T}} = 0.$$

These algebraic equalities will imply zero derivative after the derivative formula below has been proved.

```
/-- Both critical equations reuse the same support-annihilation lemma. -/
lemma critical_products {H k : ℕ} {w : Parameters I O H} (hw : D.OnRung k w) :
    w.2.transpose * (w.2 * w.1 - D.target) = 0 ∧
      (w.2 * w.1 - D.target) * w.1.transpose = 0 := by
  let c := D.prefixCoefficients k - D.sigma
  have herror : w.2 * w.1 - D.target =
      ∑ j, c j • Matrix.vecMulVec (D.u j) (D.v j) := by
    exact (D.error_eq_sandwich hw).trans
      (weighted_outer_eq (Matrix.of D.u) (Matrix.of D.v) c).symm
  have hc (j : Fin r) : c j = 0 ∨ k ≤ j.val := by
    by_cases hj : j.val < k
    · left
      simp [c, O77.RungGeometry0906.SingularData.prefixCoefficients, hj]
    · exact Or.inr (Nat.le_of_not_gt hj)
  constructor
  · rw [herror]
    exact mul_weighted_outer_eq_zero w.2.transpose (Matrix.of D.u) (Matrix.of D.v) c (fun j =>
      (hc j).imp id (fun hj => (hw.2 j hj).2))
  · have hright := mul_weighted_outer_eq_zero w.1 (Matrix.of D.v) (Matrix.of D.u) c (fun j =>
      (hc j).imp id (fun hj => (hw.2 j hj).1))
    have ht : (w.2 * w.1 - D.target).transpose =
        ∑ j, c j • Matrix.vecMulVec (D.v j) (D.u j) := by
      rw [herror]
```

```
      simp only [Matrix.transpose_sum, Matrix.transpose_smul, Matrix.transpose_vecMulVec]
    change w.1 * ∑( j, c j • Matrix.vecMulVec (D.v j) (D.u j)) = 0 at hright
    rw ←[ ht] at hright
    simpa only [Matrix.transpose_mul, Matrix.transpose_transpose,
      Matrix.transpose_zero] using congrArg Matrix.transpose hright
```

44.2 An explicit witness on every rung

For $r \le H$, let $E : \mathbb{R}^r \to \mathbb{R}^H$ be the first-coordinate inclusion. Its matrix entry is 1 precisely when the row is the embedded column index; injectivity of `Fin.castLE` proves $E^{\mathsf{T}}E = 1$. With $a_j^{(k)} = 1$ for $j < k$ and zero otherwise, define

$$A_k = E\,\mathrm{diag}(a^{(k)})V^{\mathsf{T}}, \qquad B_k = U\,\mathrm{diag}(c^{(k)})E^{\mathsf{T}}.$$

These are generally unbalanced factors. Using coefficients 1 and σ_j removes any need for square roots without changing their product.

```
/-- A concrete coordinate inclusion; its only hypothesis is the dimension bound. -/
private def hiddenFrame {H : ℕ} (h : r ≤ H) : RM H r :=
  Matrix.of (fun i j => if i = Fin.castLE h j then 1 else 0)

private lemma hiddenFrame_gram {H : ℕ} (h : r ≤ H) :
    (hiddenFrame h).transpose * hiddenFrame h = 1 := by
  have hinj : Function.Injective (Fin.castLE h) := by
    intro i j hij
    exact Fin.ext (congrArg (fun z : Fin H => z.val) hij)
  ext i j
  change ∑( a : Fin H,
    (if a = Fin.castLE h i then (1 : ℝ) else 0) *
      (if a = Fin.castLE h j then (1 : ℝ) else 0)) = if i = j then 1 else 0
  simp [hinj.eq_iff, eq_comm]

/-- The factors need not be balanced. Unit input coefficients and singular-value
output coefficients avoid an unnecessary square-root construction. -/
def rungPoint {H : ℕ} (h : r ≤ H) (k : ℕ) : Parameters I O H :=
  let E := hiddenFrame h
  let a : Fin r → ℝ := fun j => if j.val < k then 1 else 0
  (E * Matrix.diagonal a * (Matrix.of D.v),
    (Matrix.of D.u).transpose * Matrix.diagonal (D.prefixCoefficients k) * E.transpose)
```

The shared-frame composition lemma contracts $E^{\mathsf{T}}E$. Since $c_j^{(k)}a_j^{(k)} = c_j^{(k)}$, it gives $B_kA_k = \Phi_k$. For $j \ge k$, the spectral-vector contraction lemma yields $A_kv_j = 0$ and $B_k^{\mathsf{T}}u_j = 0$, proving all rung conditions. For $k \le r$, rectangular rank preservation and the cutoff count give $\operatorname{rank} A_k = \operatorname{rank} B_k = k$. The existential theorem takes these concrete matrices as witnesses; at $k = 0$ all coefficients vanish, and empty singular families are legal.

```
lemma rungPoint_mem {H : ℕ} (h : r ≤ H) (k : ℕ) :
    D.OnRung k (D.rungPoint h k) := by
  let E := hiddenFrame h
  let a : Fin r → ℝ := fun j => if j.val < k then 1 else 0
  have hE : E.transpose * E = 1 := hiddenFrame_gram h
  change ((Matrix.of D.u).transpose * Matrix.diagonal (D.prefixCoefficients k) * E.transpose) *
      (E * Matrix.diagonal a * (Matrix.of D.v)) = D.«prefix» k ∧ _
  constructor
  · have hm := sandwich_mul (Matrix.of D.u).transpose E (Matrix.of D.v).transpose hE
      (Matrix.diagonal (D.prefixCoefficients k)) (Matrix.diagonal a)
    simp only [Matrix.transpose_transpose] at hm
    rw [hm, D.prefix_eq_sandwich]
    have hdiag : Matrix.diagonal (D.prefixCoefficients k) * Matrix.diagonal a =
        Matrix.diagonal (D.prefixCoefficients k) := by
```

```
      rw [Matrix.diagonal_mul_diagonal]
      congr 1
      funext j
      by_cases hj : j.val < k <;>
        simp [a, O77.RungGeometry0906.SingularData.prefixCoefficients,
          hj]
    rw [hdiag]
  · intro j hj
    have hnot : ¬ j.val < k := Nat.not_lt.mpr hj
    constructor
    · have hc := diagonal_sandwich_mulVec E (Matrix.of D.v).transpose
        (by simpa using D.gram_v) a j
      simpa [rungPoint, E, a, hnot, Matrix.of_col] using hc
    · have hc := diagonal_sandwich_mulVec E (Matrix.of D.u).transpose
        (by simpa using D.gram_u) (D.prefixCoefficients k) j
      simpa [rungPoint, E, Matrix.transpose_mul, Matrix.transpose_transpose,
        Matrix.diagonal_transpose, Matrix.mul_assoc, Matrix.of_col,
        O77.RungGeometry0906.SingularData.prefixCoefficients, hnot] using hc

lemma rungPoint_ranks {H k : ℕ} (h : r ≤ H) (hk : k ≤ r) :
    (D.rungPoint h k).1.rank = k ∧ (D.rungPoint h k).2.rank = k := by
  let E := hiddenFrame h
  have hE : E.transpose * E = 1 := hiddenFrame_gram h
  constructor
  · have hr := rank_sandwich E (Matrix.of D.v).transpose hE (by simpa using D.gram_v)
      (Matrix.diagonal (fun j : Fin r => if j.val < k then (1 : ℝ) else 0))
    simp only [Matrix.transpose_transpose] at hr
    change (E * Matrix.diagonal
      (fun j : Fin r => if j.val < k then (1 : ℝ) else 0) * (Matrix.of D.v)).rank = k
    rw [hr]
    exact rank_cutoff_diagonal (fun _ => 1) (fun _ => one_ne_zero) hk
  · have hr := rank_sandwich (Matrix.of D.u).transpose E (by simpa using D.gram_u) hE
      (Matrix.diagonal (D.prefixCoefficients k))
    change ((Matrix.of D.u).transpose * Matrix.diagonal
      (D.prefixCoefficients k) * E.transpose).rank = k
    rw [hr]
    exact rank_cutoff_diagonal D.sigma (fun j => (D.sigma_pos j).ne') hk

/-- Every requested rung has an actual witness with the stated factor ranks. -/
theorem rung_nonempty {H k : ℕ} (h : r ≤ H) (hk : k ≤ r) :
    ∃ w : Parameters I O H, D.OnRung k w ∧ w.1.rank = k ∧ w.2.rank = k :=
  ⟨D.rungPoint h k, D.rungPoint_mem h k, D.rungPoint_ranks h ⟩hk
end O77.RungGeometry0906.SingularData

namespace O77.RungCompletion
```

44.3 Differentiating the native loss

For arbitrary $w = (A, B)$ and $d = (\dot{A}, \dot{B})$, let $E = BA - \Phi$ and $X = \dot{B}A + B\dot{A}$. Each entry of the product error on $w + td$ has derivative X_{ij} at zero. The derivative of its squared Frobenius norm is $2\sum_{i,j} E_{ij}X_{ij}$, so

$$DL_{\Phi}(w)[d] = \operatorname{tr}(E^{\mathsf{T}}X).$$

The code differentiates the finite sums by `HasDerivAt.fun_sum`, then compares the resulting scalar line derivative with the Fréchet chain rule. Uniqueness of derivatives identifies the two; no definition of a replacement gradient is introduced.

```
/-- The first derivative of the actual native loss, on an arbitrary direction.
Trace is the existing Frobenius pairing; no gradient is introduced by definition. -/
lemma loss_fderiv_apply {I O H : ℕ} (Phi : RM O I) (w d : Parameters I O H) :
    fderiv ℝ (loss Phi) w d =
```

```
      Matrix.trace ((w.2 * w.1 - Phi).transpose * (d.2 * w.1 + w.2 * d.1)) := by
  let E := w.2 * w.1 - Phi
  let X := d.2 * w.1 + w.2 * d.1
  have hentry (i : Fin O) (j : Fin I) :
      HasDerivAt (fun t : ℝ => ((w + t • d).2 * (w + t • d).1 - Phi) i j)
        (X i j) 0 := by
    have h := (HasDerivAt.fun_sum (u := Finset.univ) (fun a _ =>
      ((((hasDerivAt_id (0 : ℝ)).mul_const (d.2 i a)).const_add (w.2 i a)).mul
        (((hasDerivAt_id (0 : ℝ)).mul_const (d.1 a j)).const_add (w.1 a j))))).sub_const
      (Phi i j)
    simpa [X, Matrix.mul_apply, Matrix.sub_apply, Matrix.add_apply,
      Finset.sum_add_distrib, Pi.add_apply, Pi.smul_apply, smul_eq_mul] using h
  have hnorm : HasDerivAt
      (fun t : ℝ => frobeniusSq ((w + t • d).2 * (w + t • d).1 - Phi))
      ∑( i, ∑ j, 2 * E i j * X i j) 0 := by
    have h := HasDerivAt.fun_sum (u := Finset.univ) (fun i _ =>
      HasDerivAt.fun_sum (u := Finset.univ) (fun j _ => (hentry i j).pow 2))
    simpa [frobeniusSq, E] using h
  have hline : HasDerivAt (fun t : ℝ => loss Phi (w + t • d))
      (Matrix.trace (E.transpose * X)) 0 := by
    change HasDerivAt
      (fun t : ℝ => frobeniusSq ((w + t • d).2 * (w + t • d).1 - Phi) / 2)
      (Matrix.trace (E.transpose * X)) 0
    exact (hnorm.div_const 2).congr_deriv (by
      rw [trace_transpose_mul_eq_sum]
      simp only [mul_assoc, ← Finset.mul_sum]
      ring)
  have hpath : HasDerivAt (fun t : ℝ => w + t • d) d 0 := by
    exact (((hasDerivAt_id' (0 : ℝ)).smul_const d).const_add w).congr_deriv
      (one_smul ℝ d)
  have hf : HasFDerivAt (loss Phi) (fderiv ℝ (loss Phi) w) w :=
    (((contDiff_native_loss Phi).differentiable (by norm_num)) w).hasFDerivAt
  have hc : HasDerivAt (fun t : ℝ => loss Phi (w + t • d))
      (fderiv ℝ (loss Phi) w d) 0 :=
    hf.comp_hasDerivAt_of_eq 0 hpath (by simp only [zero_smul, add_zero])
  exact hc.unique hline
```

If $B^{\mathsf{T}}E = 0$ and $EA^{\mathsf{T}} = 0$, transposition gives $E^{\mathsf{T}}B = 0$ and $AE^{\mathsf{T}} = 0$. Expand the derivative trace into its two terms and cyclically move A in the first term. Both traces vanish. Extensionality in d proves the actual derivative is the zero continuous linear map. Applied to the rung equations, this yields, together with the previous rank and level calculations, $DL_{\Phi}(w) = 0$, $\operatorname{rank}(BA) = k$, and $L_{\Phi}(w) = \ell_k$.

```
lemma loss_fderiv_eq_zero_of_products {I O H : ℕ} (Phi : RM O I)
    (w : Parameters I O H)
    (hB : w.2.transpose * (w.2 * w.1 - Phi) = 0)
    (hA : (w.2 * w.1 - Phi) * w.1.transpose = 0) :
    fderiv ℝ (loss Phi) w = 0 := by
  let E := w.2 * w.1 - Phi
  have hEtB : E.transpose * w.2 = 0 := by
    simpa [E, Matrix.transpose_mul] using congrArg Matrix.transpose hB
  have hAEt : w.1 * E.transpose = 0 := by
    simpa [E, Matrix.transpose_mul] using congrArg Matrix.transpose hA
  apply ContinuousLinearMap.ext
  intro d
  rw [loss_fderiv_apply, Matrix.mul_add, Matrix.trace_add]
  change Matrix.trace (E.transpose * (d.2 * w.1)) +
    Matrix.trace (E.transpose * (w.2 * d.1)) = 0
  rw [Matrix.trace_mul_cycle', ← Matrix.mul_assoc, hAEt, Matrix.zero_mul,
    Matrix.trace_zero, ← Matrix.mul_assoc, hEtB, Matrix.zero_mul, Matrix.trace_zero]
  simp
```

```
/-- Rank, level and criticality now refer to the actual native matrices. -/
theorem rung_critical_rank_level {I O H r k : ℕ}
    (D : O77.RungGeometry0906.SingularData I O r) (hk : k ≤ r)
    (w : Parameters I O H) (hw : D.OnRung k w) :
    fderiv ℝ (loss D.target) w = 0 ∧
      (w.2 * w.1).rank = k ∧ loss D.target w = D.rungLevel k := by
  have hp := D.critical_products hw
  refine ⟨loss_fderiv_eq_zero_of_products D.target w hp.1 hp.2, ?_, D.loss_onRung ⟩hw
  rw [hw.1]
  exact D.prefix_rank_eq hk
```

Nonvacuity and the measured conclusion are separate. For $r < H$, the first conjunct supplies an actual point on every rung $0 \le k \le r$; it follows from the stronger witness with both factor ranks k. The second conjunct quantifies over every point on every nonoptimal rung and invokes the established native band theorem. No witness or local pair is assumed as a field of the singular data. When $r = 0$, the second conjunct is vacuous but the terminal exact-fit rung is still inhabited; the last example checks precisely this boundary.

```
/-- The complete requested saddle clause: nonvacuity and the measured conclusion
are separate conjuncts; neither is a field assumed in the input. -/
theorem saddle_rungs_nonempty_and_pair {I O H r : ℕ}
    (D : O77.RungGeometry0906.SingularData I O r) (hH : r < H) :
    ∀( k : ℕ, k ≤ r → ∃ w : Parameters I O H, D.OnRung k w) ∧
    ∀( k : ℕ, k < r → ∀ w : Parameters I O H, D.OnRung k w →
      O77.Native.HasLocalBandPair D.target w 1 1) := by
  constructor
  · intro k hk
    obtain ⟨w, hw, _, ⟩_ := D.rung_nonempty hH.le hk
    exact ⟨w, ⟩hw
  · intro k hk w hw
    exact O77.SaddleAttachment0906.rung_hasLocalBandPair D hH hk w hw

-- Boundary regression: the empty singular expansion still has a terminal exact-fit rung.
example {I O H : ℕ} (D : O77.RungGeometry0906.SingularData I O 0) :
    ∃ w : Parameters I O H, D.OnRung 0 w ∧ w.1.rank = 0 ∧ w.2.rank = 0 :=
  D.rung_nonempty (Nat.zero_le H) le_rfl

end O77.RungCompletion
end
```

45 Module `O77PowerLogUniqueness`

We now justify that a measured power-log pair is an invariant. For real exponents λ, μ, natural degrees k, l, and $\epsilon > 0$, write $g_{\lambda,k}(\epsilon) = \epsilon^{\lambda}(\log(1/\epsilon))^k$ and let `EThetaZeroPowerLogVlamk` mean positive finite two-sided comparison constants for V and this gauge at all sufficiently small positive ϵ. The gauge degree is $k = m - 1$ when the public multiplicity is m. The argument works for any function $V : \mathbb{R} \to [0, \infty]$, independently of continuity, measurability, or nonconstancy.

```
import Mathlib
import O77Transport

set_option autoImplicit false
noncomputable section
open Filter Set
open scoped Topology ENNReal

namespace O77.PowerLogUniqueness
```

```
open O77.Residual
```

45.1 Separating powers before logarithms

Substitute $\epsilon = e^{-t}$. The exact identity $g_{\lambda,k}(e^{-t}) = e^{-\lambda t}t^k$ follows from the positive-base definition of real powers and $\log(e^t) = t$. For $t > 0$ the gauge is strictly positive. Dividing two such expressions gives

$$\frac{g_{\lambda,k}(e^{-t})}{g_{\mu,l}(e^{-t})} = t^{(k:\mathbb{R})-(l:\mathbb{R})}e^{-(\lambda-\mu)t}.$$

The exponent difference is formed after casting to $\mathbb{R}$; natural subtraction would incorrectly discard its sign.

```
/-- The logarithmic substitution is exact, not an asymptotic equivalence. -/
lemma gauge_exp (lam : ℝ) (k : ℕ) (t : ℝ) :
    powerLogZeroReal lam k (Real.exp (-t)) = Real.exp (-lam*t) * t^k := by
  have hi : 1 / Real.exp (-t) = Real.exp t := by
    rw [one_div, ← Real.exp_neg, neg_neg]
  simp only [powerLogZeroReal, Real.rpow_def_of_pos (Real.exp_pos (-t)),
    hi, Real.log_exp]
  congr 1
  congr 1
  ring

lemma gauge_exp_pos (lam : ℝ) (k : ℕ) {t : ℝ} (ht : 0 < t) :
    0 < powerLogZeroReal lam k (Real.exp (-t)) := by
  rw [gauge_exp]
  exact mul_pos (Real.exp_pos _) (pow_pos ht _)

/-- Signed exponent differences are formed in ℝ, never by natural subtraction. -/
lemma gauge_exp_ratio (lam mu : ℝ) (k l : ℕ) {t : ℝ} (ht : 0 < t) :
    powerLogZeroReal lam k (Real.exp (-t)) /
        powerLogZeroReal mu l (Real.exp (-t)) =
      t^((k : ℝ)-(l : ℝ)) * Real.exp (-(lam-mu)*t) := by
  rw [gauge_exp, gauge_exp]
  calc
    (Real.exp (-lam*t) * t^k) / (Real.exp (-mu*t) * t^l) =
        (t^k / t^l) * (Real.exp (-lam*t) / Real.exp (-mu*t)) := by ring
    _ = t^((k : ℝ)-(l : ℝ)) * Real.exp ((-lam*t)-(-mu*t)) := by
      rw [Real.rpow_sub ht, Real.rpow_natCast, Real.rpow_natCast, Real.exp_sub]
    _ = _ := by congr 2; ring
```

If $\mu < \lambda$, exponential decay dominates the arbitrary real power of t, so the ratio tends to zero at infinity. The invoked theorem is `tendsto_rpow_mul_exp_neg_mul_atTop_nhds_zero`, with positive decay coefficient $\lambda - \mu$. At equal thresholds, if $k < l$, the ratio is $t^{-((l:\mathbb{R})-(k:\mathbb{R}))}$; it tends to zero by `tendsto_rpow_neg_atTop`. Each identity need hold only eventually, where $t > 0$.

```
/-- A larger threshold makes the ratio tend to zero, whatever the log degrees. -/
lemma ratio_tendsto_zero_threshold {lam mu : ℝ} (h : mu < lam) (k l : ℕ) :
    Tendsto (fun t : ℝ => powerLogZeroReal lam k (Real.exp (-t)) /
      powerLogZeroReal mu l (Real.exp (-t))) atTop (nhds 0) := by
  apply (tendsto_rpow_mul_exp_neg_mul_atTop_nhds_zero
    ((k : ℝ)-(l : ℝ)) (lam-mu) (sub_pos.mpr h)).congr'
  filter_upwards [eventually_gt_atTop (0 : ℝ)] with t ht
  exact (gauge_exp_ratio lam mu k l ht).symm

/-- At a fixed threshold a smaller logarithmic degree makes the ratio vanish. -/
lemma ratio_tendsto_zero_degree (lam : ℝ) {k l : ℕ} (h : k < l) :
    Tendsto (fun t : ℝ => powerLogZeroReal lam k (Real.exp (-t)) /
```

```
      powerLogZeroReal lam l (Real.exp (-t))) atTop (nhds 0) := by
  have hp : 0 < (l : ℝ)-(k : ℝ) := sub_pos.mpr (by exact_mod_cast h)
  apply (tendsto_rpow_neg_atTop hp).congr'
  filter_upwards [eventually_gt_atTop (0 : ℝ)] with t ht
  rw [gauge_exp_ratio lam lam k l ht]
  simp only [sub_self, neg_zero, zero_mul, Real.exp_zero, mul_one]
  congr 1
  ring
```

A real function that is eventually at least $c > 0$ cannot tend to zero: convergence would also make it eventually less than c, and the intersection of these two eventual conditions is nonempty. This elementary contradiction is the sole separation mechanism below.

```
private lemma not_vanishing_of_lower_bound {f : ℝ → ℝ} {c : ℝ}
    (hc : 0 < c) (hb : ∀ᶠ t in atTop, c ≤ f t)
    (hf : Tendsto f atTop (nhds 0)) : False := by
  have hs : ∀ᶠ t in atTop, f t < c := hf.eventually (gt_mem_nhds hc)
  obtain ⟨t, hlo, ⟩hhi := (hb.and hs).exists
  exact (not_lt_of_ge hlo) hhi
```

Suppose V has two power-log bounds. Denote the first upper constant by $A > 0$ and the second lower constant by $b > 0$. Once e^{-t} is below both cutoffs, $b\, g_{\mu,l}(e^{-t}) \le V(e^{-t}) \le A\, g_{\lambda,k}(e^{-t})$. Only the finite nonnegative expressions on the two sides are converted to real inequalities via `ENNReal.ofReal_le_ofReal_iff`; V itself is never converted to $\mathbb{R}$. Positivity of the denominator gauge then gives the eventual lower bound b/A for their ratio. Reversing the two bounds gives an analogous lower bound for the reciprocal ratio.

```
/-- Comparing two bounds for the same extended-real function. No conversion of
V to ℝ occurs; only the two finite, nonnegative comparison expressions are read in ℝ. -/
lemma ratio_eventually_bounded_below {V : ℝ → ENNReal} {lam mu : ℝ} {k l : ℕ}
    (hV : EThetaZeroPowerLog V lam k) (hW : EThetaZeroPowerLog V mu l) :
    ∃ c : ℝ, 0 < c ∧ ∀ᶠ t in atTop,
      c ≤ powerLogZeroReal lam k (Real.exp (-t)) /
        powerLogZeroReal mu l (Real.exp (-t)) := by
  obtain ⟨a, A, e, ha, haA, he, _, ⟩hb := hV
  obtain ⟨b, _, d, hb0, _, hd, _, ⟩hc := hW
  have hA : 0 < A := ha.trans_le haA
  refine ⟨b/A, div_pos hb0 hA, ?⟩_
  filter_upwards [eventually_gt_atTop (0 : ℝ),
    Real.tendsto_exp_neg_atTop_nhds_zero.eventually (gt_mem_nhds he),
    Real.tendsto_exp_neg_atTop_nhds_zero.eventually (gt_mem_nhds hd)] with t ht he' hd'
  have hcross := (hc _ (Real.exp_pos _) hd').1.trans (hb _ (Real.exp_pos _) he').2
  have hright := (mul_pos hA (gauge_exp_pos lam k ht)).le
  have hreal : b * powerLogZeroReal mu l (Real.exp (-t)) ≤
      A * powerLogZeroReal lam k (Real.exp (-t)) := by
    apply (ENNReal.ofReal_le_ofReal_iff hright).mp
    simpa only [powerLogZero, ENNReal.ofReal_mul hb0.le,
      ENNReal.ofReal_mul hA.le] using hcross
  apply (₀div_le_div_iff hA (gauge_exp_pos mu l ht)).mpr
  simpa only [mul_comm A] using hreal
```

Both ratios are bounded below by positive constants. If either threshold were larger than the other, its ratio would tend to zero, a contradiction. Thus the thresholds agree. Trichotomy of the natural degrees and the equal-threshold limit then force the degrees to agree as well. This proves fixed-window uniqueness without presupposing that either threshold is positive or that the function comes from a volume.

```
/-- Uniqueness at one fixed window; neither regularity nor nonconstancy of V is required. -/
theorem fixed_order_unique {V : ℝ → ENNReal} {lam mu : ℝ} {k l : ℕ}
    (hV : EThetaZeroPowerLog V lam k) (hW : EThetaZeroPowerLog V mu l) :
    lam = mu ∧ k = l := by
```

```
  obtain ⟨c, hc, ⟩hlower := ratio_eventually_bounded_below hV hW
  obtain ⟨d, hd, ⟩hupper := ratio_eventually_bounded_below hW hV
  have hlm : lam ≤ mu := by
    by_contra h
    exact not_vanishing_of_lower_bound hc hlower
      (ratio_tendsto_zero_threshold (lt_of_not_ge h) k l)
  have hml : mu ≤ lam := by
    by_contra h
    exact not_vanishing_of_lower_bound hd hupper
      (ratio_tendsto_zero_threshold (lt_of_not_ge h) l k)
  have he : lam = mu := le_antisymm hlm hml
  subst mu
  refine ⟨rfl, ?_⟩
  rcases lt_trichotomy k l with h | h | h
  · exact False.elim (not_vanishing_of_lower_bound hc hlower
      (ratio_tendsto_zero_degree lam h))
  · exact h
  · exact False.elim (not_vanishing_of_lower_bound hd hupper
      (ratio_tendsto_zero_degree lam h))
```

For two radius-first pairs with radius witnesses $R, S > 0$, choose the single shared radius $\delta = \min(R, S)/2$. The fixed-window theorem identifies $\lambda = \mu$ and $m - 1 = n - 1$ there. The public guards $m, n \geq 1$ are essential to recover $m = n$ from this natural-subtraction equality. The scalar regression at the end says that degrees zero and one cannot both describe the same function at threshold $1/2$; suppressing a genuine logarithm changes the invariant.

```
/-- A single radius below both witnesses suffices; the hypotheses m,n ≥ 1
are indispensable when recovering the multiplicity from its natural degree m-1. -/
theorem radius_order_unique {V : ℝ → ℝ → ENNReal} {lam mu : ℝ} {m n : ℕ}
    (hV : O77.Transport0906.RadiusOrder V lam m)
    (hW : O77.Transport0906.RadiusOrder V mu n) : lam = mu ∧ m = n := by
  obtain ⟨_, hm, R, hR, ⟩hVR := hV
  obtain ⟨_, hn, S, hS, ⟩hWS := hW
  let delta := min R S / 2
  have hmin : 0 < min R S := lt_min hR hS
  have hdelta : 0 < delta := half_pos hmin
  have hdr : delta < R := (half_lt_self hmin).trans_le (min_le_left R S)
  have hds : delta < S := (half_lt_self hmin).trans_le (min_le_right R S)
  obtain ⟨hl, ⟩hk := fixed_order_unique (hVR delta hdelta hdr) (hWS delta hdelta hds)
  exact ⟨hl, by ⟩omega

-- Regression: suppressing the logarithmic multiplicity is not admissible.
example {V : ℝ → ENNReal} (h0 : EThetaZeroPowerLog V (1/2 : ℝ) 0)
    (h1 : EThetaZeroPowerLog V (1/2 : ℝ) 1) : False := by
  have h := (fixed_order_unique h0 h1).2
  omega

end O77.PowerLogUniqueness
end
```

46 Module `O77SemanticMinimum`

Fix $I, O, H \in \mathbb{N}$ and $\Phi \in \mathbb{R}^{O \times I}$, and put $r = \operatorname{rank} \Phi$. The carrier is the original parameter space $\mathbb{R}^{H \times I} \times \mathbb{R}^{O \times H}$, with its actual band-volume predicate. Retain the numerical candidates $\Lambda(a, b)$ and $\mathfrak{m}(a, b)$ from Part I, for natural ranks a, b. Their code definitions are read as follows. Evaluate `candidateTwiceLambda` at (I, O, H, r, a, b), cast the result to $\mathbb{R}$ and divide by two to obtain $\Lambda(a, b)$. Evaluate `candidateMultiplicity` at the same six arguments to obtain $\mathfrak{m}(a, b)$. The generic lemmas in this module take a pointwise proof of these pairs on the fixed fiber.

Their Wishart wrappers instantiate that premise; the final unconditional Wishart proof is supplied later in the development. No such premise is inserted into the definition of a measured minimum.

```
import Mathlib
import O77FiberClassification
import O77PowerLogUniqueness

set_option autoImplicit false
noncomputable section
open Set
open scoped Matrix.Norms.Elementwise

namespace O77.Native
open O77.Reconstruction0905 (RM)
```

46.1 The measured relation and its minimum

The power-log separation theorem applies directly to `HasLocalBandPair`, and proves uniqueness at any center, including positive-loss centers. Define `IsThresholdMinimizerPhiw` to mean that w is an exact fit, has some actual measured pair (λ, m), and that $\lambda \le \mu$ for every actual measured pair (μ, n) at every exact fit z over this same fixed Φ. This definition contains neither Λ nor $\mathfrak{m}$.

```
/-- Native uniqueness at any center, not just on the exact-fit fiber. -/
theorem hasLocalBandPair_unique {I O H : ℕ} {Phi : RM O I}
    {w : Parameters I O H} {lam mu : ℝ} {m n : ℕ}
    (hp : HasLocalBandPair Phi w lam m) (hq : HasLocalBandPair Phi w mu n) :
    lam = mu ∧ m = n :=
  O77.PowerLogUniqueness.radius_order_unique hp hq

/-- Semantic threshold minimality. Its definition mentions only actual fits and
actual measured pairs, not the candidate formula or a chosen RLCT function. -/
def IsThresholdMinimizer {I O H : ℕ} (Phi : RM O I) (w : Parameters I O H) : Prop :=
  w.2*w.1 = Phi ∧ ∃ lam : ℝ, ∃ m : ℕ, HasLocalBandPair Phi w lam m ∧
    ∀ z : Parameters I O H, z.2*z.1 = Phi → ∀ mu : ℝ, ∀ n : ℕ,
      HasLocalBandPair Phi z mu n → lam ≤ mu
```

Suppose every exact fit z has measured pair $(\Lambda(\operatorname{rank} A_z, \operatorname{rank} B_z), \mathfrak{m}(\operatorname{rank} A_z, \operatorname{rank} B_z))$. At any fixed exact fit, uniqueness shows that having a measured pair (λ, m) is equivalent to equality with these two candidates. The forward implication compares actual pairs; the reverse substitutes their equality into the supplied existence proof. The next wrapper supplies the pointwise proof through Wishart.

```
/-- This upgrades the existence formula to a classification of the measured relation. -/
theorem exact_fit_pair_iff_of_pairs {I O H : ℕ} (Phi : RM O I)
    (hPairs : ∀ w : Parameters I O H, w.2*w.1 = Phi →
      HasLocalBandPair Phi w
        ((O77.candidateTwiceLambda I O H Phi.rank w.1.rank w.2.rank : ℝ)/2)
        (O77.candidateMultiplicity I O H Phi.rank w.1.rank w.2.rank))
    {w : Parameters I O H} (hfit : w.2*w.1 = Phi)
    (lam : ℝ) (m : ℕ) :
    HasLocalBandPair Phi w lam m ↔
      lam = (O77.candidateTwiceLambda I O H Phi.rank w.1.rank w.2.rank : ℝ)/2 ∧
      m = O77.candidateMultiplicity I O H Phi.rank w.1.rank w.2.rank := by
  have hp := hPairs w hfit
  constructor
  · intro h
    exact hasLocalBandPair_unique h hp
  · rintro ⟨rfl, rfl⟩
```

```
    exact hp

/-- Compatibility wrapper: instantiate the fixed-target pair hypothesis using Wishart. -/
theorem exact_fit_pair_iff (hW : O77.External.RealWishartEigenvalueFormula)
    {I O H : ℕ} {Phi : RM O I} {w : Parameters I O H} (hfit : w.2*w.1 = Phi)
    (lam : ℝ) (m : ℕ) :
    HasLocalBandPair Phi w lam m ↔
      lam = (O77.candidateTwiceLambda I O H Phi.rank w.1.rank w.2.rank : ℝ)/2 ∧
      m = O77.candidateMultiplicity I O H Phi.rank w.1.rank w.2.rank := by
  exact exact_fit_pair_iff_of_pairs Phi
    (fun z hz => O77.FiberClassification.exact_fit_pair_of_wishart hW hz) hfit lam m
```

If at the point one residual dimension is zero, the established neutral residual theorem supplies the candidate pair directly. Applying the same uniqueness argument gives this unconditional pointwise classification. It says nothing by itself about global minimality: the regular quadratic variables still contribute to the threshold. The following cast lemma records that division by the positive number two and the natural-to-real embedding preserve and reflect order.

```
/-- The same identification is unconditional when the actual residual is absent. -/
theorem exact_fit_pair_iff_neutral {I O H : ℕ} {Phi : RM O I}
    {w : Parameters I O H} (hfit : w.2*w.1 = Phi)
    (hz : O-w.2.rank = 0 ∨ I-w.1.rank = 0 ∨
      O77.residualHidden H Phi.rank w.1.rank w.2.rank = 0) (lam : ℝ) (m : ℕ) :
    HasLocalBandPair Phi w lam m ↔
      lam = (O77.candidateTwiceLambda I O H Phi.rank w.1.rank w.2.rank : ℝ)/2 ∧
      m = O77.candidateMultiplicity I O H Phi.rank w.1.rank w.2.rank := by
  have hp := O77.FiberClassification.exact_fit_pair_neutral hfit hz
  exact ⟨fun h => hasLocalBandPair_unique h hp, by rintro ⟨rfl, ⟩rfl; exact ⟩hp

/-- Casting and division by two preserve the numerical order in both directions. -/
private lemma natCast_half_le_iff {a b : ℕ} :
    (a : ℝ)/2 ≤ (b : ℝ)/2 ↔ a ≤ b := by
  rw [div_le_div_iff_of_pos_right (zero_lt_two : (0 : ℝ) < 2), Nat.cast_le]
```

Why numerical competitors must be realized. Measured minimality is equivalent to the numerical minimum condition on the ranks. For the forward direction, take any feasible pair (a, b). `O77.FiberClassification.feasible_iff_realized` supplies actual matrices z with those ranks over the *same* target Φ. Their candidate measured pair is available, so the measured minimum inequality applies to them. Uniqueness identifies the threshold at w, and the cast/division lemma gives the numerical inequality. Conversely, for any actual competitor z and any of its measured pairs, uniqueness identifies its threshold with the candidate; its actual ranks are feasible, so the numerical minimum inequality applies. This proves both directions without replacing the fiber by an unproved rank model. The following Wishart wrapper changes only how the pointwise pair proofs are supplied.

```
/-- Realization is used in the forward implication: every numerical competitor
must be represented by an actual point over this same fixed target. -/
theorem minimum_iff_numerical_of_pairs {I O H : ℕ} (Phi : RM O I)
    (hPairs : ∀ w : Parameters I O H, w.2*w.1 = Phi →
      HasLocalBandPair Phi w
        ((O77.candidateTwiceLambda I O H Phi.rank w.1.rank w.2.rank : ℝ)/2)
        (O77.candidateMultiplicity I O H Phi.rank w.1.rank w.2.rank))
    {w : Parameters I O H} (hfit : w.2*w.1 = Phi) :
    IsThresholdMinimizer Phi w ↔
      O77.IsNumericalThresholdMinimizer I O H Phi.rank w.1.rank w.2.rank := by
  have D := O77.Adapted0906.rankData_of_exact_fit hfit
  constructor
  · rintro ⟨_, lam, m, hp, ⟩hmin
```

```
    obtain ⟨hlam, ⟩_ := (exact_fit_pair_iff_of_pairs Phi hPairs hfit lam m).mp hp
    refine ⟨D, ?⟩_
    intro a b Dab
    obtain ⟨z, hzf, hza, ⟩hzb :=
      (O77.FiberClassification.feasible_iff_realized Phi a b).mp Dab
    have hz := hPairs z hzf
    have hle := hmin z hzf _ _ hz
    rw [hlam, hza, hzb] at hle
    exact natCast_half_le_iff.mp hle
  · intro hnum
    refine ⟨hfit, _, _, hPairs w hfit, ?⟩_
    intro z hzf mu n hp
    obtain ⟨hmu, ⟩_ := (exact_fit_pair_iff_of_pairs Phi hPairs hzf mu n).mp hp
    rw [hmu]
    have hle := hnum.2 z.1.rank z.2.rank (O77.Adapted0906.rankData_of_exact_fit hzf)
    exact natCast_half_le_iff.mpr hle

/-- Compatibility wrapper: instantiate the fixed-target pair hypothesis using Wishart. -/
theorem minimum_iff_numerical (hW : O77.External.RealWishartEigenvalueFormula)
    {I O H : ℕ} {Phi : RM O I} {w : Parameters I O H} (hfit : w.2*w.1 = Phi) :
    IsThresholdMinimizer Phi w ↔
      O77.IsNumericalThresholdMinimizer I O H Phi.rank w.1.rank w.2.rank := by
  exact minimum_iff_numerical_of_pairs Phi
    (fun z hz => O77.FiberClassification.exact_fit_pair_of_wishart hW hz) hfit
```

Compose the preceding equivalence with `O77.numerical_minimizer_iff_closed`, using feasibility of the actual ranks. The result identifies the entire measured minimum locus with `MinimumRankCondition`. This uses the established finite classification, rather than introducing a new case split or asserting that the minimum locus has constant multiplicity.

```
/-- The finite classification is imported, not reproved using a new case split. -/
theorem minimum_iff_closed_of_pairs {I O H : ℕ} (Phi : RM O I)
    (hPairs : ∀ w : Parameters I O H, w.2*w.1 = Phi →
      HasLocalBandPair Phi w
        ((O77.candidateTwiceLambda I O H Phi.rank w.1.rank w.2.rank : ℝ)/2)
        (O77.candidateMultiplicity I O H Phi.rank w.1.rank w.2.rank))
    {w : Parameters I O H} (hfit : w.2*w.1 = Phi) :
    IsThresholdMinimizer Phi w ↔
      O77.MinimumRankCondition I O H Phi.rank w.1.rank w.2.rank :=
  (minimum_iff_numerical_of_pairs Phi hPairs hfit).trans
    (O77.numerical_minimizer_iff_closed (O77.Adapted0906.rankData_of_exact_fit hfit))

/-- Compatibility wrapper: instantiate the fixed-target pair hypothesis using Wishart. -/
theorem minimum_iff_closed (hW : O77.External.RealWishartEigenvalueFormula)
    {I O H : ℕ} {Phi : RM O I} {w : Parameters I O H} (hfit : w.2*w.1 = Phi) :
    IsThresholdMinimizer Phi w ↔
      O77.MinimumRankCondition I O H Phi.rank w.1.rank w.2.rank := by
  exact minimum_iff_closed_of_pairs Phi
    (fun z hz => O77.FiberClassification.exact_fit_pair_of_wishart hW hz) hfit
```

46.2 Attainment is part of both extrema

Assume $r \le H$. The inequalities $r \le I, O$ come from the actual matrix rank; thus (r, r) is feasible. Fixed-target realization produces a point with both factor ranks r. The pointwise pair theorem supplies its measured pair, while `O77.baseline_minimizes_threshold_maximizes_multiplicity` proves that its ranks minimize the numerical threshold. The established equivalence makes this actual point a measured minimizer. The existential statement retains its exact fit, both ranks, pair, and minimality together. Its wrapper again merely supplies Wishart.

```
/-- An actual baseline point exists before it is used as a minimizing witness. -/
```

```
theorem baseline_attained_of_pairs {I O H : ℕ} (Phi : RM O I)
    (hPairs : ∀ w : Parameters I O H, w.2*w.1 = Phi →
      HasLocalBandPair Phi w
        ((O77.candidateTwiceLambda I O H Phi.rank w.1.rank w.2.rank : ℝ)/2)
        (O77.candidateMultiplicity I O H Phi.rank w.1.rank w.2.rank))
    (hrH : Phi.rank ≤ H) :
    ∃ w : Parameters I O H, w.2*w.1 = Phi ∧
      w.1.rank = Phi.rank ∧ w.2.rank = Phi.rank ∧
      HasLocalBandPair Phi w ((O77.baselineTwiceLambda I O H Phi.rank : ℝ)/2)
        (O77.candidateMultiplicity I O H Phi.rank Phi.rank Phi.rank) ∧
      IsThresholdMinimizer Phi w := by
  have hrI := Matrix.rank_le_width Phi
  have hrO := Matrix.rank_le_height Phi
  have D : O77.RankData I O H Phi.rank Phi.rank Phi.rank := by
    constructor <;> omega
  obtain ⟨w, hw, ha, ⟩hb :=
    (O77.FiberClassification.feasible_iff_realized (H := H) Phi Phi.rank Phi.rank).mp D
  refine ⟨w, hw, ha, hb, ?_, ?⟩_
  · simpa only [ha, hb, O77.baselineTwiceLambda] using
      hPairs w hw
  · apply (minimum_iff_numerical_of_pairs Phi hPairs hw).mpr
    rw [ha, hb]
    exact (O77.baseline_minimizes_threshold_maximizes_multiplicity hrI hrO hrH).1

/-- Compatibility wrapper: instantiate the fixed-target pair hypothesis using Wishart. -/
theorem baseline_attained (hW : O77.External.RealWishartEigenvalueFormula)
    {I O H : ℕ} (Phi : RM O I) (hrH : Phi.rank ≤ H) :
    ∃ w : Parameters I O H, w.2*w.1 = Phi ∧
      w.1.rank = Phi.rank ∧ w.2.rank = Phi.rank ∧
      HasLocalBandPair Phi w ((O77.baselineTwiceLambda I O H Phi.rank : ℝ)/2)
        (O77.candidateMultiplicity I O H Phi.rank Phi.rank Phi.rank) ∧
      IsThresholdMinimizer Phi w := by
  exact baseline_attained_of_pairs Phi
    (fun z hz => O77.FiberClassification.exact_fit_pair_of_wishart hW hz) hrH
```

For the set of all exponents occurring in measured pairs at exact fits, `IsLeast` requires both membership and the lower-bound property. The baseline witness proves membership. Its semantic minimality bounds every other member, and uniqueness aligns the threshold in that minimality witness with the explicitly computed baseline threshold. Thus the statement proves an attained minimum, not merely an infimum or a numerical lower bound detached from matrix points.

```
/-- This is IsLeast of the set of measured exponents, not a renamed numerical minimum. -/
theorem minimum_value_isLeast_of_pairs {I O H : ℕ} (Phi : RM O I)
    (hPairs : ∀ w : Parameters I O H, w.2*w.1 = Phi →
      HasLocalBandPair Phi w
        ((O77.candidateTwiceLambda I O H Phi.rank w.1.rank w.2.rank : ℝ)/2)
        (O77.candidateMultiplicity I O H Phi.rank w.1.rank w.2.rank))
    (hrH : Phi.rank ≤ H) :
    IsLeast {lam : ℝ | ∃ w : Parameters I O H, w.2*w.1 = Phi ∧
      ∃ m : ℕ, HasLocalBandPair Phi w lam m}
      ((O77.baselineTwiceLambda I O H Phi.rank : ℝ)/2) := by
  obtain ⟨w0, hf0, _, _, hp0, _, lam0, m0, hq0, ⟩hmin :=
    baseline_attained_of_pairs Phi hPairs hrH
  refine ⟨⟨w0, hf0, _, ⟩hp0, ?⟩_
  rintro lam ⟨w, hf, m, ⟩hp
  rw [(hasLocalBandPair_unique hp0 hq0).1]
  exact hmin w hf lam m hp

/-- Compatibility wrapper: instantiate the fixed-target pair hypothesis using Wishart. -/
theorem minimum_value_isLeast (hW : O77.External.RealWishartEigenvalueFormula)
    {I O H : ℕ} (Phi : RM O I) (hrH : Phi.rank ≤ H) :
```

```
    IsLeast {lam : ℝ | ∃ w : Parameters I O H, w.2*w.1 = Phi ∧
      ∃ m : ℕ, HasLocalBandPair Phi w lam m}
      ((O77.baselineTwiceLambda I O H Phi.rank : ℝ)/2) := by
  exact minimum_value_isLeast_of_pairs Phi
    (fun z hz => O77.FiberClassification.exact_fit_pair_of_wishart hW hz) hrH
```

At any exact fit, uniqueness identifies its measured multiplicity with $\mathfrak{m}(a,b)$. The finite inequality `O77.candidateMultiplicity_le_baseline`, applied to its feasible ranks, bounds this by $\mathfrak{m}(r,r)$. This result holds on the whole fiber, which is stronger than the restriction needed for the next theorem.

```
/-- The inherited bound is stronger than needed: it holds at every exact fit. -/
theorem multiplicity_le_baseline_of_pairs {I O H : ℕ} (Phi : RM O I)
    (hPairs : ∀ w : Parameters I O H, w.2*w.1 = Phi →
      HasLocalBandPair Phi w
        ((O77.candidateTwiceLambda I O H Phi.rank w.1.rank w.2.rank : ℝ)/2)
        (O77.candidateMultiplicity I O H Phi.rank w.1.rank w.2.rank))
    {w : Parameters I O H}
    (hf : w.2*w.1 = Phi) {lam : ℝ} {m : ℕ} (hp : HasLocalBandPair Phi w lam m) :
    m ≤ O77.candidateMultiplicity I O H Phi.rank Phi.rank Phi.rank := by
  rw [((exact_fit_pair_iff_of_pairs Phi hPairs hf lam m).mp hp).2]
  exact O77.candidateMultiplicity_le_baseline (O77.Adapted0906.rankData_of_exact_fit hf)

/-- Compatibility wrapper: instantiate the fixed-target pair hypothesis using Wishart. -/
theorem multiplicity_le_baseline (hW : O77.External.RealWishartEigenvalueFormula)
    {I O H : ℕ} {Phi : RM O I} {w : Parameters I O H}
    (hf : w.2*w.1 = Phi) {lam : ℝ} {m : ℕ} (hp : HasLocalBandPair Phi w lam m) :
    m ≤ O77.candidateMultiplicity I O H Phi.rank Phi.rank Phi.rank := by
  exact multiplicity_le_baseline_of_pairs Phi
    (fun z hz => O77.FiberClassification.exact_fit_pair_of_wishart hW hz) hf hp
```

For multiplicities occurring at measured threshold minimizers, `IsGreatest` again includes membership. The baseline point is both a measured minimizer and a point carrying multiplicity $\mathfrak{m}(r,r)$, so it proves attainment. The preceding bound proves maximality for every other member. This is a second optimization on the minimum locus; it does not claim that all of that locus has the maximal multiplicity.

```
/-- Maximal multiplicity is a separate attained comparison on the minimum locus. -/
theorem minimum_multiplicity_isGreatest_of_pairs {I O H : ℕ} (Phi : RM O I)
    (hPairs : ∀ w : Parameters I O H, w.2*w.1 = Phi →
      HasLocalBandPair Phi w
        ((O77.candidateTwiceLambda I O H Phi.rank w.1.rank w.2.rank : ℝ)/2)
        (O77.candidateMultiplicity I O H Phi.rank w.1.rank w.2.rank))
    (hrH : Phi.rank ≤ H) :
    IsGreatest {m : ℕ | ∃ w : Parameters I O H, IsThresholdMinimizer Phi w ∧
      ∃ lam : ℝ, HasLocalBandPair Phi w lam m}
      (O77.candidateMultiplicity I O H Phi.rank Phi.rank Phi.rank) := by
  obtain ⟨w0, _, _, _, hp0, ⟩hm0 := baseline_attained_of_pairs Phi hPairs hrH
  refine ⟨⟨w0, hm0, _, ⟩hp0, ?⟩_
  rintro m ⟨w, hw, lam, ⟩hp
  exact multiplicity_le_baseline_of_pairs Phi hPairs hw.1 hp

/-- Compatibility wrapper: instantiate the fixed-target pair hypothesis using Wishart. -/
theorem minimum_multiplicity_isGreatest (hW : O77.External.RealWishartEigenvalueFormula)
    {I O H : ℕ} (Phi : RM O I) (hrH : Phi.rank ≤ H) :
    IsGreatest {m : ℕ | ∃ w : Parameters I O H, IsThresholdMinimizer Phi w ∧
      ∃ lam : ℝ, HasLocalBandPair Phi w lam m}
      (O77.candidateMultiplicity I O H Phi.rank Phi.rank Phi.rank) := by
  exact minimum_multiplicity_isGreatest_of_pairs Phi
    (fun z hz => O77.FiberClassification.exact_fit_pair_of_wishart hW hz) hrH
```

46.3 The closed values

Put $\eta = (I + O + H + r) \bmod 2$. The numerical expression for the global threshold is

$$\begin{cases} [H(I-r) + Or]/2, & I + H < O + r, \\ [H(O-r) + Ir]/2, & O + H < I + r, \\ OI/2, & I + O < H + r, \\ [2(H+r)(I+O) - (I-O)^2 - (H+r)^2 + \eta]/8, & \text{otherwise}, \end{cases}$$

The definition follows the displayed order of tests. All differences in the real-valued expression are formed after casting. The equality with the baseline candidate follows from the existing four integer formulas: in the unbalanced branches the necessary natural-subtraction hypotheses are $r \le I$ or $r \le O$; in the balanced branch cast the entire integer identity first. That identity gives four times the doubled threshold, so the threshold denominator is eight.

```
/-- A numerical expression only. Minimality is proved by minimum_value_closed. -/
def globalThresholdFormula (I O H r : ℕ) : ℝ :=
  if I+H < O+r then ((H : ℝ)*((I : ℝ)-r)+(O : ℝ)*r)/2 else
  if O+H < I+r then ((H : ℝ)*((O : ℝ)-r)+(I : ℝ)*r)/2 else
  if I+O < H+r then ((O : ℝ)*I)/2 else
    (2*((H : ℝ)+r)*((I : ℝ)+O)-((I : ℝ)-O)^2-((H : ℝ)+r)^2+
      ((I+O+H+r)%2 : ℕ))/8

/-- The four already checked integer identities are cast before division. -/
lemma baseline_value_closed {I O H r : ℕ} (hrI : r ≤ I) (hrO : r ≤ O) (hrH : r ≤ H) :
    (O77.baselineTwiceLambda I O H r : ℝ)/2 = globalThresholdFormula I O H r := by
  unfold globalThresholdFormula
  split_ifs with hO hI hH
  · rw [O77.baselineTwiceLambda_output hrI hrO hrH hO]
    simp only [Nat.cast_add, Nat.cast_mul, Nat.cast_sub hrI]
  · rw [O77.baselineTwiceLambda_input hrI hrO hrH hI]
    simp only [Nat.cast_add, Nat.cast_mul, Nat.cast_sub hrO]
  · rw [O77.baselineTwiceLambda_hidden hrI hrO hrH hH]
    simp only [Nat.cast_mul]
  · have h := O77.baselineTwiceLambda_balanced hrI hrO hrH
      (by omega) (by omega) (by omega)
    have hcast := congrArg (fun z : ℤ => (z : ℝ)) h
    simp only [Int.cast_add, Int.cast_sub, Int.cast_mul, Int.cast_pow,
      Int.cast_natCast, Int.cast_ofNat] at hcast
    have hc : 4*(O77.baselineTwiceLambda I O H r : ℝ) =
        2*((H : ℝ)+r)*((I : ℝ)+O)-((I : ℝ)-O)^2-((H : ℝ)+r)^2+
          ((I+O+H+r)%2 : ℕ) := hcast
    linarith
```

Replace the baseline threshold in its `IsLeast` theorem by the just-proved closed expression. Both attainment and the comparison with every actual measured exponent are preserved under this equality. The following compatibility theorem instantiates the pointwise pair hypothesis through Wishart, with no further geometric premise.

```
theorem minimum_value_closed_of_pairs {I O H : ℕ} (Phi : RM O I)
    (hPairs : ∀ w : Parameters I O H, w.2*w.1 = Phi →
      HasLocalBandPair Phi w
        ((O77.candidateTwiceLambda I O H Phi.rank w.1.rank w.2.rank : ℝ)/2)
        (O77.candidateMultiplicity I O H Phi.rank w.1.rank w.2.rank))
    (hrH : Phi.rank ≤ H) :
    IsLeast {lam : ℝ | ∃ w : Parameters I O H, w.2*w.1 = Phi ∧
      ∃ m : ℕ, HasLocalBandPair Phi w lam m}
      (globalThresholdFormula I O H Phi.rank) := by
  rw ←[ baseline_value_closed (Matrix.rank_le_width Phi) (Matrix.rank_le_height Phi) hrH]
  exact minimum_value_isLeast_of_pairs Phi hPairs hrH
```

```
/-- Compatibility wrapper: instantiate the fixed-target pair hypothesis using Wishart. -/
theorem minimum_value_closed (hW : O77.External.RealWishartEigenvalueFormula)
    {I O H : ℕ} (Phi : RM O I) (hrH : Phi.rank ≤ H) :
    IsLeast {lam : ℝ | ∃ w : Parameters I O H, w.2*w.1 = Phi ∧
      ∃ m : ℕ, HasLocalBandPair Phi w lam m}
      (globalThresholdFormula I O H Phi.rank) := by
  exact minimum_value_closed_of_pairs Phi
    (fun z hz => O77.FiberClassification.exact_fit_pair_of_wishart hW hz) hrH
```

The finite formula `O77.baselineMultiplicity_formula` is 2 exactly when $I + O + H + r$ is odd and

$$O + r \le I + H, \qquad I + r \le O + H, \qquad H + r \le I + O;$$

otherwise it is 1. Rewrite the attained multiplicity maximum by this formula. Again this is an attained greatest value, not constancy of multiplicity along the minimum locus.

```
/-- A closed second-stage maximum; no constancy of multiplicity on the minimum locus is asserted. -/
theorem minimum_multiplicity_closed_of_pairs {I O H : ℕ} (Phi : RM O I)
    (hPairs : ∀ w : Parameters I O H, w.2*w.1 = Phi →
      HasLocalBandPair Phi w
        ((O77.candidateTwiceLambda I O H Phi.rank w.1.rank w.2.rank : ℝ)/2)
        (O77.candidateMultiplicity I O H Phi.rank w.1.rank w.2.rank))
    (hrH : Phi.rank ≤ H) :
    IsGreatest {m : ℕ | ∃ w : Parameters I O H, IsThresholdMinimizer Phi w ∧
      ∃ lam : ℝ, HasLocalBandPair Phi w lam m}
      (if (I+O+H+Phi.rank)%2 = 1 ∧ O+Phi.rank ≤ I+H ∧
        I+Phi.rank ≤ O+H ∧ H+Phi.rank ≤ I+O then 2 else 1) := by
  rw ←[ O77.baselineMultiplicity_formula
    (Matrix.rank_le_width Phi) (Matrix.rank_le_height Phi) hrH]
  exact minimum_multiplicity_isGreatest_of_pairs Phi hPairs hrH

/-- Compatibility wrapper: instantiate the fixed-target pair hypothesis using Wishart. -/
theorem minimum_multiplicity_closed (hW : O77.External.RealWishartEigenvalueFormula)
    {I O H : ℕ} (Phi : RM O I) (hrH : Phi.rank ≤ H) :
    IsGreatest {m : ℕ | ∃ w : Parameters I O H, IsThresholdMinimizer Phi w ∧
      ∃ lam : ℝ, HasLocalBandPair Phi w lam m}
      (if (I+O+H+Phi.rank)%2 = 1 ∧ O+Phi.rank ≤ I+H ∧
        I+Phi.rank ≤ O+H ∧ H+Phi.rank ≤ I+O then 2 else 1) := by
  exact minimum_multiplicity_closed_of_pairs Phi
    (fun z hz => O77.FiberClassification.exact_fit_pair_of_wishart hW hz) hrH
```

Three semantic regression examples. First, multiplicity zero is impossible because the measured predicate itself requires $m \ge 1$. Second, at the zero target in dimensions $I = O = H = 2$, the point $(A, B) = (I_2, 0)$ is not a threshold minimizer: the absent residual does not erase its regular contribution. Third, for each fixed rank-one 2×2 target, feasible ranks $(1, 1)$ and $(2, 1)$ are separately realized. Both points minimize the threshold 2, while their multiplicities are 2 and 1. These examples test the distinction between a minimum-threshold locus and a constant-pair locus, using actual fixed-target witnesses.

```
-- The all-dimensional extension does not weaken the multiplicity guard.
example {I O H : ℕ} (Phi : RM O I) (w : Parameters I O H) (lam : ℝ) :
    ¬ HasLocalBandPair Phi w lam 0 := by
  intro hp
  have h := hp.2.1
  omega

-- The absent residual at (1,0) does not make this point a threshold minimizer.
example (hW : O77.External.RealWishartEigenvalueFormula) :
    ¬ IsThresholdMinimizer (0 : RM 2 2) ((1 : RM 2 2), (0 : RM 2 2)) := by
```

```
  rw [minimum_iff_closed hW (by simp)]
  norm_num [O77.MinimumRankCondition]

/-- Same fixed rank-one target, same minimum threshold, two different multiplicities. -/
example (hW : O77.External.RealWishartEigenvalueFormula) (Phi : RM 2 2) (hr : Phi.rank = 1) :
    ∃ w z : Parameters 2 2 2, IsThresholdMinimizer Phi w ∧ IsThresholdMinimizer Phi z ∧
      HasLocalBandPair Phi w 2 2 ∧ HasLocalBandPair Phi z 2 1 := by
  have D0 : O77.RankData 2 2 2 Phi.rank 1 1 := by rw [hr]; constructor <;> decide
  have D1 : O77.RankData 2 2 2 Phi.rank 2 1 := by rw [hr]; constructor <;> decide
  obtain ⟨w, hw, ha, ⟩hb := (O77.FiberClassification.feasible_iff_realized Phi 1 1).mp D0
  obtain ⟨z, hz, hc, ⟩hd := (O77.FiberClassification.feasible_iff_realized Phi 2 1).mp D1
  refine ⟨w, z, ?_, ?_, ?_, ?⟩_
  · rw [minimum_iff_closed hW hw, hr, ha, hb]
    decide
  · rw [minimum_iff_closed hW hz, hr, hc, hd]
    decide
  · apply (exact_fit_pair_iff hW hw 2 2).mpr
    rw [hr, ha, hb]
    have ht : O77.candidateTwiceLambda 2 2 2 1 1 1 = 4 := by decide
    have hm : O77.candidateMultiplicity 2 2 2 1 1 1 = 2 := by decide
    norm_num [ht, hm]
  · apply (exact_fit_pair_iff hW hz 2 1).mpr
    rw [hr, hc, hd]
    have ht : O77.candidateTwiceLambda 2 2 2 1 2 1 = 4 := by decide
    have hm : O77.candidateMultiplicity 2 2 2 1 2 1 = 1 := by decide
    norm_num [ht, hm]

end O77.Native
end
```

47 Module `O77Answer`

This module defines the public answer independently of any Wishart, chart, rank-realization, or pair-existence package. The definitions used below refer to original-entry Lebesgue measure, the Frobenius loss, and the radius-first signed-band predicate already established. Its final theorem here is a reusable conditional assembly; the unconditional `O77.answersO77` later supplies the proved Wishart theorem to it.

```
import Mathlib
import O77SemanticMinimum
import O77RungCompletion

set_option autoImplicit false
noncomputable section
open Set
open scoped Matrix.Norms.Elementwise

namespace O77
open O77.Reconstruction0905 (RM)
open O77.Native (Parameters HasLocalBandPair IsThresholdMinimizer globalThresholdFormula)
```

47.1 Reading the exact six-clause contract

For the fixed target Φ, let $r = \operatorname{rank}\Phi$ and retain the candidates $\Lambda(a,b)$ and $\mathfrak{m}(a,b)$ defined above. The six conjuncts of `AnswersO77Phi`, in their literal code order, are:

1. Every actual exact fit $w = (A,B)$ has feasible ranks and a strictly positive candidate threshold; for every real λ and natural m, the measured pair predicate holds exactly

when $(\lambda, m) = (\Lambda(\operatorname{rank} A, \operatorname{rank} B), \mathfrak{m}(\operatorname{rank} A, \operatorname{rank} B))$.

2. Each numerical rank pair is feasible if and only if actual factors with exactly those ranks exist over this same target.
3. At every exact fit, measured threshold minimality is equivalent to the closed condition on its two ranks.
4. The displayed closed global threshold is an *attained* least element of the set of all measured thresholds on the fiber.
5. The displayed parity value is an *attained* greatest multiplicity among points minimizing the measured threshold.
6. For every supplied strictly ordered orthonormal singular expansion whose target equals Φ, all specified rungs are inhabited, and every point of every nonoptimal rung has measured pair $(1, 1)$.

The quantifier over singular data belongs only to the final conjunct. In particular the fiber assertions apply also to targets with repeated singular values, whereas the saddle clause retains its stated singular data setting. The exact-fit premise in clause three governs the entire parenthesized equivalence. The projection example immediately after the definition records that logical scope explicitly.

```
/-- The concrete answer contract. No external proposition, analytic chart,
rank-realization package or already-proved answer is a field of this definition.
The original problem's dimension assumptions belong to the theorem below. -/
def AnswersO77 {I O H : ℕ} (Phi : RM O I) : Prop :=
  ∀( w : Parameters I O H, w.2*w.1 = Phi →
    RankData I O H Phi.rank w.1.rank w.2.rank ∧
    0 < (candidateTwiceLambda I O H Phi.rank w.1.rank w.2.rank : ℝ)/2 ∧
    ∀ lam : ℝ, ∀ m : ℕ, HasLocalBandPair Phi w lam m ↔
      lam = (candidateTwiceLambda I O H Phi.rank w.1.rank w.2.rank : ℝ)/2 ∧
      m = candidateMultiplicity I O H Phi.rank w.1.rank w.2.rank) ∧
  ∀( a b : ℕ, RankData I O H Phi.rank a b ↔
    ∃ w : Parameters I O H, w.2*w.1 = Phi ∧ w.1.rank = a ∧ w.2.rank = b) ∧
  ∀( w : Parameters I O H, w.2*w.1 = Phi →
    (IsThresholdMinimizer Phi w ↔ MinimumRankCondition I O H Phi.rank w.1.rank w.2.rank)) ∧
  IsLeast {lam : ℝ | ∃ w : Parameters I O H, w.2*w.1 = Phi ∧
    ∃ m : ℕ, HasLocalBandPair Phi w lam m} (globalThresholdFormula I O H Phi.rank) ∧
  IsGreatest {m : ℕ | ∃ w : Parameters I O H, IsThresholdMinimizer Phi w ∧
    ∃ lam : ℝ, HasLocalBandPair Phi w lam m}
    (if (I+O+H+Phi.rank)%2 = 1 ∧ O+Phi.rank ≤ I+H ∧
      I+Phi.rank ≤ O+H ∧ H+Phi.rank ≤ I+O then 2 else 1) ∧
  ∀( D : O77.RungGeometry0906.SingularData I O Phi.rank, D.target = Phi →
    ∀( k : ℕ, k ≤ Phi.rank → ∃ w : Parameters I O H, D.OnRung k w) ∧
    ∀( k : ℕ, k < Phi.rank → ∀ w : Parameters I O H, D.OnRung k w →
      HasLocalBandPair Phi w 1 1))

-- The exact-fit premise must precede the entire minimum-locus equivalence.
-- This projection fails for the former unparenthesized implication/iff clause.
example {I O H : ℕ} {Phi : RM O I} (hAnswer : AnswersO77 (H := H) Phi)
    {w : Parameters I O H} (hfit : w.2*w.1 = Phi) :
    IsThresholdMinimizer Phi w ↔ MinimumRankCondition I O H Phi.rank w.1.rank w.2.rank :=
  hAnswer.2.2.1 w hfit
```

47.2 Assembling the mathematical results

Assume $I, O > 0$, $r < H$, and that the candidate measured pair has been proved at each actual point of this fixed fiber. The six branches below follow the six clauses verbatim. Feasibility uses actual matrix ranks; threshold positivity uses $H > 0$, which follows from $r < H$; uniqueness upgrades pair existence to the iff statement. Fixed-target realization supplies clause two.

The measured minimum classification and the two attained extrema supply clauses three to five. Finally rung completion and attachment supply clause six, and the equality between the singular expansion's target and Φ rewrites the loss. There is no need to assume any geometric conclusion in addition to the pointwise fiber pairs in this reusable assembly.

```
/-- Shared assembly from a measured pair proof on each actual point of this
fixed fiber. Feasible-rank realization, extrema, and all saddle assertions
are supplied by their existing proofs; none is a field of the hypothesis. -/
theorem answersO77_of_fiber_pairs {I O H : ℕ} (Phi : RM O I)
    (hPairs : ∀ w : Parameters I O H, w.2*w.1 = Phi →
      HasLocalBandPair Phi w
        ((candidateTwiceLambda I O H Phi.rank w.1.rank w.2.rank : ℝ)/2)
        (candidateMultiplicity I O H Phi.rank w.1.rank w.2.rank))
    (hI : 0 < I) (hO : 0 < O) (hrH : Phi.rank < H) :
    AnswersO77 (H := H) Phi := by
  have hH : 0 < H := (Nat.zero_le Phi.rank).trans_lt hrH
  refine ⟨?_, ?_, ?_, ?_, ?_, ?⟩_
  · intro w hf
    exact ⟨O77.Adapted0906.rankData_of_exact_fit hf,
      O77.FiberClassification.exact_fit_threshold_positive hI hO hH hf,
      O77.Native.exact_fit_pair_iff_of_pairs Phi hPairs ⟩hf
  · intro a b
    exact O77.FiberClassification.feasible_iff_realized Phi a b
  · intro w hf
    exact O77.Native.minimum_iff_closed_of_pairs Phi hPairs hf
  · exact O77.Native.minimum_value_closed_of_pairs Phi hPairs hrH.le
  · exact O77.Native.minimum_multiplicity_closed_of_pairs Phi hPairs hrH.le
  · intro D hD
    obtain ⟨hne, ⟩hp := O77.RungCompletion.saddle_rungs_nonempty_and_pair D hrH
    refine ⟨hne, ?⟩_
    intro k hk w hw
    simpa only [hD] using hp k hk w hw
```

The conditional wrapper takes the concrete Wishart eigenvalue formula and invokes `O77.FiberClassification.exact_fit_pair_of_wishart` at each exact fit. This supplies the sole non-dimension input to the preceding assembly. Its appearance here is an explicit theorem parameter, not an axiom or a field hidden in the answer definition. The later unconditional theorem replaces this parameter by its proved value. The closing example verifies that the zero target remains permitted on the original positive-dimensional domain, here $I = O = H = 2$.

```
/-- The inherited public conditional signature is unchanged. Its single
Wishart parameter is used only to construct the fixed-fiber measured pairs. -/
theorem answersO77_of_wishart (hW : External.RealWishartEigenvalueFormula)
    {I O H : ℕ} (Phi : RM O I) (hI : 0 < I) (hO : 0 < O) (hrH : Phi.rank < H) :
    AnswersO77 (H := H) Phi :=
  answersO77_of_fiber_pairs Phi
    (fun _ hf => O77.FiberClassification.exact_fit_pair_of_wishart hW hf) hI hO hrH

-- The zero target is allowed on the original positive-dimensional domain.
example (hW : External.RealWishartEigenvalueFormula) :
    AnswersO77 (H := 2) (0 : RM 2 2) :=
  answersO77_of_wishart hW 0 (by decide) (by decide) (by simp)

end O77
end
```

48 One-dimensional Gram spectra and squared radii

Throughout this section, d is a natural number, dx means product Lebesgue measure in the d real entries, and $\|x\|_2^2 = \sum_i x_i^2$. For $d > 0$ write $\omega_d = \mathrm{vol}\{x : \|x\|_2 < 1\}$ and $a_d = d\omega_d/2$. The scalar test and its radial integral are

$$f_{\rho,T}(s) = e^{-s}(1+Ts)^{-\rho}, \qquad M(A,\rho,T) = \int_0^\infty s^{A-1} f_{\rho,T}(s)\, ds.$$

The variables ρ, T will be nonnegative when positivity and integrability are used. The matrix integral $\mathcal{G}_{k,d}$ uses $e^{-\|X\|_F^2} \det(I_k + TXX^{\mathsf{T}})^{-\rho}$. The full-orthant Laguerre integral $\mathcal{J}_k(\eta,\rho,T)$ uses the existing sorted-extension convention, with density $e^{-\sum s_i} \prod s_i^{\eta} \prod_{i<j} |s_i - s_j| \prod (1+Ts_i)^{-\rho}$ off its null boundary. These are `leftGramLaplaceLIntegral` and `laguerreOrthantLIntegral`, respectively. Here we prove the case $k = 1$ directly, including $d = 1$.

```
import Mathlib
import O77

set_option autoImplicit false
noncomputable section
open Set MeasureTheory
open scoped ENNReal BigOperators Topology
namespace O77.RadialGram
open O77.Residual
```

The radial coefficient. The definition `radialMassd` is a_d, using the Euclidean unit ball, rather than the unit ball for the maximum norm on a function type. For $d > 0$, that ball has positive measure because it is nonempty and open. Its measure is finite because it lies in the compact closed unit ball. These two facts justify passing to `Measure.real` and prove $a_d > 0$.

```
/-- Half the surface mass, expressed through the actual Euclidean unit ball.
The zero-dimensional case is handled separately, not by this coefficient. -/
def radialMass (d : ℕ) : ℝ :=
  ((d : ℝ) / 2) * (volume : Measure (EuclideanSpace ℝ (Fin d))).real
    (Metric.ball 0 1)

lemma radialMass_pos {d : ℕ} (hd : 0 < d) : 0 < radialMass d := by
  have hv : 0 < (volume : Measure (EuclideanSpace ℝ (Fin d))) (Metric.ball 0 1) :=
    Metric.isOpen_ball.measure_pos volume (0, by )simp
  have hfin : (volume : Measure (EuclideanSpace ℝ (Fin d))) (Metric.ball 0 1) < ⊤ :=
    (measure_mono Metric.ball_subset_closedBall).trans_lt
      (isCompact_closedBall (0 : EuclideanSpace ℝ (Fin d)) 1).measure_lt_top
  change 0 < ((d : ℝ) / 2) * _
  exact mul_pos (by positivity) (ENNReal.toReal_pos hv.ne' hfin.ne)
```

Squared-radius integration. For any nontrivial finite-dimensional real normed space E, any additive Haar measure μ, and any real-valued function f, the displayed identity is

$$\int_E f(\|x\|^2)\, d\mu(x) = \frac{n}{2}\mu(B_1) \int_0^\infty s^{n/2-1} f(s)\, ds, \qquad n = \dim E.$$

The code first applies `integral_fun_norm_addHaar`, giving $n\mu(B_1)\int r^{n-1} f(r^2)\, dr$, and then `integral_comp_rpow_Ioi` with exponent 2. On $r > 0$, the Jacobian $2r$ and the identity $1 + 2(n/2 - 1) = n - 1$ give the factor $1/2$. The cast of $n - 1$ uses $n \geq 1$. This particular theorem is an equality of library Bochner integrals, whose definition is totalized; it is used below only after genuine integrability has been established. The later nonnegative version removes that restriction entirely.

```
/-- The squared-radius substitution, using the library's general polar formula.
This is an equality of Bochner integrals; integrability is supplied separately
before it is used to compute any nonnegative integral. -/
theorem integral_norm_sq {E : Type*} [NormedAddCommGroup E]
    [NormedSpace ℝ E] [FiniteDimensional ℝ E] [Nontrivial E]
    [MeasurableSpace E] [BorelSpace E]
    (mu : Measure E) [Measure.IsAddHaarMeasure mu] (f : ℝ → ℝ) :
    ∫ x : E, f ‖(‖x ^ 2) ∂mu) =
      ((Module.finrank ℝ E : ℝ) / 2) * mu.real (Metric.ball 0 1) *
        ∫ s in Ioi (0 : ℝ), s ^ ((Module.finrank ℝ E : ℝ) / 2 - 1) * f s := by
  let n := Module.finrank ℝ E
  have hn : 1 ≤ n := Module.finrank_pos
  have hpolar := integral_fun_norm_addHaar mu (fun r : ℝ => f (r ^ 2))
  have hsub := integral_comp_rpow_Ioi
    (fun s : ℝ => s ^ ((n : ℝ) / 2 - 1) * f s)
    (p := (2 : ℝ)) (by norm_num)
  have hpoint (r : ℝ) (hr : 0 < r) :
      (|(2 : ℝ)| * r ^ ((2 : ℝ) - 1)) *
          ((r ^ (2 : ℝ)) ^ ((n : ℝ) / 2 - 1) * f (r ^ (2 : ℝ))) =
        2 * (r ^ (n - 1) * f (r ^ 2)) := by
    have hexp : 1 + 2 * ((n : ℝ) / 2 - 1) = ((n - 1 : ℕ) : ℝ) := by
      rw [Nat.cast_sub hn]
      push_cast
      ring
    rw [abs_of_pos (by norm_num : (0 : ℝ) < 2),
      show (2 : ℝ) - 1 = 1 by norm_num, Real.rpow_one,
      ← Real.rpow_mul hr.le]
    calc
      2 * r * (r ^ (2 * ((n : ℝ) / 2 - 1)) * f (r ^ (2 : ℝ))) =
          2 * ((r ^ (1 : ℝ) * r ^ (2 * ((n : ℝ) / 2 - 1))) * f (r ^ 2)) := by
            rw [Real.rpow_one, Real.rpow_two]
            ring
      _ = _ := by rw ←[ Real.rpow_add hr, hexp, Real.rpow_natCast]
  have hsub' :
      2 * ∫ r in Ioi (0 : ℝ), r ^ (n - 1) * f (r ^ 2)) =
        ∫ s in Ioi (0 : ℝ), s ^ ((n : ℝ) / 2 - 1) * f s := by
    rw ←[ integral_const_mul]
    exact (setIntegral_congr_fun measurableSet_Ioi
      (fun r hr => (hpoint r hr).symm)).trans (by simpa only [smul_eq_mul] using hsub)
  simp only [nsmul_eq_mul, smul_eq_mul] at hpolar
  rw [hpolar]
  dsimp only [n] at hsub'
  rw ←[ hsub']
  ring
```

Gaussian domination. For $s, T \geq 0$, the base $1 + Ts$ is at least 1, so $f_{\rho,T}(s) \geq 0$; if also $\rho \geq 0$, then $f_{\rho,T}(s) \leq e^{-s}$. Positivity of the base also proves continuity of the norm-square test. Consequently $x \mapsto f_{\rho,T}(\sum_i x_i^2)$ is integrable: `quadratic_gaussian_integrable` for the identity matrix supplies the dominating Gaussian, and `Integrable.mono'` applies to its continuous, nonnegative subordinate test. This argument covers zero parameters and dimension zero.

```
/-- The determinant-power test after the single Gram eigenvalue is named. -/
def gramTest (rho T s : ℝ) : ℝ :=
  Real.exp (-s) * (1 + T * s) ^ (-rho)

lemma gramTest_nonneg {rho T s : ℝ} (hT : 0 ≤ T) (hs : 0 ≤ s) :
    0 ≤ gramTest rho T s := by
  exact mul_nonneg (Real.exp_pos _).le (Real.rpow_nonneg (by positivity) _)

lemma gramTest_le_gaussian {rho T s : ℝ}
    (hrho : 0 ≤ rho) (hT : 0 ≤ T) (hs : 0 ≤ s) :
```

```
    gramTest rho T s ≤ Real.exp (-s) := by
  exact mul_le_of_le_one_right (Real.exp_pos _).le
    (Model.factor_le_one hrho (by nlinarith [mul_nonneg hT hs]))

lemma gramTest_continuous_norm_sq {E : Type*} [SeminormedAddCommGroup E]
    (rho : ℝ) {T : ℝ} (hT : 0 ≤ T) :
    Continuous (fun x : E => gramTest rho T ‖(‖x ^ 2)) := by
  unfold gramTest
  apply Continuous.mul (by fun_prop)
  exact (by fun_prop : Continuous (fun x : E => 1 + T * |||x ^ 2)).rpow_const
    (fun x => Or.inl (ne_of_gt (by positivity)))

/-- The vector test is integrable by Gaussian domination, including rho = 0.
This discharges integrability rather than relying on a totalized integral. -/
lemma vector_gramTest_integrable (d : ℕ) {rho T : ℝ}
    (hrho : 0 ≤ rho) (hT : 0 ≤ T) :
    Integrable (fun x : Fin d → ℝ => gramTest rho T ∑( i, (x i) ^ 2))
      (coordinateLebesgue (Fin d)) := by
  have hM : Matrix.PosDef (1 : Matrix (Fin d) (Fin d) ℝ) := Matrix.PosDef.one
  have hG : Integrable (fun x : Fin d → ℝ => Real.exp ∑(-( i, (x i) ^ 2)))
      (coordinateLebesgue (Fin d)) := by
    simpa [dotProduct, pow_two] using quadratic_gaussian_integrable 1 hM
  have hcont : Continuous (fun x : Fin d → ℝ => gramTest rho T ∑( i, (x i) ^ 2)) := by
    unfold gramTest
    apply Continuous.mul (by fun_prop)
    exact (by fun_prop : Continuous (fun x : Fin d → ℝ => 1 + T * ∑ i, (x i) ^ 2)).rpow_const
      (fun x => Or.inl (ne_of_gt (by positivity)))
  refine hG.mono' hcont.aestronglyMeasurable ?_
  filter_upwards with x
  rw [Real.norm_of_nonneg (gramTest_nonneg hT (Finset.sum_nonneg fun i _ => sq_nonneg (x i)))]
  exact gramTest_le_gaussian hrho hT (Finset.sum_nonneg fun i _ => sq_nonneg (x i))
```

The vector integral in the specified measure. For $d > 0$, the exact equality is $\int_{\mathbb{R}^d} f_{\rho,T}(\sum x_i^2)\,dx = a_d M(d/2, \rho, T)$. The equivalence `WithLp.toLp2` carries the coordinate product measure to Euclidean volume by `PiLp.volume_preserving_toLp`; `EuclideanSpace.real_norm_sq_eq` identifies its squared norm. Applying the preceding radial identity then gives the result. The equality itself allows arbitrary real ρ and $T \geq 0$, as a Bochner identity; the next application imposes $\rho \geq 0$ to justify the nonnegative-integral conversion.

```
/-- Exact vector Gaussian test integral, with the entry measure transported
through the existing measure-preserving L2 type synonym. -/
theorem vector_gramTest_integral {d : ℕ} (hd : 0 < d)
    (rho : ℝ) {T : ℝ} (hT : 0 ≤ T) :
    ∫( x : Fin d → ℝ, gramTest rho T ∑( i, (x i) ^ 2)
      ∂coordinateLebesgue (Fin d)) =
      radialMass d * Model.integral ((d : ℝ) / 2) rho T := by
  let : NeZero d := ⟨Nat.ne_of_gt hd⟩
  have he : MeasurePreserving
      (WithLp.toLp 2 : (Fin d → ℝ) → EuclideanSpace ℝ (Fin d))
      (coordinateLebesgue (Fin d)) volume := by
    simpa only [coordinateLebesgue_eq_volume] using PiLp.volume_preserving_toLp (Fin d)
  have hchange :
      ∫( x : Fin d → ℝ, gramTest rho T ∑( i, (x i) ^ 2)
        ∂coordinateLebesgue (Fin d)) =
      ∫ x : EuclideanSpace ℝ (Fin d), gramTest rho T ‖(‖x ^ 2) := by
    rw ←[ he.map_eq]
    rw [integral_map he.measurable.aemeasurable
      (gramTest_continuous_norm_sq rho hT).aestronglyMeasurable]
    exact integral_congr_ae (.of_forall fun x => congrArg (gramTest rho T)
      (EuclideanSpace.real_norm_sq_eq (WithLp.toLp 2 x)).symm)
```

```
  rw [hchange, integral_norm_sq volume]
  simp only [finrank_euclideanSpace, Fintype.card_fin]
  change radialMass d * _ = radialMass d * _
  congr 1
  unfold Model.integral
  apply setIntegral_congr_fun measurableSet_Ioi
  intro s hs
  unfold Model.integrand gramTest
  ring
```

Exact adapters for one eigenvalue. A matrix with one row is a vector; `MeasurableEquiv.piUnique` and its measure-preservation theorem implement this identification exactly. Its Gram determinant is $1 + T\sum x_i^2$. On the eigenvalue side the one-coordinate ordered chamber is simply $(0,\infty)$, its Vandermonde product is 1, and $1! = 1$. The theorem `sortedLaguerre_lintegral_orthant` therefore identifies the inherited full-orthant integral with ofReal $M(\eta+1,\rho,T)$ when $\eta > -1$ and $\rho, T \geq 0$. The indicator calculation below proves this adapter for the actual sorted-extension definition, including its restriction to the positive ray.

```
lemma leftGram_one_eq_vector (d : ℕ) (rho T : ℝ) :
    leftGramLaplaceLIntegral 1 d rho T =
      ∫⁻ x : Fin d → ℝ, ENNReal.ofReal (gramTest rho T ∑( i, (x i) ^ 2))
        ∂coordinateLebesgue (Fin d) := by
  let e := MeasurableEquiv.piUnique (fun _ : Fin 1 => Fin d → ℝ)
  have he := measurePreserving_piUnique (fun _ : Fin 1 => coordinateLebesgue (Fin d))
  unfold leftGramLaplaceLIntegral
  rw [he.lintegral_map_equiv _ e]
  apply lintegral_congr_ae
  filter_upwards with Z
  simp [leftGramLaplaceIntegrand, gramTest, frobeniusSq,
    Matrix.mul_apply, e, MeasurableEquiv.piUnique, pow_two]

lemma laguerre_one_eq_model {eta rho T : ℝ}
    (heta : -1 < eta) (hrho : 0 ≤ rho) (hT : 0 ≤ T) :
    laguerreOrthantLIntegral 1 eta rho T =
      ENNReal.ofReal (Model.integral (eta + 1) rho T) := by
  rw [laguerreOrthantLIntegral, sortedLaguerre_lintegral_orthant heta hrho hT]
  simp only [Nat.factorial_one, Nat.cast_one, one_mul]
  congr 1
  let e := MeasurableEquiv.piUnique (fun _ : Fin 1 => ℝ)
  have he := measurePreserving_piUnique (fun _ : Fin 1 => (volume : Measure ℝ))
  have hset : orderedPositiveChamber 1 = e ⁻¹' Ioi 0 := by
    ext x
    simp [mem_orderedPositiveChamber_iff, e, MeasurableEquiv.piUnique]
  rw [laguerreChamberIntegral, ← integral_indicator (measurableSet_orderedPositiveChamber 1)]
  calc
    _ = ∫ x : Fin 1 → ℝ, (Ioi (0 : ℝ)).indicator (Model.integrand (eta + 1) rho T) (e x) := by
      apply integral_congr_ae
      filter_upwards with x
      by_cases hx : e x ∈ Ioi (0 : ℝ)
      · have hx' : x ∈ orderedPositiveChamber 1 := by simpa only [hset, Set.mem_preimage] using hx
        rw [Set.indicator_of_mem hx', Set.indicator_of_mem hx]
        simp [laguerreChamberIntegrand, laguerreChamberCore, commonRpowProduct,
          vandermondeFin, vandermondeList, laguerreWeightProduct,
          Model.integrand, e, MeasurableEquiv.piUnique]
        ring
      · have hx' : x ∉ orderedPositiveChamber 1 := by simpa [hset] using hx
        simp [Set.indicator_of_notMem hx, Set.indicator_of_notMem hx']
    _ = ∫ s : ℝ, (Ioi (0 : ℝ)).indicator (Model.integrand (eta + 1) rho T) s :=
      he.integral_comp e.measurableEmbedding _
    _ = _ := integral_indicator measurableSet_Ioi
```

The one-eigenvalue Wishart identity. Since $\eta_{1,d} = (d-2)/2$, one has $\eta_{1,d} > -1$ and $\eta_{1,d} + 1 = d/2$ for every $d > 0$. Convert the integrable, nonnegative vector test to an extended nonnegative integral with `ofReal_integral_eq_lintegral_ofReal`, apply the two adapters, and move the positive finite coefficient through `ENNReal.ofReal_mul`. This proves $\mathcal{G}_{1,d} = \mathrm{ofReal}(a_d)\mathcal{J}_1(\eta_{1,d}, \rho, T)$ with one coefficient chosen before both parameters.

```
/-- A genuine k = 1 Wishart identity in the original nonnegative integrals. -/
theorem leftGram_one_formula {d : ℕ} (hd : 0 < d)
    {rho T : ℝ} (hrho : 0 ≤ rho) (hT : 0 ≤ T) :
    leftGramLaplaceLIntegral 1 d rho T =
      ENNReal.ofReal (radialMass d) *
        laguerreOrthantLIntegral 1 (wishartEta 1 d) rho T := by
  have heta : -1 < wishartEta 1 d := wishartEta_gt_neg_one hd
  have hexp : wishartEta 1 d + 1 = (d : ℝ) / 2 := by
    unfold wishartEta
    norm_num
    ring
  rw [leftGram_one_eq_vector,
    ← ofReal_integral_eq_lintegral_ofReal (vector_gramTest_integrable d hrho hT)
      (.of_forall fun x => gramTest_nonneg hT (Finset.sum_nonneg fun i _ => sq_nonneg (x i))),
    vector_gramTest_integral hd rho hT, laguerre_one_eq_model heta hrho hT, hexp,
    ENNReal.ofReal_mul (radialMass_pos hd).le]
```

Calibration without assuming Wishart. At $\rho = T = 0$, the radial integral is Euler's gamma integral, by `Real.Gamma_eq_integral`. Independently, `quadratic_gaussian_integral` for the identity matrix evaluates the vector Gaussian as $\pi^{d/2}$. Comparing these expressions gives $a_d\Gamma(d/2) = \pi^{d/2}$, and `Real.Gamma_pos_of_pos` permits division. Thus $a_d = \pi^{d/2}/\Gamma(d/2)$. The final examples check $a_1 = 1$ using $\Gamma(1/2) = \sqrt{\pi}$ and recover the zero-parameter Gaussian mass. No parameterized integral identity is inferred merely from this calibration.

```
/-- The scalar radial integral at zero parameters is the Gamma integral. -/
lemma model_zero_parameters {A : ℝ} (hA : 0 < A) :
    Model.integral A 0 0 = Real.Gamma A := by
  rw [Real.Gamma_eq_integral hA]
  unfold Model.integral
  apply setIntegral_congr_fun measurableSet_Ioi
  intro s hs
  simp [Model.integrand, mul_comm]

/-- Calibration at T = 0 pins down the radial coefficient exactly. -/
theorem radialMass_eq_pi_div_gamma {d : ℕ} (hd : 0 < d) :
    radialMass d = Real.pi ^ ((d : ℝ) / 2) / Real.Gamma ((d : ℝ) / 2) := by
  have hA : 0 < (d : ℝ) / 2 := by positivity
  have hM : Matrix.PosDef (1 : Matrix (Fin d) (Fin d) ℝ) := Matrix.PosDef.one
  have hvalue : quadraticGaussianValue (1 : Matrix (Fin d) (Fin d) ℝ) =
      Real.pi ^ ((d : ℝ) / 2) := by
    unfold quadraticGaussianValue
    simp only [Fintype.card_fin, Matrix.det_one, div_one]
    rw ←[ Real.rpow_natCast, ← Real.rpow_mul Real.pi_pos.le]
    congr 1
    ring
  have hG : ∫( x : Fin d → ℝ, Real.exp ∑(-( i, (x i) ^ 2))
      ∂coordinateLebesgue (Fin d)) = Real.pi ^ ((d : ℝ) / 2) := by
    simpa only [hvalue, Matrix.one_mulVec, dotProduct, pow_two] using
      quadratic_gaussian_integral (1 : Matrix (Fin d) (Fin d) ℝ) hM
  have hrad := vector_gramTest_integral hd 0 (T := 0) (by norm_num)
  rw [model_zero_parameters hA] at hrad
  have heq : Real.pi ^ ((d : ℝ) / 2) =
      radialMass d * Real.Gamma ((d : ℝ) / 2) := by
    calc
```

```
      _ = ∫ x : Fin d → ℝ, Real.exp ∑(-( i, (x i) ^ 2))
          ∂coordinateLebesgue (Fin d) := hG.symm
      _ = _ := by simpa only [gramTest, zero_mul, add_zero, neg_zero,
        Real.rpow_zero, mul_one] using hrad
  exact (eq_div_iff (Real.Gamma_pos_of_pos hA).ne').2 heq.symm

example : radialMass 1 = 1 := by
  rw [radialMass_eq_pi_div_gamma (by decide)]
  norm_num only [Nat.cast_one]
  rw [Real.Gamma_one_half_eq, Real.sqrt_eq_rpow]
  exact div_self (Real.rpow_pos_of_pos Real.pi_pos _).ne'

example {d : ℕ} (hd : 0 < d) :
    leftGramLaplaceLIntegral 1 d 0 0 = ENNReal.ofReal (Real.pi ^ ((d : ℝ) / 2)) := by
  have hA : 0 < (d : ℝ) / 2 := by positivity
  rw [leftGram_one_eq_vector,
    ← ofReal_integral_eq_lintegral_ofReal
      (vector_gramTest_integrable d (by norm_num) (by norm_num))
      (.of_forall fun x => gramTest_nonneg (by norm_num) (by positivity)),
    vector_gramTest_integral hd 0 (by norm_num), model_zero_parameters hA,
    radialMass_eq_pi_div_gamma hd,
    ₀div_mul_cancel _ (Real.Gamma_pos_of_pos hA).ne']

end O77.RadialGram
end
```

49 Rectangular orientations, small ranks, and normalization

Fix integers $k, d \geq 0$ with $k \leq d$. We continue to write $\mathcal{G}_{k,d}(\rho, T)$ for the left Gram integral and $\mathcal{J}_k(\eta, \rho, T)$ for the full-orthant sorted-extension Laguerre integral. The exponent is the real number $\eta_{k,d} = (d - k - 1)/2$, formed after casting dimensions to $\mathbb{R}$. A concrete Wishart formula requires a single constant $C \in (0, \infty)$ such that both rectangular orientations equal $C\mathcal{J}_k$ for every $\rho, T \geq 0$. This section separates the two orientations, proves the empty case, and makes the exact quantifiers on C explicit.

```
import Mathlib
import O77RadialGram
import O77Transport

set_option autoImplicit false
noncomputable section
open Set MeasureTheory
open scoped ENNReal BigOperators

namespace O77.WishartReduction
open O77.Residual O77.Transport0906
```

Transpose is an entry permutation. Flatten a matrix to the product of its row and column index sets, apply the index swap, and unflatten. The resulting measurable equivalence evaluates exactly to matrix transposition. Each constituent preserves the specified product measures, hence so does their composition. The identity `rightGramLaplaceIntegrand_transpose` combines Frobenius invariance with Sylvester's determinant identity; changing variables by transpose then proves `right_eq_left`. No sign assumptions on the real parameters are needed for this algebraic identity between the totalized nonnegative integrands.

```
/-- Transposition as a measured entry permutation, not a normed-matrix isometry. -/
def transposeEntries {m n : Type*} [Fintype m] [Fintype n] :
```

```
    Matrix m n ℝ ≃ᵐ Matrix n m ℝ :=
  (matrixEntriesMeasurable (m := m) (n := n)).trans
    ((MeasurableEquiv.piCongrLeft (fun _ : n × m => ℝ) (Equiv.prodComm m n)).trans
      (matrixEntriesMeasurable (m := n) (n := m)).symm)

@[simp] lemma transposeEntries_apply {m n : Type*} [Fintype m] [Fintype n]
    (Z : Matrix m n ℝ) : transposeEntries Z = Z.transpose := rfl

lemma transposeEntries_preserves {m n : Type*} [Fintype m] [Fintype n] :
    MeasurePreserving (transposeEntries (m := m) (n := n))
      (matrixLebesgue m n) (matrixLebesgue n m) := by
  exact (matrixEntries_measurePreserving (m := n) (n := m)).symm.comp
    ((reindexEntries_measurePreserving (Equiv.prodComm m n)).comp
      (matrixEntries_measurePreserving (m := m) (n := n)))

/-- The large determinant in the transposed orientation is reduced by the
inherited Sylvester identity. No sign restriction on the parameters is needed. -/
theorem right_eq_left (k d : ℕ) (rho T : ℝ) :
    rightGramLaplaceLIntegral d k rho T = leftGramLaplaceLIntegral k d rho T := by
  unfold rightGramLaplaceLIntegral leftGramLaplaceLIntegral
  rw [(transposeEntries_preserves (m := Fin k) (n := Fin d)).lintegral_map_equiv
    (rightGramLaplaceIntegrand rho T) transposeEntries]
  apply lintegral_congr_ae
  filter_upwards with Z
  exact rightGramLaplaceIntegrand_transpose rho T Z
```

One orientation suffices, with the same coefficient. The forward implication extracts positivity, finiteness, and the left identity from `HasConcreteRealWishartFormula`. Conversely, supply those guards and the left identity, and use `right_eq_left` to obtain the right identity. In particular this equivalence does not introduce a coefficient depending on ρ or on T.

```
/-- Only the left identity must be supplied. Its normalization and parameter
quantifiers are unchanged, and the right identity uses that same constant. -/
theorem hasConcrete_iff_left {k d : ℕ} (hkd : k ≤ d) (C : ℝ≥∞0) :
    HasConcreteRealWishartFormula k d C ↔
      0 < C ∧ C < ⊤ ∧ ∀ rho T : ℝ, 0 ≤ rho → 0 ≤ T →
        leftGramLaplaceLIntegral k d rho T =
          C * laguerreOrthantLIntegral k (wishartEta k d) rho T := by
  constructor
  · intro h
    exact ⟨h.constant_pos, h.constant_lt_top, fun _ _ hr hT => h.left_formula hr ⟩hT
  · rintro ⟨hC, hfin, ⟩hf
    refine ⟨hkd, hC, hfin, ?⟩_
    intro rho T hr hT
    exact ⟨hf rho T hr hT, (right_eq_left k d rho T).trans (hf rho T hr hT)⟩
```

Empty and scalar spectra. For $k = 0$, both coordinate spaces are singletons of mass one. Their empty sums are zero and empty determinants and products are one, so both integrals equal one for all parameters. This constructs the concrete formula with $C = 1$. For $k = 1 \le d$, the preceding radial result constructs it with $C = \mathrm{ofReal}(a_d)$, and transpose supplies the second orientation. Thus these cases require no spectral integration theorem.

```
/-- Empty row and eigenvalue spaces both have singleton mass one. -/
lemma empty_integrals (d : ℕ) (rho T : ℝ) :
    leftGramLaplaceLIntegral 0 d rho T = 1 ∧
      laguerreOrthantLIntegral 0 (wishartEta 0 d) rho T = 1 := by
  constructor
  · unfold leftGramLaplaceLIntegral
    calc
```

```
      _ = ∫⁻ _ : Matrix (Fin 0) (Fin d) ℝ, (1 : ℝ≥∞0)
          ∂matrixLebesgue (Fin 0) (Fin d) := by
        apply lintegral_congr
        intro Z
        simp [leftGramLaplaceIntegrand, frobeniusSq]
      _ = 1 := by
        rw [lintegral_const, one_mul]
        exact Measure.pi_empty_univ _
  · simp [laguerreOrthantLIntegral, nonnegativeOrthant_zero, sortedLaguerreIntegrand,
      laguerreChamberIntegrand, laguerreChamberCore, commonRpowProduct,
      vandermondeFin, vandermondeList, laguerreWeightProduct, coordinateLebesgue]

theorem concrete_zero (d : ℕ) : ConcreteRealWishartEigenvalueFormula 0 d := by
  refine ⟨1, (hasConcrete_iff_left (Nat.zero_le d) 1).2 ⟨by norm_num, by simp, ?⟩⟩_
  intro rho T _hr _hT
  rw [(empty_integrals d rho T).1, (empty_integrals d rho T).2, mul_one]

/-- Both one-eigenvalue rectangular identities, for all nonnegative rho and T. -/
theorem concrete_one {d : ℕ} (hd : 0 < d) :
    ConcreteRealWishartEigenvalueFormula 1 d := by
  refine ⟨ENNReal.ofReal (O77.RadialGram.radialMass d),
    (hasConcrete_iff_left hd _).2 ⟨?_, ENNReal.ofReal_lt_top, ?⟩⟩_
  · exact ENNReal.ofReal_pos.2 (O77.RadialGram.radialMass_pos hd)
  · intro rho T hr hT
    exact O77.RadialGram.leftGram_one_formula hd hr hT
```

Calibrating a valid coefficient. At zero parameters the determinant factor is one, so the Gaussian row integral gives $\mathcal{G}_{k,d}(0,0) = \mathrm{ofReal}(\pi^{kd/2})$, denoted `gaussianRowsConstantkd`. Any coefficient already satisfying the concrete formula must therefore obey $C\mathcal{J}_k(\eta_{k,d}, 0, 0) = \mathrm{ofReal}(\pi^{kd/2})$. If the normalizing Laguerre integral were zero or infinite, this equality would contradict the positive finite Gaussian mass and the guards on C. Hence the normalizing integral is itself positive and finite.

```
/-- An independently computed Gaussian calibration; no Wishart assumption. -/
lemma left_zero_parameters (k d : ℕ) :
    leftGramLaplaceLIntegral k d 0 0 =
      ENNReal.ofReal (gaussianRowsConstant k d) := by
  have hg := gaussianRows_lintegral_instantiated (p := k)
    0 (by norm_num) (0 : Matrix (Fin d) (Fin 0) ℝ)
  simpa [leftGramLaplaceLIntegral, leftGramLaplaceIntegrand,
    gaussianRowsIntegrand] using hg

/-- A valid normalizer is calibrated at zero parameters, before any asymptotic
argument. The orthant integral here includes the full permutation factor. -/
lemma normalizer_calibration {k d : ℕ} {C : ℝ≥∞0}
    (hC : HasConcreteRealWishartFormula k d C) :
    C * laguerreOrthantLIntegral k (wishartEta k d) 0 0 =
      ENNReal.ofReal (gaussianRowsConstant k d) := by
  exact (hC.left_formula (by norm_num) (by norm_num)).symm.trans
    (left_zero_parameters k d)

/-- Positivity and finiteness of the normalizing integral follow from calibration
and the already explicit guards on C, not from totalizing a real integral. -/
lemma normalizing_integral_bounds {k d : ℕ} {C : ℝ≥∞0}
    (hC : HasConcreteRealWishartFormula k d C) :
    0 < laguerreOrthantLIntegral k (wishartEta k d) 0 0 ∧
      laguerreOrthantLIntegral k (wishartEta k d) 0 0 < ⊤ := by
  let N := laguerreOrthantLIntegral k (wishartEta k d) 0 0
  have hprod : C * N = ENNReal.ofReal (gaussianRowsConstant k d) :=
    normalizer_calibration hC
```

```
  have hN0 : N ≠ 0 := by
    intro hz
    rw [hz, mul_zero] at hprod
    exact (ENNReal.ofReal_pos.2 (gaussianRowsConstant_pos k d)).ne' hprod.symm
  have hNtop : N ≠ ⊤ := by
    intro hz
    rw [hz, ENNReal.mul_top hC.constant_ne_zero] at hprod
    exact ENNReal.ofReal_ne_top hprod.symm
  exact ⟨bot_lt_iff_ne_bot.2 hN0, lt_top_iff_ne_top.2 ⟩hNtop
```

Uniqueness after establishing finiteness. If C, D both satisfy the formula, calibration gives $CN = DN$, where $N = \mathcal{J}_k(\eta_{k,d}, 0, 0) \in (0, \infty)$. Only now does the proof apply `ENNReal.toReal`, cancel the positive real number N.toReal, and return to extended nonnegative numbers using finiteness of C, D. This proves uniqueness of a valid coefficient; it does not prove that an arbitrarily calibrated coefficient satisfies the parameterized identity.

```
/-- Any two constants satisfying the actual interface are equal. In particular
one cannot silently recalibrate the measure separately at different T. -/
theorem normalizer_unique {k d : ℕ} {C D : ℝ≥∞0}
    (hC : HasConcreteRealWishartFormula k d C)
    (hD : HasConcreteRealWishartFormula k d D) : C = D := by
  obtain ⟨hN0, ⟩hNtop := normalizing_integral_bounds hC
  have heq := (normalizer_calibration hC).trans (normalizer_calibration hD).symm
  have hreal := congrArg ENNReal.toReal heq
  rw [ENNReal.toReal_mul, ENNReal.toReal_mul] at hreal
  have hNreal := ENNReal.toReal_pos hN0.ne' hNtop.ne
  have hc : C.toReal = D.toReal := ₀mul_right_cancel hNreal.ne' hreal
  calc
    C = ENNReal.ofReal C.toReal := (ENNReal.ofReal_toReal hC.constant_ne_top).symm
    _ = ENNReal.ofReal D.toReal := congrArg ENNReal.ofReal hc
    _ = D := ENNReal.ofReal_toReal hD.constant_ne_top
```

The exact higher-rank interface. The global Wishart proposition is equivalent to supplying just the left identity for $2 \le k \le d$, with a positive finite C chosen before ρ, T. One direction restricts a global proof. For the converse, split into $k = 0$, $k = 1$, and $k \ge 2$, using the explicit small-rank constructions in the first two cases and the assumed higher-rank identity in the third. The examples at the end illustrate that the transposed integral can have a large determinant even when only one eigenvalue is nonzero.

```
/-- Reduction to one orientation for 2 <= k <= d.
The theorem establishes an equivalence of propositions. -/
theorem global_iff_higherRank_left :
    O77.External.RealWishartEigenvalueFormula ↔
      ∀ k d : ℕ, 2 ≤ k → k ≤ d →
        ∃ C : ℝ≥∞0, 0 < C ∧ C < ⊤ ∧
          ∀ rho T : ℝ, 0 ≤ rho → 0 ≤ T →
            leftGramLaplaceLIntegral k d rho T =
              C * laguerreOrthantLIntegral k (wishartEta k d) rho T := by
  constructor
  · intro h k d _hk hkd
    obtain ⟨C, ⟩hC := h k d hkd
    exact ⟨C, (hasConcrete_iff_left hkd C).1 ⟩hC
  · intro h k d hkd
    by_cases hk0 : k = 0
    · subst k
      exact concrete_zero d
    by_cases hk1 : k = 1
    · subst k
      exact concrete_one hkd
```

```
    obtain ⟨C, ⟩hC := h k d (by omega) hkd
    exact ⟨C, (hasConcrete_iff_left hkd C).2 ⟩hC

-- The large one-column Gram determinant is covered, not just a 1-by-1 matrix.
example : ConcreteRealWishartEigenvalueFormula 1 7 := concrete_one (by decide)
example : rightGramLaplaceLIntegral 7 0 3 5 = 1 := by
  rw [right_eq_left]
  exact (empty_integrals 7 3 5).1

end O77.WishartReduction
end
```

50 The higher-rank interface and the O77 contract

The theorem below is a conditional intermediate consumer. Its hypothesis `hHigher` asserts the higher-rank left Gram identity, with the coefficient chosen before both nonnegative parameters; its conclusion is the same six-conjunct measured `AnswersO77` contract. The proof converts `hHigher` to the global Wishart proposition by `global_iff_higherRank_left`, then applies `answersO77_of_wishart`. The matrix assumptions remain exactly $I, O > 0$ and $\operatorname{rank}\Phi < H$. This declaration does not itself discharge `hHigher`; the Gaussian Gram and spectral integration constructions below supply that input for the unconditional final theorem.

```
import Mathlib
import O77Answer
import O77WishartReduction

set_option autoImplicit false
noncomputable section
open MeasureTheory
open scoped ENNReal

namespace O77
open Residual

/-- The answer contract conditional on the higher-rank left identity.
The zero- and one-eigenvalue cases are constructed inside
`global_iff_higherRank_left`. -/
theorem answersO77_of_higherRank_left
    (hHigher : ∀ k d : ℕ, 2 ≤ k → k ≤ d →
      ∃ C : ℝ≥∞, 0 < C ∧ C < ⊤ ∧
        ∀ rho T : ℝ, 0 ≤ rho → 0 ≤ T →
          leftGramLaplaceLIntegral k d rho T =
            C * laguerreOrthantLIntegral k (wishartEta k d) rho T)
    {I O H : ℕ} (Phi : Matrix (Fin O) (Fin I) ℝ)
    (hI : 0 < I) (hO : 0 < O) (hrH : Phi.rank < H) :
    AnswersO77 (H := H) Phi :=
  answersO77_of_wishart
    (WishartReduction.global_iff_higherRank_left.mpr hHigher) Phi hI hO hrH

end O77
end
```

51 Nonnegative polar integration, including infinite tests

For the Gram induction, fix a measurable function $f : \mathbb{R} \to [0, \infty]$; only its values on $[0, \infty)$ will be used. All integrals in this section are extended nonnegative integrals, implemented by `lintegral`, and no finiteness of the test integral is assumed. The positive-dimensional formulas

concern a nontrivial finite-dimensional real normed space E with its Borel structure and additive Haar measure μ. Write $n = \dim_{\mathbb{R}} E$, $S = \{x \in E : \|x\| = 1\}$, $B_1 = \{x \in E : \|x\| < 1\}$, and let μ_S be the sphere measure `mu.toSphere`. A separate formula treats dimension zero.

```
import Mathlib
import O77RadialGram

set_option autoImplicit false
noncomputable section
open Set MeasureTheory
open scoped ENNReal BigOperators Topology

namespace O77.RadialGram
```

Polar measure decomposition. The point 0 is Haar-null in a nontrivial real vector space. On its complement, the library map `homeomorphUnitSpherePro`d sends x to $(x/\|x\|, \|x\|)$ and carries Haar measure to the product of its sphere measure `mu.toSphere` and the radial measure with density $r^{n-1}\,dr$. The code uses `measurePreserving_homeomorphUnitSpherePro`d, integrates the test $f(r)$ over that product by Tonelli, and unfolds `Measure.volumeIoiPow`. This proves $\int_E f(\|x\|)\,d\mu = \mu_S(S)\int_0^\infty r^{n-1} f(r)\,dr$, including infinite values.

```
/-- The polar formula for genuinely nonnegative tests.  Its proof takes place
entirely in `ENNReal`; in particular, no integrability hypothesis or totalized
Bochner integral is used. -/
theorem lintegral_fun_norm_addHaar_ennreal {E : Type*}
    [NormedAddCommGroup E] [NormedSpace ℝ E] [FiniteDimensional ℝ E]
    [Nontrivial E] [MeasurableSpace E] [BorelSpace E]
    (mu : Measure E) [Measure.IsAddHaarMeasure mu] (f : ℝ → ℝ≥∞0)
    (hf : Measurable f) :
    ∫⁻ x : E, f ‖‖x ∂mu) =
      mu.toSphere univ *
        ∫⁻ r in Ioi (0 : ℝ),
          ENNReal.ofReal (r ^ (Module.finrank ℝ E - 1)) * f r := by
  calc
    ∫⁻ x : E, f ‖‖x ∂mu) =
        ∫⁻ x : ({(0 : E)ᶜ} : Set E), f ‖x‖.1 ∂(mu.comap ↑()) := by
      simpa only [restrict_compl_singleton] using
        (lintegral_subtype_comap μ( := mu)
          (measurableSet_singleton (0 : E)).compl (fun x : E => f ‖‖x)).symm
    _ = ∫⁻ x : Metric.sphere (0 : E) 1 × Ioi (0 : ℝ), f x.2.1
        ∂(mu.toSphere.prod (Measure.volumeIoiPow (Module.finrank ℝ E - 1))) := by
      simpa only [homeomorphUnitSpherePro_apply_snd_coe] using
        mu.measurePreserving_homeomorphUnitSpherePro.lintegral_comp
          (f := fun x : Metric.sphere (0 : E) 1 × Ioi (0 : ℝ) => f x.2.1)
          (hf.comp (measurable_subtype_coe.comp measurable_snd))
    _ = mu.toSphere univ *
        ∫⁻ r : Ioi (0 : ℝ), f r.1
          ∂Measure.volumeIoiPow (Module.finrank ℝ E - 1) := by
      simpa only [lintegral_const, mul_comm] using
        lintegral_prod μ( := mu.toSphere)
          ν( := Measure.volumeIoiPow (Module.finrank ℝ E - 1))
          (fun x : Metric.sphere (0 : E) 1 × Ioi (0 : ℝ) => f x.2.1)
          (hf.comp (measurable_subtype_coe.comp measurable_snd)).aemeasurable
    _ = _ := by
      unfold Measure.volumeIoiPow
      congr 1
      calc
        _ = ∫⁻ r : Ioi (0 : ℝ),
            ENNReal.ofReal (r.1 ^ (Module.finrank ℝ E - 1)) * f r.1
              ∂(volume.comap (Subtype.val : Ioi (0 : ℝ) → ℝ)) :=
```

```
          lintegral_withDensity_eq_lintegral_mul _
            (f := fun r : Ioi (0 : ℝ) =>
              ENNReal.ofReal (r.1 ^ (Module.finrank ℝ E - 1)))
            (g := fun r : Ioi (0 : ℝ) => f r.1)
            (measurable_subtype_coe.pow_const _).ennreal_ofReal
            (hf.comp measurable_subtype_coe)
        _ = _ := lintegral_subtype_comap μ( := volume) measurableSet_Ioi
          (fun r : ℝ => ENNReal.ofReal (r ^ (Module.finrank ℝ E - 1)) * f r)
```

The factor $1/2$ on the positive ray. The map $r \mapsto r^2$ is injective on $(0,\infty)$, has image $(0,\infty)$ by the positive square root, and derivative $2r$. Apply `lintegral_image_eq_lintegral_abs_deriv_mul` to the test $s^{n/2-1}f(s)$. On the positive ray, real-power rules turn its pulled-back density into $2r^{n-1}f(r^2)$. Multiplication by the finite number $1/2$ gives the asserted identity. The proof never cancels an unknown integral or applies `toReal`; hence an infinite integral is harmless.

```
/-- Squared-radius substitution on the positive ray.  The test can take the
value infinity.  The factor `1/2` is multiplied in `ENNReal`, so there is no
cancellation involving a possibly infinite integral. -/
private theorem lintegral_sq_radius {n : ℕ} (hn : 0 < n)
    (f : ℝ → ℝ≥∞0) :
    ∫⁻( r in Ioi (0 : ℝ), ENNReal.ofReal (r ^ (n - 1)) * f (r ^ 2)) =
      ENNReal.ofReal (1 / 2 : ℝ) *
        ∫⁻ s in Ioi (0 : ℝ),
          ENNReal.ofReal (s ^ ((n : ℝ) / 2 - 1)) * f s := by
  have himage : (fun r : ℝ => r ^ 2) '' Ioi (0 : ℝ) = Ioi 0 := by
    apply Set.Subset.antisymm
    · rintro s ⟨r, hr, ⟩rfl
      exact sq_pos_of_pos (show 0 < r from hr)
    · intro s hs
      have hs' : 0 < s := hs
      exact ⟨Real.sqrt s, Real.sqrt_pos.2 hs', Real.sq_sqrt hs'.⟩le
  have hderiv (r : ℝ) (_hr : r ∈ Ioi (0 : ℝ)) :
      HasDerivWithinAt (fun t : ℝ => t ^ 2) (2 * r) (Ioi 0) r := by
    simpa using (hasDerivAt_pow 2 r).hasDerivWithinAt
  have hinj : Set.InjOn (fun r : ℝ => r ^ 2) (Ioi (0 : ℝ)) := by
    intro x hx y hy hxy
    have hprod : (x - y) * (x + y) = 0 := by nlinarith
    rcases mul_eq_zero.mp hprod with h | h
    · linarith
    · exact False.elim (ne_of_gt (add_pos hx hy) h)
  have hpoint (r : ℝ) (hr : 0 < r) :
      ENNReal.ofReal (|2 * r|) *
          (ENNReal.ofReal ((r ^ 2) ^ ((n : ℝ) / 2 - 1)) * f (r ^ 2)) =
        ENNReal.ofReal (2 : ℝ) *
          (ENNReal.ofReal (r ^ (n - 1)) * f (r ^ 2)) := by
    have hexp : 1 + 2 * ((n : ℝ) / 2 - 1) = ((n - 1 : ℕ) : ℝ) := by
      rw [Nat.cast_sub hn]
      push_cast
      ring
    have hpow : r * (r ^ 2) ^ ((n : ℝ) / 2 - 1) = r ^ (n - 1) := by
      calc
        r * (r ^ 2) ^ ((n : ℝ) / 2 - 1) =
            r ^ (1 : ℝ) * (r ^ (2 : ℝ)) ^ ((n : ℝ) / 2 - 1) := by
          simp only [Real.rpow_one, Real.rpow_ofNat]
        _ = r ^ (1 : ℝ) * r ^ (2 * ((n : ℝ) / 2 - 1)) := by
          rw ←[ Real.rpow_mul hr.le]
        _ = r ^ (1 + 2 * ((n : ℝ) / 2 - 1)) := by
          rw [Real.rpow_add hr]
        _ = r ^ (n - 1) := by rw [hexp, Real.rpow_natCast]
```

```
    rw [abs_of_pos (mul_pos (by norm_num) hr)]
    calc
      ENNReal.ofReal (2 * r) *
          (ENNReal.ofReal ((r ^ 2) ^ ((n : ℝ) / 2 - 1)) * f (r ^ 2)) =
          ENNReal.ofReal ((2 * r) * (r ^ 2) ^ ((n : ℝ) / 2 - 1)) * f (r ^ 2) := by
        rw [ENNReal.ofReal_mul (show (0 : ℝ) ≤ 2 * r by positivity), mul_assoc]
      _ = ENNReal.ofReal (2 * r ^ (n - 1)) * f (r ^ 2) := by
        rw [mul_assoc, hpow]
      _ = _ := by rw [ENNReal.ofReal_mul (by norm_num), mul_assoc]
  have hsub := lintegral_image_eq_lintegral_abs_deriv_mul
    measurableSet_Ioi hderiv hinj
    (fun s : ℝ => ENNReal.ofReal (s ^ ((n : ℝ) / 2 - 1)) * f s)
  rw [himage] at hsub
  have htwice :
      ∫⁻ s in Ioi (0 : ℝ),
        ENNReal.ofReal (s ^ ((n : ℝ) / 2 - 1)) * f s) =
      ENNReal.ofReal (2 : ℝ) *
        ∫⁻ r in Ioi (0 : ℝ), ENNReal.ofReal (r ^ (n - 1)) * f (r ^ 2) := by
    calc
      _ = _ := hsub
      _ = ∫⁻ r in Ioi (0 : ℝ), ENNReal.ofReal (2 : ℝ) *
          (ENNReal.ofReal (r ^ (n - 1)) * f (r ^ 2)) :=
        setLIntegral_congr_fun measurableSet_Ioi (fun r hr => hpoint r hr)
      _ = _ := lintegral_const_mul' _ _ ENNReal.ofReal_ne_top
  rw [htwice, ← mul_assoc, ← ENNReal.ofReal_mul (by norm_num)]
  norm_num
```

Squared-norm pushforward. Combine the two previous formulas with `mu.toSphere_real_apply_univ`, which gives sphere mass $n\mu(B_1)$. The resulting coefficient is $n\mu(B_1)/2$. Only this known finite sphere mass is converted from a real number; the test and its integral remain extended nonnegative. For $E = \mathbb{R}^d$ with Euclidean volume the coefficient a_d is exactly `radialMassd`.

```
/-- Exact pushforward of Haar measure by the squared norm in positive finite
dimension, for every nonnegative measurable test (including infinite tests).
For Euclidean volume the coefficient is `d/2` times the unit-ball volume.
The zero-dimensional atom is deliberately excluded from this formula. -/
theorem lintegral_norm_sq {E : Type*} [NormedAddCommGroup E]
    [NormedSpace ℝ E] [FiniteDimensional ℝ E] [Nontrivial E]
    [MeasurableSpace E] [BorelSpace E]
    (mu : Measure E) [Measure.IsAddHaarMeasure mu] (f : ℝ → ℝ≥∞0)
    (hf : Measurable f) :
    ∫⁻ x : E, f ‖(‖x ^ 2) ∂mu) =
      ENNReal.ofReal (((Module.finrank ℝ E : ℝ) / 2) *
        mu.real (Metric.ball 0 1)) *
        ∫⁻ s in Ioi (0 : ℝ),
          ENNReal.ofReal (s ^ ((Module.finrank ℝ E : ℝ) / 2 - 1)) * f s := by
  have hmass : mu.toSphere univ =
      ENNReal.ofReal ((Module.finrank ℝ E : ℝ) * mu.real (Metric.ball 0 1)) := by
    rw ←[ mu.toSphere_real_apply_univ]
    exact (ENNReal.ofReal_toReal (measure_ne_top mu.toSphere univ)).symm
  rw [lintegral_fun_norm_addHaar_ennreal mu (fun r : ℝ => f (r ^ 2))
      (hf.comp (measurable_id.pow_const 2)),
    lintegral_sq_radius Module.finrank_pos, hmass, ← mul_assoc,
    ← ENNReal.ofReal_mul (by positivity)]
  congr 1
  congr 1
  ring
```

Dimension zero is an atom. In a subsingleton seminormed group every element equals zero, so the squared-norm test is constantly $f(0)$. The integral is therefore $f(0)\mu(E)$. For the zero-dimensional coordinate Lebesgue measures used here, $\mu(E) = 1$. No singular expression s^{-1} is used to represent this atom.

```
/-- In dimension zero the squared-norm pushforward is an atom, with its
actual ambient mass.  No positive-dimensional radial density is invoked. -/
theorem lintegral_norm_sq_subsingleton {E : Type*}
    [SeminormedAddCommGroup E] [Subsingleton E] [MeasurableSpace E]
    (mu : Measure E) (f : ℝ → ℝ≥∞0) :
    ∫⁻ x : E, f ‖(‖x ^ 2) ∂mu) = f 0 * mu univ := by
  calc
    ∫⁻ x : E, f ‖(‖x ^ 2) ∂mu) = ∫⁻ _x : E, f 0 ∂mu := by
      apply lintegral_congr
      intro x
      simp [Subsingleton.elim x (0 : E)]
    _ = _ := lintegral_const _

end O77.RadialGram
end
```

52 Symmetric entry measure and Schur complements

This section fixes the symmetric-coordinate convention once and for all. For a $k \times k$ symmetric real matrix, the independent coordinates are its entries (i, j) with $i \le j$, indexed in Lean by `UpperIndexk`. Their product Lebesgue measure is `upperLebesguek`. This is the measure $\prod_{i \le j} dG_{ij}$, not the volume induced by the Frobenius metric on the symmetric subspace: the latter has an additional constant from the off-diagonal vectors of norm $\sqrt{2}$. All subsequent Gram and spectral constants refer to these independent entries.

```
import Mathlib
import O77RadialGram

set_option autoImplicit false
noncomputable section
open Set MeasureTheory Matrix
open scoped ENNReal BigOperators

namespace O77.GramBorder
open O77.Residual
```

The symmetric carrier. The function `fromUpper` fills the lower triangle by symmetry, and `upperEntries` extracts the upper entries of any square matrix. Their compositions are the identity on coordinate vectors and, respectively, on symmetric matrices. The following coordinatewise proofs also establish symmetry and continuity of `fromUpper`; no ambient-square-matrix volume assertion is involved.

```
/-- Independent entries, not the Frobenius volume on the symmetric subspace. -/
abbrev UpperIndex (k : ℕ) := {ij : Fin k × Fin k // ij.1 ≤ ij.2}
abbrev UpperCoordinates (k : ℕ) := UpperIndex k → ℝ

def upperLebesgue (k : ℕ) : Measure (UpperCoordinates k) :=
  coordinateLebesgue (UpperIndex k)

def fromUpper {k : ℕ} (a : UpperCoordinates k) : Matrix (Fin k) (Fin k) ℝ :=
  fun i j => if h : i ≤ j then a ⟨(i,j),⟩h else a ⟨(j,i),le_of_not_ge ⟩h

def upperEntries {k : ℕ} (B : Matrix (Fin k) (Fin k) ℝ) : UpperCoordinates k :=
```

```
  fun ij => B ij.1.1 ij.1.2

@[simp] lemma upperEntries_fromUpper {k : ℕ} (a : UpperCoordinates k) :
    upperEntries (fromUpper a) = a := by
  funext ij
  simp [upperEntries, fromUpper, ij.2]

lemma fromUpper_upperEntries {k : ℕ} {B : Matrix (Fin k) (Fin k) ℝ}
    (hB : B.IsSymm) : fromUpper (upperEntries B) = B := by
  ext i j
  by_cases hij : i ≤ j
  · simp [fromUpper, upperEntries, hij]
  · simpa [fromUpper, upperEntries, hij] using congrFun (congrFun hB i) j

lemma fromUpper_isSymm {k : ℕ} (a : UpperCoordinates k) :
    (fromUpper a).IsSymm := by
  ext i j
  by_cases hij : i ≤ j
  · by_cases hji : j ≤ i
    · have : i = j := le_antisymm hij hji
      subst j
      rfl
    · simp [Matrix.transpose_apply, fromUpper, hij, hji]
  · have hji : j ≤ i := le_of_not_ge hij
    simp [Matrix.transpose_apply, fromUpper, hij, hji]

lemma fromUpper_continuous (k : ℕ) :
    Continuous (fromUpper : UpperCoordinates k → Matrix (Fin k) (Fin k) ℝ) := by
  apply continuous_pi
  intro i
  apply continuous_pi
  intro j
  by_cases hij : i ≤ j <;> simp only [fromUpper, hij, dite_true, dite_false]
  all_goals exact continuous_apply _
```

The bordered matrix. Let B be an $n \times n$ real matrix, $z \in \mathbb{R}^n$, and $u \in \mathbb{R}$. The next definitions represent

$$G(B, z, u) = \begin{pmatrix} B & z \\ z^{\mathsf{T}} & u \end{pmatrix}, \qquad q_B(z) = z^{\mathsf{T}} B^{-1} z.$$

The block carrier is `Finn` $\oplus$ `Fin1`; `scalarBlock` retains the single bottom-right coordinate. The scalar Schur complement is exactly $u - q_B(z)$, not a matrix-valued expression after the one-dimensional adapter. For fixed B, q_B is a continuous quadratic polynomial in z.

```
/-- A scalar block with its sole coordinate retained explicitly. -/
def scalarBlock (u : ℝ) : Matrix (Fin 1) (Fin 1) ℝ := fun _ _ => u

lemma scalarBlock_eq_diagonal (u : ℝ) :
    scalarBlock u = Matrix.diagonal (fun _ : Fin 1 => u) := by
  ext i j
  have : i = j := Subsingleton.elim _ _
  subst j
  simp [scalarBlock]

def border {n : ℕ} (B : Matrix (Fin n) (Fin n) ℝ) (z : Fin n → ℝ) (u : ℝ) :
    Matrix (Fin n ⊕ Fin 1) (Fin n ⊕ Fin 1) ℝ :=
  Matrix.fromBlocks B (Matrix.replicateCol (Fin 1) z)
    (Matrix.replicateRow (Fin 1) z) (scalarBlock u)

def schurQuad {n : ℕ} (B : Matrix (Fin n) (Fin n) ℝ) (z : Fin n → ℝ) : ℝ :=
```

```
  (Matrix.replicateRow (Fin 1) z * ⁻B¹ * Matrix.replicateCol (Fin 1) z) 0 0

lemma schurQuad_continuous {n : ℕ} (B : Matrix (Fin n) (Fin n) ℝ) :
    Continuous (schurQuad B) := by
  unfold schurQuad
  simp only [Matrix.mul_apply, Matrix.replicateRow_apply, Matrix.replicateCol_apply]
  fun_prop

lemma scalar_schur {n : ℕ} (B : Matrix (Fin n) (Fin n) ℝ)
    (z : Fin n → ℝ) (u : ℝ) :
    scalarBlock u - Matrix.replicateRow (Fin 1) z * ⁻B¹ *
      Matrix.replicateCol (Fin 1) z = scalarBlock (u - schurQuad B z) := by
  ext i j
  have hi : i = 0 := Subsingleton.elim _ _
  have hj : j = 0 := Subsingleton.elim _ _
  subst i
  subst j
  rfl
```

Determinant and positivity. If B is invertible, `Matrix.det_fromBlocks₁₁` gives $\det G = \det B\,(u - q_B(z))$. If B is positive definite, `Matrix.PosDef.fromBlocks₁₁` identifies positive semidefiniteness of G with $u - q_B(z) \geq 0$. The strict version follows as well: positive definiteness implies positive determinant and hence $u - q_B(z) > 0$; conversely the semidefinite criterion and the nonzero determinant imply positive definiteness by `Matrix.PosSemidef.posDef_iff_det_ne_zero`. Finally, summing diagonal entries gives $\operatorname{tr} G = \operatorname{tr} B + u$. These identities remain valid for $n = 0$ with the empty determinant equal to one.

```
lemma det_border {n : ℕ} (B : Matrix (Fin n) (Fin n) ℝ)
    (hB : IsUnit B.det) (z : Fin n → ℝ) (u : ℝ) :
    (border B z u).det = B.det * (u - schurQuad B z) := by
  let : Invertible B := B.invertibleOfIsUnitDet hB
  unfold border
  rw [Matrix.₁₁det_fromBlocks, Matrix.invOf_eq_nonsing_inv, scalar_schur]
  simp [scalarBlock]

lemma posSemidef_border_iff {n : ℕ} {B : Matrix (Fin n) (Fin n) ℝ}
    (hB : B.PosDef) (z : Fin n → ℝ) (u : ℝ) :
    (border B z u).PosSemidef ↔ 0 ≤ u - schurQuad B z := by
  let : Invertible B := hB.isUnit.invertible
  have hcol : (Matrix.replicateCol (Fin 1) z).conjTranspose =
      Matrix.replicateRow (Fin 1) z := by
    ext i j
    simp [Matrix.conjTranspose_apply]
  have h := Matrix.PosDef.₁₁fromBlocks
    (Matrix.replicateCol (Fin 1) z) (scalarBlock u) hB
  rw [hcol, scalar_schur, scalarBlock_eq_diagonal (u - schurQuad B z),
    Matrix.posSemidef_diagonal_iff] at h
  simpa [border] using h

/-- The missing strict version is obtained from the library semidefinite Schur
lemma and its spectral positivity criterion, rather than postulated. -/
lemma posDef_of_posSemidef_det_ne_zero {n : Type*} [Fintype n] [DecidableEq n]
    {G : Matrix n n ℝ} (hG : G.PosSemidef) (hdet : G.det ≠ 0) : G.PosDef := by
  exact hG.posDef_iff_det_ne_zero.2 hdet

lemma posDef_border_iff {n : ℕ} {B : Matrix (Fin n) (Fin n) ℝ}
    (hB : B.PosDef) (z : Fin n → ℝ) (u : ℝ) :
    (border B z u).PosDef ↔ 0 < u - schurQuad B z := by
  have hd := det_border B (isUnit_iff_ne_zero.2 hB.det_pos.ne') z u
  constructor
```

```
  · intro h
    have hp := h.det_pos
    rw [hd] at hp
    exact (mul_pos_iff_of_pos_left hB.det_pos).1 hp
  · intro h
    apply posDef_of_posSemidef_det_ne_zero
      ((posSemidef_border_iff hB z u).2 h.le)
    rw [hd]
    exact (mul_pos hB.det_pos h).ne'

lemma trace_border {n : ℕ} (B : Matrix (Fin n) (Fin n) ℝ)
    (z : Fin n → ℝ) (u : ℝ) : (border B z u).trace = B.trace + u := by
  simp [border, Matrix.trace, Fintype.sum_sum_type, scalarBlock]
```

A measurable shear, with exact measure preservation. Let (A, μ) be a measurable space with an s-finite measure, and let $q : A \to \mathbb{R}$ be measurable. The map $(a, t) \mapsto (a, t+q(a))$ has inverse $(a, u) \mapsto (a, u - q(a))$. Both are measurable. For each fixed a, translation preserves Lebesgue measure in the last coordinate; `MeasurePreserving.skew_product` promotes these fiberwise statements to preservation of $\mu \times du$. The preimage of the epigraph $u > q(a)$ is exactly $t > 0$.

```
/-- Translation in the final coordinate, allowed to depend measurably on all
earlier coordinates. This also applies when the base includes the Gram block. -/
def shear {A : Type*} [MeasurableSpace A] (q : A → ℝ) (hq : Measurable q) :
    A × ℝ ≃ᵐ A × ℝ where
  toFun w := (w.1, w.2 + q w.1)
  invFun w := (w.1, w.2 - q w.1)
  left_inv w := by simp
  right_inv w := by simp
  measurable_toFun := measurable_fst.prodMk (measurable_snd.add (hq.comp measurable_fst))
  measurable_invFun := measurable_fst.prodMk (measurable_snd.sub (hq.comp measurable_fst))

lemma shear_preserves {A : Type*} [MeasurableSpace A]
    (mu : Measure A) [SFinite mu] (q : A → ℝ) (hq : Measurable q) :
    MeasurePreserving (shear q hq) (mu.prod volume) (mu.prod volume) := by
  exact (MeasurePreserving.id mu).skew_product
    (measurable_snd.add (hq.comp measurable_fst))
    (.of_forall fun a => (measurePreserving_add_right (volume : Measure ℝ) (q a)).map_eq)

lemma shear_preimage_epigraph {A : Type*} [MeasurableSpace A]
    (q : A → ℝ) (hq : Measurable q) :
    shear q hq ⁻¹' {w : A × ℝ | q w.1 < w.2} =
      {w : A × ℝ | 0 < w.2} := by
  ext w
  change q w.1 < w.2 + q w.1 ↔ 0 < w.2
  constructor <;> intro h <;> linarith
```

Schur change of variables. Apply the preceding measure-preserving measurable equivalence to a restricted nonnegative integral. This gives

$$\int_{t>0} F(a, t + q(a))\, d\mu(a)\, dt = \int_{u>q(a)} F(a, u)\, d\mu(a)\, du.$$

The theorem `setLIntegral_comp_preimage_emb` applies to extended nonnegative tests without a finiteness assumption. Specializing to $A = \mathbb{R}^n$ with entry measure and $q = q_B$ gives `lintegral_schur_coordinates`. The shear has unit measure factor even though it need not be a linear or orthogonal map.

```
/-- Exact nonnegative-integral Schur shear. No integrability or finiteness
is assumed and no Jacobian is inferred from a matrix norm instance. -/
theorem lintegral_shear_epigraph {A : Type*} [MeasurableSpace A]
    (mu : Measure A) [SFinite mu] (q : A → ℝ) (hq : Measurable q)
    (F : A × ℝ → ℝ≥∞0) :
    ∫⁻ w in {w : A × ℝ | 0 < w.2}, F (w.1, w.2 + q w.1) ∂mu.prod volume =
      ∫⁻ w in {w : A × ℝ | q w.1 < w.2}, F w ∂mu.prod volume := by
  have h := (shear_preserves mu q hq).setLIntegral_comp_preimage_emb
    (shear q hq).measurableEmbedding F {w : A × ℝ | q w.1 < w.2}
  rw [shear_preimage_epigraph] at h
  exact h

theorem lintegral_schur_coordinates {n : ℕ} (B : Matrix (Fin n) (Fin n) ℝ)
    (F : (Fin n → ℝ) × ℝ → ℝ≥∞0) :
    ∫⁻ w in {w : (Fin n → ℝ) × ℝ | 0 < w.2},
      F (w.1, w.2 + schurQuad B w.1) ∂(coordinateLebesgue (Fin n)).prod volume =
      ∫⁻ w in {w : (Fin n → ℝ) × ℝ | schurQuad B w.1 < w.2},
        F w ∂(coordinateLebesgue (Fin n)).prod volume := by
  simpa only [coordinateLebesgue_eq_volume] using
    lintegral_shear_epigraph (volume : Measure (Fin n → ℝ))
      (schurQuad B) (schurQuad_continuous B).measurable F
```

Dimension and density arithmetic. On $B > 0$ and $t > 0$ all determinant powers have positive bases. The Schur determinant identity therefore gives

$$(\det B)^{\eta+1/2}(\det B)^{-1/2}t^{\eta} = \det G(B, z, t + q_B(z))^{\eta}.$$

For $k \le d$, casting $d - k$ correctly yields $(d - k)/2 - 1 = \eta_{k+1,d}$. The normalization for the Gram induction is

$$A_{k,d} = \prod_{i=0}^{k-1} a_{d-i}.$$

It is positive for $k \le d$, since each dimension $d - i$ is positive. Its empty value is one and its recurrence is $A_{k+1,d} = A_{k,d}a_{d-k}$.

```
/-- Determinant exponent bookkeeping on the open Schur cone. -/
lemma schur_density_factor {n : ℕ} {B : Matrix (Fin n) (Fin n) ℝ}
    (hB : B.PosDef) (z : Fin n → ℝ) {t : ℝ} (ht : 0 < t) (eta : ℝ) :
    B.det ^ (eta + 1/2) * B.det ^ (-(1/2 : ℝ)) * t ^ eta =
      (border B z (t + schurQuad B z)).det ^ eta := by
  rw [det_border B (isUnit_iff_ne_zero.2 hB.det_pos.ne')]
  rw [add_sub_cancel_right, Real.mul_rpow hB.det_pos.le ht.le]
  rw ←[ Real.rpow_add hB.det_pos]
  congr 1
  congr 1
  ring

/-- Natural subtraction occurs only in a dimension, not in the density exponent. -/
lemma schur_exponent {k d : ℕ} (hkd : k ≤ d) :
    ((d - k : ℕ) : ℝ) / 2 - 1 = wishartEta (k+1) d := by
  rw [Nat.cast_sub hkd]
  unfold wishartEta
  push_cast
  ring

def gramMass (k d : ℕ) : ℝ :=
  ∏ i : Fin k, O77.RadialGram.radialMass (d - i.val)

lemma gramMass_pos {k d : ℕ} (hkd : k ≤ d) : 0 < gramMass k d := by
  apply Finset.prod_pos
```

```
  intro i _
  apply O77.RadialGram.radialMass_pos
  omega

@[simp] lemma gramMass_zero (d : ℕ) : gramMass 0 d = 1 := by
  simp [gramMass]

lemma gramMass_succ (k d : ℕ) :
    gramMass (k+1) d = gramMass k d * O77.RadialGram.radialMass (d-k) := by
  simp only [gramMass, Fin.prod_univ_castSucc, Fin.val_castSucc, Fin.val_last]
```

The two measures to be compared. For $k, d \in \mathbb{N}$ and a matrix test $F : \mathbb{R}^{k\times k} \to [0,\infty]$, define

$$\mathcal{Q}_{k,d}(F) = \int_{\mathbb{R}^{k\times d}} e^{-\|X\|_F^2} F(XX^{\mathsf{T}})\, dX, \qquad \mathcal{C}_{k,d}(F) = \int_{G>0} e^{-\operatorname{tr} G} (\det G)^{\eta_{k,d}} F(G) \prod_{i\le j} dG_{ij}.$$

These are `gaussianGramLIntegral` and `coneLIntegral`. In the cone integral, the negative determinant exponent is used only under restriction to $G > 0$. The test `determinantTestrhoT` is ofReal$(\det(I + TG)^{-\rho})$. Substitution gives exactly the original left Gram integral, by `ENNReal.ofReal_mul` and positivity of the Gaussian. This last adapter is an algebraic equality even for arbitrary real parameters; the Wishart application uses $\rho, T \ge 0$.

```
/-- The exact matrix integral for a nonnegative Gram test. -/
def gaussianGramLIntegral (k d : ℕ)
    (F : Matrix (Fin k) (Fin k) ℝ → ℝ≥∞0) : ℝ≥∞0 :=
  ∫⁻ X : Matrix (Fin k) (Fin d) ℝ,
    ENNReal.ofReal (Real.exp (-frobeniusSq X)) * F (X * X.transpose)
      ∂matrixLebesgue (Fin k) (Fin d)

/-- The specified upper-entry cone integral. Negative powers are evaluated
only under restriction to the positive definite cone. -/
def coneLIntegral (k d : ℕ)
    (F : Matrix (Fin k) (Fin k) ℝ → ℝ≥∞0) : ℝ≥∞0 :=
  ∫⁻ a in {a : UpperCoordinates k | (fromUpper a).PosDef},
    ENNReal.ofReal (Real.exp (-(fromUpper a).trace) *
      (fromUpper a).det ^ wishartEta k d) * F (fromUpper a) ∂upperLebesgue k

def determinantTest {k : ℕ} (rho T : ℝ)
    (G : Matrix (Fin k) (Fin k) ℝ) : ℝ≥∞0 :=
  ENNReal.ofReal ((1 + T • G).det ^ (-rho))

/-- Adapter to the actual inherited left integral, not a new Wishart predicate.
Its all-real statement is an algebraic equality of totalized integrands;
the intended downstream domain remains rho,T >= 0. -/
theorem gaussianGramLIntegral_determinantTest (k d : ℕ) (rho T : ℝ) :
    gaussianGramLIntegral k d (determinantTest rho T) =
      leftGramLaplaceLIntegral k d rho T := by
  unfold gaussianGramLIntegral leftGramLaplaceLIntegral
  apply lintegral_congr_ae
  filter_upwards with X
  exact (ENNReal.ofReal_mul (Real.exp_pos (-frobeniusSq X)).le).symm

-- The Gaussian-Gram pushforward is proved in the later O77GramPushforward
-- module; here we supply its Schur-coordinate algebra and measures.
end O77.GramBorder
end
```

53 The rank-deficient exceptional set has entry measure zero

Here X is a $k \times d$ real matrix regarded as a family of k row vectors in $\mathbb{R}^d$. We prove that its rows are independent almost everywhere in their actual product Lebesgue measure whenever $k \leq d$. The argument uses induction and proper-subspace nullity; it does not require a separately postulated polynomial-zero-set theorem. In particular the square case $k = d$ is included.

```
import Mathlib
import O77

set_option autoImplicit false
noncomputable section
open Set MeasureTheory Matrix Module
open scoped ENNReal BigOperators

namespace O77.GramRankNull
open O77.Residual
```

Borel rows and proper spans. Linear independence of a finite vector family is an open condition, by `isOpen_setOfPred_linearIndependent`, hence Borel in the row coordinates. An independent family of $k < d$ vectors has span of dimension k, by `finrank_span_eq_card`, so its span is a proper subspace. The additive Haar result `Measure.addHaar_submodule` gives that span Lebesgue measure zero. The displayed adapter replaces library volume by the manuscript's equal coordinate measure.

```
/-- Linear independence is a Borel condition in the actual row coordinates. -/
lemma measurableSet_linearIndependent_rows (k d : ℕ) :
    MeasurableSet {X : Matrix (Fin k) (Fin d) ℝ | LinearIndependent ℝ X} := by
  exact (isOpen_setOfPred_linearIndependent
    (𝕂 := ℝ) (E := Fin d → ℝ)).measurableSet

/-- Fewer than `d` independent rows span a proper subspace of ℝ`^d`. -/
lemma rowSpan_ne_top {k d : ℕ} (hkd : k < d)
    {X : Matrix (Fin k) (Fin d) ℝ} (hX : LinearIndependent ℝ X) :
    Submodule.span ℝ (Set.range X) ≠ ⊤ := by
  intro htop
  have hdim := finrank_span_eq_card hX
  rw [htop, finrank_top] at hdim
  simp only [Module.finrank_fintype_fun_eq_card, Fintype.card_fin] at hdim
  omega

/-- This is the library proper-subspace null theorem with the manuscript's
coordinate measure, not an inferred Euclidean matrix-volume convention. -/
lemma coordinateLebesgue_rowSpan {k d : ℕ} (hkd : k < d)
    {X : Matrix (Fin k) (Fin d) ℝ} (hX : LinearIndependent ℝ X) :
    coordinateLebesgue (Fin d) (Submodule.span ℝ (Set.range X) : Set (Fin d → ℝ)) = 0 := by
  rw [coordinateLebesgue_eq_volume]
  exact Measure.addHaar_submodule volume _ (rowSpan_ne_top hkd hX)
```

Inserting a row preserves the product description. The operation $(x, X) \mapsto \mathrm{Fin.cons}(x, X)$ is continuous, coordinate by coordinate. The exact measurable equivalence `MeasurableEquiv.piFinSuccAbove` at index zero splits the product of $k+1$ row spaces into a first row and the remaining k rows. Its measure-preservation theorem transfers almost-everywhere Borel row predicates through this split. The auxiliary predicate is kept abstract so that this measure argument does not depend on how linear independence is represented internally.

```
private lemma continuous_cons_rows (k d : ℕ) :
    Continuous (fun p : (Fin d → ℝ) × (Fin k → Fin d → ℝ) =>
```

```
      Fin.cons α( := fun _ : Fin (k + 1) => Fin d → ℝ) p.1 p.2) := by
  apply continuous_pi
  intro i
  refine Fin.cases ?_ (fun j => ?_) i
  · exact (continuous_fst : Continuous (Prod.fst :
      (Fin d → ℝ) × (Fin k → Fin d → ℝ) → Fin d → ℝ))
  · exact (continuous_apply j).comp
      (continuous_snd : Continuous (Prod.snd :
        (Fin d → ℝ) × (Fin k → Fin d → ℝ) → (Fin k → Fin d → ℝ)))

/-- Transport an abstract Borel row predicate through the exact entry split.
Keeping the predicate abstract avoids reducing linear independence inside a
change of product-measure carrier. -/
private lemma ae_of_ae_cons_rows {k d : ℕ}
    (P : (Fin (k+1) → Fin d → ℝ) → Prop)
    (hP : MeasurableSet {X | P X})
    (hp : ∀ᵐ p ∂(coordinateLebesgue (Fin d)).prod
        (Measure.pi (fun _ : Fin k => coordinateLebesgue (Fin d))),
      P (Fin.cons p.1 p.2)) :
    ∀ᵐ X ∂Measure.pi (fun _ : Fin (k+1) => coordinateLebesgue (Fin d)), P X := by
  have hsplit : MeasurePreserving
      (MeasurableEquiv.piFinSuccAbove (fun _ : Fin (k+1) => Fin d → ℝ) 0).symm
      ((coordinateLebesgue (Fin d)).prod
        (Measure.pi (fun _ : Fin k => coordinateLebesgue (Fin d))))
      (Measure.pi (fun _ : Fin (k+1) => coordinateLebesgue (Fin d))) :=
    (measurePreserving_piFinSuccAbove
      (fun _ : Fin (k+1) => coordinateLebesgue (Fin d)) 0).symm
  rw ←[ hsplit.map_eq]
  apply (ae_map_iff hsplit.measurable.aemeasurable hP).2
  filter_upwards [hp] with p hp
  simpa only [MeasurableEquiv.piFinSuccAbove_symm_apply, Fin.insertNthEquiv,
    Equiv.coe_fn_mk, Fin.insertNth_zero'] using hp
```

Induction in the number of rows. The empty family is independent. Suppose $k+1 \le d$. By induction the existing k rows are independent almost everywhere. For each such fixed family their span is proper, so almost every new row lies outside that span. The lemma `LinearIndependent.finCons` then gives independence of the enlarged family. The Borel property permits `Measure.ae_prod_iff_ae_ae` and `Measure.ae_ae_comm` to exchange and combine these almost-everywhere statements. Finally the insertion equivalence returns to the original row-product carrier. The public matrix theorem merely unfolds `matrixLebesgue` once.

```
/-- Carry out the induction entirely in finite function coordinates; the
public matrix-measure adapter below unfolds its measure only once. -/
private theorem ae_linearIndependent_rowFunctions {k d : ℕ} (hkd : k ≤ d) :
    ∀ᵐ X ∂Measure.pi (fun _ : Fin k => coordinateLebesgue (Fin d)),
      LinearIndependent ℝ X := by
  revert hkd
  induction k with
  | zero =>
      intro _
      exact Filter.Eventually.of_forall fun _ => linearIndependent_empty_type
  | succ k ih =>
      intro hkd
      have hi : ∀ᵐ X ∂Measure.pi (fun _ : Fin k => coordinateLebesgue (Fin d)),
          LinearIndependent ℝ X :=
        ih (by omega)
      have hiter : ∀ᵐ X ∂Measure.pi (fun _ : Fin k => coordinateLebesgue (Fin d)),
          ∀ᵐ x ∂coordinateLebesgue (Fin d), LinearIndependent ℝ (Fin.cons x X) := by
        filter_upwards [hi] with X hX
        have hout : ∀ᵐ x ∂coordinateLebesgue (Fin d),
```

```
            x ∉ Submodule.span ℝ (Set.range X) := by
          apply ae_iff.2
          have hset : {x : Fin d → ℝ | ¬ x ∉ Submodule.span ℝ (Set.range X)} =
              (Submodule.span ℝ (Set.range X) : Set (Fin d → ℝ)) := by
            ext x
            exact not_not
          rw [hset]
          exact coordinateLebesgue_rowSpan (by omega : k < d) hX
        filter_upwards [hout] with x hx
        exact hX.finCons hx
      have hm : MeasurableSet
          {p : (Fin d → ℝ) × (Fin k → Fin d → ℝ) |
            LinearIndependent ℝ (Fin.cons p.1 p.2)} :=
        (measurableSet_linearIndependent_rows (k+1) d).preimage
          (continuous_cons_rows k d).measurable
      have hp : ∀ᵐ p ∂(coordinateLebesgue (Fin d)).prod
          (Measure.pi (fun _ : Fin k => coordinateLebesgue (Fin d))),
          LinearIndependent ℝ (Fin.cons p.1 p.2) :=
        (Measure.ae_prod_iff_ae_ae hm).2 ((Measure.ae_ae_comm hm).2 hiter)
      exact ae_of_ae_cons_rows (fun X => LinearIndependent ℝ X)
        (measurableSet_linearIndependent_rows (k+1) d) hp

/-- Original entry-product Lebesgue measure almost surely gives independent
rows whenever the number of rows does not exceed the number of columns.
The successor step uses Tonelli on a Borel set and a proper row-span fiber. -/
theorem ae_linearIndependent_rows {k d : ℕ} (hkd : k ≤ d) :
    ∀ᵐ X ∂matrixLebesgue (Fin k) (Fin d), LinearIndependent ℝ X := by
  unfold matrixLebesgue
  exact ae_linearIndependent_rowFunctions hkd
```

From independent rows to the positive Gram cone. Independence is equivalent to injectivity of row combinations, namely $c \mapsto c^{\mathsf{T}}X$, via `Matrix.vecMul_injective_iff`. The library theorem `Matrix.PosDef.mul_conjTranspose_self` then proves $XX^{\mathsf{T}} > 0$; over $\mathbb{R}$ conjugate transpose is transpose. Combining this algebraic fact with the almost-everywhere independence theorem proves that the complement of the positive Gram cone is null. Since a positive definite matrix has positive determinant, the zero-Gram-determinant set is null as well. Thus later integrals may discard exactly these exceptional rows in original-entry measure.

```
/-- The algebraic adapter uses injectivity of row combinations. -/
lemma posDef_gram_of_linearIndependent_rows {k d : ℕ}
    {X : Matrix (Fin k) (Fin d) ℝ} (hX : LinearIndependent ℝ X) :
    (X * X.transpose).PosDef := by
  have hrow : LinearIndependent ℝ X.row := by simpa only [Matrix.row_def] using hX
  have hinj : Function.Injective X.vecMul := Matrix.vecMul_injective_iff.2 hrow
  have htranspose : X.conjTranspose = X.transpose := by
    ext i j
    simp [Matrix.conjTranspose_apply, Matrix.transpose_apply]
  simpa only [htranspose] using Matrix.PosDef.mul_conjTranspose_self X hinj

/-- Exact full-rank exceptional-set input for Gaussian Gram disintegration. -/
theorem ae_posDef_gram {k d : ℕ} (hkd : k ≤ d) :
    ∀ᵐ X ∂matrixLebesgue (Fin k) (Fin d), (X * X.transpose).PosDef := by
  filter_upwards [ae_linearIndependent_rows hkd] with X hX
  exact posDef_gram_of_linearIndependent_rows hX

theorem matrixLebesgue_not_posDef_gram {k d : ℕ} (hkd : k ≤ d) :
    matrixLebesgue (Fin k) (Fin d)
      {X : Matrix (Fin k) (Fin d) ℝ | ¬ (X * X.transpose).PosDef} = 0 :=
  ae_iff.1 (ae_posDef_gram hkd)
```

```
/-- The zero-determinant formulation is often convenient for measurable kernels. -/
theorem matrixLebesgue_det_gram_zero {k d : ℕ} (hkd : k ≤ d) :
    matrixLebesgue (Fin k) (Fin d)
      {X : Matrix (Fin k) (Fin d) ℝ | (X * X.transpose).det = 0} = 0 := by
  have h : ∀ᵐ X ∂matrixLebesgue (Fin k) (Fin d), (X * X.transpose).det ≠ 0 := by
    filter_upwards [ae_posDef_gram hkd] with X hX
    exact hX.det_pos.ne'
  simpa only [not_not] using (ae_iff.1 h)

end O77.GramRankNull
end
```

54 Exact independent-entry coordinates for the Schur cone

For symmetric $(n+1)\times(n+1)$ matrices we use coordinates $((a,z),u)$, where a is the vector of upper entries of an $n\times n$ matrix $B=\mathrm{fromUpper}(a)$, $z\in\mathbb{R}^n$ is its last off-diagonal column, and $u\in\mathbb{R}$ is its last diagonal entry. Their measure is $(da\,dz)\,du$, implemented by `borderLebesgue`. We construct the equivalence with all upper entries before applying the nonlinear Schur shear. This prevents confusion between independent symmetric entries and volume on the full square-matrix space.

```
import Mathlib
import O77GramBorder

set_option autoImplicit false
noncomputable section
open Set MeasureTheory Matrix
open scoped ENNReal BigOperators

namespace O77.GramCoordinates
open O77.Residual O77.GramBorder
```

Three disjoint classes of entries. The index equivalence `upperIndexBorder` partitions $i\le j$ in size $n+1$ into $j<n$, $j=n$ with $i<n$, and $i=j=n$. The inverse tests these cases, and its two inverse laws are proved by finite-index arithmetic. The measurable map `assembleUpper` composes finite-product splitting, the singleton-coordinate equivalence, and reindexing by this bijection. The corresponding library product-measure theorems show that it preserves $da\,dz\,du$ exactly.

```
abbrev BorderCoordinates (n : ℕ) := (UpperCoordinates n × (Fin n → ℝ)) × ℝ
abbrev BorderIndex (n : ℕ) := (UpperIndex n ⊕ Fin n) ⊕ Fin 1

def borderLebesgue (n : ℕ) : Measure (BorderCoordinates n) :=
  ((upperLebesgue n).prod (coordinateLebesgue (Fin n))).prod volume

/-- The three disjoint classes of independent entries: old block, last
column above the diagonal, and last diagonal entry. -/
def upperIndexBorder (n : ℕ) : BorderIndex n ≃ UpperIndex (n+1) where
  toFun
    | Sum.inl (Sum.inl ij) =>
      ⟨(ij.1.1.castSucc, ij.1.2.castSucc), by simpa using ij.2⟩
    | Sum.inl (Sum.inr i) => ⟨(i.castSucc, Fin.last n), Fin.le_last _⟩
    | Sum.inr _ => ⟨(Fin.last n, Fin.last n), le_rfl⟩
  invFun ij :=
    if hj : ij.1.2.val < n then
      Sum.inl (Sum.inl ⟨(⟨ij.1.1.val, lt_of_le_of_lt ij.2 hj⟩,
        ⟨ij.1.2.val, hj⟩), ij.2⟩)
    else if hi : ij.1.1.val < n then
```

```
      Sum.inl (Sum.inr ⟨ij.1.1.val, ⟩hi)
    else Sum.inr 0
  left_inv x := by
    rcases x with (ij | i) | u
    · simp
    · simp
    · have : u = 0 := Subsingleton.elim _ _
      subst u
      simp
  right_inv ij := by
    rcases ij with ⟨⟨i,⟩j, ⟩hij
    dsimp
    split_ifs with hj hi
    · rfl
    · apply Subtype.ext
      apply Prod.ext
      · rfl
      · apply Fin.ext
        have := j.isLt
        simp only [Fin.val_last]
        omega
    · apply Subtype.ext
      apply Prod.ext <;> apply Fin.ext
      · have := i.isLt
        simp only [Fin.val_last]
        omega
      · have := j.isLt
        simp only [Fin.val_last]
        omega

/-- Assembly is a composition of the library's exact product-coordinate
maps, not a volume claim about an embedding in all square matrices. -/
def assembleUpper (n : ℕ) : BorderCoordinates n ≃ᵐ UpperCoordinates (n+1) :=
  ((MeasurableEquiv.sumPiEquivProdPi (fun _ : UpperIndex n ⊕ Fin n => ℝ)).symm.prodCongr
    (MeasurableEquiv.funUnique (Fin 1) ℝ).symm).trans
      ((MeasurableEquiv.sumPiEquivProdPi (fun _ : BorderIndex n => ℝ)).symm.trans
        (MeasurableEquiv.piCongrLeft (fun _ : UpperIndex (n+1) => ℝ)
          (upperIndexBorder n)))

lemma assembleUpper_preserves (n : ℕ) :
    MeasurePreserving (assembleUpper n) (borderLebesgue n) (upperLebesgue (n+1)) := by
  have h1 := (measurePreserving_sumPiEquivProdPi_symm
    (fun _ : UpperIndex n ⊕ Fin n => (volume : Measure ℝ))).prod
      (measurePreserving_funUnique (volume : Measure ℝ) (Fin 1)).symm
  have h2 := measurePreserving_sumPiEquivProdPi_symm
    (fun _ : BorderIndex n => (volume : Measure ℝ))
  have h3 := measurePreserving_piCongrLeft
    (fun _ : UpperIndex (n+1) => (volume : Measure ℝ)) (upperIndexBorder n)
  exact h3.comp (h2.comp h1)
```

Checking the coordinate map on entries. Evaluation at an image index is proved with `MeasurableEquiv.piCongrLeft_apply_apply`; this matters because the map carries dependent coordinate types, rather than being definitionally ordinary precomposition. The next three lemmas identify its old upper block, final column, and final diagonal value. The row-and-column index equivalence `lastIndex` then numbers the same block matrix by `Fin(n+1)`.

```
/-- Evaluate the dependent coordinate permutation at an image index.
This avoids treating `piCongrLeft` as definitionally equal to ordinary
precomposition, which would discard its transport of the coordinate type. -/
lemma assembleUpper_apply_index (n : ℕ) (v : BorderCoordinates n)
    (x : BorderIndex n) :
```

```
    assembleUpper n v (upperIndexBorder n x) =
      Sum.elim (Sum.elim v.1.1 v.1.2) (fun _ : Fin 1 => v.2) x := by
  unfold assembleUpper
  rw [MeasurableEquiv.trans_apply, MeasurableEquiv.trans_apply,
    MeasurableEquiv.piCongrLeft_apply_apply]
  rcases x with (ij | i) | u
  · rfl
  · rfl
  · change (MeasurableEquiv.funUnique (Fin 1) ℝ).symm v.2 u = v.2
    exact congrFun ((MeasurableEquiv.funUnique (Fin 1) ℝ).symm_apply_apply
      (fun _ : Fin 1 => v.2)) u

@[simp] lemma assembleUpper_old {n : ℕ} (a : UpperCoordinates n)
    (z : Fin n → ℝ) (u : ℝ) (ij : UpperIndex n) :
    assembleUpper n ((a,z),u)
      ((ij.1.1.castSucc, ij.1.2.castSucc), by simpa using ij).2 = a ij := by
  exact assembleUpper_apply_index n ((a,z),u) (Sum.inl (Sum.inl ij))

@[simp] lemma assembleUpper_column {n : ℕ} (a : UpperCoordinates n)
    (z : Fin n → ℝ) (u : ℝ) (i : Fin n) :
    assembleUpper n ((a,z),u) ⟨(i.castSucc, Fin.last n), Fin.le_last ⟩_ = z i := by
  exact assembleUpper_apply_index n ((a,z),u) (Sum.inl (Sum.inr i))

@[simp] lemma assembleUpper_last {n : ℕ} (a : UpperCoordinates n)
    (z : Fin n → ℝ) (u : ℝ) :
    assembleUpper n ((a,z),u) ⟨(Fin.last n, Fin.last n), ⟩le_rfl = u := by
  exact assembleUpper_apply_index n ((a,z),u) (Sum.inr 0)

/-- Shared numbering of rows and columns for the existing block algebra. -/
def lastIndex (n : ℕ) : Fin n ⊕ Fin 1 ≃ Fin (n+1) where
  toFun := Sum.elim Fin.castSucc (fun _ => Fin.last n)
  invFun i := if hi : i.val < n then Sum.inl ⟨i.val,⟩hi else Sum.inr 0
  left_inv x := by
    rcases x with i | u
    · simp
    · have : u = 0 := Subsingleton.elim _ _
      subst u
      simp
  right_inv i := by
    dsimp
    split_ifs with hi
    · rfl
    · apply Fin.ext
      have := i.isLt
      change n = i.val
      omega
```

Checking the assembled symmetric matrix. Split each matrix entry according to whether its row and column are old indices or the final index. The four evaluation lemmas show that the reconstructed symmetric matrix is exactly $G(B, z, u)$. Reindexing simultaneously in rows and columns therefore proves `fromUpper_assemble_submatrix`; it preserves determinant and trace. The following two identities become

$$\det G(B, z, u) = \det B\,(u - q_B(z)), \qquad \operatorname{tr} G(B, z, u) = \operatorname{tr} B + u.$$

The determinant formula assumes $B > 0$ to obtain its invertibility.

```
lemma fromUpper_assemble_old {n : ℕ} (a : UpperCoordinates n)
    (z : Fin n → ℝ) (u : ℝ) (i j : Fin n) :
    fromUpper (assembleUpper n ((a,z),u)) i.castSucc j.castSucc = fromUpper a i j := by
  by_cases hij : i ≤ j
```

```
  · have hij' : i.castSucc ≤ j.castSucc := hij
    unfold fromUpper
    rw [dite_eq_left hij', dite_eq_left hij]
    exact assembleUpper_old a z u ⟨(i,j),⟩hij
  · have hij' : ¬ i.castSucc ≤ j.castSucc := hij
    unfold fromUpper
    rw [dite_eq_right hij', dite_eq_right hij]
    exact assembleUpper_old a z u ⟨(j,i),le_of_not_ge ⟩hij

lemma fromUpper_assemble_column {n : ℕ} (a : UpperCoordinates n)
    (z : Fin n → ℝ) (u : ℝ) (i : Fin n) :
    fromUpper (assembleUpper n ((a,z),u)) i.castSucc (Fin.last n) = z i := by
  unfold fromUpper
  rw [dite_eq_left (Fin.le_last _)]
  exact assembleUpper_column a z u i

lemma fromUpper_assemble_row {n : ℕ} (a : UpperCoordinates n)
    (z : Fin n → ℝ) (u : ℝ) (i : Fin n) :
    fromUpper (assembleUpper n ((a,z),u)) (Fin.last n) i.castSucc = z i := by
  have hi : ¬ Fin.last n ≤ i.castSucc := by simp
  simp [fromUpper, hi]

lemma fromUpper_assemble_last {n : ℕ} (a : UpperCoordinates n)
    (z : Fin n → ℝ) (u : ℝ) :
    fromUpper (assembleUpper n ((a,z),u)) (Fin.last n) (Fin.last n) = u := by
  simp [fromUpper]

lemma fromUpper_assemble_submatrix {n : ℕ} (a : UpperCoordinates n)
    (z : Fin n → ℝ) (u : ℝ) :
    (fromUpper (assembleUpper n ((a,z),u))).submatrix (lastIndex n) (lastIndex n) =
      border (fromUpper a) z u := by
  ext i j
  rcases i with i | i <;> rcases j with j | j
  · exact fromUpper_assemble_old a z u i j
  · exact fromUpper_assemble_column a z u i
  · exact fromUpper_assemble_row a z u j
  · exact fromUpper_assemble_last a z u

lemma det_fromUpper_assemble {n : ℕ} (a : UpperCoordinates n)
    (ha : (fromUpper a).PosDef) (z : Fin n → ℝ) (u : ℝ) :
    (fromUpper (assembleUpper n ((a,z),u))).det =
      (fromUpper a).det * (u - schurQuad (fromUpper a) z) := by
  rw ←[ Matrix.det_submatrix_equiv_self (lastIndex n), fromUpper_assemble_submatrix]
  exact det_border _ (isUnit_iff_ne_zero.2 ha.det_pos.ne') z u

lemma trace_fromUpper_assemble {n : ℕ} (a : UpperCoordinates n)
    (z : Fin n → ℝ) (u : ℝ) :
    (fromUpper (assembleUpper n ((a,z),u))).trace = (fromUpper a).trace + u := by
  calc
    _ = ((fromUpper (assembleUpper n ((a,z),u))).submatrix
        (lastIndex n) (lastIndex n)).trace :=
      ((lastIndex n).sum_comp
        (fun i => fromUpper (assembleUpper n ((a,z),u)) i i)).symm
    _ = _ := by rw [fromUpper_assemble_submatrix, trace_border]
```

Identifying the cone exactly. The assembled matrix is positive definite if and only if $B > 0$ and $q_B(z) < u$. In the forward direction, restriction of a positive definite quadratic form to the first n coordinates proves $B > 0$, and the Schur criterion supplies the strict inequality. In the reverse direction the bordered Schur criterion constructs positivity on the sum-index carrier, and the inverse index equivalence transports it back. Thus the statement controls the

entire cone, rather than only a subset on which a chart is available.

```
lemma posDef_fromUpper_assemble_iff {n : ℕ} (a : UpperCoordinates n)
    (z : Fin n → ℝ) (u : ℝ) :
    (fromUpper (assembleUpper n ((a,z),u))).PosDef ↔
      (fromUpper a).PosDef ∧ schurQuad (fromUpper a) z < u := by
  constructor
  · intro h
    have ha : (fromUpper a).PosDef := by
      have hsub := h.submatrix (e := Fin.castSucc) (Fin.castSucc_injective n)
      have heq : (fromUpper (assembleUpper n ((a,z),u))).submatrix
          Fin.castSucc Fin.castSucc = fromUpper a := by
        ext i j
        exact fromUpper_assemble_old a z u i j
      rwa [heq] at hsub
    refine ⟨ha, ?_⟩
    have hb := h.submatrix (lastIndex n).injective
    rw [fromUpper_assemble_submatrix, posDef_border_iff ha] at hb
    exact sub_pos.mp hb
  · rintro ⟨ha, hu⟩
    have hb := (posDef_border_iff ha z u).2 (sub_pos.mpr hu)
    rw ←[ fromUpper_assemble_submatrix a z u] at hb
    have hc := hb.submatrix (lastIndex n).symm.injective
    have heq : ((fromUpper (assembleUpper n ((a,z),u))).submatrix
        (lastIndex n) (lastIndex n)).submatrix
        (lastIndex n).symm (lastIndex n).symm =
          fromUpper (assembleUpper n ((a,z),u)) := by
      ext i j
      simp only [Matrix.submatrix_apply, Equiv.apply_symm_apply]
    rwa [heq] at hc
```

A globally measurable Schur function. The matrix inverse is globally measurable because the totalized formula $B^{-1} = (\det B)^{-1}\operatorname{adj}(B)$ is built from polynomial coordinates and scalar inverse. This fact is used only to establish measurability of $q(B,z) = z^{\mathsf{T}}B^{-1}z$. The identities involving determinant powers and actual inversion remain restricted to $B > 0$. Composing the shear $((a,z),t) \mapsto ((a,z),t+q_B(z))$ with `assembleUpper` defines `schurAssemble`, a measure-preserving measurable equivalence on the full coordinate spaces.

```
/-- Totalized inversion is used only for measurability of a global map.
Every density identity below is restricted to positive determinants. -/
lemma measurable_inverse {n : ℕ} :
    Measurable (fun B : Matrix (Fin n) (Fin n) ℝ => ⁻B¹) := by
  have hc : Continuous (fun B : Matrix (Fin n) (Fin n) ℝ => B) := continuous_id
  have h : Measurable (fun B : Matrix (Fin n) (Fin n) ℝ =>
      B.⁻det¹ • B.adjugate) :=
    hc.matrix_det.measurable.fun_inv.fun_smul hc.matrix_adjugate.measurable
  simpa only [Matrix.inv_def, Ring.inverse_eq_inv'] using h

def jointSchur (n : ℕ) (v : UpperCoordinates n × (Fin n → ℝ)) : ℝ :=
  schurQuad (fromUpper v.1) v.2

lemma jointSchur_measurable (n : ℕ) : Measurable (jointSchur n) := by
  have hb : Measurable (fun v : UpperCoordinates n × (Fin n → ℝ) =>
      (fromUpper v.1)⁻¹) :=
    measurable_inverse.comp ((fromUpper_continuous n).measurable.comp measurable_fst)
  unfold jointSchur schurQuad
  simp only [Matrix.mul_apply, Matrix.replicateRow_apply, Matrix.replicateCol_apply]
  fun_prop

/-- Full Schur coordinates, including the actual independent-entry assembly. -/
def schurAssemble (n : ℕ) : BorderCoordinates n ≃ᵐ UpperCoordinates (n+1) :=
```

```
  (shear (jointSchur n) (jointSchur_measurable n)).trans (assembleUpper n)

lemma schurAssemble_preserves (n : ℕ) :
    MeasurePreserving (schurAssemble n) (borderLebesgue n) (upperLebesgue (n+1)) := by
  let : SigmaFinite (upperLebesgue n) := by unfold upperLebesgue; infer_instance
  exact (assembleUpper_preserves n).comp
    (shear_preserves ((upperLebesgue n).prod (coordinateLebesgue (Fin n)))
      (jointSchur n) (jointSchur_measurable n))
```

The Schur cone as a product domain. The previous positivity criterion makes the preimage of the positive cone exactly $B > 0$ and $t > 0$. Therefore `setLIntegral_comp_preimage_emb` gives an equality of restricted nonnegative integrals for every test F of the assembled matrix. This coordinate change needs neither integrability nor finiteness, and the test need not be assumed measurable for this particular measurable-equivalence identity.

```
lemma schurAssemble_preimage_cone (n : ℕ) :
    schurAssemble n ⁻¹' {a : UpperCoordinates (n+1) | (fromUpper a).PosDef} =
      {w : BorderCoordinates n | (fromUpper w.1.1).PosDef ∧ 0 < w.2} := by
  ext w
  change (fromUpper (assembleUpper n (w.1, w.2 + jointSchur n w.1))).PosDef ↔ _
  rw [posDef_fromUpper_assemble_iff]
  simp [jointSchur]

/-- Exact cone change of variables for every extended nonnegative test.
The measurable-equivalence theorem does not require the test to be finite
or measurable. The matrix argument remains the actual `fromUpper` map. -/
theorem lintegral_schur_cone (n : ℕ)
    (F : Matrix (Fin (n+1)) (Fin (n+1)) ℝ → ℝ≥∞0) :
    ∫⁻( w in {w : BorderCoordinates n | (fromUpper w.1.1).PosDef ∧ 0 < w.2},
      F (fromUpper (schurAssemble n w)) ∂borderLebesgue n) =
      ∫⁻ a in {a : UpperCoordinates (n+1) | (fromUpper a).PosDef},
        F (fromUpper a) ∂upperLebesgue (n+1) := by
  have h := (schurAssemble_preserves n).setLIntegral_comp_preimage_emb
    (schurAssemble n).measurableEmbedding (fun a => F (fromUpper a))
      {a : UpperCoordinates (n+1) | (fromUpper a).PosDef}
  rw [schurAssemble_preimage_cone] at h
  exact h
```

Measurability of the symmetric cone. Every matrix reconstructed from upper entries is symmetric. Positive semidefiniteness is therefore the intersection, over all vectors x, of the closed conditions $x^{\mathsf{T}}Bx \geq 0$. Each quadratic evaluation is continuous in the independent entries, so the semidefinite cone is closed. Positive definiteness is equivalent to positive semidefiniteness together with nonzero determinant, making it Borel. This does not assert openness of the symmetric cone in the larger space of all square matrices.

```
lemma isClosed_posSemidef_upper (n : ℕ) :
    IsClosed {a : UpperCoordinates n | (fromUpper a).PosSemidef} := by
  have hh (a : UpperCoordinates n) : (fromUpper a).IsHermitian :=
    Matrix.isHermitian_iff_isSymm.2 (fromUpper_isSymm a)
  have heq : {a : UpperCoordinates n | (fromUpper a).PosSemidef} =
      ⋂ x : Fin n → ℝ,
        {a : UpperCoordinates n | 0 ≤ dotProduct (star x) (fromUpper a ᵥ* x)} := by
    ext a
    simp only [mem_ofPred_eq, mem_iInter, Matrix.posSemidef_iff_dotProduct_mulVec,
      hh a, true_and]
  rw [heq]
  apply isClosed_iInter
  intro x
  apply isClosed_le continuous_const
```

```
  have hc := fromUpper_continuous n
  unfold Matrix.mulVec dotProduct
  fun_prop

/-- Measurability is on the specified symmetric-entry carrier. It is not
an assertion that positive definite matrices are open in all square matrices. -/
lemma measurableSet_posDef_upper (n : ℕ) :
    MeasurableSet {a : UpperCoordinates n | (fromUpper a).PosDef} := by
  have heq : {a : UpperCoordinates n | (fromUpper a).PosDef} =
      {a : UpperCoordinates n | (fromUpper a).PosSemidef} n
        {a : UpperCoordinates n | (fromUpper a).det ≠ 0} := by
    ext a
    constructor
    · intro h
      exact ⟨h.posSemidef, h.det_pos.ne'⟩
    · rintro ⟨h,⟩hdet
      exact h.posDef_iff_det_ne_zero.2 hdet
  rw [heq]
  exact (isClosed_posSemidef_upper n).measurableSet.inter
    (isClosed_eq (fromUpper_continuous n).matrix_det continuous_const).measurableSet.compl
```

The density in Schur coordinates. On $B > 0, t > 0$, the determinant and trace simplify to $\det B\, t$ and $\operatorname{tr} B + t + q_B(z)$. Substitute these pointwise identities in the restricted cone integral. The result `coneLIntegral_schur` is valid for arbitrary n, d: the dimension bound is needed by the later Gram projection, while this is a coordinate identity. Its density is

$$e^{-\operatorname{tr} B - t - q_B(z)} (\det B\, t)^{\eta_{n+1,d}},$$

with all real powers evaluated at positive arguments on the domain of integration.

```
lemma det_schurAssemble {n : ℕ} (w : BorderCoordinates n)
    (hw : (fromUpper w.1.1).PosDef) :
    (fromUpper (schurAssemble n w)).det = (fromUpper w.1.1).det * w.2 := by
  change (fromUpper (assembleUpper n (w.1, w.2 + jointSchur n w.1))).det = _
  rw [det_fromUpper_assemble _ hw]
  simp [jointSchur]

lemma trace_schurAssemble {n : ℕ} (w : BorderCoordinates n) :
    (fromUpper (schurAssemble n w)).trace =
      (fromUpper w.1.1).trace + (w.2 + jointSchur n w.1) := by
  exact trace_fromUpper_assemble _ _ _

/-- Exact pullback of the inherited Gram-cone integral. The parameter d is
arbitrary because this is a coordinate identity; k<=d is needed elsewhere
for the Gaussian projection and positive normalization. All powers occur
on positive determinant factors under the displayed restriction. -/
theorem coneLIntegral_schur (n d : ℕ)
    (F : Matrix (Fin (n+1)) (Fin (n+1)) ℝ → ℝ≥∞0) :
    coneLIntegral (n+1) d F =
      ∫⁻ w in {w : BorderCoordinates n | (fromUpper w.1.1).PosDef ∧ 0 < w.2},
        ENNReal.ofReal
          (Real.exp (-((fromUpper w.1.1).trace + (w.2 + jointSchur n w.1))) *
            ((fromUpper w.1.1).det * w.2) ^ wishartEta (n+1) d) *
          F (fromUpper (schurAssemble n w)) ∂borderLebesgue n := by
  unfold coneLIntegral
  rw ←[ lintegral_schur_cone n
    (fun G => ENNReal.ofReal (Real.exp (-G.trace) * G.det ^ wishartEta (n+1) d) * F G)]
  apply setLIntegral_congr_fun
    (((measurableSet_posDef_upper n).preimage
      (measurable_fst.comp measurable_fst)).inter
        (measurableSet_Ioi.preimage measurable_snd))
```

```
  intro w hw
  dsimp only
  rw [det_schurAssemble w hw.1, trace_schurAssemble]

end O77.GramCoordinates
end
```

55 Constructing coordinates adapted to the actual rows

Let $R \in \mathbb{R}^{k\times d}$ have positive definite Gram matrix $B = RR^{\mathsf{T}}$. Its rows are independent and span a k-dimensional subspace $K \subset \mathbb{R}^d$. The next theorem constructs coordinates for this actual subspace and its orthogonal complement. It proves norm and measure identities for the resulting map; they are not fields supplied as hypotheses.

```
import Mathlib
import O77GramBorder

/-! Actual fixed-row orthogonal coordinates for the Gaussian Gram induction.
The source product has its ordinary product norm; only its L2 wrapper is
identified isometrically with the ambient Euclidean space. -/
set_option autoImplicit false
noncomputable section
open MeasureTheory Matrix
open scoped BigOperators

namespace O77.GramProjectionGeometry
```

Inner products and coordinates. In `EuclideanSpace`, the real inner product is the sum of products of coordinates; the helper below records this formula. The main theorem constructs an invertible $k \times k$ matrix M and a continuous linear equivalence

$$e : \mathbb{R}^k \times \mathbb{R}^{d-k} \longrightarrow \mathbb{R}^d$$

with $MM^{\mathsf{T}} = RR^{\mathsf{T}}$, $Re(v,u) = Mv$, and $\|e(v,u)\|_2^2 = \|v\|_2^2 + \|u\|_2^2$, preserving the exact product volume. The product carrier has its ordinary product norm; an explicit L^2 type synonym handles the orthogonal isometry.

```
private lemma inner_coordinates {n : ℕ}
    (x y : EuclideanSpace ℝ (Fin n)) :
    inner ℝ x y = ∑ i, x i * y i := by
  simp [PiLp.inner_apply, mul_comm]
```

Proof of the adapted-coordinate theorem. Define $T : \mathbb{R}^k \to \mathbb{R}^d$ by $T(c) = R^{\mathsf{T}}c$. If $T(c) = T(c')$, multiplication by R and invertibility of RR^{T} give $c = c'$. Thus $K = \operatorname{range} T$ has dimension k, and `Submodule.finrank_add_finrank_orthogonal` gives $\dim K^{\perp} = d - k$. Choose orthonormal bases b and c of these two actual spaces by `stdOrthonormalBasis`, with their index sets reindexed to the corresponding finite dimensions. Let M_{ij} be the jth coordinate of the ith row of R in b, and let $e(v,u) = b(v) + c(u)$.

Inner-product preservation by b proves $MM^{\mathsf{T}} = RR^{\mathsf{T}}$ entrywise. Its positive determinant implies $\det M \neq 0$. Orthogonality kills the $c(u)$ component when paired with any row of R, proving $Re(v,u) = Mv$. Pythagoras and the isometries of the two bases prove the squared-norm identity. Finally `Submodule.orthogonalDecomposition`, the two orthonormal-basis representations, and `WithLp.volume_preserving_toLp` prove the product-measure identity by composition. This construction includes $k = 0$ and $k = d$; one factor is then zero-dimensional.

The theorem chooses bases for each fixed R and makes no assertion that these choices vary measurably with R.

```
/-- Coordinates adapted to the actual rows of `R`, with exact Euclidean
product measure. There is no assumed orthogonal splitting or chart package.
The square and empty cases use the same construction. -/
theorem exists_projection_coordinates {k d : ℕ}
    (R : Matrix (Fin k) (Fin d) ℝ)
    (hR : (R * R.transpose).PosDef) :
    ∃ (M : Matrix (Fin k) (Fin k) ℝ)
      (e : (EuclideanSpace ℝ (Fin k) × EuclideanSpace ℝ (Fin (d - k)))
        ≃L[ℝ] EuclideanSpace ℝ (Fin d)),
      IsUnit M.det ∧ M * M.transpose = R * R.transpose ∧
      ∀( v u, R ᵥ* (fun j => e (v, u) j) = M ᵥ* (fun i => v i)) ∧
      ∀( v u, ‖e (v, u)‖ ^ 2 = ‖‖v ^ 2 + ‖‖u ^ 2) ∧
      MeasurePreserving e volume volume := by
  classical
  let T : (Fin k → ℝ) →ₗ[ℝ] EuclideanSpace ℝ (Fin d) :=
    (WithLp.linearEquiv 2 ℝ (Fin d → ℝ)).symm.toLinearMap.comp
      R.transpose.mulVecLin
  have hTi : Function.Injective T := by
    intro v w hvw
    have htr : R.transpose ᵥ* v = R.transpose ᵥ* w := by
      have h := congrArg WithLp.ofLp hvw
      exact h
    apply Matrix.mulVec_injective_of_isUnit hR.isUnit
    simpa only [Matrix.mulVec_mulVec] using congrArg (R ᵥ* ·) htr
  let K : Submodule ℝ (EuclideanSpace ℝ (Fin d)) := T.range
  have hk : Module.finrank ℝ K = k := by
    simpa [K] using (LinearMap.finrank_range_of_inj (f := T) hTi)
  have hd : Module.finrank ℝ K.orthogonal = d - k := by
    have h := K.finrank_add_finrank_orthogonal
    simp only [hk, finrank_euclideanSpace, Fintype.card_fin] at h
    omega
  let b : OrthonormalBasis (Fin k) ℝ K :=
    (stdOrthonormalBasis ℝ K).reindex (finCongr hk)
  let c : OrthonormalBasis (Fin (d - k)) ℝ K.orthogonal :=
    (stdOrthonormalBasis ℝ K.orthogonal).reindex (finCongr hd)
  let row : Fin k → K := fun i =>
    ⟨WithLp.toLp 2 (R i), by
      refine ⟨Pi.single i 1, ?⟩_
      ext j
      simp [T]⟩
  let M : Matrix (Fin k) (Fin k) ℝ := fun i j => b.repr (row i) j
  let eK : (K × K.orthogonal) ≃L[ℝ] EuclideanSpace ℝ (Fin d) :=
    (WithLp.prodContinuousLinearEquiv 2 ℝ K K.orthogonal).symm.trans
      K.orthogonalDecomposition.symm.toContinuousLinearEquiv
  let e := (b.repr.symm.toContinuousLinearEquiv.prodCongr
    c.repr.symm.toContinuousLinearEquiv).trans eK
  have he (v : EuclideanSpace ℝ (Fin k))
      (u : EuclideanSpace ℝ (Fin (d - k))) :
      e (v, u) = (b.repr.symm v : EuclideanSpace ℝ (Fin d)) +
        (c.repr.symm u : EuclideanSpace ℝ (Fin d)) := by
    rfl
  have hMM : M * M.transpose = R * R.transpose := by
    ext i j
    change ∑( a, b.repr (row i) a * b.repr (row j) a) =
      ∑ a, R i a * R j a
    calc
      _ = inner ℝ (b.repr (row i)) (b.repr (row j)) :=
        (inner_coordinates (b.repr (row i)) (b.repr (row j))).symm
      _ = inner ℝ (row i) (row j) := b.repr.inner_map_map (row i) (row j)
      _ = ∑ a, R i a * R j a :=
```

```
        inner_coordinates (WithLp.toLp 2 (R i)) (WithLp.toLp 2 (R j))
  have hM : IsUnit M.det := by
    apply isUnit_iff_ne_zero.mpr
    intro hz
    have hdet : (R * R.transpose).det = M.det * M.det := by
      rw ←[ hMM, Matrix.det_mul, Matrix.det_transpose]
    have hn := (Matrix.isUnit_iff_isUnit_det _).mp hR.isUnit
    exact hn.ne_zero (by rw [hdet, hz, zero_mul])
  have hcoord (v : EuclideanSpace ℝ (Fin k))
      (u : EuclideanSpace ℝ (Fin (d - k))) :
      R ᵥ* (fun j => e (v, u) j) = M ᵥ* (fun i => v i) := by
    ext i
    have hperp : inner ℝ (row i : EuclideanSpace ℝ (Fin d))
        (c.repr.symm u : EuclideanSpace ℝ (Fin d)) = 0 :=
      (K.mem_orthogonal _).mp (c.repr.symm u).property _ (row i).property
    calc
      (R ᵥ* (fun j => e (v, u) j)) i =
          inner ℝ (row i : EuclideanSpace ℝ (Fin d)) (e (v, u)) := by
        simp [row, inner_coordinates, Matrix.mulVec, dotProduct]
      _ = inner ℝ (row i) (b.repr.symm v) := by
        rw [he, inner_add_right, hperp, add_zero]
        rfl
      _ = inner ℝ (b.repr (row i)) v :=
        (b.repr.inner_map_eq_flip (row i) v).symm
      _ = (M ᵥ* (fun i => v i)) i := by
        simp [M, inner_coordinates, Matrix.mulVec, dotProduct]
  have hsq (v : EuclideanSpace ℝ (Fin k))
      (u : EuclideanSpace ℝ (Fin (d - k))) :
      ‖e (v, u)‖ ^ 2 = ‖‖v ^ 2 + ‖‖u ^ 2 := by
    change ‖K.orthogonalDecomposition.symm
      (WithLp.toLp 2 (b.repr.symm v, c.repr.symm u))‖ ^ 2 = _
    rw [K.orthogonalDecomposition.symm.norm_map, WithLp.prod_norm_sq_eq_of_L2]
    change ‖b.repr.symm ‖v ^ 2 + ‖c.repr.symm ‖u ^ 2 = _
    rw [b.repr.symm.norm_map, c.repr.symm.norm_map]
  have hmK : MeasurePreserving eK volume volume :=
    K.orthogonalDecomposition.symm.measurePreserving.comp
      (WithLp.volume_preserving_toLp K K.orthogonal)
  have hme : MeasurePreserving e volume volume :=
    hmK.comp (b.repr.symm.measurePreserving.prod c.repr.symm.measurePreserving)
  exact ⟨M, e, hM, hMM, hcoord, hsq, ⟩hme

end O77.GramProjectionGeometry
end
```

56 Projecting one row and integrating its orthogonal radius

Fix $R \in \mathbb{R}^{k\times d}$, put $B = RR^{\mathsf{T}} > 0$, and define $q_B(z) = z^{\mathsf{T}}B^{-1}z$. For a new row $x \in \mathbb{R}^d$, its projected entries are $z = Rx$ and its squared orthogonal radius is $t = \|x\|_2^2 - q_B(Rx)$. We prove their joint pushforward under original Euclidean volume. For the radial density we assume $k < d$, so the orthogonal dimension $d - k$ is positive. The endpoint $d - k = 1$ is included and gives exponent $-1/2$.

```
import Mathlib
import O77GramProjectionGeometry
import O77GramRadial
import O77GramBorder

set_option autoImplicit false
noncomputable section
open Set MeasureTheory Matrix
```

```
open scoped ENNReal BigOperators Topology

namespace O77.GramProjectionIntegral
open O77.Residual O77.GramBorder O77.RadialGram
```

The Schur quadratic form in projected coordinates. Multiplication of the replicated row and column blocks identifies `schurQuadBz` with $z^{\mathsf{T}}B^{-1}z$. If $B = MM^{\mathsf{T}}$ with M invertible, then

$$q_B(Mv) = v^{\mathsf{T}}M^{\mathsf{T}}(MM^{\mathsf{T}})^{-1}Mv = \|v\|_2^2.$$

The inverse-product and inverse-cancellation matrix identities below justify the middle equality. Taking determinants of $MM^{\mathsf{T}} = B$ gives $\sqrt{\det B} = |\det M|$, whence $|\det M|^{-1} = (\det B)^{-1/2}$. Positivity of B is established before any negative real power or square-root simplification.

```
lemma schurQuad_eq_dotProduct {k : ℕ}
    (B : Matrix (Fin k) (Fin k) ℝ) (z : Fin k → ℝ) :
    schurQuad B z = dotProduct z (⅟B *ᵥ z) := by
  unfold schurQuad
  rw [Matrix.mul_assoc, ← Matrix.replicateCol_mulVec]
  exact Matrix.replicateRow_mul_replicateCol_apply z (⅟B *ᵥ z) 0 0

/-- The projected quadratic norm follows from the actual nonsingular matrix
factor. This is also valid for the empty matrix. -/
lemma schurQuad_mul_self_mulVec {k : ℕ}
    (M : Matrix (Fin k) (Fin k) ℝ) (hM : IsUnit M.det)
    (v : EuclideanSpace ℝ (Fin k)) :
    schurQuad (M * M.transpose) (M *ᵥ (fun i => v i)) = ‖‖v ^ 2 := by
  have ht : IsUnit M.transpose.det := by simpa using hM
  have hc : M.transpose * (M * M.transpose)⁻¹ * M = 1 := by
    simp only [Matrix.mul_inv_rev, ← Matrix.mul_assoc]
    rw [Matrix.mul_nonsing_inv M.transpose ht, Matrix.one_mul,
      Matrix.nonsing_inv_mul M hM]
  rw [schurQuad_eq_dotProduct]
  calc
    dotProduct (M *ᵥ (fun i => v i))
        ((M * M.transpose)⁻¹ *ᵥ (M *ᵥ (fun i => v i))) =
        dotProduct ((M * M.transpose)⁻¹ *ᵥ (M *ᵥ (fun i => v i)))
          (M *ᵥ (fun i => v i)) := dotProduct_comm _ _
    _ = dotProduct (fun i => v i)
        (M.transpose *ᵥ ((M * M.transpose)⁻¹ *ᵥ (M *ᵥ (fun i => v i)))) :=
      (Matrix.dotProduct_transpose_mulVec M (fun i => v i) _).symm
    _ = dotProduct (fun i => v i)
        ((M.transpose * (M * M.transpose)⁻¹ * M) *ᵥ (fun i => v i)) := by
      simp only [Matrix.mulVec_mulVec, Matrix.mul_assoc]
    _ = dotProduct (fun i => v i) (fun i => v i) := by
      rw [hc, Matrix.one_mulVec]
    _ = ‖‖v ^ 2 := by
      simpa [dotProduct, pow_two] using (EuclideanSpace.real_norm_sq_eq v).symm

/-- The determinant factor uses its positive Gram determinant before taking a
real negative power. No cancellation of an infinite integral occurs. -/
lemma inverse_abs_det_eq_gram_rpow {k : ℕ}
    {M B : Matrix (Fin k) (Fin k) ℝ} (hB : B.PosDef)
    (hMB : M * M.transpose = B) :
    |M.det⁻|¹ = B.det ^ (-(1 / 2 : ℝ)) := by
  have hs : Real.sqrt B.det = |M.det| := by
    rw ←[ hMB, Matrix.det_mul, Matrix.det_transpose, ← pow_two,
      Real.sqrt_sq_eq_abs]
  rw [Real.rpow_neg hB.det_pos.le, ← Real.sqrt_eq_rpow, hs]
```

The projected-entry Jacobian. An invertible square matrix M pushes entry Lebesgue measure forward to $|\det M|^{-1}$ times that measure. The proof invokes the precise library statement `Real.map_matrix_volume_pi_eq_smul_volume_pi`, then `lintegral_map` and `lintegral_smul_measure`. Thus for every measurable extended nonnegative F, $\int F(Mv)\,dv = |\det M|^{-1}\int F(z)\,dz$. No cancellation involving the integral is needed; this remains valid if its value is infinite.

```
/-- Exact entry-volume change by an invertible square matrix, for measurable
nonnegative tests including tests whose integral is infinite. -/
theorem lintegral_mulVec {k : ℕ} (M : Matrix (Fin k) (Fin k) ℝ)
    (hM : IsUnit M.det) (F : (Fin k → ℝ) → ℝ≥∞0) (hF : Measurable F) :
    ∫⁻( v : Fin k → ℝ, F (M ᵥ* v) ∂coordinateLebesgue (Fin k)) =
      ENNReal.ofReal (|M.det⁻|[1]) *
        ∫⁻ z : Fin k → ℝ, F z ∂coordinateLebesgue (Fin k) := by
  simp only [coordinateLebesgue_eq_volume]
  have hm : Measure.map M.toLin' (volume : Measure (Fin k → ℝ)) =
      ENNReal.ofReal (|M.det⁻|[1]) • volume := by
    simpa only [abs_inv] using
      Real.map_matrix_volume_pi_eq_smul_volume_pi (isUnit_iff_ne_zero.mp hM)
  calc
    ∫⁻( v : Fin k → ℝ, F (M ᵥ* v)) =
        ∫⁻ z : Fin k → ℝ, F z ∂Measure.map M.toLin' volume := by
      simpa only [Matrix.toLin'_apply] using
        (lintegral_map hF M.toLin'.continuous_of_finiteDimensional.measurable).symm
    _ = _ := by rw [hm, lintegral_smul_measure]; rfl
```

The fixed-row projection theorem. For every measurable $\Psi : \mathbb{R}^k \times \mathbb{R} \to [0,\infty]$,

$$\int_{\mathbb{R}^d} \Psi\big(Rx, \|x\|_2^2 - q_B(Rx)\big)\,dx = a_{d-k}(\det B)^{-1/2}\int_{\mathbb{R}^k}\int_0^\infty t^{(d-k)/2-1}\Psi(z,t)\,dt\,dz.$$

Use the preceding constructed map $e(v,u)$. Its identities change the test to $\Psi(Mv,\|u\|_2^2)$ and its measure to $dv\,du$. Tonelli first integrates in u; the extended nonnegative squared-radius formula produces a_{d-k} and the power of t. The resulting function $H(z) = \int_0^\infty t^{(d-k)/2-1}\Psi(z,t)\,dt$ is measurable by `Measurable.lintegral_prod_right`. Transport the remaining Euclidean v coordinates to entry coordinates, and apply the preceding determinant change of variables. The squared Gram determinant relation supplies $(\det B)^{-1/2}$. All scalar coefficients are finite positive numbers; the inner integrals may be infinite.

```
/-- The fixed-row projection formula in original projected entries and squared
orthogonal radius. Its only geometric hypothesis is positive definiteness of
the actual row Gram matrix; the orthogonal coordinates are constructed by
`exists_projection_coordinates`. The strict inequality `k < d` ensures a
positive-dimensional radial factor and includes the endpoint `d - k = 1`. -/
theorem lintegral_projection {k d : ℕ}
    (R : Matrix (Fin k) (Fin d) ℝ) (hkd : k < d)
    (hR : (R * R.transpose).PosDef)
    Ψ( : (Fin k → ℝ) × ℝ → ℝ≥∞0) (Ψh : Measurable Ψ) :
    ∫⁻( x : EuclideanSpace ℝ (Fin d),
      Ψ (R ᵥ* (fun j => x j),
        |‖x ^ 2 - schurQuad (R * R.transpose) (R ᵥ* (fun j => x j)))) =
      ENNReal.ofReal (radialMass (d - k) *
        (R * R.transpose).det ^ (-(1 / 2 : ℝ))) *
      ∫⁻ z : Fin k → ℝ, ∫⁻ t in Ioi (0 : ℝ),
        ENNReal.ofReal (t ^ (((d - k : ℕ) : ℝ) / 2 - 1)) * Ψ (z, t)
          ∂volume ∂coordinateLebesgue (Fin k) := by
  classical
  let : NeZero (d - k) := ⟨Nat.ne_of_gt (Nat.sub_pos_of_lt hkd)⟩
  obtain ⟨M, e, hM, hMM, hcoord, hsq, ⟩he :=
```

```
    O77.GramProjectionGeometry.exists_projection_coordinates R hR
  let H : (Fin k → ℝ) → ℝ≥∞0 := fun z =>
    ∫⁻ t in Ioi (0 : ℝ),
      ENNReal.ofReal (t ^ (((d - k : ℕ) : ℝ) / 2 - 1)) * Ψ (z, t)
  have hH : Measurable H := by
    apply Measurable.lintegral_prod_right
    exact ((measurable_rpow_const (((d - k : ℕ) : ℝ) / 2 - 1)).comp
      measurable_snd).ennreal_ofReal.mul Ψh
  have hMc : Measurable
      (fun v : EuclideanSpace ℝ (Fin k) => M ᵥ* (fun i => v i)) :=
    M.mulVecLin.continuous_of_finiteDimensional.measurable.comp
      (PiLp.volume_preserving_ofLp (Fin k)).measurable
  have hprod : Measurable
      (fun w : EuclideanSpace ℝ (Fin k) × EuclideanSpace ℝ (Fin (d - k)) =>
        Ψ (M ᵥ* (fun i => w.1 i), ‖w‖.2 ^ 2)) :=
    Ψh.comp ((hMc.comp measurable_fst).prodMk (measurable_snd.norm.pow_const 2))
  have hrad (v : EuclideanSpace ℝ (Fin k)) :
      ∫⁻( u : EuclideanSpace ℝ (Fin (d - k)),
        Ψ (M ᵥ* (fun i => v i), ‖‖u ^ 2)) =
      ENNReal.ofReal (radialMass (d - k)) * H (M ᵥ* (fun i => v i)) := by
    simpa only [finrank_euclideanSpace, Fintype.card_fin, radialMass, H] using
      lintegral_norm_sq (volume : Measure (EuclideanSpace ℝ (Fin (d - k))))
        (fun t => Ψ (M ᵥ* (fun i => v i), t))
        (Ψh.comp (measurable_const.prodMk measurable_id))
  have hmatrix :
      ∫⁻( v : EuclideanSpace ℝ (Fin k), H (M ᵥ* (fun i => v i))) =
      ENNReal.ofReal ((R * R.transpose).det ^ (-(1 / 2 : ℝ))) *
        ∫⁻ z : Fin k → ℝ, H z ∂coordinateLebesgue (Fin k) := by
    calc
      _ = ∫⁻ v : Fin k → ℝ, H (M ᵥ* v) ∂coordinateLebesgue (Fin k) := by
        simpa only [coordinateLebesgue_eq_volume, Function.comp_apply,
          Matrix.mulVecLin_apply] using
          (PiLp.volume_preserving_ofLp (Fin k)).lintegral_comp
            (hH.comp M.mulVecLin.continuous_of_finiteDimensional.measurable)
      _ = _ := by
        rw [lintegral_mulVec M hM H hH, inverse_abs_det_eq_gram_rpow hR hMM]
  calc
    _ = ∫⁻ w : EuclideanSpace ℝ (Fin k) × EuclideanSpace ℝ (Fin (d - k)),
        Ψ (R ᵥ* (fun j => e w j),
          ‖e ‖w ^ 2 - schurQuad (R * R.transpose) (R ᵥ* (fun j => e w j))) :=
      (he.lintegral_comp_emb e.toHomeomorph.toMeasurableEquiv.measurableEmbedding _).symm
    _ = ∫⁻ w : EuclideanSpace ℝ (Fin k) × EuclideanSpace ℝ (Fin (d - k)),
        Ψ (M ᵥ* (fun i => w.1 i), ‖w‖.2 ^ 2) := by
      apply lintegral_congr
      intro w
      rw [hcoord w.1 w.2, hsq w.1 w.2, ← hMM,
        schurQuad_mul_self_mulVec M hM]
      have hsub : ‖w‖.1 ^ 2 + ‖w‖.2 ^ 2 - ‖w‖.1 ^ 2 = ‖w‖.2 ^ 2 := by ring
      rw [hsub]
    _ = ∫⁻ v : EuclideanSpace ℝ (Fin k),
        ∫⁻ u : EuclideanSpace ℝ (Fin (d - k)),
          Ψ (M ᵥ* (fun i => v i), ‖‖u ^ 2) :=
      lintegral_prod _ hprod.aemeasurable
    _ = ∫⁻ v : EuclideanSpace ℝ (Fin k),
        ENNReal.ofReal (radialMass (d - k)) * H (M ᵥ* (fun i => v i)) :=
      lintegral_congr hrad
    _ = ENNReal.ofReal (radialMass (d - k)) *
        ∫⁻ v : EuclideanSpace ℝ (Fin k), H (M ᵥ* (fun i => v i)) :=
      lintegral_const_mul' _ _ ENNReal.ofReal_ne_top
    _ = _ := by
      rw [hmatrix, ← mul_assoc,
        ← ENNReal.ofReal_mul (radialMass_pos (Nat.sub_pos_of_lt hkd)).le]
```

Adding the Gaussian weight. For a measurable nonnegative border test $F(z,u)$, use

$$\Psi(z,t) = e^{-q_B(z)-t} F(z, t + q_B(z)).$$

At the projected point, $q_B(Rx) + t = \|x\|_2^2$, so the left side becomes $\int e^{-\|x\|_2^2} F(Rx, \|x\|_2^2)\, dx$. On the right, the exponent $(d-k)/2 - 1$ equals $\eta_{k+1,d}$. This proves precisely the weighted border formula needed in the row induction. Measurability follows from continuity of the fixed quadratic form and measurability of F; no integrability assumption is introduced.

```
/-- Gaussian specialization of the projection formula, retaining an arbitrary
nonnegative measurable border test. The test need not be integrable. -/
theorem lintegral_projection_gaussian {k d : ℕ}
    (R : Matrix (Fin k) (Fin d) ℝ) (hkd : k < d)
    (hR : (R * R.transpose).PosDef)
    (F : (Fin k → ℝ) × ℝ → ℝ≥∞0) (hF : Measurable F) :
    ∫⁻ x : EuclideanSpace ℝ (Fin d),
      ENNReal.ofReal (Real.exp ‖(-‖x ^ 2)) *
        F (R ᵥ* (fun j => x j), ‖‖x ^ 2)) =
      ENNReal.ofReal (radialMass (d - k) *
        (R * R.transpose).det ^ (-(1 / 2 : ℝ))) *
      ∫⁻ z : Fin k → ℝ, ∫⁻ t in Ioi (0 : ℝ),
        ENNReal.ofReal (Real.exp (-(schurQuad (R * R.transpose) z + t)) *
          t ^ wishartEta (k + 1) d) *
          F (z, t + schurQuad (R * R.transpose) z)
            ∂volume ∂coordinateLebesgue (Fin k) := by
  let Ψ : (Fin k → ℝ) × ℝ → ℝ≥∞0 := fun w =>
    ENNReal.ofReal (Real.exp (-(schurQuad (R * R.transpose) w.1 + w.2))) *
      F (w.1, w.2 + schurQuad (R * R.transpose) w.1)
  have Ψh : Measurable Ψ := by
    have hq : Measurable (fun w : (Fin k → ℝ) × ℝ =>
        schurQuad (R * R.transpose) w.1) :=
      (schurQuad_continuous (R * R.transpose)).measurable.comp measurable_fst
    exact (Real.measurable_exp.comp (hq.add measurable_snd).neg).ennreal_ofReal.mul
      (hF.comp (measurable_fst.prodMk (measurable_snd.add hq)))
  have h := lintegral_projection R hkd hR Ψ Ψh
  calc
    _ = ∫⁻ x : EuclideanSpace ℝ (Fin d),
        Ψ (R ᵥ* (fun j => x j),
          ‖‖x ^ 2 - schurQuad (R * R.transpose) (R ᵥ* (fun j => x j))) := by
      apply lintegral_congr
      intro x
      dsimp only Ψ[]
      have hleft : schurQuad (R * R.transpose) (R ᵥ* (fun j => x j)) +
          ‖(‖x ^ 2 - schurQuad (R * R.transpose) (R ᵥ* (fun j => x j))) =
          ‖‖x ^ 2 := by ring
      rw [hleft, sub_add_cancel]
    _ = _ := h.trans (by
      congr 1
      apply lintegral_congr
      intro z
      apply lintegral_congr
      intro t
      rw [schur_exponent hkd.le]
      dsimp only Ψ[]
      rw [ENNReal.ofReal_mul (Real.exp_pos _).le]
      ac_rfl)

end O77.GramProjectionIntegral
end
```

57 Splitting the last matrix row in entry measure

Fix $k, d \in \mathbb{N}$. For $R \in \mathbb{R}^{k\times d}$ and $x \in \mathbb{R}^d$, let $[R; x] \in \mathbb{R}^{(k+1)\times d}$ be the matrix formed by appending x as the last row. All entry spaces retain product Lebesgue measure. The row recurrence below uses a measurable test $F : \mathbb{R}^{(k+1)\times(k+1)} \to [0, \infty]$ in $\mathcal{Q}_{k+1,d}(F)$. It connects the literal row map to the previously fixed symmetric-entry enumeration.

```
import Mathlib
import O77GramCoordinates

set_option autoImplicit false
noncomputable section
open Set MeasureTheory Matrix
open scoped ENNReal BigOperators

namespace O77.GramRows
open O77.Residual O77.GramBorder O77.GramCoordinates
```

Exact row assembly. The measurable equivalence `rowAssemble` first regards the last row as a singleton-indexed family, combines the old and new families on a sum index, and reindexes by `lastIndex`. The library's finite-product equivalences preserve the row-product measures at every step. Evaluation on old indices and on the last index verifies that this map is exactly $[R; x]$. Consequently its squared Frobenius norm is $\|R\|_F^2 + \sum_j x_j^2$.

```
/-- Original matrix-entry row assembly. The final row is a whole vector;
this map does not use a subspace volume or a matrix default norm. -/
def rowAssemble (k d : ℕ) :
    (Matrix (Fin k) (Fin d) ℝ × (Fin d → ℝ)) ≃ᵐ
      Matrix (Fin (k+1)) (Fin d) ℝ :=
  ((MeasurableEquiv.refl (Matrix (Fin k) (Fin d) ℝ)).prodCongr
    (MeasurableEquiv.funUnique (Fin 1) (Fin d → ℝ)).symm).trans
      ((MeasurableEquiv.sumPiEquivProdPi
        (fun _ : Fin k ⊕ Fin 1 => Fin d → ℝ)).symm.trans
          (MeasurableEquiv.piCongrLeft (fun _ : Fin (k+1) => Fin d → ℝ)
            (lastIndex k)))

lemma rowAssemble_preserves (k d : ℕ) :
    MeasurePreserving (rowAssemble k d)
      ((matrixLebesgue (Fin k) (Fin d)).prod (coordinateLebesgue (Fin d)))
      (matrixLebesgue (Fin (k+1)) (Fin d)) := by
  have h1 := (MeasurePreserving.id (matrixLebesgue (Fin k) (Fin d))).prod
    (measurePreserving_funUnique (coordinateLebesgue (Fin d)) (Fin 1)).symm
  have h2 := measurePreserving_sumPiEquivProdPi_symm
    (fun _ : Fin k ⊕ Fin 1 => coordinateLebesgue (Fin d))
  have h3 := measurePreserving_piCongrLeft
    (fun _ : Fin (k+1) => coordinateLebesgue (Fin d)) (lastIndex k)
  exact h3.comp (h2.comp h1)

@[simp] lemma rowAssemble_old {k d : ℕ}
    (R : Matrix (Fin k) (Fin d) ℝ) (x : Fin d → ℝ) (i : Fin k) :
    rowAssemble k d (R,x) i.castSucc = R i := by
  change (MeasurableEquiv.piCongrLeft
    (fun _ : Fin (k+1) => Fin d → ℝ) (lastIndex k)
      (Sum.elim R (fun _ : Fin 1 => x))) (lastIndex k (Sum.inl i)) = _
  exact MeasurableEquiv.piCongrLeft_apply_apply _ _ _

@[simp] lemma rowAssemble_last {k d : ℕ}
    (R : Matrix (Fin k) (Fin d) ℝ) (x : Fin d → ℝ) :
    rowAssemble k d (R,x) (Fin.last k) = x := by
  change (MeasurableEquiv.piCongrLeft
```

```
    (fun _ : Fin (k+1) => Fin d → ℝ) (lastIndex k)
      (Sum.elim R (fun _ : Fin 1 => x))) (lastIndex k (Sum.inr 0)) = _
  exact MeasurableEquiv.piCongrLeft_apply_apply _ _ _

lemma frobeniusSq_rowAssemble {k d : ℕ}
    (R : Matrix (Fin k) (Fin d) ℝ) (x : Fin d → ℝ) :
    frobeniusSq (rowAssemble k d (R,x)) = frobeniusSq R + ∑ j, (x j)^2 := by
  simp only [frobeniusSq, Fin.sum_univ_castSucc, rowAssemble_old, rowAssemble_last]
```

The actual Gram border. The Gram matrix RR^{T} is symmetric. Direct matrix multiplication in the four blocks gives

$$[R;x][R;x]^{\mathsf{T}} = \begin{pmatrix} RR^{\mathsf{T}} & Rx \\ (Rx)^{\mathsf{T}} & \|x\|_2^2 \end{pmatrix}.$$

The formal identity is expressed through `fromUpper` and `assembleUpper`, so its row and column numbering agrees exactly with the cone coordinate theorem. The proof checks the old block, the final column, its symmetric row, and the final diagonal entry separately.

```
lemma gram_isSymm {k d : ℕ} (R : Matrix (Fin k) (Fin d) ℝ) :
    (R * R.transpose).IsSymm := by
  simp [Matrix.IsSymm, Matrix.transpose_mul]

/-- Exact Gram border identity in the same `Fin (k+1)` numbering as the
upper-entry cone transport. -/
lemma gram_rowAssemble {k d : ℕ}
    (R : Matrix (Fin k) (Fin d) ℝ) (x : Fin d → ℝ) :
    rowAssemble k d (R,x) * (rowAssemble k d (R,x)).transpose =
      fromUpper (assembleUpper k
        ((upperEntries (R * R.transpose), R ᵥ* x), ∑ j, (x j)^2)) := by
  ext i j
  rcases (lastIndex k).surjective i with ⟨i,⟩rfl
  rcases (lastIndex k).surjective j with ⟨j,⟩rfl
  rcases i with i | i <;> rcases j with j | j
  · change _ = fromUpper (assembleUpper k _) i.castSucc j.castSucc
    rw [fromUpper_assemble_old, fromUpper_upperEntries (gram_isSymm R)]
    simp [Matrix.mul_apply, Matrix.transpose_apply, lastIndex]
  · change _ = fromUpper (assembleUpper k _) i.castSucc (Fin.last k)
    rw [fromUpper_assemble_column]
    simp [Matrix.mul_apply, Matrix.transpose_apply, Matrix.mulVec, dotProduct, lastIndex]
  · change _ = fromUpper (assembleUpper k _) (Fin.last k) j.castSucc
    rw [fromUpper_assemble_row]
    simp [Matrix.mul_apply, Matrix.transpose_apply, Matrix.mulVec, dotProduct,
      lastIndex, mul_comm]
  · change _ = fromUpper (assembleUpper k _) (Fin.last k) (Fin.last k)
    rw [fromUpper_assemble_last]
    simp [Matrix.mul_apply, Matrix.transpose_apply, lastIndex, pow_two]
```

Tonelli for the original Gaussian Gram integral. If F is measurable, the Gram test multiplied by the Gaussian is measurable because the Gram entries and Frobenius square are polynomial functions. Change variables by `rowAssemble_preserves`, apply `lintegral_prod`, and separate the Gaussian using $e^{-\|[R;x]\|_F^2} = e^{-\|R\|_F^2} e^{-\|x\|_2^2}$. Substitution of the border identity gives `gaussianGramLIntegral_rows`. This equality is in extended nonnegative integrals, so no unproved integrability hypothesis is hidden in the row split.

```
lemma measurable_gaussianGramIntegrand {k d : ℕ}
    (F : Matrix (Fin k) (Fin k) ℝ → ℝ≥∞0) (hF : Measurable F) :
    Measurable (fun X : Matrix (Fin k) (Fin d) ℝ =>
      ENNReal.ofReal (Real.exp (-frobeniusSq X)) * F (X * X.transpose)) := by
```

```
  have hX : Measurable (fun X : Matrix (Fin k) (Fin d) ℝ => X) := measurable_id
  exact continuous_frobeniusSq.neg.rexp.measurable.ennreal_ofReal.mul
    (hF.comp (hX.o77MatrixMul hX.o77MatrixTranspose))

/-- Tonelli row split of the original entry integral. No integrability or
finite-value hypothesis is imposed on the nonnegative Gram test. -/
theorem gaussianGramLIntegral_rows (k d : ℕ)
    (F : Matrix (Fin (k+1)) (Fin (k+1)) ℝ → ℝ≥∞0) (hF : Measurable F) :
    gaussianGramLIntegral (k+1) d F =
      ∫⁻ R : Matrix (Fin k) (Fin d) ℝ,
        ENNReal.ofReal (Real.exp (-frobeniusSq R)) *
          ∫⁻ x : Fin d → ℝ, ENNReal.ofReal (Real.exp ∑(-( j, (x j)^2))) *
            F (fromUpper (assembleUpper k
              ((upperEntries (R * R.transpose), R ᵥ* x), ∑ j, (x j)^2)))
            ∂coordinateLebesgue (Fin d)
        ∂matrixLebesgue (Fin k) (Fin d) := by
  let g := fun X : Matrix (Fin (k+1)) (Fin d) ℝ =>
    ENNReal.ofReal (Real.exp (-frobeniusSq X)) * F (X * X.transpose)
  have hg : Measurable g := measurable_gaussianGramIntegrand F hF
  change ∫⁻( X, g X ∂matrixLebesgue (Fin (k+1)) (Fin d)) = _
  have hp : Measurable (fun p : Matrix (Fin k) (Fin d) ℝ × (Fin d → ℝ) =>
      g (rowAssemble k d p)) := hg.comp (rowAssemble k d).measurable
  calc
    _ = ∫⁻ p : Matrix (Fin k) (Fin d) ℝ × (Fin d → ℝ),
        g (rowAssemble k d p)
          ∂(matrixLebesgue (Fin k) (Fin d)).prod (coordinateLebesgue (Fin d)) :=
      ((rowAssemble_preserves k d).lintegral_comp hg).symm
    _ = ∫⁻ R : Matrix (Fin k) (Fin d) ℝ,
        ∫⁻( x : Fin d → ℝ, g (rowAssemble k d (R,x)) ∂coordinateLebesgue (Fin d))
          ∂matrixLebesgue (Fin k) (Fin d) :=
      lintegral_prod (fun p => g (rowAssemble k d p)) hp.aemeasurable
    _ = _ := by
      apply lintegral_congr
      intro R
      rw ←[ lintegral_const_mul' _ _ ENNReal.ofReal_ne_top]
      apply lintegral_congr
      intro x
      dsimp [g]
      rw [frobeniusSq_rowAssemble, gram_rowAssemble, neg_add, Real.exp_add,
        ENNReal.ofReal_mul (Real.exp_pos _).le, mul_assoc]

end O77.GramRows
end
```

58 A measurable border kernel and its cone identity

Fix $k, d \in \mathbb{N}$ and a measurable test $F : \mathbb{R}^{(k+1)\times(k+1)} \to [0,\infty]$. The row projection formula must yield a measurable function of the old Gram matrix $B \in \mathbb{R}^{k\times k}$. For $z \in \mathbb{R}^k$ and $u \in \mathbb{R}$, use the established notation $q_B(z) = z^{\mathsf{T}} B^{-1} z$ and $G(B,z,u) = \left(\begin{smallmatrix} B & z \\ z^{\mathsf{T}} & u \end{smallmatrix}\right)$. Put

$$K_{k,d}F(B) = (\det B)^{-1/2} \int_{\mathbb{R}^k} \int_0^\infty e^{-q_B(z)-t} t^{\eta_{k+1,d}} F(G(B,z,t+q_B(z)))\, dt\, dz.$$

The formal definitions extend this expression measurably to all square matrices by extracting and reconstructing upper entries. On $B > 0$ the expression agrees with the displayed one. The factor a_{d-k} is kept outside this kernel.

```
import Mathlib
import O77GramCoordinates
```

```
set_option autoImplicit false
noncomputable section
open Set MeasureTheory Matrix
open scoped ENNReal BigOperators

namespace O77.GramKernel
open O77.Residual O77.GramBorder O77.GramCoordinates
```

The kernel is an explicit integral, not a choice of coordinates. The definitions `borderTest` and `borderKernel` use the fixed entry assembly and the globally measurable totalized inverse. To prove their measurability, compose upper-entry extraction, assembly, and the test F, and then multiply by the exponential and real-power densities. Applying `Measurable.lintegral_prod_right'` first to t and then to z proves measurability of $B \mapsto K_{k,d}F(B)$. This is the reason the later induction does not need a measurable choice of orthonormal bases depending on B.

```
/-- The actual bordered matrix, with the inherited upper-entry enumeration.
The extension away from symmetric positive definite B is used only to make
the induction test a globally measurable function. -/
def borderTest {k : ℕ}
    (F : Matrix (Fin (k+1)) (Fin (k+1)) ℝ → ℝ≥∞0)
    (B : Matrix (Fin k) (Fin k) ℝ) (z : Fin k → ℝ) (t : ℝ) : ℝ≥∞0 :=
  F (fromUpper (assembleUpper k ((upperEntries B,z),t + schurQuad B z)))

/-- This kernel includes the determinant factor from the fixed-row
projection; its radial normalization is kept outside the kernel. -/
def borderKernel (k d : ℕ)
    (F : Matrix (Fin (k+1)) (Fin (k+1)) ℝ → ℝ≥∞0)
    (B : Matrix (Fin k) (Fin k) ℝ) : ℝ≥∞0 :=
  ENNReal.ofReal (B.det ^ (-(1/2 : ℝ))) *
    ∫⁻ z : Fin k → ℝ, ∫⁻( t in Ioi (0 : ℝ),
      ENNReal.ofReal (Real.exp (-(schurQuad B z + t)) *
        t ^ wishartEta (k+1) d) * borderTest F B z t) ∂coordinateLebesgue (Fin k)

lemma measurable_schurQuad (k : ℕ) :
    Measurable (fun v : Matrix (Fin k) (Fin k) ℝ × (Fin k → ℝ) =>
      schurQuad v.1 v.2) := by
  have hb : Measurable (fun v : Matrix (Fin k) (Fin k) ℝ × (Fin k → ℝ) =>
      v⁻.1¹) := measurable_inverse.comp measurable_fst
  unfold schurQuad
  simp only [Matrix.mul_apply, Matrix.replicateRow_apply, Matrix.replicateCol_apply]
  fun_prop

lemma measurable_borderTest {k : ℕ}
    {F : Matrix (Fin (k+1)) (Fin (k+1)) ℝ → ℝ≥∞0} (hF : Measurable F) :
    Measurable (fun w : (Matrix (Fin k) (Fin k) ℝ × (Fin k → ℝ)) × ℝ =>
      borderTest F w.1.1 w.1.2 w.2) := by
  have hu : Measurable (upperEntries : Matrix (Fin k) (Fin k) ℝ →
      UpperCoordinates k) := by
    unfold upperEntries
    fun_prop
  have ha : Measurable (fun w : (Matrix (Fin k) (Fin k) ℝ × (Fin k → ℝ)) × ℝ =>
      ((upperEntries w.1.1,w.1.2),w.2 + schurQuad w.1.1 w.1.2)) :=
    ((hu.comp (measurable_fst.comp measurable_fst)).prodMk
      (measurable_snd.comp measurable_fst)).prodMk
        (measurable_snd.add ((measurable_schurQuad k).comp measurable_fst))
  exact hF.comp ((fromUpper_continuous (k+1)).measurable.comp
    ((assembleUpper k).measurable.comp ha))

lemma measurable_borderIntegrand (k d : ℕ)
```

```
    {F : Matrix (Fin (k+1)) (Fin (k+1)) ℝ → ℝ≥∞0} (hF : Measurable F) :
    Measurable (fun w : (Matrix (Fin k) (Fin k) ℝ × (Fin k → ℝ)) × ℝ =>
      ENNReal.ofReal (Real.exp (-(schurQuad w.1.1 w.1.2 + w.2)) *
        w.2 ^ wishartEta (k+1) d) * borderTest F w.1.1 w.1.2 w.2) := by
  have he := Real.measurable_exp.comp
    (((measurable_schurQuad k).comp measurable_fst).add measurable_snd).neg
  have hp : Measurable
      (fun w : (Matrix (Fin k) (Fin k) ℝ × (Fin k → ℝ)) × ℝ =>
        w.2 ^ wishartEta (k+1) d) :=
    (measurable_rpow_const (wishartEta (k+1) d)).comp measurable_snd
  exact (he.mul hp).ennreal_ofReal.mul (measurable_borderTest hF)

theorem measurable_borderKernel (k d : ℕ)
    {F : Matrix (Fin (k+1)) (Fin (k+1)) ℝ → ℝ≥∞0} (hF : Measurable F) :
    Measurable (borderKernel k d F) := by
  have hinner := (measurable_borderIntegrand k d hF).lintegral_prod_right'
    ν( := (volume : Measure ℝ).restrict (Ioi 0))
  have houter := hinner.lintegral_prod_right'
    ν( := coordinateLebesgue (Fin k))
  have hdet : Measurable (fun B : Matrix (Fin k) (Fin k) ℝ =>
      ENNReal.ofReal (B.det ^ (-(1/2 : ℝ)))) :=
    ((measurable_rpow_const (-(1/2 : ℝ))).comp
      (continuous_id.matrix_det.measurable)).ennreal_ofReal
  exact hdet.mul houter
```

Compatibility and exponent arithmetic. On $B = \mathrm{fromUpper}(a)$, extraction of upper entries returns a, so the border test is exactly the test of `schurAssemble`. The real exponent identity is $\eta_{k,d} - 1/2 = \eta_{k+1,d}$. Hence for $B > 0$ and $t > 0$,

$$\left[e^{-\operatorname{tr} B}(\det B)^{\eta_{k,d}}\right](\det B)^{-1/2}\left[e^{-q_B(z)-t}t^{\eta_{k+1,d}}\right] = e^{-\operatorname{tr} B - t - q_B(z)}(\det B\, t)^{\eta_{k+1,d}}.$$

The proof combines real powers only at positive bases and combines Gaussian factors using `Real.exp_add`; it does not cancel extended integrals.

```
@[simp] lemma borderTest_fromUpper {k : ℕ}
    (F : Matrix (Fin (k+1)) (Fin (k+1)) ℝ → ℝ≥∞0)
    (a : UpperCoordinates k) (z : Fin k → ℝ) (t : ℝ) :
    borderTest F (fromUpper a) z t =
      F (fromUpper (schurAssemble k ((a,z),t))) := by
  change F (fromUpper (assembleUpper k
      ((upperEntries (fromUpper a),z),t + schurQuad (fromUpper a) z))) =
    F (fromUpper (assembleUpper k ((a,z),t + schurQuad (fromUpper a) z)))
  rw [upperEntries_fromUpper]

lemma wishartEta_step (k d : ℕ) :
    wishartEta k d + (-(1/2 : ℝ)) = wishartEta (k+1) d := by
  unfold wishartEta
  push_cast
  ring

/-- The exponent cancellation is performed on positive real determinants,
before converting to ENNReal; no cancellation of an integral occurs. -/
lemma border_density_factor {k : ℕ} (d : ℕ)
    {B : Matrix (Fin k) (Fin k) ℝ} (hB : B.PosDef)
    (z : Fin k → ℝ) {t : ℝ} (ht : 0 < t) :
    (Real.exp (-B.trace) * B.det ^ wishartEta k d) *
        B.det ^ (-(1/2 : ℝ)) *
          (Real.exp (-(schurQuad B z + t)) * t ^ wishartEta (k+1) d) =
      Real.exp (-(B.trace + (t + schurQuad B z))) *
        (B.det * t) ^ wishartEta (k+1) d := by
  have hp : B.det ^ wishartEta k d * B.det ^ (-(1/2 : ℝ)) =
```

```
      B.det ^ wishartEta (k+1) d := by
    rw ←[ Real.rpow_add hB.det_pos, wishartEta_step]
  have he : Real.exp (-(B.trace + (t + schurQuad B z))) =
      Real.exp (-B.trace) * Real.exp (-(schurQuad B z + t)) := by
    rw ←[ Real.exp_add]
    congr 1
    ring
  rw [he, Real.mul_rpow hB.det_pos.le ht.le]
  calc
    _ = (Real.exp (-B.trace) * Real.exp (-(schurQuad B z + t))) *
        (B.det ^ wishartEta k d * B.det ^ (-(1/2 : ℝ))) *
          t ^ wishartEta (k+1) d := by ring
    _ = _ := by rw [hp]; ring
```

Iterating the Schur cone integral. The Schur density is measurable because trace, determinant, assembly, and the joint Schur function are measurable. Its domain is the product $(\{B > 0\} \times \mathbb{R}^k) \times (0, \infty)$. Applying `setLIntegral_prod` twice to `coneLIntegral_schur` writes the cone integral as the ordered integrals in a, z, t displayed below. The Borel cone result and the σ-finiteness of coordinate Lebesgue measures supply the exact hypotheses.

```
lemma measurable_schurDensity (k d : ℕ)
    {F : Matrix (Fin (k+1)) (Fin (k+1)) ℝ → ℝ≥∞0} (hF : Measurable F) :
    Measurable (fun w : BorderCoordinates k =>
      ENNReal.ofReal
        (Real.exp (-((fromUpper w.1.1).trace + (w.2 + jointSchur k w.1))) *
          ((fromUpper w.1.1).det * w.2) ^ wishartEta (k+1) d) *
        F (fromUpper (schurAssemble k w))) := by
  have hc : Continuous (fun w : BorderCoordinates k => fromUpper w.1.1) :=
    (fromUpper_continuous k).comp (continuous_fst.comp continuous_fst)
  have htr : Measurable (fun w : BorderCoordinates k => (fromUpper w.1.1).trace) :=
    hc.matrix_trace.measurable
  have he := Real.measurable_exp.comp
    (htr.add (measurable_snd.add ((jointSchur_measurable k).comp measurable_fst))).neg
  have hp := (measurable_rpow_const (wishartEta (k+1) d)).comp
    (hc.matrix_det.measurable.mul measurable_snd)
  exact (he.mul hp).ennreal_ofReal.mul
    (hF.comp ((fromUpper_continuous (k+1)).measurable.comp (schurAssemble k).measurable))

/-- Tonelli disintegrates the specified upper-entry cone measure into the
old cone, the last off-diagonal column, and the positive Schur complement. -/
theorem coneLIntegral_schur_iterated (k d : ℕ)
    (F : Matrix (Fin (k+1)) (Fin (k+1)) ℝ → ℝ≥∞0) (hF : Measurable F) :
    coneLIntegral (k+1) d F =
      ∫⁻ a in {a : UpperCoordinates k | (fromUpper a).PosDef},
        ∫⁻( z : Fin k → ℝ, ∫⁻( t in Ioi (0 : ℝ),
          ENNReal.ofReal
            (Real.exp (-((fromUpper a).trace + (t + schurQuad (fromUpper a) z))) *
              ((fromUpper a).det * t) ^ wishartEta (k+1) d) *
            F (fromUpper (schurAssemble k ((a,z),t)))) ∂coordinateLebesgue (Fin k))
          ∂upperLebesgue k := by
  let : SigmaFinite (upperLebesgue k) := by unfold upperLebesgue; infer_instance
  let W : BorderCoordinates k → ℝ≥∞0 := fun w =>
    ENNReal.ofReal
      (Real.exp (-((fromUpper w.1.1).trace + (w.2 + jointSchur k w.1))) *
        ((fromUpper w.1.1).det * w.2) ^ wishartEta (k+1) d) *
      F (fromUpper (schurAssemble k w))
  have hW : Measurable W := measurable_schurDensity k d hF
  have hs : {w : BorderCoordinates k | (fromUpper w.1.1).PosDef ∧ 0 < w.2} =
      ({a : UpperCoordinates k | (fromUpper a).PosDef} ×ˢ univ) ×ˢ Ioi (0 : ℝ) := by
    ext w
    simp only [mem_ofPred_eq, mem_prod, mem_univ, and_true, mem_Ioi]
```

```
  rw [coneLIntegral_schur, hs]
  change ∫⁻( w in
      ({a : UpperCoordinates k | (fromUpper a).PosDef} ×ˢ univ) ×ˢ Ioi (0 : ℝ),
      W w ∂((upperLebesgue k).prod (coordinateLebesgue (Fin k))).prod volume) = _
  rw [setLIntegral_prod _ hW.aemeasurable]
  rw [setLIntegral_prod _
    (hW.lintegral_prod_right' ν( := (volume : Measure ℝ).restrict (Ioi 0))).aemeasurable]
  simp only [Measure.restrict_univ, W, jointSchur]
```

The cone kernel recurrence. We obtain

$$\mathcal{C}_{k,d}(K_{k,d}F) = \mathcal{C}_{k+1,d}(F).$$

Expand the left side and move its finite pointwise old-cone density through the z and t integrals by `lintegral_const_mul'`. On each positive-definite B and each $t > 0$, the preceding density identity turns this integrand into that of the iterated Schur cone formula. The border-test compatibility identifies the matrix supplied to F. This proves the equality for every measurable extended nonnegative F, even with infinite integral, and for all natural k, d; the condition $k < d$ will be imposed only in the separate row-projection step.

```
/-- Exact Gaussian-Gram induction kernel identity. It holds for all k,d:
the dimension restriction is needed by the separate projection theorem.
All tests may take the value infinity; this proof uses only Tonelli and
finite pointwise density factors. -/
theorem coneLIntegral_borderKernel (k d : ℕ)
    (F : Matrix (Fin (k+1)) (Fin (k+1)) ℝ → ℝ≥∞0) (hF : Measurable F) :
    coneLIntegral k d (borderKernel k d F) = coneLIntegral (k+1) d F := by
  rw [coneLIntegral_schur_iterated k d F hF]
  unfold coneLIntegral
  apply setLIntegral_congr_fun (measurableSet_posDef_upper k)
  intro a ha
  let B := fromUpper a
  let c : ℝ≥∞0 := ENNReal.ofReal
    ((Real.exp (-B.trace) * B.det ^ wishartEta k d) * B.det ^ (-(1/2 : ℝ)))
  have h0 : 0 ≤ Real.exp (-B.trace) * B.det ^ wishartEta k d :=
    mul_nonneg (Real.exp_pos _).le (Real.rpow_nonneg ha.det_pos.le _)
  have h1 : 0 ≤ (Real.exp (-B.trace) * B.det ^ wishartEta k d) *
      B.det ^ (-(1/2 : ℝ)) :=
    mul_nonneg h0 (Real.rpow_nonneg ha.det_pos.le _)
  change ENNReal.ofReal (Real.exp (-B.trace) * B.det ^ wishartEta k d) *
    (ENNReal.ofReal (B.det ^ (-(1/2 : ℝ))) *
      ∫⁻ z : Fin k → ℝ, ∫⁻( t in Ioi (0 : ℝ),
        ENNReal.ofReal (Real.exp (-(schurQuad B z + t)) *
          t ^ wishartEta (k+1) d) * borderTest F B z t) ∂coordinateLebesgue (Fin k)) = _
  rw ←[ mul_assoc, ← ENNReal.ofReal_mul h0]
  change c * ∫⁻( z : Fin k → ℝ, ∫⁻( t in Ioi (0 : ℝ),
    ENNReal.ofReal (Real.exp (-(schurQuad B z + t)) *
      t ^ wishartEta (k+1) d) * borderTest F B z t) ∂coordinateLebesgue (Fin k)) = _
  rw ←[ lintegral_const_mul' c _ ENNReal.ofReal_ne_top]
  apply lintegral_congr
  intro z
  rw ←[ lintegral_const_mul' c _ ENNReal.ofReal_ne_top]
  apply setLIntegral_congr_fun measurableSet_Ioi
  intro t ht
  dsimp only
  rw ←[ mul_assoc]
  change (ENNReal.ofReal
      ((Real.exp (-B.trace) * B.det ^ wishartEta k d) * B.det ^ (-(1/2 : ℝ))) *
    ENNReal.ofReal (Real.exp (-(schurQuad B z + t)) * t ^ wishartEta (k+1) d)) *
      borderTest F B z t = _
```

```
  rw ←[ ENNReal.ofReal_mul h1, border_density_factor d ha z ht]
  simp only [B, borderTest_fromUpper]

end O77.GramKernel
end
```

59 The Gaussian Gram pushforward theorem

Fix natural numbers k, d with $k \le d$. Let $F : \mathbb{R}^{k\times k} \to [0,\infty]$ be Borel measurable. Only its values on symmetric matrices occur in the identity below, matching the full-matrix test type of the Lean theorem. Matrix-entry measure on $X \in \mathbb{R}^{k\times d}$ is denoted by dX, and dS is product Lebesgue measure in the independent entries S_{ij}, $i \le j$, of a symmetric matrix S. Put $\eta_{k,d} = (d-k-1)/2$, with real subtraction. The row decomposition and Schur-coordinate recurrence prove

$$\int_{\mathbb{R}^{k\times d}} e^{-\|X\|_F^2} F(XX^{\mathsf{T}})\,dX = \left(\prod_{i=0}^{k-1} \frac{(d-i)\omega_{d-i}}{2}\right) \int_{S>0} e^{-\operatorname{tr} S} \det(S)^{\eta_{k,d}} F(S)\,dS.$$

Here $S > 0$ means positive definite, and ω_j denotes the volume of the Euclidean unit ball in $\mathbb{R}^j$. Every $d-i$ in the product is positive. Its radial factor $(d-i)\omega_{d-i}/2 = \pi^{(d-i)/2}/\Gamma((d-i)/2)$ is the one obtained by substituting squared radius in polar integration. For $k = 0$ the product is one and both integrals equal the value of F at the empty matrix. Every integral in this section is nonnegative and may initially be infinite.

```
import Mathlib
import O77GramRows
import O77GramKernel
import O77GramRankNull
import O77GramProjectionIntegral

set_option autoImplicit false
noncomputable section
open Set MeasureTheory Matrix
open scoped ENNReal BigOperators

namespace O77.GramPushforward
open O77.Residual O77.GramBorder O77.GramCoordinates O77.GramKernel
open O77.GramRows O77.RadialGram
```

Eliminating the fixed-row auxiliary bases. Fix R with $RR^{\mathsf{T}} > 0$ and $k < d$. Apply the weighted projection theorem to the function $(z,u) \mapsto F(G(RR^{\mathsf{T}}, z, u))$, where $G(B,z,u) = \left(\begin{smallmatrix} B & z \\ z^{\mathsf{T}} & u \end{smallmatrix}\right)$ is the bordered matrix in the fixed upper-entry assembly. Transport Euclidean x coordinates to original entries by `PiLp.volume_preserving_ofLp`; the squared norm is the sum of squared entries. The resulting identity is

$$\int_{\mathbb{R}^d} e^{-\|x\|_2^2} F([R;x][R;x]^{\mathsf{T}})\,dx = a_{d-k} K_{k,d} F(RR^{\mathsf{T}}).$$

The right side is the already constructed measurable kernel. Thus although the proof of the fixed-row theorem chose orthonormal bases, no such choice appears in the expression subsequently integrated over R.

```
/-- The fixed-row formula is rewritten into the explicit measurable Gram
kernel before integration in R. No measurable choice of orthonormal bases
depending on R is required or asserted. -/
```

```
theorem gaussian_row_eq_borderKernel {k d : ℕ}
    (R : Matrix (Fin k) (Fin d) ℝ) (hkd : k < d)
    (hR : (R * R.transpose).PosDef)
    (F : Matrix (Fin (k+1)) (Fin (k+1)) ℝ → ℝ≥∞0) (hF : Measurable F) :
    ∫⁻( x : Fin d → ℝ, ENNReal.ofReal (Real.exp ∑(-( j, (x j)^2))) *
      F (fromUpper (assembleUpper k
        ((upperEntries (R * R.transpose), R ᵥ* x), ∑ j, (x j)^2)))
        ∂coordinateLebesgue (Fin d)) =
      ENNReal.ofReal (radialMass (d-k)) * borderKernel k d F (R * R.transpose) := by
  let G : (Fin k → ℝ) × ℝ → ℝ≥∞0 := fun w =>
    F (fromUpper (assembleUpper k ((upperEntries (R * R.transpose),w.1),w.2)))
  have hG : Measurable G :=
    hF.comp ((fromUpper_continuous (k+1)).measurable.comp
      ((assembleUpper k).measurable.comp
        ((measurable_const.prodMk measurable_fst).prodMk measurable_snd)))
  have he : MeasurePreserving
      (fun x : EuclideanSpace ℝ (Fin d) => fun j => x j)
      volume (coordinateLebesgue (Fin d)) := by
    simpa only [coordinateLebesgue_eq_volume] using PiLp.volume_preserving_ofLp (Fin d)
  have hg : Measurable (fun x : Fin d → ℝ =>
      ENNReal.ofReal (Real.exp ∑(-( j, (x j)^2))) * G (R ᵥ* x, ∑ j, (x j)^2)) := by
    apply Measurable.mul
    · fun_prop
    · apply hG.comp
      unfold Matrix.mulVec dotProduct
      fun_prop
  calc
    _ = ∫⁻ x : EuclideanSpace ℝ (Fin d),
        ENNReal.ofReal (Real.exp ∑(-( j, (x j)^2))) *
          G (R ᵥ* (fun j => x j), ∑ j, (x j)^2) := (he.lintegral_comp hg).symm
    _ = ∫⁻ x : EuclideanSpace ℝ (Fin d),
        ENNReal.ofReal (Real.exp ‖(-‖x^2)) * G (R ᵥ* (fun j => x j), ‖‖x^2) := by
      apply lintegral_congr
      intro x
      rw ←[ EuclideanSpace.real_norm_sq_eq x]
    _ = _ := by
      simpa only [ENNReal.ofReal_mul (radialMass_pos (Nat.sub_pos_of_lt hkd)).le,
        mul_assoc, G, borderKernel, borderTest] using
        O77.GramProjectionIntegral.lintegral_projection_gaussian R hkd hR G hG
```

The matrix recurrence. Tonelli's row split expresses $\mathcal{Q}_{k+1,d}(F)$ as a Gaussian integral over R of the preceding new-row integral. The rows R with non-positive-definite Gram matrix form a null set by `O77.GramRankNull.ae_posDef_gram`; apply the fixed-row identity everywhere else. Since a_{d-k} is finite, move it outside the integral. This proves $\mathcal{Q}_{k+1,d}(F) = a_{d-k}\mathcal{Q}_{k,d}(K_{k,d}F)$. The only discarded set is the one already proved null in entry measure.

```
/-- The induction step is an equality of actual matrix integrals. The only
exceptional rows discarded are the null set already proved in entry measure. -/
theorem gaussianGramLIntegral_succ {k d : ℕ} (hkd : k < d)
    (F : Matrix (Fin (k+1)) (Fin (k+1)) ℝ → ℝ≥∞0) (hF : Measurable F) :
    gaussianGramLIntegral (k+1) d F =
      ENNReal.ofReal (radialMass (d-k)) *
        gaussianGramLIntegral k d (borderKernel k d F) := by
  rw [gaussianGramLIntegral_rows k d F hF]
  calc
    _ = ∫⁻ R : Matrix (Fin k) (Fin d) ℝ,
        ENNReal.ofReal (Real.exp (-frobeniusSq R)) *
          (ENNReal.ofReal (radialMass (d-k)) * borderKernel k d F (R * R.transpose))
          ∂matrixLebesgue (Fin k) (Fin d) := by
      apply lintegral_congr_ae
      filter_upwards [O77.GramRankNull.ae_posDef_gram hkd.le] with R hR
```

```
      rw [gaussian_row_eq_borderKernel R hkd hR F hF]
    _ = ∫⁻ R : Matrix (Fin k) (Fin d) ℝ,
        ENNReal.ofReal (radialMass (d-k)) *
          (ENNReal.ofReal (Real.exp (-frobeniusSq R)) * borderKernel k d F (R * R.transpose))
          ∂matrixLebesgue (Fin k) (Fin d) := by
      apply lintegral_congr
      intro R
      exact mul_left_comm _ _ _
    _ = _ := lintegral_const_mul' _ _ ENNReal.ofReal_ne_top
```

The empty starting case. In dimension $k = 0$, the matrix-entry and symmetric-entry spaces each consist of one point of mass one. The unique square matrix is positive definite in the vacuous zero-dimensional sense and has determinant one. Therefore both $\mathcal{Q}_{0,d}(F)$ and $\mathcal{C}_{0,d}(F)$ equal $F(0)$, including when $F(0) = \infty$.

```
/-- Both empty-coordinate integrals have their actual singleton mass one. -/
lemma gaussianGram_cone_zero (d : ℕ)
    (F : Matrix (Fin 0) (Fin 0) ℝ → ℝ≥∞0) :
    gaussianGramLIntegral 0 d F = F 0 ∧ coneLIntegral 0 d F = F 0 := by
  have hzero (B : Matrix (Fin 0) (Fin 0) ℝ) : B = 0 := Subsingleton.elim _ _
  have hpd : (0 : Matrix (Fin 0) (Fin 0) ℝ).PosDef := by
    rw ←[ hzero (1 : Matrix (Fin 0) (Fin 0) ℝ)]
    exact Matrix.PosDef.one
  constructor
  · unfold gaussianGramLIntegral
    calc
      _ = ∫⁻ _ : Matrix (Fin 0) (Fin d) ℝ, F 0
          ∂matrixLebesgue (Fin 0) (Fin d) := by
        apply lintegral_congr
        intro X
        simp [hzero, frobeniusSq]
      _ = F 0 := by
        have hmass : matrixLebesgue (Fin 0) (Fin d) Set.univ = 1 :=
          Measure.pi_empty_univ _
        rw [lintegral_const, hmass, mul_one]
  · simp [coneLIntegral, hzero, hpd, Matrix.trace, upperLebesgue, coordinateLebesgue]
```

Proof of the full Gram pushforward. Induct on k, retaining the arbitrary measurable test in the induction hypothesis. The empty case is the preceding calculation. If $k+1 \le d$, the matrix recurrence and measurability of $K_{k,d}F$ allow the induction hypothesis to be applied to that kernel:

$$\mathcal{Q}_{k+1,d}(F) = a_{d-k}A_{k,d}\mathcal{C}_{k,d}(K_{k,d}F) = a_{d-k}A_{k,d}\mathcal{C}_{k+1,d}(F) = A_{k+1,d}\mathcal{C}_{k+1,d}(F).$$

The middle equality is the cone kernel recurrence, and the last is the finite-product recurrence for A. This includes the square case: the final step $k+1 = d$ uses radial dimension $d-k = 1$, never zero, and the final cone exponent is $\eta_{d,d} = -1/2$. Positivity and finiteness of $A_{k,d}$ were proved independently, so all scalar manipulations are valid even if the test integral is infinite. No Gram-density or spectral theorem is a hypothesis of this result.

```
/-- Gaussian Gram pushforward in the specified independent upper entries.
The test is arbitrary measurable and nonnegative, including infinite tests.
No Wishart, spectral integration, chart, or projection proposition is a
parameter. -/
theorem gaussianGramLIntegral_eq_gramMass_mul_cone (k d : ℕ) (hkd : k ≤ d)
    (F : Matrix (Fin k) (Fin k) ℝ → ℝ≥∞0) (hF : Measurable F) :
    gaussianGramLIntegral k d F = ENNReal.ofReal (gramMass k d) * coneLIntegral k d F := by
  induction k generalizing d with
```

```
  | zero =>
      rw [(gaussianGram_cone_zero d F).1, (gaussianGram_cone_zero d F).2, gramMass_zero]
      simp
  | succ k ih =>
      have hk : k ≤ d := by omega
      rw [gaussianGramLIntegral_succ (by omega : k < d) F hF,
        ih d hk (borderKernel k d F) (measurable_borderKernel k d hF),
        coneLIntegral_borderKernel k d F hF, gramMass_succ,
        ENNReal.ofReal_mul (gramMass_pos hk).le]
      ac_rfl
```

The determinant-test consumer. The product $A_{k,d}$ is strictly positive for $k \le d$ and finite because it is real. The test $G \mapsto \mathrm{ofReal}(\det(I+TG)^{-\rho})$ is measurable for any real parameters: the determinant is continuous and real powers are measurable. Substituting it in the Gram pushforward, then applying the earlier exact integrand adapter, gives

$$\mathcal{G}_{k,d}(\rho,T) = A_{k,d}\mathcal{C}_{k,d}(\mathrm{determinantTest}(\rho,T)).$$

This particular equality of totalized nonnegative integrands holds for arbitrary real parameters. The concrete Wishart theorem uses its nonnegative-parameter specialization.

```
lemma gram_normalizer_pos_finite {k d : ℕ} (hkd : k ≤ d) :
    0 < ENNReal.ofReal (gramMass k d) ∧ ENNReal.ofReal (gramMass k d) < ⊤ :=
  ⟨ENNReal.ofReal_pos.2 (gramMass_pos hkd), ENNReal.ofReal_lt_top⟩

lemma measurable_determinantTest (k : ℕ) (rho T : ℝ) :
    Measurable (determinantTest (k := k) rho T) := by
  unfold determinantTest
  apply Measurable.ennreal_ofReal
  apply (measurable_rpow_const (-rho)).comp
  exact (show Continuous (fun G : Matrix (Fin k) (Fin k) ℝ => 1 + T • G) by
    fun_prop).matrix_det.measurable

/-- Exact consumer of the inherited left integral. The intended Wishart
domain is rho,T >= 0; this equality itself also holds for the totalized
nonnegative determinant test at arbitrary real parameters. -/
theorem leftGram_eq_cone {k d : ℕ} (hkd : k ≤ d) (rho T : ℝ) :
    leftGramLaplaceLIntegral k d rho T =
      ENNReal.ofReal (gramMass k d) * coneLIntegral k d (determinantTest rho T) := by
  rw ←[ gaussianGramLIntegral_determinantTest]
  exact gaussianGramLIntegral_eq_gramMass_mul_cone k d hkd _
    (measurable_determinantTest k rho T)

end O77.GramPushforward
end
```

60 From the Gram cone to the exact Wishart interface

The Gram pushforward just proved fixes the matrix-to-cone coefficient $A_{k,d} > 0$. The remaining interface in this section concerns the cone-to-eigenvalue identity, for the existing sorted-extension integral on the full nonnegative orthant. We first calibrate the cone mass independently of spectral integration, then prove the precise equivalence between that spectral interface and the global Wishart proposition. The subsequent spectral modules construct the spectral interface used by the unconditional O77 theorem.

```
import Mathlib
import O77GramPushforward
import O77WishartAnswer
```

```
set_option autoImplicit false
noncomputable section
open Set MeasureTheory Matrix
open scoped ENNReal BigOperators

namespace O77.GramWishart
open O77.Residual O77.GramBorder O77.GramPushforward
```

The cone's zero-parameter mass. Setting $\rho = T = 0$ in the established matrix-to-cone identity gives

$$A_{k,d}\,\mathcal{C}_{k,d}(1) = \pi^{kd/2}.$$

The right side is the independently evaluated entry Gaussian. Since $A_{k,d}$ is positive and finite, the cone mass cannot be zero or infinite; either possibility contradicts this equality using extended nonnegative arithmetic. Only after these bounds are proved does `cone_zero_parameters_real` pass to real values and divide, obtaining $\mathcal{C}_{k,d}(1) = \pi^{kd/2}/A_{k,d} > 0$. The argument includes $k = 0$ and $k = d$.

```
/-- Calibration of the actual upper-entry cone integral, obtained from
the Gaussian Gram theorem without any eigenvalue-integration assumption. -/
theorem cone_normalizer_calibration {k d : ℕ} (hkd : k ≤ d) :
    ENNReal.ofReal (gramMass k d) * coneLIntegral k d (determinantTest 0 0) =
      ENNReal.ofReal (gaussianRowsConstant k d) :=
  (leftGram_eq_cone hkd 0 0).symm.trans
    (O77.WishartReduction.left_zero_parameters k d)

/-- Positive finite cone mass, including k=0 and the square case k=d.
No totalized real integral and no cancellation are used to obtain the bounds. -/
theorem cone_zero_parameters_bounds {k d : ℕ} (hkd : k ≤ d) :
    0 < coneLIntegral k d (determinantTest 0 0) ∧
      coneLIntegral k d (determinantTest 0 0) < ⊤ := by
  let N := coneLIntegral k d (determinantTest 0 0)
  have hA0 := (gram_normalizer_pos_finite hkd).1
  have hprod : ENNReal.ofReal (gramMass k d) * N =
      ENNReal.ofReal (gaussianRowsConstant k d) := cone_normalizer_calibration hkd
  have hN0 : N ≠ 0 := by
    intro hz
    rw [hz, mul_zero] at hprod
    exact (ENNReal.ofReal_pos.2 (gaussianRowsConstant_pos k d)).ne' hprod.symm
  have hNtop : N ≠ ⊤ := by
    intro hz
    rw [hz, ENNReal.mul_top hA0.ne'] at hprod
    exact ENNReal.ofReal_ne_top hprod.symm
  exact ⟨bot_lt_iff_ne_bot.2 hN0, lt_top_iff_ne_top.2 ⟩hNtop

/-- Real cancellation is used only after the cone mass is positive and finite. -/
theorem cone_zero_parameters_real {k d : ℕ} (hkd : k ≤ d) :
    0 < (coneLIntegral k d (determinantTest 0 0)).toReal ∧
      (coneLIntegral k d (determinantTest 0 0)).toReal =
        gaussianRowsConstant k d / gramMass k d := by
  obtain ⟨hN0,⟩hNfin := cone_zero_parameters_bounds hkd
  refine ⟨ENNReal.toReal_pos hN0.ne' hNfin.ne, ?_⟩
  have hreal := congrArg ENNReal.toReal (cone_normalizer_calibration hkd)
  rw [ENNReal.toReal_mul, ENNReal.toReal_ofReal (gramMass_pos hkd).le,
    ENNReal.toReal_ofReal (gaussianRowsConstant_pos k d).le] at hreal
  apply (eq_div_iff (gramMass_pos hkd).ne').2
  simpa only [mul_comm] using hreal
```

The exact spectral interface. The proposition below requires, for every $2 \le k \le d$, a single $D_{k,d} \in (0,\infty)$ such that

$$\mathcal{C}_{k,d}(\mathrm{determinantTest}(\rho,T)) = D_{k,d}\mathcal{J}_k(\eta_{k,d},\rho,T) \quad \text{for all } \rho,T \ge 0.$$

The existential quantifier on $D_{k,d}$ precedes both parameters. The integral on the right is the full orthant with its inherited sorted-extension convention, so a chamber permutation factor cannot be omitted. This declaration is a definition of the intermediate proposition, not a supplied proof of it.

```
/-- The cone-to-eigenvalue interface. The target is the inherited
sorted-extension integral on the FULL nonnegative orthant. Its positive
finite normalizer is chosen before BOTH test parameters. This definition
does not assert an inhabitant or provide a proof of the spectral formula. -/
def HigherRankConeEigenvalueFormula : Prop :=
  ∀ k d : ℕ, 2 ≤ k → k ≤ d →
    ∃ D : ℝ≥∞0, 0 < D ∧ D < ⊤ ∧
      ∀ rho T : ℝ, 0 ≤ rho → 0 ≤ T →
        coneLIntegral k d (determinantTest rho T) =
          D * laguerreOrthantLIntegral k (wishartEta k d) rho T
```

Composing the two density changes. Assuming the displayed spectral interface, choose $C_{k,d} = A_{k,d}D_{k,d}$. Its positivity and finiteness follow from those of both factors. Substitution into the matrix-to-cone formula gives the required left Gram identity for every nonnegative ρ, T. Conversely, suppose the higher-rank left identity has a valid coefficient $C_{k,d}$. Since $A_{k,d} \in (0,\infty)$ is already known, its extended inverse is positive and finite, and $D_{k,d} = A_{k,d}^{-1}C_{k,d}$ gives the cone identity. The cancellation uses `ENNReal.inv_mul_cancel_left` with both required nonzero and noninfinite guards; it does not require the individual test integral to be finite.

```
/-- The Gram density theorem supplies exactly the missing matrix-to-cone
factor. The spectral premise remains explicit and is not inferred from
zero-parameter calibration. -/
theorem higherRank_left_of_cone (hS : HigherRankConeEigenvalueFormula) :
    ∀ k d : ℕ, 2 ≤ k → k ≤ d →
      ∃ C : ℝ≥∞0, 0 < C ∧ C < ⊤ ∧
        ∀ rho T : ℝ, 0 ≤ rho → 0 ≤ T →
          leftGramLaplaceLIntegral k d rho T =
            C * laguerreOrthantLIntegral k (wishartEta k d) rho T := by
  intro k d hk hkd
  obtain ⟨D,hD0,hDfin,⟩hD := hS k d hk hkd
  obtain ⟨hA0,⟩hAfin := gram_normalizer_pos_finite hkd
  refine ⟨ENNReal.ofReal (gramMass k d) * D,
    bot_lt_iff_ne_bot.2 (mul_ne_zero hA0.ne' hD0.ne'),
    ENNReal.mul_lt_top hAfin hDfin, ?⟩_
  intro rho T hr hT
  rw [leftGram_eq_cone hkd, hD rho T hr hT, mul_assoc]

/-- Conversely the Gram multiplier can be removed because it has already
been proved positive and finite. Thus the spectral identity gives an
equivalent condition, with the same parameter quantifiers. -/
theorem cone_of_higherRank_left
    (hHigher : ∀ k d : ℕ, 2 ≤ k → k ≤ d →
      ∃ C : ℝ≥∞0, 0 < C ∧ C < ⊤ ∧
        ∀ rho T : ℝ, 0 ≤ rho → 0 ≤ T →
          leftGramLaplaceLIntegral k d rho T =
            C * laguerreOrthantLIntegral k (wishartEta k d) rho T) :
    HigherRankConeEigenvalueFormula := by
  intro k d hk hkd
  obtain ⟨C,hC0,hCfin,⟩hC := hHigher k d hk hkd
  obtain ⟨hA0,⟩hAfin := gram_normalizer_pos_finite hkd
```

```
  let A := ENNReal.ofReal (gramMass k d)
  have hAi0 : 0 < ⁻A¹ := ENNReal.inv_pos.2 hAfin.ne
  have hAifin : ⁻A¹ < ⊤ := ENNReal.inv_lt_top.2 hA0
  refine ⟨⁻A¹ * C, bot_lt_iff_ne_bot.2 (mul_ne_zero hAi0.ne' hC0.ne'),
    ENNReal.mul_lt_top hAifin hCfin, ?⟩_
  intro rho T hr hT
  calc
    coneLIntegral k d (determinantTest rho T) =
        ⁻A¹ * (A * coneLIntegral k d (determinantTest rho T)) :=
      (ENNReal.inv_mul_cancel_left hA0.ne' hAfin.ne).symm
    _ = ⁻A¹ * leftGramLaplaceLIntegral k d rho T := by
      rw [leftGram_eq_cone hkd]
    _ = (⁻A¹ * C) * laguerreOrthantLIntegral k (wishartEta k d) rho T := by
      rw [hC rho T hr hT, mul_assoc]
```

The equivalence and its O77 consumer. Combining the two implications with `global_iff_higherRank_left` proves that `External.RealWishartEigenvalueFormula` is equivalent to `HigherRankConeEigenvalueFormula`. The small cases $k = 0, 1$ are supplied by the earlier explicit constructions. Finally `answersO77_of_cone_eigenvalues` composes this spectral interface with the established measured O77 answer. Its spectral premise is visible in its type; the later spectral integration proof supplies an inhabitant for the unconditional final theorem. None of these equivalences alone proves a density formula by zero-parameter calibration.

```
/-- This equivalence relates the Wishart and cone-to-eigenvalue formulas.
It is not a proof of either side. The k=0,1 cases are supplied by the
inherited reduction when the reverse direction is used. -/
theorem global_iff_cone_eigenvalues :
    O77.External.RealWishartEigenvalueFormula ↔ HigherRankConeEigenvalueFormula := by
  constructor
  · intro hW
    exact cone_of_higherRank_left
      (O77.WishartReduction.global_iff_higherRank_left.mp hW)
  · intro hS
    exact O77.WishartReduction.global_iff_higherRank_left.mpr
      (higherRank_left_of_cone hS)

end O77.GramWishart

namespace O77

/-- The six-conjunct measured answer from the cone-to-eigenvalue formula. -/
theorem answersO77_of_cone_eigenvalues
    (hS : GramWishart.HigherRankConeEigenvalueFormula)
    {I O H : ℕ} (Phi : Matrix (Fin O) (Fin I) ℝ)
    (hI : 0 < I) (hO : 0 < O) (hrH : Phi.rank < H) :
    AnswersO77 (H := H) Phi :=
  answersO77_of_higherRank_left
    (GramWishart.higherRank_left_of_cone hS) Phi hI hO hrH

end O77
end
```

61 The spectral tangent in independent upper entries

Throughout this module, $k \geq 0$ and $E_k = \mathbb{R}^{\{(i,j):0\leq i\leq j<k\}}$ is the existing `UpperCoordinatesk`. Its measure is the product of ordinary real Lebesgue measures. Write $S(a) = \mathrm{fromUpper}(a)$ for symmetric assembly. The diagonal of a will encode eigenvalue displacement, while its strict

upper part encodes angular displacement. These two roles share one carrier but must not be confused. In particular the skew basis vectors are $E_{ij} - E_{ji}$, with no division by $\sqrt{2}$.
Implementation namespace and dependencies. Module `O77SpectralDifferential`; imports `Mathlib`, `O77GramBorder`.

```
import Mathlib
import O77GramBorder

set_option autoImplicit false
noncomputable section
open Matrix
open scoped BigOperators

namespace O77.SpectralDifferential
open O77.GramBorder
```

61.1 Diagonal and angular coordinates

Define $v(a)_i = a_{ii}$ and the skew matrix $K(a)$ by $K(a)_{ij} = a_{ij}$ for $i < j$, $K(a)_{ji} = -a_{ij}$, and $K(a)_{ii} = 0$. The next three elementary lemmas verify these entries and $K(a)^{\mathsf{T}} = -K(a)$ by the three cases $i < j$, $i = j$, $j < i$. The functions are actual coordinate extraction and assembly maps; neither orthogonality nor a proposed differential is built into them.

```
/-- The diagonal coordinates are eigenvalue velocities. -/
def diagonalVelocity {k : ℕ} (a : UpperCoordinates k) : Fin k → ℝ :=
  fun i => a ⟨(i, i), le_rfl⟩

/-- Strict upper coordinates are unnormalized skew-matrix entries. -/
def skew {k : ℕ} (a : UpperCoordinates k) : Matrix (Fin k) (Fin k) ℝ :=
  fun i j => if hij : i < j then a ⟨(i, j), hij.le⟩
    else if hji : j < i then -a ⟨(j, i), hji.le⟩ else 0

@[simp] theorem skew_diag {k : ℕ} (a : UpperCoordinates k) (i : Fin k) :
    skew a i i = 0 := by simp [skew]

theorem skew_upper {k : ℕ} (a : UpperCoordinates k) {i j : Fin k}
    (hij : i < j) : skew a i j = a ⟨(i, j), hij.le⟩ := by
  simp [skew, hij]

theorem skew_transpose {k : ℕ} (a : UpperCoordinates k) :
    (skew a).transpose = -skew a := by
  ext i j
  rcases lt_trichotomy i j with hij | hij | hji
  · simp [skew, Matrix.transpose_apply, hij, not_lt_of_ge hij.le]
  · subst j; simp
  · simp [skew, Matrix.transpose_apply, hji, not_lt_of_ge hji.le]
```

61.2 The commutator is a diagonal coordinate map

For a real tuple s, let $\mathcal{T}_s : E_k \to E_k$ multiply diagonal coordinates by 1 and strict upper coordinates by $s_j - s_i$. The Lean linear map is `Matrix.toLin'` of this diagonal matrix. Direct multiplication of a matrix by a diagonal matrix gives

$$\operatorname{upperEntries}\big(\operatorname{diag} v(a) + K(a)\operatorname{diag} s - \operatorname{diag} sK(a)\big) = \mathcal{T}_s a.$$

For $i < j$, the relevant entry is $K_{ij}s_j - s_iK_{ij}$; on the diagonal only v_i remains. The calculation supplies the linear map whose determinant will be computed; a later argument identifies it with the derivative of an actual map.

```
def weight {k : ℕ} (s : Fin k → ℝ) (ij : UpperIndex k) : ℝ :=
  if ij.1.1 = ij.1.2 then 1 else s ij.1.2 - s ij.1.1

/-- A genuine linear map on the inherited independent upper-entry coordinates. -/
def tangentMap {k : ℕ} (s : Fin k → ℝ) :
    UpperCoordinates k →ₗ[ℝ] UpperCoordinates k :=
  Matrix.toLin' (Matrix.diagonal (weight s))

@[simp] theorem tangentMap_apply {k : ℕ} (s : Fin k → ℝ)
    (a : UpperCoordinates k) (ij : UpperIndex k) :
    tangentMap s a ij = weight s ij * a ij := by
  simp [tangentMap, Matrix.toLin'_apply, Matrix.mulVec, dotProduct,
    Matrix.diagonal_apply]

/-- The commutator computation uses ordinary upper entries, with no sqrt(2) scaling. -/
theorem upperEntries_commutator {k : ℕ} (s : Fin k → ℝ)
    (a : UpperCoordinates k) :
    upperEntries (Matrix.diagonal (diagonalVelocity a) +
      skew a * Matrix.diagonal s - Matrix.diagonal s * skew a) =
      tangentMap s a := by
  funext ij
  rcases ij with ⟨⟨i, ⟩j, ⟩hij
  by_cases heq : i = j
  · subst j
    simp [upperEntries, diagonalVelocity, weight]
  · have hlt : i < j := lt_of_le_of_ne hij heq
    simp only [upperEntries, Matrix.sub_apply, Matrix.add_apply,
      Matrix.mul_diagonal, Matrix.diagonal_mul, Matrix.diagonal_apply_ne _ heq,
      tangentMap_apply, weight, ite_eq_right heq, skew_upper a hlt]
    ring
```

61.3 Signed determinant and the inherited Vandermonde

The determinant of a diagonal linear map is the product of its coordinate weights, by `LinearMap.det_toLin'` and `Matrix.det_diagonal`. Extending the upper-index product to all pairs inserts only factors 1, so

$$\det \mathcal{T}_s = \prod_{i<j} (s_j - s_i).$$

The implementation next proves that this product is exactly the existing list-based `077.Residual.vandermondeFins`. It splits off the first coordinate and uses the recursion

$$\Delta_{k+1}(s) = \left(\prod_{j=0}^{k-1} (s_{j+1} - s_0) \right) \Delta_k(s_1, \ldots, s_k).$$

The parenthesized difference is the body of the finite product. Induction using `List.ofFn_succ` matches the original list recursion. This adapter is valid for every tuple, without positivity or sorting.

```
theorem det_tangentMap {k : ℕ} (s : Fin k → ℝ) :
    LinearMap.det (tangentMap s) = ∏ ij : UpperIndex k, weight s ij := by
  simp [tangentMap, LinearMap.det_toLin', Matrix.det_diagonal]

def signedVandermonde {k : ℕ} (s : Fin k → ℝ) : ℝ :=
  ∏ ij : Fin k × Fin k, if ij.1 < ij.2 then s ij.2 - s ij.1 else 1

theorem det_eq_signedVandermonde {k : ℕ} (s : Fin k → ℝ) :
    LinearMap.det (tangentMap s) = signedVandermonde s := by
  rw [det_tangentMap]
```

```
  apply Fintype.prod_of_injective
    (fun ij : UpperIndex k => ij.1) Subtype.val_injective
  · intro ij hij
    have hnle : ¬ij.1 ≤ ij.2 := by
      intro hle
      exact hij ⟨⟨ij, ⟩hle, ⟩rfl
    simp [not_lt_of_ge (le_of_not_ge hnle)]
  · intro ij
    by_cases heq : ij.1.1 = ij.1.2
    · simp [weight, heq]
    · have hlt : ij.1.1 < ij.1.2 := lt_of_le_of_ne ij.2 heq
      simp [weight, heq, hlt]

theorem signedVandermonde_succ {k : ℕ} (s : Fin (k + 1) → ℝ) :
    signedVandermonde s = ∏( j : Fin k, (s j.succ - s 0)) *
      signedVandermonde (fun i : Fin k => s i.succ) := by
  simp [signedVandermonde, Fintype.prod_prod_type, Fin.prod_univ_succ]

/-- Exact adapter to the list-based Vandermonde already used by the manuscript. -/
theorem det_eq_vandermondeFin {k : ℕ} (s : Fin k → ℝ) :
    LinearMap.det (tangentMap s) = O77.Residual.vandermondeFin s := by
  rw [det_eq_signedVandermonde]
  induction k with
  | zero => simp [signedVandermonde, O77.Residual.vandermondeFin,
      O77.Residual.vandermondeList]
  | succ k ih =>
      rw [signedVandermonde_succ, ih]
      simpa only [O77.Residual.vandermondeFin, List.ofFn_succ,
        O77.Residual.vandermondeList, List.map_ofFn, List.prod_ofFn,
        Function.comp_apply] using
        mul_comm ∏( j : Fin k, (s j.succ - s 0))
          (O77.Residual.vandermondeList (List.ofFn fun j : Fin k => s j.succ))
```

61.4 Absolute determinant and chamber convention

Set $\Delta_{\mathrm{abs}}(s) = \prod_{i<j} |s_j - s_i|$. Taking absolute values factorwise yields $|\det \mathcal{T}_s| = \Delta_{\mathrm{abs}}(s) = |\,\mathrm{vandermondeFin}(s)|$. The proof again uses the same finite upper-coordinate indexing; no normalization constant is inserted. On the nonnegative nondecreasing chamber the inherited Vandermonde is nonnegative, so its absolute value can be removed by `vandermondeFin_nonneg_le`. That chamber lemma retains its original nonnegativity hypothesis even though positivity of the individual differences is the elementary reason for the sign.

```
/-- This is the full real Vandermonde, one factor for each strict upper pair. -/
def absoluteVandermonde {k : ℕ} (s : Fin k → ℝ) : ℝ :=
  ∏ ij : Fin k × Fin k, if ij.1 < ij.2 then |s ij.2 - s ij.1| else 1

theorem abs_det_tangentMap {k : ℕ} (s : Fin k → ℝ) :
    |LinearMap.det (tangentMap s)| = absoluteVandermonde s := by
  rw [det_tangentMap, Finset.abs_prod]
  apply Fintype.prod_of_injective
    (fun ij : UpperIndex k => ij.1) Subtype.val_injective
  · intro ij hij
    have hnle : ¬ij.1 ≤ ij.2 := by
      intro hle
      exact hij ⟨⟨ij, ⟩hle, ⟩rfl
    simp [not_lt_of_ge (le_of_not_ge hnle)]
  · intro ij
    by_cases heq : ij.1.1 = ij.1.2
    · simp [weight, heq]
    · have hlt : ij.1.1 < ij.1.2 := lt_of_le_of_ne ij.2 heq
```

```
      simp [weight, heq, hlt]

theorem absoluteVandermonde_eq_abs_vandermondeFin {k : ℕ} (s : Fin k → ℝ) :
    absoluteVandermonde s = |O77.Residual.vandermondeFin s| := by
  rw ←[ abs_det_tangentMap, det_eq_vandermondeFin]

theorem abs_det_eq_abs_vandermondeFin {k : ℕ} (s : Fin k → ℝ) :
    |LinearMap.det (tangentMap s)| = |O77.Residual.vandermondeFin s| := by
  rw [det_eq_vandermondeFin]

theorem abs_det_eq_vandermondeFin_on_chamber {k : ℕ} (s : Fin k → ℝ)
    (hpos : ∀ i, 0 ≤ s i)
    (hsorted : ∀ {i j : Fin k}, i < j → s i ≤ s j) :
    |LinearMap.det (tangentMap s)| = O77.Residual.vandermondeFin s := by
  rw [det_eq_vandermondeFin,
    abs_of_nonneg (O77.Residual.vandermondeFin_nonneg_le s hpos hsorted).1]
```

61.5 Simple spectra and empty dimensions

The determinant is nonzero exactly when s is injective. Indeed, each strict upper factor is a difference of two distinct entries; conversely, an equality between different entries makes one such factor zero. `LinearMap.isUnit_iff_isUnit_det` converts this criterion to invertibility of $\mathcal{T}_s$. The absolute determinant is positive under the same hypothesis, while a strictly increasing tuple makes the signed determinant positive. For $k = 0$ the determinant is an empty product, hence 1; for $k = 1$ its only coordinate weight is 1. No nonempty-index assumption is needed.

```
theorem det_ne_zero_iff_injective {k : ℕ} (s : Fin k → ℝ) :
    LinearMap.det (tangentMap s) ≠ 0 ↔ Function.Injective s := by
  rw [det_tangentMap, Finset.prod_ne_zero_iff]
  constructor
  · intro h i j hs
    by_contra hij
    rcases lt_or_gt_of_ne hij with hlt | hgt
    · have hn := h ⟨(i, j), hlt.⟩le (Finset.mem_univ _)
      exact hn (by simp [weight, hij, hs])
    · have hn := h ⟨(j, i), hgt.⟩le (Finset.mem_univ _)
      exact hn (by simp [weight, Ne.symm hij, hs])
  · intro hs ij _
    by_cases hij : ij.1.1 = ij.1.2
    · simp [weight, hij]
    · simp only [weight, ite_eq_right hij, sub_ne_zero]
      exact fun h => hij (hs h).symm

theorem tangentMap_isUnit_iff {k : ℕ} (s : Fin k → ℝ) :
    IsUnit (tangentMap s) ↔ Function.Injective s := by
  rw [LinearMap.isUnit_iff_isUnit_det, isUnit_iff_ne_zero,
    det_ne_zero_iff_injective]

theorem absoluteVandermonde_pos_iff {k : ℕ} (s : Fin k → ℝ) :
    0 < absoluteVandermonde s ↔ Function.Injective s := by
  rw ←[ abs_det_tangentMap, abs_pos, det_ne_zero_iff_injective]

theorem det_tangentMap_pos_of_strictMono {k : ℕ} (s : Fin k → ℝ)
    (hs : StrictMono s) : 0 < LinearMap.det (tangentMap s) := by
  rw [det_tangentMap]
  apply Finset.prod_pos
  intro ij _
  by_cases hij : ij.1.1 = ij.1.2
  · simp [weight, hij]
  · simp only [weight, ite_eq_right hij]
```

```
    exact sub_pos.mpr (hs (lt_of_le_of_ne ij.2 hij))

@[simp] theorem det_tangentMap_zero (s : Fin 0 → ℝ) :
    LinearMap.det (tangentMap s) = 1 := by simp [det_tangentMap]

@[simp] theorem det_tangentMap_one (s : Fin 1 → ℝ) :
    LinearMap.det (tangentMap s) = 1 := by
  rw [det_tangentMap]
  apply Finset.prod_eq_one
  intro ij _
  simp [weight, Subsingleton.elim ij.1.1 ij.1.2]
```

61.6 A polynomial curve with the required tangent

Define

$$P_{s,a}(t) = (I + tK(a))\,\mathrm{diag}(s + tv(a))(I + tK(a))^{\mathsf{T}}.$$

This is a polynomial curve; $I + tK(a)$ need not be orthogonal for $t \neq 0$. The entry formula writes it as one finite sum of triple scalar products. The two affine-derivative helpers prove $(b + tc)' = c$ first in a real normed vector space and then for real entries. They will be reused by the genuine Cayley chart, so the polynomial calculation is only a preparatory differential computation.

```
/-- A polynomial curve with the orthogonal-conjugation tangent at zero.
It is not asserted to remain in the orthogonal group at nonzero times. -/
def conjugationCurve {k : ℕ} (s : Fin k → ℝ)
    (a : UpperCoordinates k) (t : ℝ) : Matrix (Fin k) (Fin k) ℝ :=
  (1 + t • skew a) * Matrix.diagonal (s + t • diagonalVelocity a) *
    (1 + t • skew a).transpose

theorem conjugationCurve_apply {k : ℕ} (s : Fin k → ℝ)
    (a : UpperCoordinates k) (t : ℝ) (i j : Fin k) :
    conjugationCurve s a t i j =
      ∑ l : Fin k,
        (((1 : Matrix (Fin k) (Fin k) ℝ) i l + t * skew a i l) *
          (s l + t * diagonalVelocity a l)) *
        ((1 : Matrix (Fin k) (Fin k) ℝ) j l + t * skew a j l) := by
  rw [conjugationCurve, Matrix.mul_apply]
  simp only [Matrix.mul_diagonal, Matrix.transpose_apply, Matrix.add_apply,
    Matrix.smul_apply, smul_eq_mul, Pi.add_apply, Pi.smul_apply]

/-- Affine derivatives shared by the matrix and coordinate Cayley curves. -/
theorem affine_smul_hasDerivAt {E : Type*} [NormedAddCommGroup E]
    [NormedSpace ℝ E] (b c : E) :
    HasDerivAt (fun t : ℝ => b + t • c) c 0 := by
  simpa only [one_smul] using
    ((hasDerivAt_id' (0 : ℝ)).smul_const c).const_add b

/-- Scalar specialization for entrywise derivatives of the spectral charts. -/
theorem affine_hasDerivAt (b c : ℝ) :
    HasDerivAt (fun t : ℝ => b + t * c) c 0 := by
  simpa only [smul_eq_mul] using affine_smul_hasDerivAt b c
```

61.7 Differentiation, rather than an assumed tangent formula

Apply the scalar product rule to each summand of the entry formula. At $t = 0$, the identity matrices collapse the sums and skewness replaces K_{ji} by $-K_{ij}$. The result is

$$P'_{s,a}(0) = \mathrm{diag}\, v(a) + K(a)\,\mathrm{diag}\, s - \mathrm{diag}\, sK(a).$$

The theorem `hasDerivAt_pi` assembles the entry derivatives into a derivative in E_k; the previously proved commutator identity then gives $\mathcal{T}_s a$. Thus the code establishes an actual derivative, not only an algebraic expression intended to be one.

```
theorem conjugationCurve_entry_hasDerivAt {k : ℕ} (s : Fin k → ℝ)
    (a : UpperCoordinates k) (i j : Fin k) :
    HasDerivAt (fun t : ℝ => conjugationCurve s a t i j)
      ((Matrix.diagonal (diagonalVelocity a) + skew a * Matrix.diagonal s -
        Matrix.diagonal s * skew a) i j) 0 := by
  have hsum := HasDerivAt.fun_sum (u := Finset.univ) (fun l (_ : l ∈ Finset.univ) =>
    ((affine_hasDerivAt ((1 : Matrix (Fin k) (Fin k) ℝ) i l) (skew a i l)).mul
      (affine_hasDerivAt (s l) (diagonalVelocity a l))).mul
      (affine_hasDerivAt ((1 : Matrix (Fin k) (Fin k) ℝ) j l) (skew a j l)))
  have hskew : skew a j i = -skew a i j :=
    congrFun (congrFun (skew_transpose a) i) j
  apply (hsum.congr_of_eventuallyEq (Filter.Eventually.of_forall
    (fun t => conjugationCurve_apply s a t i j))).congr_deriv
  dsimp only [Pi.mul_apply]
  simp only [zero_mul, add_zero, Matrix.add_apply, Matrix.sub_apply,
    Matrix.mul_diagonal, Matrix.diagonal_mul]
  simp only [Finset.sum_add_distrib, Matrix.one_apply,
    mul_ite, ite_mul, mul_zero, zero_mul, mul_one, one_mul,
    Finset.sum_ite_eq, Finset.mem_univ, ite_true]
  by_cases hij : i = j
  · subst j
    simp [diagonalVelocity]
  · (simp [hij, hskew]; ring)

/-- The actual derivative of the upper-entry curve is the endomorphism whose
determinant was computed above; it is not an assumed Hessian/Jacobian formula. -/
theorem upper_conjugationCurve_hasDerivAt {k : ℕ} (s : Fin k → ℝ)
    (a : UpperCoordinates k) :
    HasDerivAt (fun t : ℝ => upperEntries (conjugationCurve s a t))
      (tangentMap s a) 0 := by
  rw ←[ upperEntries_commutator s a, hasDerivAt_pi]
  intro ij
  exact conjugationCurve_entry_hasDerivAt s a ij.1.1 ij.1.2
```

61.8 Linearity of the coordinate operations

Diagonal extraction commutes with scalar multiplication and sends zero to zero. Skew assembly likewise sends zero to zero and respects addition and scalar multiplication. The latter identities are proved entrywise through the defining order cases. These helpers ensure that a line $t \mapsto ta$ in E_k is sent to the literal matrices and tuples used in the preceding curve; no implicit change of coordinates occurs.

```
@[simp] theorem diagonalVelocity_smul {k : ℕ} (t : ℝ) (a : UpperCoordinates k) :
    diagonalVelocity (t • a) = t • diagonalVelocity a := rfl

@[simp] theorem diagonalVelocity_zero (k : ℕ) :
    diagonalVelocity (0 : UpperCoordinates k) = 0 := rfl

@[simp] theorem skew_zero (k : ℕ) : skew (0 : UpperCoordinates k) = 0 := by
  ext i j
  simp [skew]

/-- Addition in the independent coordinates is addition of skew matrices. -/
theorem skew_add {k : ℕ} (a b : UpperCoordinates k) :
    skew (a + b) = skew a + skew b := by
  ext i j
  rw [Matrix.add_apply]
```

```
  simp only [skew, Pi.add_apply]
  split_ifs <;> ring

@[simp] theorem skew_smul {k : ℕ} (t : ℝ) (a : UpperCoordinates k) :
    skew (t • a) = t • skew a := by
  ext i j
  rw [Matrix.smul_apply]
  change skew (t • a) i j = t * skew a i j
  simp only [skew, Pi.smul_apply, smul_eq_mul]
  split_ifs <;> ring
```

61.9 From directional derivatives to the full derivative

Let $\mathcal{P}_s(a) = \mathrm{upperEntries}((I + K(a))\,\mathrm{diag}(s + v(a))(I + K(a))^{\mathsf{T}})$. Then $\mathcal{P}_s(0) = \mathrm{upperEntries}(\mathrm{diag}\, s)$ and $\mathcal{P}_s(ta) = \mathrm{upperEntries}(P_{s,a}(t))$. Each entry is a finite polynomial expression, so the code proves global Fréchet differentiability entrywise. This existence step matters: directional derivatives alone would not establish Fréchet differentiability. The two private helpers expose the linear coordinate functions to the library's differentiability automation.

```
/-- The full polynomial parameter map; no orthogonality away from zero is claimed. -/
def spectralPolynomial {k : ℕ} (s : Fin k → ℝ) (a : UpperCoordinates k) :
    UpperCoordinates k :=
  upperEntries ((1 + skew a) * Matrix.diagonal (s + diagonalVelocity a) *
    (1 + skew a).transpose)

@[simp] theorem spectralPolynomial_zero {k : ℕ} (s : Fin k → ℝ) :
    spectralPolynomial s 0 = upperEntries (Matrix.diagonal s) := by
  simp [spectralPolynomial]

theorem spectralPolynomial_smul {k : ℕ} (s : Fin k → ℝ)
    (a : UpperCoordinates k) (t : ℝ) :
    spectralPolynomial s (t • a) = upperEntries (conjugationCurve s a t) := by
  simp [spectralPolynomial, conjugationCurve]

@[fun_prop] private theorem diagonalVelocity_entry_differentiable {k : ℕ} (i : Fin k) :
    Differentiable ℝ (fun a : UpperCoordinates k => diagonalVelocity a i) := by
  unfold diagonalVelocity
  fun_prop

@[fun_prop] private theorem skew_entry_differentiable {k : ℕ} (i j : Fin k) :
    Differentiable ℝ (fun a : UpperCoordinates k => skew a i j) := by
  unfold skew
  split_ifs <;> fun_prop

theorem spectralPolynomial_differentiable {k : ℕ} (s : Fin k → ℝ) :
    Differentiable ℝ (spectralPolynomial s) := by
  apply differentiable_pi.2
  intro ij
  have heq : (fun a : UpperCoordinates k => spectralPolynomial s a ij) =
      (fun a => ∑ l : Fin k,
        (((1 : Matrix (Fin k) (Fin k) ℝ) ij.1.1 l + skew a ij.1.1 l) *
          (s l + diagonalVelocity a l)) *
        ((1 : Matrix (Fin k) (Fin k) ℝ) ij.1.2 l + skew a ij.1.2 l)) := by
    funext a
    simpa [spectralPolynomial, conjugationCurve, upperEntries] using
      conjugationCurve_apply s a 1 ij.1.1 ij.1.2
  rw [heq]
  fun_prop
```

61.10 Identification of the full derivative

Let $L = D\mathcal{P}_s(0)$, whose existence was just proved. Compose L with the line $t \mapsto ta$. The chain rule says that this curve has derivative La; the explicit curve computation says its derivative is $\mathcal{T}_s a$. Uniqueness of derivatives gives equality for every a, and extensionality of continuous linear maps gives $L = \mathcal{T}_s$. The final two lemmas substitute this identity into the determinant. This completes the polynomial tangent calculation; the next modules supply a genuinely orthogonal chart and its measure-theoretic use.

```
/-- The entire polynomial map has the computed Frechet derivative at zero.
Differentiability is established first; directional uniqueness then identifies
its actual derivative. This remains distinct from an orthogonal-group chart. -/
theorem spectralPolynomial_hasFDerivAt {k : ℕ} (s : Fin k → ℝ) :
    HasFDerivAt (spectralPolynomial s) (tangentMap s).toContinuousLinearMap 0 := by
  have hd : HasFDerivAt (spectralPolynomial s)
      (fderiv ℝ (spectralPolynomial s) (0 : UpperCoordinates k)) 0 :=
    (spectralPolynomial_differentiable s).differentiableAt.hasFDerivAt
  apply hd.congr_fderiv
  apply ContinuousLinearMap.ext
  intro a
  have hline : HasDerivAt (fun t : ℝ => t • a) a 0 := by
    simpa using (hasDerivAt_id (0 : ℝ)).smul_const a
  have hcomp := hd.comp_hasDerivAt_of_eq (0 : ℝ) hline (by simp)
  have hcurve : HasDerivAt (fun t : ℝ => upperEntries (conjugationCurve s a t))
      (fderiv ℝ (spectralPolynomial s) 0 a) 0 := by
    simpa only [Function.comp_def, spectralPolynomial_smul] using hcomp
  exact hcurve.unique (upper_conjugationCurve_hasDerivAt s a)

theorem det_fderiv_spectralPolynomial {k : ℕ} (s : Fin k → ℝ) :
    LinearMap.det (fderiv ℝ (spectralPolynomial s) 0).toLinearMap =
      O77.Residual.vandermondeFin s := by
  rw [(spectralPolynomial_hasFDerivAt s).fderiv]
  exact det_eq_vandermondeFin s

theorem abs_det_fderiv_spectralPolynomial {k : ℕ} (s : Fin k → ℝ) :
    |LinearMap.det (fderiv ℝ (spectralPolynomial s) 0).toLinearMap| =
      |O77.Residual.vandermondeFin s| := by
  rw [det_fderiv_spectralPolynomial]

end O77.SpectralDifferential
end
```

62 Half-normalized Cayley coordinates

All matrices here are real $k \times k$ matrices, with $k \geq 0$. The module constructs an actual coordinate chart on the orthogonal group using algebraic inverse identities. Matrix inverse in Lean is a total function, so every cancellation below explicitly supplies the required determinant nonsingularity. The final bijection restricts the source to skew matrices and specifies the exact image.

Implementation namespace and dependencies. Module `O77OrthogonalCayley`; imports `Mathlib`.

```
import Mathlib

set_option autoImplicit false
noncomputable section
open Matrix

namespace O77.OrthogonalCayley
```

62.1 The convention and its normalization

Write $A(K) = I - K/2$, $B(K) = I + K/2$, and

$$C(K) = A(K)^{-1}B(K), \qquad H(Q) = 2(Q - I)(Q + I)^{-1}.$$

These are `minusHalf`, `plusHalf`, `cayley`, and `inverseCayley`. The factor $1/2$ makes the differential of C at zero the identity on skew matrices; omitting it would change the angular Jacobian. At this point these are total matrix expressions, not assertions that every denominator is invertible.

```
/-- The half factor makes the Cayley differential at zero the identity. -/
def minusHalf {k : ℕ} (K : Matrix (Fin k) (Fin k) ℝ) :
    Matrix (Fin k) (Fin k) ℝ := 1 - (1 / 2 : ℝ) • K

def plusHalf {k : ℕ} (K : Matrix (Fin k) (Fin k) ℝ) :
    Matrix (Fin k) (Fin k) ℝ := 1 + (1 / 2 : ℝ) • K

def cayley {k : ℕ} (K : Matrix (Fin k) (Fin k) ℝ) :
    Matrix (Fin k) (Fin k) ℝ := (minusHalf K)⁻¹ * plusHalf K

def inverseCayley {k : ℕ} (Q : Matrix (Fin k) (Fin k) ℝ) :
    Matrix (Fin k) (Fin k) ℝ :=
  (2 : ℝ) • ((Q - 1) * (Q + 1)⁻¹)
```

62.2 Global nonsingularity on the skew subspace

If $K^{\mathsf{T}} = -K$, then $x^{\mathsf{T}}Kx = 0$: transposition changes this scalar to its negative. If $A(K)x = 0$, pairing with x therefore gives $x^{\mathsf{T}}x = 0$, hence $x = 0$. The proof obtains a nonzero kernel vector from failure of invertibility via `Matrix.mulVec_injective_iff_isUnit`, then contradicts positive definiteness of the identity matrix. It follows that $A(K)$ is invertible. Since $A(K)^{\mathsf{T}} = B(K)$, the same holds for $B(K)$. This argument includes $k = 0$, where no nonzero vector exists.

```
theorem dotProduct_skew_mulVec {k : ℕ}
    (K : Matrix (Fin k) (Fin k) ℝ) (hK : K.transpose = -K)
    (x : Fin k → ℝ) : dotProduct x (K ᵥ* x) = 0 := by
  have h := Matrix.dotProduct_transpose_mulVec K x x
  rw [hK, Matrix.neg_mulVec, dotProduct_neg] at h
  linarith

/-- Global nonsingularity on the skew locus, without a nonempty-index assumption. -/
theorem isUnit_det_minusHalf {k : ℕ}
    (K : Matrix (Fin k) (Fin k) ℝ) (hK : K.transpose = -K) :
    IsUnit (minusHalf K).det := by
  apply (Matrix.isUnit_iff_isUnit_det _).mp
  by_contra hu
  obtain ⟨x, hx, ⟩hzero : ∃ x : Fin k → ℝ,
      x ≠ 0 ∧ minusHalf K ᵥ* x = 0 := by
    obtain ⟨a, b, heq, ⟩hab := Function.not_injective_iff.mp
      (Matrix.mulVec_injective_iff_isUnit.not.mpr hu)
    refine ⟨a - b, sub_ne_zero.mpr hab, ?⟩
    simp only [Matrix.mulVec_sub, heq, sub_self]
  have hdot := congrArg (dotProduct x) hzero
  have hself : dotProduct x x = 0 := by
    simpa [minusHalf, Matrix.sub_mulVec, Matrix.smul_mulVec,
      dotProduct_sub, dotProduct_smul, dotProduct_skew_mulVec K hK x]
      using hdot
  have hpos := (Matrix.PosDef.one :
    (1 : Matrix (Fin k) (Fin k) ℝ).PosDef).dotProduct_mulVec_pos hx
  simp [hself] at hpos
```

```
theorem transpose_minusHalf {k : ℕ}
    (K : Matrix (Fin k) (Fin k) ℝ) (hK : K.transpose = -K) :
    (minusHalf K).transpose = plusHalf K := by
  simp [minusHalf, plusHalf, hK]

theorem transpose_plusHalf {k : ℕ}
    (K : Matrix (Fin k) (Fin k) ℝ) (hK : K.transpose = -K) :
    (plusHalf K).transpose = minusHalf K := by
  simp [minusHalf, plusHalf, hK, sub_eq_add_neg]

theorem isUnit_det_plusHalf {k : ℕ}
    (K : Matrix (Fin k) (Fin k) ℝ) (hK : K.transpose = -K) :
    IsUnit (plusHalf K).det := by
  rw ←[ transpose_minusHalf K hK, Matrix.det_transpose]
  exact isUnit_det_minusHalf K hK
```

62.3 Commutation and orthogonality

The factors A and B commute because both are polynomials in K. Multiplying by A^{-1} shows that A^{-1} commutes with B, giving both $AC = B$ and $CA = B$. Transposition then gives $C^{\mathsf{T}} = B^{-1}A$. Cancelling the proved invertible factors yields $C^{\mathsf{T}}C = I$, and `mul_eq_one_comm` gives $CC^{\mathsf{T}} = I$. Finally $\det B = \det A$ because $B = A^{\mathsf{T}}$, so $\det C = (\det A)^{-1} \det B = 1$. The origin is sent to I. These are identities about the actual matrices, not orthogonality fields assumed in a chart package.

```
theorem minusHalf_mul_plusHalf {k : ℕ}
    (K : Matrix (Fin k) (Fin k) ℝ) :
    minusHalf K * plusHalf K = plusHalf K * minusHalf K := by
  unfold minusHalf plusHalf
  simp only [sub_mul, mul_add, add_mul, mul_sub, one_mul, mul_one]
  abel

private theorem inv_mul_comm_of_mul_comm {k : ℕ}
    (A B : Matrix (Fin k) (Fin k) ℝ) (hA : IsUnit A.det)
    (hAB : A * B = B * A) : ⅟A * B = B * ⅟A := by
  calc
    ⅟A * B = ⅟A * B * (A * ⅟A) := by
      rw [A.mul_nonsing_inv hA, mul_one]
    _ = ⅟A * (B * A) * ⅟A := by simp only [mul_assoc]
    _ = ⅟A * (A * B) * ⅟A := by rw ←[ hAB]
    _ = B * ⅟A := by rw [A.nonsing_inv_mul_cancel_left B hA]

theorem cayley_eq_plusHalf_mul_inv {k : ℕ}
    (K : Matrix (Fin k) (Fin k) ℝ) (hK : K.transpose = -K) :
    cayley K = plusHalf K * (minusHalf K)⁻¹ :=
  inv_mul_comm_of_mul_comm _ _ (isUnit_det_minusHalf K hK)
    (minusHalf_mul_plusHalf K)

theorem minusHalf_mul_cayley {k : ℕ}
    (K : Matrix (Fin k) (Fin k) ℝ) (hK : K.transpose = -K) :
    minusHalf K * cayley K = plusHalf K := by
  exact (minusHalf K).mul_nonsing_inv_cancel_left _
    (isUnit_det_minusHalf K hK)

theorem cayley_mul_minusHalf {k : ℕ}
    (K : Matrix (Fin k) (Fin k) ℝ) (hK : K.transpose = -K) :
    cayley K * minusHalf K = plusHalf K := by
  rw [cayley_eq_plusHalf_mul_inv K hK]
  exact (minusHalf K).nonsing_inv_mul_cancel_right _
    (isUnit_det_minusHalf K hK)
```

```
theorem transpose_cayley {k : ℕ}
    (K : Matrix (Fin k) (Fin k) ℝ) (hK : K.transpose = -K) :
    (cayley K).transpose = (plusHalf K)⁻¹ * minusHalf K := by
  rw [cayley_eq_plusHalf_mul_inv K hK, Matrix.transpose_mul,
    Matrix.transpose_nonsing_inv, transpose_minusHalf K hK,
    transpose_plusHalf K hK]

theorem transpose_cayley_mul_cayley {k : ℕ}
    (K : Matrix (Fin k) (Fin k) ℝ) (hK : K.transpose = -K) :
    (cayley K).transpose * cayley K = 1 := by
  rw [transpose_cayley K hK, cayley]
  calc
    (plusHalf K)⁻¹ * minusHalf K * ((minusHalf K)⁻¹ * plusHalf K) =
        (plusHalf K)⁻¹ * (minusHalf K * (minusHalf K)⁻¹) * plusHalf K := by
      simp only [mul_assoc]
    _ = 1 := by
      rw [(minusHalf K).mul_nonsing_inv (isUnit_det_minusHalf K hK),
        mul_one, (plusHalf K).nonsing_inv_mul (isUnit_det_plusHalf K hK)]

theorem cayley_mul_transpose_cayley {k : ℕ}
    (K : Matrix (Fin k) (Fin k) ℝ) (hK : K.transpose = -K) :
    cayley K * (cayley K).transpose = 1 :=
  mul_eq_one_comm.mp (transpose_cayley_mul_cayley K hK)

theorem det_cayley {k : ℕ}
    (K : Matrix (Fin k) (Fin k) ℝ) (hK : K.transpose = -K) :
    (cayley K).det = 1 := by
  rw [cayley, Matrix.det_mul, ← transpose_minusHalf K hK, Matrix.det_transpose]
  exact (minusHalf K).det_nonsing_inv_mul_det (isUnit_det_minusHalf K hK)

@[simp] theorem cayley_zero {k : ℕ} :
    cayley (0 : Matrix (Fin k) (Fin k) ℝ) = 1 := by
  simp [cayley, minusHalf, plusHalf]
```

62.4 The inverse formula on Cayley images

The identities $B + A = 2I$ and $B - A = K$ imply

$$(C(K) + I)(A(K)/2) = I.$$

Thus $C(K) + I$ is invertible and its inverse is exactly $A(K)/2$. Substitution into H gives $H(C(K)) = K$. These calculations prove both injectivity of the Cayley map on skew matrices and one of its chart inverse identities. They do not assume that every orthogonal matrix belongs to its image.

```
theorem plusHalf_add_minusHalf {k : ℕ}
    (K : Matrix (Fin k) (Fin k) ℝ) :
    plusHalf K + minusHalf K = (2 : ℝ) • 1 := by
  ext i j
  simp only [plusHalf, minusHalf, Matrix.add_apply, Matrix.sub_apply,
    Matrix.smul_apply, smul_eq_mul]
  ring

theorem plusHalf_sub_minusHalf {k : ℕ}
    (K : Matrix (Fin k) (Fin k) ℝ) :
    plusHalf K - minusHalf K = K := by
  ext i j
  simp only [plusHalf, minusHalf, Matrix.add_apply, Matrix.sub_apply,
    Matrix.smul_apply, smul_eq_mul]
  ring
```

```
theorem add_one_mul_half_minusHalf {k : ℕ}
    (K : Matrix (Fin k) (Fin k) ℝ) (hK : K.transpose = -K) :
    (cayley K + 1) * ((1 / 2 : ℝ) • minusHalf K) = 1 := by
  rw [Matrix.mul_smul, add_mul, cayley_mul_minusHalf K hK, one_mul,
    plusHalf_add_minusHalf K, smul_smul]
  norm_num

theorem isUnit_det_cayley_add_one {k : ℕ}
    (K : Matrix (Fin k) (Fin k) ℝ) (hK : K.transpose = -K) :
    IsUnit (cayley K + 1).det :=
  Matrix.isUnit_det_of_right_inverse (add_one_mul_half_minusHalf K hK)

theorem inv_cayley_add_one {k : ℕ}
    (K : Matrix (Fin k) (Fin k) ℝ) (hK : K.transpose = -K) :
    (cayley K + 1)⁻¹ = (1 / 2 : ℝ) • minusHalf K :=
  Matrix.inv_eq_right_inv (add_one_mul_half_minusHalf K hK)

theorem inverseCayley_cayley {k : ℕ}
    (K : Matrix (Fin k) (Fin k) ℝ) (hK : K.transpose = -K) :
    inverseCayley (cayley K) = K := by
  rw [inverseCayley, inv_cayley_add_one K hK, Matrix.mul_smul,
    sub_mul, cayley_mul_minusHalf K hK, one_mul,
    plusHalf_sub_minusHalf K, smul_smul]
  norm_num
```

62.5 Recovering every frame in the specified image

Now assume $Q^{\mathsf{T}}Q = I$ and $Q + I$ invertible. The definition gives $H(Q)(Q + I) = 2(Q - I)$. To prove skewness, expand

$$(Q^{\mathsf{T}} - I)(Q + I) = -(Q^{\mathsf{T}} + I)(Q - I)$$

and cancel the invertible outer factors $(Q + I)^{\mathsf{T}}$ and $Q + I$. This yields $H(Q)^{\mathsf{T}} = -H(Q)$. Rearranging the preceding product identity gives $(I - H(Q)/2)Q = I + H(Q)/2$, so nonsingularity on the skew subspace now implies $C(H(Q)) = Q$. Consequently

$$C : \{K : K^{\mathsf{T}} = -K\} \longrightarrow \{Q : Q^{\mathsf{T}}Q = I,\ \det(Q + I) \neq 0\}$$

is a bijection. The final `Set.BijOn` records membership, injectivity, and surjectivity with these exact sets.

```
theorem inverseCayley_mul_add_one {k : ℕ}
    (Q : Matrix (Fin k) (Fin k) ℝ) (hQ : IsUnit (Q + 1).det) :
    inverseCayley Q * (Q + 1) = (2 : ℝ) • (Q - 1) := by
  rw [inverseCayley, Matrix.smul_mul,
    (Q + 1).nonsing_inv_mul_cancel_right (Q - 1) hQ]

theorem transpose_inverseCayley {k : ℕ}
    (Q : Matrix (Fin k) (Fin k) ℝ)
    (horth : Q.transpose * Q = 1) (hQ : IsUnit (Q + 1).det) :
    (inverseCayley Q).transpose = -inverseCayley Q := by
  let := Matrix.invertibleOfIsUnitDet (Q + 1) hQ
  let := Matrix.invertibleOfIsUnitDet (Q + 1).transpose
    ((Q + 1).isUnit_det_transpose hQ)
  apply Matrix.mul_right_injective_of_invertible (Q + 1).transpose
  apply Matrix.mul_left_injective_of_invertible (Q + 1)
  have hpoly : (Q.transpose - 1) * (Q + 1) =
      -((Q.transpose + 1) * (Q - 1)) := by
    simp only [sub_mul, mul_add, add_mul, mul_sub, horth, mul_one, one_mul]
```

```
    abel
  calc
    ((Q + 1).transpose * (inverseCayley Q).transpose) * (Q + 1) =
        (inverseCayley Q * (Q + 1)).transpose * (Q + 1) := by
      rw [Matrix.transpose_mul]
    _ = (2 : ℝ) • ((Q.transpose - 1) * (Q + 1)) := by
      rw [inverseCayley_mul_add_one Q hQ]
      simp only [Matrix.transpose_smul, Matrix.transpose_sub,
        Matrix.transpose_one, Matrix.smul_mul]
    _ = -(2 : ℝ) • ((Q.transpose + 1) * (Q - 1)) := by
      rw [hpoly, smul_neg, neg_smul]
    _ = ((Q + 1).transpose * -inverseCayley Q) * (Q + 1) := by
      rw [mul_assoc, neg_mul, inverseCayley_mul_add_one Q hQ,
        mul_neg, Matrix.mul_smul]
      simp only [Matrix.transpose_add, Matrix.transpose_one, neg_smul]

theorem cayley_inverseCayley {k : ℕ}
    (Q : Matrix (Fin k) (Fin k) ℝ)
    (horth : Q.transpose * Q = 1) (hQ : IsUnit (Q + 1).det) :
    cayley (inverseCayley Q) = Q := by
  have hskew := transpose_inverseCayley Q horth hQ
  have hprod : inverseCayley Q * Q =
      (2 : ℝ) • (Q - 1) - inverseCayley Q := by
    apply eq_sub_iff_add_eq.mpr
    simpa only [mul_add, mul_one] using inverseCayley_mul_add_one Q hQ
  have hsolve : minusHalf (inverseCayley Q) * Q =
      plusHalf (inverseCayley Q) := by
    unfold minusHalf plusHalf
    rw [sub_mul, one_mul, Matrix.smul_mul, hprod]
    ext i j
    simp only [Matrix.add_apply, Matrix.sub_apply, Matrix.smul_apply, smul_eq_mul]
    ring
  unfold cayley
  rw ←[ hsolve]
  exact (minusHalf (inverseCayley Q)).nonsing_inv_mul_cancel_left Q
    (isUnit_det_minusHalf _ hskew)

/-- Exact algebraic chart image; openness and differentiability are separate clients. -/
theorem cayley_bijOn {k : ℕ} :
    Set.BijOn (@cayley k) {K | K.transpose = -K}
      {Q | Q.transpose * Q = 1 ∧ IsUnit (Q + 1).det} := by
  refine ⟨?_, ?_, ?⟩
  · intro K hK
    exact ⟨transpose_cayley_mul_cayley K hK, isUnit_det_cayley_add_one K ⟩hK
  · intro K hK L hL heq
    have h := congrArg inverseCayley heq
    simpa only [inverseCayley_cayley K hK, inverseCayley_cayley L hL] using h
  · intro Q hQ
    exact ⟨inverseCayley Q, transpose_inverseCayley Q hQ.1 hQ.2,
      cayley_inverseCayley Q hQ.1 hQ⟩.2

end O77.OrthogonalCayley
end
```

63 Orthogonal conjugation in upper-entry measure

Let $E_k = \mathrm{UpperCoordinates}(k)$, $S(a) = \mathrm{fromUpper}(a)$, and $da = \mathrm{upperLebesgue}(k)$, the product measure in independent upper entries. This module proves that a fixed orthogonal conjugation preserves this measure. It does not replace it by the Euclidean measure obtained

from a Frobenius-orthonormal basis of symmetric matrices.
Implementation namespace and dependencies. Module `O77ConjugationVolume`; imports `Mathlib`, `O77GramBorder`.

```
import Mathlib
import O77GramBorder

set_option autoImplicit false
noncomputable section
open Set MeasureTheory Matrix
open scoped ENNReal BigOperators

namespace O77.ConjugationVolume
open O77.Residual O77.GramBorder
```

63.1 The actual linear coordinate action

Symmetric assembly is linear, as checked separately on upper and lower entries. Define

$$c_Q(a) = \mathrm{upperEntries}(QS(a)Q^{\mathsf{T}}).$$

The next block packages this actual function as `conjugationLinearQ`, using linearity of assembly and matrix multiplication. For every Q, the conjugated matrix is symmetric; hence extracting and reassembling its upper entries recovers precisely $QS(a)Q^{\mathsf{T}}$. This round trip is necessary for the later inverse and invariant calculations.

```
@[simp] lemma fromUpper_add {k : ℕ} (a b : UpperCoordinates k) :
    fromUpper (a + b) = fromUpper a + fromUpper b := by
  ext i j
  by_cases hij : i ≤ j <;> simp [fromUpper, hij]

@[simp] lemma fromUpper_smul {k : ℕ} (c : ℝ) (a : UpperCoordinates k) :
    fromUpper (c • a) = c • fromUpper a := by
  ext i j
  by_cases hij : i ≤ j <;> simp [fromUpper, hij]

/-- The actual coordinate action on independent upper entries. -/
def conjugateUpper {k : ℕ} (Q : Matrix (Fin k) (Fin k) ℝ)
    (a : UpperCoordinates k) : UpperCoordinates k :=
  upperEntries (Q * fromUpper a * Q.transpose)

def conjugationLinear {k : ℕ} (Q : Matrix (Fin k) (Fin k) ℝ) :
    UpperCoordinates k →ₗ[ℝ] UpperCoordinates k where
  toFun := conjugateUpper Q
  map_add' a b := by
    exact congrArg upperEntries
      (show Q * fromUpper (a + b) * Q.transpose =
        Q * fromUpper a * Q.transpose + Q * fromUpper b * Q.transpose by
        rw [fromUpper_add, Matrix.mul_add, Matrix.add_mul])
  map_smul' c a := by
    exact congrArg upperEntries
      (show Q * fromUpper (c • a) * Q.transpose =
        c • (Q * fromUpper a * Q.transpose) by
        rw [fromUpper_smul, Matrix.mul_smul, Matrix.smul_mul])

@[simp] lemma conjugationLinear_apply {k : ℕ}
    (Q : Matrix (Fin k) (Fin k) ℝ) (a : UpperCoordinates k) :
    conjugationLinear Q a = conjugateUpper Q a := rfl

lemma conjugate_isSymm {k : ℕ} (Q : Matrix (Fin k) (Fin k) ℝ)
    (a : UpperCoordinates k) : (Q * fromUpper a * Q.transpose).IsSymm := by
```

```
  change (Q * fromUpper a * Q.transpose).transpose = _
  rw [Matrix.transpose_mul, Matrix.transpose_mul, Matrix.transpose_transpose,
    fromUpper_isSymm]
  simp only [Matrix.mul_assoc]

@[simp] lemma fromUpper_conjugateUpper {k : ℕ}
    (Q : Matrix (Fin k) (Fin k) ℝ) (a : UpperCoordinates k) :
    fromUpper (conjugateUpper Q a) = Q * fromUpper a * Q.transpose :=
  fromUpper_upperEntries (conjugate_isSymm Q a)
```

63.2 Inverse and spectral invariants

Suppose $Q^{\mathsf{T}}Q = I$. Then also $QQ^{\mathsf{T}} = I$, and direct multiplication proves $c_{Q^{\mathsf{T}}} c_Q = \mathrm{id}$ and the reverse identity. These functions form `conjugationEquivQhQ`. Trace cyclicity gives $\operatorname{tr} S(c_Q a) = \operatorname{tr} S(a)$. Taking determinants of orthogonality gives $(\det Q)^2 = 1$, whence $\det S(c_Q a) = \det S(a)$. Positive definiteness is equivalent on both sides by invertible congruence; the code applies `posDef_star_right_conjugate_iff` after identifying real matrix adjoint with transpose. These facts concern all symmetric matrices, without a spectral simplicity assumption.

```
lemma conjugateUpper_transpose_inverse {k : ℕ}
    (Q : Matrix (Fin k) (Fin k) ℝ) (hQ : Q.transpose * Q = 1)
    (a : UpperCoordinates k) :
    conjugateUpper Q.transpose (conjugateUpper Q a) = a := by
  unfold conjugateUpper
  rw [fromUpper_upperEntries (conjugate_isSymm Q a), Matrix.transpose_transpose]
  have hm : Q.transpose * (Q * fromUpper a * Q.transpose) * Q =
      (Q.transpose * Q) * fromUpper a * (Q.transpose * Q) := by
    noncomm_ring
  rw [hm, hQ, Matrix.one_mul, Matrix.mul_one, upperEntries_fromUpper]

def conjugationEquiv {k : ℕ} (Q : Matrix (Fin k) (Fin k) ℝ)
    (hQ : Q.transpose * Q = 1) : UpperCoordinates k ≃ₗ[ℝ] UpperCoordinates k where
  __ := conjugationLinear Q
  invFun := conjugateUpper Q.transpose
  left_inv := conjugateUpper_transpose_inverse Q hQ
  right_inv a := by
    change conjugateUpper Q (conjugateUpper Q.transpose a) = a
    have hQt : Q.transpose.transpose * Q.transpose = 1 := by
      simpa only [Matrix.transpose_transpose] using (mul_eq_one_comm.mp hQ)
    simpa only [Matrix.transpose_transpose] using
      conjugateUpper_transpose_inverse Q.transpose hQt a

lemma trace_conjugateUpper {k : ℕ} (Q : Matrix (Fin k) (Fin k) ℝ)
    (hQ : Q.transpose * Q = 1) (a : UpperCoordinates k) :
    (fromUpper (conjugateUpper Q a)).trace = (fromUpper a).trace := by
  rw [fromUpper_conjugateUpper, Matrix.trace_mul_cycle, hQ, Matrix.one_mul]

lemma det_conjugateUpper {k : ℕ} (Q : Matrix (Fin k) (Fin k) ℝ)
    (hQ : Q.transpose * Q = 1) (a : UpperCoordinates k) :
    (fromUpper (conjugateUpper Q a)).det = (fromUpper a).det := by
  have hd := congrArg Matrix.det hQ
  simp only [Matrix.det_mul, Matrix.det_transpose, Matrix.det_one] at hd
  rw [fromUpper_conjugateUpper, Matrix.det_mul, Matrix.det_mul, Matrix.det_transpose]
  calc
    Q.det * (fromUpper a).det * Q.det = (Q.det * Q.det) * (fromUpper a).det := by ring
    _ = (fromUpper a).det := by rw [hd, one_mul]

lemma posDef_conjugateUpper_iff {k : ℕ} (Q : Matrix (Fin k) (Fin k) ℝ)
    (hQ : Q.transpose * Q = 1) (a : UpperCoordinates k) :
    (fromUpper (conjugateUpper Q a)).PosDef ↔ (fromUpper a).PosDef := by
```

```
  have hu : IsUnit Q := (Matrix.isUnit_iff_isUnit_det Q).mpr
    (Matrix.isUnit_det_of_left_inverse hQ)
  have hstar : star Q = Q.transpose := by
    ext i j
    simp only [Matrix.star_eq_conjTranspose, Matrix.conjTranspose_apply,
      Matrix.transpose_apply, star_trivial]
  rw [fromUpper_conjugateUpper, ← hstar]
  exact hu.posDef_star_right_conjugate_iff
```

63.3 A nondegenerate pairing without changing the measure

Define $\beta(a,b) = \operatorname{tr}(S(a)S(b))$. Linearity of the trace and assembly gives a bilinear form. Symmetry shows

$$\beta(a,a) = \sum_{i,j} S(a)_{ij}^2 = \sum_i a_{ii}^2 + 2\sum_{i<j} a_{ij}^2.$$

If this is zero, every term in the finite nonnegative sum vanishes, and therefore every independent coordinate vanishes. This proves nondegeneracy in both arguments by testing the alleged radical vector against itself. The coordinate matrix of β thus has weights 1 on diagonal coordinates and 2 on strict upper coordinates. We use this form algebraically; it does not redefine the norm or the measure.

```
/-- The trace pairing is used only algebraically. We do not replace the
original coordinate norm or declare the independent-entry volume Frobenius. -/
def upperTraceForm (k : ℕ) : LinearMap.BilinForm ℝ (UpperCoordinates k) where
  toFun a :=
    { toFun := fun b => (fromUpper a * fromUpper b).trace
      map_add' := by
        intro b c
        simp only [fromUpper_add, Matrix.mul_add, Matrix.trace_add]
      map_smul' := by
        intro c b
        change (fromUpper a * fromUpper (c • b)).trace =
          c • (fromUpper a * fromUpper b).trace
        rw [fromUpper_smul, Matrix.mul_smul, Matrix.trace_smul] }
  map_add' := by
    intro a b
    apply LinearMap.ext
    intro c
    change (fromUpper (a + b) * fromUpper c).trace =
      (fromUpper a * fromUpper c).trace + (fromUpper b * fromUpper c).trace
    rw [fromUpper_add, Matrix.add_mul, Matrix.trace_add]
  map_smul' := by
    intro c a
    apply LinearMap.ext
    intro b
    change (fromUpper (c • a) * fromUpper b).trace =
      c • (fromUpper a * fromUpper b).trace
    rw [fromUpper_smul, Matrix.smul_mul, Matrix.trace_smul]

@[simp] lemma upperTraceForm_apply {k : ℕ} (a b : UpperCoordinates k) :
    upperTraceForm k a b = (fromUpper a * fromUpper b).trace := rfl

lemma upperTraceForm_self {k : ℕ} (a : UpperCoordinates k) :
    upperTraceForm k a a = ∑ i, ∑ j, (fromUpper a i j) ^ 2 := by
  simp only [upperTraceForm_apply, Matrix.trace, Matrix.diag_apply, Matrix.mul_apply]
  apply Finset.sum_congr rfl
  intro i _hi
  apply Finset.sum_congr rfl
  intro j _hj
```

```
  have hs : fromUpper a j i = fromUpper a i j :=
    congrFun (congrFun (fromUpper_isSymm a) i) j
  rw [hs, pow_two]

lemma eq_zero_of_upperTraceForm_self_eq_zero {k : ℕ}
    {a : UpperCoordinates k} (ha : upperTraceForm k a a = 0) : a = 0 := by
  rw [upperTraceForm_self] at ha
  have hrow : ∀ i ∈ Finset.univ, ∑( j, (fromUpper a i j) ^ 2) = 0 :=
    (Finset.sum_eq_zero_iff_of_nonneg
      (fun i _hi => Finset.sum_nonneg (fun j _hj => sq_nonneg _))).mp ha
  funext ij
  have hentry : (fromUpper a ij.1.1 ij.1.2) ^ 2 = 0 :=
    (Finset.sum_eq_zero_iff_of_nonneg (fun j _hj => sq_nonneg _)).mp
      (hrow ij.1.1 (Finset.mem_univ _)) ij.1.2 (Finset.mem_univ _)
  have hz : fromUpper a ij.1.1 ij.1.2 = 0 := by
    simpa only [sq_eq_zero_iff] using hentry
  simpa only [fromUpper, dite_eq_left ij.2, Pi.zero_apply] using hz

lemma upperTraceForm_nondegenerate (k : ℕ) : (upperTraceForm k).Nondegenerate := by
  constructor
  · intro a ha
    exact eq_zero_of_upperTraceForm_self_eq_zero (ha a)
  · intro a ha
    exact eq_zero_of_upperTraceForm_self_eq_zero (ha a)
```

63.4 The absolute coordinate determinant is one

Orthogonality and trace cyclicity imply $\beta(c_Q a, c_Q b) = \beta(a, b)$. Let A be the coordinate matrix of c_Q and D the coordinate matrix of β in the same independent upper-entry basis. Then $A^{\mathsf{T}} D A = D$. `LinearMap.BilinForm.nondegenerate_iff_det_ne_zero` gives $\det D \neq 0$ before any cancellation. Taking determinants and cancelling $\det D$ yields $(\det A)^2 = 1$, hence $|\det c_Q| = 1$. The argument retains the off-diagonal weights in D throughout, which avoids assuming that the standard upper-entry basis is Frobenius-orthonormal.

```
lemma upperTraceForm_conjugation {k : ℕ}
    (Q : Matrix (Fin k) (Fin k) ℝ) (hQ : Q.transpose * Q = 1) :
    (upperTraceForm k).comp (conjugationLinear Q) (conjugationLinear Q) =
      upperTraceForm k := by
  apply LinearMap.BilinForm.ext
  intro a b
  change (fromUpper (conjugateUpper Q a) *
      fromUpper (conjugateUpper Q b)).trace = (fromUpper a * fromUpper b).trace
  rw [fromUpper_conjugateUpper, fromUpper_conjugateUpper]
  have hm : (Q * fromUpper a * Q.transpose) * (Q * fromUpper b * Q.transpose) =
      Q * (fromUpper a * (Q.transpose * Q) * fromUpper b) * Q.transpose := by
    noncomm_ring
  rw [hm, hQ, Matrix.mul_one, Matrix.trace_mul_cycle]
  simp only ←[ Matrix.mul_assoc, hQ, Matrix.one_mul]

/-- Absolute determinant in the original independent-entry basis. The
nondegenerate trace pairing has weights one on diagonal entries and two on
off-diagonal entries; its determinant cancels without renormalizing volume. -/
theorem abs_det_conjugationLinear {k : ℕ}
    (Q : Matrix (Fin k) (Fin k) ℝ) (hQ : Q.transpose * Q = 1) :
    |LinearMap.det (conjugationLinear Q)| = 1 := by
  classical
  let D := LinearMap.BilinForm.toMatrix' (upperTraceForm k)
  let A := LinearMap.toMatrix' (conjugationLinear Q)
  have hD : D.det ≠ 0 := by
    have h := (LinearMap.BilinForm.nondegenerate_iff_det_ne_zero
```

```
      (Pi.basisFun ℝ (UpperIndex k))).mp (upperTraceForm_nondegenerate k)
    simpa only [LinearMap.BilinForm.toMatrix_basisFun] using h
  have hA : A.transpose * D * A = D := by
    dsimp only [A, D]
    rw ←[ LinearMap.BilinForm.toMatrix'_comp, upperTraceForm_conjugation Q hQ]
  have hdet := congrArg Matrix.det hA
  simp only [Matrix.det_mul, Matrix.det_transpose] at hdet
  have hsq : A.det ^ 2 = 1 := by
    apply (₀mul_right_cancel hD)
    calc
      A.det ^ 2 * D.det = A.det * D.det * A.det := by ring
      _ = D.det := hdet
      _ = 1 * D.det := (one_mul _).symm
  have hsq' : LinearMap.det (conjugationLinear Q) ^ 2 = 1 := by
    simpa only [A, LinearMap.det_toMatrix'] using hsq
  rcases sq_eq_one_iff.mp hsq' with h | h <;> simp only [h, abs_one, abs_neg]
```

63.5 Exact measure preservation

The linear map is continuous in finite dimension and has nonzero determinant. Apply `Real.map_linearMap_volume_pi_eq_smul_volume_pi`: the pushforward coordinate volume is multiplied by $|\det c_Q|^{-1}$. The preceding result makes that factor 1. The adapter `coordinateLebesgue_eq_volume` identifies the explicitly chosen product measure with the library coordinate volume, so the conclusion is exactly $(c_Q)_* da = da$. In dimension zero both spaces have mass one, and the same literal determinant computation applies.

```
/-- Exact preservation of the product measure in independent upper entries,
including the mass-one empty coordinate space. -/
theorem conjugateUpper_preserves {k : ℕ}
    (Q : Matrix (Fin k) (Fin k) ℝ) (hQ : Q.transpose * Q = 1) :
    MeasurePreserving (conjugateUpper Q) (upperLebesgue k) (upperLebesgue k) := by
  have ha := abs_det_conjugationLinear Q hQ
  have hd : LinearMap.det (conjugationLinear Q) ≠ 0 := by
    intro hz
    rw [hz, abs_zero] at ha
    exact zero_ne_one ha
  simp only [upperLebesgue, coordinateLebesgue_eq_volume]
  refine ⟨(conjugationLinear Q).continuous_of_finiteDimensional.measurable, ?⟩
  change Measure.map (conjugationLinear Q) volume = volume
  rw [Real.map_linearMap_volume_pi_eq_smul_volume_pi hd]
  simp only [abs_inv, ha, inv_one, ENNReal.ofReal_one, one_smul]

end O77.ConjugationVolume
end
```

64 The genuine orthogonal spectral map

Retain E_k, $v(a)$, and $K(a)$ from the spectral tangent module. With the proved Cayley transform $C(K) = (I - K/2)^{-1}(I + K/2)$, define $Q(a) = C(K(a))$. The spectral coordinate map is

$$\mathcal{S}_s(a) = \mathrm{upperEntries}(Q(a)\,\mathrm{diag}(s + v(a))Q(a)^{\mathsf{T}}).$$

Unlike the preparatory polynomial map, its conjugating frame is orthogonal at every parameter. All derivatives and invariants below refer to this actual function.

Implementation namespace and dependencies. Module `O77OrthogonalSpectral`; imports `Mathlib`, `O77Analytic`, `O77OrthogonalCayley`, `O77SpectralDifferential`.

```
import Mathlib
import O77Analytic
import O77OrthogonalCayley
import O77SpectralDifferential

set_option autoImplicit false
noncomputable section
open Matrix
open scoped BigOperators Matrix.Norms.Elementwise

namespace O77.OrthogonalSpectral
open O77.GramBorder O77.SpectralDifferential O77.OrthogonalCayley
```

64.1 Frames and the value at the center

The definitions `frame` and `orthogonalSpectral` are the functions just displayed. Skewness of $K(a)$ discharges the hypotheses of the Cayley algebra, giving both $Q(a)^{\mathsf{T}}Q(a) = I$ and $Q(a)Q(a)^{\mathsf{T}} = I$. At $a = 0$, the frame is I and the spectral map equals the upper entries of $\operatorname{diag} s$. No ordering, positivity, or distinctness of s is needed for these identities.

```
/-- Actual orthogonal frames; the half in the Cayley convention gives unit
skew velocity at zero. -/
def frame {k : ℕ} (a : UpperCoordinates k) : Matrix (Fin k) (Fin k) ℝ :=
  cayley (skew a)

def orthogonalSpectral {k : ℕ} (s : Fin k → ℝ) (a : UpperCoordinates k) :
    UpperCoordinates k :=
  upperEntries (frame a * Matrix.diagonal (s + diagonalVelocity a) *
    (frame a).transpose)

@[simp] theorem frame_zero (k : ℕ) : frame (0 : UpperCoordinates k) = 1 := by
  simp [frame]

theorem transpose_frame_mul_frame {k : ℕ} (a : UpperCoordinates k) :
    (frame a).transpose * frame a = 1 :=
  transpose_cayley_mul_cayley (skew a) (skew_transpose a)

theorem frame_mul_transpose_frame {k : ℕ} (a : UpperCoordinates k) :
    frame a * (frame a).transpose = 1 :=
  cayley_mul_transpose_cayley (skew a) (skew_transpose a)

@[simp] theorem orthogonalSpectral_zero {k : ℕ} (s : Fin k → ℝ) :
    orthogonalSpectral s 0 = upperEntries (Matrix.diagonal s) := by
  simp [orthogonalSpectral]
```

64.2 Analyticity with discharged inverse guards

Coordinate projections and the signed upper-entry assembly are analytic. For every a, skewness proves $\det(I - K(a)/2) \neq 0$. The established `O77.Reconstruction0905.entrywise_inverse_analyticAt` therefore makes the inverse matrix analytic near a; finite sums and products make Q analytic. Expanding only the outer multiplication gives

$$\mathcal{S}_s(a)_{ij} = \sum_l Q(a)_{il}(s_l + v_l(a))Q(a)_{jl}.$$

Each summand is analytic, proving analyticity of the full spectral map at every point. The proof uses the inverse on a genuine open nonsingular neighborhood, rather than relying on totalized inverse values at singular matrices.

```
private theorem upper_coordinate_analyticAt {k : ℕ} (a : UpperCoordinates k)
    (ij : UpperIndex k) : AnalyticAt ℝ (fun b : UpperCoordinates k => b ij) a :=
  analyticAt_pi_iff.mp
    (analyticAt_id : AnalyticAt ℝ (id : UpperCoordinates k → UpperCoordinates k) a) ij

private theorem skew_entry_analyticAt {k : ℕ} (a : UpperCoordinates k)
    (i j : Fin k) : AnalyticAt ℝ (fun b : UpperCoordinates k => skew b i j) a := by
  unfold skew
  split_ifs
  · exact upper_coordinate_analyticAt a _
  · exact (upper_coordinate_analyticAt a _).neg
  · exact analyticAt_const

/-- Analyticity uses the actual adjugate/determinant inverse already present
in the historical analytic module, with its nonsingularity guard discharged. -/
theorem frame_analyticAt {k : ℕ} (a : UpperCoordinates k) :
    AnalyticAt ℝ (@frame k) a := by
  have hminus : AnalyticAt ℝ (fun b : UpperCoordinates k =>
      (minusHalf (skew b) : O77.Reconstruction0905.RM k k)) a := by
    apply AnalyticAt.pi
    intro i
    apply AnalyticAt.pi
    intro j
    change AnalyticAt ℝ (fun b : UpperCoordinates k =>
      (1 : Matrix (Fin k) (Fin k) ℝ) i j - (1 / 2 : ℝ) * skew b i j) a
    exact analyticAt_const.sub
      ((analyticAt_const (v := (1 / 2 : ℝ))).mul (skew_entry_analyticAt a i j))
  have hinv := O77.Reconstruction0905.entrywise_inverse_analyticAt hminus
    (isUnit_iff_ne_zero.mp (isUnit_det_minusHalf (skew a) (skew_transpose a)))
  apply AnalyticAt.pi
  intro i
  apply AnalyticAt.pi
  intro j
  change AnalyticAt ℝ (fun b : UpperCoordinates k =>
    ∑ l : Fin k, (minusHalf (skew b))⁻¹ i l * plusHalf (skew b) l j) a
  apply Finset.analyticAt_fun_sum Finset.univ
  intro l _
  have hplus : AnalyticAt ℝ
      (fun b : UpperCoordinates k => plusHalf (skew b) l j) a := by
    change AnalyticAt ℝ (fun b : UpperCoordinates k =>
      (1 : Matrix (Fin k) (Fin k) ℝ) l j + (1 / 2 : ℝ) * skew b l j) a
    exact analyticAt_const.add
      ((analyticAt_const (v := (1 / 2 : ℝ))).mul (skew_entry_analyticAt a l j))
  exact (O77.Reconstruction0905.entry_analyticAt hinv i l).mul hplus

/-- Entry evaluation is shared by analyticity and differentiation.  Only the
outer matrix product is expanded; the inner diagonal product stays intact. -/
theorem orthogonalSpectral_apply {k : ℕ} (s : Fin k → ℝ)
    (a : UpperCoordinates k) (ij : UpperIndex k) :
    orthogonalSpectral s a ij =
      ∑ l : Fin k, (frame a ij.1.1 l * (s l + diagonalVelocity a l)) *
        frame a ij.1.2 l := by
  change (frame a * Matrix.diagonal (s + diagonalVelocity a) *
    (frame a).transpose) ij.1.1 ij.1.2 = _
  rw [Matrix.mul_apply]
  simp only [Matrix.mul_diagonal, Matrix.transpose_apply, Pi.add_apply]

theorem orthogonalSpectral_analyticAt {k : ℕ} (s : Fin k → ℝ)
    (a : UpperCoordinates k) : AnalyticAt ℝ (orthogonalSpectral s) a := by
  have hf (i j : Fin k) :
      AnalyticAt ℝ (fun b : UpperCoordinates k => frame b i j) a :=
    analyticAt_pi_iff.mp (analyticAt_pi_iff.mp (frame_analyticAt a) i) j
```

```
  apply AnalyticAt.pi
  intro ij
  simp only [orthogonalSpectral_apply]
  apply Finset.analyticAt_fun_sum Finset.univ
  intro l _
  exact ((hf ij.1.1 l).mul (analyticAt_const.add
    (upper_coordinate_analyticAt a ⟨(l, l), ⟩le_rfl))).mul (hf ij.1.2 l)
```

64.3 Differentiate the defining inverse equation

For a fixed direction a, the affine matrices $I \mp tK(a)/2$ have entry derivatives $\mp K(a)_{ij}/2$. Analyticity supplies existence of the entry derivatives of $Q(ta)$. Differentiating the actual identity

$$(I - tK(a)/2)Q(ta) = I + tK(a)/2$$

at zero gives $-K(a)/2 + Q'(0) = K(a)/2$, since $Q(0) = I$. Thus $Q'(0) = K(a)$. The implementation derives this by finite sums of scalar product rules and uniqueness of derivatives, making the normalizing half factors explicit.

```
private theorem minusHalf_curve_entry_hasDerivAt {k : ℕ}
    (a : UpperCoordinates k) (i j : Fin k) :
    HasDerivAt (fun t : ℝ => minusHalf (skew (t • a)) i j)
      (-(1 / 2 : ℝ) * skew a i j) 0 := by
  have heq : (fun t : ℝ => minusHalf (skew (t • a)) i j) =
      (fun t => (1 : Matrix (Fin k) (Fin k) ℝ) i j +
        t * (-(1 / 2 : ℝ) * skew a i j)) := by
    funext t
    rw [skew_smul]
    change (1 : Matrix (Fin k) (Fin k) ℝ) i j -
      (1 / 2 : ℝ) * (t * skew a i j) = _
    ring
  rw [heq]
  exact affine_hasDerivAt _ _

private theorem plusHalf_curve_entry_hasDerivAt {k : ℕ}
    (a : UpperCoordinates k) (i j : Fin k) :
    HasDerivAt (fun t : ℝ => plusHalf (skew (t • a)) i j)
      ((1 / 2 : ℝ) * skew a i j) 0 := by
  have heq : (fun t : ℝ => plusHalf (skew (t • a)) i j) =
      (fun t => (1 : Matrix (Fin k) (Fin k) ℝ) i j +
        t * ((1 / 2 : ℝ) * skew a i j)) := by
    funext t
    rw [skew_smul]
    change (1 : Matrix (Fin k) (Fin k) ℝ) i j +
      (1 / 2 : ℝ) * (t * skew a i j) = _
    ring
  rw [heq]
  exact affine_hasDerivAt _ _

/-- Differentiate (I-K/2) C(K)=I+K/2.  The inverse derivative is obtained
from the actual inverse equation, after analyticity has supplied existence. -/
theorem frame_curve_entry_hasDerivAt {k : ℕ} (a : UpperCoordinates k)
    (i j : Fin k) : HasDerivAt (fun t : ℝ => frame (t • a) i j)
      (skew a i j) 0 := by
  let dc : Matrix (Fin k) (Fin k) ℝ :=
    fun l m => deriv (fun t : ℝ => frame (t • a) l m) 0
  have hc (l m : Fin k) : HasDerivAt (fun t : ℝ => frame (t • a) l m)
      (dc l m) 0 := by
    have ha : AnalyticAt ℝ (fun t : ℝ => frame (t • a) l m) 0 := by
      have he := analyticAt_pi_iff.mp
        (analyticAt_pi_iff.mp (frame_analyticAt (0 : UpperCoordinates k)) l) m
```

```
      have ht : AnalyticAt ℝ (fun t : ℝ => t • a) 0 := by fun_prop
      exact he.comp_of_eq ht (by simp)
    exact ha.differentiableAt.hasDerivAt
  have hsum := HasDerivAt.fun_sum (u := Finset.univ)
    (fun l (_ : l ∈ Finset.univ) =>
      (minusHalf_curve_entry_hasDerivAt a i l).mul (hc l j))
  have heq : (fun t : ℝ => ∑ l : Fin k,
      minusHalf (skew (t • a)) i l * frame (t • a) l j) =
      (fun t => plusHalf (skew (t • a)) i j) := by
    funext t
    exact congrFun (congrFun
      (minusHalf_mul_cayley (skew (t • a)) (skew_transpose (t • a))) i) j
  have hsum' : HasDerivAt
      (fun t : ℝ => ∑ l : Fin k,
        minusHalf (skew (t • a)) i l * frame (t • a) l j)
      ∑( l : Fin k, (-(1 / 2 : ℝ) * skew a i l * frame ((0 : ℝ) • a) l j +
        minusHalf (skew ((0 : ℝ) • a)) i l * dc l j)) 0 := by
    simpa only [Pi.mul_apply] using hsum
  rw [heq] at hsum'
  have hv := hsum'.unique (plusHalf_curve_entry_hasDerivAt a i j)
  have hv' : -(1 / 2 : ℝ) * skew a i j + dc i j =
      (1 / 2 : ℝ) * skew a i j := by
    simpa [zero_smul, frame_zero, minusHalf, Matrix.one_apply,
      Finset.sum_add_distrib, mul_ite, ite_mul] using hv
  exact (hc i j).congr_deriv (by linarith)
```

64.4 The strict derivative and Vandermonde at zero

Differentiate the three factors in $\mathcal{S}_s(ta)$. Using the frame derivative gives $\operatorname{diag} v(a) + K(a)\operatorname{diag} s - \operatorname{diag} sK(a)$, whose upper entries are $\mathcal{T}_s a$. Analyticity already supplies a Fréchet derivative, and equality of its value on every direction with this curve derivative identifies it with $\mathcal{T}_s$. Analyticity also supplies a strict Fréchet derivative; substitution gives the same linear map in that stronger statement. The two determinant corollaries use the already proved adapter to `vandermondeFin`. Distinct eigenvalues will be needed only when invoking the inverse function theorem.

```
theorem orthogonalSpectral_curve_hasDerivAt {k : ℕ} (s : Fin k → ℝ)
    (a : UpperCoordinates k) :
    HasDerivAt (fun t : ℝ => orthogonalSpectral s (t • a)) (tangentMap s a) 0 := by
  rw [hasDerivAt_pi]
  intro ij
  have hd (l : Fin k) : HasDerivAt
      (fun t : ℝ => s l + diagonalVelocity (t • a) l)
      (diagonalVelocity a l) 0 := by
    change HasDerivAt (fun t : ℝ => s l + t * diagonalVelocity a l)
      (diagonalVelocity a l) 0
    exact affine_hasDerivAt _ _
  have hs := HasDerivAt.fun_sum (u := Finset.univ)
    (fun l (_ : l ∈ Finset.univ) =>
      ((frame_curve_entry_hasDerivAt a ij.1.1 l).mul (hd l)).mul
        (frame_curve_entry_hasDerivAt a ij.1.2 l))
  have hskew : skew a ij.1.2 ij.1.1 = -skew a ij.1.1 ij.1.2 :=
    congrFun (congrFun (skew_transpose a) ij.1.1) ij.1.2
  apply (hs.congr_of_eventuallyEq (Filter.Eventually.of_forall
    (fun t => orthogonalSpectral_apply s (t • a) ij))).congr_deriv
  rw ←[ upperEntries_commutator s a]
  simp only [Pi.mul_apply, zero_smul, frame_zero, diagonalVelocity_zero, Pi.zero_apply,
    add_zero, upperEntries, Matrix.add_apply, Matrix.sub_apply,
    Matrix.mul_diagonal, Matrix.diagonal_mul]
  simp only [Finset.sum_add_distrib, Matrix.one_apply,
```

```
    mul_ite, ite_mul, mul_zero, zero_mul, mul_one, one_mul,
    Finset.sum_ite_eq, Finset.mem_univ, ite_true]
  by_cases hij : ij.1.1 = ij.1.2
  · simp [hij]
  · simp only [Matrix.diagonal_apply_ne _ hij, ite_eq_right hij, hskew]
    ring

/-- Frechet differentiation of the actual orthogonal chart, in the inherited
upper-entry coordinate space. -/
theorem orthogonalSpectral_hasFDerivAt {k : ℕ} (s : Fin k → ℝ) :
    HasFDerivAt (orthogonalSpectral s) (tangentMap s).toContinuousLinearMap 0 := by
  have hd := (orthogonalSpectral_analyticAt s (0 : UpperCoordinates k)).differentiableAt.hasFDerivAt
  apply hd.congr_fderiv
  apply ContinuousLinearMap.ext
  intro a
  have hline : HasDerivAt (fun t : ℝ => t • a) a 0 := by
    simpa using (hasDerivAt_id (0 : ℝ)).smul_const a
  have hc := hd.comp_hasDerivAt_of_eq (0 : ℝ) hline (by simp)
  exact hc.unique (orthogonalSpectral_curve_hasDerivAt s a)

theorem orthogonalSpectral_hasStrictFDerivAt {k : ℕ} (s : Fin k → ℝ) :
    HasStrictFDerivAt (orthogonalSpectral s)
      (tangentMap s).toContinuousLinearMap 0 := by
  simpa only [(orthogonalSpectral_hasFDerivAt s).fderiv] using
    (orthogonalSpectral_analyticAt s (0 : UpperCoordinates k)).hasStrictFDerivAt

theorem det_fderiv_orthogonalSpectral {k : ℕ} (s : Fin k → ℝ) :
    LinearMap.det (fderiv ℝ (orthogonalSpectral s) 0).toLinearMap =
      O77.Residual.vandermondeFin s := by
  rw [(orthogonalSpectral_hasFDerivAt s).fderiv]
  exact det_eq_vandermondeFin s

theorem abs_det_fderiv_orthogonalSpectral {k : ℕ} (s : Fin k → ℝ) :
    |LinearMap.det (fderiv ℝ (orthogonalSpectral s) 0).toLinearMap| =
      |O77.Residual.vandermondeFin s| := by
  rw [det_fderiv_orthogonalSpectral]
```

64.5 Reconstruction and positivity

Symmetry of $Q\,\mathrm{diag}(q)Q^{\mathsf{T}}$, where $q = s + v(a)$, makes upper-entry reconstruction exact. If every $q_i > 0$, the diagonal matrix is positive definite. The identity $QQ^{\mathsf{T}} = I$ proves injectivity of row multiplication by Q; `Matrix.PosDef.mul_mul_conjTranspose_same` then proves that the actual reconstructed image is positive definite. This implication is what permits later density formulas to be used on the positive cone.

```
/-- The decoded upper coordinates are the actual symmetric conjugate. -/
theorem fromUpper_orthogonalSpectral {k : ℕ} (s : Fin k → ℝ)
    (a : UpperCoordinates k) :
    fromUpper (orthogonalSpectral s a) =
      frame a * Matrix.diagonal (s + diagonalVelocity a) * (frame a).transpose := by
  apply fromUpper_upperEntries
  show (frame a * Matrix.diagonal (s + diagonalVelocity a) *
    (frame a).transpose).transpose = _
  simp [Matrix.transpose_mul, Matrix.mul_assoc]

theorem orthogonalSpectral_posDef {k : ℕ} (s : Fin k → ℝ)
    (a : UpperCoordinates k) (hs : ∀ i, 0 < s i + diagonalVelocity a i) :
    (fromUpper (orthogonalSpectral s a)).PosDef := by
  have hq : Function.Injective (frame a).vecMul := by
    intro x y hxy
    have h := congrArg (fun z => Matrix.vecMul z (frame a).transpose) hxy
```

```
    simpa only [Matrix.vecMul_vecMul, frame_mul_transpose_frame,
      Matrix.vecMul_one] using h
  rw [fromUpper_orthogonalSpectral]
  have hD : (Matrix.diagonal (s + diagonalVelocity a)).PosDef :=
    Matrix.posDef_diagonal_iff.mpr (by simpa only [Pi.add_apply] using hs)
  simpa only [Matrix.conjTranspose_eq_transpose_of_trivial] using
    hD.mul_mul_conjTranspose_same hq
```

64.6 Trace, determinant, and the determinant test

For $G = S(\mathcal{S}_s(a))$ and $q = s + v(a)$, trace cyclicity gives $\operatorname{tr} G = \sum_i q_i$. Determinant multiplicativity and orthogonality prove the reusable identity $\det(QDQ^{\mathsf{T}}) = \det D$ for every square matrix D. It gives $\det G = \prod_i q_i$. Finally the exact matrix identity

$$I + TG = Q \operatorname{diag}(1 + Tq) Q^{\mathsf{T}}$$

yields $\det(I + TG) = \prod_i (1 + Tq_i)$. These algebraic statements hold for every real T. Positivity will be required only when factoring real powers of these determinants in the density adapter.

```
theorem trace_orthogonalSpectral {k : ℕ} (s : Fin k → ℝ)
    (a : UpperCoordinates k) :
    (fromUpper (orthogonalSpectral s a)).trace =
      ∑ i : Fin k, (s i + diagonalVelocity a i) := by
  rw [fromUpper_orthogonalSpectral, Matrix.trace_mul_cycle,
    transpose_frame_mul_frame, one_mul, Matrix.trace_diagonal]
  rfl

theorem det_frame_conjugate {k : ℕ} (a : UpperCoordinates k)
    (D : Matrix (Fin k) (Fin k) ℝ) :
    (frame a * D * (frame a).transpose).det = D.det := by
  have hdet : (frame a).det * (frame a).transpose.det = 1 := by
    rw ←[ Matrix.det_mul, frame_mul_transpose_frame, Matrix.det_one]
  rw [Matrix.det_mul, Matrix.det_mul]
  calc
    (frame a).det * D.det *
        (frame a).transpose.det =
      ((frame a).det * (frame a).transpose.det) *
        D.det := by ring
    _ = _ := by rw [hdet, one_mul]

theorem det_orthogonalSpectral {k : ℕ} (s : Fin k → ℝ)
    (a : UpperCoordinates k) :
    (fromUpper (orthogonalSpectral s a)).det =
      ∏ i : Fin k, (s i + diagonalVelocity a i) := by
  rw [fromUpper_orthogonalSpectral, det_frame_conjugate, Matrix.det_diagonal]
  rfl

/-- This identity is the pointwise bridge for the inherited determinant test;
neither its algebra nor the normalization depends on positivity of T. -/
theorem shifted_orthogonalSpectral {k : ℕ} (s : Fin k → ℝ)
    (a : UpperCoordinates k) (T : ℝ) :
    1 + T • fromUpper (orthogonalSpectral s a) =
      frame a * Matrix.diagonal (fun i => 1 + T *
        (s i + diagonalVelocity a i)) * (frame a).transpose := by
  have hd : Matrix.diagonal (fun i : Fin k =>
      1 + T * (s i + diagonalVelocity a i)) =
      1 + T • Matrix.diagonal (s + diagonalVelocity a) := by
    ext i j
    by_cases hij : i = j <;> simp [hij]
  rw [hd, Matrix.mul_add, Matrix.add_mul, Matrix.mul_smul,
    Matrix.smul_mul, mul_one, frame_mul_transpose_frame,
```

```
    fromUpper_orthogonalSpectral]

theorem det_shifted_orthogonalSpectral {k : ℕ} (s : Fin k → ℝ)
    (a : UpperCoordinates k) (T : ℝ) :
    (1 + T • fromUpper (orthogonalSpectral s a)).det =
      ∏ i : Fin k, (1 + T * (s i + diagonalVelocity a i)) := by
  rw [shifted_orthogonalSpectral, det_frame_conjugate, Matrix.det_diagonal]

end O77.OrthogonalSpectral
end
```

65 Null zero loci of real polynomials

This module proves polynomial nullity in the actual finite product measure `coordinateLebesgue`. Its conclusion applies to every finite coordinate type, including the empty type. The proof explicitly controls exceptional fibers during induction; it does not assert that every specialization of a nonzero multivariate polynomial is nonzero.

Implementation namespace and dependencies. Module `O77PolynomialNull`; imports `Mathlib`, `O77`.

```
import Mathlib
import O77

set_option autoImplicit false
noncomputable section
open Set MeasureTheory

namespace O77.PolynomialNull
open O77.Residual
```

65.1 The one-variable starting point

A nonzero real polynomial has finitely many roots by `Polynomial.finite_setOfPred_isRoot`. Finite subsets of the real line are Lebesgue null, so its evaluation is nonzero almost everywhere. The helper following this fact proves continuity of $(t, x) \mapsto \mathrm{Fin.cons}(t, x)$: each output coordinate is either the first projection or one coordinate of the second projection. This provides the measurability needed for the induction.

```
/-- The exceptional univariate fiber is finite, hence Lebesgue null. -/
lemma ae_polynomial_eval_ne_zero (p : Polynomial ℝ) (hp : p ≠ 0) :
    ∀ᵐ t ∂(volume : Measure ℝ), p.eval t ≠ 0 := by
  apply ae_iff.2
  simpa only [not_not, Polynomial.IsRoot] using
    (Polynomial.finite_setOfPred_isRoot hp).measure_zero (volume : Measure ℝ)

private lemma continuous_cons (n : ℕ) :
    Continuous (fun z : ℝ × (Fin n → ℝ) =>
      Fin.cons α( := fun _ : Fin (n+1) => ℝ) z.1 z.2) := by
  apply continuous_pi
  intro i
  refine Fin.cases ?_ (fun j => ?_) i
  · simpa only [Fin.cons_zero] using
      (continuous_fst : Continuous (Prod.fst : ℝ × (Fin n → ℝ) → ℝ))
  · simpa only [Fin.cons_succ, Function.comp_def] using (continuous_apply j).comp
      (continuous_snd : Continuous (Prod.snd : ℝ × (Fin n → ℝ) → Fin n → ℝ))
```

65.2 Induction with one fixed nonzero coefficient

For $n = 0$, all coordinate assignments coincide. If a nonzero polynomial vanished at the unique assignment, polynomial extensionality would make it zero. For $n + 1$, use `MvPolynomial.finSuccEquiv` to write $P(t, x) = \sum_j P_j(x)t^j$. Since $P \neq 0$, choose one index j with $P_j \neq 0$ before specializing x. By induction, $P_j(x) \neq 0$ for almost every x, so for those x the specialized polynomial in t is nonzero and its roots are null.

Polynomial evaluation is continuous, hence the relevant predicate on $\mathbb{R} \times \mathbb{R}^n$ is measurable. The two almost-everywhere product lemmas `Measure.ae_prod_iff_ae_ae` and `Measure.ae_ae_comm` assemble the fiberwise assertion. Finally `measurePreserving_piFinSuccAbove` transports it to the exact coordinate measure on $\mathbb{R}^{n+1}$. Degree loss under specialization causes no problem: the selected coefficient is enough.

```
/-- A nonzero real polynomial in `n` actual real coordinates is almost surely
nonzero.  The induction selects one nonzero coefficient polynomial before
applying Tonelli; it does not assume the specialized polynomial is nonzero
at every choice of the remaining coordinates. -/
theorem ae_eval_ne_zero_fin {n : ℕ} (p : MvPolynomial (Fin n) ℝ)
    (hp : p ≠ 0) :
    ∀ᵐ x ∂coordinateLebesgue (Fin n), MvPolynomial.eval x p ≠ 0 := by
  induction n with
  | zero =>
      apply Filter.Eventually.of_forall
      intro x hx
      apply hp
      apply MvPolynomial.funext
      intro y
      have hy : y = x := Subsingleton.elim _ _
      simpa only [hy, map_zero] using hx
  | succ n ih =>
      let q : Polynomial (MvPolynomial (Fin n) ℝ) :=
        MvPolynomial.finSuccEquiv ℝ n p
      have hq : q ≠ 0 := by
        intro hq
        apply hp
        apply (MvPolynomial.finSuccEquiv ℝ n).injective
        simpa only [map_zero] using hq
      obtain ⟨j, ⟩hj : ∃ j : ℕ, q.coeff j ≠ 0 := by
        by_contra! h
        apply hq
        apply Polynomial.ext
        intro j
        simpa only [Polynomial.coeff_zero] using h j
      have hcoeff : ∀ᵐ x ∂coordinateLebesgue (Fin n),
          MvPolynomial.eval x (q.coeff j) ≠ 0 := ih (q.coeff j) hj
      have hiter : ∀ᵐ x ∂coordinateLebesgue (Fin n),
          ∀ᵐ t ∂(volume : Measure ℝ),
            MvPolynomial.eval (Fin.cons t x) p ≠ 0 := by
        filter_upwards [hcoeff] with x hx
        have hslice : Polynomial.map (MvPolynomial.eval x) q ≠ 0 := by
          intro hz
          apply hx
          have h := congrArg (fun P : Polynomial ℝ => P.coeff j) hz
          simpa only [Polynomial.coeff_map, Polynomial.coeff_zero] using h
        simpa only [MvPolynomial.eval_eq_eval_mv_eval', q] using
          ae_polynomial_eval_ne_zero (Polynomial.map (MvPolynomial.eval x) q) hslice
      have hm : MeasurableSet
          {z : ℝ × (Fin n → ℝ) | MvPolynomial.eval (Fin.cons z.1 z.2) p ≠ 0} :=
        (((p.continuous_eval.comp (continuous_cons n)).measurable
          (measurableSet_singleton 0)).compl)
      have hprod : ∀ᵐ z ∂(volume : Measure ℝ).prod (coordinateLebesgue (Fin n)),
```

```
          MvPolynomial.eval (Fin.cons z.1 z.2) p ≠ 0 :=
        (Measure.ae_prod_iff_ae_ae hm).2 ((Measure.ae_ae_comm hm).2 hiter)
      have hsplit :=
        (measurePreserving_piFinSuccAbove (fun _ : Fin (n+1) => (volume : Measure ℝ)) 0).symm
      change ∀ᵐ x ∂Measure.pi (fun _ : Fin (n+1) => (volume : Measure ℝ)),
        MvPolynomial.eval x p ≠ 0
      rw ←[ hsplit.map_eq]
      apply (ae_map_iff hsplit.measurable.aemeasurable
        ((p.continuous_eval.measurable (measurableSet_singleton 0)).compl)).2
      simpa only [MeasurableEquiv.piFinSuccAbove_symm_apply, Fin.insertNthEquiv,
        Equiv.coe_fn_mk, Fin.insertNth_zero', coordinateLebesgue,
        Set.mem_preimage, Set.mem_singleton_iff] using hprod
```

65.3 Arbitrary finite coordinate indices

Choose a finite equivalence $e : \mathrm{Fin}(|J|) \simeq J$. Rename the variables of P along e^{-1}; injectivity of the polynomial renaming equivalence preserves nonzeroness. Evaluation is compatible with the inverse coordinate permutation by `MvPolynomial.eval_rename`. The exact product-measure permutation `measurePreserving_piCongrLeft` transports the finite-numerical case to $\mathbb{R}^J$. The final theorem converts the almost-everywhere nonvanishing assertion to

$$\mathrm{coordinateLebesgue}(J)\{x : P(x) = 0\} = 0.$$

This is a measured theorem about the original coordinates, not an appeal to an unspecified algebraic dimension notion.

```
/-- Finite reindexing transports the original entry-product measure exactly.
The index type need not be nonempty. -/
theorem ae_eval_ne_zero ι{ : Type*} [Fintype ι]
    (p : MvPolynomial ι ℝ) (hp : p ≠ 0) :
    ∀ᵐ x ∂coordinateLebesgue ι, MvPolynomial.eval x p ≠ 0 := by
  classical
  let e : Fin (Fintype.card ι) ≃ ι := (Fintype.equivFin ι).symm
  let q : MvPolynomial (Fin (Fintype.card ι)) ℝ :=
    MvPolynomial.renameEquiv ℝ e.symm p
  have hq : q ≠ 0 := by
    intro hq
    apply hp
    apply (MvPolynomial.renameEquiv ℝ e.symm).injective
    simpa only [map_zero] using hq
  have h := ae_eval_ne_zero_fin q hq
  have heval (x : Fin (Fintype.card ι) → ℝ) :
      MvPolynomial.eval (MeasurableEquiv.piCongrLeft (fun _ : ι => ℝ) e x) p =
        MvPolynomial.eval x q := by
    change MvPolynomial.eval (MeasurableEquiv.piCongrLeft (fun _ : ι => ℝ) e x) p =
      MvPolynomial.eval x (MvPolynomial.rename e.symm p)
    rw [MvPolynomial.eval_rename]
    apply congrArg (fun v : ι → ℝ => MvPolynomial.eval v p)
    funext i
    simpa only [Equiv.apply_symm_apply, Function.comp_apply] using
      MeasurableEquiv.piCongrLeft_apply_apply β( := fun _ : ι => ℝ) e x (e.symm i)
  have hpres := measurePreserving_piCongrLeft (fun _ : ι => (volume : Measure ℝ)) e
  change ∀ᵐ x ∂Measure.pi (fun _ : ι => (volume : Measure ℝ)),
    MvPolynomial.eval x p ≠ 0
  rw ←[ hpres.map_eq]
  apply (ae_map_iff hpres.measurable.aemeasurable
    ((p.continuous_eval.measurable (measurableSet_singleton 0)).compl)).2
  simpa only [Set.mem_preimage, Set.mem_singleton_iff, heval, coordinateLebesgue] using h

theorem zero_locus_null ι{ : Type*} [Fintype ι]
```

```
    (p : MvPolynomial ι ℝ) (hp : p ≠ 0) :
    coordinateLebesgue ι {x : ι → ℝ | MvPolynomial.eval x p = 0} = 0 := by
  simpa only [not_not] using (ae_iff.1 (ae_eval_ne_zero p hp))

end O77.PolynomialNull
end
```

66 Repeated spectra and the singular boundary

We work in independent symmetric upper entries $a \in E_k$ with $S(a) = \mathrm{fromUpper}(a)$ and measure da. Two exceptional sets must be controlled separately: repeated eigenvalues and zero determinant. Both will be zero loci of explicitly nonzero polynomials in these very coordinates, so the preceding polynomial nullity theorem applies without a normalization change.

Implementation namespace and dependencies. Module `O77SpectralExceptional`; imports `Mathlib`, `O77GramBorder`, `O77PolynomialNull`.

```
import Mathlib
import O77GramBorder
import O77PolynomialNull

set_option autoImplicit false
noncomputable section
open Set MeasureTheory Matrix
open scoped ENNReal BigOperators

namespace O77.SpectralExceptional
open O77.Residual O77.GramBorder
```

66.1 The universal symmetric matrix and fixed resultant sizes

Let $\mathbb{S}$ be the symmetric matrix over $\mathbb{R}[X_{ij} : i \le j]$ whose (i,j) entry is X_{ij} for $i \le j$ and X_{ji} otherwise. Evaluation at a gives exactly $S(a)$. Put $f = \chi_{\mathbb{S}}$ and

$$R_k = \mathrm{Res}_{k,k-1}(f, f').$$

The sizes of the Sylvester matrix are fixed before evaluation; in Lean $k-1$ is natural subtraction, hence 0 at $k = 0$. Evaluation commutes with the determinant, characteristic polynomial, and derivative. The specialized characteristic polynomial is monic of degree k, and its derivative has natural degree $k-1$, so the evaluated fixed-size resultant agrees with the ordinary resultant. These facts are supplied by `Polynomial.resultant_map_map`, `Matrix.charpoly_map`, and the characteristic-polynomial and derivative degree lemmas.

```
/-- The universal symmetric matrix uses exactly the independent upper entries.
No Frobenius rescaling is inserted in the off-diagonal coordinates. -/
def universalSymmetric (k : ℕ) :
    Matrix (Fin k) (Fin k) (MvPolynomial (UpperIndex k) ℝ) :=
  fun i j => if h : i ≤ j then MvPolynomial.X ⟨(i,j),⟩h
    else MvPolynomial.X ⟨(j,i),le_of_not_ge ⟩h

@[simp] lemma universalSymmetric_map_eval {k : ℕ} (a : UpperCoordinates k) :
    (universalSymmetric k).map (MvPolynomial.eval a) = fromUpper a := by
  ext i j
  by_cases hij : i ≤ j <;> simp [universalSymmetric, fromUpper, hij]

/-- Explicit fixed sizes are essential: specializing coefficients must not
silently change the Sylvester matrix size. Characteristic polynomials are
monic of degree k, and over the reals their derivatives have degree k-1. -/
```

```
def spectralResultant (k : ℕ) : MvPolynomial (UpperIndex k) ℝ :=
  let f := (universalSymmetric k).charpoly
  f.resultant f.derivative k (k-1)

lemma eval_spectralResultant {k : ℕ} (a : UpperCoordinates k) :
    MvPolynomial.eval a (spectralResultant k) =
      (fromUpper a).charpoly.resultant (fromUpper a).charpoly.derivative := by
  let f := (universalSymmetric k).charpoly
  have hmap : f.map (MvPolynomial.eval a) = (fromUpper a).charpoly := by
    change (universalSymmetric k).charpoly.map (MvPolynomial.eval a) = _
    rw ←[ Matrix.charpoly_map, universalSymmetric_map_eval]
  change MvPolynomial.eval a (f.resultant f.derivative k (k-1)) = _
  rw ←[ Polynomial.resultant_map_map f f.derivative k (k-1) (MvPolynomial.eval a),
    ← Polynomial.derivative_map, hmap]
  have hn : (fromUpper a).charpoly.natDegree = k := by
    simp [Matrix.charpoly_natDegree_eq_dim]
  have hd : (fromUpper a).charpoly.derivative.natDegree = k-1 := by
    rw [Polynomial.natDegree_derivative, hn]
  rw [hn, hd]
```

66.2 Detecting separability and proving nonzeroness

For the monic characteristic polynomial, the resultant criterion `Polynomial.isUnit_resultant_iff_isCoprime` gives $R_k(a) \neq 0$ precisely when $\chi_{S(a)}$ is separable. Its negation identifies the repeated-root set with $\{R_k = 0\}$. To prove that R_k is not the zero polynomial, evaluate at $\mathrm{diag}(0, 1, \ldots, k-1)$. Its characteristic polynomial is the product of distinct linear factors, hence separable by `Polynomial.separable_prod_X_sub_C_iff`. The resulting nonzero evaluation witnesses nonzeroness of R_k. This also covers $k = 0$: the characteristic polynomial is 1 and is separable.

```
lemma eval_spectralResultant_ne_zero_iff {k : ℕ} (a : UpperCoordinates k) :
    MvPolynomial.eval a (spectralResultant k) ≠ 0 ↔
      (fromUpper a).charpoly.Separable := by
  rw [eval_spectralResultant, ← isUnit_iff_ne_zero]
  exact Polynomial.isUnit_resultant_iff_isCoprime (Matrix.charpoly_monic _)

lemma eval_spectralResultant_eq_zero_iff {k : ℕ} (a : UpperCoordinates k) :
    MvPolynomial.eval a (spectralResultant k) = 0 ↔
      ¬ (fromUpper a).charpoly.Separable := by
  simpa only [not_not] using not_congr (eval_spectralResultant_ne_zero_iff a)

lemma diagonal_nat_charpoly_separable (k : ℕ) :
    (Matrix.diagonal (fun i : Fin k => (i.val : ℝ))).charpoly.Separable := by
  rw [Matrix.charpoly_diagonal, Polynomial.separable_prod_X_sub_C_iff]
  intro i j hij
  apply Fin.ext
  have hij' : (i.val : ℝ) = (j.val : ℝ) := hij
  exact_mod_cast hij'

theorem spectralResultant_ne_zero (k : ℕ) : spectralResultant k ≠ 0 := by
  let D := Matrix.diagonal (fun i : Fin k => (i.val : ℝ))
  let a := upperEntries D
  have hD : D.IsSymm := Matrix.isSymm_diagonal _
  have hsep : (fromUpper a).charpoly.Separable := by
    rw [fromUpper_upperEntries hD]
    exact diagonal_nat_charpoly_separable k
  have hval := (eval_spectralResultant_ne_zero_iff a).2 hsep
  intro hz
  exact hval (by rw [hz, map_zero])
```

66.3 Closed exceptional locus and nullity

Polynomial evaluation is continuous, so the zero locus of R_k is closed and its complement is open. Since $R_k \neq 0$, polynomial nullity gives separability of $\chi_{S(a)}$ almost everywhere and measure zero for its failure set. The code first proves the closedness and openness statements and then the measured assertions; no measurability of an arbitrarily chosen eigenbasis is required for this step.

```
/-- The exceptional set is algebraic and hence closed before any measure
argument. This also shows that the simple-spectrum locus is open. -/
theorem isClosed_not_separable_charpoly (k : ℕ) :
    IsClosed {a : UpperCoordinates k | ¬ (fromUpper a).charpoly.Separable} := by
  have h : IsClosed {a : UpperCoordinates k |
      MvPolynomial.eval a (spectralResultant k) = 0} :=
    isClosed_eq (spectralResultant k).continuous_eval continuous_const
  simpa only [eval_spectralResultant_eq_zero_iff] using h

theorem isOpen_separable_charpoly (k : ℕ) :
    IsOpen {a : UpperCoordinates k | (fromUpper a).charpoly.Separable} := by
  simpa only [Set.compl_ofPred, not_not] using
    (isClosed_not_separable_charpoly k).isOpen_compl

theorem ae_separable_charpoly (k : ℕ) :
    ∀ᵐ a ∂upperLebesgue k, (fromUpper a).charpoly.Separable := by
  have h := O77.PolynomialNull.ae_eval_ne_zero
    (spectralResultant k) (spectralResultant_ne_zero k)
  filter_upwards [h] with a ha
  exact (eval_spectralResultant_ne_zero_iff a).1 ha

theorem upperLebesgue_not_separable_charpoly (k : ℕ) :
    upperLebesgue k {a | ¬ (fromUpper a).charpoly.Separable} = 0 :=
  ae_iff.1 (ae_separable_charpoly k)
```

66.4 The actual Mathlib eigenvalues

Real symmetry implies Hermitian symmetry. Define `eigenvaluesa` to be the eigenvalues supplied by Mathlib for the actual matrix $S(a)$, with multiplicities. The Hermitian characteristic polynomial factorization and the separability criterion for a product of linear factors show

$$\chi_{S(a)} \text{ separable} \iff \text{eigenvalues}(a) \text{ injective}.$$

Thus the preceding null locus is exactly the locus of repeated actual eigenvalues. The definition does not prescribe an arbitrary tuple as the spectrum, and no sorting convention is needed here.

```
lemma fromUpper_isHermitian {k : ℕ} (a : UpperCoordinates k) :
    (fromUpper a).IsHermitian :=
  Matrix.isHermitian_iff_isSymm.2 (fromUpper_isSymm a)

/-- These are Mathlib's actual Hermitian eigenvalues, with multiplicities;
the definition does not assign a chosen proposed spectrum. -/
def eigenvalues {k : ℕ} (a : UpperCoordinates k) : Fin k → ℝ :=
  (fromUpper_isHermitian a).eigenvalues

lemma separable_charpoly_iff_injective_eigenvalues {k : ℕ}
    (a : UpperCoordinates k) :
    (fromUpper a).charpoly.Separable ↔ Function.Injective (eigenvalues a) := by
  rw [(fromUpper_isHermitian a).charpoly_eq]
  simpa [eigenvalues] using
    (Polynomial.separable_prod_X_sub_C_iff
```

```
      (f := (fromUpper_isHermitian a).eigenvalues))

theorem ae_injective_eigenvalues (k : ℕ) :
    ∀ᵐ a ∂upperLebesgue k, Function.Injective (eigenvalues a) := by
  filter_upwards [ae_separable_charpoly k] with a ha
  exact (separable_charpoly_iff_injective_eigenvalues a).1 ha

theorem upperLebesgue_repeated_eigenvalues (k : ℕ) :
    upperLebesgue k {a | ¬ Function.Injective (eigenvalues a)} = 0 :=
  ae_iff.1 (ae_injective_eigenvalues k)
```

66.5 A separate polynomial for singular matrices

Define $D_k = \det \mathbb{S}$. Evaluation commutes with the determinant, so $D_k(a) = \det S(a)$. Evaluation at the identity matrix gives 1, proving $D_k \neq 0$. Polynomial nullity therefore gives $\det S(a) \neq 0$ almost everywhere and $da\{a : \det S(a) = 0\} = 0$. This is a statement about symmetric upper coordinates; nullity of rank-deficient rectangular matrices is a different result used in the Gram pushforward.

```
/-- The singular symmetric boundary has a separate nonzero polynomial;
this is a statement about upper-entry measure, not rectangular matrices. -/
def determinantPolynomial (k : ℕ) : MvPolynomial (UpperIndex k) ℝ :=
  (universalSymmetric k).det

@[simp] lemma eval_determinantPolynomial {k : ℕ} (a : UpperCoordinates k) :
    MvPolynomial.eval a (determinantPolynomial k) = (fromUpper a).det := by
  rw [determinantPolynomial, RingHom.map_det]
  change ((universalSymmetric k).map (MvPolynomial.eval a)).det = _
  rw [universalSymmetric_map_eval]

theorem determinantPolynomial_ne_zero (k : ℕ) : determinantPolynomial k ≠ 0 := by
  let a : UpperCoordinates k := upperEntries (1 : Matrix (Fin k) (Fin k) ℝ)
  have hval : MvPolynomial.eval a (determinantPolynomial k) = 1 := by
    rw [eval_determinantPolynomial, fromUpper_upperEntries Matrix.isSymm_one,
      Matrix.det_one]
  intro hz
  rw [hz, map_zero] at hval
  exact zero_ne_one hval

theorem ae_det_ne_zero (k : ℕ) :
    ∀ᵐ a ∂upperLebesgue k, (fromUpper a).det ≠ 0 := by
  simpa only [upperLebesgue, eval_determinantPolynomial] using
    (O77.PolynomialNull.ae_eval_ne_zero
      (determinantPolynomial k) (determinantPolynomial_ne_zero k))

theorem upperLebesgue_det_zero (k : ℕ) :
    upperLebesgue k {a | (fromUpper a).det = 0} = 0 := by
  simpa only [not_not] using (ae_iff.1 (ae_det_ne_zero k))
```

66.6 Nullity persists under arbitrary nonnegative densities

The library theorem `withDensity_absolutelyContinuous` says that $da.\mathrm{withDensity}(w)$ is absolutely continuous with respect to da, for every function $w : E_k \to [0, \infty]$. Consequently both exceptional sets remain null after multiplying by the cone density. No positivity, finiteness, or measurability premise on w is needed for this absolute-continuity statement. In particular it does not justify cancellation or conversion of any infinite integral to a real number; it only allows removal of already null sets.

```
/-- All densities on the inherited upper-entry measure retain both null
```

```
exceptional sets. No positivity, finiteness, or measurability of the density
is needed for absolute continuity of `withDensity`. -/
theorem withDensity_repeated_eigenvalues (k : ℕ)
    (w : UpperCoordinates k → ℝ≥∞0) :
    (upperLebesgue k).withDensity w
      {a | ¬ Function.Injective (eigenvalues a)} = 0 :=
  withDensity_absolutelyContinuous (upperLebesgue k) w
    (upperLebesgue_repeated_eigenvalues k)

theorem withDensity_det_zero (k : ℕ) (w : UpperCoordinates k → ℝ≥∞0) :
    (upperLebesgue k).withDensity w {a | (fromUpper a).det = 0} = 0 :=
  withDensity_absolutelyContinuous (upperLebesgue k) w (upperLebesgue_det_zero k)

end O77.SpectralExceptional
end
```

67 Constructed local inverses and exact local integration

Fix $s \in \mathbb{R}^k$ with pairwise distinct entries. The actual spectral map $\mathcal{S}_s : E_k \to E_k$ has strict derivative $\mathcal{T}_s$ at zero. The next steps construct its local inverse from that derivative, and then use the injective change-of-variables theorem in the same upper-entry product measure. Positivity of s is imposed only for windows required to remain inside the positive-definite cone. *Implementation namespace and dependencies.* Module `O77SpectralLocal`; imports `Mathlib`, `O77OrthogonalSpectral`, `O77Transport`.

```
import Mathlib
import O77OrthogonalSpectral
import O77Transport

set_option autoImplicit false
noncomputable section
open Set Filter MeasureTheory
open scoped Topology ENNReal

namespace O77.SpectralLocal
open O77.Residual O77.GramBorder O77.SpectralDifferential
open O77.OrthogonalSpectral
```

67.1 The explicit inverse of the tangent

Injectivity of s makes every coordinate weight w_{ij} of $\mathcal{T}_s$ nonzero. Its inverse is therefore $b_{ij} \mapsto w_{ij}^{-1} b_{ij}$. The code checks both inverse identities coordinatewise and packages the result as a linear equivalence `tangentEquiv`. Finite dimensionality makes this a continuous linear equivalence. Its forward map is definitionally the already computed tangent map, which supplies the exact derivative interface required by the inverse theorem.

```
lemma weight_ne_zero {k : ℕ} {s : Fin k → ℝ}
    (hs : Function.Injective s) (ij : UpperIndex k) : weight s ij ≠ 0 := by
  by_cases hij : ij.1.1 = ij.1.2
  · simp [weight, hij]
  · have hne : s ij.1.2 ≠ s ij.1.1 := fun h => hij (hs h).symm
    simpa [weight, hij] using sub_ne_zero.mpr hne

/-- The inverse of the actual spectral tangent map, in the same entry basis. -/
def tangentEquiv {k : ℕ} (s : Fin k → ℝ) (hs : Function.Injective s) :
    UpperCoordinates k ≃ₗ[ℝ] UpperCoordinates k where
  toFun := tangentMap s
  invFun b ij := (weight s ij)⁻¹ * b ij
```

```
  left_inv a := by
    funext ij
    simp only [tangentMap_apply, ← mul_assoc,
      ₀inv_mul_cancel (weight_ne_zero hs ij), one_mul]
  right_inv a := by
    funext ij
    simp only [tangentMap_apply, ← mul_assoc,
      ₀mul_inv_cancel (weight_ne_zero hs ij), one_mul]
  map_add' := (tangentMap s).map_add
  map_smul' := (tangentMap s).map_smul

def tangentContinuousEquiv {k : ℕ} (s : Fin k → ℝ)
    (hs : Function.Injective s) : UpperCoordinates k ≃L[ℝ] UpperCoordinates k :=
  (tangentEquiv s hs).toContinuousLinearEquiv

@[simp] lemma tangentContinuousEquiv_toContinuousLinearMap {k : ℕ}
    (s : Fin k → ℝ) (hs : Function.Injective s) :
    (tangentContinuousEquiv s hs : UpperCoordinates k →L[ℝ] UpperCoordinates k) =
      (tangentMap s).toContinuousLinearMap := by
  ext a ij
  rfl
```

67.2 The local homeomorphism is constructed

Replace the tangent map in the established strict derivative by the continuous linear equivalence just constructed. The library theorem `HasStrictFDerivAt.toOpenPartialHomeomorph` then constructs e_s, an open partial homeomorphism whose forward function is exactly $\mathcal{S}_s$. Zero belongs to its source, and upperEntries(diag s) belongs to its target. The two inverse identities hold on these specified source and target sets; in particular $\mathcal{S}_s$ is injective on the source. The library local inverse theorem also gives the strict derivative of e_s^{-1} at the central image, equal to $\mathcal{T}_s^{-1}$. No chart or inverse package is an input hypothesis.

```
theorem strictDerivative_equiv {k : ℕ} (s : Fin k → ℝ)
    (hs : Function.Injective s) :
    HasStrictFDerivAt (orthogonalSpectral s)
      (tangentContinuousEquiv s hs : UpperCoordinates k →L[ℝ] UpperCoordinates k) 0 := by
  simpa only [tangentContinuousEquiv_toContinuousLinearMap] using
    orthogonalSpectral_hasStrictFDerivAt s

/-- The library inverse theorem constructs the chart; no inverse or chart
package is an input. Its forward map is the actual orthogonal spectral map. -/
def localChart {k : ℕ} (s : Fin k → ℝ) (hs : Function.Injective s) :
    OpenPartialHomeomorph (UpperCoordinates k) (UpperCoordinates k) :=
  (strictDerivative_equiv s hs).toOpenPartialHomeomorph (orthogonalSpectral s)

@[simp] theorem localChart_apply {k : ℕ} (s : Fin k → ℝ)
    (hs : Function.Injective s) (a : UpperCoordinates k) :
    localChart s hs a = orthogonalSpectral s a := rfl

theorem zero_mem_source {k : ℕ} (s : Fin k → ℝ) (hs : Function.Injective s) :
    (0 : UpperCoordinates k) ∈ (localChart s hs).source :=
  (strictDerivative_equiv s hs).mem_toOpenPartialHomeomorph_source

theorem diagonal_mem_target {k : ℕ} (s : Fin k → ℝ)
    (hs : Function.Injective s) :
    upperEntries (Matrix.diagonal s) ∈ (localChart s hs).target := by
  have h := (localChart s hs).map_source (zero_mem_source s hs)
  simpa only [localChart_apply, orthogonalSpectral_zero] using h

theorem localChart_left_inverse {k : ℕ} (s : Fin k → ℝ)
```

```
    (hs : Function.Injective s) {a : UpperCoordinates k}
    (ha : a ∈ (localChart s hs).source) :
    (localChart s hs).symm (orthogonalSpectral s a) = a :=
  (localChart s hs).left_inv ha

theorem localChart_right_inverse {k : ℕ} (s : Fin k → ℝ)
    (hs : Function.Injective s) {b : UpperCoordinates k}
    (hb : b ∈ (localChart s hs).target) :
    orthogonalSpectral s ((localChart s hs).symm b) = b :=
  (localChart s hs).right_inv hb

theorem orthogonalSpectral_injOn_source {k : ℕ} (s : Fin k → ℝ)
    (hs : Function.Injective s) :
    InjOn (orthogonalSpectral s) (localChart s hs).source :=
  (localChart s hs).injOn

theorem localChart_symm_hasStrictFDerivAt {k : ℕ} (s : Fin k → ℝ)
    (hs : Function.Injective s) :
    HasStrictFDerivAt (localChart s hs).symm
      ((tangentContinuousEquiv s hs).symm : UpperCoordinates k →L[ℝ] UpperCoordinates k)
      (upperEntries (Matrix.diagonal s)) := by
  simpa only [HasStrictFDerivAt.localInverse_def, localChart,
    orthogonalSpectral_zero] using (strictDerivative_equiv s hs).to_localInverse
```

67.3 The exact local Jacobian formula

Let $A \subseteq \mathrm{source}(e_s)$ be measurable and let $F : E_k \to [0,\infty]$ be any test function. Analyticity supplies a Fréchet derivative at every point, and the constructed chart supplies injectivity on A. The library theorem `lintegral_image_eq_lintegral_abs_det_fderiv_mul` gives

$$\int_{\mathcal{S}_s(A)} F(b)\,db = \int_A |\det D\mathcal{S}_s(a)| F(\mathcal{S}_s(a))\,da.$$

These are nonnegative lower integrals, so no Bochner integrability or finiteness is presumed. The measure is identified as additive Haar measure through the coordinate-volume adapter. The companion theorem `measurable_image_of_fderivWithin` proves measurability of the actual image set, rather than assuming it as a separate premise.

```
/-- Exact nonnegative change of variables on every measurable subset of the
constructed chart source. No integrability or finiteness premise is needed. -/
theorem lintegral_image {k : ℕ} (s : Fin k → ℝ) (hs : Function.Injective s)
    {S : Set (UpperCoordinates k)} (hS : MeasurableSet S)
    (hsub : S ⊆ (localChart s hs).source) (F : UpperCoordinates k → ℝ≥∞0) :
    ∫⁻ b in orthogonalSpectral s '' S, F b ∂upperLebesgue k =
      ∫⁻ a in S, ENNReal.ofReal |(fderiv ℝ (orthogonalSpectral s) a).det| *
        F (orthogonalSpectral s a) ∂upperLebesgue k := by
  let _ : Measure.IsAddHaarMeasure (upperLebesgue k) := by
    simpa only [upperLebesgue, coordinateLebesgue_eq_volume] using
      (inferInstance : Measure.IsAddHaarMeasure (volume : Measure (UpperCoordinates k)))
  exact lintegral_image_eq_lintegral_abs_det_fderiv_mul (upperLebesgue k) hS
    (fun a _ => (orthogonalSpectral_analyticAt s a).differentiableAt.hasFDerivAt.hasFDerivWithinAt)
    ((orthogonalSpectral_injOn_source s hs).mono hsub) F

theorem measurable_image {k : ℕ} (s : Fin k → ℝ) (hs : Function.Injective s)
    {S : Set (UpperCoordinates k)} (hS : MeasurableSet S)
    (hsub : S ⊆ (localChart s hs).source) :
    MeasurableSet (orthogonalSpectral s '' S) :=
  measurable_image_of_fderivWithin hS
    (fun a _ => (orthogonalSpectral_analyticAt s a).differentiableAt.hasFDerivAt.hasFDerivWithinAt)
    ((orthogonalSpectral_injOn_source s hs).mono hsub)
```

67.4 Choose one common neighborhood first

At zero the determinant is the nonzero Vandermonde of s. Continuity of the derivative determinant supplies an open neighborhood on which

$$\tfrac{1}{2}|\Delta(s)| \le |\det D\mathcal{S}_s(a)| \le 2|\Delta(s)|.$$

Intersect it with the source of e_s. This chooses one common open window before any measurable subset or test function is selected. If every $s_i > 0$, intersect once more with $\{a : \forall i,\ s_i + v_i(a) > 0\}$. This is a finite intersection of open sets containing zero, and all its images are positive definite. Both restrictions retain the same determinant bounds.

```
/-- One common open neighborhood is chosen before any measurable set or test
function. The Jacobian bounds concern the original upper-entry measure. -/
theorem exists_common_window {k : ℕ} (s : Fin k → ℝ)
    (hs : Function.Injective s) :
    ∃ U : Set (UpperCoordinates k), IsOpen U ∧ 0 ∈ U ∧
      U ⊆ (localChart s hs).source ∧
      ∀ a ∈ U, |vandermondeFin s| / 2 ≤ |(fderiv ℝ (orthogonalSpectral s) a).det| ∧
        |(fderiv ℝ (orthogonalSpectral s) a).det| ≤ 2 * |vandermondeFin s| := by
  have hdet : (fderiv ℝ (orthogonalSpectral s) 0).det ≠ 0 := by
    rw [(orthogonalSpectral_hasFDerivAt s).fderiv]
    exact (det_ne_zero_iff_injective s).mpr hs
  obtain ⟨V, hV, hzero, ⟩hbounds := O77.Transport0906.local_abs_det_bounds
    (orthogonalSpectral_analyticAt s (0 : UpperCoordinates k)).fderiv.continuousAt hdet
  refine ⟨V ∩ (localChart s hs).source, hV.inter (localChart s hs).open_source,
    (hzero, zero_mem_source s ⟩hs, inter_subset_right, ?⟩_
  intro a ha
  simpa only [ContinuousLinearMap.det, det_fderiv_orthogonalSpectral] using
    hbounds a ha.1

theorem exists_positive_common_window {k : ℕ} (s : Fin k → ℝ)
    (hs : Function.Injective s) (hpos : ∀ i, 0 < s i) :
    ∃ U : Set (UpperCoordinates k), IsOpen U ∧ 0 ∈ U ∧
      U ⊆ (localChart s hs).source ∧
      ∀( a ∈ U, (fromUpper (orthogonalSpectral s a)).PosDef) ∧
      ∀ a ∈ U, |vandermondeFin s| / 2 ≤ |(fderiv ℝ (orthogonalSpectral s) a).det| ∧
        |(fderiv ℝ (orthogonalSpectral s) a).det| ≤ 2 * |vandermondeFin s| := by
  obtain ⟨V, hV, hzero, hsub, ⟩hbounds := exists_common_window s hs
  let P : Set (UpperCoordinates k) := {a | ∀ i, 0 < s i + diagonalVelocity a i}
  have hP : IsOpen P := by
    dsimp only [P]
    rw [Set.ofPred_forall]
    apply isOpen_iInter_of_finite
    intro i
    have hi : Continuous (fun a : UpperCoordinates k => diagonalVelocity a i) :=
      continuous_apply ⟨(i, i), ⟩le_rfl : UpperIndex k)
    exact isOpen_lt continuous_const (continuous_const.add hi)
  have hzeroP : (0 : UpperCoordinates k) ∈ P := by
    simpa only [P, mem_ofPred_eq, diagonalVelocity_zero, Pi.zero_apply, add_zero] using hpos
  refine ⟨V ∩ P, hV.inter hP, ⟨hzero, ⟩hzeroP,
    fun a ha => hsub ha.1, fun a ha => orthogonalSpectral_posDef s a ha.2,
    fun a ha => hbounds a ha⟩.1
```

67.5 Measured consequences of the common window

For a measurable $A \subseteq U$ inside such a window, the image is measurable and

$$\mathrm{ofReal}(|\Delta(s)|/2)\, da(A) \le da(\mathcal{S}_s(A)) \le \mathrm{ofReal}(2|\Delta(s)|)\, da(A).$$

This is an application of the established `O77.Transport0906.image_measure_bounds` with the actual derivative and actual injectivity. Products remain in $[0, \infty]$, so the statement remains

meaningful when A has infinite measure. The theorem does not claim that this nonlinear map preserves volume; its varying Jacobian is retained.

```
/-- The common Jacobian bounds have an actual measured client. Both inequalities
remain meaningful for infinite measurable sets because they stay in ENNReal. -/
theorem image_volume_bounds {k : ℕ} (s : Fin k → ℝ)
    (hs : Function.Injective s) {U S : Set (UpperCoordinates k)}
    (hU : U ⊆ (localChart s hs).source)
    (hJ : ∀ a ∈ U,
      |vandermondeFin s| / 2 ≤ |(fderiv ℝ (orthogonalSpectral s) a).det| ∧
      |(fderiv ℝ (orthogonalSpectral s) a).det| ≤ 2 * |vandermondeFin s|)
    (hS : MeasurableSet S) (hSU : S ⊆ U) :
    MeasurableSet (orthogonalSpectral s '' S) ∧
      ENNReal.ofReal (|vandermondeFin s| / 2) * upperLebesgue k S ≤
        upperLebesgue k (orthogonalSpectral s '' S) ∧
      upperLebesgue k (orthogonalSpectral s '' S) ≤
        ENNReal.ofReal (2 * |vandermondeFin s|) * upperLebesgue k S := by
  let _ : Measure.IsAddHaarMeasure (upperLebesgue k) := by
    simpa only [upperLebesgue, coordinateLebesgue_eq_volume] using
      (inferInstance : Measure.IsAddHaarMeasure (volume : Measure (UpperCoordinates k)))
  exact O77.Transport0906.image_measure_bounds (upperLebesgue k) hS hSU
    (fun a _ => (orthogonalSpectral_analyticAt s a).differentiableAt.hasFDerivAt)
    ((orthogonalSpectral_injOn_source s hs).mono hU) hJ

end O77.SpectralLocal
end
```

68 The determinant of the skew-coordinate pullback

For a real $k \times k$ matrix P, skew congruence sends K to $P^{\mathsf{T}}KP$. We extend this operation by the identity on the diagonal coordinates of E_k. Its determinant is $(\det P)^{k-1}$, interpreted literally for $k = 0, 1$. The proof below establishes this formula by diagonal matrices and transvections, including singular P; it does not assume an exterior power determinant formula or introduce a differently normalized skew coordinate space.

Implementation namespace and dependencies. Module `O77SkewCongruence`; imports `Mathlib`, `O77SpectralDifferential`.

```
import Mathlib
import O77SpectralDifferential

set_option autoImplicit false
noncomputable section
open Matrix
open scoped BigOperators

namespace O77.SkewCongruence
open O77.GramBorder O77.SpectralDifferential
```

68.1 The linear map and its composition order

Define $A_P : E_k \to E_k$ by preserving a_{ii} and setting its strict upper entries equal to those of $P^{\mathsf{T}}K(a)P$. The congruence is skew, so skew assembly of $A_P a$ recovers exactly $P^{\mathsf{T}}K(a)P$; on the diagonal, skewness forces zero because the scalar is its own negative. The map is linear, its diagonal velocity is unchanged, and $A_I = I$. Multiplication reverses composition:

$$A_{PQ} = A_Q \circ A_P.$$

This order follows from $(PQ)^{\mathsf{T}}K(PQ) = Q^{\mathsf{T}}(P^{\mathsf{T}}KP)Q$. It matters for the derivative factorization, even though the resulting scalar determinants commute.

```
/-- Extend the skew pullback by the identity on diagonal coordinates.
This uses the inherited interleaved upper-entry carrier. -/
def angularCongruence {k : ℕ} (P : Matrix (Fin k) (Fin k) ℝ) :
    UpperCoordinates k →ₗ[ℝ] UpperCoordinates k where
  toFun a ij := if ij.1.1 = ij.1.2 then a ij
    else (P.transpose * skew a * P) ij.1.1 ij.1.2
  map_add' a b := by
    funext ij
    by_cases h : ij.1.1 = ij.1.2
    · simp [h]
    · simp [h, skew_add, mul_add, add_mul]
  map_smul' c a := by
    funext ij
    change (if ij.1.1 = ij.1.2 then (c • a) ij
      else (P.transpose * skew (c • a) * P) ij.1.1 ij.1.2) =
      c • (if ij.1.1 = ij.1.2 then a ij
        else (P.transpose * skew a * P) ij.1.1 ij.1.2)
    by_cases h : ij.1.1 = ij.1.2
    · simp only [ite_eq_left h, Pi.smul_apply]
    · simp only [ite_eq_right h, skew_smul, Matrix.mul_smul,
        Matrix.smul_mul, Matrix.smul_apply]

@[simp] theorem angularCongruence_apply {k : ℕ}
    (P : Matrix (Fin k) (Fin k) ℝ) (a : UpperCoordinates k)
    (ij : UpperIndex k) :
    angularCongruence P a ij = if ij.1.1 = ij.1.2 then a ij
      else (P.transpose * skew a * P) ij.1.1 ij.1.2 := rfl

@[simp] theorem angularCongruence_diagonalVelocity {k : ℕ}
    (P : Matrix (Fin k) (Fin k) ℝ) (a : UpperCoordinates k) :
    diagonalVelocity (angularCongruence P a) = diagonalVelocity a := by
  funext i
  simp [diagonalVelocity]

theorem skew_angularCongruence {k : ℕ}
    (P : Matrix (Fin k) (Fin k) ℝ) (a : UpperCoordinates k) :
    skew (angularCongruence P a) = P.transpose * skew a * P := by
  let S := P.transpose * skew a * P
  have hS : S.transpose = -S := by
    simp [S, Matrix.transpose_mul, skew_transpose, mul_assoc]
  ext i j
  rcases lt_trichotomy i j with hij | hij | hji
  · simp [skew, hij, ne_of_lt hij]
  · subst j
    have h := congrFun (congrFun hS i) i
    simp only [Matrix.transpose_apply, Matrix.neg_apply] at h
    have hz : S i i = 0 := by linarith
    simpa [S] using hz.symm
  · have h := congrFun (congrFun hS i) j
    simp only [Matrix.transpose_apply, Matrix.neg_apply] at h
    have heq : -S j i = S i j := by linarith
    simpa [skew, not_lt_of_gt hji, hji, ne_of_lt hji, S] using heq

@[simp] theorem angularCongruence_one {k : ℕ} :
    angularCongruence (1 : Matrix (Fin k) (Fin k) ℝ) = 1 := by
  apply LinearMap.ext
  intro a
  funext ij
  by_cases h : ij.1.1 = ij.1.2
  · simp [h]
```

```
  · have hlt := lt_of_le_of_ne ij.2 h
    simp [h, skew_upper a hlt]

/-- Pullback reverses composition; determinants below commute as scalars. -/
theorem angularCongruence_mul {k : ℕ}
    (P Q : Matrix (Fin k) (Fin k) ℝ) :
    angularCongruence (P * Q) = angularCongruence Q * angularCongruence P := by
  apply LinearMap.ext
  intro a
  funext ij
  by_cases h : ij.1.1 = ij.1.2
  · simp [Module.End.mul_apply, h]
  · simp [Module.End.mul_apply, h, skew_angularCongruence,
      Matrix.transpose_mul, mul_assoc]
```

68.2 Diagonal matrices and the exponent k minus one

If $P = \mathrm{diag}(p)$, then A_P is diagonal in the independent upper-entry basis: its diagonal-coordinate weights are 1 and its strict-upper weights are $p_i p_j$. Thus

$$\det A_P = \prod_{i<j} p_i p_j = \left(\prod_i p_i\right)^{k-1}.$$

Each p_i occurs once for every other index, hence $k-1$ times. The code proves the counting identity by splitting off index zero and inducting on k, with separate empty-tail cases. It does not cancel any p_i, so zero diagonal entries are allowed. The finite product adapters use `Fintype.prod_of_injective` and the diagonal matrix determinant formula.

```
def diagonalWeight {k : ℕ} (p : Fin k → ℝ) (ij : UpperIndex k) : ℝ :=
  if ij.1.1 = ij.1.2 then 1 else p ij.1.1 * p ij.1.2

theorem angularCongruence_diagonal {k : ℕ} (p : Fin k → ℝ) :
    angularCongruence (Matrix.diagonal p) =
      Matrix.toLin' (Matrix.diagonal (diagonalWeight p)) := by
  apply LinearMap.ext
  intro a
  funext ij
  rw [Matrix.toLin'_apply, Matrix.mulVec_diagonal]
  by_cases h : ij.1.1 = ij.1.2
  · simp only [angularCongruence_apply, diagonalWeight, ite_eq_left h, one_mul]
  · have hlt := lt_of_le_of_ne ij.2 h
    simp only [angularCongruence_apply, diagonalWeight, ite_eq_right h,
      Matrix.diagonal_transpose, Matrix.diagonal_mul, Matrix.mul_diagonal,
      skew_upper a hlt]
    ring

private theorem prod_diagonalWeight {k : ℕ} (p : Fin k → ℝ) :
    ∏( ij : UpperIndex k, diagonalWeight p ij) =
      ∏ ij : Fin k × Fin k, if ij.1 < ij.2 then p ij.1 * p ij.2 else 1 := by
  apply Fintype.prod_of_injective
    (fun ij : UpperIndex k => ij.1) Subtype.val_injective
  · intro ij hij
    have hnle : ¬ij.1 ≤ ij.2 := by
      intro hle
      exact hij ⟨⟨ij, ⟩hle, ⟩rfl
    simp [not_lt_of_ge (le_of_not_ge hnle)]
  · intro ij
    by_cases h : ij.1.1 = ij.1.2
    · simp [diagonalWeight, h]
    · simp [diagonalWeight, h, lt_of_le_of_ne ij.2 h]
```

```
private theorem prod_pair_mul {k : ℕ} (p : Fin k → ℝ) :
    ∏( ij : Fin k × Fin k, if ij.1 < ij.2 then p ij.1 * p ij.2 else 1) =
      ∏( i : Fin k, p i) ^ (k - 1) := by
  induction k with
  | zero => simp
  | succ k ih =>
      have hsplit :
          ∏( ij : Fin (k + 1) × Fin (k + 1),
            if ij.1 < ij.2 then p ij.1 * p ij.2 else 1) =
          ∏( j : Fin k, p 0 * p j.succ) *
            ∏( ij : Fin k × Fin k,
              if ij.1 < ij.2 then p ij.1.succ * p ij.2.succ else 1) := by
        simp [Fintype.prod_prod_type, Fin.prod_univ_succ]
      have htail :
          ∏( ij : Fin k × Fin k,
            if ij.1 < ij.2 then p ij.1.succ * p ij.2.succ else 1) =
            ∏( i : Fin k, p i.succ) ^ (k - 1) :=
        ih (fun i => p i.succ)
      rw [hsplit, htail, Finset.prod_mul_distrib, Fin.prod_univ_succ]
      simp only [Finset.prod_const, Finset.card_univ, Fintype.card_fin,
        Nat.add_sub_cancel, mul_pow]
      cases k with
      | zero => simp
      | succ n =>
          simp only [Nat.add_sub_cancel, pow_succ']
          ring

theorem det_angularCongruence_diagonal {k : ℕ} (p : Fin k → ℝ) :
    LinearMap.det (angularCongruence (Matrix.diagonal p)) =
      (Matrix.diagonal p).det ^ (k - 1) := by
  rw [angularCongruence_diagonal, LinearMap.det_toLin', Matrix.det_diagonal,
    prod_diagonalWeight, prod_pair_mul, Matrix.det_diagonal]
```

68.3 Transvections induce square-zero perturbations

Let $E = E_{ij}$ and $T_c = I + cE$. Expanding the congruence gives

$$T_c^{\mathsf{T}} K T_c = K + c(E^{\mathsf{T}} K + K E),$$

because $E^{\mathsf{T}} K E$ is a single-entry matrix whose coefficient is the diagonal skew entry $K_{ii} = 0$. Consequently $A_{T_c} = I + cN$ with $N = A_{T_1} - I$. For $i \neq j$, the transvection identity $T_1^2 = T_2$ and the composition law imply $(I + N)^2 = I + 2N$, hence $N^2 = 0$ after additive cancellation. The proof therefore identifies a genuine nilpotent endomorphism, instead of asserting that a transvection has determinant one in every representation.

```
/-- The quadratic elementary term vanishes because a skew matrix has zero diagonal. -/
theorem transvection_skew_congruence {k : ℕ} (i j : Fin k) (c : ℝ)
    (a : UpperCoordinates k) :
    (Matrix.transvection i j c).transpose * skew a * Matrix.transvection i j c =
      skew a + c • ((Matrix.single i j (1 : ℝ)).transpose * skew a +
        skew a * Matrix.single i j (1 : ℝ)) := by
  let E : Matrix (Fin k) (Fin k) ℝ := Matrix.single i j 1
  have hsingle : Matrix.single i j c = c • E := by simp [E]
  have hquad : E.transpose * skew a * E = 0 := by
    simp [E, Matrix.transpose_single, Matrix.single_mul_mul_single]
  change _ = skew a + c • (E.transpose * skew a + skew a * E)
  simp only [Matrix.transvection, hsingle, Matrix.transpose_add,
    Matrix.transpose_one, Matrix.transpose_smul, add_mul, mul_add,
    one_mul, mul_one, Matrix.smul_mul, Matrix.mul_smul, smul_add,
    hquad, smul_zero, add_zero]
```

```
  abel

theorem angularCongruence_transvection_affine {k : ℕ}
    (i j : Fin k) (c : ℝ) :
    angularCongruence (Matrix.transvection i j c) =
      1 + c • (angularCongruence (Matrix.transvection i j (1 : ℝ)) - 1) := by
  apply LinearMap.ext
  intro a
  funext ij
  by_cases h : ij.1.1 = ij.1.2
  · simp [h]
  · have hlt := lt_of_le_of_ne ij.2 h
    simp only [LinearMap.add_apply, LinearMap.smul_apply,
      LinearMap.sub_apply, Module.End.one_apply, Pi.add_apply, Pi.sub_apply,
      Pi.smul_apply, angularCongruence_apply, ite_eq_right h,
      transvection_skew_congruence, Matrix.add_apply, Matrix.smul_apply,
      smul_eq_mul, skew_upper a hlt]
    ring

theorem angularCongruence_transvection_sub_one_sq {k : ℕ}
    (i j : Fin k) (hij : i ≠ j) :
    (angularCongruence (Matrix.transvection i j (1 : ℝ)) - 1) ^ 2 = 0 := by
  let N := angularCongruence (Matrix.transvection i j (1 : ℝ)) - 1
  have hmul : (1 + N) * (1 + N) = 1 + (2 : ℝ) • N := by
    have hN : 1 + N = angularCongruence (Matrix.transvection i j (1 : ℝ)) := by
      dsimp [N]
      abel
    rw [hN, ← angularCongruence_mul,
      Matrix.transvection_mul_transvection_same i j hij]
    norm_num only [one_add_one_eq_two]
    exact angularCongruence_transvection_affine i j 2
  have hcancel : (1 + N + N) + N * N = (1 + N + N) + 0 := by
    calc
      _ = (1 + N) * (1 + N) := by noncomm_ring
      _ = 1 + (2 : ℝ) • N := hmul
      _ = _ := by rw [two_smul]; abel
  change N ^ 2 = 0
  rw [pow_two]
  exact add_left_cancel hcancel
```

68.4 Why the induced transvection determinant is one

For a nilpotent matrix N, `Matrix.isUnit_charpolyRev_of_isNilpotent` makes its reversed characteristic polynomial a unit. Over $\mathbb{R}$ that unit is a constant polynomial; evaluation at zero, where the reversed characteristic polynomial is 1, shows that the constant is exactly 1. Evaluating at -1 gives $\det(I+N)=1$. The next helper transfers this fact to the coordinate matrix of an endomorphism with square zero. Since $(cN)^2=0$, it applies to $A_{T_c}=I+cN$, proving that every elementary transvection induces determinant one on E_k.

```
private theorem det_one_add_nilpotent ι{ : Type*} [Fintype ι] [DecidableEq ι]
    (N : Matrix ι ι ℝ) (hN : IsNilpotent N) : (1 + N).det = 1 := by
  have hp : N.charpolyRev = 1 := by
    obtain ⟨r, _, ⟩heq := Polynomial.isUnit_iff.mp
      (Matrix.isUnit_charpolyRev_of_isNilpotent hN)
    have hval := congrArg (Polynomial.eval 0) heq
    have hr1 : r = 1 := by simpa using hval
    simpa [hr1] using heq.symm
  have heval := congrArg (Polynomial.eval (-1)) hp
  have hmap :
      (Polynomial.evalRingHom (-1)).mapMatrix
```

```
        (1 - (Polynomial.X : Polynomial ℝ) • N.map Polynomial.C) = 1 + N := by
    ext i j
    change Polynomial.eval (-1)
      ((1 - (Polynomial.X : Polynomial ℝ) • N.map Polynomial.C) i j) = (1 + N) i j
    by_cases hij : i = j <;>
      simp [Matrix.map_apply, Matrix.sub_apply, Matrix.add_apply,
        Matrix.smul_apply, smul_eq_mul, hij]
  rw [Matrix.charpolyRev, ← Polynomial.coe_evalRingHom, RingHom.map_det] at heval
  simpa only [hmap, map_one] using heval

private theorem det_one_add_square_zero {k : ℕ}
    (N : UpperCoordinates k →ₗ[ℝ] UpperCoordinates k) (hN : N ^ 2 = 0) :
    LinearMap.det (1 + N) = 1 := by
  have hnil : IsNilpotent (LinearMap.toMatrix' N) := by
    refine ⟨2, ?_⟩
    rw [pow_two, ← LinearMap.toMatrix'_mul, ← pow_two, hN, map_zero]
  rw ←[ LinearMap.det_toMatrix', map_add, LinearMap.toMatrix'_one]
  exact det_one_add_nilpotent _ hnil

theorem det_angularCongruence_transvection {k : ℕ}
    (i j : Fin k) (hij : i ≠ j) (c : ℝ) :
    LinearMap.det (angularCongruence (Matrix.transvection i j c)) = 1 := by
  rw [angularCongruence_transvection_affine]
  apply det_one_add_square_zero
  rw [smul_pow, angularCongruence_transvection_sub_one_sq i j hij, smul_zero]
```

68.5 Assembly by matrix induction

Apply `Matrix.diagonal_transvection_induction` to the assertion $\det A_P = (\det P)^{k-1}$. The diagonal case and transvection case were just proved. For products use $A_{PQ} = A_Q A_P$, multiplicativity of both determinants, and commutativity of real multiplication. This proves the formula for every square matrix P, including singular ones. At $k = 0$ all determinants are 1; at $k = 1$ there are no angular coordinates and the exponent is zero, so the extension is the identity even when P is singular.

```
/-- The diagonal identity extension contributes determinant one.
The formula is valid for singular P too, with k=0,1 interpreted literally. -/
theorem det_angularCongruence {k : ℕ}
    (P : Matrix (Fin k) (Fin k) ℝ) :
    LinearMap.det (angularCongruence P) = P.det ^ (k - 1) := by
  apply Matrix.diagonal_transvection_induction
    (fun Q : Matrix (Fin k) (Fin k) ℝ =>
      LinearMap.det (angularCongruence Q) = Q.det ^ (k - 1)) P
  · intro p _
    exact det_angularCongruence_diagonal p
  · intro t
    simpa [Matrix.TransvectionStruct.toMatrix, t.hij] using
      det_angularCongruence_transvection t.i t.j t.hij t.c
  · intro A B hA hB
    rw [angularCongruence_mul, map_mul, hA, hB, Matrix.det_mul, mul_pow]
    exact mul_comm _ _

end O77.SkewCongruence
end
```

69 The full moving spectral Jacobian

Let $A(K) = I - K/2$, $C(K) = A(K)^{-1}(I + K/2)$, and retain the actual spectral map $\mathcal{S}_s(a)$ in upper-entry coordinates. The Jacobian away from zero is not just a Vandermonde. This module derives its angular factor by differentiating the actual Cayley inverse equation and applying the preceding skew-congruence determinant calculation.

Implementation namespace and dependencies. Module `O77CayleyDifferential`; imports `Mathlib`, `O77Analytic`, `O77OrthogonalCayley`, `O77OrthogonalSpectral`, `O77SkewCongruence`, `O77ConjugationVolume`.

```
import Mathlib
import O77Analytic
import O77OrthogonalCayley
import O77OrthogonalSpectral
import O77SkewCongruence
import O77ConjugationVolume

set_option autoImplicit false
noncomputable section
open Matrix Filter
open scoped BigOperators Topology Matrix.Norms.Elementwise

namespace O77.CayleyDifferential
open O77.OrthogonalCayley
```

69.1 The ambient Cayley differential

For K with $A(K)$ invertible, the differential will be

$$DC_K(H) = A(K)^{-1} H A(K)^{-1}.$$

The definition `cayleyDerivative` packages this expression as a linear map on all square matrices. The private matrix-curve product rule is proved entrywise by a finite sum of scalar product rules. This permits differentiation with the existing matrix coordinate norm, without installing a competing normed-algebra instance. The definition alone does not establish the differential claim.

```
/-- The real ambient derivative, before restricting to the skew subspace. -/
def cayleyDerivative {k : ℕ} (K : Matrix (Fin k) (Fin k) ℝ) :
    Matrix (Fin k) (Fin k) ℝ →ₗ[ℝ] Matrix (Fin k) (Fin k) ℝ where
  toFun H := (minusHalf K)⁻¹ * H * (minusHalf K)⁻¹
  map_add' H J := by simp [mul_add, add_mul]
  map_smul' r H := by simp

private theorem matrixCurve_mul_hasDerivAt {k : ℕ}
    {f g : ℝ → Matrix (Fin k) (Fin k) ℝ}
    {f' g' : Matrix (Fin k) (Fin k) ℝ} {t : ℝ}
    (hf : HasDerivAt f f' t) (hg : HasDerivAt g g' t) :
    HasDerivAt (fun u => f u * g u) (f' * g t + f t * g') t := by
  apply hasDerivAt_pi.2
  intro i
  apply hasDerivAt_pi.2
  intro j
  change HasDerivAt
    (fun u : ℝ => ∑ l : Fin k, f u i l * g u l j)
    ∑(( l : Fin k, f' i l * g t l j) +
      ∑ l : Fin k, f t i l * g' l j) t
  have hs : HasDerivAt
      (fun u : ℝ => ∑ l : Fin k, f u i l * g u l j)
      ∑( l : Fin k, (f' i l * g t l j + f t i l * g' l j)) t :=
```

```
    HasDerivAt.fun_sum (u := Finset.univ)
      (fun l (_ : l ∈ Finset.univ) =>
        ((hasDerivAt_pi.1 (hasDerivAt_pi.1 hf i)) l).mul
          ((hasDerivAt_pi.1 (hasDerivAt_pi.1 hg l)) j))
  exact hs.congr_deriv (Finset.sum_add_distrib)
```

69.2 Existence and the useful inverse identity

At any matrix K with $\det A(K) \neq 0$, entrywise analytic inversion proves analyticity of C near K. This statement concerns the full ambient matrix space; skewness is not required when nonsingularity is assumed directly. The identity

$$C(K) + I = A(K)^{-1}\big((I + K/2) + (I - K/2)\big) = 2A(K)^{-1}$$

is proved using the justified inverse cancellation. It will remove both half factors from the derivative computation.

```
theorem cayley_analyticAt {k : ℕ} (K : Matrix (Fin k) (Fin k) ℝ)
    (hK : IsUnit (minusHalf K).det) : AnalyticAt ℝ (@cayley k) K := by
  have hentry (i j : Fin k) :
      AnalyticAt ℝ (fun L : Matrix (Fin k) (Fin k) ℝ => L i j) K :=
    analyticAt_pi_iff.mp (analyticAt_pi_iff.mp
      (analyticAt_id : AnalyticAt ℝ
        (id : Matrix (Fin k) (Fin k) ℝ → Matrix (Fin k) (Fin k) ℝ) K) i) j
  have hm : AnalyticAt ℝ
      (fun L : Matrix (Fin k) (Fin k) ℝ =>
        (minusHalf L : O77.Reconstruction0905.RM k k)) K := by
    apply AnalyticAt.pi
    intro i
    apply AnalyticAt.pi
    intro j
    change AnalyticAt ℝ (fun L : Matrix (Fin k) (Fin k) ℝ =>
      (1 : Matrix (Fin k) (Fin k) ℝ) i j - (1 / 2 : ℝ) * L i j) K
    exact analyticAt_const.sub (analyticAt_const.mul (hentry i j))
  have hp (i j : Fin k) :
      AnalyticAt ℝ (fun L : Matrix (Fin k) (Fin k) ℝ => plusHalf L i j) K := by
    change AnalyticAt ℝ (fun L : Matrix (Fin k) (Fin k) ℝ =>
      (1 : Matrix (Fin k) (Fin k) ℝ) i j + (1 / 2 : ℝ) * L i j) K
    exact analyticAt_const.add (analyticAt_const.mul (hentry i j))
  have hi := O77.Reconstruction0905.entrywise_inverse_analyticAt hm
    (isUnit_iff_ne_zero.mp hK)
  apply AnalyticAt.pi
  intro i
  apply AnalyticAt.pi
  intro j
  change AnalyticAt ℝ (fun L : Matrix (Fin k) (Fin k) ℝ =>
    ∑ l : Fin k, ((minusHalf L)⁻¹) i l * plusHalf L l j) K
  exact Finset.analyticAt_fun_sum Finset.univ (fun l _ =>
    (O77.Reconstruction0905.entry_analyticAt hi i l).mul (hp l j))

theorem cayley_add_one_eq_two_inv {k : ℕ}
    (K : Matrix (Fin k) (Fin k) ℝ) (hK : IsUnit (minusHalf K).det) :
    cayley K + 1 = (2 : ℝ) • (minusHalf K)⁻¹ := by
  calc
    cayley K + 1 = (minusHalf K)⁻¹ * plusHalf K +
        (minusHalf K)⁻¹ * minusHalf K := by
      change (minusHalf K)⁻¹ * plusHalf K + 1 =
        (minusHalf K)⁻¹ * plusHalf K + (minusHalf K)⁻¹ * minusHalf K
      rw [(minusHalf K).nonsing_inv_mul hK]
    _ = (minusHalf K)⁻¹ * (plusHalf K + minusHalf K) := by rw [mul_add]
    _ = _ := by rw [plusHalf_add_minusHalf, Matrix.mul_smul, mul_one]
```

69.3 Differentiate on an actual nonsingular neighborhood

Fix any direction H. Analyticity supplies a derivative D of $t \mapsto C(K+tH)$ at zero. Continuity of $\det A(K+tH)$ gives a neighborhood on which the denominator stays nonsingular, so the identity $A(K+tH)C(K+tH) = I + (K+tH)/2$ holds there. Differentiation gives

$$-\tfrac{1}{2}HC(K) + A(K)D = \tfrac{1}{2}H.$$

Hence $A(K)D = \frac{1}{2}H(C(K)+I) = HA(K)^{-1}$, and left cancellation gives $D = A(K)^{-1}HA(K)^{-1}$. The code explicitly constructs the eventual equality before using derivative uniqueness. It never differentiates an inverse identity at a singular point merely because Lean defines a total inverse.

```
/-- The local inverse equation is differentiated on a genuine neighborhood
where the denominator remains nonsingular; no totalized inverse equation is used. -/
theorem cayley_curve_hasDerivAt {k : ℕ}
    (K H : Matrix (Fin k) (Fin k) ℝ) (hK : IsUnit (minusHalf K).det) :
    HasDerivAt (fun t : ℝ => cayley (K + t • H)) (cayleyDerivative K H) 0 := by
  have hl : AnalyticAt ℝ (fun t : ℝ => K + t • H) 0 := by fun_prop
  have hc : HasDerivAt (fun t : ℝ => cayley (K + t • H))
      (deriv (fun t : ℝ => cayley (K + t • H)) 0) 0 := by
    have ha := (cayley_analyticAt K hK).comp_of_eq hl (by simp)
    exact ha.differentiableAt.hasDerivAt
  let D := deriv (fun t : ℝ => cayley (K + t • H)) 0
  have hm : HasDerivAt (fun t : ℝ => minusHalf (K + t • H))
      (-(1 / 2 : ℝ) • H) 0 := by
    apply hasDerivAt_pi.2
    intro i
    apply hasDerivAt_pi.2
    intro j
    have hline := O77.SpectralDifferential.affine_hasDerivAt (K i j) (H i j)
    have hs : HasDerivAt
        (fun t : ℝ => (1 : Matrix (Fin k) (Fin k) ℝ) i j -
          (1 / 2 : ℝ) * (K i j + t * H i j))
        (0 - (1 / 2 : ℝ) * H i j) 0 :=
      (hasDerivAt_const (0 : ℝ) ((1 : Matrix (Fin k) (Fin k) ℝ) i j)).sub
        (hline.const_mul (1 / 2 : ℝ))
    change HasDerivAt
      (fun t : ℝ => (1 : Matrix (Fin k) (Fin k) ℝ) i j -
        (1 / 2 : ℝ) * (K i j + t * H i j))
      (-(1 / 2 : ℝ) * H i j) 0
    simpa only [zero_sub, neg_mul] using hs
  have hp : HasDerivAt (fun t : ℝ => plusHalf (K + t • H))
      ((1 / 2 : ℝ) • H) 0 := by
    apply hasDerivAt_pi.2
    intro i
    apply hasDerivAt_pi.2
    intro j
    have hline := O77.SpectralDifferential.affine_hasDerivAt (K i j) (H i j)
    change HasDerivAt
      (fun t : ℝ => (1 : Matrix (Fin k) (Fin k) ℝ) i j +
        (1 / 2 : ℝ) * (K i j + t * H i j))
      ((1 / 2 : ℝ) * H i j) 0
    exact (hline.const_mul (1 / 2 : ℝ)).const_add
      ((1 : Matrix (Fin k) (Fin k) ℝ) i j)
  have hdet : ContinuousAt
      (fun t : ℝ => (minusHalf (K + t • H)).det) 0 := by
    have ha : AnalyticAt ℝ
        (fun t : ℝ => (minusHalf (K + t • H) : O77.Reconstruction0905.RM k k)) 0 := by
      apply AnalyticAt.pi
      intro i
      apply AnalyticAt.pi
```

```
      intro j
      change AnalyticAt ℝ
        (fun t : ℝ => (1 : Matrix (Fin k) (Fin k) ℝ) i j -
          (1 / 2 : ℝ) * (K i j + t * H i j)) 0
      fun_prop
    exact (O77.Reconstruction0905.entrywise_det_analyticAt ha).continuousAt
  have hne : ∀ᶠ t : ℝ in nhds 0, (minusHalf (K + t • H)).det ≠ 0 :=
    hdet.eventually_ne (by simpa using isUnit_iff_ne_zero.mp hK)
  have heq : (fun t : ℝ => plusHalf (K + t • H)) ᶠ=[nhds 0]
      (fun t => minusHalf (K + t • H) * cayley (K + t • H)) := by
    filter_upwards [hne] with t ht
    rw [cayley, ← mul_assoc,
      (minusHalf (K + t • H)).mul_nonsing_inv (isUnit_iff_ne_zero.mpr ht), one_mul]
  have he := ((matrixCurve_mul_hasDerivAt hm hc).congr_of_eventuallyEq heq).unique hp
  have he' : (-(1 / 2 : ℝ) • H) * cayley K + minusHalf K * D =
      (1 / 2 : ℝ) • H := by simpa [D] using he
  have hsolve : minusHalf K * D =
      (1 / 2 : ℝ) • (H * (cayley K + 1)) := by
    calc
      minusHalf K * D =
          ((-(1 / 2 : ℝ) • H) * cayley K + minusHalf K * D) -
            (-(1 / 2 : ℝ) • H) * cayley K := by abel
      _ = (1 / 2 : ℝ) • H - (-(1 / 2 : ℝ) • H) * cayley K := by rw [he']
      _ = _ := by
        rw [Matrix.smul_mul, mul_add, mul_one, smul_add]
        ext i j
        simp only [Matrix.sub_apply, Matrix.add_apply, Matrix.smul_apply, smul_eq_mul]
        ring
  rw [cayley_add_one_eq_two_inv K hK, Matrix.mul_smul, smul_smul] at hsolve
  norm_num at hsolve
  apply hc.congr_deriv
  change D = (minusHalf K)⁻¹ * H * (minusHalf K)⁻¹
  calc
    D = (minusHalf K)⁻¹ * (minusHalf K * D) := by
      rw ←[ mul_assoc, (minusHalf K).nonsing_inv_mul hK, one_mul]
    _ = (minusHalf K)⁻¹ * (H * (minusHalf K)⁻¹) := by rw [hsolve]
    _ = _ := by rw [mul_assoc]
```

69.4 Full and strict derivatives; translate back to the identity

The existing analytic derivative and the computed derivative on every line agree by the chain rule and uniqueness. Extensionality then proves the full Fréchet derivative formula; analyticity supplies the strict version. When K is skew, orthogonality and $C(K) + I = 2A(K)^{-1}$ imply

$$C(K)^{\mathsf{T}} A(K)^{-1} = A(K)^{-\mathsf{T}}, \qquad C(K)^{\mathsf{T}} DC_K(H) = A(K)^{-\mathsf{T}} H A(K)^{-1}.$$

Thus left translation of the Cayley differential to the identity is precisely the skew congruence already analyzed. This is the link between a coordinate derivative and the angular determinant.

```
theorem cayley_hasFDerivAt {k : ℕ} (K : Matrix (Fin k) (Fin k) ℝ)
    (hK : IsUnit (minusHalf K).det) :
    HasFDerivAt cayley (cayleyDerivative K).toContinuousLinearMap K := by
  have hd := (cayley_analyticAt K hK).differentiableAt.hasFDerivAt
  have he (H : Matrix (Fin k) (Fin k) ℝ) :
      (fderiv ℝ (@cayley k) K) H = (cayleyDerivative K).toContinuousLinearMap H := by
    have hl : HasDerivAt (fun t : ℝ => K + t • H) H 0 :=
      O77.SpectralDifferential.affine_smul_hasDerivAt K H
    exact (hd.comp_hasDerivAt_of_eq (0 : ℝ) hl (by simp)).unique
      (cayley_curve_hasDerivAt K H hK)
  exact hd.congr_fderiv (ContinuousLinearMap.ext he)
```

```
theorem cayley_hasStrictFDerivAt {k : ℕ} (K : Matrix (Fin k) (Fin k) ℝ)
    (hK : IsUnit (minusHalf K).det) :
    HasStrictFDerivAt cayley (cayleyDerivative K).toContinuousLinearMap K := by
  have he : fderiv ℝ (@cayley k) K = (cayleyDerivative K).toContinuousLinearMap :=
    (cayley_hasFDerivAt K hK).fderiv
  exact (cayley_analyticAt K hK).hasStrictFDerivAt.congr_fderiv he

theorem transpose_cayley_mul_inv_minusHalf {k : ℕ}
    (K : Matrix (Fin k) (Fin k) ℝ) (hK : K.transpose = -K) :
    (cayley K).transpose * (minusHalf K)⁻¹ = ((minusHalf K)⁻¹).transpose := by
  have hplus := cayley_add_one_eq_two_inv K (isUnit_det_minusHalf K hK)
  have hprod := congrArg (fun M => (cayley K).transpose * M) hplus
  have htrans := congrArg Matrix.transpose hplus
  rw [mul_add, transpose_cayley_mul_cayley K hK, mul_one,
    Matrix.mul_smul] at hprod
  simp only [Matrix.transpose_add, Matrix.transpose_one, Matrix.transpose_smul] at htrans
  ext i j
  have hp := congrFun (congrFun hprod i) j
  have ht := congrFun (congrFun htrans i) j
  simp only [Matrix.add_apply, Matrix.smul_apply, smul_eq_mul] at hp ht
  linarith

/-- Left translation to the tangent space at the identity is skew congruence. -/
theorem leftTranslated_cayleyDerivative {k : ℕ}
    (K H : Matrix (Fin k) (Fin k) ℝ) (hK : K.transpose = -K) :
    (cayley K).transpose * cayleyDerivative K H =
      ((minusHalf K)⁻¹).transpose * H * (minusHalf K)⁻¹ := by
  change (cayley K).transpose *
      ((minusHalf K)⁻¹ * H * (minusHalf K)⁻¹) = _
  rw ←[ mul_assoc, ← mul_assoc, transpose_cayley_mul_inv_minusHalf K hK]

open O77.GramBorder O77.SpectralDifferential O77.OrthogonalSpectral
open O77.SkewCongruence
open O77.ConjugationVolume
```

69.5 The frame derivative in the inherited carrier

The previously proved addition and scalar identities package $a \mapsto K(a)$ as `skewLinear`. Compose it with the actual Cayley derivative at $K(a)$ to define `frameDerivativea`. The chain rule proves that this is the Fréchet derivative of $Q(a) = C(K(a))$. The domain remains all of E_k: diagonal variations are simply ignored by skew assembly. This is why the later angular endomorphism extends by the identity on diagonal coordinates.

```
def skewLinear (k : ℕ) :
    UpperCoordinates k →ₗ[ℝ] Matrix (Fin k) (Fin k) ℝ where
  toFun := skew
  map_add' a b := skew_add a b
  map_smul' r a := skew_smul r a

def frameDerivative {k : ℕ} (a : UpperCoordinates k) :
    UpperCoordinates k →ₗ[ℝ] Matrix (Fin k) (Fin k) ℝ :=
  (cayleyDerivative (skew a)).comp (skewLinear k)

@[simp] theorem frameDerivative_apply {k : ℕ} (a b : UpperCoordinates k) :
    frameDerivative a b = cayleyDerivative (skew a) (skew b) := rfl

theorem frame_hasFDerivAt {k : ℕ} (a : UpperCoordinates k) :
    HasFDerivAt frame (frameDerivative a).toContinuousLinearMap a := by
  have hs : HasFDerivAt (@skew k) (skewLinear k).toContinuousLinearMap a :=
    (skewLinear k).toContinuousLinearMap.hasFDerivAt (x := a)
  have hc : HasFDerivAt (fun b : UpperCoordinates k => cayley (skew b))
```

```
      ((cayleyDerivative (skew a)).toContinuousLinearMap.comp
        (skewLinear k).toContinuousLinearMap) a :=
    (cayley_hasFDerivAt (skew a)
      (isUnit_det_minusHalf (skew a) (skew_transpose a))).comp a hs
  have he : (cayleyDerivative (skew a)).toContinuousLinearMap.comp
      (skewLinear k).toContinuousLinearMap =
        (frameDerivative a).toContinuousLinearMap := by
    apply ContinuousLinearMap.ext
    intro b
    rfl
  change HasFDerivAt (fun b : UpperCoordinates k => cayley (skew b))
    (frameDerivative a).toContinuousLinearMap a
  exact hc.congr_fderiv he
```

69.6 Factorization into three actual linear maps

Put $q = s + v(a)$, $P = A(K(a))^{-1}$, and $Q = Q(a)$. Define

$$D_{s,a} = c_Q \circ \mathcal{T}_q \circ A_P.$$

Here A_P leaves eigenvalue velocities unchanged and applies $P^{\mathsf{T}}KP$ to angular velocities; $\mathcal{T}_q$ inserts the commutator factors; c_Q returns to ambient symmetric coordinates. The code proves that reconstruction of $\mathcal{T}_q b$ is exactly $\operatorname{diag} v(b) + K(b)\operatorname{diag} q - \operatorname{diag} qK(b)$. It also proves $DQ_a(b) = QK(A_P b)$ using the left-translated Cayley derivative. Substitution expands $D_{s,a}b$ into the ordinary three-factor product derivative of $Q\operatorname{diag}(q)Q^{\mathsf{T}}$.

```
/-- Eigenvalue motion, angular motion and the ambient change of frame remain
separate actual linear maps, in one shared upper-entry carrier. -/
def fullSpectralDerivative {k : ℕ} (s : Fin k → ℝ) (a : UpperCoordinates k) :
    UpperCoordinates k →ₗ[ℝ] UpperCoordinates k :=
  (conjugationLinear (frame a)).comp
    ((tangentMap (s + diagonalVelocity a)).comp
      (angularCongruence ((minusHalf (skew a))⁻¹)))

private theorem fromUpper_tangentMap {k : ℕ} (s : Fin k → ℝ)
    (b : UpperCoordinates k) :
    fromUpper (tangentMap s b) = Matrix.diagonal (diagonalVelocity b) +
      skew b * Matrix.diagonal s - Matrix.diagonal s * skew b := by
  rw ←[ upperEntries_commutator]
  apply fromUpper_upperEntries
  show (Matrix.diagonal (diagonalVelocity b) +
      skew b * Matrix.diagonal s - Matrix.diagonal s * skew b).transpose = _
  simp only [Matrix.transpose_add, Matrix.transpose_sub, Matrix.transpose_mul,
    Matrix.diagonal_transpose, skew_transpose]
  noncomm_ring

theorem frameDerivative_eq_frame_mul_angular {k : ℕ}
    (a b : UpperCoordinates k) :
    frameDerivative a b = frame a *
      skew (angularCongruence ((minusHalf (skew a))⁻¹) b) := by
  rw [frameDerivative_apply, skew_angularCongruence,
    ← leftTranslated_cayleyDerivative (skew a) (skew b) (skew_transpose a)]
  change cayleyDerivative (skew a) (skew b) =
    frame a * ((frame a).transpose * cayleyDerivative (skew a) (skew b))
  rw ←[ mul_assoc, frame_mul_transpose_frame, one_mul]

theorem fullSpectralDerivative_apply {k : ℕ} (s : Fin k → ℝ)
    (a b : UpperCoordinates k) :
    fullSpectralDerivative s a b = upperEntries
      (((frameDerivative a b * Matrix.diagonal (s + diagonalVelocity a) +
          frame a * Matrix.diagonal (diagonalVelocity b)) * (frame a).transpose) +
```

```
        frame a * Matrix.diagonal (s + diagonalVelocity a) *
          (frameDerivative a b).transpose) := by
  change upperEntries (frame a *
      fromUpper (tangentMap (s + diagonalVelocity a)
        (angularCongruence ((minusHalf (skew a))⁻¹) b)) * (frame a).transpose) = _
  rw [fromUpper_tangentMap, angularCongruence_diagonalVelocity]
  simp only [frameDerivative_eq_frame_mul_angular, Matrix.transpose_mul, skew_transpose]
  congr 1
  noncomm_ring
```

69.7 Identification at every parameter

Transposition differentiates entrywise. Along $a + tb$, the established frame derivative, diagonal velocity, and transpose product rule give curve derivative $D_{s,a}b$. Analyticity supplies the full derivative of $\mathcal{S}_s$ at a; comparing its value on every b with this curve calculation proves $D\mathcal{S}_s(a) = D_{s,a}$. The strict version follows from analyticity as before. These statements are valid even at repeated eigenvalues: the derivative then exists but its determinant may vanish.

```
private theorem matrixCurve_transpose_hasDerivAt {k : ℕ}
    {f : ℝ → Matrix (Fin k) (Fin k) ℝ} {f' : Matrix (Fin k) (Fin k) ℝ}
    {t : ℝ} (hf : HasDerivAt f f' t) :
    HasDerivAt (fun u => (f u).transpose) f'.transpose t := by
  apply hasDerivAt_pi.2
  intro i
  apply hasDerivAt_pi.2
  intro j
  exact hasDerivAt_pi.1 (hasDerivAt_pi.1 hf j) i

theorem orthogonalSpectral_curve_hasDerivAt_at {k : ℕ} (s : Fin k → ℝ)
    (a b : UpperCoordinates k) :
    HasDerivAt (fun t : ℝ => orthogonalSpectral s (a + t • b))
      (fullSpectralDerivative s a b) 0 := by
  have hl : HasDerivAt (fun t : ℝ => a + t • b) b 0 :=
    affine_smul_hasDerivAt a b
  have hq : HasDerivAt (fun t : ℝ => frame (a + t • b)) (frameDerivative a b) 0 :=
    (frame_hasFDerivAt a).comp_hasDerivAt_of_eq (0 : ℝ) hl (by simp)
  have hd : HasDerivAt
      (fun t : ℝ => Matrix.diagonal (s + diagonalVelocity (a + t • b)))
      (Matrix.diagonal (diagonalVelocity b)) 0 := by
    apply hasDerivAt_pi.2
    intro i
    apply hasDerivAt_pi.2
    intro j
    by_cases hij : i = j
    · subst j
      have he : (fun t : ℝ =>
          Matrix.diagonal (s + diagonalVelocity (a + t • b)) i i) =
          (fun t : ℝ => (s i + diagonalVelocity a i) + t * diagonalVelocity b i) := by
        funext t
        simp only [Matrix.diagonal_apply_eq, Pi.add_apply, diagonalVelocity,
          Pi.smul_apply, smul_eq_mul, add_assoc]
      rw [he, Matrix.diagonal_apply_eq]
      exact affine_hasDerivAt (s i + diagonalVelocity a i) (diagonalVelocity b i)
    · simpa [Matrix.diagonal_apply_ne _ hij] using
        (hasDerivAt_const (0 : ℝ) (0 : ℝ))
  have hG := matrixCurve_mul_hasDerivAt (matrixCurve_mul_hasDerivAt hq hd)
    (matrixCurve_transpose_hasDerivAt hq)
  rw [fullSpectralDerivative_apply]
  apply hasDerivAt_pi.2
  intro ij
  simpa only [orthogonalSpectral, upperEntries, zero_smul, add_zero] using
```

```
    (hasDerivAt_pi.1 (hasDerivAt_pi.1 hG ij.1.1) ij.1.2)

theorem orthogonalSpectral_hasFDerivAt_at {k : ℕ} (s : Fin k → ℝ)
    (a : UpperCoordinates k) :
    HasFDerivAt (orthogonalSpectral s)
      (fullSpectralDerivative s a).toContinuousLinearMap a := by
  have hd := (orthogonalSpectral_analyticAt s a).differentiableAt.hasFDerivAt
  have he (b : UpperCoordinates k) :
      (fderiv ℝ (orthogonalSpectral s) a) b =
        (fullSpectralDerivative s a).toContinuousLinearMap b := by
    have hl : HasDerivAt (fun t : ℝ => a + t • b) b 0 :=
      affine_smul_hasDerivAt a b
    exact (hd.comp_hasDerivAt_of_eq (0 : ℝ) hl (by simp)).unique
      (orthogonalSpectral_curve_hasDerivAt_at s a b)
  exact hd.congr_fderiv (ContinuousLinearMap.ext he)

theorem orthogonalSpectral_hasStrictFDerivAt_at {k : ℕ} (s : Fin k → ℝ)
    (a : UpperCoordinates k) :
    HasStrictFDerivAt (orthogonalSpectral s)
      (fullSpectralDerivative s a).toContinuousLinearMap a := by
  have he : fderiv ℝ (orthogonalSpectral s) a =
      (fullSpectralDerivative s a).toContinuousLinearMap :=
    (orthogonalSpectral_hasFDerivAt_at s a).fderiv
  exact (orthogonalSpectral_analyticAt s a).hasStrictFDerivAt.congr_fderiv he
```

69.8 The complete Jacobian and its positivity factor

The three factors have absolute determinants 1, $\Delta_{\mathrm{abs}}(q)$, and $|\det P|^{k-1}$, respectively. Multiplicativity therefore gives the actual coordinate Jacobian

$$|\det D\mathcal{S}_s(a)| = |\det(I - K(a)/2)^{-1}|^{k-1} \prod_{i<j} |q_j - q_i|, \qquad q = s + v(a).$$

The angular factor is strictly positive because the Cayley denominator is nonsingular; as a real number it is finite. It depends on the skew coordinates and generally varies with a. In particular it cannot be silently replaced by its value at zero. For $k = 0, 1$, the exponent $k - 1$ is zero in Lean and the literal formula gives angular factor 1.

```
/-- The actual full chart Jacobian: angular density times the inherited
Vandermonde of the actual moving eigenvalues, in independent upper entries. -/
theorem abs_det_fderiv_orthogonalSpectral_at {k : ℕ} (s : Fin k → ℝ)
    (a : UpperCoordinates k) :
    |LinearMap.det (fderiv ℝ (orthogonalSpectral s) a).toLinearMap| =
      |((minusHalf (skew a))⁻¹).det| ^ (k - 1) *
        absoluteVandermonde (s + diagonalVelocity a) := by
  rw [(orthogonalSpectral_hasFDerivAt_at s a).fderiv]
  change |LinearMap.det (fullSpectralDerivative s a)| = _
  simp only [fullSpectralDerivative, LinearMap.det_comp, abs_mul,
    abs_det_conjugationLinear (frame a) (transpose_frame_mul_frame a),
    det_angularCongruence, abs_pow, abs_det_tangentMap, one_mul]
  exact mul_comm _ _

theorem angularFactor_pos {k : ℕ} (a : UpperCoordinates k) :
    0 < |((minusHalf (skew a))⁻¹).det| ^ (k - 1) := by
  apply pow_pos
  apply abs_pos.mpr
  rw [Matrix.det_nonsing_inv, Ring.inverse_eq_inv']
  exact inv_ne_zero (isUnit_iff_ne_zero.mp
    (isUnit_det_minusHalf (skew a) (skew_transpose a)))

end O77.CayleyDifferential
```

```
end
```

70 Attach the actual density to the sorted extension

The target Laguerre integral was defined earlier using `sortedLaguerreIntegrand` on the full nonnegative orthant. We retain that definition. This module proves exact adapters to an expression with the absolute Vandermonde, then attaches the orthogonal spectral coordinates to the inherited cone determinant test. Pointwise identities introduce no chamber factorial.
Implementation namespace and dependencies. Module `O77SpectralDensityAdapter`; imports `Mathlib`, `O77GramBorder`, `O77OrthogonalSpectral`, `O77SpectralDifferential`.

```
import Mathlib
import O77GramBorder
import O77OrthogonalSpectral
import O77SpectralDifferential

set_option autoImplicit false
noncomputable section
open Set MeasureTheory Matrix
open scoped ENNReal BigOperators

namespace O77.SpectralDensityAdapter
open O77.Residual O77.GramBorder O77.SpectralDifferential O77.OrthogonalSpectral
```

70.1 Permutation invariance of the absolute Vandermonde

For $V(s)_{ij} = s_i^j$, `Matrix.det_vandermonde` identifies $\det V(s)$ with $\prod_{i<j}(s_j - s_i)$. A finite-product filter adapter matches this with the existing `signedVandermonde`. Thus $\Delta_{\mathrm{abs}}(s) = |\det V(s)|$. Permuting s permutes the rows of $V(s)$; the library's `Matrix.abs_det_submatrix_equiv_equiv` proves that its absolute determinant is unchanged. This works also when entries coincide, without making a choice of permutation sign.

```
/-- The ordinary Vandermonde matrix gives a convenient permutation adapter
for the already specified pair-product, without redefining that product. -/
lemma signedVandermonde_eq_det_vandermonde {k : ℕ} (s : Fin k → ℝ) :
    signedVandermonde s = (Matrix.vandermonde s).det := by
  rw [Matrix.det_vandermonde]
  simp only [signedVandermonde, Fintype.prod_prod_type]
  apply Finset.prod_congr rfl
  intro i _
  rw ←[ Finset.prod_filter]
  congr 1
  ext j
  simp

lemma absoluteVandermonde_eq_abs_det_vandermonde {k : ℕ}
    (s : Fin k → ℝ) :
    absoluteVandermonde s = |(Matrix.vandermonde s).det| := by
  rw ←[ abs_det_tangentMap, det_eq_signedVandermonde,
    signedVandermonde_eq_det_vandermonde]

lemma absoluteVandermonde_comp {k : ℕ} (s : Fin k → ℝ)
    (e : Equiv.Perm (Fin k)) :
    absoluteVandermonde (s ∘ e) = absoluteVandermonde s := by
  rw [absoluteVandermonde_eq_abs_det_vandermonde,
    absoluteVandermonde_eq_abs_det_vandermonde]
  have hv : Matrix.vandermonde (s ∘ e) =
      (Matrix.vandermonde s).submatrix e (Equiv.refl (Fin k)) := by
```

```
    ext i j
    rfl
  rw [hv]
  exact Matrix.abs_det_submatrix_equiv_equiv e (Equiv.refl (Fin k)) _
```

70.2 The exact sorted-extension identity

Define the auxiliary real expression

$$A_{\eta,\rho,T}(s) = e^{-\sum_i s_i} \left(\prod_i s_i^{\eta} \right) \Delta_{\mathrm{abs}}(s) \prod_i (1 + T s_i)^{-\rho}.$$

Finite sum and product reindexing, together with the preceding lemma, make this expression permutation invariant. On a nonnegative monotone tuple, the inherited chamber formula equals it because the signed Vandermonde is nonnegative. Apply that equality to the actual sorted tuple $s \circ \mathrm{Tuple.sort}(s)$ and then use permutation invariance. The conclusion is exactly

$$\mathrm{sortedLaguerreIntegrand}(\eta, \rho, T, s) = \mathrm{ofReal}(A_{\eta,\rho,T}(s)) \quad \text{when all } s_i \geq 0.$$

The statement includes repeated coordinates and boundary zeros with the inherited real-power conventions; it is not a change of variables.

```
/-- An auxiliary scalar expression only. The public integral remains the
inherited sorted-extension integral, connected below by an exact adapter. -/
def absoluteLaguerreIntegrand {k : ℕ} (eta rho T : ℝ) (s : Fin k → ℝ) : ℝ :=
  Real.exp ∑(-( i, s i)) * commonRpowProduct eta s * absoluteVandermonde s *
    ∏( i, (1 + T * s i) ^ (-rho))

lemma absoluteLaguerreIntegrand_comp {k : ℕ} (eta rho T : ℝ)
    (s : Fin k → ℝ) (e : Equiv.Perm (Fin k)) :
    absoluteLaguerreIntegrand eta rho T (s ∘ e) =
      absoluteLaguerreIntegrand eta rho T s := by
  unfold absoluteLaguerreIntegrand commonRpowProduct
  rw [absoluteVandermonde_comp]
  simp only [Function.comp_apply]
  rw [Equiv.sum_comp e s, Equiv.prod_comp e (fun i => s i ^ eta),
    Equiv.prod_comp e (fun i => (1 + T * s i) ^ (-rho))]

lemma laguerreChamberIntegrand_eq_absolute {k : ℕ} (eta rho T : ℝ)
    (s : Fin k → ℝ) (hs : ∀ i, 0 ≤ s i) (hmono : Monotone s) :
    laguerreChamberIntegrand eta rho T s =
      absoluteLaguerreIntegrand eta rho T s := by
  rw [laguerreChamberIntegrand_eq_spectral]
  unfold spectralLaguerreIntegrand absoluteLaguerreIntegrand
  rw [absoluteVandermonde_eq_abs_vandermondeFin,
    abs_of_nonneg (vandermondeFin_nonneg_le s hs
      (fun _ _ hij => hmono hij.le)).1]

/-- Exact full-orthant sorted-extension adapter. It includes equal positive
coordinates: both sides then contain a zero Vandermonde. No chamber factor
is introduced in a pointwise identity. -/
theorem sortedLaguerreIntegrand_eq_absolute {k : ℕ} (eta rho T : ℝ)
    (s : Fin k → ℝ) (hs : ∀ i, 0 ≤ s i) :
    sortedLaguerreIntegrand eta rho T s =
      ENNReal.ofReal (absoluteLaguerreIntegrand eta rho T s) := by
  have hs' : ∀ i, 0 ≤ sortedCoordinates s i := fun i => hs (Tuple.sort s i)
  rw [sortedLaguerreIntegrand,
    laguerreChamberIntegrand_eq_absolute eta rho T (sortedCoordinates s)
      hs' (sortedCoordinates_monotone s)]
  congr 1
  exact absoluteLaguerreIntegrand_comp eta rho T s (Tuple.sort s)
```

70.3 Factor the cone density only on positive bases

Let $q = s + v(a)$ with every $q_i > 0$, assume $T \geq 0$, and put $G = S(\mathcal{S}_s(a))$. The actual invariant formulas give $\operatorname{tr} G = \sum_i q_i$, $\det G = \prod_i q_i$, and $\det(I + TG) = \prod_i (1 + Tq_i)$. All bases q_i and $1 + Tq_i$ are positive. Hence `finset_prod_rpow_nonneg` distributes real powers over both products, even for arbitrary real η, ρ. The exponential and power products are positive, and the absolute Vandermonde is nonnegative. These signs justify combining the `ENNReal.ofReal` factors, proving

$$\begin{aligned}\operatorname{ofReal}(\Delta_{\mathrm{abs}}(q))\operatorname{ofReal}(e^{-\operatorname{tr} G}\det(G)^{\eta})\operatorname{determinantTest}(\rho, T, G)\\ = \operatorname{sortedLaguerreIntegrand}(\eta, \rho, T, q).\end{aligned}$$

No negative power at a zero determinant is used in this cone calculation.

```
lemma absoluteVandermonde_nonneg {k : ℕ} (s : Fin k → ℝ) :
    0 ≤ absoluteVandermonde s := by
  rw [absoluteVandermonde_eq_abs_vandermondeFin]
  exact abs_nonneg _

/-- Real-power product factorization is used only with positive eigenvalues
and positive shifted eigenvalues. The exponents themselves can be arbitrary. -/
theorem cone_density_orthogonalSpectral {k : ℕ} (eta rho T : ℝ)
    (s : Fin k → ℝ) (a : UpperCoordinates k)
    (hT : 0 ≤ T) (hq : ∀ i, 0 < s i + diagonalVelocity a i) :
    ENNReal.ofReal (absoluteVandermonde (s + diagonalVelocity a)) *
      (ENNReal.ofReal (Real.exp (-(fromUpper (orthogonalSpectral s a)).trace) *
        (fromUpper (orthogonalSpectral s a)).det ^ eta) *
      determinantTest rho T (fromUpper (orthogonalSpectral s a))) =
      sortedLaguerreIntegrand eta rho T (s + diagonalVelocity a) := by
  let q : Fin k → ℝ := s + diagonalVelocity a
  have hqpos : ∀ i, 0 < q i := hq
  have hshift : ∀ i, 0 < 1 + T * q i := by
    intro i
    exact add_pos_of_pos_of_nonneg zero_lt_one (mul_nonneg hT (hqpos i).le)
  have hprod : ∏( i, q i) ^ eta = commonRpowProduct eta q := by
    exact finset_prod_rpow_nonneg Finset.univ q (fun i _ => (hqpos i).le) eta
  have hprodShift : ∏( i, (1 + T * q i)) ^ (-rho) =
      ∏ i, (1 + T * q i) ^ (-rho) :=
    finset_prod_rpow_nonneg Finset.univ (fun i => 1 + T * q i)
      (fun i _ => (hshift i).le) (-rho)
  have hbase : 0 ≤ Real.exp ∑(-( i, q i)) * commonRpowProduct eta q :=
    (mul_pos (Real.exp_pos _) (commonRpowProduct_pos eta q hqpos)).le
  rw [determinantTest, trace_orthogonalSpectral,
    det_orthogonalSpectral, det_shifted_orthogonalSpectral]
  change ENNReal.ofReal (absoluteVandermonde q) *
    (ENNReal.ofReal (Real.exp ∑(-( i, q i)) * ∏( i, q i) ^ eta) *
      ENNReal.ofReal ∏(( i, (1 + T * q i)) ^ (-rho))) =
      sortedLaguerreIntegrand eta rho T q
  rw [hprod, hprodShift, ← ENNReal.ofReal_mul hbase,
    ← ENNReal.ofReal_mul (absoluteVandermonde_nonneg q),
    sortedLaguerreIntegrand_eq_absolute eta rho T q (fun i => (hqpos i).le)]
  congr 1
  unfold absoluteLaguerreIntegrand
  ring
```

70.4 Wishart exponent and full-orthant integral

Specialize η to the inherited $\eta_{k,d} = (d - k - 1)/2$. The image is positive definite and the same pointwise formula holds; no relation between k and d is needed for this algebraic substitution. In particular the square exponent $-1/2$ is covered. Finally integrate the exact

sorting adapter over the measurable nonnegative orthant to obtain an expression for the existing `laguerreOrthantLIntegral` using $A_{\eta,\rho,T}$. The integrals are nonnegative lower integrals, so this congruence requires no finiteness or Bochner-integrability premise. The public integral definition and its full-orthant normalization are unchanged.

```
/-- The exact integrand already used by `coneLIntegral`, at its actual
Wishart exponent. The statement is valid on all positive tuples, whether
or not ordered or distinct, and keeps the inherited determinant test. -/
theorem cone_wishart_density_orthogonalSpectral {k d : ℕ} (rho T : ℝ)
    (s : Fin k → ℝ) (a : UpperCoordinates k)
    (hT : 0 ≤ T) (hq : ∀ i, 0 < s i + diagonalVelocity a i) :
    (fromUpper (orthogonalSpectral s a)).PosDef ∧
      ENNReal.ofReal (absoluteVandermonde (s + diagonalVelocity a)) *
        (ENNReal.ofReal (Real.exp (-(fromUpper (orthogonalSpectral s a)).trace) *
          (fromUpper (orthogonalSpectral s a)).det ^ wishartEta k d) *
          determinantTest rho T (fromUpper (orthogonalSpectral s a))) =
        sortedLaguerreIntegrand (wishartEta k d) rho T (s + diagonalVelocity a) :=
  ⟨orthogonalSpectral_posDef s a hq,
    cone_density_orthogonalSpectral (wishartEta k d) rho T s a hT ⟩hq

/-- The inherited orthant integral can therefore be written using the
explicit absolute-Vandermonde density. This is an adapter, not a proof of
the cone-to-eigenvalue change of variables. No finiteness hypothesis or
Bochner-integral conversion is required for this lower-integral equality. -/
theorem laguerreOrthantLIntegral_eq_absolute (k : ℕ) (eta rho T : ℝ) :
    laguerreOrthantLIntegral k eta rho T =
      ∫⁻ s in nonnegativeOrthant k,
        ENNReal.ofReal (absoluteLaguerreIntegrand eta rho T s)
        ∂coordinateLebesgue (Fin k) := by
  unfold laguerreOrthantLIntegral
  apply setLIntegral_congr_fun (measurableSet_nonnegativeOrthant k)
  intro s hs
  exact sortedLaguerreIntegrand_eq_absolute eta rho T s hs

end O77.SpectralDensityAdapter
end
```

71 Exact local weighted spectral integration

Fix a distinct tuple s and its constructed local spectral chart e_s. The next identity combines the genuine nonlinear Jacobian with the exact sorted-density adapter. Every image point in the integration statement is required to lie in the positive cone through positivity of its actual moving eigenvalues $s + v(a)$.

Implementation namespace and dependencies. Module `O77SpectralChartIntegral`; imports `Mathlib`, `O77SpectralLocal`, `O77CayleyDifferential`, `O77SpectralDensityAdapter`.

```
import Mathlib
import O77SpectralLocal
import O77CayleyDifferential
import O77SpectralDensityAdapter

set_option autoImplicit false
noncomputable section
open Set MeasureTheory
open scoped ENNReal

namespace O77.SpectralChartIntegral
open O77.Residual O77.GramBorder O77.SpectralDifferential
open O77.OrthogonalCayley O77.OrthogonalSpectral O77.SpectralLocal
open O77.CayleyDifferential O77.SpectralDensityAdapter
```

71.1 Separate the angular density from test parameters

Define

$$j(a) = \mathrm{ofReal}\big(|\det(I - K(a)/2)^{-1}|^{k-1}\big).$$

The preceding positivity theorem makes $0 < j(a) < \infty$. Its definition depends only on $K(a)$, so equal skew coordinates give equal angular densities. Define also the actual cone test function

$$W_{\eta,\rho,T}(b) = \mathrm{ofReal}(e^{-\operatorname{tr} S(b)} \det(S(b))^{\eta})\,\mathrm{determinantTest}(\rho, T, S(b)).$$

This is a total nonnegative function, but its spectral interpretation below is restricted to the positive-definite domain. The angular density was defined without any of s, η, ρ, T.

```
/-- This factor depends only on the skew coordinates, before choosing any
eigenvalues or determinant-test parameters. -/
def angularDensity {k : ℕ} (a : UpperCoordinates k) : ℝ≥∞0 :=
  ENNReal.ofReal (|((minusHalf (skew a))⁻¹).det| ^ (k - 1))

theorem angularDensity_pos_lt_top {k : ℕ} (a : UpperCoordinates k) :
    0 < angularDensity a ∧ angularDensity a < ⊤ :=
  ⟨ENNReal.ofReal_pos.mpr (angularFactor_pos a), ENNReal.ofReal_lt_top⟩

theorem angularDensity_eq_of_skew_eq {k : ℕ} {a b : UpperCoordinates k}
    (h : skew a = skew b) : angularDensity a = angularDensity b := by
  simp only [angularDensity, h]

/-- The actual cone density on its positive-definite domain. Outside that
domain this is merely a total function; all change-of-variable clients below
explicitly keep their entire image in the cone. -/
def coneTestDensity {k : ℕ} (eta rho T : ℝ) (b : UpperCoordinates k) : ℝ≥∞0 :=
  ENNReal.ofReal (Real.exp (-(fromUpper b).trace) * (fromUpper b).det ^ eta) *
    determinantTest rho T (fromUpper b)
```

71.2 The pointwise weighted Jacobian

For $T \geq 0$ and $q_i = s_i + v_i(a) > 0$, multiply the full Jacobian formula by $W_{\eta,\rho,T}(\mathcal{S}_s(a))$. Positivity of the angular factor justifies factoring it through `ENNReal.ofReal_mul`. The remaining Vandermonde times cone density is exactly the inherited sorted Laguerre density by the preceding adapter. Therefore

$$\mathrm{ofReal}(|\det D\mathcal{S}_s(a)|)W_{\eta,\rho,T}(\mathcal{S}_s(a)) = j(a)\,\mathrm{sortedLaguerreIntegrand}(\eta, \rho, T, q).$$

The tuple in this formula is q, not the central tuple s.

```
theorem weighted_jacobian {k : ℕ} (eta rho T : ℝ)
    (s : Fin k → ℝ) (a : UpperCoordinates k)
    (hT : 0 ≤ T) (hq : ∀ i, 0 < s i + diagonalVelocity a i) :
    ENNReal.ofReal |(fderiv ℝ (orthogonalSpectral s) a).det| *
      coneTestDensity eta rho T (orthogonalSpectral s a) =
      angularDensity a * sortedLaguerreIntegrand eta rho T (s + diagonalVelocity a) := by
  rw [ContinuousLinearMap.det, abs_det_fderiv_orthogonalSpectral_at,
    ENNReal.ofReal_mul (angularFactor_pos a).le, mul_assoc]
  change angularDensity a *
      (ENNReal.ofReal (absoluteVandermonde (s + diagonalVelocity a)) *
        (ENNReal.ofReal (Real.exp (-(fromUpper (orthogonalSpectral s a)).trace) *
          (fromUpper (orthogonalSpectral s a)).det ^ eta) *
          determinantTest rho T (fromUpper (orthogonalSpectral s a)))) = _
  rw [cone_density_orthogonalSpectral eta rho T s a hT hq]
```

71.3 Integrate over a fixed injective window

Let A be measurable, contained in the source of e_s, and suppose $s_i + v_i(a) > 0$ for every $a \in A$ and every i. The local Jacobian theorem applied to W and pointwise congruence give

$$\int_{\mathcal{S}_s(A)} W_{\eta,\rho,T}(b)\, db = \int_A j(a)\, \mathrm{sortedLaguerreIntegrand}(\eta, \rho, T, s + v(a))\, da.$$

The chart and window are fixed before choosing the real exponents and $T \geq 0$. The next lemma separately proves that the entire image is positive definite. The final theorem specializes the exponent to $\eta_{k,d}$. All integrals remain nonnegative, with no finiteness assumption. A global constant cannot be inferred from this local formula until chart overlaps and angular integration are accounted for.

```
/-- Exact local cone-to-sorted-Laguerre integration, including the full angular
factor. The source window is fixed before eta, rho and T, and no integral is
assumed finite. This is not a global angular normalization theorem. -/
theorem local_cone_integral {k : ℕ} (s : Fin k → ℝ)
    (hs : Function.Injective s) {S : Set (UpperCoordinates k)}
    (hS : MeasurableSet S) (hsub : S ⊆ (localChart s hs).source)
    (hpos : ∀ a ∈ S, ∀ i, 0 < s i + diagonalVelocity a i)
    (eta rho T : ℝ) (hT : 0 ≤ T) :
    ∫⁻( b in orthogonalSpectral s '' S, coneTestDensity eta rho T b ∂upperLebesgue k) =
      ∫⁻ a in S, angularDensity a *
        sortedLaguerreIntegrand eta rho T (s + diagonalVelocity a) ∂upperLebesgue k := by
  rw [O77.SpectralLocal.lintegral_image s hs hS hsub]
  apply setLIntegral_congr_fun hS
  intro a ha
  exact weighted_jacobian eta rho T s a hT (hpos a ha)

theorem local_image_posDef {k : ℕ} (s : Fin k → ℝ)
    {S : Set (UpperCoordinates k)}
    (hpos : ∀ a ∈ S, ∀ i, 0 < s i + diagonalVelocity a i) :
    orthogonalSpectral s '' S ⊆ {b | (fromUpper b).PosDef} := by
  rintro b ⟨a, ha, ⟩rfl
  exact orthogonalSpectral_posDef s a (hpos a ha)

theorem local_wishart_cone_integral {k d : ℕ} (s : Fin k → ℝ)
    (hs : Function.Injective s) {S : Set (UpperCoordinates k)}
    (hS : MeasurableSet S) (hsub : S ⊆ (localChart s hs).source)
    (hpos : ∀ a ∈ S, ∀ i, 0 < s i + diagonalVelocity a i)
    (rho T : ℝ) (hT : 0 ≤ T) :
    ∫⁻( b in orthogonalSpectral s '' S,
        coneTestDensity (wishartEta k d) rho T b ∂upperLebesgue k) =
      ∫⁻ a in S, angularDensity a *
        sortedLaguerreIntegrand (wishartEta k d) rho T
          (s + diagonalVelocity a) ∂upperLebesgue k :=
  local_cone_integral s hs hS hsub hpos (wishartEta k d) rho T hT

end O77.SpectralChartIntegral
end
```

72 Charts at every actual simple-spectrum matrix

The local chart above is centered at a diagonal matrix. This module moves it to every simple-spectrum symmetric matrix using its actual Mathlib spectral decomposition. The result is an open cover with a countable subcover, and an exact local integration formula after the move. Covering sets are not asserted to be disjoint.

Implementation namespace and dependencies. Module `O77SpectralCoverage`; imports `Mathlib`, `O77SpectralLocal`, `O77ConjugationVolume`, `O77SpectralExceptional`.

```
import Mathlib
import O77SpectralLocal
import O77ConjugationVolume
import O77SpectralExceptional

set_option autoImplicit false
noncomputable section
open Set MeasureTheory Matrix
open scoped Topology ENNReal

namespace O77.SpectralCoverage
open O77.GramBorder O77.SpectralLocal O77.OrthogonalSpectral
open O77.ConjugationVolume O77.SpectralExceptional
```

72.1 Move a chart by a fixed orthogonal conjugation

For $Q^{\mathsf{T}}Q = I$, the previously constructed linear equivalence c_Q is continuous with continuous inverse in finite dimension. Postcompose the local chart e_s with that global homeomorphism to obtain `movedChart`. Its source is unchanged, and its forward map is exactly $u \mapsto c_Q(\mathcal{S}_s(u))$. The lemma `movedChart_source` records this identity of source sets, so all local injectivity and window properties retain their original hypotheses.

```
def conjugationContinuousEquiv {k : ℕ} (Q : Matrix (Fin k) (Fin k) ℝ)
    (hQ : Q.transpose * Q = 1) : UpperCoordinates k ≃L[ℝ] UpperCoordinates k :=
  (conjugationEquiv Q hQ).toContinuousLinearEquiv

@[simp] lemma conjugationContinuousEquiv_apply {k : ℕ}
    (Q : Matrix (Fin k) (Fin k) ℝ) (hQ : Q.transpose * Q = 1)
    (a : UpperCoordinates k) :
    conjugationContinuousEquiv Q hQ a = conjugateUpper Q a := rfl

/-- Transport the constructed local inverse chart by an actual orthogonal
conjugation. The source is unchanged, and its forward function is explicit. -/
def movedChart {k : ℕ} (Q : Matrix (Fin k) (Fin k) ℝ)
    (hQ : Q.transpose * Q = 1) (s : Fin k → ℝ) (hs : Function.Injective s) :
    OpenPartialHomeomorph (UpperCoordinates k) (UpperCoordinates k) :=
  (localChart s hs).transHomeomorph (conjugationContinuousEquiv Q hQ).toHomeomorph

@[simp] lemma movedChart_apply {k : ℕ} (Q : Matrix (Fin k) (Fin k) ℝ)
    (hQ : Q.transpose * Q = 1) (s : Fin k → ℝ) (hs : Function.Injective s)
    (u : UpperCoordinates k) :
    movedChart Q hQ s hs u = conjugateUpper Q (orthogonalSpectral s u) := rfl

@[simp] lemma movedChart_source {k : ℕ} (Q : Matrix (Fin k) (Fin k) ℝ)
    (hQ : Q.transpose * Q = 1) (s : Fin k → ℝ) (hs : Function.Injective s) :
    (movedChart Q hQ s hs).source = (localChart s hs).source := rfl
```

72.2 Use the matrix supplied by the spectral theorem

Define Q_a as Mathlib's `eigenvectorUnitary` of the actual Hermitian matrix $S(a)$. Over $\mathbb{R}$, adjoint equals transpose; unitarity thus gives $Q_a^{\mathsf{T}}Q_a = I$. `Matrix.IsHermitian.spectral_theorem` yields

$$S(a) = Q_a \,\mathrm{diag}(s_a) Q_a^{\mathsf{T}}, \qquad s_a = \mathrm{eigenvalues}(a).$$

Extracting upper entries gives exact reconstruction of a. Both the tuple and the frame come from the actual matrix; a supplied diagonalization is not an extra premise of the resulting

chart theorem.

```
/-- Mathlib's actual eigenvector matrix for the assembled symmetric matrix. -/
def eigenMatrix {k : ℕ} (a : UpperCoordinates k) : Matrix (Fin k) (Fin k) ℝ :=
  (fromUpper_isHermitian a).eigenvectorUnitary

private lemma real_star_eq_transpose {k : ℕ} (Q : Matrix (Fin k) (Fin k) ℝ) :
    star Q = Q.transpose := by
  ext i j
  simp only [Matrix.star_eq_conjTranspose, Matrix.conjTranspose_apply,
    Matrix.transpose_apply, star_trivial]

lemma eigenMatrix_orthogonal {k : ℕ} (a : UpperCoordinates k) :
    (eigenMatrix a).transpose * eigenMatrix a = 1 := by
  rw ←[ real_star_eq_transpose]
  exact Unitary.coe_star_mul_self (fromUpper_isHermitian a).eigenvectorUnitary

lemma spectral_decomposition {k : ℕ} (a : UpperCoordinates k) :
    fromUpper a = eigenMatrix a * Matrix.diagonal (eigenvalues a) *
      (eigenMatrix a).transpose := by
  have h : fromUpper a = eigenMatrix a * Matrix.diagonal (eigenvalues a) *
      star (eigenMatrix a) := by
    simpa [eigenMatrix, eigenvalues, Unitary.conjStarAlgAut_apply] using
      (fromUpper_isHermitian a).spectral_theorem
  simpa only [real_star_eq_transpose] using h

lemma spectral_reconstruction {k : ℕ} (a : UpperCoordinates k) :
    conjugateUpper (eigenMatrix a) (upperEntries (Matrix.diagonal (eigenvalues a))) = a := by
  unfold conjugateUpper
  rw [fromUpper_upperEntries (Matrix.isSymm_diagonal _),
    ← spectral_decomposition a, upperEntries_fromUpper]
```

72.3 Construct and cover every simple-spectrum center

The subtype `SimplePointk` consists of $a \in E_k$ whose actual characteristic polynomial is separable. For such a, the actual s_a is injective, so define E_a by moving e_{s_a} with Q_a. Reconstruction gives $E_a(0) = a$; zero is in its source, hence a is in its open target. The targets therefore cover the simple-spectrum locus. Finite-dimensional coordinate space is second countable and hereditarily Lindelöf, so `HereditarilyLindelofSpace.isLindelof` and `elim_countable_subcover` select a countable subfamily of these actual targets. The conclusion is a covering inclusion, not an equality or a disjoint decomposition.

```
abbrev SimplePoint (k : ℕ) :=
  {a : UpperCoordinates k // (fromUpper a).charpoly.Separable}

/-- No supplied diagonalization or chart package occurs in this interface. -/
def chartAt {k : ℕ} (a : SimplePoint k) :
    OpenPartialHomeomorph (UpperCoordinates k) (UpperCoordinates k) :=
  movedChart (eigenMatrix a.1) (eigenMatrix_orthogonal a.1) (eigenvalues a.1)
    ((separable_charpoly_iff_injective_eigenvalues a.1).1 a.2)

@[simp] lemma chartAt_zero {k : ℕ} (a : SimplePoint k) :
    chartAt a 0 = a.1 := by
  change conjugateUpper (eigenMatrix a.1) (orthogonalSpectral (eigenvalues a.1) 0) = a.1
  rw [orthogonalSpectral_zero, spectral_reconstruction]

lemma zero_mem_chartAt_source {k : ℕ} (a : SimplePoint k) :
    (0 : UpperCoordinates k) ∈ (chartAt a).source :=
  zero_mem_source _ _

lemma center_mem_chartAt_target {k : ℕ} (a : SimplePoint k) :
```

```
    a.1 ∈ (chartAt a).target := by
  simpa only [chartAt_zero] using (chartAt a).map_source (zero_mem_chartAt_source a)

theorem simple_locus_subset_chart_targets (k : ℕ) :
    {a : UpperCoordinates k | (fromUpper a).charpoly.Separable} ⊆
      ⋃ a : SimplePoint k, (chartAt a).target := by
  intro a ha
  exact mem_iUnion.2 ⟨⟨a, ⟩ha, center_mem_chartAt_target ⟨a, ⟩⟩ha

/-- Finite-dimensional second countability gives an actual countable subcover
of targets of the constructed charts. No countable family is assumed. -/
theorem exists_countable_chart_cover (k : ℕ) :
    ∃ C : Set (SimplePoint k), C.Countable ∧
      {a : UpperCoordinates k | (fromUpper a).charpoly.Separable} ⊆
        ⋃ a ∈ C, (chartAt a).target := by
  exact (HereditarilyLindelofSpace.isLindelof
    {a : UpperCoordinates k | (fromUpper a).charpoly.Separable}).elim_countable_subcover
      (fun a : SimplePoint k => (chartAt a).target)
      (fun a => (chartAt a).open_target) (simple_locus_subset_chart_targets k)
```

72.4 Transport the local integral without a new factor

For a measurable A in the moved source, its image is $c_Q(\mathcal{S}_s(A))$. The previously proved exact measure preservation of c_Q changes the integral over this image to the integral of $F \circ c_Q$ over $\mathcal{S}_s(A)$. Apply the local Jacobian theorem to obtain

$$\int_{E(A)} F(b)\,db = \int_A |\det D\mathcal{S}_s(u)| F(E(u))\,du.$$

The library `MeasurePreserving.setLIntegral_comp_emb` handles the fixed conjugation as a measurable embedding. Its absolute coordinate determinant is one, so no additional density is introduced. The test function is nonnegative and no finiteness is required.

```
/-- Fixed conjugation has absolute coordinate determinant one, so the exact
local nonnegative integral retains the unmoved chart's Jacobian. -/
theorem lintegral_moved_image {k : ℕ} (Q : Matrix (Fin k) (Fin k) ℝ)
    (hQ : Q.transpose * Q = 1) (s : Fin k → ℝ) (hs : Function.Injective s)
    {S : Set (UpperCoordinates k)} (hS : MeasurableSet S)
    (hsub : S ⊆ (movedChart Q hQ s hs).source) (F : UpperCoordinates k → ℝ≥∞0) :
    ∫⁻( b in movedChart Q hQ s hs '' S, F b ∂upperLebesgue k) =
      ∫⁻ u in S, ENNReal.ofReal |(fderiv ℝ (orthogonalSpectral s) u).det| *
        F (movedChart Q hQ s hs u) ∂upperLebesgue k := by
  have himage : movedChart Q hQ s hs '' S =
      conjugateUpper Q '' (orthogonalSpectral s '' S) := by
    rw [Set.image_image]
    rfl
  rw [himage]
  calc
    _ = ∫⁻ b in orthogonalSpectral s '' S, F (conjugateUpper Q b) ∂upperLebesgue k :=
      ((conjugateUpper_preserves Q hQ).setLIntegral_comp_emb
        (conjugationContinuousEquiv Q hQ).toHomeomorph.toMeasurableEquiv.measurableEmbedding
        F (orthogonalSpectral s '' S)).symm
    _ = _ := O77.SpectralLocal.lintegral_image s hs hS hsub (fun b => F (conjugateUpper Q b))
```

72.5 Positive windows at arbitrary centers

If $S(a)$ is also positive definite, the Hermitian spectral theorem's `posDef_iff_eigenvalues_pos` makes every actual eigenvalue positive. Apply the common positive window theorem at s_a and move it by c_{Q_a}. Positive definiteness is preserved by this conjugation, and the same lower

and upper Jacobian bounds survive. The window is chosen once at the fixed center before selecting subsets or test functions. This provides local analytic control everywhere on the positive simple-spectrum cone; global overlap counting is a separate step in the subsequent integration modules.

```
/-- At every positive-definite simple-spectrum center, one common open source
window stays in the positive cone and has the inherited positive Jacobian
bounds. The point's actual eigenvalues determine these constants. -/
theorem exists_positive_window_at {k : ℕ} (a : SimplePoint k)
    (ha : (fromUpper a.1).PosDef) :
    ∃ U : Set (UpperCoordinates k), IsOpen U ∧ 0 ∈ U ∧
      U ⊆ (chartAt a).source ∧
      ∀( u ∈ U, (fromUpper (chartAt a u)).PosDef) ∧
      ∀ u ∈ U,
        |O77.Residual.vandermondeFin (eigenvalues a.1)| / 2 ≤
          |(fderiv ℝ (orthogonalSpectral (eigenvalues a.1)) u).det| ∧
        |(fderiv ℝ (orthogonalSpectral (eigenvalues a.1)) u).det| ≤
          2 * |O77.Residual.vandermondeFin (eigenvalues a.1)| := by
  have hp : ∀ i, 0 < eigenvalues a.1 i :=
    (fromUpper_isHermitian a.1).posDef_iff_eigenvalues_pos.mp ha
  obtain ⟨U, hU, hzero, hsub, hpos, ⟩hbounds :=
    exists_positive_common_window (eigenvalues a.1)
      ((separable_charpoly_iff_injective_eigenvalues a.1).1 a.2) hp
  refine ⟨U, hU, hzero, hsub, ?_, ⟩hbounds
  intro u hu
  exact (posDef_conjugateUpper_iff (eigenMatrix a.1)
    (eigenMatrix_orthogonal a.1) _).2 (hpos u hu)

end O77.SpectralCoverage
end
```

73 The exact finite ambiguity of an eigenframe

Fix an injective real tuple $s : \mathrm{Fin}(k) \to \mathbb{R}$ and $D = \mathrm{diag}(s)$. No positivity or order assumption is needed. The tuple is held fixed: this module counts sign choices of its eigenvectors, not permutations of eigenvalues. The assertions concern all actual orthogonal matrices and include $k = 0$.

Implementation namespace and dependencies. Module `O77SpectralStabilizer`; imports `Mathlib`.

```
import Mathlib

set_option autoImplicit false
noncomputable section
open Set Matrix
open scoped BigOperators

namespace O77.SpectralStabilizer
```

73.1 Commuting matrices and diagonal sign matrices

If $QD = DQ$, its (i, j) entry says $Q_{ij}(s_j - s_i) = 0$. For $i \neq j$, injectivity of s makes the difference nonzero, so $Q_{ij} = 0$; thus Q is diagonal. If instead orthogonality and $QDQ^{\mathsf{T}} = D$ are given, right multiplication by Q gives $QD = DQ$. Conversely, a diagonal matrix with entries ε_i in $\{-1, 1\}$ satisfies both orthogonality and stabilization because $\varepsilon_i^2 = 1$. These facts give the two directions needed for the exact stabilizer description.

```
/-- Distinct diagonal entries force every commuting real matrix to be
```

```
diagonal. No positivity or ordering of the entries is required. -/
lemma eq_diagonal_of_commute {k : ℕ} (s : Fin k → ℝ)
    (hs : Function.Injective s) (Q : Matrix (Fin k) (Fin k) ℝ)
    (hcomm : Q * Matrix.diagonal s = Matrix.diagonal s * Q) :
    Q = Matrix.diagonal (fun i => Q i i) := by
  ext i j
  by_cases hij : i = j
  · subst j
    simp
  · have heq := congrArg (fun M : Matrix (Fin k) (Fin k) ℝ => M i j) hcomm
    simp only [Matrix.mul_diagonal, Matrix.diagonal_mul] at heq
    have hdiff : s j - s i ≠ 0 := by
      intro h
      exact hij (hs (sub_eq_zero.mp h)).symm
    have hmul : Q i j * (s j - s i) = 0 := by nlinarith [heq]
    have hzero : Q i j = 0 := (mul_eq_zero.mp hmul).resolve_right hdiff
    simp [hij, hzero]

lemma commute_of_orthogonal_stabilizes {k : ℕ} (s : Fin k → ℝ)
    (Q : Matrix (Fin k) (Fin k) ℝ) (hQ : Q.transpose * Q = 1)
    (hstab : Q * Matrix.diagonal s * Q.transpose = Matrix.diagonal s) :
    Q * Matrix.diagonal s = Matrix.diagonal s * Q := by
  have h := congrArg (fun M : Matrix (Fin k) (Fin k) ℝ => M * Q) hstab
  simpa only [Matrix.mul_assoc, hQ, Matrix.mul_one] using h

lemma sign_diagonal_properties {k : ℕ} (s eps : Fin k → ℝ)
    (heps : ∀ i, eps i = -1 ∨ eps i = 1) :
    (Matrix.diagonal eps).transpose * Matrix.diagonal eps = 1 ∧
      Matrix.diagonal eps * Matrix.diagonal s *
        (Matrix.diagonal eps).transpose = Matrix.diagonal s := by
  have hmul : ∀ i, eps i * eps i = 1 := by
    intro i
    rcases heps i with h | h <;> rw [h] <;> norm_num
  constructor
  · rw [Matrix.diagonal_transpose, Matrix.diagonal_mul_diagonal]
    ext i j
    by_cases hij : i = j
    · subst j
      simp [hmul]
    · simp [hij]
  · rw [Matrix.diagonal_transpose, Matrix.diagonal_mul_diagonal,
      Matrix.diagonal_mul_diagonal]
    apply congrArg Matrix.diagonal
    funext i
    calc
      eps i * s i * eps i = s i * (eps i * eps i) := by ring
      _ = s i := by rw [hmul, mul_one]
```

73.2 The full stabilizer, with both implications

The preceding commutation lemma makes an orthogonal stabilizer Q diagonal. Its diagonal orthogonality equations give $Q_{ii}^2 = 1$, hence $Q_{ii} = 1$ or -1. Therefore

$$Q^{\mathsf{T}}Q = I \ \wedge \ QDQ^{\mathsf{T}} = D \quad \Longleftrightarrow \quad Q = \operatorname{diag}(\varepsilon), \qquad \varepsilon \in \{-1, 1\}^k.$$

The reverse direction reuses the explicit diagonal calculation. This characterizes the entire matrix subtype, so the later cardinality argument does not merely count a proposed collection of examples.

```
/-- The entire orthogonal stabilizer of a diagonal matrix with distinct
entries consists exactly of the independent column-sign changes. -/
```

```
theorem orthogonal_stabilizer_iff {k : ℕ} (s : Fin k → ℝ)
    (hs : Function.Injective s) (Q : Matrix (Fin k) (Fin k) ℝ) :
    (Q.transpose * Q = 1 ∧
      Q * Matrix.diagonal s * Q.transpose = Matrix.diagonal s) ↔
      ∃ eps : Fin k → ℝ, ∀( i, eps i = -1 ∨ eps i = 1) ∧
        Q = Matrix.diagonal eps := by
  constructor
  · rintro ⟨hQ,⟩hstab
    have hdiag := eq_diagonal_of_commute s hs Q
      (commute_of_orthogonal_stabilizes s Q hQ hstab)
    refine ⟨fun i => Q i i, ?_, ⟩hdiag
    intro i
    have hentry := congrArg (fun M : Matrix (Fin k) (Fin k) ℝ => M i i) hQ
    rw [hdiag] at hentry
    simp only [Matrix.diagonal_transpose, Matrix.diagonal_mul_diagonal,
      Matrix.diagonal_apply_eq, Matrix.one_apply_eq] at hentry
    have hsq : (Q i i) ^ 2 = 1 := by nlinarith [hentry]
    rcases sq_eq_one_iff.mp hsq with h | h
    · exact Or.inr h
    · exact Or.inl h
  · rintro ⟨eps,heps,⟩rfl
    exact sign_diagonal_properties s eps heps
```

73.3 Two frames for the same fixed tuple

If orthogonal Q, R give the same conjugate of D, then $E = Q^{\mathsf{T}}R$ is orthogonal and stabilizes D. Indeed, insert $QQ^{\mathsf{T}} = I$ in both products and use the equality of the two conjugates. The stabilizer characterization gives $E = \operatorname{diag}(\varepsilon)$, hence $R = Q\operatorname{diag}(\varepsilon)$. Thus two eigenframes for the same fixed distinct tuple differ only by independent column signs. If the tuple were allowed to be permuted, an additional permutation ambiguity would enter; it is absent from this precise statement.

```
lemma relative_frame_properties {k : ℕ} (s : Fin k → ℝ)
    (Q R : Matrix (Fin k) (Fin k) ℝ)
    (hQ : Q.transpose * Q = 1) (hR : R.transpose * R = 1)
    (hQR : R * Matrix.diagonal s * R.transpose =
      Q * Matrix.diagonal s * Q.transpose) :
    (Q.transpose * R).transpose * (Q.transpose * R) = 1 ∧
      (Q.transpose * R) * Matrix.diagonal s *
        (Q.transpose * R).transpose = Matrix.diagonal s := by
  have hQQ : Q * Q.transpose = 1 := mul_eq_one_comm.mp hQ
  constructor
  · rw [Matrix.transpose_mul, Matrix.transpose_transpose]
    calc
      (R.transpose * Q) * (Q.transpose * R) =
        R.transpose * (Q * Q.transpose) * R := by noncomm_ring
      _ = 1 := by rw [hQQ, mul_one, hR]
  · rw [Matrix.transpose_mul, Matrix.transpose_transpose]
    calc
      (Q.transpose * R) * Matrix.diagonal s * (R.transpose * Q) =
        Q.transpose * (R * Matrix.diagonal s * R.transpose) * Q := by noncomm_ring
      _ = Q.transpose * (Q * Matrix.diagonal s * Q.transpose) * Q := by rw [hQR]
      _ = (Q.transpose * Q) * Matrix.diagonal s * (Q.transpose * Q) := by
        noncomm_ring
      _ = Matrix.diagonal s := by rw [hQ, one_mul, mul_one]

/-- For the same fixed ordered distinct spectrum, two actual orthogonal
eigenframes differ by exactly a right diagonal sign matrix. The theorem
does not permit a permutation of the given spectrum. -/
theorem eigenframes_differ_by_signs {k : ℕ} (s : Fin k → ℝ)
```

```
    (hs : Function.Injective s) (Q R : Matrix (Fin k) (Fin k) ℝ)
    (hQ : Q.transpose * Q = 1) (hR : R.transpose * R = 1)
    (hQR : R * Matrix.diagonal s * R.transpose =
      Q * Matrix.diagonal s * Q.transpose) :
    ∃ eps : Fin k → ℝ, ∀( i, eps i = -1 ∨ eps i = 1) ∧
      R = Q * Matrix.diagonal eps := by
  obtain ⟨eps,heps,⟩hdiag := (orthogonal_stabilizer_iff s hs (Q.transpose * R)).1
    (relative_frame_properties s Q R hQ hR hQR)
  refine ⟨eps,heps, ?⟩_
  have hQQ : Q * Q.transpose = 1 := mul_eq_one_comm.mp hQ
  calc
    R = (Q * Q.transpose) * R := by rw [hQQ, one_mul]
    _ = Q * (Q.transpose * R) := Matrix.mul_assoc _ _ _
    _ = Q * Matrix.diagonal eps := by rw [hdiag]
```

73.4 A finite equivalence with the actual stabilizer

Encode signs by Boolean tuples, assigning 1 to true and -1 to false. Evaluation at each coordinate proves injectivity of this encoding; deciding whether a given sign is 1 supplies its inverse. The code packages the resulting map to diagonal matrices as an equivalence

$$\{\mathrm{false},\mathrm{true}\}^k \simeq \{Q : Q^{\mathsf{T}}Q = I,\ QDQ^{\mathsf{T}} = D\}.$$

Surjectivity uses the already proved characterization of every stabilizer. Transporting finite cardinality through this actual equivalence gives 2^k, including the unique empty tuple when $k = 0$. The uses of `decide` here encode Boolean signs through proved identities; they do not replace a mathematical proof by native evaluation.

```
/-- An explicit finite indexing of the actual sign choices. -/
def signVector {k : ℕ} (b : Fin k → Bool) : Fin k → ℝ :=
  fun i => if b i then 1 else -1

lemma signVector_signs {k : ℕ} (b : Fin k → Bool) :
    ∀ i, signVector b i = -1 ∨ signVector b i = 1 := by
  intro i
  cases h : b i <;> simp [signVector, h]

lemma signVector_injective (k : ℕ) :
    Function.Injective (signVector (k := k)) := by
  classical
  intro b c hbc
  funext i
  have decode (d : Fin k → Bool) : decide (signVector d i = 1) = d i := by
    cases hd : d i <;> norm_num [signVector, hd]
  exact (decode b).symm.trans
    ((congrArg (fun x : ℝ => decide (x = 1)) (congrFun hbc i)).trans (decode c))

lemma exists_signVector {k : ℕ} (eps : Fin k → ℝ)
    (heps : ∀ i, eps i = -1 ∨ eps i = 1) :
    ∃ b : Fin k → Bool, signVector b = eps := by
  classical
  refine ⟨fun i => decide (eps i = 1), ?⟩_
  funext i
  rcases heps i with h | h <;> norm_num [signVector, h]

/-- This is an equivalence with the actual matrix stabilizer subtype;
it does not merely count a list of proposed stabilizing matrices. -/
def stabilizerEquivSigns {k : ℕ} (s : Fin k → ℝ) (hs : Function.Injective s) :
    (Fin k → Bool) ≃
      {Q : Matrix (Fin k) (Fin k) ℝ // Q.transpose * Q = 1 ∧
```

```
        Q * Matrix.diagonal s * Q.transpose = Matrix.diagonal s} := by
  refine Equiv.ofBijective
    (fun b => ⟨Matrix.diagonal (signVector b),
      sign_diagonal_properties s (signVector b) (signVector_signs b)⟩) ?_
  constructor
  · intro b c hbc
    apply signVector_injective k
    apply Matrix.diagonal_injective
    exact congrArg Subtype.val hbc
  · intro Q
    obtain ⟨eps,heps,⟩hdiag := (orthogonal_stabilizer_iff s hs Q.val).1 Q.property
    obtain ⟨b,⟩hb := exists_signVector eps heps
    refine ⟨b,Subtype.ext ?⟩_
    change Matrix.diagonal (signVector b) = Q.val
    rw [hb]
    exact hdiag.symm

theorem card_orthogonal_stabilizer {k : ℕ} (s : Fin k → ℝ)
    (hs : Function.Injective s) :
    Nat.card {Q : Matrix (Fin k) (Fin k) ℝ // Q.transpose * Q = 1 ∧
      Q * Matrix.diagonal s * Q.transpose = Matrix.diagonal s} = 2 ^ k := by
  rw ←[ Nat.card_congr (stabilizerEquivSigns s hs)]
  simp [Nat.card_eq_fintype_card]
```

73.5 The full fiber count and the Cayley boundary

For a fixed orthogonal Q, multiplication by Q^{T} maps the full fixed-spectrum eigenframe fiber to the stabilizer; its inverse is multiplication by Q. The block proves both inverse identities and membership equations explicitly. This equivalence itself does not require distinct eigenvalues; distinctness is used only when applying the stabilizer count. Therefore the full fiber has exactly 2^k frames when s is injective.

This is a count in the whole orthogonal group. A single Cayley chart need not contain all these frames, and the number it contains can vary. Consequently this theorem alone does not authorize division of a Cayley-chart integral by 2^k. The global proof accounts for actual Cayley fibers separately, and the full-orthant permutation factor is also a separate issue.

```
/-- Translation by one actual orthogonal eigenframe identifies its full
fixed-spectrum fiber with the stabilizer. The equivalence itself requires
no distinctness hypothesis; that hypothesis enters only in the count. -/
def eigenframeEquivStabilizer {k : ℕ} (s : Fin k → ℝ)
    (Q : Matrix (Fin k) (Fin k) ℝ) (hQ : Q.transpose * Q = 1) :
    {R : Matrix (Fin k) (Fin k) ℝ // R.transpose * R = 1 ∧
      R * Matrix.diagonal s * R.transpose = Q * Matrix.diagonal s * Q.transpose} ≃
    {E : Matrix (Fin k) (Fin k) ℝ // E.transpose * E = 1 ∧
      E * Matrix.diagonal s * E.transpose = Matrix.diagonal s} where
  toFun R := ⟨Q.transpose * R.val,
    relative_frame_properties s Q R.val hQ R.property.1 R.property⟩.2
  invFun E := ⟨Q * E.val, by
    constructor
    · rw [Matrix.transpose_mul]
      calc
        (E.val.transpose * Q.transpose) * (Q * E.val) =
          E.val.transpose * (Q.transpose * Q) * E.val := by noncomm_ring
        _ = 1 := by rw [hQ, mul_one, E.property.1]
    · rw [Matrix.transpose_mul]
      calc
        (Q * E.val) * Matrix.diagonal s * (E.val.transpose * Q.transpose) =
          Q * (E.val * Matrix.diagonal s * E.val.transpose) * Q.transpose := by
            noncomm_ring
        _ = Q * Matrix.diagonal s * Q.transpose := by rw [E.property⟩.2]
```

```
  left_inv R := by
    apply Subtype.ext
    change Q * (Q.transpose * R.val) = R.val
    rw ←[ Matrix.mul_assoc, mul_eq_one_comm.mp hQ, one_mul]
  right_inv E := by
    apply Subtype.ext
    change Q.transpose * (Q * E.val) = E.val
    rw ←[ Matrix.mul_assoc, hQ, one_mul]

theorem card_fixed_spectrum_eigenframes {k : ℕ} (s : Fin k → ℝ)
    (hs : Function.Injective s) (Q : Matrix (Fin k) (Fin k) ℝ)
    (hQ : Q.transpose * Q = 1) :
    Nat.card {R : Matrix (Fin k) (Fin k) ℝ // R.transpose * R = 1 ∧
      R * Matrix.diagonal s * R.transpose =
        Q * Matrix.diagonal s * Q.transpose} = 2 ^ k := by
  rw [Nat.card_congr (eigenframeEquivStabilizer s Q hQ)]
  exact card_orthogonal_stabilizer s hs

end O77.SpectralStabilizer
end
```

74 Every spectral orbit meets the Cayley chart

Fix $k \geq 0$. Matrices in this section are real $k \times k$ matrices, D_b denotes the diagonal sign matrix with ith entry 1 for $b_i = \mathsf{true}$ and -1 otherwise, and $C(K) = (I - K/2)^{-1}(I + K/2)$ is the half-scaled Cayley transform. The chart condition on an orthogonal matrix R is $\det(R+I) \neq 0$. Its inverse is $K = 2(R - I)(R + I)^{-1}$, proved in `O77.OrthogonalCayley.cayley_inverseCayley` and `O77.OrthogonalCayley.inverseCayley_cayley`. Our immediate goal is coverage by this one Cayley parametrization after diagonal signs are forgotten under conjugation. No measure assertion is needed here.

```
import Mathlib
import O77SpectralStabilizer
import O77OrthogonalCayley

set_option autoImplicit false
noncomputable section
open Matrix
open scoped BigOperators

namespace O77.CayleySignCover

open O77.SpectralStabilizer O77.OrthogonalCayley
```

Summing column choices. For two possible columns $c_j(\mathsf{false})$ and $c_j(\mathsf{true})$ at each index j, determinant multilinearity gives

$$\sum_b \det(c_1(b_1), \ldots, c_k(b_k)) = \det\big(c_1(\mathsf{false}) + c_1(\mathsf{true}), \ldots, c_k(\mathsf{false}) + c_k(\mathsf{true})\big).$$

The formal proof expands the Leibniz formula `Matrix.det_apply'`, exchanges two finite sums, and uses `Fintype.prod_sum`. Thus the relevant multilinearity is proved for the actual column arrangement, without an orientation convention being left implicit. Empty columns and the unique empty sign choice give $1 = 1$.

```
/-- Summing independent column choices commutes with the determinant.
The Boolean index is the actual finite set of choices, including the
```

```
unique empty choice when `k = 0`. -/
lemma sum_det_column_choices {k : ℕ}
    (C : Fin k → Bool → Fin k → ℝ) :
    ∑( b : Fin k → Bool, Matrix.det (Matrix.of (fun i j => C j (b j) i))) =
      Matrix.det (Matrix.of (fun i j => ∑ c : Bool, C j c i)) := by
  classical
  simp only [Matrix.det_apply', Matrix.of_apply]
  rw [Finset.sum_comm]
  apply Finset.sum_congr rfl
  intro sigma _
  rw ←[ Finset.mul_sum]
  exact congrArg (fun x : ℝ => ((Equiv.Perm.sign sigma : ℤ) : ℝ) * x)
    (Fintype.prod_sum κ( := fun _ : Fin k => Bool)
      (fun (i : Fin k) (c : Bool) => C i c (sigma i))).symm
```

A nonzero signed branch exists. Apply the preceding identity with $c_j(b_j) = e_j + (D_b)_{jj} Q_{\cdot j}$. The two choices add to $2e_j$, so

$$\sum_{b:\mathrm{Fin}(k)\to\mathrm{Bool}} \det(QD_b + I) = 2^k.$$

This identity holds for every Q, including singular and nonorthogonal matrices. Since $2^k \neq 0$, at least one summand is nonzero. Over $\mathbb{R}$ this is exactly the unit-determinant condition used in `exists_sign_regular`. The proof does not select a sign measurably and does not assert a constant number of successful signs.

```
/-- The average determinant of all independent signed-column
perturbations of the identity is one. No invertibility or orthogonality
assumption on `Q` is used. -/
theorem sum_det_signs {k : ℕ} (Q : Matrix (Fin k) (Fin k) ℝ) :
    ∑( b : Fin k → Bool,
      (Q * Matrix.diagonal (signVector b) + 1).det) = (2 : ℝ) ^ k := by
  classical
  have hbool (x y : ℝ) :
      ∑( c : Bool, (x * (if c then 1 else -1) + y)) = 2 * y := by
    calc
      ∑( c : Bool, (x * (if c then 1 else -1) + y)) =
          (x * 1 + y) + (x * (-1) + y) := Fintype.sum_bool _
      _ = 2 * y := by ring
  calc
    ∑( b : Fin k → Bool,
        (Q * Matrix.diagonal (signVector b) + 1).det) =
        ∑ b : Fin k → Bool,
          Matrix.det (Matrix.of (fun i j => Q i j * (if b j then 1 else -1) +
            (1 : Matrix (Fin k) (Fin k) ℝ) i j)) := by
      apply Finset.sum_congr rfl
      intro b _
      apply congrArg (fun M : Matrix (Fin k) (Fin k) ℝ => M.det)
      funext i j
      simp [signVector]
    _ = Matrix.det (Matrix.of (fun i j => ∑ c : Bool,
        (Q i j * (if c then 1 else -1) +
          (1 : Matrix (Fin k) (Fin k) ℝ) i j))) :=
      sum_det_column_choices (fun j c i =>
        Q i j * (if c then 1 else -1) +
          (1 : Matrix (Fin k) (Fin k) ℝ) i j)
    _ = ((2 : ℝ) • (1 : Matrix (Fin k) (Fin k) ℝ)).det := by
      apply congrArg (fun M : Matrix (Fin k) (Fin k) ℝ => M.det)
      funext i j
      exact hbool (Q i j) ((1 : Matrix (Fin k) (Fin k) ℝ) i j)
```

```
    _ = (2 : ℝ) ^ k := by simp

/-- A regular signed branch exists at every matrix, not merely almost
every orthogonal frame. This proves existence, not constant multiplicity. -/
theorem exists_sign_regular {k : ℕ} (Q : Matrix (Fin k) (Fin k) ℝ) :
    ∃ b : Fin k → Bool,
      IsUnit (Q * Matrix.diagonal (signVector b) + 1).det := by
  classical
  by_contra h
  have hz : ∀ b : Fin k → Bool,
      (Q * Matrix.diagonal (signVector b) + 1).det = 0 := by
    intro b
    by_contra hb
    exact h ⟨b, isUnit_iff_ne_zero.mpr ⟩hb
  have hsum := sum_det_signs Q
  simp only [hz, Finset.sum_const_zero] at hsum
  exact (pow_ne_zero k (by norm_num : (2 : ℝ) ≠ 0)) hsum.symm
```

Signs preserve the orthogonal condition. If $Q^TQ = I$, then $(QD_b)^TQD_b = D_b^TD_b = I$. The matrix multiplications below retain their order; the last equality uses the entrywise identities $(D_b)_{ii}^2 = 1$ established in `sign_diagonal_properties`.

```
lemma sign_adjusted_orthogonal {k : ℕ}
    (Q : Matrix (Fin k) (Fin k) ℝ) (hQ : Q.transpose * Q = 1)
    (b : Fin k → Bool) :
    (Q * Matrix.diagonal (signVector b)).transpose *
      (Q * Matrix.diagonal (signVector b)) = 1 := by
  rw [Matrix.transpose_mul]
  calc
    ((Matrix.diagonal (signVector b)).transpose * Q.transpose) *
        (Q * Matrix.diagonal (signVector b)) =
        (Matrix.diagonal (signVector b)).transpose * (Q.transpose * Q) *
          Matrix.diagonal (signVector b) := by noncomm_ring
    _ = 1 := by
      rw [hQ, mul_one]
      exact (sign_diagonal_properties (0 : Fin k → ℝ)
        (signVector b) (signVector_signs b)).1
```

Exact decomposition and conjugation coverage. Choose b with $\det(QD_b + I) \neq 0$ and put $R = QD_b$. The inverse Cayley identities give a skew matrix $K = 2(R - I)(R + I)^{-1}$ with $C(K) = R$. Since $D_b^2 = I$, $Q = C(K)D_b$. Moreover $D_b \operatorname{diag}(s) D_b^T = \operatorname{diag}(s)$ for every vector s. Hence

$$C(K)\operatorname{diag}(s)C(K)^T = Q\operatorname{diag}(s)Q^T.$$

The final theorem requires only orthogonality of Q. It does not require positive, distinct, or sorted entries of s. This distinction matters: coverage precedes, and is separate from, the finite-fiber argument.

```
/-- Every orthogonal frame is a Cayley frame followed by an actual
diagonal sign matrix. The sign choice is allowed to depend on the frame. -/
theorem exists_cayley_sign_decomposition {k : ℕ}
    (Q : Matrix (Fin k) (Fin k) ℝ) (hQ : Q.transpose * Q = 1) :
    ∃ (b : Fin k → Bool) (K : Matrix (Fin k) (Fin k) ℝ),
      K.transpose = -K ∧ Q = cayley K * Matrix.diagonal (signVector b) := by
  obtain ⟨b, ⟩hb := exists_sign_regular Q
  have hR := sign_adjusted_orthogonal Q hQ b
  have hskew := transpose_inverseCayley
    (Q * Matrix.diagonal (signVector b)) hR hb
  have hrec := cayley_inverseCayley
```

```
    (Q * Matrix.diagonal (signVector b)) hR hb
  have hDD : Matrix.diagonal (signVector b) *
      Matrix.diagonal (signVector b) = 1 := by
    simpa only [Matrix.diagonal_transpose] using
      (sign_diagonal_properties (0 : Fin k → ℝ)
        (signVector b) (signVector_signs b)).1
  refine ⟨b, inverseCayley (Q * Matrix.diagonal (signVector b)), hskew, ?⟩_
  rw [hrec, Matrix.mul_assoc, hDD, mul_one]

/-- Passing to conjugation removes the selected column signs. Thus one
global Cayley map reaches every orthogonal conjugate of every diagonal
matrix, even though its finite fiber multiplicity can vary. -/
theorem exists_cayley_conjugate {k : ℕ}
    (s : Fin k → ℝ) (Q : Matrix (Fin k) (Fin k) ℝ)
    (hQ : Q.transpose * Q = 1) :
    ∃ K : Matrix (Fin k) (Fin k) ℝ, K.transpose = -K ∧
      cayley K * Matrix.diagonal s * (cayley K).transpose =
        Q * Matrix.diagonal s * Q.transpose := by
  obtain ⟨b, ⟩hb := exists_sign_regular Q
  have hR := sign_adjusted_orthogonal Q hQ b
  have hskew := transpose_inverseCayley
    (Q * Matrix.diagonal (signVector b)) hR hb
  refine ⟨inverseCayley (Q * Matrix.diagonal (signVector b)), hskew, ?⟩_
  rw [cayley_inverseCayley (Q * Matrix.diagonal (signVector b)) hR hb,
    Matrix.transpose_mul]
  calc
    (Q * Matrix.diagonal (signVector b)) * Matrix.diagonal s *
        ((Matrix.diagonal (signVector b)).transpose * Q.transpose) =
        Q * (Matrix.diagonal (signVector b) * Matrix.diagonal s *
          (Matrix.diagonal (signVector b)).transpose) * Q.transpose := by
      noncomm_ring
    _ = Q * Matrix.diagonal s * Q.transpose := by
      rw [(sign_diagonal_properties s (signVector b) (signVector_signs b)).2]

end O77.CayleySignCover
end
```

75 Counting actual Cayley fibers

Continue with the same $C(K)$ and D_b. For a real square matrix Q define

$$\mathcal{A}(Q) = \{b : \det(QD_b + I) \neq 0\}, \qquad N(Q) = \#\mathcal{A}(Q).$$

These are respectively `admissibleSigns` and `branchCount`. They count only the sign representatives in the Cayley chart. In particular, $N(Q)$ must not be replaced by the full 2^k-element diagonal stabilizer. The following proof identifies $N(Q)$ with an actual fiber when the indexed spectrum is injective. For example, in dimension two $N(I) = 1$: only the all-positive sign matrix makes $I + D_b$ invertible. For $Q = \left(\begin{smallmatrix} 0 & -1 \\ 1 & 0 \end{smallmatrix}\right)$, the signs $D_b = I$ and $D_b = -I$ both give determinant 2, while the two mixed-sign choices give determinant zero; hence $N(Q) = 2$. This direct calculation explains why the subsequent integral must use the actual varying count.

```
import Mathlib
import O77SpectralStabilizer
import O77OrthogonalSpectral

set_option autoImplicit false
noncomputable section
open Set Matrix MeasureTheory
open scoped BigOperators
```

```
namespace O77.CayleyFibers

open O77.SpectralStabilizer O77.OrthogonalCayley
```

The finite set and the two geometric carriers. The finite sign set is a filter of all Boolean vectors by the explicit determinant condition. For fixed s and Q, `regularEigenframes` consists of orthogonal R with $R\,\mathrm{diag}(s)R^T = Q\,\mathrm{diag}(s)Q^T$ and $\det(R+I) \neq 0$. The type `cayleySpectralFiber` consists of skew matrices K satisfying the corresponding conjugation equation for $C(K)$. Both carriers include the actual matrix equation as part of membership.

```
/-- Column-sign choices whose resulting frame belongs to the actual Cayley chart. -/
def admissibleSigns {k : ℕ} (Q : Matrix (Fin k) (Fin k) ℝ) :
    Finset (Fin k → Bool) := by
  classical
  exact Finset.univ.filter (fun b =>
    IsUnit (Q * Matrix.diagonal (signVector b) + 1).det)

/-- This is a variable chart branch count, not the full orthogonal stabilizer size. -/
def branchCount {k : ℕ} (Q : Matrix (Fin k) (Fin k) ℝ) : ℕ :=
  (admissibleSigns Q).card

@[simp] theorem mem_admissibleSigns {k : ℕ}
    (Q : Matrix (Fin k) (Fin k) ℝ) (b : Fin k → Bool) :
    b ∈ admissibleSigns Q ↔
      IsUnit (Q * Matrix.diagonal (signVector b) + 1).det := by
  classical
  simp [admissibleSigns]

abbrev regularEigenframes {k : ℕ} (s : Fin k → ℝ)
    (Q : Matrix (Fin k) (Fin k) ℝ) :=
  {R : Matrix (Fin k) (Fin k) ℝ // R.transpose * R = 1 ∧
    R * Matrix.diagonal s * R.transpose =
      Q * Matrix.diagonal s * Q.transpose ∧ IsUnit (R + 1).det}

abbrev cayleySpectralFiber {k : ℕ} (s : Fin k → ℝ)
    (Q : Matrix (Fin k) (Fin k) ℝ) :=
  {K : Matrix (Fin k) (Fin k) ℝ // K.transpose = -K ∧
    cayley K * Matrix.diagonal s * (cayley K).transpose =
      Q * Matrix.diagonal s * Q.transpose}
```

From signs to frames. Every sign matrix is orthogonal and commutes with $\mathrm{diag}(s)$. Consequently QD_b is orthogonal when Q is, and gives the same conjugate of $\mathrm{diag}(s)$. The two statements are proved simultaneously by reassociating products and applying the two diagonal-sign identities.

```
theorem signed_frame_properties {k : ℕ} (s : Fin k → ℝ)
    (Q : Matrix (Fin k) (Fin k) ℝ) (hQ : Q.transpose * Q = 1)
    (b : Fin k → Bool) :
    (Q * Matrix.diagonal (signVector b)).transpose *
        (Q * Matrix.diagonal (signVector b)) = 1 ∧
      (Q * Matrix.diagonal (signVector b)) * Matrix.diagonal s *
        (Q * Matrix.diagonal (signVector b)).transpose =
          Q * Matrix.diagonal s * Q.transpose := by
  let D := Matrix.diagonal (signVector b)
  have hD := sign_diagonal_properties s (signVector b) (signVector_signs b)
  change (Q * D).transpose * (Q * D) = 1 ∧
    (Q * D) * Matrix.diagonal s * (Q * D).transpose =
      Q * Matrix.diagonal s * Q.transpose
```

```
  constructor
  · rw [Matrix.transpose_mul]
    calc
      (D.transpose * Q.transpose) * (Q * D) =
          D.transpose * (Q.transpose * Q) * D := by noncomm_ring
      _ = 1 := by rw [hQ, mul_one]; exact hD.1
  · rw [Matrix.transpose_mul]
    calc
      (Q * D) * Matrix.diagonal s * (D.transpose * Q.transpose) =
          Q * (D * Matrix.diagonal s * D.transpose) * Q.transpose := by
            noncomm_ring
      _ = Q * Matrix.diagonal s * Q.transpose := by rw [hD.2]
```

A bijection with regular eigenframes. Assume now that s is injective and $Q^TQ = I$. The map $b \mapsto QD_b$ sends $\mathcal{A}(Q)$ to regular eigenframes. Injectivity follows by left multiplication by Q^T, followed by injectivity of the diagonal-matrix and Boolean-sign maps. For surjectivity, `eigenframes_differ_by_signs` says that two orthogonal frames for the same injectively indexed spectrum differ by a unique diagonal sign matrix. The regularity hypothesis on the given frame says exactly that this sign belongs to $\mathcal{A}(Q)$.

```
/-- The finite sign list is in bijection with the actual regular eigenframes,
including their orthogonality and their full matrix-product equation. -/
def admissibleSignsEquivRegularFrames {k : ℕ} (s : Fin k → ℝ)
    (hs : Function.Injective s) (Q : Matrix (Fin k) (Fin k) ℝ)
    (hQ : Q.transpose * Q = 1) :
    {b : Fin k → Bool // b ∈ admissibleSigns Q} ≃ regularEigenframes s Q := by
  refine Equiv.ofBijective (fun b =>
    (Q * Matrix.diagonal (signVector b.val),
      (signed_frame_properties s Q hQ b.val).1,
      (signed_frame_properties s Q hQ b.val).2,
      (mem_admissibleSigns Q b.val).1 b.⟩property) ?_
  constructor
  · intro b c hbc
    apply Subtype.ext
    apply signVector_injective k
    apply Matrix.diagonal_injective
    have h := congrArg
      (fun R : regularEigenframes s Q => Q.transpose * R.val) hbc
    simpa only ←[ Matrix.mul_assoc, hQ, one_mul] using h
  · intro R
    obtain ⟨eps, heps, ⟩hR := eigenframes_differ_by_signs s hs Q R.val
      hQ R.property.1 R.property.2.1
    obtain ⟨b, ⟩hb := exists_signVector eps heps
    refine ⟨⟨b, ?⟩_, Subtype.ext ?⟩_
    · apply (mem_admissibleSigns Q b).2
      rw [hb, ← hR]
      exact R.property.2.2
    · change Q * Matrix.diagonal (signVector b) = R.val
      rw [hb]
      exact hR.symm
```

A bijection with skew-matrix solutions. Independently of distinctness of s, the maps $K \mapsto C(K)$ and $R \mapsto 2(R - I)(R + I)^{-1}$ restrict to inverse maps between the skew fiber and the regular-frame carrier. Skewness, orthogonality, and both inverse equations come from the previously proved Cayley identities. Composing this equivalence with the preceding one gives an equivalence with $\mathcal{A}(Q)$, hence the two cardinality identities. The use of `Nat.card` occurs after an equivalence to an explicitly finite set, so it does not conceal the totalized cardinality

of an infinite fiber.

```
/-- Restricting the two explicit Cayley inverse identities gives an equivalence
of actual skew-matrix fibers with regular eigenframes. Distinctness is not used here. -/
def cayleyFiberEquivRegularFrames {k : ℕ} (s : Fin k → ℝ)
    (Q : Matrix (Fin k) (Fin k) ℝ) :
    cayleySpectralFiber s Q ≃ regularEigenframes s Q where
  toFun K := ⟨cayley K.val,
    transpose_cayley_mul_cayley K.val K.property.1,
    K.property.2, isUnit_det_cayley_add_one K.val K.property⟩.1
  invFun R := ⟨inverseCayley R.val,
    transpose_inverseCayley R.val R.property.1 R.property.2.2, by
      rw [cayley_inverseCayley R.val R.property.1 R.property.2.2]
      exact R.property⟩.2.1
  left_inv K := Subtype.ext (inverseCayley_cayley K.val K.property.1)
  right_inv R := Subtype.ext
    (cayley_inverseCayley R.val R.property.1 R.property.2.2)

def cayleyFiberEquivAdmissibleSigns {k : ℕ} (s : Fin k → ℝ)
    (hs : Function.Injective s) (Q : Matrix (Fin k) (Fin k) ℝ)
    (hQ : Q.transpose * Q = 1) :
    cayleySpectralFiber s Q ≃ {b : Fin k → Bool // b ∈ admissibleSigns Q} :=
  (cayleyFiberEquivRegularFrames s Q).trans
    (admissibleSignsEquivRegularFrames s hs Q hQ).symm

theorem card_regularEigenframes {k : ℕ} (s : Fin k → ℝ)
    (hs : Function.Injective s) (Q : Matrix (Fin k) (Fin k) ℝ)
    (hQ : Q.transpose * Q = 1) :
    Nat.card (regularEigenframes s Q) = branchCount Q := by
  rw ←[ Nat.card_congr (admissibleSignsEquivRegularFrames s hs Q hQ)]
  exact Nat.card_eq_finsetCard (admissibleSigns Q)

theorem card_cayleySpectralFiber {k : ℕ} (s : Fin k → ℝ)
    (hs : Function.Injective s) (Q : Matrix (Fin k) (Fin k) ℝ)
    (hQ : Q.transpose * Q = 1) :
    Nat.card (cayleySpectralFiber s Q) = branchCount Q := by
  rw [Nat.card_congr (cayleyFiberEquivRegularFrames s Q)]
  exact card_regularEigenframes s hs Q hQ
```

Bounds and positivity. The inclusion $\mathcal{A}(Q) \subseteq \mathrm{Bool}^k$ gives $N(Q) \le 2^k$. If $Q+I$ is invertible, the all-true vector belongs to $\mathcal{A}(Q)$, so $N(Q) > 0$. In particular every Cayley frame $C(K)$ has positive count. The final adapter applies this to `framea`, where the strictly upper entries of a determine a skew matrix and its Cayley transform.

```
theorem branchCount_le {k : ℕ} (Q : Matrix (Fin k) (Fin k) ℝ) :
    branchCount Q ≤ 2 ^ k := by
  classical
  calc
    branchCount Q ≤ (Finset.univ : Finset (Fin k → Bool)).card :=
      Finset.card_le_card (Finset.subset_univ _)
    _ = 2 ^ k := by simp

theorem branchCount_pos_of_regular {k : ℕ}
    (Q : Matrix (Fin k) (Fin k) ℝ) (hQ : IsUnit (Q + 1).det) :
    0 < branchCount Q := by
  classical
  apply Finset.card_pos.mpr
  refine ⟨fun _ => true, (mem_admissibleSigns Q _).2 ?⟩_
  have hD : Matrix.diagonal (signVector (fun _ : Fin k => true)) = 1 := by
    change Matrix.diagonal (1 : Fin k → ℝ) = 1
    simp
```

```
  rw [hD, Matrix.mul_one]
  exact hQ

theorem branchCount_cayley_pos {k : ℕ}
    (K : Matrix (Fin k) (Fin k) ℝ) (hK : K.transpose = -K) :
    0 < branchCount (cayley K) :=
  branchCount_pos_of_regular _ (isUnit_det_cayley_add_one K hK)

theorem branchCount_frame_pos {k : ℕ} (a : O77.GramBorder.UpperCoordinates k) :
    0 < branchCount (O77.OrthogonalSpectral.frame a) :=
  branchCount_cayley_pos _ (O77.SpectralDifferential.skew_transpose a)
```

What invariance of the count means. For an injective fixed s, if Q and R yield the same conjugated matrix, their sets of regular eigenframes are literally equal. The cardinality theorem therefore gives $N(Q) = N(R)$. Right multiplication by any sign matrix preserves the count; the proof can choose the auxiliary distinct spectrum $s_i = i$ and apply this invariance. Equality of skew coordinates likewise preserves the associated frame and its count. None of these statements says that N is constant over all orthogonal matrices.

```
/-- Equal actual matrices with the same injective indexed spectrum have identical
regular-frame sets. This invariance does not assert that the count is constant in Q. -/
theorem branchCount_eq_of_same_spectral_matrix {k : ℕ} (s : Fin k → ℝ)
    (hs : Function.Injective s) (Q R : Matrix (Fin k) (Fin k) ℝ)
    (hQ : Q.transpose * Q = 1) (hR : R.transpose * R = 1)
    (hQR : Q * Matrix.diagonal s * Q.transpose =
      R * Matrix.diagonal s * R.transpose) :
    branchCount Q = branchCount R := by
  have heq : regularEigenframes s Q = regularEigenframes s R := by
    unfold regularEigenframes
    rw [hQR]
  rw ←[ card_regularEigenframes s hs Q hQ,
    ← card_regularEigenframes s hs R hR, heq]

theorem branchCount_right_sign {k : ℕ}
    (Q : Matrix (Fin k) (Fin k) ℝ) (hQ : Q.transpose * Q = 1)
    (b : Fin k → Bool) :
    branchCount (Q * Matrix.diagonal (signVector b)) = branchCount Q := by
  let s : Fin k → ℝ := fun i => (i.val : ℝ)
  have hs : Function.Injective s := by
    intro i j hij
    apply Fin.ext
    change (i.val : ℝ) = (j.val : ℝ) at hij
    exact_mod_cast hij
  have hsign := signed_frame_properties s Q hQ b
  exact branchCount_eq_of_same_spectral_matrix s hs _ Q hsign.1 hQ hsign.2

theorem branchCount_frame_eq_of_skew_eq {k : ℕ}
    (a b : O77.GramBorder.UpperCoordinates k)
    (hab : O77.SpectralDifferential.skew a = O77.SpectralDifferential.skew b) :
    branchCount (O77.OrthogonalSpectral.frame a) =
      branchCount (O77.OrthogonalSpectral.frame b) := by
  unfold O77.OrthogonalSpectral.frame
  rw [hab]

open scoped Classical in
```

Measurability without choosing eigenvectors. Write $N(Q)$ as the finite sum of the indicators of $\det(QD_b + I) \neq 0$. Each determinant is continuous in the matrix entries, so

its nonzero locus is open and measurable. The indicator sum is therefore measurable as a natural-number-valued function. This proof avoids measurable selection of an eigenbasis entirely; it is the measurability needed for reciprocal weighting later.

```
theorem branchCount_eq_sum {k : ℕ} (Q : Matrix (Fin k) (Fin k) ℝ) :
    branchCount Q = ∑ b : Fin k → Bool,
      if IsUnit (Q * Matrix.diagonal (signVector b) + 1).det then 1 else 0 := by
  classical
  change (admissibleSigns Q).card = _
  simpa only [mem_admissibleSigns] using
    (Finset.card_eq_sum_ite (Finset.subset_univ (admissibleSigns Q)))

/-- Measurability is on matrix-entry coordinates. No measurable eigenbasis
selection and no constancy of the branch count is assumed. -/
theorem measurable_branchCount (k : ℕ) :
    Measurable (branchCount (k := k)) := by
  classical
  change Measurable (fun Q : Matrix (Fin k) (Fin k) ℝ => branchCount Q)
  simp_rw [branchCount_eq_sum]
  apply Finset.measurable_sum
  intro b _
  have hc : Continuous (fun Q : Matrix (Fin k) (Fin k) ℝ =>
      (Q * Matrix.diagonal (signVector b) + 1).det) :=
    ((continuous_id.mul continuous_const).add continuous_const).matrix_det
  have hm : MeasurableSet {Q : Matrix (Fin k) (Fin k) ℝ |
      IsUnit (Q * Matrix.diagonal (signVector b) + 1).det} := by
    have hset : {Q : Matrix (Fin k) (Fin k) ℝ |
        IsUnit (Q * Matrix.diagonal (signVector b) + 1).det} =
        (fun Q : Matrix (Fin k) (Fin k) ℝ =>
          (Q * Matrix.diagonal (signVector b) + 1).det) ⁻¹' {x : ℝ | x ≠ 0} :=
      Set.ext (fun _ => isUnit_iff_ne_zero)
    rw [hset]
    exact ((isOpen_ne : IsOpen {x : ℝ | x ≠ 0}).preimage hc).measurableSet
  exact Measurable.ite hm measurable_const measurable_const

end O77.CayleyFibers
end
```

76 Separating eigenvalue and angular entry coordinates

Continue to use $E_k = \mathbb{R}^{\{(i,j):0\le i\le j<k\}}$ for the independent upper-entry coordinates of a symmetric matrix. Let $\mathcal{K}_k = \mathbb{R}^{\{(i,j):0\le i<j<k\}}$ be the independent angular coordinates. Both carry product Lebesgue measure in these entries. Write $s(a)_i = a_{ii}$ for `diagonalVelocitya`, $z(a)_{ij} = a_{ij}$ for $i < j$, and $K(a)$ for the skew matrix with those strictly upper entries. Our angular density is

$$j(a) = \left|\det(I - K(a)/2)^{-1}\right|^{k-1}.$$

Here and below the exponent at $k = 0$ is the natural number $k - 1 = 0$. The point of the module is an exact product-measure decomposition, not an appeal to an unspecified Euclidean volume on symmetric matrices.

```
import Mathlib
import O77SpectralChartIntegral

set_option autoImplicit false
noncomputable section
open Set MeasureTheory
open scoped ENNReal
```

```
namespace O77.SpectralSplit
open O77.Residual O77.GramBorder O77.SpectralDifferential
open O77.OrthogonalCayley O77.SpectralChartIntegral
```

The coordinate permutation. Every upper index is uniquely diagonal or strictly upper. The equivalence `upperIndexSplit` sends i to (i, i) and a strict upper pair to itself; its inverse separates the cases $i = j$ and $i < j$. Combining this equivalence with the function-space equivalence for a disjoint union gives

$$\mathrm{join} : \mathbb{R}^k \times \mathcal{K}_k \longrightarrow E_k.$$

It inserts s on the diagonal and z above it. The library results `measurePreserving_piCongrLeft` and `measurePreserving_sumPiEquivProdPi_symm` prove preservation of the specified product measures. There is no factor $\sqrt{2}$ on off-diagonal entries: these are independent entry measures, not Frobenius-normalized volume on the symmetric subspace.

```
/-- Independent strict upper entries, without Frobenius rescaling. -/
abbrev StrictUpperIndex (k : ℕ) :=
  {ij : Fin k × Fin k // ij.1 < ij.2}

abbrev AngularCoordinates (k : ℕ) := StrictUpperIndex k → ℝ

def angularLebesgue (k : ℕ) : Measure (AngularCoordinates k) :=
  coordinateLebesgue (StrictUpperIndex k)

def angularEntries {k : ℕ} (a : UpperCoordinates k) : AngularCoordinates k :=
  fun ij => a ⟨ij.1, ij.2.⟩le

/-- Each independent symmetric entry is either diagonal or strictly upper. -/
def upperIndexSplit (k : ℕ) : Fin k ⊕ StrictUpperIndex k ≃ UpperIndex k where
  toFun
    | Sum.inl i => ⟨(i, i), ⟩le_rfl
    | Sum.inr ij => ⟨ij.1, ij.2.⟩le
  invFun ij :=
    if h : ij.1.1 = ij.1.2 then Sum.inl ij.1.1
    else Sum.inr ⟨ij.1, lt_of_le_of_ne ij.2 ⟩h
  left_inv x := by
    rcases x with i | ij
    · simp
    · simp [ij.2.ne]
  right_inv ij := by
    rcases ij with ⟨⟨i, ⟩j, ⟩hij
    dsimp
    split_ifs with heq
    · subst j
      rfl
    · rfl

/-- Joining the two families is an exact finite-coordinate permutation. -/
def join (k : ℕ) :
    ((Fin k → ℝ) × AngularCoordinates k) ≃ᵐ UpperCoordinates k :=
  (MeasurableEquiv.sumPiEquivProdPi
    (fun _ : Fin k ⊕ StrictUpperIndex k => ℝ)).symm.trans
      (MeasurableEquiv.piCongrLeft (fun _ : UpperIndex k => ℝ)
        (upperIndexSplit k))

theorem join_preserves (k : ℕ) :
    MeasurePreserving (join k)
      ((coordinateLebesgue (Fin k)).prod (angularLebesgue k))
      (upperLebesgue k) := by
```

```
  exact (measurePreserving_piCongrLeft
      (fun _ : UpperIndex k => (volume : Measure ℝ)) (upperIndexSplit k)).comp
    (measurePreserving_sumPiEquivProdPi_symm
      (fun _ : Fin k ⊕ StrictUpperIndex k => (volume : Measure ℝ)))
```

Evaluation and reconstruction. The dependent-index evaluation lemma for `MeasurableEquiv.piCongrLeft` gives both entry formulas for $\mathrm{join}(s, z)$. Thus $s(\mathrm{join}(s, z)) = s$ and $z(\mathrm{join}(s, z)) = z$. Checking diagonal and strict upper indices separately proves $\mathrm{join}(s(a), z(a)) = a$ and identifies the inverse map. The product-argument variants expose these identities before Tonelli splits a product variable into components.

```
/-- Evaluation at an image index uses the dependent-coordinate adapter
explicitly. No definitional identification with precomposition is needed. -/
theorem join_apply_index {k : ℕ} (s : Fin k → ℝ)
    (z : AngularCoordinates k) (x : Fin k ⊕ StrictUpperIndex k) :
    join k (s, z) (upperIndexSplit k x) = Sum.elim s z x := by
  change (MeasurableEquiv.piCongrLeft (fun _ : UpperIndex k => ℝ)
    (upperIndexSplit k))
      ((MeasurableEquiv.sumPiEquivProdPi
        (fun _ : Fin k ⊕ StrictUpperIndex k => ℝ)).symm (s, z))
      (upperIndexSplit k x) = Sum.elim s z x
  exact (MeasurableEquiv.piCongrLeft_apply_apply
    β( := fun _ : UpperIndex k => ℝ) (upperIndexSplit k)
    ((MeasurableEquiv.sumPiEquivProdPi
      (fun _ : Fin k ⊕ StrictUpperIndex k => ℝ)).symm (s, z)) x).trans
    (by cases x <;> rfl)

@[simp] theorem join_diagonal {k : ℕ} (s : Fin k → ℝ)
    (z : AngularCoordinates k) (i : Fin k) :
    join k (s, z) ⟨(i, i), ⟩le_rfl = s i := by
  exact join_apply_index s z (Sum.inl i)

@[simp] theorem join_strictUpper {k : ℕ} (s : Fin k → ℝ)
    (z : AngularCoordinates k) (ij : StrictUpperIndex k) :
    join k (s, z) ⟨ij.1, ij.2.⟩le = z ij := by
  exact join_apply_index s z (Sum.inr ij)

@[simp] theorem diagonalVelocity_join {k : ℕ} (s : Fin k → ℝ)
    (z : AngularCoordinates k) : diagonalVelocity (join k (s, z)) = s := by
  funext i
  exact join_diagonal s z i

@[simp] theorem angularEntries_join {k : ℕ} (s : Fin k → ℝ)
    (z : AngularCoordinates k) : angularEntries (join k (s, z)) = z := by
  funext ij
  exact join_strictUpper s z ij

/-- Product arguments occur before Tonelli has introduced their components.
These adapters expose both entry families without unfolding the permutation. -/
theorem diagonalVelocity_join_prod {k : ℕ}
    (w : (Fin k → ℝ) × AngularCoordinates k) :
    diagonalVelocity (join k w) = w.1 := by
  rcases w with ⟨s, ⟩z
  exact diagonalVelocity_join s z

theorem angularEntries_join_prod {k : ℕ}
    (w : (Fin k → ℝ) × AngularCoordinates k) :
    angularEntries (join k w) = w.2 := by
  rcases w with ⟨s, ⟩z
  exact angularEntries_join s z
```

```
@[simp] theorem join_split {k : ℕ} (a : UpperCoordinates k) :
    join k (diagonalVelocity a, angularEntries a) = a := by
  funext ij
  rcases ij with ⟨⟨i, ⟩j, ⟩hij
  by_cases heq : i = j
  · subst j
    exact join_diagonal _ _ i
  · exact join_strictUpper _ _ ⟨(i, j), lt_of_le_of_ne hij ⟩heq

@[simp] theorem join_symm_apply {k : ℕ} (a : UpperCoordinates k) :
    (join k).symm a = (diagonalVelocity a, angularEntries a) := by
  apply (join k).injective
  change (join k).toEquiv ((join k).toEquiv.symm a) =
    join k (diagonalVelocity a, angularEntries a)
  rw [Equiv.apply_symm_apply, join_split]
```

Linear and topological versions of the same map. The same finite coordinate permutation is a linear equivalence; finite dimensionality gives its continuous linear equivalence. The displayed equality lemmas identify these maps with the already measure-preserving `join`; a separate coordinate change is not introduced. Since $K(a)$ ignores diagonal entries, $K(\mathrm{join}(s,z)) = K(\mathrm{join}(t,z))$ for all s,t. The angular density therefore has the same independence from s.

```
/-- The same entry map as a linear equivalence; continuity does not assert
that an ambient matrix norm is Frobenius. -/
def joinLinearEquiv (k : ℕ) :
    ((Fin k → ℝ) × AngularCoordinates k) ≃ₗ[ℝ] UpperCoordinates k :=
  (LinearEquiv.sumArrowLequivProdArrow
    (Fin k) (StrictUpperIndex k) ℝ ℝ).symm.trans
      (LinearEquiv.piCongrLeft ℝ (fun _ : UpperIndex k => ℝ)
        (upperIndexSplit k))

@[simp] theorem joinLinearEquiv_apply {k : ℕ}
    (w : (Fin k → ℝ) × AngularCoordinates k) :
    joinLinearEquiv k w = join k w := rfl

def joinContinuousLinearEquiv (k : ℕ) :
    ((Fin k → ℝ) × AngularCoordinates k) ≃L[ℝ] UpperCoordinates k :=
  (joinLinearEquiv k).toContinuousLinearEquiv

@[simp] theorem joinContinuousLinearEquiv_apply {k : ℕ}
    (w : (Fin k → ℝ) × AngularCoordinates k) :
    joinContinuousLinearEquiv k w = join k w := rfl

theorem skew_join {k : ℕ} (s t : Fin k → ℝ) (z : AngularCoordinates k) :
    skew (join k (s, z)) = skew (join k (t, z)) := by
  ext i j
  unfold skew
  split_ifs with hij hji
  · exact (join_strictUpper s z ⟨(i, j), ⟩hij).trans
      (join_strictUpper t z ⟨(i, j), ⟩hij).symm
  · rw [join_strictUpper s z ⟨(j, i), ⟩hji,
      join_strictUpper t z ⟨(j, i), ⟩hji]
  · rfl

theorem angularDensity_join {k : ℕ} (s : Fin k → ℝ)
    (z : AngularCoordinates k) :
    angularDensity (join k (s, z)) = angularDensity (join k (0, z)) :=
  angularDensity_eq_of_skew_eq (skew_join s 0 z)
```

Zero angular dimension. If $k \leq 1$, no indices satisfy $i < j$. The angular space is then a singleton, its product Lebesgue measure has total mass one by `Measure.pi_empty_univ`, and the exponent $k - 1$ is zero. Thus the uncorrected angular density and its integral are both one. Empty coordinate spaces are not assigned zero mass.

```
theorem strictUpperIndex_isEmpty (k : ℕ) (hk : k ≤ 1) :
    IsEmpty (StrictUpperIndex k) := by
  constructor
  intro ij
  have hi := ij.1.1.isLt
  have hj := ij.1.2.isLt
  have hij := ij.2
  omega

theorem angularLebesgue_univ_of_le_one (k : ℕ) (hk : k ≤ 1) :
    angularLebesgue k Set.univ = 1 := by
  let := strictUpperIndex_isEmpty k hk
  exact Measure.pi_empty_univ (fun _ : StrictUpperIndex k => (volume : Measure ℝ))

theorem angularDensity_eq_one_of_le_one {k : ℕ} (hk : k ≤ 1)
    (a : UpperCoordinates k) : angularDensity a = 1 := by
  have he : k - 1 = 0 := by omega
  simp [angularDensity, he]

theorem angular_mass_of_le_one (k : ℕ) (hk : k ≤ 1) :
    ∫⁻ z, angularDensity (join k (0, z)) ∂angularLebesgue k) = 1 := by
  simp_rw [angularDensity_eq_one_of_le_one hk]
  simp [angularLebesgue_univ_of_le_one k hk]
```

Measurability of the entries and density. Coordinate projections give measurability of $s(a)$ and $z(a)$. The matrix $I - K(a)/2$ is continuous entrywise. To establish measurability of its inverse without prematurely using nonsingularity, the proof uses the library adjugate formula for the total matrix inverse. Determinant, absolute value, natural power, and `ENNReal.ofReal` then give measurability of j. Its stronger continuity is established later using the everywhere nonzero determinant on skew matrices.

```
theorem measurable_angularEntries (k : ℕ) : Measurable (@angularEntries k) := by
  apply Measurable.of_eval
  intro ij
  exact measurable_pi_apply _

theorem measurable_diagonalVelocity (k : ℕ) : Measurable (@diagonalVelocity k) := by
  apply Measurable.of_eval
  intro i
  exact measurable_pi_apply _

/-- The same matrix-valued continuous denominator is used by both the
measurable and continuous angular-density proofs. -/
theorem continuous_minusHalf_skew (k : ℕ) :
    Continuous (fun a : UpperCoordinates k => minusHalf (skew a)) := by
  have hsk : Continuous (@skew k) := by
    apply continuous_pi
    intro i
    apply continuous_pi
    intro j
    unfold skew
    split_ifs <;> fun_prop
  change Continuous (fun a : UpperCoordinates k =>
    (1 : Matrix (Fin k) (Fin k) ℝ) - (1 / 2 : ℝ) • skew a)
  exact (continuous_const : Continuous (fun _ : UpperCoordinates k =>
```

```
    (1 : Matrix (Fin k) (Fin k) ℝ))).fun_sub
      ((continuous_const : Continuous (fun _ : UpperCoordinates k =>
        (1 / 2 : ℝ))).fun_smul hsk)

theorem measurable_angularDensity (k : ℕ) : Measurable (@angularDensity k) := by
  have hm := continuous_minusHalf_skew k
  have hi : Measurable (fun a : UpperCoordinates k => (minusHalf (skew a))⁻¹) := by
    have hinv : Measurable (fun a : UpperCoordinates k =>
        (minusHalf (skew a)).det⁻¹ • (minusHalf (skew a)).adjugate) :=
      hm.matrix_det.measurable.fun_inv.fun_smul hm.matrix_adjugate.measurable
    simpa only [Matrix.inv_def, Ring.inverse_eq_inv'] using hinv
  have hd : Measurable (fun a : UpperCoordinates k => ((minusHalf (skew a))⁻¹).det) := by
    simpa only [Function.comp_def] using
      (show Measurable (fun M : Matrix (Fin k) (Fin k) ℝ => M.det) from
        continuous_id.matrix_det.measurable).comp hi
  change Measurable (fun a : UpperCoordinates k =>
    ENNReal.ofReal (|((minusHalf (skew a))⁻¹).det| ^ (k - 1)))
  exact (hd.abs.pow_const (k - 1)).ennreal_ofReal
```

Transport on an arbitrary radial window. For $S \subseteq \mathbb{R}^k$, the preimage of $s^{-1}(S)$ under join is exactly $S \times \mathcal{K}_k$. The measurable-equivalence change-of-variables theorem therefore gives, for every nonnegative extended-valued test H,

$$\int_{s(a)\in S} H(a)\, da = \int_{S\times\mathcal{K}_k} H(\mathrm{join}(s,z))\, ds\, dz.$$

The equality itself needs no integrability or finiteness hypothesis.

```
/-- Every nonnegative test is transported by the measurable equivalence;
no integrability or finiteness convention enters this equality. -/
theorem lintegral_radial_preimage {k : ℕ} (S : Set (Fin k → ℝ))
    (H : UpperCoordinates k → ℝ≥∞0) :
    ∫⁻ a in diagonalVelocity ⁻¹' S, H a ∂upperLebesgue k) =
      ∫⁻ w in S ×ˢ Set.univ, H (join k w)
        ∂((coordinateLebesgue (Fin k)).prod (angularLebesgue k)) := by
  have hp : join k ⁻¹' (diagonalVelocity ⁻¹' S) = S ×ˢ Set.univ := by
    ext w
    simp only [Set.mem_preimage, diagonalVelocity_join_prod,
      Set.mem_prod, Set.mem_univ, and_true]
  have h := (join_preserves k).setLIntegral_comp_preimage_emb
    (join k).measurableEmbedding H (diagonalVelocity ⁻¹' S)
  rw [hp] at h
  exact h.symm
```

Tonelli separation. For measurable nonnegative extended-valued W and F, Tonelli gives

$$\int_{s(a)\in S} W(z(a))F(s(a))\, da = \left(\int_{\mathcal{K}_k} W(z)\, dz\right)\left(\int_S F(s)\, ds\right).$$

The proof supplies sigma-finiteness of the two finite product Lebesgue measures and invokes `lintegral_prod_mul`; the underlying product integration source is cited in [15]. Replacing $W(z)$ by $j(\mathrm{join}(0,z))W(z)$ proves the angular-density variant. This step uses the established independence of j from the diagonal, not an informal claim that a matrix space is already a product.

```
/-- Exact nonnegative Fubini separation on any radial window.
Both sides remain meaningful if one or both factors are infinite. -/
theorem lintegral_separated {k : ℕ} (S : Set (Fin k → ℝ))
    (W : AngularCoordinates k → ℝ≥∞0)
```

```
    (F : (Fin k → ℝ) → ℝ≥∞0) (hW : Measurable W) (hF : Measurable F) :
    ∫⁻( a in diagonalVelocity ⁻¹' S,
      W (angularEntries a) * F (diagonalVelocity a) ∂upperLebesgue k) =
      ∫⁻( z, W z ∂angularLebesgue k) *
        ∫⁻ s in S, F s ∂coordinateLebesgue (Fin k) := by
  let : SigmaFinite (angularLebesgue k) := by
    unfold angularLebesgue
    infer_instance
  let : SigmaFinite (coordinateLebesgue (Fin k)) := by
    unfold coordinateLebesgue
    infer_instance
  rw [lintegral_radial_preimage S]
  simp only [angularEntries_join_prod, diagonalVelocity_join_prod]
  rw ←[ Measure.restrict_prod_eq_prod_univ]
  simpa only [mul_comm] using
    (lintegral_prod_mul μ( := (coordinateLebesgue (Fin k)).restrict S)
      ν( := angularLebesgue k) hF.aemeasurable hW.aemeasurable)

/-- Inserting the actual angular Jacobian preserves the separation. The
extra weight W can later be a measurable reciprocal branch count. -/
theorem lintegral_angular_separated {k : ℕ} (S : Set (Fin k → ℝ))
    (W : AngularCoordinates k → ℝ≥∞0)
    (F : (Fin k → ℝ) → ℝ≥∞0) (hW : Measurable W) (hF : Measurable F) :
    ∫⁻( a in diagonalVelocity ⁻¹' S,
      (angularDensity a * W (angularEntries a)) * F (diagonalVelocity a)
      ∂upperLebesgue k) =
      ∫⁻( z, angularDensity (join k (0, z)) * W z ∂angularLebesgue k) *
        ∫⁻ s in S, F s ∂coordinateLebesgue (Fin k) := by
  have hj : Measurable (fun z : AngularCoordinates k => join k (0, z)) := by
    simpa only [Function.comp_def, id_eq] using
      (join k).measurable.comp (measurable_const.prodMk measurable_id)
  have hA : Measurable (fun z : AngularCoordinates k =>
      angularDensity (join k (0, z)) * W z) := by
    simpa only [Function.comp_def] using
      ((measurable_angularDensity k).comp hj).fun_mul hW
  have heq (a : UpperCoordinates k) :
      angularDensity a = angularDensity (join k (0, angularEntries a)) := by
    calc
      _ = angularDensity (join k (diagonalVelocity a, angularEntries a)) := by
        rw [join_split]
      _ = _ := angularDensity_join _ _
  calc
    _ = ∫⁻ a in diagonalVelocity ⁻¹' S,
        (angularDensity (join k (0, angularEntries a)) * W (angularEntries a)) *
          F (diagonalVelocity a) ∂upperLebesgue k := by
      apply lintegral_congr
      intro a
      exact congrArg (fun c : ℝ≥∞0 =>
        (c * W (angularEntries a)) * F (diagonalVelocity a)) (heq a)
    _ = _ := lintegral_separated S
      (fun z => angularDensity (join k (0, z)) * W z) F hA hF
```

Adapter for the actual sorted extension. Let $S_k = \{0 < s_1 < \cdots < s_k\}$ and let $L_{\eta,\rho,T}$ denote exactly `sortedLaguerreIntegrand`, the inherited permutation-invariant sorted extension. On S_k it equals the measurable chamber expression by `sortedLaguerreIntegrand_eq_on_orderedPositiveChamber`. Replace both restricted integrands by this chamber expression and apply the previous separation theorem. This proves the displayed identity for all real η, ρ, T: no power identity outside the positive chamber and no finiteness assertion is used in this adapter.

```
/-- The inherited sorted extension is used literally. Equality with the
```

```
measurable chamber formula is applied only on the positive ordered chamber;
eta, rho and T are arbitrary here because no power identities are used. -/
theorem lintegral_ordered_angular_sorted {k : ℕ}
    (W : AngularCoordinates k → ℝ≥∞0) (hW : Measurable W) (eta rho T : ℝ) :
    ∫⁻( a in diagonalVelocity ⁻¹' strictOrderedPositive k,
      (angularDensity a * W (angularEntries a)) *
        sortedLaguerreIntegrand eta rho T (diagonalVelocity a)
      ∂upperLebesgue k) =
      ∫⁻( z, angularDensity (join k (0, z)) * W z ∂angularLebesgue k) *
        ∫⁻ s in strictOrderedPositive k,
          sortedLaguerreIntegrand eta rho T s ∂coordinateLebesgue (Fin k) := by
  have hS := measurableSet_strictOrderedPositive k
  have heq (s : Fin k → ℝ) (hs : s ∈ strictOrderedPositive k) :
      sortedLaguerreIntegrand eta rho T s =
        ENNReal.ofReal (laguerreChamberIntegrand eta rho T s) :=
    sortedLaguerreIntegrand_eq_on_orderedPositiveChamber eta rho T
      (strictOrderedPositive_subset_orderedPositiveChamber k hs)
  have hleft :
      ∫⁻( a in diagonalVelocity ⁻¹' strictOrderedPositive k,
        (angularDensity a * W (angularEntries a)) *
          sortedLaguerreIntegrand eta rho T (diagonalVelocity a)
        ∂upperLebesgue k) =
      ∫⁻ a in diagonalVelocity ⁻¹' strictOrderedPositive k,
        (angularDensity a * W (angularEntries a)) *
          ENNReal.ofReal (laguerreChamberIntegrand eta rho T (diagonalVelocity a))
        ∂upperLebesgue k := by
    apply setLIntegral_congr_fun (hS.preimage (measurable_diagonalVelocity k))
    intro a ha
    change (angularDensity a * W (angularEntries a)) *
        sortedLaguerreIntegrand eta rho T (diagonalVelocity a) =
      (angularDensity a * W (angularEntries a)) *
        ENNReal.ofReal (laguerreChamberIntegrand eta rho T (diagonalVelocity a))
    exact congrArg ((angularDensity a * W (angularEntries a)) * ·) (heq _ ha)
  have hright :
      ∫⁻( s in strictOrderedPositive k,
        sortedLaguerreIntegrand eta rho T s ∂coordinateLebesgue (Fin k)) =
      ∫⁻ s in strictOrderedPositive k,
        ENNReal.ofReal (laguerreChamberIntegrand eta rho T s)
        ∂coordinateLebesgue (Fin k) :=
    setLIntegral_congr_fun hS heq
  rw [hleft, hright]
  exact lintegral_angular_separated (strictOrderedPositive k) W
    (fun s => ENNReal.ofReal (laguerreChamberIntegrand eta rho T s)) hW
    (measurable_laguerreChamberIntegrand eta rho T).ennreal_ofReal

end O77.SpectralSplit
end
```

77 Ordered spectra and the full-orthant factor

Use independent upper coordinates $b \in E_k$ and write $S(b)$ for the associated real symmetric matrix, `fromUpperb`. The positive ordered chamber is $S_k = \{s : 0 < s_1 < \cdots < s_k\}$. This module supplies the order rigidity that fixes eigenvalues in an actual fiber, constructs ordered eigenframes from the spectral theorem, and converts the ordered integral to the full orthant integral.

```
import Mathlib
import O77SpectralCoverage

set_option autoImplicit false
```

```
noncomputable section
open Set MeasureTheory Matrix
open scoped ENNReal BigOperators

namespace O77.SpectralOrder
open O77.Residual O77.GramBorder O77.SpectralExceptional O77.SpectralCoverage
```

Recovering the spectrum from the actual matrix. If $Q^TQ = I$, then QGQ^T and G have the same characteristic polynomial; the code proves this using `Matrix.charpoly_mul_comm`. For a diagonal matrix its characteristic polynomial is $\prod_i (X - s_i)$, whose real root set is exactly the range of s. Consequently two strictly increasing vectors whose orthogonal conjugates are equal have equal ranges and hence are equal, by `StrictMono.eq_of_range_eq`. Both orthogonality assumptions are retained. This identifies eigenvalues from the matrix equation and does not assume the desired equality of the diagonal coordinates.

```
lemma charpoly_orthogonal_conjugate {k : ℕ}
    (Q G : Matrix (Fin k) (Fin k) ℝ) (hQ : Q.transpose * Q = 1) :
    (Q * G * Q.transpose).charpoly = G.charpoly := by
  rw [Matrix.charpoly_mul_comm, ← Matrix.mul_assoc, hQ, Matrix.one_mul]

lemma isRoot_charpoly_diagonal_iff {k : ℕ} (s : Fin k → ℝ) (x : ℝ) :
    (Matrix.diagonal s).charpoly.IsRoot x ↔ x ∈ Set.range s := by
  classical
  simp only [Matrix.charpoly_diagonal, Polynomial.IsRoot, Polynomial.eval_prod,
    Polynomial.eval_sub, Polynomial.eval_X, Polynomial.eval_C,
    Finset.prod_eq_zero_iff, Finset.mem_univ, true_and, sub_eq_zero, Set.mem_range]
  constructor
  · rintro ⟨i, ⟩hi
    exact ⟨i, hi.⟩symm
  · rintro ⟨i, ⟩hi
    exact ⟨i, hi.⟩symm

lemma range_eq_of_diagonal_charpoly_eq {k : ℕ} (s t : Fin k → ℝ)
    (h : (Matrix.diagonal s).charpoly = (Matrix.diagonal t).charpoly) :
    Set.range s = Set.range t := by
  ext x
  rw ←[ isRoot_charpoly_diagonal_iff, ← isRoot_charpoly_diagonal_iff, h]

/-- Increasing enumeration eliminates every permutation ambiguity. The
library order-rigidity theorem is applied to actual characteristic roots. -/
theorem ordered_spectrum_unique {k : ℕ} (s t : Fin k → ℝ)
    (hs : StrictMono s) (ht : StrictMono t)
    (Q R : Matrix (Fin k) (Fin k) ℝ)
    (hQ : Q.transpose * Q = 1) (hR : R.transpose * R = 1)
    (hG : Q * Matrix.diagonal s * Q.transpose =
      R * Matrix.diagonal t * R.transpose) : s = t := by
  apply hs.eq_of_range_eq ht
  apply range_eq_of_diagonal_charpoly_eq s t
  have h := congrArg Matrix.charpoly hG
  simpa only [charpoly_orthogonal_conjugate Q _ hQ,
    charpoly_orthogonal_conjugate R _ hR] using h
```

Sorting the eigenframe together with its eigenvalues. Reindexing the columns of Q by a permutation σ preserves orthogonality. Reindexing the entries of s by the same permutation preserves $Q\,\mathrm{diag}(s)Q^T$. The two matrix submatrix identities below prove these assertions exactly. For a positive-definite matrix with separable characteristic polynomial, the spectral theorem gives an orthogonal eigenframe and positive eigenvalues. Separability gives injectivity

of their list. `Tuple.sort` makes that list monotone; injectivity makes it strictly monotone. Applying the same sorting permutation to the frame columns produces the stated positive ordered spectral decomposition.

```
lemma permuteColumns_orthogonal {k : ℕ} (Q : Matrix (Fin k) (Fin k) ℝ)
    (hQ : Q.transpose * Q = 1) σ( : Equiv.Perm (Fin k)) :
    (Q.submatrix id σ).transpose * Q.submatrix id σ = 1 := by
  change Q.transpose.submatrix σ id * Q.submatrix id σ = 1
  have h := Matrix.submatrix_mul_equiv Q.transpose Q σ (Equiv.refl (Fin k)) σ
  simpa only [Equiv.coe_refl, hQ, Matrix.submatrix_one_equiv] using h

lemma permuteColumns_conjugation {k : ℕ}
    (Q : Matrix (Fin k) (Fin k) ℝ) (s : Fin k → ℝ) σ( : Equiv.Perm (Fin k)) :
    Q.submatrix id σ * Matrix.diagonal (s ∘ σ) * (Q.submatrix id σ).transpose =
      Q * Matrix.diagonal s * Q.transpose := by
  rw ←[ Matrix.submatrix_diagonal_equiv s σ]
  change (Q.submatrix id σ * (Matrix.diagonal s).submatrix σ σ) *
    Q.transpose.submatrix σ id = _
  rw [Matrix.submatrix_mul_equiv, Matrix.submatrix_mul_equiv, Matrix.submatrix_id_id]

/-- Actual existence on each positive-definite simple-spectrum matrix. Both
the eigenvalues and frame come from the pinned spectral theorem, then the
same finite sorting permutation is applied to eigenvalues and frame columns. -/
theorem exists_positive_ordered_frame {k : ℕ} {b : UpperCoordinates k}
    (hpos : (fromUpper b).PosDef) (hsimple : (fromUpper b).charpoly.Separable) :
    ∃ s : Fin k → ℝ, ∃ Q : Matrix (Fin k) (Fin k) ℝ,
      StrictMono s ∧ ∀( i, 0 < s i) ∧ Q.transpose * Q = 1 ∧
        fromUpper b = Q * Matrix.diagonal s * Q.transpose := by
  let v := eigenvalues b
  let σ := Tuple.sort v
  have hv : Function.Injective v :=
    (separable_charpoly_iff_injective_eigenvalues b).1 hsimple
  have hmono : StrictMono (v ∘ σ) :=
    (Tuple.monotone_sort v).strictMono_of_injective (hv.comp σ.injective)
  have hvpos : ∀ i, 0 < v i :=
    (fromUpper_isHermitian b).posDef_iff_eigenvalues_pos.mp hpos
  refine ⟨v ∘ σ, (eigenMatrix b).submatrix id σ, hmono,
    (fun i => hvpos σ( i)),
    permuteColumns_orthogonal (eigenMatrix b) (eigenMatrix_orthogonal b) σ, ?⟩_
  rw [permuteColumns_conjugation]
  exact spectral_decomposition b
```

Why the divisor is $k!$. Write $J_k(\eta, \rho, T)$ for the inherited full nonnegative orthant integral, with its literal sorted extension. The already proved chamber decomposition, `symmetric_integral_chambers_instantiated`, says

$$J_k(\eta, \rho, T) = k! \int_{S_k} L_{\eta,\rho,T}(s)\, ds.$$

It accounts for the null sets of zero and tied coordinates. Since the extended-real natural number $k!$ is positive and finite, it can be cancelled to obtain the inverse-factor identity below even when the integral is infinite. No integral is cancelled. The same statement includes $k = 0$, where $0! = 1$ and there is one empty coordinate vector.

```
/-- The inherited chamber decomposition already handles tied and zero
coordinates almost everywhere. This inverse-factor adapter uses its
nonnegative-integral version, so no restriction on eta, rho or T is needed.
Only the positive finite factorial is cancelled, never an integral. -/
theorem strictSortedLIntegral_eq_factorial_inv_mul (k : ℕ) (eta rho T : ℝ) :
    ∫⁻( s in strictOrderedPositive k, sortedLaguerreIntegrand eta rho T s
```

```
      ∂coordinateLebesgue (Fin k)) =
      (k.factorial : ℝ≥∞0)⁻¹ * laguerreOrthantLIntegral k eta rho T := by
  have hzero : (k.factorial : ℝ≥∞0) ≠ 0 := by
    exact_mod_cast Nat.factorial_ne_zero k
  have htop : (k.factorial : ℝ≥∞0) ≠ ⊤ := by simp
  have hfull : laguerreOrthantLIntegral k eta rho T = (k.factorial : ℝ≥∞0) *
      ∫⁻ s in strictOrderedPositive k, sortedLaguerreIntegrand eta rho T s
        ∂coordinateLebesgue (Fin k) :=
    symmetric_integral_chambers_instantiated
      (sortedLaguerreIntegrand (k := k) eta rho T)
      (sortedLaguerreIntegrand_permutationInvariant eta rho T)
  rw [hfull, ← mul_assoc, ENNReal.inv_mul_cancel hzero htop, one_mul]

end O77.SpectralOrder
end
```

78 The spectral map is locally invertible on its ordered source

For $a \in E_k$, let $s(a)$ be its diagonal, $K(a)$ the skew matrix from its strict upper entries, and $Q(a) = C(K(a))$. Define the actual map $F : E_k \to E_k$ by

$$S(F(a)) = Q(a)\,\mathrm{diag}(s(a))Q(a)^T, \qquad D_k = \{a : s(a) \in S_k\}.$$

These are `spectralMap` and `orderedSource`. The angular coordinates in D_k are unrestricted. We now construct local inverse charts at every point of D_k, which is the input needed for the global counted change of variables.

```
import Mathlib
import O77CayleyDifferential
import O77SpectralLocal

set_option autoImplicit false
noncomputable section
open Set Filter MeasureTheory
open scoped Topology ENNReal

namespace O77.SpectralRegular

open O77.GramBorder O77.SpectralDifferential O77.OrthogonalSpectral
open O77.CayleyDifferential
```

The domain is genuinely open. The map is `orthogonalSpectral` with zero base spectrum, so its diagonal input is the eigenvalue vector itself. Positivity of each diagonal coordinate and the finitely many strict order inequalities define open sets. Their finite intersection is D_k, hence is measurable. The definitions include $k = 0$ and $k = 1$, with the expected vacuous ordering conditions.

```
/-- The actual global Cayley spectral map, in independent upper entries.
Its diagonal input coordinates are the eigenvalues themselves. -/
def spectralMap {k : ℕ} : UpperCoordinates k → UpperCoordinates k :=
  orthogonalSpectral (0 : Fin k → ℝ)

/-- Positive eigenvalues with one strict increasing order. The angular
coordinates are unrestricted. Empty and singleton index sets are retained. -/
def orderedSource (k : ℕ) : Set (UpperCoordinates k) :=
  diagonalVelocity ⁻¹' O77.Residual.strictOrderedPositive k

@[simp] theorem mem_orderedSource {k : ℕ} (a : UpperCoordinates k) :
```

```
    a ∈ orderedSource k ↔
      ∀( i, 0 < diagonalVelocity a i) ∧ StrictMono (diagonalVelocity a) := Iff.rfl

theorem isOpen_orderedSource (k : ℕ) : IsOpen (orderedSource k) := by
  have hdiag (i : Fin k) :
      Continuous (fun a : UpperCoordinates k => diagonalVelocity a i) :=
    continuous_apply (((i, i), )le_rfl : UpperIndex k)
  have hpos : IsOpen {a : UpperCoordinates k | ∀ i, 0 < diagonalVelocity a i} := by
    rw [Set.ofPred_forall]
    apply isOpen_iInter_of_finite
    intro i
    exact isOpen_lt continuous_const (hdiag i)
  have hmono : IsOpen {a : UpperCoordinates k | StrictMono (diagonalVelocity a)} := by
    change IsOpen {a : UpperCoordinates k |
      ∀ i j : Fin k, i < j → diagonalVelocity a i < diagonalVelocity a j}
    rw [Set.ofPred_forall]
    apply isOpen_iInter_of_finite
    intro i
    rw [Set.ofPred_forall]
    apply isOpen_iInter_of_finite
    intro j
    by_cases hij : i < j
    · simpa only [hij, true_implies] using
        (isOpen_lt (hdiag i) (hdiag j))
    · simp only [hij, false_implies, ofPred_true, isOpen_univ]
  exact hpos.inter hmono

theorem measurableSet_orderedSource (k : ℕ) : MeasurableSet (orderedSource k) :=
  (isOpen_orderedSource k).measurableSet
```

The full derivative is nonsingular. The previously established analytic Cayley formula makes F analytic everywhere. Its actual derivative at arbitrary a satisfies

$$|\det DF(a)| = j(a) \prod_{i<j} |s(a)_j - s(a)_i|.$$

On D_k, $j(a) > 0$ and strict order makes every Vandermonde factor nonzero. Thus the determinant of the actual derivative is nonzero. The proof invokes `abs_det_fderiv_orthogonalSpectral_at`; it does not extrapolate the derivative at the origin to an arbitrary angular point.

```
theorem spectralMap_analyticAt {k : ℕ} (a : UpperCoordinates k) :
    AnalyticAt ℝ (@spectralMap k) a :=
  orthogonalSpectral_analyticAt (0 : Fin k → ℝ) a

theorem spectralMap_continuous {k : ℕ} : Continuous (@spectralMap k) :=
  continuous_iff_continuousAt.mpr (fun a => (spectralMap_analyticAt a).continuousAt)

/-- Nonsingularity holds at every point of the ordered source. Its proof
uses the full derivative away from the chart center, not the derivative at zero. -/
theorem det_fderiv_ne_zero {k : ℕ} (a : UpperCoordinates k)
    (ha : a ∈ orderedSource k) : (fderiv ℝ (@spectralMap k) a).det ≠ 0 := by
  apply abs_pos.mp
  change 0 < |LinearMap.det
    (fderiv ℝ (orthogonalSpectral (0 : Fin k → ℝ)) a).toLinearMap|
  rw [abs_det_fderiv_orthogonalSpectral_at]
  simp only [zero_add]
  exact mul_pos (angularFactor_pos a)
    ((absoluteVandermonde_pos_iff (diagonalVelocity a)).mpr ha.2.injective)
```

From the derivative to a local inverse. Finite-dimensional linear algebra turns $DF(a)$ into a continuous linear equivalence using `toContinuousLinearEquivOfDetNeZero`. Analyticity supplies a strict derivative, which is the hypothesis of the library inverse function theorem `HasStrictFDerivAt.toOpenPartialHomeomorph`. The resulting `localChartAt` has forward map exactly F. The following membership, inverse, and injectivity lemmas are the actual properties of that chart; no chart package is assumed by a caller.

```
/-- The inverse differential is constructed from the actual nonzero
determinant, using finite-dimensional linear algebra. -/
def derivativeEquiv {k : ℕ} (a : UpperCoordinates k)
    (ha : a ∈ orderedSource k) :
    UpperCoordinates k ≃L[ℝ] UpperCoordinates k :=
  (fderiv ℝ (@spectralMap k) a).toContinuousLinearEquivOfDetNeZero
    (det_fderiv_ne_zero a ha)

@[simp] theorem derivativeEquiv_toContinuousLinearMap {k : ℕ}
    (a : UpperCoordinates k) (ha : a ∈ orderedSource k) :
    (derivativeEquiv a ha : UpperCoordinates k →L[ℝ] UpperCoordinates k) =
      fderiv ℝ (@spectralMap k) a := by
  simp [derivativeEquiv]

theorem strictDerivative_equiv {k : ℕ} (a : UpperCoordinates k)
    (ha : a ∈ orderedSource k) :
    HasStrictFDerivAt (@spectralMap k)
      (derivativeEquiv a ha : UpperCoordinates k →L[ℝ] UpperCoordinates k) a := by
  simpa only [derivativeEquiv_toContinuousLinearMap] using
    (spectralMap_analyticAt a).hasStrictFDerivAt

/-- A genuine inverse-function chart at every ordered input point. Its
forward map is definitionally the same global spectral map. -/
def localChartAt {k : ℕ} (a : UpperCoordinates k)
    (ha : a ∈ orderedSource k) :
    OpenPartialHomeomorph (UpperCoordinates k) (UpperCoordinates k) :=
  (strictDerivative_equiv a ha).toOpenPartialHomeomorph (@spectralMap k)

@[simp] theorem localChartAt_apply {k : ℕ}
    (a : UpperCoordinates k) (ha : a ∈ orderedSource k) (b : UpperCoordinates k) :
    localChartAt a ha b = spectralMap b := rfl

theorem mem_localChartAt_source {k : ℕ} (a : UpperCoordinates k)
    (ha : a ∈ orderedSource k) : a ∈ (localChartAt a ha).source :=
  (strictDerivative_equiv a ha).mem_toOpenPartialHomeomorph_source

theorem image_mem_localChartAt_target {k : ℕ} (a : UpperCoordinates k)
    (ha : a ∈ orderedSource k) : spectralMap a ∈ (localChartAt a ha).target :=
  (strictDerivative_equiv a ha).image_mem_toOpenPartialHomeomorph_target

theorem localChartAt_left_inverse {k : ℕ}
    (a : UpperCoordinates k) (ha : a ∈ orderedSource k)
    {b : UpperCoordinates k} (hb : b ∈ (localChartAt a ha).source) :
    (localChartAt a ha).symm (spectralMap b) = b :=
  (localChartAt a ha).left_inv hb

theorem localChartAt_right_inverse {k : ℕ}
    (a : UpperCoordinates k) (ha : a ∈ orderedSource k)
    {b : UpperCoordinates k} (hb : b ∈ (localChartAt a ha).target) :
    spectralMap ((localChartAt a ha).symm b) = b :=
  (localChartAt a ha).right_inv hb

theorem spectralMap_injOn_localChartAt_source {k : ℕ}
    (a : UpperCoordinates k) (ha : a ∈ orderedSource k) :
    InjOn (@spectralMap k) (localChartAt a ha).source :=
```

```
  (localChartAt a ha).injOn
```

Keeping the local neighborhood inside the ordered source. Intersect the inverse-chart source with D_k. This open set contains a, lies in D_k, and remains a set of injectivity for F. `exists_injective_open_neighborhood` packages precisely these four properties. The neighborhood depends on a but on no density parameters. Finally positivity of $s(a)$ and orthogonality of $Q(a)$ show that $S(F(a))$ is positive definite; this follows from the already proved positive-definiteness property of `orthogonalSpectral`.

```
/-- A common open source window is chosen from the actual inverse chart
and then intersected with the ordered domain. No density parameters occur. -/
def localWindow {k : ℕ} (a : UpperCoordinates k)
    (ha : a ∈ orderedSource k) : Set (UpperCoordinates k) :=
  (localChartAt a ha).source ∩ orderedSource k

theorem isOpen_localWindow {k : ℕ} (a : UpperCoordinates k)
    (ha : a ∈ orderedSource k) : IsOpen (localWindow a ha) :=
  (localChartAt a ha).open_source.inter (isOpen_orderedSource k)

theorem mem_localWindow {k : ℕ} (a : UpperCoordinates k)
    (ha : a ∈ orderedSource k) : a ∈ localWindow a ha :=
  ⟨mem_localChartAt_source a ha, ha⟩

theorem localWindow_subset {k : ℕ} (a : UpperCoordinates k)
    (ha : a ∈ orderedSource k) : localWindow a ha ⊆ orderedSource k :=
  inter_subset_right

theorem spectralMap_injOn_localWindow {k : ℕ}
    (a : UpperCoordinates k) (ha : a ∈ orderedSource k) :
    InjOn (@spectralMap k) (localWindow a ha) :=
  (spectralMap_injOn_localChartAt_source a ha).mono inter_subset_left

theorem exists_injective_open_neighborhood {k : ℕ}
    (a : UpperCoordinates k) (ha : a ∈ orderedSource k) :
    ∃ U : Set (UpperCoordinates k), IsOpen U ∧ a ∈ U ∧
      U ⊆ orderedSource k ∧ InjOn (@spectralMap k) U :=
  ⟨localWindow a ha, isOpen_localWindow a ha, mem_localWindow a ha,
    localWindow_subset a ha, spectralMap_injOn_localWindow a ha⟩

theorem spectralMap_posDef {k : ℕ} (a : UpperCoordinates k)
    (ha : a ∈ orderedSource k) : (fromUpper (spectralMap a)).PosDef := by
  apply orthogonalSpectral_posDef (0 : Fin k → ℝ) a
  simpa only [Pi.zero_apply, zero_add] using ha.1

end O77.SpectralRegular
end
```

79 A change-of-variables formula with the actual fiber count

This module proves the measure-theoretic tool used in global spectral integration. Initially $f : E \to F$ is an arbitrary map and $D \subseteq E$. Define the extended-valued fiber count

$$m_D(y) = \sum_{x \in D,\ f(x)=y} 1 \in [0, \infty].$$

For the integration statements, take $F = E$, where E is a finite-dimensional real normed vector space, and let μ be an additive Haar measure on E. The same measure is used in source and

target. The argument sums the library's injective Jacobian formula over disjoint source pieces; it never assumes disjoint target images or a constant covering degree.

```
import Mathlib

set_option autoImplicit false
noncomputable section
open Set MeasureTheory
open scoped Topology ENNReal

namespace O77.CountedJacobian
```

Elementary properties of the actual fiber count. `fiberMultiplicity` sums over the subtype whose membership includes both $x \in D$ and $f(x) = y$. It is therefore the extended cardinality of the actual fiber. It vanishes outside $f(D)$, is positive on $f(D)$, and agrees with the usual natural cardinality when that fiber has a `Fintype` instance. In particular, a finite fiber gives $m_D(y) < \infty$. These properties are proved before any inverse of m_D is introduced.

```
/-- The actual fiber, with both domain membership and the product equation. -/
def fiberMultiplicity {E F : Type*} (f : E → F) (D : Set E) (y : F) : ℝ≥∞0 :=
  ∑' _ : {x : E // x ∈ D ∧ f x = y}, (1 : ℝ≥∞0)

theorem fiberMultiplicity_eq_encard {E F : Type*} (f : E → F)
    (D : Set E) (y : F) :
    fiberMultiplicity f D y = ({x | x ∈ D ∧ f x = y} : Set E).encard :=
  ENNReal.tsum_set_one _

theorem fiberMultiplicity_eq_zero_of_not_mem_image {E F : Type*}
    (f : E → F) (D : Set E) {y : F} (hy : y ∉ f '' D) :
    fiberMultiplicity f D y = 0 := by
  have hempty : ({x | x ∈ D ∧ f x = y} : Set E) = ∅ := by
    apply Set.Subset.antisymm
    · intro x hx
      exact (hy ⟨x, hx.1, hx⟩.2).elim
    · exact Set.empty_subset _
  simp [fiberMultiplicity_eq_encard, hempty]

theorem fiberMultiplicity_pos {E F : Type*} (f : E → F) (D : Set E)
    {y : F} (hy : y ∈ f '' D) : 0 < fiberMultiplicity f D y := by
  rcases hy with ⟨x, hx, ⟩hxy
  exact lt_of_lt_of_le (by norm_num : (0 : ℝ≥∞0) < 1)
    (ENNReal.le_tsum ⟨x, hx, ⟩hxy)

theorem fiberMultiplicity_eq_card {E F : Type*} (f : E → F)
    (D : Set E) (y : F)
    [Fintype {x : E // x ∈ D ∧ f x = y}] :
    fiberMultiplicity f D y =
      (Fintype.card {x : E // x ∈ D ∧ f x = y} : ℝ≥∞0) := by
  simp [fiberMultiplicity, tsum_fintype]

theorem fiberMultiplicity_lt_top {E F : Type*} (f : E → F)
    (D : Set E) (y : F) (hy : ({x | x ∈ D ∧ f x = y} : Set E).Finite) :
    fiberMultiplicity f D y < ⊤ := by
  let _ : Fintype {x : E // x ∈ D ∧ f x = y} := hy.fintype
  exact (fiberMultiplicity_eq_card f D y).trans_lt (ENNReal.natCast_lt_top _)
```

Counting source pieces whose images contain a point. Suppose $D = \bigcup_i P_i$, the P_i are pairwise disjoint, and f is injective on each P_i. For fixed y, a piece contributes one fiber point exactly when $y \in f(P_i)$. Choosing that point gives a bijection between such indices and the

actual fiber: disjointness proves injectivity of the choice map, while the cover and piecewise injectivity prove surjectivity. Therefore

$$m_D(y) = \sum_i \mathbf{1}_{f(P_i)}(y).$$

The proof needs no measurable selection. If the index type is countable and each image is measurable, this formula proves measurability of m_D as a countable sum of measurable indicators.

```
/-- Disjoint source pieces count actual fibers, even though their images overlap. -/
theorem fiberMultiplicity_eq_tsum_indicator {E F ι : Type*}
    (f : E → F) (D : Set E) (P : ι → Set E)
    (hcover : ⋃( i, P i) = D)
    (hdisj : Pairwise (fun i j => Disjoint (P i) (P j)))
    (hinj : ∀ i, Set.InjOn f (P i)) (y : F) :
    fiberMultiplicity f D y =
      ∑' i, (f '' P i).indicator (fun _ => (1 : ℝ≥∞0)) y := by
  classical
  let A := {i : ι // y ∈ f '' P i}
  let B := {x : E // x ∈ D ∧ f x = y}
  have hsub (i : ι) : P i ⊆ D := by
    rw ←[ hcover]
    exact Set.subset_iUnion P i
  let pick : A → B := fun i =>
    ⟨Classical.choose i.2,
      hsub i.1 (Classical.choose_spec i.2).1,
      (Classical.choose_spec i.2)).2
  have hpick_mem (i : A) : (pick i).1 ∈ P i.1 :=
    (Classical.choose_spec i.2).1
  have hpick_inj : Function.Injective pick := by
    intro i j hij
    apply Subtype.ext
    by_contra hne
    have hx : (pick i).1 = (pick j).1 := congrArg Subtype.val hij
    exact Set.disjoint_left.1 (hdisj hne) (hpick_mem i)
      (by simpa only [hx] using hpick_mem j)
  have hpick_surj : Function.Surjective pick := by
    intro x
    have hxU : x.1 ∈ ⋃ i, P i := by rw [hcover]; exact x.2.1
    obtain ⟨i, ⟩hi := Set.mem_iUnion.1 hxU
    let a : A := ⟨i, x.1, hi, x⟩.2.2
    refine ⟨a, Subtype.ext ?⟩_
    exact hinj i (hpick_mem a) hi ((pick a).2.2.trans x.2.2.symm)
  calc
    fiberMultiplicity f D y = ∑' _ : A, (1 : ℝ≥∞0) := by
      exact ((Equiv.ofBijective pick ⟨hpick_inj, ⟩hpick_surj).tsum_eq
        (fun _ : B => (1 : ℝ≥∞0))).symm
    _ = ∑' i : ι, ({j : ι | y ∈ f '' P j}).indicator
        (fun _ => (1 : ℝ≥∞0)) i :=
      tsum_subtype {i : ι | y ∈ f '' P i} (fun _ : ι => (1 : ℝ≥∞0))
    _ = ∑' i, (f '' P i).indicator (fun _ => (1 : ℝ≥∞0)) y := by
      apply tsum_congr
      intro i
      exact Set.indicator_const_eq_indicator_const Iff.rfl

section Measurability
variable {E F ι : Type*} [MeasurableSpace F] [Countable ι]

/-- No measurable choice of inverse or eigenframe is needed. -/
theorem measurable_fiberMultiplicity_of_partition
    (f : E → F) (D : Set E) (P : ι → Set E)
    (hcover : ⋃( i, P i) = D)
```

```
    (hdisj : Pairwise (fun i j => Disjoint (P i) (P j)))
    (hinj : ∀ i, Set.InjOn f (P i))
    (himage : ∀ i, MeasurableSet (f '' P i)) :
    Measurable (fiberMultiplicity f D) := by
  have heq : fiberMultiplicity f D = fun y =>
      ∑' i, (f '' P i).indicator (fun _ => (1 : ℝ≥∞0)) y := by
    funext y
    exact fiberMultiplicity_eq_tsum_indicator f D P hcover hdisj hinj y
  rw [heq]
  exact Measurable.tsum (fun i => measurable_const.indicator (himage i))

end Measurability

section Partition
variable {E : Type*} [TopologicalSpace E] [SecondCountableTopology E]
  [MeasurableSpace E] [BorelSpace E]
```

Constructing a disjoint measurable partition. Assume E is second countable with its Borel measurable structure, D is measurable, and every $x \in D$ lies in an open set on which f is injective. A countable subcover $U_0, U_1, \dots$ exists by the Lindelof property, using `HereditarilyLindelofSpace.isLindelof` and `indexed_countable_subcover`. Define

$$P_n = D \cap \left(U_n \setminus \bigcup_{j<n} U_j \right).$$

These sets are measurable, pairwise disjoint, cover D, and inherit injectivity from U_n. The code uses `disjointed` for this first-occurrence construction and treats $D = \varnothing$ separately. For a finite-dimensional vector space, an invertible strict derivative at each point supplies the required open neighborhoods through the inverse function theorem.

```
/-- First-hit disjointification of a countable local injectivity cover. The domain
need only be measurable; the injectivity neighborhoods themselves are open. -/
theorem exists_measurable_injective_partition {F : Type*}
    (f : E → F) (D : Set E) (hD : MeasurableSet D)
    (hlocal : ∀ x ∈ D, ∃ U : Set E,
      IsOpen U ∧ x ∈ U ∧ Set.InjOn f U) :
    ∃ P : ℕ → Set E, ∀( n, MeasurableSet (P n)) ∧
      Pairwise (fun i j => Disjoint (P i) (P j)) ∧
      ⋃( n, P n) = D ∧ ∀( n, Set.InjOn f (P n)) := by
  classical
  rcases D.eq_empty_or_nonempty with rfl | hne
  · refine ⟨fun _ => ∅, by simp, ?_, by simp, ?⟩_
    · intro i j _hij
      simp
    · intro n
      exact Set.injOn_empty f
  let _ : Nonempty D := hne.to_subtype
  choose U hopen hmem hinj using fun x : D => hlocal x.1 x.2
  have hcover : D ⊆ ⋃ x : D, U x := by
    intro x hx
    exact Set.mem_iUnion.2 ⟨⟨x, ⟩hx, hmem ⟨x, ⟩⟩hx
  obtain ⟨seq, ⟩hseq :=
    (HereditarilyLindelofSpace.isLindelof D).indexed_countable_subcover U hopen hcover
  let V : ℕ → Set E := fun n => U (seq n)
  let P : ℕ → Set E := fun n => D ∩ disjointed V n
  have hV : ∀ n, MeasurableSet (V n) := fun n => (hopen (seq n)).measurableSet
  have hdisj : Pairwise (fun i j => Disjoint (P i) (P j)) :=
    pairwise_disjoint_mono (disjoint_disjointed V) (fun n => inter_subset_right)
  refine ⟨P, (fun n => hD.inter (MeasurableSet.disjointed hV n)), hdisj, ?_, ?⟩_
```

```
  · apply Set.Subset.antisymm
    · exact Set.iUnion_subset (fun n => Set.inter_subset_left)
    · intro x hx
      have hxV : x ∈ ⋃ n, disjointed V n := by
        rw [iUnion_disjointed]
        exact hseq hx
      obtain ⟨n, ⟩hn := Set.mem_iUnion.1 hxV
      exact Set.mem_iUnion.2 ⟨n, hx, ⟩hn
  · intro n
    exact (hinj (seq n)).mono ((Set.inter_subset_right).trans (disjointed_subset V n))

end Partition

section Differentiable
variable {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E]
  [FiniteDimensional ℝ E] [MeasurableSpace E] [BorelSpace E]

/-- The actual inverse function theorem supplies the local injectivity cover. -/
theorem exists_partition_of_strictDerivative
    (f : E → E) (D : Set E) (hD : MeasurableSet D)
    (hstrict : ∀ x ∈ D, ∃ L : E ≃L[ℝ] E,
      HasStrictFDerivAt f (L : E →L[ℝ] E) x) :
    ∃ P : ℕ → Set E, ∀( n, MeasurableSet (P n)) ∧
      Pairwise (fun i j => Disjoint (P i) (P j)) ∧
      ⋃( n, P n) = D ∧ ∀( n, Set.InjOn f (P n)) := by
  apply exists_measurable_injective_partition f D hD
  intro x hx
  obtain ⟨L, ⟩hL := hstrict x hx
  refine ⟨(hL.toOpenPartialHomeomorph f).source,
    (hL.toOpenPartialHomeomorph f).open_source,
    hL.mem_toOpenPartialHomeomorph_source, ?⟩_
  exact (hL.toOpenPartialHomeomorph f).injOn

variable μ( : Measure E) [Measure.IsAddHaarMeasure μ]
```

Summing the injective Jacobian identities. Suppose $Df(x) = f'(x)$ at every $x \in D$ and $g : E \to [0, \infty]$ is measurable. For the preceding measurable partition, the library results `measurable_image_of_fderivWithin` and `lintegral_image_eq_lintegral_abs_det_fderiv_mul`, from the pinned Jacobian source [14], give measurability of $f(P_n)$ and

$$\int_{P_n} |\det f'(x)| g(f(x))\, d\mu(x) = \int_{f(P_n)} g(y)\, d\mu(y).$$

Sum over the disjoint source pieces, replace each target restriction by its indicator, and interchange a nonnegative countable sum with an integral using `lintegral_tsum`. The count identity above yields

$$\int_D |\det f'(x)| g(f(x))\, d\mu(x) = \int_E m_D(y) g(y)\, d\mu(y).$$

This is a nonnegative integral identity. It is valid even when either side is infinite, so no Bochner-integrability or totalized-real-integral argument enters.

```
/-- Area formula proved by summing injective Jacobian identities. Images may
overlap; the actual fiber multiplicity records that overlap exactly. -/
theorem lintegral_jacobian_eq_fiberMultiplicity
    (f : E → E) (f' : E → E →L[ℝ] E) (D : Set E) (P : ℕ → Set E)
    (hP : ∀ n, MeasurableSet (P n))
    (hdisj : Pairwise (fun i j => Disjoint (P i) (P j)))
    (hcover : ⋃( n, P n) = D) (hinj : ∀ n, Set.InjOn f (P n))
```

```
    (hderiv : ∀ x ∈ D, HasFDerivAt f (f' x) x)
    (g : E → ℝ≥∞0) (hg : Measurable g) :
    ∫⁻( x in D, ENNReal.ofReal |(f' x).det| * g (f x) ∂μ) =
      ∫⁻ y, fiberMultiplicity f D y * g y ∂μ := by
  classical
  have hsub (n : ℕ) : P n ⊆ D := by
    rw ←[ hcover]
    exact Set.subset_iUnion P n
  have hderivP (n : ℕ) : ∀ x ∈ P n, HasFDerivWithinAt f (f' x) (P n) x :=
    fun x hx => (hderiv x (hsub n hx)).hasFDerivWithinAt
  have himage (n : ℕ) : MeasurableSet (f '' P n) :=
    measurable_image_of_fderivWithin (hP n) (hderivP n) (hinj n)
  calc
    ∫⁻( x in D, ENNReal.ofReal |(f' x).det| * g (f x) ∂μ) =
        ∑' n, ∫⁻ x in P n, ENNReal.ofReal |(f' x).det| * g (f x) ∂μ := by
      rw ←[ hcover]
      exact lintegral_iUnion hP hdisj _
    _ = ∑' n, ∫⁻ y in f '' P n, g y ∂μ := by
      apply tsum_congr
      intro n
      exact (lintegral_image_eq_lintegral_abs_det_fderiv_mul μ
        (hP n) (hderivP n) (hinj n) g).symm
    _ = ∑' n, ∫⁻ y, (f '' P n).indicator g y ∂μ := by
      apply tsum_congr
      intro n
      exact (lintegral_indicator (himage n) g).symm
    _ = ∫⁻ y, ∑' n, (f '' P n).indicator g y ∂μ :=
      (lintegral_tsum (fun n => (hg.indicator (himage n)).aemeasurable)).symm
    _ = ∫⁻ y, fiberMultiplicity f D y * g y ∂μ := by
      apply lintegral_congr
      intro y
      rw [fiberMultiplicity_eq_tsum_indicator f D P hcover hdisj hinj,
        ← ENNReal.tsum_mul_right]
      apply tsum_congr
      intro n
      by_cases hy : y ∈ f '' P n <;> simp [hy]
```

Removing overlaps with reciprocal multiplicity. Assume now that every fiber over $f(D)$ is finite. The earlier lemmas give $0 < m_D(y) < \infty$ on $f(D)$, and measurability of m_D makes $m_D^{-1} g$ an admissible nonnegative test. Apply the previous identity to this test. On the image, $m_D(y)(m_D(y)^{-1} g(y)) = g(y)$, using `ENNReal.mul_inv_cancel_left` with both nonzero and nontop hypotheses already proved. Outside the image $m_D = 0$, so the integrand is zero. Thus

$$\int_D \frac{|\det f'(x)|}{m_D(f(x))}\, g(f(x))\, d\mu(x) = \int_{f(D)} g(y)\, d\mu(y).$$

The formula includes infinite values of g; there is no cancellation of an infinite integral and no division by an infinite fiber count.

```
/-- Reciprocal actual multiplicity removes the overlap. Finiteness and positivity
are established before the ENNReal cancellation, including for infinite test values. -/
theorem lintegral_reciprocal_fiberMultiplicity
    (f : E → E) (f' : E → E →L[ℝ] E) (D : Set E) (P : ℕ → Set E)
    (hP : ∀ n, MeasurableSet (P n))
    (hdisj : Pairwise (fun i j => Disjoint (P i) (P j)))
    (hcover : ⋃( n, P n) = D) (hinj : ∀ n, Set.InjOn f (P n))
    (hderiv : ∀ x ∈ D, HasFDerivAt f (f' x) x)
    (hfinite : ∀ y ∈ f '' D, ({x | x ∈ D ∧ f x = y} : Set E).Finite)
    (g : E → ℝ≥∞0) (hg : Measurable g) :
    ∫⁻( x in D, ENNReal.ofReal |(f' x).det| *
```

```
      ((fiberMultiplicity f D (f x))⁻¹ * g (f x)) ∂μ) =
      ∫⁻ y in f '' D, g y ∂μ := by
  classical
  have hsub (n : ℕ) : P n ⊆ D := by
    rw ←[ hcover]
    exact Set.subset_iUnion P n
  have himage (n : ℕ) : MeasurableSet (f '' P n) :=
    measurable_image_of_fderivWithin (hP n)
      (fun x hx => (hderiv x (hsub n hx)).hasFDerivWithinAt) (hinj n)
  have hm : Measurable (fiberMultiplicity f D) :=
    measurable_fiberMultiplicity_of_partition f D P hcover hdisj hinj himage
  have htarget : MeasurableSet (f '' D) := by
    rw ←[ hcover, Set.image_iUnion]
    exact MeasurableSet.iUnion himage
  let weighted : E → ℝ≥∞0 := fun y => (fiberMultiplicity f D y)⁻¹ * g y
  have hw : Measurable weighted := hm.fun_inv.fun_mul hg
  calc
    _ = ∫⁻ y, fiberMultiplicity f D y * weighted y ∂μ :=
      lintegral_jacobian_eq_fiberMultiplicity μ f f' D P hP hdisj hcover hinj
        hderiv weighted hw
    _ = ∫⁻ y, (f '' D).indicator g y ∂μ := by
      apply lintegral_congr
      intro y
      dsimp only [weighted]
      by_cases hy : y ∈ f '' D
      · rw [Set.indicator_of_mem hy]
        exact ENNReal.mul_inv_cancel_left
          (fiberMultiplicity_pos f D hy).ne'
          (fiberMultiplicity_lt_top f D y (hfinite y hy)).ne
      · rw [fiberMultiplicity_eq_zero_of_not_mem_image f D hy,
          Set.indicator_of_notMem hy, zero_mul]
    _ = _ := lintegral_indicator htarget g
```

Interfaces that construct their own partitions. The next four wrappers consume measurable D, local open injectivity, and actual derivatives. They construct the partition internally and return respectively the counted formula, the reciprocal formula, measurability of the fiber count, and measurability of the image. They add no geometric assumption beyond their displayed hypotheses. The final wrapper uses invertible strict derivatives directly and identifies their linear maps with `fderiv`; this is the interface used by the global spectral integral.

```
/-- Local injectivity and actual derivatives suffice; the countable measurable
partition is constructed, not supplied by the client. -/
theorem lintegral_jacobian_of_localInjective
    (f : E → E) (f' : E → E →L[ℝ] E) (D : Set E) (hD : MeasurableSet D)
    (hlocal : ∀ x ∈ D, ∃ U : Set E,
      IsOpen U ∧ x ∈ U ∧ Set.InjOn f U)
    (hderiv : ∀ x ∈ D, HasFDerivAt f (f' x) x)
    (g : E → ℝ≥∞0) (hg : Measurable g) :
    ∫⁻( x in D, ENNReal.ofReal |(f' x).det| * g (f x) ∂μ) =
      ∫⁻ y, fiberMultiplicity f D y * g y ∂μ := by
  obtain ⟨P, hP, hdisj, hcover, ⟩hinj :=
    exists_measurable_injective_partition f D hD hlocal
  exact lintegral_jacobian_eq_fiberMultiplicity μ f f' D P
    hP hdisj hcover hinj hderiv g hg

theorem lintegral_reciprocal_of_localInjective
    (f : E → E) (f' : E → E →L[ℝ] E) (D : Set E) (hD : MeasurableSet D)
    (hlocal : ∀ x ∈ D, ∃ U : Set E,
      IsOpen U ∧ x ∈ U ∧ Set.InjOn f U)
    (hderiv : ∀ x ∈ D, HasFDerivAt f (f' x) x)
```

```
    (hfinite : ∀ y ∈ f '' D, ({x | x ∈ D ∧ f x = y} : Set E).Finite)
    (g : E → ℝ≥∞0) (hg : Measurable g) :
    ∫⁻( x in D, ENNReal.ofReal |(f' x).det| *
      ((fiberMultiplicity f D (f x))⁻¹ * g (f x)) ∂μ) =
      ∫⁻ y in f '' D, g y ∂μ := by
  obtain ⟨P, hP, hdisj, hcover, ⟩hinj :=
    exists_measurable_injective_partition f D hD hlocal
  exact lintegral_reciprocal_fiberMultiplicity μ f f' D P
    hP hdisj hcover hinj hderiv hfinite g hg

theorem measurable_fiberMultiplicity_of_localInjective
    (f : E → E) (f' : E → E →L[ℝ] E) (D : Set E) (hD : MeasurableSet D)
    (hlocal : ∀ x ∈ D, ∃ U : Set E,
      IsOpen U ∧ x ∈ U ∧ Set.InjOn f U)
    (hderiv : ∀ x ∈ D, HasFDerivAt f (f' x) x) :
    Measurable (fiberMultiplicity f D) := by
  obtain ⟨P, hP, hdisj, hcover, ⟩hinj :=
    exists_measurable_injective_partition f D hD hlocal
  apply measurable_fiberMultiplicity_of_partition f D P hcover hdisj hinj
  intro n
  apply measurable_image_of_fderivWithin (f' := f') (hP n) _ (hinj n)
  intro x hx
  have hxD : x ∈ D := by
    rw ←[ hcover]
    exact Set.mem_iUnion.2 ⟨n, ⟩hx
  exact (hderiv x hxD).hasFDerivWithinAt

theorem measurable_image_of_localInjective
    (f : E → E) (f' : E → E →L[ℝ] E) (D : Set E) (hD : MeasurableSet D)
    (hlocal : ∀ x ∈ D, ∃ U : Set E,
      IsOpen U ∧ x ∈ U ∧ Set.InjOn f U)
    (hderiv : ∀ x ∈ D, HasFDerivAt f (f' x) x) :
    MeasurableSet (f '' D) := by
  obtain ⟨P, hP, hdisj, hcover, ⟩hinj :=
    exists_measurable_injective_partition f D hD hlocal
  rw ←[ hcover, Set.image_iUnion]
  apply MeasurableSet.iUnion
  intro n
  apply measurable_image_of_fderivWithin (f' := f') (hP n) _ (hinj n)
  intro x hx
  have hxD : x ∈ D := by
    rw ←[ hcover]
    exact Set.mem_iUnion.2 ⟨n, ⟩hx
  exact (hderiv x hxD).hasFDerivWithinAt

/-- Actual invertible strict derivatives are another sufficient local interface. -/
theorem lintegral_reciprocal_of_strictDerivative
    (f : E → E) (D : Set E) (hD : MeasurableSet D)
    (hstrict : ∀ x ∈ D, ∃ L : E ≃L[ℝ] E,
      HasStrictFDerivAt f (L : E →L[ℝ] E) x)
    (hfinite : ∀ y ∈ f '' D, ({x | x ∈ D ∧ f x = y} : Set E).Finite)
    (g : E → ℝ≥∞0) (hg : Measurable g) :
    ∫⁻( x in D, ENNReal.ofReal |(fderiv ℝ f x).det| *
      ((fiberMultiplicity f D (f x))⁻¹ * g (f x)) ∂μ) =
      ∫⁻ y in f '' D, g y ∂μ := by
  obtain ⟨P, hP, hdisj, hcover, ⟩hinj :=
    exists_partition_of_strictDerivative f D hD hstrict
  apply lintegral_reciprocal_fiberMultiplicity μ f (fderiv ℝ f) D P
    hP hdisj hcover hinj _ hfinite g hg
  intro x hx
  obtain ⟨L, ⟩hL := hstrict x hx
  simpa only [hL.hasFDerivAt.fderiv] using hL.hasFDerivAt
```

```
end Differentiable
end O77.CountedJacobian
end
```

80 The image and finite fibers of the global spectral map

Retain the map F given by `spectralMap`, its ordered source D_k given by `orderedSourcek`, the diagonal vector $s(a)$ given by `diagonalVelocitya`, and the frame $Q(a) = C(K(a))$. The source and target are both the independent upper-entry space E_k. This module proves two global facts needed by the counted formula: every positive-definite simple-spectrum matrix is in $F(D_k)$, and the fiber over $F(a)$ has exactly $N(Q(a))$ elements. The word "fiber" includes the moving eigenvalue coordinates; those coordinates must be proved to be fixed by the image equation.

```
import Mathlib
import O77CayleyFibers
import O77SpectralSplit
import O77SpectralOrder
import O77SpectralRegular
import O77CayleySignCover
import O77CountedJacobian

set_option autoImplicit false
noncomputable section
open Set Matrix MeasureTheory
open scoped ENNReal

namespace O77.GlobalSpectralFibers

open O77.GramBorder O77.SpectralDifferential O77.OrthogonalCayley
open O77.OrthogonalSpectral O77.CayleyFibers O77.SpectralSplit
open O77.SpectralOrder O77.SpectralRegular O77.CayleySignCover
```

Reassembling a point from a spectrum and a skew matrix. For a skew matrix K, `skewEntriesK` records its strictly upper entries and joinSkew(s, K) inserts these entries together with diagonal s. Its associated skew matrix is exactly K: strict upper entries agree by construction, the lower entries agree by skew symmetry, and diagonal entries vanish because $K_{ii} = -K_{ii}$. Conversely joinSkew$(s(a), K(a)) = a$. The associated orthogonal frame is consequently $C(K)$.

```
/-- The independent upper entries of an actual skew matrix. -/
def skewEntries {k : ℕ} (K : Matrix (Fin k) (Fin k) ℝ) : AngularCoordinates k :=
  fun ij => K ij.1.1 ij.1.2

/-- Assemble the eigenvalue coordinates and the independent entries of K. -/
def joinSkew {k : ℕ} (s : Fin k → ℝ) (K : Matrix (Fin k) (Fin k) ℝ) :
    UpperCoordinates k := join k (s, skewEntries K)

@[simp] theorem diagonalVelocity_joinSkew {k : ℕ} (s : Fin k → ℝ)
    (K : Matrix (Fin k) (Fin k) ℝ) : diagonalVelocity (joinSkew s K) = s :=
  diagonalVelocity_join s (skewEntries K)

theorem skew_joinSkew {k : ℕ} (s : Fin k → ℝ)
    (K : Matrix (Fin k) (Fin k) ℝ) (hK : K.transpose = -K) :
    skew (joinSkew s K) = K := by
  ext i j
```

```
  rcases lt_trichotomy i j with hij | hij | hji
  · exact (skew_upper (joinSkew s K) hij).trans
      (join_strictUpper s (skewEntries K) ⟨(i, j), ⟩hij)
  · subst j
    have hentry : K i i = -K i i :=
      congrArg (fun M : Matrix (Fin k) (Fin k) ℝ => M i i) hK
    calc
      skew (joinSkew s K) i i = 0 := skew_diag (joinSkew s K) i
      _ = K i i := by linarith
  · have hentry : K i j = -K j i :=
      congrArg (fun M : Matrix (Fin k) (Fin k) ℝ => M j i) hK
    have hskew : skew (joinSkew s K) i j = -skew (joinSkew s K) j i :=
      congrArg (fun M : Matrix (Fin k) (Fin k) ℝ => M j i)
        (skew_transpose (joinSkew s K))
    exact hskew.trans
      ((congrArg (fun x : ℝ => -x)
        ((skew_upper (joinSkew s K) hji).trans
          (join_strictUpper s (skewEntries K) ⟨(j, i), ⟩hji))).trans hentry.symm)

@[simp] theorem skewEntries_skew {k : ℕ} (a : UpperCoordinates k) :
    skewEntries (skew a) = angularEntries a := by
  funext ij
  simp [skewEntries, skew, angularEntries, ij.2]

@[simp] theorem joinSkew_diagonalVelocity_skew {k : ℕ}
    (a : UpperCoordinates k) : joinSkew (diagonalVelocity a) (skew a) = a := by
  simp [joinSkew]

theorem frame_joinSkew {k : ℕ} (s : Fin k → ℝ)
    (K : Matrix (Fin k) (Fin k) ℝ) (hK : K.transpose = -K) :
    frame (joinSkew s K) = cayley K := by
  unfold frame
  rw [skew_joinSkew s K hK]
```

The coordinate equation is the matrix equation. Applying `upperEntries` shows that `fromUpper` is injective. Thus $F(a) = F(b)$ is equivalent to the equality

$$Q(a)\,\mathrm{diag}(s(a))Q(a)^T = Q(b)\,\mathrm{diag}(s(b))Q(b)^T.$$

If $a, b \in D_k$, their two diagonal vectors are strictly increasing, so `ordered_spectrum_unique` gives $s(a) = s(b)$. This is the step that rules out additional eigenvalue or permutation choices in the full source fiber.

```
theorem fromUpper_injective (k : ℕ) : Function.Injective (@fromUpper k) := by
  intro a b hab
  have h := congrArg upperEntries hab
  simpa only [upperEntries_fromUpper] using h

theorem fromUpper_spectralMap {k : ℕ} (a : UpperCoordinates k) :
    fromUpper (spectralMap a) =
      frame a * Matrix.diagonal (diagonalVelocity a) * (frame a).transpose := by
  simpa only [spectralMap, zero_add] using
    fromUpper_orthogonalSpectral (0 : Fin k → ℝ) a

theorem spectralMap_eq_iff {k : ℕ} (a b : UpperCoordinates k) :
    spectralMap a = spectralMap b ↔
      frame a * Matrix.diagonal (diagonalVelocity a) * (frame a).transpose =
        frame b * Matrix.diagonal (diagonalVelocity b) * (frame b).transpose := by
  constructor
  · intro h
    simpa only [fromUpper_spectralMap] using congrArg fromUpper h
```

```
  · intro h
    apply fromUpper_injective k
    simpa only [fromUpper_spectralMap] using h

/-- The source equation itself fixes the moving ordered eigenvalue coordinates. -/
theorem ordered_diagonal_eq {k : ℕ} (a b : UpperCoordinates k)
    (ha : a ∈ orderedSource k) (hb : b ∈ orderedSource k)
    (hab : spectralMap a = spectralMap b) : diagonalVelocity a = diagonalVelocity b :=
  ordered_spectrum_unique (diagonalVelocity a) (diagonalVelocity b) ha.2 hb.2
    (frame a) (frame b) (transpose_frame_mul_frame a) (transpose_frame_mul_frame b)
    ((spectralMap_eq_iff a b).1 hab)
```

A bijection for the entire source fiber. For $a \in D_k$, map a point b in the fiber over $F(a)$ to $K(b)$. The just-proved equality $s(b) = s(a)$ shows that this belongs to the fixed-spectrum Cayley fiber for $(s(a), Q(a))$. Its inverse sends a skew solution K to $\mathrm{joinSkew}(s(a), K)$. The reconstruction identities verify both inverse equations and all source-membership conditions. Composing with the admissible-sign bijection identifies the full source fiber with $\mathcal{A}(Q(a))$.

```
abbrev orderedFiber {k : ℕ} (y : UpperCoordinates k) :=
  {b : UpperCoordinates k // b ∈ orderedSource k ∧ spectralMap b = y}

/-- This equivalence fixes the eigenvalues by ordered-spectrum uniqueness
and retains every actual skew solution. It counts the full source fiber. -/
def orderedFiberEquivCayleyFiber {k : ℕ} (a : UpperCoordinates k)
    (ha : a ∈ orderedSource k) :
    orderedFiber (spectralMap a) ≃ cayleySpectralFiber (diagonalVelocity a) (frame a) where
  toFun b := ⟨skew b.val, skew_transpose b.val, by
    have hdiag := ordered_diagonal_eq b.val a b.property.1 ha b.property.2
    have hmat := (spectralMap_eq_iff b.val a).1 b.property.2
    rw [hdiag] at hmat
    exact ⟩hmat
  invFun K := ⟨joinSkew (diagonalVelocity a) K.val, by
    constructor
    · simpa only [mem_orderedSource, diagonalVelocity_joinSkew] using ha
    · apply (spectralMap_eq_iff _ a).2
      rw [diagonalVelocity_joinSkew, frame_joinSkew _ _ K.property.1]
      exact K.property).2
  left_inv b := by
    apply Subtype.ext
    change joinSkew (diagonalVelocity a) (skew b.val) = b.val
    rw ←[ ordered_diagonal_eq b.val a b.property.1 ha b.property.2,
      joinSkew_diagonalVelocity_skew]
  right_inv K := by
    apply Subtype.ext
    exact skew_joinSkew (diagonalVelocity a) K.val K.property.1

def orderedFiberEquivAdmissibleSigns {k : ℕ} (a : UpperCoordinates k)
    (ha : a ∈ orderedSource k) :
    orderedFiber (spectralMap a) ≃
      {b : Fin k → Bool // b ∈ admissibleSigns (frame a)} :=
  (orderedFiberEquivCayleyFiber a ha).trans
    (cayleyFiberEquivAdmissibleSigns (diagonalVelocity a) ha.2.injective
      (frame a) (transpose_frame_mul_frame a))
```

Finite cardinality before numerical conversion. Transfer the finite type structure from $\mathcal{A}(Q(a))$ through this equivalence. This proves finiteness of every fiber over the image and

$$\#\{b \in D_k : F(b) = F(a)\} = N(Q(a)), \qquad m_{D_k}(F(a)) = N(Q(a)) \quad \text{in } [0, \infty].$$

Two ordered source points with the same image therefore have the same count, also proved directly from the fixed-spectrum frame invariance. This remains compatible with variation of the count between distinct target matrices.

```
/-- Finiteness is established before converting cardinalities to numbers. -/
@[instance_reducible] def orderedFiberFintype {k : ℕ} (a : UpperCoordinates k)
    (ha : a ∈ orderedSource k) : Fintype (orderedFiber (spectralMap a)) :=
  Fintype.ofEquiv {b : Fin k → Bool // b ∈ admissibleSigns (frame a)}
    (orderedFiberEquivAdmissibleSigns a ha).symm

theorem card_orderedFiber {k : ℕ} (a : UpperCoordinates k)
    (ha : a ∈ orderedSource k) :
    Nat.card (orderedFiber (spectralMap a)) = branchCount (frame a) := by
  rw [Nat.card_congr (orderedFiberEquivCayleyFiber a ha)]
  exact card_cayleySpectralFiber (diagonalVelocity a) ha.2.injective
    (frame a) (transpose_frame_mul_frame a)

theorem finite_fiber_of_mem_image {k : ℕ} (y : UpperCoordinates k)
    (hy : y ∈ spectralMap '' orderedSource k) :
    ({b : UpperCoordinates k | b ∈ orderedSource k ∧ spectralMap b = y} :
      Set (UpperCoordinates k)).Finite := by
  rcases hy with ⟨a, ha, ⟩rfl
  exact Set.finite_def.mpr ⟨orderedFiberFintype a ⟩ha

theorem fiberMultiplicity_at {k : ℕ} (a : UpperCoordinates k)
    (ha : a ∈ orderedSource k) :
    O77.CountedJacobian.fiberMultiplicity spectralMap (orderedSource k) (spectralMap a) =
      (branchCount (frame a) : ℝ≥∞0) := by
  let _ : Fintype (orderedFiber (spectralMap a)) := orderedFiberFintype a ha
  rw [O77.CountedJacobian.fiberMultiplicity_eq_card]
  change (Fintype.card (orderedFiber (spectralMap a)) : ℝ≥∞0) = _
  rw ←[ Nat.card_eq_fintype_card, card_orderedFiber a ha]

theorem branchCount_frame_eq_of_spectralMap_eq {k : ℕ} (a b : UpperCoordinates k)
    (ha : a ∈ orderedSource k) (hb : b ∈ orderedSource k)
    (hab : spectralMap a = spectralMap b) :
    branchCount (frame a) = branchCount (frame b) := by
  have hdiag := ordered_diagonal_eq a b ha hb hab
  have hmat := (spectralMap_eq_iff a b).1 hab
  rw ←[ hdiag] at hmat
  exact branchCount_eq_of_same_spectral_matrix (diagonalVelocity a) ha.2.injective
    (frame a) (frame b) (transpose_frame_mul_frame a) (transpose_frame_mul_frame b) hmat
```

Identifying the image exactly. The characteristic polynomial of $S(F(a))$ is $\prod_i (X - s(a)_i)$; strict order implies its separability by `Polynomial.separable_prod_X_sub_C_iff`. Along with positive definiteness, this proves one inclusion in

$$F(D_k) = \{b : S(b) \text{ is positive definite and its characteristic polynomial is separable}\}.$$

Conversely, take a positive ordered spectral decomposition $S(b) = Q\,\mathrm{diag}(s)Q^T$. The signed Cayley coverage theorem produces a skew K with the same conjugate. Then $\mathrm{joinSkew}(s, K) \in D_k$ maps to b. This proves pointwise coverage of the whole simple-spectrum cone; the removal of the repeated-spectrum null set is a separate measure argument in the final module.

```
theorem spectralMap_separable {k : ℕ} (a : UpperCoordinates k)
    (ha : a ∈ orderedSource k) : (fromUpper (spectralMap a)).charpoly.Separable := by
  rw [fromUpper_spectralMap,
    charpoly_orthogonal_conjugate (frame a) _ (transpose_frame_mul_frame a),
    Matrix.charpoly_diagonal, Polynomial.separable_prod_X_sub_C_iff]
  exact ha.2.injective
```

```
/-- One global Cayley map reaches every positive-definite simple-spectrum
matrix. The sign cover proves surjectivity without an a.e. branch-count claim. -/
theorem image_orderedSource_eq (k : ℕ) :
    spectralMap '' orderedSource k =
      {b : UpperCoordinates k | (fromUpper b).PosDef ∧
        (fromUpper b).charpoly.Separable} := by
  ext b
  constructor
  · rintro ⟨a, ha, ⟩rfl
    exact ⟨spectralMap_posDef a ha, spectralMap_separable a ⟩ha
  · rintro ⟨hpos, ⟩hsimple
    obtain ⟨s, Q, hs, hspos, hQ, ⟩hdec := exists_positive_ordered_frame hpos hsimple
    obtain ⟨K, hK, ⟩hconj := exists_cayley_conjugate s Q hQ
    refine ⟨joinSkew s K, ?_, ?⟩_
    · simpa only [mem_orderedSource, diagonalVelocity_joinSkew] using And.intro hspos hs
    · apply fromUpper_injective k
      rw [fromUpper_spectralMap, diagonalVelocity_joinSkew, frame_joinSkew s K hK,
        hconj]
      exact hdec.symm

end O77.GlobalSpectralFibers
end
```

81 Angular density and parameter independence

For $a \in E_k$, write $n(a) = N(Q(a))$, with N the admissible-sign count and $Q(a) = C(K(a))$. The full angular Jacobian is $j(a) = |\det(I - K(a)/2)^{-1}|^{k-1}$. We now define its corrected version $\widetilde{j}(a) = j(a)/n(a)$ in the nonnegative extended reals. These objects depend only on angular coordinates, before any eigenvalues, Wishart dimensions, or test parameters are chosen.

```
import Mathlib
import O77CayleyFibers
import O77SpectralSplit

set_option autoImplicit false
noncomputable section
open Set MeasureTheory
open scoped ENNReal Matrix.Norms.Elementwise

namespace O77.CayleyWeights
open O77.Residual O77.GramBorder O77.SpectralDifferential
open O77.OrthogonalCayley O77.OrthogonalSpectral O77.CayleyDifferential
open O77.SpectralChartIntegral O77.CayleyFibers O77.SpectralSplit
```

A positive finite measurable denominator. Every $Q(a)$ lies in the regular Cayley chart, so the all-true sign choice shows $n(a) > 0$. Its definition as a finite cardinality gives $n(a) < \infty$. The frame map is analytic and continuous; composition with the previously established measurable branch-count function, followed by the natural-number inclusion into $[0, \infty]$, proves measurability of n.

```
/-- The actual finite number of regular sign representatives of this frame.
It is not replaced by a constant, even on the simple-spectrum locus. -/
def sourceMultiplicity {k : ℕ} (a : UpperCoordinates k) : ℝ≥∞0 :=
  (branchCount (frame a) : ℝ≥∞0)

theorem sourceMultiplicity_pos_lt_top {k : ℕ} (a : UpperCoordinates k) :
    0 < sourceMultiplicity a ∧ sourceMultiplicity a < ⊤ := by
```

```
  constructor
  · change (0 : ℝ≥∞0) < (branchCount (frame a) : ℝ≥∞0)
    exact_mod_cast branchCount_pos_of_regular (frame a)
      (isUnit_det_cayley_add_one (skew a) (skew_transpose a))
  · exact ENNReal.natCast_lt_top _

theorem continuous_frame (k : ℕ) : Continuous (@frame k) :=
  continuous_iff_continuousAt.mpr (fun a => (frame_analyticAt a).continuousAt)

theorem measurable_sourceMultiplicity (k : ℕ) : Measurable (@sourceMultiplicity k) := by
  change Measurable (fun a : UpperCoordinates k => (branchCount (frame a) : ℝ≥∞0))
  simpa only [Function.comp_def] using
    (measurable_of_countable (fun n : ℕ => (n : ℝ≥∞0))).comp
      ((measurable_branchCount k).comp (continuous_frame k).measurable)
```

Continuity of the angular factor. The determinant of $I - K(a)/2$ is continuous and nowhere zero for skew $K(a)$. Its reciprocal is therefore continuous by `Continuous.fun_inv₀`; the determinant-of-inverse identity, absolute value, and natural power give continuity of j. The corrected density is positive finite because its numerator and denominator are positive finite, and it is measurable as their quotient. Both numerator and denominator depend only on $K(a)$, so the same is true of $\widetilde{j}$.

```
theorem continuous_angularDensity (k : ℕ) : Continuous (@angularDensity k) := by
  have hd : Continuous (fun a : UpperCoordinates k => (minusHalf (skew a)).det) :=
    (continuous_minusHalf_skew k).matrix_det
  have hi : Continuous (fun a : UpperCoordinates k => (minusHalf (skew a)).⁻det¹) :=
    hd.₀fun_inv (fun a => isUnit_iff_ne_zero.mp
      (isUnit_det_minusHalf (skew a) (skew_transpose a)))
  change Continuous (fun a : UpperCoordinates k =>
    ENNReal.ofReal (|((minusHalf (skew a))⁻¹).det| ^ (k - 1)))
  simp only [Matrix.det_nonsing_inv, Ring.inverse_eq_inv']
  simpa only [Function.comp_def] using
    ENNReal.continuous_ofReal.comp (hi.abs.fun_pow (k - 1))

/-- Reciprocal multiplicity corrects the actual Cayley covering map.
The definition is made before the spectrum, eta, rho, T, or a test function. -/
def correctedAngularDensity {k : ℕ} (a : UpperCoordinates k) : ℝ≥∞0 :=
  angularDensity a / sourceMultiplicity a

theorem correctedAngularDensity_pos_lt_top {k : ℕ} (a : UpperCoordinates k) :
    0 < correctedAngularDensity a ∧ correctedAngularDensity a < ⊤ := by
  obtain ⟨ha0, ⟩hat := angularDensity_pos_lt_top a
  obtain ⟨hn0, ⟩hnt := sourceMultiplicity_pos_lt_top a
  exact ⟨ENNReal.div_pos ha0.ne' hnt.ne,
    ENNReal.div_lt_top hat.ne hn0.ne⟩'

theorem measurable_correctedAngularDensity (k : ℕ) :
    Measurable (@correctedAngularDensity k) := by
  change Measurable (fun a : UpperCoordinates k => angularDensity a / sourceMultiplicity a)
  exact (continuous_angularDensity k).measurable.fun_div (measurable_sourceMultiplicity k)

theorem sourceMultiplicity_eq_of_skew_eq {k : ℕ} {a b : UpperCoordinates k}
    (h : skew a = skew b) : sourceMultiplicity a = sourceMultiplicity b := by
  simp only [sourceMultiplicity, frame, h]

theorem correctedAngularDensity_eq_of_skew_eq {k : ℕ}
    {a b : UpperCoordinates k} (h : skew a = skew b) :
    correctedAngularDensity a = correctedAngularDensity b := by
  rw [correctedAngularDensity, correctedAngularDensity,
    angularDensity_eq_of_skew_eq h, sourceMultiplicity_eq_of_skew_eq h]
```

The corrected pointwise Jacobian. Let $q = s + s(a)$ have positive entries, let $T \geq 0$, and let $\eta, \rho \in \mathbb{R}$. Write $H_{\eta,\rho,T}(G) = e^{-\operatorname{tr} G}(\det G)^{\eta} \det(I + TG)^{-\rho}$ on the positive-definite cone. The preceding `weighted_jacobian` identity, which already includes the absolute Vandermonde and the exact sorted extension, gives

$$|\det D\,\mathrm{orthogonalSpectral}_s(a)|\, n(a)^{-1} H_{\eta,\rho,T}(S(\mathrm{orthogonalSpectral}_s(a))) = \widetilde{j}(a) L_{\eta,\rho,T}(q).$$

The proof only reassociates multiplication and division in the nonnegative extended reals. It neither integrates nor assumes that an integral is finite. Positivity of q and $T \geq 0$ are retained by the underlying density adapter.

```
/-- This is the exact corrected pointwise Jacobian. It includes the full
angular determinant and is valid before any finiteness of integrals is known. -/
theorem corrected_weighted_jacobian {k : ℕ} (eta rho T : ℝ)
    (s : Fin k → ℝ) (a : UpperCoordinates k)
    (hT : 0 ≤ T) (hq : ∀ i, 0 < s i + diagonalVelocity a i) :
    ENNReal.ofReal |(fderiv ℝ (orthogonalSpectral s) a).det| *
        ((sourceMultiplicity a)⁻¹ *
          coneTestDensity eta rho T (orthogonalSpectral s a)) =
      correctedAngularDensity a *
        sortedLaguerreIntegrand eta rho T (s + diagonalVelocity a) := by
  calc
    _ = (ENNReal.ofReal |(fderiv ℝ (orthogonalSpectral s) a).det| *
          coneTestDensity eta rho T (orthogonalSpectral s a)) /
          sourceMultiplicity a := by simp only [div_eq_mul_inv]; ac_rfl
    _ = (angularDensity a *
          sortedLaguerreIntegrand eta rho T (s + diagonalVelocity a)) /
          sourceMultiplicity a := by rw [weighted_jacobian eta rho T s a hT hq]
    _ = _ := by unfold correctedAngularDensity; exact ENNReal.mul_div_right_comm
```

Removing the radial coordinates from the correction. For $z \in \mathcal{K}_k$, set $W(z) = n(\mathrm{join}(0, z))^{-1}$. This is measurable, and independence of n from the diagonal gives $\widetilde{j}(a) = j(a)W(z(a))$. Hence the single angular mass

$$A_k = \int_{\mathcal{K}_k} \widetilde{j}(\mathrm{join}(0, z))\, dz$$

is defined before d, η, ρ, T. Tonelli separation with this exact reciprocal weight proves

$$\int_{D_k} \widetilde{j}(a) L_{\eta,\rho,T}(s(a))\, da = A_k \int_{S_k} L_{\eta,\rho,T}(s)\, ds.$$

At this stage A_k is merely a nonnegative extended real. Its finiteness is proved from the global identity in the following module, rather than being assumed in its definition.

```
def reciprocalWeight {k : ℕ} (z : AngularCoordinates k) : ℝ≥∞0 :=
  (sourceMultiplicity (join k (0, z)))⁻¹

theorem measurable_reciprocalWeight (k : ℕ) : Measurable (@reciprocalWeight k) := by
  change Measurable (fun z : AngularCoordinates k => (sourceMultiplicity (join k (0, z)))⁻¹)
  simpa only [Function.comp_def, id_eq] using
    ((measurable_sourceMultiplicity k).comp
      ((join k).measurable.comp (measurable_const.prodMk measurable_id))).fun_inv

theorem correctedAngularDensity_eq_angular_mul_reciprocal {k : ℕ}
    (a : UpperCoordinates k) :
    correctedAngularDensity a = angularDensity a * reciprocalWeight (angularEntries a) := by
  have heq : sourceMultiplicity a =
      sourceMultiplicity (join k (0, angularEntries a)) := by
```

```
    calc
      _ = sourceMultiplicity (join k (diagonalVelocity a, angularEntries a)) := by
        rw [join_split]
      _ = _ := sourceMultiplicity_eq_of_skew_eq (skew_join _ _ _)
  simp only [correctedAngularDensity, div_eq_mul_inv, reciprocalWeight, heq]

/-- A single nonnegative angular integral, fixed before every radial parameter.
Its positivity and finiteness are proved only after the global area identity. -/
def angularMass (k : ℕ) : ℝ≥∞0 :=
  ∫⁻ z : AngularCoordinates k, correctedAngularDensity (join k (0, z))
    ∂angularLebesgue k

theorem angularMass_eq (k : ℕ) :
    angularMass k =
      ∫⁻ z : AngularCoordinates k, angularDensity (join k (0, z)) * reciprocalWeight z
        ∂angularLebesgue k := by
  unfold angularMass
  apply lintegral_congr
  intro z
  simpa only [angularEntries_join] using
    correctedAngularDensity_eq_angular_mul_reciprocal (join k (0, z))

/-- The corrected Jacobian separates in the exact inherited entry measures.
The radial factor is the literal sorted-extension integrand. -/
theorem lintegral_ordered_corrected {k : ℕ} (eta rho T : ℝ) :
    ∫⁻( a in diagonalVelocity ⁻¹' strictOrderedPositive k,
      correctedAngularDensity a *
        sortedLaguerreIntegrand eta rho T (diagonalVelocity a) ∂upperLebesgue k) =
      angularMass k *
        ∫⁻ s in strictOrderedPositive k,
          sortedLaguerreIntegrand eta rho T s ∂coordinateLebesgue (Fin k) := by
  simp_rw [correctedAngularDensity_eq_angular_mul_reciprocal]
  rw [angularMass_eq]
  exact lintegral_ordered_angular_sorted reciprocalWeight
    (measurable_reciprocalWeight k) eta rho T
```

A safe calibration lemma. Suppose $M = AJ$ in $[0, \infty]$ and $0 < M < \infty$. Neither factor can be zero, since then $M = 0$. Once both are nonzero, neither can be infinite, since then their product is infinite. Therefore both factors are positive finite. The code proves these four exclusions directly, before invoking any inverse or passage to a real number. This elementary lemma will obtain the bounds on A_k from a known positive finite cone integral.

```
/-- Positive finite calibration of a nonnegative product forces BOTH
factors to be positive finite. No inverse or real-part cancellation occurs
before these facts have been proved. -/
theorem factors_pos_lt_top_of_calibration {M A J : ℝ≥∞0}
    (hM0 : 0 < M) (hMt : M < ⊤) (hcal : M = A * J) :
    (0 < A ∧ A < ⊤) ∧ (0 < J ∧ J < ⊤) := by
  have hA0 : A ≠ 0 := by
    intro h
    rw [h, zero_mul] at hcal
    exact hM0.ne' hcal
  have hJ0 : J ≠ 0 := by
    intro h
    rw [h, mul_zero] at hcal
    exact hM0.ne' hcal
  have hAt : A ≠ ⊤ := by
    intro h
    rw [h, ENNReal.top_mul hJ0] at hcal
    exact hMt.ne hcal
  have hJt : J ≠ ⊤ := by
```

```
    intro h
    rw [h, ENNReal.mul_top hA0] at hcal
    exact hMt.ne hcal
  exact ⟨(pos_iff_ne_zero.mpr hA0, lt_top_iff_ne_top.mpr ⟩hAt,
    (pos_iff_ne_zero.mpr hJ0, lt_top_iff_ne_top.mpr ⟩)hJt

end O77.CayleyWeights
end
```

82 Global spectral integration and the unconditional answer

Fix natural numbers k, d, a real number ρ, and $T \geq 0$. Put $\eta_{k,d} = (d-k-1)/2$, using real subtraction. Let $\mathcal{P}_k$ be the cone of positive-definite real symmetric $k \times k$ matrices, with measure dS in the independent upper entries. The cone integral considered here is

$$\int_{\mathcal{P}_k} e^{-\operatorname{tr} S} \det(S)^{\eta_{k,d}} \det(I_k + TS)^{-\rho}\, dS.$$

In Lean this is `coneLIntegralkd(determinantTestrhoT)`. For the proof we use the same spectral notation as in Part I: K is a real skew-symmetric $k \times k$ matrix, $\mathcal{C}_k = \{s \in \mathbb{R}^k : 0 < s_1 < \cdots < s_k\}$, and $s \in \mathcal{C}_k$. Define

$$Q(K) = (I_k - K/2)^{-1}(I_k + K/2), \qquad \Theta(K, s) = Q(K)\operatorname{diag}(s)Q(K)^{\mathsf{T}}.$$

The source uses independent entries K_{ij} for $i < j$ and the coordinates s_i. The Lean type `UpperCoordinatesk` stores these in a single upper triangular array: its diagonal stores s, and its strict upper entries store K. Thus passing between the two descriptions is a coordinate permutation, with no density factor.

For an orthogonal matrix Q, put

$$n(Q) = \#\{\varepsilon \in \{-1, 1\}^k : \det(Q\operatorname{diag}(\varepsilon) + I_k) \neq 0\}.$$

The preceding fiber theorem proves that $1 \leq n(Q) \leq 2^k$ and that $n(Q(K))$ is the cardinality of the actual fiber of Θ through (K, s). The Jacobian contributes $a_k(K) = |\det(I_k - K/2)|^{-(k-1)}$ for $k \geq 1$, with $a_0 = 1$. Define the angular integral, initially in $[0, \infty]$, by

$$D_k^{\mathrm{ord}} = \int_{\mathrm{Skew}_k(\mathbb{R})} \frac{a_k(K)}{n(Q(K))}\, dK,$$

where dK is product measure in the independent skew entries. The Laguerre density on $\mathcal{C}_k$ is

$$\mathcal{L}_{\eta,\rho,T}(s) = e^{-\sum_i s_i} \prod_i s_i^{\eta} \prod_{i<j} (s_j - s_i) \prod_i (1 + Ts_i)^{-\rho},$$

with $1 \leq i \leq k$ and $1 \leq i < j \leq k$ in all products. Its sorted extension defines the already introduced full nonnegative orthant integral $J_k(\eta, \rho, T)$. We prove that the cone integral is $(D_k^{\mathrm{ord}}/k!)J_k(\eta_{k,d}, \rho, T)$, then establish that this constant is positive and finite.

```
import Mathlib
import O77GlobalSpectralFibers
import O77CayleyWeights
import O77GramWishart

set_option autoImplicit false
noncomputable section
open Set MeasureTheory
```

```
open scoped ENNReal

namespace O77.GlobalSpectralIntegral
open O77.Residual O77.GramBorder O77.GramPushforward O77.GramWishart
open O77.SpectralDifferential O77.OrthogonalSpectral O77.SpectralExceptional
open O77.SpectralChartIntegral O77.SpectralRegular O77.SpectralOrder
open O77.CountedJacobian O77.GlobalSpectralFibers O77.CayleyWeights
```

The density and the omitted exceptional set. Trace and determinant are continuous in the independent upper entries; real powers and the determinant test are measurable. Their product therefore defines a measurable nonnegative density. Outside the cone this is a total function, but all integration below restricts it to the cone or its actual spectral preimage. The image of Θ is exactly the positive-definite simple-spectrum locus. The previously proved `ae_separable_charpoly` says that repeated-spectrum matrices form a null set in upper-entry measure. Thus the image of Θ and the positive-definite cone agree almost everywhere before any weighting is introduced. This follows from the null zero set of the nonzero `spectralResultant`: the resultant of the universal characteristic polynomial and its derivative. It does not require choosing eigenvectors measurably.

```
theorem measurable_coneTestDensity (k : ℕ) (eta rho T : ℝ) :
    Measurable (coneTestDensity (k := k) eta rho T) := by
  have he : Continuous (fun a : UpperCoordinates k => Real.exp (-(fromUpper a).trace)) :=
    (fromUpper_continuous k).matrix_trace.neg.rexp
  have hp : Measurable (fun a : UpperCoordinates k => (fromUpper a).det ^ eta) := by
    simpa only [Function.comp_def] using
      (measurable_rpow_const eta).comp (fromUpper_continuous k).matrix_det.measurable
  change Measurable (fun a : UpperCoordinates k =>
    ENNReal.ofReal (Real.exp (-(fromUpper a).trace) * (fromUpper a).det ^ eta) *
      determinantTest rho T (fromUpper a))
  simpa only [Function.comp_def] using
    (he.measurable.fun_mul hp).ennreal_ofReal.fun_mul
      ((measurable_determinantTest k rho T).comp (fromUpper_continuous k).measurable)

/-- The omitted target points are the actual repeated-spectrum null set.
This equality concerns the inherited upper-entry measure, before weighting. -/
theorem image_orderedSource_ae_cone (k : ℕ) :
    spectralMap '' orderedSource k ᵐ=[upperLebesgue k]
      {b : UpperCoordinates k | (fromUpper b).PosDef} := by
  filter_upwards [ae_separable_charpoly k] with b hb
  simp only [image_orderedSource_eq, Set.mem_ofPred_eq, hb, and_true]
```

Applying the area formula to the actual map. The upper-entry product measure is an additive Haar measure. On $\mathrm{Skew}_k(\mathbb{R}) \times \mathcal{C}_k$, the derivative of Θ is invertible, and every fiber over its image is finite. In the packed upper-entry coordinates these are exactly the hypotheses of `lintegral_reciprocal_of_strictDerivative`. Apply that theorem to the nonnegative cone density, replacing the cone by the image of Θ using their almost-everywhere equality. The trace and determinants on $\Theta(K, s)$ are respectively $\sum_i s_i$, $\prod_i s_i$, and $\prod_i (1 + T s_i)$. The fiber cardinality is $n(Q(K))$, and the Jacobian is $a_k(K) \prod_{i<j}(s_j - s_i)$. Consequently the exact formula is

$$\int_{\mathcal{P}_k} e^{-\operatorname{tr} S} \det(S)^{\eta_{k,d}} \det(I_k + TS)^{-\rho}\, dS = \int_{\mathrm{Skew}_k(\mathbb{R})\times\mathcal{C}_k} \frac{a_k(K)}{n(Q(K))} \mathcal{L}_{\eta_{k,d},\rho,T}(s)\, dK\, ds.$$

No integral has yet been assumed finite. Since $T \geq 0$ and $s_i > 0$, the factors $1 + T s_i$ are positive for every real exponent $-\rho$. Thus this nonnegative identity permits arbitrary $\rho \in \mathbb{R}$

and $k, d \in \mathbb{N}$, including $k > d$; the Wishart application will subsequently impose $k \le d$ and $\rho \ge 0$.

```
/-- Exact global integration with reciprocal ACTUAL fiber multiplicity.
No finiteness of an integral, constant covering degree, or angular normalizer
is supplied as a hypothesis. -/
theorem cone_eq_corrected_ordered (k d : ℕ) (rho T : ℝ) (hT : 0 ≤ T) :
    coneLIntegral k d (determinantTest rho T) =
      ∫⁻ a in orderedSource k, correctedAngularDensity a *
        sortedLaguerreIntegrand (wishartEta k d) rho T (diagonalVelocity a)
        ∂upperLebesgue k := by
  let _ : Measure.IsAddHaarMeasure (upperLebesgue k) := by
    simpa only [upperLebesgue, coordinateLebesgue_eq_volume] using
      (inferInstance : Measure.IsAddHaarMeasure (volume : Measure (UpperCoordinates k)))
  have harea := lintegral_reciprocal_of_strictDerivative (upperLebesgue k)
    (@spectralMap k) (orderedSource k) (measurableSet_orderedSource k)
    (fun a ha => ⟨derivativeEquiv a ha, strictDerivative_equiv a ⟩ha)
    (fun b hb => finite_fiber_of_mem_image b hb)
    (coneTestDensity (wishartEta k d) rho T)
    (measurable_coneTestDensity k (wishartEta k d) rho T)
  calc
    coneLIntegral k d (determinantTest rho T) =
        ∫⁻ b in spectralMap '' orderedSource k,
          coneTestDensity (wishartEta k d) rho T b ∂upperLebesgue k := by
      exact setLIntegral_congr
        (f := coneTestDensity (wishartEta k d) rho T)
        (image_orderedSource_ae_cone k).symm
    _ = ∫⁻ a in orderedSource k,
        ENNReal.ofReal |(fderiv ℝ (@spectralMap k) a).det| *
          ((fiberMultiplicity spectralMap (orderedSource k) (spectralMap a))⁻¹ *
            coneTestDensity (wishartEta k d) rho T (spectralMap a))
          ∂upperLebesgue k := harea.symm
    _ = _ := by
      apply setLIntegral_congr_fun (measurableSet_orderedSource k)
      intro a ha
      change ENNReal.ofReal |(fderiv ℝ (@spectralMap k) a).det| *
          ((fiberMultiplicity spectralMap (orderedSource k) (spectralMap a))⁻¹ *
            coneTestDensity (wishartEta k d) rho T (spectralMap a)) =
        correctedAngularDensity a *
          sortedLaguerreIntegrand (wishartEta k d) rho T (diagonalVelocity a)
      rw [fiberMultiplicity_at a ha]
      simpa only [spectralMap, sourceMultiplicity, zero_add] using
        corrected_weighted_jacobian (wishartEta k d) rho T
          (0 : Fin k → ℝ) a hT
          (by simpa only [Pi.zero_apply, zero_add] using ha.1)
```

Separate first, calibrate second. The first factor on the right depends only on K, and the Laguerre density only on s. Tonelli therefore gives

$$\int_{\mathcal{P}_k} e^{-\operatorname{tr} S} \det(S)^{\eta_{k,d}} \det(I_k + TS)^{-\rho}\, dS = D_k^{\mathrm{ord}} \int_{\mathcal{C}_k} \mathcal{L}_{\eta_{k,d},\rho,T}(s)\, ds \qquad (T \ge 0).$$

To determine the bounds on D_k^{ord}, specialize this already parameterized identity to $d = k$ and $\rho = T = 0$. The Gaussian Gram formula identifies the left side as $\pi^{k^2/2}/A_{k,k}$, where $A_{k,k} = \prod_{i=0}^{k-1} \pi^{(k-i)/2}/\Gamma((k-i)/2)$ is positive and finite. Thus the product on the right is positive and finite. In $[0, \infty]$, a positive finite product has both factors positive and finite, so $0 < D_k^{\mathrm{ord}} < \infty$. In the formal proof the corresponding cone bound is `cone_zero_parameters_bounds`. This argument includes $k = 0$ and the square exponent $-1/2$. Calibration supplies bounds on an established normalizer; it is not used to infer the parameterized identity.

```
/-- The angular integral is selected before d, rho and T. This is already
the parameterized identity, although its normalizer bounds are proved next. -/
theorem cone_eq_angularMass_mul_ordered (k d : ℕ) (rho T : ℝ) (hT : 0 ≤ T) :
    coneLIntegral k d (determinantTest rho T) =
      angularMass k *
        ∫⁻ s in strictOrderedPositive k,
          sortedLaguerreIntegrand (wishartEta k d) rho T s
          ∂coordinateLebesgue (Fin k) := by
  rw [cone_eq_corrected_ordered k d rho T hT]
  exact lintegral_ordered_corrected (wishartEta k d) rho T

/-- Calibration at d=k and zero test parameters proves a positive finite
angular mass. The square-case exponent is -1/2, not a nonnegative exponent.
The already parameterized identity precedes this calibration. -/
theorem angularMass_pos_lt_top (k : ℕ) :
    0 < angularMass k ∧ angularMass k < ⊤ := by
  obtain ⟨hM0, ⟩hMt := cone_zero_parameters_bounds (k := k) (d := k) le_rfl
  exact (factors_pos_lt_top_of_calibration hM0 hMt
    (cone_eq_angularMass_mul_ordered k k 0 0 le_rfl)).1
```

The exact full-orthant normalizer. The constant $D_k^{\mathrm{ord}}/k!$ is positive and finite. The ordered-chamber identity `strictSortedLIntegral_eq_factorial_inv_mul` then gives

$$\int_{\mathcal{P}_k} e^{-\operatorname{tr} S} \det(S)^{\eta_{k,d}} \det(I_k + TS)^{-\rho}\, dS = \frac{D_k^{\mathrm{ord}}}{k!} J_k(\eta_{k,d}, \rho, T) \qquad (T \geq 0).$$

The normalizer depends only on k and is selected before d, ρ, T. The proof uses the inherited sorted-extension integral, with its exact boundary convention. For $k > d$ or unrestricted ρ, the equality remains an extended nonnegative integral identity; it makes no additional finiteness assertion for those parameters.

```
/-- Full nonnegative orthant normalization in independent upper entries.
The factorial comes only from the inherited chamber decomposition. -/
def spectralConstant (k : ℕ) : ℝ≥∞0 :=
  angularMass k / (k.factorial : ℝ≥∞0)

theorem spectralConstant_pos_lt_top (k : ℕ) :
    0 < spectralConstant k ∧ spectralConstant k < ⊤ := by
  obtain ⟨hA0, ⟩hAt := angularMass_pos_lt_top k
  have hfact : (k.factorial : ℝ≥∞0) ≠ 0 := by
    exact_mod_cast Nat.factorial_ne_zero k
  exact ⟨ENNReal.div_pos hA0.ne' (ENNReal.natCast_ne_top _),
    ENNReal.div_lt_top hAt.ne ⟩hfact

/-- Exact cone-to-eigenvalue identity for the literal inherited sorted
extension on the full orthant. In fact rho is arbitrary and no k<=d premise
is needed for this nonnegative identity; integrals may then be infinite. -/
theorem cone_eq_spectralConstant_mul_laguerre (k d : ℕ)
    (rho T : ℝ) (hT : 0 ≤ T) :
    coneLIntegral k d (determinantTest rho T) =
      spectralConstant k * laguerreOrthantLIntegral k (wishartEta k d) rho T := by
  rw [cone_eq_angularMass_mul_ordered k d rho T hT,
    strictSortedLIntegral_eq_factorial_inv_mul]
  simp only [spectralConstant, div_eq_mul_inv, mul_assoc]
```

Discharging the Wishart premise. The preceding theorem supplies the positive finite constant $D_k^{\mathrm{ord}}/k!$ required by `HigherRankConeEigenvalueFormula` for every $2 \leq k \leq d$. The already proved equivalence `global_iff_cone_eigenvalues` composes this spectral identity with

the Gaussian Gram pushforward, the transpose adapter, and the zero- and one-row instances. It produces an actual proof of `O77.External.RealWishartEigenvalueFormula`. In particular, the global Wishart proposition is the conclusion of a theorem here, rather than an extra axiom or a parameter left for the caller.

```
/-- The higher-rank cone-to-eigenvalue formula. -/
theorem higherRankConeEigenvalueFormula : HigherRankConeEigenvalueFormula := by
  intro k d _hk _hkd
  obtain ⟨hD0, ⟩hDt := spectralConstant_pos_lt_top k
  exact ⟨spectralConstant k, hD0, hDt,
    fun rho T _hrho hT => cone_eq_spectralConstant_mul_laguerre k d rho T ⟩hT

theorem realWishartEigenvalueFormula : O77.External.RealWishartEigenvalueFormula :=
  global_iff_cone_eigenvalues.mpr higherRankConeEigenvalueFormula

end O77.GlobalSpectralIntegral

namespace O77
```

The final theorem has only the problem's dimension hypotheses. For $I, O, H \in \mathbb{N}$ and $\Phi \in \mathbb{R}^{O\times I}$, the final declaration assumes $I > 0$, $O > 0$, and $\operatorname{rank}\Phi < H$. It applies `answersO77_of_wishart` to the actual proof of Wishart just constructed. Its conclusion is the unchanged six-conjunct `AnswersO77` contract: exact-fit pair classification, fixed-target rank realization, the full threshold-minimum locus, attained minimum threshold, attained greatest multiplicity on that locus, and the inhabited singular-data rungs with pair $(1,1)$ at every nonoptimal rung point. The supplied singular data retain their strict positive singular-value order and both discarded-mode annihilation conditions. There is no Wishart, local-chart, measured-pair, or rank-realization parameter in the final theorem. Its mathematical declaration and proof script are preserved from the certification anchor identified in the front matter; editorial comments have been brought up to date.

```
/-- The six-conjunct measured answer, with no Wishart parameter. -/
theorem answersO77 {I O H : ℕ} (Phi : Matrix (Fin O) (Fin I) ℝ)
    (hI : 0 < I) (hO : 0 < O) (hrH : Phi.rank < H) : AnswersO77 (H := H) Phi :=
  answersO77_of_wishart GlobalSpectralIntegral.realWishartEigenvalueFormula
    Phi hI hO hrH

end O77
end
```

83 Supplementary modules outside the terminal dependency closure

The following six modules give an alternate small-residual route and numerical comparisons with published multiplicity expressions. The unconditional theorem above uses the preceding 55-module closure; these supplementary routes are retained separately for reference. Their mathematical declarations are preserved. A stray continuation of an English comment in `O77LowRankAnswer` is removed as obsolete editorial prose; it was outside the terminal export.

84 Auxiliary module `O77DimensionLocalResidual`

This supplementary module is an alternate route outside the terminal theorem's dependency closure.

```
import Mathlib
import O77WishartReduction

set_option autoImplicit false
noncomputable section
open Set MeasureTheory
open scoped ENNReal BigOperators

namespace O77.DimensionLocalResidual
open O77.Residual
```

A single residual rectangle. Let $Y \in \mathbb{R}^{p\times h}$ and $Z \in \mathbb{R}^{h\times q}$. Fix a concrete Wishart normalizer for $k = \min(h, q)$ and $d = \max(h, q)$. Integrating the p Gaussian rows produces the positive row constant; choosing the smaller Gram matrix gives the same k eigenvalues in either orientation. Passing from the full orthant to the ordered chamber contributes exactly $k!$. The following equality retains all three constants and assumes $T \geq 0$.

```
/-- One fixed-dimensional normalizer works in either rectangular orientation.
The constant is selected before the Laplace parameter. -/
theorem residualGaussianLaplace_eq_orderedLaguerre {p q h : ℕ} {C : ℝ≥∞0}
    (hW : HasConcreteRealWishartFormula (min h q) (max h q) C)
    (T : ℝ) (hT : 0 ≤ T) :
    residualGaussianLaplaceLIntegral p q h T =
      ENNReal.ofReal (gaussianRowsConstant p h) * C *
        (O77.Arithmetic.chamberK h q).factorial *
        ENNReal.ofReal (laguerreChamberIntegral
          (k := O77.Arithmetic.chamberK h q)
          (residualSpectralEta q h) (residualSpectralRho p) T) := by
  rcases le_total h q with hhq | hqh
  · have hC : HasConcreteRealWishartFormula h q C := by
      simpa only [min_eq_left hhq, max_eq_right hhq] using hW
    exact residualGaussianLaplace_eq_orderedLaguerre_left hhq hC T hT
  · have hC : HasConcreteRealWishartFormula q h C := by
      simpa only [min_eq_right hqh, max_eq_left hqh] using hW
    exact residualGaussianLaplace_eq_orderedLaguerre_right hqh hC T hT
```

Power–log order and finite rescaling. Assume $p > 0$ and the Wishart proposition only for this fixed rectangle. Choose its normalizer C and multiply it by the Gaussian row constant and $k!$. Each factor is nonzero and finite in $[0, \infty]$. The ordered chamber estimate can therefore be multiplied by this constant. The preceding exact integral equality identifies the result with the Gaussian matrix integral. No global Wishart proposition is inferred from this one instance.

```
/-- The Gaussian order needs only this residual's concrete Wishart instance.
The row exponent needs p>0; no unused positivity premises on q or h occur. -/
theorem gaussianLaplace_powerLog_bounds {p q h : ℕ} (hp : 0 < p)
    (hW : ConcreteRealWishartEigenvalueFormula (min h q) (max h q)) :
    LaplacePowerLogBounds (residualGaussianLaplaceLIntegral p q h)
      ((O77.residualMin p q h : ℝ) / 2)
      (O77.residualMultiplicity p q h - 1) := by
  obtain ⟨C, ⟩hC := hW
  let K : ℝ≥∞0 := ENNReal.ofReal (gaussianRowsConstant p h) * C *
    (O77.Arithmetic.chamberK h q).factorial
  have hgauss0 : ENNReal.ofReal (gaussianRowsConstant p h) ≠ 0 :=
    (ENNReal.ofReal_pos.2 (gaussianRowsConstant_pos p h)).ne'
  have hfactorial0 : ((O77.Arithmetic.chamberK h q).factorial : ℝ≥∞0) ≠ 0 := by
    exact_mod_cast Nat.factorial_ne_zero (O77.Arithmetic.chamberK h q)
  have hK0 : K ≠ 0 := mul_ne_zero
    (mul_ne_zero hgauss0 hC.constant_ne_zero) hfactorial0
  have hKtop : K ≠ ⊤ := ENNReal.mul_ne_top
    (ENNReal.mul_ne_top ENNReal.ofReal_ne_top hC.constant_ne_top)
    (ENNReal.natCast_ne_top _)
  have hscaled := LaplacePowerLogBounds.const_mul hK0 hKtop
    (residual_laguerreChamber_laplacePowerLogBounds (p := p) (q := q) (h := h) hp)
  apply LaplacePowerLogBounds.congr_of_eq_of_ge_two hscaled
  intro T hT
  have hT0 : 0 ≤ T := by linarith
  simpa only [K, mul_assoc] using
    (residualGaussianLaplace_eq_orderedLaguerre (p := p) hC T hT0).symm
```

Localization and strict balls. The next two results take a Gaussian power–log bound with positive exponent as their only analytic premise. Quartic homogeneous localization gives the same order for the Laplace integral on each closed ball; finite ball measure permits the Laplace-to-sublevel theorem. The open-ball mass lies between the closed-half-ball and closed-ball masses. Applying the same-window comparison with scale factors one preserves the exponent and logarithmic degree.

```
/-- A Gaussian order is localized by the established quartic homogeneous
comparison, then converted using the finite-measure Laplace/sublevel bridge.
This lemma carries no Wishart assumption. -/
theorem closedBallSublevel_order_of_gaussian {p q h : ℕ} {lam : ℝ} {degree : ℕ}
    (hlam : 0 < lam)
    (hG : LaplacePowerLogBounds (residualGaussianLaplaceLIntegral p q h) lam degree)
    {R : ℝ} (hR : 0 < R) :
    EThetaZeroPowerLog
      (sublevelMass (residualQuadraticBallMeasure p q h R) residualKernel)
      lam degree := by
  obtain ⟨a, A, ha, hatop, hA, hAtop, ⟩hcomp :=
    homogeneous_gaussianLaplace_comparable_ball_instantiated p q h hR
  have hball := LaplacePowerLogBounds.of_two_sided_comparison
    ha.ne' hatop.ne hA.ne' hAtop.ne hcomp hG
  apply volumeOrder_of_laplaceOrder (residualQuadraticBallMeasure p q h R)
    measurable_residualKernel residualKernel_nonneg hlam degree
  apply LaplacePowerLogBounds.congr_of_eq_of_ge_two hball
  intro T _hT
  exact residualBallLaplaceLIntegral_eq_laplaceIntegral p q h R T

/-- Open balls use the existing two-window comparison. The closed inner
radius R/2 and closed outer radius R are fixed before epsilon. -/
theorem openBallSublevel_order_of_gaussian {p q h : ℕ} {lam : ℝ} {degree : ℕ}
    (hlam : 0 < lam)
    (hG : LaplacePowerLogBounds (residualGaussianLaplaceLIntegral p q h) lam degree)
    {R : ℝ} (hR : 0 < R) :
```

```
    EThetaZeroPowerLog
      (sublevelMass (residualOpenQuadraticBallMeasure p q h R) residualKernel)
      lam degree := by
  apply O77.Transport0906.order_of_two_window_bounds
    (closedBallSublevel_order_of_gaussian hlam hG (half_pos hR))
    (closedBallSublevel_order_of_gaussian hlam hG hR)
    (lo := 1) (hi := 1) (by norm_num) (by norm_num)
  intro eps _heps
  constructor
  · simpa only [ENNReal.ofReal_one, one_mul] using
      residualClosedHalfBall_sublevel_le_openBall p q h hR eps
  · simpa only [ENNReal.ofReal_one, one_mul] using
      residualOpenBall_sublevel_le_closedBall p q h R eps
```

Recovering the radius-first predicate. When all three dimensions are positive, the finite residual minimum is positive and its multiplicity is at least one. Use outer radius 1 and apply the open-ball result for each $0 < R < 1$. The radius is fixed before its comparison constants. When any factor dimension vanishes, the kernel is identically zero; the already established neutral measured pair and the numerical zero-dimension formulas agree.

```
/-- The complete radius-first residual predicate follows from a single
Gaussian power-log input in the positive-dimensional case. -/
theorem residualBandPair_of_gaussian {p q h : ℕ}
    (hp : 0 < p) (hq : 0 < q) (hh : 0 < h)
    (hG : LaplacePowerLogBounds (residualGaussianLaplaceLIntegral p q h)
      ((O77.residualMin p q h : ℝ) / 2)
      (O77.residualMultiplicity p q h - 1)) :
    HasResidualBandPairAtOrigin p q h ((O77.residualMin p q h : ℝ) / 2)
      (O77.residualMultiplicity p q h) := by
  have hmin : 0 < (O77.residualMin p q h : ℝ) := by
    exact_mod_cast O77.Arithmetic.residualMin_positive p q h hp hq hh
  have hlam : 0 < (O77.residualMin p q h : ℝ) / 2 := half_pos hmin
  refine (hlam.le, (O77.Arithmetic.residualMultiplicity_one_or_two p q h).1,
    1, by norm_num, ?)_
  intro R hR _hRone
  exact openBallSublevel_order_of_gaussian hlam hG hR

/-- The proposed numerical residual pair agrees with the directly proved
neutral measured pair whenever a factor is absent. -/
lemma residualBandPair_formula_of_zero_dimension {p q h : ℕ}
    (hz : p = 0 ∨ q = 0 ∨ h = 0) :
    HasResidualBandPairAtOrigin p q h ((O77.residualMin p q h : ℝ) / 2)
      (O77.residualMultiplicity p q h) := by
  have hmin := O77.Arithmetic.residualMin_zero p q h hz
  have hmult := O77.Arithmetic.residualMultiplicity_zero p q h hz
  simpa only [hmin, hmult, Nat.cast_zero, zero_div] using
    residualBandPair_neutral_of_zero_dimension hz
```

The concrete instance and the scalar branch. Split off the zero-factor cases first. Otherwise all dimensions are positive and the concrete Wishart instance gives the Gaussian order, hence the measured pair. If $\min(h, q) \le 1$, it is either zero or one: use the empty-space identity or the proved radial identity respectively. The final scalar specialization evaluates the finite table to $(1/2, 2)$.

```
/-- Dimension-local input; it is never promoted to a global Wishart premise.
All zero-factor cases reuse the direct neutral measured proof. -/
theorem residualBandPair_of_concrete (p q h : ℕ)
    (hW : ConcreteRealWishartEigenvalueFormula (min h q) (max h q)) :
    HasResidualBandPairAtOrigin p q h ((O77.residualMin p q h : ℝ) / 2)
```

```
      (O77.residualMultiplicity p q h) := by
  by_cases hz : p = 0 ∨ q = 0 ∨ h = 0
  · exact residualBandPair_formula_of_zero_dimension hz
  · have hp : 0 < p := Nat.pos_of_ne_zero (fun hp => hz (Or.inl hp))
    have hq : 0 < q := Nat.pos_of_ne_zero (fun hq => hz (Or.inr (Or.inl hq)))
    have hh : 0 < h := Nat.pos_of_ne_zero (fun hh => hz (Or.inr (Or.inr hh)))
    exact residualBandPair_of_gaussian hp hq hh (gaussianLaplace_powerLog_bounds hp hW)

theorem concrete_of_min_le_one {h q : ℕ} (hsmall : min h q ≤ 1) :
    ConcreteRealWishartEigenvalueFormula (min h q) (max h q) := by
  by_cases hz : min h q = 0
  · simpa only [hz] using O77.WishartReduction.concrete_zero (max h q)
  · have hone : min h q = 1 := by omega
    have hle : min h q ≤ max h q := (min_le_left h q).trans (le_max_left h q)
    have hpos : 0 < max h q := by omega
    simpa only [hone] using O77.WishartReduction.concrete_one hpos

/-- An unconditional origin-pair attempt for every residual with at most one
Gram eigenvalue, including both rectangular orientations and empty factors. -/
theorem residualBandPair_of_min_le_one (p q h : ℕ) (hsmall : min h q ≤ 1) :
    HasResidualBandPairAtOrigin p q h ((O77.residualMin p q h : ℝ) / 2)
      (O77.residualMultiplicity p q h) :=
  residualBandPair_of_concrete p q h (concrete_of_min_le_one hsmall)

/-- The zero-output branch is also unconditional when both Gram dimensions
are large; it uses the actual identically-zero kernel. -/
theorem residualBandPair_of_zero_or_min_le_one (p q h : ℕ)
    (hs : p = 0 ∨ min h q ≤ 1) :
    HasResidualBandPairAtOrigin p q h ((O77.residualMin p q h : ℝ) / 2)
      (O77.residualMultiplicity p q h) := by
  rcases hs with hp | hsmall
  · exact residualBandPair_formula_of_zero_dimension (Or.inl hp)
  · exact residualBandPair_of_min_le_one p q h hsmall

theorem scalar_residual_pair : HasResidualBandPairAtOrigin 1 1 1 (1 / 2 : ℝ) 2 := by
  have hmin : O77.residualMin 1 1 1 = 1 := by decide
  have hmult : O77.residualMultiplicity 1 1 1 = 2 := by decide
  simpa only [hmin, hmult, Nat.cast_one] using
    residualBandPair_of_min_le_one 1 1 1 (by decide)

end O77.DimensionLocalResidual
end
```

85 Auxiliary module `O77ResidualTranspose`

This supplementary module is an alternate route outside the terminal theorem's dependency closure.

```
import Mathlib
import O77WishartReduction

set_option autoImplicit false
noncomputable section
open Set MeasureTheory
open scoped ENNReal

namespace O77.ResidualTranspose
open O77.Residual O77.WishartReduction
```

The exact coordinate equivalence. Define $\tau(Y, Z) = (Z^{\mathsf{T}}, Y^{\mathsf{T}})$. It is a measurable equivalence from residual dimensions (p, q, h) to (q, p, h), obtained by two entry transpositions and a product swap. Applying it twice is the identity, including in empty dimensions.

```
/-- Transpose each matrix and exchange the two factors. This is an exact
permutation of the original real matrix entries. -/
def transposePair (p q h : ℕ) : ResidualSpace p q h ≃ᵐ ResidualSpace q p h :=
  ((transposeEntries (m := Fin p) (n := Fin h)).prodCongr
    (transposeEntries (m := Fin h) (n := Fin q))).trans MeasurableEquiv.prodComm

@[simp] theorem transposePair_apply {p q h : ℕ} (w : ResidualSpace p q h) :
    transposePair p q h w = (w.2.transpose, w.1.transpose) := rfl

@[simp] theorem transposePair_symm_apply {p q h : ℕ} (w : ResidualSpace q p h) :
    (transposePair p q h).symm w = transposePair q p h w := rfl

@[simp] theorem transposePair_transposePair {p q h : ℕ} (w : ResidualSpace p q h) :
    transposePair q p h (transposePair p q h w) = w := rfl

@[simp] theorem transposePair_zero (p q h : ℕ) :
    transposePair p q h (0 : ResidualSpace p q h) = 0 := rfl
```

Measure, norm and kernel. Each entry transposition preserves the specified product Lebesgue measure, and the product swap does too. The Frobenius squared norm is unchanged by transposition; swapping the two summands preserves the ambient squared norm. Finally $(YZ)^{\mathsf{T}} = Z^{\mathsf{T}}Y^{\mathsf{T}}$ proves equality of the residual kernels. None of these assertions uses a default norm on matrices.

```
/-- The measure identity is proved from the two entry-transposition maps
and product-measure swap; no matrix or product norm convention is used. -/
theorem transposePair_preserves (p q h : ℕ) :
    MeasurePreserving (transposePair p q h)
      (residualMatrixLebesgue p q h) (residualMatrixLebesgue q p h) := by
  have ht := (transposeEntries_preserves (m := Fin p) (n := Fin h)).prod
    (transposeEntries_preserves (m := Fin h) (n := Fin q))
  exact (Measure.measurePreserving_swap
    μ( := matrixLebesgue (Fin h) (Fin p))
    ν( := matrixLebesgue (Fin q) (Fin h))).comp ht

@[simp] theorem ambientNormSq_transposePair {p q h : ℕ}
    (w : ResidualSpace p q h) :
    residualAmbientNormSq (transposePair p q h w) = residualAmbientNormSq w := by
  simp only [transposePair_apply, residualAmbientNormSq,
    frobeniusSq_transpose, add_comm]

@[simp] theorem kernel_transposePair {p q h : ℕ} (w : ResidualSpace p q h) :
    residualKernel (transposePair p q h w) = residualKernel w := by
  change frobeniusSq (w.2.transpose * w.1.transpose) = frobeniusSq (w.1 * w.2)
  rw ←[ Matrix.transpose_mul, frobeniusSq_transpose]
```

Corresponding strict sets. Substitute the squared-norm identity into the inequality $\mathcal{N} < R^2$ and the kernel identity into $K < \varepsilon$. This gives the exact preimages of the quadratic balls and sublevels; bijectivity also gives their images. The identity for all real R concerns the displayed quadratic set, not a metric ball with negative radius.

```
/-- The radius convention is the inherited inequality normSq < R^2.
This equality is valid even for R<=0 and does not identify a negative R
with a metric-ball radius. -/
theorem preimage_openBall (p q h : ℕ) (R : ℝ) :
```

```
    transposePair p q h ⁻¹' residualOpenQuadraticBall q p h R =
      residualOpenQuadraticBall p q h R := by
  ext w
  simp only [Set.mem_preimage, residualOpenQuadraticBall, Set.mem_setOf_eq,
    ambientNormSq_transposePair]

theorem image_openBall (p q h : ℕ) (R : ℝ) :
    transposePair p q h '' residualOpenQuadraticBall p q h R =
      residualOpenQuadraticBall q p h R := by
  rw ←[ preimage_openBall p q h R]
  exact Set.image_preimage_eq _ (transposePair p q h).surjective

theorem preimage_kernel_sublevel (p q h : ℕ) (eps : ℝ) :
    transposePair p q h ⁻¹' {w : ResidualSpace q p h | residualKernel w < eps} =
      {w : ResidualSpace p q h | residualKernel w < eps} := by
  ext w
  simp only [Set.mem_preimage, Set.mem_setOf_eq, kernel_transposePair]
```

Restricting the measure. Restrict the measure-preserving equivalence to the corresponding ball sets, using `MeasurePreserving.restrict_preimage_emb`. Applying the resulting measure identity to the strict kernel sublevel gives equality of the two sublevel masses for every R, ε. Function extensionality packages these pointwise equalities.

```
/-- Restricting the exact coordinate measures to corresponding open
quadratic balls preserves the same transpose equivalence. -/
theorem openBall_preserves (p q h : ℕ) (R : ℝ) :
    MeasurePreserving (transposePair p q h)
      (residualOpenQuadraticBallMeasure p q h R)
      (residualOpenQuadraticBallMeasure q p h R) := by
  have ht := (transposePair_preserves p q h).restrict_preimage_emb
    (transposePair p q h).measurableEmbedding (residualOpenQuadraticBall q p h R)
  simpa only [preimage_openBall, residualOpenQuadraticBallMeasure] using ht

/-- Exact equality of strict residual sublevel masses, for all real
radii and epsilon. There is no estimate, null-boundary argument, or
epsilon rescaling in this transport. -/
theorem sublevelMass_eq (p q h : ℕ) (R eps : ℝ) :
    sublevelMass (residualOpenQuadraticBallMeasure p q h R) residualKernel eps =
      sublevelMass (residualOpenQuadraticBallMeasure q p h R) residualKernel eps := by
  have ht := (openBall_preserves p q h R).measure_preimage_equiv
    {w : ResidualSpace q p h | residualKernel w < eps}
  rw [preimage_kernel_sublevel] at ht
  exact ht

theorem sublevelMass_function_eq (p q h : ℕ) (R : ℝ) :
    sublevelMass (residualOpenQuadraticBallMeasure p q h R) residualKernel =
      sublevelMass (residualOpenQuadraticBallMeasure q p h R) residualKernel := by
  funext eps
  exact sublevelMass_eq p q h R eps
```

Transport of the complete predicate. Rewrite the mass function inside the radius-first definition. The same radii and the same constants work, and the public guards $\lambda \geq 0$, $m \geq 1$ are unchanged. Only afterwards use the independently proved arithmetic symmetries of d_0 and μ_0 to rewrite the candidate pair.

```
/-- The whole radius-first measured predicate is invariant, including
its public lambda>=0 and m>=1 guards. Every radius and comparison
constant is retained unchanged. -/
theorem bandPair_iff (p q h : ℕ) (lam : ℝ) (m : ℕ) :
```

```
    HasResidualBandPairAtOrigin p q h lam m ↔
      HasResidualBandPairAtOrigin q p h lam m := by
  simp only [HasResidualBandPairAtOrigin, sublevelMass_function_eq p q h]

/-- Candidate arithmetic is an adapter after measured transport. The
natural arithmetic symmetries are inherited checked theorems; this does
not assign a measured pair by definition. -/
theorem candidateBandPair_iff (p q h : ℕ) :
    HasResidualBandPairAtOrigin p q h
      ((O77.residualMin p q h : ℝ) / 2) (O77.residualMultiplicity p q h) ↔
    HasResidualBandPairAtOrigin q p h
      ((O77.residualMin q p h : ℝ) / 2) (O77.residualMultiplicity q p h) := by
  rw [bandPair_iff p q h,
    O77.Arithmetic.residualMin_transpose p q h,
    O77.Arithmetic.residualMultiplicity_transpose p q h]

end O77.ResidualTranspose
end
```

86 Auxiliary module `O77LowRankFiber`

This supplementary module is an alternate route outside the terminal theorem's dependency closure.

```
import Mathlib
import O77DimensionLocalResidual
import O77ResidualTranspose
import O77FiberClassification
import O77PowerLogUniqueness

/-! A local analytic dependency for actual exact-fit fibers.
No global Gram or spectral pushforward module is imported. -/
set_option autoImplicit false
noncomputable section
open scoped Matrix.Norms.Elementwise

namespace O77.LowRankFiber
open O77.Residual
open O77.Native (Parameters HasLocalBandPair)
open O77.Reconstruction0905 (RM)
```

Three small-dimension alternatives. If $q \le 1$ or $h \le 1$, the smaller Gram dimension is at most one, so the preceding radial consumer applies. If $p \le 1$, first exchange the two residual factors by transposition, apply the same consumer, and transport the measured pair back. The numerical transpose identities are used after the measured transport.

```
/-- Transpose transport supplies the output-small branch; the other two
branches use only the zero/one-eigenvalue radial formulas. -/
theorem residualBandPair_of_small (p q h : ℕ)
    (hsmall : p ≤ 1 ∨ q ≤ 1 ∨ h ≤ 1) :
    HasResidualBandPairAtOrigin p q h ((O77.residualMin p q h : ℝ)/2)
      (O77.residualMultiplicity p q h) := by
  rcases hsmall with hp | hq | hh
  · apply (O77.ResidualTranspose.bandPair_iff p q h _ _).mpr
    have ht := O77.DimensionLocalResidual.residualBandPair_of_min_le_one
      q p h ((min_le_right h p).trans hp)
    simpa only [O77.Arithmetic.residualMin_transpose p q h,
      O77.Arithmetic.residualMultiplicity_transpose p q h] using ht
  · exact O77.DimensionLocalResidual.residualBandPair_of_min_le_one
```

```
      p q h ((min_le_right h q).trans hq)
  · exact O77.DimensionLocalResidual.residualBandPair_of_min_le_one
      p q h ((min_le_left h q).trans hh)
```

From a residual instance to the actual fiber point. For an exact fit $w = (A, B)$, the canonical chart uses $p = O - \operatorname{rank} B$, $q = I - \operatorname{rank} A$, and $h = H + \operatorname{rank} \Phi - (\operatorname{rank} A + \operatorname{rank} B)$. Supply the residual pair in these actual dimensions to `exact_fit_pair_of_residual`. This proves the native pair either from one concrete Wishart instance or from one of the small-dimension alternatives. No spectral assumption on Φ is involved in the fiber statement.

```
/-- Only the concrete Wishart identity of this point's residual rectangle is
an analytic parameter. The statement still uses the original native bands. -/
theorem exact_fit_pair_of_concrete {I O H : ℕ}
    {Phi : RM O I} {w : Parameters I O H} (hfit : w.2*w.1 = Phi)
    (hW : ConcreteRealWishartEigenvalueFormula
      (min (O77.residualHidden H Phi.rank w.1.rank w.2.rank) (I-w.1.rank))
      (max (O77.residualHidden H Phi.rank w.1.rank w.2.rank) (I-w.1.rank))) :
    HasLocalBandPair Phi w
      ((O77.candidateTwiceLambda I O H Phi.rank w.1.rank w.2.rank : ℝ)/2)
      (O77.candidateMultiplicity I O H Phi.rank w.1.rank w.2.rank) := by
  exact O77.FiberClassification.exact_fit_pair_of_residual hfit
    (O77.DimensionLocalResidual.residualBandPair_of_concrete _ _ _ hW)

/-- Unconditional in the written dependency graph when any actual residual
dimension is at most one. No restriction on the target's singular values. -/
theorem exact_fit_pair_of_small {I O H : ℕ}
    {Phi : RM O I} {w : Parameters I O H} (hfit : w.2*w.1 = Phi)
    (hsmall : O-w.2.rank ≤ 1 ∨ I-w.1.rank ≤ 1 ∨
      O77.residualHidden H Phi.rank w.1.rank w.2.rank ≤ 1) :
    HasLocalBandPair Phi w
      ((O77.candidateTwiceLambda I O H Phi.rank w.1.rank w.2.rank : ℝ)/2)
      (O77.candidateMultiplicity I O H Phi.rank w.1.rank w.2.rank) := by
  exact O77.FiberClassification.exact_fit_pair_of_residual hfit
    (residualBandPair_of_small _ _ _ hsmall)
```

Existence versus uniqueness. The small-dimension theorem supplies one measured pair. Radius-first power–log uniqueness identifies every other pair with it. Conversely, equality of the two components lets one substitute the known pair. This proves an equivalence of measured relations; it does not define the relation by the formula.

```
/-- This classifies the measured relation; existence is not built into its
definition, and the public multiplicity guard remains in RadiusOrder. -/
theorem exact_fit_pair_iff_of_small {I O H : ℕ}
    {Phi : RM O I} {w : Parameters I O H} (hfit : w.2*w.1 = Phi)
    (hsmall : O-w.2.rank ≤ 1 ∨ I-w.1.rank ≤ 1 ∨
      O77.residualHidden H Phi.rank w.1.rank w.2.rank ≤ 1)
    (lam : ℝ) (m : ℕ) :
    HasLocalBandPair Phi w lam m ↔
      lam = (O77.candidateTwiceLambda I O H Phi.rank w.1.rank w.2.rank : ℝ)/2 ∧
      m = O77.candidateMultiplicity I O H Phi.rank w.1.rank w.2.rank := by
  have hp := exact_fit_pair_of_small hfit hsmall
  constructor
  · intro h
    exact O77.PowerLogUniqueness.radius_order_unique h hp
  · rintro ⟨rfl, ⟩rfl
    exact hp
```

A condition on the target alone. Feasibility gives $a, b \geq r$, hence $O - b \leq O - r$, $I - a \leq I - r$, and $H + r - (a + b) \leq H - r$. Thus a small remaining target dimension guarantees the required alternative at every point of its fiber. The final theorem combines this inequality with actual rank necessity; the examples check a scalar logarithm and an asymmetric transpose case.

```
/-- Feasibility makes a target-wide small codimension sufficient at every
rank stratum. Subtraction is taken only after the full H+r sum. -/
lemma rank_shape_small {I O H r a b : ℕ} (D : O77.RankData I O H r a b)
    (hsmall : O-r ≤ 1 ∨ I-r ≤ 1 ∨ H-r ≤ 1) :
    O-b ≤ 1 ∨ I-a ≤ 1 ∨ O77.residualHidden H r a b ≤ 1 := by
  rcases D with ⟨hra, hrb, hai, hah, hbo, hbh, ⟩hab
  unfold O77.residualHidden
  omega

/-- A whole fixed-target fiber is classified using the scalar radial branch
if min(O-r,I-r,H-r) is at most one. Auxiliary empty dimensions are included. -/
theorem exact_fit_pair_iff_of_target_small {I O H : ℕ}
    {Phi : RM O I}
    (hsmall : O-Phi.rank ≤ 1 ∨ I-Phi.rank ≤ 1 ∨ H-Phi.rank ≤ 1)
    {w : Parameters I O H} (hfit : w.2*w.1 = Phi) (lam : ℝ) (m : ℕ) :
    HasLocalBandPair Phi w lam m ↔
      lam = (O77.candidateTwiceLambda I O H Phi.rank w.1.rank w.2.rank : ℝ)/2 ∧
      m = O77.candidateMultiplicity I O H Phi.rank w.1.rank w.2.rank :=
  exact_fit_pair_iff_of_small hfit
    (rank_shape_small (O77.Adapted0906.rankData_of_exact_fit hfit) hsmall) lam m

-- Source-level regression: the logarithmic scalar pair needs no global input.
example : HasLocalBandPair (0 : RM 1 1) ((0 : RM 1 1), (0 : RM 1 1))
    (1/2 : ℝ) 2 := by
  have hp := exact_fit_pair_of_small (Phi := (0 : RM 1 1))
    (w := ((0 : RM 1 1), (0 : RM 1 1))) (by simp) (Or.inl (by simp))
  have ht : O77.candidateTwiceLambda 1 1 1 0 0 0 = 1 := by decide
  have hm : O77.candidateMultiplicity 1 1 1 0 0 0 = 2 := by decide
  simpa only [Matrix.rank_zero, ht, hm, Nat.cast_one] using hp

-- This asymmetric branch specifically requires the new residual transpose.
example : HasResidualBandPairAtOrigin 1 4 3
    ((O77.residualMin 1 4 3 : ℝ)/2) (O77.residualMultiplicity 1 4 3) :=
  residualBandPair_of_small 1 4 3 (Or.inl (by decide))

end O77.LowRankFiber
end
```

87 Auxiliary module `O77LowRankAnswer`

This supplementary module is an alternate route outside the terminal theorem's dependency closure.

```
import Mathlib
import O77LowRankFiber
import O77Answer

/-! Independent small-residual route to the unchanged six-clause contract.
No higher-rank Gram or global spectral module is in this import closure. -/
set_option autoImplicit false
noncomputable section
open scoped Matrix.Norms.Elementwise

namespace O77
```

```
open O77.Reconstruction0905 (RM)
```

The same six-clause conclusion. Assume $I, O > 0$, $\operatorname{rank}\Phi < H$, and at least one of $O - \operatorname{rank}\Phi$, $I - \operatorname{rank}\Phi$, $H - \operatorname{rank}\Phi$ is at most one. For every exact fit, rank necessity gives feasibility, and the preceding lemma transfers the small-dimension condition to its residual blocks. The proved local residual pair supplies the fiber-pair argument of `answersO77_of_fiber_pairs`. Rank realization, uniqueness, attained extrema and all saddle clauses are then supplied by that assembly theorem.

```
/-- The same full answer follows from the scalar radial input whenever the
fixed target has at most one remaining input, output, or hidden dimension.
No Wishart, chart, rank-realization, or already-measured-pair premise occurs. -/
theorem answersO77_of_target_small {I O H : ℕ} (Phi : RM O I)
    (hI : 0 < I) (hO : 0 < O) (hrH : Phi.rank < H)
    (hsmall : O-Phi.rank ≤ 1 ∨ I-Phi.rank ≤ 1 ∨ H-Phi.rank ≤ 1) :
    AnswersO77 (H := H) Phi := by
  apply answersO77_of_fiber_pairs Phi ?_ hI hO hrH
  intro w hf
  exact LowRankFiber.exact_fit_pair_of_small hf
    (LowRankFiber.rank_shape_small (Adapted0906.rankData_of_exact_fit hf) hsmall)

-- All clauses, including attained extrema and all nonoptimal rung points.
example : AnswersO77 (H := 3) (0 : RM 1 4) := by
  apply answersO77_of_target_small 0 (by decide) (by decide) (by simp)
  exact Or.inl (by simp)

-- The two minimizing strata in the multiplicity counterexample.
example (Phi : RM 2 2) (hr : Phi.rank = 1) : AnswersO77 (H := 2) Phi := by
  apply answersO77_of_target_small Phi (by decide) (by decide) (by omega)
  exact Or.inl (by omega)

end O77
end
```

88 Auxiliary module `O77PublishedInputAudit`

This supplementary module is an alternate route outside the terminal theorem's dependency closure.

```
import Mathlib
import O77LowRankFiber

/-! Source regressions for a published-input specification audit. -/
set_option autoImplicit false
noncomputable section
open scoped Matrix.Norms.Elementwise

namespace O77.PublishedInputAudit
open O77.Native (HasLocalBandPair)
open O77.Reconstruction0905 (RM)
```

A measured scalar control. The small-residual fiber theorem gives the actual pair $(1/2, 2)$ for the scalar zero target at $(0, 0)$. The finite table evaluates to numerator one and multiplicity two. Applying radius-first uniqueness excludes multiplicity one at that same exponent and point. This comparison is about the measured predicate, not only candidate numbers.

```
/-- An actual native measured scalar pair, obtained from the one-eigenvalue
```

```
consumer. This theorem has no global Wishart or spectral integration premise. -/
theorem scalar_origin_pair :
    HasLocalBandPair (0 : RM 1 1) ((0 : RM 1 1), (0 : RM 1 1))
      (1 / 2 : ℝ) 2 := by
  have hp := O77.LowRankFiber.exact_fit_pair_of_small (Phi := (0 : RM 1 1))
    (w := ((0 : RM 1 1), (0 : RM 1 1))) (by simp) (Or.inl (by simp))
  have ht : O77.candidateTwiceLambda 1 1 1 0 0 0 = 1 := by decide
  have hm : O77.candidateMultiplicity 1 1 1 0 0 0 = 2 := by decide
  simpa only [Matrix.rank_zero, ht, hm, Nat.cast_one] using hp

/-- Radius-first uniqueness excludes logarithmic multiplicity one at the
same actual scalar point; this is not only a comparison of candidate formulas. -/
theorem scalar_origin_not_pair_one :
    ¬ HasLocalBandPair (0 : RM 1 1) ((0 : RM 1 1), (0 : RM 1 1))
      (1 / 2 : ℝ) 1 := by
  intro h
  have hm := (O77.PowerLogUniqueness.radius_order_unique h scalar_origin_pair).2
  norm_num at hm
```

A numerical transcription, with a deliberately narrow meaning. The function below is $m^2\{S/m\}(1-\{S/m\})$, where $\{x\} = x - \lfloor x \rfloor$. It is a numeric expression used in the literature comparison [9], not a definition of a learning multiplicity. Direct evaluation gives one at $(m, S) = (2, 3)$ and zero at $(2, 6)$. The final two-by-two calculation evaluates only the candidate table. Matching a paper's dimension conventions to these numeric arguments is an ordinary source-reading obligation, not proved by these evaluations.

```
/-- The numeric expression printed in the publication under review.
Arguments are only numeric placeholders here: matching them to the paper's
matrix dimensions and regime is a separate ordinary specification check.
No formula is installed as a definition of learning multiplicity. -/
def lr24DisplayedExpression (m S : ℕ) : ℝ :=
  (m : ℝ)^2 * Int.fract ((S : ℝ) / (m : ℝ)) *
    (1 - Int.fract ((S : ℝ) / (m : ℝ)))

theorem lr24DisplayedExpression_two_three : lr24DisplayedExpression 2 3 = 1 := by
  norm_num [lr24DisplayedExpression, Int.fract_div_natCast_eq_div_natCast_mod]

theorem lr24DisplayedExpression_two_six : lr24DisplayedExpression 2 6 = 0 := by
  norm_num [lr24DisplayedExpression, Int.fract_div_natCast_eq_div_natCast_mod]

/-- Arithmetic only. This statement does not assert a native pair for the
two-by-two origin and introduces no higher-rank Wishart dependency. -/
theorem two_by_two_origin_candidates :
    ((O77.candidateTwiceLambda 2 2 2 0 0 0 : ℝ) / 2 = (3 / 2 : ℝ)) ∧
      O77.candidateMultiplicity 2 2 2 0 0 0 = 1 := by
  have ht : O77.candidateTwiceLambda 2 2 2 0 0 0 = 3 := by decide
  have hm : O77.candidateMultiplicity 2 2 2 0 0 0 = 1 := by decide
  refine ⟨?_, ⟩hm
  norm_num [ht]

end O77.PublishedInputAudit
end
```

89 Auxiliary module `O77MultiplicityTranslation`

This supplementary module is an alternate route outside the terminal theorem's dependency closure.

```
import Mathlib
```

```
import O77PublishedInputAudit

/-! Arithmetic transcription audit for the published multiplicity
formula. No theorem below assumes or defines a measured learning coefficient. -/
set_option autoImplicit false
noncomputable section

namespace O77.MultiplicityTranslation
```

Ceiling versus fractional part. For every real x, write $u = \{x\}$. If $u = 0$, then $x = \lceil x \rceil$ and both products below vanish. If $u \neq 0$, then $\lceil x \rceil = x + 1 - u$, so the two factors on the left are $1 - u$ and u. Multiplication proves the identity, including integral arguments where the individual factors differ from their nonintegral expressions.

```
/-- The product identity remains valid at integral arguments: both products
are then zero, although the individual ceiling and fractional-part factors
are interchanged relative to the nonintegral case. -/
theorem ceil_gap_product (x : ℝ) :
    ((Int.ceil x : ℝ) - x) * (1 + x - (Int.ceil x : ℝ)) =
      Int.fract x * (1 - Int.fract x) := by
  by_cases hx : Int.fract x = 0
  · have hc : (Int.ceil x : ℝ) = x :=
      (Int.ceil_eq_self_iff_mem x).2 (Int.fract_eq_zero_iff.mp hx)
    rw [hc, hx]
    ring
  · rw [Int.ceil_eq_add_one_sub_fract hx]
    ring
```

The additive constant survives the translation. Let $j > 0$ and $x = S/j$. Put $a = S - j\lceil x \rceil + j = j(1 + x - \lceil x \rceil)$. Then $j - a = j(\lceil x \rceil - x)$, so the preceding identity gives

$$a(j - a) + 1 = j^2\{x\}(1 - \{x\}) + 1.$$

The proof first justifies division by the nonzero real cast of j, then uses the ceiling identity and ring algebra. It proves a transcription identity, not a paper-specific identification of active dimensions or measured coefficients.

```
/-- Pure algebraic translation of Aoyagi's expression `a * (j - a) + 1`.
This numerical identity makes no identification of paper-specific
dimensions or active index sets. -/
theorem aoyagi_expression_eq_displayed_add_one (j S : ℕ) (hj : 0 < j) :
    let a : ℝ := (S : ℝ) - (j : ℝ) *
      (Int.ceil ((S : ℝ) / (j : ℝ)) : ℝ) + (j : ℝ)
    a * ((j : ℝ) - a) + 1 =
      O77.PublishedInputAudit.lr24DisplayedExpression j S + 1 := by
  let x : ℝ := (S : ℝ) / (j : ℝ)
  have hj0 : (j : ℝ) ≠ 0 := by exact_mod_cast (Nat.ne_of_gt hj)
  have hx : (S : ℝ) = (j : ℝ) * x := by
    dsimp [x]
    field_simp [hj0] <;> ring
  change ((S : ℝ) - (j : ℝ) * (Int.ceil x : ℝ) + (j : ℝ)) *
      ((j : ℝ) - ((S : ℝ) - (j : ℝ) * (Int.ceil x : ℝ) + (j : ℝ))) + 1 =
    (j : ℝ)^2 * Int.fract x * (1 - Int.fract x) + 1
  calc
    _ = (j : ℝ)^2 *
        (((Int.ceil x : ℝ) - x) * (1 + x - (Int.ceil x : ℝ))) + 1 := by
      rw [hx]
      ring
    _ = _ := by
```

```
      rw [ceil_gap_product]
      ring
```

Two controls on the transcription. Adding one to the preceding numeric specializations gives two at $(2, 3)$ and one at $(2, 6)$. The integral-ratio case demonstrates why the additive constant cannot be discarded while retaining a positive candidate multiplicity. These calculations make no new assumption in the O77 theorem.

```
/-- The corrected transcription gives the double scalar pole's
multiplicity in the numerical specialization used by the audit. -/
theorem corrected_two_three :
    O77.PublishedInputAudit.lr24DisplayedExpression 2 3 + 1 = 2 := by
  rw [O77.PublishedInputAudit.lr24DisplayedExpression_two_three]
  norm_num

/-- At an integral ratio the fractional-part product vanishes, so retaining
`+ 1` is necessary even for positivity of a candidate pole multiplicity. -/
theorem corrected_two_six :
    O77.PublishedInputAudit.lr24DisplayedExpression 2 6 + 1 = 1 := by
  rw [O77.PublishedInputAudit.lr24DisplayedExpression_two_six]
  norm_num

end O77.MultiplicityTranslation
end
```

90 Certification anchor and reproducibility

90.1 Statement review reported on 17 October 2026

Niels Feld reports that he has personally reviewed the main formal statement, `O77.answersO77`, its six-clause conclusion `O77.AnswersO77`, and the upstream definitions needed to interpret that conclusion. In his judgment, they express an answer to the natural-language O77 question in its two-sided band-volume formulation, under the hypotheses stated in Part I. This is a report of his own specification review, not a claim of independent refereeing of the whole development or complete personal understanding of its proof.

The reviewed public Lean artifact is `O77Full_2026_10_06_20_50_UTC.lean`, with SHA-256

```
54dd6627468f7f1ad19784b8085e567806d0a087095a1f56a99bb920436acacc
```

The repository snapshot previously announced in MAIS issue #12 is `f4766ac0a2875cce22aa835c59b6b02beb02b698` at `https://github.com/nfeld/mais-o77-lean`. The specification review concerns the actual loss, entry norm and measure, radius-first band bounds, rank arithmetic, attained extrema, and both discarded-mode annihilation conditions in rung membership. These are the definitions displayed together in Part I. It does not extend the formal conclusion to meromorphic continuation, zeta poles, or training dynamics.

Existing mechanical evidence. The public log `recheck-20261007T222112Z-GHTD7O.log` records the same Lean source hash, the pinned Lean and Mathlib versions, successful compilation/build status, and a successful separate module kernel replay. The Lake build reused cached results. The replay used the imported environment; it was not a `--fresh` replay of all imported constants or a Comparator comparison. The seven strict axiom guards are preserved. The theorem's ordinary hypotheses must still be read in its type; an axiom list alone does not specify them.

Preservation in the present editorial update. The parent TeX is `mais-o77-blueprint_2026_10_06_20_50_UTC.tex`, SHA-256

```
a6f28e92a3299916a329b2518487fcbe633d20f6fc7397071c3f06757ff830c1
```

All 61 extracted modules, the 55-module terminal import closure, and the extractor are preserved byte for byte. No mathematical definition, theorem statement, proof script, or axiom guard changes. A fresh standalone extraction changes only the header comment recording the source TeX hash. The existing public Lean file should therefore be retained under its current filename and hash. This pass updates the review record and typesets the documents; it adds no new Lean execution receipt. The companion lecture source and README record the same scope of review.

90.2 Historical provenance of the 6 October 2026 edition

The following provenance statement is retained verbatim from the parent edition. Its references to "this edition" and to review still in progress describe that historical checkpoint; the dated statement review above is the current record.

The parent of this editorial pass is the recovered `mais-o77-blueprint_2026_10_06_15_09_UTC.tex`, SHA-256

```
2745b80496e7aac536dff90bb59b4be628b84b3a5fdb7da7b8a74560bab6f8bc
```

and its standalone `O77Full_2026_10_06_15_09_UTC.lean` has SHA-256

```
eb6e48a2be566b525e5195140f98a40dd196590b306c56a3d66fff88740c0b03
```

The reattached 13:40 TeX agrees byte for byte with that revision's frozen parent, SHA-256

```
1daf5b95367ad45854728b46831327ebbe7874c55b538b90c6a0caeabde5ac39
```

Both parent files are preserved. The certification anchor remains the supplied `O77Full_2026_09_08_13_24_UTC.lean`, SHA-256

```
7f39d428e4a203b7178db470051dae6ceb3ad7c1d9b4257466e61ce108db1914
```

Feld reports successful kernel checking of that standalone development. The seven strict axiom guards are retained verbatim, including those for the exact Wishart theorem and `O77.answersO77`; their permitted set is precisely `propext`, `Classical.choice`, and `Quot.sound`. Their docstrings are functional test data, unlike ordinary explanatory comments. The final theorem has only the hypotheses displayed in Part I. Human review of its correspondence with O77 and of the exposition remains in progress.

The pinned environment is Lean 4.34.0-rc2, commit `6a10ac8c22beadecabdbb0919c2b50214762f91d`, and Mathlib commit `664cbbe52927ce6acaa39ca35498336116e123b8`. An archival reproduction should retain the exact command, complete output, exit status, source hash and environment versions. Kernel validity and fidelity to the intended mathematical statement are separate checks. The complete statement package in Part I is designed for the latter.

Preservation in this edition. The terminal export selects the same 55-module import closure. After removal of comments by a string-aware, nested-comment-aware textual lexer, its text agrees exactly with the certification anchor, including whitespace outside comments. Every mathematical declaration, proof script, import, command and module order is thus preserved. The seven guard docstrings are also compared separately. The standalone header records the certification anchor and the SHA-256 of the actual TeX from which it is extracted. Relative to the reattached checkpoint, module hashes change only where explanatory comments change. The further prose revisions in this edition preserve all extracted module bytes from the recovered 15:09 edition. The six supplementary modules remain outside the terminal closure and are unchanged in this pass.

The editorial pass also checks labels, references, citation keys, extraction dependencies and document generation. These textual and PDF checks support reproducibility; the author's kernel-certification evidence remains attached to the identified Lean source. The parent checkpoint preserves the earlier editorial history.

90.3 Filesystem-only extraction

Save this document as `blueprint.tex`. The command below writes all 61 modules to `development`. Add the argument `full` to select only the terminal closure; use an output filename and `full-single` to reproduce the standalone program with the same mathematical code and updated comments. The other retained profiles are `conditional`, `single`, `gram`, and `gram-single`. Selection follows actual imports. The script checks its restricted namespace/section transformations but is not a Lean parser. Ordinary display-only listings in Part I are excluded. Extractable environments with historical names such as `Attempt` are identified by the manifest, not by a current verification claim.

```
python3 - blueprint.tex development <<'PYDEVELOPMENT'
from pathlib import Path
import hashlib, re, sys
mode = sys.argv[3] if len(sys.argv) == 4 else "all"
if len(sys.argv) not in (3, 4) or mode not in (
    "all", "conditional", "single", "gram", "gram-single", "full", "full-single"
):
```

```
    raise SystemExit("profile must be all, conditional, single, gram, "
                     "gram-single, full, or full-single")
source_bytes = Path(sys.argv[1]).read_bytes()
text = source_bytes.decode("utf-8")
out = Path(sys.argv[2])
profiles = [
    ("O77", "LeanCode|MathlibDraft"),
    ("O77AdaptedSections", "AdaptedSectionsAttempt"),
    ("O77NumberedCoordinates", "NumberedCoordinateAttempt"),
    ("O77Analytic", "ReconstructionCode"),
    ("O77CanonicalRanks", "CanonicalRankAttempt"),
    ("O77Transport", "TransportAttempt"),
    ("O77NormalForm", "NormalFormAttempt"),
    ("O77Coordinates", "CoordinateAttempt"),
    ("O77Canonical", "CanonicalAttempt"),
    ("O77BasisChange", "BasisChangeAttempt"),
    ("O77FiberRanks", "FiberRankAttempt"),
    ("O77FiberClassification", "FiberClassificationAttempt"),
    ("O77SaddlePreparation", "SaddlePreparationAttempt"),
    ("O77MatrixFrames", "MatrixFramesAttempt"),
    ("O77RungGeometry", "RungGeometryAttempt"),
    ("O77SaddleAttachment", "SaddleAttachmentAttempt"),
    ("O77RungCompletion", "RungCompletionAttempt"),
    ("O77PowerLogUniqueness", "PowerLogUniquenessAttempt"),
    ("O77SemanticMinimum", "SemanticMinimumAttempt"),
    ("O77Answer", "AnswerAttempt"),
    ("O77RadialGram", "RadialGramAttempt"),
    ("O77WishartReduction", "WishartReductionAttempt"),
    ("O77DimensionLocalResidual", "DimensionLocalResidualAttempt"),
    ("O77ResidualTranspose", "ResidualTransposeAttempt"),
    ("O77LowRankFiber", "LowRankFiberAttempt"),
    ("O77LowRankAnswer", "LowRankAnswerAttempt"),
    ("O77PublishedInputAudit", "PublishedInputAuditAttempt"),
    ("O77MultiplicityTranslation", "MultiplicityTranslationAttempt"),
    ("O77WishartAnswer", "WishartAnswerAttempt"),
    ("O77GramRadial", "GramRadialAttempt"),
    ("O77GramBorder", "GramBorderAttempt"),
    ("O77GramRankNull", "GramRankNullAttempt"),
    ("O77GramCoordinates", "GramCoordinateAttempt"),
    ("O77GramProjectionGeometry", "GramProjectionGeometryAttempt"),
    ("O77GramProjectionIntegral", "GramProjectionIntegralAttempt"),
    ("O77GramRows", "GramRowsAttempt"),
    ("O77GramKernel", "GramKernelAttempt"),
    ("O77GramPushforward", "GramPushforwardAttempt"),
    ("O77GramWishart", "GramWishartAttempt"),
    ("O77SpectralDifferential", "SpectralDifferentialAttempt"),
    ("O77OrthogonalCayley", "OrthogonalCayleyAttempt"),
    ("O77ConjugationVolume", "ConjugationVolumeAttempt"),
    ("O77OrthogonalSpectral", "OrthogonalSpectralAttempt"),
    ("O77PolynomialNull", "PolynomialNullAttempt"),
    ("O77SpectralExceptional", "SpectralExceptionalAttempt"),
    ("O77SpectralLocal", "SpectralLocalAttempt"),
    ("O77SkewCongruence", "SkewCongruenceAttempt"),
    ("O77CayleyDifferential", "CayleyDifferentialAttempt"),
    ("O77SpectralDensityAdapter", "SpectralDensityAdapterAttempt"),
    ("O77SpectralChartIntegral", "SpectralChartIntegralAttempt"),
    ("O77SpectralCoverage", "SpectralCoverageAttempt"),
    ("O77SpectralStabilizer", "SpectralStabilizerAttempt"),
    ("O77CayleySignCover", "CayleySignCoverAttempt"),
    ("O77CayleyFibers", "CayleyFibersAttempt"),
    ("O77SpectralSplit", "SpectralSplitAttempt"),
    ("O77SpectralOrder", "SpectralOrderAttempt"),
```

```
    ("O77SpectralRegular", "SpectralRegularAttempt"),
    ("O77CountedJacobian", "CountedJacobianAttempt"),
    ("O77GlobalSpectralFibers", "GlobalSpectralFibersAttempt"),
    ("O77CayleyWeights", "CayleyWeightsAttempt"),
    ("O77GlobalSpectralIntegral", "GlobalSpectralIntegralAttempt"),
]
# Filesystem-only support to be embedded in the manuscript's extractor.
# This recognizes the restricted command forms in the actual selected source.
# It is not a Lean parser, elaborator, kernel, or transitive axiom audit.
def source_mask(code):
    out = list(code)
    i, depth, quoted = 0, 0, False
    while i < len(code):
        if depth:
            if code.startswith('/-', i):
                out[i:i+2] = '  '
                depth += 1
                i += 2
            elif code.startswith('-/', i):
                out[i:i+2] = '  '
                depth -= 1
                i += 2
            else:
                if code[i] != '\n':
                    out[i] = ' '
                i += 1
        elif quoted:
            if code[i] == '\\':
                out[i] = ' '
                i += 1
                if i < len(code) and code[i] != '\n':
                    out[i] = ' '
                i += 1
            elif code[i] == '"':
                out[i] = ' '
                quoted = False
                i += 1
            else:
                if code[i] != '\n':
                    out[i] = ' '
                i += 1
        elif code.startswith('/-', i):
            out[i:i+2] = '  '
            depth = 1
            i += 2
        elif code.startswith('--', i):
            j = code.find('\n', i)
            if j < 0:
                j = len(code)
            out[i:j] = ' ' * (j-i)
            i = j
        elif code[i] == '"':
            out[i] = ' '
            quoted = True
            i += 1
        else:
            i += 1
    if depth or quoted:
        raise SystemExit('unterminated comment/string in selected source')
    return ''.join(out)

def standalone_body(module, data, seen):
```

```
code = data.decode('utf-8')
pure = source_mask(code)
forbidden = (
    r'\b(sorry|sorryAx|admit|axiom|opaque|unsafe|unsafeCast|'
    r'native_decide|ofReduceBool|ofReduceNat|eval_decide|'
    r'implemented_by|extern|run_tac|run_elab|elab|macro|'
    r'initialize|builtin_initialize|mkSorry|addDecl|addAndCompile)\b')
if re.search(forbidden, pure):
    raise SystemExit('forbidden source token in ' + module)
if re.search(r'(?m)^\s*(#|syntax\b|elab_rules\b|macro_rules\b)', pure):
    raise SystemExit('unexpected command extension in ' + module)
options = {'autoImplicit': 'false',
           'linter.style.haveILetI': 'false'}
stack, kept = [], []
started = False
for no, (line, masked) in enumerate(zip(
        code.splitlines(keepends=True),
        pure.splitlines(keepends=True)), 1):
    q = masked.strip()
    if re.match(r'(?:(?:public|private|meta)\s+)?import\b', q):
        if started:
            raise SystemExit('late import in ' + module)
        im = re.fullmatch(r'import\s+([A-Za-z_][\w.]*)', q)
        if not im:
            raise SystemExit('unsupported import in ' + module)
        dep = im[1]
        if dep.startswith('O77'):
            if dep not in seen:
                raise SystemExit('forward/missing import ' + dep)
        elif not (dep == 'Std' or dep == 'Mathlib'
                  or dep.startswith('Mathlib.')):
            raise SystemExit('unexpected external import ' + dep)
        continue
    if q:
        started = True
    if re.match(r'set_option\b', q):
        fields = q.split()
        if len(fields) != 3 or options.get(fields[1]) != fields[2]:
            raise SystemExit('unexpected option in ' + module)
    m = re.fullmatch(r'namespace\s+([A-Za-z_][\w.]*)', q)
    if m:
        stack.extend(('namespace', n) for n in m[1].split('.'))
    else:
        m = re.fullmatch(
            r'(?:noncomputable\s+)?section(?:\s+([A-Za-z_][\w.]*))?', q)
        if m:
            names = m[1].split('.') if m[1] else ['']
            stack.extend(('section', n) for n in names)
        else:
            m = re.fullmatch(r'end(?:\s+([A-Za-z_][\w.]*))?', q)
            if m:
                names = m[1].split('.') if m[1] else ['']
                for name in reversed(names):
                    if not stack or stack[-1][1] != name:
                        raise SystemExit('scope mismatch in ' + module
                                         + ':' + str(no))
                    stack.pop()
            elif re.match(
                    r'(?:(?:public|noncomputable)\s+)?'
                    r'(namespace|section|end)\b', q):
                raise SystemExit('unsupported scope in ' + module)
    kept.append(line)
```

```
    if any(kind != 'section' or name for kind, name in stack):
        raise SystemExit('unclosed named scope in ' + module)
    return ''.join(kept), len(stack)

def write_standalone(out, emitted, selected, source_bytes, gram=False, full=False):
    title = ("Full O77 development" if full else
             "Gram-extended O77 development" if gram else
             "Conditional O77 development")
    endpoint = (
        "The final endpoint has no Wishart or cone-eigenvalue premise."
        if full else
        "The endpoint assumes HigherRankConeEigenvalueFormula."
        if gram else
        "The published Wishart input is an explicit theorem parameter.")
    parts = [
        '/-\n' + title + '.\n'
        'Self-contained project code; the pinned Mathlib is required.\n'
        'Lean 4.34.0-rc2: 6a10ac8c22beadecabdbb0919c2b50214762f91d\n'
        'Mathlib: 664cbbe52927ce6acaa39ca35498336116e123b8\n'
        + endpoint + '\n'
        'Certification anchor: O77Full_2026_09_08_13_24_UTC.lean\n'
        'Anchor SHA-256: '
        '7f39d428e4a203b7178db470051dae6ceb3ad7c1d9b4257466e61ce108db1914\n'
        'Mathematical code is preserved; editorial comments are updated.\n'
        'From the pinned project: lake env lean THIS_FILE.lean\n'
        'Source TeX SHA-256: ' + hashlib.sha256(source_bytes).hexdigest()
        + '\n-/\n\nimport Mathlib\n\nset_option autoImplicit false\n']
    seen = set()
    for module, (_, data) in emitted.items():
        if module not in selected:
            continue
        body, depth = standalone_body(module, data, seen)
        tag = 'O77File_' + module
        parts.append('\n/- BEGIN ' + module + '\nOriginal module SHA-256: '
                     + hashlib.sha256(data).hexdigest() + '\n-/\n')
        parts.append('section ' + tag + '\n')
        parts.append(body)
        if not body.endswith('\n'):
            parts.append('\n')
        if depth:
            parts.append('-- Close anonymous sections formerly closed at EOF.\n')
            parts.append('end\n' * depth)
        parts.append('end ' + tag + '\n-- END ' + module + '\n')
        seen.add(module)
        print('single:', module, 'anonymous EOF scopes:', depth)
    if seen != selected:
        raise SystemExit('incomplete single-file closure')
    parts.append('''
-- This explicit type test supplies no extra geometric hypothesis.
example (hW : O77.External.RealWishartEigenvalueFormula)
    {I O H : Nat} (Phi : Matrix (Fin O) (Fin I) Real)
    (hI : 0 < I) (hO : 0 < O) (hrH : Phi.rank < H) :
    O77.AnswersO77 (H := H) Phi :=
  O77.answersO77_of_wishart hW Phi hI hO hrH

-- Print the actual formal statement and measured definitions for inspection.
#check @O77.answersO77_of_wishart
#print O77.AnswersO77
#print O77.Native.IsThresholdMinimizer
#print O77.candidateTwiceLambda
#print O77.candidateMultiplicity
#print O77.regularDim
```

```
#print O77.residualHidden
#print O77.residualCost
#print O77.residualMin
#print O77.residualMinimizerList
#print O77.residualMultiplicity
#print O77.Native.HasLocalBandPair
#print O77.Transport0906.RadiusOrder
#print O77.Residual.EThetaZeroPowerLog
#print O77.Residual.powerLogZero
#print O77.Residual.powerLogZeroReal
#print O77.Native.entryMeasure
#print O77.Residual.matrixLebesgue
#print O77.Residual.coordinateLebesgue
#print O77.Native.ambientSq
#print O77.Native.entryNorm
#print O77.Native.entryBall
#print O77.Native.loss
#print O77.Native.bandSet
#print O77.Native.bandVolume
#print O77.RungGeometry0906.SingularData
#print O77.RungGeometry0906.SingularData.OnRung
#print O77.External.RealWishartEigenvalueFormula
#print O77.Residual.ConcreteRealWishartEigenvalueFormula
#print O77.Residual.HasConcreteRealWishartFormula

-- Axiom guard for the conditional endpoint.
-- It rejects extra axioms, including sorryAx and native reduction axioms.
-- An ordinary theorem parameter such as hW is not an axiom dependency.
/--
info: 'O77.answersO77_of_wishart' depends on axioms:
  [propext, Classical.choice, Quot.sound]
-/
#guard_msgs (whitespace := lax) in
#print axioms O77.answersO77_of_wishart
''')
    if gram:
        parts.append('''
-- The Gram theorem is stated for the actual entry measure and arbitrary
-- measurable nonnegative tests. No integrability or Wishart premise is added.
example (k d : Nat) (hkd : k ≤ d)
    (F : Matrix (Fin k) (Fin k) Real → ENNReal) (hF : Measurable F) :
    O77.GramBorder.gaussianGramLIntegral k d F =
      ENNReal.ofReal (O77.GramBorder.gramMass k d) *
        O77.GramBorder.coneLIntegral k d F :=
  O77.GramPushforward.gaussianGramLIntegral_eq_gramMass_mul_cone k d hkd F hF

-- Conditional cone-to-eigenvalue consumer with the same answer type.
example (hS : O77.GramWishart.HigherRankConeEigenvalueFormula)
    {I O H : Nat} (Phi : Matrix (Fin O) (Fin I) Real)
    (hI : 0 < I) (hO : 0 < O) (hrH : Phi.rank < H) :
    O77.AnswersO77 (H := H) Phi :=
  O77.answersO77_of_cone_eigenvalues hS Phi hI hO hrH

#check @O77.GramPushforward.gaussianGramLIntegral_eq_gramMass_mul_cone
#check @O77.answersO77_of_cone_eigenvalues
#print O77.GramWishart.HigherRankConeEigenvalueFormula

/--
info: 'O77.GramPushforward.gaussianGramLIntegral_eq_gramMass_mul_cone' depends on axioms:
  [propext, Classical.choice, Quot.sound]
-/
#guard_msgs (whitespace := lax) in
```

```
#print axioms O77.GramPushforward.gaussianGramLIntegral_eq_gramMass_mul_cone

/--
info: 'O77.GramWishart.global_iff_cone_eigenvalues' depends on axioms:
  [propext, Classical.choice, Quot.sound]
-/
#guard_msgs (whitespace := lax) in
#print axioms O77.GramWishart.global_iff_cone_eigenvalues

/--
info: 'O77.answersO77_of_cone_eigenvalues' depends on axioms:
  [propext, Classical.choice, Quot.sound]
-/
#guard_msgs (whitespace := lax) in
#print axioms O77.answersO77_of_cone_eigenvalues
''')
    if full:
        parts.append('''
-- Full endpoint type tests: the Wishart input and O77 conclusion are supplied
-- by proof terms, with no new mathematical parameter or project axiom.
example : O77.External.RealWishartEigenvalueFormula :=
  O77.GlobalSpectralIntegral.realWishartEigenvalueFormula

example {I O H : Nat} (Phi : Matrix (Fin O) (Fin I) Real)
    (hI : 0 < I) (hO : 0 < O) (hrH : Phi.rank < H) :
    O77.AnswersO77 (H := H) Phi :=
  O77.answersO77 Phi hI hO hrH

#check @O77.GlobalSpectralIntegral.cone_eq_spectralConstant_mul_laguerre
#check @O77.GlobalSpectralIntegral.realWishartEigenvalueFormula
#check @O77.answersO77

/--
info: 'O77.GlobalSpectralIntegral.higherRankConeEigenvalueFormula' depends on axioms:
  [propext, Classical.choice, Quot.sound]
-/
#guard_msgs (whitespace := lax) in
#print axioms O77.GlobalSpectralIntegral.higherRankConeEigenvalueFormula

/--
info: 'O77.GlobalSpectralIntegral.realWishartEigenvalueFormula' depends on axioms:
  [propext, Classical.choice, Quot.sound]
-/
#guard_msgs (whitespace := lax) in
#print axioms O77.GlobalSpectralIntegral.realWishartEigenvalueFormula

/--
info: 'O77.answersO77' depends on axioms:
  [propext, Classical.choice, Quot.sound]
-/
#guard_msgs (whitespace := lax) in
#print axioms O77.answersO77
''')
    result = ''.join(parts).encode('utf-8')
    out.parent.mkdir(parents=True, exist_ok=True)
    out.write_bytes(result)
    if out.read_bytes() != result:
        raise SystemExit('single-file write mismatch')
    print(out, len(result.splitlines()), 'lines')
    print('SHA-256:', hashlib.sha256(result).hexdigest())

emitted = {}
```

```
for module, names in profiles:
    pat = (r"(?ms)^\\begin\{(" + names + r")\}\r?\n(.*?)"
           r"^\\end\{\1\}[ \t]*$")
    blocks = re.findall(pat, text)
    starts = re.findall(
        r"(?m)^\\begin\{(?:" + names + r")\}[ \t]*$", text)
    ends = re.findall(
        r"(?m)^\\end\{(?:" + names + r")\}[ \t]*$", text)
    if not blocks or len(blocks) != len(starts) \
            or len(blocks) != len(ends):
        raise SystemExit("malformed blocks for " + module)
    data = "".join(body for _, body in blocks).encode("utf-8")
    emitted[module] = (len(blocks), data)
selected = set()
def visit(module):
    if module in selected:
        return
    if module not in emitted:
        raise SystemExit("missing project import " + module)
    selected.add(module)
    body = emitted[module][1].decode("utf-8")
    for dep in re.findall(r"(?m)^import (O77\w*)\s*$", body):
        visit(dep)
target_module = ("O77GlobalSpectralIntegral" if mode.startswith("full") else
                 "O77GramWishart" if mode.startswith("gram") else "O77Answer")
for target in (list(emitted) if mode == "all" else [target_module]):
    visit(target)
if mode in ("single", "gram-single", "full-single"):
    write_standalone(out, emitted, selected, source_bytes,
                     gram=mode in ("gram-single", "full-single"),
                     full=mode == "full-single")
    raise SystemExit(0)
out.mkdir(parents=True, exist_ok=True)
for module, (block_count, data) in emitted.items():
    if module not in selected:
        continue
    path = out / (module + ".lean")
    path.write_bytes(data)
    if path.read_bytes() != data:
        raise SystemExit("write mismatch for " + module)
    print(module, block_count, len(data.splitlines()))
    print("  SHA-256:", hashlib.sha256(data).hexdigest())
PYDEVELOPMENT
```

References

[1] MAIS project, *MAIS-O77: Learning coefficients on the matrix factorization fiber and saddles*, public problem page. The repository credits Claude Fable 5 directed by Lionel Levine, audited by GPT 5.6 Sol. Version inspected 7 September 2026, repository commit `9dd29f8bf5ccd1e7701e300039b09ed4096b6516`.
https://github.com/lionellevine/MAIS/blob/9dd29f8bf5ccd1e7701e300039b09ed4096b6516/open-problems/MAIS-O77.md

[2] MAIS project, *Effective loss and learning dynamics: selection among the strata of a singular optimal set* (MAIS-A7), draft dated 13 July 2026, especially Section 2.4 and Problem 3.9. The source credits Claude Fable 5, audited by GPT 5.6 Sol. Version inspected 7 September 2026 at repository commit `9dd29f8bf5ccd1e7701e300039b09ed4096b6516`.
https://github.com/lionellevine/MAIS/blob/9dd29f8bf5ccd1e7701e300039b09ed4096b6516/agendas/A7/MAIS-A7.tex

[3] R. Sneiderman (`Robby955`), *MAIS-O70: candidate complete solution, local RLCT table for reduced-rank regression*, issue #3, opened 3 August 2026; issue text and its comment by `mbrcic` inspected 6 October 2026. Sneiderman reports development with Claude Opus 5.
https://github.com/Math-for-AI-Safety/MAIS/issues/3

[4] S. Novikov (`kumino`), *MAIS-O77: candidate complete solution—fiber rank strata and the universal saddle pair*, issue #12, opened 4 August 2026; issue text and its comment by `mbrcic` inspected 6 October 2026. The issue credits OpenAI Codex for the generated construction, drafting, and AI audit, under Novikov's direction; it defers priority for the overlapping fiber formula to Sneiderman's issue #3.
https://github.com/Math-for-AI-Safety/MAIS/issues/12

[5] S. Novikov (`kumino`), *MAIS-O7: candidate negative resolution—scalar counterexample to opposing staircases*, issue #5, opened 4 August 2026; issue text and its comment by `mbrcic` inspected 6 October 2026. Generated with OpenAI Codex under Novikov's direction. This is a scalar O7 result, not a general O77 saddle classification.
https://github.com/Math-for-AI-Safety/MAIS/issues/5

[6] M. Brcic, *AI Safety Formalization Atlas*, software and accompanying mathematical formalizations, repository commit `d79e42591eca197f9d948d26fac25bab109b1728`, inspected 6 October 2026. Author name as recorded in the repository's `CITATION.cff`. O77 source: `AISafetyAtlas/Conjectures/MAIS/O77.lean` and `O77Proof.lean`; saddle endpoint: `O77Chart.lean`. See also Brcic's formalization reports in MAIS issues #3, #5, and #12.
https://github.com/mbrcic/ai-safety-formalization-atlas/tree/d79e42591eca197f9d948d26fac25bab109b1728
https://github.com/Math-for-AI-Safety/MAIS/issues/3#issuecomment-5541316052
https://github.com/Math-for-AI-Safety/MAIS/issues/5#issuecomment-5561573741
https://github.com/Math-for-AI-Safety/MAIS/issues/12#issuecomment-5561574420

[7] R. Sneiderman, *Local learning coefficients of reduced-rank regression*, August 2026, 81-page user-supplied PDF `mais_o70_local_rlct_svd.pdf`. Theorem 5.1, Section 6, Theorem 8.1, Corollary 11.2 and Appendix A are the principal source locations. Candidate preprint; not treated as independently refereed or formally checked.

[8] M. Aoyagi and S. Watanabe, *Stochastic complexities of reduced rank regression in Bayesian estimation*, Neural Networks **18**(7) (2005), 924–933. Background on the global reduced-rank regression invariant; no local-volume theorem from this paper is assumed in the final Lean theorem.

[9] S. Pepin Lehalleur and R. Rimányi, *Geometry of the fibers of the multiplication map of deep linear neural networks*, arXiv:2411.19920v2, 11 December 2024 (first version 29 November 2024). The inspected Section 8 is used for the published-statement audit, not as an assumed all-point O77 theorem; version 1 was also checked for the displayed multiplicity.
https://arxiv.org/html/2411.19920v2

[10] I. Dumitriu and A. Edelman, *Matrix Models for Beta Ensembles*, Journal of Mathematical Physics **43** (2002), 5830–5847, DOI: 10.1063/1.1507823. The inspected source is arXiv:math-ph/0206043v1 (25 June 2002), equations (3)–(4), Theorem 3.1 and equation (24); the end of the proof of Theorem 3.4 fixes the unordered-eigenvalue convention. Source inspected 7 September 2026.
https://arxiv.org/abs/math-ph/0206043v1
https://doi.org/10.1063/1.1507823

[11] M. Aoyagi, *Singular leaning coefficients and efficiency in learning theory*, arXiv:2501.12747v1, 22 January 2025, Section 3, Definition 3 and Remark 1. The title and version are as inspected; this later preprint is not identified with an independently checked copy of the cited 2024 journal theorem. Source inspected 7 September 2026.
https://arxiv.org/html/2501.12747v1

[12] R. J. Muirhead, *Aspects of Multivariate Statistical Theory*, Wiley, 1982, Chapter 3.2. Cited through Sneiderman's Proposition 8.11 for the real Wishart density and eigenvalue transformation; theorem-number correspondence not independently checked here.

[13] A. T. James, *Normal multivariate analysis and the orthogonal group*, Annals of Mathematical Statistics **25**(1) (1954), 40–75. Cited through the supplied draft for the eigenvalue Jacobian; no page-specific primary-source verification claimed here.

[14] Mathlib contributors, `Mathlib/MeasureTheory/Function/Jacobian.lean`, source at pinned commit `664cbbe52927ce6acaa39ca35498336116e123b8`. Re-retrieved 7 September 2026; its Git blob `fe7664d0202314c1be5c7f038cf5fe2a4eb75f31` agrees with the earlier recorded blob.
https://github.com/leanprover-community/mathlib4/blob/664cbbe52927ce6acaa39ca35498336116e123b8/Mathlib/MeasureTheory/Function/Jacobian.lean

[15] Mathlib contributors, `Mathlib/MeasureTheory/Integral/Prod.lean`, source at pinned commit `664cbbe52927ce6acaa39ca35498336116e123b8`. Re-retrieved 7 September 2026; its Git blob `7484a64e40beacdd472da61f858f02c8ffb62863` agrees with the earlier recorded blob.
https://github.com/leanprover-community/mathlib4/blob/664cbbe52927ce6acaa39ca35498336116e123b8/Mathlib/MeasureTheory/Integral/Prod.lean

[16] Mathlib contributors, *Inverse function theorem, C^r case*, `Mathlib/Analysis/Calculus/InverseFunctionTheorem/ContDiff.lean`, source at pinned commit `664cbbe52927ce6acaa39ca35498336116e123b8`, retrieved 7 September 2026.
https://github.com/leanprover-community/mathlib4/blob/664cbbe52927ce6acaa39ca35498336116e123b8/Mathlib/Analysis/Calculus/InverseFunctionTheorem/ContDiff.lean

[17] P. Massot et al., *Lean blueprints*, official README, retrieved 31 August 2026; blob `b611158b316e074e467e24901a430ccbaf8ed23e`.
https://github.com/PatrickMassot/leanblueprint

[18] Mathlib contributors, pinned commit `664cbbe52927ce6acaa39ca35498336116e123b8`. Its toolchain file reads `leanprover/lean4:v4.34.0-rc2`; the official cache-snapshot artifact is the cache used for the checked profile.
https://github.com/leanprover-community/mathlib4/tree/664cbbe52927ce6acaa39ca35498336116e123b8

[19] Mathlib contributors, `Mathlib/Analysis/Calculus/ImplicitContDiff.lean`, source at pinned commit `664cbbe52927ce6acaa39ca35498336116e123b8`, retrieved 7 September 2026; Git blob `74332e2d31d4a6543bb76f65e97b1bf82670ebd8`. In particular the implicit-function equation and `ContDiffAt.contDiffAt_implicitFunction`.
https://github.com/leanprover-community/mathlib4/blob/664cbbe52927ce6acaa39ca35498336116e123b8/Mathlib/Analysis/Calculus/ImplicitContDiff.lean

[20] Mathlib contributors, `Mathlib/LinearAlgebra/Matrix/Rank.lean`, source at pinned commit `664cbbe52927ce6acaa39ca35498336116e123b8`, retrieved 7 September 2026; Git blob `31faae9f47694f8f386f96f3529dfb84a0d6fb2a`. Natural/cardinal/extended rank and

the documented square-matrix normal form are distinguished.
https://github.com/leanprover-community/mathlib4/blob/664cbbe52927ce6acaa39ca35498336116e123b8/Mathlib/LinearAlgebra/Matrix/Rank.lean

[21] Mathlib contributors, matrix norm instances and block/inverse APIs, source at pinned commit 664cbbe52927ce6acaa39ca35498336116e123b8, retrieved 7 September 2026.
https://github.com/leanprover-community/mathlib4/blob/664cbbe52927ce6acaa39ca35498336116e123b8/Mathlib/Analysis/Matrix/Normed.lean
https://github.com/leanprover-community/mathlib4/blob/664cbbe52927ce6acaa39ca35498336116e123b8/Mathlib/Data/Matrix/Block.lean
https://github.com/leanprover-community/mathlib4/blob/664cbbe52927ce6acaa39ca35498336116e123b8/Mathlib/LinearAlgebra/Matrix/NonsingularInverse.lean

[22] Mathlib contributors, Euclidean ball volume and parameter integrals, source at pinned commit 664cbbe52927ce6acaa39ca35498336116e123b8, retrieved 7 September 2026.
https://github.com/leanprover-community/mathlib4/blob/664cbbe52927ce6acaa39ca35498336116e123b8/Mathlib/MeasureTheory/Measure/Lebesgue/VolumeOfBalls.lean
https://github.com/leanprover-community/mathlib4/blob/664cbbe52927ce6acaa39ca35498336116e123b8/Mathlib/Analysis/Calculus/ParametricIntegral.lean

[23] Mathlib contributors, Mathlib/Analysis/Calculus/ContDiff/Defs.lean, commit 664cbbe52927ce6acaa39ca35498336116e123b8, implementation notes and definition of ContDiffWithinAt. The cited implementation notes and definition explicitly permit the neighbourhood to depend on the finite derivative order at order ∞.
https://github.com/leanprover-community/mathlib4/blob/664cbbe52927ce6acaa39ca35498336116e123b8/Mathlib/Analysis/Calculus/ContDiff/Defs.lean

[24] Lean contributors, scope management, private names, command printing and axiom inspection, source at commit 6a10ac8c22beadecabdbb0919c2b50214762f91d (Lean 4.34.0-rc2), inspected 7 September 2026. In particular src/Lean/Elab/BuiltinCommand.lean, src/Lean/Elab/Command/Scope.lean, src/Lean/PrivateName.lean, src/Lean/Elab/Print.lean, src/Lean/Elab/GuardMsgs.lean, and src/Lean/Util/CollectAxioms.lean.
https://github.com/leanprover/lean4/tree/6a10ac8c22beadecabdbb0919c2b50214762f91d/src/Lean